

FitlyzerC

Generated by Doxygen 1.17.0



|                                |           |
|--------------------------------|-----------|
| <b>1 FitlyzerC</b>             | <b>1</b>  |
| 1.1 Download                   | 1         |
| 1.2 What is FitlyzerC?         | 1         |
| 1.3 Features                   | 1         |
| 1.3.1 Activity Analysis        | 1         |
| 1.3.2 Interactive Charts       | 2         |
| 1.3.3 Mapping                  | 2         |
| 1.3.4 Athlete Management       | 2         |
| 1.3.5 Video Export             | 3         |
| 1.3.6 Cross Platform           | 3         |
| 1.4 Screenshots                | 3         |
| 1.5 Current Development Status | 3         |
| 1.5.1 Planned Areas            | 3         |
| 1.6 Technology Stack           | 4         |
| 1.7 Project Structure          | 4         |
| 1.8 Build Requirements         | 4         |
| 1.8.1 Required Software        | 4         |
| 1.9 Building                   | 4         |
| 1.9.1 macOS                    | 4         |
| 1.9.1.1 Dependencies           | 4         |
| 1.9.1.2 Configure              | 5         |
| 1.9.1.3 Build                  | 5         |
| 1.9.1.4 Create DMG             | 5         |
| 1.9.2 Windows                  | 5         |
| 1.9.2.1 Configure              | 5         |
| 1.9.2.2 Build                  | 5         |
| 1.9.2.3 Create Installer       | 5         |
| 1.9.3 Linux                    | 5         |
| 1.9.3.1 Arch Linux             | 5         |
| 1.9.3.2 Configure              | 5         |
| 1.9.3.3 Build                  | 5         |
| 1.9.3.4 AppImage               | 5         |
| 1.9.4 One-Command Packaging    | 6         |
| 1.9.5 Garmin FIT SDK           | 6         |
| 1.9.6 FFmpeg Bundling          | 6         |
| 1.9.7 Running                  | 6         |
| 1.9.8 Contributing             | 7         |
| 1.9.9 License                  | 7         |
| <b>2 Namespace Index</b>       | <b>9</b>  |
| 2.1 Namespace List             | 9         |
| <b>3 Hierarchical Index</b>    | <b>11</b> |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 3.1 Class Hierarchy                                                   | 11        |
| <b>4 Class Index</b>                                                  | <b>15</b> |
| 4.1 Class List                                                        | 15        |
| <b>5 File Index</b>                                                   | <b>21</b> |
| 5.1 File List                                                         | 21        |
| <b>6 Namespace Documentation</b>                                      | <b>27</b> |
| 6.1 ActivityAnalysisFlags Namespace Reference                         | 27        |
| 6.1.1 Detailed Description                                            | 27        |
| 6.1.2 Variable Documentation                                          | 27        |
| 6.1.2.1 HasClimbs                                                     | 27        |
| 6.1.2.2 HasHistogram                                                  | 27        |
| 6.1.2.3 HasIntervals                                                  | 28        |
| 6.1.2.4 HasPowerCurve                                                 | 28        |
| 6.1.2.5 HasVideo                                                      | 28        |
| 6.1.2.6 None                                                          | 28        |
| 6.2 AnalysisVersions Namespace Reference                              | 28        |
| 6.2.1 Detailed Description                                            | 28        |
| 6.2.2 Variable Documentation                                          | 28        |
| 6.2.2.1 ClimbDetection                                                | 28        |
| 6.2.2.2 IntervalDetection                                             | 28        |
| 6.3 anonymous_namespace{ActivityBrowser.cpp} Namespace Reference      | 29        |
| 6.3.1 Variable Documentation                                          | 29        |
| 6.3.1.1 kIsoDateRole                                                  | 29        |
| 6.3.1.2 kNotesRole                                                    | 29        |
| 6.3.1.3 kWeatherRole                                                  | 29        |
| 6.4 anonymous_namespace{AnalysisWorker.cpp} Namespace Reference       | 29        |
| 6.4.1 Function Documentation                                          | 29        |
| 6.4.1.1 applySqlitePragmas()                                          | 29        |
| 6.5 anonymous_namespace{AthleteDialog.cpp} Namespace Reference        | 29        |
| 6.5.1 Function Documentation                                          | 30        |
| 6.5.1.1 displayDateOrRaw()                                            | 30        |
| 6.5.1.2 toIsoOrRaw()                                                  | 30        |
| 6.5.2 Variable Documentation                                          | 30        |
| 6.5.2.1 kEntryIdRole                                                  | 30        |
| 6.6 anonymous_namespace{ChartController.cpp} Namespace Reference      | 30        |
| 6.6.1 Function Documentation                                          | 30        |
| 6.6.1.1 fmtDur()                                                      | 30        |
| 6.6.1.2 rangeLabelForZone()                                           | 30        |
| 6.7 anonymous_namespace{ClimbCandidateFinder.cpp} Namespace Reference | 30        |
| 6.7.1 Function Documentation                                          | 30        |

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 6.7.1.1 segmentMeters()                                                    | 30 |
| 6.8 anonymous_namespace{ClimbDetector.cpp} Namespace Reference             | 31 |
| 6.8.1 Function Documentation                                               | 31 |
| 6.8.1.1 buildFilteredAltitudeMeters()                                      | 31 |
| 6.8.1.2 clampValue()                                                       | 31 |
| 6.8.1.3 estimateSpeedKmhAt()                                               | 31 |
| 6.8.1.4 isFinite()                                                         | 31 |
| 6.9 anonymous_namespace{ClimbMetrics.cpp} Namespace Reference              | 31 |
| 6.9.1 Function Documentation                                               | 32 |
| 6.9.1.1 classifyClimbCategory()                                            | 32 |
| 6.9.1.2 classifyClimbShape()                                               | 32 |
| 6.9.1.3 computeLinearRegressionSlope()                                     | 32 |
| 6.9.1.4 computeNormalizedPower()                                           | 32 |
| 6.9.1.5 computeSliceStats()                                                | 32 |
| 6.9.1.6 isFinite()                                                         | 32 |
| 6.10 anonymous_namespace{ClimbRepository.cpp} Namespace Reference          | 33 |
| 6.10.1 Function Documentation                                              | 33 |
| 6.10.1.1 recordFromQuery()                                                 | 33 |
| 6.10.2 Variable Documentation                                              | 33 |
| 6.10.2.1 kSelectCols                                                       | 33 |
| 6.11 anonymous_namespace{ColorProvider.cpp} Namespace Reference            | 33 |
| 6.11.1 Function Documentation                                              | 33 |
| 6.11.1.1 interpolateColor()                                                | 33 |
| 6.12 anonymous_namespace{DatabaseMigrationManager.cpp} Namespace Reference | 33 |
| 6.12.1 Function Documentation                                              | 34 |
| 6.12.1.1 columnExists()                                                    | 34 |
| 6.12.1.2 ensureActivityTimeColumns()                                       | 34 |
| 6.13 anonymous_namespace{DateFormatter.cpp} Namespace Reference            | 34 |
| 6.13.1 Function Documentation                                              | 34 |
| 6.13.1.1 datePatternForFormat()                                            | 34 |
| 6.14 anonymous_namespace{FfmpegPath.cpp} Namespace Reference               | 34 |
| 6.14.1 Function Documentation                                              | 34 |
| 6.14.1.1 absoluteExecutablePath()                                          | 34 |
| 6.15 anonymous_namespace{FitParser.cpp} Namespace Reference                | 34 |
| 6.15.1 Function Documentation                                              | 35 |
| 6.15.1.1 fitTimestampToIsoUtc()                                            | 35 |
| 6.16 anonymous_namespace{ImportWorker.cpp} Namespace Reference             | 35 |
| 6.16.1 Function Documentation                                              | 35 |
| 6.16.1.1 applySqlitePragmas()                                              | 35 |
| 6.17 anonymous_namespace{MainWindowChartActions.cpp} Namespace Reference   | 35 |
| 6.17.1 Variable Documentation                                              | 35 |
| 6.17.1.1 kChartPresetCustom                                                | 35 |

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| 6.17.1.2 kChartPresetHeartRateFocus . . . . .                                           | 35 |
| 6.17.1.3 kChartPresetPowerAnalysis . . . . .                                            | 35 |
| 6.17.1.4 kChartPresetRaceReview . . . . .                                               | 35 |
| 6.18 anonymous_namespace{MainWindowClimbInteractions.cpp} Namespace Reference . . . . . | 36 |
| 6.18.1 Function Documentation . . . . .                                                 | 36 |
| 6.18.1.1 fmtDur() . . . . .                                                             | 36 |
| 6.18.2 Variable Documentation . . . . .                                                 | 36 |
| 6.18.2.1 kClimbEndRole . . . . .                                                        | 36 |
| 6.18.2.2 kClimbStartRole . . . . .                                                      | 36 |
| 6.19 anonymous_namespace{MainWindowClimbViews.cpp} Namespace Reference . . . . .        | 36 |
| 6.19.1 Function Documentation . . . . .                                                 | 37 |
| 6.19.1.1 fmtDur() . . . . .                                                             | 37 |
| 6.19.2 Variable Documentation . . . . .                                                 | 37 |
| 6.19.2.1 kClimbEndRole . . . . .                                                        | 37 |
| 6.19.2.2 kClimbStartRole . . . . .                                                      | 37 |
| 6.20 anonymous_namespace{MainWindowDatabase.cpp} Namespace Reference . . . . .          | 37 |
| 6.20.1 Function Documentation . . . . .                                                 | 38 |
| 6.20.1.1 mapLegacyTabsToPageIndex() . . . . .                                           | 38 |
| 6.20.2 Variable Documentation . . . . .                                                 | 38 |
| 6.20.2.1 kChartPresetCustom . . . . .                                                   | 38 |
| 6.20.2.2 kLegacyAnalysisCalendar . . . . .                                              | 38 |
| 6.20.2.3 kLegacyAnalysisCharts . . . . .                                                | 38 |
| 6.20.2.4 kLegacyAnalysisClimbing . . . . .                                              | 38 |
| 6.20.2.5 kLegacyAnalysisFitness . . . . .                                               | 38 |
| 6.20.2.6 kLegacyAnalysisHistogram . . . . .                                             | 38 |
| 6.20.2.7 kLegacyAnalysisPowerCurve . . . . .                                            | 38 |
| 6.20.2.8 kLegacyAnalysisZones . . . . .                                                 | 38 |
| 6.20.2.9 kLegacyTabActivities . . . . .                                                 | 38 |
| 6.20.2.10 kLegacyTabAnalysis . . . . .                                                  | 39 |
| 6.20.2.11 kMaxRecentDatabases . . . . .                                                 | 39 |
| 6.20.2.12 kPageActivities . . . . .                                                     | 39 |
| 6.20.2.13 kPageCalendar . . . . .                                                       | 39 |
| 6.20.2.14 kPageCharts . . . . .                                                         | 39 |
| 6.20.2.15 kPageClimbs . . . . .                                                         | 39 |
| 6.20.2.16 kPageFitness . . . . .                                                        | 39 |
| 6.20.2.17 kPageIntervals . . . . .                                                      | 39 |
| 6.20.2.18 kPagePower . . . . .                                                          | 39 |
| 6.20.2.19 kPageVideo . . . . .                                                          | 39 |
| 6.21 anonymous_namespace{MainWindowIntervals.cpp} Namespace Reference . . . . .         | 39 |
| 6.21.1 Function Documentation . . . . .                                                 | 40 |
| 6.21.1.1 fmtDur() . . . . .                                                             | 40 |
| 6.21.2 Variable Documentation . . . . .                                                 | 40 |

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| 6.21.2.1 kIntervalEndRole . . . . .                                                   | 40 |
| 6.21.2.2 kIntervalStartRole . . . . .                                                 | 40 |
| 6.22 anonymous_namespace{MainWindowStatus.cpp} Namespace Reference . . . . .          | 40 |
| 6.22.1 Function Documentation . . . . .                                               | 40 |
| 6.22.1.1 formatActivityDateLabel() . . . . .                                          | 40 |
| 6.23 anonymous_namespace{MainWindowUiBuild.cpp} Namespace Reference . . . . .         | 40 |
| 6.23.1 Enumeration Type Documentation . . . . .                                       | 41 |
| 6.23.1.1 ChartPresetId . . . . .                                                      | 41 |
| 6.23.2 Function Documentation . . . . .                                               | 41 |
| 6.23.2.1 wrapWithNoActivityState() . . . . .                                          | 41 |
| 6.23.3 Variable Documentation . . . . .                                               | 41 |
| 6.23.3.1 kClimbEndRole . . . . .                                                      | 41 |
| 6.23.3.2 kClimbStartRole . . . . .                                                    | 41 |
| 6.24 anonymous_namespace{MainWindowVideo.cpp} Namespace Reference . . . . .           | 41 |
| 6.24.1 Function Documentation . . . . .                                               | 41 |
| 6.24.1.1 sanitizeVideoFilePart() . . . . .                                            | 41 |
| 6.25 anonymous_namespace{MapFitMath.cpp} Namespace Reference . . . . .                | 41 |
| 6.25.1 Function Documentation . . . . .                                               | 42 |
| 6.25.1.1 latToTileY() . . . . .                                                       | 42 |
| 6.25.1.2 normalizeLongitude() . . . . .                                               | 42 |
| 6.25.1.3 tileYToLat() . . . . .                                                       | 42 |
| 6.25.2 Variable Documentation . . . . .                                               | 42 |
| 6.25.2.1 kMaxMercatorLat . . . . .                                                    | 42 |
| 6.25.2.2 kMinLatSpan . . . . .                                                        | 42 |
| 6.25.2.3 kMinLonSpan . . . . .                                                        | 42 |
| 6.25.2.4 kPi . . . . .                                                                | 42 |
| 6.25.2.5 kTilePx . . . . .                                                            | 42 |
| 6.26 anonymous_namespace{RideChartSelectionManager.cpp} Namespace Reference . . . . . | 43 |
| 6.26.1 Variable Documentation . . . . .                                               | 43 |
| 6.26.1.1 kClimbHandleCenterOffsetYPx . . . . .                                        | 43 |
| 6.26.1.2 kClimbHandleGrabRadiusPx . . . . .                                           | 43 |
| 6.26.1.3 kClimbHandleRadiusPx . . . . .                                               | 43 |
| 6.27 anonymous_namespace{RideChartWidget.cpp} Namespace Reference . . . . .           | 43 |
| 6.28 anonymous_namespace{TileCache.cpp} Namespace Reference . . . . .                 | 43 |
| 6.28.1 Function Documentation . . . . .                                               | 43 |
| 6.28.1.1 defaultDiskCacheRoot() . . . . .                                             | 43 |
| 6.29 anonymous_namespace{VideoExportDialog.cpp} Namespace Reference . . . . .         | 43 |
| 6.29.1 Function Documentation . . . . .                                               | 44 |
| 6.29.1.1 formatVideoLength() . . . . .                                                | 44 |
| 6.29.1.2 makeDefaultFilename() . . . . .                                              | 44 |
| 6.29.1.3 mapStyleName() . . . . .                                                     | 44 |
| 6.30 anonymous_namespace{VideoRenderer.cpp} Namespace Reference . . . . .             | 44 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 6.30.1 Variable Documentation                                       | 44 |
| 6.30.1.1 kPi                                                        | 44 |
| 6.30.1.2 kTilePx                                                    | 44 |
| 6.31 anonymous_namespace{VideoRenderJob.cpp} Namespace Reference    | 44 |
| 6.31.1 Function Documentation                                       | 45 |
| 6.31.1.1 routeColor()                                               | 45 |
| 6.31.2 Variable Documentation                                       | 45 |
| 6.31.2.1 kTilePx                                                    | 45 |
| 6.31.2.2 kVideoExportTileCache                                      | 45 |
| 6.32 anonymous_namespace{VideoTileProvider.cpp} Namespace Reference | 45 |
| 6.32.1 Function Documentation                                       | 45 |
| 6.32.1.1 defaultPrimaryTileCacheRoot()                              | 45 |
| 6.32.1.2 defaultVideoExportCacheRoot()                              | 45 |
| 6.33 anonymous_namespace{WelcomeWidget.cpp} Namespace Reference     | 45 |
| 6.33.1 Function Documentation                                       | 46 |
| 6.33.1.1 makeCard()                                                 | 46 |
| 6.33.1.2 makeHeading()                                              | 46 |
| 6.33.1.3 makePrimaryBtn()                                           | 46 |
| 6.33.1.4 makeSecondaryBtn()                                         | 46 |
| 6.34 anonymous_namespace{WorkoutController.cpp} Namespace Reference | 46 |
| 6.34.1 Function Documentation                                       | 46 |
| 6.34.1.1 activityDateForFtp()                                       | 46 |
| 6.34.1.2 resolveAthleteFtpForDate()                                 | 46 |
| 6.35 anonymous_namespace{ZoneCalculator.cpp} Namespace Reference    | 47 |
| 6.35.1 Function Documentation                                       | 47 |
| 6.35.1.1 sampleAvailable()                                          | 47 |
| 6.35.1.2 sampleDurationSeconds()                                    | 47 |
| 6.35.1.3 valueForMetric()                                           | 47 |
| 6.36 ClimbDefaults Namespace Reference                              | 47 |
| 6.36.1 Detailed Description                                         | 47 |
| 6.36.2 Variable Documentation                                       | 47 |
| 6.36.2.1 kDipDistance                                               | 47 |
| 6.36.2.2 kDipMeters                                                 | 48 |
| 6.36.2.3 kMinGain                                                   | 48 |
| 6.36.2.4 kMinGradient                                               | 48 |
| 6.36.2.5 kMinLength                                                 | 48 |
| 6.36.2.6 kSmoothing                                                 | 48 |
| 6.36.2.7 kStartGrad                                                 | 48 |
| 6.37 Platform Namespace Reference                                   | 48 |
| 6.37.1 Detailed Description                                         | 48 |
| 6.37.2 Function Documentation                                       | 48 |
| 6.37.2.1 applicationDataDirectory()                                 | 48 |



|                                              |           |
|----------------------------------------------|-----------|
| 6.37.2.2 defaultBackupPath()                 | 48        |
| 6.37.2.3 defaultDatabaseDirectory()          | 49        |
| 6.37.2.4 defaultDatabasePath()               | 49        |
| 6.38 SqlQueryHelper Namespace Reference      | 49        |
| 6.38.1 Detailed Description                  | 49        |
| 6.38.2 Function Documentation                | 49        |
| 6.38.2.1 bindBoolValue()                     | 49        |
| 6.38.2.2 selectExistsEver()                  | 50        |
| 6.38.2.3 selectExistsLocked()                | 50        |
| 6.38.2.4 selectExistsNonDeleted()            | 51        |
| 6.38.3 Variable Documentation                | 51        |
| 6.38.3.1 kWhereDeleted                       | 51        |
| 6.38.3.2 kWhereLocked                        | 52        |
| 6.38.3.3 kWhereNotDeleted                    | 52        |
| <b>7 Class Documentation</b>                 | <b>53</b> |
| 7.1 AboutDialog Class Reference              | 53        |
| 7.1.1 Detailed Description                   | 54        |
| 7.1.2 Constructor & Destructor Documentation | 54        |
| 7.1.2.1 AboutDialog()                        | 54        |
| 7.2 ActivitiesPage Class Reference           | 54        |
| 7.2.1 Detailed Description                   | 55        |
| 7.2.2 Constructor & Destructor Documentation | 56        |
| 7.2.2.1 ActivitiesPage()                     | 56        |
| 7.2.3 Member Function Documentation          | 57        |
| 7.2.3.1 toolbar()                            | 57        |
| 7.2.4 Member Data Documentation              | 57        |
| 7.2.4.1 m_toolbar                            | 57        |
| 7.3 Activity Struct Reference                | 57        |
| 7.3.1 Detailed Description                   | 58        |
| 7.3.2 Member Function Documentation          | 59        |
| 7.3.2.1 isValid()                            | 59        |
| 7.3.3 Member Data Documentation              | 59        |
| 7.3.3.1 analysisFlags                        | 59        |
| 7.3.3.2 analysisVersion                      | 59        |
| 7.3.3.3 athleteId                            | 59        |
| 7.3.3.4 avgCadence                           | 59        |
| 7.3.3.5 avgHr                                | 59        |
| 7.3.3.6 avgPower                             | 59        |
| 7.3.3.7 avgSpeed                             | 59        |
| 7.3.3.8 bike                                 | 60        |
| 7.3.3.9 distanceM                            | 60        |

|                                                     |    |
|-----------------------------------------------------|----|
| 7.3.3.10 durationSec                                | 60 |
| 7.3.3.11 elevationGain                              | 60 |
| 7.3.3.12 endTime                                    | 60 |
| 7.3.3.13 equipment                                  | 60 |
| 7.3.3.14 fatigue                                    | 60 |
| 7.3.3.15 fileName                                   | 60 |
| 7.3.3.16 fingerprint                                | 60 |
| 7.3.3.17 fitHash                                    | 61 |
| 7.3.3.18 id                                         | 61 |
| 7.3.3.19 importedAt                                 | 61 |
| 7.3.3.20 importSource                               | 61 |
| 7.3.3.21 maxHr                                      | 61 |
| 7.3.3.22 maxPower                                   | 61 |
| 7.3.3.23 normalizedPower                            | 61 |
| 7.3.3.24 notes                                      | 61 |
| 7.3.3.25 rpe                                        | 62 |
| 7.3.3.26 sleepHours                                 | 62 |
| 7.3.3.27 sport                                      | 62 |
| 7.3.3.28 startTime                                  | 62 |
| 7.3.3.29 temperature                                | 62 |
| 7.3.3.30 weather                                    | 62 |
| 7.3.3.31 weatherNotes                               | 62 |
| 7.3.3.32 weightKg                                   | 62 |
| 7.3.3.33 wind                                       | 62 |
| 7.4 ActivityAnalysisCacheRepository Class Reference | 63 |
| 7.4.1 Detailed Description                          | 63 |
| 7.4.2 Constructor & Destructor Documentation        | 63 |
| 7.4.2.1 ActivityAnalysisCacheRepository()           | 63 |
| 7.4.3 Member Function Documentation                 | 63 |
| 7.4.3.1 getCache()                                  | 63 |
| 7.4.3.2 hasCache()                                  | 64 |
| 7.4.3.3 invalidateCache()                           | 64 |
| 7.4.3.4 invalidateKey()                             | 64 |
| 7.4.3.5 setCache()                                  | 65 |
| 7.4.4 Member Data Documentation                     | 65 |
| 7.4.4.1 m_db                                        | 65 |
| 7.5 ActivityAnalysisSettings Struct Reference       | 65 |
| 7.5.1 Detailed Description                          | 66 |
| 7.5.2 Member Function Documentation                 | 66 |
| 7.5.2.1 isValid()                                   | 66 |
| 7.5.3 Member Data Documentation                     | 66 |
| 7.5.3.1 activityId                                  | 66 |

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| 7.5.3.2 climbMinGain . . . . .                                                                | 66 |
| 7.5.3.3 climbMinGradient . . . . .                                                            | 66 |
| 7.5.3.4 climbMinLength . . . . .                                                              | 66 |
| 7.5.3.5 intervalRestThreshold . . . . .                                                       | 67 |
| 7.5.3.6 intervalWorkThreshold . . . . .                                                       | 67 |
| 7.6 ActivityAnalysisSettingsRepository Class Reference . . . . .                              | 67 |
| 7.6.1 Detailed Description . . . . .                                                          | 67 |
| 7.6.2 Constructor & Destructor Documentation . . . . .                                        | 67 |
| 7.6.2.1 ActivityAnalysisSettingsRepository() . . . . .                                        | 67 |
| 7.6.3 Member Function Documentation . . . . .                                                 | 68 |
| 7.6.3.1 insertOrUpdate() . . . . .                                                            | 68 |
| 7.6.3.2 settingsForActivity() . . . . .                                                       | 68 |
| 7.6.4 Member Data Documentation . . . . .                                                     | 68 |
| 7.6.4.1 m_db . . . . .                                                                        | 68 |
| 7.7 ActivityBrowser Class Reference . . . . .                                                 | 68 |
| 7.7.1 Detailed Description . . . . .                                                          | 70 |
| 7.7.2 Constructor & Destructor Documentation . . . . .                                        | 70 |
| 7.7.2.1 ActivityBrowser() . . . . .                                                           | 70 |
| 7.7.3 Member Function Documentation . . . . .                                                 | 70 |
| 7.7.3.1 applyFilters() . . . . .                                                              | 70 |
| 7.7.3.2 deleteActivity() . . . . .                                                            | 70 |
| 7.7.3.3 dragEnterEvent() . . . . .                                                            | 70 |
| 7.7.3.4 dropEvent() . . . . .                                                                 | 70 |
| 7.7.3.5 editNotes() . . . . .                                                                 | 70 |
| 7.7.3.6 focusSearch() . . . . .                                                               | 70 |
| 7.7.3.7 refresh() . . . . .                                                                   | 71 |
| 7.7.3.8 showContextMenu() . . . . .                                                           | 71 |
| 7.7.4 Member Data Documentation . . . . .                                                     | 71 |
| 7.7.4.1 m_athleteId . . . . .                                                                 | 71 |
| 7.7.4.2 m_dbManager . . . . .                                                                 | 71 |
| 7.7.4.3 m_emptyStateLabel . . . . .                                                           | 71 |
| 7.7.4.4 m_rangeFilter . . . . .                                                               | 71 |
| 7.7.4.5 m_searchEdit . . . . .                                                                | 71 |
| 7.7.4.6 m_table . . . . .                                                                     | 71 |
| 7.8 anonymous_namespace{ActivityBrowser.cpp}::ActivityDataTableItem Class Reference . . . . . | 72 |
| 7.8.1 Detailed Description . . . . .                                                          | 72 |
| 7.8.2 Member Function Documentation . . . . .                                                 | 72 |
| 7.8.2.1 operator<() . . . . .                                                                 | 72 |
| 7.9 ActivityRepository Class Reference . . . . .                                              | 73 |
| 7.9.1 Detailed Description . . . . .                                                          | 73 |
| 7.9.2 Constructor & Destructor Documentation . . . . .                                        | 73 |
| 7.9.2.1 ActivityRepository() . . . . .                                                        | 73 |

|                                               |    |
|-----------------------------------------------|----|
| 7.9.3 Member Function Documentation           | 74 |
| 7.9.3.1 deleteActivity()                      | 74 |
| 7.9.3.2 getActivity()                         | 74 |
| 7.9.3.3 hasFingerprintMatch()                 | 74 |
| 7.9.3.4 hasFitHash()                          | 74 |
| 7.9.3.5 insertActivity()                      | 75 |
| 7.9.3.6 listActivities()                      | 76 |
| 7.9.3.7 setImportSource()                     | 76 |
| 7.9.3.8 updateActivityProperties()            | 77 |
| 7.9.3.9 updateAnalysisFlags()                 | 77 |
| 7.9.3.10 updateFingerprint()                  | 78 |
| 7.9.3.11 updateNotes()                        | 78 |
| 7.9.4 Member Data Documentation               | 78 |
| 7.9.4.1 m_db                                  | 78 |
| 7.10 ActivityViewController Class Reference   | 79 |
| 7.10.1 Detailed Description                   | 80 |
| 7.10.2 Constructor & Destructor Documentation | 81 |
| 7.10.2.1 ActivityViewController()             | 81 |
| 7.10.3 Member Function Documentation          | 81 |
| 7.10.3.1 formatActivityDateLabel()            | 81 |
| 7.10.3.2 loadActivity()                       | 81 |
| 7.10.3.3 onWorkoutLoaded                      | 82 |
| 7.10.3.4 populateActivityDetails()            | 83 |
| 7.10.3.5 refreshActivity()                    | 83 |
| 7.10.3.6 setAthleteContext()                  | 84 |
| 7.10.3.7 setAthleteHeaderWidget()             | 84 |
| 7.10.3.8 setStatusBarLabels()                 | 84 |
| 7.10.3.9 syncAnalysisViews()                  | 84 |
| 7.10.3.10 updateActivityDisplay()             | 85 |
| 7.10.3.11 updateActivityMetadata()            | 86 |
| 7.10.3.12 updateStatsDisplay()                | 86 |
| 7.10.3.13 updateStatusBarInformation()        | 86 |
| 7.10.4 Member Data Documentation              | 87 |
| 7.10.4.1 m_activityCountLabel                 | 87 |
| 7.10.4.2 m_athleteHeader                      | 87 |
| 7.10.4.3 m_athleteStatusLabel                 | 87 |
| 7.10.4.4 m_controller                         | 87 |
| 7.10.4.5 m_currentAthleteId                   | 87 |
| 7.10.4.6 m_currentAthleteName                 | 87 |
| 7.10.4.7 m_dbManager                          | 87 |
| 7.10.4.8 m_dbStatusLabel                      | 87 |
| 7.11 AnalysisQueue Class Reference            | 87 |

|                                                         |    |
|---------------------------------------------------------|----|
| 7.11.1 Detailed Description . . . . .                   | 89 |
| 7.11.2 Constructor & Destructor Documentation . . . . . | 89 |
| 7.11.2.1 AnalysisQueue() . . . . .                      | 89 |
| 7.11.2.2 ~AnalysisQueue() . . . . .                     | 90 |
| 7.11.3 Member Function Documentation . . . . .          | 90 |
| 7.11.3.1 enqueue() . . . . .                            | 90 |
| 7.11.3.2 isBusy() . . . . .                             | 90 |
| 7.11.3.3 onTaskCompleted . . . . .                      | 90 |
| 7.11.3.4 onTaskFailed . . . . .                         | 90 |
| 7.11.3.5 pendingCount() . . . . .                       | 90 |
| 7.11.3.6 pendingCountChanged . . . . .                  | 91 |
| 7.11.3.7 processNext() . . . . .                        | 91 |
| 7.11.3.8 setDatabasePath() . . . . .                    | 91 |
| 7.11.3.9 taskCompleted . . . . .                        | 91 |
| 7.11.3.10 taskFailed . . . . .                          | 92 |
| 7.11.4 Member Data Documentation . . . . .              | 92 |
| 7.11.4.1 m_busy . . . . .                               | 92 |
| 7.11.4.2 m_dbPath . . . . .                             | 92 |
| 7.11.4.3 m_queue . . . . .                              | 92 |
| 7.11.4.4 m_thread . . . . .                             | 92 |
| 7.11.4.5 m_worker . . . . .                             | 92 |
| 7.12 AnalysisWorker Class Reference . . . . .           | 93 |
| 7.12.1 Detailed Description . . . . .                   | 94 |
| 7.12.2 Constructor & Destructor Documentation . . . . . | 94 |
| 7.12.2.1 AnalysisWorker() . . . . .                     | 94 |
| 7.12.3 Member Function Documentation . . . . .          | 94 |
| 7.12.3.1 processTask . . . . .                          | 94 |
| 7.12.3.2 setDatabasePath() . . . . .                    | 95 |
| 7.12.3.3 taskCompleted . . . . .                        | 95 |
| 7.12.3.4 taskFailed . . . . .                           | 96 |
| 7.12.4 Member Data Documentation . . . . .              | 96 |
| 7.12.4.1 m_dbPath . . . . .                             | 96 |
| 7.13 AppSettings Class Reference . . . . .              | 97 |
| 7.13.1 Detailed Description . . . . .                   | 97 |
| 7.13.2 Member Function Documentation . . . . .          | 97 |
| 7.13.2.1 dateFormat() . . . . .                         | 97 |
| 7.13.2.2 dateFormatKey() . . . . .                      | 97 |
| 7.13.2.3 instance() . . . . .                           | 97 |
| 7.13.2.4 sanitizeDateFormat() . . . . .                 | 98 |
| 7.13.2.5 setDateFormat() . . . . .                      | 98 |
| 7.13.2.6 setTileCacheSize() . . . . .                   | 98 |
| 7.13.2.7 tileCacheSize() . . . . .                      | 98 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.13.2.8 tileCacheSizeKey()                   | 99  |
| 7.14 Athlete Struct Reference                 | 99  |
| 7.14.1 Detailed Description                   | 99  |
| 7.14.2 Member Function Documentation          | 99  |
| 7.14.2.1 fullName()                           | 99  |
| 7.14.2.2 isValid()                            | 100 |
| 7.14.3 Member Data Documentation              | 100 |
| 7.14.3.1 createdAt                            | 100 |
| 7.14.3.2 dateOfBirth                          | 100 |
| 7.14.3.3 email                                | 100 |
| 7.14.3.4 firstName                            | 100 |
| 7.14.3.5 ftpWatts                             | 100 |
| 7.14.3.6 heightCm                             | 101 |
| 7.14.3.7 id                                   | 101 |
| 7.14.3.8 lastName                             | 101 |
| 7.15 AthleteDialog Class Reference            | 101 |
| 7.15.1 Detailed Description                   | 102 |
| 7.15.2 Constructor & Destructor Documentation | 103 |
| 7.15.2.1 AthleteDialog()                      | 103 |
| 7.15.3 Member Function Documentation          | 103 |
| 7.15.3.1 addFtpRow                            | 103 |
| 7.15.3.2 addWeightRow                         | 103 |
| 7.15.3.3 deleteFtpRow                         | 103 |
| 7.15.3.4 deleteWeightRow                      | 103 |
| 7.15.3.5 loadAthlete()                        | 103 |
| 7.15.3.6 onAccepted                           | 104 |
| 7.15.3.7 populateFtpTable()                   | 104 |
| 7.15.3.8 populateWeightTable()                | 104 |
| 7.15.3.9 savedAthleteld()                     | 104 |
| 7.15.4 Member Data Documentation              | 104 |
| 7.15.4.1 m_athleteld                          | 104 |
| 7.15.4.2 m_dob                                | 104 |
| 7.15.4.3 m_email                              | 104 |
| 7.15.4.4 m_firstName                          | 104 |
| 7.15.4.5 m_ftp                                | 104 |
| 7.15.4.6 m_ftpTable                           | 104 |
| 7.15.4.7 m_height                             | 105 |
| 7.15.4.8 m_lastName                           | 105 |
| 7.15.4.9 m_repo                               | 105 |
| 7.15.4.10 m_savedId                           | 105 |
| 7.15.4.11 m_weightTable                       | 105 |
| 7.16 AthleteHeaderWidget Class Reference      | 105 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.16.1 Detailed Description                   | 106 |
| 7.16.2 Constructor & Destructor Documentation | 106 |
| 7.16.2.1 AthleteHeaderWidget()                | 106 |
| 7.16.3 Member Function Documentation          | 106 |
| 7.16.3.1 setRideMetrics()                     | 106 |
| 7.16.3.2 setSummary()                         | 106 |
| 7.16.4 Member Data Documentation              | 107 |
| 7.16.4.1 m_metricsLabel                       | 107 |
| 7.16.4.2 m_nameLabel                          | 107 |
| 7.16.4.3 m_statsLabel                         | 107 |
| 7.17 AthleteListDialog Class Reference        | 107 |
| 7.17.1 Detailed Description                   | 108 |
| 7.17.2 Constructor & Destructor Documentation | 108 |
| 7.17.2.1 AthleteListDialog()                  | 108 |
| 7.17.3 Member Function Documentation          | 108 |
| 7.17.3.1 deleteAthlete                        | 108 |
| 7.17.3.2 editAthlete                          | 108 |
| 7.17.3.3 newAthlete                           | 108 |
| 7.17.3.4 refresh                              | 108 |
| 7.17.3.5 selectAthlete                        | 109 |
| 7.17.3.6 selectedAthleteld()                  | 109 |
| 7.17.4 Member Data Documentation              | 109 |
| 7.17.4.1 m_deleteBtn                          | 109 |
| 7.17.4.2 m_editBtn                            | 109 |
| 7.17.4.3 m_repo                               | 109 |
| 7.17.4.4 m_selectBtn                          | 109 |
| 7.17.4.5 m_selectedId                         | 109 |
| 7.17.4.6 m_table                              | 109 |
| 7.18 AthleteRepository Class Reference        | 109 |
| 7.18.1 Detailed Description                   | 110 |
| 7.18.2 Constructor & Destructor Documentation | 110 |
| 7.18.2.1 AthleteRepository()                  | 110 |
| 7.18.3 Member Function Documentation          | 110 |
| 7.18.3.1 addFtpEntry()                        | 110 |
| 7.18.3.2 addWeightEntry()                     | 111 |
| 7.18.3.3 deleteAthlete()                      | 111 |
| 7.18.3.4 deleteFtpEntry()                     | 111 |
| 7.18.3.5 deleteWeightEntry()                  | 112 |
| 7.18.3.6 getAthlete()                         | 112 |
| 7.18.3.7 getFtpForDate()                      | 113 |
| 7.18.3.8 getFtpHistory()                      | 113 |
| 7.18.3.9 getWeightHistory()                   | 113 |

---

|                                               |     |
|-----------------------------------------------|-----|
| 7.18.3.10 insertAthlete()                     | 114 |
| 7.18.3.11 listAthletes()                      | 114 |
| 7.18.3.12 updateAthlete()                     | 114 |
| 7.18.4 Member Data Documentation              | 114 |
| 7.18.4.1 m_db                                 | 114 |
| 7.19 BasePage Class Reference                 | 115 |
| 7.19.1 Detailed Description                   | 116 |
| 7.19.1.1 Adding a new feature as a new page   | 116 |
| 7.19.2 Constructor & Destructor Documentation | 116 |
| 7.19.2.1 BasePage()                           | 116 |
| 7.19.3 Member Function Documentation          | 116 |
| 7.19.3.1 onActivated()                        | 116 |
| 7.20 CalendarPage Class Reference             | 116 |
| 7.20.1 Detailed Description                   | 117 |
| 7.20.2 Constructor & Destructor Documentation | 118 |
| 7.20.2.1 CalendarPage()                       | 118 |
| 7.21 CalendarWidget Class Reference           | 118 |
| 7.21.1 Detailed Description                   | 119 |
| 7.21.2 Constructor & Destructor Documentation | 120 |
| 7.21.2.1 CalendarWidget()                     | 120 |
| 7.21.3 Member Function Documentation          | 120 |
| 7.21.3.1 addPlannedWorkout                    | 120 |
| 7.21.3.2 deleteSelectedWorkout                | 120 |
| 7.21.3.3 editSelectedWorkout                  | 120 |
| 7.21.3.4 onDateChanged                        | 120 |
| 7.21.3.5 recolorMonth                         | 120 |
| 7.21.3.6 refresh()                            | 120 |
| 7.21.3.7 reloadDayList()                      | 120 |
| 7.21.3.8 setAthleteId()                       | 120 |
| 7.21.3.9 setDatabaseManager()                 | 120 |
| 7.21.3.10 updateMonthFormatting()             | 121 |
| 7.21.4 Member Data Documentation              | 121 |
| 7.21.4.1 m_addButton                          | 121 |
| 7.21.4.2 m_athleteId                          | 121 |
| 7.21.4.3 m_calendar                           | 121 |
| 7.21.4.4 m_colorByCombo                       | 121 |
| 7.21.4.5 m_dayList                            | 121 |
| 7.21.4.6 m_dbManager                          | 121 |
| 7.21.4.7 m_deleteButton                       | 121 |
| 7.21.4.8 m_editButton                         | 121 |
| 7.22 ChartController Class Reference          | 121 |
| 7.22.1 Detailed Description                   | 125 |

---



|                                               |     |
|-----------------------------------------------|-----|
| 7.22.2 Constructor & Destructor Documentation | 125 |
| 7.22.2.1 ChartController()                    | 125 |
| 7.22.3 Member Function Documentation          | 125 |
| 7.22.3.1 applyChartSmoothing()                | 125 |
| 7.22.3.2 applyColorMetric()                   | 125 |
| 7.22.3.3 connectChartSignals()                | 126 |
| 7.22.3.4 currentColorMetric()                 | 126 |
| 7.22.3.5 handleWorkoutLoaded()                | 127 |
| 7.22.3.6 refreshCharts()                      | 127 |
| 7.22.3.7 resetAllZoom()                       | 128 |
| 7.22.3.8 setAnalysisTabWidgets()              | 128 |
| 7.22.3.9 setAthleteId()                       | 128 |
| 7.22.3.10 setChartControls()                  | 128 |
| 7.22.3.11 setChartWidgets()                   | 128 |
| 7.22.3.12 setClimbCharts()                    | 129 |
| 7.22.3.13 setClimbingTabStack()               | 129 |
| 7.22.3.14 setClimbs()                         | 129 |
| 7.22.3.15 setColorContext()                   | 129 |
| 7.22.3.16 setColorMetric()                    | 130 |
| 7.22.3.17 setIntervals()                      | 130 |
| 7.22.3.18 setZonesTable()                     | 130 |
| 7.22.3.19 syncChartCrosshair()                | 130 |
| 7.22.3.20 syncChartZoom()                     | 131 |
| 7.22.3.21 updateAnalysisEmptyStates()         | 131 |
| 7.22.3.22 updateCharts()                      | 131 |
| 7.22.3.23 updateColorLegend()                 | 132 |
| 7.22.3.24 updateColorLegendDisplay()          | 133 |
| 7.22.3.25 updateFitnessChart()                | 134 |
| 7.22.3.26 updateHistogram()                   | 134 |
| 7.22.3.27 updatePowerCurve()                  | 134 |
| 7.22.3.28 updateZoneAvailability()            | 135 |
| 7.22.3.29 updateZonesTab()                    | 136 |
| 7.22.4 Member Data Documentation              | 136 |
| 7.22.4.1 m_activityTabStack                   | 136 |
| 7.22.4.2 m_altitudeChart                      | 137 |
| 7.22.4.3 m_analysisTabWidget                  | 137 |
| 7.22.4.4 m_autoSmoothingCheck                 | 137 |
| 7.22.4.5 m_cadenceChart                       | 137 |
| 7.22.4.6 m_calendarTabStack                   | 137 |
| 7.22.4.7 m_climbAltitudeChart                 | 137 |
| 7.22.4.8 m_climbCadenceChart                  | 137 |
| 7.22.4.9 m_climbHrChart                       | 137 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.22.4.10 m_climbingTabStack                  | 137 |
| 7.22.4.11 m_climbOverlayEnabledCheck          | 137 |
| 7.22.4.12 m_climbOverlayMetricCombo           | 137 |
| 7.22.4.13 m_climbPowerChart                   | 137 |
| 7.22.4.14 m_climbs                            | 138 |
| 7.22.4.15 m_climbSpeedChart                   | 138 |
| 7.22.4.16 m_colorContext                      | 138 |
| 7.22.4.17 m_colorLegendLayout                 | 138 |
| 7.22.4.18 m_colorMetric                       | 138 |
| 7.22.4.19 m_colorMetricCombo                  | 138 |
| 7.22.4.20 m_controller                        | 138 |
| 7.22.4.21 m_currentAthleteld                  | 138 |
| 7.22.4.22 m_dbManager                         | 138 |
| 7.22.4.23 m_fitnessChart                      | 138 |
| 7.22.4.24 m_fitnessTabStack                   | 138 |
| 7.22.4.25 m_histogram                         | 138 |
| 7.22.4.26 m_histogramTabStack                 | 139 |
| 7.22.4.27 m_hrChart                           | 139 |
| 7.22.4.28 m_intervals                         | 139 |
| 7.22.4.29 m_powerChart                        | 139 |
| 7.22.4.30 m_powerCurve                        | 139 |
| 7.22.4.31 m_powerCurveTabStack                | 139 |
| 7.22.4.32 m_powerSmoothingCombo               | 139 |
| 7.22.4.33 m_smoothingAmount                   | 139 |
| 7.22.4.34 m_smoothingEnabled                  | 139 |
| 7.22.4.35 m_speedChart                        | 139 |
| 7.22.4.36 m_zonesTable                        | 139 |
| 7.22.4.37 m_zonesTabStack                     | 139 |
| 7.23 ChartsPage Class Reference               | 140 |
| 7.23.1 Detailed Description                   | 141 |
| 7.23.2 Constructor & Destructor Documentation | 141 |
| 7.23.2.1 ChartsPage()                         | 141 |
| 7.24 Climb Struct Reference                   | 141 |
| 7.24.1 Detailed Description                   | 143 |
| 7.24.2 Member Data Documentation              | 143 |
| 7.24.2.1 aerobicDecouplingPct                 | 143 |
| 7.24.2.2 averageCadence                       | 143 |
| 7.24.2.3 averageGradient                      | 143 |
| 7.24.2.4 averageHeartRate                     | 143 |
| 7.24.2.5 averagePower                         | 143 |
| 7.24.2.6 averageSpeed                         | 143 |
| 7.24.2.7 averageWattsPerKg                    | 144 |

|                                                  |     |
|--------------------------------------------------|-----|
| 7.24.2.8 category                                | 144 |
| 7.24.2.9 climbCategory                           | 144 |
| 7.24.2.10 climbScore                             | 144 |
| 7.24.2.11 difficultyScore                        | 144 |
| 7.24.2.12 durationSeconds                        | 144 |
| 7.24.2.13 elevationGainM                         | 144 |
| 7.24.2.14 endDistanceKm                          | 144 |
| 7.24.2.15 endElevationM                          | 144 |
| 7.24.2.16 endSeconds                             | 145 |
| 7.24.2.17 gradePercentile                        | 145 |
| 7.24.2.18 hrDriftPct                             | 145 |
| 7.24.2.19 id                                     | 145 |
| 7.24.2.20 lengthKm                               | 145 |
| 7.24.2.21 maximumGradient                        | 145 |
| 7.24.2.22 name                                   | 145 |
| 7.24.2.23 normalizedPower                        | 145 |
| 7.24.2.24 powerFadePct                           | 146 |
| 7.24.2.25 powerFirstHalf                         | 146 |
| 7.24.2.26 powerSecondHalf                        | 146 |
| 7.24.2.27 quarterMetrics                         | 146 |
| 7.24.2.28 shapeClass                             | 146 |
| 7.24.2.29 startDistanceKm                        | 146 |
| 7.24.2.30 startElevationM                        | 146 |
| 7.24.2.31 startSeconds                           | 146 |
| 7.24.2.32 vam                                    | 147 |
| 7.24.2.33 wattsPerKgRank                         | 147 |
| 7.25 ClimbDetector Class Reference               | 147 |
| 7.25.1 Detailed Description                      | 148 |
| 7.25.2 Member Typedef Documentation              | 148 |
| 7.25.2.1 Config                                  | 148 |
| 7.25.3 Member Function Documentation             | 148 |
| 7.25.3.1 buildCumulativeDistanceMeters()         | 148 |
| 7.25.3.2 categorizeClimb()                       | 148 |
| 7.25.3.3 computeAdaptiveSmoothingWindowsMeters() | 149 |
| 7.25.3.4 computeClimbScore()                     | 149 |
| 7.25.3.5 computeLocalGradientPct()               | 150 |
| 7.25.3.6 computeVAM()                            | 151 |
| 7.25.3.7 detect()                                | 151 |
| 7.25.3.8 mergeFragmentedClimbs()                 | 152 |
| 7.25.3.9 smoothAltitudeByDistance()              | 153 |
| 7.26 ClimbDetectorConfig Struct Reference        | 154 |
| 7.26.1 Detailed Description                      | 155 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.26.2 Member Data Documentation              | 155 |
| 7.26.2.1 adaptiveSmoothingMaxMeters           | 155 |
| 7.26.2.2 adaptiveSmoothingMinMeters           | 155 |
| 7.26.2.3 altitudeSpikeVerticalSpeedMps        | 155 |
| 7.26.2.4 continueGradient                     | 156 |
| 7.26.2.5 durationScoreWeight                  | 156 |
| 7.26.2.6 elevationSmoothingMeters             | 156 |
| 7.26.2.7 enableClimbMerging                   | 156 |
| 7.26.2.8 endGradient                          | 156 |
| 7.26.2.9 gainScoreWeight                      | 156 |
| 7.26.2.10 gradePersistenceMinMeters           | 156 |
| 7.26.2.11 gradePersistenceThreshold           | 156 |
| 7.26.2.12 gradeScoreWeight                    | 156 |
| 7.26.2.13 gradientWindowHalfMeters            | 157 |
| 7.26.2.14 maxDipDistanceMeters                | 157 |
| 7.26.2.15 maxDipMeters                        | 157 |
| 7.26.2.16 maxDipPctOfGain                     | 157 |
| 7.26.2.17 maxPlateauDistanceMeters            | 157 |
| 7.26.2.18 maxSmallDipsCount                   | 157 |
| 7.26.2.19 maxStartInterruptionMeters          | 157 |
| 7.26.2.20 mergeMaxGapElevationM               | 157 |
| 7.26.2.21 mergeMaxGapMeters                   | 157 |
| 7.26.2.22 minAverageGradient                  | 158 |
| 7.26.2.23 minClimbScore                       | 158 |
| 7.26.2.24 minElevationGainM                   | 158 |
| 7.26.2.25 minLengthKm                         | 158 |
| 7.26.2.26 minStartContinuousDistanceMeters    | 158 |
| 7.26.2.27 plateauMinGradient                  | 158 |
| 7.26.2.28 requireGradePersistence             | 158 |
| 7.26.2.29 smallDipThresholdMeters             | 158 |
| 7.26.2.30 startGradient                       | 158 |
| 7.26.2.31 useAdaptiveSmoothing                | 159 |
| 7.26.2.32 useScoringDetection                 | 159 |
| 7.26.2.33 vamScoreBoost                       | 159 |
| 7.27 ClimbEditCommand Class Reference         | 159 |
| 7.27.1 Detailed Description                   | 160 |
| 7.27.2 Constructor & Destructor Documentation | 160 |
| 7.27.2.1 ClimbEditCommand()                   | 160 |
| 7.27.3 Member Function Documentation          | 161 |
| 7.27.3.1 apply()                              | 161 |
| 7.27.3.2 description()                        | 161 |
| 7.27.3.3 redo()                               | 161 |

|                                                                                               |     |
|-----------------------------------------------------------------------------------------------|-----|
| 7.27.3.4 undo()                                                                               | 161 |
| 7.27.4 Member Data Documentation                                                              | 161 |
| 7.27.4.1 m_after                                                                              | 161 |
| 7.27.4.2 m_before                                                                             | 161 |
| 7.27.4.3 m_connName                                                                           | 162 |
| 7.27.4.4 m_refresh                                                                            | 162 |
| 7.28 RideChartSelectionManager::ClimbHandleHitResult Struct Reference                         | 162 |
| 7.28.1 Detailed Description                                                                   | 162 |
| 7.28.2 Member Data Documentation                                                              | 162 |
| 7.28.2.1 hit                                                                                  | 162 |
| 7.28.2.2 startHandle                                                                          | 162 |
| 7.29 ClimbMetrics Class Reference                                                             | 162 |
| 7.29.1 Detailed Description                                                                   | 162 |
| 7.29.2 Member Function Documentation                                                          | 163 |
| 7.29.2.1 build()                                                                              | 163 |
| 7.30 anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem Class Reference | 164 |
| 7.30.1 Detailed Description                                                                   | 164 |
| 7.30.2 Constructor & Destructor Documentation                                                 | 164 |
| 7.30.2.1 ClimbNameTableItem()                                                                 | 164 |
| 7.30.3 Member Function Documentation                                                          | 165 |
| 7.30.3.1 operator<()                                                                          | 165 |
| 7.30.3.2 trailingNumber()                                                                     | 165 |
| 7.31 anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem Class Reference        | 165 |
| 7.31.1 Detailed Description                                                                   | 166 |
| 7.31.2 Constructor & Destructor Documentation                                                 | 166 |
| 7.31.2.1 ClimbNameTableItem()                                                                 | 166 |
| 7.31.3 Member Function Documentation                                                          | 166 |
| 7.31.3.1 operator<()                                                                          | 166 |
| 7.31.3.2 trailingNumber()                                                                     | 166 |
| 7.32 ClimbQuarterMetrics Struct Reference                                                     | 167 |
| 7.32.1 Detailed Description                                                                   | 167 |
| 7.32.2 Member Data Documentation                                                              | 167 |
| 7.32.2.1 averageCadence                                                                       | 167 |
| 7.32.2.2 averageHeartRate                                                                     | 167 |
| 7.32.2.3 averagePower                                                                         | 167 |
| 7.33 ClimbRecord Struct Reference                                                             | 167 |
| 7.33.1 Detailed Description                                                                   | 168 |
| 7.33.2 Member Data Documentation                                                              | 168 |
| 7.33.2.1 activityId                                                                           | 168 |
| 7.33.2.2 algorithmVersion                                                                     | 169 |
| 7.33.2.3 averageGradient                                                                      | 169 |
| 7.33.2.4 avgCadence                                                                           | 169 |

|                                                                                                  |     |
|--------------------------------------------------------------------------------------------------|-----|
| 7.33.2.5 avgHr                                                                                   | 169 |
| 7.33.2.6 avgPower                                                                                | 169 |
| 7.33.2.7 deleted                                                                                 | 169 |
| 7.33.2.8 elevationGainM                                                                          | 169 |
| 7.33.2.9 endSeconds                                                                              | 169 |
| 7.33.2.10 favorite                                                                               | 169 |
| 7.33.2.11 id                                                                                     | 170 |
| 7.33.2.12 lengthKm                                                                               | 170 |
| 7.33.2.13 locked                                                                                 | 170 |
| 7.33.2.14 name                                                                                   | 170 |
| 7.33.2.15 notes                                                                                  | 170 |
| 7.33.2.16 np                                                                                     | 170 |
| 7.33.2.17 originalEndSeconds                                                                     | 170 |
| 7.33.2.18 originalStartSeconds                                                                   | 170 |
| 7.33.2.19 rating                                                                                 | 171 |
| 7.33.2.20 source                                                                                 | 171 |
| 7.33.2.21 startSeconds                                                                           | 171 |
| 7.33.2.22 uuid                                                                                   | 171 |
| 7.33.2.23 vam                                                                                    | 171 |
| 7.34 ClimbRepository Class Reference                                                             | 171 |
| 7.34.1 Detailed Description                                                                      | 172 |
| 7.34.2 Constructor & Destructor Documentation                                                    | 172 |
| 7.34.2.1 ClimbRepository()                                                                       | 172 |
| 7.34.3 Member Function Documentation                                                             | 172 |
| 7.34.3.1 climbsForActivity()                                                                     | 172 |
| 7.34.3.2 deleteClimbsForActivity()                                                               | 173 |
| 7.34.3.3 deletedClimbsForActivity()                                                              | 173 |
| 7.34.3.4 getClimb()                                                                              | 173 |
| 7.34.3.5 hardDeleteClimb()                                                                       | 174 |
| 7.34.3.6 hasClimbs()                                                                             | 174 |
| 7.34.3.7 hasClimbsEver()                                                                         | 174 |
| 7.34.3.8 hasLockedClimbs()                                                                       | 175 |
| 7.34.3.9 insertClimb()                                                                           | 176 |
| 7.34.3.10 softDeleteClimb()                                                                      | 177 |
| 7.34.3.11 updateClimb()                                                                          | 177 |
| 7.34.4 Member Data Documentation                                                                 | 178 |
| 7.34.4.1 m_db                                                                                    | 178 |
| 7.35 anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem Class Reference | 178 |
| 7.35.1 Detailed Description                                                                      | 179 |
| 7.35.2 Constructor & Destructor Documentation                                                    | 179 |
| 7.35.2.1 ClimbSortKeyTableItem()                                                                 | 179 |

|                                                                                           |     |
|-------------------------------------------------------------------------------------------|-----|
| 7.35.3 Member Function Documentation                                                      | 179 |
| 7.35.3.1 operator<()                                                                      | 179 |
| 7.35.4 Member Data Documentation                                                          | 179 |
| 7.35.4.1 m_sortKey                                                                        | 179 |
| 7.36 anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem Class Reference | 180 |
| 7.36.1 Detailed Description                                                               | 180 |
| 7.36.2 Constructor & Destructor Documentation                                             | 180 |
| 7.36.2.1 ClimbSortKeyTableItem()                                                          | 180 |
| 7.36.3 Member Function Documentation                                                      | 181 |
| 7.36.3.1 operator<()                                                                      | 181 |
| 7.36.4 Member Data Documentation                                                          | 181 |
| 7.36.4.1 m_sortKey                                                                        | 181 |
| 7.37 ClimbsPage Class Reference                                                           | 181 |
| 7.37.1 Detailed Description                                                               | 182 |
| 7.37.2 Constructor & Destructor Documentation                                             | 183 |
| 7.37.2.1 ClimbsPage()                                                                     | 183 |
| 7.37.3 Member Data Documentation                                                          | 183 |
| 7.37.3.1 m_toolbar                                                                        | 183 |
| 7.38 ColorContext Struct Reference                                                        | 183 |
| 7.38.1 Detailed Description                                                               | 184 |
| 7.38.2 Member Data Documentation                                                          | 184 |
| 7.38.2.1 altitudeMax                                                                      | 184 |
| 7.38.2.2 altitudeMin                                                                      | 184 |
| 7.38.2.3 ftp                                                                              | 184 |
| 7.38.2.4 hasAltitudeRange                                                                 | 184 |
| 7.38.2.5 heartRateMax                                                                     | 184 |
| 7.38.2.6 heartRateRest                                                                    | 184 |
| 7.38.2.7 thresholdHeartRate                                                               | 184 |
| 7.39 ColorProvider Class Reference                                                        | 185 |
| 7.39.1 Detailed Description                                                               | 185 |
| 7.39.2 Member Function Documentation                                                      | 185 |
| 7.39.2.1 colorForAltitude()                                                               | 185 |
| 7.39.2.2 colorForCadence()                                                                | 185 |
| 7.39.2.3 colorForHeartRate()                                                              | 185 |
| 7.39.2.4 colorForMetric()                                                                 | 185 |
| 7.39.2.5 colorForPower()                                                                  | 186 |
| 7.39.2.6 colorForSpeed()                                                                  | 186 |
| 7.39.2.7 colorForValue()                                                                  | 186 |
| 7.40 CreateDatabaseDialog Class Reference                                                 | 186 |
| 7.40.1 Detailed Description                                                               | 187 |
| 7.40.2 Constructor & Destructor Documentation                                             | 187 |
| 7.40.2.1 CreateDatabaseDialog()                                                           | 187 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.40.3 Member Function Documentation          | 188 |
| 7.40.3.1 accept                               | 188 |
| 7.40.3.2 browseLocation                       | 188 |
| 7.40.3.3 databasePath()                       | 188 |
| 7.40.3.4 defaultAthleteFirstName()            | 188 |
| 7.40.3.5 defaultAthleteLastName()             | 188 |
| 7.40.3.6 normalizedDatabaseName()             | 188 |
| 7.40.3.7 shouldCreateDefaultAthlete()         | 188 |
| 7.40.3.8 updateAthleteFields                  | 188 |
| 7.40.4 Member Data Documentation              | 188 |
| 7.40.4.1 m_browseButton                       | 188 |
| 7.40.4.2 m_buttonBox                          | 188 |
| 7.40.4.3 m_createAthleteCheck                 | 188 |
| 7.40.4.4 m_firstNameEdit                      | 189 |
| 7.40.4.5 m_lastNameEdit                       | 189 |
| 7.40.4.6 m_locationEdit                       | 189 |
| 7.40.4.7 m_nameEdit                           | 189 |
| 7.41 DailyLoadPoint Struct Reference          | 189 |
| 7.41.1 Detailed Description                   | 189 |
| 7.41.2 Member Data Documentation              | 189 |
| 7.41.2.1 tss                                  | 189 |
| 7.42 DatabaseConnectionGuard Class Reference  | 189 |
| 7.42.1 Detailed Description                   | 190 |
| 7.42.2 Constructor & Destructor Documentation | 190 |
| 7.42.2.1 DatabaseConnectionGuard()            | 190 |
| 7.42.2.2 ~DatabaseConnectionGuard()           | 190 |
| 7.42.3 Member Function Documentation          | 190 |
| 7.42.3.1 database()                           | 190 |
| 7.42.4 Member Data Documentation              | 191 |
| 7.42.4.1 m_connectionName                     | 191 |
| 7.43 DatabaseManager Class Reference          | 191 |
| 7.43.1 Detailed Description                   | 192 |
| 7.43.2 Constructor & Destructor Documentation | 192 |
| 7.43.2.1 ~DatabaseManager()                   | 192 |
| 7.43.3 Member Function Documentation          | 192 |
| 7.43.3.1 applySchema()                        | 192 |
| 7.43.3.2 cleanupFailedOpen()                  | 192 |
| 7.43.3.3 close()                              | 192 |
| 7.43.3.4 create()                             | 192 |
| 7.43.3.5 database()                           | 193 |
| 7.43.3.6 isOpen()                             | 193 |
| 7.43.3.7 open()                               | 193 |



|                                               |     |
|-----------------------------------------------|-----|
| 7.43.3.8 path()                               | 194 |
| 7.43.3.9 upgradeSchema()                      | 194 |
| 7.43.4 Member Data Documentation              | 195 |
| 7.43.4.1 m_connectionName                     | 195 |
| 7.43.4.2 m_db                                 | 195 |
| 7.43.4.3 m_path                               | 195 |
| 7.44 DateFormatter Class Reference            | 195 |
| 7.44.1 Detailed Description                   | 195 |
| 7.44.2 Member Function Documentation          | 195 |
| 7.44.2.1 formatDate()                         | 195 |
| 7.44.2.2 formatDateTime()                     | 196 |
| 7.44.2.3 parseDate()                          | 197 |
| 7.44.2.4 qtDatePattern()                      | 197 |
| 7.44.2.5 toIsoDate()                          | 198 |
| 7.44.2.6 toIsoDateTime()                      | 198 |
| 7.45 DuplicateActivityInfo Struct Reference   | 199 |
| 7.45.1 Detailed Description                   | 199 |
| 7.45.2 Member Data Documentation              | 199 |
| 7.45.2.1 existingActivityId                   | 199 |
| 7.45.2.2 existingDurationSeconds              | 199 |
| 7.45.2.3 existingStartUtc                     | 199 |
| 7.45.2.4 newDurationSeconds                   | 199 |
| 7.45.2.5 newStartUtc                          | 199 |
| 7.46 EditActivityDialog Class Reference       | 199 |
| 7.46.1 Detailed Description                   | 200 |
| 7.46.2 Constructor & Destructor Documentation | 201 |
| 7.46.2.1 EditActivityDialog()                 | 201 |
| 7.46.3 Member Function Documentation          | 201 |
| 7.46.3.1 updatedActivity()                    | 201 |
| 7.46.4 Member Data Documentation              | 201 |
| 7.46.4.1 m_bikeEdit                           | 201 |
| 7.46.4.2 m_equipmentEdit                      | 201 |
| 7.46.4.3 m_fatigueSpin                        | 201 |
| 7.46.4.4 m_initial                            | 201 |
| 7.46.4.5 m_notesEdit                          | 201 |
| 7.46.4.6 m_rpeSpin                            | 201 |
| 7.46.4.7 m_sleepSpin                          | 201 |
| 7.46.4.8 m_temperatureSpin                    | 201 |
| 7.46.4.9 m_weatherEdit                        | 201 |
| 7.46.4.10 m_weightSpin                        | 202 |
| 7.46.4.11 m_windEdit                          | 202 |
| 7.47 EditNotesDialog Class Reference          | 202 |

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| 7.47.1 Detailed Description                                                          | 203 |
| 7.47.2 Constructor & Destructor Documentation                                        | 203 |
| 7.47.2.1 EditNotesDialog()                                                           | 203 |
| 7.47.3 Member Function Documentation                                                 | 203 |
| 7.47.3.1 notes()                                                                     | 203 |
| 7.47.3.2 weatherNotes()                                                              | 203 |
| 7.47.4 Member Data Documentation                                                     | 203 |
| 7.47.4.1 m_notes                                                                     | 203 |
| 7.47.4.2 m_weather                                                                   | 203 |
| 7.48 anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard Class Reference | 204 |
| 7.48.1 Detailed Description                                                          | 204 |
| 7.48.2 Constructor & Destructor Documentation                                        | 204 |
| 7.48.2.1 ExportTileCleanupGuard()                                                    | 204 |
| 7.48.2.2 ~ExportTileCleanupGuard()                                                   | 204 |
| 7.48.3 Member Data Documentation                                                     | 205 |
| 7.48.3.1 m_provider                                                                  | 205 |
| 7.49 anonymous_namespace{FitParser.cpp}::FitFileIdListener Class Reference           | 205 |
| 7.49.1 Detailed Description                                                          | 206 |
| 7.49.2 Member Function Documentation                                                 | 206 |
| 7.49.2.1 OnMesg()                                                                    | 206 |
| 7.49.2.2 timeCreated()                                                               | 206 |
| 7.49.3 Member Data Documentation                                                     | 206 |
| 7.49.3.1 m_timeCreated                                                               | 206 |
| 7.50 FitnessChartWidget Class Reference                                              | 206 |
| 7.50.1 Detailed Description                                                          | 207 |
| 7.50.2 Constructor & Destructor Documentation                                        | 207 |
| 7.50.2.1 FitnessChartWidget()                                                        | 207 |
| 7.50.3 Member Function Documentation                                                 | 208 |
| 7.50.3.1 clearChart()                                                                | 208 |
| 7.50.3.2 setTimeline()                                                               | 208 |
| 7.50.4 Member Data Documentation                                                     | 208 |
| 7.50.4.1 m_atlSeries                                                                 | 208 |
| 7.50.4.2 m_axisX                                                                     | 208 |
| 7.50.4.3 m_axisY                                                                     | 208 |
| 7.50.4.4 m_ctlSeries                                                                 | 208 |
| 7.50.4.5 m_tsbSeries                                                                 | 208 |
| 7.51 FitnessMetricsPoint Struct Reference                                            | 208 |
| 7.51.1 Detailed Description                                                          | 209 |
| 7.51.2 Member Data Documentation                                                     | 209 |
| 7.51.2.1 atl                                                                         | 209 |
| 7.51.2.2 ctl                                                                         | 209 |
| 7.51.2.3 tsb                                                                         | 209 |

|                                                                                       |     |
|---------------------------------------------------------------------------------------|-----|
| 7.52 FitnessPage Class Reference . . . . .                                            | 209 |
| 7.52.1 Detailed Description . . . . .                                                 | 210 |
| 7.52.2 Constructor & Destructor Documentation . . . . .                               | 211 |
| 7.52.2.1 FitnessPage() . . . . .                                                      | 211 |
| 7.53 FitParser Class Reference . . . . .                                              | 211 |
| 7.53.1 Detailed Description . . . . .                                                 | 211 |
| 7.53.2 Member Function Documentation . . . . .                                        | 211 |
| 7.53.2.1 load() . . . . .                                                             | 211 |
| 7.54 FitRecordListener Class Reference . . . . .                                      | 212 |
| 7.54.1 Detailed Description . . . . .                                                 | 213 |
| 7.54.2 Constructor & Destructor Documentation . . . . .                               | 214 |
| 7.54.2.1 FitRecordListener() . . . . .                                                | 214 |
| 7.54.3 Member Function Documentation . . . . .                                        | 214 |
| 7.54.3.1 firstTimestamp() . . . . .                                                   | 214 |
| 7.54.3.2 lastTimestamp() . . . . .                                                    | 214 |
| 7.54.3.3 OnMesg() . . . . .                                                           | 215 |
| 7.54.4 Member Data Documentation . . . . .                                            | 215 |
| 7.54.4.1 m_firstTimestamp . . . . .                                                   | 215 |
| 7.54.4.2 m_lastTimestamp . . . . .                                                    | 215 |
| 7.54.4.3 m_rideData . . . . .                                                         | 215 |
| 7.55 MapFitMath::FitResult Struct Reference . . . . .                                 | 215 |
| 7.55.1 Detailed Description . . . . .                                                 | 216 |
| 7.55.2 Member Data Documentation . . . . .                                            | 216 |
| 7.55.2.1 centerLat . . . . .                                                          | 216 |
| 7.55.2.2 centerLon . . . . .                                                          | 216 |
| 7.55.2.3 lonSpanDegrees . . . . .                                                     | 216 |
| 7.55.2.4 maxLat . . . . .                                                             | 216 |
| 7.55.2.5 maxLon . . . . .                                                             | 216 |
| 7.55.2.6 minLat . . . . .                                                             | 216 |
| 7.55.2.7 minLon . . . . .                                                             | 216 |
| 7.55.2.8 zoom . . . . .                                                               | 216 |
| 7.56 anonymous_namespace{FitParser.cpp}::FitSessionListener Class Reference . . . . . | 217 |
| 7.56.1 Detailed Description . . . . .                                                 | 217 |
| 7.56.2 Member Function Documentation . . . . .                                        | 217 |
| 7.56.2.1 OnMesg() . . . . .                                                           | 217 |
| 7.56.2.2 startTime() . . . . .                                                        | 218 |
| 7.56.3 Member Data Documentation . . . . .                                            | 218 |
| 7.56.3.1 m_startTime . . . . .                                                        | 218 |
| 7.57 FtpEntry Struct Reference . . . . .                                              | 218 |
| 7.57.1 Detailed Description . . . . .                                                 | 218 |
| 7.57.2 Member Data Documentation . . . . .                                            | 218 |
| 7.57.2.1 athleteId . . . . .                                                          | 218 |

|                                                      |     |
|------------------------------------------------------|-----|
| 7.57.2.2 effectiveFrom                               | 218 |
| 7.57.2.3 ftpWatts                                    | 219 |
| 7.57.2.4 id                                          | 219 |
| 7.57.2.5 notes                                       | 219 |
| 7.58 FTPZones Class Reference                        | 219 |
| 7.58.1 Detailed Description                          | 219 |
| 7.58.2 Member Typedef Documentation                  | 219 |
| 7.58.2.1 ZoneArray                                   | 219 |
| 7.58.3 Member Function Documentation                 | 220 |
| 7.58.3.1 compute()                                   | 220 |
| 7.58.3.2 defaultZones()                              | 220 |
| 7.58.4 Member Data Documentation                     | 221 |
| 7.58.4.1 kZoneCount                                  | 221 |
| 7.59 GpsCacheData Struct Reference                   | 221 |
| 7.59.1 Detailed Description                          | 221 |
| 7.59.2 Member Data Documentation                     | 221 |
| 7.59.2.1 firstRecord                                 | 221 |
| 7.59.2.2 lastRecord                                  | 221 |
| 7.59.2.3 records                                     | 222 |
| 7.60 MainWindow::ImportBatchSummary Struct Reference | 222 |
| 7.60.1 Detailed Description                          | 222 |
| 7.60.2 Member Data Documentation                     | 222 |
| 7.60.2.1 duplicates                                  | 222 |
| 7.60.2.2 errors                                      | 222 |
| 7.60.2.3 failed                                      | 222 |
| 7.60.2.4 imported                                    | 222 |
| 7.60.2.5 queued                                      | 222 |
| 7.60.2.6 showResultDialog                            | 222 |
| 7.60.2.7 sourceLabel                                 | 223 |
| 7.61 ImportController Class Reference                | 223 |
| 7.61.1 Detailed Description                          | 225 |
| 7.61.2 Constructor & Destructor Documentation        | 225 |
| 7.61.2.1 ImportController()                          | 225 |
| 7.61.3 Member Function Documentation                 | 225 |
| 7.61.3.1 canImportActivities()                       | 225 |
| 7.61.3.2 finalizeBatches()                           | 226 |
| 7.61.3.3 importQueue()                               | 226 |
| 7.61.3.4 onActivityImportComplete()                  | 226 |
| 7.61.3.5 progressModel()                             | 227 |
| 7.61.3.6 queueFilesForImport()                       | 227 |
| 7.61.3.7 setImportQueue()                            | 228 |
| 7.61.3.8 setProgressModel()                          | 228 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.61.3.9 setStatusWidget()                    | 229 |
| 7.61.3.10 updateImportAvailability()          | 229 |
| 7.61.4 Member Data Documentation              | 229 |
| 7.61.4.1 m_controller                         | 229 |
| 7.61.4.2 m_dbManager                          | 229 |
| 7.61.4.3 m_importQueue                        | 230 |
| 7.61.4.4 m_progressModel                      | 230 |
| 7.61.4.5 m_statusWidget                       | 230 |
| 7.62 ImportJob Struct Reference               | 230 |
| 7.62.1 Detailed Description                   | 230 |
| 7.62.2 Member Enumeration Documentation       | 231 |
| 7.62.2.1 State                                | 231 |
| 7.62.3 Member Data Documentation              | 231 |
| 7.62.3.1 athleteId                            | 231 |
| 7.62.3.2 batchId                              | 231 |
| 7.62.3.3 errorMessage                         | 231 |
| 7.62.3.4 filePath                             | 231 |
| 7.62.3.5 importSource                         | 231 |
| 7.62.3.6 jobId                                | 231 |
| 7.62.3.7 progress                             | 232 |
| 7.62.3.8 state                                | 232 |
| 7.63 ImportProgressModel Class Reference      | 232 |
| 7.63.1 Detailed Description                   | 233 |
| 7.63.2 Constructor & Destructor Documentation | 233 |
| 7.63.2.1 ImportProgressModel()                | 233 |
| 7.63.3 Member Function Documentation          | 233 |
| 7.63.3.1 activeJobs()                         | 233 |
| 7.63.3.2 attachQueue()                        | 233 |
| 7.63.3.3 completedJobs()                      | 234 |
| 7.63.3.4 currentFileName()                    | 234 |
| 7.63.3.5 failedJobs()                         | 234 |
| 7.63.3.6 progress()                           | 234 |
| 7.63.3.7 statusText()                         | 234 |
| 7.63.4 Member Data Documentation              | 234 |
| 7.63.4.1 m_activeJobs                         | 234 |
| 7.63.4.2 m_completedJobs                      | 235 |
| 7.63.4.3 m_currentFileName                    | 235 |
| 7.63.4.4 m_failedJobs                         | 235 |
| 7.63.4.5 m_progress                           | 235 |
| 7.63.4.6 m_statusText                         | 235 |
| 7.64 ImportQueue Class Reference              | 235 |
| 7.64.1 Detailed Description                   | 237 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.64.2 Constructor & Destructor Documentation | 237 |
| 7.64.2.1 ImportQueue()                        | 237 |
| 7.64.2.2 ~ImportQueue()                       | 237 |
| 7.64.3 Member Function Documentation          | 237 |
| 7.64.3.1 activeCountUnsafe()                  | 237 |
| 7.64.3.2 enqueue()                            | 237 |
| 7.64.3.3 isBusy()                             | 238 |
| 7.64.3.4 jobFailed                            | 238 |
| 7.64.3.5 onWorkerCompleted                    | 239 |
| 7.64.3.6 onWorkerFailed                       | 239 |
| 7.64.3.7 onWorkerProgress                     | 239 |
| 7.64.3.8 pendingCount()                       | 239 |
| 7.64.3.9 processNext()                        | 239 |
| 7.64.3.10 setDatabasePath()                   | 239 |
| 7.64.4 Member Data Documentation              | 240 |
| 7.64.4.1 m_busy                               | 240 |
| 7.64.4.2 m_dbPath                             | 240 |
| 7.64.4.3 m_queue                              | 240 |
| 7.64.4.4 m_runningJob                         | 240 |
| 7.64.4.5 m_thread                             | 240 |
| 7.64.4.6 m_worker                             | 240 |
| 7.65 ImportRecord Struct Reference            | 240 |
| 7.65.1 Detailed Description                   | 241 |
| 7.65.2 Member Data Documentation              | 241 |
| 7.65.2.1 activityId                           | 241 |
| 7.65.2.2 errorMessage                         | 241 |
| 7.65.2.3 fileName                             | 241 |
| 7.65.2.4 id                                   | 241 |
| 7.65.2.5 importedAt                           | 241 |
| 7.65.2.6 status                               | 241 |
| 7.66 ImportRepository Class Reference         | 241 |
| 7.66.1 Detailed Description                   | 242 |
| 7.66.2 Constructor & Destructor Documentation | 242 |
| 7.66.2.1 ImportRepository()                   | 242 |
| 7.66.3 Member Function Documentation          | 242 |
| 7.66.3.1 getRecentImports()                   | 242 |
| 7.66.3.2 logImport()                          | 242 |
| 7.66.4 Member Data Documentation              | 243 |
| 7.66.4.1 m_db                                 | 243 |
| 7.67 ImportStatusWidget Class Reference       | 243 |
| 7.67.1 Detailed Description                   | 244 |
| 7.67.2 Constructor & Destructor Documentation | 244 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.67.2.1 ImportStatusWidget()                 | 244 |
| 7.67.3 Member Function Documentation          | 244 |
| 7.67.3.1 setModel()                           | 244 |
| 7.67.3.2 updateFromModel()                    | 245 |
| 7.67.4 Member Data Documentation              | 245 |
| 7.67.4.1 m_detailLabel                        | 245 |
| 7.67.4.2 m_model                              | 245 |
| 7.67.4.3 m_progressBar                        | 245 |
| 7.67.4.4 m_titleLabel                         | 245 |
| 7.68 ImportWorker Class Reference             | 245 |
| 7.68.1 Detailed Description                   | 247 |
| 7.68.2 Constructor & Destructor Documentation | 247 |
| 7.68.2.1 ImportWorker()                       | 247 |
| 7.68.3 Member Function Documentation          | 247 |
| 7.68.3.1 failed                               | 247 |
| 7.68.3.2 processJob                           | 247 |
| 7.68.3.3 setDatabasePath()                    | 248 |
| 7.68.4 Member Data Documentation              | 248 |
| 7.68.4.1 m_dbPath                             | 248 |
| 7.69 Interval Struct Reference                | 249 |
| 7.69.1 Detailed Description                   | 249 |
| 7.69.2 Member Data Documentation              | 249 |
| 7.69.2.1 averageCadence                       | 249 |
| 7.69.2.2 averageHeartRate                     | 249 |
| 7.69.2.3 averagePower                         | 249 |
| 7.69.2.4 averageSpeed                         | 250 |
| 7.69.2.5 durationSeconds                      | 250 |
| 7.69.2.6 endSeconds                           | 250 |
| 7.69.2.7 label                                | 250 |
| 7.69.2.8 maximumPower                         | 250 |
| 7.69.2.9 normalizedPower                      | 250 |
| 7.69.2.10 startSeconds                        | 250 |
| 7.70 IntervalDetector Class Reference         | 250 |
| 7.70.1 Detailed Description                   | 251 |
| 7.70.2 Member Typedef Documentation           | 251 |
| 7.70.2.1 Config                               | 251 |
| 7.70.3 Member Function Documentation          | 251 |
| 7.70.3.1 buildInterval()                      | 251 |
| 7.70.3.2 detect()                             | 251 |
| 7.70.3.3 removeOverlappingAndDuplicates()     | 252 |
| 7.70.3.4 smoothPower()                        | 252 |
| 7.71 IntervalDetectorConfig Struct Reference  | 252 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.71.1 Detailed Description                   | 253 |
| 7.71.2 Member Data Documentation              | 253 |
| 7.71.2.1 ftp                                  | 253 |
| 7.71.2.2 minRecoverySeconds                   | 253 |
| 7.71.2.3 minWorkSeconds                       | 253 |
| 7.71.2.4 smoothingWindow                      | 253 |
| 7.71.2.5 workThresholdPct                     | 253 |
| 7.72 IntervalEditorDialog Class Reference     | 253 |
| 7.72.1 Detailed Description                   | 254 |
| 7.72.2 Constructor & Destructor Documentation | 255 |
| 7.72.2.1 IntervalEditorDialog()               | 255 |
| 7.72.3 Member Function Documentation          | 255 |
| 7.72.3.1 intervalName()                       | 255 |
| 7.72.3.2 intervalNotes()                      | 255 |
| 7.72.3.3 intervalType()                       | 255 |
| 7.72.4 Member Data Documentation              | 255 |
| 7.72.4.1 m_endSpin                            | 255 |
| 7.72.4.2 m_nameEdit                           | 255 |
| 7.72.4.3 m_notesEdit                          | 256 |
| 7.72.4.4 m_startSpin                          | 256 |
| 7.72.4.5 m_typeEdit                           | 256 |
| 7.73 IntervalRecord Struct Reference          | 256 |
| 7.73.1 Detailed Description                   | 257 |
| 7.73.2 Member Data Documentation              | 257 |
| 7.73.2.1 activityId                           | 257 |
| 7.73.2.2 algorithmVersion                     | 257 |
| 7.73.2.3 avgCadence                           | 257 |
| 7.73.2.4 avgHr                                | 257 |
| 7.73.2.5 avgPower                             | 257 |
| 7.73.2.6 deleted                              | 257 |
| 7.73.2.7 endSample                            | 258 |
| 7.73.2.8 id                                   | 258 |
| 7.73.2.9 locked                               | 258 |
| 7.73.2.10 name                                | 258 |
| 7.73.2.11 notes                               | 258 |
| 7.73.2.12 np                                  | 258 |
| 7.73.2.13 source                              | 258 |
| 7.73.2.14 startSample                         | 258 |
| 7.73.2.15 type                                | 259 |
| 7.73.2.16 uuid                                | 259 |
| 7.74 IntervalRepository Class Reference       | 259 |
| 7.74.1 Detailed Description                   | 259 |



|                                               |     |
|-----------------------------------------------|-----|
| 7.74.2 Constructor & Destructor Documentation | 260 |
| 7.74.2.1 IntervalRepository()                 | 260 |
| 7.74.3 Member Function Documentation          | 260 |
| 7.74.3.1 deleteInterval()                     | 260 |
| 7.74.3.2 deleteIntervalsForActivity()         | 260 |
| 7.74.3.3 hasLockedIntervals()                 | 260 |
| 7.74.3.4 insertInterval()                     | 261 |
| 7.74.3.5 listIntervalsForActivity()           | 262 |
| 7.74.3.6 softDeleteInterval()                 | 262 |
| 7.74.3.7 updateInterval()                     | 262 |
| 7.74.4 Member Data Documentation              | 263 |
| 7.74.4.1 m_db                                 | 263 |
| 7.75 IntervalsPage Class Reference            | 263 |
| 7.75.1 Detailed Description                   | 264 |
| 7.75.2 Constructor & Destructor Documentation | 264 |
| 7.75.2.1 IntervalsPage()                      | 264 |
| 7.76 MainWindow Class Reference               | 265 |
| 7.76.1 Detailed Description                   | 271 |
| 7.76.2 Constructor & Destructor Documentation | 271 |
| 7.76.2.1 MainWindow()                         | 271 |
| 7.76.3 Member Function Documentation          | 272 |
| 7.76.3.1 activityCount()                      | 272 |
| 7.76.3.2 addRecentDatabase()                  | 272 |
| 7.76.3.3 applyChartAndMapUpdates()            | 273 |
| 7.76.3.4 applyChartControlDrivenUpdates()     | 273 |
| 7.76.3.5 applyChartHeight()                   | 273 |
| 7.76.3.6 applyChartPreset()                   | 274 |
| 7.76.3.7 applyDetectedClimbsToCharts()        | 274 |
| 7.76.3.8 applyEstimatedFtpForCurrentAthlete() | 274 |
| 7.76.3.9 applyWeightMetricsToClimbs()         | 274 |
| 7.76.3.10 assignClimbWeightMetrics()          | 274 |
| 7.76.3.11 backupDatabase                      | 274 |
| 7.76.3.12 buildColorContext()                 | 274 |
| 7.76.3.13 buildStatusBar()                    | 274 |
| 7.76.3.14 buildToolBar()                      | 274 |
| 7.76.3.15 buildUI()                           | 274 |
| 7.76.3.16 canImportActivities()               | 274 |
| 7.76.3.17 clearIntervalSummary()              | 275 |
| 7.76.3.18 closeEvent()                        | 275 |
| 7.76.3.19 configureFolderWatcher()            | 275 |
| 7.76.3.20 createDatabase                      | 275 |
| 7.76.3.21 createOrOpenDefaultDatabase()       | 275 |

|                                                 |     |
|-------------------------------------------------|-----|
| 7.76.3.22 createVideo                           | 275 |
| 7.76.3.23 currentColorMetric()                  | 275 |
| 7.76.3.24 decreaseChartHeight()                 | 275 |
| 7.76.3.25 defaultDatabasePathForAutoCreate()    | 275 |
| 7.76.3.26 detectClimbsAndRefresh()              | 276 |
| 7.76.3.27 dragEnterEvent()                      | 276 |
| 7.76.3.28 dropEvent()                           | 276 |
| 7.76.3.29 editCurrentActivityProperties()       | 276 |
| 7.76.3.30 editSelectedClimbBoundaries()         | 276 |
| 7.76.3.31 enqueueImportBatch()                  | 276 |
| 7.76.3.32 ensureAthlete()                       | 277 |
| 7.76.3.33 ensureDatabase()                      | 277 |
| 7.76.3.34 ensureImportReady()                   | 277 |
| 7.76.3.35 estimatedFtpFromCurrentRide()         | 277 |
| 7.76.3.36 eventFilter()                         | 277 |
| 7.76.3.37 finalizeImportBatches()               | 277 |
| 7.76.3.38 finalizeIntervalSelectionAndSummary() | 277 |
| 7.76.3.39 fitFilesFromMimeData()                | 278 |
| 7.76.3.40 hideWelcomeScreen()                   | 278 |
| 7.76.3.41 importActivities                      | 278 |
| 7.76.3.42 importFiles()                         | 278 |
| 7.76.3.43 importFilesInternal()                 | 279 |
| 7.76.3.44 importQueuedStatusMessage()           | 279 |
| 7.76.3.45 increaseChartHeight()                 | 279 |
| 7.76.3.46 joinSelectedClimbs()                  | 279 |
| 7.76.3.47 loadSettings()                        | 280 |
| 7.76.3.48 manageAthletes                        | 280 |
| 7.76.3.49 monitoredDirectories()                | 280 |
| 7.76.3.50 navigateToInterval()                  | 280 |
| 7.76.3.51 onActivityImported                    | 280 |
| 7.76.3.52 onAthleteSelectionChanged             | 280 |
| 7.76.3.53 onClimbBoundaryEdited()               | 280 |
| 7.76.3.54 onClimbRowDoubleClicked()             | 280 |
| 7.76.3.55 onClimbSelectionChanged()             | 280 |
| 7.76.3.56 onClimbTableContextMenuRequested()    | 280 |
| 7.76.3.57 onIntervalRowDoubleClicked()          | 281 |
| 7.76.3.58 onIntervalSelectionChanged()          | 281 |
| 7.76.3.59 onNewClimbRequested()                 | 281 |
| 7.76.3.60 onWatchDirectoryChanged()             | 281 |
| 7.76.3.61 onWorkoutLoaded                       | 281 |
| 7.76.3.62 openDatabase                          | 281 |
| 7.76.3.63 openDatabasePath()                    | 281 |

|                                                    |     |
|----------------------------------------------------|-----|
| 7.76.3.64 openRecentDatabase                       | 281 |
| 7.76.3.65 openSettingsDialog                       | 281 |
| 7.76.3.66 prepareImportFileBatch()                 | 281 |
| 7.76.3.67 previewFitFile                           | 281 |
| 7.76.3.68 refreshAthleteSelector()                 | 282 |
| 7.76.3.69 refreshClimbViews()                      | 282 |
| 7.76.3.70 refreshMapRideDataFromCurrentState()     | 282 |
| 7.76.3.71 removeSelectedClimbs()                   | 282 |
| 7.76.3.72 resetAllZoom()                           | 282 |
| 7.76.3.73 resolveActiveActivityWeightKg()          | 282 |
| 7.76.3.74 restoreClimbSelection()                  | 282 |
| 7.76.3.75 revertSelectedClimbToAuto                | 282 |
| 7.76.3.76 saveSettings()                           | 283 |
| 7.76.3.77 scanWatchDirectories()                   | 283 |
| 7.76.3.78 scheduleActivityBrowserRefresh()         | 283 |
| 7.76.3.79 selectedClimbIndicesFromTable()          | 283 |
| 7.76.3.80 selectIntervalRowOffset()                | 283 |
| 7.76.3.81 setupShortcuts()                         | 283 |
| 7.76.3.82 showAbout                                | 283 |
| 7.76.3.83 showWelcomeScreen()                      | 283 |
| 7.76.3.84 syncAthleteContextToControllers()        | 283 |
| 7.76.3.85 syncChartContextToController()           | 283 |
| 7.76.3.86 triggerDetectClimbs                      | 284 |
| 7.76.3.87 triggerDetectIntervals                   | 284 |
| 7.76.3.88 updateActivityNotesFromCurrentActivity() | 284 |
| 7.76.3.89 updateAthleteLabel()                     | 284 |
| 7.76.3.90 updateChartAnalysisEmptyStates()         | 284 |
| 7.76.3.91 updateClimbingTab()                      | 284 |
| 7.76.3.92 updateClimbQuarterCharts()               | 285 |
| 7.76.3.93 updateClimbRowStyles()                   | 285 |
| 7.76.3.94 updateImportAvailability()               | 285 |
| 7.76.3.95 updateIntervalRowStyles()                | 285 |
| 7.76.3.96 updateIntervals()                        | 285 |
| 7.76.3.97 updateIntervalSummary()                  | 285 |
| 7.76.3.98 updateLapsSummaryFromCurrentActivity()   | 286 |
| 7.76.3.99 updateRecentDatabaseMenu()               | 286 |
| 7.76.3.100 updateStatsLabel()                      | 286 |
| 7.76.3.101 updateStatusBarInfo()                   | 286 |
| 7.76.3.102 updateWelcomeScreenVisibility()         | 286 |
| 7.76.4 Member Data Documentation                   | 286 |
| 7.76.4.1 m_actionsManager                          | 286 |
| 7.76.4.2 m_activitiesPage                          | 286 |

|                                       |     |
|---------------------------------------|-----|
| 7.76.4.3 m_activityBrowser            | 286 |
| 7.76.4.4 m_activityCountLabel         | 286 |
| 7.76.4.5 m_activityNotesView          | 286 |
| 7.76.4.6 m_activityTabStack           | 286 |
| 7.76.4.7 m_activityViewController     | 286 |
| 7.76.4.8 m_altitudeChart              | 287 |
| 7.76.4.9 m_analysisQueue              | 287 |
| 7.76.4.10 m_analysisTabWidget         | 287 |
| 7.76.4.11 m_athleteCombo              | 287 |
| 7.76.4.12 m_athleteHeader             | 287 |
| 7.76.4.13 m_athleteLabelAct           | 287 |
| 7.76.4.14 m_athletesMenu              | 287 |
| 7.76.4.15 m_athleteStatusLabel        | 287 |
| 7.76.4.16 m_autoSmoothingCheck        | 287 |
| 7.76.4.17 m_cadenceChart              | 287 |
| 7.76.4.18 m_calendarPageContent       | 287 |
| 7.76.4.19 m_calendarTabStack          | 287 |
| 7.76.4.20 m_calendarWidget            | 288 |
| 7.76.4.21 m_chartController           | 288 |
| 7.76.4.22 m_chartHeight               | 288 |
| 7.76.4.23 m_chartHeightDecreaseButton | 288 |
| 7.76.4.24 m_chartHeightIncreaseButton | 288 |
| 7.76.4.25 m_chartMapSplit             | 288 |
| 7.76.4.26 m_chartPresetCombo          | 288 |
| 7.76.4.27 m_chartsContainer           | 288 |
| 7.76.4.28 m_chartScroll               | 288 |
| 7.76.4.29 m_chartsLayout              | 288 |
| 7.76.4.30 m_chartsPageContent         | 288 |
| 7.76.4.31 m_climbAltitudeChart        | 288 |
| 7.76.4.32 m_climbCadenceChart         | 289 |
| 7.76.4.33 m_climbDipDistanceSpin      | 289 |
| 7.76.4.34 m_climbDipMetersSpin        | 289 |
| 7.76.4.35 m_climbHrChart              | 289 |
| 7.76.4.36 m_climbingTabStack          | 289 |
| 7.76.4.37 m_climbMinGainSpin          | 289 |
| 7.76.4.38 m_climbMinGradientSpin      | 289 |
| 7.76.4.39 m_climbMinLengthSpin        | 289 |
| 7.76.4.40 m_climbOverlayEnabledCheck  | 289 |
| 7.76.4.41 m_climbOverlayMetricCombo   | 289 |
| 7.76.4.42 m_climbPowerChart           | 289 |
| 7.76.4.43 m_climbQuarterCadenceChart  | 289 |
| 7.76.4.44 m_climbQuarterHrChart       | 290 |

|                                                    |     |
|----------------------------------------------------|-----|
| 7.76.4.45 m_climbQuarterPowerChart . . . . .       | 290 |
| 7.76.4.46 m_climbSmoothingSpin . . . . .           | 290 |
| 7.76.4.47 m_climbsPageContent . . . . .            | 290 |
| 7.76.4.48 m_climbSpeedChart . . . . .              | 290 |
| 7.76.4.49 m_climbsTable . . . . .                  | 290 |
| 7.76.4.50 m_climbStartGradientSpin . . . . .       | 290 |
| 7.76.4.51 m_climbSummaryLabel . . . . .            | 290 |
| 7.76.4.52 m_colorLegendLayout . . . . .            | 290 |
| 7.76.4.53 m_colorLegendWidget . . . . .            | 290 |
| 7.76.4.54 m_colorMetricCombo . . . . .             | 290 |
| 7.76.4.55 m_controller . . . . .                   | 290 |
| 7.76.4.56 m_currentAthleteId . . . . .             | 291 |
| 7.76.4.57 m_currentAthleteName . . . . .           | 291 |
| 7.76.4.58 m_dbManager . . . . .                    | 291 |
| 7.76.4.59 m_dbStatusLabel . . . . .                | 291 |
| 7.76.4.60 m_detectedClimbs . . . . .               | 291 |
| 7.76.4.61 m_firstLaunchCompleted . . . . .         | 291 |
| 7.76.4.62 m_fitChartsButton . . . . .              | 291 |
| 7.76.4.63 m_fitnessChart . . . . .                 | 291 |
| 7.76.4.64 m_fitnessPageContent . . . . .           | 291 |
| 7.76.4.65 m_fitnessTabStack . . . . .              | 291 |
| 7.76.4.66 m_followIntervalSelectionCheck . . . . . | 291 |
| 7.76.4.67 m_histogram . . . . .                    | 291 |
| 7.76.4.68 m_histogramPageContent . . . . .         | 292 |
| 7.76.4.69 m_histogramTabStack . . . . .            | 292 |
| 7.76.4.70 m_hrChart . . . . .                      | 292 |
| 7.76.4.71 m_importAct . . . . .                    | 292 |
| 7.76.4.72 m_importBatchSummaries . . . . .         | 292 |
| 7.76.4.73 m_importController . . . . .             | 292 |
| 7.76.4.74 m_importProgressModel . . . . .          | 292 |
| 7.76.4.75 m_importQueue . . . . .                  | 292 |
| 7.76.4.76 m_importRefreshTimer . . . . .           | 292 |
| 7.76.4.77 m_importStatusWidget . . . . .           | 292 |
| 7.76.4.78 m_intervalsPageContent . . . . .         | 292 |
| 7.76.4.79 m_intervalsTable . . . . .               | 292 |
| 7.76.4.80 m_intervalSummaryLabel . . . . .         | 293 |
| 7.76.4.81 m_knownWatchedFitFiles . . . . .         | 293 |
| 7.76.4.82 m_lapsTable . . . . .                    | 293 |
| 7.76.4.83 m_lastOpenDir . . . . .                  | 293 |
| 7.76.4.84 m_mapAutoContrastCheck . . . . .         | 293 |
| 7.76.4.85 m_mapController . . . . .                | 293 |
| 7.76.4.86 m_mapRenderer . . . . .                  | 293 |

|                                                         |     |
|---------------------------------------------------------|-----|
| 7.76.4.87 m_mapStyleCombo . . . . .                     | 293 |
| 7.76.4.88 m_navigationController . . . . .              | 293 |
| 7.76.4.89 m_navigationSidebar . . . . .                 | 293 |
| 7.76.4.90 m_pageStack . . . . .                         | 293 |
| 7.76.4.91 m_pdcPageContent . . . . .                    | 293 |
| 7.76.4.92 m_pdcWidget . . . . .                         | 294 |
| 7.76.4.93 m_powerChart . . . . .                        | 294 |
| 7.76.4.94 m_powerCurveTabStack . . . . .                | 294 |
| 7.76.4.95 m_powerSmoothingCombo . . . . .               | 294 |
| 7.76.4.96 m_previewAct . . . . .                        | 294 |
| 7.76.4.97 m_recentDbMenu . . . . .                      | 294 |
| 7.76.4.98 m_showAltitude . . . . .                      | 294 |
| 7.76.4.99 m_showCadence . . . . .                       | 294 |
| 7.76.4.100 m_showHR . . . . .                           | 294 |
| 7.76.4.101 m_showPower . . . . .                        | 294 |
| 7.76.4.102 m_showSpeed . . . . .                        | 294 |
| 7.76.4.103 m_speedChart . . . . .                       | 294 |
| 7.76.4.104 m_suppressClimbAutoZoom . . . . .            | 295 |
| 7.76.4.105 m_toolbarImportAct . . . . .                 | 295 |
| 7.76.4.106 m_toolbarSearchAct . . . . .                 | 295 |
| 7.76.4.107 m_undoManager . . . . .                      | 295 |
| 7.76.4.108 m_useEstimatedFtpButton . . . . .            | 295 |
| 7.76.4.109 m_watcher . . . . .                          | 295 |
| 7.76.4.110 m_watchFolderEnabled . . . . .               | 295 |
| 7.76.4.111 m_watchFolderPath . . . . .                  | 295 |
| 7.76.4.112 m_watchRescanTimer . . . . .                 | 295 |
| 7.76.4.113 m_welcomeWidget . . . . .                    | 295 |
| 7.76.4.114 m_zonesPageContent . . . . .                 | 295 |
| 7.76.4.115 m_zonesTable . . . . .                       | 295 |
| 7.76.4.116 m_zonesTabStack . . . . .                    | 296 |
| 7.77 MainWindowActions Class Reference . . . . .        | 296 |
| 7.77.1 Detailed Description . . . . .                   | 298 |
| 7.77.2 Constructor & Destructor Documentation . . . . . | 299 |
| 7.77.2.1 MainWindowActions() . . . . .                  | 299 |
| 7.77.3 Member Function Documentation . . . . .          | 299 |
| 7.77.3.1 athleteCombo() . . . . .                       | 299 |
| 7.77.3.2 buildMenus() . . . . .                         | 299 |
| 7.77.3.3 buildToolbar() . . . . .                       | 300 |
| 7.77.3.4 createAnalysisMenu() . . . . .                 | 300 |
| 7.77.3.5 createEditMenu() . . . . .                     | 301 |
| 7.77.3.6 createFileMenu() . . . . .                     | 301 |
| 7.77.3.7 createHelpMenu() . . . . .                     | 301 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.77.3.8 createToolsMenu()                    | 302 |
| 7.77.3.9 createVideoAction()                  | 302 |
| 7.77.3.10 createViewMenu()                    | 302 |
| 7.77.3.11 importAction()                      | 302 |
| 7.77.3.12 previewFitAction()                  | 302 |
| 7.77.3.13 recentDatabaseMenu()                | 302 |
| 7.77.3.14 setupShortcuts()                    | 303 |
| 7.77.3.15 toolbarImportAction()               | 303 |
| 7.77.4 Member Data Documentation              | 303 |
| 7.77.4.1 m_aboutAct                           | 303 |
| 7.77.4.2 m_athleteCombo                       | 303 |
| 7.77.4.3 m_backupDatabaseAct                  | 303 |
| 7.77.4.4 m_createDatabaseAct                  | 303 |
| 7.77.4.5 m_createVideoAct                     | 303 |
| 7.77.4.6 m_detectClimbsAct                    | 303 |
| 7.77.4.7 m_detectIntervalsAct                 | 303 |
| 7.77.4.8 m_exitAct                            | 303 |
| 7.77.4.9 m_fullscreenAct                      | 303 |
| 7.77.4.10 m_importAct                         | 304 |
| 7.77.4.11 m_mainWindow                        | 304 |
| 7.77.4.12 m_manageAthletesAct                 | 304 |
| 7.77.4.13 m_openDatabaseAct                   | 304 |
| 7.77.4.14 m_openSettingsAct                   | 304 |
| 7.77.4.15 m_previewFitAct                     | 304 |
| 7.77.4.16 m_recentDbMenu                      | 304 |
| 7.77.4.17 m_revertClimbAct                    | 304 |
| 7.77.4.18 m_toolbarImportAct                  | 304 |
| 7.77.4.19 m_toolbarSearchAct                  | 304 |
| 7.78 MapController Class Reference            | 304 |
| 7.78.1 Detailed Description                   | 306 |
| 7.78.2 Constructor & Destructor Documentation | 307 |
| 7.78.2.1 MapController()                      | 307 |
| 7.78.3 Member Function Documentation          | 307 |
| 7.78.3.1 applyColorMetric()                   | 307 |
| 7.78.3.2 connectMapSignals()                  | 307 |
| 7.78.3.3 fitToTrack()                         | 308 |
| 7.78.3.4 fitToVisibleRange()                  | 308 |
| 7.78.3.5 highlightSegment()                   | 308 |
| 7.78.3.6 mapRenderer()                        | 308 |
| 7.78.3.7 refreshMap()                         | 308 |
| 7.78.3.8 setAutoRouteContrast()               | 309 |
| 7.78.3.9 setCurrentTime()                     | 309 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.78.3.10 setMapRenderer()                    | 309 |
| 7.78.3.11 setVisibleTimeRange()               | 310 |
| 7.78.3.12 updateMap()                         | 310 |
| 7.78.3.13 updateRouteColoring()               | 310 |
| 7.78.3.14 zoomIn()                            | 311 |
| 7.78.3.15 zoomOut()                           | 311 |
| 7.78.4 Member Data Documentation              | 311 |
| 7.78.4.1 m_controller                         | 311 |
| 7.78.4.2 m_dbManager                          | 311 |
| 7.78.4.3 m_mapRenderer                        | 311 |
| 7.79 MapRenderer Class Reference              | 311 |
| 7.79.1 Detailed Description                   | 314 |
| 7.79.2 Member Enumeration Documentation       | 314 |
| 7.79.2.1 DragHandle                           | 314 |
| 7.79.3 Constructor & Destructor Documentation | 314 |
| 7.79.3.1 MapRenderer()                        | 314 |
| 7.79.4 Member Function Documentation          | 314 |
| 7.79.4.1 adjustRouteColorForStyle()           | 314 |
| 7.79.4.2 autoRouteContrast()                  | 314 |
| 7.79.4.3 fitToBounds()                        | 314 |
| 7.79.4.4 fitToTrack                           | 314 |
| 7.79.4.5 fitToVisibleRange                    | 315 |
| 7.79.4.6 gpsRecordAtTime()                    | 315 |
| 7.79.4.7 latLonToTile()                       | 315 |
| 7.79.4.8 mapStyle()                           | 315 |
| 7.79.4.9 mouseDoubleClickEvent()              | 315 |
| 7.79.4.10 mouseMoveEvent()                    | 315 |
| 7.79.4.11 mousePressEvent()                   | 315 |
| 7.79.4.12 mouseReleaseEvent()                 | 315 |
| 7.79.4.13 nearestGpsRecord()                  | 315 |
| 7.79.4.14 paintEvent()                        | 315 |
| 7.79.4.15 rebuildGpsCache()                   | 316 |
| 7.79.4.16 rebuildSegmentColors()              | 316 |
| 7.79.4.17 setAutoRouteContrast()              | 316 |
| 7.79.4.18 setCurrentTime()                    | 316 |
| 7.79.4.19 setHighlightedElapsedSeconds        | 316 |
| 7.79.4.20 setMapStyle()                       | 316 |
| 7.79.4.21 setRideData()                       | 316 |
| 7.79.4.22 setRouteColorMode()                 | 316 |
| 7.79.4.23 setTileCacheSize()                  | 316 |
| 7.79.4.24 setVisibleTimeRange()               | 316 |
| 7.79.4.25 tileToScreen()                      | 317 |



|                                               |     |
|-----------------------------------------------|-----|
| 7.79.4.26 wheelEvent()                        | 317 |
| 7.79.4.27 zoomIn                              | 317 |
| 7.79.4.28 zoomOut                             | 317 |
| 7.79.5 Member Data Documentation              | 317 |
| 7.79.5.1 m_autoFitVisibleRange                | 317 |
| 7.79.5.2 m_autoRouteContrast                  | 317 |
| 7.79.5.3 m_centerLat                          | 317 |
| 7.79.5.4 m_centerLon                          | 317 |
| 7.79.5.5 m_colorContext                       | 317 |
| 7.79.5.6 m_currentTimeSeconds                 | 317 |
| 7.79.5.7 m_draggingSelection                  | 317 |
| 7.79.5.8 m_dragHandle                         | 318 |
| 7.79.5.9 m_endMarkerRect                      | 318 |
| 7.79.5.10 m_firstGpsRecord                    | 318 |
| 7.79.5.11 m_gpsRecords                        | 318 |
| 7.79.5.12 m_hasGps                            | 318 |
| 7.79.5.13 m_highlightElapsedSeconds           | 318 |
| 7.79.5.14 m_lastGpsRecord                     | 318 |
| 7.79.5.15 m_maxLat                            | 318 |
| 7.79.5.16 m_maxLon                            | 318 |
| 7.79.5.17 m_minLat                            | 318 |
| 7.79.5.18 m_minLon                            | 318 |
| 7.79.5.19 m_minZoom                           | 318 |
| 7.79.5.20 m_panning                           | 319 |
| 7.79.5.21 m_panStart                          | 319 |
| 7.79.5.22 m_rideData                          | 319 |
| 7.79.5.23 m_routeColorMode                    | 319 |
| 7.79.5.24 m_segmentColors                     | 319 |
| 7.79.5.25 m_startMarkerRect                   | 319 |
| 7.79.5.26 m_tileCache                         | 319 |
| 7.79.5.27 m_tileRepaintTimer                  | 319 |
| 7.79.5.28 m_userMovedMap                      | 319 |
| 7.79.5.29 m_visibleEndSeconds                 | 319 |
| 7.79.5.30 m_visibleStartSeconds               | 319 |
| 7.79.5.31 m_zoom                              | 319 |
| 7.80 NavigationController Class Reference     | 320 |
| 7.80.1 Detailed Description                   | 321 |
| 7.80.2 Member Typedef Documentation           | 321 |
| 7.80.2.1 Page                                 | 321 |
| 7.80.3 Constructor & Destructor Documentation | 321 |
| 7.80.3.1 NavigationController()               | 321 |
| 7.80.4 Member Function Documentation          | 322 |

|          |                                        |     |
|----------|----------------------------------------|-----|
| 7.80.4.1 | currentPage()                          | 322 |
| 7.80.4.2 | navigateTo()                           | 322 |
| 7.80.4.3 | restoreNavigationState()               | 323 |
| 7.80.4.4 | saveNavigationState()                  | 323 |
| 7.80.4.5 | setNavigationSidebar()                 | 323 |
| 7.80.4.6 | setPageStack()                         | 324 |
| 7.80.5   | Member Data Documentation              | 324 |
| 7.80.5.1 | m_currentPage                          | 324 |
| 7.80.5.2 | m_pageStack                            | 324 |
| 7.80.5.3 | m_sidebar                              | 324 |
| 7.81     | NavigationSidebar Class Reference      | 325 |
| 7.81.1   | Detailed Description                   | 326 |
| 7.81.2   | Member Enumeration Documentation       | 326 |
| 7.81.2.1 | Page                                   | 326 |
| 7.81.3   | Constructor & Destructor Documentation | 326 |
| 7.81.3.1 | NavigationSidebar()                    | 326 |
| 7.81.4   | Member Function Documentation          | 326 |
| 7.81.4.1 | currentPage()                          | 326 |
| 7.81.4.2 | setCurrentPage()                       | 326 |
| 7.81.5   | Member Data Documentation              | 327 |
| 7.81.5.1 | m_list                                 | 327 |
| 7.82     | PlaceholderPage Class Reference        | 327 |
| 7.82.1   | Detailed Description                   | 328 |
| 7.82.2   | Constructor & Destructor Documentation | 328 |
| 7.82.2.1 | PlaceholderPage()                      | 328 |
| 7.83     | PlannedWorkoutDialog Class Reference   | 328 |
| 7.83.1   | Detailed Description                   | 329 |
| 7.83.2   | Constructor & Destructor Documentation | 329 |
| 7.83.2.1 | PlannedWorkoutDialog() [1/2]           | 329 |
| 7.83.2.2 | PlannedWorkoutDialog() [2/2]           | 329 |
| 7.83.3   | Member Function Documentation          | 329 |
| 7.83.3.1 | draft()                                | 329 |
| 7.83.4   | Member Data Documentation              | 330 |
| 7.83.4.1 | m_dateEdit                             | 330 |
| 7.83.4.2 | m_durationSpin                         | 330 |
| 7.83.4.3 | m_id                                   | 330 |
| 7.83.4.4 | m_notesEdit                            | 330 |
| 7.83.4.5 | m_titleEdit                            | 330 |
| 7.83.4.6 | m_tssSpin                              | 330 |
| 7.83.4.7 | m_typeEdit                             | 330 |
| 7.84     | PlannedWorkoutDraft Struct Reference   | 330 |
| 7.84.1   | Detailed Description                   | 330 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.84.2 Member Data Documentation              | 331 |
| 7.84.2.1 date                                 | 331 |
| 7.84.2.2 durationSeconds                      | 331 |
| 7.84.2.3 id                                   | 331 |
| 7.84.2.4 notes                                | 331 |
| 7.84.2.5 targetTss                            | 331 |
| 7.84.2.6 title                                | 331 |
| 7.84.2.7 type                                 | 331 |
| 7.85 PowerCurve Class Reference               | 331 |
| 7.85.1 Detailed Description                   | 331 |
| 7.85.2 Member Function Documentation          | 332 |
| 7.85.2.1 bestMeanPower()                      | 332 |
| 7.85.2.2 computeStandard()                    | 332 |
| 7.85.2.3 standardDurations()                  | 333 |
| 7.86 PowerCurvePoint Struct Reference         | 334 |
| 7.86.1 Detailed Description                   | 334 |
| 7.86.2 Member Data Documentation              | 334 |
| 7.86.2.1 bestPower                            | 334 |
| 7.86.2.2 durationSeconds                      | 334 |
| 7.87 PowerCurveWidget Class Reference         | 334 |
| 7.87.1 Detailed Description                   | 335 |
| 7.87.2 Constructor & Destructor Documentation | 335 |
| 7.87.2.1 PowerCurveWidget()                   | 335 |
| 7.87.3 Member Function Documentation          | 335 |
| 7.87.3.1 clearChart()                         | 335 |
| 7.87.3.2 setRideData()                        | 336 |
| 7.87.3.3 setRideDataWithComparisons()         | 336 |
| 7.88 PowerHistogramWidget Class Reference     | 337 |
| 7.88.1 Detailed Description                   | 338 |
| 7.88.2 Constructor & Destructor Documentation | 338 |
| 7.88.2.1 PowerHistogramWidget()               | 338 |
| 7.88.3 Member Function Documentation          | 338 |
| 7.88.3.1 clearChart()                         | 338 |
| 7.88.3.2 setRideData()                        | 339 |
| 7.89 PowerPage Class Reference                | 339 |
| 7.89.1 Detailed Description                   | 340 |
| 7.89.2 Constructor & Destructor Documentation | 341 |
| 7.89.2.1 PowerPage()                          | 341 |
| 7.90 PowerZone Struct Reference               | 341 |
| 7.90.1 Detailed Description                   | 342 |
| 7.90.2 Member Data Documentation              | 342 |
| 7.90.2.1 lowerPct                             | 342 |

|                                               |     |
|-----------------------------------------------|-----|
| 7.90.2.2 name                                 | 342 |
| 7.90.2.3 number                               | 342 |
| 7.90.2.4 seconds                              | 342 |
| 7.90.2.5 upperPct                             | 342 |
| 7.91 RideChartWidget Class Reference          | 342 |
| 7.91.1 Detailed Description                   | 346 |
| 7.91.2 Member Enumeration Documentation       | 346 |
| 7.91.2.1 ClimbHandleDrag                      | 346 |
| 7.91.3 Constructor & Destructor Documentation | 346 |
| 7.91.3.1 RideChartWidget()                    | 346 |
| 7.91.4 Member Function Documentation          | 347 |
| 7.91.4.1 addClimbBackgroundBands()            | 347 |
| 7.91.4.2 addIntervalBackgroundBands()         | 347 |
| 7.91.4.3 addMetricOverlayBands()              | 347 |
| 7.91.4.4 addZoneBackgroundBands()             | 347 |
| 7.91.4.5 buildSampleIndices()                 | 347 |
| 7.91.4.6 clearBackgroundSeries()              | 347 |
| 7.91.4.7 clearChart()                         | 347 |
| 7.91.4.8 climbBoundaryEdited                  | 347 |
| 7.91.4.9 climbClicked                         | 348 |
| 7.91.4.10 computeSmoothedPowerSeries()        | 348 |
| 7.91.4.11 computeSmoothedSeries()             | 348 |
| 7.91.4.12 crosshairMoved                      | 349 |
| 7.91.4.13 drawForeground()                    | 349 |
| 7.91.4.14 fitToData                           | 349 |
| 7.91.4.15 hasData()                           | 349 |
| 7.91.4.16 intervalSelectionRequested          | 350 |
| 7.91.4.17 leaveEvent()                        | 350 |
| 7.91.4.18 mouseDoubleClickEvent()             | 350 |
| 7.91.4.19 mouseMoveEvent()                    | 351 |
| 7.91.4.20 mousePressEvent()                   | 351 |
| 7.91.4.21 mouseReleaseEvent()                 | 352 |
| 7.91.4.22 newClimbRequested                   | 352 |
| 7.91.4.23 rebuildChart()                      | 352 |
| 7.91.4.24 resetZoom                           | 353 |
| 7.91.4.25 resizeEvent()                       | 353 |
| 7.91.4.26 setAutoSmoothingEnabled()           | 353 |
| 7.91.4.27 setClimbEditingEnabled()            | 353 |
| 7.91.4.28 setClimbs()                         | 354 |
| 7.91.4.29 setCrosshair                        | 354 |
| 7.91.4.30 setIntervals()                      | 354 |
| 7.91.4.31 setMetricOverlay()                  | 354 |

|                                            |     |
|--------------------------------------------|-----|
| 7.91.4.32 setPowerSmoothingSeconds()       | 355 |
| 7.91.4.33 setRideData()                    | 355 |
| 7.91.4.34 setSelectedClimbIndex()          | 355 |
| 7.91.4.35 setXRange                        | 355 |
| 7.91.4.36 updateXTicks()                   | 356 |
| 7.91.4.37 updateYAxis()                    | 356 |
| 7.91.4.38 updateYAxisForVisibleRange()     | 356 |
| 7.91.4.39 visibleRangeChanged              | 356 |
| 7.91.4.40 visibleRangeEndMinutes()         | 357 |
| 7.91.4.41 visibleRangeStartMinutes()       | 357 |
| 7.91.4.42 wheelEvent()                     | 357 |
| 7.91.4.43 xRangeChanged                    | 358 |
| 7.91.5 Member Data Documentation           | 358 |
| 7.91.5.1 cachedAxisMax                     | 358 |
| 7.91.5.2 cachedAxisMin                     | 358 |
| 7.91.5.3 cachedMaxX                        | 358 |
| 7.91.5.4 cachedMinX                        | 358 |
| 7.91.5.5 m_autoSmoothing                   | 358 |
| 7.91.5.6 m_axisX                           | 358 |
| 7.91.5.7 m_axisXDisplay                    | 359 |
| 7.91.5.8 m_axisY                           | 359 |
| 7.91.5.9 m_axisYOverlay                    | 359 |
| 7.91.5.10 m_backgroundSeries               | 359 |
| 7.91.5.11 m_climbEditingEnabled            | 359 |
| 7.91.5.12 m_climbHandleDrag                | 359 |
| 7.91.5.13 m_climbs                         | 359 |
| 7.91.5.14 m_colorContext                   | 359 |
| 7.91.5.15 m_colorMetric                    | 359 |
| 7.91.5.16 m_colorSeries                    | 359 |
| 7.91.5.17 m_crosshairX                     | 359 |
| 7.91.5.18 m_crosshairY                     | 359 |
| 7.91.5.19 m_dataMaxY                       | 360 |
| 7.91.5.20 m_dataMinY                       | 360 |
| 7.91.5.21 m_dragOriginalClimbEndSec        | 360 |
| 7.91.5.22 m_dragOriginalClimbStartSec      | 360 |
| 7.91.5.23 m_effectivePowerSmoothingSeconds | 360 |
| 7.91.5.24 m_hasData                        | 360 |
| 7.91.5.25 m_intervalEndMin                 | 360 |
| 7.91.5.26 m_intervals                      | 360 |
| 7.91.5.27 m_intervalSelecting              | 360 |
| 7.91.5.28 m_intervalStartMin               | 360 |
| 7.91.5.29 m_isRebuilding                   | 360 |

|                                                      |     |
|------------------------------------------------------|-----|
| 7.91.5.30 m_metric . . . . .                         | 360 |
| 7.91.5.31 m_metricOverlayEnabled . . . . .           | 361 |
| 7.91.5.32 m_metricOverlayMetric . . . . .            | 361 |
| 7.91.5.33 m_newClimbCreating . . . . .               | 361 |
| 7.91.5.34 m_newClimbCurrentSec . . . . .             | 361 |
| 7.91.5.35 m_newClimbDragging . . . . .               | 361 |
| 7.91.5.36 m_newClimbStartSec . . . . .               | 361 |
| 7.91.5.37 m_origMaxX . . . . .                       | 361 |
| 7.91.5.38 m_overlaySeries . . . . .                  | 361 |
| 7.91.5.39 m_rawSeries . . . . .                      | 361 |
| 7.91.5.40 m_referenceSeries . . . . .                | 361 |
| 7.91.5.41 m_requestedPowerSmoothingSeconds . . . . . | 361 |
| 7.91.5.42 m_rideData . . . . .                       | 361 |
| 7.91.5.43 m_selectedClimbIndex . . . . .             | 362 |
| 7.91.5.44 m_series . . . . .                         | 362 |
| 7.91.5.45 m_smoothedSeriesCache . . . . .            | 362 |
| 7.91.5.46 m_syncing . . . . .                        | 362 |
| 7.91.5.47 m_tooltipDisplayValues . . . . .           | 362 |
| 7.91.5.48 m_tooltipPoints . . . . .                  | 362 |
| 7.91.5.49 m_tooltipRawValues . . . . .               | 362 |
| 7.91.5.50 valid . . . . .                            | 362 |
| 7.92 RideData Class Reference . . . . .              | 362 |
| 7.92.1 Detailed Description . . . . .                | 363 |
| 7.92.2 Member Data Documentation . . . . .           | 363 |
| 7.92.2.1 activityEndTimeUtc . . . . .                | 363 |
| 7.92.2.2 activityStartTimeUtc . . . . .              | 363 |
| 7.92.2.3 fileTimestampUtc . . . . .                  | 363 |
| 7.92.2.4 records . . . . .                           | 363 |
| 7.93 RideDataLoader Class Reference . . . . .        | 363 |
| 7.93.1 Detailed Description . . . . .                | 363 |
| 7.93.2 Member Function Documentation . . . . .       | 364 |
| 7.93.2.1 loadActivitySamples() . . . . .             | 364 |
| 7.94 RideDataSerializer Class Reference . . . . .    | 364 |
| 7.94.1 Detailed Description . . . . .                | 364 |
| 7.94.2 Member Function Documentation . . . . .       | 364 |
| 7.94.2.1 loadRideFromDatabase() . . . . .            | 364 |
| 7.94.2.2 saveRideToDatabase() . . . . .              | 364 |
| 7.95 RideRecord Struct Reference . . . . .           | 365 |
| 7.95.1 Detailed Description . . . . .                | 365 |
| 7.95.2 Member Data Documentation . . . . .           | 365 |
| 7.95.2.1 altitude . . . . .                          | 365 |
| 7.95.2.2 cadence . . . . .                           | 366 |

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| 7.95.2.3 elapsedSeconds . . . . .                                                 | 366 |
| 7.95.2.4 hasAltitude . . . . .                                                    | 366 |
| 7.95.2.5 hasCadence . . . . .                                                     | 366 |
| 7.95.2.6 hasGps . . . . .                                                         | 366 |
| 7.95.2.7 hasHeartRate . . . . .                                                   | 366 |
| 7.95.2.8 hasPower . . . . .                                                       | 366 |
| 7.95.2.9 hasSpeed . . . . .                                                       | 366 |
| 7.95.2.10 heartRate . . . . .                                                     | 367 |
| 7.95.2.11 latitude . . . . .                                                      | 367 |
| 7.95.2.12 longitude . . . . .                                                     | 367 |
| 7.95.2.13 power . . . . .                                                         | 367 |
| 7.95.2.14 speed . . . . .                                                         | 367 |
| 7.96 anonymous_namespace{ClimbMetrics.cpp}::SliceStats Struct Reference . . . . . | 367 |
| 7.96.1 Detailed Description . . . . .                                             | 367 |
| 7.96.2 Member Data Documentation . . . . .                                        | 368 |
| 7.96.2.1 avgCadence . . . . .                                                     | 368 |
| 7.96.2.2 avgHeartRate . . . . .                                                   | 368 |
| 7.96.2.3 avgPower . . . . .                                                       | 368 |
| 7.97 SmoothingEngine Class Reference . . . . .                                    | 368 |
| 7.97.1 Detailed Description . . . . .                                             | 368 |
| 7.97.2 Member Function Documentation . . . . .                                    | 368 |
| 7.97.2.1 smoothPower() . . . . .                                                  | 368 |
| 7.98 TileCache Class Reference . . . . .                                          | 369 |
| 7.98.1 Detailed Description . . . . .                                             | 370 |
| 7.98.2 Constructor & Destructor Documentation . . . . .                           | 370 |
| 7.98.2.1 TileCache() . . . . .                                                    | 370 |
| 7.98.3 Member Function Documentation . . . . .                                    | 370 |
| 7.98.3.1 attribution() . . . . .                                                  | 370 |
| 7.98.3.2 clearMemoryCache() . . . . .                                             | 371 |
| 7.98.3.3 diskCacheRoot() . . . . .                                                | 371 |
| 7.98.3.4 diskTilePath() . . . . .                                                 | 371 |
| 7.98.3.5 diskTilePathForRoot() [1/2] . . . . .                                    | 371 |
| 7.98.3.6 diskTilePathForRoot() [2/2] . . . . .                                    | 371 |
| 7.98.3.7 dispatchNextDownloads() . . . . .                                        | 371 |
| 7.98.3.8 isTileCachedOnDisk() . . . . .                                           | 371 |
| 7.98.3.9 key() . . . . .                                                          | 371 |
| 7.98.3.10 keyForStyle() . . . . .                                                 | 372 |
| 7.98.3.11 mapStyle() . . . . .                                                    | 372 |
| 7.98.3.12 mapStyleName() . . . . .                                                | 372 |
| 7.98.3.13 maxZoom() . . . . .                                                     | 372 |
| 7.98.3.14 onReplyFinished . . . . .                                               | 372 |
| 7.98.3.15 setMapStyle() . . . . .                                                 | 372 |

|                                                    |     |
|----------------------------------------------------|-----|
| 7.98.3.16 setMemoryCacheSize()                     | 372 |
| 7.98.3.17 tile()                                   | 372 |
| 7.98.3.18 tileBlocking()                           | 372 |
| 7.98.4 Member Data Documentation                   | 373 |
| 7.98.4.1 kMaxConcurrentDownloads                   | 373 |
| 7.98.4.2 m_activeDownloads                         | 373 |
| 7.98.4.3 m_cache                                   | 373 |
| 7.98.4.4 m_diskCacheRoot                           | 373 |
| 7.98.4.5 m_diskFallbackRoots                       | 373 |
| 7.98.4.6 m_downloadQueue                           | 373 |
| 7.98.4.7 m_mapStyle                                | 373 |
| 7.98.4.8 m_nam                                     | 373 |
| 7.98.4.9 m_pending                                 | 373 |
| 7.98.4.10 m_provider                               | 373 |
| 7.99 TileCacheConfig Struct Reference              | 373 |
| 7.99.1 Detailed Description                        | 374 |
| 7.99.2 Member Data Documentation                   | 374 |
| 7.99.2.1 diskCacheRoot                             | 374 |
| 7.99.2.2 diskFallbackRoots                         | 374 |
| 7.99.2.3 maxTilesInMemory                          | 374 |
| 7.100 TileProvider Struct Reference                | 374 |
| 7.100.1 Detailed Description                       | 374 |
| 7.100.2 Member Data Documentation                  | 374 |
| 7.100.2.1 attribution                              | 374 |
| 7.100.2.2 maxZoom                                  | 374 |
| 7.100.2.3 name                                     | 375 |
| 7.100.2.4 urlTemplate                              | 375 |
| 7.101 TileCache::TileRequest Struct Reference      | 375 |
| 7.101.1 Detailed Description                       | 375 |
| 7.101.2 Member Data Documentation                  | 375 |
| 7.101.2.1 x                                        | 375 |
| 7.101.2.2 y                                        | 375 |
| 7.101.2.3 z                                        | 375 |
| 7.102 VideoRenderer::TileViewport Struct Reference | 375 |
| 7.102.1 Detailed Description                       | 376 |
| 7.102.2 Member Function Documentation              | 376 |
| 7.102.2.1 operator==( )                            | 376 |
| 7.102.3 Member Data Documentation                  | 376 |
| 7.102.3.1 nX                                       | 376 |
| 7.102.3.2 nY                                       | 376 |
| 7.102.3.3 tileX0                                   | 376 |
| 7.102.3.4 tileY0                                   | 376 |



|                                                |     |
|------------------------------------------------|-----|
| 7.103 TrainingLoad Class Reference             | 376 |
| 7.103.1 Detailed Description                   | 376 |
| 7.103.2 Member Function Documentation          | 377 |
| 7.103.2.1 fitnessTimeline()                    | 377 |
| 7.103.2.2 intensityFactor()                    | 377 |
| 7.103.2.3 normalizedPower()                    | 378 |
| 7.103.2.4 trainingStressScore()                | 378 |
| 7.104 TrainingMetrics Class Reference          | 379 |
| 7.104.1 Detailed Description                   | 379 |
| 7.104.2 Member Function Documentation          | 380 |
| 7.104.2.1 compute()                            | 380 |
| 7.105 TrainingMetricsSummary Struct Reference  | 380 |
| 7.105.1 Detailed Description                   | 381 |
| 7.105.2 Member Data Documentation              | 381 |
| 7.105.2.1 averageHeartRate                     | 381 |
| 7.105.2.2 averagePower                         | 381 |
| 7.105.2.3 efficiencyFactor                     | 381 |
| 7.105.2.4 intensityFactor                      | 381 |
| 7.105.2.5 normalizedPower                      | 381 |
| 7.105.2.6 trainingStressScore                  | 381 |
| 7.105.2.7 variabilityIndex                     | 382 |
| 7.106 UndoCommand Class Reference              | 382 |
| 7.106.1 Detailed Description                   | 382 |
| 7.106.2 Member Function Documentation          | 382 |
| 7.106.2.1 description()                        | 382 |
| 7.106.2.2 redo()                               | 383 |
| 7.106.2.3 undo()                               | 383 |
| 7.107 UndoManager Class Reference              | 383 |
| 7.107.1 Detailed Description                   | 384 |
| 7.107.2 Constructor & Destructor Documentation | 384 |
| 7.107.2.1 UndoManager()                        | 384 |
| 7.107.3 Member Function Documentation          | 385 |
| 7.107.3.1 canRedo()                            | 385 |
| 7.107.3.2 canUndo()                            | 385 |
| 7.107.3.3 clear()                              | 385 |
| 7.107.3.4 notify()                             | 386 |
| 7.107.3.5 push()                               | 386 |
| 7.107.3.6 redo                                 | 386 |
| 7.107.3.7 redoDescription()                    | 386 |
| 7.107.3.8 undo                                 | 386 |
| 7.107.3.9 undoDescription()                    | 386 |
| 7.107.4 Member Data Documentation              | 386 |

|                                                          |     |
|----------------------------------------------------------|-----|
| 7.107.4.1 m_maxStackSize . . . . .                       | 386 |
| 7.107.4.2 m_redoStack . . . . .                          | 386 |
| 7.107.4.3 m_undoStack . . . . .                          | 387 |
| 7.108 VideoExportDialog Class Reference . . . . .        | 387 |
| 7.108.1 Detailed Description . . . . .                   | 389 |
| 7.108.2 Constructor & Destructor Documentation . . . . . | 389 |
| 7.108.2.1 VideoExportDialog() . . . . .                  | 389 |
| 7.108.2.2 ~VideoExportDialog() . . . . .                 | 389 |
| 7.108.3 Member Function Documentation . . . . .          | 389 |
| 7.108.3.1 cancelOrClose . . . . .                        | 389 |
| 7.108.3.2 chooseOutputFile . . . . .                     | 389 |
| 7.108.3.3 collectSettings() . . . . .                    | 389 |
| 7.108.3.4 onFinished . . . . .                           | 389 |
| 7.108.3.5 onFrameProgressChanged . . . . .               | 390 |
| 7.108.3.6 onProgressChanged . . . . .                    | 390 |
| 7.108.3.7 onStageChanged . . . . .                       | 390 |
| 7.108.3.8 onTileStatsChanged . . . . .                   | 390 |
| 7.108.3.9 setAllOverlayChecks() . . . . .                | 390 |
| 7.108.3.10 startExport . . . . .                         | 390 |
| 7.108.3.11 updateVideoLengthLabel() . . . . .            | 390 |
| 7.108.4 Member Data Documentation . . . . .              | 390 |
| 7.108.4.1 m_altitudeCheck . . . . .                      | 390 |
| 7.108.4.2 m_athleteCheck . . . . .                       | 390 |
| 7.108.4.3 m_cadenceCheck . . . . .                       | 390 |
| 7.108.4.4 m_cancelButton . . . . .                       | 391 |
| 7.108.4.5 m_chartsCheck . . . . .                        | 391 |
| 7.108.4.6 m_defaults . . . . .                           | 391 |
| 7.108.4.7 m_deleteTemporaryTilesCheck . . . . .          | 391 |
| 7.108.4.8 m_entireActivityRadio . . . . .                | 391 |
| 7.108.4.9 m_etaLabel . . . . .                           | 391 |
| 7.108.4.10 m_exportButton . . . . .                      | 391 |
| 7.108.4.11 m_fixedZoomSpin . . . . .                     | 391 |
| 7.108.4.12 m_fpsSpin . . . . .                           | 391 |
| 7.108.4.13 m_frameLabel . . . . .                        | 391 |
| 7.108.4.14 m_hrCheck . . . . .                           | 391 |
| 7.108.4.15 m_job . . . . .                               | 391 |
| 7.108.4.16 m_legendCheck . . . . .                       | 392 |
| 7.108.4.17 m_mapStyleCombo . . . . .                     | 392 |
| 7.108.4.18 m_outputEdit . . . . .                        | 392 |
| 7.108.4.19 m_powerCheck . . . . .                        | 392 |
| 7.108.4.20 m_progressBar . . . . .                       | 392 |
| 7.108.4.21 m_resolutionCombo . . . . .                   | 392 |

|                                                      |     |
|------------------------------------------------------|-----|
| 7.108.4.22 m_routeColorCombo . . . . .               | 392 |
| 7.108.4.23 m_running . . . . .                       | 392 |
| 7.108.4.24 m_selectedSegmentRadio . . . . .          | 392 |
| 7.108.4.25 m_speedCheck . . . . .                    | 392 |
| 7.108.4.26 m_speedSpin . . . . .                     | 392 |
| 7.108.4.27 m_stageLabel . . . . .                    | 392 |
| 7.108.4.28 m_telemetryPanelCheck . . . . .           | 393 |
| 7.108.4.29 m_thread . . . . .                        | 393 |
| 7.108.4.30 m_tileStatsLabel . . . . .                | 393 |
| 7.108.4.31 m_timeCheck . . . . .                     | 393 |
| 7.108.4.32 m_videoLengthLabel . . . . .              | 393 |
| 7.109 VideoExportSettings Struct Reference . . . . . | 393 |
| 7.109.1 Detailed Description . . . . .               | 395 |
| 7.109.2 Member Data Documentation . . . . .          | 395 |
| 7.109.2.1 activityName . . . . .                     | 395 |
| 7.109.2.2 athleteName . . . . .                      | 395 |
| 7.109.2.3 autoZoom . . . . .                         | 395 |
| 7.109.2.4 colorContext . . . . .                     | 395 |
| 7.109.2.5 deleteTemporaryTilesAfterExport . . . . .  | 395 |
| 7.109.2.6 ffmpegPath . . . . .                       | 395 |
| 7.109.2.7 fixedZoom . . . . .                        | 395 |
| 7.109.2.8 followAthlete . . . . .                    | 396 |
| 7.109.2.9 fps . . . . .                              | 396 |
| 7.109.2.10 height . . . . .                          | 396 |
| 7.109.2.11 mapStyle . . . . .                        | 396 |
| 7.109.2.12 outputPath . . . . .                      | 396 |
| 7.109.2.13 overlayAltitude . . . . .                 | 396 |
| 7.109.2.14 overlayAthleteName . . . . .              | 396 |
| 7.109.2.15 overlayCadence . . . . .                  | 396 |
| 7.109.2.16 overlayCharts . . . . .                   | 396 |
| 7.109.2.17 overlayHeartRate . . . . .                | 397 |
| 7.109.2.18 overlayPower . . . . .                    | 397 |
| 7.109.2.19 overlayRouteLegend . . . . .              | 397 |
| 7.109.2.20 overlaySpeed . . . . .                    | 397 |
| 7.109.2.21 overlayTelemetryPanel . . . . .           | 397 |
| 7.109.2.22 overlayTime . . . . .                     | 397 |
| 7.109.2.23 playbackSpeed . . . . .                   | 397 |
| 7.109.2.24 rideData . . . . .                        | 397 |
| 7.109.2.25 routeColorMetric . . . . .                | 398 |
| 7.109.2.26 segmentEndSeconds . . . . .               | 398 |
| 7.109.2.27 segmentStartSeconds . . . . .             | 398 |
| 7.109.2.28 singleRouteColor . . . . .                | 398 |

---

|                                                          |     |
|----------------------------------------------------------|-----|
| 7.109.2.29 useSelectedSegment . . . . .                  | 398 |
| 7.109.2.30 width . . . . .                               | 398 |
| 7.110 VideoPage Class Reference . . . . .                | 398 |
| 7.110.1 Detailed Description . . . . .                   | 400 |
| 7.110.2 Constructor & Destructor Documentation . . . . . | 400 |
| 7.110.2.1 VideoPage() . . . . .                          | 400 |
| 7.111 VideoRenderJob Class Reference . . . . .           | 400 |
| 7.111.1 Detailed Description . . . . .                   | 402 |
| 7.111.2 Constructor & Destructor Documentation . . . . . | 402 |
| 7.111.2.1 VideoRenderJob() . . . . .                     | 402 |
| 7.111.3 Member Function Documentation . . . . .          | 402 |
| 7.111.3.1 cancel() . . . . .                             | 402 |
| 7.111.3.2 finished . . . . .                             | 402 |
| 7.111.3.3 frameProgressChanged . . . . .                 | 403 |
| 7.111.3.4 progressChanged . . . . .                      | 403 |
| 7.111.3.5 run() . . . . .                                | 404 |
| 7.111.3.6 stageChanged . . . . .                         | 404 |
| 7.111.3.7 tileStatsChanged . . . . .                     | 405 |
| 7.111.4 Member Data Documentation . . . . .              | 405 |
| 7.111.4.1 m_cancelRequested . . . . .                    | 405 |
| 7.111.4.2 m_settings . . . . .                           | 405 |
| 7.112 VideoTileId Struct Reference . . . . .             | 406 |
| 7.112.1 Detailed Description . . . . .                   | 406 |
| 7.112.2 Member Function Documentation . . . . .          | 406 |
| 7.112.2.1 operator==( ) . . . . .                        | 406 |
| 7.112.3 Member Data Documentation . . . . .              | 406 |
| 7.112.3.1 x . . . . .                                    | 406 |
| 7.112.3.2 y . . . . .                                    | 406 |
| 7.112.3.3 z . . . . .                                    | 406 |
| 7.113 VideoTileIdHash Struct Reference . . . . .         | 407 |
| 7.113.1 Detailed Description . . . . .                   | 407 |
| 7.113.2 Member Function Documentation . . . . .          | 407 |
| 7.113.2.1 operator()( ) . . . . .                        | 407 |
| 7.114 VideoTileProvider Class Reference . . . . .        | 407 |
| 7.114.1 Detailed Description . . . . .                   | 408 |
| 7.114.2 Constructor & Destructor Documentation . . . . . | 408 |
| 7.114.2.1 VideoTileProvider() . . . . .                  | 408 |
| 7.114.3 Member Function Documentation . . . . .          | 409 |
| 7.114.3.1 cleanupExportCache() . . . . .                 | 409 |
| 7.114.3.2 clearMemoryCache() . . . . .                   | 409 |
| 7.114.3.3 exportCacheRoot() . . . . .                    | 409 |
| 7.114.3.4 getTile() . . . . .                            | 409 |

|                                                |     |
|------------------------------------------------|-----|
| 7.114.3.5 isTileCachedOnDisk()                 | 410 |
| 7.114.3.6 requiredTileCount()                  | 410 |
| 7.114.3.7 requiredTilesAvailableOnDisk()       | 411 |
| 7.114.3.8 setRequiredTiles()                   | 411 |
| 7.114.3.9 shouldDeleteExportCacheAfterExport() | 411 |
| 7.114.3.10 stats()                             | 411 |
| 7.114.4 Member Data Documentation              | 411 |
| 7.114.4.1 m_deleteTemporaryTilesAfterExport    | 411 |
| 7.114.4.2 m_downloadedTiles                    | 411 |
| 7.114.4.3 m_requiredTiles                      | 412 |
| 7.114.4.4 m_requiredTilesOnDisk                | 412 |
| 7.114.4.5 m_tileCache                          | 412 |
| 7.115 VideoTileProviderConfig Struct Reference | 412 |
| 7.115.1 Detailed Description                   | 412 |
| 7.115.2 Member Data Documentation              | 412 |
| 7.115.2.1 deleteTemporaryTilesAfterExport      | 412 |
| 7.115.2.2 exportCacheRoot                      | 412 |
| 7.115.2.3 memoryCacheTiles                     | 412 |
| 7.115.2.4 style                                | 413 |
| 7.116 VideoTileStats Struct Reference          | 413 |
| 7.116.1 Detailed Description                   | 413 |
| 7.116.2 Member Data Documentation              | 413 |
| 7.116.2.1 downloadedTiles                      | 413 |
| 7.116.2.2 onDiskTiles                          | 413 |
| 7.116.2.3 requiredTiles                        | 413 |
| 7.117 WeightEntry Struct Reference             | 413 |
| 7.117.1 Detailed Description                   | 414 |
| 7.117.2 Member Data Documentation              | 414 |
| 7.117.2.1 athleteId                            | 414 |
| 7.117.2.2 effectiveFrom                        | 414 |
| 7.117.2.3 id                                   | 414 |
| 7.117.2.4 weightKg                             | 414 |
| 7.118 WelcomeWidget Class Reference            | 414 |
| 7.118.1 Detailed Description                   | 416 |
| 7.118.2 Constructor & Destructor Documentation | 416 |
| 7.118.2.1 WelcomeWidget()                      | 416 |
| 7.118.3 Member Function Documentation          | 417 |
| 7.118.3.1 setFirstLaunch()                     | 417 |
| 7.118.4 Member Data Documentation              | 417 |
| 7.118.4.1 m_createDbButton                     | 417 |
| 7.118.4.2 m_importButton                       | 417 |
| 7.118.4.3 m_manageAthletesButton               | 417 |

|                                                          |     |
|----------------------------------------------------------|-----|
| 7.118.4.4 m_openDbButton . . . . .                       | 417 |
| 7.118.4.5 m_subtitleLabel . . . . .                      | 417 |
| 7.119 WorkoutAnalyzer Class Reference . . . . .          | 417 |
| 7.119.1 Detailed Description . . . . .                   | 418 |
| 7.119.2 Member Function Documentation . . . . .          | 418 |
| 7.119.2.1 analyze() . . . . .                            | 418 |
| 7.120 WorkoutController Class Reference . . . . .        | 418 |
| 7.120.1 Detailed Description . . . . .                   | 420 |
| 7.120.2 Constructor & Destructor Documentation . . . . . | 420 |
| 7.120.2.1 WorkoutController() . . . . .                  | 420 |
| 7.120.3 Member Function Documentation . . . . .          | 420 |
| 7.120.3.1 climbs() . . . . .                             | 420 |
| 7.120.3.2 currentActivityId() . . . . .                  | 421 |
| 7.120.3.3 currentAthleteId() . . . . .                   | 421 |
| 7.120.3.4 efficiencyFactor() . . . . .                   | 421 |
| 7.120.3.5 ftp() . . . . .                                | 421 |
| 7.120.3.6 importFile() . . . . .                         | 421 |
| 7.120.3.7 intensityFactor() . . . . .                    | 421 |
| 7.120.3.8 intervals() . . . . .                          | 421 |
| 7.120.3.9 loadActivity() . . . . .                       | 421 |
| 7.120.3.10 loadFile() . . . . .                          | 421 |
| 7.120.3.11 loadOrGenerateClimbs() . . . . .              | 421 |
| 7.120.3.12 loadOrGenerateIntervals() . . . . .           | 422 |
| 7.120.3.13 normalizedPower() . . . . .                   | 422 |
| 7.120.3.14 pdcPoints() . . . . .                         | 422 |
| 7.120.3.15 powerZones() . . . . .                        | 422 |
| 7.120.3.16 reanalyze() . . . . .                         | 422 |
| 7.120.3.17 reloadClimbsFromDatabase() . . . . .          | 422 |
| 7.120.3.18 rideData() . . . . .                          | 422 |
| 7.120.3.19 setAnalysisQueue() . . . . .                  | 422 |
| 7.120.3.20 setCurrentAthlete() . . . . .                 | 422 |
| 7.120.3.21 setDatabaseManager() . . . . .                | 422 |
| 7.120.3.22 setFTP() . . . . .                            | 422 |
| 7.120.3.23 statistics() . . . . .                        | 423 |
| 7.120.3.24 trainingStressScore() . . . . .               | 423 |
| 7.120.3.25 variabilityIndex() . . . . .                  | 423 |
| 7.120.4 Member Data Documentation . . . . .              | 423 |
| 7.120.4.1 m_analysisQueue . . . . .                      | 423 |
| 7.120.4.2 m_analyzer . . . . .                           | 423 |
| 7.120.4.3 m_climbs . . . . .                             | 423 |
| 7.120.4.4 m_currentActivityId . . . . .                  | 423 |
| 7.120.4.5 m_currentAthleteId . . . . .                   | 423 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| 7.120.4.6 m_dbManager . . . . .                           | 423 |
| 7.120.4.7 m_ef . . . . .                                  | 423 |
| 7.120.4.8 m_ftp . . . . .                                 | 423 |
| 7.120.4.9 m_if . . . . .                                  | 423 |
| 7.120.4.10 m_intervals . . . . .                          | 424 |
| 7.120.4.11 m_np . . . . .                                 | 424 |
| 7.120.4.12 m_pdcPoints . . . . .                          | 424 |
| 7.120.4.13 m_powerZones . . . . .                         | 424 |
| 7.120.4.14 m_rideData . . . . .                           | 424 |
| 7.120.4.15 m_statistics . . . . .                         | 424 |
| 7.120.4.16 m_tss . . . . .                                | 424 |
| 7.120.4.17 m_vi . . . . .                                 | 424 |
| 7.121 WorkoutPersistenceService Class Reference . . . . . | 424 |
| 7.121.1 Detailed Description . . . . .                    | 424 |
| 7.121.2 Member Function Documentation . . . . .           | 425 |
| 7.121.2.1 loadRide() . . . . .                            | 425 |
| 7.121.2.2 saveRide() . . . . .                            | 425 |
| 7.122 WorkoutStatistics Struct Reference . . . . .        | 425 |
| 7.122.1 Detailed Description . . . . .                    | 426 |
| 7.122.2 Member Data Documentation . . . . .               | 426 |
| 7.122.2.1 averageCadence . . . . .                        | 426 |
| 7.122.2.2 averageHeartRate . . . . .                      | 426 |
| 7.122.2.3 averagePower . . . . .                          | 426 |
| 7.122.2.4 averageSpeed . . . . .                          | 426 |
| 7.122.2.5 durationSeconds . . . . .                       | 426 |
| 7.122.2.6 elevationGainM . . . . .                        | 426 |
| 7.122.2.7 maximumCadence . . . . .                        | 426 |
| 7.122.2.8 maximumHeartRate . . . . .                      | 427 |
| 7.122.2.9 maximumPower . . . . .                          | 427 |
| 7.122.2.10 maximumSpeed . . . . .                         | 427 |
| 7.122.2.11 movingTimeSeconds . . . . .                    | 427 |
| 7.122.2.12 totalDistanceKm . . . . .                      | 427 |
| 7.123 Zone Struct Reference . . . . .                     | 427 |
| 7.123.1 Detailed Description . . . . .                    | 427 |
| 7.123.2 Member Data Documentation . . . . .               | 428 |
| 7.123.2.1 color . . . . .                                 | 428 |
| 7.123.2.2 maxValuE . . . . .                              | 428 |
| 7.123.2.3 minValuE . . . . .                              | 428 |
| 7.123.2.4 name . . . . .                                  | 428 |
| 7.124 ZoneCalculator Class Reference . . . . .            | 428 |
| 7.124.1 Detailed Description . . . . .                    | 429 |
| 7.124.2 Member Function Documentation . . . . .           | 429 |

|                                         |            |
|-----------------------------------------|------------|
| 7.124.2.1 altitudeZones()               | 429        |
| 7.124.2.2 cadenceZones()                | 429        |
| 7.124.2.3 computeDistribution()         | 430        |
| 7.124.2.4 hasMetricSamples()            | 431        |
| 7.124.2.5 heartRateZones()              | 431        |
| 7.124.2.6 powerZones()                  | 432        |
| 7.124.2.7 speedZones()                  | 432        |
| 7.124.2.8 zonesForMetric()              | 433        |
| 7.125 ZoneDistribution Struct Reference | 434        |
| 7.125.1 Detailed Description            | 435        |
| 7.125.2 Member Data Documentation       | 435        |
| 7.125.2.1 percent                       | 435        |
| 7.125.2.2 seconds                       | 435        |
| 7.125.2.3 zone                          | 435        |
| <b>8 File Documentation</b>             | <b>437</b> |
| 8.1 ClimbCandidateFinder.cpp            | 437        |
| 8.2 ClimbCandidateFinder.h              | 438        |
| 8.3 ClimbClassifier.cpp                 | 438        |
| 8.4 ClimbClassifier.h                   | 438        |
| 8.5 ClimbMerger.cpp                     | 438        |
| 8.6 ClimbMerger.h                       | 439        |
| 8.7 ClimbDetector.cpp                   | 439        |
| 8.8 ClimbDetector.h                     | 446        |
| 8.9 ClimbMetrics.cpp                    | 447        |
| 8.10 ClimbMetrics.h                     | 453        |
| 8.11 FTPZones.cpp                       | 454        |
| 8.12 FTPZones.h                         | 455        |
| 8.13 IntervalDetector.cpp               | 455        |
| 8.14 IntervalDetector.h                 | 458        |
| 8.15 PowerCurve.cpp                     | 459        |
| 8.16 PowerCurve.h                       | 460        |
| 8.17 AnalysisQueue.cpp                  | 460        |
| 8.18 AnalysisQueue.h                    | 461        |
| 8.19 AnalysisWorker.cpp                 | 462        |
| 8.20 AnalysisWorker.h                   | 465        |
| 8.21 SmoothingEngine.cpp                | 465        |
| 8.22 SmoothingEngine.h                  | 466        |
| 8.23 TrainingLoad.cpp                   | 466        |
| 8.24 TrainingLoad.h                     | 467        |
| 8.25 TrainingMetrics.cpp                | 468        |
| 8.26 TrainingMetrics.h                  | 468        |



|                                                       |     |
|-------------------------------------------------------|-----|
| 8.27 WorkoutAnalyzer.cpp . . . . .                    | 469 |
| 8.28 WorkoutAnalyzer.h . . . . .                      | 470 |
| 8.29 RideChartDataBuilder.cpp . . . . .               | 471 |
| 8.30 RideChartDataBuilder.h . . . . .                 | 472 |
| 8.31 FitnessChartWidget.cpp . . . . .                 | 473 |
| 8.32 FitnessChartWidget.h . . . . .                   | 474 |
| 8.33 Metric.h . . . . .                               | 474 |
| 8.34 PowerCurveWidget.cpp . . . . .                   | 475 |
| 8.35 PowerCurveWidget.h . . . . .                     | 477 |
| 8.36 PowerHistogramWidget.cpp . . . . .               | 478 |
| 8.37 PowerHistogramWidget.h . . . . .                 | 479 |
| 8.38 RideChartRenderer.cpp . . . . .                  | 479 |
| 8.39 RideChartRenderer.h . . . . .                    | 480 |
| 8.40 RideChartWidget.cpp . . . . .                    | 480 |
| 8.41 RideChartWidget.h . . . . .                      | 501 |
| 8.42 RideChartSelectionManager.cpp . . . . .          | 503 |
| 8.43 RideChartSelectionManager.h . . . . .            | 504 |
| 8.44 ActivityAnalysisFlags.h . . . . .                | 504 |
| 8.45 AnalysisVersions.h . . . . .                     | 505 |
| 8.46 AppSettings.cpp . . . . .                        | 505 |
| 8.47 AppSettings.h . . . . .                          | 506 |
| 8.48 DateFormatter.cpp . . . . .                      | 506 |
| 8.49 DateFormatter.h . . . . .                        | 507 |
| 8.50 ClimbEditCommand.cpp . . . . .                   | 508 |
| 8.51 ClimbEditCommand.h . . . . .                     | 508 |
| 8.52 UndoCommand.h . . . . .                          | 509 |
| 8.53 UndoManager.cpp . . . . .                        | 509 |
| 8.54 UndoManager.h . . . . .                          | 510 |
| 8.55 ColorProvider.cpp . . . . .                      | 511 |
| 8.56 ColorProvider.h . . . . .                        | 512 |
| 8.57 ZoneCalculator.cpp . . . . .                     | 512 |
| 8.58 ZoneCalculator.h . . . . .                       | 515 |
| 8.59 ZoneDefinition.cpp . . . . .                     | 515 |
| 8.60 ZoneDefinition.h . . . . .                       | 516 |
| 8.61 ActivityAnalysisCacheRepository.cpp . . . . .    | 517 |
| 8.62 ActivityAnalysisCacheRepository.h . . . . .      | 517 |
| 8.63 ActivityAnalysisSettingsRepository.cpp . . . . . | 518 |
| 8.64 ActivityAnalysisSettingsRepository.h . . . . .   | 518 |
| 8.65 ActivityRepository.cpp . . . . .                 | 519 |
| 8.66 ActivityRepository.h . . . . .                   | 523 |
| 8.67 AthleteRepository.cpp . . . . .                  | 524 |
| 8.68 AthleteRepository.h . . . . .                    | 526 |

|                                    |     |
|------------------------------------|-----|
| 8.69 ClimbRepository.cpp           | 528 |
| 8.70 ClimbRepository.h             | 530 |
| 8.71 DatabaseConnectionFactory.cpp | 532 |
| 8.72 DatabaseConnectionFactory.h   | 532 |
| 8.73 DatabaseConnectionGuard.cpp   | 532 |
| 8.74 DatabaseConnectionGuard.h     | 533 |
| 8.75 DatabaseManager.cpp           | 533 |
| 8.76 DatabaseManager.h             | 535 |
| 8.77 DatabaseSchema.cpp            | 535 |
| 8.78 DatabaseSchema.h              | 535 |
| 8.79 ImportRepository.cpp          | 539 |
| 8.80 ImportRepository.h            | 540 |
| 8.81 IntervalRepository.cpp        | 540 |
| 8.82 IntervalRepository.h          | 542 |
| 8.83 DatabaseMigrationManager.cpp  | 543 |
| 8.84 DatabaseMigrationManager.h    | 547 |
| 8.85 SqlQueryHelper.h              | 547 |
| 8.86 FitParser.cpp                 | 547 |
| 8.87 FitParser.h                   | 550 |
| 8.88 FitRecordListener.cpp         | 550 |
| 8.89 FitRecordListener.h           | 552 |
| 8.90 RideData.h                    | 552 |
| 8.91 fitlyzerC.cpp                 | 553 |
| 8.92 AboutDialog.cpp               | 553 |
| 8.93 AboutDialog.h                 | 554 |
| 8.94 ActivityBrowser.cpp           | 554 |
| 8.95 ActivityBrowser.h             | 558 |
| 8.96 AthleteDialog.cpp             | 559 |
| 8.97 AthleteDialog.h               | 562 |
| 8.98 AthleteHeaderWidget.cpp       | 563 |
| 8.99 AthleteHeaderWidget.h         | 564 |
| 8.100 AthleteListDialog.cpp        | 564 |
| 8.101 AthleteListDialog.h          | 566 |
| 8.102 CalendarWidget.cpp           | 566 |
| 8.103 CalendarWidget.h             | 570 |
| 8.104 ActivityViewController.cpp   | 571 |
| 8.105 ActivityViewController.h     | 573 |
| 8.106 ChartController.cpp          | 574 |
| 8.107 ChartController.h            | 581 |
| 8.108 ImportController.cpp         | 583 |
| 8.109 ImportController.h           | 584 |
| 8.110 MainWindowActions.cpp        | 585 |

|                                                 |     |
|-------------------------------------------------|-----|
| 8.111 MainWindowActions.h . . . . .             | 586 |
| 8.112 MapController.cpp . . . . .               | 588 |
| 8.113 MapController.h . . . . .                 | 589 |
| 8.114 NavigationController.cpp . . . . .        | 590 |
| 8.115 NavigationController.h . . . . .          | 591 |
| 8.116 CreateDatabaseDialog.cpp . . . . .        | 591 |
| 8.117 CreateDatabaseDialog.h . . . . .          | 593 |
| 8.118 EditActivityDialog.cpp . . . . .          | 594 |
| 8.119 EditActivityDialog.h . . . . .            | 595 |
| 8.120 EditNotesDialog.cpp . . . . .             | 595 |
| 8.121 EditNotesDialog.h . . . . .               | 596 |
| 8.122 ImportStatusWidget.cpp . . . . .          | 596 |
| 8.123 ImportStatusWidget.h . . . . .            | 597 |
| 8.124 IntervalEditorDialog.cpp . . . . .        | 598 |
| 8.125 IntervalEditorDialog.h . . . . .          | 598 |
| 8.126 MainWindow.cpp . . . . .                  | 599 |
| 8.127 MainWindow.h . . . . .                    | 602 |
| 8.128 MainWindowAthlete.cpp . . . . .           | 607 |
| 8.129 MainWindowChartActions.cpp . . . . .      | 612 |
| 8.130 MainWindowCharts.cpp . . . . .            | 614 |
| 8.131 MainWindowClimbInteractions.cpp . . . . . | 615 |
| 8.132 MainWindowClimbViews.cpp . . . . .        | 625 |
| 8.133 MainWindowContext.cpp . . . . .           | 629 |
| 8.134 MainWindowDatabase.cpp . . . . .          | 630 |
| 8.135 MainWindowDetection.cpp . . . . .         | 634 |
| 8.136 MainWindowImport.cpp . . . . .            | 637 |
| 8.137 MainWindowIntervals.cpp . . . . .         | 640 |
| 8.138 MainWindowReadiness.cpp . . . . .         | 645 |
| 8.139 MainWindowStatus.cpp . . . . .            | 648 |
| 8.140 MainWindowUi.cpp . . . . .                | 649 |
| 8.141 MainWindowUiBuild.cpp . . . . .           | 651 |
| 8.142 MainWindowVideo.cpp . . . . .             | 665 |
| 8.143 MainWindowWatch.cpp . . . . .             | 666 |
| 8.144 NavigationSidebar.cpp . . . . .           | 668 |
| 8.145 NavigationSidebar.h . . . . .             | 669 |
| 8.146 ActivitiesPage.cpp . . . . .              | 670 |
| 8.147 ActivitiesPage.h . . . . .                | 671 |
| 8.148 BasePage.h . . . . .                      | 671 |
| 8.149 CalendarPage.cpp . . . . .                | 671 |
| 8.150 CalendarPage.h . . . . .                  | 672 |
| 8.151 ChartsPage.cpp . . . . .                  | 672 |
| 8.152 ChartsPage.h . . . . .                    | 672 |

|                                |     |
|--------------------------------|-----|
| 8.153 ClimbsPage.cpp           | 672 |
| 8.154 ClimbsPage.h             | 673 |
| 8.155 FitnessPage.cpp          | 673 |
| 8.156 FitnessPage.h            | 674 |
| 8.157 IntervalsPage.cpp        | 674 |
| 8.158 IntervalsPage.h          | 674 |
| 8.159 PlaceholderPage.h        | 675 |
| 8.160 PowerPage.cpp            | 675 |
| 8.161 PowerPage.h              | 675 |
| 8.162 VideoPage.cpp            | 676 |
| 8.163 VideoPage.h              | 677 |
| 8.164 PlannedWorkoutDialog.cpp | 677 |
| 8.165 PlannedWorkoutDialog.h   | 678 |
| 8.166 WelcomeWidget.cpp        | 679 |
| 8.167 WelcomeWidget.h          | 681 |
| 8.168 WorkoutController.cpp    | 682 |
| 8.169 WorkoutController.h      | 687 |
| 8.170 ImportJob.cpp            | 689 |
| 8.171 ImportJob.h              | 689 |
| 8.172 ImportProgressModel.cpp  | 690 |
| 8.173 ImportProgressModel.h    | 691 |
| 8.174 ImportQueue.cpp          | 691 |
| 8.175 ImportQueue.h            | 693 |
| 8.176 ImportWorker.cpp         | 694 |
| 8.177 ImportWorker.h           | 697 |
| 8.178 DiskTileCache.cpp        | 698 |
| 8.179 DiskTileCache.h          | 698 |
| 8.180 GpsCacheBuilder.cpp      | 699 |
| 8.181 GpsCacheBuilder.h        | 699 |
| 8.182 MapFitMath.cpp           | 699 |
| 8.183 MapFitMath.h             | 701 |
| 8.184 MapRenderer.cpp          | 701 |
| 8.185 MapRenderer.h            | 711 |
| 8.186 TileDownloader.cpp       | 712 |
| 8.187 TileDownloader.h         | 713 |
| 8.188 TileProviderRegistry.cpp | 713 |
| 8.189 TileProviderRegistry.h   | 714 |
| 8.190 RouteColorizer.cpp       | 714 |
| 8.191 RouteColorizer.h         | 715 |
| 8.192 TileCache.cpp            | 715 |
| 8.193 TileCache.h              | 719 |
| 8.194 RideDataLoader.cpp       | 720 |

---

|                                               |            |
|-----------------------------------------------|------------|
| 8.195 RideDataLoader.h . . . . .              | 721        |
| 8.196 RideDataSerializer.cpp . . . . .        | 721        |
| 8.197 RideDataSerializer.h . . . . .          | 723        |
| 8.198 Platform.cpp . . . . .                  | 724        |
| 8.199 Platform.h . . . . .                    | 724        |
| 8.200 Platform_linux.cpp . . . . .            | 724        |
| 8.201 Platform_macos.cpp . . . . .            | 725        |
| 8.202 Platform_windows.cpp . . . . .          | 725        |
| 8.203 FfmpegPath.cpp . . . . .                | 725        |
| 8.204 FfmpegPath.h . . . . .                  | 726        |
| 8.205 UuidGenerator.h . . . . .               | 726        |
| 8.206 VideoEncoder.cpp . . . . .              | 727        |
| 8.207 VideoEncoder.h . . . . .                | 727        |
| 8.208 OverlayRenderer.cpp . . . . .           | 727        |
| 8.209 OverlayRenderer.h . . . . .             | 728        |
| 8.210 VideoRenderer.cpp . . . . .             | 728        |
| 8.211 VideoRenderer.h . . . . .               | 730        |
| 8.212 VideoExportDialog.cpp . . . . .         | 731        |
| 8.213 VideoExportDialog.h . . . . .           | 736        |
| 8.214 VideoExportSettings.h . . . . .         | 737        |
| 8.215 VideoRenderJob.cpp . . . . .            | 738        |
| 8.216 VideoRenderJob.h . . . . .              | 747        |
| 8.217 VideoTileProvider.cpp . . . . .         | 748        |
| 8.218 VideoTileProvider.h . . . . .           | 749        |
| 8.219 WorkoutPersistenceService.cpp . . . . . | 750        |
| 8.220 WorkoutPersistenceService.h . . . . .   | 751        |
| <b>Index</b>                                  | <b>753</b> |



# Chapter 1

## FitlyzerC

---

**Modern Garmin FIT analysis for cyclists, runners, triathletes, coaches, and data enthusiasts.**

---

### Warning

**FitlyzerC is pre-release software.**

Database schemas, internal data structures, and project formats may change between releases. Upgrading may require recreating databases or re-importing activities. Backward compatibility is not guaranteed yet.

## 1.1 Download

Prebuilt binaries are available from the [Releases](#) page.

| Platform | Package        |
|----------|----------------|
| Windows  | NSIS Installer |
| macOS    | DMG            |
| Linux    | ApplImage      |

---

## 1.2 What is FitlyzerC?

FitlyzerC is a native desktop application for analyzing Garmin FIT activity files.

Unlike cloud-based training platforms, FitlyzerC focuses on local-first analysis, high-performance visualization, and complete ownership of your training data. The application is written in modern C++23 and Qt 6 and runs on Windows, macOS, and Linux.

The project is designed primarily for cycling and currently focuses on detailed ride analysis, interval inspection, power-based training metrics, GPS visualization, and athlete management.

---

## 1.3 Features

### 1.3.1 Activity Analysis

- Import Garmin FIT activities
- Decode FIT files using the Garmin FIT SDK
- Power, heart rate, cadence, speed, elevation and GPS analysis

- Automatic interval detection
- Power curve generation
- Power histograms
- Training load calculations
- Fitness tracking metrics
- Workout review tools
- Ride notes

### 1.3.2 Interactive Charts

- Zoomable and pannable charts
- Synchronized chart navigation
- Power analysis
- Fitness trend visualization
- Ride timeline inspection
- Smoothed data visualization

### 1.3.3 Mapping

- Interactive route display
- Multiple map styles
- GPS track visualization
- Segment selection directly on the map
- Draggable selection markers
- Route coloring by:
  - Power
  - Heart Rate
  - Cadence
  - Speed
  - Altitude
  - Gradient

### 1.3.4 Athlete Management

- Multiple athlete profiles
- Athlete-specific FTP settings
- Training history
- Calendar-based activity browsing
- Planned workouts



### 1.3.5 Video Export

- Animated ride playback
- Map-based activity visualization
- Chart synchronization
- Power zone overlays
- MP4 export through FFmpeg
- Custom export settings

### 1.3.6 Cross Platform

- Windows
  - macOS
  - Linux
- 

## 1.4 Screenshots

## 1.5 Current Development Status

---

FitlyzerC is under active development.  
Recent milestones include:

- Linux ApplImage packaging
- Reworked map tile system
- Additional map styles
- Improved route rendering
- Movable interval markers
- Segment selection on maps
- Video export support
- FFmpeg integration
- Multiple video rendering improvements

### 1.5.1 Planned Areas

- Additional training metrics
  - Reporting features
  - More export formats
  - Better workout planning
  - UI/UX refinements
  - Performance improvements
-

## 1.6 Technology Stack

| Component    | Technology     |
|--------------|----------------|
| Language     | C++23          |
| GUI          | Qt 6           |
| Charts       | Qt Charts      |
| Database     | SQLite         |
| Networking   | Qt Network     |
| Build System | CMake          |
| Packaging    | CPack          |
| FIT Parsing  | Garmin FIT SDK |

## 1.7 Project Structure

```
src/  
  analysis/      Training metrics, power curves, interval detection  
  charts/       Interactive chart widgets  
  core/         Zone calculations and shared logic  
  database/     SQLite repositories and schema management  
  fit/          Garmin FIT decoding  
  gui/          User interface  
  maps/         Tile engine and route rendering  
  model/        Serialization helpers  
  platform/     OS-specific functionality  
  video/        Video export pipeline  
  reports/      Reporting infrastructure
```

## 1.8 Build Requirements

### 1.8.1 Required Software

| Component                     | Version                                                                                                              |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| CMake                         | 3.21+                                                                                                                |
| Qt                            | 6.x                                                                                                                  |
| Compiler                      | C++23 capable                                                                                                        |
| Git                           | Current                                                                                                              |
| Ninja                         | Recommended                                                                                                          |
| FFmpeg (for bundled releases) | Source tree in <code>third_party/ffmpeg</code> or prebuilt binary in <code>resources/ffmpeg/&lt;platform&gt;/</code> |

## 1.9 Building

### 1.9.1 macOS

#### 1.9.1.1 Dependencies

```
brew install cmake ninja qt  
export PATH="/opt/homebrew/opt/qt/bin:$PATH"
```

### 1.9.1.2 Configure

```
cmake --preset macos-release
```

### 1.9.1.3 Build

```
cmake --build --preset macos-release
```

### 1.9.1.4 Create DMG

```
cmake --build --preset macos-dmg
```

---

## 1.9.2 Windows

Install:

- Visual Studio 2026
- Qt 6 (MSVC)
- CMake
- Ninja

### 1.9.2.1 Configure

```
cmake --preset windows-release
```

### 1.9.2.2 Build

```
cmake --build --preset windows-release
```

### 1.9.2.3 Create Installer

```
cmake --build --preset windows-nsis
```

---

## 1.9.3 Linux

### 1.9.3.1 Arch Linux

```
sudo pacman -S \  
    base-devel \  
    cmake \  
    ninja \  
    qt6-base \  
    qt6-charts \  
    git
```

### 1.9.3.2 Configure

```
cmake --preset linux-release
```

### 1.9.3.3 Build

```
cmake --build --preset linux-release
```

### 1.9.3.4 AppImage

Install:

```
sudo pacman -S patchelf  
yay -S appimagetool-bin  
cmake --preset linux-appimage  
cmake --build --preset linux-appimage
```

---

## 1.9.4 One-Command Packaging

```
cmake -P scripts/package.cmake
```

Supported outputs:

| Platform | Output    |
|----------|-----------|
| Windows  | Setup.exe |
| macOS    | DMG       |
| Linux    | ApplImage |

## 1.9.5 Garmin FIT SDK

By default, CMake automatically downloads the Garmin FIT SDK.

Repository:

<https://github.com/garmin/fit-cpp-sdk>

To use a local SDK copy:

```
cmake \
  -DGARMIN_FIT_SDK_DIR=/path/to/sdk \
  -S . \
  -B build
```

Supported locations:

```
fit-sdk/
fit-sdk/src/
fit-sdk/cpp-sdk/src/
```

## 1.9.6 FFmpeg Bundling

Release builds are configured to bundle FFmpeg with the app so end users do not need a system FFmpeg installation.

Bundling priority:

1. Build from FFmpeg source in `third_party/ffmpeg`
2. Auto-download FFmpeg from <https://git.ffmpeg.org/ffmpeg.git> when the local source tree is missing
3. Use prebuilt executable in `resources/ffmpeg/<platform>/`
4. Fallback to system `ffmpeg` if neither source nor bundled binary is available

Default prebuilt lookup paths:

```
resources/ffmpeg/macos/ffmpeg
resources/ffmpeg/linux/ffmpeg
resources/ffmpeg/windows/ffmpeg.exe
```

Useful CMake options:

```
-DFITLYZERC_BUNDLE_FFMPEG=ON
-DFITLYZERC_FFMPEG_SOURCE_DIR=third_party/ffmpeg
-DFITLYZERC_AUTO_DOWNLOAD_FFMPEG=ON
-DFITLYZERC_FFMPEG_GIT_REPOSITORY=https://git.ffmpeg.org/ffmpeg.git
-DFITLYZERC_FFMPEG_GIT_TAG=n7.0
-DFITLYZERC_FFMPEG_BINARY_PATH=/absolute/path/to/ffmpeg
```

If you want fully reproducible builds, point `FITLYZERC_FFMPEG_SOURCE_DIR` at a local FFmpeg checkout or pin `FITLYZERC_FFMPEG_GIT_TAG` to a specific release tag/commit.

Disable bundling (not recommended for release artifacts):

```
cmake -DFITLYZERC_BUNDLE_FFMPEG=OFF --preset <your-preset>
```

## 1.9.7 Running

Typical workflow:

1. Create an athlete
2. Import FIT activities

3. Browse activities in the calendar
  4. Inspect charts and maps
  5. Select intervals
  6. Analyze performance metrics
  7. Export video visualizations
- 

### 1.9.8 Contributing

Issues, bug reports, feature requests and pull requests are welcome.  
If you encounter a problem, include:

- Operating system
  - Application version
  - FIT file characteristics
  - Steps to reproduce
- 

### 1.9.9 License

Informal license:

Do whatever you want with it, provide attribution and link back to the repository.  
Except dopers. They cannot use it.



## Chapter 2

# Namespace Index

### 2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| <a href="#">ActivityAnalysisFlags</a>                                           |    |
| Bitmask flags for analysis completion tracking                                  | 27 |
| <a href="#">AnalysisVersions</a>                                                |    |
| Algorithm version tracking for analysis results                                 | 28 |
| <a href="#">anonymous_namespace{ActivityBrowser.cpp}</a>                        | 29 |
| <a href="#">anonymous_namespace{AnalysisWorker.cpp}</a>                         | 29 |
| <a href="#">anonymous_namespace{AthleteDialog.cpp}</a>                          | 29 |
| <a href="#">anonymous_namespace{ChartController.cpp}</a>                        | 30 |
| <a href="#">anonymous_namespace{ClimbCandidateFinder.cpp}</a>                   | 30 |
| <a href="#">anonymous_namespace{ClimbDetector.cpp}</a>                          | 31 |
| <a href="#">anonymous_namespace{ClimbMetrics.cpp}</a>                           | 31 |
| <a href="#">anonymous_namespace{ClimbRepository.cpp}</a>                        | 33 |
| <a href="#">anonymous_namespace{ColorProvider.cpp}</a>                          | 33 |
| <a href="#">anonymous_namespace{DatabaseMigrationManager.cpp}</a>               | 33 |
| <a href="#">anonymous_namespace{DateFormatter.cpp}</a>                          | 34 |
| <a href="#">anonymous_namespace{FfmpegPath.cpp}</a>                             | 34 |
| <a href="#">anonymous_namespace{FitParser.cpp}</a>                              | 34 |
| <a href="#">anonymous_namespace{ImportWorker.cpp}</a>                           | 35 |
| <a href="#">anonymous_namespace{MainWindowChartActions.cpp}</a>                 | 35 |
| <a href="#">anonymous_namespace{MainWindowClimbInteractions.cpp}</a>            | 36 |
| <a href="#">anonymous_namespace{MainWindowClimbViews.cpp}</a>                   | 36 |
| <a href="#">anonymous_namespace{MainWindowDatabase.cpp}</a>                     | 37 |
| <a href="#">anonymous_namespace{MainWindowIntervals.cpp}</a>                    | 39 |
| <a href="#">anonymous_namespace{MainWindowStatus.cpp}</a>                       | 40 |
| <a href="#">anonymous_namespace{MainWindowUiBuild.cpp}</a>                      | 40 |
| <a href="#">anonymous_namespace{MainWindowVideo.cpp}</a>                        | 41 |
| <a href="#">anonymous_namespace{MapFitMath.cpp}</a>                             | 41 |
| <a href="#">anonymous_namespace{RideChartSelectionManager.cpp}</a>              | 43 |
| <a href="#">anonymous_namespace{RideChartWidget.cpp}</a>                        | 43 |
| <a href="#">anonymous_namespace{TileCache.cpp}</a>                              | 43 |
| <a href="#">anonymous_namespace{VideoExportDialog.cpp}</a>                      | 43 |
| <a href="#">anonymous_namespace{VideoRenderer.cpp}</a>                          | 44 |
| <a href="#">anonymous_namespace{VideoRenderJob.cpp}</a>                         | 44 |
| <a href="#">anonymous_namespace{VideoTileProvider.cpp}</a>                      | 45 |
| <a href="#">anonymous_namespace{WelcomeWidget.cpp}</a>                          | 45 |
| <a href="#">anonymous_namespace{WorkoutController.cpp}</a>                      | 46 |
| <a href="#">anonymous_namespace{ZoneCalculator.cpp}</a>                         | 47 |
| <a href="#">ClimbDefaults</a>                                                   |    |
| Default climb detection values (mirrors <a href="#">ClimbDetector</a> defaults) | 47 |

|                                                                                            |                    |
|--------------------------------------------------------------------------------------------|--------------------|
| <a href="#">Platform</a>                                                                   |                    |
| Platform-specific abstractions and utilities . . . . .                                     | <a href="#">48</a> |
| <a href="#">SqlQueryHelper</a>                                                             |                    |
| Common SQL patterns and query builders to reduce duplication across repositories . . . . . | <a href="#">49</a> |



## Chapter 3

# Hierarchical Index

### 3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| Activity                                                        | 57  |
| ActivityAnalysisCacheRepository                                 | 63  |
| ActivityAnalysisSettings                                        | 65  |
| ActivityAnalysisSettingsRepository                              | 67  |
| ActivityRepository                                              | 73  |
| AppSettings                                                     | 97  |
| Athlete                                                         | 99  |
| AthleteRepository                                               | 109 |
| Climb                                                           | 141 |
| ClimbDetector                                                   | 147 |
| ClimbDetectorConfig                                             | 154 |
| RideChartSelectionManager::ClimbHandleHitResult                 | 162 |
| ClimbMetrics                                                    | 162 |
| ClimbQuarterMetrics                                             | 167 |
| ClimbRecord                                                     | 167 |
| ClimbRepository                                                 | 171 |
| ColorContext                                                    | 183 |
| ColorProvider                                                   | 185 |
| DailyLoadPoint                                                  | 189 |
| DatabaseConnectionGuard                                         | 189 |
| DatabaseManager                                                 | 191 |
| DateFormatter                                                   | 195 |
| DuplicateActivityInfo                                           | 199 |
| anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard | 204 |
| fit::FileIdMesgListener                                         |     |
| anonymous_namespace{FitParser.cpp}::FitFileIdListener           | 205 |
| FitnessMetricsPoint                                             | 208 |
| FitParser                                                       | 211 |
| MapFitMath::FitResult                                           | 215 |
| FtpEntry                                                        | 218 |
| FTPZones                                                        | 219 |
| GpsCacheData                                                    | 221 |
| MainWindow::ImportBatchSummary                                  | 222 |
| ImportJob                                                       | 230 |
| ImportRecord                                                    | 240 |
| ImportRepository                                                | 241 |
| Interval                                                        | 249 |
| IntervalDetector                                                | 250 |
| IntervalDetectorConfig                                          | 252 |
| IntervalRecord                                                  | 256 |

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| IntervalRepository                                                                | 259 |
| PlannedWorkoutDraft                                                               | 330 |
| PowerCurve                                                                        | 331 |
| PowerCurvePoint                                                                   | 334 |
| PowerZone                                                                         | 341 |
| QChartView                                                                        |     |
| FitnessChartWidget                                                                | 206 |
| PowerCurveWidget                                                                  | 334 |
| PowerHistogramWidget                                                              | 337 |
| RideChartWidget                                                                   | 342 |
| QDialog                                                                           |     |
| AboutDialog                                                                       | 53  |
| AthleteDialog                                                                     | 101 |
| AthleteListDialog                                                                 | 107 |
| CreateDatabaseDialog                                                              | 186 |
| EditActivityDialog                                                                | 199 |
| EditNotesDialog                                                                   | 202 |
| IntervalEditorDialog                                                              | 253 |
| PlannedWorkoutDialog                                                              | 328 |
| VideoExportDialog                                                                 | 387 |
| QMainWindow                                                                       |     |
| MainWindow                                                                        | 265 |
| QObject                                                                           |     |
| ActivityViewController                                                            | 79  |
| AnalysisQueue                                                                     | 87  |
| AnalysisWorker                                                                    | 93  |
| ChartController                                                                   | 121 |
| ImportController                                                                  | 223 |
| ImportProgressModel                                                               | 232 |
| ImportQueue                                                                       | 235 |
| ImportWorker                                                                      | 245 |
| MainWindowActions                                                                 | 296 |
| MapController                                                                     | 304 |
| NavigationController                                                              | 320 |
| TileCache                                                                         | 369 |
| UndoManager                                                                       | 383 |
| VideoRenderJob                                                                    | 400 |
| WorkoutController                                                                 | 418 |
| QTableWidgetItem                                                                  |     |
| anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableWidgetItem             | 72  |
| anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableWidgetItem    | 164 |
| anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableWidgetItem | 178 |
| anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableWidgetItem           | 165 |
| anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableWidgetItem        | 180 |
| QWidget                                                                           |     |
| ActivityBrowser                                                                   | 68  |
| AthleteHeaderWidget                                                               | 105 |
| BasePage                                                                          | 115 |
| ActivitiesPage                                                                    | 54  |
| CalendarPage                                                                      | 116 |
| ChartsPage                                                                        | 140 |
| ClimbsPage                                                                        | 181 |
| FitnessPage                                                                       | 209 |
| IntervalsPage                                                                     | 263 |
| PlaceholderPage                                                                   | 327 |
| PowerPage                                                                         | 339 |
| VideoPage                                                                         | 398 |
| CalendarWidget                                                                    | 118 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| ImportStatusWidget . . . . .                                     | 243 |
| MapRenderer . . . . .                                            | 311 |
| NavigationSidebar . . . . .                                      | 325 |
| WelcomeWidget . . . . .                                          | 414 |
| fit::RecordMesgListener                                          |     |
| FitRecordListener . . . . .                                      | 212 |
| RideData . . . . .                                               | 362 |
| RideDataLoader . . . . .                                         | 363 |
| RideDataSerializer . . . . .                                     | 364 |
| RideRecord . . . . .                                             | 365 |
| fit::SessionMesgListener                                         |     |
| anonymous_namespace{FitParser.cpp}::FitSessionListener . . . . . | 217 |
| anonymous_namespace{ClimbMetrics.cpp}::SliceStats . . . . .      | 367 |
| SmoothingEngine . . . . .                                        | 368 |
| TileCacheConfig . . . . .                                        | 373 |
| TileProvider . . . . .                                           | 374 |
| TileCache::TileRequest . . . . .                                 | 375 |
| VideoRenderer::TileViewport . . . . .                            | 375 |
| TrainingLoad . . . . .                                           | 376 |
| TrainingMetrics . . . . .                                        | 379 |
| TrainingMetricsSummary . . . . .                                 | 380 |
| UndoCommand . . . . .                                            | 382 |
| ClimbEditCommand . . . . .                                       | 159 |
| VideoExportSettings . . . . .                                    | 393 |
| VideoTileId . . . . .                                            | 406 |
| VideoTileIdHash . . . . .                                        | 407 |
| VideoTileProvider . . . . .                                      | 407 |
| VideoTileProviderConfig . . . . .                                | 412 |
| VideoTileStats . . . . .                                         | 413 |
| WeightEntry . . . . .                                            | 413 |
| WorkoutAnalyzer . . . . .                                        | 417 |
| WorkoutPersistenceService . . . . .                              | 424 |
| WorkoutStatistics . . . . .                                      | 425 |
| Zone . . . . .                                                   | 427 |
| ZoneCalculator . . . . .                                         | 428 |
| ZoneDistribution . . . . .                                       | 434 |



# Chapter 4

## Class Index

### 4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                                                 |                                                                        |     |
|---------------------------------------------------------------------------------|------------------------------------------------------------------------|-----|
| <a href="#">AboutDialog</a>                                                     | Displays application about/splash dialog . . . . .                     | 53  |
| <a href="#">ActivitiesPage</a>                                                  | Top-level page for activity browsing, search, and management . . . . . | 54  |
| <a href="#">Activity</a>                                                        | Represents a single recorded activity or workout . . . . .             | 57  |
| <a href="#">ActivityAnalysisCacheRepository</a>                                 | Caches expensive analysis results per activity . . . . .               | 63  |
| <a href="#">ActivityAnalysisSettings</a>                                        | Analysis settings per activity . . . . .                               | 65  |
| <a href="#">ActivityAnalysisSettingsRepository</a>                              | Repository for per-activity analysis settings . . . . .                | 67  |
| <a href="#">ActivityBrowser</a>                                                 | Browse and manage activities . . . . .                                 | 68  |
| <a href="#">anonymous_namespace{ActivityBrowser.cpp}::ActivityDateTableItem</a> |                                                                        | 72  |
| <a href="#">ActivityRepository</a>                                              | Repository for activity record persistence . . . . .                   | 73  |
| <a href="#">ActivityViewController</a>                                          | Manages activity loading, switching, and display operations . . . . .  | 79  |
| <a href="#">AnalysisQueue</a>                                                   | Manages sequential background analysis task queue . . . . .            | 87  |
| <a href="#">AnalysisWorker</a>                                                  | Worker thread for background activity analysis . . . . .               | 93  |
| <a href="#">AppSettings</a>                                                     | Application-wide settings singleton . . . . .                          | 97  |
| <a href="#">Athlete</a>                                                         | Represents an athlete profile . . . . .                                | 99  |
| <a href="#">AthleteDialog</a>                                                   | Dialog for creating or editing athlete profile . . . . .               | 101 |
| <a href="#">AthleteHeaderWidget</a>                                             | Header bar displaying current athlete and ride metrics . . . . .       | 105 |
| <a href="#">AthleteListDialog</a>                                               | Dialog for browsing and managing athletes . . . . .                    | 107 |
| <a href="#">AthleteRepository</a>                                               | Repository for athlete profile data persistence . . . . .              | 109 |
| <a href="#">BasePage</a>                                                        | Abstract base class for all top-level page widgets . . . . .           | 115 |
| <a href="#">CalendarPage</a>                                                    | Top-level page for activity and workout planning . . . . .             | 116 |

|                                                                                             |     |
|---------------------------------------------------------------------------------------------|-----|
| <a href="#">CalendarWidget</a>                                                              |     |
| Calendar widget for activity scheduling and display                                         | 118 |
| <a href="#">ChartController</a>                                                             |     |
| Manages all chart widgets in the analysis view                                              | 121 |
| <a href="#">ChartsPage</a>                                                                  |     |
| Top-level page for activity chart inspection                                                | 140 |
| <a href="#">Climb</a>                                                                       |     |
| Represents a detected climb with comprehensive metrics                                      | 141 |
| <a href="#">ClimbDetector</a>                                                               |     |
| Detects and categorizes climbs in cycling activities                                        | 147 |
| <a href="#">ClimbDetectorConfig</a>                                                         |     |
| Configuration parameters for climb detection                                                | 154 |
| <a href="#">ClimbEditCommand</a>                                                            |     |
| Undo/redo command for climb boundary or field edits                                         | 159 |
| <a href="#">RideChartSelectionManager::ClimbHandleHitResult</a>                             | 162 |
| <a href="#">ClimbMetrics</a>                                                                |     |
| Computes comprehensive climb metrics from activity data                                     | 162 |
| <a href="#">anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem</a>    | 164 |
| <a href="#">anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem</a>           | 165 |
| <a href="#">ClimbQuarterMetrics</a>                                                         |     |
| Metrics for a quarter of a climb segment                                                    | 167 |
| <a href="#">ClimbRecord</a>                                                                 |     |
| Database representation of a stored climb                                                   | 167 |
| <a href="#">ClimbRepository</a>                                                             |     |
| Repository for climb record persistence                                                     | 171 |
| <a href="#">anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem</a> | 178 |
| <a href="#">anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem</a>        | 180 |
| <a href="#">ClimbsPage</a>                                                                  |     |
| Top-level page for climb detection and analysis                                             | 181 |
| <a href="#">ColorContext</a>                                                                |     |
| Context parameters for determining zone colors based on athlete metrics                     | 183 |
| <a href="#">ColorProvider</a>                                                               |     |
| Provides color mapping for athletic metrics                                                 | 185 |
| <a href="#">CreateDatabaseDialog</a>                                                        |     |
| Dialog for creating a new database                                                          | 186 |
| <a href="#">DailyLoadPoint</a>                                                              |     |
| Daily training stress score value                                                           | 189 |
| <a href="#">DatabaseConnectionGuard</a>                                                     |     |
| RAII guard for temporary SQLite database connections                                        | 189 |
| <a href="#">DatabaseManager</a>                                                             |     |
| Manages a single database connection and schema for FitlyzerC                               | 191 |
| <a href="#">DateFormatter</a>                                                               |     |
| Provides date/time formatting consistent with user settings                                 | 195 |
| <a href="#">DuplicateActivityInfo</a>                                                       | 199 |
| <a href="#">EditActivityDialog</a>                                                          |     |
| Dialog for editing activity metadata                                                        | 199 |
| <a href="#">EditNotesDialog</a>                                                             |     |
| Dialog for editing activity notes and weather                                               | 202 |
| <a href="#">anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard</a>             | 204 |
| <a href="#">anonymous_namespace{FitParser.cpp}::FitFileIdListener</a>                       | 205 |
| <a href="#">FitnessChartWidget</a>                                                          |     |
| Displays fitness metrics timeline (CTL, ATL, TSB)                                           | 206 |
| <a href="#">FitnessMetricsPoint</a>                                                         |     |
| Fitness metrics for a single day                                                            | 208 |
| <a href="#">FitnessPage</a>                                                                 |     |
| Top-level page for fitness tracking                                                         | 209 |
| <a href="#">FitParser</a>                                                                   |     |
| Parses FIT files and extracts activity recording data                                       | 211 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| <a href="#">FitRecordListener</a>                                           |     |
| Listener for FIT record messages during parsing . . . . .                   | 212 |
| <a href="#">MapFitMath::FitResult</a>                                       |     |
| Result of fitting GPS bounds to map viewport . . . . .                      | 215 |
| <a href="#">anonymous_namespace{FitParser.cpp}::FitSessionListener</a>      | 217 |
| <a href="#">FtpEntry</a>                                                    |     |
| Represents a historical FTP entry . . . . .                                 | 218 |
| <a href="#">FTPZones</a>                                                    |     |
| Analyzes time spent in FTP-based power zones . . . . .                      | 219 |
| <a href="#">GpsCacheData</a>                                                | 221 |
| <a href="#">MainWindow::ImportBatchSummary</a>                              | 222 |
| <a href="#">ImportController</a>                                            |     |
| Manages activity import operations and UI . . . . .                         | 223 |
| <a href="#">ImportJob</a>                                                   |     |
| Represents a single background FIT import request . . . . .                 | 230 |
| <a href="#">ImportProgressModel</a>                                         |     |
| UI-facing aggregate state for background import queue activity . . . . .    | 232 |
| <a href="#">ImportQueue</a>                                                 |     |
| Sequential queue that dispatches background FIT import jobs . . . . .       | 235 |
| <a href="#">ImportRecord</a>                                                |     |
| Log entry for an activity import . . . . .                                  | 240 |
| <a href="#">ImportRepository</a>                                            |     |
| Repository for import log persistence . . . . .                             | 241 |
| <a href="#">ImportStatusWidget</a>                                          |     |
| Toolbar widget showing live background import queue progress . . . . .      | 243 |
| <a href="#">ImportWorker</a>                                                |     |
| Worker that imports FIT files in a dedicated background thread . . . . .    | 245 |
| <a href="#">Interval</a>                                                    |     |
| Represents a single work or recovery interval . . . . .                     | 249 |
| <a href="#">IntervalDetector</a>                                            |     |
| Detects work and recovery intervals in activity data . . . . .              | 250 |
| <a href="#">IntervalDetectorConfig</a>                                      |     |
| Configuration parameters for interval detection . . . . .                   | 252 |
| <a href="#">IntervalEditorDialog</a>                                        |     |
| Dialog for editing interval metadata . . . . .                              | 253 |
| <a href="#">IntervalRecord</a>                                              |     |
| Database representation of an interval record . . . . .                     | 256 |
| <a href="#">IntervalRepository</a>                                          |     |
| Repository for interval record persistence . . . . .                        | 259 |
| <a href="#">IntervalsPage</a>                                               |     |
| Top-level page for interval analysis and management . . . . .               | 263 |
| <a href="#">MainWindow</a>                                                  |     |
| Main application window . . . . .                                           | 265 |
| <a href="#">MainWindowActions</a>                                           |     |
| Manages menus, toolbars, actions, and keyboard shortcuts . . . . .          | 296 |
| <a href="#">MapController</a>                                               |     |
| Manages map rendering and interaction . . . . .                             | 304 |
| <a href="#">MapRenderer</a>                                                 |     |
| Interactive map widget for displaying and editing activity routes . . . . . | 311 |
| <a href="#">NavigationController</a>                                        |     |
| Manages top-level page navigation in the main window . . . . .              | 320 |
| <a href="#">NavigationSidebar</a>                                           |     |
| Main navigation sidebar . . . . .                                           | 325 |
| <a href="#">PlaceholderPage</a>                                             |     |
| A placeholder page shown for features not yet fully migrated . . . . .      | 327 |
| <a href="#">PlannedWorkoutDialog</a>                                        |     |
| Dialog for creating or editing planned workouts . . . . .                   | 328 |

|                                                                                 |     |
|---------------------------------------------------------------------------------|-----|
| <a href="#">PlannedWorkoutDraft</a>                                             |     |
| Draft data for a planned workout . . . . .                                      | 330 |
| <a href="#">PowerCurve</a>                                                      |     |
| Computes power duration curve (PDC) for an activity . . . . .                   | 331 |
| <a href="#">PowerCurvePoint</a>                                                 |     |
| A single point on a power duration curve . . . . .                              | 334 |
| <a href="#">PowerCurveWidget</a>                                                |     |
| Displays power output over duration curve . . . . .                             | 334 |
| <a href="#">PowerHistogramWidget</a>                                            |     |
| Displays power distribution histogram . . . . .                                 | 337 |
| <a href="#">PowerPage</a>                                                       |     |
| Top-level page for power analysis . . . . .                                     | 339 |
| <a href="#">PowerZone</a>                                                       |     |
| Represents a single power zone with time spent . . . . .                        | 341 |
| <a href="#">RideChartWidget</a>                                                 |     |
| Interactive chart displaying ride data by selected metric . . . . .             | 342 |
| <a href="#">RideData</a>                                                        |     |
| Container for all recorded samples and metadata for a single activity . . . . . | 362 |
| <a href="#">RideDataLoader</a>                                                  |     |
| Loads activity ride data samples from the database . . . . .                    | 363 |
| <a href="#">RideDataSerializer</a>                                              |     |
| Persists ride data to database . . . . .                                        | 364 |
| <a href="#">RideRecord</a>                                                      |     |
| A single recorded sample from an activity recording . . . . .                   | 365 |
| <a href="#">anonymous_namespace{ClimbMetrics.cpp}::SliceStats</a>               | 367 |
| <a href="#">SmoothingEngine</a>                                                 |     |
| Provides data smoothing algorithms for activity metrics . . . . .               | 368 |
| <a href="#">TileCache</a>                                                       |     |
| Caches map tiles in memory and on disk . . . . .                                | 369 |
| <a href="#">TileCacheConfig</a>                                                 |     |
| Configuration for tile caching behavior . . . . .                               | 373 |
| <a href="#">TileProvider</a>                                                    |     |
| Tile provider configuration . . . . .                                           | 374 |
| <a href="#">TileCache::TileRequest</a>                                          | 375 |
| <a href="#">VideoRenderer::TileViewPort</a>                                     | 375 |
| <a href="#">TrainingLoad</a>                                                    |     |
| Computes training load metrics from activity power data . . . . .               | 376 |
| <a href="#">TrainingMetrics</a>                                                 |     |
| Computes training metrics from activity data . . . . .                          | 379 |
| <a href="#">TrainingMetricsSummary</a>                                          |     |
| Summary of training-related metrics for an activity . . . . .                   | 380 |
| <a href="#">UndoCommand</a>                                                     |     |
| Abstract base class for all undo/redo operations . . . . .                      | 382 |
| <a href="#">UndoManager</a>                                                     |     |
| Central undo/redo stack manager . . . . .                                       | 383 |
| <a href="#">VideoExportDialog</a>                                               |     |
| Dialog for configuring and launching activity video export . . . . .            | 387 |
| <a href="#">VideoExportSettings</a>                                             |     |
| Configuration for activity video export . . . . .                               | 393 |
| <a href="#">VideoPage</a>                                                       |     |
| Top-level page for video creation and export . . . . .                          | 398 |
| <a href="#">VideoRenderJob</a>                                                  |     |
| Renders activity video with map overlay and data visualization . . . . .        | 400 |
| <a href="#">VideoTileId</a>                                                     |     |
| Unique identifier for a map tile (z/x/y coordinates) . . . . .                  | 406 |
| <a href="#">VideoTileIdHash</a>                                                 |     |
| Hash function for <a href="#">VideoTileId</a> . . . . .                         | 407 |



|                                                                                          |     |
|------------------------------------------------------------------------------------------|-----|
| <a href="#">VideoTileProvider</a>                                                        |     |
| Manages map tiles for video export . . . . .                                             | 407 |
| <a href="#">VideoTileProviderConfig</a>                                                  |     |
| Configuration for video tile provider . . . . .                                          | 412 |
| <a href="#">VideoTileStats</a>                                                           |     |
| Tile availability statistics for export . . . . .                                        | 413 |
| <a href="#">WeightEntry</a>                                                              |     |
| Represents a historical weight entry . . . . .                                           | 413 |
| <a href="#">WelcomeWidget</a>                                                            |     |
| Dashboard welcome screen shown when no database is open or no activities exist . . . . . | 414 |
| <a href="#">WorkoutAnalyzer</a>                                                          |     |
| Analyzes ride data to compute workout statistics . . . . .                               | 417 |
| <a href="#">WorkoutController</a>                                                        |     |
| Manages activity loading, import, and analysis . . . . .                                 | 418 |
| <a href="#">WorkoutPersistenceService</a>                                                |     |
| . . . . .                                                                                | 424 |
| <a href="#">WorkoutStatistics</a>                                                        |     |
| Statistics summary for a workout activity . . . . .                                      | 425 |
| <a href="#">Zone</a>                                                                     |     |
| Represents a single training or power zone with bounds and color . . . . .               | 427 |
| <a href="#">ZoneCalculator</a>                                                           |     |
| Calculates training zones for various metrics . . . . .                                  | 428 |
| <a href="#">ZoneDistribution</a>                                                         |     |
| Distribution of time or distance spent in a specific zone . . . . .                      | 434 |



# Chapter 5

## File Index

### 5.1 File List

Here is a list of all documented files with brief descriptions:

|                                                              |     |
|--------------------------------------------------------------|-----|
| src/ <a href="#">fitlyzerC.cpp</a>                           | 553 |
| src/analysis/ <a href="#">ClimbDetector.cpp</a>              | 439 |
| src/analysis/ <a href="#">ClimbDetector.h</a>                | 446 |
| src/analysis/ <a href="#">ClimbMetrics.cpp</a>               | 447 |
| src/analysis/ <a href="#">ClimbMetrics.h</a>                 | 453 |
| src/analysis/ <a href="#">FTPZones.cpp</a>                   | 454 |
| src/analysis/ <a href="#">FTPZones.h</a>                     | 455 |
| src/analysis/ <a href="#">IntervalDetector.cpp</a>           | 455 |
| src/analysis/ <a href="#">IntervalDetector.h</a>             | 458 |
| src/analysis/ <a href="#">PowerCurve.cpp</a>                 | 459 |
| src/analysis/ <a href="#">PowerCurve.h</a>                   | 460 |
| src/analysis/ <a href="#">SmoothingEngine.cpp</a>            | 465 |
| src/analysis/ <a href="#">SmoothingEngine.h</a>              | 466 |
| src/analysis/ <a href="#">TrainingLoad.cpp</a>               | 466 |
| src/analysis/ <a href="#">TrainingLoad.h</a>                 | 467 |
| src/analysis/ <a href="#">TrainingMetrics.cpp</a>            | 468 |
| src/analysis/ <a href="#">TrainingMetrics.h</a>              | 468 |
| src/analysis/ <a href="#">WorkoutAnalyzer.cpp</a>            | 469 |
| src/analysis/ <a href="#">WorkoutAnalyzer.h</a>              | 470 |
| src/analysis/climb/ <a href="#">ClimbCandidateFinder.cpp</a> | 437 |
| src/analysis/climb/ <a href="#">ClimbCandidateFinder.h</a>   | 438 |
| src/analysis/climb/ <a href="#">ClimbClassifier.cpp</a>      | 438 |
| src/analysis/climb/ <a href="#">ClimbClassifier.h</a>        | 438 |
| src/analysis/climb/ <a href="#">ClimbMerger.cpp</a>          | 438 |
| src/analysis/climb/ <a href="#">ClimbMerger.h</a>            | 439 |
| src/analysis/queue/ <a href="#">AnalysisQueue.cpp</a>        | 460 |
| src/analysis/queue/ <a href="#">AnalysisQueue.h</a>          | 461 |
| src/analysis/queue/ <a href="#">AnalysisWorker.cpp</a>       | 462 |
| src/analysis/queue/ <a href="#">AnalysisWorker.h</a>         | 465 |
| src/charts/ <a href="#">FitnessChartWidget.cpp</a>           | 473 |
| src/charts/ <a href="#">FitnessChartWidget.h</a>             | 474 |
| src/charts/ <a href="#">Metric.h</a>                         | 474 |
| src/charts/ <a href="#">PowerCurveWidget.cpp</a>             | 475 |
| src/charts/ <a href="#">PowerCurveWidget.h</a>               | 477 |
| src/charts/ <a href="#">PowerHistogramWidget.cpp</a>         | 478 |
| src/charts/ <a href="#">PowerHistogramWidget.h</a>           | 479 |
| src/charts/ <a href="#">RideChartWidget.cpp</a>              | 480 |
| src/charts/ <a href="#">RideChartWidget.h</a>                | 501 |
| src/charts/data/ <a href="#">RideChartDataBuilder.cpp</a>    | 471 |
| src/charts/data/ <a href="#">RideChartDataBuilder.h</a>      | 472 |

|                                                       |     |
|-------------------------------------------------------|-----|
| src/charts/rendering/RideChartRenderer.cpp            | 479 |
| src/charts/rendering/RideChartRenderer.h              | 480 |
| src/charts/selection/RideChartSelectionManager.cpp    | 503 |
| src/charts/selection/RideChartSelectionManager.h      | 504 |
| src/core/ActivityAnalysisFlags.h                      | 504 |
| src/core/AnalysisVersions.h                           | 505 |
| src/core/settings/AppSettings.cpp                     | 505 |
| src/core/settings/AppSettings.h                       | 506 |
| src/core/settings/DateFormatter.cpp                   | 506 |
| src/core/settings/DateFormatter.h                     | 507 |
| src/core/undo/ClimbEditCommand.cpp                    | 508 |
| src/core/undo/ClimbEditCommand.h                      | 508 |
| src/core/undo/UndoCommand.h                           | 509 |
| src/core/undo/UndoManager.cpp                         | 509 |
| src/core/undo/UndoManager.h                           | 510 |
| src/core/zones/ColorProvider.cpp                      | 511 |
| src/core/zones/ColorProvider.h                        | 512 |
| src/core/zones/ZoneCalculator.cpp                     | 512 |
| src/core/zones/ZoneCalculator.h                       | 515 |
| src/core/zones/ZoneDefinition.cpp                     | 515 |
| src/core/zones/ZoneDefinition.h                       | 516 |
| src/database/ActivityAnalysisCacheRepository.cpp      | 517 |
| src/database/ActivityAnalysisCacheRepository.h        | 517 |
| src/database/ActivityAnalysisSettingsRepository.cpp   | 518 |
| src/database/ActivityAnalysisSettingsRepository.h     | 518 |
| src/database/ActivityRepository.cpp                   | 519 |
| src/database/ActivityRepository.h                     | 523 |
| src/database/AthleteRepository.cpp                    | 524 |
| src/database/AthleteRepository.h                      | 526 |
| src/database/ClimbRepository.cpp                      | 528 |
| src/database/ClimbRepository.h                        | 530 |
| src/database/DatabaseConnectionGuard.cpp              | 532 |
| src/database/DatabaseConnectionGuard.h                | 533 |
| src/database/DatabaseManager.cpp                      | 533 |
| src/database/DatabaseManager.h                        | 535 |
| src/database/DatabaseSchema.cpp                       | 535 |
| src/database/DatabaseSchema.h                         | 535 |
| src/database/ImportRepository.cpp                     | 539 |
| src/database/ImportRepository.h                       | 540 |
| src/database/IntervalRepository.cpp                   | 540 |
| src/database/IntervalRepository.h                     | 542 |
| src/database/SqlQueryHelper.h                         | 547 |
| src/database/connection/DatabaseConnectionFactory.cpp | 532 |
| src/database/connection/DatabaseConnectionFactory.h   | 532 |
| src/database/migrations/DatabaseMigrationManager.cpp  | 543 |
| src/database/migrations/DatabaseMigrationManager.h    | 547 |
| src/fit/FitParser.cpp                                 | 547 |
| src/fit/FitParser.h                                   | 550 |
| src/fit/FitRecordListener.cpp                         | 550 |
| src/fit/FitRecordListener.h                           | 552 |
| src/fit/RideData.h                                    | 552 |
| src/gui/AboutDialog.cpp                               | 553 |
| src/gui/AboutDialog.h                                 | 554 |
| src/gui/ActivityBrowser.cpp                           | 554 |
| src/gui/ActivityBrowser.h                             | 558 |
| src/gui/AthleteDialog.cpp                             | 559 |
| src/gui/AthleteDialog.h                               | 562 |
| src/gui/AthleteHeaderWidget.cpp                       | 563 |

|                                                |     |
|------------------------------------------------|-----|
| src/gui/AthleteHeaderWidget.h                  | 564 |
| src/gui/AthleteListDialog.cpp                  | 564 |
| src/gui/AthleteListDialog.h                    | 566 |
| src/gui/CalendarWidget.cpp                     | 566 |
| src/gui/CalendarWidget.h                       | 570 |
| src/gui/CreateDatabaseDialog.cpp               | 591 |
| src/gui/CreateDatabaseDialog.h                 | 593 |
| src/gui/EditActivityDialog.cpp                 | 594 |
| src/gui/EditActivityDialog.h                   | 595 |
| src/gui/EditNotesDialog.cpp                    | 595 |
| src/gui/EditNotesDialog.h                      | 596 |
| src/gui/ImportStatusWidget.cpp                 | 596 |
| src/gui/ImportStatusWidget.h                   | 597 |
| src/gui/IntervalEditorDialog.cpp               | 598 |
| src/gui/IntervalEditorDialog.h                 | 598 |
| src/gui/MainWindow.cpp                         | 599 |
| src/gui/MainWindow.h                           | 602 |
| src/gui/MainWindowAthlete.cpp                  | 607 |
| src/gui/MainWindowChartActions.cpp             | 612 |
| src/gui/MainWindowCharts.cpp                   | 614 |
| src/gui/MainWindowClimbInteractions.cpp        | 615 |
| src/gui/MainWindowClimbViews.cpp               | 625 |
| src/gui/MainWindowContext.cpp                  | 629 |
| src/gui/MainWindowDatabase.cpp                 | 630 |
| src/gui/MainWindowDetection.cpp                | 634 |
| src/gui/MainWindowImport.cpp                   | 637 |
| src/gui/MainWindowIntervals.cpp                | 640 |
| src/gui/MainWindowReadiness.cpp                | 645 |
| src/gui/MainWindowStatus.cpp                   | 648 |
| src/gui/MainWindowUi.cpp                       | 649 |
| src/gui/MainWindowUiBuild.cpp                  | 651 |
| src/gui/MainWindowVideo.cpp                    | 665 |
| src/gui/MainWindowWatch.cpp                    | 666 |
| src/gui/NavigationSidebar.cpp                  | 668 |
| src/gui/NavigationSidebar.h                    | 669 |
| src/gui/PlannedWorkoutDialog.cpp               | 677 |
| src/gui/PlannedWorkoutDialog.h                 | 678 |
| src/gui/WelcomeWidget.cpp                      | 679 |
| src/gui/WelcomeWidget.h                        | 681 |
| src/gui/WorkoutController.cpp                  | 682 |
| src/gui/WorkoutController.h                    | 687 |
| src/gui/controllers/ActivityViewController.cpp | 571 |
| src/gui/controllers/ActivityViewController.h   | 573 |
| src/gui/controllers/ChartController.cpp        | 574 |
| src/gui/controllers/ChartController.h          | 581 |
| src/gui/controllers/ImportController.cpp       | 583 |
| src/gui/controllers/ImportController.h         | 584 |
| src/gui/controllers/MainWindowActions.cpp      | 585 |
| src/gui/controllers/MainWindowActions.h        | 586 |
| src/gui/controllers/MapController.cpp          | 588 |
| src/gui/controllers/MapController.h            | 589 |
| src/gui/controllers/NavigationController.cpp   | 590 |
| src/gui/controllers/NavigationController.h     | 591 |
| src/gui/pages/ActivitiesPage.cpp               | 670 |
| src/gui/pages/ActivitiesPage.h                 | 671 |
| src/gui/pages/BasePage.h                       | 671 |
| src/gui/pages/CalendarPage.cpp                 | 671 |
| src/gui/pages/CalendarPage.h                   | 672 |

|                                             |     |
|---------------------------------------------|-----|
| src/gui/pages/ChartsPage.cpp                | 672 |
| src/gui/pages/ChartsPage.h                  | 672 |
| src/gui/pages/ClimbsPage.cpp                | 672 |
| src/gui/pages/ClimbsPage.h                  | 673 |
| src/gui/pages/FitnessPage.cpp               | 673 |
| src/gui/pages/FitnessPage.h                 | 674 |
| src/gui/pages/IntervalsPage.cpp             | 674 |
| src/gui/pages/IntervalsPage.h               | 674 |
| src/gui/pages/PlaceholderPage.h             | 675 |
| src/gui/pages/PowerPage.cpp                 | 675 |
| src/gui/pages/PowerPage.h                   | 675 |
| src/gui/pages/VideoPage.cpp                 | 676 |
| src/gui/pages/VideoPage.h                   | 677 |
| src/import/ImportJob.cpp                    | 689 |
| src/import/ImportJob.h                      | 689 |
| src/import/ImportProgressModel.cpp          | 690 |
| src/import/ImportProgressModel.h            | 691 |
| src/import/ImportQueue.cpp                  | 691 |
| src/import/ImportQueue.h                    | 693 |
| src/import/ImportWorker.cpp                 | 694 |
| src/import/ImportWorker.h                   | 697 |
| src/maps/MapFitMath.cpp                     | 699 |
| src/maps/MapFitMath.h                       | 701 |
| src/maps/MapRenderer.cpp                    | 701 |
| src/maps/MapRenderer.h                      | 711 |
| src/maps/TileCache.cpp                      | 715 |
| src/maps/TileCache.h                        | 719 |
| src/maps/cache/DiskTileCache.cpp            | 698 |
| src/maps/cache/DiskTileCache.h              | 698 |
| src/maps/cache/GpsCacheBuilder.cpp          | 699 |
| src/maps/cache/GpsCacheBuilder.h            | 699 |
| src/maps/network/TileDownloader.cpp         | 712 |
| src/maps/network/TileDownloader.h           | 713 |
| src/maps/providers/TileProviderRegistry.cpp | 713 |
| src/maps/providers/TileProviderRegistry.h   | 714 |
| src/maps/rendering/RouteColorizer.cpp       | 714 |
| src/maps/rendering/RouteColorizer.h         | 715 |
| src/model/RideDataLoader.cpp                | 720 |
| src/model/RideDataLoader.h                  | 721 |
| src/model/RideDataSerializer.cpp            | 721 |
| src/model/RideDataSerializer.h              | 723 |
| src/platform/Platform.cpp                   | 724 |
| src/platform/Platform.h                     | 724 |
| src/platform/Platform_linux.cpp             | 724 |
| src/platform/Platform_macos.cpp             | 725 |
| src/platform/Platform_windows.cpp           | 725 |
| src/utls/FmpegPath.cpp                      | 725 |
| src/utls/FmpegPath.h                        | 726 |
| src/utls/UuidGenerator.h                    | 726 |
| src/video/VideoExportDialog.cpp             | 731 |
| src/video/VideoExportDialog.h               | 736 |
| src/video/VideoExportSettings.h             | 737 |
| src/video/VideoRenderJob.cpp                | 738 |
| src/video/VideoRenderJob.h                  | 747 |
| src/video/VideoTileProvider.cpp             | 748 |
| src/video/VideoTileProvider.h               | 749 |
| src/video/encoding/VideoEncoder.cpp         | 727 |
| src/video/encoding/VideoEncoder.h           | 727 |

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| src/video/overlay/ <a href="#">OverlayRenderer.cpp</a> . . . . .      | 727 |
| src/video/overlay/ <a href="#">OverlayRenderer.h</a> . . . . .        | 728 |
| src/video/rendering/ <a href="#">VideoRenderer.cpp</a> . . . . .      | 728 |
| src/video/rendering/ <a href="#">VideoRenderer.h</a> . . . . .        | 730 |
| src/workouts/ <a href="#">WorkoutPersistenceService.cpp</a> . . . . . | 750 |
| src/workouts/ <a href="#">WorkoutPersistenceService.h</a> . . . . .   | 751 |





## Chapter 6

# Namespace Documentation

### 6.1 ActivityAnalysisFlags Namespace Reference

Bitmask flags for analysis completion tracking.

#### Variables

- constexpr int [None](#) = 0  
*No analysis flags set.*
- constexpr int [HasClimbs](#) = 1 << 0  
*Climb detection has been computed.*
- constexpr int [HasIntervals](#) = 1 << 1  
*Interval detection has been computed.*
- constexpr int [HasPowerCurve](#) = 1 << 2  
*Power curve has been computed.*
- constexpr int [HasHistogram](#) = 1 << 3  
*Power histogram has been computed.*
- constexpr int [HasVideo](#) = 1 << 4  
*Video export has been created.*

#### 6.1.1 Detailed Description

Bitmask flags for analysis completion tracking.

Flags are stored in `activities.analysis_flags` to track which analysis products have been computed, allowing the UI to warn on outdated results.

#### 6.1.2 Variable Documentation

##### 6.1.2.1 HasClimbs

```
int ActivityAnalysisFlags::HasClimbs = 1 << 0 [constexpr]
```

[Climb](#) detection has been computed.

Definition at line 22 of file [ActivityAnalysisFlags.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

##### 6.1.2.2 HasHistogram

```
int ActivityAnalysisFlags::HasHistogram = 1 << 3 [constexpr]
```

Power histogram has been computed.

Definition at line 37 of file [ActivityAnalysisFlags.h](#).

### 6.1.2.3 HasIntervals

```
int ActivityAnalysisFlags::HasIntervals = 1 << 1 [constexpr]
```

[Interval](#) detection has been computed.

Definition at line 27 of file [ActivityAnalysisFlags.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

### 6.1.2.4 HasPowerCurve

```
int ActivityAnalysisFlags::HasPowerCurve = 1 << 2 [constexpr]
```

Power curve has been computed.

Definition at line 32 of file [ActivityAnalysisFlags.h](#).

### 6.1.2.5 HasVideo

```
int ActivityAnalysisFlags::HasVideo = 1 << 4 [constexpr]
```

Video export has been created.

Definition at line 42 of file [ActivityAnalysisFlags.h](#).

### 6.1.2.6 None

```
int ActivityAnalysisFlags::None = 0 [constexpr]
```

No analysis flags set.

Definition at line 17 of file [ActivityAnalysisFlags.h](#).

## 6.2 AnalysisVersions Namespace Reference

Algorithm version tracking for analysis results.

### Variables

- constexpr int [ClimbDetection](#) = 1  
*Climb detection algorithm version.*
- constexpr int [IntervalDetection](#) = 1  
*Interval detection algorithm version.*

### 6.2.1 Detailed Description

Algorithm version tracking for analysis results.

Version constants are incremented when algorithms change to track data compatibility and warn on outdated results.

### 6.2.2 Variable Documentation

#### 6.2.2.1 ClimbDetection

```
int AnalysisVersions::ClimbDetection = 1 [constexpr]
```

[Climb](#) detection algorithm version.

Definition at line 17 of file [AnalysisVersions.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

#### 6.2.2.2 IntervalDetection

```
int AnalysisVersions::IntervalDetection = 1 [constexpr]
```

[Interval](#) detection algorithm version.

Definition at line 22 of file [AnalysisVersions.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

## 6.3 anonymous\_namespace{ActivityBrowser.cpp} Namespace Reference

### Classes

- class [ActivityDateTableItem](#)

### Variables

- constexpr int [kNotesRole](#) = Qt::UserRole + 1
- constexpr int [kWeatherRole](#) = Qt::UserRole + 2
- constexpr int [kIsoDateRole](#) = Qt::UserRole + 3

### 6.3.1 Variable Documentation

#### 6.3.1.1 kIsoDateRole

int anonymous\_namespace{ActivityBrowser.cpp}::kIsoDateRole = Qt::UserRole + 3 [constexpr]  
Definition at line 27 of file [ActivityBrowser.cpp](#).

#### 6.3.1.2 kNotesRole

int anonymous\_namespace{ActivityBrowser.cpp}::kNotesRole = Qt::UserRole + 1 [constexpr]  
Definition at line 25 of file [ActivityBrowser.cpp](#).

#### 6.3.1.3 kWeatherRole

int anonymous\_namespace{ActivityBrowser.cpp}::kWeatherRole = Qt::UserRole + 2 [constexpr]  
Definition at line 26 of file [ActivityBrowser.cpp](#).

## 6.4 anonymous\_namespace{AnalysisWorker.cpp} Namespace Reference

### Functions

- void [applySqlitePragmas](#) (QSqlDatabase &db)

### 6.4.1 Function Documentation

#### 6.4.1.1 applySqlitePragmas()

void anonymous\_namespace{AnalysisWorker.cpp}::applySqlitePragmas (  
    QSqlDatabase & db)

Definition at line 14 of file [AnalysisWorker.cpp](#).

## 6.5 anonymous\_namespace{AthleteDialog.cpp} Namespace Reference

### Functions

- QString [displayDateOrRaw](#) (const QString &raw)
- QString [tolsoOrRaw](#) (const QString &text)

### Variables

- constexpr int [kEntryIdRole](#) = Qt::UserRole

## 6.5.1 Function Documentation

### 6.5.1.1 displayDateOrRaw()

```
QString anonymous_namespace{AthleteDialog.cpp}::displayDateOrRaw (
    const QString & raw)
```

Definition at line 27 of file [AthleteDialog.cpp](#).

### 6.5.1.2 toIsoOrRaw()

```
QString anonymous_namespace{AthleteDialog.cpp}::toIsoOrRaw (
    const QString & text)
```

Definition at line 33 of file [AthleteDialog.cpp](#).

## 6.5.2 Variable Documentation

### 6.5.2.1 kEntryIdRole

```
int anonymous_namespace{AthleteDialog.cpp}::kEntryIdRole = Qt::UserRole [constexpr]
```

Definition at line 25 of file [AthleteDialog.cpp](#).

## 6.6 anonymous\_namespace{ChartController.cpp} Namespace Reference

### Functions

- QString [fmtDur](#) (double seconds)
- QString [rangeLabelForZone](#) (const [Zone](#) &zone, ColorMetric metric)

## 6.6.1 Function Documentation

### 6.6.1.1 fmtDur()

```
QString anonymous_namespace{ChartController.cpp}::fmtDur (
    double seconds)
```

Definition at line 36 of file [ChartController.cpp](#).

### 6.6.1.2 rangeLabelForZone()

```
QString anonymous_namespace{ChartController.cpp}::rangeLabelForZone (
    const Zone & zone,
    ColorMetric metric)
```

Definition at line 49 of file [ChartController.cpp](#).

## 6.7 anonymous\_namespace{ClimbCandidateFinder.cpp} Namespace Reference

### Functions

- double [segmentMeters](#) (double lat1, double lon1, double lat2, double lon2)

## 6.7.1 Function Documentation

### 6.7.1.1 segmentMeters()

```
double anonymous_namespace{ClimbCandidateFinder.cpp}::segmentMeters (
    double lat1,
    double lon1,
```

```
double lat2,
double lon2)
```

Definition at line 11 of file [ClimbCandidateFinder.cpp](#).

## 6.8 anonymous\_namespace{ClimbDetector.cpp} Namespace Reference

### Functions

- bool [isFinite](#) (double value)
- double [clampValue](#) (double value, double lo, double hi)
- std::vector< double > [buildFilteredAltitudeMeters](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, double maxVerticalSpeedMps)
- double [estimateSpeedKmhAt](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, int index)

### 6.8.1 Function Documentation

#### 6.8.1.1 buildFilteredAltitudeMeters()

```
std::vector< double > anonymous_namespace{ClimbDetector.cpp}::buildFilteredAltitudeMeters (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    double maxVerticalSpeedMps)
```

Definition at line 26 of file [ClimbDetector.cpp](#).

#### 6.8.1.2 clampValue()

```
double anonymous_namespace{ClimbDetector.cpp}::clampValue (
    double value,
    double lo,
    double hi)
```

Definition at line 21 of file [ClimbDetector.cpp](#).

#### 6.8.1.3 estimateSpeedKmhAt()

```
double anonymous_namespace{ClimbDetector.cpp}::estimateSpeedKmhAt (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    int index)
```

Definition at line 74 of file [ClimbDetector.cpp](#).

#### 6.8.1.4 isFinite()

```
bool anonymous_namespace{ClimbDetector.cpp}::isFinite (
    double value)
```

Definition at line 16 of file [ClimbDetector.cpp](#).

## 6.9 anonymous\_namespace{ClimbMetrics.cpp} Namespace Reference

### Classes

- struct [SliceStats](#)

### Functions

- bool [isFinite](#) (double value)
- double [computeLinearRegressionSlope](#) (const std::vector< double > &x, const std::vector< double > &y, double \*interceptOut=nullptr)

- QString [classifyClimbCategory](#) (double gainM, double difficultyScore)
- QString [classifyClimbShape](#) (const std::vector< double > &gradients, double lengthKm, double gainM, double avgGradient, double maxGradient)
- [SliceStats](#) [computeSliceStats](#) (const [RideData](#) &rideData, int startIdx, int endIdx, const std::vector< double > &basis, double sliceStart, double sliceEnd)
- double [computeNormalizedPower](#) (const std::vector< double > &power)

## 6.9.1 Function Documentation

### 6.9.1.1 [classifyClimbCategory\(\)](#)

```
QString anonymous_namespace{ClimbMetrics.cpp}::classifyClimbCategory (
    double gainM,
    double difficultyScore)
```

Definition at line 64 of file [ClimbMetrics.cpp](#).

### 6.9.1.2 [classifyClimbShape\(\)](#)

```
QString anonymous_namespace{ClimbMetrics.cpp}::classifyClimbShape (
    const std::vector< double > & gradients,
    double lengthKm,
    double gainM,
    double avgGradient,
    double maxGradient)
```

Definition at line 77 of file [ClimbMetrics.cpp](#).

### 6.9.1.3 [computeLinearRegressionSlope\(\)](#)

```
double anonymous_namespace{ClimbMetrics.cpp}::computeLinearRegressionSlope (
    const std::vector< double > & x,
    const std::vector< double > & y,
    double * interceptOut = nullptr)
```

Definition at line 24 of file [ClimbMetrics.cpp](#).

### 6.9.1.4 [computeNormalizedPower\(\)](#)

```
double anonymous_namespace{ClimbMetrics.cpp}::computeNormalizedPower (
    const std::vector< double > & power)
```

Definition at line 194 of file [ClimbMetrics.cpp](#).

### 6.9.1.5 [computeSliceStats\(\)](#)

```
SliceStats anonymous_namespace{ClimbMetrics.cpp}::computeSliceStats (
    const RideData & rideData,
    int startIdx,
    int endIdx,
    const std::vector< double > & basis,
    double sliceStart,
    double sliceEnd)
```

Definition at line 141 of file [ClimbMetrics.cpp](#).

### 6.9.1.6 [isFinite\(\)](#)

```
bool anonymous_namespace{ClimbMetrics.cpp}::isFinite (
    double value)
```

Definition at line 19 of file [ClimbMetrics.cpp](#).

## 6.10 anonymous\_namespace{ClimbRepository.cpp} Namespace Reference

### Functions

- [ClimbRecord recordFromQuery](#) (QSqlQuery &q)

### Variables

- const char \* [kSelectCols](#)

### 6.10.1 Function Documentation

#### 6.10.1.1 recordFromQuery()

[ClimbRecord](#) anonymous\_namespace{ClimbRepository.cpp}::recordFromQuery (  
QSqlQuery & q)

Definition at line 24 of file [ClimbRepository.cpp](#).

### 6.10.2 Variable Documentation

#### 6.10.2.1 kSelectCols

const char\* anonymous\_namespace{ClimbRepository.cpp}::kSelectCols

Initial value:

```
=
"id, activity_id, COALESCE(uuid,"), start_seconds, end_seconds, "
" COALESCE(original_start_seconds,-1.0), COALESCE(original_end_seconds,-1.0), "
" COALESCE(name,"), COALESCE(elevation_gain_m,0.0), COALESCE(length_km,0.0), "
" COALESCE(average_gradient,0.0), "
" COALESCE(avg_power,0.0), COALESCE(np,0.0), COALESCE(avg_hr,0.0), "
" COALESCE(avg_cadence,0.0), COALESCE(vam,0.0), "
" COALESCE(source,'auto'), COALESCE(locked,0), COALESCE(deleted,0), "
" COALESCE(algorithm_version,1), "
" COALESCE(notes,"), COALESCE(favorite,0), COALESCE(rating,0) "
```

Definition at line 13 of file [ClimbRepository.cpp](#).

## 6.11 anonymous\_namespace{ColorProvider.cpp} Namespace Reference

### Functions

- QColor [interpolateColor](#) (const QColor &a, const QColor &b, double t)

### 6.11.1 Function Documentation

#### 6.11.1.1 interpolateColor()

QColor anonymous\_namespace{ColorProvider.cpp}::interpolateColor (  
const QColor & a,  
const QColor & b,  
double t)

Definition at line 12 of file [ColorProvider.cpp](#).

## 6.12 anonymous\_namespace{DatabaseMigrationManager.cpp} Namespace Reference

### Functions

- bool [columnExists](#) (QSqlDatabase &db, const QString &tableName, const QString &columnName)
- bool [ensureActivityTimeColumns](#) (QSqlDatabase &db, QString \*errorOut)

## 6.12.1 Function Documentation

### 6.12.1.1 columnExists()

```
bool anonymous_namespace{DatabaseMigrationManager.cpp}::columnExists (
    QSqlDatabase & db,
    const QString & tableName,
    const QString & columnName)
```

Definition at line 13 of file [DatabaseMigrationManager.cpp](#).

### 6.12.1.2 ensureActivityTimeColumns()

```
bool anonymous_namespace{DatabaseMigrationManager.cpp}::ensureActivityTimeColumns (
    QSqlDatabase & db,
    QString * errorOut)
```

Definition at line 25 of file [DatabaseMigrationManager.cpp](#).

## 6.13 anonymous\_namespace{DateFormatter.cpp} Namespace Reference

### Functions

- QString [datePatternForFormat](#) (DateFormat format)

## 6.13.1 Function Documentation

### 6.13.1.1 datePatternForFormat()

```
QString anonymous_namespace{DateFormatter.cpp}::datePatternForFormat (
    DateFormat format)
```

Definition at line 12 of file [DateFormatter.cpp](#).

## 6.14 anonymous\_namespace{FfmpegPath.cpp} Namespace Reference

### Functions

- QString [absoluteExecutablePath](#) (const QString &candidate)

## 6.14.1 Function Documentation

### 6.14.1.1 absoluteExecutablePath()

```
QString anonymous_namespace{FfmpegPath.cpp}::absoluteExecutablePath (
    const QString & candidate)
```

Definition at line 13 of file [FfmpegPath.cpp](#).

## 6.15 anonymous\_namespace{FitParser.cpp} Namespace Reference

### Classes

- class [FitSessionListener](#)
- class [FitFileIdListener](#)

### Functions

- QString [fitTimestampToIsoUtc](#) (FIT\_DATE\_TIME timestamp)



### 6.15.1 Function Documentation

#### 6.15.1.1 fitTimestampToIsoUtc()

```
QString anonymous_namespace{FitParser.cpp}::fitTimestampToIsoUtc (
    FIT_DATE_TIME timestamp)
```

Definition at line 21 of file [FitParser.cpp](#).

## 6.16 anonymous\_namespace{ImportWorker.cpp} Namespace Reference

### Functions

- void [applySqlitePragmas](#) (QSqlDatabase &db)

### 6.16.1 Function Documentation

#### 6.16.1.1 applySqlitePragmas()

```
void anonymous_namespace{ImportWorker.cpp}::applySqlitePragmas (
    QSqlDatabase & db)
```

Definition at line 24 of file [ImportWorker.cpp](#).

## 6.17 anonymous\_namespace{MainWindowChartActions.cpp} Namespace Reference

### Variables

- static constexpr int [kChartPresetCustom](#) = 0
- static constexpr int [kChartPresetPowerAnalysis](#) = 1
- static constexpr int [kChartPresetHeartRateFocus](#) = 2
- static constexpr int [kChartPresetRaceReview](#) = 3

### 6.17.1 Variable Documentation

#### 6.17.1.1 kChartPresetCustom

```
int anonymous_namespace{MainWindowChartActions.cpp}::kChartPresetCustom = 0 [static], [constexpr]
```

Definition at line 22 of file [MainWindowChartActions.cpp](#).

#### 6.17.1.2 kChartPresetHeartRateFocus

```
int anonymous_namespace{MainWindowChartActions.cpp}::kChartPresetHeartRateFocus = 2 [static],
[constexpr]
```

Definition at line 24 of file [MainWindowChartActions.cpp](#).

#### 6.17.1.3 kChartPresetPowerAnalysis

```
int anonymous_namespace{MainWindowChartActions.cpp}::kChartPresetPowerAnalysis = 1 [static],
[constexpr]
```

Definition at line 23 of file [MainWindowChartActions.cpp](#).

#### 6.17.1.4 kChartPresetRaceReview

```
int anonymous_namespace{MainWindowChartActions.cpp}::kChartPresetRaceReview = 3 [static],
[constexpr]
```

Definition at line 25 of file [MainWindowChartActions.cpp](#).

## 6.18 anonymous\_namespace{MainWindowClimbInteractions.cpp} Namespace Reference

### Classes

- class [ClimbNameTableItem](#)
- class [ClimbSortKeyTableItem](#)

### Functions

- static QString [fmtDur](#) (double seconds)

### Variables

- static constexpr int [kClimbStartRole](#) = Qt::UserRole + 3
- static constexpr int [kClimbEndRole](#) = Qt::UserRole + 4

### 6.18.1 Function Documentation

#### 6.18.1.1 [fmtDur\(\)](#)

```
QString anonymous_namespace{MainWindowClimbInteractions.cpp}::fmtDur (  
    double seconds) [static]
```

Definition at line 30 of file [MainWindowClimbInteractions.cpp](#).

### 6.18.2 Variable Documentation

#### 6.18.2.1 [kClimbEndRole](#)

```
int anonymous_namespace{MainWindowClimbInteractions.cpp}::kClimbEndRole = Qt::UserRole + 4  
[static], [constexpr]
```

Definition at line 44 of file [MainWindowClimbInteractions.cpp](#).

#### 6.18.2.2 [kClimbStartRole](#)

```
int anonymous_namespace{MainWindowClimbInteractions.cpp}::kClimbStartRole = Qt::UserRole + 3  
[static], [constexpr]
```

Definition at line 43 of file [MainWindowClimbInteractions.cpp](#).

## 6.19 anonymous\_namespace{MainWindowClimbViews.cpp} Namespace Reference

### Classes

- class [ClimbNameTableItem](#)
- class [ClimbSortKeyTableItem](#)

### Functions

- QString [fmtDur](#) (double seconds)

### Variables

- static constexpr int [kClimbStartRole](#) = Qt::UserRole + 3
- static constexpr int [kClimbEndRole](#) = Qt::UserRole + 4

## 6.19.1 Function Documentation

### 6.19.1.1 fmtDur()

```
QString anonymous_namespace{MainWindowClimbViews.cpp}::fmtDur (
    double seconds)
```

Definition at line 27 of file [MainWindowClimbViews.cpp](#).

## 6.19.2 Variable Documentation

### 6.19.2.1 kClimbEndRole

```
int anonymous_namespace{MainWindowClimbViews.cpp}::kClimbEndRole = Qt::UserRole + 4 [static],
[constexpr]
```

Definition at line 41 of file [MainWindowClimbViews.cpp](#).

### 6.19.2.2 kClimbStartRole

```
int anonymous_namespace{MainWindowClimbViews.cpp}::kClimbStartRole = Qt::UserRole + 3 [static],
[constexpr]
```

Definition at line 40 of file [MainWindowClimbViews.cpp](#).

## 6.20 anonymous\_namespace{MainWindowDatabase.cpp} Namespace Reference

### Functions

- int [mapLegacyTabsToPageIndex](#) (int legacyActiveTab, int legacyAnalysisTab)  
*Maps legacy activeTab/analysisTab settings to the new page index.*

### Variables

- static constexpr int [kMaxRecentDatabases](#) = 8  
*Maximum number of recent databases to remember.*
- static constexpr int [kChartPresetCustom](#) = 0  
*Default chart preset identifier value.*
- static constexpr int [kLegacyTabActivities](#) = 0
- static constexpr int [kLegacyTabAnalysis](#) = 1
- static constexpr int [kLegacyAnalysisCharts](#) = 0
- static constexpr int [kLegacyAnalysisZones](#) = 1
- static constexpr int [kLegacyAnalysisHistogram](#) = 2
- static constexpr int [kLegacyAnalysisPowerCurve](#) = 3
- static constexpr int [kLegacyAnalysisCalendar](#) = 4
- static constexpr int [kLegacyAnalysisFitness](#) = 5
- static constexpr int [kLegacyAnalysisClimbing](#) = 6
- static constexpr int [kPageActivities](#) = 0
- static constexpr int [kPageCharts](#) = 1
- static constexpr int [kPagePower](#) = 2
- static constexpr int [kPageIntervals](#) = 3
- static constexpr int [kPageClimbs](#) = 4
- static constexpr int [kPageFitness](#) = 5
- static constexpr int [kPageCalendar](#) = 6
- static constexpr int [kPageVideo](#) = 7

## 6.20.1 Function Documentation

### 6.20.1.1 mapLegacyTabsToPageIndex()

```
int anonymous_namespace{MainWindowDatabase.cpp}::mapLegacyTabsToPageIndex (
    int legacyActiveTab,
    int legacyAnalysisTab)
```

Maps legacy activeTab/analysisTab settings to the new page index.

Definition at line 53 of file [MainWindowDatabase.cpp](#).

## 6.20.2 Variable Documentation

### 6.20.2.1 kChartPresetCustom

```
int anonymous_namespace{MainWindowDatabase.cpp}::kChartPresetCustom = 0 [static], [constexpr]
```

Default chart preset identifier value.

Definition at line 27 of file [MainWindowDatabase.cpp](#).

### 6.20.2.2 kLegacyAnalysisCalendar

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisCalendar = 4 [static], [constexpr]
```

Definition at line 38 of file [MainWindowDatabase.cpp](#).

### 6.20.2.3 kLegacyAnalysisCharts

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisCharts = 0 [static], [constexpr]
```

Definition at line 34 of file [MainWindowDatabase.cpp](#).

### 6.20.2.4 kLegacyAnalysisClimbing

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisClimbing = 6 [static], [constexpr]
```

Definition at line 40 of file [MainWindowDatabase.cpp](#).

### 6.20.2.5 kLegacyAnalysisFitness

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisFitness = 5 [static], [constexpr]
```

Definition at line 39 of file [MainWindowDatabase.cpp](#).

### 6.20.2.6 kLegacyAnalysisHistogram

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisHistogram = 2 [static], [constexpr]
```

Definition at line 36 of file [MainWindowDatabase.cpp](#).

### 6.20.2.7 kLegacyAnalysisPowerCurve

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisPowerCurve = 3 [static],
[constexpr]
```

Definition at line 37 of file [MainWindowDatabase.cpp](#).

### 6.20.2.8 kLegacyAnalysisZones

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyAnalysisZones = 1 [static], [constexpr]
```

Definition at line 35 of file [MainWindowDatabase.cpp](#).

### 6.20.2.9 kLegacyTabActivities

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyTabActivities = 0 [static], [constexpr]
```

Definition at line 30 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.10 kLegacyTabAnalysis

```
int anonymous_namespace{MainWindowDatabase.cpp}::kLegacyTabAnalysis = 1 [static], [constexpr]
```

Definition at line 31 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.11 kMaxRecentDatabases

```
int anonymous_namespace{MainWindowDatabase.cpp}::kMaxRecentDatabases = 8 [static], [constexpr]
```

Maximum number of recent databases to remember.  
Definition at line 24 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.12 kPageActivities

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageActivities = 0 [static], [constexpr]
```

Definition at line 43 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.13 kPageCalendar

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageCalendar = 6 [static], [constexpr]
```

Definition at line 49 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.14 kPageCharts

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageCharts = 1 [static], [constexpr]
```

Definition at line 44 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.15 kPageClimbs

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageClimbs = 4 [static], [constexpr]
```

Definition at line 47 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.16 kPageFitness

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageFitness = 5 [static], [constexpr]
```

Definition at line 48 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.17 kPageIntervals

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageIntervals = 3 [static], [constexpr]
```

Definition at line 46 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.18 kPagePower

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPagePower = 2 [static], [constexpr]
```

Definition at line 45 of file [MainWindowDatabase.cpp](#).

#### 6.20.2.19 kPageVideo

```
int anonymous_namespace{MainWindowDatabase.cpp}::kPageVideo = 7 [static], [constexpr]
```

Definition at line 50 of file [MainWindowDatabase.cpp](#).

## 6.21 anonymous\_namespace{MainWindowIntervals.cpp} Namespace Reference

### Functions

- static QString [fmtDur](#) (double seconds)

**Variables**

- static constexpr int [kIntervalStartRole](#) = Qt::UserRole + 1
- static constexpr int [kIntervalEndRole](#) = Qt::UserRole + 2

**6.21.1 Function Documentation****6.21.1.1 fmtDur()**

```
QString anonymous_namespace{MainWindowIntervals.cpp}::fmtDur (
    double seconds) [static]
```

Definition at line 28 of file [MainWindowIntervals.cpp](#).

**6.21.2 Variable Documentation****6.21.2.1 kIntervalEndRole**

```
int anonymous_namespace{MainWindowIntervals.cpp}::kIntervalEndRole = Qt::UserRole + 2 [static],
[constexpr]
```

Definition at line 42 of file [MainWindowIntervals.cpp](#).

**6.21.2.2 kIntervalStartRole**

```
int anonymous_namespace{MainWindowIntervals.cpp}::kIntervalStartRole = Qt::UserRole + 1 [static],
[constexpr]
```

Definition at line 41 of file [MainWindowIntervals.cpp](#).

**6.22 anonymous\_namespace{MainWindowStatus.cpp} Namespace Reference****Functions**

- QString [formatActivityDateLabel](#) (const QString &isoDate)

**6.22.1 Function Documentation****6.22.1.1 formatActivityDateLabel()**

```
QString anonymous_namespace{MainWindowStatus.cpp}::formatActivityDateLabel (
    const QString & isoDate)
```

Definition at line 17 of file [MainWindowStatus.cpp](#).

**6.23 anonymous\_namespace{MainWindowUiBuild.cpp} Namespace Reference****Enumerations**

- enum **ChartPresetId** { **ChartPresetCustom** = 0 , **ChartPresetPowerAnalysis** , **ChartPresetHeartRate**↵  
**Focus** , **ChartPresetRaceReview** }

**Functions**

- QWidget \* [wrapWithNoActivityState](#) (QWidget \*content, QStackedLayout \*\*outStack, const QString &message=QStringLiteral("No **Activity** Selected"))

### Variables

- static constexpr int [kClimbStartRole](#) = Qt::UserRole + 3
- static constexpr int [kClimbEndRole](#) = Qt::UserRole + 4

## 6.23.1 Enumeration Type Documentation

### 6.23.1.1 ChartPresetId

enum anonymous\_namespace{MainWindowUiBuild.cpp}::ChartPresetId  
Definition at line 73 of file [MainWindowUiBuild.cpp](#).

## 6.23.2 Function Documentation

### 6.23.2.1 wrapWithNoActivityState()

```
QWidget * anonymous_namespace{MainWindowUiBuild.cpp}::wrapWithNoActivityState (
    QWidget * content,
    QStackedLayout ** outStack,
    const QString & message = QStringLiteral("No Activity Selected"))
```

Definition at line 84 of file [MainWindowUiBuild.cpp](#).

## 6.23.3 Variable Documentation

### 6.23.3.1 kClimbEndRole

```
int anonymous_namespace{MainWindowUiBuild.cpp}::kClimbEndRole = Qt::UserRole + 4 [static],
[constexpr]
```

Definition at line 82 of file [MainWindowUiBuild.cpp](#).

### 6.23.3.2 kClimbStartRole

```
int anonymous_namespace{MainWindowUiBuild.cpp}::kClimbStartRole = Qt::UserRole + 3 [static],
[constexpr]
```

Definition at line 81 of file [MainWindowUiBuild.cpp](#).

## 6.24 anonymous\_namespace{MainWindowVideo.cpp} Namespace Reference

### Functions

- QString [sanitizeVideoFilePart](#) (QString text)

## 6.24.1 Function Documentation

### 6.24.1.1 sanitizeVideoFilePart()

```
QString anonymous_namespace{MainWindowVideo.cpp}::sanitizeVideoFilePart (
    QString text)
```

Definition at line 24 of file [MainWindowVideo.cpp](#).

## 6.25 anonymous\_namespace{MapFitMath.cpp} Namespace Reference

### Functions

- double [normalizeLongitude](#) (double lon)
- double [latToTileY](#) (double lat)
- double [tileYToLat](#) (double tileY)

## Variables

- constexpr double `kPi` = 3.14159265358979323846
- constexpr int `kTilePx` = 256
- constexpr double `kMinLatSpan` = 0.001
- constexpr double `kMinLonSpan` = 0.001
- constexpr double `kMaxMercatorLat` = 85.051129

## 6.25.1 Function Documentation

### 6.25.1.1 latToTileY()

```
double anonymous_namespace{MapFitMath.cpp}::latToTileY (  
    double lat)
```

Definition at line 25 of file [MapFitMath.cpp](#).

### 6.25.1.2 normalizeLongitude()

```
double anonymous_namespace{MapFitMath.cpp}::normalizeLongitude (  
    double lon)
```

Definition at line 17 of file [MapFitMath.cpp](#).

### 6.25.1.3 tileYToLat()

```
double anonymous_namespace{MapFitMath.cpp}::tileYToLat (  
    double tileY)
```

Definition at line 32 of file [MapFitMath.cpp](#).

## 6.25.2 Variable Documentation

### 6.25.2.1 kMaxMercatorLat

```
double anonymous_namespace{MapFitMath.cpp}::kMaxMercatorLat = 85.051129 [constexpr]
```

Definition at line 15 of file [MapFitMath.cpp](#).

### 6.25.2.2 kMinLatSpan

```
double anonymous_namespace{MapFitMath.cpp}::kMinLatSpan = 0.001 [constexpr]
```

Definition at line 13 of file [MapFitMath.cpp](#).

### 6.25.2.3 kMinLonSpan

```
double anonymous_namespace{MapFitMath.cpp}::kMinLonSpan = 0.001 [constexpr]
```

Definition at line 14 of file [MapFitMath.cpp](#).

### 6.25.2.4 kPi

```
double anonymous_namespace{MapFitMath.cpp}::kPi = 3.14159265358979323846 [constexpr]
```

Definition at line 11 of file [MapFitMath.cpp](#).

### 6.25.2.5 kTilePx

```
int anonymous_namespace{MapFitMath.cpp}::kTilePx = 256 [constexpr]
```

Definition at line 12 of file [MapFitMath.cpp](#).



## 6.26 anonymous\_namespace{RideChartSelectionManager.cpp} Namespace Reference

### Variables

- constexpr double [kClimbHandleCenterOffsetYPx](#) = 10.0
- constexpr double [kClimbHandleRadiusPx](#) = 8.0
- constexpr double [kClimbHandleGrabRadiusPx](#) = 24.0

### 6.26.1 Variable Documentation

#### 6.26.1.1 kClimbHandleCenterOffsetYPx

```
double anonymous_namespace{RideChartSelectionManager.cpp}::kClimbHandleCenterOffsetYPx = 10.0
[constexpr]
```

Definition at line 11 of file [RideChartSelectionManager.cpp](#).

#### 6.26.1.2 kClimbHandleGrabRadiusPx

```
double anonymous_namespace{RideChartSelectionManager.cpp}::kClimbHandleGrabRadiusPx = 24.↵
0 [constexpr]
```

Definition at line 13 of file [RideChartSelectionManager.cpp](#).

#### 6.26.1.3 kClimbHandleRadiusPx

```
double anonymous_namespace{RideChartSelectionManager.cpp}::kClimbHandleRadiusPx = 8.0 [constexpr]
```

Definition at line 12 of file [RideChartSelectionManager.cpp](#).

## 6.27 anonymous\_namespace{RideChartWidget.cpp} Namespace Reference

## 6.28 anonymous\_namespace{TileCache.cpp} Namespace Reference

### Functions

- QString [defaultDiskCacheRoot](#) ()

### 6.28.1 Function Documentation

#### 6.28.1.1 defaultDiskCacheRoot()

```
QString anonymous_namespace{TileCache.cpp}::defaultDiskCacheRoot ()
```

Definition at line 23 of file [TileCache.cpp](#).

## 6.29 anonymous\_namespace{VideoExportDialog.cpp} Namespace Reference

### Functions

- QString [mapStyleName](#) (MapStyle style)
- QString [makeDefaultFilename](#) (const [VideoExportSettings](#) &defaults)
- QString [formatVideoLength](#) (double seconds)

## 6.29.1 Function Documentation

### 6.29.1.1 formatVideoLength()

```
QString anonymous_namespace{VideoExportDialog.cpp}::formatVideoLength (
    double seconds)
```

Definition at line 73 of file [VideoExportDialog.cpp](#).

### 6.29.1.2 makeDefaultFilename()

```
QString anonymous_namespace{VideoExportDialog.cpp}::makeDefaultFilename (
    const VideoExportSettings & defaults)
```

Definition at line 50 of file [VideoExportDialog.cpp](#).

### 6.29.1.3 mapStyleName()

```
QString anonymous_namespace{VideoExportDialog.cpp}::mapStyleName (
    MapStyle style)
```

Definition at line 35 of file [VideoExportDialog.cpp](#).

## 6.30 anonymous\_namespace{VideoRenderer.cpp} Namespace Reference

### Variables

- constexpr int [kTilePx](#) = 256
- constexpr double [kPi](#) = 3.14159265358979323846

## 6.30.1 Variable Documentation

### 6.30.1.1 kPi

```
double anonymous_namespace{VideoRenderer.cpp}::kPi = 3.14159265358979323846 [constexpr]
```

Definition at line 12 of file [VideoRenderer.cpp](#).

### 6.30.1.2 kTilePx

```
int anonymous_namespace{VideoRenderer.cpp}::kTilePx = 256 [constexpr]
```

Definition at line 11 of file [VideoRenderer.cpp](#).

## 6.31 anonymous\_namespace{VideoRenderJob.cpp} Namespace Reference

### Classes

- class [ExportTileCleanupGuard](#)

### Functions

- QColor [routeColor](#) (const [RideRecord](#) &rec, ColorMetric metric, const [ColorContext](#) &context, const QColor &singleColor)

### Variables

- static constexpr int [kTilePx](#) = 256
- static constexpr int [kVideoExportTileCache](#) = 96

### 6.31.1 Function Documentation

#### 6.31.1.1 routeColor()

```
QColor anonymous_namespace{VideoRenderJob.cpp}::routeColor (
    const RideRecord & rec,
    ColorMetric metric,
    const ColorContext & context,
    const QColor & singleColor)
```

Definition at line 37 of file [VideoRenderJob.cpp](#).

### 6.31.2 Variable Documentation

#### 6.31.2.1 kTilePx

```
int anonymous_namespace{VideoRenderJob.cpp}::kTilePx = 256 [static], [constexpr]
```

Definition at line 34 of file [VideoRenderJob.cpp](#).

#### 6.31.2.2 kVideoExportTileCache

```
int anonymous_namespace{VideoRenderJob.cpp}::kVideoExportTileCache = 96 [static], [constexpr]
```

Definition at line 35 of file [VideoRenderJob.cpp](#).

## 6.32 anonymous\_namespace{VideoTileProvider.cpp} Namespace Reference

### Functions

- QString [defaultVideoExportCacheRoot](#) ()
- QString [defaultPrimaryTileCacheRoot](#) ()

### 6.32.1 Function Documentation

#### 6.32.1.1 defaultPrimaryTileCacheRoot()

```
QString anonymous_namespace{VideoTileProvider.cpp}::defaultPrimaryTileCacheRoot ()
```

Definition at line 21 of file [VideoTileProvider.cpp](#).

#### 6.32.1.2 defaultVideoExportCacheRoot()

```
QString anonymous_namespace{VideoTileProvider.cpp}::defaultVideoExportCacheRoot ()
```

Definition at line 13 of file [VideoTileProvider.cpp](#).

## 6.33 anonymous\_namespace{WelcomeWidget.cpp} Namespace Reference

### Functions

- QFrame \* [makeCard](#) (QWidget \*parent)  
*Creates a styled card frame for dashboard sections.*
- QLabel \* [makeHeading](#) (const QString &text, QWidget \*parent)  
*Creates a section heading label.*
- QPushButton \* [makePrimaryBtn](#) (const QString &text, QWidget \*parent)  
*Creates a primary action button.*
- QPushButton \* [makeSecondaryBtn](#) (const QString &text, QWidget \*parent)  
*Creates a secondary action button.*

### 6.33.1 Function Documentation

#### 6.33.1.1 makeCard()

```
QFrame * anonymous_namespace{WelcomeWidget.cpp}::makeCard (
    QWidget * parent)
```

Creates a styled card frame for dashboard sections.

Definition at line 17 of file [WelcomeWidget.cpp](#).

#### 6.33.1.2 makeHeading()

```
QLabel * anonymous_namespace{WelcomeWidget.cpp}::makeHeading (
    const QString & text,
    QWidget * parent)
```

Creates a section heading label.

Definition at line 30 of file [WelcomeWidget.cpp](#).

#### 6.33.1.3 makePrimaryBtn()

```
QPushButton * anonymous_namespace{WelcomeWidget.cpp}::makePrimaryBtn (
    const QString & text,
    QWidget * parent)
```

Creates a primary action button.

Definition at line 39 of file [WelcomeWidget.cpp](#).

#### 6.33.1.4 makeSecondaryBtn()

```
QPushButton * anonymous_namespace{WelcomeWidget.cpp}::makeSecondaryBtn (
    const QString & text,
    QWidget * parent)
```

Creates a secondary action button.

Definition at line 54 of file [WelcomeWidget.cpp](#).

## 6.34 anonymous\_namespace{WorkoutController.cpp} Namespace Reference

### Functions

- QDate [activityDateForFtp](#) (const [Activity](#) &activity)
- int [resolveAthleteFtpForDate](#) ([AthleteRepository](#) &repo, int athleteId, const QDate &date)

### 6.34.1 Function Documentation

#### 6.34.1.1 activityDateForFtp()

```
QDate anonymous_namespace{WorkoutController.cpp}::activityDateForFtp (
    const Activity & activity)
```

Definition at line 32 of file [WorkoutController.cpp](#).

#### 6.34.1.2 resolveAthleteFtpForDate()

```
int anonymous_namespace{WorkoutController.cpp}::resolveAthleteFtpForDate (
    AthleteRepository & repo,
    int athleteId,
    const QDate & date)
```

Definition at line 40 of file [WorkoutController.cpp](#).

## 6.35 anonymous\_namespace{ZoneCalculator.cpp} Namespace Reference

### Functions

- double [valueForMetric](#) (const [RideRecord](#) &record, ColorMetric metric)
- bool [sampleAvailable](#) (const [RideRecord](#) &record, ColorMetric metric)
- double [sampleDurationSeconds](#) (const std::vector< [RideRecord](#) > &records, int index)

### 6.35.1 Function Documentation

#### 6.35.1.1 sampleAvailable()

```
bool anonymous_namespace{ZoneCalculator.cpp}::sampleAvailable (
    const RideRecord & record,
    ColorMetric metric)
```

Definition at line 27 of file [ZoneCalculator.cpp](#).

#### 6.35.1.2 sampleDurationSeconds()

```
double anonymous_namespace{ZoneCalculator.cpp}::sampleDurationSeconds (
    const std::vector< RideRecord > & records,
    int index)
```

Definition at line 43 of file [ZoneCalculator.cpp](#).

#### 6.35.1.3 valueForMetric()

```
double anonymous_namespace{ZoneCalculator.cpp}::valueForMetric (
    const RideRecord & record,
    ColorMetric metric)
```

Definition at line 11 of file [ZoneCalculator.cpp](#).

## 6.36 ClimbDefaults Namespace Reference

Default climb detection values (mirrors [ClimbDetector](#) defaults).

### Variables

- static constexpr double [kMinLength](#) = 0.15
- static constexpr double [kMinGain](#) = 10.0
- static constexpr double [kMinGradient](#) = 4.0
- static constexpr double [kStartGrad](#) = 1.5
- static constexpr double [kDipMeters](#) = 10.0
- static constexpr double [kDipDistance](#) = 200.0
- static constexpr double [kSmoothing](#) = 50.0

### 6.36.1 Detailed Description

Default climb detection values (mirrors [ClimbDetector](#) defaults).

### 6.36.2 Variable Documentation

#### 6.36.2.1 kDipDistance

```
double ClimbDefaults::kDipDistance = 200.0 [static], [constexpr]
```

Definition at line 40 of file [MainWindowAthlete.cpp](#).

### 6.36.2.2 kDipMeters

```
double ClimbDefaults::kDipMeters = 10.0 [static], [constexpr]
```

Definition at line 39 of file [MainWindowAthlete.cpp](#).

### 6.36.2.3 kMinGain

```
double ClimbDefaults::kMinGain = 10.0 [static], [constexpr]
```

Definition at line 36 of file [MainWindowAthlete.cpp](#).

### 6.36.2.4 kMinGradient

```
double ClimbDefaults::kMinGradient = 4.0 [static], [constexpr]
```

Definition at line 37 of file [MainWindowAthlete.cpp](#).

### 6.36.2.5 kMinLength

```
double ClimbDefaults::kMinLength = 0.15 [static], [constexpr]
```

Definition at line 35 of file [MainWindowAthlete.cpp](#).

### 6.36.2.6 kSmoothing

```
double ClimbDefaults::kSmoothing = 50.0 [static], [constexpr]
```

Definition at line 41 of file [MainWindowAthlete.cpp](#).

### 6.36.2.7 kStartGrad

```
double ClimbDefaults::kStartGrad = 1.5 [static], [constexpr]
```

Definition at line 38 of file [MainWindowAthlete.cpp](#).

## 6.37 Platform Namespace Reference

Platform-specific abstractions and utilities.

### Functions

- QString [applicationDataDirectory](#) ()
- QString [defaultDatabaseDirectory](#) ()
- QString [defaultDatabasePath](#) ()
- QString [defaultBackupPath](#) (const QString &databasePath)
- void [openInFileManager](#) (const QString &path)

### 6.37.1 Detailed Description

Platform-specific abstractions and utilities.

Provides OS-agnostic access to system paths and file operations, with implementation specific to macOS, Linux, or Windows.

### 6.37.2 Function Documentation

#### 6.37.2.1 applicationDataDirectory()

```
QString Platform::applicationDataDirectory ()
```

Definition at line 12 of file [Platform.cpp](#).

#### 6.37.2.2 defaultBackupPath()

```
QString Platform::defaultBackupPath (  
    const QString & databasePath)
```

Definition at line 36 of file [Platform.cpp](#).

### 6.37.2.3 defaultDatabaseDirectory()

QString Platform::defaultDatabaseDirectory ()  
 Definition at line 22 of file [Platform.cpp](#).

### 6.37.2.4 defaultDatabasePath()

QString Platform::defaultDatabasePath ()  
 Definition at line 31 of file [Platform.cpp](#).

## 6.38 SqlQueryHelper Namespace Reference

Common SQL patterns and query builders to reduce duplication across repositories.

### Functions

- void [bindBoolValue](#) (QSqlQuery &query, const QString &placeholder, bool value)  
*Binds a boolean value to a query placeholder (0 or 1).*
- QString [selectExistsNonDeleted](#) (const QString &tableName)  
*Generates EXISTS query for non-deleted records.*
- QString [selectExistsLocked](#) (const QString &tableName)  
*Generates EXISTS query for locked records.*
- QString [selectExistsEver](#) (const QString &tableName)  
*Generates EXISTS query regardless of deletion state.*

### Variables

- constexpr std::string\_view [kWhereNotDeleted](#) = "COALESCE(deleted,0)=0"  
*SQL predicate for non-deleted rows.*
- constexpr std::string\_view [kWhereDeleted](#) = "COALESCE(deleted,0)=1"  
*SQL predicate for deleted rows.*
- constexpr std::string\_view [kWhereLocked](#) = "locked=1"  
*SQL predicate for locked rows.*

### 6.38.1 Detailed Description

Common SQL patterns and query builders to reduce duplication across repositories.  
 Helper utilities for SQL query generation and binding.

### 6.38.2 Function Documentation

#### 6.38.2.1 bindBoolValue()

```
void SqlQueryHelper::bindBoolValue (
    QSqlQuery & query,
    const QString & placeholder,
    bool value) [inline]
```

Binds a boolean value to a query placeholder (0 or 1).

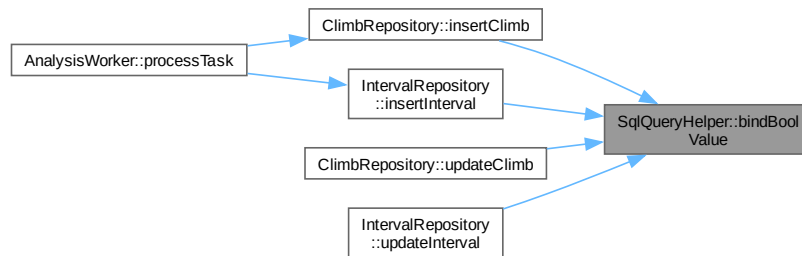
#### Parameters

|                    |                                     |
|--------------------|-------------------------------------|
| <i>query</i>       | SQL query to bind to.               |
| <i>placeholder</i> | Placeholder name (e.g., ":locked"). |
| <i>value</i>       | Boolean value to bind.              |

Definition at line 32 of file [SqlQueryHelper.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [IntervalRepository::insertInterval\(\)](#), [ClimbRepository::updateClimb\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

Here is the caller graph for this function:



### 6.38.2.2 selectExistsEver()

```
QString SqlQueryHelper::selectExistsEver (
    const QString & tableName) [inline]
```

Generates EXISTS query regardless of deletion state.

#### Parameters

|                  |                 |
|------------------|-----------------|
| <i>tableName</i> | Table to query. |
|------------------|-----------------|

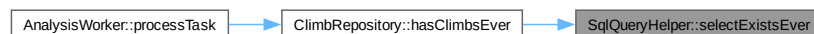
#### Returns

Query string (requires :id parameter).

Definition at line 65 of file [SqlQueryHelper.h](#).

Referenced by [ClimbRepository::hasClimbsEver\(\)](#).

Here is the caller graph for this function:



### 6.38.2.3 selectExistsLocked()

```
QString SqlQueryHelper::selectExistsLocked (
    const QString & tableName) [inline]
```

Generates EXISTS query for locked records.

#### Parameters

|                  |                 |
|------------------|-----------------|
| <i>tableName</i> | Table to query. |
|------------------|-----------------|



**Returns**

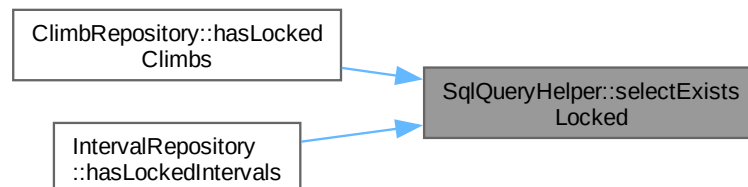
Query string (requires :id parameter).

Definition at line 53 of file [SqlQueryHelper.h](#).

References [kWhereLocked](#), and [kWhereNotDeleted](#).

Referenced by [ClimbRepository::hasLockedClimbs\(\)](#), and [IntervalRepository::hasLockedIntervals\(\)](#).

Here is the caller graph for this function:

**6.38.2.4 selectExistsNonDeleted()**

```
QString SqlQueryHelper::selectExistsNonDeleted (
    const QString & tableName) [inline]
```

Generates EXISTS query for non-deleted records.

**Parameters**

|                  |                 |
|------------------|-----------------|
| <i>tableName</i> | Table to query. |
|------------------|-----------------|

**Returns**

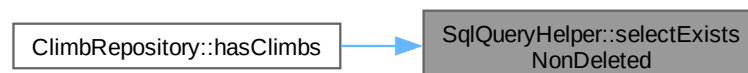
Query string (requires :id parameter).

Definition at line 42 of file [SqlQueryHelper.h](#).

References [kWhereNotDeleted](#).

Referenced by [ClimbRepository::hasClimbs\(\)](#).

Here is the caller graph for this function:

**6.38.3 Variable Documentation****6.38.3.1 kWhereDeleted**

```
std::string_view SqlQueryHelper::kWhereDeleted = "COALESCE(deleted,0)=1" [inline], [constexpr]
```

SQL predicate for deleted rows.

Definition at line 21 of file [SqlQueryHelper.h](#).

### 6.38.3.2 kWhereLocked

```
std::string_view SqlQueryHelper::kWhereLocked = "locked=1" [inline], [constexpr]
```

SQL predicate for locked rows.

Definition at line 24 of file [SqlQueryHelper.h](#).

Referenced by [selectExistsLocked\(\)](#).

### 6.38.3.3 kWhereNotDeleted

```
std::string_view SqlQueryHelper::kWhereNotDeleted = "COALESCE(deleted,0)=0" [inline], [constexpr]
```

SQL predicate for non-deleted rows.

Definition at line 18 of file [SqlQueryHelper.h](#).

Referenced by [selectExistsLocked\(\)](#), and [selectExistsNonDeleted\(\)](#).

## Chapter 7

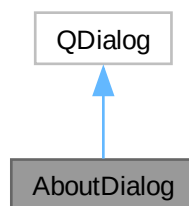
# Class Documentation

### 7.1 AboutDialog Class Reference

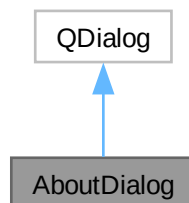
Displays application about/splash dialog.

```
#include <AboutDialog.h>
```

Inheritance diagram for AboutDialog:



Collaboration diagram for AboutDialog:



#### Public Member Functions

- [AboutDialog](#) (QWidget \*parent=nullptr)  
*Constructs about dialog.*

### 7.1.1 Detailed Description

Displays application about/splash dialog.

Definition at line 11 of file [AboutDialog.h](#).

### 7.1.2 Constructor & Destructor Documentation

#### 7.1.2.1 AboutDialog()

```
AboutDialog::AboutDialog (  
    QWidget * parent = nullptr) [explicit]
```

Constructs about dialog.

#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 13 of file [AboutDialog.cpp](#).

The documentation for this class was generated from the following files:

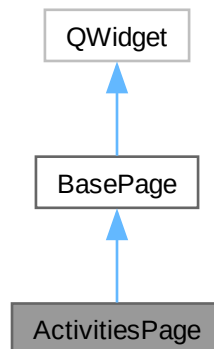
- [src/gui/AboutDialog.h](#)
- [src/gui/AboutDialog.cpp](#)

## 7.2 ActivitiesPage Class Reference

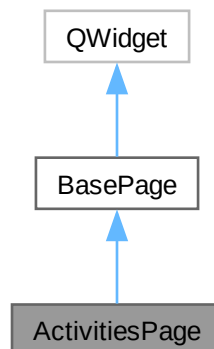
Top-level page for activity browsing, search, and management.

```
#include <ActivitiesPage.h>
```

Inheritance diagram for ActivitiesPage:



Collaboration diagram for ActivitiesPage:



### Signals

- void **importRequested** ()  
*Emitted when the user requests an import.*
- void **detectClimbsRequested** ()  
*Emitted when the user requests climb detection.*
- void **detectIntervalsRequested** ()  
*Emitted when the user requests interval detection.*
- void **recalculateMetricsRequested** ()  
*Emitted when the user requests metric recalculation.*
- void **editActivityRequested** ()  
*Emitted when the user requests the activity properties dialog.*

### Public Member Functions

- [ActivitiesPage](#) ([ActivityBrowser](#) \*browser, QWidget \*parent=nullptr)  
*Constructs the Activities page.*
- QToolBar \* [toolbar](#) () const  
*Returns the page toolbar (used by [MainWindow](#) to enable/disable actions).*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

### Private Attributes

- QToolBar \* [m\\_toolbar](#) = nullptr

## 7.2.1 Detailed Description

Top-level page for activity browsing, search, and management.  
Hosts the [ActivityBrowser](#) widget and a context toolbar with:

- Import: open file dialog to import FIT files
- Analyze: dropdown with Detect Climbs, Detect Intervals, and Recalculate Metrics
- Edit: open activity properties dialog
- Export: export selected activity (future)

The toolbar actions emit signals that [MainWindow](#) connects to.  
Definition at line 23 of file [ActivitiesPage.h](#).

## 7.2.2 Constructor & Destructor Documentation

### 7.2.2.1 ActivitiesPage()

```
ActivitiesPage::ActivitiesPage (
    ActivityBrowser * browser,
    QWidget * parent = nullptr) [explicit]
```

Constructs the Activities page.

#### Parameters

|                |                                                                                |
|----------------|--------------------------------------------------------------------------------|
| <i>browser</i> | The <a href="#">ActivityBrowser</a> widget (ownership transfers to this page). |
| <i>parent</i>  | Parent widget.                                                                 |

Wraps the pre-built [ActivityBrowser](#) in a page that also provides a context toolbar with Import, Analyze (dropdown), and Edit actions.

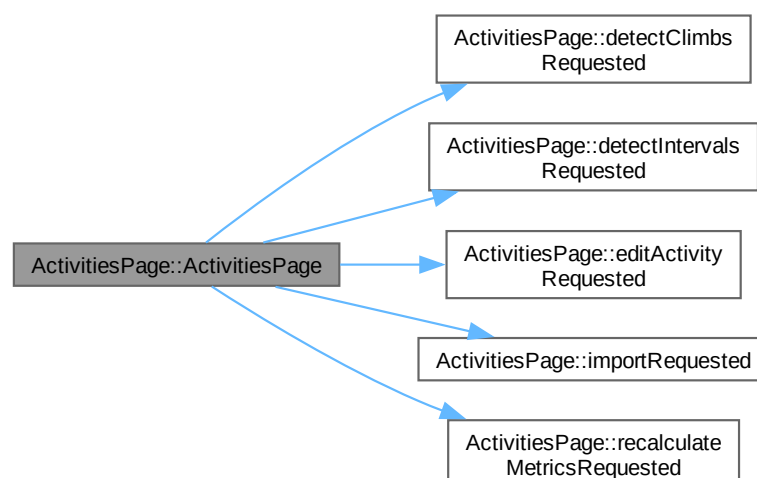
#### Parameters

|                |                                                                    |
|----------------|--------------------------------------------------------------------|
| <i>browser</i> | <a href="#">ActivityBrowser</a> widget (ownership transfers here). |
| <i>parent</i>  | Parent widget.                                                     |

Definition at line 23 of file [ActivitiesPage.cpp](#).

References [detectClimbsRequested\(\)](#), [detectIntervalsRequested\(\)](#), [editActivityRequested\(\)](#), [importRequested\(\)](#), and [recalculateMetricsRequested\(\)](#).

Here is the call graph for this function:



## 7.2.3 Member Function Documentation

### 7.2.3.1 toolbar()

`QToolBar * ActivitiesPage::toolbar () const [inline]`

Returns the page toolbar (used by [MainWindow](#) to enable/disable actions).

Definition at line 36 of file [ActivitiesPage.h](#).

## 7.2.4 Member Data Documentation

### 7.2.4.1 m\_toolbar

`QToolBar* ActivitiesPage::m_toolbar = nullptr [private]`

Definition at line 55 of file [ActivitiesPage.h](#).

The documentation for this class was generated from the following files:

- [src/gui/pages/ActivitiesPage.h](#)
- [src/gui/pages/ActivitiesPage.cpp](#)

## 7.3 Activity Struct Reference

Represents a single recorded activity or workout.

`#include <ActivityRepository.h>`

### Public Member Functions

- `bool isValid () const`  
*Checks if activity data is valid.*

### Public Attributes

- `int id = -1`  
*Unique activity identifier.*
- `int athleteId = -1`  
*Associated athlete ID.*
- `QString fitHash`  
*SHA-256 hash of original FIT file for duplicate detection.*
- `QString fileName`  
*Original filename.*
- `QString sport`  
*Sport type (cycling, running, etc.).*
- `QString startTime`  
*ISO8601 activity start time.*
- `QString endTime`  
*ISO8601 activity end time.*
- `double durationSec = 0.0`  
*Total duration in seconds.*
- `double distanceM = 0.0`  
*Total distance in meters.*
- `double avgPower = 0.0`  
*Average power in watts.*
- `double maxPower = 0.0`  
*Maximum power in watts.*
- `double normalizedPower = 0.0`  
*Normalized power in watts.*

- double `avgHr` = 0.0  
*Average heart rate in bpm.*
- double `maxHr` = 0.0  
*Maximum heart rate in bpm.*
- double `avgCadence` = 0.0  
*Average cadence in rpm.*
- double `avgSpeed` = 0.0  
*Average speed in km/h.*
- double `elevationGain` = 0.0  
*Total elevation gain in meters.*
- QString `notes`  
*User activity notes.*
- QString `weatherNotes`  
*Weather/environmental notes.*
- double `temperature` = 0.0  
*Temperature in Celsius.*
- QString `weather`  
*Weather description.*
- QString `wind`  
*Wind conditions.*
- int `rpe` = 0  
*Rate of perceived exertion (1-10).*
- int `fatigue` = 0  
*Fatigue level (1-10).*
- double `sleepHours` = 0.0  
*Sleep before activity in hours.*
- double `weightKg` = 0.0  
*Athlete weight in kg.*
- QString `bike`  
*Bike used.*
- QString `equipment`  
*Equipment used.*
- QString `importedAt`  
*ISO8601 import timestamp.*
- int `analysisVersion` = 1  
*Analysis algorithm version used.*
- QString `fingerprint`  
*Content hash for detecting meaningful changes.*
- int `analysisFlags` = 0  
*Bitmask of analysis flags (climbs, intervals, etc.).*
- QString `importSource`  
*Source of import (file, web, api, etc.).*

### 7.3.1 Detailed Description

Represents a single recorded activity or workout.

Contains all activity metadata, measurements, and athlete information.

Definition at line 16 of file [ActivityRepository.h](#).



## 7.3.2 Member Function Documentation

### 7.3.2.1 isValid()

```
bool Activity::isValid () const [inline]
```

Checks if activity data is valid.

#### Returns

True if activity has a valid positive ID.

Definition at line 121 of file [ActivityRepository.h](#).

## 7.3.3 Member Data Documentation

### 7.3.3.1 analysisFlags

```
int Activity::analysisFlags = 0
```

Bitmask of analysis flags (climbs, intervals, etc.).

Definition at line 112 of file [ActivityRepository.h](#).

### 7.3.3.2 analysisVersion

```
int Activity::analysisVersion = 1
```

Analysis algorithm version used.

Definition at line 106 of file [ActivityRepository.h](#).

### 7.3.3.3 athleteId

```
int Activity::athleteId = -1
```

Associated athlete ID.

Definition at line 22 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.4 avgCadence

```
double Activity::avgCadence = 0.0
```

Average cadence in rpm.

Definition at line 61 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.5 avgHr

```
double Activity::avgHr = 0.0
```

Average heart rate in bpm.

Definition at line 55 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.6 avgPower

```
double Activity::avgPower = 0.0
```

Average power in watts.

Definition at line 46 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.7 avgSpeed

```
double Activity::avgSpeed = 0.0
```

Average speed in km/h.

Definition at line 64 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.8 bike

```
QString Activity::bike
```

Bike used.

Definition at line 97 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

### 7.3.3.9 distanceM

```
double Activity::distanceM = 0.0
```

Total distance in meters.

Definition at line 43 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.10 durationSec

```
double Activity::durationSec = 0.0
```

Total duration in seconds.

Definition at line 40 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.11 elevationGain

```
double Activity::elevationGain = 0.0
```

Total elevation gain in meters.

Definition at line 67 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.12 endTime

```
QString Activity::endTime
```

ISO8601 activity end time.

Definition at line 37 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.13 equipment

```
QString Activity::equipment
```

Equipment used.

Definition at line 100 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

### 7.3.3.14 fatigue

```
int Activity::fatigue = 0
```

Fatigue level (1-10).

Definition at line 88 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

### 7.3.3.15 fileName

```
QString Activity::fileName
```

Original filename.

Definition at line 28 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.3.3.16 fingerprint

```
QString Activity::fingerprint
```

Content hash for detecting meaningful changes.

Definition at line 109 of file [ActivityRepository.h](#).

#### 7.3.3.17 fitHash

```
QString Activity::fitHash
```

SHA-256 hash of original FIT file for duplicate detection.

Definition at line 25 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.18 id

```
int Activity::id = -1
```

Unique activity identifier.

Definition at line 19 of file [ActivityRepository.h](#).

#### 7.3.3.19 importedAt

```
QString Activity::importedAt
```

ISO8601 import timestamp.

Definition at line 103 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.20 importSource

```
QString Activity::importSource
```

Source of import (file, web, api, etc.).

Definition at line 115 of file [ActivityRepository.h](#).

#### 7.3.3.21 maxHr

```
double Activity::maxHr = 0.0
```

Maximum heart rate in bpm.

Definition at line 58 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.22 maxPower

```
double Activity::maxPower = 0.0
```

Maximum power in watts.

Definition at line 49 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.23 normalizedPower

```
double Activity::normalizedPower = 0.0
```

Normalized power in watts.

Definition at line 52 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.24 notes

```
QString Activity::notes
```

User activity notes.

Definition at line 70 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.25 rpe

```
int Activity::rpe = 0
```

Rate of perceived exertion (1-10).

Definition at line 85 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.26 sleepHours

```
double Activity::sleepHours = 0.0
```

Sleep before activity in hours.

Definition at line 91 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.27 sport

```
QString Activity::sport
```

Sport type (cycling, running, etc.).

Definition at line 31 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.28 startTime

```
QString Activity::startTime
```

ISO8601 activity start time.

Definition at line 34 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.3.3.29 temperature

```
double Activity::temperature = 0.0
```

Temperature in Celsius.

Definition at line 76 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.30 weather

```
QString Activity::weather
```

Weather description.

Definition at line 79 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.31 weatherNotes

```
QString Activity::weatherNotes
```

Weather/environmental notes.

Definition at line 73 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.32 weightKg

```
double Activity::weightKg = 0.0
```

[Athlete](#) weight in kg.

Definition at line 94 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

#### 7.3.3.33 wind

```
QString Activity::wind
```

Wind conditions.

Definition at line 82 of file [ActivityRepository.h](#).

Referenced by [ActivityRepository::insertActivity\(\)](#), and [ActivityRepository::updateActivityProperties\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ActivityRepository.h](#)

## 7.4 ActivityAnalysisCacheRepository Class Reference

Caches expensive analysis results per activity.

```
#include <ActivityAnalysisCacheRepository.h>
```

### Public Member Functions

- [ActivityAnalysisCacheRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- bool [setCache](#) (int activityId, const QString &key, const QByteArray &value)  
*Stores analysis result cache.*
- QByteArray [getCache](#) (int activityId, const QString &key)  
*Retrieves cached analysis result.*
- bool [hasCache](#) (int activityId, const QString &key)  
*Checks if cache entry exists.*
- bool [invalidateCache](#) (int activityId)  
*Invalidates all cache for an activity.*
- bool [invalidateKey](#) (int activityId, const QString &key)  
*Invalidates specific cache key for an activity.*

### Private Attributes

- QSqlDatabase & [m\\_db](#)

### 7.4.1 Detailed Description

Caches expensive analysis results per activity.

Stores pre-computed power curves, histograms, best efforts, and other analysis data to improve responsiveness on repeated access.

Definition at line 17 of file [ActivityAnalysisCacheRepository.h](#).

### 7.4.2 Constructor & Destructor Documentation

#### 7.4.2.1 ActivityAnalysisCacheRepository()

```
ActivityAnalysisCacheRepository::ActivityAnalysisCacheRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

#### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 9 of file [ActivityAnalysisCacheRepository.cpp](#).

### 7.4.3 Member Function Documentation

#### 7.4.3.1 getCache()

```
QByteArray ActivityAnalysisCacheRepository::getCache (  
    int activityId,  
    const QString & key)
```

Retrieves cached analysis result.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
| <i>key</i>        | Cache key name.              |

#### Returns

Cached data, or empty if not found.

Definition at line 25 of file [ActivityAnalysisCacheRepository.cpp](#).

#### 7.4.3.2 hasCache()

```
bool ActivityAnalysisCacheRepository::hasCache (  
    int activityId,  
    const QString & key)
```

Checks if cache entry exists.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
| <i>key</i>        | Cache key name.              |

#### Returns

True if entry exists.

Definition at line 36 of file [ActivityAnalysisCacheRepository.cpp](#).

#### 7.4.3.3 invalidateCache()

```
bool ActivityAnalysisCacheRepository::invalidateCache (  
    int activityId)
```

Invalidates all cache for an activity.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

#### Returns

True on success.

Definition at line 45 of file [ActivityAnalysisCacheRepository.cpp](#).

#### 7.4.3.4 invalidateKey()

```
bool ActivityAnalysisCacheRepository::invalidateKey (  
    int activityId,  
    const QString & key)
```

Invalidates specific cache key for an activity.

**Parameters**

|                   |                 |
|-------------------|-----------------|
| <i>activityId</i> | Activity ID.    |
| <i>key</i>        | Cache key name. |

**Returns**

True on success.

Definition at line 53 of file [ActivityAnalysisCacheRepository.cpp](#).

**7.4.3.5 setCache()**

```
bool ActivityAnalysisCacheRepository::setCache (
    int activityId,
    const QString & key,
    const QByteArray & value)
```

Stores analysis result cache.

**Parameters**

|                   |                        |
|-------------------|------------------------|
| <i>activityId</i> | Activity ID.           |
| <i>key</i>        | Cache key name.        |
| <i>value</i>      | Serialized cache data. |

**Returns**

True on success.

Definition at line 13 of file [ActivityAnalysisCacheRepository.cpp](#).

**7.4.4 Member Data Documentation****7.4.4.1 m\_db**

```
QSqlDatabase& ActivityAnalysisCacheRepository::m_db [private]
```

Definition at line 67 of file [ActivityAnalysisCacheRepository.h](#).

The documentation for this class was generated from the following files:

- [src/database/ActivityAnalysisCacheRepository.h](#)
- [src/database/ActivityAnalysisCacheRepository.cpp](#)

**7.5 ActivityAnalysisSettings Struct Reference**

Analysis settings per activity.

```
#include <ActivityAnalysisSettingsRepository.h>
```

**Public Member Functions**

- bool [isValid](#) () const  
*Checks if settings are valid.*

## Public Attributes

- int `activityId` = -1  
*Associated activity ID.*
- double `climbMinGain` = 30.0  
*Minimum climb elevation gain in meters.*
- double `climbMinLength` = 0.5  
*Minimum climb length in kilometers.*
- double `climbMinGradient` = 3.0  
*Minimum climb gradient percentage.*
- double `intervalWorkThreshold` = 0.0  
*Work interval power threshold (fraction of FTP).*
- double `intervalRestThreshold` = 0.0  
*Recovery interval power threshold (fraction of FTP).*

### 7.5.1 Detailed Description

Analysis settings per activity.

Definition at line 14 of file [ActivityAnalysisSettingsRepository.h](#).

### 7.5.2 Member Function Documentation

#### 7.5.2.1 isValid()

```
bool ActivityAnalysisSettings::isValid () const [inline]
```

Checks if settings are valid.

Definition at line 37 of file [ActivityAnalysisSettingsRepository.h](#).

References [activityId](#).

### 7.5.3 Member Data Documentation

#### 7.5.3.1 activityId

```
int ActivityAnalysisSettings::activityId = -1
```

Associated activity ID.

Definition at line 17 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), [isValid\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

#### 7.5.3.2 climbMinGain

```
double ActivityAnalysisSettings::climbMinGain = 30.0
```

Minimum climb elevation gain in meters.

Definition at line 20 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

#### 7.5.3.3 climbMinGradient

```
double ActivityAnalysisSettings::climbMinGradient = 3.0
```

Minimum climb gradient percentage.

Definition at line 26 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

#### 7.5.3.4 climbMinLength

```
double ActivityAnalysisSettings::climbMinLength = 0.5
```

Minimum climb length in kilometers.

Definition at line 23 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).



### 7.5.3.5 intervalRestThreshold

double ActivityAnalysisSettings::intervalRestThreshold = 0.0

Recovery interval power threshold (fraction of FTP).

Definition at line 32 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

### 7.5.3.6 intervalWorkThreshold

double ActivityAnalysisSettings::intervalWorkThreshold = 0.0

Work interval power threshold (fraction of FTP).

Definition at line 29 of file [ActivityAnalysisSettingsRepository.h](#).

Referenced by [ActivityAnalysisSettingsRepository::insertOrUpdate\(\)](#), and [ActivityAnalysisSettingsRepository::settingsForActivity\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ActivityAnalysisSettingsRepository.h](#)

## 7.6 ActivityAnalysisSettingsRepository Class Reference

Repository for per-activity analysis settings.

```
#include <ActivityAnalysisSettingsRepository.h>
```

### Public Member Functions

- [ActivityAnalysisSettingsRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- bool [insertOrUpdate](#) (const [ActivityAnalysisSettings](#) &settings)  
*Saves or updates activity settings.*
- [ActivityAnalysisSettings settingsForActivity](#) (int activityId)  
*Retrieves settings for an activity.*

### Private Attributes

- QSqlDatabase & [m\\_db](#)

### 7.6.1 Detailed Description

Repository for per-activity analysis settings.

Stores custom climb detection and interval detection parameters per activity.

Definition at line 45 of file [ActivityAnalysisSettingsRepository.h](#).

### 7.6.2 Constructor & Destructor Documentation

#### 7.6.2.1 ActivityAnalysisSettingsRepository()

```
ActivityAnalysisSettingsRepository::ActivityAnalysisSettingsRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 9 of file [ActivityAnalysisSettingsRepository.cpp](#).

## 7.6.3 Member Function Documentation

### 7.6.3.1 insertOrUpdate()

```
bool ActivityAnalysisSettingsRepository::insertOrUpdate (
    const ActivityAnalysisSettings & settings)
```

Saves or updates activity settings.

#### Parameters

|                 |                   |
|-----------------|-------------------|
| <i>settings</i> | Settings to save. |
|-----------------|-------------------|

#### Returns

True on success.

Definition at line 13 of file [ActivityAnalysisSettingsRepository.cpp](#).

References [ActivityAnalysisSettings::activityId](#), [ActivityAnalysisSettings::climbMinGain](#), [ActivityAnalysisSettings::climbMinGradient](#), [ActivityAnalysisSettings::climbMinLength](#), [ActivityAnalysisSettings::intervalRestThreshold](#), and [ActivityAnalysisSettings::intervalWork](#).

### 7.6.3.2 settingsForActivity()

```
ActivityAnalysisSettings ActivityAnalysisSettingsRepository::settingsForActivity (
    int activityId)
```

Retrieves settings for an activity.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

#### Returns

Settings, or invalid if not found.

Definition at line 34 of file [ActivityAnalysisSettingsRepository.cpp](#).

References [ActivityAnalysisSettings::activityId](#), [ActivityAnalysisSettings::climbMinGain](#), [ActivityAnalysisSettings::climbMinGradient](#), [ActivityAnalysisSettings::climbMinLength](#), [ActivityAnalysisSettings::intervalRestThreshold](#), and [ActivityAnalysisSettings::intervalWork](#).

## 7.6.4 Member Data Documentation

### 7.6.4.1 m\_db

```
QSqlDatabase& ActivityAnalysisSettingsRepository::m_db [private]
```

Definition at line 69 of file [ActivityAnalysisSettingsRepository.h](#).

The documentation for this class was generated from the following files:

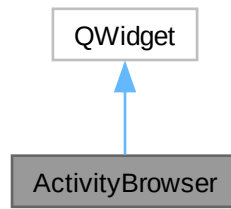
- src/database/ActivityAnalysisSettingsRepository.h
- src/database/ActivityAnalysisSettingsRepository.cpp

## 7.7 ActivityBrowser Class Reference

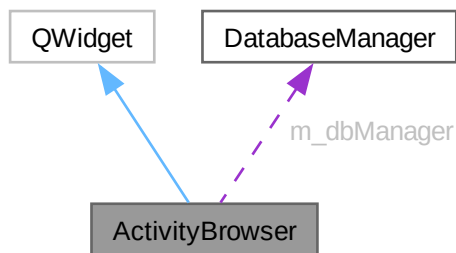
Browse and manage activities.

```
#include <ActivityBrowser.h>
```

Inheritance diagram for ActivityBrowser:



Collaboration diagram for ActivityBrowser:



### Signals

- void **activitySelected** (int activityId)
- void **activityDeleted** (int activityId)
- void **fitFilesDropped** (const QStringList &filePaths)

### Public Member Functions

- [ActivityBrowser](#) ([DatabaseManager](#) \*dbManager, QWidget \*parent=nullptr)
- void [refresh](#) (int athleteId=-1)
- void [focusSearch](#) ()

### Protected Member Functions

- void [dragEnterEvent](#) (QDragEnterEvent \*event) override
- void [dropEvent](#) (QDropEvent \*event) override

### Private Member Functions

- void [applyFilters](#) ()
- void [showContextMenu](#) (const QPoint &pos)
- void [editNotes](#) (int activityId)
- void [deleteActivity](#) (int activityId)

## Private Attributes

- `DatabaseManager * m_dbManager = nullptr`
- `QLabel * m_emptyStateLabel = nullptr`
- `QLineEdit * m_searchEdit = nullptr`
- `QComboBox * m_rangeFilter = nullptr`
- `QTableWidget * m_table = nullptr`
- `int m_athleteId = -1`

### 7.7.1 Detailed Description

Browse and manage activities.

Displays activities in a filterable table and provides context-menu actions.

The widget accepts local `.fit` drops and emits `fitFilesDropped()` so the main window can enqueue background imports.

Definition at line 26 of file [ActivityBrowser.h](#).

### 7.7.2 Constructor & Destructor Documentation

#### 7.7.2.1 ActivityBrowser()

```
ActivityBrowser::ActivityBrowser (
    DatabaseManager * dbManager,
    QWidget * parent = nullptr) [explicit]
```

Definition at line 65 of file [ActivityBrowser.cpp](#).

### 7.7.3 Member Function Documentation

#### 7.7.3.1 applyFilters()

```
void ActivityBrowser::applyFilters () [private]
```

Definition at line 229 of file [ActivityBrowser.cpp](#).

#### 7.7.3.2 deleteActivity()

```
void ActivityBrowser::deleteActivity (
    int activityId) [private]
```

Definition at line 354 of file [ActivityBrowser.cpp](#).

#### 7.7.3.3 dragEnterEvent()

```
void ActivityBrowser::dragEnterEvent (
    QDragEnterEvent * event) [override], [protected]
```

Definition at line 139 of file [ActivityBrowser.cpp](#).

#### 7.7.3.4 dropEvent()

```
void ActivityBrowser::dropEvent (
    QDropEvent * event) [override], [protected]
```

Definition at line 156 of file [ActivityBrowser.cpp](#).

#### 7.7.3.5 editNotes()

```
void ActivityBrowser::editNotes (
    int activityId) [private]
```

Definition at line 326 of file [ActivityBrowser.cpp](#).

#### 7.7.3.6 focusSearch()

```
void ActivityBrowser::focusSearch ()
```

Definition at line 130 of file [ActivityBrowser.cpp](#).

#### 7.7.3.7 refresh()

```
void ActivityBrowser::refresh (
    int athleteId = -1)
```

Definition at line 179 of file [ActivityBrowser.cpp](#).

#### 7.7.3.8 showContextMenu()

```
void ActivityBrowser::showContextMenu (
    const QPoint & pos) [private]
```

Definition at line 302 of file [ActivityBrowser.cpp](#).

### 7.7.4 Member Data Documentation

#### 7.7.4.1 m\_athleteId

```
int ActivityBrowser::m_athleteId = -1 [private]
```

Definition at line 62 of file [ActivityBrowser.h](#).

#### 7.7.4.2 m\_dbManager

```
DatabaseManager* ActivityBrowser::m_dbManager = nullptr [private]
```

Definition at line 57 of file [ActivityBrowser.h](#).

#### 7.7.4.3 m\_emptyStateLabel

```
QLabel* ActivityBrowser::m_emptyStateLabel = nullptr [private]
```

Definition at line 58 of file [ActivityBrowser.h](#).

#### 7.7.4.4 m\_rangeFilter

```
QComboBox* ActivityBrowser::m_rangeFilter = nullptr [private]
```

Definition at line 60 of file [ActivityBrowser.h](#).

#### 7.7.4.5 m\_searchEdit

```
QLineEdit* ActivityBrowser::m_searchEdit = nullptr [private]
```

Definition at line 59 of file [ActivityBrowser.h](#).

#### 7.7.4.6 m\_table

```
QTableWidget* ActivityBrowser::m_table = nullptr [private]
```

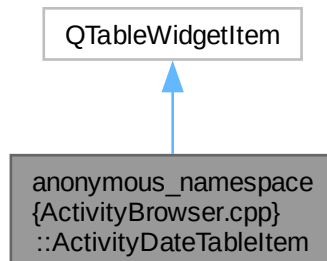
Definition at line 61 of file [ActivityBrowser.h](#).

The documentation for this class was generated from the following files:

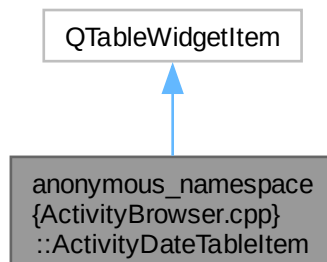
- [src/gui/ActivityBrowser.h](#)
- [src/gui/ActivityBrowser.cpp](#)

## 7.8 anonymous\_namespace{ActivityBrowser.cpp}::ActivityDataTableItem Class Reference

Inheritance diagram for anonymous\_namespace{ActivityBrowser.cpp}::ActivityDataTableItem:



Collaboration diagram for anonymous\_namespace{ActivityBrowser.cpp}::ActivityDataTableItem:



### Public Member Functions

- bool `operator<` (const QTableWidgetItem &other) const override

### 7.8.1 Detailed Description

Definition at line 29 of file [ActivityBrowser.cpp](#).

### 7.8.2 Member Function Documentation

#### 7.8.2.1 operator<()

```
bool anonymous_namespace{ActivityBrowser.cpp}::ActivityDataTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 34 of file [ActivityBrowser.cpp](#).

The documentation for this class was generated from the following file:

- `src/gui/ActivityBrowser.cpp`

## 7.9 ActivityRepository Class Reference

Repository for activity record persistence.

```
#include <ActivityRepository.h>
```

### Public Member Functions

- [ActivityRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- int [insertActivity](#) (const [Activity](#) &a)  
*Inserts a new activity into the database.*
- [Activity](#) [getActivity](#) (int activityId)  
*Retrieves an activity by ID.*
- QList< [Activity](#) > [listActivities](#) (int athleteId)  
*Lists all activities for an athlete.*
- bool [deleteActivity](#) (int activityId)  
*Deletes an activity.*
- bool [hasFitHash](#) (const QString &hash)  
*Checks if activity exists by FIT file hash.*
- bool [updateNotes](#) (int activityId, const QString &notes, const QString &weatherNotes)  
*Updates activity notes and weather information.*
- bool [updateActivityProperties](#) (int activityId, const [Activity](#) &a)  
*Updates comprehensive activity properties.*
- bool [updateAnalysisFlags](#) (int activityId, int flags)  
*Updates analysis flags for an activity.*
- bool [updateFingerprint](#) (int activityId, const QString &fingerprint)  
*Updates activity fingerprint for duplicate detection.*
- bool [hasFingerprintMatch](#) (const QString &fingerprint)  
*Checks if fingerprint already exists.*
- bool [setImportSource](#) (int activityId, const QString &source)  
*Sets the import source label for an activity.*

### Private Attributes

- QSqlDatabase & [m\\_db](#)

### 7.9.1 Detailed Description

Repository for activity record persistence.

Provides CRUD operations and queries for activities in the database.

Definition at line 129 of file [ActivityRepository.h](#).

### 7.9.2 Constructor & Destructor Documentation

#### 7.9.2.1 ActivityRepository()

```
ActivityRepository::ActivityRepository (  
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

#### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 13 of file [ActivityRepository.cpp](#).

### 7.9.3 Member Function Documentation

#### 7.9.3.1 deleteActivity()

```
bool ActivityRepository::deleteActivity (
    int activityId)
```

Deletes an activity.

##### Parameters

|                   |                                        |
|-------------------|----------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID to delete. |
|-------------------|----------------------------------------|

##### Returns

True if deletion succeeded.

Definition at line 154 of file [ActivityRepository.cpp](#).

#### 7.9.3.2 getActivity()

```
Activity ActivityRepository::getActivity (
    int activityId)
```

Retrieves an activity by ID.

##### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

##### Returns

[Activity](#) record, or invalid [Activity](#) if not found.

Definition at line 123 of file [ActivityRepository.cpp](#).

#### 7.9.3.3 hasFingerprintMatch()

```
bool ActivityRepository::hasFingerprintMatch (
    const QString & fingerprint)
```

Checks if fingerprint already exists.

##### Parameters

|                    |                       |
|--------------------|-----------------------|
| <i>fingerprint</i> | Fingerprint to check. |
|--------------------|-----------------------|

##### Returns

True if match found.

Definition at line 189 of file [ActivityRepository.cpp](#).

#### 7.9.3.4 hasFitHash()

```
bool ActivityRepository::hasFitHash (
    const QString & hash)
```

Checks if activity exists by FIT file hash.



**Parameters**

|             |                           |
|-------------|---------------------------|
| <i>hash</i> | SHA-256 hash of FIT file. |
|-------------|---------------------------|

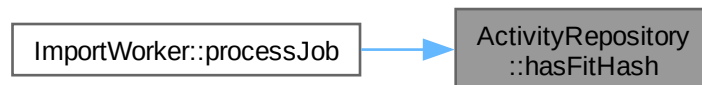
**Returns**

True if hash already exists in database.

Definition at line 162 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.9.3.5 insertActivity()**

```
int ActivityRepository::insertActivity (
    const Activity & a)
```

Inserts a new activity into the database.

**Parameters**

|          |                                     |
|----------|-------------------------------------|
| <i>a</i> | <a href="#">Activity</a> to insert. |
|----------|-------------------------------------|

**Returns**

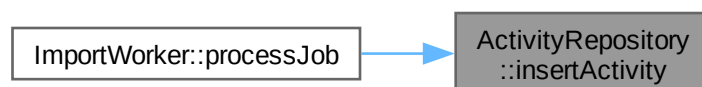
ID of inserted activity, or -1 on error.

Definition at line 70 of file [ActivityRepository.cpp](#).

References [Activity::athleteId](#), [Activity::avgCadence](#), [Activity::avgHr](#), [Activity::avgPower](#), [Activity::avgSpeed](#), [Activity::bike](#), [Activity::distanceM](#), [Activity::durationSec](#), [Activity::elevationGain](#), [Activity::endTime](#), [Activity::equipment](#), [Activity::fatigue](#), [Activity::fileName](#), [Activity::fitHash](#), [Activity::importedAt](#), [Activity::maxHr](#), [Activity::maxPower](#), [Activity::normalizedPower](#), [Activity::notes](#), [Activity::rpe](#), [Activity::sleepHours](#), [Activity::sport](#), [Activity::startTime](#), [Activity::temperature](#), [Activity::weather](#), [Activity::weatherNotes](#), [Activity::weightKg](#), and [Activity::wind](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



### 7.9.3.6 listActivities()

```
QList< Activity > ActivityRepository::listActivities (
    int athleteId)
```

Lists all activities for an athlete.

#### Parameters

|                  |                              |
|------------------|------------------------------|
| <i>athleteId</i> | <a href="#">Activity</a> ID. |
|------------------|------------------------------|

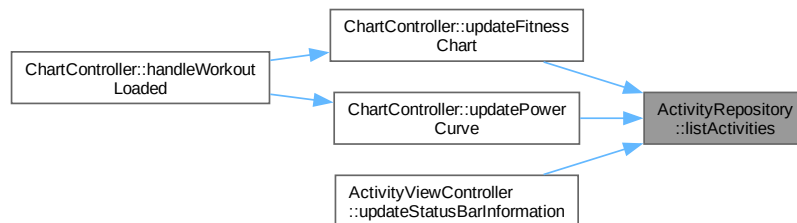
#### Returns

List of activities.

Definition at line 133 of file [ActivityRepository.cpp](#).

Referenced by [ChartController::updateFitnessChart\(\)](#), [ChartController::updatePowerCurve\(\)](#), and [ActivityViewController::updateStatus](#)

Here is the caller graph for this function:



### 7.9.3.7 setImportSource()

```
bool ActivityRepository::setImportSource (
    int activityId,
    const QString & source)
```

Sets the import source label for an activity.

#### Parameters

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID.                                     |
| <i>source</i>     | Import source label (e.g., "manual_import", "directory_import"). |

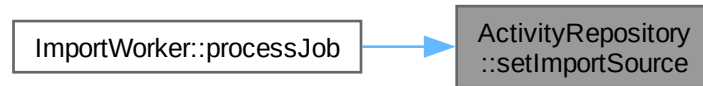
**Returns**

True if update succeeded.

Definition at line 199 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.9.3.8 updateActivityProperties()**

```
bool ActivityRepository::updateActivityProperties (
    int activityId,
    const Activity & a)
```

Updates comprehensive activity properties.

**Parameters**

|                   |                                           |
|-------------------|-------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID.              |
| <i>a</i>          | <a href="#">Activity</a> with new values. |

**Returns**

True if update succeeded.

Definition at line 220 of file [ActivityRepository.cpp](#).

References [Activity::bike](#), [Activity::equipment](#), [Activity::fatigue](#), [Activity::notes](#), [Activity::rpe](#), [Activity::sleepHours](#), [Activity::temperature](#), [Activity::weather](#), [Activity::weatherNotes](#), [Activity::weightKg](#), and [Activity::wind](#).

**7.9.3.9 updateAnalysisFlags()**

```
bool ActivityRepository::updateAnalysisFlags (
    int activityId,
    int flags)
```

Updates analysis flags for an activity.

**Parameters**

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
| <i>flags</i>      | Analysis flag bitmask.       |

**Returns**

True if update succeeded.

Definition at line 171 of file [ActivityRepository.cpp](#).

### 7.9.3.10 updateFingerprint()

```
bool ActivityRepository::updateFingerprint (
    int activityId,
    const QString & fingerprint)
```

Updates activity fingerprint for duplicate detection.

#### Parameters

|                    |                     |
|--------------------|---------------------|
| <i>activityId</i>  | Activity ID.        |
| <i>fingerprint</i> | Fingerprint string. |

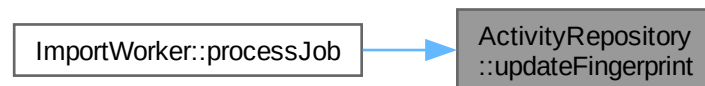
#### Returns

True if update succeeded.

Definition at line 180 of file [ActivityRepository.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



### 7.9.3.11 updateNotes()

```
bool ActivityRepository::updateNotes (
    int activityId,
    const QString & notes,
    const QString & weatherNotes)
```

Updates activity notes and weather information.

#### Parameters

|                     |                       |
|---------------------|-----------------------|
| <i>activityId</i>   | Activity ID.          |
| <i>notes</i>        | Activity notes.       |
| <i>weatherNotes</i> | Weather observations. |

#### Returns

True if update succeeded.

Definition at line 208 of file [ActivityRepository.cpp](#).

## 7.9.4 Member Data Documentation

### 7.9.4.1 m\_db

```
QSqlDatabase& ActivityRepository::m_db [private]
```

Definition at line 223 of file [ActivityRepository.h](#).

The documentation for this class was generated from the following files:

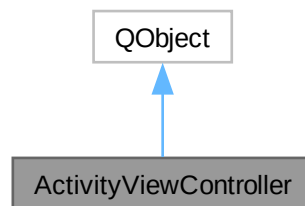
- [src/database/ActivityRepository.h](#)
- [src/database/ActivityRepository.cpp](#)

## 7.10 ActivityViewController Class Reference

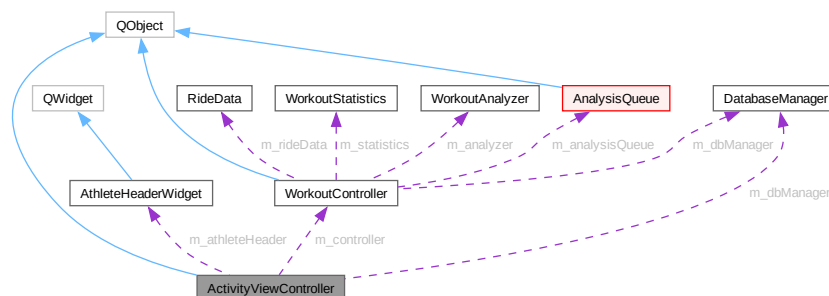
Manages activity loading, switching, and display operations.

```
#include <ActivityViewController.h>
```

Inheritance diagram for ActivityViewController:



Collaboration diagram for ActivityViewController:



### Signals

- void **activityLoaded** (int activityId)  
*Emitted when an activity has been successfully loaded.*
- void **activityDisplayUpdated** ()  
*Emitted when the activity display has been updated.*
- void **errorOccurred** (const QString &message)  
*Emitted when an error occurs during activity operations.*

### Public Member Functions

- [ActivityViewController](#) ([WorkoutController](#) \*controller, [DatabaseManager](#) \*dbManager, QObject \*parent=nullptr)  
*Constructs the activity view controller.*
- **~ActivityViewController** () override

*Destructor.*

- bool [loadActivity](#) (int activityId, QString &outError)  
*Loads an activity by ID.*
- void [refreshActivity](#) ()  
*Refreshes the display of the currently loaded activity.*
- void [updateActivityDisplay](#) ()  
*Updates all activity-related UI elements.*
- void [setAthleteContext](#) (int athleteId, const QString &athleteName)  
*Sets the athlete context information.*
- void [setAthleteHeaderWidget](#) (class [AthleteHeaderWidget](#) \*athleteHeader)  
*Sets the athlete header widget for display updates.*
- void [setStatusBarLabels](#) (class QLabel \*dbStatusLabel, class QLabel \*athleteStatusLabel, class QLabel \*activityCountLabel)  
*Sets the status bar widgets for updates.*
- void [updateStatsDisplay](#) ()  
*Updates activity metadata and stats display.*
- void [updateStatusBarInformation](#) ()  
*Updates status bar information.*

### Private Slots

- void [onWorkoutLoaded](#) ()  
*Slot for when [WorkoutController](#) loads a new activity.*

### Private Member Functions

- void [updateActivityMetadata](#) ()  
*Helper to update activity metadata in UI.*
- void [populateActivityDetails](#) ()  
*Helper to populate activity detail panels.*
- void [syncAnalysisViews](#) ()  
*Helper to sync charts and map with current activity.*
- QString [formatActivityDateLabel](#) (const QString &rawDate) const  
*Helper to format activity date for display.*

### Private Attributes

- class [AthleteHeaderWidget](#) \* [m\\_athleteHeader](#) = nullptr
- class QLabel \* [m\\_dbStatusLabel](#) = nullptr
- class QLabel \* [m\\_athleteStatusLabel](#) = nullptr
- class QLabel \* [m\\_activityCountLabel](#) = nullptr
- int [m\\_currentAthleteId](#) = -1
- QString [m\\_currentAthleteName](#)
- [WorkoutController](#) \* [m\\_controller](#)
- [DatabaseManager](#) \* [m\\_dbManager](#)

## 7.10.1 Detailed Description

Manages activity loading, switching, and display operations.  
Handles activity-specific UI updates including:

- Loading and switching between activities
- Refreshing activity display
- Updating activity metadata
- Populating activity detail panels

Definition at line 21 of file [ActivityViewController.h](#).

## 7.10.2 Constructor & Destructor Documentation

### 7.10.2.1 ActivityViewController()

```
ActivityViewController::ActivityViewController (
    WorkoutController * controller,
    DatabaseManager * dbManager,
    QObject * parent = nullptr) [explicit]
```

Constructs the activity view controller.

#### Parameters

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <i>controller</i> | Pointer to <a href="#">WorkoutController</a> for data access. |
| <i>dbManager</i>  | Pointer to <a href="#">DatabaseManager</a> for queries.       |
| <i>parent</i>     | Parent object.                                                |

Definition at line 21 of file [ActivityViewController.cpp](#).

References [onWorkoutLoaded\(\)](#).

Here is the call graph for this function:



## 7.10.3 Member Function Documentation

### 7.10.3.1 formatActivityDateLabel()

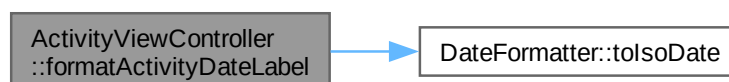
```
QString ActivityViewController::formatActivityDateLabel (
    const QString & rawDate) const [private]
```

Helper to format activity date for display.

Definition at line 197 of file [ActivityViewController.cpp](#).

References [DateFormatter::toISOString\(\)](#).

Here is the call graph for this function:



### 7.10.3.2 loadActivity()

```
bool ActivityViewController::loadActivity (
    int activityId,
```

```
QString & outError)
```

Loads an activity by ID.

#### Parameters

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>activityId</i> | ID of activity to load.                     |
| <i>outError</i>   | Output string for error message on failure. |

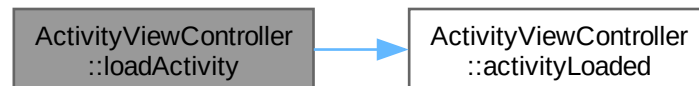
#### Returns

True if successfully loaded, false otherwise.

Definition at line 78 of file [ActivityViewController.cpp](#).

References [activityLoaded\(\)](#).

Here is the call graph for this function:



#### 7.10.3.3 onWorkoutLoaded

```
void ActivityViewController::onWorkoutLoaded () [private], [slot]
```

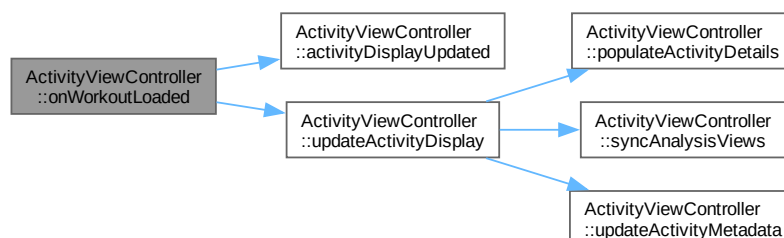
Slot for when [WorkoutController](#) loads a new activity.

Definition at line 155 of file [ActivityViewController.cpp](#).

References [activityDisplayUpdated\(\)](#), and [updateActivityDisplay\(\)](#).

Referenced by [ActivityViewController\(\)](#).

Here is the call graph for this function:





Here is the caller graph for this function:



#### 7.10.3.4 populateActivityDetails()

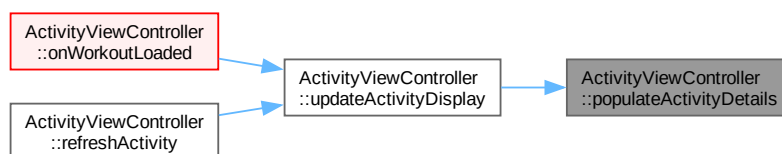
```
void ActivityViewController::populateActivityDetails () [private]
```

Helper to populate activity detail panels.

Definition at line 180 of file [ActivityViewController.cpp](#).

Referenced by [updateActivityDisplay\(\)](#).

Here is the caller graph for this function:



#### 7.10.3.5 refreshActivity()

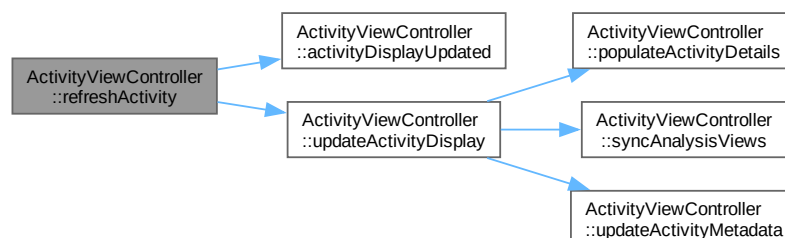
```
void ActivityViewController::refreshActivity ()
```

Refreshes the display of the currently loaded activity.

Definition at line 96 of file [ActivityViewController.cpp](#).

References [activityDisplayUpdated\(\)](#), and [updateActivityDisplay\(\)](#).

Here is the call graph for this function:



### 7.10.3.6 setAthleteContext()

```
void ActivityViewController::setAthleteContext (
    int athleteId,
    const QString & athleteName)
```

Sets the athlete context information.

#### Parameters

|                    |                       |
|--------------------|-----------------------|
| <i>athleteId</i>   | Current athlete ID.   |
| <i>athleteName</i> | Current athlete name. |

Definition at line 45 of file [ActivityViewController.cpp](#).

### 7.10.3.7 setAthleteHeaderWidget()

```
void ActivityViewController::setAthleteHeaderWidget (
    class AthleteHeaderWidget * athleteHeader)
```

Sets the athlete header widget for display updates.

#### Parameters

|                      |                                   |
|----------------------|-----------------------------------|
| <i>athleteHeader</i> | Pointer to athlete header widget. |
|----------------------|-----------------------------------|

Definition at line 54 of file [ActivityViewController.cpp](#).

### 7.10.3.8 setStatusBarLabels()

```
void ActivityViewController::setStatusBarLabels (
    class QLabel * dbStatusLabel,
    class QLabel * athleteStatusLabel,
    class QLabel * activityCountLabel)
```

Sets the status bar widgets for updates.

#### Parameters

|                           |                            |
|---------------------------|----------------------------|
| <i>dbStatusLabel</i>      | Label for database status. |
| <i>athleteStatusLabel</i> | Label for athlete status.  |
| <i>activityCountLabel</i> | Label for activity count.  |

Definition at line 62 of file [ActivityViewController.cpp](#).

### 7.10.3.9 syncAnalysisViews()

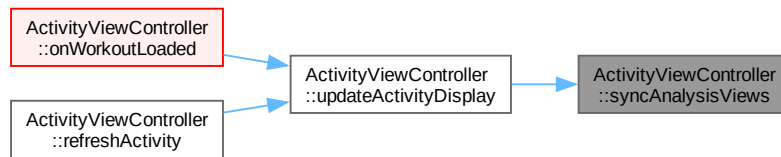
```
void ActivityViewController::syncAnalysisViews () [private]
```

Helper to sync charts and map with current activity.

Definition at line 188 of file [ActivityViewController.cpp](#).

Referenced by [updateActivityDisplay\(\)](#).

Here is the caller graph for this function:



### 7.10.3.10 updateActivityDisplay()

```
void ActivityViewController::updateActivityDisplay ()
```

Updates all activity-related UI elements.

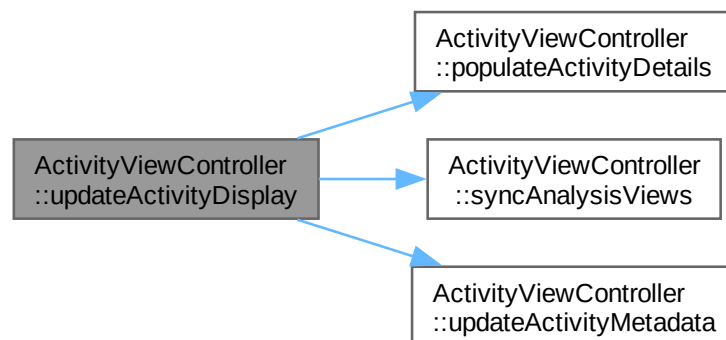
Called after loading a new activity or when data changes.

Definition at line 105 of file [ActivityViewController.cpp](#).

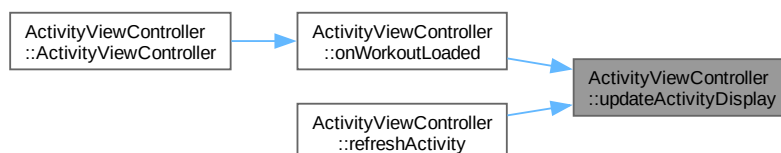
References [populateActivityDetails\(\)](#), [syncAnalysisViews\(\)](#), and [updateActivityMetadata\(\)](#).

Referenced by [onWorkoutLoaded\(\)](#), and [refreshActivity\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.10.3.11 updateActivityMetadata()

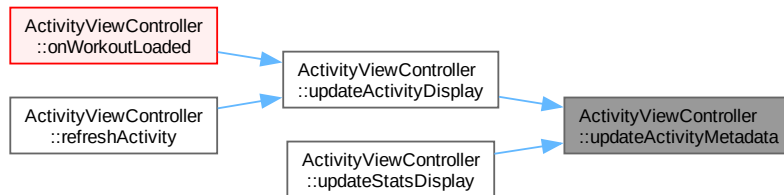
```
void ActivityViewController::updateActivityMetadata () [private]
```

Helper to update activity metadata in UI.

Definition at line 164 of file [ActivityViewController.cpp](#).

Referenced by [updateActivityDisplay\(\)](#), and [updateStatsDisplay\(\)](#).

Here is the caller graph for this function:



### 7.10.3.12 updateStatsDisplay()

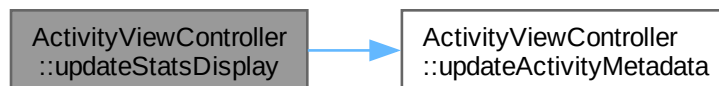
```
void ActivityViewController::updateStatsDisplay ()
```

Updates activity metadata and stats display.

Definition at line 115 of file [ActivityViewController.cpp](#).

References [updateActivityMetadata\(\)](#).

Here is the call graph for this function:



### 7.10.3.13 updateStatusBarInformation()

```
void ActivityViewController::updateStatusBarInformation ()
```

Updates status bar information.

Definition at line 123 of file [ActivityViewController.cpp](#).

References [ActivityRepository::listActivities\(\)](#).

Here is the call graph for this function:



## 7.10.4 Member Data Documentation

### 7.10.4.1 m\_activityCountLabel

```
class QLabel* ActivityViewController::m_activityCountLabel = nullptr [private]
```

Definition at line 127 of file [ActivityViewController.h](#).

### 7.10.4.2 m\_athleteHeader

```
class AthleteHeaderWidget* ActivityViewController::m_athleteHeader = nullptr [private]
```

Definition at line 124 of file [ActivityViewController.h](#).

### 7.10.4.3 m\_athleteStatusLabel

```
class QLabel* ActivityViewController::m_athleteStatusLabel = nullptr [private]
```

Definition at line 126 of file [ActivityViewController.h](#).

### 7.10.4.4 m\_controller

```
WorkoutController* ActivityViewController::m_controller [private]
```

Definition at line 133 of file [ActivityViewController.h](#).

### 7.10.4.5 m\_currentAthleteId

```
int ActivityViewController::m_currentAthleteId = -1 [private]
```

Definition at line 130 of file [ActivityViewController.h](#).

### 7.10.4.6 m\_currentAthleteName

```
QString ActivityViewController::m_currentAthleteName [private]
```

Definition at line 131 of file [ActivityViewController.h](#).

### 7.10.4.7 m\_dbManager

```
DatabaseManager* ActivityViewController::m_dbManager [private]
```

Definition at line 134 of file [ActivityViewController.h](#).

### 7.10.4.8 m\_dbStatusLabel

```
class QLabel* ActivityViewController::m_dbStatusLabel = nullptr [private]
```

Definition at line 125 of file [ActivityViewController.h](#).

The documentation for this class was generated from the following files:

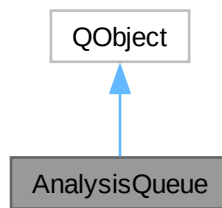
- [src/gui/controllers/ActivityViewController.h](#)
- [src/gui/controllers/ActivityViewController.cpp](#)

## 7.11 AnalysisQueue Class Reference

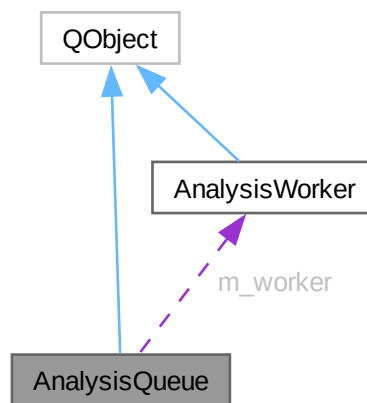
Manages sequential background analysis task queue.

```
#include <AnalysisQueue.h>
```

Inheritance diagram for AnalysisQueue:



Collaboration diagram for AnalysisQueue:



## Signals

- void `taskCompleted` (int activityId)  
*signal Emitted when a task completes successfully.*
- void `taskFailed` (int activityId, const QString &error)  
*signal Emitted when a task fails.*
- void `pendingCountChanged` (int count)  
*signal Emitted when pending task count changes.*

## Public Member Functions

- `AnalysisQueue` (QObject \*parent=nullptr)  
*Constructs analysis queue.*
- void `setDatabasePath` (const QString &path)  
*Sets the database path for analysis operations.*
- void `enqueue` (int activityId)  
*Queues an activity for background analysis.*

- int [pendingCount](#) () const  
*Gets the number of pending analysis tasks.*
- bool [isBusy](#) () const  
*Checks if analysis is currently running.*

### Private Slots

- void [onTaskCompleted](#) (int activityId)
- void [onTaskFailed](#) (int activityId, const QString &error)

### Private Member Functions

- void [processNext](#) ()

### Private Attributes

- QThread \* [m\\_thread](#) = nullptr
- [AnalysisWorker](#) \* [m\\_worker](#) = nullptr
- QQueue< int > [m\\_queue](#)
- bool [m\\_busy](#) = false
- QString [m\\_dbPath](#)

## 7.11.1 Detailed Description

Manages sequential background analysis task queue.

Processes activities one at a time in a background thread, detecting climbs, intervals, and computing analysis metrics.

Definition at line 19 of file [AnalysisQueue.h](#).

## 7.11.2 Constructor & Destructor Documentation

### 7.11.2.1 AnalysisQueue()

```
AnalysisQueue::AnalysisQueue (
    QObject * parent = nullptr) [explicit]
```

Constructs analysis queue.

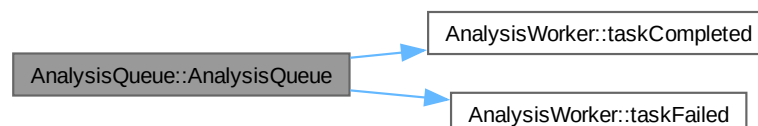
### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent object. |
|---------------|----------------|

Definition at line 9 of file [AnalysisQueue.cpp](#).

References [AnalysisWorker::taskCompleted\(\)](#), and [AnalysisWorker::taskFailed\(\)](#).

Here is the call graph for this function:



### 7.11.2.2 ~AnalysisQueue()

AnalysisQueue::~~AnalysisQueue () [override]  
 Definition at line 24 of file [AnalysisQueue.cpp](#).

## 7.11.3 Member Function Documentation

### 7.11.3.1 enqueue()

```
void AnalysisQueue::enqueue (
    int activityId)
```

Queues an activity for background analysis.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID to analyze. |
|-------------------|-----------------------------------------|

Definition at line 40 of file [AnalysisQueue.cpp](#).  
 References [pendingCountChanged\(\)](#).  
 Here is the call graph for this function:



### 7.11.3.2 isBusy()

```
bool AnalysisQueue::isBusy () const
```

Checks if analysis is currently running.

#### Returns

True if a task is being processed.

Definition at line 53 of file [AnalysisQueue.cpp](#).

### 7.11.3.3 onTaskCompleted

```
void AnalysisQueue::onTaskCompleted (
    int activityId) [private], [slot]
```

Definition at line 72 of file [AnalysisQueue.cpp](#).

### 7.11.3.4 onTaskFailed

```
void AnalysisQueue::onTaskFailed (
    int activityId,
    const QString & error) [private], [slot]
```

Definition at line 79 of file [AnalysisQueue.cpp](#).

### 7.11.3.5 pendingCount()

```
int AnalysisQueue::pendingCount () const
```

Gets the number of pending analysis tasks.



**Returns**

Count of queued activities.

Definition at line 48 of file [AnalysisQueue.cpp](#).

**7.11.3.6 pendingCountChanged**

```
void AnalysisQueue::pendingCountChanged (
    int count) [signal]
```

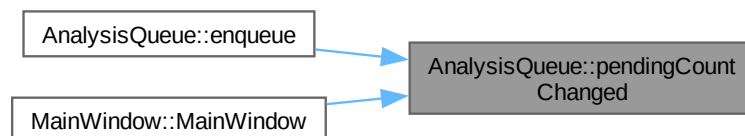
\signal Emitted when pending task count changes.

**Parameters**

|              |                                   |
|--------------|-----------------------------------|
| <i>count</i> | Number of pending analysis tasks. |
|--------------|-----------------------------------|

Referenced by [enqueue\(\)](#), and [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:

**7.11.3.7 processNext()**

```
void AnalysisQueue::processNext () [private]
```

Definition at line 58 of file [AnalysisQueue.cpp](#).

**7.11.3.8 setDatabasePath()**

```
void AnalysisQueue::setDatabasePath (
    const QString & path)
```

Sets the database path for analysis operations.

**Parameters**

|             |                     |
|-------------|---------------------|
| <i>path</i> | Database file path. |
|-------------|---------------------|

Definition at line 31 of file [AnalysisQueue.cpp](#).

**7.11.3.9 taskCompleted**

```
void AnalysisQueue::taskCompleted (
    int activityId) [signal]
```

\signal Emitted when a task completes successfully.

**Parameters**

|                   |                                                |
|-------------------|------------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID that was analyzed. |
|-------------------|------------------------------------------------|

Referenced by [MainWindow::MainWindow\(\)](#).  
Here is the caller graph for this function:

**7.11.3.10 taskFailed**

```
void AnalysisQueue::taskFailed (
    int activityId,
    const QString & error) [signal]
```

\signal Emitted when a task fails.

**Parameters**

|                   |                                          |
|-------------------|------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID that failed. |
| <i>error</i>      | Error message describing the failure.    |

**7.11.4 Member Data Documentation****7.11.4.1 m\_busy**

```
bool AnalysisQueue::m_busy = false [private]
```

Definition at line 78 of file [AnalysisQueue.h](#).

**7.11.4.2 m\_dbPath**

```
QString AnalysisQueue::m_dbPath [private]
```

Definition at line 79 of file [AnalysisQueue.h](#).

**7.11.4.3 m\_queue**

```
QQueue<int> AnalysisQueue::m_queue [private]
```

Definition at line 77 of file [AnalysisQueue.h](#).

**7.11.4.4 m\_thread**

```
QThread* AnalysisQueue::m_thread = nullptr [private]
```

Definition at line 75 of file [AnalysisQueue.h](#).

**7.11.4.5 m\_worker**

```
AnalysisWorker* AnalysisQueue::m_worker = nullptr [private]
```

Definition at line 76 of file [AnalysisQueue.h](#).

The documentation for this class was generated from the following files:

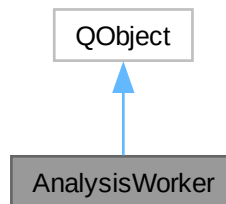
- `src/analysis/queue/AnalysisQueue.h`
- `src/analysis/queue/AnalysisQueue.cpp`

## 7.12 AnalysisWorker Class Reference

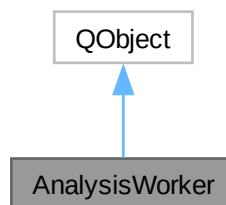
Worker thread for background activity analysis.

```
#include <AnalysisWorker.h>
```

Inheritance diagram for AnalysisWorker:



Collaboration diagram for AnalysisWorker:



### Public Slots

- void `processTask` (int activityId)  
*Processes an activity analysis task.*

### Signals

- void `taskCompleted` (int activityId)  
*signal Emitted when a task completes successfully.*
- void `taskFailed` (int activityId, const QString &error)  
*signal Emitted when a task fails.*

### Public Member Functions

- `AnalysisWorker` (QObject \*parent=nullptr)  
*Constructs analysis worker.*

- void [setDatabasePath](#) (const QString &path)  
*Sets database path for analysis operations.*

### Private Attributes

- QString [m\\_dbPath](#)

## 7.12.1 Detailed Description

Worker thread for background activity analysis.

Processes analysis tasks (detect climbs, intervals, compute metrics) synchronously in a dedicated thread.

Definition at line 15 of file [AnalysisWorker.h](#).

## 7.12.2 Constructor & Destructor Documentation

### 7.12.2.1 AnalysisWorker()

```
AnalysisWorker::AnalysisWorker (
    QObject * parent = nullptr) [explicit]
```

Constructs analysis worker.

#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent object. |
|---------------|----------------|

Definition at line 37 of file [AnalysisWorker.cpp](#).

## 7.12.3 Member Function Documentation

### 7.12.3.1 processTask

```
void AnalysisWorker::processTask (
    int activityId) [slot]
```

Processes an activity analysis task.

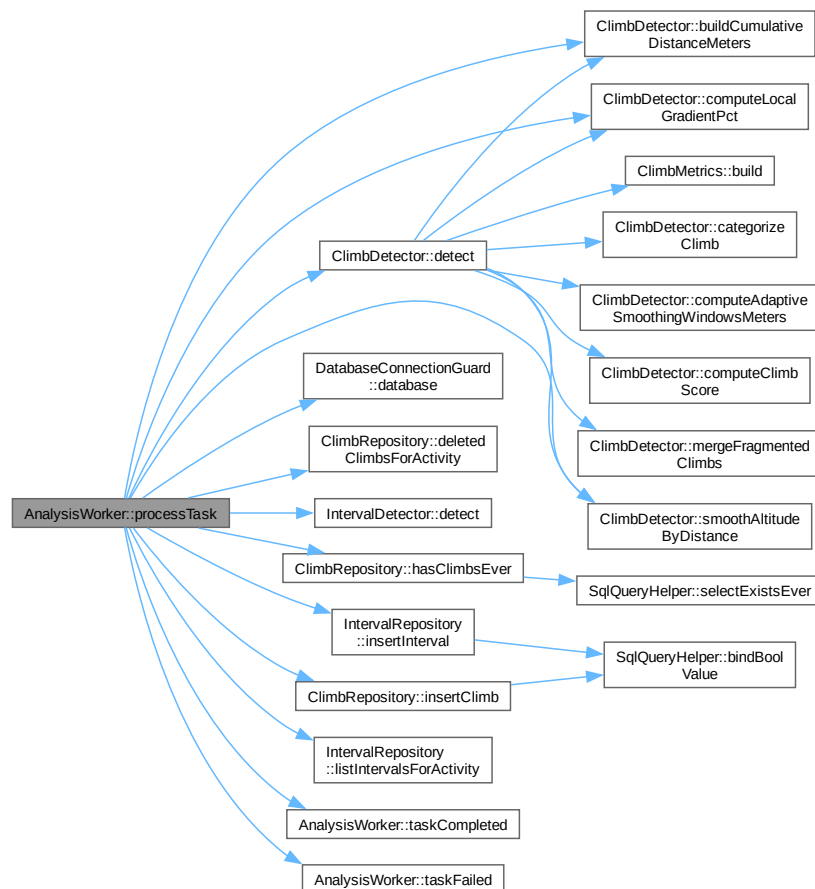
#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID to analyze. |
|-------------------|-----------------------------------------|

Definition at line 46 of file [AnalysisWorker.cpp](#).

References [ClimbRecord::activityId](#), [IntervalRecord::activityId](#), [ClimbRecord::algorithmVersion](#), [IntervalRecord::algorithmVersion](#), [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [IntervalRecord::avgCadence](#), [ClimbRecord::avgHr](#), [IntervalRecord::avgHr](#), [ClimbRecord::avgPower](#), [IntervalRecord::avgPower](#), [ClimbDetector::buildCumulativeDistanceMeters\(\)](#), [AnalysisVersions::ClimbDetection](#), [ClimbDetector::computeLocalGradientPct\(\)](#), [DatabaseConnectionGuard::database\(\)](#), [ClimbRecord::deleted](#), [IntervalRecord::deleted](#), [ClimbRepository::deletedClimbsForActivity\(\)](#), [ClimbDetector::detect\(\)](#), [IntervalDetector::detect\(\)](#), [ClimbRecord::elevationGainM](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [IntervalRecord::endSample](#), [ClimbRecord::endSeconds](#), [IntervalDetectorConfig::ftp](#), [ClimbDetectorConfig::gradientWindowHalfMeters](#), [ActivityAnalysisFlags::HasClimbs](#), [ClimbRepository::hasClimbsEver\(\)](#), [ActivityAnalysisFlags::HasIntervals](#), [ClimbRepository::insertClimb\(\)](#), [IntervalRepository::insertInterval\(\)](#), [AnalysisVersions::IntervalDetection](#), [ClimbRecord::lengthKm](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [ClimbRecord::locked](#), [IntervalRecord::locked](#), [ClimbDetectorConfig::minAverageGradient](#), [ClimbDetectorConfig::minElevationGainM](#), [ClimbDetectorConfig::minLengthKm](#), [ClimbRecord::name](#), [IntervalRecord::name](#), [ClimbRecord::np](#), [IntervalRecord::np](#), [ClimbRecord::originalEndSeconds](#), [ClimbRecord::originalStartSeconds](#), [RideData::records](#), [ClimbDetector::smoothAltitudeByDistance\(\)](#), [ClimbRecord::source](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [ClimbRecord::startSeconds](#), [taskCompleted\(\)](#), [taskFailed\(\)](#), [IntervalRecord::type](#), [ClimbRecord::uuid](#), [IntervalRecord::uuid](#), and [ClimbRecord::vam](#).

Here is the call graph for this function:



### 7.12.3.2 setDatabasePath()

```
void AnalysisWorker::setDatabasePath (
    const QString & path)
```

Sets database path for analysis operations.

#### Parameters

|             |                     |
|-------------|---------------------|
| <i>path</i> | Database file path. |
|-------------|---------------------|

Definition at line 41 of file [AnalysisWorker.cpp](#).

### 7.12.3.3 taskCompleted

```
void AnalysisWorker::taskCompleted (
    int activityId) [signal]
```

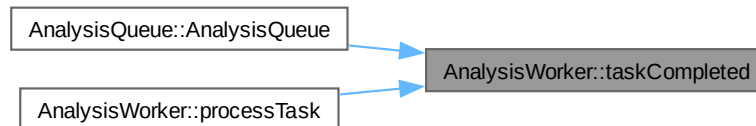
\signal Emitted when a task completes successfully.

**Parameters**

|                   |                                                |
|-------------------|------------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID that was analyzed. |
|-------------------|------------------------------------------------|

Referenced by [AnalysisQueue::AnalysisQueue\(\)](#), and [processTask\(\)](#).

Here is the caller graph for this function:

**7.12.3.4 taskFailed**

```
void AnalysisWorker::taskFailed (
    int activityId,
    const QString & error) [signal]
```

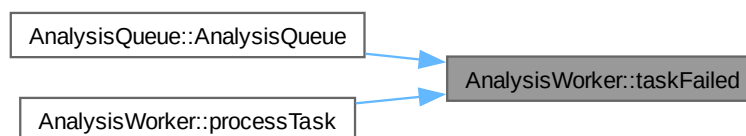
\signal Emitted when a task fails.

**Parameters**

|                   |                                          |
|-------------------|------------------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID that failed. |
| <i>error</i>      | Error message describing the failure.    |

Referenced by [AnalysisQueue::AnalysisQueue\(\)](#), and [processTask\(\)](#).

Here is the caller graph for this function:

**7.12.4 Member Data Documentation****7.12.4.1 m\_dbPath**

```
QString AnalysisWorker::m_dbPath [private]
```

Definition at line 49 of file [AnalysisWorker.h](#).

The documentation for this class was generated from the following files:

- src/analysis/queue/AnalysisWorker.h
- src/analysis/queue/AnalysisWorker.cpp

## 7.13 AppSettings Class Reference

Application-wide settings singleton.

```
#include <AppSettings.h>
```

### Public Member Functions

- `DateFormat` [dateFormat](#) () const  
*Gets the current date format preference.*
- `void` [setDateFormat](#) (DateFormat format)  
*Sets the date format preference.*
- `int` [tileCacheSize](#) () const  
*Gets the tile cache size.*
- `void` [setTileCacheSize](#) (int tiles)  
*Sets the tile cache size.*

### Static Public Member Functions

- `static` [AppSettings](#) & [instance](#) ()  
*Returns the global settings instance.*

### Static Private Member Functions

- `static` `QString` [dateFormatKey](#) ()
- `static` `DateFormat` [sanitizeDateFormat](#) (int rawValue)
- `static` `QString` [tileCacheSizeKey](#) ()

### 7.13.1 Detailed Description

Application-wide settings singleton.

Manages persistent user preferences including date formatting and cache sizes.

Definition at line 26 of file [AppSettings.h](#).

### 7.13.2 Member Function Documentation

#### 7.13.2.1 dateFormat()

```
DateFormat AppSettings::dateFormat () const
```

Gets the current date format preference.

Returns

Current DateFormat.

Definition at line 14 of file [AppSettings.cpp](#).

#### 7.13.2.2 dateFormatKey()

```
QString AppSettings::dateFormatKey () [static], [private]
```

Definition at line 27 of file [AppSettings.cpp](#).

#### 7.13.2.3 instance()

```
AppSettings & AppSettings::instance () [static]
```

Returns the global settings instance.

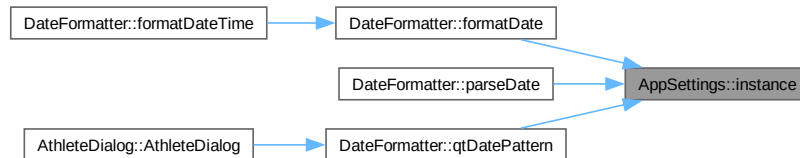
**Returns**

Reference to the singleton [AppSettings](#).

Definition at line 8 of file [AppSettings.cpp](#).

Referenced by [DateFormatter::formatDate\(\)](#), [DateFormatter::parseDate\(\)](#), and [DateFormatter::qtDatePattern\(\)](#).

Here is the caller graph for this function:

**7.13.2.4 sanitizeDateFormat()**

```
DateFormat AppSettings::sanitizeDateFormat (
    int rawValue) [static], [private]
```

Definition at line 32 of file [AppSettings.cpp](#).

**7.13.2.5 setDateFormat()**

```
void AppSettings::setDateFormat (
    DateFormat format)
```

Sets the date format preference.

**Parameters**

|               |                 |
|---------------|-----------------|
| <i>format</i> | New DateFormat. |
|---------------|-----------------|

Definition at line 21 of file [AppSettings.cpp](#).

**7.13.2.6 setTileCacheSize()**

```
void AppSettings::setTileCacheSize (
    int tiles)
```

Sets the tile cache size.

**Parameters**

|              |                                      |
|--------------|--------------------------------------|
| <i>tiles</i> | Number of tiles to cache (128-2048). |
|--------------|--------------------------------------|

Definition at line 61 of file [AppSettings.cpp](#).

**7.13.2.7 tileCacheSize()**

```
int AppSettings::tileCacheSize () const
```

Gets the tile cache size.

**Returns**

Number of tiles to keep in RAM (128-2048).

Definition at line 49 of file [AppSettings.cpp](#).



### 7.13.2.8 tileCacheSizeKey()

QString AppSettings::tileCacheSizeKey () [static], [private]  
Definition at line 67 of file [AppSettings.cpp](#).

The documentation for this class was generated from the following files:

- src/core/settings/AppSettings.h
- src/core/settings/AppSettings.cpp

## 7.14 Athlete Struct Reference

Represents an athlete profile.

```
#include <AthleteRepository.h>
```

### Public Member Functions

- QString [fullName](#) () const  
*Returns full name (first + space + last).*
- bool [isValid](#) () const  
*Checks if athlete data is valid.*

### Public Attributes

- int [id](#) = -1  
*Unique athlete identifier.*
- QString [firstName](#)  
*Athlete first name.*
- QString [lastName](#)  
*Athlete last name.*
- QString [dateOfBirth](#)  
*Date of birth (ISO 8601 YYYY-MM-DD).*
- QString [email](#)  
*Email address.*
- double [heightCm](#) = 0.0  
*Height in centimeters.*
- int [ftpWatts](#) = 250  
*Functional threshold power in watts.*
- QString [createdAt](#)  
*ISO8601 creation timestamp.*

### 7.14.1 Detailed Description

Represents an athlete profile.

Definition at line 15 of file [AthleteRepository.h](#).

### 7.14.2 Member Function Documentation

#### 7.14.2.1 fullName()

QString Athlete::fullName () const [inline]

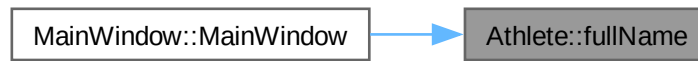
Returns full name (first + space + last).

Definition at line 44 of file [AthleteRepository.h](#).

References [firstName](#), and [lastName](#).

Referenced by [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



#### 7.14.2.2 isValid()

```
bool Athlete::isValid () const [inline]
```

Checks if athlete data is valid.

Definition at line 49 of file [AthleteRepository.h](#).

### 7.14.3 Member Data Documentation

#### 7.14.3.1 createdAt

```
QString Athlete::createdAt
```

ISO8601 creation timestamp.

Definition at line 39 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), and [AthleteRepository::listAthletes\(\)](#).

#### 7.14.3.2 dateOfBirth

```
QString Athlete::dateOfBirth
```

Date of birth (ISO 8601 YYYY-MM-DD).

Definition at line 27 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.3 email

```
QString Athlete::email
```

Email address.

Definition at line 30 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.4 firstName

```
QString Athlete::firstName
```

[Athlete](#) first name.

Definition at line 21 of file [AthleteRepository.h](#).

Referenced by [fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.5 ftpWatts

```
int Athlete::ftpWatts = 250
```

Functional threshold power in watts.

Definition at line 36 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.6 heightCm

```
double Athlete::heightCm = 0.0
```

Height in centimeters.

Definition at line 33 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.7 id

```
int Athlete::id = -1
```

Unique athlete identifier.

Definition at line 18 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

#### 7.14.3.8 lastName

```
QString Athlete::lastName
```

[Athlete](#) last name.

Definition at line 24 of file [AthleteRepository.h](#).

Referenced by [fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [AthleteRepository::insertAthlete\(\)](#), [AthleteRepository::listAthletes\(\)](#), and [AthleteRepository::updateAthlete\(\)](#).

The documentation for this struct was generated from the following file:

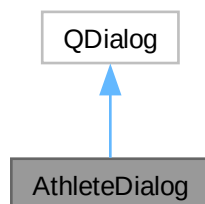
- [src/database/AthleteRepository.h](#)

## 7.15 AthleteDialog Class Reference

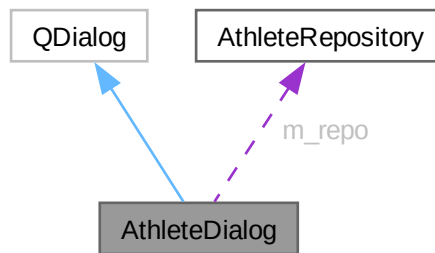
Dialog for creating or editing athlete profile.

```
#include <AthleteDialog.h>
```

Inheritance diagram for AthleteDialog:



Collaboration diagram for AthleteDialog:



### Public Member Functions

- [AthleteDialog](#) ([AthleteRepository](#) &repo, int athleteId=-1, QWidget \*parent=nullptr)  
*Constructs athlete edit dialog.*
- int [savedAthleteId](#) () const  
*Gets ID of saved athlete.*

### Private Slots

- void [onAccepted](#) ()
- void [addFtpRow](#) ()
- void [deleteFtpRow](#) ()
- void [addWeightRow](#) ()
- void [deleteWeightRow](#) ()

### Private Member Functions

- void [loadAthlete](#) ()
- void [populateFtpTable](#) (const QList< [FtpEntry](#) > &entries)
- void [populateWeightTable](#) (const QList< [WeightEntry](#) > &entries)

### Private Attributes

- [AthleteRepository](#) & [m\\_repo](#)
- int [m\\_athleteId](#)
- int [m\\_savedId](#) = -1
- QLineEdit \* [m\\_firstName](#) = nullptr
- QLineEdit \* [m\\_lastName](#) = nullptr
- QLineEdit \* [m\\_dob](#) = nullptr
- QLineEdit \* [m\\_email](#) = nullptr
- QDoubleSpinBox \* [m\\_height](#) = nullptr
- QSpinBox \* [m ftp](#) = nullptr
- QWidget \* [m\\_ftpTable](#) = nullptr
- QWidget \* [m\\_weightTable](#) = nullptr

## 7.15.1 Detailed Description

Dialog for creating or editing athlete profile.  
Allows entry of personal info, FTP history, and weight history.  
Definition at line 21 of file [AthleteDialog.h](#).

## 7.15.2 Constructor & Destructor Documentation

### 7.15.2.1 AthleteDialog()

```
AthleteDialog::AthleteDialog (
    AthleteRepository & repo,
    int athleteId = -1,
    QWidget * parent = nullptr) [explicit]
```

Constructs athlete edit dialog.

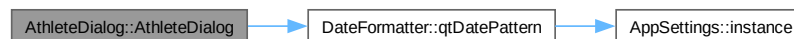
#### Parameters

|                  |                                        |
|------------------|----------------------------------------|
| <i>repo</i>      | Athlete repository.                    |
| <i>athleteId</i> | Athlete ID to edit (-1 to create new). |
| <i>parent</i>    | Parent widget.                         |

Definition at line 40 of file [AthleteDialog.cpp](#).

References [DateFormatter::qtDatePattern\(\)](#).

Here is the call graph for this function:



## 7.15.3 Member Function Documentation

### 7.15.3.1 addFtpRow

```
void AthleteDialog::addFtpRow () [private], [slot]
```

Definition at line 175 of file [AthleteDialog.cpp](#).

### 7.15.3.2 addWeightRow

```
void AthleteDialog::addWeightRow () [private], [slot]
```

Definition at line 196 of file [AthleteDialog.cpp](#).

### 7.15.3.3 deleteFtpRow

```
void AthleteDialog::deleteFtpRow () [private], [slot]
```

Definition at line 186 of file [AthleteDialog.cpp](#).

### 7.15.3.4 deleteWeightRow

```
void AthleteDialog::deleteWeightRow () [private], [slot]
```

Definition at line 206 of file [AthleteDialog.cpp](#).

### 7.15.3.5 loadAthlete()

```
void AthleteDialog::loadAthlete () [private]
```

Definition at line 132 of file [AthleteDialog.cpp](#).

#### 7.15.3.6 onAccepted

```
void AthleteDialog::onAccepted () [private], [slot]
```

Definition at line 216 of file [AthleteDialog.cpp](#).

#### 7.15.3.7 populateFtpTable()

```
void AthleteDialog::populateFtpTable (  
    const QList< FtpEntry > & entries) [private]
```

Definition at line 146 of file [AthleteDialog.cpp](#).

#### 7.15.3.8 populateWeightTable()

```
void AthleteDialog::populateWeightTable (  
    const QList< WeightEntry > & entries) [private]
```

Definition at line 161 of file [AthleteDialog.cpp](#).

#### 7.15.3.9 savedAthleteld()

```
int AthleteDialog::savedAthleteId () const [inline]
```

Gets ID of saved athlete.

Returns

[Athlete](#) ID (valid after Accepted).

Definition at line 39 of file [AthleteDialog.h](#).

### 7.15.4 Member Data Documentation

#### 7.15.4.1 m\_athleteld

```
int AthleteDialog::m_athleteId [private]
```

Definition at line 54 of file [AthleteDialog.h](#).

#### 7.15.4.2 m\_dob

```
QLineEdit* AthleteDialog::m_dob = nullptr [private]
```

Definition at line 59 of file [AthleteDialog.h](#).

#### 7.15.4.3 m\_email

```
QLineEdit* AthleteDialog::m_email = nullptr [private]
```

Definition at line 60 of file [AthleteDialog.h](#).

#### 7.15.4.4 m\_firstName

```
QLineEdit* AthleteDialog::m_firstName = nullptr [private]
```

Definition at line 57 of file [AthleteDialog.h](#).

#### 7.15.4.5 m\_ftp

```
QSpinBox* AthleteDialog::m_ftp = nullptr [private]
```

Definition at line 62 of file [AthleteDialog.h](#).

#### 7.15.4.6 m\_ftpTable

```
QTableWidget* AthleteDialog::m_ftpTable = nullptr [private]
```

Definition at line 63 of file [AthleteDialog.h](#).

#### 7.15.4.7 m\_height

`QDoubleSpinBox* AthleteDialog::m_height = nullptr [private]`  
Definition at line 61 of file [AthleteDialog.h](#).

#### 7.15.4.8 m\_lastName

`QLineEdit* AthleteDialog::m_lastName = nullptr [private]`  
Definition at line 58 of file [AthleteDialog.h](#).

#### 7.15.4.9 m\_repo

`AthleteRepository& AthleteDialog::m_repo [private]`  
Definition at line 53 of file [AthleteDialog.h](#).

#### 7.15.4.10 m\_savedId

`int AthleteDialog::m_savedId = -1 [private]`  
Definition at line 55 of file [AthleteDialog.h](#).

#### 7.15.4.11 m\_weightTable

`QTableWidget* AthleteDialog::m_weightTable = nullptr [private]`  
Definition at line 64 of file [AthleteDialog.h](#).

The documentation for this class was generated from the following files:

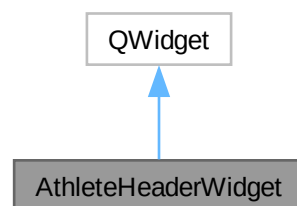
- `src/gui/AthleteDialog.h`
- `src/gui/AthleteDialog.cpp`

## 7.16 AthleteHeaderWidget Class Reference

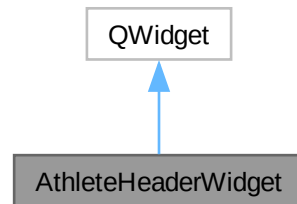
Header bar displaying current athlete and ride metrics.

```
#include <AthleteHeaderWidget.h>
```

Inheritance diagram for AthleteHeaderWidget:



Collaboration diagram for `AthleteHeaderWidget`:



### Public Member Functions

- [AthleteHeaderWidget](#) (`QWidget *parent=nullptr`)
- void [setSummary](#) (`const QString &athleteName`, `int ftpWatts`, `int activityCount`, `const QString &lastActivityDate`)
- void [setRideMetrics](#) (`const QString &metricsText`)

### Private Attributes

- `QLabel * m_nameLabel = nullptr`
- `QLabel * m_statsLabel = nullptr`
- `QLabel * m_metricsLabel = nullptr`

## 7.16.1 Detailed Description

Header bar displaying current athlete and ride metrics.

Shows athlete name, FTP, activity count, and current ride metrics (NP, IF, TSS).

Definition at line 15 of file [AthleteHeaderWidget.h](#).

## 7.16.2 Constructor & Destructor Documentation

### 7.16.2.1 AthleteHeaderWidget()

```
AthleteHeaderWidget::AthleteHeaderWidget (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 9 of file [AthleteHeaderWidget.cpp](#).

## 7.16.3 Member Function Documentation

### 7.16.3.1 setRideMetrics()

```
void AthleteHeaderWidget::setRideMetrics (
    const QString & metricsText)
```

Definition at line 60 of file [AthleteHeaderWidget.cpp](#).

### 7.16.3.2 setSummary()

```
void AthleteHeaderWidget::setSummary (
    const QString & athleteName,
    int ftpWatts,
    int activityCount,
    const QString & lastActivityDate)
```

Definition at line 47 of file [AthleteHeaderWidget.cpp](#).



## 7.16.4 Member Data Documentation

### 7.16.4.1 m\_metricsLabel

QLabel\* AthleteHeaderWidget::m\_metricsLabel = nullptr [private]

Definition at line 32 of file [AthleteHeaderWidget.h](#).

### 7.16.4.2 m\_nameLabel

QLabel\* AthleteHeaderWidget::m\_nameLabel = nullptr [private]

Definition at line 30 of file [AthleteHeaderWidget.h](#).

### 7.16.4.3 m\_statsLabel

QLabel\* AthleteHeaderWidget::m\_statsLabel = nullptr [private]

Definition at line 31 of file [AthleteHeaderWidget.h](#).

The documentation for this class was generated from the following files:

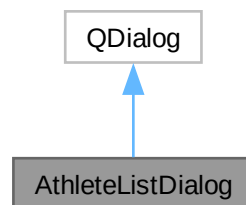
- src/gui/AthleteHeaderWidget.h
- src/gui/AthleteHeaderWidget.cpp

## 7.17 AthleteListDialog Class Reference

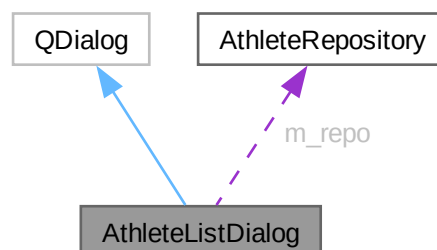
Dialog for browsing and managing athletes.

```
#include <AthleteListDialog.h>
```

Inheritance diagram for AthleteListDialog:



Collaboration diagram for AthleteListDialog:



## Public Member Functions

- [AthleteListDialog](#) ([AthleteRepository](#) &repo, [QWidget](#) \*parent=nullptr)
- int [selectedAthleteId](#) () const

## Private Slots

- void [refresh](#) ()
- void [newAthlete](#) ()
- void [editAthlete](#) ()
- void [deleteAthlete](#) ()
- void [selectAthlete](#) ()

## Private Attributes

- [AthleteRepository](#) & [m\\_repo](#)
- int [m\\_selectedId](#) = -1
- [QTableWidget](#) \* [m\\_table](#) = nullptr
- [QPushButton](#) \* [m\\_editBtn](#) = nullptr
- [QPushButton](#) \* [m\\_deleteBtn](#) = nullptr
- [QPushButton](#) \* [m\\_selectBtn](#) = nullptr

### 7.17.1 Detailed Description

Dialog for browsing and managing athletes.

Lists all athletes, allows selection as current athlete, and provides create/edit/delete operations.

Definition at line 19 of file [AthleteListDialog.h](#).

### 7.17.2 Constructor & Destructor Documentation

#### 7.17.2.1 AthleteListDialog()

```
AthleteListDialog::AthleteListDialog (  
    AthleteRepository & repo,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 19 of file [AthleteListDialog.cpp](#).

### 7.17.3 Member Function Documentation

#### 7.17.3.1 deleteAthlete

```
void AthleteListDialog::deleteAthlete () [private], [slot]
```

Definition at line 111 of file [AthleteListDialog.cpp](#).

#### 7.17.3.2 editAthlete

```
void AthleteListDialog::editAthlete () [private], [slot]
```

Definition at line 101 of file [AthleteListDialog.cpp](#).

#### 7.17.3.3 newAthlete

```
void AthleteListDialog::newAthlete () [private], [slot]
```

Definition at line 94 of file [AthleteListDialog.cpp](#).

#### 7.17.3.4 refresh

```
void AthleteListDialog::refresh () [private], [slot]
```

Definition at line 77 of file [AthleteListDialog.cpp](#).

### 7.17.3.5 selectAthlete

void AthleteListDialog::selectAthlete () [private], [slot]  
 Definition at line 128 of file [AthleteListDialog.cpp](#).

### 7.17.3.6 selectedAthleteId()

int AthleteListDialog::selectedAthleteId () const [inline]  
 Definition at line 26 of file [AthleteListDialog.h](#).

## 7.17.4 Member Data Documentation

### 7.17.4.1 m\_deleteBtn

QPushButton\* AthleteListDialog::m\_deleteBtn = nullptr [private]  
 Definition at line 41 of file [AthleteListDialog.h](#).

### 7.17.4.2 m\_editBtn

QPushButton\* AthleteListDialog::m\_editBtn = nullptr [private]  
 Definition at line 40 of file [AthleteListDialog.h](#).

### 7.17.4.3 m\_repo

[AthleteRepository](#)& AthleteListDialog::m\_repo [private]  
 Definition at line 36 of file [AthleteListDialog.h](#).

### 7.17.4.4 m\_selectBtn

QPushButton\* AthleteListDialog::m\_selectBtn = nullptr [private]  
 Definition at line 42 of file [AthleteListDialog.h](#).

### 7.17.4.5 m\_selectedId

int AthleteListDialog::m\_selectedId = -1 [private]  
 Definition at line 37 of file [AthleteListDialog.h](#).

### 7.17.4.6 m\_table

QTableWidget\* AthleteListDialog::m\_table = nullptr [private]  
 Definition at line 39 of file [AthleteListDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/AthleteListDialog.h](#)
- [src/gui/AthleteListDialog.cpp](#)

## 7.18 AthleteRepository Class Reference

Repository for athlete profile data persistence.

#include <AthleteRepository.h>

### Public Member Functions

- [AthleteRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- int [insertAthlete](#) (const [Athlete](#) &a)  
*Inserts a new athlete.*
- bool [updateAthlete](#) (const [Athlete](#) &a)  
*Updates an existing athlete.*

- bool [deleteAthlete](#) (int athleteId)  
*Deletes an athlete.*
- QList< [Athlete](#) > [listAthletes](#) ()  
*Lists all athletes.*
- [Athlete](#) [getAthlete](#) (int athleteId)  
*Retrieves an athlete by ID.*
- int [addFtpEntry](#) (const [FtpEntry](#) &e)  
*Adds an FTP history entry.*
- bool [deleteFtpEntry](#) (int entryId)  
*Deletes an FTP history entry.*
- QList< [FtpEntry](#) > [getFtpHistory](#) (int athleteId)  
*Retrieves FTP history for an athlete.*
- int [getFtpForDate](#) (int athleteId, const QDate &date)  
*Gets FTP for an athlete on a specific date.*
- int [addWeightEntry](#) (const [WeightEntry](#) &e)  
*Adds a weight history entry.*
- bool [deleteWeightEntry](#) (int entryId)  
*Deletes a weight history entry.*
- QList< [WeightEntry](#) > [getWeightHistory](#) (int athleteId)  
*Retrieves weight history for an athlete.*

#### Private Attributes

- QSqlDatabase & [m\\_db](#)

### 7.18.1 Detailed Description

Repository for athlete profile data persistence.  
Provides CRUD operations for athletes and their FTP/weight history.  
Definition at line 96 of file [AthleteRepository.h](#).

### 7.18.2 Constructor & Destructor Documentation

#### 7.18.2.1 AthleteRepository()

```
AthleteRepository::AthleteRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

#### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 11 of file [AthleteRepository.cpp](#).

### 7.18.3 Member Function Documentation

#### 7.18.3.1 addFtpEntry()

```
int AthleteRepository::addFtpEntry (
    const FtpEntry & e)
```

Adds an FTP history entry.

**Parameters**

|          |                   |
|----------|-------------------|
| <i>e</i> | FTP entry to add. |
|----------|-------------------|

**Returns**

ID of inserted entry, or -1 on error.

Definition at line 106 of file [AthleteRepository.cpp](#).

References [FtpEntry::athleteId](#), [FtpEntry::effectiveFrom](#), [FtpEntry::ftpWatts](#), and [FtpEntry::notes](#).

**7.18.3.2 addWeightEntry()**

```
int AthleteRepository::addWeightEntry (  
    const WeightEntry & e)
```

Adds a weight history entry.

**Parameters**

|          |                      |
|----------|----------------------|
| <i>e</i> | Weight entry to add. |
|----------|----------------------|

**Returns**

ID of inserted entry, or -1 on error.

Definition at line 167 of file [AthleteRepository.cpp](#).

References [WeightEntry::athleteId](#), [WeightEntry::effectiveFrom](#), and [WeightEntry::weightKg](#).

**7.18.3.3 deleteAthlete()**

```
bool AthleteRepository::deleteAthlete (  
    int athleteId)
```

Deletes an athlete.

**Parameters**

|                  |                             |
|------------------|-----------------------------|
| <i>athleteId</i> | <a href="#">Athlete</a> ID. |
|------------------|-----------------------------|

**Returns**

True if deletion succeeded.

Definition at line 51 of file [AthleteRepository.cpp](#).

**7.18.3.4 deleteFtpEntry()**

```
bool AthleteRepository::deleteFtpEntry (  
    int entryId)
```

Deletes an FTP history entry.

**Parameters**

|                |           |
|----------------|-----------|
| <i>entryId</i> | Entry ID. |
|----------------|-----------|

**Returns**

True if deletion succeeded.

Definition at line 120 of file [AthleteRepository.cpp](#).

**7.18.3.5 deleteWeightEntry()**

```
bool AthleteRepository::deleteWeightEntry (
    int entryId)
```

Deletes a weight history entry.

**Parameters**

|                |           |
|----------------|-----------|
| <i>entryId</i> | Entry ID. |
|----------------|-----------|

**Returns**

True if deletion succeeded.

Definition at line 180 of file [AthleteRepository.cpp](#).

**7.18.3.6 getAthlete()**

```
Athlete AthleteRepository::getAthlete (
    int athleteId)
```

Retrieves an athlete by ID.

**Parameters**

|                  |                             |
|------------------|-----------------------------|
| <i>athleteId</i> | <a href="#">Athlete</a> ID. |
|------------------|-----------------------------|

**Returns**

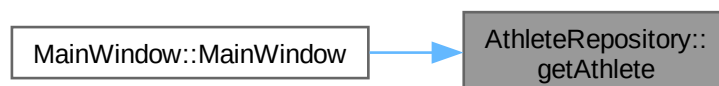
[Athlete](#) record.

Definition at line 81 of file [AthleteRepository.cpp](#).

References [Athlete::createdAt](#), [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

Referenced by [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



### 7.18.3.7 getFtpForDate()

```
int AthleteRepository::getFtpForDate (
    int athleteId,
    const QDate & date)
```

Gets FTP for an athlete on a specific date.

#### Parameters

|                  |             |
|------------------|-------------|
| <i>athleteId</i> | Athlete ID. |
| <i>date</i>      | Query date. |

#### Returns

FTP in watts, or 0 if no entry found on or before date.

Definition at line 149 of file [AthleteRepository.cpp](#).

### 7.18.3.8 getFtpHistory()

```
QList< FtpEntry > AthleteRepository::getFtpHistory (
    int athleteId)
```

Retrieves FTP history for an athlete.

#### Parameters

|                  |             |
|------------------|-------------|
| <i>athleteId</i> | Athlete ID. |
|------------------|-------------|

#### Returns

List of FTP entries.

Definition at line 128 of file [AthleteRepository.cpp](#).

References [FtpEntry::athleteId](#), [FtpEntry::effectiveFrom](#), [FtpEntry::ftpWatts](#), [FtpEntry::id](#), and [FtpEntry::notes](#).

### 7.18.3.9 getWeightHistory()

```
QList< WeightEntry > AthleteRepository::getWeightHistory (
    int athleteId)
```

Retrieves weight history for an athlete.

#### Parameters

|                  |             |
|------------------|-------------|
| <i>athleteId</i> | Athlete ID. |
|------------------|-------------|

#### Returns

Weight entries sorted by date.

Definition at line 188 of file [AthleteRepository.cpp](#).

References [WeightEntry::athleteId](#), [WeightEntry::effectiveFrom](#), [WeightEntry::id](#), and [WeightEntry::weightKg](#).

### 7.18.3.10 insertAthlete()

```
int AthleteRepository::insertAthlete (  
    const Athlete & a)
```

Inserts a new athlete.

#### Parameters

|          |                    |
|----------|--------------------|
| <i>a</i> | Athlete to insert. |
|----------|--------------------|

#### Returns

ID of inserted athlete, or -1 on error.

Definition at line 17 of file [AthleteRepository.cpp](#).

References [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), and [Athlete::lastName](#).

### 7.18.3.11 listAthletes()

```
QList< Athlete > AthleteRepository::listAthletes ()
```

Lists all athletes.

#### Returns

List of athlete records.

Definition at line 59 of file [AthleteRepository.cpp](#).

References [Athlete::createdAt](#), [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

### 7.18.3.12 updateAthlete()

```
bool AthleteRepository::updateAthlete (  
    const Athlete & a)
```

Updates an existing athlete.

#### Parameters

|          |                              |
|----------|------------------------------|
| <i>a</i> | Athlete with updated values. |
|----------|------------------------------|

#### Returns

True if update succeeded.

Definition at line 35 of file [AthleteRepository.cpp](#).

References [Athlete::dateOfBirth](#), [Athlete::email](#), [Athlete::firstName](#), [Athlete::ftpWatts](#), [Athlete::heightCm](#), [Athlete::id](#), and [Athlete::lastName](#).

## 7.18.4 Member Data Documentation

### 7.18.4.1 m\_db

```
QSqlDatabase& AthleteRepository::m_db [private]
```

Definition at line 190 of file [AthleteRepository.h](#).

The documentation for this class was generated from the following files:

- [src/database/AthleteRepository.h](#)
- [src/database/AthleteRepository.cpp](#)

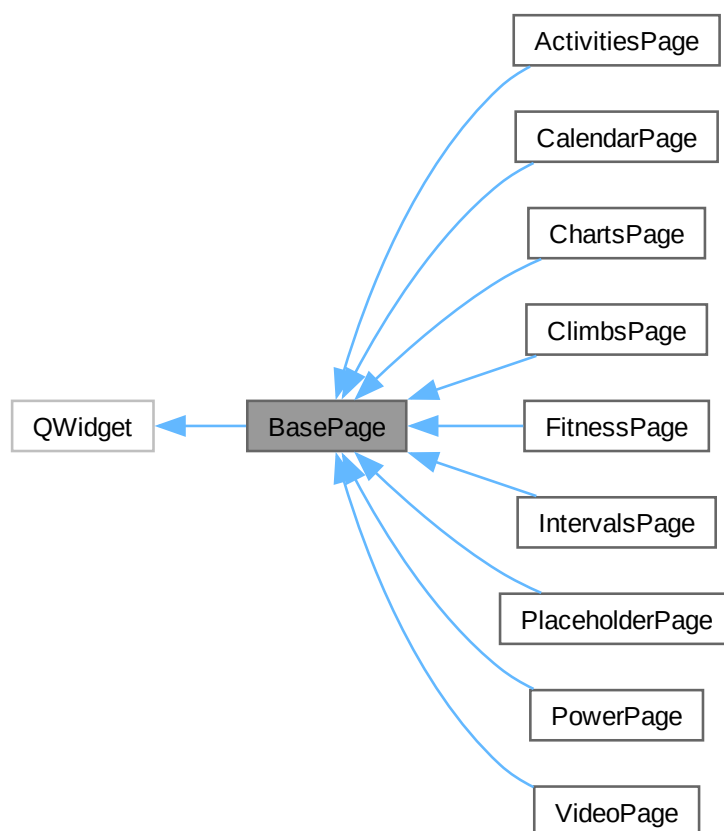


## 7.19 BasePage Class Reference

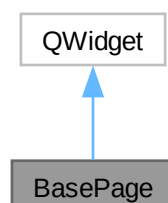
Abstract base class for all top-level page widgets.

```
#include <BasePage.h>
```

Inheritance diagram for BasePage:



Collaboration diagram for BasePage:



## Public Member Functions

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()

Called by [MainWindow](#) when this page becomes the active page.

### 7.19.1 Detailed Description

Abstract base class for all top-level page widgets.

Each page corresponds to one entry in [NavigationSidebar::Page](#) and occupies one slot in the main window's QStackedWidget (m\_pageStack).

Subclasses are responsible for building their own UI in the constructor. They may optionally override [onActivated\(\)](#) to refresh data when the user navigates to the page.

#### 7.19.1.1 Adding a new feature as a new page

1. Create MyFeaturePage : public [BasePage](#) in src/gui/pages/.
2. Add the corresponding entry to [NavigationSidebar::Page](#) (append at end).
3. Add the matching item label to [NavigationSidebar::NavigationSidebar\(\)](#).
4. Append `m_pageStack->addWidget (new MyFeaturePage (...))` in `buildUI()` at `index == static_cast<int>(Page::MyFeature)`.
5. Update [NavigationController::restoreNavigationState\(\)](#) to clamp to the new last valid page value.
6. Add the new source files to CMakeLists.txt.

[MainWindow](#) does NOT need to be modified to add new page-scoped features. Inter-page navigation (e.g. selecting an activity opens Charts) is done via [NavigationController::navigateTo\(\)](#) or [NavigationSidebar::setCurrentPage\(\)](#). Definition at line 32 of file [BasePage.h](#).

### 7.19.2 Constructor & Destructor Documentation

#### 7.19.2.1 BasePage()

```
BasePage::BasePage (
    QWidget * parent = nullptr) [inline], [explicit]
```

Definition at line 37 of file [BasePage.h](#).

### 7.19.3 Member Function Documentation

#### 7.19.3.1 onActivated()

```
virtual void BasePage::onActivated () [inline], [virtual]
```

Called by [MainWindow](#) when this page becomes the active page.

Override to refresh stale data or update UI state. The default implementation does nothing.

Definition at line 45 of file [BasePage.h](#).

The documentation for this class was generated from the following file:

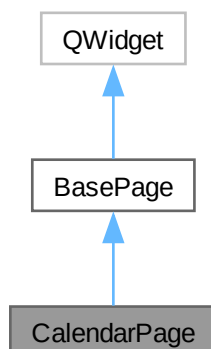
- src/gui/pages/BasePage.h

## 7.20 CalendarPage Class Reference

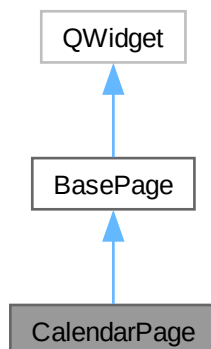
Top-level page for activity and workout planning.

```
#include <CalendarPage.h>
```

Inheritance diagram for CalendarPage:



Collaboration diagram for CalendarPage:



### Public Member Functions

- [CalendarPage](#) ([CalendarWidget](#) \*calendarWidget, [QWidget](#) \*parent=nullptr)  
*Constructs the Calendar page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) ([QWidget](#) \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

#### 7.20.1 Detailed Description

Top-level page for activity and workout planning.  
Keeps the calendar focused on planning:

- [Activity](#) calendar browsing
- Planned workout scheduling

No ride-analysis widgets appear on this page.  
Definition at line 18 of file [CalendarPage.h](#).

## 7.20.2 Constructor & Destructor Documentation

### 7.20.2.1 CalendarPage()

```
CalendarPage::CalendarPage (
    CalendarWidget * calendarWidget,
    QWidget * parent = nullptr) [explicit]
```

Constructs the Calendar page.

#### Parameters

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| <i>calendarWidget</i> | The <a href="#">CalendarWidget</a> (ownership transfers here). |
| <i>parent</i>         | Parent widget.                                                 |
| <i>calendarWidget</i> | <a href="#">CalendarWidget</a> to embed.                       |
| <i>parent</i>         | Parent widget.                                                 |

Definition at line 14 of file [CalendarPage.cpp](#).

The documentation for this class was generated from the following files:

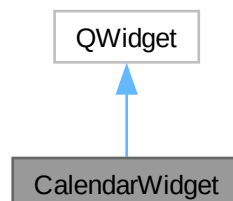
- [src/gui/pages/CalendarPage.h](#)
- [src/gui/pages/CalendarPage.cpp](#)

## 7.21 CalendarWidget Class Reference

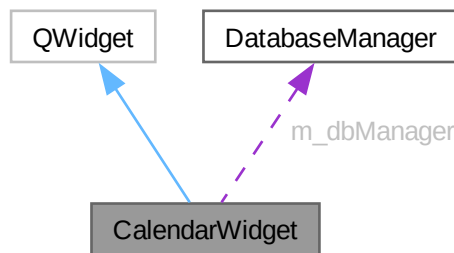
Calendar widget for activity scheduling and display.

```
#include <CalendarWidget.h>
```

Inheritance diagram for CalendarWidget:



Collaboration diagram for CalendarWidget:



### Public Member Functions

- [CalendarWidget](#) (QWidget \*parent=nullptr)
- void [setDatabaseManager](#) (DatabaseManager \*dbManager)
- void [setAthleteld](#) (int athleteId)
- void [refresh](#) ()

### Private Slots

- void [onDateChanged](#) ()
- void [addPlannedWorkout](#) ()
- void [editSelectedWorkout](#) ()
- void [deleteSelectedWorkout](#) ()
- void [recolorMonth](#) ()

### Private Member Functions

- void [reloadDayList](#) (const QDate &date)
- void [updateMonthFormatting](#) ()

### Private Attributes

- QCalendarWidget \* [m\\_calendar](#) = nullptr
- QComboBox \* [m\\_colorByCombo](#) = nullptr
- QListWidget \* [m\\_dayList](#) = nullptr
- QPushButton \* [m\\_addButton](#) = nullptr
- QPushButton \* [m\\_editButton](#) = nullptr
- QPushButton \* [m\\_deleteButton](#) = nullptr
- DatabaseManager \* [m\\_dbManager](#) = nullptr
- int [m\\_athleteId](#) = -1

#### 7.21.1 Detailed Description

Calendar widget for activity scheduling and display.

Shows activities by date with customizable coloring and planned workout support.

Definition at line 22 of file [CalendarWidget.h](#).

## 7.21.2 Constructor & Destructor Documentation

### 7.21.2.1 CalendarWidget()

```
CalendarWidget::CalendarWidget (  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 22 of file [CalendarWidget.cpp](#).

## 7.21.3 Member Function Documentation

### 7.21.3.1 addPlannedWorkout

```
void CalendarWidget::addPlannedWorkout () [private], [slot]
```

Definition at line 95 of file [CalendarWidget.cpp](#).

### 7.21.3.2 deleteSelectedWorkout

```
void CalendarWidget::deleteSelectedWorkout () [private], [slot]
```

Definition at line 185 of file [CalendarWidget.cpp](#).

### 7.21.3.3 editSelectedWorkout

```
void CalendarWidget::editSelectedWorkout () [private], [slot]
```

Definition at line 130 of file [CalendarWidget.cpp](#).

### 7.21.3.4 onDateChanged

```
void CalendarWidget::onDateChanged () [private], [slot]
```

Definition at line 88 of file [CalendarWidget.cpp](#).

### 7.21.3.5 recolorMonth

```
void CalendarWidget::recolorMonth () [private], [slot]
```

Definition at line 210 of file [CalendarWidget.cpp](#).

### 7.21.3.6 refresh()

```
void CalendarWidget::refresh ()
```

Definition at line 80 of file [CalendarWidget.cpp](#).

### 7.21.3.7 reloadDayList()

```
void CalendarWidget::reloadDayList (  
    const QDate & date) [private]
```

Definition at line 215 of file [CalendarWidget.cpp](#).

### 7.21.3.8 setAthleteId()

```
void CalendarWidget::setAthleteId (  
    int athleteId)
```

Definition at line 74 of file [CalendarWidget.cpp](#).

### 7.21.3.9 setDatabaseManager()

```
void CalendarWidget::setDatabaseManager (  
    DatabaseManager * dbManager)
```

Definition at line 68 of file [CalendarWidget.cpp](#).

### 7.21.3.10 updateMonthFormatting()

void CalendarWidget::updateMonthFormatting () [private]  
Definition at line 257 of file [CalendarWidget.cpp](#).

## 7.21.4 Member Data Documentation

### 7.21.4.1 m\_addButton

QPushButton\* CalendarWidget::m\_addButton = nullptr [private]  
Definition at line 50 of file [CalendarWidget.h](#).

### 7.21.4.2 m\_athleteId

int CalendarWidget::m\_athleteId = -1 [private]  
Definition at line 55 of file [CalendarWidget.h](#).

### 7.21.4.3 m\_calendar

QCalendarWidget\* CalendarWidget::m\_calendar = nullptr [private]  
Definition at line 47 of file [CalendarWidget.h](#).

### 7.21.4.4 m\_colorByCombo

QComboBox\* CalendarWidget::m\_colorByCombo = nullptr [private]  
Definition at line 48 of file [CalendarWidget.h](#).

### 7.21.4.5 m\_dayList

QListWidget\* CalendarWidget::m\_dayList = nullptr [private]  
Definition at line 49 of file [CalendarWidget.h](#).

### 7.21.4.6 m\_dbManager

[DatabaseManager](#)\* CalendarWidget::m\_dbManager = nullptr [private]  
Definition at line 54 of file [CalendarWidget.h](#).

### 7.21.4.7 m\_deleteButton

QPushButton\* CalendarWidget::m\_deleteButton = nullptr [private]  
Definition at line 52 of file [CalendarWidget.h](#).

### 7.21.4.8 m\_editButton

QPushButton\* CalendarWidget::m\_editButton = nullptr [private]  
Definition at line 51 of file [CalendarWidget.h](#).

The documentation for this class was generated from the following files:

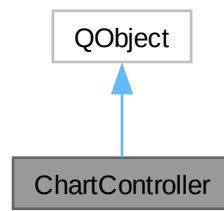
- [src/gui/CalendarWidget.h](#)
- [src/gui/CalendarWidget.cpp](#)

## 7.22 ChartController Class Reference

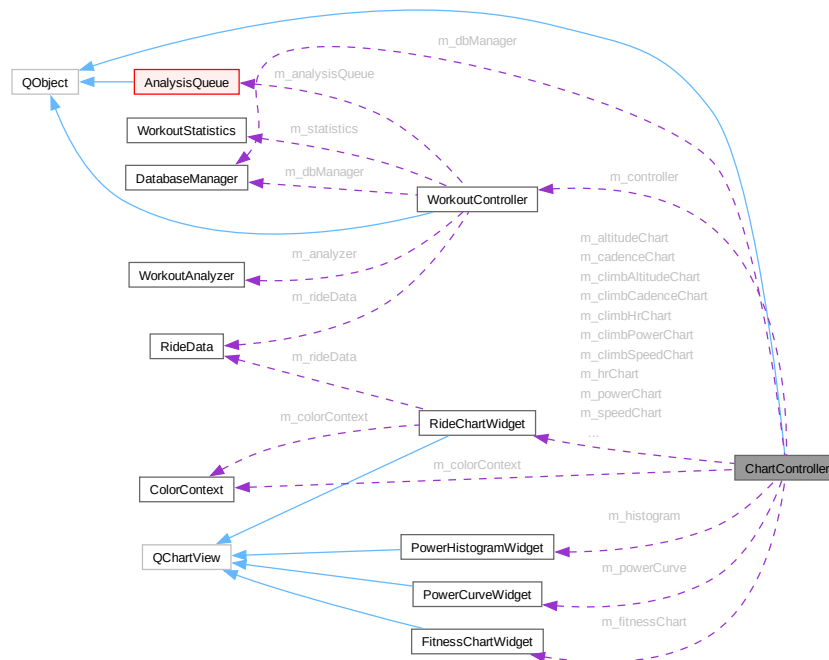
Manages all chart widgets in the analysis view.

```
#include <ChartController.h>
```

Inheritance diagram for ChartController:



Collaboration diagram for ChartController:



## Signals

- void **chartsUpdated** ()  
*Emitted when charts have been successfully updated.*
- void **errorOccurred** (const QString &message)  
*Emitted when an error occurs during chart operations.*

## Public Member Functions

- **ChartController** ([WorkoutController](#) \*controller, [DatabaseManager](#) \*dbManager, QObject \*parent=nullptr)  
*Constructs the chart controller.*
- **~ChartController** () override  
*Destructor.*



- void [setChartWidgets](#) ([RideChartWidget](#) \*powerChart, [RideChartWidget](#) \*hrChart, [RideChartWidget](#) \*cadenceChart, [RideChartWidget](#) \*speedChart, [RideChartWidget](#) \*altitudeChart, [PowerHistogramWidget](#) \*histogram, [PowerCurveWidget](#) \*powerCurve, [FitnessChartWidget](#) \*fitnessChart)  
*Sets the main ride chart widgets managed by this controller.*
- void [setAnalysisTabWidgets](#) ([QTabWidget](#) \*analysisTabWidget, [QStackedLayout](#) \*activityTabStack, [QStackedLayout](#) \*zonesTabStack, [QStackedLayout](#) \*histogramTabStack, [QStackedLayout](#) \*powerCurveTabStack, [QStackedLayout](#) \*calendarTabStack, [QStackedLayout](#) \*fitnessTabStack)  
*Sets the analysis tab container and layout widgets.*
- void [setClimbingTabStack](#) ([QStackedLayout](#) \*climbingTabStack)  
*Sets the climbing tab stacked layout.*
- void [setZonesTable](#) ([QTableWidget](#) \*zonesTable)  
*Sets the zones table widget for zone distribution display.*
- void [setClimbCharts](#) ([RideChartWidget](#) \*climbPowerChart, [RideChartWidget](#) \*climbHrChart, [RideChartWidget](#) \*climbCadenceChart, [RideChartWidget](#) \*climbSpeedChart, [RideChartWidget](#) \*climbAltitudeChart)  
*Sets the climb chart widgets.*
- void [setChartControls](#) ([QComboBox](#) \*colorMetricCombo, [QComboBox](#) \*powerSmoothingCombo, [QCheckBox](#) \*autoSmoothingCheck, [QCheckBox](#) \*climbOverlayEnabledCheck, [QComboBox](#) \*climbOverlayMetricCombo, [QLayout](#) \*colorLegendLayout)  
*Sets the chart control widgets.*
- void [setAthleteId](#) (int athleteId)  
*Sets athlete context for power curve queries.*
- void [updateCharts](#) ()  
*Updates all managed charts with current activity data.*
- void [refreshCharts](#) ()  
*Refreshes chart display without reloading data.*
- void [resetAllZoom](#) ()  
*Resets zoom on all charts to fit data.*
- void [updateHistogram](#) ()  
*Updates power histogram display.*
- void [updateFitnessChart](#) ()  
*Updates fitness chart display.*
- void [updateAnalysisEmptyStates](#) ()  
*Updates empty state indicators for all chart tabs.*
- void [updateZonesTab](#) ()  
*Updates zones table with zone distribution.*
- void [updatePowerCurve](#) ()  
*Updates power curve with historical athlete data.*
- void [updateColorLegend](#) ()  
*Updates color legend with zone colors.*
- void [applyChartSmoothing](#) ()  
*Applies chart smoothing settings to all charts.*
- void [syncChartZoom](#) ()  
*Synchronizes chart zoom levels across all ride charts.*
- void [syncChartCrosshair](#) ()  
*Synchronizes crosshair position across all ride charts.*
- void [setIntervals](#) (const std::vector< [Interval](#) > &intervals)  
*Sets intervals for display on power chart.*
- void [setClimbs](#) (const std::vector< [Climb](#) > &climbs)  
*Sets climbs for display on charts.*
- void [setColorMetric](#) (int colorMetric)  
*Sets the color metric for chart visualization.*
- void [setColorContext](#) (const [ColorContext](#) &colorContext)

*Sets the color context for chart rendering.*

- void [updateZoneAvailability](#) ()

*Updates zone availability display.*

- void [handleWorkoutLoaded](#) ()

*Runs full chart update sequence after a workout is loaded.*

### Private Member Functions

- void [connectChartSignals](#) ()

*Helper to connect chart widget signals for synchronization.*

- void [applyColorMetric](#) ()

*Helper to apply current color metric to charts.*

- void [updateColorLegendDisplay](#) ()

*Helper to update color legend display.*

- int [currentColorMetric](#) () const

*Retrieves current color metric from stored value.*

### Private Attributes

- [WorkoutController](#) \* [m\\_controller](#)
- [DatabaseManager](#) \* [m\\_dbManager](#)
- [RideChartWidget](#) \* [m\\_powerChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_hrChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_cadenceChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_speedChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_altitudeChart](#) = nullptr
- [PowerHistogramWidget](#) \* [m\\_histogram](#) = nullptr
- [PowerCurveWidget](#) \* [m\\_powerCurve](#) = nullptr
- [FitnessChartWidget](#) \* [m\\_fitnessChart](#) = nullptr
- [QTabWidget](#) \* [m\\_analysisTabWidget](#) = nullptr
- [QStackedLayout](#) \* [m\\_activityTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_zonesTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_histogramTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_powerCurveTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_calendarTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_fitnessTabStack](#) = nullptr
- [QStackedLayout](#) \* [m\\_climbingTabStack](#) = nullptr
- [std::vector](#)< [Interval](#) > [m\\_intervals](#)
- [std::vector](#)< [Climb](#) > [m\\_climbs](#)
- int [m\\_colorMetric](#) = 0
- [ColorContext](#) [m\\_colorContext](#)
- bool [m\\_smoothingEnabled](#) = false
- int [m\\_smoothingAmount](#) = 0
- [RideChartWidget](#) \* [m\\_climbPowerChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_climbHrChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_climbCadenceChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_climbSpeedChart](#) = nullptr
- [RideChartWidget](#) \* [m\\_climbAltitudeChart](#) = nullptr
- [QTableWidget](#) \* [m\\_zonesTable](#) = nullptr
- [QComboBox](#) \* [m\\_colorMetricCombo](#) = nullptr
- [QComboBox](#) \* [m\\_powerSmoothingCombo](#) = nullptr
- [QCheckBox](#) \* [m\\_autoSmoothingCheck](#) = nullptr
- [QCheckBox](#) \* [m\\_climbOverlayEnabledCheck](#) = nullptr
- [QComboBox](#) \* [m\\_climbOverlayMetricCombo](#) = nullptr
- [QLayout](#) \* [m\\_colorLegendLayout](#) = nullptr
- int [m\\_currentAthletelId](#) = -1

### 7.22.1 Detailed Description

Manages all chart widgets in the analysis view.  
Definition at line 31 of file [ChartController.h](#).

### 7.22.2 Constructor & Destructor Documentation

#### 7.22.2.1 ChartController()

```
ChartController::ChartController (
    WorkoutController * controller,
    DatabaseManager * dbManager,
    QObject * parent = nullptr) [explicit]
```

Constructs the chart controller.

#### Parameters

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <i>controller</i> | Pointer to <a href="#">WorkoutController</a> for data access. |
| <i>dbManager</i>  | Pointer to <a href="#">DatabaseManager</a> for queries.       |
| <i>parent</i>     | Parent object.                                                |

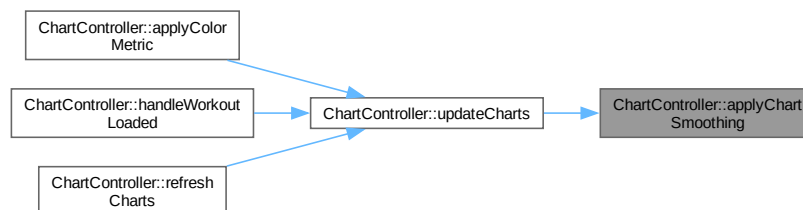
Definition at line 68 of file [ChartController.cpp](#).

### 7.22.3 Member Function Documentation

#### 7.22.3.1 applyChartSmoothing()

```
void ChartController::applyChartSmoothing ()
```

Applies chart smoothing settings to all charts.  
Helper to apply smoothing settings to all charts.  
Definition at line 711 of file [ChartController.cpp](#).  
Referenced by [updateCharts\(\)](#).  
Here is the caller graph for this function:

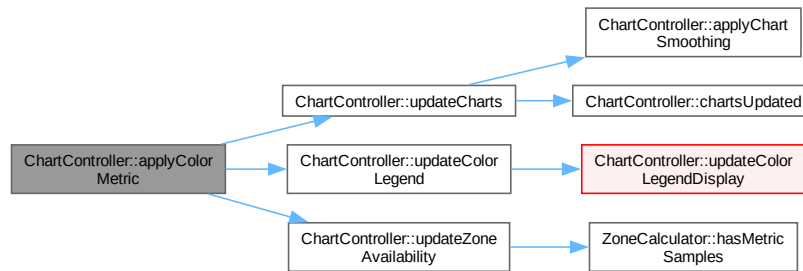


#### 7.22.3.2 applyColorMetric()

```
void ChartController::applyColorMetric () [private]
```

Helper to apply current color metric to charts.  
Definition at line 573 of file [ChartController.cpp](#).  
References [updateCharts\(\)](#), [updateColorLegend\(\)](#), and [updateZoneAvailability\(\)](#).

Here is the call graph for this function:



### 7.22.3.3 connectChartSignals()

```
void ChartController::connectChartSignals () [private]
```

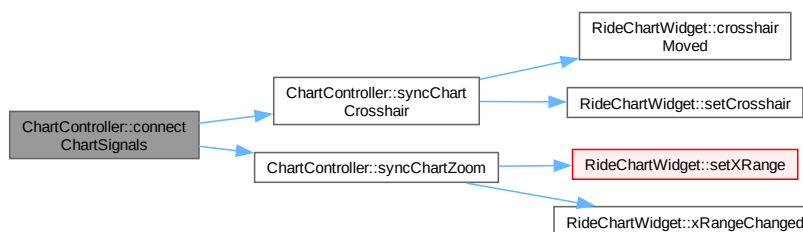
Helper to connect chart widget signals for synchronization.

Definition at line 429 of file [ChartController.cpp](#).

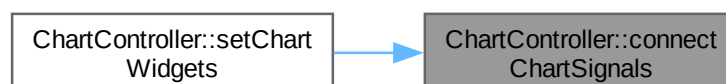
References [syncChartCrosshair\(\)](#), and [syncChartZoom\(\)](#).

Referenced by [setChartWidgets\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.4 currentColorMetric()

```
int ChartController::currentColorMetric () const [private]
```

Retrieves current color metric from stored value.

Definition at line 487 of file [ChartController.cpp](#).

### 7.22.3.5 handleWorkoutLoaded()

```
void ChartController::handleWorkoutLoaded ()
```

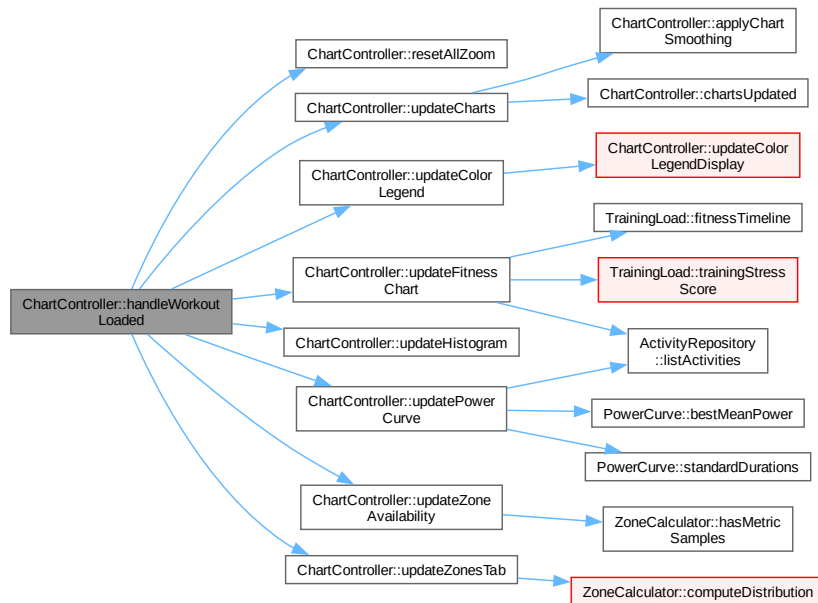
Runs full chart update sequence after a workout is loaded.

Runs full chart update sequence after workout load.

Definition at line 551 of file [ChartController.cpp](#).

References [resetAllZoom\(\)](#), [updateCharts\(\)](#), [updateColorLegend\(\)](#), [updateFitnessChart\(\)](#), [updateHistogram\(\)](#), [updatePowerCurve\(\)](#), [updateZoneAvailability\(\)](#), and [updateZonesTab\(\)](#).

Here is the call graph for this function:



### 7.22.3.6 refreshCharts()

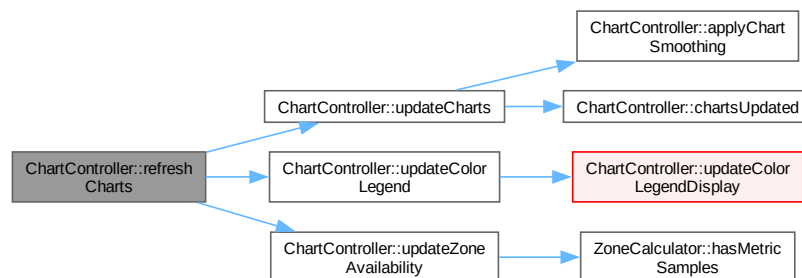
```
void ChartController::refreshCharts ()
```

Refreshes chart display without reloading data.

Definition at line 262 of file [ChartController.cpp](#).

References [updateCharts\(\)](#), [updateColorLegend\(\)](#), and [updateZoneAvailability\(\)](#).

Here is the call graph for this function:



### 7.22.3.7 resetAllZoom()

```
void ChartController::resetAllZoom ()
```

Resets zoom on all charts to fit data.

Definition at line 272 of file [ChartController.cpp](#).

Referenced by [handleWorkoutLoaded\(\)](#).

Here is the caller graph for this function:



### 7.22.3.8 setAnalysisTabWidgets()

```
void ChartController::setAnalysisTabWidgets (
    QTabWidget * analysisTabWidget,
    QStackedLayout * activityTabStack,
    QStackedLayout * zonesTabStack,
    QStackedLayout * histogramTabStack,
    QStackedLayout * powerCurveTabStack,
    QStackedLayout * calendarTabStack,
    QStackedLayout * fitnessTabStack)
```

Sets the analysis tab container and layout widgets.

Sets the analysis tab-related widgets.

Definition at line 111 of file [ChartController.cpp](#).

### 7.22.3.9 setAthleteId()

```
void ChartController::setAthleteId (
    int athleteId)
```

Sets athlete context for power curve queries.

Definition at line 184 of file [ChartController.cpp](#).

### 7.22.3.10 setChartControls()

```
void ChartController::setChartControls (
    QComboBox * colorMetricCombo,
    QComboBox * powerSmoothingCombo,
    QCheckBox * autoSmoothingCheck,
    QCheckBox * climbOverlayEnabledCheck,
    QComboBox * climbOverlayMetricCombo,
    QLayout * colorLegendLayout)
```

Sets the chart control widgets.

Definition at line 165 of file [ChartController.cpp](#).

### 7.22.3.11 setChartWidgets()

```
void ChartController::setChartWidgets (
    RideChartWidget * powerChart,
    RideChartWidget * hrChart,
    RideChartWidget * cadenceChart,
    RideChartWidget * speedChart,
    RideChartWidget * altitudeChart,
```

```

    PowerHistogramWidget * histogram,
    PowerCurveWidget * powerCurve,
    FitnessChartWidget * fitnessChart)

```

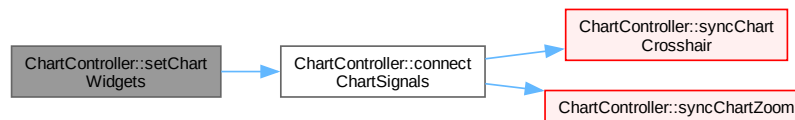
Sets the main ride chart widgets managed by this controller.

Sets the chart widgets managed by this controller.

Definition at line 86 of file [ChartController.cpp](#).

References [connectChartSignals\(\)](#).

Here is the call graph for this function:



### 7.22.3.12 setClimbCharts()

```

void ChartController::setClimbCharts (
    RideChartWidget * climbPowerChart,
    RideChartWidget * climbHrChart,
    RideChartWidget * climbCadenceChart,
    RideChartWidget * climbSpeedChart,
    RideChartWidget * climbAltitudeChart)

```

Sets the climb chart widgets.

Definition at line 148 of file [ChartController.cpp](#).

### 7.22.3.13 setClimbingTabStack()

```

void ChartController::setClimbingTabStack (
    QStackedLayout * climbingTabStack)

```

Sets the climbing tab stacked layout.

Sets climbing tab stacked layout.

Definition at line 132 of file [ChartController.cpp](#).

### 7.22.3.14 setClimbs()

```

void ChartController::setClimbs (
    const std::vector< Climb > & climbs)

```

Sets climbs for display on charts.

#### Parameters

|               |                            |
|---------------|----------------------------|
| <i>climbs</i> | List of climbs to display. |
|---------------|----------------------------|

Definition at line 503 of file [ChartController.cpp](#).

### 7.22.3.15 setColorContext()

```

void ChartController::setColorContext (
    const ColorContext & colorContext)

```

Sets the color context for chart rendering.

**Parameters**

|                     |                          |
|---------------------|--------------------------|
| <i>colorContext</i> | Color context structure. |
|---------------------|--------------------------|

Definition at line 519 of file [ChartController.cpp](#).

**7.22.3.16 setColorMetric()**

```
void ChartController::setColorMetric (
    int colorMetric)
```

Sets the color metric for chart visualization.

**Parameters**

|                    |                                 |
|--------------------|---------------------------------|
| <i>colorMetric</i> | Color metric enumeration value. |
|--------------------|---------------------------------|

Definition at line 511 of file [ChartController.cpp](#).

**7.22.3.17 setIntervals()**

```
void ChartController::setIntervals (
    const std::vector< Interval > & intervals)
```

Sets intervals for display on power chart.

**Parameters**

|                  |                               |
|------------------|-------------------------------|
| <i>intervals</i> | List of intervals to display. |
|------------------|-------------------------------|

Definition at line 495 of file [ChartController.cpp](#).

**7.22.3.18 setZonesTable()**

```
void ChartController::setZonesTable (
    QTableWidgetItem * zonesTable)
```

Sets the zones table widget for zone distribution display.

Definition at line 140 of file [ChartController.cpp](#).

**7.22.3.19 syncChartCrosshair()**

```
void ChartController::syncChartCrosshair ()
```

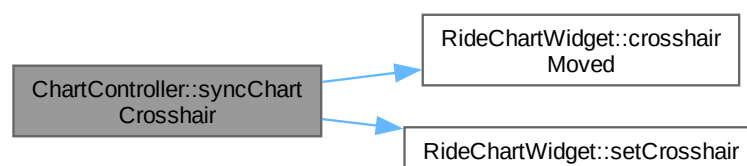
Synchronizes crosshair position across all ride charts.

Definition at line 406 of file [ChartController.cpp](#).

References [RideChartWidget::crosshairMoved\(\)](#), and [RideChartWidget::setCrosshair\(\)](#).

Referenced by [connectChartSignals\(\)](#).

Here is the call graph for this function:





Here is the caller graph for this function:



### 7.22.3.20 syncChartZoom()

```
void ChartController::syncChartZoom ()
```

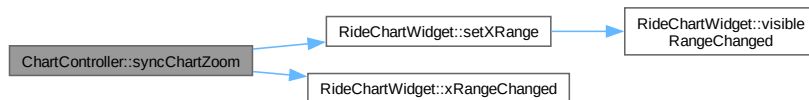
Synchronizes chart zoom levels across all ride charts.

Definition at line 383 of file [ChartController.cpp](#).

References [RideChartWidget::setXRange\(\)](#), and [RideChartWidget::xRangeChanged\(\)](#).

Referenced by [connectChartSignals\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.21 updateAnalysisEmptyStates()

```
void ChartController::updateAnalysisEmptyStates ()
```

Updates empty state indicators for all chart tabs.

Definition at line 356 of file [ChartController.cpp](#).

### 7.22.3.22 updateCharts()

```
void ChartController::updateCharts ()
```

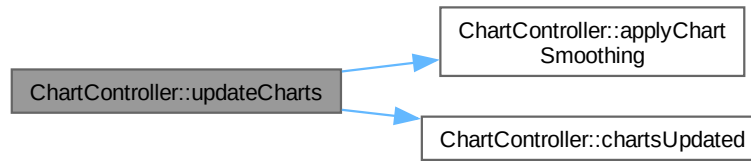
Updates all managed charts with current activity data.

Definition at line 192 of file [ChartController.cpp](#).

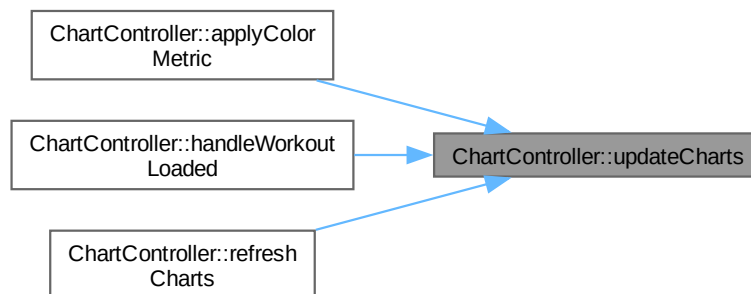
References [applyChartSmoothing\(\)](#), and [chartsUpdated\(\)](#).

Referenced by [applyColorMetric\(\)](#), [handleWorkoutLoaded\(\)](#), and [refreshCharts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.23 updateColorLegend()

```
void ChartController::updateColorLegend ()
```

Updates color legend with zone colors.

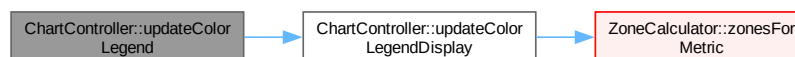
Updates color legend widget with current zone colors.

Definition at line 703 of file [ChartController.cpp](#).

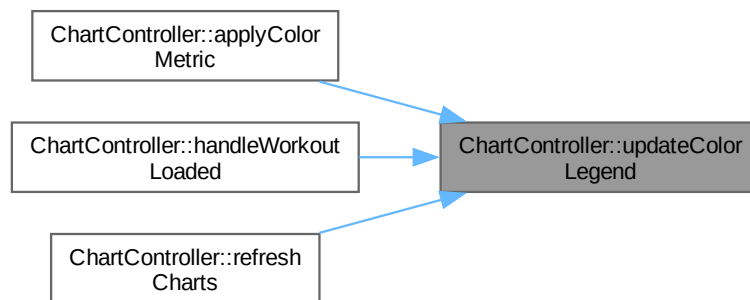
References [updateColorLegendDisplay\(\)](#).

Referenced by [applyColorMetric\(\)](#), [handleWorkoutLoaded\(\)](#), and [refreshCharts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.24 updateColorLegendDisplay()

```
void ChartController::updateColorLegendDisplay () [private]
```

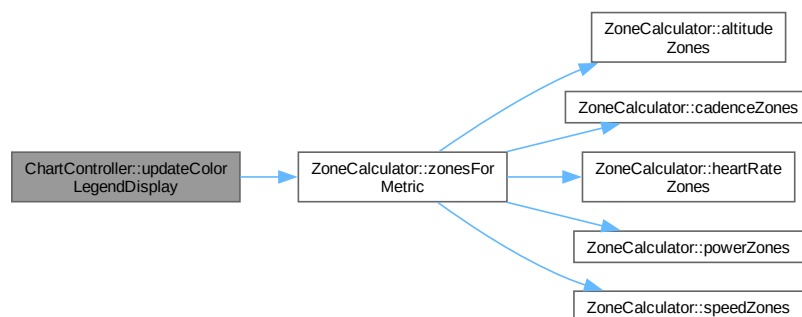
Helper to update color legend display.

Definition at line 438 of file [ChartController.cpp](#).

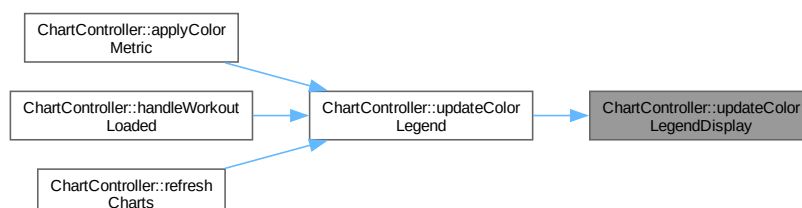
References [ZoneCalculator::zonesForMetric\(\)](#).

Referenced by [updateColorLegend\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.25 updateFitnessChart()

```
void ChartController::updateFitnessChart ()
```

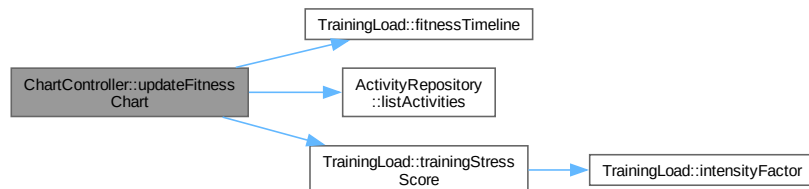
Updates fitness chart display.

Definition at line 295 of file [ChartController.cpp](#).

References [TrainingLoad::fitnessTimeline\(\)](#), [ActivityRepository::listActivities\(\)](#), [TrainingLoad::trainingStressScore\(\)](#), and [DailyLoadPoint::tss](#).

Referenced by [handleWorkoutLoaded\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.26 updateHistogram()

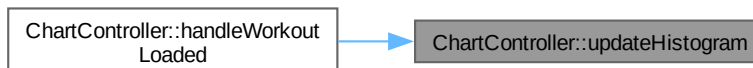
```
void ChartController::updateHistogram ()
```

Updates power histogram display.

Definition at line 284 of file [ChartController.cpp](#).

Referenced by [handleWorkoutLoaded\(\)](#).

Here is the caller graph for this function:



### 7.22.3.27 updatePowerCurve()

```
void ChartController::updatePowerCurve ()
```

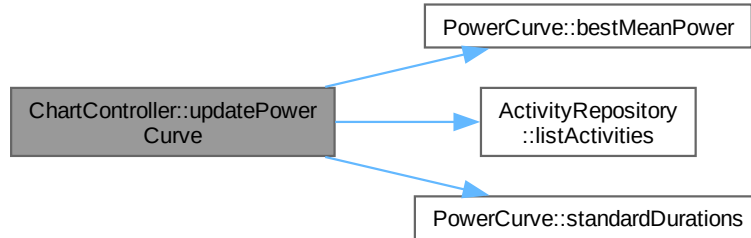
Updates power curve with historical athlete data.

Updates power curve display with athlete's historical power data.

Definition at line 627 of file [ChartController.cpp](#).

References [PowerCurve::bestMeanPower\(\)](#), [ActivityRepository::listActivities\(\)](#), [RideData::records](#), and [PowerCurve::standardDuration](#).  
Referenced by [handleWorkoutLoaded\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.28 updateZoneAvailability()

```
void ChartController::updateZoneAvailability ()
```

Updates zone availability display.

Updates zone/power sub-tab availability based on loaded activity data.

`m_analysisTabWidget` is now the Power page's sub-tab widget with three tabs: [0] Zones, [1] Histogram, [2] Power Curve. Calendar and Fitness are top-level pages and are always accessible.

Definition at line 531 of file [ChartController.cpp](#).

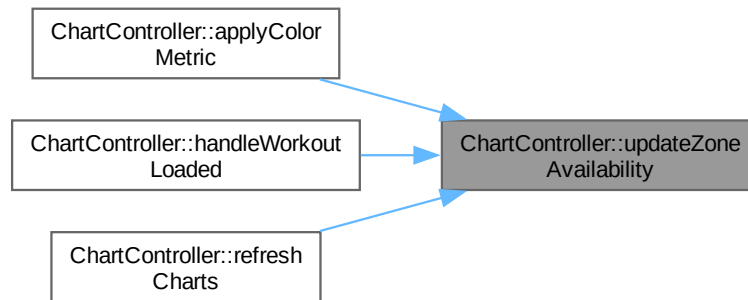
References [ZoneCalculator::hasMetricSamples\(\)](#).

Referenced by [applyColorMetric\(\)](#), [handleWorkoutLoaded\(\)](#), and [refreshCharts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.22.3.29 updateZonesTab()

```
void ChartController::updateZonesTab ()
```

Updates zones table with zone distribution.

Updates zones table with zone distribution for current activity.

Definition at line 583 of file [ChartController.cpp](#).

References [Zone::color](#), [ZoneCalculator::computeDistribution\(\)](#), [Zone::name](#), [ZoneDistribution::percent](#), [ZoneDistribution::seconds](#), and [ZoneDistribution::zone](#).

Referenced by [handleWorkoutLoaded\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 7.22.4 Member Data Documentation

### 7.22.4.1 m\_activityTabStack

```
QStackedLayout* ChartController::m_activityTabStack = nullptr [private]
```

Definition at line 233 of file [ChartController.h](#).

#### 7.22.4.2 m\_altitudeChart

`RideChartWidget* ChartController::m_altitudeChart = nullptr [private]`  
Definition at line 226 of file [ChartController.h](#).

#### 7.22.4.3 m\_analysisTabWidget

`QTabWidget* ChartController::m_analysisTabWidget = nullptr [private]`  
Definition at line 232 of file [ChartController.h](#).

#### 7.22.4.4 m\_autoSmoothingCheck

`QCheckBox* ChartController::m_autoSmoothingCheck = nullptr [private]`  
Definition at line 260 of file [ChartController.h](#).

#### 7.22.4.5 m\_cadenceChart

`RideChartWidget* ChartController::m_cadenceChart = nullptr [private]`  
Definition at line 224 of file [ChartController.h](#).

#### 7.22.4.6 m\_calendarTabStack

`QStackedLayout* ChartController::m_calendarTabStack = nullptr [private]`  
Definition at line 237 of file [ChartController.h](#).

#### 7.22.4.7 m\_climbAltitudeChart

`RideChartWidget* ChartController::m_climbAltitudeChart = nullptr [private]`  
Definition at line 254 of file [ChartController.h](#).

#### 7.22.4.8 m\_climbCadenceChart

`RideChartWidget* ChartController::m_climbCadenceChart = nullptr [private]`  
Definition at line 252 of file [ChartController.h](#).

#### 7.22.4.9 m\_climbHrChart

`RideChartWidget* ChartController::m_climbHrChart = nullptr [private]`  
Definition at line 251 of file [ChartController.h](#).

#### 7.22.4.10 m\_climbingTabStack

`QStackedLayout* ChartController::m_climbingTabStack = nullptr [private]`  
Definition at line 239 of file [ChartController.h](#).

#### 7.22.4.11 m\_climbOverlayEnabledCheck

`QCheckBox* ChartController::m_climbOverlayEnabledCheck = nullptr [private]`  
Definition at line 261 of file [ChartController.h](#).

#### 7.22.4.12 m\_climbOverlayMetricCombo

`QComboBox* ChartController::m_climbOverlayMetricCombo = nullptr [private]`  
Definition at line 262 of file [ChartController.h](#).

#### 7.22.4.13 m\_climbPowerChart

`RideChartWidget* ChartController::m_climbPowerChart = nullptr [private]`  
Definition at line 250 of file [ChartController.h](#).

#### 7.22.4.14 m\_climbs

`std::vector<Climb> ChartController::m_climbs [private]`  
Definition at line 243 of file [ChartController.h](#).

#### 7.22.4.15 m\_climbSpeedChart

`RideChartWidget* ChartController::m_climbSpeedChart = nullptr [private]`  
Definition at line 253 of file [ChartController.h](#).

#### 7.22.4.16 m\_colorContext

`ColorContext ChartController::m_colorContext [private]`  
Definition at line 245 of file [ChartController.h](#).

#### 7.22.4.17 m\_colorLegendLayout

`QLayout* ChartController::m_colorLegendLayout = nullptr [private]`  
Definition at line 263 of file [ChartController.h](#).

#### 7.22.4.18 m\_colorMetric

`int ChartController::m_colorMetric = 0 [private]`  
Definition at line 244 of file [ChartController.h](#).

#### 7.22.4.19 m\_colorMetricCombo

`QComboBox* ChartController::m_colorMetricCombo = nullptr [private]`  
Definition at line 258 of file [ChartController.h](#).

#### 7.22.4.20 m\_controller

`WorkoutController* ChartController::m_controller [private]`  
Definition at line 218 of file [ChartController.h](#).

#### 7.22.4.21 m\_currentAthleteId

`int ChartController::m_currentAthleteId = -1 [private]`  
Definition at line 266 of file [ChartController.h](#).

#### 7.22.4.22 m\_dbManager

`DatabaseManager* ChartController::m_dbManager [private]`  
Definition at line 219 of file [ChartController.h](#).

#### 7.22.4.23 m\_fitnessChart

`FitnessChartWidget* ChartController::m_fitnessChart = nullptr [private]`  
Definition at line 229 of file [ChartController.h](#).

#### 7.22.4.24 m\_fitnessTabStack

`QStackedLayout* ChartController::m_fitnessTabStack = nullptr [private]`  
Definition at line 238 of file [ChartController.h](#).

#### 7.22.4.25 m\_histogram

`PowerHistogramWidget* ChartController::m_histogram = nullptr [private]`  
Definition at line 227 of file [ChartController.h](#).



#### 7.22.4.26 m\_histogramTabStack

`QStackedLayout* ChartController::m_histogramTabStack = nullptr [private]`  
Definition at line 235 of file [ChartController.h](#).

#### 7.22.4.27 m\_hrChart

`RideChartWidget* ChartController::m_hrChart = nullptr [private]`  
Definition at line 223 of file [ChartController.h](#).

#### 7.22.4.28 m\_intervals

`std::vector<Interval> ChartController::m_intervals [private]`  
Definition at line 242 of file [ChartController.h](#).

#### 7.22.4.29 m\_powerChart

`RideChartWidget* ChartController::m_powerChart = nullptr [private]`  
Definition at line 222 of file [ChartController.h](#).

#### 7.22.4.30 m\_powerCurve

`PowerCurveWidget* ChartController::m_powerCurve = nullptr [private]`  
Definition at line 228 of file [ChartController.h](#).

#### 7.22.4.31 m\_powerCurveTabStack

`QStackedLayout* ChartController::m_powerCurveTabStack = nullptr [private]`  
Definition at line 236 of file [ChartController.h](#).

#### 7.22.4.32 m\_powerSmoothingCombo

`QComboBox* ChartController::m_powerSmoothingCombo = nullptr [private]`  
Definition at line 259 of file [ChartController.h](#).

#### 7.22.4.33 m\_smoothingAmount

`int ChartController::m_smoothingAmount = 0 [private]`  
Definition at line 247 of file [ChartController.h](#).

#### 7.22.4.34 m\_smoothingEnabled

`bool ChartController::m_smoothingEnabled = false [private]`  
Definition at line 246 of file [ChartController.h](#).

#### 7.22.4.35 m\_speedChart

`RideChartWidget* ChartController::m_speedChart = nullptr [private]`  
Definition at line 225 of file [ChartController.h](#).

#### 7.22.4.36 m\_zonesTable

`QTableWidget* ChartController::m_zonesTable = nullptr [private]`  
Definition at line 255 of file [ChartController.h](#).

#### 7.22.4.37 m\_zonesTabStack

`QStackedLayout* ChartController::m_zonesTabStack = nullptr [private]`  
Definition at line 234 of file [ChartController.h](#).

The documentation for this class was generated from the following files:

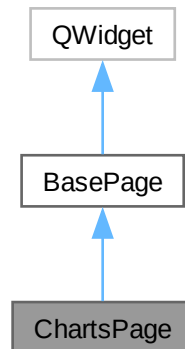
- `src/gui/controllers/ChartController.h`
- `src/gui/controllers/ChartController.cpp`

## 7.23 ChartsPage Class Reference

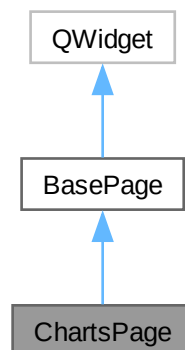
Top-level page for activity chart inspection.

```
#include <ChartsPage.h>
```

Inheritance diagram for ChartsPage:



Collaboration diagram for ChartsPage:



### Public Member Functions

- [ChartsPage](#) (`QWidget *colorBarWidget`, `QWidget *contentWidget`, `QWidget *parent=nullptr`)  
*Constructs the Charts page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (`QWidget *parent=nullptr`)

- virtual void [onActivated](#) ()

Called by [MainWindow](#) when this page becomes the active page.

### 7.23.1 Detailed Description

Top-level page for activity chart inspection.

Hosts the ride charts (Power / HR / Cadence / Speed / Altitude), the route map, and the intervals / laps / notes panels. Map controls are kept isolated from chart controls on this page.

Chart controls (metrics, smoothing, presets, zoom) live in the toolbar area at the top of this page. Map controls (style, fit-to-track, zoom) are embedded in the map panel itself.

Definition at line 20 of file [ChartsPage.h](#).

### 7.23.2 Constructor & Destructor Documentation

#### 7.23.2.1 ChartsPage()

```
ChartsPage::ChartsPage (
    QWidget * colorBarWidget,
    QWidget * contentWidget,
    QWidget * parent = nullptr) [explicit]
```

Constructs the Charts page.

#### Parameters

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| <i>colorBarWidget</i> | Widget containing the "Color By" metric control and the color-legend strip. |
| <i>contentWidget</i>  | Pre-assembled chart+map+intervals widget.                                   |
| <i>parent</i>         | Parent widget.                                                              |

Lays out the color-by control bar above the main chart+map+intervals content area.

#### Parameters

|                       |                                                                                    |
|-----------------------|------------------------------------------------------------------------------------|
| <i>colorBarWidget</i> | Widget containing the "Color By" metric selector and the color-legend strip.       |
| <i>contentWidget</i>  | Pre-assembled splitter widget with charts, map, intervals, laps, and notes panels. |
| <i>parent</i>         | Parent widget.                                                                     |

Definition at line 19 of file [ChartsPage.cpp](#).

The documentation for this class was generated from the following files:

- [src/gui/pages/ChartsPage.h](#)
- [src/gui/pages/ChartsPage.cpp](#)

## 7.24 Climb Struct Reference

Represents a detected climb with comprehensive metrics.

```
#include <ClimbMetrics.h>
```

#### Public Attributes

- int [id](#) = -1  
*Unique climb identifier (database row id).*
- double [startSeconds](#) = 0.0  
*Climb start time in seconds from activity start.*
- double [endSeconds](#) = 0.0  
*Climb end time in seconds from activity start.*

- double `startDistanceKm` = 0.0  
*Climb start distance in km from activity start.*
- double `endDistanceKm` = 0.0  
*Climb end distance in km from activity start.*
- double `durationSeconds` = 0.0  
*Climb duration in seconds.*
- double `lengthKm` = 0.0  
*Climb length in km.*
- double `elevationGainM` = 0.0  
*Total elevation gain in meters.*
- double `startElevationM` = 0.0  
*Starting elevation in meters.*
- double `endElevationM` = 0.0  
*Ending elevation in meters.*
- double `averageGradient` = 0.0  
*Average gradient percentage.*
- double `maximumGradient` = 0.0  
*Maximum gradient percentage.*
- double `climbScore` = 0.0  
*Composite climb score.*
- double `gradePercentile` = 0.0  
*Percentile of climb with grade above threshold.*
- ClimbCategory `climbCategory` = ClimbCategory::FalsePositive  
*Climb category (Major, Medium, Punchy, FalsePositive).*
- double `averagePower` = 0.0  
*Average power in watts.*
- double `averageWattsPerKg` = 0.0  
*Average power to weight in watts/kg.*
- double `normalizedPower` = 0.0  
*Normalized power in watts.*
- double `averageHeartRate` = 0.0  
*Average heart rate in bpm.*
- double `averageCadence` = 0.0  
*Average cadence in rpm.*
- double `averageSpeed` = 0.0  
*Average speed in km/h.*
- double `vam` = 0.0  
*VAM (Vertical Ascent Meter) in m/h.*
- double `powerFirstHalf` = 0.0  
*Average power in first half.*
- double `powerSecondHalf` = 0.0  
*Average power in second half.*
- double `powerFadePct` = 0.0  
*Power fade percentage from first to second half.*
- double `aerobicDecouplingPct` = 0.0  
*Aerobic decoupling percentage.*
- double `hrDriftPct` = 0.0  
*Heart rate drift percentage.*
- double `difficultyScore` = 0.0  
*Overall difficulty score.*
- std::array< ClimbQuarterMetrics, 4 > `quarterMetrics` {}

*Metrics for each quarter of climb.*

- QString [category](#)  
*Category description string.*
- QString [shapeClass](#)  
*Shape classification (steep, ramp, var, etc.).*
- QString [wattsPerKgRank](#)  
*Watts/kg performance ranking.*
- QString [name](#)  
*Climb name.*

### 7.24.1 Detailed Description

Represents a detected climb with comprehensive metrics.

Contains temporal, spatial, effort, and performance data for a single detected climb.

Definition at line 44 of file [ClimbMetrics.h](#).

### 7.24.2 Member Data Documentation

#### 7.24.2.1 aerobicDecouplingPct

```
double Climb::aerobicDecouplingPct = 0.0
```

Aerobic decoupling percentage.

Definition at line 122 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.2 averageCadence

```
double Climb::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 104 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.3 averageGradient

```
double Climb::averageGradient = 0.0
```

Average gradient percentage.

Definition at line 77 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

#### 7.24.2.4 averageHeartRate

```
double Climb::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 101 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.5 averagePower

```
double Climb::averagePower = 0.0
```

Average power in watts.

Definition at line 92 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.6 averageSpeed

```
double Climb::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 107 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.7 averageWattsPerKg

```
double Climb::averageWattsPerKg = 0.0
```

Average power to weight in watts/kg.

Definition at line 95 of file [ClimbMetrics.h](#).

#### 7.24.2.8 category

```
QString Climb::category
```

Category description string.

Definition at line 134 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.9 climbCategory

```
ClimbCategory Climb::climbCategory = ClimbCategory::FalsePositive
```

[Climb](#) category (Major, Medium, Punchy, FalsePositive).

Definition at line 89 of file [ClimbMetrics.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.24.2.10 climbScore

```
double Climb::climbScore = 0.0
```

Composite climb score.

Definition at line 83 of file [ClimbMetrics.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.24.2.11 difficultyScore

```
double Climb::difficultyScore = 0.0
```

Overall difficulty score.

Definition at line 128 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.12 durationSeconds

```
double Climb::durationSeconds = 0.0
```

[Climb](#) duration in seconds.

Definition at line 62 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.13 elevationGainM

```
double Climb::elevationGainM = 0.0
```

Total elevation gain in meters.

Definition at line 68 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

#### 7.24.2.14 endDistanceKm

```
double Climb::endDistanceKm = 0.0
```

[Climb](#) end distance in km from activity start.

Definition at line 59 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.15 endElevationM

```
double Climb::endElevationM = 0.0
```

Ending elevation in meters.

Definition at line 74 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.16 endSeconds

```
double Climb::endSeconds = 0.0
```

[Climb](#) end time in seconds from activity start.

Definition at line 53 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::drawForeground\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [RideChartWidget::mousePressEvent\(\)](#), and [RideChartWidget::mouseReleaseEvent\(\)](#).

#### 7.24.2.17 gradePercentile

```
double Climb::gradePercentile = 0.0
```

Percentile of climb with grade above threshold.

Definition at line 86 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

#### 7.24.2.18 hrDriftPct

```
double Climb::hrDriftPct = 0.0
```

Heart rate drift percentage.

Definition at line 125 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.19 id

```
int Climb::id = -1
```

Unique climb identifier (database row id).

Definition at line 47 of file [ClimbMetrics.h](#).

#### 7.24.2.20 lengthKm

```
double Climb::lengthKm = 0.0
```

[Climb](#) length in km.

Definition at line 65 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

#### 7.24.2.21 maximumGradient

```
double Climb::maximumGradient = 0.0
```

Maximum gradient percentage.

Definition at line 80 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

#### 7.24.2.22 name

```
QString Climb::name
```

[Climb](#) name.

Definition at line 143 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.23 normalizedPower

```
double Climb::normalizedPower = 0.0
```

Normalized power in watts.

Definition at line 98 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.24 powerFadePct

```
double Climb::powerFadePct = 0.0
```

Power fade percentage from first to second half.

Definition at line 119 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.25 powerFirstHalf

```
double Climb::powerFirstHalf = 0.0
```

Average power in first half.

Definition at line 113 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.26 powerSecondHalf

```
double Climb::powerSecondHalf = 0.0
```

Average power in second half.

Definition at line 116 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.27 quarterMetrics

```
std::array<ClimbQuarterMetrics, 4> Climb::quarterMetrics {}
```

Metrics for each quarter of climb.

Definition at line 131 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.28 shapeClass

```
QString Climb::shapeClass
```

Shape classification (steep, ramp, var, etc.).

Definition at line 137 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.29 startDistanceKm

```
double Climb::startDistanceKm = 0.0
```

[Climb](#) start distance in km from activity start.

Definition at line 56 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.30 startElevationM

```
double Climb::startElevationM = 0.0
```

Starting elevation in meters.

Definition at line 71 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#).

#### 7.24.2.31 startSeconds

```
double Climb::startSeconds = 0.0
```

[Climb](#) start time in seconds from activity start.

Definition at line 50 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::drawForeground\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [RideChartWidget::mousePressEvent\(\)](#), and [RideChartWidget::mouseReleaseEvent\(\)](#).



### 7.24.2.32 vam

double Climb::vam = 0.0

VAM (Vertical Ascent Meter) in m/h.

Definition at line 110 of file [ClimbMetrics.h](#).

Referenced by [ClimbMetrics::build\(\)](#), and [ClimbDetector::detect\(\)](#).

### 7.24.2.33 wattsPerKgRank

QString Climb::wattsPerKgRank

Watts/kg performance ranking.

Definition at line 140 of file [ClimbMetrics.h](#).

The documentation for this struct was generated from the following file:

- [src/analysis/ClimbMetrics.h](#)

## 7.25 ClimbDetector Class Reference

Detects and categorizes climbs in cycling activities.

#include <ClimbDetector.h>

### Public Types

- using [Config](#) = [ClimbDetectorConfig](#)  
*Configuration typedef for climb detection parameters.*

### Static Public Member Functions

- static std::vector< [Climb](#) > [detect](#) (const [RideData](#) &rideData, const [Config](#) &config=[Config](#){})  
*Detects climbs in activity data using multi-phase algorithm.*
- static std::vector< double > [buildCumulativeDistanceMeters](#) (const [RideData](#) &rideData)  
*Builds cumulative distance array for activity.*
- static std::vector< double > [smoothAltitudeByDistance](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, double windowMeters, const std::vector< double > &adaptiveWindowMeters={}, double altitudeSpikeVerticalSpeedMps=8.0)  
*Smooths altitude data using distance-based rolling window.*
- static std::vector< double > [computeAdaptiveSmoothingWindowsMeters](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, const [Config](#) &config)  
*Computes adaptive smoothing windows for elevation data.*
- static std::vector< double > [computeLocalGradientPct](#) (const std::vector< double > &smoothedAltitudeMeters, const std::vector< double > &cumulativeDistanceMeters, double halfWindowMeters)  
*Computes local gradient at each point.*
- static double [computeClimbScore](#) (double gainMeters, double averageGrade, double distanceKm, double continuityRatio, double vam)  
*Computes scoring-based climb difficulty metric.*
- static double [computeVAM](#) (double elevationGainM, double durationSeconds)  
*Computes vertical ascent metric (VAM).*
- static ClimbCategory [categorizeClimb](#) (double gainMeters, double lengthKm, double averageGrade)  
*Categorizes a climb by difficulty.*
- static std::vector< [Climb](#) > [mergeFragmentedClimbs](#) (std::vector< [Climb](#) > &candidates, const [Config](#) &config)  
*Merges fragmented climb candidates into continuous climbs.*

### 7.25.1 Detailed Description

Detects and categorizes climbs in cycling activities.

Provides static methods for analyzing activity elevation data to identify climbs using multi-phase detection including gradient thresholds, scoring, and merge logic.

Definition at line 100 of file [ClimbDetector.h](#).

### 7.25.2 Member Typedef Documentation

#### 7.25.2.1 Config

```
using ClimbDetector::Config = ClimbDetectorConfig
```

Configuration typedef for climb detection parameters.

Definition at line 104 of file [ClimbDetector.h](#).

### 7.25.3 Member Function Documentation

#### 7.25.3.1 buildCumulativeDistanceMeters()

```
std::vector< double > ClimbDetector::buildCumulativeDistanceMeters (
    const RideData & rideData) [static]
```

Builds cumulative distance array for activity.

##### Parameters

|                 |                |
|-----------------|----------------|
| <i>rideData</i> | Activity data. |
|-----------------|----------------|

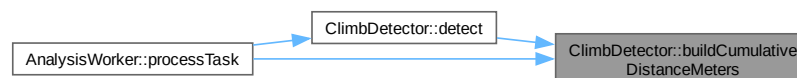
##### Returns

Array of cumulative distances in meters.

Definition at line 103 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



#### 7.25.3.2 categorizeClimb()

```
ClimbCategory ClimbDetector::categorizeClimb (
    double gainMeters,
    double lengthKm,
    double averageGrade) [static]
```

Categorizes a climb by difficulty.

##### Parameters

|                     |                                             |
|---------------------|---------------------------------------------|
| <i>gainMeters</i>   | Elevation gain in meters.                   |
| <i>lengthKm</i>     | <a href="#">Climb</a> length in kilometers. |
| <i>averageGrade</i> | Average gradient percentage.                |

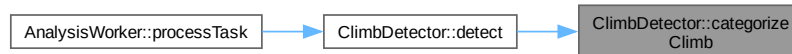
**Returns**

Category classification.

Definition at line 270 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.25.3.3 computeAdaptiveSmoothingWindowsMeters()**

```

std::vector< double > ClimbDetector::computeAdaptiveSmoothingWindowsMeters (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    const Config & config) [static]
  
```

Computes adaptive smoothing windows for elevation data.

**Parameters**

|                                 |                                                   |
|---------------------------------|---------------------------------------------------|
| <i>rideData</i>                 | <a href="#">Activity</a> data.                    |
| <i>cumulativeDistanceMeters</i> | Cumulative distances.                             |
| <i>config</i>                   | Configuration with adaptive smoothing parameters. |

**Returns**

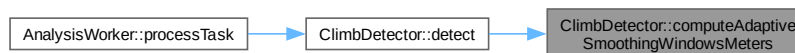
Array of adaptive window sizes.

Definition at line 168 of file [ClimbDetector.cpp](#).

References [ClimbDetectorConfig::adaptiveSmoothingMaxMeters](#), [ClimbDetectorConfig::adaptiveSmoothingMinMeters](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [RideData::records](#), and [ClimbDetectorConfig::useAdaptiveSmoothing](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.25.3.4 computeClimbScore()**

```

double ClimbDetector::computeClimbScore (
    double gainMeters,
    double averageGrade,
    double distanceKm,
    double continuityRatio,
    double vam) [static]
  
```

Computes scoring-based climb difficulty metric.

**Parameters**

|                        |                                               |
|------------------------|-----------------------------------------------|
| <i>gainMeters</i>      | Elevation gain in meters.                     |
| <i>averageGrade</i>    | Average gradient percentage.                  |
| <i>distanceKm</i>      | <a href="#">Climb</a> distance in kilometers. |
| <i>continuityRatio</i> | Ratio of continuous vs interrupted climbing.  |
| <i>vam</i>             | Vertical ascent speed in meters/hour.         |

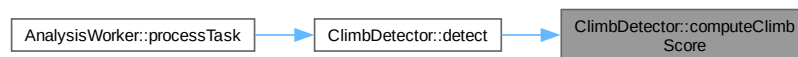
**Returns**

Computed climb score.

Definition at line 241 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.25.3.5 computeLocalGradientPct()**

```

std::vector< double > ClimbDetector::computeLocalGradientPct (
    const std::vector< double > & smoothedAltitudeMeters,
    const std::vector< double > & cumulativeDistanceMeters,
    double halfWindowMeters) [static]
  
```

Computes local gradient at each point.

**Parameters**

|                                 |                                          |
|---------------------------------|------------------------------------------|
| <i>smoothedAltitudeMeters</i>   | Smoothed elevation data.                 |
| <i>cumulativeDistanceMeters</i> | Cumulative distances.                    |
| <i>halfWindowMeters</i>         | Half-width of gradient window in meters. |

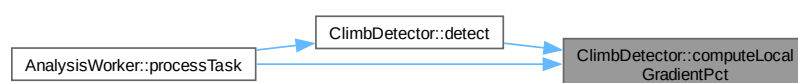
**Returns**

Array of gradient percentages.

Definition at line 200 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



### 7.25.3.6 computeVAM()

```
double ClimbDetector::computeVAM (
    double elevationGainM,
    double durationSeconds) [static]
```

Computes vertical ascent metric (VAM).

#### Parameters

|                        |                           |
|------------------------|---------------------------|
| <i>elevationGainM</i>  | Elevation gain in meters. |
| <i>durationSeconds</i> | Duration in seconds.      |

#### Returns

VAM in meters/hour.

Definition at line 257 of file [ClimbDetector.cpp](#).

### 7.25.3.7 detect()

```
std::vector< Climb > ClimbDetector::detect (
    const RideData & rideData,
    const Config & config = Config{}) [static]
```

Detects climbs in activity data using multi-phase algorithm.

#### Parameters

|                 |                                          |
|-----------------|------------------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> recording data. |
| <i>config</i>   | Detection configuration parameters.      |

#### Returns

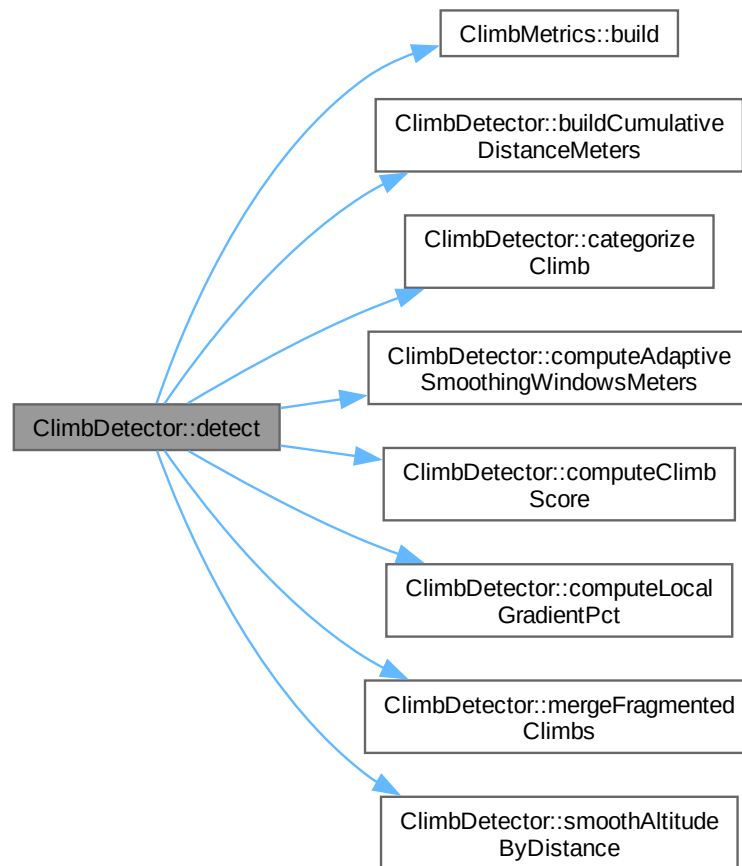
Vector of detected climbs.

Definition at line 286 of file [ClimbDetector.cpp](#).

References [ClimbDetectorConfig::altitudeSpikeVerticalSpeedMps](#), [Climb::averageGradient](#), [ClimbMetrics::build\(\)](#), [buildCumulativeDistanceMeters\(\)](#), [categorizeClimb\(\)](#), [Climb::climbCategory](#), [Climb::climbScore](#), [computeAdaptiveSmoothingWindows](#), [computeClimbScore\(\)](#), [computeLocalGradientPct\(\)](#), [ClimbDetectorConfig::continueGradient](#), [Climb::elevationGainM](#), [ClimbDetectorConfig::elevationSmoothingMeters](#), [ClimbDetectorConfig::enableClimbMerging](#), [ClimbDetectorConfig::endGradient](#), [Climb::gradePercentile](#), [RideRecord::hasAltitude](#), [Climb::lengthKm](#), [ClimbDetectorConfig::maxDipDistanceMeters](#), [ClimbDetectorConfig::maxDipMeters](#), [ClimbDetectorConfig::maxDipPctOfGain](#), [Climb::maximumGradient](#), [ClimbDetectorConfig::maxPlateauDistanceMeters](#), [ClimbDetectorConfig::maxSmallDipsCount](#), [ClimbDetectorConfig::maxStartInterrupt](#), [mergeFragmentedClimbs\(\)](#), [ClimbDetectorConfig::minAverageGradient](#), [ClimbDetectorConfig::minClimbScore](#), [ClimbDetectorConfig::minElevationGainM](#), [ClimbDetectorConfig::minLengthKm](#), [ClimbDetectorConfig::minStartContinuousDistanceM](#), [ClimbDetectorConfig::plateauMinGradient](#), [RideData::records](#), [ClimbDetectorConfig::smallDipThresholdMeters](#), [smoothAltitudeByDistance\(\)](#), [ClimbDetectorConfig::startGradient](#), [ClimbDetectorConfig::useScoringDetection](#), and [Climb::vam](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.25.3.8 mergeFragmentedClimbs()

```

std::vector< Climb > ClimbDetector::mergeFragmentedClimbs (
    std::vector< Climb > & candidates,
    const Config & config) [static]
  
```

Merges fragmented climb candidates into continuous climbs.

**Parameters**

|                   |                                     |
|-------------------|-------------------------------------|
| <i>candidates</i> | Vector of initial climb candidates. |
| <i>config</i>     | Merge configuration parameters.     |

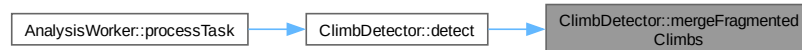
**Returns**

Merged climb results.

Definition at line 279 of file [ClimbDetector.cpp](#).

Referenced by [detect\(\)](#).

Here is the caller graph for this function:

**7.25.3.9 smoothAltitudeByDistance()**

```

std::vector< double > ClimbDetector::smoothAltitudeByDistance (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    double windowMeters,
    const std::vector< double > & adaptiveWindowMeters = {},
    double altitudeSpikeVerticalSpeedMps = 8.0) [static]
  
```

Smooths altitude data using distance-based rolling window.

**Parameters**

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| <i>rideData</i>                      | <a href="#">Activity</a> data.       |
| <i>cumulativeDistanceMeters</i>      | Cumulative distances.                |
| <i>windowMeters</i>                  | Fixed smoothing window in meters.    |
| <i>adaptiveWindowMeters</i>          | Optional adaptive window per sample. |
| <i>altitudeSpikeVerticalSpeedMps</i> | Spike detection threshold in m/s.    |

**Returns**

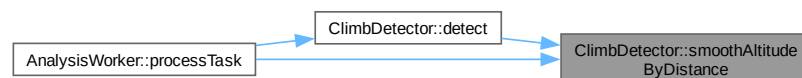
Smoothed altitude array.

Definition at line 108 of file [ClimbDetector.cpp](#).

References [RideData::records](#).

Referenced by [detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- src/analysis/ClimbDetector.h
- src/analysis/ClimbDetector.cpp

## 7.26 ClimbDetectorConfig Struct Reference

Configuration parameters for climb detection.

```
#include <ClimbDetector.h>
```

### Public Attributes

- double `minLengthKm` = 0.15  
*Minimum climb length in km (legacy).*
- double `minElevationGainM` = 10.0  
*Minimum elevation gain in meters (legacy).*
- double `minAverageGradient` = 4.0  
*Minimum average gradient (legacy).*
- double `startGradient` = 1.5  
*Gradient threshold to start climb detection (%).*
- double `continueGradient` = 1.0  
*Gradient threshold to continue climb (%).*
- double `endGradient` = 0.5  
*Gradient threshold to end climb (%).*
- double `minStartContinuousDistanceMeters` = 100.0  
*Minimum continuous distance at start gradient (meters).*
- double `maxStartInterruptionMeters` = 30.0  
*Maximum interruption when starting climb (meters).*
- double `maxDipMeters` = 10.0  
*Maximum dip elevation loss (meters).*
- double `maxDipDistanceMeters` = 200.0  
*Maximum dip horizontal distance (meters).*
- double `maxDipPctOfGain` = 10.0  
*Maximum dip as percentage of gain (%).*
- double `maxSmallDipsCount` = 3  
*Maximum number of small dips (< 5m) allowed.*
- double `smallDipThresholdMeters` = 5.0  
*Threshold for small dip detection (meters).*
- double `maxPlateauDistanceMeters` = 500.0  
*Maximum distance for plateau detection (meters).*
- double `plateauMinGradient` = -1.0  
*Minimum gradient for plateau (%).*
- double `elevationSmoothingMeters` = 50.0  
*Elevation smoothing window (meters).*
- bool `useAdaptiveSmoothing` = true  
*Enable adaptive smoothing based on grade changes.*
- double `adaptiveSmoothingMinMeters` = 25.0  
*Minimum adaptive smoothing window (meters).*
- double `adaptiveSmoothingMaxMeters` = 100.0  
*Maximum adaptive smoothing window (meters).*
- double `gradientWindowHalfMeters` = 25.0  
*Half-window for gradient computation (meters).*



- double `altitudeSpikeVerticalSpeedMps` = 8.0  
*Altitude spike detection threshold (m/s).*
- bool `useScoringDetection` = true  
*Enable scoring-based climb detection.*
- double `minClimbScore` = 50.0  
*Minimum composite climb score.*
- double `gainScoreWeight` = 0.5  
*Weight for elevation gain in score.*
- double `gradeScoreWeight` = 4.0  
*Weight for average grade in score.*
- double `durationScoreWeight` = 2.0  
*Weight for duration in score.*
- double `vamScoreBoost` = 800.0  
*VAM threshold for score boost.*
- bool `requireGradePersistence` = true  
*Enable grade persistence requirement.*
- double `gradePersistenceThreshold` = 4.0  
*Minimum gradient for persistence (%).*
- double `gradePersistenceMinMeters` = 50.0  
*Minimum distance at grade for persistence (meters).*
- bool `enableClimbMerging` = true  
*Enable merge of fragmented climbs.*
- double `mergeMaxGapMeters` = 50.0  
*Maximum gap between climbs to merge (meters).*
- double `mergeMaxGapElevationM` = 10.0  
*Maximum elevation gap to allow merge (meters).*

### 7.26.1 Detailed Description

Configuration parameters for climb detection.

Contains tunable thresholds and flags used to control climb detection behavior, including gradient thresholds, distance tolerances, and elevation smoothing parameters.

Definition at line 16 of file `ClimbDetector.h`.

### 7.26.2 Member Data Documentation

#### 7.26.2.1 adaptiveSmoothingMaxMeters

```
double ClimbDetectorConfig::adaptiveSmoothingMaxMeters = 100.0
```

Maximum adaptive smoothing window (meters).

Definition at line 59 of file `ClimbDetector.h`.

Referenced by `ClimbDetector::computeAdaptiveSmoothingWindowsMeters()`.

#### 7.26.2.2 adaptiveSmoothingMinMeters

```
double ClimbDetectorConfig::adaptiveSmoothingMinMeters = 25.0
```

Minimum adaptive smoothing window (meters).

Definition at line 57 of file `ClimbDetector.h`.

Referenced by `ClimbDetector::computeAdaptiveSmoothingWindowsMeters()`.

#### 7.26.2.3 altitudeSpikeVerticalSpeedMps

```
double ClimbDetectorConfig::altitudeSpikeVerticalSpeedMps = 8.0
```

Altitude spike detection threshold (m/s).

Definition at line 64 of file `ClimbDetector.h`.

Referenced by `ClimbDetector::detect()`.

#### 7.26.2.4 continueGradient

```
double ClimbDetectorConfig::continueGradient = 1.0
```

Gradient threshold to continue climb (%).

Definition at line 28 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.26.2.5 durationScoreWeight

```
double ClimbDetectorConfig::durationScoreWeight = 2.0
```

Weight for duration in score.

Definition at line 75 of file [ClimbDetector.h](#).

#### 7.26.2.6 elevationSmoothingMeters

```
double ClimbDetectorConfig::elevationSmoothingMeters = 50.0
```

Elevation smoothing window (meters).

Definition at line 53 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#), [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

#### 7.26.2.7 enableClimbMerging

```
bool ClimbDetectorConfig::enableClimbMerging = true
```

Enable merge of fragmented climbs.

Definition at line 87 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.26.2.8 endGradient

```
double ClimbDetectorConfig::endGradient = 0.5
```

Gradient threshold to end climb (%).

Definition at line 30 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.26.2.9 gainScoreWeight

```
double ClimbDetectorConfig::gainScoreWeight = 0.5
```

Weight for elevation gain in score.

Definition at line 71 of file [ClimbDetector.h](#).

#### 7.26.2.10 gradePersistenceMinMeters

```
double ClimbDetectorConfig::gradePersistenceMinMeters = 50.0
```

Minimum distance at grade for persistence (meters).

Definition at line 84 of file [ClimbDetector.h](#).

#### 7.26.2.11 gradePersistenceThreshold

```
double ClimbDetectorConfig::gradePersistenceThreshold = 4.0
```

Minimum gradient for persistence (%).

Definition at line 82 of file [ClimbDetector.h](#).

#### 7.26.2.12 gradeScoreWeight

```
double ClimbDetectorConfig::gradeScoreWeight = 4.0
```

Weight for average grade in score.

Definition at line 73 of file [ClimbDetector.h](#).

### 7.26.2.13 gradientWindowHalfMeters

```
double ClimbDetectorConfig::gradientWindowHalfMeters = 25.0
```

Half-window for gradient computation (meters).

Definition at line 61 of file [ClimbDetector.h](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

### 7.26.2.14 maxDipDistanceMeters

```
double ClimbDetectorConfig::maxDipDistanceMeters = 200.0
```

Maximum dip horizontal distance (meters).

Definition at line 39 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.15 maxDipMeters

```
double ClimbDetectorConfig::maxDipMeters = 10.0
```

Maximum dip elevation loss (meters).

Definition at line 37 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.16 maxDipPctOfGain

```
double ClimbDetectorConfig::maxDipPctOfGain = 10.0
```

Maximum dip as percentage of gain (%).

Definition at line 41 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.17 maxPlateauDistanceMeters

```
double ClimbDetectorConfig::maxPlateauDistanceMeters = 500.0
```

Maximum distance for plateau detection (meters).

Definition at line 48 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.18 maxSmallDipsCount

```
double ClimbDetectorConfig::maxSmallDipsCount = 3
```

Maximum number of small dips (< 5m) allowed.

Definition at line 43 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.19 maxStartInterruptionMeters

```
double ClimbDetectorConfig::maxStartInterruptionMeters = 30.0
```

Maximum interruption when starting climb (meters).

Definition at line 34 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.20 mergeMaxGapElevationM

```
double ClimbDetectorConfig::mergeMaxGapElevationM = 10.0
```

Maximum elevation gap to allow merge (meters).

Definition at line 91 of file [ClimbDetector.h](#).

### 7.26.2.21 mergeMaxGapMeters

```
double ClimbDetectorConfig::mergeMaxGapMeters = 50.0
```

Maximum gap between climbs to merge (meters).

Definition at line 89 of file [ClimbDetector.h](#).

### 7.26.2.22 minAverageGradient

```
double ClimbDetectorConfig::minAverageGradient = 4.0
```

Minimum average gradient (legacy).

Definition at line 23 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.26.2.23 minClimbScore

```
double ClimbDetectorConfig::minClimbScore = 50.0
```

Minimum composite climb score.

Definition at line 69 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.24 minElevationGainM

```
double ClimbDetectorConfig::minElevationGainM = 10.0
```

Minimum elevation gain in meters (legacy).

Definition at line 21 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.26.2.25 minLengthKm

```
double ClimbDetectorConfig::minLengthKm = 0.15
```

Minimum climb length in km (legacy).

Definition at line 19 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.26.2.26 minStartContinuousDistanceMeters

```
double ClimbDetectorConfig::minStartContinuousDistanceMeters = 100.0
```

Minimum continuous distance at start gradient (meters).

Definition at line 32 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.27 plateauMinGradient

```
double ClimbDetectorConfig::plateauMinGradient = -1.0
```

Minimum gradient for plateau (%).

Definition at line 50 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.28 requireGradePersistence

```
bool ClimbDetectorConfig::requireGradePersistence = true
```

Enable grade persistence requirement.

Definition at line 80 of file [ClimbDetector.h](#).

### 7.26.2.29 smallDipThresholdMeters

```
double ClimbDetectorConfig::smallDipThresholdMeters = 5.0
```

Threshold for small dip detection (meters).

Definition at line 45 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

### 7.26.2.30 startGradient

```
double ClimbDetectorConfig::startGradient = 1.5
```

Gradient threshold to start climb detection (%).

Definition at line 26 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.26.2.31 useAdaptiveSmoothing

```
bool ClimbDetectorConfig::useAdaptiveSmoothing = true
```

Enable adaptive smoothing based on grade changes.

Definition at line 55 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#).

#### 7.26.2.32 useScoringDetection

```
bool ClimbDetectorConfig::useScoringDetection = true
```

Enable scoring-based climb detection.

Definition at line 67 of file [ClimbDetector.h](#).

Referenced by [ClimbDetector::detect\(\)](#).

#### 7.26.2.33 vamScoreBoost

```
double ClimbDetectorConfig::vamScoreBoost = 800.0
```

VAM threshold for score boost.

Definition at line 77 of file [ClimbDetector.h](#).

The documentation for this struct was generated from the following file:

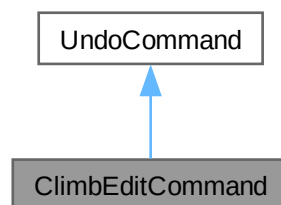
- [src/analysis/ClimbDetector.h](#)

## 7.27 ClimbEditCommand Class Reference

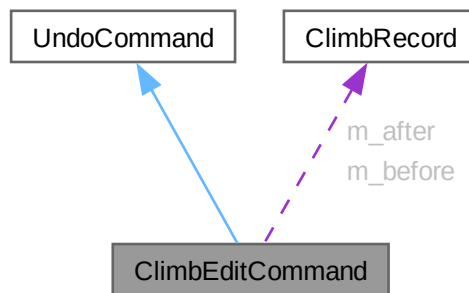
Undo/redo command for climb boundary or field edits.

```
#include <ClimbEditCommand.h>
```

Inheritance diagram for ClimbEditCommand:



Collaboration diagram for `ClimbEditCommand`:



### Public Member Functions

- `ClimbEditCommand` (const `ClimbRecord` &before, const `ClimbRecord` &after, const `QString` &dbConnection←Name, std::function< void()> refreshCallback)  
*Constructs climb edit command.*
- void `undo` () override  
*Reverses the command (undo operation).*
- void `redo` () override  
*Re-applies the command (redo operation).*
- `QString` `description` () const override  
*Returns a human-readable description of the operation.*

### Private Member Functions

- void `apply` (const `ClimbRecord` &record)

### Private Attributes

- `ClimbRecord` `m_before`
- `ClimbRecord` `m_after`
- `QString` `m_connName`
- std::function< void()> `m_refresh`

#### 7.27.1 Detailed Description

Undo/redo command for climb boundary or field edits.

Stores before/after `ClimbRecord` pairs and applies changes through the `ClimbRepository` during undo/redo operations.

Definition at line 19 of file `ClimbEditCommand.h`.

#### 7.27.2 Constructor & Destructor Documentation

##### 7.27.2.1 `ClimbEditCommand()`

```

ClimbEditCommand::ClimbEditCommand (
    const ClimbRecord & before,
    const ClimbRecord & after,

```

```
const QString & dbConnectionName,
std::function< void()> refreshCallback)
```

Constructs climb edit command.

#### Parameters

|                         |                                          |
|-------------------------|------------------------------------------|
| <i>before</i>           | Previous climb record state.             |
| <i>after</i>            | New climb record state.                  |
| <i>dbConnectionName</i> | Database connection name.                |
| <i>refreshCallback</i>  | Function to call after applying changes. |

Definition at line 8 of file [ClimbEditCommand.cpp](#).

### 7.27.3 Member Function Documentation

#### 7.27.3.1 apply()

```
void ClimbEditCommand::apply (
    const ClimbRecord & record) [private]
```

Definition at line 33 of file [ClimbEditCommand.cpp](#).

#### 7.27.3.2 description()

```
QString ClimbEditCommand::description () const [override], [virtual]
```

Returns a human-readable description of the operation.

#### Returns

Short description shown in menus/status bar.

Reimplemented from [UndoCommand](#).

Definition at line 28 of file [ClimbEditCommand.cpp](#).

#### 7.27.3.3 redo()

```
void ClimbEditCommand::redo () [override], [virtual]
```

Re-applies the command (redo operation).

Implements [UndoCommand](#).

Definition at line 23 of file [ClimbEditCommand.cpp](#).

#### 7.27.3.4 undo()

```
void ClimbEditCommand::undo () [override], [virtual]
```

Reverses the command (undo operation).

Implements [UndoCommand](#).

Definition at line 18 of file [ClimbEditCommand.cpp](#).

### 7.27.4 Member Data Documentation

#### 7.27.4.1 m\_after

```
ClimbRecord ClimbEditCommand::m_after [private]
```

Definition at line 42 of file [ClimbEditCommand.h](#).

#### 7.27.4.2 m\_before

```
ClimbRecord ClimbEditCommand::m_before [private]
```

Definition at line 41 of file [ClimbEditCommand.h](#).

### 7.27.4.3 m\_connName

QString ClimbEditCommand::m\_connName [private]  
 Definition at line 43 of file [ClimbEditCommand.h](#).

### 7.27.4.4 m\_refresh

std::function<void()> ClimbEditCommand::m\_refresh [private]  
 Definition at line 44 of file [ClimbEditCommand.h](#).

The documentation for this class was generated from the following files:

- src/core/undo/ClimbEditCommand.h
- src/core/undo/ClimbEditCommand.cpp

## 7.28 RideChartSelectionManager::ClimbHandleHitResult Struct Reference

### Public Attributes

- bool [hit](#) = false
- bool [startHandle](#) = true

### 7.28.1 Detailed Description

Definition at line 15 of file [RideChartSelectionManager.h](#).

### 7.28.2 Member Data Documentation

#### 7.28.2.1 hit

bool RideChartSelectionManager::ClimbHandleHitResult::hit = false  
 Definition at line 17 of file [RideChartSelectionManager.h](#).

#### 7.28.2.2 startHandle

bool RideChartSelectionManager::ClimbHandleHitResult::startHandle = true  
 Definition at line 18 of file [RideChartSelectionManager.h](#).

The documentation for this struct was generated from the following file:

- src/charts/selection/RideChartSelectionManager.h

## 7.29 ClimbMetrics Class Reference

Computes comprehensive climb metrics from activity data.

```
#include <ClimbMetrics.h>
```

### Static Public Member Functions

- static [Climb build](#) (const [RideData](#) &rideData, const std::vector< double > &cumulativeDistanceMeters, const std::vector< double > &smoothedAltitudeMeters, const std::vector< double > &localGradientPct, int startIdx, int endIdx, int climbOrdinal)

*Builds a complete [Climb](#) object from activity data.*

### 7.29.1 Detailed Description

Computes comprehensive climb metrics from activity data.

Definition at line 149 of file [ClimbMetrics.h](#).



## 7.29.2 Member Function Documentation

### 7.29.2.1 build()

```
Climb ClimbMetrics::build (
    const RideData & rideData,
    const std::vector< double > & cumulativeDistanceMeters,
    const std::vector< double > & smoothedAltitudeMeters,
    const std::vector< double > & localGradientPct,
    int startIdx,
    int endIdx,
    int climbOrdinal) [static]
```

Builds a complete **Climb** object from activity data.

#### Parameters

|                                 |                                        |
|---------------------------------|----------------------------------------|
| <i>rideData</i>                 | <b>Activity</b> recording data.        |
| <i>cumulativeDistanceMeters</i> | Distance array.                        |
| <i>smoothedAltitudeMeters</i>   | Elevation array (smoothed).            |
| <i>localGradientPct</i>         | Gradient array.                        |
| <i>startIdx</i>                 | Index of climb start.                  |
| <i>endIdx</i>                   | Index of climb end.                    |
| <i>climbOrdinal</i>             | Ordinal position of climb in activity. |

#### Returns

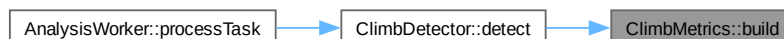
Fully populated **Climb** structure with all metrics.

Definition at line 221 of file **ClimbMetrics.cpp**.

References **Climb::aerobicDecouplingPct**, **RideRecord::altitude**, **Climb::averageCadence**, **Climb::averageGradient**, **Climb::averageHeartRate**, **Climb::averagePower**, **Climb::averageSpeed**, **RideRecord::cadence**, **Climb::category**, **Climb::difficultyScore**, **Climb::durationSeconds**, **RideRecord::elapsedSeconds**, **Climb::elevationGainM**, **Climb::endDistanceKm**, **Climb::endElevationM**, **Climb::endSeconds**, **Climb::gradePercentile**, **RideRecord::hasAltitude**, **RideRecord::hasCadence**, **RideRecord::hasHeartRate**, **RideRecord::hasPower**, **RideRecord::hasSpeed**, **RideRecord::heartRate**, **Climb::hrDriftPct**, **Climb::lengthKm**, **Climb::maximumGradient**, **Climb::name**, **Climb::normalizedPower**, **RideRecord::power**, **Climb::powerFadePct**, **Climb::powerFirstHalf**, **Climb::powerSecondHalf**, **Climb::quarterMetrics**, **RideData::records**, **Climb::shapeClass**, **RideRecord::speed**, **Climb::startDistanceKm**, **Climb::startElevationM**, **Climb::startSeconds**, and **Climb::vam**.

Referenced by **ClimbDetector::detect()**.

Here is the caller graph for this function:

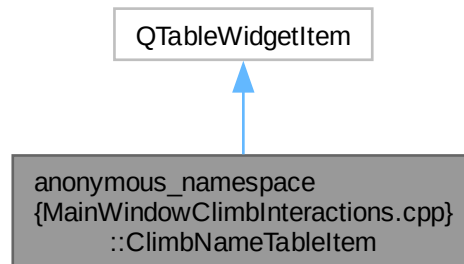


The documentation for this class was generated from the following files:

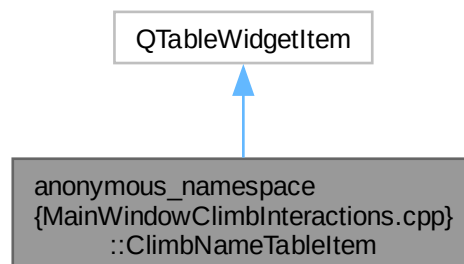
- src/analysis/ClimbMetrics.h
- src/analysis/ClimbMetrics.cpp

## 7.30 anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem Class Reference

Inheritance diagram for anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem:



Collaboration diagram for anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem:



### Public Member Functions

- [ClimbNameTableItem](#) (const QString &text)
- bool [operator<](#) (const QTableWidgetItem &other) const override

### Static Private Member Functions

- static int [trailingNumber](#) (const QString &text, bool \*ok)

### 7.30.1 Detailed Description

Definition at line 46 of file [MainWindowClimbInteractions.cpp](#).

### 7.30.2 Constructor & Destructor Documentation

#### 7.30.2.1 ClimbNameTableItem()

```
anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem::ClimbNameTableItem (
    const QString & text) [inline], [explicit]
```

Definition at line 49 of file [MainWindowClimbInteractions.cpp](#).

### 7.30.3 Member Function Documentation

#### 7.30.3.1 operator<()

```
bool anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 53 of file [MainWindowClimbInteractions.cpp](#).

#### 7.30.3.2 trailingNumber()

```
int anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem::trailingNumber (
    const QString & text,
    bool * ok) [inline], [static], [private]
```

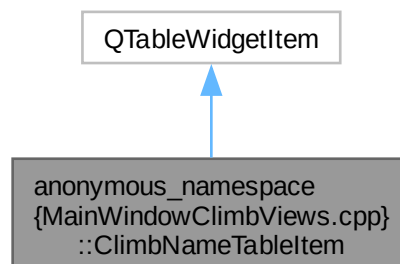
Definition at line 67 of file [MainWindowClimbInteractions.cpp](#).

The documentation for this class was generated from the following file:

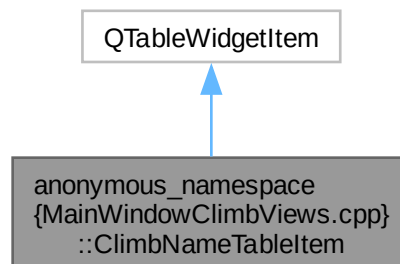
- [src/gui/MainWindowClimbInteractions.cpp](#)

## 7.31 anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem Class Reference

Inheritance diagram for anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem:



Collaboration diagram for `anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem`:



### Public Member Functions

- [ClimbNameTableItem](#) (const QString &text)
- bool [operator<](#) (const QTableWidgetItem &other) const override

### Static Private Member Functions

- static int [trailingNumber](#) (const QString &text, bool \*ok)

## 7.31.1 Detailed Description

Definition at line 43 of file [MainWindowClimbViews.cpp](#).

## 7.31.2 Constructor & Destructor Documentation

### 7.31.2.1 ClimbNameTableItem()

```
anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem::ClimbNameTableItem (
    const QString & text) [inline], [explicit]
```

Definition at line 46 of file [MainWindowClimbViews.cpp](#).

## 7.31.3 Member Function Documentation

### 7.31.3.1 operator<()

```
bool anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 50 of file [MainWindowClimbViews.cpp](#).

### 7.31.3.2 trailingNumber()

```
int anonymous_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem::trailingNumber (
    const QString & text,
    bool * ok) [inline], [static], [private]
```

Definition at line 64 of file [MainWindowClimbViews.cpp](#).

The documentation for this class was generated from the following file:

- `src/gui/MainWindowClimbViews.cpp`

## 7.32 ClimbQuarterMetrics Struct Reference

Metrics for a quarter of a climb segment.

```
#include <ClimbMetrics.h>
```

### Public Attributes

- double [averagePower](#) = 0.0  
*Average power during quarter.*
- double [averageHeartRate](#) = 0.0  
*Average heart rate during quarter.*
- double [averageCadence](#) = 0.0  
*Average cadence during quarter.*

### 7.32.1 Detailed Description

Metrics for a quarter of a climb segment.

Definition at line 27 of file [ClimbMetrics.h](#).

### 7.32.2 Member Data Documentation

#### 7.32.2.1 averageCadence

```
double ClimbQuarterMetrics::averageCadence = 0.0
```

Average cadence during quarter.

Definition at line 36 of file [ClimbMetrics.h](#).

#### 7.32.2.2 averageHeartRate

```
double ClimbQuarterMetrics::averageHeartRate = 0.0
```

Average heart rate during quarter.

Definition at line 33 of file [ClimbMetrics.h](#).

#### 7.32.2.3 averagePower

```
double ClimbQuarterMetrics::averagePower = 0.0
```

Average power during quarter.

Definition at line 30 of file [ClimbMetrics.h](#).

The documentation for this struct was generated from the following file:

- src/analysis/ClimbMetrics.h

## 7.33 ClimbRecord Struct Reference

Database representation of a stored climb.

```
#include <ClimbRepository.h>
```

### Public Attributes

- int [id](#) = -1  
*Unique climb record ID.*
- int [activityId](#) = -1  
*Associated activity ID.*
- QString [uuid](#)  
*Stable UUID, generated once on insert.*
- double [startSeconds](#) = 0.0  
*Climb start time in seconds from activity start.*

- double `endSeconds` = 0.0  
*Climb end time in seconds from activity start.*
- double `originalStartSeconds` = -1.0  
*Start bounds at first auto-detection (-1 if not set).*
- double `originalEndSeconds` = -1.0  
*End bounds at first auto-detection (-1 if not set).*
- QString `name`  
*Climb name.*
- double `elevationGainM` = 0.0  
*Elevation gain in meters.*
- double `lengthKm` = 0.0  
*Climb length in kilometers.*
- double `averageGradient` = 0.0  
*Average gradient percentage.*
- double `avgPower` = 0.0  
*Average power in watts (cached).*
- double `np` = 0.0  
*Normalized power in watts (cached).*
- double `avgHr` = 0.0  
*Average heart rate in bpm (cached).*
- double `avgCadence` = 0.0  
*Average cadence in rpm (cached).*
- double `vam` = 0.0  
*VAM (Vertical Ascent Meter) in m/h (cached).*
- QString `source` = QStringLiteral("auto")  
*Source of climb (auto, manual, edited).*
- bool `locked` = false  
*Whether climb is locked from editing.*
- bool `deleted` = false  
*Soft-delete flag.*
- int `algorithmVersion` = 1  
*Algorithm version used for detection.*
- QString `notes`  
*User notes about climb.*
- bool `favorite` = false  
*Whether climb is marked as favorite.*
- int `rating` = 0  
*User rating (0-5).*

### 7.33.1 Detailed Description

Database representation of a stored climb.

Contains all climb metrics and lifecycle information persisted in the database.

Definition at line 16 of file [ClimbRepository.h](#).

### 7.33.2 Member Data Documentation

#### 7.33.2.1 activityId

```
int ClimbRecord::activityId = -1
```

Associated activity ID.

Definition at line 22 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.33.2.2 algorithmVersion

```
int ClimbRecord::algorithmVersion = 1
```

Algorithm version used for detection.

Definition at line 76 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.33.2.3 averageGradient

```
double ClimbRecord::averageGradient = 0.0
```

Average gradient percentage.

Definition at line 49 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.4 avgCadence

```
double ClimbRecord::avgCadence = 0.0
```

Average cadence in rpm (cached).

Definition at line 61 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.5 avgHr

```
double ClimbRecord::avgHr = 0.0
```

Average heart rate in bpm (cached).

Definition at line 58 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.6 avgPower

```
double ClimbRecord::avgPower = 0.0
```

Average power in watts (cached).

Definition at line 52 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.7 deleted

```
bool ClimbRecord::deleted = false
```

Soft-delete flag.

Definition at line 73 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.8 elevationGainM

```
double ClimbRecord::elevationGainM = 0.0
```

Elevation gain in meters.

Definition at line 43 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.9 endSeconds

```
double ClimbRecord::endSeconds = 0.0
```

[Climb](#) end time in seconds from activity start.

Definition at line 31 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.10 favorite

```
bool ClimbRecord::favorite = false
```

Whether climb is marked as favorite.

Definition at line 82 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.11 id

```
int ClimbRecord::id = -1
```

Unique climb record ID.

Definition at line 19 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.12 lengthKm

```
double ClimbRecord::lengthKm = 0.0
```

[Climb](#) length in kilometers.

Definition at line 46 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.13 locked

```
bool ClimbRecord::locked = false
```

Whether climb is locked from editing.

Definition at line 70 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.14 name

```
QString ClimbRecord::name
```

[Climb](#) name.

Definition at line 40 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.15 notes

```
QString ClimbRecord::notes
```

User notes about climb.

Definition at line 79 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.16 np

```
double ClimbRecord::np = 0.0
```

Normalized power in watts (cached).

Definition at line 55 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

#### 7.33.2.17 originalEndSeconds

```
double ClimbRecord::originalEndSeconds = -1.0
```

End bounds at first auto-detection (-1 if not set).

Definition at line 37 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

#### 7.33.2.18 originalStartSeconds

```
double ClimbRecord::originalStartSeconds = -1.0
```

Start bounds at first auto-detection (-1 if not set).

Definition at line 34 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).



### 7.33.2.19 rating

```
int ClimbRecord::rating = 0
```

User rating (0-5).

Definition at line 85 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.20 source

```
QString ClimbRecord::source = QStringLiteral("auto")
```

Source of climb (auto, manual, edited).

Definition at line 67 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.21 startSeconds

```
double ClimbRecord::startSeconds = 0.0
```

[Climb](#) start time in seconds from activity start.

Definition at line 28 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

### 7.33.2.22 uuid

```
QString ClimbRecord::uuid
```

Stable UUID, generated once on insert.

Definition at line 25 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), and [AnalysisWorker::processTask\(\)](#).

### 7.33.2.23 vam

```
double ClimbRecord::vam = 0.0
```

VAM (Vertical Ascent Meter) in m/h (cached).

Definition at line 64 of file [ClimbRepository.h](#).

Referenced by [ClimbRepository::insertClimb\(\)](#), [AnalysisWorker::processTask\(\)](#), and [ClimbRepository::updateClimb\(\)](#).

The documentation for this struct was generated from the following file:

- `src/database/ClimbRepository.h`

## 7.34 ClimbRepository Class Reference

Repository for climb record persistence.

```
#include <ClimbRepository.h>
```

### Public Member Functions

- [ClimbRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- int [insertClimb](#) (const [ClimbRecord](#) &climb)  
*Inserts a new climb record into the database.*
- bool [updateClimb](#) (const [ClimbRecord](#) &climb)  
*Updates an existing climb record.*
- bool [softDeleteClimb](#) (int climbId)  
*Soft-deletes a climb (marks deleted=1, locked=1).*
- bool [hardDeleteClimb](#) (int climbId)  
*Hard-deletes a climb row (permanent removal).*
- bool [deleteClimbsForActivity](#) (int activityId)  
*Deletes all climbs for an activity (hard delete).*

- [ClimbRecord](#) `getClimb` (int climbId)  
*Retrieves a single climb by ID.*
- `QList< ClimbRecord > climbsForActivity` (int activityId)  
*Lists all non-deleted climbs for an activity.*
- `QList< ClimbRecord > deletedClimbsForActivity` (int activityId)  
*Lists all deleted climbs for an activity.*
- bool `hasClimbs` (int activityId)  
*Checks if non-deleted climbs exist for an activity.*
- bool `hasClimbsEver` (int activityId)  
*Checks if analysis has been run for an activity.*
- bool `hasLockedClimbs` (int activityId)  
*Checks if any locked non-deleted climbs exist.*

#### Private Attributes

- `QSqlDatabase & m_db`

### 7.34.1 Detailed Description

Repository for climb record persistence.  
Provides CRUD operations and queries for climbs in the database.  
Definition at line 93 of file [ClimbRepository.h](#).

### 7.34.2 Constructor & Destructor Documentation

#### 7.34.2.1 ClimbRepository()

```
ClimbRepository::ClimbRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

#### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 55 of file [ClimbRepository.cpp](#).

### 7.34.3 Member Function Documentation

#### 7.34.3.1 climbsForActivity()

```
QList< ClimbRecord > ClimbRepository::climbsForActivity (
    int activityId)
```

Lists all non-deleted climbs for an activity.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

#### Returns

List of climb records.

Definition at line 187 of file [ClimbRepository.cpp](#).

**7.34.3.2 deleteClimbsForActivity()**

```
bool ClimbRepository::deleteClimbsForActivity (
    int activityId)
```

Deletes all climbs for an activity (hard delete).

**Parameters**

|                   |              |
|-------------------|--------------|
| <i>activityId</i> | Activity ID. |
|-------------------|--------------|

**Returns**

True if successful.

Definition at line 169 of file [ClimbRepository.cpp](#).

**7.34.3.3 deletedClimbsForActivity()**

```
QList< ClimbRecord > ClimbRepository::deletedClimbsForActivity (
    int activityId)
```

Lists all deleted climbs for an activity.

**Parameters**

|                   |              |
|-------------------|--------------|
| <i>activityId</i> | Activity ID. |
|-------------------|--------------|

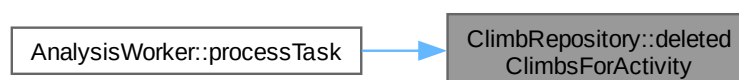
**Returns**

List of deleted climb records.

Definition at line 201 of file [ClimbRepository.cpp](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:

**7.34.3.4 getClimb()**

```
ClimbRecord ClimbRepository::getClimb (
    int climbId)
```

Retrieves a single climb by ID.

**Parameters**

|                |           |
|----------------|-----------|
| <i>climbId</i> | Climb ID. |
|----------------|-----------|

**Returns**

[Climb](#) record.

Definition at line 177 of file [ClimbRepository.cpp](#).

**7.34.3.5 hardDeleteClimb()**

```
bool ClimbRepository::hardDeleteClimb (
    int climbId)
```

Hard-deletes a climb row (permanent removal).

**Parameters**

|                |                                     |
|----------------|-------------------------------------|
| <i>climbId</i> | <a href="#">Climb</a> ID to delete. |
|----------------|-------------------------------------|

**Returns**

True if successful.

Definition at line 161 of file [ClimbRepository.cpp](#).

**7.34.3.6 hasClimbs()**

```
bool ClimbRepository::hasClimbs (
    int activityId)
```

Checks if non-deleted climbs exist for an activity.

**Parameters**

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

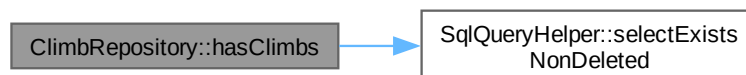
**Returns**

True if any non-deleted climbs found.

Definition at line 215 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsNonDeleted\(\)](#).

Here is the call graph for this function:

**7.34.3.7 hasClimbsEver()**

```
bool ClimbRepository::hasClimbsEver (
    int activityId)
```

Checks if analysis has been run for an activity.

**Parameters**

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

**Returns**

True if any climb records exist (deleted or not).

Definition at line 223 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsEver\(\)](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.34.3.8 hasLockedClimbs()**

```
bool ClimbRepository::hasLockedClimbs (  
    int activityId)
```

Checks if any locked non-deleted climbs exist.

**Parameters**

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

**Returns**

True if locked climbs found.

Definition at line 231 of file [ClimbRepository.cpp](#).

References [SqlQueryHelper::selectExistsLocked\(\)](#).

Here is the call graph for this function:

**7.34.3.9 insertClimb()**

```
int ClimbRepository::insertClimb (
    const ClimbRecord & climb)
```

Inserts a new climb record into the database.

**Parameters**

|              |                                         |
|--------------|-----------------------------------------|
| <i>climb</i> | <a href="#">Climb</a> record to insert. |
|--------------|-----------------------------------------|

**Returns**

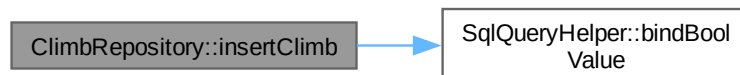
ID of inserted record, or -1 on error.

Definition at line 59 of file [ClimbRepository.cpp](#).

References [ClimbRecord::activityId](#), [ClimbRecord::algorithmVersion](#), [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [ClimbRecord::avgHr](#), [ClimbRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [ClimbRecord::deleted](#), [ClimbRecord::elevationGain](#), [ClimbRecord::endSeconds](#), [ClimbRecord::favorite](#), [ClimbRecord::lengthKm](#), [ClimbRecord::locked](#), [ClimbRecord::name](#), [ClimbRecord::notes](#), [ClimbRecord::np](#), [ClimbRecord::originalEndSeconds](#), [ClimbRecord::originalStartSeconds](#), [ClimbRecord::rating](#), [ClimbRecord::source](#), [ClimbRecord::startSeconds](#), [ClimbRecord::uuid](#), and [ClimbRecord::vam](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 7.34.3.10 softDeleteClimb()

```
bool ClimbRepository::softDeleteClimb (  
    int climbId)
```

Soft-deletes a climb (marks deleted=1, locked=1).

##### Parameters

|                |                                     |
|----------------|-------------------------------------|
| <i>climbId</i> | <a href="#">Climb</a> ID to delete. |
|----------------|-------------------------------------|

##### Returns

True if successful.

Definition at line 153 of file [ClimbRepository.cpp](#).

#### 7.34.3.11 updateClimb()

```
bool ClimbRepository::updateClimb (  
    const ClimbRecord & climb)
```

Updates an existing climb record.

##### Parameters

|              |                                                   |
|--------------|---------------------------------------------------|
| <i>climb</i> | <a href="#">Climb</a> record with updated values. |
|--------------|---------------------------------------------------|

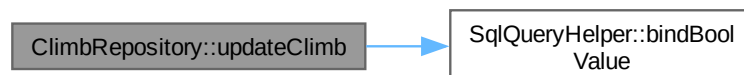
**Returns**

True if update succeeded.

Definition at line 107 of file [ClimbRepository.cpp](#).

References [ClimbRecord::averageGradient](#), [ClimbRecord::avgCadence](#), [ClimbRecord::avgHr](#), [ClimbRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [ClimbRecord::deleted](#), [ClimbRecord::elevationGainM](#), [ClimbRecord::endSeconds](#), [ClimbRecord::favorite](#), [ClimbRecord::id](#), [ClimbRecord::lengthKm](#), [ClimbRecord::locked](#), [ClimbRecord::name](#), [ClimbRecord::notes](#), [ClimbRecord::np](#), [ClimbRecord::rating](#), [ClimbRecord::source](#), [ClimbRecord::startSeconds](#), and [ClimbRecord::vam](#).

Here is the call graph for this function:



## 7.34.4 Member Data Documentation

### 7.34.4.1 m\_db

`QSqlDatabase& ClimbRepository::m_db [private]`

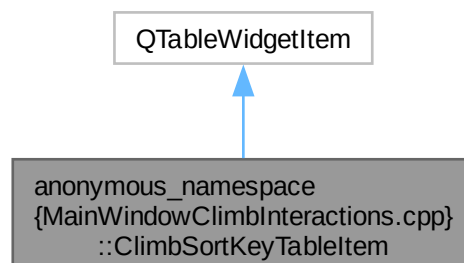
Definition at line 180 of file [ClimbRepository.h](#).

The documentation for this class was generated from the following files:

- `src/database/ClimbRepository.h`
- `src/database/ClimbRepository.cpp`

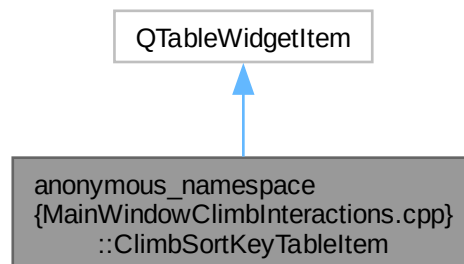
## 7.35 anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem Class Reference

Inheritance diagram for `anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem`:





Collaboration diagram for anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem:



### Public Member Functions

- [ClimbSortKeyTableItem](#) (const QString &text, double sortKey)
- bool [operator<](#) (const QTableWidgetItem &other) const override

### Private Attributes

- double [m\\_sortKey](#) = std::numeric\_limits<double>::quiet\_NaN()

## 7.35.1 Detailed Description

Definition at line 92 of file [MainWindowClimbInteractions.cpp](#).

## 7.35.2 Constructor & Destructor Documentation

### 7.35.2.1 ClimbSortKeyTableItem()

```
anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem::ClimbSortKeyTableItem (
    const QString & text,
    double sortKey) [inline], [explicit]
```

Definition at line 95 of file [MainWindowClimbInteractions.cpp](#).

## 7.35.3 Member Function Documentation

### 7.35.3.1 operator<()

```
bool anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 100 of file [MainWindowClimbInteractions.cpp](#).

## 7.35.4 Member Data Documentation

### 7.35.4.1 m\_sortKey

```
double anonymous_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem::m_sortKey
= std::numeric_limits<double>::quiet_NaN() [private]
```

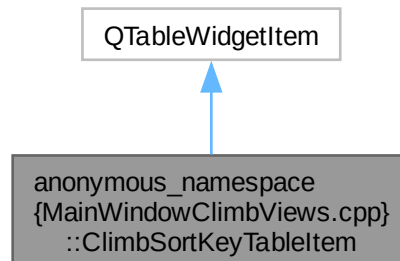
Definition at line 119 of file [MainWindowClimbInteractions.cpp](#).

The documentation for this class was generated from the following file:

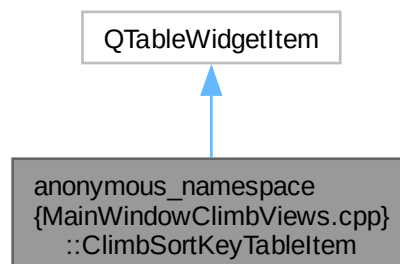
- [src/gui/MainWindowClimbInteractions.cpp](#)

## 7.36 anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem Class Reference

Inheritance diagram for anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem:



Collaboration diagram for anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem:



### Public Member Functions

- [ClimbSortKeyTableItem](#) (const QString &text, double sortKey)
- bool [operator<](#) (const QTableWidgetItem &other) const override

### Private Attributes

- double [m\\_sortKey](#) = std::numeric\_limits<double>::quiet\_NaN()

### 7.36.1 Detailed Description

Definition at line 89 of file [MainWindowClimbViews.cpp](#).

### 7.36.2 Constructor & Destructor Documentation

#### 7.36.2.1 ClimbSortKeyTableItem()

```
anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem::ClimbSortKeyTableItem (
    const QString & text,
```

```
double sortKey) [inline], [explicit]
```

Definition at line 92 of file [MainWindowClimbViews.cpp](#).

### 7.36.3 Member Function Documentation

#### 7.36.3.1 operator<()

```
bool anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem::operator< (
    const QTableWidgetItem & other) const [inline], [override]
```

Definition at line 97 of file [MainWindowClimbViews.cpp](#).

### 7.36.4 Member Data Documentation

#### 7.36.4.1 m\_sortKey

```
double anonymous_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem::m_sortKey = std::numeric_limits<double>::quiet_NaN() [private]
```

Definition at line 116 of file [MainWindowClimbViews.cpp](#).

The documentation for this class was generated from the following file:

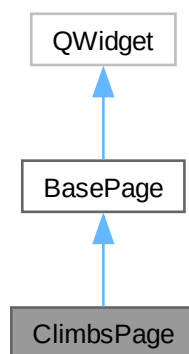
- [src/gui/MainWindowClimbViews.cpp](#)

## 7.37 ClimbsPage Class Reference

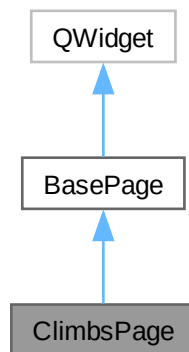
Top-level page for climb detection and analysis.

```
#include <ClimbsPage.h>
```

Inheritance diagram for ClimbsPage:



Collaboration diagram for ClimbsPage:



### Signals

- void **addRequested** ()
- void **editRequested** ()
- void **mergeRequested** ()
- void **deleteRequested** ()
- void **undoRequested** ()
- void **redoRequested** ()

### Public Member Functions

- [ClimbsPage](#) (QWidget \*contentWidget, QWidget \*parent=nullptr)  
*Constructs the Climbs page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

### Private Attributes

- QToolBar \* [m\\_toolbar](#) = nullptr

## 7.37.1 Detailed Description

Top-level page for climb detection and analysis.

Hosts the climb altitude chart (with boundary-editing enabled), the climbs table, and the per-climb statistics summary.

Toolbar actions: Add, Edit, Merge, Delete, Undo, Redo.

Definition at line 18 of file [ClimbsPage.h](#).

## 7.37.2 Constructor & Destructor Documentation

### 7.37.2.1 ClimbsPage()

```
ClimbsPage::ClimbsPage (
    QWidget * contentWidget,
    QWidget * parent = nullptr) [explicit]
```

Constructs the Climbs page.

#### Parameters

|                      |                                            |
|----------------------|--------------------------------------------|
| <i>contentWidget</i> | Pre-assembled climb charts + table widget. |
| <i>parent</i>        | Parent widget.                             |

Adds a climb-context toolbar above the main climb content area.

#### Parameters

|                      |                                                                           |
|----------------------|---------------------------------------------------------------------------|
| <i>contentWidget</i> | Pre-assembled splitter with climb charts and the recognised-climbs table. |
| <i>parent</i>        | Parent widget.                                                            |

Definition at line 19 of file [ClimbsPage.cpp](#).

## 7.37.3 Member Data Documentation

### 7.37.3.1 m\_toolbar

```
QToolBar* ClimbsPage::m_toolbar = nullptr [private]
```

Definition at line 40 of file [ClimbsPage.h](#).

The documentation for this class was generated from the following files:

- [src/gui/pages/ClimbsPage.h](#)
- [src/gui/pages/ClimbsPage.cpp](#)

## 7.38 ColorContext Struct Reference

Context parameters for determining zone colors based on athlete metrics.

```
#include <ZoneDefinition.h>
```

#### Public Attributes

- int [ftp](#) = 250  
*Functional threshold power in watts.*
- int [heartRateMax](#) = 190  
*Maximum heart rate in bpm.*
- int [heartRateRest](#) = 60  
*Resting heart rate in bpm.*
- int [thresholdHeartRate](#) = 0  
*Heart rate threshold in bpm (for HR zones).*
- double [altitudeMin](#) = 0.0  
*Minimum altitude in meters (for altitude coloring).*
- double [altitudeMax](#) = 0.0  
*Maximum altitude in meters (for altitude coloring).*
- bool [hasAltitudeRange](#) = false  
*Whether altitude range is valid for coloring.*

### 7.38.1 Detailed Description

Context parameters for determining zone colors based on athlete metrics.  
Definition at line 59 of file [ZoneDefinition.h](#).

### 7.38.2 Member Data Documentation

#### 7.38.2.1 altitudeMax

```
double ColorContext::altitudeMax = 0.0
```

Maximum altitude in meters (for altitude coloring).

Definition at line 77 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

#### 7.38.2.2 altitudeMin

```
double ColorContext::altitudeMin = 0.0
```

Minimum altitude in meters (for altitude coloring).

Definition at line 74 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

#### 7.38.2.3 ftp

```
int ColorContext::ftp = 250
```

Functional threshold power in watts.

Definition at line 62 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

#### 7.38.2.4 hasAltitudeRange

```
bool ColorContext::hasAltitudeRange = false
```

Whether altitude range is valid for coloring.

Definition at line 80 of file [ZoneDefinition.h](#).

#### 7.38.2.5 heartRateMax

```
int ColorContext::heartRateMax = 190
```

Maximum heart rate in bpm.

Definition at line 65 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

#### 7.38.2.6 heartRateRest

```
int ColorContext::heartRateRest = 60
```

Resting heart rate in bpm.

Definition at line 68 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

#### 7.38.2.7 thresholdHeartRate

```
int ColorContext::thresholdHeartRate = 0
```

Heart rate threshold in bpm (for HR zones).

Definition at line 71 of file [ZoneDefinition.h](#).

Referenced by [ZoneCalculator::zonesForMetric\(\)](#).

The documentation for this struct was generated from the following file:

- [src/core/zones/ZoneDefinition.h](#)

## 7.39 ColorProvider Class Reference

Provides color mapping for athletic metrics.

```
#include <ColorProvider.h>
```

### Static Public Member Functions

- static QColor [colorForMetric](#) (ColorMetric metric, double value, const [ColorContext](#) &context)
- static QColor [colorForPower](#) (double watts, int ftp)
- static QColor [colorForHeartRate](#) (double bpm, int heartRateMax, int heartRateRest=60, int thresholdHeartRate=0)
- static QColor [colorForCadence](#) (double cadence)
- static QColor [colorForSpeed](#) (double speedKmh)
- static QColor [colorForAltitude](#) (double altitude, double minAltitude, double maxAltitude)
- static QColor [colorForValue](#) (double value, const std::vector< [Zone](#) > &zones, const QColor &fallback=QColor("#94a3b8"))

### 7.39.1 Detailed Description

Provides color mapping for athletic metrics.

Generates colors for power, heart rate, cadence, speed, and altitude based on zones.

Definition at line 17 of file [ColorProvider.h](#).

### 7.39.2 Member Function Documentation

#### 7.39.2.1 colorForAltitude()

```
QColor ColorProvider::colorForAltitude (
    double altitude,
    double minAltitude,
    double maxAltitude) [static]
```

Definition at line 75 of file [ColorProvider.cpp](#).

#### 7.39.2.2 colorForCadence()

```
QColor ColorProvider::colorForCadence (
    double cadence) [static]
```

Definition at line 65 of file [ColorProvider.cpp](#).

#### 7.39.2.3 colorForHeartRate()

```
QColor ColorProvider::colorForHeartRate (
    double bpm,
    int heartRateMax,
    int heartRateRest = 60,
    int thresholdHeartRate = 0) [static]
```

Definition at line 55 of file [ColorProvider.cpp](#).

#### 7.39.2.4 colorForMetric()

```
QColor ColorProvider::colorForMetric (
    ColorMetric metric,
    double value,
    const ColorContext & context) [static]
```

Definition at line 23 of file [ColorProvider.cpp](#).

#### 7.39.2.5 colorForPower()

```
QColor ColorProvider::colorForPower (
    double watts,
    int ftp) [static]
```

Definition at line 50 of file [ColorProvider.cpp](#).

#### 7.39.2.6 colorForSpeed()

```
QColor ColorProvider::colorForSpeed (
    double speedKmh) [static]
```

Definition at line 70 of file [ColorProvider.cpp](#).

#### 7.39.2.7 colorForValue()

```
QColor ColorProvider::colorForValue (
    double value,
    const std::vector< Zone > & zones,
    const QColor & fallback = QColor("#94a3b8")) [static]
```

Definition at line 90 of file [ColorProvider.cpp](#).

The documentation for this class was generated from the following files:

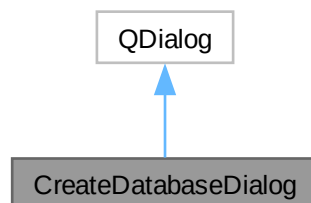
- [src/core/zones/ColorProvider.h](#)
- [src/core/zones/ColorProvider.cpp](#)

## 7.40 CreateDatabaseDialog Class Reference

Dialog for creating a new database.

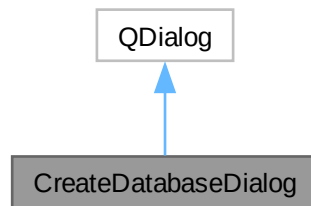
```
#include <CreateDatabaseDialog.h>
```

Inheritance diagram for CreateDatabaseDialog:





Collaboration diagram for CreateDatabaseDialog:



### Public Member Functions

- [CreateDatabaseDialog](#) (const QString &defaultDirectory, QWidget \*parent=nullptr)
- QString [databasePath](#) () const
- bool [shouldCreateDefaultAthlete](#) () const
- QString [defaultAthleteFirstName](#) () const
- QString [defaultAthleteLastName](#) () const

### Private Slots

- void [browseLocation](#) ()
- void [updateAthleteFields](#) ()
- void [accept](#) () override

### Private Member Functions

- QString [normalizedDatabaseName](#) () const

### Private Attributes

- QLineEdit \* [m\\_nameEdit](#) = nullptr
- QLineEdit \* [m\\_locationEdit](#) = nullptr
- QCheckBox \* [m\\_createAthleteCheck](#) = nullptr
- QLineEdit \* [m\\_firstNameEdit](#) = nullptr
- QLineEdit \* [m\\_lastNameEdit](#) = nullptr
- QPushButton \* [m\\_browseButton](#) = nullptr
- QDialogButtonBox \* [m\\_buttonBox](#) = nullptr

## 7.40.1 Detailed Description

Dialog for creating a new database.

Prompts for database name, location, and optionally creates a default athlete.

Definition at line 18 of file [CreateDatabaseDialog.h](#).

## 7.40.2 Constructor & Destructor Documentation

### 7.40.2.1 CreateDatabaseDialog()

```

CreateDatabaseDialog::CreateDatabaseDialog (
    const QString & defaultDirectory,
    QWidget * parent = nullptr) [explicit]
  
```

Definition at line 19 of file [CreateDatabaseDialog.cpp](#).

### 7.40.3 Member Function Documentation

#### 7.40.3.1 accept

```
void CreateDatabaseDialog::accept () [override], [private], [slot]
```

Definition at line 110 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.2 browseLocation

```
void CreateDatabaseDialog::browseLocation () [private], [slot]
```

Definition at line 93 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.3 databasePath()

```
QString CreateDatabaseDialog::databasePath () const
```

Definition at line 69 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.4 defaultAthleteFirstName()

```
QString CreateDatabaseDialog::defaultAthleteFirstName () const
```

Definition at line 83 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.5 defaultAthleteLastName()

```
QString CreateDatabaseDialog::defaultAthleteLastName () const
```

Definition at line 88 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.6 normalizedDatabaseName()

```
QString CreateDatabaseDialog::normalizedDatabaseName () const [private]
```

Definition at line 145 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.7 shouldCreateDefaultAthlete()

```
bool CreateDatabaseDialog::shouldCreateDefaultAthlete () const
```

Definition at line 78 of file [CreateDatabaseDialog.cpp](#).

#### 7.40.3.8 updateAthleteFields

```
void CreateDatabaseDialog::updateAthleteFields () [private], [slot]
```

Definition at line 101 of file [CreateDatabaseDialog.cpp](#).

### 7.40.4 Member Data Documentation

#### 7.40.4.1 m\_browseButton

```
QPushButton* CreateDatabaseDialog::m_browseButton = nullptr [private]
```

Definition at line 46 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.2 m\_buttonBox

```
QDialogButtonBox* CreateDatabaseDialog::m_buttonBox = nullptr [private]
```

Definition at line 47 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.3 m\_createAthleteCheck

```
QCheckBox* CreateDatabaseDialog::m_createAthleteCheck = nullptr [private]
```

Definition at line 43 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.4 m\_firstNameEdit

QLineEdit\* CreateDatabaseDialog::m\_firstNameEdit = nullptr [private]  
 Definition at line 44 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.5 m\_lastNameEdit

QLineEdit\* CreateDatabaseDialog::m\_lastNameEdit = nullptr [private]  
 Definition at line 45 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.6 m\_locationEdit

QLineEdit\* CreateDatabaseDialog::m\_locationEdit = nullptr [private]  
 Definition at line 42 of file [CreateDatabaseDialog.h](#).

#### 7.40.4.7 m\_nameEdit

QLineEdit\* CreateDatabaseDialog::m\_nameEdit = nullptr [private]  
 Definition at line 41 of file [CreateDatabaseDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/CreateDatabaseDialog.h](#)
- [src/gui/CreateDatabaseDialog.cpp](#)

## 7.41 DailyLoadPoint Struct Reference

Daily training stress score value.

```
#include <TrainingLoad.h>
```

### Public Attributes

- double [tss](#) = 0.0  
*Training stress score for day.*

#### 7.41.1 Detailed Description

Daily training stress score value.

Definition at line 13 of file [TrainingLoad.h](#).

#### 7.41.2 Member Data Documentation

##### 7.41.2.1 tss

```
double DailyLoadPoint::tss = 0.0
```

Training stress score for day.

Definition at line 16 of file [TrainingLoad.h](#).

Referenced by [ChartController::updateFitnessChart\(\)](#).

The documentation for this struct was generated from the following file:

- [src/analysis/TrainingLoad.h](#)

## 7.42 DatabaseConnectionGuard Class Reference

RAII guard for temporary SQLite database connections.

```
#include <DatabaseConnectionGuard.h>
```

## Public Member Functions

- [DatabaseConnectionGuard](#) (const QString &connectionName)  
*Constructs guard with connection name.*
- **DatabaseConnectionGuard** (const [DatabaseConnectionGuard](#) &)=delete
- [DatabaseConnectionGuard](#) & **operator=** (const [DatabaseConnectionGuard](#) &)=delete
- QSqlDatabase [database](#) () const  
*Gets the database connection.*

## Private Attributes

- QString [m\\_connectionName](#)

### 7.42.1 Detailed Description

RAII guard for temporary SQLite database connections.

Ensures connections are properly closed and removed from Qt's connection registry, even if exceptions occur during processing.

Definition at line 15 of file [DatabaseConnectionGuard.h](#).

### 7.42.2 Constructor & Destructor Documentation

#### 7.42.2.1 DatabaseConnectionGuard()

```
DatabaseConnectionGuard::DatabaseConnectionGuard (
    const QString & connectionName) [explicit]
```

Constructs guard with connection name.

#### Parameters

|                       |                                  |
|-----------------------|----------------------------------|
| <i>connectionName</i> | Unique name for this connection. |
|-----------------------|----------------------------------|

Definition at line 6 of file [DatabaseConnectionGuard.cpp](#).

#### 7.42.2.2 ~DatabaseConnectionGuard()

```
DatabaseConnectionGuard::~~DatabaseConnectionGuard ()
```

Definition at line 16 of file [DatabaseConnectionGuard.cpp](#).

### 7.42.3 Member Function Documentation

#### 7.42.3.1 database()

```
QSqlDatabase DatabaseConnectionGuard::database () const
```

Gets the database connection.

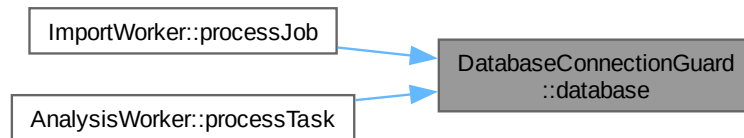
**Returns**

QSqlDatabase for this guard.

Definition at line 33 of file [DatabaseConnectionGuard.cpp](#).

Referenced by [ImportWorker::processJob\(\)](#), and [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:

**7.42.4 Member Data Documentation****7.42.4.1 m\_connectionName**

QString DatabaseConnectionGuard::m\_connectionName [private]

Definition at line 36 of file [DatabaseConnectionGuard.h](#).

The documentation for this class was generated from the following files:

- src/database/DatabaseConnectionGuard.h
- src/database/DatabaseConnectionGuard.cpp

**7.43 DatabaseManager Class Reference**

Manages a single database connection and schema for FitlyzerC.

```
#include <DatabaseManager.h>
```

**Public Member Functions**

- bool [open](#) (const QString &path, QString \*errorOut=nullptr)  
*Opens an existing database file.*
- bool [create](#) (const QString &path, QString \*errorOut=nullptr)  
*Creates a new database file at the specified path.*
- void [close](#) ()  
*Closes the database connection.*
- bool [isOpen](#) () const  
*Checks if database connection is open.*
- QSqlDatabase [database](#) () const  
*Gets the underlying SQL database connection.*
- QString [path](#) () const  
*Gets the database file path.*

**Private Member Functions**

- bool [applySchema](#) (QString \*errorOut)
- bool [upgradeSchema](#) (int fromVersion, QString \*errorOut)
- void [cleanupFailedOpen](#) ()

### Private Attributes

- QString [m\\_path](#)
- QString [m\\_connectionName](#)
- QSqlDatabase [m\\_db](#)

### 7.43.1 Detailed Description

Manages a single database connection and schema for FitlyzerC.

Handles opening, creating, and closing SQLite database connections, applying schema initialization and upgrades as needed.

Definition at line 20 of file [DatabaseManager.h](#).

### 7.43.2 Constructor & Destructor Documentation

#### 7.43.2.1 ~DatabaseManager()

```
DatabaseManager::~DatabaseManager ()
```

Definition at line 14 of file [DatabaseManager.cpp](#).

### 7.43.3 Member Function Documentation

#### 7.43.3.1 applySchema()

```
bool DatabaseManager::applySchema (
    QString * errorOut) [private]
```

Definition at line 21 of file [DatabaseManager.cpp](#).

#### 7.43.3.2 cleanupFailedOpen()

```
void DatabaseManager::cleanupFailedOpen () [private]
```

Definition at line 75 of file [DatabaseManager.cpp](#).

#### 7.43.3.3 close()

```
void DatabaseManager::close ()
```

Closes the database connection.

Definition at line 95 of file [DatabaseManager.cpp](#).

Referenced by [open\(\)](#).

Here is the caller graph for this function:



#### 7.43.3.4 create()

```
bool DatabaseManager::create (
    const QString & path,
    QString * errorOut = nullptr)
```

Creates a new database file at the specified path.

### Parameters

|             |                             |
|-------------|-----------------------------|
| <i>path</i> | Path for new database file. |
|-------------|-----------------------------|

|                 |                                |
|-----------------|--------------------------------|
| <i>errorOut</i> | Optional error message output. |
|-----------------|--------------------------------|

**Returns**

True if successfully created, false otherwise.

Definition at line 89 of file [DatabaseManager.cpp](#).

References [open\(\)](#), and [path\(\)](#).

Here is the call graph for this function:

**7.43.3.5 database()**

```
QSqlDatabase DatabaseManager::database () const
```

Gets the underlying SQL database connection.

**Returns**

`QSqlDatabase` handle (valid only when [isOpen\(\)](#) returns true).

Definition at line 115 of file [DatabaseManager.cpp](#).

**7.43.3.6 isOpen()**

```
bool DatabaseManager::isOpen () const
```

Checks if database connection is open.

**Returns**

True if connected to a database.

Definition at line 110 of file [DatabaseManager.cpp](#).

**7.43.3.7 open()**

```
bool DatabaseManager::open (
    const QString & path,
    QString * errorOut = nullptr)
```

Opens an existing database file.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>path</i>     | Path to database file.         |
| <i>errorOut</i> | Optional error message output. |

**Returns**

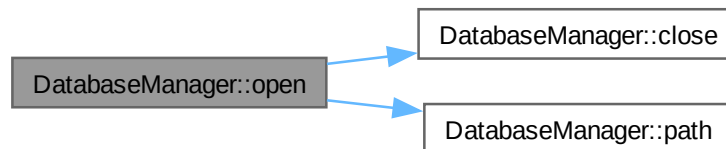
True if successfully opened, false otherwise.

Definition at line 33 of file [DatabaseManager.cpp](#).

References [close\(\)](#), and [path\(\)](#).

Referenced by [create\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.43.3.8 path()**

```
QString DatabaseManager::path () const [inline]
```

Gets the database file path.

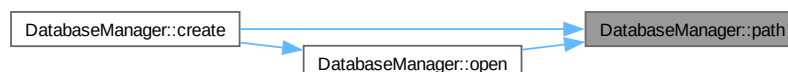
**Returns**

Path to the database file.

Definition at line 63 of file [DatabaseManager.h](#).

Referenced by [create\(\)](#), and [open\(\)](#).

Here is the caller graph for this function:

**7.43.3.9 upgradeSchema()**

```
bool DatabaseManager::upgradeSchema (
    int fromVersion,
    QString * errorOut) [private]
```



Definition at line 26 of file [DatabaseManager.cpp](#).

### 7.43.4 Member Data Documentation

#### 7.43.4.1 m\_connectionName

QString DatabaseManager::m\_connectionName [private]

Definition at line 71 of file [DatabaseManager.h](#).

#### 7.43.4.2 m\_db

QSqlDatabase DatabaseManager::m\_db [private]

Definition at line 72 of file [DatabaseManager.h](#).

#### 7.43.4.3 m\_path

QString DatabaseManager::m\_path [private]

Definition at line 70 of file [DatabaseManager.h](#).

The documentation for this class was generated from the following files:

- src/database/DatabaseManager.h
- src/database/DatabaseManager.cpp

## 7.44 DateFormatter Class Reference

Provides date/time formatting consistent with user settings.

```
#include <DateFormatter.h>
```

### Static Public Member Functions

- static QString [formatDate](#) (const QDate &date)  
*Formats a date according to current settings.*
- static QString [formatDateTime](#) (const QDateTime &dateTime)  
*Formats a date/time according to current settings.*
- static QString [tolsoDate](#) (const QDate &date)  
*Formats a date as ISO 8601.*
- static QString [tolsoDateTime](#) (const QDateTime &dateTime)  
*Formats a date/time as ISO 8601.*
- static QDate [parseDate](#) (const QString &text)  
*Parses a date string.*
- static QString [qtDatePattern](#) ()  
*Gets the Qt date format pattern.*

### 7.44.1 Detailed Description

Provides date/time formatting consistent with user settings.

Definition at line 13 of file [DateFormatter.h](#).

### 7.44.2 Member Function Documentation

#### 7.44.2.1 formatDate()

```
QString DateFormatter::formatDate (
    const QDate & date) [static]
```

Formats a date according to current settings.

**Parameters**

|             |                 |
|-------------|-----------------|
| <i>date</i> | Date to format. |
|-------------|-----------------|

**Returns**

Formatted date string.

Definition at line 30 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Referenced by [formatDateTime\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.44.2.2 formatDateTime()**

```
QString DateFormatter::formatDateTime (
    const QDateTime & dateTime) [static]
```

Formats a date/time according to current settings.

**Parameters**

|                 |                      |
|-----------------|----------------------|
| <i>dateTime</i> | Date/time to format. |
|-----------------|----------------------|

**Returns**

Formatted date/time string.

Definition at line 38 of file [DateFormatter.cpp](#).

References [formatDate\(\)](#).

Here is the call graph for this function:



### 7.44.2.3 parseDate()

```
QDate DateFormatter::parseDate (
    const QString & text) [static]
```

Parses a date string.

#### Parameters

|             |                       |
|-------------|-----------------------|
| <i>text</i> | Date string to parse. |
|-------------|-----------------------|

#### Returns

Parsed QDate.

Definition at line 57 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Here is the call graph for this function:



### 7.44.2.4 qtDatePattern()

```
QString DateFormatter::qtDatePattern () [static]
```

Gets the Qt date format pattern.

#### Returns

Format pattern string.

Definition at line 91 of file [DateFormatter.cpp](#).

References [AppSettings::instance\(\)](#).

Referenced by [AthleteDialog::AthleteDialog\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 7.44.2.5 toIsoDate()

```
QString DateFormatter::toIsoDate (
    const QDate & date) [static]
```

Formats a date as ISO 8601.

##### Parameters

|             |                 |
|-------------|-----------------|
| <i>date</i> | Date to format. |
|-------------|-----------------|

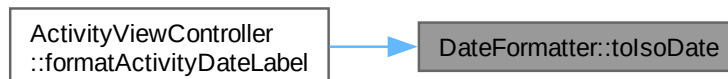
##### Returns

ISO date string (YYYY-MM-DD).

Definition at line 47 of file [DateFormatter.cpp](#).

Referenced by [ActivityViewController::formatActivityDateLabel\(\)](#).

Here is the caller graph for this function:



#### 7.44.2.6 toIsoDateTime()

```
QString DateFormatter::toIsoDateTime (
    const QDateTime & dateTime) [static]
```

Formats a date/time as ISO 8601.

##### Parameters

|                 |                      |
|-----------------|----------------------|
| <i>dateTime</i> | Date/time to format. |
|-----------------|----------------------|

##### Returns

ISO date/time string.

Definition at line 52 of file [DateFormatter.cpp](#).

The documentation for this class was generated from the following files:

- [src/core/settings/DateFormatter.h](#)
- [src/core/settings/DateFormatter.cpp](#)

## 7.45 DuplicateActivityInfo Struct Reference

### Public Attributes

- int [existingActivityId](#) = -1
- QString [existingStartUtc](#)
- double [existingDurationSeconds](#) = 0.0
- QString [newStartUtc](#)
- double [newDurationSeconds](#) = 0.0

### 7.45.1 Detailed Description

Definition at line 37 of file [WorkoutController.h](#).

### 7.45.2 Member Data Documentation

#### 7.45.2.1 existingActivityId

```
int DuplicateActivityInfo::existingActivityId = -1
```

Definition at line 40 of file [WorkoutController.h](#).

#### 7.45.2.2 existingDurationSeconds

```
double DuplicateActivityInfo::existingDurationSeconds = 0.0
```

Definition at line 46 of file [WorkoutController.h](#).

#### 7.45.2.3 existingStartUtc

```
QString DuplicateActivityInfo::existingStartUtc
```

Definition at line 43 of file [WorkoutController.h](#).

#### 7.45.2.4 newDurationSeconds

```
double DuplicateActivityInfo::newDurationSeconds = 0.0
```

Definition at line 52 of file [WorkoutController.h](#).

#### 7.45.2.5 newStartUtc

```
QString DuplicateActivityInfo::newStartUtc
```

Definition at line 49 of file [WorkoutController.h](#).

The documentation for this struct was generated from the following file:

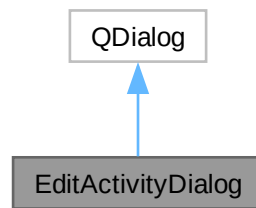
- [src/gui/WorkoutController.h](#)

## 7.46 EditActivityDialog Class Reference

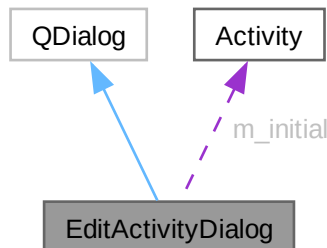
Dialog for editing activity metadata.

```
#include <EditActivityDialog.h>
```

Inheritance diagram for EditActivityDialog:



Collaboration diagram for EditActivityDialog:



### Public Member Functions

- `EditActivityDialog` (const [Activity](#) &activity, QWidget \*parent=nullptr)
- `Activity updatedActivity ()` const

### Private Attributes

- [Activity](#) `m_initial`
- `QDoubleSpinBox * m_temperatureSpin = nullptr`
- `QLineEdit * m_weatherEdit = nullptr`
- `QLineEdit * m_windEdit = nullptr`
- `QSpinBox * m_rpeSpin = nullptr`
- `QSpinBox * m_fatigueSpin = nullptr`
- `QDoubleSpinBox * m_sleepSpin = nullptr`
- `QDoubleSpinBox * m_weightSpin = nullptr`
- `QLineEdit * m_bikeEdit = nullptr`
- `QLineEdit * m_equipmentEdit = nullptr`
- `QTextEdit * m_notesEdit = nullptr`

## 7.46.1 Detailed Description

Dialog for editing activity metadata.

Allows editing of weather, temperature, RPE, fatigue, sleep, weight, bike, equipment, and notes.

Definition at line 21 of file [EditActivityDialog.h](#).

## 7.46.2 Constructor & Destructor Documentation

### 7.46.2.1 EditActivityDialog()

```
EditActivityDialog::EditActivityDialog (  
    const Activity & activity,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 14 of file [EditActivityDialog.cpp](#).

## 7.46.3 Member Function Documentation

### 7.46.3.1 updatedActivity()

```
Activity EditActivityDialog::updatedActivity () const
```

Definition at line 79 of file [EditActivityDialog.cpp](#).

## 7.46.4 Member Data Documentation

### 7.46.4.1 m\_bikeEdit

```
QLineEdit* EditActivityDialog::m_bikeEdit = nullptr [private]
```

Definition at line 40 of file [EditActivityDialog.h](#).

### 7.46.4.2 m\_equipmentEdit

```
QLineEdit* EditActivityDialog::m_equipmentEdit = nullptr [private]
```

Definition at line 41 of file [EditActivityDialog.h](#).

### 7.46.4.3 m\_fatigueSpin

```
QSpinBox* EditActivityDialog::m_fatigueSpin = nullptr [private]
```

Definition at line 37 of file [EditActivityDialog.h](#).

### 7.46.4.4 m\_initial

```
Activity EditActivityDialog::m_initial [private]
```

Definition at line 31 of file [EditActivityDialog.h](#).

### 7.46.4.5 m\_notesEdit

```
QTextEdit* EditActivityDialog::m_notesEdit = nullptr [private]
```

Definition at line 42 of file [EditActivityDialog.h](#).

### 7.46.4.6 m\_rpeSpin

```
QSpinBox* EditActivityDialog::m_rpeSpin = nullptr [private]
```

Definition at line 36 of file [EditActivityDialog.h](#).

### 7.46.4.7 m\_sleepSpin

```
QDoubleSpinBox* EditActivityDialog::m_sleepSpin = nullptr [private]
```

Definition at line 38 of file [EditActivityDialog.h](#).

### 7.46.4.8 m\_temperatureSpin

```
QDoubleSpinBox* EditActivityDialog::m_temperatureSpin = nullptr [private]
```

Definition at line 33 of file [EditActivityDialog.h](#).

### 7.46.4.9 m\_weatherEdit

```
QLineEdit* EditActivityDialog::m_weatherEdit = nullptr [private]
```

Definition at line 34 of file [EditActivityDialog.h](#).

#### 7.46.4.10 m\_weightSpin

`QDoubleSpinBox* EditActivityDialog::m_weightSpin = nullptr [private]`  
Definition at line 39 of file [EditActivityDialog.h](#).

#### 7.46.4.11 m\_windEdit

`QLineEdit* EditActivityDialog::m_windEdit = nullptr [private]`  
Definition at line 35 of file [EditActivityDialog.h](#).

The documentation for this class was generated from the following files:

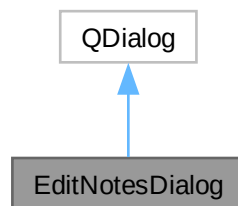
- [src/gui/EditActivityDialog.h](#)
- [src/gui/EditActivityDialog.cpp](#)

## 7.47 EditNotesDialog Class Reference

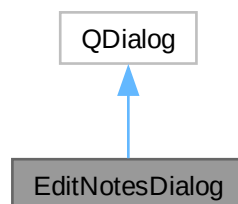
Dialog for editing activity notes and weather.

`#include <EditNotesDialog.h>`

Inheritance diagram for EditNotesDialog:



Collaboration diagram for EditNotesDialog:



### Public Member Functions

- [EditNotesDialog](#) (const QString &notes, const QString &weatherNotes, QWidget \*parent=nullptr)
- QString [notes](#) () const
- QString [weatherNotes](#) () const



### Private Attributes

- QTextEdit \* [m\\_notes](#) = nullptr
- QLineEdit \* [m\\_weather](#) = nullptr

### 7.47.1 Detailed Description

Dialog for editing activity notes and weather.

Definition at line 15 of file [EditNotesDialog.h](#).

### 7.47.2 Constructor & Destructor Documentation

#### 7.47.2.1 EditNotesDialog()

```
EditNotesDialog::EditNotesDialog (  
    const QString & notes,  
    const QString & weatherNotes,  
    QWidget * parent = nullptr) [explicit]
```

Definition at line 13 of file [EditNotesDialog.cpp](#).

### 7.47.3 Member Function Documentation

#### 7.47.3.1 notes()

```
QString EditNotesDialog::notes () const
```

Definition at line 42 of file [EditNotesDialog.cpp](#).

#### 7.47.3.2 weatherNotes()

```
QString EditNotesDialog::weatherNotes () const
```

Definition at line 47 of file [EditNotesDialog.cpp](#).

### 7.47.4 Member Data Documentation

#### 7.47.4.1 m\_notes

```
QTextEdit* EditNotesDialog::m_notes = nullptr [private]
```

Definition at line 28 of file [EditNotesDialog.h](#).

#### 7.47.4.2 m\_weather

```
QLineEdit* EditNotesDialog::m_weather = nullptr [private]
```

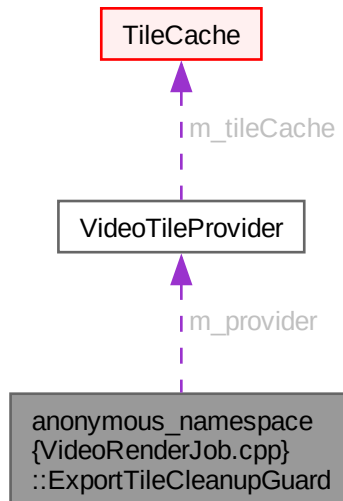
Definition at line 29 of file [EditNotesDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/EditNotesDialog.h](#)
- [src/gui/EditNotesDialog.cpp](#)

## 7.48 anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard Class Reference

Collaboration diagram for anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard:



### Public Member Functions

- [ExportTileCleanupGuard](#) ([VideoTileProvider](#) &provider)

### Private Attributes

- [VideoTileProvider](#) & `m_provider`

### 7.48.1 Detailed Description

Definition at line 53 of file [VideoRenderJob.cpp](#).

### 7.48.2 Constructor & Destructor Documentation

#### 7.48.2.1 ExportTileCleanupGuard()

```
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::ExportTileCleanupGuard (
    VideoTileProvider & provider) [inline], [explicit]
```

Definition at line 56 of file [VideoRenderJob.cpp](#).

#### 7.48.2.2 ~ExportTileCleanupGuard()

```
anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::~~ExportTileCleanupGuard ()
[inline]
```

Definition at line 60 of file [VideoRenderJob.cpp](#).

### 7.48.3 Member Data Documentation

#### 7.48.3.1 m\_provider

`VideoTileProvider& anonymous_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard::m_provider`  
[private]

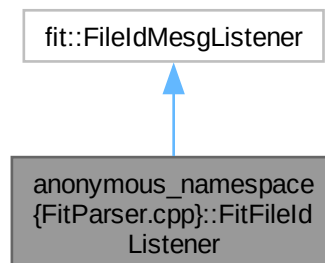
Definition at line 67 of file [VideoRenderJob.cpp](#).

The documentation for this class was generated from the following file:

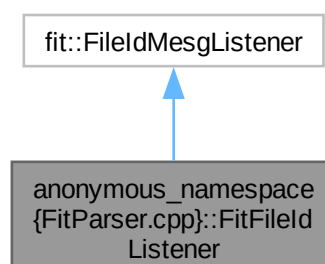
- `src/video/VideoRenderJob.cpp`

## 7.49 anonymous\_namespace{FitParser.cpp}::FitFileIdListener Class Reference

Inheritance diagram for anonymous\_namespace{FitParser.cpp}::FitFileIdListener:



Collaboration diagram for anonymous\_namespace{FitParser.cpp}::FitFileIdListener:



### Public Member Functions

- void [OnMesg](#) (fit::FileIdMesg &mesg) override
- FIT\_DATE\_TIME [timeCreated](#) () const

### Private Attributes

- FIT\_DATE\_TIME [m\\_timeCreated](#) = FIT\_DATE\_TIME\_INVALID

### 7.49.1 Detailed Description

Definition at line 47 of file [FitParser.cpp](#).

### 7.49.2 Member Function Documentation

#### 7.49.2.1 OnMesg()

```
void anonymous_namespace{FitParser.cpp}::FitFileIdListener::OnMesg (
    fit::FileIdMesg & msg) [inline], [override]
```

Definition at line 50 of file [FitParser.cpp](#).

#### 7.49.2.2 timeCreated()

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitFileIdListener::timeCreated () const
[inline]
```

Definition at line 56 of file [FitParser.cpp](#).

### 7.49.3 Member Data Documentation

#### 7.49.3.1 m\_timeCreated

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitFileIdListener::m_timeCreated = FIT_↵
DATE_TIME_INVALID [private]
```

Definition at line 59 of file [FitParser.cpp](#).

The documentation for this class was generated from the following file:

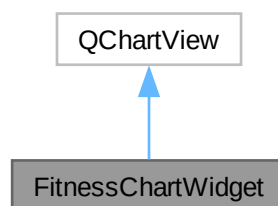
- [src/fit/FitParser.cpp](#)

## 7.50 FitnessChartWidget Class Reference

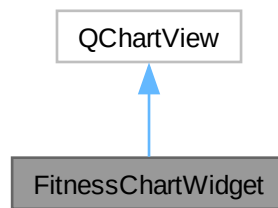
Displays fitness metrics timeline (CTL, ATL, TSB).

```
#include <FitnessChartWidget.h>
```

Inheritance diagram for FitnessChartWidget:



Collaboration diagram for FitnessChartWidget:



### Public Member Functions

- [FitnessChartWidget](#) (`QWidget *parent=nullptr`)  
*Constructs fitness chart widget.*
- void [setTimeline](#) (`const std::vector< FitnessMetricsPoint > &timeline`)  
*Sets fitness metrics timeline to display.*
- void [clearChart](#) ()  
*Clears the chart.*

### Private Attributes

- `QLineSeries * m\_ctlSeries = nullptr`
- `QLineSeries * m\_atlSeries = nullptr`
- `QLineSeries * m\_tsbSeries = nullptr`
- `QValueAxis * m\_axisX = nullptr`
- `QValueAxis * m\_axisY = nullptr`

## 7.50.1 Detailed Description

Displays fitness metrics timeline (CTL, ATL, TSB).

Shows cumulative training load, acute training load, and training stress balance over a date range using a line chart.

Definition at line 21 of file [FitnessChartWidget.h](#).

## 7.50.2 Constructor & Destructor Documentation

### 7.50.2.1 FitnessChartWidget()

```

FitnessChartWidget::FitnessChartWidget (
    QWidget * parent = nullptr) [explicit]

```

Constructs fitness chart widget.

#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 13 of file [FitnessChartWidget.cpp](#).

### 7.50.3 Member Function Documentation

#### 7.50.3.1 clearChart()

```
void FitnessChartWidget::clearChart ()
```

Clears the chart.

Definition at line 93 of file [FitnessChartWidget.cpp](#).

#### 7.50.3.2 setTimeline()

```
void FitnessChartWidget::setTimeline (
    const std::vector< FitnessMetricsPoint > & timeline)
```

Sets fitness metrics timeline to display.

#### Parameters

|                 |                              |
|-----------------|------------------------------|
| <i>timeline</i> | Timeline of fitness metrics. |
|-----------------|------------------------------|

Definition at line 56 of file [FitnessChartWidget.cpp](#).

### 7.50.4 Member Data Documentation

#### 7.50.4.1 m\_atlSeries

```
QLineSeries* FitnessChartWidget::m_atlSeries = nullptr [private]
```

Definition at line 45 of file [FitnessChartWidget.h](#).

#### 7.50.4.2 m\_axisX

```
QValueAxis* FitnessChartWidget::m_axisX = nullptr [private]
```

Definition at line 47 of file [FitnessChartWidget.h](#).

#### 7.50.4.3 m\_axisY

```
QValueAxis* FitnessChartWidget::m_axisY = nullptr [private]
```

Definition at line 48 of file [FitnessChartWidget.h](#).

#### 7.50.4.4 m\_ctlSeries

```
QLineSeries* FitnessChartWidget::m_ctlSeries = nullptr [private]
```

Definition at line 44 of file [FitnessChartWidget.h](#).

#### 7.50.4.5 m\_tsbSeries

```
QLineSeries* FitnessChartWidget::m_tsbSeries = nullptr [private]
```

Definition at line 46 of file [FitnessChartWidget.h](#).

The documentation for this class was generated from the following files:

- [src/charts/FitnessChartWidget.h](#)
- [src/charts/FitnessChartWidget.cpp](#)

## 7.51 FitnessMetricsPoint Struct Reference

Fitness metrics for a single day.

```
#include <TrainingLoad.h>
```

### Public Attributes

- double `atl` = 0.0  
*Acute Training Load (7-day EWMA).*
- double `ctl` = 0.0  
*Chronic Training Load (42-day EWMA).*
- double `tsb` = 0.0  
*Training Stress Balance (ctl - atl).*

### 7.51.1 Detailed Description

Fitness metrics for a single day.

Represents Acute Training Load (ATL), Chronic Training Load (CTL), and Training Stress Balance (TSB) calculated from Training Stress Score.

Definition at line 25 of file [TrainingLoad.h](#).

### 7.51.2 Member Data Documentation

#### 7.51.2.1 `atl`

```
double FitnessMetricsPoint::atl = 0.0
```

Acute Training Load (7-day EWMA).

Definition at line 28 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

#### 7.51.2.2 `ctl`

```
double FitnessMetricsPoint::ctl = 0.0
```

Chronic Training Load (42-day EWMA).

Definition at line 31 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

#### 7.51.2.3 `tsb`

```
double FitnessMetricsPoint::tsb = 0.0
```

Training Stress Balance (ctl - atl).

Definition at line 34 of file [TrainingLoad.h](#).

Referenced by [TrainingLoad::fitnessTimeline\(\)](#).

The documentation for this struct was generated from the following file:

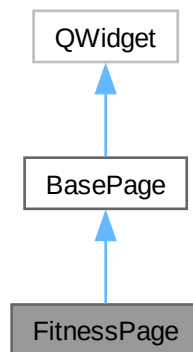
- `src/analysis/TrainingLoad.h`

## 7.52 FitnessPage Class Reference

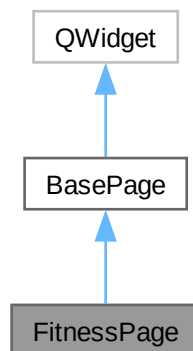
Top-level page for fitness tracking.

```
#include <FitnessPage.h>
```

Inheritance diagram for `FitnessPage`:



Collaboration diagram for `FitnessPage`:



### Public Member Functions

- [FitnessPage](#) ([FitnessChartWidget](#) \*fitnessChart, [QWidget](#) \*parent=nullptr)  
*Constructs the Fitness page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) ([QWidget](#) \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

#### 7.52.1 Detailed Description

Top-level page for fitness tracking.

Displays CTL (Chronic Training Load), ATL (Acute Training Load), TSB (Training Stress Balance), and FTP history.



Future additions: Readiness score, fatigue/recovery estimates, training recommendations.  
 Definition at line 18 of file [FitnessPage.h](#).

## 7.52.2 Constructor & Destructor Documentation

### 7.52.2.1 FitnessPage()

```

FitnessPage::FitnessPage (
    FitnessChartWidget * fitnessChart,
    QWidget * parent = nullptr) [explicit]

```

Constructs the Fitness page.

#### Parameters

|                     |                                                                    |
|---------------------|--------------------------------------------------------------------|
| <i>fitnessChart</i> | The <a href="#">FitnessChartWidget</a> (ownership transfers here). |
| <i>parent</i>       | Parent widget.                                                     |
| <i>fitnessChart</i> | <a href="#">FitnessChartWidget</a> to embed in this page.          |
| <i>parent</i>       | Parent widget.                                                     |

Definition at line 14 of file [FitnessPage.cpp](#).

The documentation for this class was generated from the following files:

- [src/gui/pages/FitnessPage.h](#)
- [src/gui/pages/FitnessPage.cpp](#)

## 7.53 FitParser Class Reference

Parses FIT files and extracts activity recording data.

```
#include <FitParser.h>
```

### Public Member Functions

- [RideData load](#) (const QString &fileName)  
*Loads activity data from a FIT file.*

### Static Private Member Functions

- static double **semicirclesToDegrees** (long value)

### 7.53.1 Detailed Description

Parses FIT files and extracts activity recording data.

Definition at line 13 of file [FitParser.h](#).

## 7.53.2 Member Function Documentation

### 7.53.2.1 load()

```

RideData FitParser::load (
    const QString & fileName)

```

Loads activity data from a FIT file.

#### Parameters

|                 |                    |
|-----------------|--------------------|
| <i>fileName</i> | Path to .fit file. |
|-----------------|--------------------|

### Returns

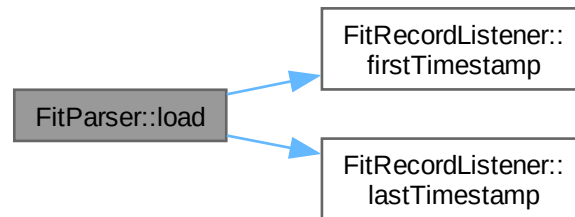
[RideData](#) with parsed activity records.

Definition at line 124 of file [FitParser.cpp](#).

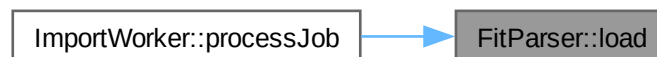
References [RideData::activityEndTimeUtc](#), [RideData::activityStartTimeUtc](#), [RideData::fileTimestampUtc](#), [FitRecordListener::firstTimestamp](#), [FitRecordListener::lastTimestamp\(\)](#), and [RideData::records](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

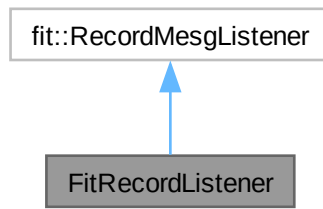
- `src/fit/FitParser.h`
- `src/fit/FitParser.cpp`

## 7.54 FitRecordListener Class Reference

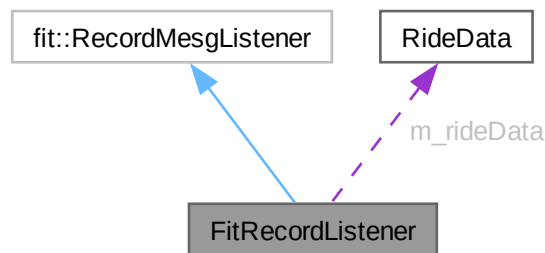
Listener for FIT record messages during parsing.

```
#include <FitRecordListener.h>
```

Inheritance diagram for FitRecordListener:



Collaboration diagram for FitRecordListener:



### Public Member Functions

- [FitRecordListener](#) ([RideData](#) &rideData)  
*Constructs listener with reference to ride data accumulator.*
- void [OnMesg](#) (fit::RecordMesg &mesg) override  
*Processes a single FIT record message.*
- FIT\_DATE\_TIME [firstTimestamp](#) () const  
*Gets the first timestamp from records.*
- FIT\_DATE\_TIME [lastTimestamp](#) () const  
*Gets the last timestamp from records.*

### Private Attributes

- [RideData](#) & [m\\_rideData](#)
- FIT\_DATE\_TIME [m\\_firstTimestamp](#) = FIT\_DATE\_TIME\_INVALID
- FIT\_DATE\_TIME [m\\_lastTimestamp](#) = FIT\_DATE\_TIME\_INVALID

#### 7.54.1 Detailed Description

Listener for FIT record messages during parsing.

Processes individual FIT record messages and accumulates sample data into [RideData](#).

Definition at line 15 of file [FitRecordListener.h](#).

## 7.54.2 Constructor & Destructor Documentation

### 7.54.2.1 FitRecordListener()

```
FitRecordListener::FitRecordListener (
    RideData & rideData) [explicit]
```

Constructs listener with reference to ride data accumulator.

#### Parameters

|                 |                                                           |
|-----------------|-----------------------------------------------------------|
| <i>rideData</i> | Reference to <a href="#">RideData</a> object to populate. |
|-----------------|-----------------------------------------------------------|

Definition at line 8 of file [FitRecordListener.cpp](#).

## 7.54.3 Member Function Documentation

### 7.54.3.1 firstTimestamp()

```
FIT_DATE_TIME FitRecordListener::firstTimestamp () const [inline]
```

Gets the first timestamp from records.

#### Returns

First timestamp value.

Definition at line 37 of file [FitRecordListener.h](#).

Referenced by [FitParser::load\(\)](#).

Here is the caller graph for this function:



### 7.54.3.2 lastTimestamp()

```
FIT_DATE_TIME FitRecordListener::lastTimestamp () const [inline]
```

Gets the last timestamp from records.

#### Returns

Last timestamp value.

Definition at line 43 of file [FitRecordListener.h](#).

Referenced by [FitParser::load\(\)](#).

Here is the caller graph for this function:



### 7.54.3.3 OnMesg()

```
void FitRecordListener::OnMesg (
    fit::RecordMesg & mesg) [override]
```

Processes a single FIT record message.

#### Parameters

|             |                     |
|-------------|---------------------|
| <i>mesg</i> | FIT record message. |
|-------------|---------------------|

Definition at line 14 of file [FitRecordListener.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [RideRecord::elapsedSeconds](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [RideRecord::latitude](#), [RideRecord::longitude](#), [RideRecord::power](#), and [RideRecord::speed](#).

## 7.54.4 Member Data Documentation

### 7.54.4.1 m\_firstTimestamp

```
FIT_DATE_TIME FitRecordListener::m_firstTimestamp = FIT_DATE_TIME_INVALID [private]
```

Definition at line 48 of file [FitRecordListener.h](#).

### 7.54.4.2 m\_lastTimestamp

```
FIT_DATE_TIME FitRecordListener::m_lastTimestamp = FIT_DATE_TIME_INVALID [private]
```

Definition at line 49 of file [FitRecordListener.h](#).

### 7.54.4.3 m\_rideData

```
RideData& FitRecordListener::m_rideData [private]
```

Definition at line 46 of file [FitRecordListener.h](#).

The documentation for this class was generated from the following files:

- [src/fit/FitRecordListener.h](#)
- [src/fit/FitRecordListener.cpp](#)

## 7.55 MapFitMath::FitResult Struct Reference

Result of fitting GPS bounds to map viewport.

```
#include <MapFitMath.h>
```

#### Public Attributes

- double [minLat](#) = 0.0  
*Minimum latitude of track bounds.*
- double [maxLat](#) = 0.0  
*Maximum latitude of track bounds.*
- double [minLon](#) = 0.0  
*Minimum longitude of track bounds.*
- double [maxLon](#) = 0.0  
*Maximum longitude of track bounds.*
- double [centerLat](#) = 0.0  
*Center latitude of track bounds.*
- double [centerLon](#) = 0.0  
*Center longitude of track bounds.*
- double [lonSpanDegrees](#) = 0.0  
*Longitude span in degrees for centering calculation.*

- int `zoom` = 1

*Recommended zoom level to fit bounds in viewport.*

### 7.55.1 Detailed Description

Result of fitting GPS bounds to map viewport.

Definition at line 11 of file [MapFitMath.h](#).

### 7.55.2 Member Data Documentation

#### 7.55.2.1 centerLat

```
double MapFitMath::FitResult::centerLat = 0.0
```

Center latitude of track bounds.

Definition at line 26 of file [MapFitMath.h](#).

#### 7.55.2.2 centerLon

```
double MapFitMath::FitResult::centerLon = 0.0
```

Center longitude of track bounds.

Definition at line 29 of file [MapFitMath.h](#).

#### 7.55.2.3 lonSpanDegrees

```
double MapFitMath::FitResult::lonSpanDegrees = 0.0
```

Longitude span in degrees for centering calculation.

Definition at line 32 of file [MapFitMath.h](#).

#### 7.55.2.4 maxLat

```
double MapFitMath::FitResult::maxLat = 0.0
```

Maximum latitude of track bounds.

Definition at line 17 of file [MapFitMath.h](#).

#### 7.55.2.5 maxLon

```
double MapFitMath::FitResult::maxLon = 0.0
```

Maximum longitude of track bounds.

Definition at line 23 of file [MapFitMath.h](#).

#### 7.55.2.6 minLat

```
double MapFitMath::FitResult::minLat = 0.0
```

Minimum latitude of track bounds.

Definition at line 14 of file [MapFitMath.h](#).

#### 7.55.2.7 minLon

```
double MapFitMath::FitResult::minLon = 0.0
```

Minimum longitude of track bounds.

Definition at line 20 of file [MapFitMath.h](#).

#### 7.55.2.8 zoom

```
int MapFitMath::FitResult::zoom = 1
```

Recommended zoom level to fit bounds in viewport.

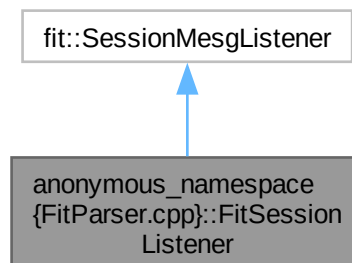
Definition at line 35 of file [MapFitMath.h](#).

The documentation for this struct was generated from the following file:

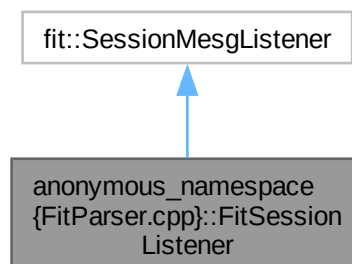
- `src/maps/MapFitMath.h`

## 7.56 anonymous\_namespace{FitParser.cpp}::FitSessionListener Class Reference

Inheritance diagram for anonymous\_namespace{FitParser.cpp}::FitSessionListener:



Collaboration diagram for anonymous\_namespace{FitParser.cpp}::FitSessionListener:



### Public Member Functions

- void [OnMesg](#) (fit::SessionMesg &mesg) override
- FIT\_DATE\_TIME [startTime](#) () const

### Private Attributes

- FIT\_DATE\_TIME [m\\_startTime](#) = FIT\_DATE\_TIME\_INVALID

### 7.56.1 Detailed Description

Definition at line 32 of file [FitParser.cpp](#).

### 7.56.2 Member Function Documentation

#### 7.56.2.1 OnMesg()

```
void anonymous_namespace{FitParser.cpp}::FitSessionListener::OnMesg (
    fit::SessionMesg & mesg) [inline], [override]
```

Definition at line 35 of file [FitParser.cpp](#).

### 7.56.2.2 `startTime()`

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitSessionListener::startTime () const [inline]
```

Definition at line 41 of file [FitParser.cpp](#).

## 7.56.3 Member Data Documentation

### 7.56.3.1 `m_startTime`

```
FIT_DATE_TIME anonymous_namespace{FitParser.cpp}::FitSessionListener::m_startTime = FIT_DATE←
_TIME_INVALID [private]
```

Definition at line 44 of file [FitParser.cpp](#).

The documentation for this class was generated from the following file:

- [src/fit/FitParser.cpp](#)

## 7.57 FtpEntry Struct Reference

Represents a historical FTP entry.

```
#include <AthleteRepository.h>
```

### Public Attributes

- `int id = -1`  
*Unique FTP entry identifier.*
- `int athleteId = -1`  
*Associated athlete ID.*
- `int ftpWatts = 0`  
*FTP value in watts.*
- `QString effectiveFrom`  
*Date FTP becomes effective (ISO 8601 YYYY-MM-DD).*
- `QString notes`  
*Optional notes about FTP change.*

### 7.57.1 Detailed Description

Represents a historical FTP entry.

Definition at line 55 of file [AthleteRepository.h](#).

## 7.57.2 Member Data Documentation

### 7.57.2.1 `athleteId`

```
int FtpEntry::athleteId = -1
```

Associated athlete ID.

Definition at line 61 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

### 7.57.2.2 `effectiveFrom`

```
QString FtpEntry::effectiveFrom
```

Date FTP becomes effective (ISO 8601 YYYY-MM-DD).

Definition at line 67 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).



### 7.57.2.3 ftpWatts

```
int FtpEntry::ftpWatts = 0
```

FTP value in watts.

Definition at line 64 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

### 7.57.2.4 id

```
int FtpEntry::id = -1
```

Unique FTP entry identifier.

Definition at line 58 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getFtpHistory\(\)](#).

### 7.57.2.5 notes

```
QString FtpEntry::notes
```

Optional notes about FTP change.

Definition at line 70 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addFtpEntry\(\)](#), and [AthleteRepository::getFtpHistory\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/AthleteRepository.h](#)

## 7.58 FTPZones Class Reference

Analyzes time spent in FTP-based power zones.

```
#include <FTPZones.h>
```

### Public Types

- using [ZoneArray](#) = std::array<[PowerZone](#), [kZoneCount](#)>  
*Array container type for zone definitions.*

### Static Public Member Functions

- static [ZoneArray](#) [defaultZones](#) ()  
*Returns the default power zone definitions.*
- static [ZoneArray](#) [compute](#) (const [RideData](#) &rideData, double ftp)  
*Computes power zones for an activity.*

### Static Public Attributes

- static constexpr int [kZoneCount](#) = 7  
*Number of FTP power zones (always 7 for standard zones).*

### 7.58.1 Detailed Description

Analyzes time spent in FTP-based power zones.

Definition at line 35 of file [FTPZones.h](#).

### 7.58.2 Member Typedef Documentation

#### 7.58.2.1 ZoneArray

```
using FTPZones::ZoneArray = std::array<PowerZone, kZoneCount>
```

Array container type for zone definitions.

Definition at line 42 of file [FTPZones.h](#).

### 7.58.3 Member Function Documentation

#### 7.58.3.1 compute()

```
FTPZones::ZoneArray FTPZones::compute (
    const RideData & rideData,
    double ftp) [static]
```

Computes power zones for an activity.

##### Parameters

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>rideData</i> | Activity data.                               |
| <i>ftp</i>      | Athlete functional threshold power in watts. |

##### Returns

Array of zones with time spent in each.

Definition at line 22 of file [FTPZones.cpp](#).

References [defaultZones\(\)](#), and [RideData::records](#).

Here is the call graph for this function:



#### 7.58.3.2 defaultZones()

```
FTPZones::ZoneArray FTPZones::defaultZones () [static]
```

Returns the default power zone definitions.

##### Returns

Array of standard FTP-based zones.

Definition at line 9 of file [FTPZones.cpp](#).

Referenced by [compute\(\)](#).

Here is the caller graph for this function:



## 7.58.4 Member Data Documentation

### 7.58.4.1 kZoneCount

```
int FTPZones::kZoneCount = 7 [static], [constexpr]
```

Number of FTP power zones (always 7 for standard zones).

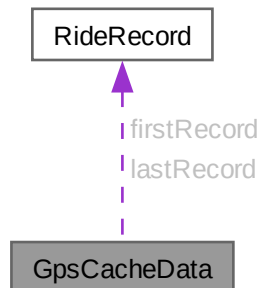
Definition at line 39 of file [FTPZones.h](#).

The documentation for this class was generated from the following files:

- [src/analysis/FTPZones.h](#)
- [src/analysis/FTPZones.cpp](#)

## 7.59 GpsCacheData Struct Reference

Collaboration diagram for GpsCacheData:



### Public Attributes

- `std::vector< const RideRecord * >` [records](#)
- `const RideRecord * firstRecord` = nullptr
- `const RideRecord * lastRecord` = nullptr

### 7.59.1 Detailed Description

Definition at line 9 of file [GpsCacheBuilder.h](#).

## 7.59.2 Member Data Documentation

### 7.59.2.1 firstRecord

```
const RideRecord* GpsCacheData::firstRecord = nullptr
```

Definition at line 12 of file [GpsCacheBuilder.h](#).

### 7.59.2.2 lastRecord

```
const RideRecord* GpsCacheData::lastRecord = nullptr
```

Definition at line 13 of file [GpsCacheBuilder.h](#).

### 7.59.2.3 records

```
std::vector<const RideRecord*> GpsCacheData::records
```

Definition at line 11 of file [GpsCacheBuilder.h](#).

The documentation for this struct was generated from the following file:

- [src/maps/cache/GpsCacheBuilder.h](#)

## 7.60 MainWindow::ImportBatchSummary Struct Reference

### Public Attributes

- int [queued](#) = 0
- int [imported](#) = 0
- int [duplicates](#) = 0
- int [failed](#) = 0
- bool [showResultDialog](#) = false
- QString [sourceLabel](#)
- QStringList [errors](#)

### 7.60.1 Detailed Description

Definition at line 400 of file [MainWindow.h](#).

### 7.60.2 Member Data Documentation

#### 7.60.2.1 duplicates

```
int MainWindow::ImportBatchSummary::duplicates = 0
```

Definition at line 404 of file [MainWindow.h](#).

#### 7.60.2.2 errors

```
QStringList MainWindow::ImportBatchSummary::errors
```

Definition at line 408 of file [MainWindow.h](#).

#### 7.60.2.3 failed

```
int MainWindow::ImportBatchSummary::failed = 0
```

Definition at line 405 of file [MainWindow.h](#).

#### 7.60.2.4 imported

```
int MainWindow::ImportBatchSummary::imported = 0
```

Definition at line 403 of file [MainWindow.h](#).

#### 7.60.2.5 queued

```
int MainWindow::ImportBatchSummary::queued = 0
```

Definition at line 402 of file [MainWindow.h](#).

#### 7.60.2.6 showResultDialog

```
bool MainWindow::ImportBatchSummary::showResultDialog = false
```

Definition at line 406 of file [MainWindow.h](#).

### 7.60.2.7 sourceLabel

QString MainWindow::ImportBatchSummary::sourceLabel

Definition at line 407 of file [MainWindow.h](#).

The documentation for this struct was generated from the following file:

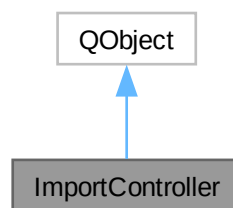
- src/gui/MainWindow.h

## 7.61 ImportController Class Reference

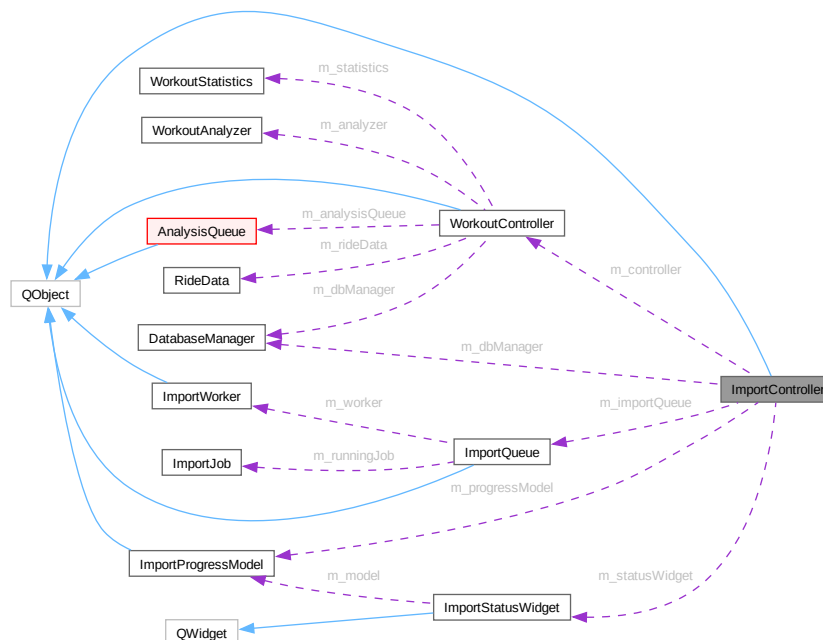
Manages activity import operations and UI.

#include <ImportController.h>

Inheritance diagram for ImportController:



Collaboration diagram for ImportController:



### Signals

- void **filesQueued** (int count, const QString &source)

- *Emitted when files are queued for import.*
- void **activityImported** (int activityId)  
*Emitted when an activity has been successfully imported.*
- void **importBatchesCompleted** ()  
*Emitted when import batches complete.*
- void **importAvailabilityChanged** (bool available)  
*Emitted when import availability changes.*
- void **errorOccurred** (const QString &message)  
*Emitted when an error occurs during import.*

## Public Member Functions

- **ImportController** ([WorkoutController](#) \*controller, [DatabaseManager](#) \*dbManager, QObject \*parent=nullptr)  
*Constructs the import controller.*
- **~ImportController** () override  
*Destructor.*
- void **setImportQueue** ([ImportQueue](#) \*importQueue)  
*Sets the import queue.*
- void **setProgressModel** ([ImportProgressModel](#) \*progressModel)  
*Sets the import progress model.*
- void **setStatusWidget** ([ImportStatusWidget](#) \*statusWidget)  
*Sets the import status widget.*
- void **queueFilesForImport** (const QStringList &filePaths, int athleteId, bool showResultDialog, const QString &sourceLabel)  
*Queues a batch of files for import.*
- bool **canImportActivities** () const  
*Returns whether import operations can proceed.*
- void **updateImportAvailability** ()  
*Updates import availability status based on current conditions.*
- [ImportProgressModel](#) \* **progressModel** () const  
*Gets the import progress model.*
- [ImportQueue](#) \* **importQueue** () const  
*Gets the import queue.*

## Private Member Functions

- void **finalizeBatches** ()  
*Helper to finalize import batch summaries.*
- void **onActivityImportComplete** (int activityId)  
*Helper to handle activity import completion.*

## Private Attributes

- [WorkoutController](#) \* **m\_controller**
- [DatabaseManager](#) \* **m\_dbManager**
- [ImportQueue](#) \* **m\_importQueue** = nullptr
- [ImportProgressModel](#) \* **m\_progressModel** = nullptr
- [ImportStatusWidget](#) \* **m\_statusWidget** = nullptr

### 7.61.1 Detailed Description

Manages activity import operations and UI.

Handles:

- Import queue management
- Import status widget
- Import progress tracking
- Import completion and results
- Import error handling
- File batch import coordination

Definition at line 27 of file [ImportController.h](#).

### 7.61.2 Constructor & Destructor Documentation

#### 7.61.2.1 ImportController()

```
ImportController::ImportController (
    WorkoutController * controller,
    DatabaseManager * dbManager,
    QObject * parent = nullptr) [explicit]
```

Constructs the import controller.

#### Parameters

|                   |                                                                    |
|-------------------|--------------------------------------------------------------------|
| <i>controller</i> | Pointer to <a href="#">WorkoutController</a> for data access.      |
| <i>dbManager</i>  | Pointer to <a href="#">DatabaseManager</a> for import destination. |
| <i>parent</i>     | Parent object.                                                     |

Definition at line 17 of file [ImportController.cpp](#).

### 7.61.3 Member Function Documentation

#### 7.61.3.1 canImportActivities()

```
bool ImportController::canImportActivities () const
```

Returns whether import operations can proceed.

#### Returns

True if database and athlete are ready for import.

Definition at line 80 of file [ImportController.cpp](#).

Referenced by [updateImportAvailability\(\)](#).

Here is the caller graph for this function:



### 7.61.3.2 finalizeBatches()

```
void ImportController::finalizeBatches () [private]
```

Helper to finalize import batch summaries.

Definition at line 119 of file [ImportController.cpp](#).

References [importBatchesCompleted\(\)](#).

Here is the call graph for this function:



### 7.61.3.3 importQueue()

```
ImportQueue * ImportController::importQueue () const
```

Gets the import queue.

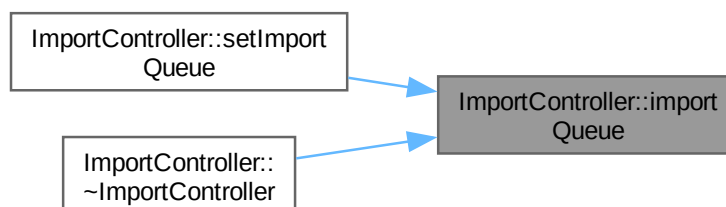
#### Returns

Pointer to [ImportQueue](#), or nullptr if not set.

Definition at line 111 of file [ImportController.cpp](#).

Referenced by [setImportQueue\(\)](#), and [~ImportController\(\)](#).

Here is the caller graph for this function:



### 7.61.3.4 onActivityImportComplete()

```
void ImportController::onActivityImportComplete (
    int activityId) [private]
```

Helper to handle activity import completion.

Definition at line 128 of file [ImportController.cpp](#).

References [activityImported\(\)](#).



Here is the call graph for this function:



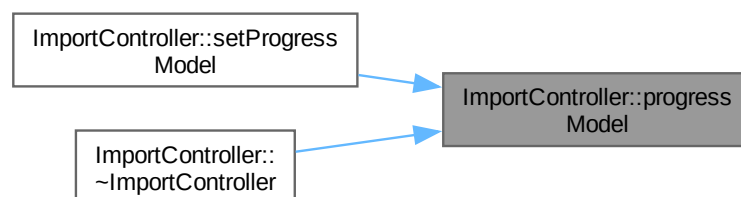
### 7.61.3.5 progressModel()

`ImportProgressModel * ImportController::progressModel () const`  
 Gets the import progress model.

#### Returns

Pointer to [ImportProgressModel](#), or nullptr if not set.

Definition at line 102 of file [ImportController.cpp](#).  
 Referenced by [setProgressModel\(\)](#), and [~ImportController\(\)](#).  
 Here is the caller graph for this function:



### 7.61.3.6 queueFilesForImport()

```

void ImportController::queueFilesForImport (
    const QStringList & filePaths,
    int athleteId,
    bool showResultDialog,
    const QString & sourceLabel)
  
```

Queues a batch of files for import.

#### Parameters

|                         |                                       |
|-------------------------|---------------------------------------|
| <i>filePaths</i>        | List of FIT file paths to import.     |
| <i>athleteId</i>        | ID of target athlete.                 |
| <i>showResultDialog</i> | Whether to show import result dialog. |
| <i>sourceLabel</i>      | Label describing import source.       |

Definition at line 66 of file [ImportController.cpp](#).

References [filesQueued\(\)](#).

Here is the call graph for this function:



### 7.61.3.7 setImportQueue()

```
void ImportController::setImportQueue (
    ImportQueue * importQueue)
```

Sets the import queue.

#### Parameters

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <i>importQueue</i> | Pointer to <a href="#">ImportQueue</a> instance. |
|--------------------|--------------------------------------------------|

Definition at line 36 of file [ImportController.cpp](#).

References [importQueue\(\)](#).

Here is the call graph for this function:



### 7.61.3.8 setProgressModel()

```
void ImportController::setProgressModel (
    ImportProgressModel * progressModel)
```

Sets the import progress model.

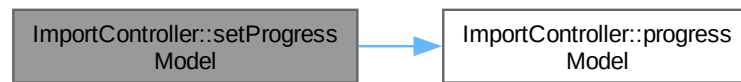
#### Parameters

|                      |                                                          |
|----------------------|----------------------------------------------------------|
| <i>progressModel</i> | Pointer to <a href="#">ImportProgressModel</a> instance. |
|----------------------|----------------------------------------------------------|

Definition at line 45 of file [ImportController.cpp](#).

References [progressModel\(\)](#).

Here is the call graph for this function:



### 7.61.3.9 setStatusWidget()

```
void ImportController::setStatusWidget (
    ImportStatusWidget * statusWidget)
```

Sets the import status widget.

#### Parameters

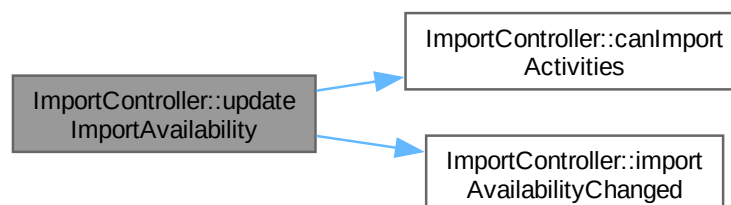
|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>statusWidget</i> | Pointer to <a href="#">ImportStatusWidget</a> instance. |
|---------------------|---------------------------------------------------------|

Definition at line 54 of file [ImportController.cpp](#).

### 7.61.3.10 updateImportAvailability()

```
void ImportController::updateImportAvailability ()
```

Updates import availability status based on current conditions.  
 Definition at line 91 of file [ImportController.cpp](#).  
 References [canImportActivities\(\)](#), and [importAvailabilityChanged\(\)](#).  
 Here is the call graph for this function:



## 7.61.4 Member Data Documentation

### 7.61.4.1 m\_controller

```
WorkoutController* ImportController::m_controller [private]
```

Definition at line 125 of file [ImportController.h](#).

### 7.61.4.2 m\_dbManager

```
DatabaseManager* ImportController::m_dbManager [private]
```

Definition at line 126 of file [ImportController.h](#).

### 7.61.4.3 m\_importQueue

`ImportQueue*` `ImportController::m_importQueue = nullptr` [private]  
 Definition at line 127 of file [ImportController.h](#).

### 7.61.4.4 m\_progressModel

`ImportProgressModel*` `ImportController::m_progressModel = nullptr` [private]  
 Definition at line 128 of file [ImportController.h](#).

### 7.61.4.5 m\_statusWidget

`ImportStatusWidget*` `ImportController::m_statusWidget = nullptr` [private]  
 Definition at line 129 of file [ImportController.h](#).

The documentation for this class was generated from the following files:

- `src/gui/controllers/ImportController.h`
- `src/gui/controllers/ImportController.cpp`

## 7.62 ImportJob Struct Reference

Represents a single background FIT import request.

`#include <ImportJob.h>`

### Public Types

- enum class [State](#) {  
[Queued](#) , [Parsing](#) , [Storing](#) , [Completed](#) ,  
[Failed](#) }  
*Processing stage used for worker/queue state reporting.*

### Public Attributes

- `QString` [jobId](#)  
*Unique import job id.*
- `QString` [batchId](#)  
*Caller-defined batch id for grouped summaries.*
- `QString` [filePath](#)  
*Absolute path to FIT file.*
- `QString` [athleteId](#)  
*Target athlete id.*
- `QString` [importSource](#)  
*Source tag (e.g. manual, drag\_drop, watch\_folder).*
- `State` [state](#) = `State::Queued`
- `double` [progress](#) = 0.0  
*Best-effort progress percentage [0..100].*
- `QString` [errorMessage](#)  
*User-facing failure details when state is Failed.*

### 7.62.1 Detailed Description

Represents a single background FIT import request.

Definition at line 10 of file [ImportJob.h](#).

## 7.62.2 Member Enumeration Documentation

### 7.62.2.1 State

enum class `ImportJob::State` [strong]

Processing stage used for worker/queue state reporting.

#### Enumerator

|           |                                          |
|-----------|------------------------------------------|
| Queued    | Waiting in queue.                        |
| Parsing   | FIT file parsing and duplicate checks.   |
| Storing   | Activity/sample persistence in database. |
| Completed | Successfully imported.                   |
| Failed    | Failed or rejected as duplicate.         |

Definition at line 13 of file [ImportJob.h](#).

## 7.62.3 Member Data Documentation

### 7.62.3.1 athleteId

QString `ImportJob::athleteId`

Target athlete id.

Definition at line 25 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.62.3.2 batchId

QString `ImportJob::batchId`

Caller-defined batch id for grouped summaries.

Definition at line 23 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.62.3.3 errorMessage

QString `ImportJob::errorMessage`

User-facing failure details when state is Failed.

Definition at line 29 of file [ImportJob.h](#).

### 7.62.3.4 filePath

QString `ImportJob::filePath`

Absolute path to FIT file.

Definition at line 24 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.62.3.5 importSource

QString `ImportJob::importSource`

Source tag (e.g. manual, drag\_drop, watch\_folder).

Definition at line 26 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.62.3.6 jobId

QString `ImportJob::jobId`

Unique import job id.

Definition at line 22 of file [ImportJob.h](#).

Referenced by [ImportQueue::enqueue\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.62.3.7 progress

`double ImportJob::progress = 0.0`  
Best-effort progress percentage [0..100].  
Definition at line 28 of file [ImportJob.h](#).  
Referenced by [ImportQueue::enqueue\(\)](#).

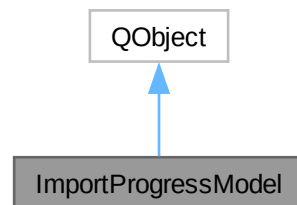
### 7.62.3.8 state

`State ImportJob::state = State::Queued`  
Definition at line 27 of file [ImportJob.h](#).  
The documentation for this struct was generated from the following file:

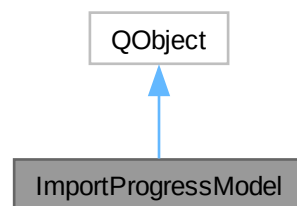
- `src/import/ImportJob.h`

## 7.63 ImportProgressModel Class Reference

UI-facing aggregate state for background import queue activity.  
`#include <ImportProgressModel.h>`  
Inheritance diagram for ImportProgressModel:



Collaboration diagram for ImportProgressModel:



### Signals

- `void changed ()`  
*Emitted whenever visible model state changes.*

## Public Member Functions

- [ImportProgressModel](#) (QObject \*parent=nullptr)
- void [attachQueue](#) ([ImportQueue](#) \*queue)  
*Connects queue signals and starts mirroring import state.*
- int [activeJobs](#) () const
- int [completedJobs](#) () const
- int [failedJobs](#) () const
- double [progress](#) () const
- QString [currentFileName](#) () const
- QString [statusText](#) () const

## Private Attributes

- int [m\\_activeJobs](#) = 0
- int [m\\_completedJobs](#) = 0
- int [m\\_failedJobs](#) = 0
- double [m\\_progress](#) = 0.0
- QString [m\\_currentFileName](#)
- QString [m\\_statusText](#)

### 7.63.1 Detailed Description

UI-facing aggregate state for background import queue activity. Tracks currently active jobs and latest progress text for status widgets. Definition at line 15 of file [ImportProgressModel.h](#).

### 7.63.2 Constructor & Destructor Documentation

#### 7.63.2.1 ImportProgressModel()

```
ImportProgressModel::ImportProgressModel (
    QObject * parent = nullptr) [explicit]
```

Definition at line 9 of file [ImportProgressModel.cpp](#).

### 7.63.3 Member Function Documentation

#### 7.63.3.1 activeJobs()

```
int ImportProgressModel::activeJobs () const [inline]
```

Definition at line 24 of file [ImportProgressModel.h](#).

#### 7.63.3.2 attachQueue()

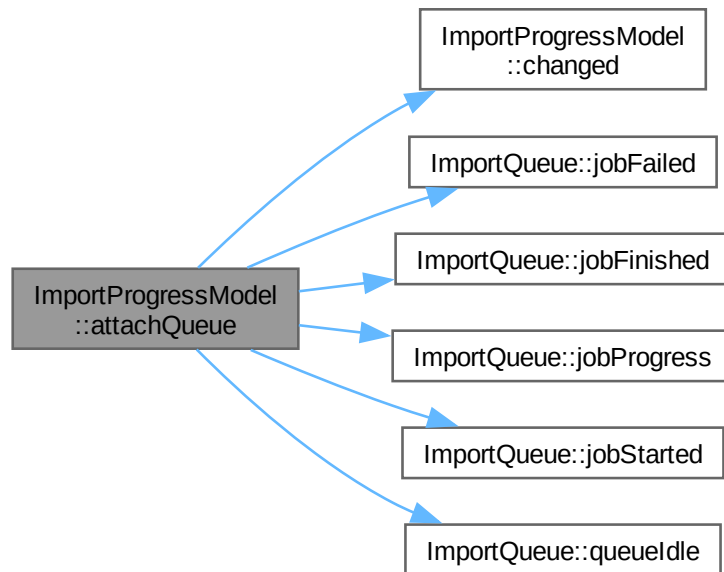
```
void ImportProgressModel::attachQueue (
    ImportQueue * queue)
```

Connects queue signals and starts mirroring import state.

Definition at line 13 of file [ImportProgressModel.cpp](#).

References [changed\(\)](#), [ImportQueue::jobFailed\(\)](#), [ImportQueue::jobFinished\(\)](#), [ImportQueue::jobProgress\(\)](#), [ImportQueue::jobStarted\(\)](#), and [ImportQueue::queueIdle\(\)](#).

Here is the call graph for this function:



#### 7.63.3.3 completedJobs()

```
int ImportProgressModel::completedJobs () const [inline]
```

Definition at line 25 of file [ImportProgressModel.h](#).

#### 7.63.3.4 currentFileName()

```
QString ImportProgressModel::currentFileName () const [inline]
```

Definition at line 28 of file [ImportProgressModel.h](#).

#### 7.63.3.5 failedJobs()

```
int ImportProgressModel::failedJobs () const [inline]
```

Definition at line 26 of file [ImportProgressModel.h](#).

#### 7.63.3.6 progress()

```
double ImportProgressModel::progress () const [inline]
```

Definition at line 27 of file [ImportProgressModel.h](#).

#### 7.63.3.7 statusText()

```
QString ImportProgressModel::statusText () const [inline]
```

Definition at line 29 of file [ImportProgressModel.h](#).

### 7.63.4 Member Data Documentation

#### 7.63.4.1 m\_activeJobs

```
int ImportProgressModel::m_activeJobs = 0 [private]
```

Definition at line 36 of file [ImportProgressModel.h](#).



#### 7.63.4.2 m\_completedJobs

```
int ImportProgressModel::m_completedJobs = 0 [private]
```

Definition at line 37 of file [ImportProgressModel.h](#).

#### 7.63.4.3 m\_currentFileName

```
QString ImportProgressModel::m_currentFileName [private]
```

Definition at line 40 of file [ImportProgressModel.h](#).

#### 7.63.4.4 m\_failedJobs

```
int ImportProgressModel::m_failedJobs = 0 [private]
```

Definition at line 38 of file [ImportProgressModel.h](#).

#### 7.63.4.5 m\_progress

```
double ImportProgressModel::m_progress = 0.0 [private]
```

Definition at line 39 of file [ImportProgressModel.h](#).

#### 7.63.4.6 m\_statusText

```
QString ImportProgressModel::m_statusText [private]
```

Definition at line 41 of file [ImportProgressModel.h](#).

The documentation for this class was generated from the following files:

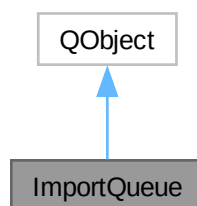
- [src/import/ImportProgressModel.h](#)
- [src/import/ImportProgressModel.cpp](#)

## 7.64 ImportQueue Class Reference

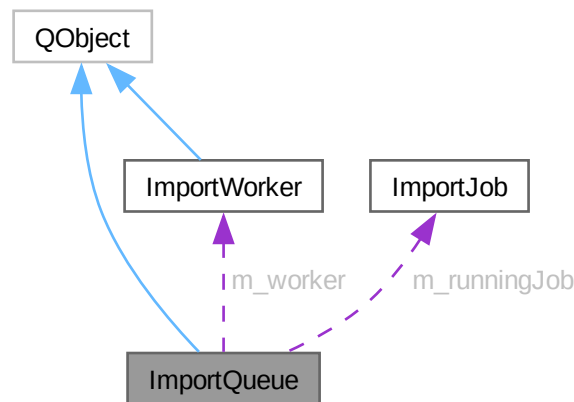
Sequential queue that dispatches background FIT import jobs.

```
#include <ImportQueue.h>
```

Inheritance diagram for ImportQueue:



Collaboration diagram for ImportQueue:



## Signals

- void **jobQueued** (const QString &jobId, const QString &batchId, const QString &filePath, int activeCount)  
*Emitted when a job enters the queue.*
- void **jobStarted** (const QString &jobId, const QString &batchId, const QString &filePath, int activeCount)  
*Emitted when a queued job begins processing.*
- void **jobProgress** (const QString &jobId, const QString &batchId, const QString &filePath, double percentage, const QString &statusText, int activeCount)  
*Emitted with progress updates from the running job.*
- void **jobFinished** (const QString &jobId, const QString &batchId, const QString &filePath, int activityId, int activeCount)  
*Emitted when a job successfully creates an activity.*
- void **jobFailed** (const QString &jobId, const QString &batchId, const QString &filePath, const QString &reason, const QString &errorMessage, int activeCount)  
*Emitted when a job fails or is rejected as duplicate.*
- void **queueStateChanged** (int runningCount, int pendingCount)  
*Emitted whenever running/pending counts change.*
- void **queueIdle** ()  
*Emitted after the last queued/running job is fully processed.*

## Public Member Functions

- **ImportQueue** (QObject \*parent=nullptr)
- void **setDatabasePath** (const QString &path)  
*Sets the SQLite database path used by the worker.*
- void **enqueue** (const QString &filePath, const QString &athleteId, const QString &batchId, const QString &importSource)  
*Enqueues a file import request.*
- int **pendingCount** () const  
*Returns active jobs count (queued + currently running).*
- bool **isBusy** () const  
*Returns true when a job is currently running.*

### Private Slots

- void [onWorkerProgress](#) (const QString &jobId, const QString &batchId, const QString &filePath, double percentage, const QString &statusText)
- void [onWorkerCompleted](#) (const QString &jobId, const QString &batchId, const QString &filePath, int activityId)
- void [onWorkerFailed](#) (const QString &jobId, const QString &batchId, const QString &filePath, const QString &reason, const QString &errorMessage)

### Private Member Functions

- void [processNext](#) ()
- int [activeCountUnsafe](#) () const

### Private Attributes

- QThread \* [m\\_thread](#) = nullptr
- ImportWorker \* [m\\_worker](#) = nullptr
- QQueue< ImportJob > [m\\_queue](#)
- bool [m\\_busy](#) = false
- QString [m\\_dbPath](#)
- ImportJob [m\\_runningJob](#)

## 7.64.1 Detailed Description

Sequential queue that dispatches background FIT import jobs.

Jobs are processed one-at-a-time on a worker thread. The queue emits lifecycle signals for UI updates and batch bookkeeping.

Definition at line 20 of file [ImportQueue.h](#).

## 7.64.2 Constructor & Destructor Documentation

### 7.64.2.1 ImportQueue()

```
ImportQueue::ImportQueue (
    QObject * parent = nullptr) [explicit]
```

Definition at line 10 of file [ImportQueue.cpp](#).

### 7.64.2.2 ~ImportQueue()

```
ImportQueue::~ImportQueue () [override]
```

Definition at line 27 of file [ImportQueue.cpp](#).

## 7.64.3 Member Function Documentation

### 7.64.3.1 activeCountUnsafe()

```
int ImportQueue::activeCountUnsafe () const [private]
```

Definition at line 140 of file [ImportQueue.cpp](#).

### 7.64.3.2 enqueue()

```
void ImportQueue::enqueue (
    const QString & filePath,
    const QString & athleteId,
    const QString & batchId,
    const QString & importSource)
```

Enqueues a file import request.

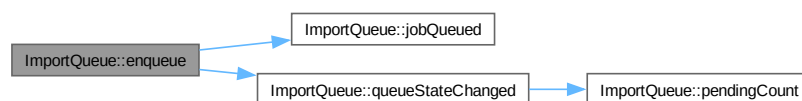
**Parameters**

|                     |                                                   |
|---------------------|---------------------------------------------------|
| <i>filePath</i>     | Absolute file path to a FIT file.                 |
| <i>athleteId</i>    | Target athlete id as string.                      |
| <i>batchId</i>      | Caller-defined batch id for grouped summaries.    |
| <i>importSource</i> | Source metadata saved with the imported activity. |

Definition at line 43 of file [ImportQueue.cpp](#).

References [ImportJob::athleteId](#), [ImportJob::batchId](#), [ImportJob::filePath](#), [ImportJob::importSource](#), [ImportJob::jobId](#), [jobQueued\(\)](#), [ImportJob::progress](#), [ImportJob::Queued](#), and [queueStateChanged\(\)](#).

Here is the call graph for this function:

**7.64.3.3 isBusy()**

```
bool ImportQueue::isBusy () const
```

Returns true when a job is currently running.

Definition at line 71 of file [ImportQueue.cpp](#).

**7.64.3.4 jobFailed**

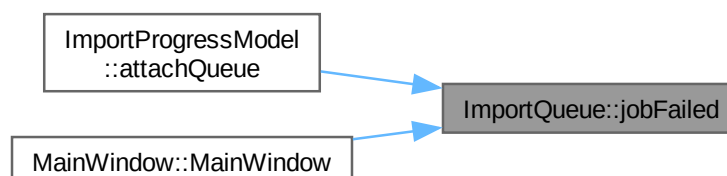
```
void ImportQueue::jobFailed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
    const QString & errorMessage,
    int activeCount) [signal]
```

Emitted when a job fails or is rejected as duplicate.

*reason* is a stable category such as *duplicate*, *time\_overlap*, or *error*; *errorMessage* provides user-facing details.

Referenced by [ImportProgressModel::attachQueue\(\)](#), and [MainWindow::MainWindow\(\)](#).

Here is the caller graph for this function:



**7.64.3.5 onWorkerCompleted**

```
void ImportQueue::onWorkerCompleted (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    int activityId) [private], [slot]
```

Definition at line 85 of file [ImportQueue.cpp](#).

**7.64.3.6 onWorkerFailed**

```
void ImportQueue::onWorkerFailed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
    const QString & errorMessage) [private], [slot]
```

Definition at line 103 of file [ImportQueue.cpp](#).

**7.64.3.7 onWorkerProgress**

```
void ImportQueue::onWorkerProgress (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    double percentage,
    const QString & statusText) [private], [slot]
```

Definition at line 76 of file [ImportQueue.cpp](#).

**7.64.3.8 pendingCount()**

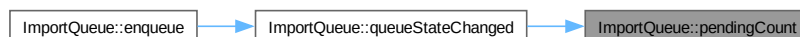
```
int ImportQueue::pendingCount () const
```

Returns active jobs count (queued + currently running).

Definition at line 66 of file [ImportQueue.cpp](#).

Referenced by [queueStateChanged\(\)](#).

Here is the caller graph for this function:

**7.64.3.9 processNext()**

```
void ImportQueue::processNext () [private]
```

Definition at line 122 of file [ImportQueue.cpp](#).

**7.64.3.10 setDatabasePath()**

```
void ImportQueue::setDatabasePath (
    const QString & path)
```

Sets the SQLite database path used by the worker.

Definition at line 34 of file [ImportQueue.cpp](#).

## 7.64.4 Member Data Documentation

### 7.64.4.1 m\_busy

```
bool ImportQueue::m_busy = false [private]
```

Definition at line 116 of file [ImportQueue.h](#).

### 7.64.4.2 m\_dbPath

```
QString ImportQueue::m_dbPath [private]
```

Definition at line 117 of file [ImportQueue.h](#).

### 7.64.4.3 m\_queue

```
QQueue<ImportJob> ImportQueue::m_queue [private]
```

Definition at line 115 of file [ImportQueue.h](#).

### 7.64.4.4 m\_runningJob

```
ImportJob ImportQueue::m_runningJob [private]
```

Definition at line 118 of file [ImportQueue.h](#).

### 7.64.4.5 m\_thread

```
QThread* ImportQueue::m_thread = nullptr [private]
```

Definition at line 113 of file [ImportQueue.h](#).

### 7.64.4.6 m\_worker

```
ImportWorker* ImportQueue::m_worker = nullptr [private]
```

Definition at line 114 of file [ImportQueue.h](#).

The documentation for this class was generated from the following files:

- [src/import/ImportQueue.h](#)
- [src/import/ImportQueue.cpp](#)

## 7.65 ImportRecord Struct Reference

Log entry for an activity import.

```
#include <ImportRepository.h>
```

### Public Attributes

- int [id](#) = -1  
*Unique import log entry ID.*
- QString [fileName](#)  
*Name of imported file.*
- QString [importedAt](#)  
*ISO8601 import timestamp.*
- QString [status](#)  
*Import status (success, duplicate, error).*
- int [activityId](#) = -1  
*ID of imported activity (or -1 if failed).*
- QString [errorMessage](#)  
*Error message if import failed.*

### 7.65.1 Detailed Description

Log entry for an activity import.

Definition at line 14 of file [ImportRepository.h](#).

### 7.65.2 Member Data Documentation

#### 7.65.2.1 activityId

```
int ImportRecord::activityId = -1
```

ID of imported activity (or -1 if failed).

Definition at line 29 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.65.2.2 errorMessage

```
QString ImportRecord::errorMessage
```

Error message if import failed.

Definition at line 32 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.65.2.3 fileName

```
QString ImportRecord::fileName
```

Name of imported file.

Definition at line 20 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.65.2.4 id

```
int ImportRecord::id = -1
```

Unique import log entry ID.

Definition at line 17 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#).

#### 7.65.2.5 importedAt

```
QString ImportRecord::importedAt
```

ISO8601 import timestamp.

Definition at line 23 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), and [ImportRepository::logImport\(\)](#).

#### 7.65.2.6 status

```
QString ImportRecord::status
```

Import status (success, duplicate, error).

Definition at line 26 of file [ImportRepository.h](#).

Referenced by [ImportRepository::getRecentImports\(\)](#), [ImportRepository::logImport\(\)](#), and [ImportWorker::processJob\(\)](#).

The documentation for this struct was generated from the following file:

- [src/database/ImportRepository.h](#)

## 7.66 ImportRepository Class Reference

Repository for import log persistence.

```
#include <ImportRepository.h>
```

## Public Member Functions

- [ImportRepository](#) (QSqlDatabase &db)  
*Constructs repository with database connection.*
- int [logImport](#) (const [ImportRecord](#) &r)  
*Logs an import attempt.*
- QList< [ImportRecord](#) > [getRecentImports](#) (int limit=50)  
*Retrieves recent import log entries.*

## Private Attributes

- QSqlDatabase & [m\\_db](#)

## 7.66.1 Detailed Description

Repository for import log persistence.

Tracks all import attempts including successes, duplicates, and errors.

Definition at line 40 of file [ImportRepository.h](#).

## 7.66.2 Constructor & Destructor Documentation

### 7.66.2.1 ImportRepository()

```
ImportRepository::ImportRepository (
    QSqlDatabase & db) [explicit]
```

Constructs repository with database connection.

#### Parameters

|           |                      |
|-----------|----------------------|
| <i>db</i> | Database connection. |
|-----------|----------------------|

Definition at line 10 of file [ImportRepository.cpp](#).

## 7.66.3 Member Function Documentation

### 7.66.3.1 getRecentImports()

```
QList< ImportRecord > ImportRepository::getRecentImports (
    int limit = 50)
```

Retrieves recent import log entries.

#### Parameters

|              |                                                   |
|--------------|---------------------------------------------------|
| <i>limit</i> | Maximum number of entries to return (default 50). |
|--------------|---------------------------------------------------|

#### Returns

List of import records.

Definition at line 31 of file [ImportRepository.cpp](#).

References [ImportRecord::activityId](#), [ImportRecord::errorMessage](#), [ImportRecord::fileName](#), [ImportRecord::id](#), [ImportRecord::importedAt](#), and [ImportRecord::status](#).

### 7.66.3.2 logImport()

```
int ImportRepository::logImport (
    const ImportRecord & r)
```

Logs an import attempt.



**Parameters**

|          |                       |
|----------|-----------------------|
| <i>r</i> | Import record to log. |
|----------|-----------------------|

**Returns**

ID of log entry, or -1 on error.

Definition at line 14 of file [ImportRepository.cpp](#).

References [ImportRecord::activityId](#), [ImportRecord::errorMessage](#), [ImportRecord::fileName](#), [ImportRecord::importedAt](#), and [ImportRecord::status](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.66.4 Member Data Documentation****7.66.4.1 m\_db**

`QSqlDatabase& ImportRepository::m_db [private]`

Definition at line 64 of file [ImportRepository.h](#).

The documentation for this class was generated from the following files:

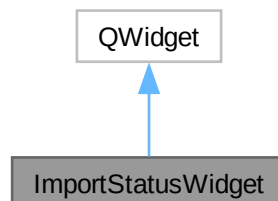
- `src/database/ImportRepository.h`
- `src/database/ImportRepository.cpp`

**7.67 ImportStatusWidget Class Reference**

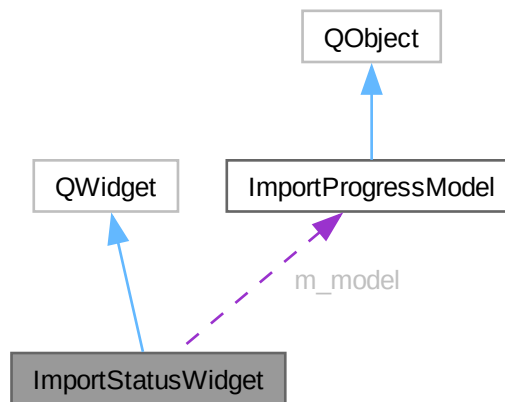
Toolbar widget showing live background import queue progress.

```
#include <ImportStatusWidget.h>
```

Inheritance diagram for `ImportStatusWidget`:



Collaboration diagram for ImportStatusWidget:



### Public Member Functions

- [ImportStatusWidget](#) (QWidget \*parent=nullptr)
- void [setModel](#) (ImportProgressModel \*model)  
*Binds a progress model; disconnects any previously bound model.*

### Private Member Functions

- void [updateFromModel](#) ()

### Private Attributes

- ImportProgressModel \* [m\\_model](#) = nullptr
- QLabel \* [m\\_titleLabel](#) = nullptr
- QLabel \* [m\\_detailLabel](#) = nullptr
- QProgressBar \* [m\\_progressBar](#) = nullptr

## 7.67.1 Detailed Description

Toolbar widget showing live background import queue progress.  
Definition at line 14 of file [ImportStatusWidget.h](#).

## 7.67.2 Constructor & Destructor Documentation

### 7.67.2.1 ImportStatusWidget()

```
ImportStatusWidget::ImportStatusWidget (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 11 of file [ImportStatusWidget.cpp](#).

## 7.67.3 Member Function Documentation

### 7.67.3.1 setModel()

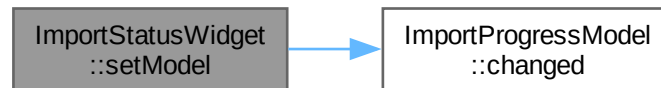
```
void ImportStatusWidget::setModel (
    ImportProgressModel * model)
```

Binds a progress model; disconnects any previously bound model.

Definition at line 36 of file [ImportStatusWidget.cpp](#).

References [ImportProgressModel::changed\(\)](#).

Here is the call graph for this function:



### 7.67.3.2 updateFromModel()

```
void ImportStatusWidget::updateFromModel () [private]
```

Definition at line 51 of file [ImportStatusWidget.cpp](#).

## 7.67.4 Member Data Documentation

### 7.67.4.1 m\_detailLabel

```
QLabel* ImportStatusWidget::m_detailLabel = nullptr [private]
```

Definition at line 28 of file [ImportStatusWidget.h](#).

### 7.67.4.2 m\_model

```
ImportProgressModel* ImportStatusWidget::m_model = nullptr [private]
```

Definition at line 26 of file [ImportStatusWidget.h](#).

### 7.67.4.3 m\_progressBar

```
QProgressBar* ImportStatusWidget::m_progressBar = nullptr [private]
```

Definition at line 29 of file [ImportStatusWidget.h](#).

### 7.67.4.4 m\_titleLabel

```
QLabel* ImportStatusWidget::m_titleLabel = nullptr [private]
```

Definition at line 27 of file [ImportStatusWidget.h](#).

The documentation for this class was generated from the following files:

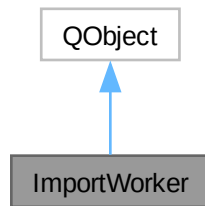
- [src/gui/ImportStatusWidget.h](#)
- [src/gui/ImportStatusWidget.cpp](#)

## 7.68 ImportWorker Class Reference

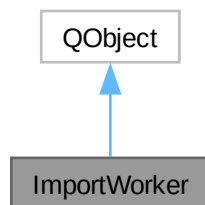
Worker that imports FIT files in a dedicated background thread.

```
#include <ImportWorker.h>
```

Inheritance diagram for ImportWorker:



Collaboration diagram for ImportWorker:



### Public Slots

- void `processJob` (const `ImportJob` &job)  
*Processes one queued import job.*

### Signals

- void **progressChanged** (const `QString` &jobId, const `QString` &batchId, const `QString` &filePath, double percentage, const `QString` &statusText)  
*Progress update for the currently running import job.*
- void **completed** (const `QString` &jobId, const `QString` &batchId, const `QString` &filePath, int activityId)  
*Emitted when an activity was imported successfully.*
- void `failed` (const `QString` &jobId, const `QString` &batchId, const `QString` &filePath, const `QString` &reason, const `QString` &errorMessage)  
*Emitted when import fails or is rejected as duplicate.*

### Public Member Functions

- `ImportWorker` (`QObject` \*parent=nullptr)
- void `setDatabasePath` (const `QString` &path)  
*Sets the SQLite database path used for imports.*

**Private Attributes**

- QString [m\\_dbPath](#)

**7.68.1 Detailed Description**

Worker that imports FIT files in a dedicated background thread.

The worker parses FIT payloads, performs duplicate checks, writes activity and sample rows, and emits progress/↵ final status signals.

Definition at line 16 of file [ImportWorker.h](#).

**7.68.2 Constructor & Destructor Documentation****7.68.2.1 ImportWorker()**

```
ImportWorker::ImportWorker (
    QObject * parent = nullptr) [explicit]
```

Definition at line 35 of file [ImportWorker.cpp](#).

**7.68.3 Member Function Documentation****7.68.3.1 failed**

```
void ImportWorker::failed (
    const QString & jobId,
    const QString & batchId,
    const QString & filePath,
    const QString & reason,
    const QString & errorMessage) [signal]
```

Emitted when import fails or is rejected as duplicate.

reason is a machine-readable category (duplicate, time\_overlap, error) and errorMessage contains details.

Referenced by [processJob\(\)](#).

Here is the caller graph for this function:

**7.68.3.2 processJob**

```
void ImportWorker::processJob (
    const ImportJob & job) [slot]
```

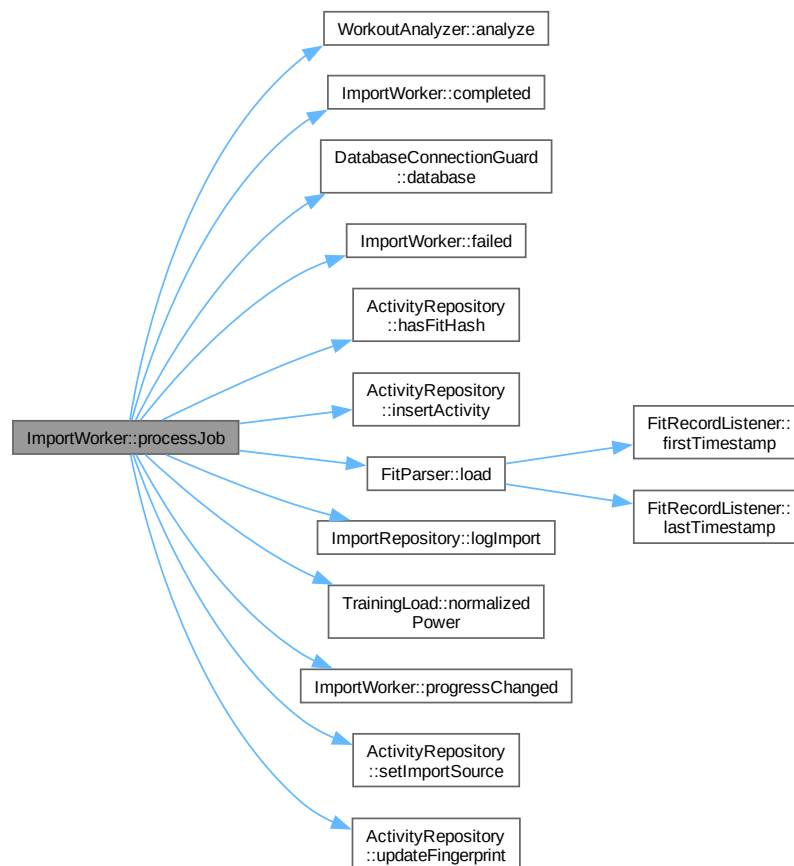
Processes one queued import job.

Definition at line 44 of file [ImportWorker.cpp](#).

References [RideData::activityEndTimeUtc](#), [ImportRecord::activityId](#), [RideData::activityStartTimeUtc](#), [RideRecord::altitude](#), [WorkoutAnalyzer::analyze\(\)](#), [Activity::athleteId](#), [ImportJob::athleteId](#), [WorkoutStatistics::averageCadence](#), [WorkoutStatistics::averageHeartRate](#), [WorkoutStatistics::averagePower](#), [WorkoutStatistics::averageSpeed](#), [Activity::avgCadence](#), [Activity::avgHr](#), [Activity::avgPower](#), [Activity::avgSpeed](#), [ImportJob::batchId](#), [RideRecord::cadence](#), [completed\(\)](#), [DatabaseConnectionGuard::database\(\)](#), [Activity::distanceM](#), [Activity::durationSec](#), [WorkoutStatistics::durationSeconds](#), [RideRecord::elapsedSeconds](#), [Activity::elevationGain](#), [WorkoutStatistics::elevationGainM](#), [Activity::endTime](#), [ImportRecord::errorMessage](#), [failed\(\)](#), [Activity::fileName](#), [ImportRecord::fileName](#), [ImportJob::filePath](#), [Activity::fitHash](#),

[RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [ActivityRepository::hasFitHash\(\)](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [Activity::importedAt](#), [ImportJob::importSource](#), [ActivityRepository::insertActivity\(\)](#), [ImportJob::jobId](#), [RideRecord::latitude](#), [FitParser::load\(\)](#), [ImportRepository::logImport\(\)](#), [RideRecord::longitude](#), [Activity::maxHr](#), [WorkoutStatistics::maximumHeartRate](#), [WorkoutStatistics::maximumPower](#), [Activity::maxPower](#), [Activity::normalizedPower](#), [TrainingLoad::normalizedPower\(\)](#), [RideRecord::power](#), [progressChanged\(\)](#), [RideData::records](#), [ActivityRepository::setImportSource\(\)](#), [RideRecord::speed](#), [Activity::sport](#), [Activity::startTime](#), [ImportRecord::status](#), [WorkoutStatistics::totalDistanceKm](#), and [ActivityRepository::updateFingerprint](#)

Here is the call graph for this function:



### 7.68.3.3 setDatabasePath()

```
void ImportWorker::setDatabasePath (
    const QString & path)
```

Sets the SQLite database path used for imports.

Definition at line 39 of file [ImportWorker.cpp](#).

## 7.68.4 Member Data Documentation

### 7.68.4.1 m\_dbPath

```
QString ImportWorker::m_dbPath [private]
```

Definition at line 54 of file [ImportWorker.h](#).

The documentation for this class was generated from the following files:

- `src/import/ImportWorker.h`
- `src/import/ImportWorker.cpp`

## 7.69 Interval Struct Reference

Represents a single work or recovery interval.

```
#include <IntervalDetector.h>
```

### Public Attributes

- double [startSeconds](#) = 0.0  
*Interval start time in seconds from activity start.*
- double [endSeconds](#) = 0.0  
*Interval end time in seconds from activity start.*
- double [durationSeconds](#) = 0.0  
*Interval duration in seconds.*
- double [averagePower](#) = 0.0  
*Average power in watts.*
- double [maximumPower](#) = 0.0  
*Maximum power in watts.*
- double [normalizedPower](#) = 0.0  
*Normalized power in watts.*
- double [averageHeartRate](#) = 0.0  
*Average heart rate in bpm.*
- double [averageCadence](#) = 0.0  
*Average cadence in rpm.*
- double [averageSpeed](#) = 0.0  
*Average speed in km/h.*
- std::string [label](#)  
*Interval type label (Work or Recovery).*

### 7.69.1 Detailed Description

Represents a single work or recovery interval.

Definition at line 14 of file [IntervalDetector.h](#).

### 7.69.2 Member Data Documentation

#### 7.69.2.1 averageCadence

```
double Interval::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 38 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.2 averageHeartRate

```
double Interval::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 35 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.3 averagePower

```
double Interval::averagePower = 0.0
```

Average power in watts.

Definition at line 26 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.4 averageSpeed

```
double Interval::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 41 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.5 durationSeconds

```
double Interval::durationSeconds = 0.0
```

[Interval](#) duration in seconds.

Definition at line 23 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.6 endSeconds

```
double Interval::endSeconds = 0.0
```

[Interval](#) end time in seconds from activity start.

Definition at line 20 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.7 label

```
std::string Interval::label
```

[Interval](#) type label (Work or Recovery).

Definition at line 44 of file [IntervalDetector.h](#).

#### 7.69.2.8 maximumPower

```
double Interval::maximumPower = 0.0
```

Maximum power in watts.

Definition at line 29 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.9 normalizedPower

```
double Interval::normalizedPower = 0.0
```

Normalized power in watts.

Definition at line 32 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

#### 7.69.2.10 startSeconds

```
double Interval::startSeconds = 0.0
```

[Interval](#) start time in seconds from activity start.

Definition at line 17 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::removeOverlappingAndDuplicates\(\)](#).

The documentation for this struct was generated from the following file:

- [src/analysis/IntervalDetector.h](#)

## 7.70 IntervalDetector Class Reference

Detects work and recovery intervals in activity data.

```
#include <IntervalDetector.h>
```

### Public Types

- using [Config](#) = [IntervalDetectorConfig](#)  
*Interval detection configuration structure.*



### Static Public Member Functions

- static `std::vector< Interval > detect` (const [RideData](#) &rideData, const [Config](#) &config=[Config](#){})  
*Detects intervals in activity data.*
- static `std::vector< Interval > removeOverlappingAndDuplicates` (std::vector< [Interval](#) > intervals)  
*Removes duplicate and overlapping intervals.*

### Static Private Member Functions

- static `std::vector< double > smoothPower` (const [RideData](#) &rideData, int window)
- static [Interval](#) `buildInterval` (const [RideData](#) &rideData, int startIdx, int endIdx, const std::string &label)

## 7.70.1 Detailed Description

Detects work and recovery intervals in activity data.

Identifies distinct work intervals (above power threshold) and recovery periods based on configurable FTP-derived thresholds.

Definition at line 74 of file [IntervalDetector.h](#).

## 7.70.2 Member Typedef Documentation

### 7.70.2.1 Config

using [IntervalDetector::Config](#) = [IntervalDetectorConfig](#)

[Interval](#) detection configuration structure.

Definition at line 78 of file [IntervalDetector.h](#).

## 7.70.3 Member Function Documentation

### 7.70.3.1 buildInterval()

```
Interval IntervalDetector::buildInterval (
    const RideData & rideData,
    int startIdx,
    int endIdx,
    const std::string & label) [static], [private]
```

Definition at line 34 of file [IntervalDetector.cpp](#).

### 7.70.3.2 detect()

```
std::vector< Interval > IntervalDetector::detect (
    const RideData & rideData,
    const Config & config = Config{}) [static]
```

Detects intervals in activity data.

#### Parameters

|                 |                                          |
|-----------------|------------------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> recording data. |
| <i>config</i>   | Detection parameters.                    |

#### Returns

List of detected intervals.

Definition at line 92 of file [IntervalDetector.cpp](#).

References [IntervalDetectorConfig::ftp](#), [IntervalDetectorConfig::minRecoverySeconds](#), [IntervalDetectorConfig::minWorkSeconds](#), [RideData::records](#), [IntervalDetectorConfig::smoothingWindow](#), and [IntervalDetectorConfig::workThresholdPct](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



### 7.70.3.3 removeOverlappingAndDuplicates()

```
std::vector< Interval > IntervalDetector::removeOverlappingAndDuplicates (
    std::vector< Interval > intervals) [static]
```

Removes duplicate and overlapping intervals.

#### Parameters

|                  |                             |
|------------------|-----------------------------|
| <i>intervals</i> | List of intervals to clean. |
|------------------|-----------------------------|

#### Returns

Cleaned list (within 1 second tolerance).

Definition at line 146 of file [IntervalDetector.cpp](#).

References [Interval::averageCadence](#), [Interval::averageHeartRate](#), [Interval::averagePower](#), [Interval::averageSpeed](#), [Interval::durationSeconds](#), [Interval::endSeconds](#), [Interval::maximumPower](#), [Interval::normalizedPower](#), and [Interval::startSeconds](#).

### 7.70.3.4 smoothPower()

```
std::vector< double > IntervalDetector::smoothPower (
    const RideData & rideData,
    int window) [static], [private]
```

Definition at line 9 of file [IntervalDetector.cpp](#).

The documentation for this class was generated from the following files:

- [src/analysis/IntervalDetector.h](#)
- [src/analysis/IntervalDetector.cpp](#)

## 7.71 IntervalDetectorConfig Struct Reference

Configuration parameters for interval detection.

```
#include <IntervalDetector.h>
```

#### Public Attributes

- double [ftp](#) = 250.0  
*Functional threshold power in watts.*
- double [workThresholdPct](#) = 0.76  
*Work threshold as fraction of FTP.*
- double [minWorkSeconds](#) = 60.0  
*Minimum work interval duration in seconds.*
- double [minRecoverySeconds](#) = 20.0

*Minimum recovery interval duration in seconds.*

- `int smoothingWindow = 30`

*Rolling average window width in samples.*

### 7.71.1 Detailed Description

Configuration parameters for interval detection.

Definition at line 50 of file [IntervalDetector.h](#).

### 7.71.2 Member Data Documentation

#### 7.71.2.1 ftp

```
double IntervalDetectorConfig::ftp = 250.0
```

Functional threshold power in watts.

Definition at line 53 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#), and [AnalysisWorker::processTask\(\)](#).

#### 7.71.2.2 minRecoverySeconds

```
double IntervalDetectorConfig::minRecoverySeconds = 20.0
```

Minimum recovery interval duration in seconds.

Definition at line 62 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

#### 7.71.2.3 minWorkSeconds

```
double IntervalDetectorConfig::minWorkSeconds = 60.0
```

Minimum work interval duration in seconds.

Definition at line 59 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

#### 7.71.2.4 smoothingWindow

```
int IntervalDetectorConfig::smoothingWindow = 30
```

Rolling average window width in samples.

Definition at line 65 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

#### 7.71.2.5 workThresholdPct

```
double IntervalDetectorConfig::workThresholdPct = 0.76
```

Work threshold as fraction of FTP.

Definition at line 56 of file [IntervalDetector.h](#).

Referenced by [IntervalDetector::detect\(\)](#).

The documentation for this struct was generated from the following file:

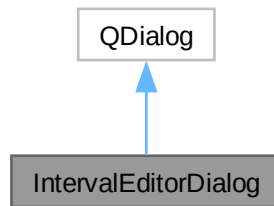
- `src/analysis/IntervalDetector.h`

## 7.72 IntervalEditorDialog Class Reference

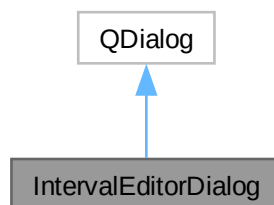
Dialog for editing interval metadata.

```
#include <IntervalEditorDialog.h>
```

Inheritance diagram for IntervalEditorDialog:



Collaboration diagram for IntervalEditorDialog:



### Public Member Functions

- [IntervalEditorDialog](#) (double startSeconds, double endSeconds, QWidget \*parent=nullptr)  
*Constructs interval editor dialog.*
- QString [intervalName](#) () const  
*Gets entered interval name.*
- QString [intervalType](#) () const  
*Gets entered interval type.*
- QString [intervalNotes](#) () const  
*Gets entered interval notes.*

### Private Attributes

- QLineEdit \* [m\\_nameEdit](#) = nullptr
- QLineEdit \* [m\\_typeEdit](#) = nullptr
- QTextEdit \* [m\\_notesEdit](#) = nullptr
- QDoubleSpinBox \* [m\\_startSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_endSpin](#) = nullptr

### 7.72.1 Detailed Description

Dialog for editing interval metadata.

Allows entry of interval name, type, and notes.

Definition at line 18 of file [IntervalEditorDialog.h](#).

## 7.72.2 Constructor & Destructor Documentation

### 7.72.2.1 IntervalEditorDialog()

```
IntervalEditorDialog::IntervalEditorDialog (  
    double startSeconds,  
    double endSeconds,  
    QWidget * parent = nullptr) [explicit]
```

Constructs interval editor dialog.

#### Parameters

|                     |                        |
|---------------------|------------------------|
| <i>startSeconds</i> | Start time in seconds. |
| <i>endSeconds</i>   | End time in seconds.   |
| <i>parent</i>       | Parent widget.         |

Definition at line 13 of file [IntervalEditorDialog.cpp](#).

## 7.72.3 Member Function Documentation

### 7.72.3.1 intervalName()

```
QString IntervalEditorDialog::intervalName () const
```

Gets entered interval name.

#### Returns

[Interval](#) name.

Definition at line 54 of file [IntervalEditorDialog.cpp](#).

### 7.72.3.2 intervalNotes()

```
QString IntervalEditorDialog::intervalNotes () const
```

Gets entered interval notes.

#### Returns

Notes text.

Definition at line 64 of file [IntervalEditorDialog.cpp](#).

### 7.72.3.3 intervalType()

```
QString IntervalEditorDialog::intervalType () const
```

Gets entered interval type.

#### Returns

[Interval](#) type.

Definition at line 59 of file [IntervalEditorDialog.cpp](#).

## 7.72.4 Member Data Documentation

### 7.72.4.1 m\_endSpin

```
QDoubleSpinBox* IntervalEditorDialog::m_endSpin = nullptr [private]
```

Definition at line 56 of file [IntervalEditorDialog.h](#).

### 7.72.4.2 m\_nameEdit

```
QLineEdit* IntervalEditorDialog::m_nameEdit = nullptr [private]
```

Definition at line 52 of file [IntervalEditorDialog.h](#).

#### 7.72.4.3 m\_notesEdit

QTextEdit\* IntervalEditorDialog::m\_notesEdit = nullptr [private]  
 Definition at line 54 of file [IntervalEditorDialog.h](#).

#### 7.72.4.4 m\_startSpin

QDoubleSpinBox\* IntervalEditorDialog::m\_startSpin = nullptr [private]  
 Definition at line 55 of file [IntervalEditorDialog.h](#).

#### 7.72.4.5 m\_typeEdit

QLineEdit\* IntervalEditorDialog::m\_typeEdit = nullptr [private]  
 Definition at line 53 of file [IntervalEditorDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/IntervalEditorDialog.h](#)
- [src/gui/IntervalEditorDialog.cpp](#)

## 7.73 IntervalRecord Struct Reference

Database representation of an interval record.

#include <IntervalRepository.h>

### Public Attributes

- int [id](#) = -1  
*Unique interval record ID.*
- int [activityId](#) = -1  
*Associated activity ID.*
- int [startSample](#) = 0  
*Starting sample index.*
- int [endSample](#) = 0  
*Ending sample index.*
- QString [name](#)  
*Interval name.*
- QString [type](#)  
*Interval type (Work, Recovery, etc.).*
- double [avgPower](#) = 0.0  
*Average power in watts.*
- double [np](#) = 0.0  
*Normalized power in watts.*
- double [avgHr](#) = 0.0  
*Average heart rate in bpm.*
- double [avgCadence](#) = 0.0  
*Average cadence in rpm.*
- QString [notes](#)  
*User notes about interval.*
- QString [source](#) = QStringLiteral("auto")  
*Source of interval (auto, manual, edited).*
- bool [locked](#) = false  
*Whether interval is locked from editing.*
- bool [deleted](#) = false  
*Soft-delete flag.*

- int [algorithmVersion](#) = 1  
*Algorithm version used for detection.*
- QString [uuid](#)  
*Stable UUID, generated once on insert.*

### 7.73.1 Detailed Description

Database representation of an interval record.  
Definition at line 14 of file [IntervalRepository.h](#).

### 7.73.2 Member Data Documentation

#### 7.73.2.1 activityId

```
int IntervalRecord::activityId = -1
```

Associated activity ID.

Definition at line 20 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

#### 7.73.2.2 algorithmVersion

```
int IntervalRecord::algorithmVersion = 1
```

Algorithm version used for detection.

Definition at line 59 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

#### 7.73.2.3 avgCadence

```
double IntervalRecord::avgCadence = 0.0
```

Average cadence in rpm.

Definition at line 44 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

#### 7.73.2.4 avgHr

```
double IntervalRecord::avgHr = 0.0
```

Average heart rate in bpm.

Definition at line 41 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

#### 7.73.2.5 avgPower

```
double IntervalRecord::avgPower = 0.0
```

Average power in watts.

Definition at line 35 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

#### 7.73.2.6 deleted

```
bool IntervalRecord::deleted = false
```

Soft-delete flag.

Definition at line 56 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.7 endSample

```
int IntervalRecord::endSample = 0
```

Ending sample index.

Definition at line 26 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.8 id

```
int IntervalRecord::id = -1
```

Unique interval record ID.

Definition at line 17 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::listIntervalsForActivity\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.9 locked

```
bool IntervalRecord::locked = false
```

Whether interval is locked from editing.

Definition at line 53 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.10 name

```
QString IntervalRecord::name
```

[Interval](#) name.

Definition at line 29 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.11 notes

```
QString IntervalRecord::notes
```

User notes about interval.

Definition at line 47 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.12 np

```
double IntervalRecord::np = 0.0
```

Normalized power in watts.

Definition at line 38 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.13 source

```
QString IntervalRecord::source = QStringLiteral("auto")
```

Source of interval (auto, manual, edited).

Definition at line 50 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.14 startSample

```
int IntervalRecord::startSample = 0
```

Starting sample index.

Definition at line 23 of file [IntervalRepository.h](#).



Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.15 type

`QString IntervalRecord::type`

[Interval](#) type (Work, Recovery, etc.).

Definition at line 32 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), [AnalysisWorker::processTask\(\)](#), and [IntervalRepository::updateInterval\(\)](#).

### 7.73.2.16 uuid

`QString IntervalRecord::uuid`

Stable UUID, generated once on insert.

Definition at line 62 of file [IntervalRepository.h](#).

Referenced by [IntervalRepository::insertInterval\(\)](#), [IntervalRepository::listIntervalsForActivity\(\)](#), and [AnalysisWorker::processTask\(\)](#).

The documentation for this struct was generated from the following file:

- `src/database/IntervalRepository.h`

## 7.74 IntervalRepository Class Reference

Repository for interval record persistence.

```
#include <IntervalRepository.h>
```

### Public Member Functions

- [IntervalRepository](#) (QSqlDatabase &db)  
*Constructs interval repository.*
- int [insertInterval](#) (const [IntervalRecord](#) &interval)  
*Inserts a new interval record.*
- bool [updateInterval](#) (const [IntervalRecord](#) &interval)  
*Updates an existing interval record.*
- bool [softDeleteInterval](#) (int intervalId)  
*Soft-delete: marks deleted=1.*
- bool [deleteInterval](#) (int intervalId)  
*Hard-delete (for full reset before re-detect).*
- bool [deleteIntervalsForActivity](#) (int activityId)  
*Deletes all intervals for an activity.*
- QList< [IntervalRecord](#) > [listIntervalsForActivity](#) (int activityId)  
*Returns non-deleted intervals only.*
- bool [hasLockedIntervals](#) (int activityId)  
*Checks if activity has locked intervals.*

### Private Attributes

- QSqlDatabase & [m\\_db](#)

### 7.74.1 Detailed Description

Repository for interval record persistence.

Provides CRUD operations and queries for work/recovery intervals in the database.

Definition at line 70 of file [IntervalRepository.h](#).

## 7.74.2 Constructor & Destructor Documentation

### 7.74.2.1 IntervalRepository()

```
IntervalRepository::IntervalRepository (
    QSqlDatabase & db) [explicit]
```

Constructs interval repository.

#### Parameters

|           |                         |
|-----------|-------------------------|
| <i>db</i> | SQL database reference. |
|-----------|-------------------------|

Definition at line 10 of file [IntervalRepository.cpp](#).

## 7.74.3 Member Function Documentation

### 7.74.3.1 deleteInterval()

```
bool IntervalRepository::deleteInterval (
    int intervalId)
```

Hard-delete (for full reset before re-detect).

Definition at line 92 of file [IntervalRepository.cpp](#).

### 7.74.3.2 deleteIntervalsForActivity()

```
bool IntervalRepository::deleteIntervalsForActivity (
    int activityId)
```

Deletes all intervals for an activity.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

#### Returns

True if deletion succeeded.

Definition at line 101 of file [IntervalRepository.cpp](#).

### 7.74.3.3 hasLockedIntervals()

```
bool IntervalRepository::hasLockedIntervals (
    int activityId)
```

Checks if activity has locked intervals.

#### Parameters

|                   |                              |
|-------------------|------------------------------|
| <i>activityId</i> | <a href="#">Activity</a> ID. |
|-------------------|------------------------------|

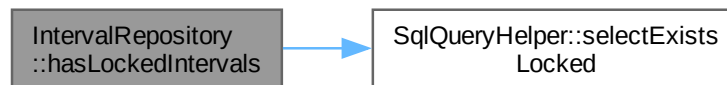
#### Returns

True if locked intervals exist.

Definition at line 148 of file [IntervalRepository.cpp](#).

References [SqlQueryHelper::selectExistsLocked\(\)](#).

Here is the call graph for this function:



#### 7.74.3.4 insertInterval()

```
int IntervalRepository::insertInterval (
    const IntervalRecord & interval)
```

Inserts a new interval record.

##### Parameters

|                 |                                     |
|-----------------|-------------------------------------|
| <i>interval</i> | <a href="#">Interval</a> to insert. |
|-----------------|-------------------------------------|

##### Returns

New interval ID, or -1 on error.

Definition at line 14 of file [IntervalRepository.cpp](#).

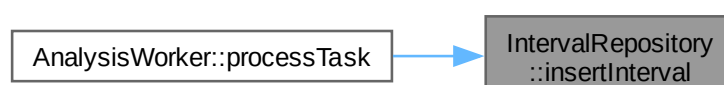
References [IntervalRecord::activityId](#), [IntervalRecord::algorithmVersion](#), [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [IntervalRecord::type](#), and [IntervalRecord::uuid](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.74.3.5 listIntervalsForActivity()

```
QList< IntervalRecord > IntervalRepository::listIntervalsForActivity (
    int activityId)
```

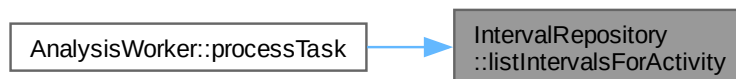
Returns non-deleted intervals only.

Definition at line 109 of file [IntervalRepository.cpp](#).

References [IntervalRecord::activityId](#), [IntervalRecord::algorithmVersion](#), [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::id](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), [IntervalRecord::type](#), and [IntervalRecord::uuid](#).

Referenced by [AnalysisWorker::processTask\(\)](#).

Here is the caller graph for this function:



### 7.74.3.6 softDeleteInterval()

```
bool IntervalRepository::softDeleteInterval (
    int intervalId)
```

Soft-delete: marks deleted=1.

Definition at line 84 of file [IntervalRepository.cpp](#).

### 7.74.3.7 updateInterval()

```
bool IntervalRepository::updateInterval (
    const IntervalRecord & interval)
```

Updates an existing interval record.

#### Parameters

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>interval</i> | <a href="#">Interval</a> with updated data. |
|-----------------|---------------------------------------------|

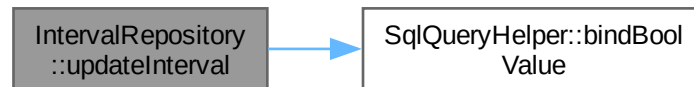
**Returns**

True if update succeeded.

Definition at line 48 of file [IntervalRepository.cpp](#).

References [IntervalRecord::avgCadence](#), [IntervalRecord::avgHr](#), [IntervalRecord::avgPower](#), [SqlQueryHelper::bindBoolValue\(\)](#), [IntervalRecord::deleted](#), [IntervalRecord::endSample](#), [IntervalRecord::id](#), [IntervalRecord::locked](#), [IntervalRecord::name](#), [IntervalRecord::notes](#), [IntervalRecord::np](#), [IntervalRecord::source](#), [IntervalRecord::startSample](#), and [IntervalRecord::type](#).

Here is the call graph for this function:

**7.74.4 Member Data Documentation****7.74.4.1 m\_db**

`QSqlDatabase& IntervalRepository::m_db [private]`

Definition at line 116 of file [IntervalRepository.h](#).

The documentation for this class was generated from the following files:

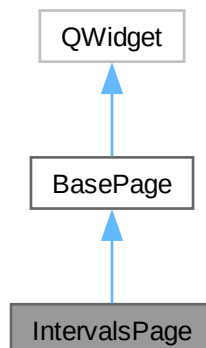
- `src/database/IntervalRepository.h`
- `src/database/IntervalRepository.cpp`

**7.75 IntervalsPage Class Reference**

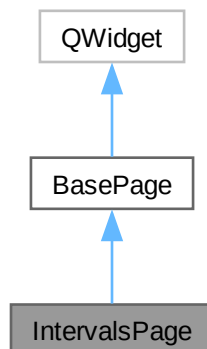
Top-level page for interval analysis and management.

```
#include <IntervalsPage.h>
```

Inheritance diagram for `IntervalsPage`:



Collaboration diagram for IntervalsPage:



### Public Member Functions

- [IntervalsPage](#) (QWidget \*intervalsContent, QWidget \*parent=nullptr)  
*Constructs the Intervals page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

## 7.75.1 Detailed Description

Top-level page for interval analysis and management.

Displays the interval table (auto-detected and manually saved intervals) with summary statistics for the selected interval.

Toolbar actions: Add, Edit, Delete.

Future additions: [Interval](#) Comparison, [Interval](#) Library, Structured Workout Builder.

Definition at line 23 of file [IntervalsPage.h](#).

## 7.75.2 Constructor & Destructor Documentation

### 7.75.2.1 IntervalsPage()

```

IntervalsPage::IntervalsPage (
    QWidget * intervalsContent,
    QWidget * parent = nullptr) [explicit]
  
```

Constructs the Intervals page.

### Parameters

|                         |                                                            |
|-------------------------|------------------------------------------------------------|
| <i>intervalsContent</i> | Pre-assembled intervals panel (table + toolbar + summary). |
| <i>parent</i>           | Parent widget.                                             |

If intervalsContent is provided, embeds it in a full-height layout. Otherwise shows a placeholder label directing the user to the Charts page where interval selection and chart-context navigation are available.

**Parameters**

|                         |                                                         |
|-------------------------|---------------------------------------------------------|
| <i>intervalsContent</i> | Assembled intervals widget, or nullptr for placeholder. |
| <i>parent</i>           | Parent widget.                                          |

Definition at line 18 of file [IntervalsPage.cpp](#).

The documentation for this class was generated from the following files:

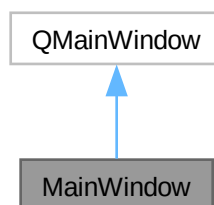
- src/gui/pages/IntervalsPage.h
- src/gui/pages/IntervalsPage.cpp

## 7.76 MainWindow Class Reference

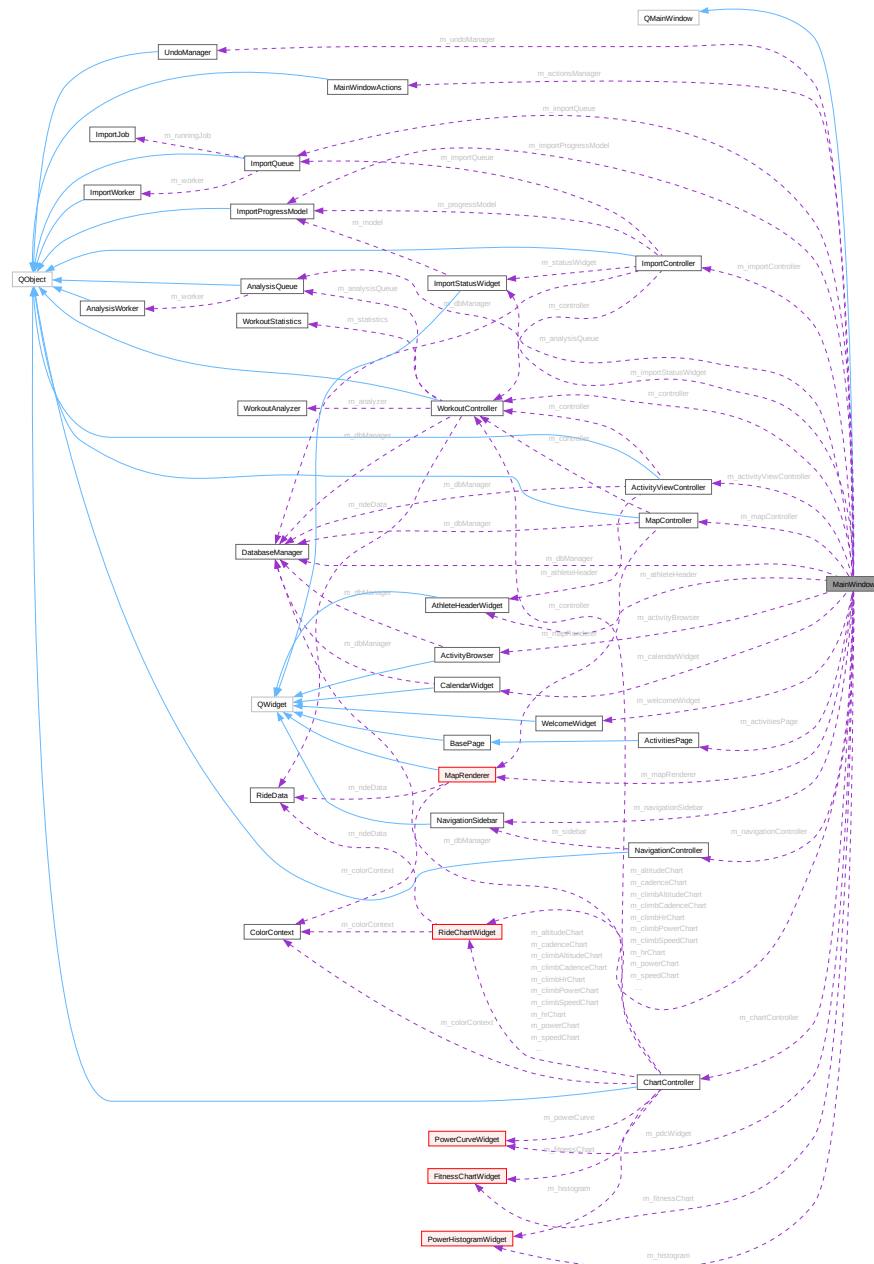
Main application window.

```
#include <MainWindow.h>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



## Classes

- struct [ImportBatchSummary](#)

## Public Member Functions

- [MainWindow](#) (QWidget \*parent=nullptr)  
*Constructs main window.*

## Protected Member Functions

- void [closeEvent](#) (QCloseEvent \*event) override  
*Handles application close event.*



- bool [eventFilter](#) (QObject \*watched, QEvent \*event) override  
*Filters events for custom handling.*
- void [dragEnterEvent](#) (QDragEnterEvent \*event) override  
*Accepts drag-enter when the payload contains local .fit files.*
- void [dropEvent](#) (QDropEvent \*event) override  
*Extracts dropped .fit files and forwards them to the import queue.*

### Private Slots

- void [importActivities](#) ()
- void [previewFitFile](#) ()
- void [onWorkoutLoaded](#) ()
- void [onActivityImported](#) (int activityId)
- void [openDatabase](#) ()
- void [createDatabase](#) ()
- void [manageAthletes](#) ()
- void [backupDatabase](#) ()
- void [openSettingsDialog](#) ()
- void [openRecentDatabase](#) ()
- void [onAthleteSelectionChanged](#) (int index)
- void [createVideo](#) ()
- void [showAbout](#) ()
- void [triggerDetectClimbs](#) ()
- void [triggerDetectIntervals](#) ()
- void [revertSelectedClimbToAuto](#) ()

### Private Member Functions

- void [buildUI](#) ()
- void [buildToolBar](#) ()
- void [buildStatusBar](#) ()
- void [setupShortcuts](#) ()
- void [saveSettings](#) ()
- void [loadSettings](#) ()
- void [addRecentDatabase](#) (const QString &path)
- void [updateRecentDatabaseMenu](#) ()
- void [openDatabasePath](#) (const QString &path)
- bool [canImportActivities](#) () const  
*Returns whether importing is currently allowed.*
- void [updateImportAvailability](#) ()  
*Enables/disables import actions based on current readiness.*
- void [importFiles](#) (const QStringList &filePaths)  
*Queues a batch of file paths for background import.*
- void [enqueueImportBatch](#) (const QStringList &filePaths, bool showResultDialog, const QString &sourceLabel)
- QStringList [prepareImportFileBatch](#) (const QStringList &filePaths)
- QString [importQueuedStatusMessage](#) (int queuedCount) const
- QStringList [fitFilesFromMimeData](#) (const QMimeData \*mimeData) const  
*Returns deduplicated local .fit files from MIME drop/paste data.*
- void [scheduleActivityBrowserRefresh](#) ()
- void [finalizeImportBatches](#) ()  
*Shows a per-batch import result summary when queue work completes.*
- void [refreshAthleteSelector](#) ()
- void [resetAllZoom](#) ()

- void [increaseChartHeight](#) ()
- void [decreaseChartHeight](#) ()
- void [applyChartHeight](#) ()
- void [updateStatsLabel](#) ()
- void [updateIntervals](#) ()
- void [updateClimbingTab](#) ()
- void [detectClimbsAndRefresh](#) ()
- void [refreshClimbViews](#) (double preferredStartSeconds=-1.0, double preferredEndSeconds=-1.0)
- void [applyDetectedClimbsToCharts](#) ()
- void [restoreClimbSelection](#) (int previousRow, double preferredStartSeconds, double preferredEndSeconds)
- void [updateClimbQuarterCharts](#) (const [Climb](#) \*climb)
- double [resolveActiveActivityWeightKg](#) () const
- void [assignClimbWeightMetrics](#) ([Climb](#) &climb, double riderWeightKg) const
- void [applyWeightMetricsToClimbs](#) (std::vector< [Climb](#) > &climbs, double riderWeightKg) const
- std::vector< int > [selectedClimbIndicesFromTable](#) () const
- void [onClimbTableContextMenuRequested](#) (const QPoint &pos)
- void [removeSelectedClimbs](#) ()
- void [editSelectedClimbBoundaries](#) ()
- void [joinSelectedClimbs](#) ()
- void [updateClimbRowStyles](#) ()
- void [onClimbSelectionChanged](#) ()
- void [onClimbRowDoubleClicked](#) (QTableWidgetItem \*item)
- void [onClimbBoundaryEdited](#) (double oldStartSeconds, double oldEndSeconds, double newStartSeconds, double newEndSeconds)
- void [onNewClimbRequested](#) (double startSeconds, double endSeconds)
- void [onIntervalSelectionChanged](#) ()
- void [onIntervalRowDoubleClicked](#) (QTableWidgetItem \*item)
- void [navigateToInterval](#) (double startSeconds, double endSeconds, bool exactZoom)
- void [selectIntervalRowOffset](#) (int delta)
- void [updateIntervalRowStyles](#) ()
- void [finalizeIntervalSelectionAndSummary](#) (int previousRow)
- void [updateIntervalSummary](#) (double startSeconds, double endSeconds)
- void [clearIntervalSummary](#) ()
- void [updateLapsSummaryFromCurrentActivity](#) ()
- void [updateActivityNotesFromCurrentActivity](#) ()
- bool [ensureDatabase](#) ()
- bool [ensureAthlete](#) ()
- bool [ensureImportReady](#) ()

*Ensures database path, athlete selection, and queue state are import-ready.*

- bool [createOrOpenDefaultDatabase](#) ()
- QString [defaultDatabasePathForAutoCreate](#) () const
- void [showWelcomeScreen](#) ()
- void [hideWelcomeScreen](#) ()
- void [updateWelcomeScreenVisibility](#) ()
- int [activityCount](#) () const
- void [updateAthleteLabel](#) ()
- void [updateStatusBarInfo](#) ()
- ColorMetric [currentColorMetric](#) () const
- [ColorContext](#) [buildColorContext](#) () const
- void [applyChartPreset](#) (int presetId)
- void [editCurrentActivityProperties](#) ()
- double [estimatedFtpFromCurrentRide](#) () const
- void [applyEstimatedFtpForCurrentAthlete](#) ()
- void [importFilesInternal](#) (const QStringList &filePaths, bool showResultDialog, const QString &sourceLabel)

*Internal import entry point shared by menu action and drag-drop.*

- void [configureFolderWatcher](#) ()
- QStringList [monitoredDirectories](#) () const
- void [scanWatchDirectories](#) (bool initialScan)
- void [onWatchDirectoryChanged](#) (const QString &path)
- void [syncAthleteContextToControllers](#) ()

*Syncs athlete context to all controllers that need it.*

- void [syncChartContextToController](#) (bool includeLegendLayout)

*Syncs current chart-related context to [ChartController](#).*

- void [applyChartControlDrivenUpdates](#) (bool includeLegendLayout, bool applySmoothing=false)

*Applies controller chart updates based on current UI control state.*

- void [applyChartAndMapUpdates](#) (bool includeLegendLayout, bool applySmoothing)

*Applies chart control-driven updates and refreshes map coloring.*

- void [updateChartAnalysisEmptyStates](#) ()

*Updates analysis tab empty-state visibility through [ChartController](#).*

- void [refreshMapRideDataFromCurrentState](#) ()

*Refreshes map route coloring based on current ride data and metric.*

### Private Attributes

- [UndoManager](#) \* [m\\_undoManager](#) = nullptr
- [AnalysisQueue](#) \* [m\\_analysisQueue](#) = nullptr
- [ImportQueue](#) \* [m\\_importQueue](#) = nullptr
- [ImportProgressModel](#) \* [m\\_importProgressModel](#) = nullptr
- [WorkoutController](#) \* [m\\_controller](#) = nullptr
- class [ActivityViewController](#) \* [m\\_activityViewController](#) = nullptr
- class [ChartController](#) \* [m\\_chartController](#) = nullptr
- class [MapController](#) \* [m\\_mapController](#) = nullptr
- class [ImportController](#) \* [m\\_importController](#) = nullptr
- class [NavigationController](#) \* [m\\_navigationController](#) = nullptr
- [NavigationSidebar](#) \* [m\\_navigationSidebar](#) = nullptr
- class [MainWindowActions](#) \* [m\\_actionsManager](#) = nullptr
- [DatabaseManager](#) [m\\_dbManager](#)
- int [m\\_currentAthleteId](#) = -1
- QString [m\\_currentAthleteName](#)
- QMenu \* [m\\_athletesMenu](#) = nullptr
- QAction \* [m\\_athleteLabelAct](#) = nullptr
- QAction \* [m\\_importAct](#) = nullptr
- QAction \* [m\\_toolbarImportAct](#) = nullptr
- QAction \* [m\\_toolbarSearchAct](#) = nullptr
- QAction \* [m\\_previewAct](#) = nullptr
- QMenu \* [m\\_recentDbMenu](#) = nullptr
- QString [m\\_lastOpenDir](#)
- [WelcomeWidget](#) \* [m\\_welcomeWidget](#) = nullptr
- bool [m\\_firstLaunchCompleted](#) = false
- QFileSystemWatcher \* [m\\_watcher](#) = nullptr
- QTimer \* [m\\_watchRescanTimer](#) = nullptr
- QTimer \* [m\\_importRefreshTimer](#) = nullptr
- bool [m\\_watchFolderEnabled](#) = true
- QString [m\\_watchFolderPath](#)
- QSet< QString > [m\\_knownWatchedFitFiles](#)
- QComboBox \* [m\\_athleteCombo](#) = nullptr
- QComboBox \* [m\\_colorMetricCombo](#) = nullptr
- QComboBox \* [m\\_chartPresetCombo](#) = nullptr

- `QComboBox * m_powerSmoothingCombo = nullptr`
- `QCheckBox * m_autoSmoothingCheck = nullptr`
- `ImportStatusWidget * m_importStatusWidget = nullptr`
- `QLabel * m_dbStatusLabel = nullptr`
- `QLabel * m_athleteStatusLabel = nullptr`
- `QLabel * m_activityCountLabel = nullptr`
- `AthleteHeaderWidget * m_athleteHeader = nullptr`
- `QStackedWidget * m_pageStack = nullptr`
- `QWidget * m_chartsPageContent = nullptr`  
*Charts+map+intervals content.*
- `QWidget * m_zonesPageContent = nullptr`  
*Zones (Power page sub-tab).*
- `QWidget * m_histogramPageContent = nullptr`  
*Histogram (Power page sub-tab).*
- `QWidget * m_pdcPageContent = nullptr`  
*Power Curve (Power page sub-tab).*
- `QWidget * m_calendarPageContent = nullptr`  
*Calendar content.*
- `QWidget * m_fitnessPageContent = nullptr`  
*Fitness chart content.*
- `QWidget * m_climbsPageContent = nullptr`  
*Climb charts + table content.*
- `QWidget * m_intervalsPageContent = nullptr`  
*Intervals table + tools content.*
- `QTabWidget * m_analysisTabWidget = nullptr`
- `QStackedLayout * m_activityTabStack = nullptr`
- `QStackedLayout * m_zonesTabStack = nullptr`
- `QStackedLayout * m_histogramTabStack = nullptr`
- `QStackedLayout * m_powerCurveTabStack = nullptr`
- `QStackedLayout * m_calendarTabStack = nullptr`
- `QStackedLayout * m_fitnessTabStack = nullptr`
- `QStackedLayout * m_climbingTabStack = nullptr`
- `QScrollArea * m_chartScroll = nullptr`
- `QWidget * m_chartsContainer = nullptr`
- `QVBoxLayout * m_chartsLayout = nullptr`
- `QSplitter * m_chartMapSplit = nullptr`
- `int m_chartHeight = 220`
- `RideChartWidget * m_powerChart = nullptr`
- `RideChartWidget * m_hrChart = nullptr`
- `RideChartWidget * m_cadenceChart = nullptr`
- `RideChartWidget * m_speedChart = nullptr`
- `RideChartWidget * m_altitudeChart = nullptr`
- `RideChartWidget * m_climbAltitudeChart = nullptr`
- `RideChartWidget * m_climbPowerChart = nullptr`
- `RideChartWidget * m_climbHrChart = nullptr`
- `RideChartWidget * m_climbCadenceChart = nullptr`
- `RideChartWidget * m_climbSpeedChart = nullptr`
- `QChartView * m_climbQuarterPowerChart = nullptr`
- `QChartView * m_climbQuarterHrChart = nullptr`
- `QChartView * m_climbQuarterCadenceChart = nullptr`
- `QPushButton * m_fitChartsButton = nullptr`
- `QPushButton * m_chartHeightIncreaseButton = nullptr`
- `QPushButton * m_chartHeightDecreaseButton = nullptr`

- QPushButton \* [m\\_useEstimatedFtpButton](#) = nullptr
- QWidget \* [m\\_colorLegendWidget](#) = nullptr
- QHBoxLayout \* [m\\_colorLegendLayout](#) = nullptr
- QCheckBox \* [m\\_showPower](#) = nullptr
- QCheckBox \* [m\\_showHR](#) = nullptr
- QCheckBox \* [m\\_showCadence](#) = nullptr
- QCheckBox \* [m\\_showSpeed](#) = nullptr
- QCheckBox \* [m\\_showAltitude](#) = nullptr
- QTableWidget \* [m\\_zonesTable](#) = nullptr
- PowerHistogramWidget \* [m\\_histogram](#) = nullptr
- PowerCurveWidget \* [m\\_pdcWidget](#) = nullptr
- FitnessChartWidget \* [m\\_fitnessChart](#) = nullptr
- QTableWidget \* [m\\_intervalsTable](#) = nullptr
- QTableWidget \* [m\\_lapsTable](#) = nullptr
- QTextEdit \* [m\\_activityNotesView](#) = nullptr
- QCheckBox \* [m\\_followIntervalSelectionCheck](#) = nullptr
- QLabel \* [m\\_intervalSummaryLabel](#) = nullptr
- QTableWidget \* [m\\_climbsTable](#) = nullptr
- QLabel \* [m\\_climbSummaryLabel](#) = nullptr
- QDoubleSpinBox \* [m\\_climbMinLengthSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbMinGainSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbMinGradientSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbStartGradientSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbDipMetersSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbDipDistanceSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_climbSmoothingSpin](#) = nullptr
- QCheckBox \* [m\\_climbOverlayEnabledCheck](#) = nullptr
- QComboBox \* [m\\_climbOverlayMetricCombo](#) = nullptr
- std::vector< [Climb](#) > [m\\_detectedClimbs](#)
- bool [m\\_suppressClimbAutoZoom](#) = false
- MapRenderer \* [m\\_mapRenderer](#) = nullptr
- QComboBox \* [m\\_mapStyleCombo](#) = nullptr
- QCheckBox \* [m\\_mapAutoContrastCheck](#) = nullptr
- CalendarWidget \* [m\\_calendarWidget](#) = nullptr
- ActivityBrowser \* [m\\_activityBrowser](#) = nullptr
- class ActivitiesPage \* [m\\_activitiesPage](#) = nullptr
- QHash< QString, [ImportBatchSummary](#) > [m\\_importBatchSummaries](#)

### 7.76.1 Detailed Description

Main application window.

Provides integrated UI for activity browsing, analysis, charting, mapping, and athlete/profile management.

Definition at line 63 of file [MainWindow.h](#).

### 7.76.2 Constructor & Destructor Documentation

#### 7.76.2.1 MainWindow()

```
MainWindow::MainWindow (
    QWidget * parent = nullptr) [explicit]
```

Constructs main window.

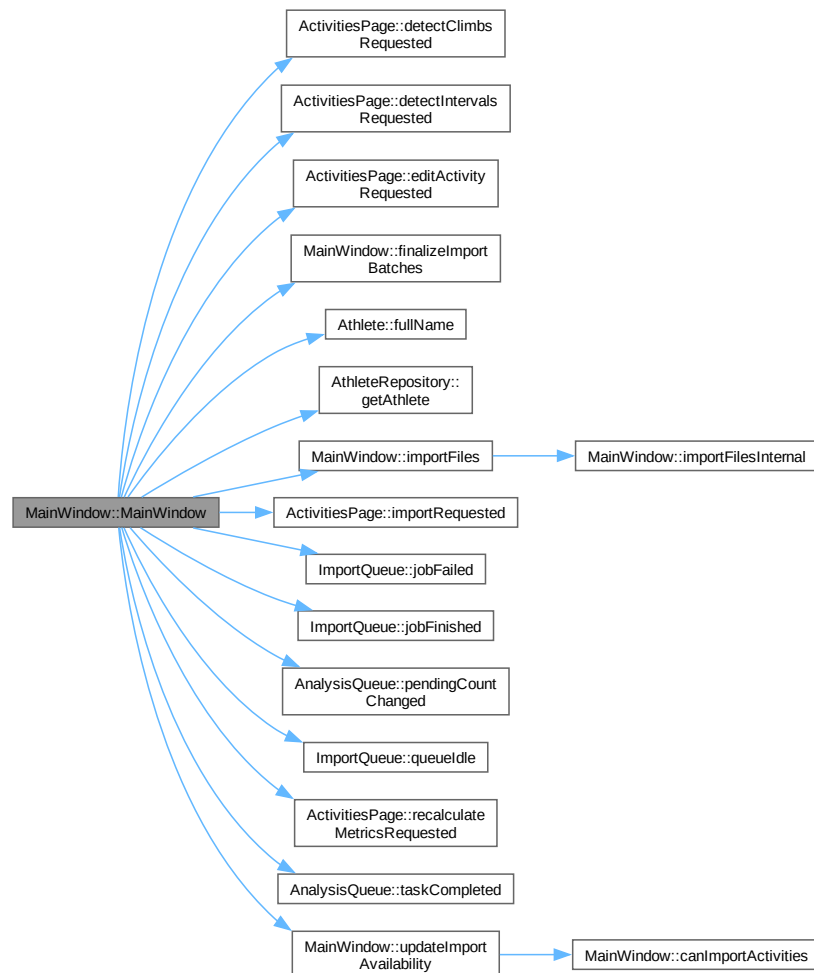
#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 29 of file [MainWindow.cpp](#).

References [ActivitiesPage::detectClimbsRequested\(\)](#), [ActivitiesPage::detectIntervalsRequested\(\)](#), [ActivitiesPage::editActivityRequested\(\)](#), [finalizeImportBatches\(\)](#), [Athlete::fullName\(\)](#), [AthleteRepository::getAthlete\(\)](#), [importFiles\(\)](#), [ActivitiesPage::importRequested\(\)](#), [ImportQueue::jobFailed\(\)](#), [ImportQueue::jobFinished\(\)](#), [AnalysisQueue::pendingCountChanged\(\)](#), [ImportQueue::queueIdle\(\)](#), [ActivitiesPage::recalculateMetricsRequested\(\)](#), [AnalysisQueue::taskCompleted\(\)](#), and [updateImportAvailability\(\)](#).

Here is the call graph for this function:



## 7.76.3 Member Function Documentation

### 7.76.3.1 activityCount()

```
int MainWindow::activityCount () const [private]
```

Definition at line 211 of file [MainWindowReadiness.cpp](#).

### 7.76.3.2 addRecentDatabase()

```
void MainWindow::addRecentDatabase (
    const QString & path) [private]
```

Definition at line 211 of file [MainWindowDatabase.cpp](#).

### 7.76.3.3 applyChartAndMapUpdates()

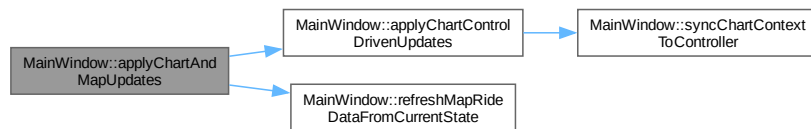
```
void MainWindow::applyChartAndMapUpdates (
    bool includeLegendLayout,
    bool applySmoothing) [private]
```

Applies chart control-driven updates and refreshes map coloring.

Definition at line 51 of file [MainWindowCharts.cpp](#).

References [applyChartControlDrivenUpdates\(\)](#), and [refreshMapRideDataFromCurrentState\(\)](#).

Here is the call graph for this function:



### 7.76.3.4 applyChartControlDrivenUpdates()

```
void MainWindow::applyChartControlDrivenUpdates (
    bool includeLegendLayout,
    bool applySmoothing = false) [private]
```

Applies controller chart updates based on current UI control state.

Definition at line 37 of file [MainWindowCharts.cpp](#).

References [syncChartContextToController\(\)](#).

Referenced by [applyChartAndMapUpdates\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 7.76.3.5 applyChartHeight()

```
void MainWindow::applyChartHeight () [private]
```

Definition at line 48 of file [MainWindowChartActions.cpp](#).

#### 7.76.3.6 applyChartPreset()

```
void MainWindow::applyChartPreset (
    int presetId) [private]
```

Definition at line 92 of file [MainWindowChartActions.cpp](#).

#### 7.76.3.7 applyDetectedClimbsToCharts()

```
void MainWindow::applyDetectedClimbsToCharts () [private]
```

Definition at line 209 of file [MainWindowClimbViews.cpp](#).

#### 7.76.3.8 applyEstimatedFtpForCurrentAthlete()

```
void MainWindow::applyEstimatedFtpForCurrentAthlete () [private]
```

Definition at line 158 of file [MainWindowChartActions.cpp](#).

#### 7.76.3.9 applyWeightMetricsToClimbs()

```
void MainWindow::applyWeightMetricsToClimbs (
    std::vector< Climb > & climbs,
    double riderWeightKg) const [private]
```

Definition at line 177 of file [MainWindowDetection.cpp](#).

#### 7.76.3.10 assignClimbWeightMetrics()

```
void MainWindow::assignClimbWeightMetrics (
    Climb & climb,
    double riderWeightKg) const [private]
```

Definition at line 154 of file [MainWindowDetection.cpp](#).

#### 7.76.3.11 backupDatabase

```
void MainWindow::backupDatabase () [private], [slot]
```

Definition at line 357 of file [MainWindowAthlete.cpp](#).

#### 7.76.3.12 buildColorContext()

```
ColorContext MainWindow::buildColorContext () const [private]
```

Definition at line 51 of file [MainWindowContext.cpp](#).

#### 7.76.3.13 buildStatusBar()

```
void MainWindow::buildStatusBar () [private]
```

Definition at line 110 of file [MainWindowUi.cpp](#).

#### 7.76.3.14 buildToolBar()

```
void MainWindow::buildToolBar () [private]
```

Definition at line 22 of file [MainWindowUi.cpp](#).

#### 7.76.3.15 buildUI()

```
void MainWindow::buildUI () [private]
```

Definition at line 111 of file [MainWindowUiBuild.cpp](#).

#### 7.76.3.16 canImportActivities()

```
bool MainWindow::canImportActivities () const [private]
```

Returns whether importing is currently allowed.

Requires an open database and a selected athlete.



Definition at line 19 of file [MainWindowReadiness.cpp](#).  
Referenced by [updateImportAvailability\(\)](#).  
Here is the caller graph for this function:



### 7.76.3.17 clearIntervalSummary()

`void MainWindow::clearIntervalSummary () [private]`  
Definition at line 179 of file [MainWindowIntervals.cpp](#).

### 7.76.3.18 closeEvent()

`void MainWindow::closeEvent (`  
`QCloseEvent * event) [override], [protected]`  
Handles application close event.  
Definition at line 258 of file [MainWindow.cpp](#).

### 7.76.3.19 configureFolderWatcher()

`void MainWindow::configureFolderWatcher () [private]`  
Definition at line 37 of file [MainWindowWatch.cpp](#).

### 7.76.3.20 createDatabase

`void MainWindow::createDatabase () [private], [slot]`  
Definition at line 293 of file [MainWindowDatabase.cpp](#).

### 7.76.3.21 createOrOpenDefaultDatabase()

`bool MainWindow::createOrOpenDefaultDatabase () [private]`  
Definition at line 24 of file [MainWindowReadiness.cpp](#).

### 7.76.3.22 createVideo

`void MainWindow::createVideo () [private], [slot]`  
Definition at line 36 of file [MainWindowVideo.cpp](#).

### 7.76.3.23 currentColorMetric()

`ColorMetric MainWindow::currentColorMetric () const [private]`  
Definition at line 43 of file [MainWindowContext.cpp](#).

### 7.76.3.24 decreaseChartHeight()

`void MainWindow::decreaseChartHeight () [private]`  
Definition at line 42 of file [MainWindowChartActions.cpp](#).

### 7.76.3.25 defaultDatabasePathForAutoCreate()

`QString MainWindow::defaultDatabasePathForAutoCreate () const [private]`  
Definition at line 164 of file [MainWindowReadiness.cpp](#).

### 7.76.3.26 detectClimbsAndRefresh()

void MainWindow::detectClimbsAndRefresh () [private]  
 Definition at line 82 of file [MainWindowDetection.cpp](#).

### 7.76.3.27 dragEnterEvent()

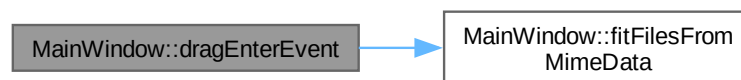
```
void MainWindow::dragEnterEvent (
    QDragEnterEvent * event) [override], [protected]
```

Accepts drag-enter when the payload contains local .fit files.

Definition at line 15 of file [MainWindowWatch.cpp](#).

References [fitFilesFromMimeData\(\)](#).

Here is the call graph for this function:



### 7.76.3.28 dropEvent()

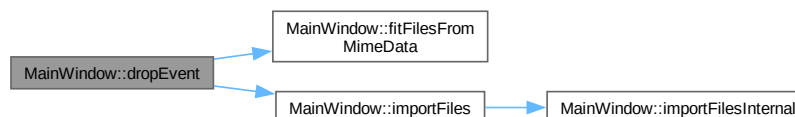
```
void MainWindow::dropEvent (
    QDropEvent * event) [override], [protected]
```

Extracts dropped .fit files and forwards them to the import queue.

Definition at line 24 of file [MainWindowWatch.cpp](#).

References [fitFilesFromMimeData\(\)](#), and [importFiles\(\)](#).

Here is the call graph for this function:



### 7.76.3.29 editCurrentActivityProperties()

void MainWindow::editCurrentActivityProperties () [private]  
 Definition at line 239 of file [MainWindowDetection.cpp](#).

### 7.76.3.30 editSelectedClimbBoundaries()

void MainWindow::editSelectedClimbBoundaries () [private]  
 Definition at line 228 of file [MainWindowClimbInteractions.cpp](#).

### 7.76.3.31 enqueueImportBatch()

```
void MainWindow::enqueueImportBatch (
    const QStringList & filePaths,
```

```
bool showResultDialog,
    const QString & sourceLabel) [private]
```

Definition at line 53 of file [MainWindowImport.cpp](#).

#### 7.76.3.32 ensureAthlete()

```
bool MainWindow::ensureAthlete () [private]
```

Definition at line 83 of file [MainWindowReadiness.cpp](#).

#### 7.76.3.33 ensureDatabase()

```
bool MainWindow::ensureDatabase () [private]
```

Definition at line 64 of file [MainWindowReadiness.cpp](#).

#### 7.76.3.34 ensureImportReady()

```
bool MainWindow::ensureImportReady () [private]
```

Ensures database path, athlete selection, and queue state are import-ready.

Definition at line 159 of file [MainWindowReadiness.cpp](#).

#### 7.76.3.35 estimatedFtpFromCurrentRide()

```
double MainWindow::estimatedFtpFromCurrentRide () const [private]
```

Definition at line 150 of file [MainWindowChartActions.cpp](#).

#### 7.76.3.36 eventFilter()

```
bool MainWindow::eventFilter (
    QObject * watched,
    QEvent * event) [override], [protected]
```

Filters events for custom handling.

Definition at line 168 of file [MainWindowUi.cpp](#).

#### 7.76.3.37 finalizeImportBatches()

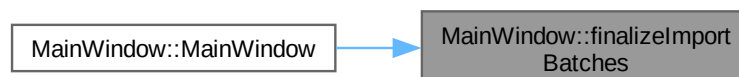
```
void MainWindow::finalizeImportBatches () [private]
```

Shows a per-batch import result summary when queue work completes.

Definition at line 95 of file [MainWindowImport.cpp](#).

Referenced by [MainWindow\(\)](#).

Here is the caller graph for this function:



#### 7.76.3.38 finalizeIntervalSelectionAndSummary()

```
void MainWindow::finalizeIntervalSelectionAndSummary (
    int previousRow) [private]
```

Definition at line 377 of file [MainWindowIntervals.cpp](#).

### 7.76.3.39 fitFilesFromMimeData()

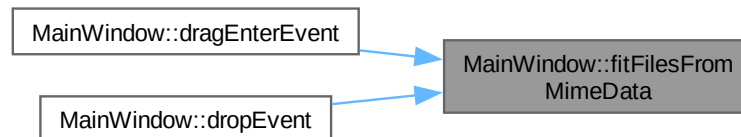
```
QStringList MainWindow::fitFilesFromMimeData (
    const QMimeData * mimeData) const [private]
```

Returns deduplicated local `.fit` files from MIME drop/paste data.

Definition at line 135 of file [MainWindowImport.cpp](#).

Referenced by [dragEnterEvent\(\)](#), and [dropEvent\(\)](#).

Here is the caller graph for this function:



### 7.76.3.40 hideWelcomeScreen()

```
void MainWindow::hideWelcomeScreen () [private]
```

Definition at line 203 of file [MainWindowReadiness.cpp](#).

### 7.76.3.41 importActivities

```
void MainWindow::importActivities () [private], [slot]
```

Definition at line 154 of file [MainWindowImport.cpp](#).

### 7.76.3.42 importFiles()

```
void MainWindow::importFiles (
    const QStringList & filePaths) [private]
```

Queues a batch of file paths for background import.

Definition at line 18 of file [MainWindowImport.cpp](#).

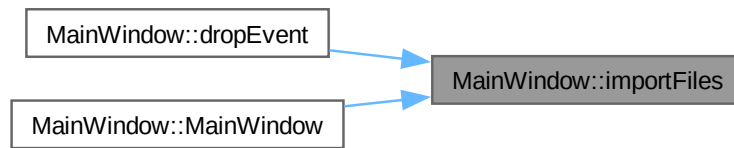
References [importFilesInternal\(\)](#).

Referenced by [dropEvent\(\)](#), and [MainWindow\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 7.76.3.43 importFilesInternal()

```
void MainWindow::importFilesInternal (
    const QStringList & filePaths,
    bool showResultDialog,
    const QString & sourceLabel) [private]
```

Internal import entry point shared by menu action and drag-drop.

##### Parameters

|                         |                                                                      |
|-------------------------|----------------------------------------------------------------------|
| <i>filePaths</i>        | Candidate files to import (only <code>.fit</code> files are queued). |
| <i>showResultDialog</i> | Whether to display completion summary dialogs.                       |
| <i>sourceLabel</i>      | Source tag stored in import metadata (e.g. "manual", "drop").        |

Definition at line 23 of file [MainWindowImport.cpp](#).

Referenced by [importFiles\(\)](#).

Here is the caller graph for this function:



#### 7.76.3.44 importQueuedStatusMessage()

```
QString MainWindow::importQueuedStatusMessage (
    int queuedCount) const [private]
```

Definition at line 82 of file [MainWindowImport.cpp](#).

#### 7.76.3.45 increaseChartHeight()

```
void MainWindow::increaseChartHeight () [private]
```

Definition at line 36 of file [MainWindowChartActions.cpp](#).

#### 7.76.3.46 joinSelectedClimbs()

```
void MainWindow::joinSelectedClimbs () [private]
```

Definition at line 258 of file [MainWindowClimbInteractions.cpp](#).

#### 7.76.3.47 loadSettings()

```
void MainWindow::loadSettings () [private]
```

Definition at line 117 of file [MainWindowDatabase.cpp](#).

#### 7.76.3.48 manageAthletes

```
void MainWindow::manageAthletes () [private], [slot]
```

Definition at line 44 of file [MainWindowAthlete.cpp](#).

#### 7.76.3.49 monitoredDirectories()

```
QStringList MainWindow::monitoredDirectories () const [private]
```

Definition at line 82 of file [MainWindowWatch.cpp](#).

#### 7.76.3.50 navigateToInterval()

```
void MainWindow::navigateToInterval (
    double startSeconds,
    double endSeconds,
    bool exactZoom) [private]
```

Definition at line 104 of file [MainWindowIntervals.cpp](#).

#### 7.76.3.51 onActivityImported

```
void MainWindow::onActivityImported (
    int activityId) [private], [slot]
```

Definition at line 180 of file [MainWindowImport.cpp](#).

#### 7.76.3.52 onAthleteSelectionChanged

```
void MainWindow::onAthleteSelectionChanged (
    int index) [private], [slot]
```

Definition at line 410 of file [MainWindowAthlete.cpp](#).

#### 7.76.3.53 onClimbBoundaryEdited()

```
void MainWindow::onClimbBoundaryEdited (
    double oldStartSeconds,
    double oldEndSeconds,
    double newStartSeconds,
    double newEndSeconds) [private]
```

Definition at line 471 of file [MainWindowClimbInteractions.cpp](#).

#### 7.76.3.54 onClimbRowDoubleClicked()

```
void MainWindow::onClimbRowDoubleClicked (
    QTableWidgetItem * item) [private]
```

Definition at line 419 of file [MainWindowClimbInteractions.cpp](#).

#### 7.76.3.55 onClimbSelectionChanged()

```
void MainWindow::onClimbSelectionChanged () [private]
```

Definition at line 340 of file [MainWindowClimbInteractions.cpp](#).

#### 7.76.3.56 onClimbTableContextMenuRequested()

```
void MainWindow::onClimbTableContextMenuRequested (
    const QPoint & pos) [private]
```

Definition at line 156 of file [MainWindowClimbInteractions.cpp](#).

**7.76.3.57 onIntervalRowDoubleClicked()**

```
void MainWindow::onIntervalRowDoubleClicked (
    QTableWidgetItem * item) [private]
```

Definition at line 85 of file [MainWindowIntervals.cpp](#).

**7.76.3.58 onIntervalSelectionChanged()**

```
void MainWindow::onIntervalSelectionChanged () [private]
```

Definition at line 45 of file [MainWindowIntervals.cpp](#).

**7.76.3.59 onNewClimbRequested()**

```
void MainWindow::onNewClimbRequested (
    double startSeconds,
    double endSeconds) [private]
```

Definition at line 733 of file [MainWindowClimbInteractions.cpp](#).

**7.76.3.60 onWatchDirectoryChanged()**

```
void MainWindow::onWatchDirectoryChanged (
    const QString & path) [private]
```

Definition at line 146 of file [MainWindowWatch.cpp](#).

**7.76.3.61 onWorkoutLoaded**

```
void MainWindow::onWorkoutLoaded () [private], [slot]
```

Definition at line 73 of file [MainWindowChartActions.cpp](#).

**7.76.3.62 openDatabase**

```
void MainWindow::openDatabase () [private], [slot]
```

Definition at line 283 of file [MainWindowDatabase.cpp](#).

**7.76.3.63 openDatabasePath()**

```
void MainWindow::openDatabasePath (
    const QString & path) [private]
```

Definition at line 245 of file [MainWindowDatabase.cpp](#).

**7.76.3.64 openRecentDatabase**

```
void MainWindow::openRecentDatabase () [private], [slot]
```

Definition at line 401 of file [MainWindowAthlete.cpp](#).

**7.76.3.65 openSettingsDialog**

```
void MainWindow::openSettingsDialog () [private], [slot]
```

Definition at line 73 of file [MainWindowAthlete.cpp](#).

**7.76.3.66 prepareImportFileBatch()**

```
QStringList MainWindow::prepareImportFileBatch (
    const QStringList & filePaths) [private]
```

Definition at line 34 of file [MainWindowImport.cpp](#).

**7.76.3.67 previewFitFile**

```
void MainWindow::previewFitFile () [private], [slot]
```

Definition at line 166 of file [MainWindowImport.cpp](#).

**7.76.3.68 refreshAthleteSelector()**

```
void MainWindow::refreshAthleteSelector () [private]
```

Definition at line 12 of file [MainWindowContext.cpp](#).

**7.76.3.69 refreshClimbViews()**

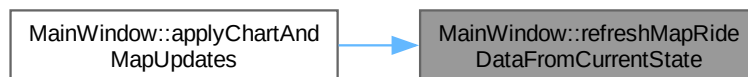
```
void MainWindow::refreshClimbViews (
    double preferredStartSeconds = -1.0,
    double preferredEndSeconds = -1.0) [private]
```

Definition at line 141 of file [MainWindowClimbViews.cpp](#).

**7.76.3.70 refreshMapRideDataFromCurrentState()**

```
void MainWindow::refreshMapRideDataFromCurrentState () [private]
```

Refreshes map route coloring based on current ride data and metric.  
Definition at line 66 of file [MainWindowCharts.cpp](#).  
Referenced by [applyChartAndMapUpdates\(\)](#).  
Here is the caller graph for this function:

**7.76.3.71 removeSelectedClimbs()**

```
void MainWindow::removeSelectedClimbs () [private]
```

Definition at line 194 of file [MainWindowClimbInteractions.cpp](#).

**7.76.3.72 resetAllZoom()**

```
void MainWindow::resetAllZoom () [private]
```

Definition at line 28 of file [MainWindowChartActions.cpp](#).

**7.76.3.73 resolveActiveActivityWeightKg()**

```
double MainWindow::resolveActiveActivityWeightKg () const [private]
```

Definition at line 113 of file [MainWindowDetection.cpp](#).

**7.76.3.74 restoreClimbSelection()**

```
void MainWindow::restoreClimbSelection (
    int previousRow,
    double preferredStartSeconds,
    double preferredEndSeconds) [private]
```

Definition at line 229 of file [MainWindowClimbViews.cpp](#).

**7.76.3.75 revertSelectedClimbToAuto**

```
void MainWindow::revertSelectedClimbToAuto () [private], [slot]
```

Definition at line 693 of file [MainWindowClimbInteractions.cpp](#).



**7.76.3.76 saveSettings()**

```
void MainWindow::saveSettings () [private]
```

Definition at line 77 of file [MainWindowDatabase.cpp](#).

**7.76.3.77 scanWatchDirectories()**

```
void MainWindow::scanWatchDirectories (
    bool initialScan) [private]
```

Definition at line 106 of file [MainWindowWatch.cpp](#).

**7.76.3.78 scheduleActivityBrowserRefresh()**

```
void MainWindow::scheduleActivityBrowserRefresh () [private]
```

Definition at line 89 of file [MainWindowImport.cpp](#).

**7.76.3.79 selectedClimbIndicesFromTable()**

```
std::vector< int > MainWindow::selectedClimbIndicesFromTable () const [private]
```

Definition at line 123 of file [MainWindowClimbInteractions.cpp](#).

**7.76.3.80 selectIntervalRowOffset()**

```
void MainWindow::selectIntervalRowOffset (
    int delta) [private]
```

Definition at line 140 of file [MainWindowIntervals.cpp](#).

**7.76.3.81 setupShortcuts()**

```
void MainWindow::setupShortcuts () [private]
```

Definition at line 121 of file [MainWindowUi.cpp](#).

**7.76.3.82 showAbout**

```
void MainWindow::showAbout () [private], [slot]
```

Definition at line 264 of file [MainWindowDetection.cpp](#).

**7.76.3.83 showWelcomeScreen()**

```
void MainWindow::showWelcomeScreen () [private]
```

Definition at line 195 of file [MainWindowReadiness.cpp](#).

**7.76.3.84 syncAthleteContextToControllers()**

```
void MainWindow::syncAthleteContextToControllers () [private]
```

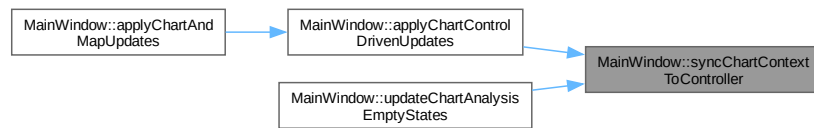
Syncs athlete context to all controllers that need it.  
Definition at line 62 of file [MainWindowStatus.cpp](#).

**7.76.3.85 syncChartContextToController()**

```
void MainWindow::syncChartContextToController (
    bool includeLegendLayout) [private]
```

Syncs current chart-related context to [ChartController](#).  
Definition at line 8 of file [MainWindowCharts.cpp](#).  
Referenced by [applyChartControlDrivenUpdates\(\)](#), and [updateChartAnalysisEmptyStates\(\)](#).

Here is the caller graph for this function:



#### 7.76.3.86 triggerDetectClimbs

```
void MainWindow::triggerDetectClimbs () [private], [slot]
```

Definition at line 19 of file [MainWindowDetection.cpp](#).

#### 7.76.3.87 triggerDetectIntervals

```
void MainWindow::triggerDetectIntervals () [private], [slot]
```

Definition at line 183 of file [MainWindowDetection.cpp](#).

#### 7.76.3.88 updateActivityNotesFromCurrentActivity()

```
void MainWindow::updateActivityNotesFromCurrentActivity () [private]
```

Definition at line 395 of file [MainWindowIntervals.cpp](#).

#### 7.76.3.89 updateAthleteLabel()

```
void MainWindow::updateAthleteLabel () [private]
```

Definition at line 392 of file [MainWindowAthlete.cpp](#).

#### 7.76.3.90 updateChartAnalysisEmptyStates()

```
void MainWindow::updateChartAnalysisEmptyStates () [private]
```

Updates analysis tab empty-state visibility through [ChartController](#).

Definition at line 57 of file [MainWindowCharts.cpp](#).

References [syncChartContextToController\(\)](#).

Here is the call graph for this function:



#### 7.76.3.91 updateClimbingTab()

```
void MainWindow::updateClimbingTab () [private]
```

Definition at line 120 of file [MainWindowClimbViews.cpp](#).

**7.76.3.92 updateClimbQuarterCharts()**

```
void MainWindow::updateClimbQuarterCharts (  
    const Climb * climb) [private]
```

Definition at line 260 of file [MainWindowClimbViews.cpp](#).

**7.76.3.93 updateClimbRowStyles()**

```
void MainWindow::updateClimbRowStyles () [private]
```

Definition at line 319 of file [MainWindowClimbInteractions.cpp](#).

**7.76.3.94 updateImportAvailability()**

```
void MainWindow::updateImportAvailability () [private]
```

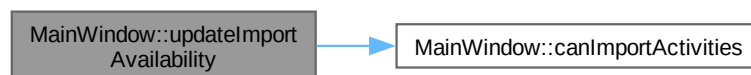
Enables/disables import actions based on current readiness.

Definition at line 173 of file [MainWindowReadiness.cpp](#).

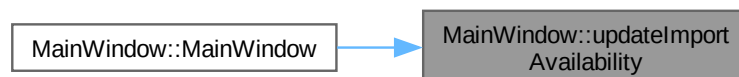
References [canImportActivities\(\)](#).

Referenced by [MainWindow\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.76.3.95 updateIntervalRowStyles()**

```
void MainWindow::updateIntervalRowStyles () [private]
```

Definition at line 158 of file [MainWindowIntervals.cpp](#).

**7.76.3.96 updateIntervals()**

```
void MainWindow::updateIntervals () [private]
```

Definition at line 272 of file [MainWindowIntervals.cpp](#).

**7.76.3.97 updateIntervalSummary()**

```
void MainWindow::updateIntervalSummary (  
    double startSeconds,  
    double endSeconds) [private]
```

Definition at line 185 of file [MainWindowIntervals.cpp](#).

#### 7.76.3.98 updateLapsSummaryFromCurrentActivity()

void MainWindow::updateLapsSummaryFromCurrentActivity () [private]  
Definition at line 365 of file [MainWindowIntervals.cpp](#).

#### 7.76.3.99 updateRecentDatabaseMenu()

void MainWindow::updateRecentDatabaseMenu () [private]  
Definition at line 222 of file [MainWindowDatabase.cpp](#).

#### 7.76.3.100 updateStatsLabel()

void MainWindow::updateStatsLabel () [private]  
Definition at line 71 of file [MainWindowStatus.cpp](#).

#### 7.76.3.101 updateStatusBarInfo()

void MainWindow::updateStatusBarInfo () [private]  
Definition at line 33 of file [MainWindowStatus.cpp](#).

#### 7.76.3.102 updateWelcomeScreenVisibility()

void MainWindow::updateWelcomeScreenVisibility () [private]  
Definition at line 221 of file [MainWindowReadiness.cpp](#).

### 7.76.4 Member Data Documentation

#### 7.76.4.1 m\_actionsManager

class [MainWindowActions](#)\* MainWindow::m\_actionsManager = nullptr [private]  
Definition at line 244 of file [MainWindow.h](#).

#### 7.76.4.2 m\_activitiesPage

class [ActivitiesPage](#)\* MainWindow::m\_activitiesPage = nullptr [private]  
Definition at line 398 of file [MainWindow.h](#).

#### 7.76.4.3 m\_activityBrowser

[ActivityBrowser](#)\* MainWindow::m\_activityBrowser = nullptr [private]  
Definition at line 395 of file [MainWindow.h](#).

#### 7.76.4.4 m\_activityCountLabel

QLabel\* MainWindow::m\_activityCountLabel = nullptr [private]  
Definition at line 284 of file [MainWindow.h](#).

#### 7.76.4.5 m\_activityNotesView

QTextEdit\* MainWindow::m\_activityNotesView = nullptr [private]  
Definition at line 367 of file [MainWindow.h](#).

#### 7.76.4.6 m\_activityTabStack

QStackedLayout\* MainWindow::m\_activityTabStack = nullptr [private]  
Definition at line 307 of file [MainWindow.h](#).

#### 7.76.4.7 m\_activityViewController

class [ActivityViewController](#)\* MainWindow::m\_activityViewController = nullptr [private]  
Definition at line 238 of file [MainWindow.h](#).

#### 7.76.4.8 m\_altitudeChart

`RideChartWidget*` MainWindow::m\_altitudeChart = nullptr [private]  
Definition at line 325 of file [MainWindow.h](#).

#### 7.76.4.9 m\_analysisQueue

`AnalysisQueue*` MainWindow::m\_analysisQueue = nullptr [private]  
Definition at line 228 of file [MainWindow.h](#).

#### 7.76.4.10 m\_analysisTabWidget

`QTabWidget*` MainWindow::m\_analysisTabWidget = nullptr [private]  
Definition at line 306 of file [MainWindow.h](#).

#### 7.76.4.11 m\_athleteCombo

`QComboBox*` MainWindow::m\_athleteCombo = nullptr [private]  
Definition at line 274 of file [MainWindow.h](#).

#### 7.76.4.12 m\_athleteHeader

`AthleteHeaderWidget*` MainWindow::m\_athleteHeader = nullptr [private]  
Definition at line 287 of file [MainWindow.h](#).

#### 7.76.4.13 m\_athleteLabelAct

`QAction*` MainWindow::m\_athleteLabelAct = nullptr [private]  
Definition at line 253 of file [MainWindow.h](#).

#### 7.76.4.14 m\_athletesMenu

`QMenu*` MainWindow::m\_athletesMenu = nullptr [private]  
Definition at line 252 of file [MainWindow.h](#).

#### 7.76.4.15 m\_athleteStatusLabel

`QLabel*` MainWindow::m\_athleteStatusLabel = nullptr [private]  
Definition at line 283 of file [MainWindow.h](#).

#### 7.76.4.16 m\_autoSmoothingCheck

`QCheckBox*` MainWindow::m\_autoSmoothingCheck = nullptr [private]  
Definition at line 278 of file [MainWindow.h](#).

#### 7.76.4.17 m\_cadenceChart

`RideChartWidget*` MainWindow::m\_cadenceChart = nullptr [private]  
Definition at line 323 of file [MainWindow.h](#).

#### 7.76.4.18 m\_calendarPageContent

`QWidget*` MainWindow::m\_calendarPageContent = nullptr [private]  
Calendar content.  
Definition at line 300 of file [MainWindow.h](#).

#### 7.76.4.19 m\_calendarTabStack

`QStackedLayout*` MainWindow::m\_calendarTabStack = nullptr [private]  
Definition at line 311 of file [MainWindow.h](#).

#### 7.76.4.20 m\_calendarWidget

`CalendarWidget* MainWindow::m_calendarWidget = nullptr [private]`  
Definition at line 392 of file [MainWindow.h](#).

#### 7.76.4.21 m\_chartController

`class ChartController* MainWindow::m_chartController = nullptr [private]`  
Definition at line 239 of file [MainWindow.h](#).

#### 7.76.4.22 m\_chartHeight

`int MainWindow::m_chartHeight = 220 [private]`  
Definition at line 320 of file [MainWindow.h](#).

#### 7.76.4.23 m\_chartHeightDecreaseButton

`QPushButton* MainWindow::m_chartHeightDecreaseButton = nullptr [private]`  
Definition at line 340 of file [MainWindow.h](#).

#### 7.76.4.24 m\_chartHeightIncreaseButton

`QPushButton* MainWindow::m_chartHeightIncreaseButton = nullptr [private]`  
Definition at line 339 of file [MainWindow.h](#).

#### 7.76.4.25 m\_chartMapSplit

`QSplitter* MainWindow::m_chartMapSplit = nullptr [private]`  
Definition at line 319 of file [MainWindow.h](#).

#### 7.76.4.26 m\_chartPresetCombo

`QComboBox* MainWindow::m_chartPresetCombo = nullptr [private]`  
Definition at line 276 of file [MainWindow.h](#).

#### 7.76.4.27 m\_chartsContainer

`QWidget* MainWindow::m_chartsContainer = nullptr [private]`  
Definition at line 317 of file [MainWindow.h](#).

#### 7.76.4.28 m\_chartScroll

`QScrollArea* MainWindow::m_chartScroll = nullptr [private]`  
Definition at line 316 of file [MainWindow.h](#).

#### 7.76.4.29 m\_chartsLayout

`QVBoxLayout* MainWindow::m_chartsLayout = nullptr [private]`  
Definition at line 318 of file [MainWindow.h](#).

#### 7.76.4.30 m\_chartsPageContent

`QWidget* MainWindow::m_chartsPageContent = nullptr [private]`  
Charts+map+intervals content.  
Definition at line 296 of file [MainWindow.h](#).

#### 7.76.4.31 m\_climbAltitudeChart

`RideChartWidget* MainWindow::m_climbAltitudeChart = nullptr [private]`  
Definition at line 328 of file [MainWindow.h](#).

#### 7.76.4.32 m\_climbCadenceChart

`RideChartWidget* MainWindow::m_climbCadenceChart = nullptr [private]`  
Definition at line 331 of file [MainWindow.h](#).

#### 7.76.4.33 m\_climbDipDistanceSpin

`QDoubleSpinBox* MainWindow::m_climbDipDistanceSpin = nullptr [private]`  
Definition at line 379 of file [MainWindow.h](#).

#### 7.76.4.34 m\_climbDipMetersSpin

`QDoubleSpinBox* MainWindow::m_climbDipMetersSpin = nullptr [private]`  
Definition at line 378 of file [MainWindow.h](#).

#### 7.76.4.35 m\_climbHrChart

`RideChartWidget* MainWindow::m_climbHrChart = nullptr [private]`  
Definition at line 330 of file [MainWindow.h](#).

#### 7.76.4.36 m\_climbingTabStack

`QStackedLayout* MainWindow::m_climbingTabStack = nullptr [private]`  
Definition at line 313 of file [MainWindow.h](#).

#### 7.76.4.37 m\_climbMinGainSpin

`QDoubleSpinBox* MainWindow::m_climbMinGainSpin = nullptr [private]`  
Definition at line 375 of file [MainWindow.h](#).

#### 7.76.4.38 m\_climbMinGradientSpin

`QDoubleSpinBox* MainWindow::m_climbMinGradientSpin = nullptr [private]`  
Definition at line 376 of file [MainWindow.h](#).

#### 7.76.4.39 m\_climbMinLengthSpin

`QDoubleSpinBox* MainWindow::m_climbMinLengthSpin = nullptr [private]`  
Definition at line 374 of file [MainWindow.h](#).

#### 7.76.4.40 m\_climbOverlayEnabledCheck

`QCheckBox* MainWindow::m_climbOverlayEnabledCheck = nullptr [private]`  
Definition at line 381 of file [MainWindow.h](#).

#### 7.76.4.41 m\_climbOverlayMetricCombo

`QComboBox* MainWindow::m_climbOverlayMetricCombo = nullptr [private]`  
Definition at line 382 of file [MainWindow.h](#).

#### 7.76.4.42 m\_climbPowerChart

`RideChartWidget* MainWindow::m_climbPowerChart = nullptr [private]`  
Definition at line 329 of file [MainWindow.h](#).

#### 7.76.4.43 m\_climbQuarterCadenceChart

`QChartView* MainWindow::m_climbQuarterCadenceChart = nullptr [private]`  
Definition at line 335 of file [MainWindow.h](#).

#### 7.76.4.44 m\_climbQuarterHrChart

QChartView\* MainWindow::m\_climbQuarterHrChart = nullptr [private]  
Definition at line 334 of file [MainWindow.h](#).

#### 7.76.4.45 m\_climbQuarterPowerChart

QChartView\* MainWindow::m\_climbQuarterPowerChart = nullptr [private]  
Definition at line 333 of file [MainWindow.h](#).

#### 7.76.4.46 m\_climbSmoothingSpin

QDoubleSpinBox\* MainWindow::m\_climbSmoothingSpin = nullptr [private]  
Definition at line 380 of file [MainWindow.h](#).

#### 7.76.4.47 m\_climbsPageContent

QWidget\* MainWindow::m\_climbsPageContent = nullptr [private]  
[Climb](#) charts + table content.  
Definition at line 302 of file [MainWindow.h](#).

#### 7.76.4.48 m\_climbSpeedChart

RideChartWidget\* MainWindow::m\_climbSpeedChart = nullptr [private]  
Definition at line 332 of file [MainWindow.h](#).

#### 7.76.4.49 m\_climbsTable

QTableWidget\* MainWindow::m\_climbsTable = nullptr [private]  
Definition at line 372 of file [MainWindow.h](#).

#### 7.76.4.50 m\_climbStartGradientSpin

QDoubleSpinBox\* MainWindow::m\_climbStartGradientSpin = nullptr [private]  
Definition at line 377 of file [MainWindow.h](#).

#### 7.76.4.51 m\_climbSummaryLabel

QLabel\* MainWindow::m\_climbSummaryLabel = nullptr [private]  
Definition at line 373 of file [MainWindow.h](#).

#### 7.76.4.52 m\_colorLegendLayout

QHBoxLayout\* MainWindow::m\_colorLegendLayout = nullptr [private]  
Definition at line 343 of file [MainWindow.h](#).

#### 7.76.4.53 m\_colorLegendWidget

QWidget\* MainWindow::m\_colorLegendWidget = nullptr [private]  
Definition at line 342 of file [MainWindow.h](#).

#### 7.76.4.54 m\_colorMetricCombo

QComboBox\* MainWindow::m\_colorMetricCombo = nullptr [private]  
Definition at line 275 of file [MainWindow.h](#).

#### 7.76.4.55 m\_controller

WorkoutController\* MainWindow::m\_controller = nullptr [private]  
Definition at line 235 of file [MainWindow.h](#).



**7.76.4.56 m\_currentAthleteId**

```
int MainWindow::m_currentAthleteId = -1 [private]
```

Definition at line 248 of file [MainWindow.h](#).

**7.76.4.57 m\_currentAthleteName**

```
QString MainWindow::m_currentAthleteName [private]
```

Definition at line 249 of file [MainWindow.h](#).

**7.76.4.58 m\_dbManager**

```
DatabaseManager MainWindow::m_dbManager [private]
```

Definition at line 247 of file [MainWindow.h](#).

**7.76.4.59 m\_dbStatusLabel**

```
QLabel* MainWindow::m_dbStatusLabel = nullptr [private]
```

Definition at line 282 of file [MainWindow.h](#).

**7.76.4.60 m\_detectedClimbs**

```
std::vector<Climb> MainWindow::m_detectedClimbs [private]
```

Definition at line 383 of file [MainWindow.h](#).

**7.76.4.61 m\_firstLaunchCompleted**

```
bool MainWindow::m_firstLaunchCompleted = false [private]
```

Definition at line 263 of file [MainWindow.h](#).

**7.76.4.62 m\_fitChartsButton**

```
QPushButton* MainWindow::m_fitChartsButton = nullptr [private]
```

Definition at line 338 of file [MainWindow.h](#).

**7.76.4.63 m\_fitnessChart**

```
FitnessChartWidget* MainWindow::m_fitnessChart = nullptr [private]
```

Definition at line 362 of file [MainWindow.h](#).

**7.76.4.64 m\_fitnessPageContent**

```
QWidget* MainWindow::m_fitnessPageContent = nullptr [private]
```

Fitness chart content.  
Definition at line 301 of file [MainWindow.h](#).

**7.76.4.65 m\_fitnessTabStack**

```
QStackedLayout* MainWindow::m_fitnessTabStack = nullptr [private]
```

Definition at line 312 of file [MainWindow.h](#).

**7.76.4.66 m\_followIntervalSelectionCheck**

```
QCheckBox* MainWindow::m_followIntervalSelectionCheck = nullptr [private]
```

Definition at line 368 of file [MainWindow.h](#).

**7.76.4.67 m\_histogram**

```
PowerHistogramWidget* MainWindow::m_histogram = nullptr [private]
```

Definition at line 356 of file [MainWindow.h](#).

#### 7.76.4.68 m\_histogramPageContent

`QWidget* MainWindow::m_histogramPageContent = nullptr [private]`

Histogram (Power page sub-tab).

Definition at line 298 of file [MainWindow.h](#).

#### 7.76.4.69 m\_histogramTabStack

`QStackedLayout* MainWindow::m_histogramTabStack = nullptr [private]`

Definition at line 309 of file [MainWindow.h](#).

#### 7.76.4.70 m\_hrChart

`RideChartWidget* MainWindow::m_hrChart = nullptr [private]`

Definition at line 322 of file [MainWindow.h](#).

#### 7.76.4.71 m\_importAct

`QAction* MainWindow::m_importAct = nullptr [private]`

Definition at line 254 of file [MainWindow.h](#).

#### 7.76.4.72 m\_importBatchSummaries

`QHash<QString, ImportBatchSummary> MainWindow::m_importBatchSummaries [private]`

Definition at line 411 of file [MainWindow.h](#).

#### 7.76.4.73 m\_importController

`class ImportController* MainWindow::m_importController = nullptr [private]`

Definition at line 241 of file [MainWindow.h](#).

#### 7.76.4.74 m\_importProgressModel

`ImportProgressModel* MainWindow::m_importProgressModel = nullptr [private]`

Definition at line 232 of file [MainWindow.h](#).

#### 7.76.4.75 m\_importQueue

`ImportQueue* MainWindow::m_importQueue = nullptr [private]`

Definition at line 231 of file [MainWindow.h](#).

#### 7.76.4.76 m\_importRefreshTimer

`QTimer* MainWindow::m_importRefreshTimer = nullptr [private]`

Definition at line 268 of file [MainWindow.h](#).

#### 7.76.4.77 m\_importStatusWidget

`ImportStatusWidget* MainWindow::m_importStatusWidget = nullptr [private]`

Definition at line 279 of file [MainWindow.h](#).

#### 7.76.4.78 m\_intervalsPageContent

`QWidget* MainWindow::m_intervalsPageContent = nullptr [private]`

Intervals table + tools content.

Definition at line 303 of file [MainWindow.h](#).

#### 7.76.4.79 m\_intervalsTable

`QTableWidget* MainWindow::m_intervalsTable = nullptr [private]`

Definition at line 365 of file [MainWindow.h](#).

#### 7.76.4.80 m\_intervalSummaryLabel

QLabel\* MainWindow::m\_intervalSummaryLabel = nullptr [private]  
Definition at line 369 of file [MainWindow.h](#).

#### 7.76.4.81 m\_knownWatchedFitFiles

QSet<QString> MainWindow::m\_knownWatchedFitFiles [private]  
Definition at line 271 of file [MainWindow.h](#).

#### 7.76.4.82 m\_lapsTable

QTableWidget\* MainWindow::m\_lapsTable = nullptr [private]  
Definition at line 366 of file [MainWindow.h](#).

#### 7.76.4.83 m\_lastOpenDir

QString MainWindow::m\_lastOpenDir [private]  
Definition at line 259 of file [MainWindow.h](#).

#### 7.76.4.84 m\_mapAutoContrastCheck

QCheckBox\* MainWindow::m\_mapAutoContrastCheck = nullptr [private]  
Definition at line 389 of file [MainWindow.h](#).

#### 7.76.4.85 m\_mapController

class [MapController](#)\* MainWindow::m\_mapController = nullptr [private]  
Definition at line 240 of file [MainWindow.h](#).

#### 7.76.4.86 m\_mapRenderer

[MapRenderer](#)\* MainWindow::m\_mapRenderer = nullptr [private]  
Definition at line 387 of file [MainWindow.h](#).

#### 7.76.4.87 m\_mapStyleCombo

QComboBox\* MainWindow::m\_mapStyleCombo = nullptr [private]  
Definition at line 388 of file [MainWindow.h](#).

#### 7.76.4.88 m\_navigationController

class [NavigationController](#)\* MainWindow::m\_navigationController = nullptr [private]  
Definition at line 242 of file [MainWindow.h](#).

#### 7.76.4.89 m\_navigationSidebar

[NavigationSidebar](#)\* MainWindow::m\_navigationSidebar = nullptr [private]  
Definition at line 243 of file [MainWindow.h](#).

#### 7.76.4.90 m\_pageStack

QStackedWidget\* MainWindow::m\_pageStack = nullptr [private]  
Definition at line 292 of file [MainWindow.h](#).

#### 7.76.4.91 m\_pdcPageContent

QWidget\* MainWindow::m\_pdcPageContent = nullptr [private]  
Power Curve (Power page sub-tab).  
Definition at line 299 of file [MainWindow.h](#).

#### 7.76.4.92 m\_pdcWidget

`PowerCurveWidget* MainWindow::m_pdcWidget = nullptr [private]`  
Definition at line 359 of file [MainWindow.h](#).

#### 7.76.4.93 m\_powerChart

`RideChartWidget* MainWindow::m_powerChart = nullptr [private]`  
Definition at line 321 of file [MainWindow.h](#).

#### 7.76.4.94 m\_powerCurveTabStack

`QStackedLayout* MainWindow::m_powerCurveTabStack = nullptr [private]`  
Definition at line 310 of file [MainWindow.h](#).

#### 7.76.4.95 m\_powerSmoothingCombo

`QComboBox* MainWindow::m_powerSmoothingCombo = nullptr [private]`  
Definition at line 277 of file [MainWindow.h](#).

#### 7.76.4.96 m\_previewAct

`QAction* MainWindow::m_previewAct = nullptr [private]`  
Definition at line 257 of file [MainWindow.h](#).

#### 7.76.4.97 m\_recentDbMenu

`QMenu* MainWindow::m_recentDbMenu = nullptr [private]`  
Definition at line 258 of file [MainWindow.h](#).

#### 7.76.4.98 m\_showAltitude

`QCheckBox* MainWindow::m_showAltitude = nullptr [private]`  
Definition at line 350 of file [MainWindow.h](#).

#### 7.76.4.99 m\_showCadence

`QCheckBox* MainWindow::m_showCadence = nullptr [private]`  
Definition at line 348 of file [MainWindow.h](#).

#### 7.76.4.100 m\_showHR

`QCheckBox* MainWindow::m_showHR = nullptr [private]`  
Definition at line 347 of file [MainWindow.h](#).

#### 7.76.4.101 m\_showPower

`QCheckBox* MainWindow::m_showPower = nullptr [private]`  
Definition at line 346 of file [MainWindow.h](#).

#### 7.76.4.102 m\_showSpeed

`QCheckBox* MainWindow::m_showSpeed = nullptr [private]`  
Definition at line 349 of file [MainWindow.h](#).

#### 7.76.4.103 m\_speedChart

`RideChartWidget* MainWindow::m_speedChart = nullptr [private]`  
Definition at line 324 of file [MainWindow.h](#).

**7.76.4.104 m\_suppressClimbAutoZoom**

`bool MainWindow::m_suppressClimbAutoZoom = false [private]`  
Definition at line 384 of file [MainWindow.h](#).

**7.76.4.105 m\_toolbarImportAct**

`QAction* MainWindow::m_toolbarImportAct = nullptr [private]`  
Definition at line 255 of file [MainWindow.h](#).

**7.76.4.106 m\_toolbarSearchAct**

`QAction* MainWindow::m_toolbarSearchAct = nullptr [private]`  
Definition at line 256 of file [MainWindow.h](#).

**7.76.4.107 m\_undoManager**

`UndoManager* MainWindow::m_undoManager = nullptr [private]`  
Definition at line 225 of file [MainWindow.h](#).

**7.76.4.108 m\_useEstimatedFtpButton**

`QPushButton* MainWindow::m_useEstimatedFtpButton = nullptr [private]`  
Definition at line 341 of file [MainWindow.h](#).

**7.76.4.109 m\_watcher**

`QFileSystemWatcher* MainWindow::m_watcher = nullptr [private]`  
Definition at line 266 of file [MainWindow.h](#).

**7.76.4.110 m\_watchFolderEnabled**

`bool MainWindow::m_watchFolderEnabled = true [private]`  
Definition at line 269 of file [MainWindow.h](#).

**7.76.4.111 m\_watchFolderPath**

`QString MainWindow::m_watchFolderPath [private]`  
Definition at line 270 of file [MainWindow.h](#).

**7.76.4.112 m\_watchRescanTimer**

`QTimer* MainWindow::m_watchRescanTimer = nullptr [private]`  
Definition at line 267 of file [MainWindow.h](#).

**7.76.4.113 m\_welcomeWidget**

`WelcomeWidget* MainWindow::m_welcomeWidget = nullptr [private]`  
Definition at line 262 of file [MainWindow.h](#).

**7.76.4.114 m\_zonesPageContent**

`QWidget* MainWindow::m_zonesPageContent = nullptr [private]`  
Zones (Power page sub-tab).  
Definition at line 297 of file [MainWindow.h](#).

**7.76.4.115 m\_zonesTable**

`QTableWidget* MainWindow::m_zonesTable = nullptr [private]`  
Definition at line 353 of file [MainWindow.h](#).

#### 7.76.4.116 `m_zonesTabStack`

`QStackedLayout* MainWindow::m_zonesTabStack = nullptr [private]`

Definition at line 308 of file [MainWindow.h](#).

The documentation for this class was generated from the following files:

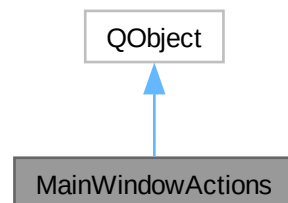
- `src/gui/MainWindow.h`
- `src/gui/MainWindow.cpp`
- `src/gui/MainWindowAthlete.cpp`
- `src/gui/MainWindowChartActions.cpp`
- `src/gui/MainWindowCharts.cpp`
- `src/gui/MainWindowClimbInteractions.cpp`
- `src/gui/MainWindowClimbViews.cpp`
- `src/gui/MainWindowContext.cpp`
- `src/gui/MainWindowDatabase.cpp`
- `src/gui/MainWindowDetection.cpp`
- `src/gui/MainWindowImport.cpp`
- `src/gui/MainWindowIntervals.cpp`
- `src/gui/MainWindowReadiness.cpp`
- `src/gui/MainWindowStatus.cpp`
- `src/gui/MainWindowUi.cpp`
- `src/gui/MainWindowUiBuild.cpp`
- `src/gui/MainWindowVideo.cpp`
- `src/gui/MainWindowWatch.cpp`

## 7.77 `MainWindowActions` Class Reference

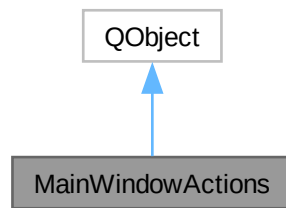
Manages menus, toolbars, actions, and keyboard shortcuts.

`#include <MainWindowActions.h>`

Inheritance diagram for `MainWindowActions`:



Collaboration diagram for MainWindowActions:



## Signals

- void **importTriggered** ()  
*Emitted when import action is triggered.*
- void **createVideoTriggered** ()  
*Emitted when create video action is triggered.*
- void **previewFitTriggered** ()  
*Emitted when preview FIT action is triggered.*
- void **openDatabaseRequested** ()  
*Emitted when database management is requested.*
- void **createDatabaseRequested** ()  
*Emitted when create database is requested.*
- void **manageAthletesRequested** ()  
*Emitted when athlete management is requested.*
- void **openSettingsRequested** ()  
*Emitted when settings dialog is requested.*
- void **showAboutRequested** ()  
*Emitted when about dialog is requested.*
- void **detectClimbsRequested** ()  
*Emitted when detect climbs is requested.*
- void **detectIntervalsRequested** ()  
*Emitted when detect intervals is requested.*
- void **athleteSelectionChanged** (int athleteId)  
*Emitted when athlete selection changes.*

## Public Member Functions

- [MainWindowActions](#) (QMainWindow \*mainWindow, QObject \*parent=nullptr)  
*Constructs the actions manager.*
- **~MainWindowActions** () override  
*Destructor.*
- void [buildMenus](#) ()  
*Creates and sets up all menus.*
- void [buildToolbar](#) ()  
*Creates and sets up the main toolbar.*
- void [setupShortcuts](#) ()  
*Sets up keyboard shortcuts.*

- QAction \* [importAction](#) () const
- QAction \* [toolbarImportAction](#) () const
- QAction \* [createVideoAction](#) () const
- QAction \* [previewFitAction](#) () const
- QMenu \* [recentDatabaseMenu](#) () const
- QComboBox \* [athleteCombo](#) () const

### Private Member Functions

- void [createFileMenu](#) ()  
*Helper to create File menu.*
- void [createEditMenu](#) ()  
*Helper to create Edit menu.*
- void [createViewMenu](#) ()  
*Helper to create View menu.*
- void [createAnalysisMenu](#) ()  
*Helper to create Analysis menu.*
- void [createToolsMenu](#) ()  
*Helper to create Tools menu.*
- void [createHelpMenu](#) ()  
*Helper to create Help menu.*

### Private Attributes

- QMainWindow \* [m\\_mainWindow](#) = nullptr
- QAction \* [m\\_importAct](#) = nullptr
- QAction \* [m\\_previewFitAct](#) = nullptr
- QAction \* [m\\_openDatabaseAct](#) = nullptr
- QAction \* [m\\_createDatabaseAct](#) = nullptr
- QMenu \* [m\\_recentDbMenu](#) = nullptr
- QAction \* [m\\_backupDatabaseAct](#) = nullptr
- QAction \* [m\\_exitAct](#) = nullptr
- QAction \* [m\\_manageAthletesAct](#) = nullptr
- QAction \* [m\\_openSettingsAct](#) = nullptr
- QAction \* [m\\_fullscreenAct](#) = nullptr
- QAction \* [m\\_detectClimbsAct](#) = nullptr
- QAction \* [m\\_detectIntervalsAct](#) = nullptr
- QAction \* [m\\_revertClimbAct](#) = nullptr
- QAction \* [m\\_createVideoAct](#) = nullptr
- QAction \* [m\\_aboutAct](#) = nullptr
- QAction \* [m\\_toolbarImportAct](#) = nullptr
- QAction \* [m\\_toolbarSearchAct](#) = nullptr
- QComboBox \* [m\\_athleteCombo](#) = nullptr

### 7.77.1 Detailed Description

Manages menus, toolbars, actions, and keyboard shortcuts.  
Handles:

- File menu (import, open database, create database)
- Edit menu (athlete management, settings)
- View menu (full screen)
- Analysis menu (detect climbs, detect intervals)



- Tools menu (create video, athlete management)
- Help menu (about)
- Main toolbar (import, search, athlete selector, import status)
- Keyboard shortcuts (F11, Ctrl+O, Ctrl+Z, etc.)

Definition at line 28 of file [MainWindowActions.h](#).

## 7.77.2 Constructor & Destructor Documentation

### 7.77.2.1 MainWindowActions()

```
MainWindowActions::MainWindowActions (
    QMainWindow * mainWindow,
    QObject * parent = nullptr) [explicit]
```

Constructs the actions manager.

#### Parameters

|                   |                             |
|-------------------|-----------------------------|
| <i>mainWindow</i> | Pointer to the main window. |
| <i>parent</i>     | Parent object.              |

Definition at line 19 of file [MainWindowActions.cpp](#).

## 7.77.3 Member Function Documentation

### 7.77.3.1 athleteCombo()

```
QComboBox * MainWindowActions::athleteCombo () const [inline]
```

Definition at line 68 of file [MainWindowActions.h](#).

### 7.77.3.2 buildMenus()

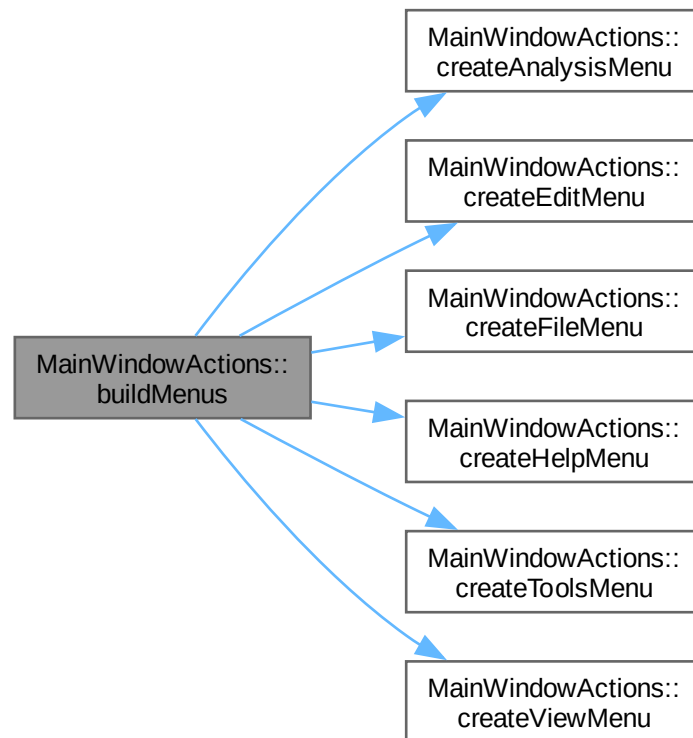
```
void MainWindowActions::buildMenus ()
```

Creates and sets up all menus.

Definition at line 33 of file [MainWindowActions.cpp](#).

References [createAnalysisMenu\(\)](#), [createEditMenu\(\)](#), [createFileMenu\(\)](#), [createHelpMenu\(\)](#), [createToolsMenu\(\)](#), and [createViewMenu\(\)](#).

Here is the call graph for this function:



### 7.77.3.3 buildToolbar()

```
void MainWindowActions::buildToolbar ()
```

Creates and sets up the main toolbar.

Definition at line 49 of file [MainWindowActions.cpp](#).

### 7.77.3.4 createAnalysisMenu()

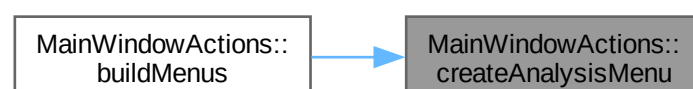
```
void MainWindowActions::createAnalysisMenu () [private]
```

Helper to create Analysis menu.

Definition at line 107 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.5 createEditMenu()

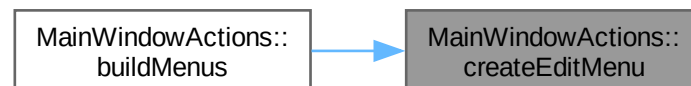
```
void MainWindowActions::createEditMenu () [private]
```

Helper to create Edit menu.

Definition at line 81 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.6 createFileMenu()

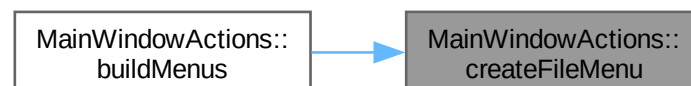
```
void MainWindowActions::createFileMenu () [private]
```

Helper to create File menu.

Definition at line 68 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.7 createHelpMenu()

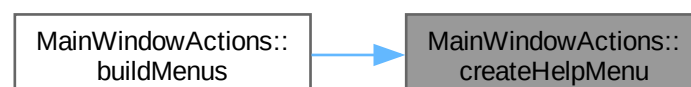
```
void MainWindowActions::createHelpMenu () [private]
```

Helper to create Help menu.

Definition at line 133 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.8 createToolsMenu()

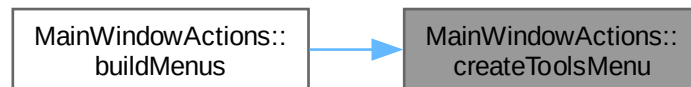
```
void MainWindowActions::createToolsMenu () [private]
```

Helper to create Tools menu.

Definition at line 120 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.9 createVideoAction()

```
QAction * MainWindowActions::createVideoAction () const [inline]
```

Definition at line 63 of file [MainWindowActions.h](#).

### 7.77.3.10 createViewMenu()

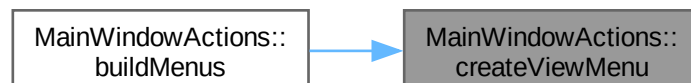
```
void MainWindowActions::createViewMenu () [private]
```

Helper to create View menu.

Definition at line 94 of file [MainWindowActions.cpp](#).

Referenced by [buildMenus\(\)](#).

Here is the caller graph for this function:



### 7.77.3.11 importAction()

```
QAction * MainWindowActions::importAction () const [inline]
```

Definition at line 61 of file [MainWindowActions.h](#).

### 7.77.3.12 previewFitAction()

```
QAction * MainWindowActions::previewFitAction () const [inline]
```

Definition at line 64 of file [MainWindowActions.h](#).

### 7.77.3.13 recentDatabaseMenu()

```
QMenu * MainWindowActions::recentDatabaseMenu () const [inline]
```

Definition at line 65 of file [MainWindowActions.h](#).

#### 7.77.3.14 setupShortcuts()

```
void MainWindowActions::setupShortcuts ()
```

Sets up keyboard shortcuts.

Definition at line 60 of file [MainWindowActions.cpp](#).

#### 7.77.3.15 toolbarImportAction()

```
QAction * MainWindowActions::toolbarImportAction () const [inline]
```

Definition at line 62 of file [MainWindowActions.h](#).

### 7.77.4 Member Data Documentation

#### 7.77.4.1 m\_aboutAct

```
QAction* MainWindowActions::m_aboutAct = nullptr [private]
```

Definition at line 150 of file [MainWindowActions.h](#).

#### 7.77.4.2 m\_athleteCombo

```
QComboBox* MainWindowActions::m_athleteCombo = nullptr [private]
```

Definition at line 155 of file [MainWindowActions.h](#).

#### 7.77.4.3 m\_backupDatabaseAct

```
QAction* MainWindowActions::m_backupDatabaseAct = nullptr [private]
```

Definition at line 131 of file [MainWindowActions.h](#).

#### 7.77.4.4 m\_createDatabaseAct

```
QAction* MainWindowActions::m_createDatabaseAct = nullptr [private]
```

Definition at line 129 of file [MainWindowActions.h](#).

#### 7.77.4.5 m\_createVideoAct

```
QAction* MainWindowActions::m_createVideoAct = nullptr [private]
```

Definition at line 147 of file [MainWindowActions.h](#).

#### 7.77.4.6 m\_detectClimbsAct

```
QAction* MainWindowActions::m_detectClimbsAct = nullptr [private]
```

Definition at line 142 of file [MainWindowActions.h](#).

#### 7.77.4.7 m\_detectIntervalsAct

```
QAction* MainWindowActions::m_detectIntervalsAct = nullptr [private]
```

Definition at line 143 of file [MainWindowActions.h](#).

#### 7.77.4.8 m\_exitAct

```
QAction* MainWindowActions::m_exitAct = nullptr [private]
```

Definition at line 132 of file [MainWindowActions.h](#).

#### 7.77.4.9 m\_fullscreenAct

```
QAction* MainWindowActions::m_fullscreenAct = nullptr [private]
```

Definition at line 139 of file [MainWindowActions.h](#).

#### 7.77.4.10 m\_importAct

`QAction* MainWindowActions::m_importAct = nullptr [private]`  
Definition at line 126 of file [MainWindowActions.h](#).

#### 7.77.4.11 m\_mainWindow

`QMainWindow* MainWindowActions::m_mainWindow = nullptr [private]`  
Definition at line 123 of file [MainWindowActions.h](#).

#### 7.77.4.12 m\_manageAthletesAct

`QAction* MainWindowActions::m_manageAthletesAct = nullptr [private]`  
Definition at line 135 of file [MainWindowActions.h](#).

#### 7.77.4.13 m\_openDatabaseAct

`QAction* MainWindowActions::m_openDatabaseAct = nullptr [private]`  
Definition at line 128 of file [MainWindowActions.h](#).

#### 7.77.4.14 m\_openSettingsAct

`QAction* MainWindowActions::m_openSettingsAct = nullptr [private]`  
Definition at line 136 of file [MainWindowActions.h](#).

#### 7.77.4.15 m\_previewFitAct

`QAction* MainWindowActions::m_previewFitAct = nullptr [private]`  
Definition at line 127 of file [MainWindowActions.h](#).

#### 7.77.4.16 m\_recentDbMenu

`QMenu* MainWindowActions::m_recentDbMenu = nullptr [private]`  
Definition at line 130 of file [MainWindowActions.h](#).

#### 7.77.4.17 m\_revertClimbAct

`QAction* MainWindowActions::m_revertClimbAct = nullptr [private]`  
Definition at line 144 of file [MainWindowActions.h](#).

#### 7.77.4.18 m\_toolbarImportAct

`QAction* MainWindowActions::m_toolbarImportAct = nullptr [private]`  
Definition at line 153 of file [MainWindowActions.h](#).

#### 7.77.4.19 m\_toolbarSearchAct

`QAction* MainWindowActions::m_toolbarSearchAct = nullptr [private]`  
Definition at line 154 of file [MainWindowActions.h](#).

The documentation for this class was generated from the following files:

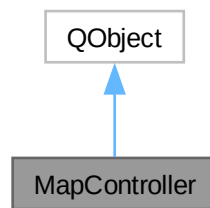
- [src/gui/controllers/MainWindowActions.h](#)
- [src/gui/controllers/MainWindowActions.cpp](#)

## 7.78 MapController Class Reference

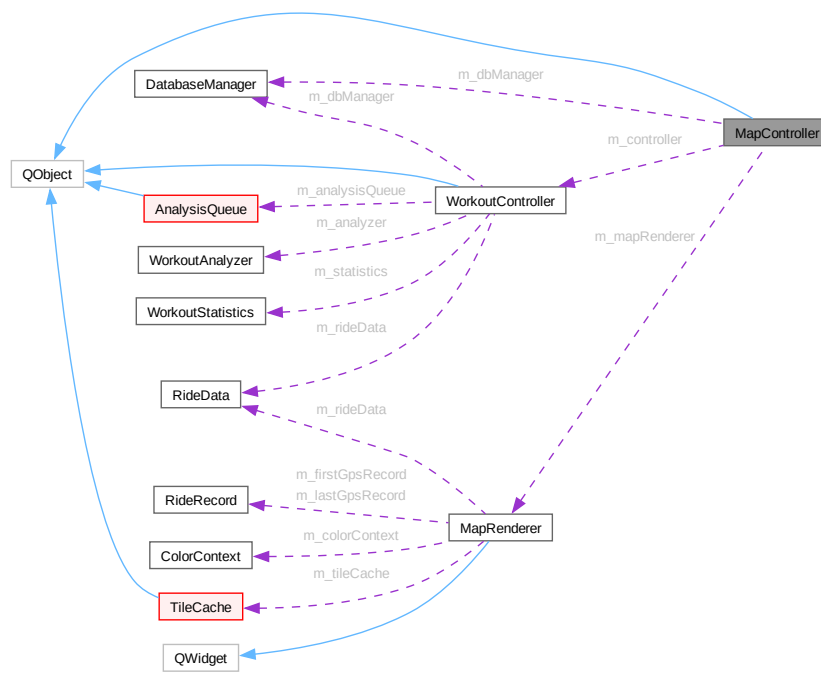
Manages map rendering and interaction.

```
#include <MapController.h>
```

Inheritance diagram for MapController:



Collaboration diagram for MapController:



## Signals

- void **mapUpdated** ()  
*Emitted when map has been successfully updated.*
- void **errorOccurred** (const QString &message)  
*Emitted when an error occurs during map operations.*

## Public Member Functions

- **MapController** (**WorkoutController** \*controller, **DatabaseManager** \*dbManager, QObject \*parent=nullptr)  
*Constructs the map controller.*
- **~MapController** () override  
*Destructor.*

- void [setMapRenderer](#) ([MapRenderer](#) \*mapRenderer)  
*Sets the [MapRenderer](#) widget managed by this controller.*
- [MapRenderer](#) \* [mapRenderer](#) () const  
*Gets the managed map renderer widget.*
- void [updateMap](#) ()  
*Updates map display with current activity route data.*
- void [refreshMap](#) ()  
*Refreshes map rendering without reloading data.*
- void [fitToTrack](#) ()  
*Fits map view to entire track.*
- void [fitToVisibleRange](#) ()  
*Fits map view to visible time range.*
- void [zoomIn](#) ()  
*Zooms in on the map.*
- void [zoomOut](#) ()  
*Zooms out on the map.*
- void [setCurrentTime](#) (double timeSeconds)  
*Sets the current crosshair time position.*
- void [setVisibleTimeRange](#) (double startSeconds, double endSeconds)  
*Sets the visible time range on the map.*
- void [highlightSegment](#) (double startSeconds, double endSeconds)  
*Highlights a segment on the map.*
- void [updateRouteColoring](#) ()  
*Updates route coloring based on current metric.*
- void [setAutoRouteContrast](#) (bool enabled)  
*Enables/disables automatic route contrast adjustment.*

### Private Member Functions

- void [applyColorMetric](#) ()  
*Helper to apply current color metric to map.*
- void [connectMapSignals](#) ()  
*Helper to connect map signals.*

### Private Attributes

- [WorkoutController](#) \* [m\\_controller](#)
- [DatabaseManager](#) \* [m\\_dbManager](#)
- [MapRenderer](#) \* [m\\_mapRenderer](#) = nullptr

## 7.78.1 Detailed Description

Manages map rendering and interaction.

Handles:

- [MapRenderer](#) widget management
- Map styles (light, street, satellite, dark, terrain, topographic, minimal)
- Route coloring based on metrics
- Map refresh and synchronization
- Segment highlighting
- Zoom and pan operations

Definition at line 23 of file [MapController.h](#).



## 7.78.2 Constructor & Destructor Documentation

### 7.78.2.1 MapController()

```
MapController::MapController (
    WorkoutController * controller,
    DatabaseManager * dbManager,
    QObject * parent = nullptr) [explicit]
```

Constructs the map controller.

#### Parameters

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <i>controller</i> | Pointer to <a href="#">WorkoutController</a> for data access. |
| <i>dbManager</i>  | Pointer to <a href="#">DatabaseManager</a> for queries.       |
| <i>parent</i>     | Parent object.                                                |

Definition at line 15 of file [MapController.cpp](#).

## 7.78.3 Member Function Documentation

### 7.78.3.1 applyColorMetric()

```
void MapController::applyColorMetric () [private]
```

Helper to apply current color metric to map.

Definition at line 162 of file [MapController.cpp](#).

Referenced by [updateRouteColoring\(\)](#).

Here is the caller graph for this function:



### 7.78.3.2 connectMapSignals()

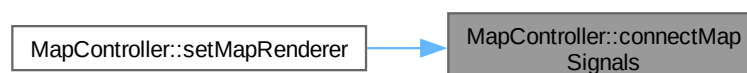
```
void MapController::connectMapSignals () [private]
```

Helper to connect map signals.

Definition at line 170 of file [MapController.cpp](#).

Referenced by [setMapRenderer\(\)](#).

Here is the caller graph for this function:



**7.78.3.3 fitToTrack()**

```
void MapController::fitToTrack ()
```

Fits map view to entire track.

Definition at line 70 of file [MapController.cpp](#).

**7.78.3.4 fitToVisibleRange()**

```
void MapController::fitToVisibleRange ()
```

Fits map view to visible time range.

Definition at line 80 of file [MapController.cpp](#).

**7.78.3.5 highlightSegment()**

```
void MapController::highlightSegment (
    double startSeconds,
    double endSeconds)
```

Highlights a segment on the map.

**Parameters**

|                     |                                |
|---------------------|--------------------------------|
| <i>startSeconds</i> | Segment start time in seconds. |
| <i>endSeconds</i>   | Segment end time in seconds.   |

Definition at line 135 of file [MapController.cpp](#).

**7.78.3.6 mapRenderer()**

```
MapRenderer * MapController::mapRenderer () const
```

Gets the managed map renderer widget.

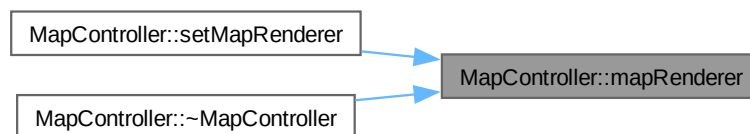
**Returns**

Pointer to [MapRenderer](#), or nullptr if not set.

Definition at line 44 of file [MapController.cpp](#).

Referenced by [setMapRenderer\(\)](#), and [~MapController\(\)](#).

Here is the caller graph for this function:

**7.78.3.7 refreshMap()**

```
void MapController::refreshMap ()
```

Refreshes map rendering without reloading data.

Definition at line 61 of file [MapController.cpp](#).

References [mapUpdated\(\)](#).

Here is the call graph for this function:



### 7.78.3.8 setAutoRouteContrast()

```
void MapController::setAutoRouteContrast (
    bool enabled)
```

Enables/disables automatic route contrast adjustment.

#### Parameters

|                |                                  |
|----------------|----------------------------------|
| <i>enabled</i> | Whether to enable auto contrast. |
|----------------|----------------------------------|

Definition at line 152 of file [MapController.cpp](#).

### 7.78.3.9 setCurrentTime()

```
void MapController::setCurrentTime (
    double timeSeconds)
```

Sets the current crosshair time position.

#### Parameters

|                    |                                |
|--------------------|--------------------------------|
| <i>timeSeconds</i> | Time in seconds (-1 to clear). |
|--------------------|--------------------------------|

Definition at line 111 of file [MapController.cpp](#).

### 7.78.3.10 setMapRenderer()

```
void MapController::setMapRenderer (
    MapRenderer * mapRenderer)
```

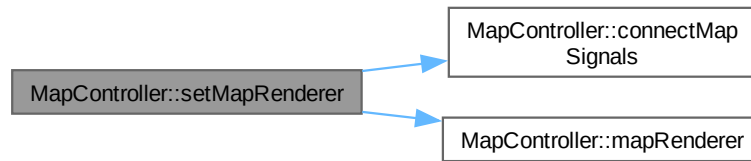
Sets the [MapRenderer](#) widget managed by this controller.

#### Parameters

|                    |                                                |
|--------------------|------------------------------------------------|
| <i>mapRenderer</i> | Pointer to <a href="#">MapRenderer</a> widget. |
|--------------------|------------------------------------------------|

Definition at line 34 of file [MapController.cpp](#).  
References [connectMapSignals\(\)](#), and [mapRenderer\(\)](#).

Here is the call graph for this function:



#### 7.78.3.11 setVisibleTimeRange()

```
void MapController::setVisibleTimeRange (
    double startSeconds,
    double endSeconds)
```

Sets the visible time range on the map.

##### Parameters

|                     |                        |
|---------------------|------------------------|
| <i>startSeconds</i> | Start time in seconds. |
| <i>endSeconds</i>   | End time in seconds.   |

Definition at line 123 of file [MapController.cpp](#).

#### 7.78.3.12 updateMap()

```
void MapController::updateMap ()
```

Updates map display with current activity route data.  
Definition at line 52 of file [MapController.cpp](#).  
References [mapUpdated\(\)](#).

Here is the call graph for this function:



#### 7.78.3.13 updateRouteColoring()

```
void MapController::updateRouteColoring ()
```

Updates route coloring based on current metric.  
Definition at line 143 of file [MapController.cpp](#).  
References [applyColorMetric\(\)](#).

Here is the call graph for this function:



#### 7.78.3.14 zoomIn()

```
void MapController::zoomIn ()
```

Zooms in on the map.

Definition at line 90 of file [MapController.cpp](#).

#### 7.78.3.15 zoomOut()

```
void MapController::zoomOut ()
```

Zooms out on the map.

Definition at line 100 of file [MapController.cpp](#).

### 7.78.4 Member Data Documentation

#### 7.78.4.1 m\_controller

```
WorkoutController* MapController::m_controller [private]
```

Definition at line 131 of file [MapController.h](#).

#### 7.78.4.2 m\_dbManager

```
DatabaseManager* MapController::m_dbManager [private]
```

Definition at line 132 of file [MapController.h](#).

#### 7.78.4.3 m\_mapRenderer

```
MapRenderer* MapController::m_mapRenderer = nullptr [private]
```

Definition at line 133 of file [MapController.h](#).

The documentation for this class was generated from the following files:

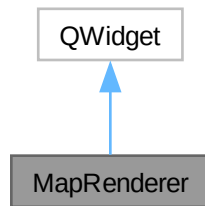
- [src/gui/controllers/MapController.h](#)
- [src/gui/controllers/MapController.cpp](#)

## 7.79 MapRenderer Class Reference

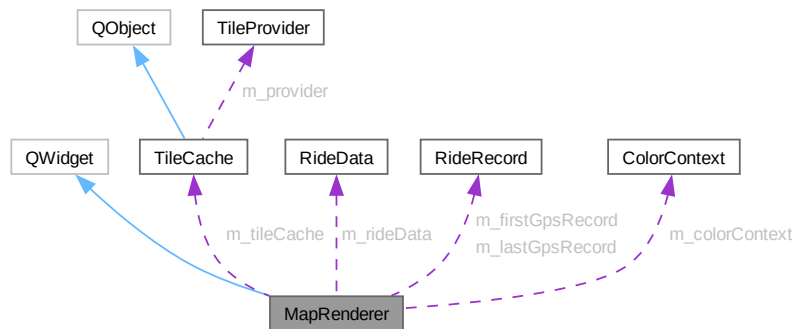
Interactive map widget for displaying and editing activity routes.

```
#include <MapRenderer.h>
```

Inheritance diagram for MapRenderer:



Collaboration diagram for MapRenderer:



### Public Slots

- void [fitToTrack](#) ()
- void [fitToVisibleRange](#) ()
- void [zoomIn](#) ()
- void [zoomOut](#) ()
- void [setHighlightedElapsedSeconds](#) (double elapsedSeconds)

### Signals

- void **segmentSelectionChanged** (double startSeconds, double endSeconds)
- void **segmentSelectionFinished** (double startSeconds, double endSeconds)

### Public Member Functions

- [MapRenderer](#) (QWidget \*parent=nullptr)
- void [setRideData](#) (const [RideData](#) &rideData, ColorMetric colorMetric=ColorMetric::None, const [ColorContext](#) &colorContext={})
- void [setVisibleTimeRange](#) (double startSeconds, double endSeconds)
- void [setCurrentTime](#) (double seconds)
- void [setRouteColorMode](#) (RouteColorMode mode, const [ColorContext](#) &colorContext={})
- void [setMapStyle](#) (MapStyle style)

- MapStyle [mapStyle](#) () const
- void [setAutoRouteContrast](#) (bool enabled)
- bool [autoRouteContrast](#) () const
- void [setTileCacheSize](#) (int maxTiles)

### Protected Member Functions

- void [paintEvent](#) (QPaintEvent \*event) override
- void [wheelEvent](#) (QWheelEvent \*event) override
- void [mouseDoubleClickEvent](#) (QMouseEvent \*event) override
- void [mousePressEvent](#) (QMouseEvent \*event) override
- void [mouseMoveEvent](#) (QMouseEvent \*event) override
- void [mouseReleaseEvent](#) (QMouseEvent \*event) override

### Private Types

- enum class **DragHandle** { **None** , **Start** , **End** }

### Private Member Functions

- void [fitToBounds](#) (double minLat, double maxLat, double minLon, double maxLon, bool updateMinZoom)
- QPointF [tileToScreen](#) (const QPointF &tilePt) const
- QColor [adjustRouteColorForStyle](#) (const QColor &color) const
- const [RideRecord](#) \* [nearestGpsRecord](#) (const QPointF &screenPos) const
- void [rebuildGpsCache](#) ()
- void [rebuildSegmentColors](#) ()
- const [RideRecord](#) \* [gpsRecordAtTime](#) (double elapsedSeconds) const

### Static Private Member Functions

- static QPointF [latLonToTile](#) (double lat, double lon, int zoom)

### Private Attributes

- [TileCache](#) [m\\_tileCache](#)
- [RideData](#) [m\\_rideData](#)
- std::vector< const [RideRecord](#) \* > [m\\_gpsRecords](#)
- std::vector< QColor > [m\\_segmentColors](#)
- const [RideRecord](#) \* [m\\_firstGpsRecord](#) = nullptr
- const [RideRecord](#) \* [m\\_lastGpsRecord](#) = nullptr
- bool [m\\_hasGps](#) = false
- RouteColorMode [m\\_routeColorMode](#) = RouteColorMode::None
- [ColorContext](#) [m\\_colorContext](#)
- double [m\\_minLat](#) = 90.0
- double [m\\_maxLat](#) = -90.0
- double [m\\_minLon](#) = 180.0
- double [m\\_maxLon](#) = -180.0
- double [m\\_centerLat](#) = 51.5
- double [m\\_centerLon](#) = 0.0
- int [m\\_zoom](#) = 13
- int [m\\_minZoom](#) = 1
- QPointF [m\\_panStart](#)
- bool [m\\_panning](#) = false
- DragHandle [m\\_dragHandle](#) = DragHandle::None
- QRectF [m\\_startMarkerRect](#)
- QRectF [m\\_endMarkerRect](#)

- bool `m_draggingSelection` = false
- bool `m_userMovedMap` = false
- bool `m_autoFitVisibleRange` = true
- bool `m_autoRouteContrast` = true
- double `m_highlightElapsedSeconds` = -1.0
- double `m_visibleStartSeconds` = -1.0
- double `m_visibleEndSeconds` = -1.0
- double `m_currentTimeSeconds` = -1.0
- QTimer `m_tileRepaintTimer`

### 7.79.1 Detailed Description

Interactive map widget for displaying and editing activity routes.

Renders map tiles with activity GPS trace, supports color-coding by metrics, time-range selection, and interacts with the main UI.

Definition at line 35 of file [MapRenderer.h](#).

### 7.79.2 Member Enumeration Documentation

#### 7.79.2.1 DragHandle

```
enum class MapRenderer::DragHandle [strong], [private]
```

Definition at line 85 of file [MapRenderer.h](#).

### 7.79.3 Constructor & Destructor Documentation

#### 7.79.3.1 MapRenderer()

```
MapRenderer::MapRenderer (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 61 of file [MapRenderer.cpp](#).

### 7.79.4 Member Function Documentation

#### 7.79.4.1 adjustRouteColorForStyle()

```
QColor MapRenderer::adjustRouteColorForStyle (
    const QColor & color) const [private]
```

Definition at line 265 of file [MapRenderer.cpp](#).

#### 7.79.4.2 autoRouteContrast()

```
bool MapRenderer::autoRouteContrast() const [inline]
```

Definition at line 50 of file [MapRenderer.h](#).

#### 7.79.4.3 fitToBounds()

```
void MapRenderer::fitToBounds (
    double minLat,
    double maxLat,
    double minLon,
    double maxLon,
    bool updateMinZoom) [private]
```

Definition at line 386 of file [MapRenderer.cpp](#).

#### 7.79.4.4 fitToTrack

```
void MapRenderer::fitToTrack () [slot]
```

Definition at line 316 of file [MapRenderer.cpp](#).



#### 7.79.4.5 fitToVisibleRange

```
void MapRenderer::fitToVisibleRange () [slot]
```

Definition at line 340 of file [MapRenderer.cpp](#).

#### 7.79.4.6 gpsRecordAtTime()

```
const RideRecord * MapRenderer::gpsRecordAtTime (  
    double elapsedSeconds) const [private]
```

Definition at line 158 of file [MapRenderer.cpp](#).

#### 7.79.4.7 latLonToTile()

```
QPointF MapRenderer::latLonToTile (  
    double lat,  
    double lon,  
    int zoom) [static], [private]
```

Definition at line 42 of file [MapRenderer.cpp](#).

#### 7.79.4.8 mapStyle()

```
MapStyle MapRenderer::mapStyle () const [inline]
```

Definition at line 48 of file [MapRenderer.h](#).

#### 7.79.4.9 mouseDoubleClickEvent()

```
void MapRenderer::mouseDoubleClickEvent (  
    QMouseEvent * event) [override], [protected]
```

Definition at line 672 of file [MapRenderer.cpp](#).

#### 7.79.4.10 mouseMoveEvent()

```
void MapRenderer::mouseMoveEvent (  
    QMouseEvent * event) [override], [protected]
```

Definition at line 730 of file [MapRenderer.cpp](#).

#### 7.79.4.11 mousePressEvent()

```
void MapRenderer::mousePressEvent (  
    QMouseEvent * event) [override], [protected]
```

Definition at line 698 of file [MapRenderer.cpp](#).

#### 7.79.4.12 mouseReleaseEvent()

```
void MapRenderer::mouseReleaseEvent (  
    QMouseEvent * event) [override], [protected]
```

Definition at line 804 of file [MapRenderer.cpp](#).

#### 7.79.4.13 nearestGpsRecord()

```
const RideRecord * MapRenderer::nearestGpsRecord (  
    const QPointF & screenPos) const [private]
```

Definition at line 291 of file [MapRenderer.cpp](#).

#### 7.79.4.14 paintEvent()

```
void MapRenderer::paintEvent (  
    QPaintEvent * event) [override], [protected]
```

Definition at line 434 of file [MapRenderer.cpp](#).

#### 7.79.4.15 rebuildGpsCache()

```
void MapRenderer::rebuildGpsCache () [private]
```

Definition at line 149 of file [MapRenderer.cpp](#).

#### 7.79.4.16 rebuildSegmentColors()

```
void MapRenderer::rebuildSegmentColors () [private]
```

Definition at line 81 of file [MapRenderer.cpp](#).

#### 7.79.4.17 setAutoRouteContrast()

```
void MapRenderer::setAutoRouteContrast (
    bool enabled)
```

Definition at line 252 of file [MapRenderer.cpp](#).

#### 7.79.4.18 setCurrentTime()

```
void MapRenderer::setCurrentTime (
    double seconds)
```

Definition at line 228 of file [MapRenderer.cpp](#).

#### 7.79.4.19 setHighlightedElapsedSeconds

```
void MapRenderer::setHighlightedElapsedSeconds (
    double elapsedSeconds) [slot]
```

Definition at line 427 of file [MapRenderer.cpp](#).

#### 7.79.4.20 setMapStyle()

```
void MapRenderer::setMapStyle (
    MapStyle style)
```

Definition at line 242 of file [MapRenderer.cpp](#).

#### 7.79.4.21 setRideData()

```
void MapRenderer::setRideData (
    const RideData & rideData,
    ColorMetric colorMetric = ColorMetric::None,
    const ColorContext & colorContext = {})
```

Definition at line 178 of file [MapRenderer.cpp](#).

#### 7.79.4.22 setRouteColorMode()

```
void MapRenderer::setRouteColorMode (
    RouteColorMode mode,
    const ColorContext & colorContext = {})
```

Definition at line 234 of file [MapRenderer.cpp](#).

#### 7.79.4.23 setTileCacheSize()

```
void MapRenderer::setTileCacheSize (
    int maxTiles)
```

Definition at line 260 of file [MapRenderer.cpp](#).

#### 7.79.4.24 setVisibleTimeRange()

```
void MapRenderer::setVisibleTimeRange (
    double startSeconds,
    double endSeconds)
```

Definition at line 217 of file [MapRenderer.cpp](#).

#### 7.79.4.25 tileToScreen()

```
QPointF MapRenderer::tileToScreen (
    const QPointF & tilePt) const [private]
```

Definition at line 52 of file [MapRenderer.cpp](#).

#### 7.79.4.26 wheelEvent()

```
void MapRenderer::wheelEvent (
    QWheelEvent * event) [override], [protected]
```

Definition at line 631 of file [MapRenderer.cpp](#).

#### 7.79.4.27 zoomIn

```
void MapRenderer::zoomIn () [slot]
```

Definition at line 411 of file [MapRenderer.cpp](#).

#### 7.79.4.28 zoomOut

```
void MapRenderer::zoomOut () [slot]
```

Definition at line 419 of file [MapRenderer.cpp](#).

### 7.79.5 Member Data Documentation

#### 7.79.5.1 m\_autoFitVisibleRange

```
bool MapRenderer::m_autoFitVisibleRange = true [private]
```

Definition at line 131 of file [MapRenderer.h](#).

#### 7.79.5.2 m\_autoRouteContrast

```
bool MapRenderer::m_autoRouteContrast = true [private]
```

Definition at line 132 of file [MapRenderer.h](#).

#### 7.79.5.3 m\_centerLat

```
double MapRenderer::m_centerLat = 51.5 [private]
```

Definition at line 119 of file [MapRenderer.h](#).

#### 7.79.5.4 m\_centerLon

```
double MapRenderer::m_centerLon = 0.0 [private]
```

Definition at line 120 of file [MapRenderer.h](#).

#### 7.79.5.5 m\_colorContext

```
ColorContext MapRenderer::m_colorContext [private]
```

Definition at line 114 of file [MapRenderer.h](#).

#### 7.79.5.6 m\_currentTimeSeconds

```
double MapRenderer::m_currentTimeSeconds = -1.0 [private]
```

Definition at line 136 of file [MapRenderer.h](#).

#### 7.79.5.7 m\_draggingSelection

```
bool MapRenderer::m_draggingSelection = false [private]
```

Definition at line 129 of file [MapRenderer.h](#).

#### 7.79.5.8 m\_dragHandle

`DragHandle MapRenderer::m_dragHandle = DragHandle::None [private]`  
Definition at line 126 of file [MapRenderer.h](#).

#### 7.79.5.9 m\_endMarkerRect

`QRectF MapRenderer::m_endMarkerRect [private]`  
Definition at line 128 of file [MapRenderer.h](#).

#### 7.79.5.10 m\_firstGpsRecord

`const RideRecord* MapRenderer::m_firstGpsRecord = nullptr [private]`  
Definition at line 110 of file [MapRenderer.h](#).

#### 7.79.5.11 m\_gpsRecords

`std::vector<const RideRecord*> MapRenderer::m_gpsRecords [private]`  
Definition at line 106 of file [MapRenderer.h](#).

#### 7.79.5.12 m\_hasGps

`bool MapRenderer::m_hasGps = false [private]`  
Definition at line 112 of file [MapRenderer.h](#).

#### 7.79.5.13 m\_highlightElapsedSeconds

`double MapRenderer::m_highlightElapsedSeconds = -1.0 [private]`  
Definition at line 133 of file [MapRenderer.h](#).

#### 7.79.5.14 m\_lastGpsRecord

`const RideRecord* MapRenderer::m_lastGpsRecord = nullptr [private]`  
Definition at line 111 of file [MapRenderer.h](#).

#### 7.79.5.15 m\_maxLat

`double MapRenderer::m_maxLat = -90.0 [private]`  
Definition at line 116 of file [MapRenderer.h](#).

#### 7.79.5.16 m\_maxLon

`double MapRenderer::m_maxLon = -180.0 [private]`  
Definition at line 117 of file [MapRenderer.h](#).

#### 7.79.5.17 m\_minLat

`double MapRenderer::m_minLat = 90.0 [private]`  
Definition at line 116 of file [MapRenderer.h](#).

#### 7.79.5.18 m\_minLon

`double MapRenderer::m_minLon = 180.0 [private]`  
Definition at line 117 of file [MapRenderer.h](#).

#### 7.79.5.19 m\_minZoom

`int MapRenderer::m_minZoom = 1 [private]`  
Definition at line 122 of file [MapRenderer.h](#).

#### 7.79.5.20 m\_panning

`bool MapRenderer::m_panning = false [private]`  
Definition at line 125 of file [MapRenderer.h](#).

#### 7.79.5.21 m\_panStart

`QPointF MapRenderer::m_panStart [private]`  
Definition at line 124 of file [MapRenderer.h](#).

#### 7.79.5.22 m\_rideData

`RideData MapRenderer::m_rideData [private]`  
Definition at line 102 of file [MapRenderer.h](#).

#### 7.79.5.23 m\_routeColorMode

`RouteColorMode MapRenderer::m_routeColorMode = RouteColorMode::None [private]`  
Definition at line 113 of file [MapRenderer.h](#).

#### 7.79.5.24 m\_segmentColors

`std::vector<QColor> MapRenderer::m_segmentColors [private]`  
Definition at line 109 of file [MapRenderer.h](#).

#### 7.79.5.25 m\_startMarkerRect

`QRectF MapRenderer::m_startMarkerRect [private]`  
Definition at line 127 of file [MapRenderer.h](#).

#### 7.79.5.26 m\_tileCache

`TileCache MapRenderer::m_tileCache [private]`  
Definition at line 101 of file [MapRenderer.h](#).

#### 7.79.5.27 m\_tileRepaintTimer

`QTimer MapRenderer::m_tileRepaintTimer [private]`  
Definition at line 137 of file [MapRenderer.h](#).

#### 7.79.5.28 m\_userMovedMap

`bool MapRenderer::m_userMovedMap = false [private]`  
Definition at line 130 of file [MapRenderer.h](#).

#### 7.79.5.29 m\_visibleEndSeconds

`double MapRenderer::m_visibleEndSeconds = -1.0 [private]`  
Definition at line 135 of file [MapRenderer.h](#).

#### 7.79.5.30 m\_visibleStartSeconds

`double MapRenderer::m_visibleStartSeconds = -1.0 [private]`  
Definition at line 134 of file [MapRenderer.h](#).

#### 7.79.5.31 m\_zoom

`int MapRenderer::m_zoom = 13 [private]`  
Definition at line 121 of file [MapRenderer.h](#).

The documentation for this class was generated from the following files:

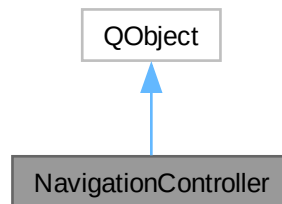
- `src/maps/MapRenderer.h`
- `src/maps/MapRenderer.cpp`

## 7.80 NavigationController Class Reference

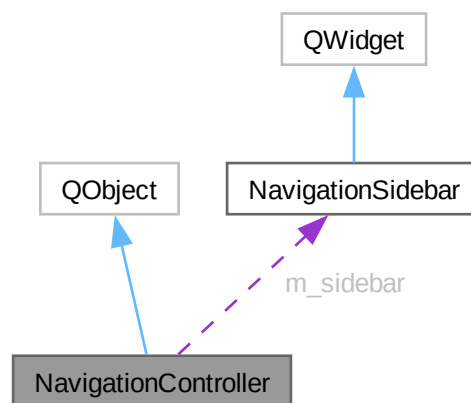
Manages top-level page navigation in the main window.

```
#include <NavigationController.h>
```

Inheritance diagram for NavigationController:



Collaboration diagram for NavigationController:



### Public Types

- using `Page` = `NavigationSidebar::Page`  
*Convenience alias so callers do not need to include `NavigationSidebar.h`.*

### Signals

- void **pageChanged** (`NavigationSidebar::Page` page)  
*Emitted whenever the active page changes.*
- void **navigationStateRestored** ()  
*Emitted once after `restoreNavigationState()` completes.*

**Public Member Functions**

- [NavigationController](#) (QObject \*parent=nullptr)  
*Constructs the navigation controller.*
- [~NavigationController](#) () override  
*Destructor.*
- void [setPageStack](#) (QStackedWidget \*stack)  
*Sets the stacked widget that holds one widget per [Page](#).*
- void [setNavigationSidebar](#) ([NavigationSidebar](#) \*sidebar)  
*Sets and connects the navigation sidebar.*
- void [navigateTo](#) ([Page](#) page)  
*Navigates to the given page.*
- [Page](#) [currentPage](#) () const  
*Returns the currently active page.*
- void [saveNavigationState](#) ()  
*Saves the active page to QSettings.*
- void [restoreNavigationState](#) ()  
*Restores the active page from QSettings.*

**Private Attributes**

- QStackedWidget \* [m\\_pageStack](#) = nullptr
- [NavigationSidebar](#) \* [m\\_sidebar](#) = nullptr
- [Page](#) [m\\_currentPage](#) = [Page::Activities](#)

**7.80.1 Detailed Description**

Manages top-level page navigation in the main window.  
Owns the relationship between:

- [NavigationSidebar](#) (user input)
- QStackedWidget (page display)

The legacy QTabWidget / analysis-sub-tab hierarchy has been removed. Each entry in [NavigationSidebar::Page](#) corresponds to one index in the page stack. Call [setPageStack\(\)](#) once after both the sidebar and the stack are constructed, then call [connectSidebar\(\)](#) to wire them.  
Definition at line 24 of file [NavigationController.h](#).

**7.80.2 Member Typedef Documentation****7.80.2.1 Page**

using [NavigationController::Page](#) = [NavigationSidebar::Page](#)  
Convenience alias so callers do not need to include [NavigationSidebar.h](#).  
Definition at line 30 of file [NavigationController.h](#).

**7.80.3 Constructor & Destructor Documentation****7.80.3.1 NavigationController()**

```
NavigationController::NavigationController (
    QObject * parent = nullptr) [explicit]
```

Constructs the navigation controller.

**Parameters**

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent object. |
|---------------|----------------|

Definition at line 12 of file [NavigationController.cpp](#).

**7.80.4 Member Function Documentation****7.80.4.1 currentPage()**

[NavigationController::Page](#) [NavigationController::currentPage](#) () const

Returns the currently active page.

Definition at line 74 of file [NavigationController.cpp](#).

**7.80.4.2 navigateTo()**

```
void NavigationController::navigateTo (
    Page page)
```

Navigates to the given page.

**Parameters**

|             |              |
|-------------|--------------|
| <i>page</i> | Target page. |
|-------------|--------------|

Updates the page stack and the sidebar selection, then emits `pageChanged`.

**Parameters**

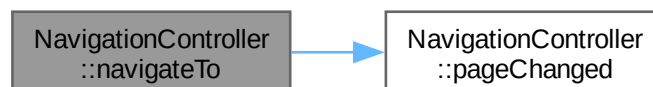
|             |              |
|-------------|--------------|
| <i>page</i> | Target page. |
|-------------|--------------|

Definition at line 54 of file [NavigationController.cpp](#).

References [pageChanged\(\)](#).

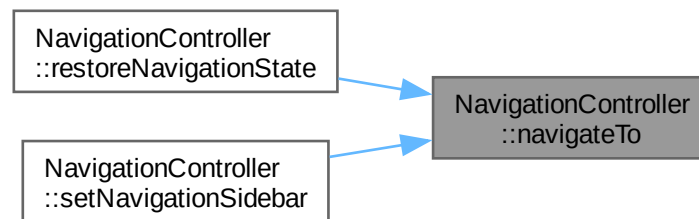
Referenced by [restoreNavigationState\(\)](#), and [setNavigationSidebar\(\)](#).

Here is the call graph for this function:





Here is the caller graph for this function:



#### 7.80.4.3 restoreNavigationState()

```
void NavigationController::restoreNavigationState ()
```

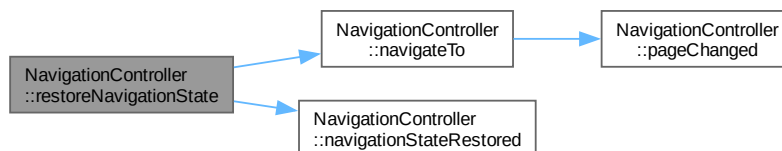
Restores the active page from QSettings.

Clamps the stored value to the valid [Page](#) range so that stale settings written by an older version of the application cannot cause out-of-bounds access. When new [Page](#) values are added, update the upper bound here.

Definition at line 95 of file [NavigationController.cpp](#).

References [navigateTo\(\)](#), and [navigationStateRestored\(\)](#).

Here is the call graph for this function:



#### 7.80.4.4 saveNavigationState()

```
void NavigationController::saveNavigationState ()
```

Saves the active page to QSettings.

Saves the active page index to QSettings.

Definition at line 82 of file [NavigationController.cpp](#).

#### 7.80.4.5 setNavigationSidebar()

```
void NavigationController::setNavigationSidebar (
    NavigationSidebar * sidebar)
```

Sets and connects the navigation sidebar.

Sets and wires the navigation sidebar.

After this call, selecting a row in the sidebar automatically switches the page stack to the corresponding index.

#### Parameters

|                |                                                    |
|----------------|----------------------------------------------------|
| <i>sidebar</i> | Pointer to the <a href="#">NavigationSidebar</a> . |
|----------------|----------------------------------------------------|

Selecting a row in the sidebar immediately navigates to the corresponding page in the stack.

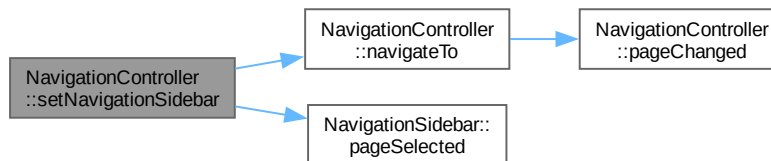
### Parameters

|                |                                                |
|----------------|------------------------------------------------|
| <i>sidebar</i> | Pointer to <a href="#">NavigationSidebar</a> . |
|----------------|------------------------------------------------|

Definition at line 38 of file [NavigationController.cpp](#).

References [navigateTo\(\)](#), and [NavigationSidebar::pageSelected\(\)](#).

Here is the call graph for this function:



#### 7.80.4.6 setPageStack()

```
void NavController::setPageStack (
    QStackedWidget * stack)
```

Sets the stacked widget that holds one widget per [Page](#).

Sets the page stacked widget.

### Parameters

|              |                                                               |
|--------------|---------------------------------------------------------------|
| <i>stack</i> | Pointer to the QStackedWidget used as the page container.     |
| <i>stack</i> | Pointer to the QStackedWidget that hosts one widget per page. |

Definition at line 26 of file [NavigationController.cpp](#).

## 7.80.5 Member Data Documentation

### 7.80.5.1 m\_currentPage

```
Page NavController::m_currentPage = Page::Activities [private]
```

Definition at line 76 of file [NavigationController.h](#).

### 7.80.5.2 m\_pageStack

```
QStackedWidget* NavController::m_pageStack = nullptr [private]
```

Definition at line 73 of file [NavigationController.h](#).

### 7.80.5.3 m\_sidebar

```
NavigationSidebar* NavController::m_sidebar = nullptr [private]
```

Definition at line 74 of file [NavigationController.h](#).

The documentation for this class was generated from the following files:

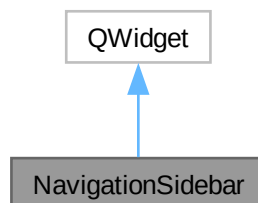
- [src/gui/controllers/NavController.h](#)
- [src/gui/controllers/NavController.cpp](#)

## 7.81 NavigationSidebar Class Reference

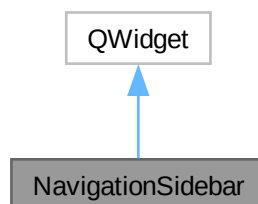
Main navigation sidebar.

```
#include <NavigationSidebar.h>
```

Inheritance diagram for NavigationSidebar:



Collaboration diagram for NavigationSidebar:



### Public Types

- enum class [Page](#) {  
    [Activities](#) = 0 , [Charts](#) , [Power](#) , [Intervals](#) ,  
    [Climbs](#) , [Fitness](#) , [Calendar](#) , [Video](#) }

*Top-level application pages.*

### Signals

- void **pageSelected** ([NavigationSidebar::Page](#) page)  
*Emitted when the user selects a different page.*

### Public Member Functions

- [NavigationSidebar](#) ([QWidget](#) \*parent=nullptr)
- void **setCurrentPage** ([Page](#) page)  
*Selects the given page without emitting pageSelected.*
- [Page](#) **currentPage** () const  
*Returns the currently selected page.*

## Private Attributes

- `QListWidget * m_list = nullptr`

### 7.81.1 Detailed Description

Main navigation sidebar.

Displays the top-level pages available in the application. The [Page](#) enum maps 1-to-1 to list-widget rows, so values must remain contiguous and start at zero.

Definition at line 18 of file [NavigationSidebar.h](#).

### 7.81.2 Member Enumeration Documentation

#### 7.81.2.1 Page

```
enum class NavigationSidebar::Page [strong]
```

Top-level application pages.

Matches the order in which items appear in the sidebar list. Future pages (Reports, Readiness, Settings) should be appended at the end so that existing values are not renumbered.

#### Enumerator

|            |                                                  |
|------------|--------------------------------------------------|
| Activities | <a href="#">Activity</a> browser and management. |
| Charts     | <a href="#">Activity</a> charts and map.         |
| Power      | Power curve, histogram, and zones.               |
| Intervals  | <a href="#">Interval</a> table and editor.       |
| Climbs     | <a href="#">Climb</a> detection and analysis.    |
| Fitness    | Fitness, CTL/ATL/TSB, and FTP history.           |
| Calendar   | <a href="#">Activity</a> and workout calendar.   |
| Video      | Video creation and export.                       |

Definition at line 30 of file [NavigationSidebar.h](#).

### 7.81.3 Constructor & Destructor Documentation

#### 7.81.3.1 NavigationSidebar()

```
NavigationSidebar::NavigationSidebar (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 9 of file [NavigationSidebar.cpp](#).

### 7.81.4 Member Function Documentation

#### 7.81.4.1 currentPage()

```
NavigationSidebar::Page NavigationSidebar::currentPage () const
```

Returns the currently selected page.

Definition at line 71 of file [NavigationSidebar.cpp](#).

References [Activities](#).

#### 7.81.4.2 setCurrentPage()

```
void NavigationSidebar::setCurrentPage (
    Page page)
```

Selects the given page without emitting `pageSelected`.

Definition at line 64 of file [NavigationSidebar.cpp](#).

## 7.81.5 Member Data Documentation

### 7.81.5.1 m\_list

`QListWidget* NavigationSidebar::m_list = nullptr [private]`

Definition at line 55 of file [NavigationSidebar.h](#).

The documentation for this class was generated from the following files:

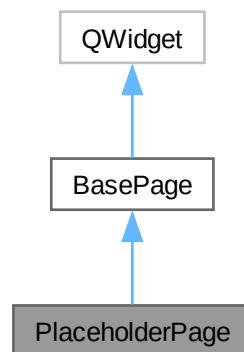
- `src/gui/NavigationSidebar.h`
- `src/gui/NavigationSidebar.cpp`

## 7.82 PlaceholderPage Class Reference

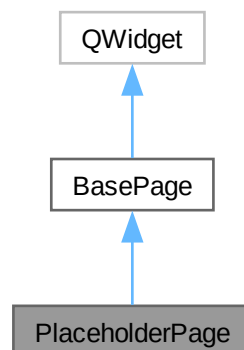
A placeholder page shown for features not yet fully migrated.

```
#include <PlaceholderPage.h>
```

Inheritance diagram for PlaceholderPage:



Collaboration diagram for PlaceholderPage:



## Public Member Functions

- [PlaceholderPage](#) (const QString &text, QWidget \*parent=nullptr)  
*Constructs a placeholder page with the given label text.*

## Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

### 7.82.1 Detailed Description

A placeholder page shown for features not yet fully migrated.

Displays a centred label so the page is never blank. Replace this widget with the real page implementation as each phase is completed.

Definition at line 16 of file [PlaceholderPage.h](#).

### 7.82.2 Constructor & Destructor Documentation

#### 7.82.2.1 PlaceholderPage()

```
PlaceholderPage::PlaceholderPage (
    const QString & text,
    QWidget * parent = nullptr) [inline], [explicit]
```

Constructs a placeholder page with the given label text.

#### Parameters

|               |                                                   |
|---------------|---------------------------------------------------|
| <i>text</i>   | Descriptive text shown in the centre of the page. |
| <i>parent</i> | Parent widget.                                    |

Definition at line 26 of file [PlaceholderPage.h](#).

The documentation for this class was generated from the following file:

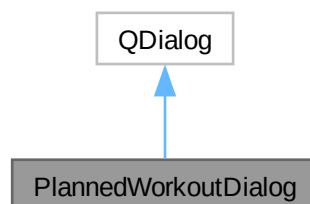
- src/gui/pages/PlaceholderPage.h

## 7.83 PlannedWorkoutDialog Class Reference

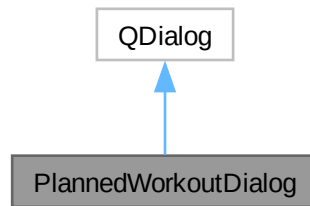
Dialog for creating or editing planned workouts.

```
#include <PlannedWorkoutDialog.h>
```

Inheritance diagram for PlannedWorkoutDialog:



Collaboration diagram for PlannedWorkoutDialog:



### Public Member Functions

- [PlannedWorkoutDialog](#) (QWidget \*parent=nullptr)
- [PlannedWorkoutDialog](#) (const [PlannedWorkoutDraft](#) &draft, QWidget \*parent=nullptr)
- [PlannedWorkoutDraft](#) draft () const

### Private Attributes

- QDateEdit \* [m\\_dateEdit](#) = nullptr
- QLineEdit \* [m\\_titleEdit](#) = nullptr
- QLineEdit \* [m\\_typeEdit](#) = nullptr
- QDoubleSpinBox \* [m\\_durationSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_tssSpin](#) = nullptr
- QTextEdit \* [m\\_notesEdit](#) = nullptr
- int [m\\_id](#) = -1

## 7.83.1 Detailed Description

Dialog for creating or editing planned workouts.  
Definition at line 46 of file [PlannedWorkoutDialog.h](#).

## 7.83.2 Constructor & Destructor Documentation

### 7.83.2.1 PlannedWorkoutDialog() [1/2]

```
PlannedWorkoutDialog::PlannedWorkoutDialog (
    QWidget * parent = nullptr) [explicit]
```

Definition at line 16 of file [PlannedWorkoutDialog.cpp](#).

### 7.83.2.2 PlannedWorkoutDialog() [2/2]

```
PlannedWorkoutDialog::PlannedWorkoutDialog (
    const PlannedWorkoutDraft & draft,
    QWidget * parent = nullptr) [explicit]
```

Definition at line 20 of file [PlannedWorkoutDialog.cpp](#).

## 7.83.3 Member Function Documentation

### 7.83.3.1 draft()

```
PlannedWorkoutDraft PlannedWorkoutDialog::draft () const
```

Definition at line 69 of file [PlannedWorkoutDialog.cpp](#).

## 7.83.4 Member Data Documentation

### 7.83.4.1 m\_dateEdit

`QDateEdit* PlannedWorkoutDialog::m_dateEdit = nullptr [private]`

Definition at line 58 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.2 m\_durationSpin

`QDoubleSpinBox* PlannedWorkoutDialog::m_durationSpin = nullptr [private]`

Definition at line 61 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.3 m\_id

`int PlannedWorkoutDialog::m_id = -1 [private]`

Definition at line 64 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.4 m\_notesEdit

`QTextEdit* PlannedWorkoutDialog::m_notesEdit = nullptr [private]`

Definition at line 63 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.5 m\_titleEdit

`QLineEdit* PlannedWorkoutDialog::m_titleEdit = nullptr [private]`

Definition at line 59 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.6 m\_tssSpin

`QDoubleSpinBox* PlannedWorkoutDialog::m_tssSpin = nullptr [private]`

Definition at line 62 of file [PlannedWorkoutDialog.h](#).

### 7.83.4.7 m\_typeEdit

`QLineEdit* PlannedWorkoutDialog::m_typeEdit = nullptr [private]`

Definition at line 60 of file [PlannedWorkoutDialog.h](#).

The documentation for this class was generated from the following files:

- [src/gui/PlannedWorkoutDialog.h](#)
- [src/gui/PlannedWorkoutDialog.cpp](#)

## 7.84 PlannedWorkoutDraft Struct Reference

Draft data for a planned workout.

```
#include <PlannedWorkoutDialog.h>
```

### Public Attributes

- `int id = -1`
- `QDate date`
- `QString title`
- `QString type`
- `double durationSeconds = 0.0`
- `double targetTss = 0.0`
- `QString notes`

### 7.84.1 Detailed Description

Draft data for a planned workout.

Definition at line 14 of file [PlannedWorkoutDialog.h](#).



## 7.84.2 Member Data Documentation

### 7.84.2.1 date

`QDate PlannedWorkoutDraft::date`

Definition at line 20 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.2 durationSeconds

`double PlannedWorkoutDraft::durationSeconds = 0.0`

Definition at line 29 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.3 id

`int PlannedWorkoutDraft::id = -1`

Definition at line 17 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.4 notes

`QString PlannedWorkoutDraft::notes`

Definition at line 35 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.5 targetTss

`double PlannedWorkoutDraft::targetTss = 0.0`

Definition at line 32 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.6 title

`QString PlannedWorkoutDraft::title`

Definition at line 23 of file [PlannedWorkoutDialog.h](#).

### 7.84.2.7 type

`QString PlannedWorkoutDraft::type`

Definition at line 26 of file [PlannedWorkoutDialog.h](#).

The documentation for this struct was generated from the following file:

- [src/gui/PlannedWorkoutDialog.h](#)

## 7.85 PowerCurve Class Reference

Computes power duration curve (PDC) for an activity.

`#include <PowerCurve.h>`

### Static Public Member Functions

- static `std::vector< double >` [standardDurations](#) ()  
*Returns standard test durations in seconds.*
- static `double` [bestMeanPower](#) (const [RideData](#) &rideData, double durationSeconds)  
*Calculates best mean power over a duration.*
- static `std::vector< PowerCurvePoint >` [computeStandard](#) (const [RideData](#) &rideData)  
*Computes power duration curve for standard durations.*

### 7.85.1 Detailed Description

Computes power duration curve (PDC) for an activity.

Analyzes best sustained power outputs across a range of durations.

Definition at line 27 of file [PowerCurve.h](#).

## 7.85.2 Member Function Documentation

### 7.85.2.1 bestMeanPower()

```
double PowerCurve::bestMeanPower (
    const RideData & rideData,
    double durationSeconds) [static]
```

Calculates best mean power over a duration.

#### Parameters

|                        |                                |
|------------------------|--------------------------------|
| <i>rideData</i>        | <a href="#">Activity</a> data. |
| <i>durationSeconds</i> | Duration window in seconds.    |

#### Returns

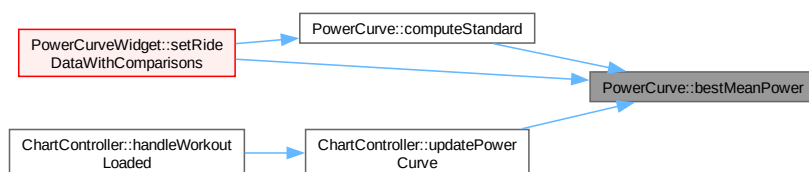
Best mean power in watts.

Definition at line 18 of file [PowerCurve.cpp](#).

References [RideData::records](#).

Referenced by [computeStandard\(\)](#), [PowerCurveWidget::setRideDataWithComparisons\(\)](#), and [ChartController::updatePowerCurve\(\)](#).

Here is the caller graph for this function:



### 7.85.2.2 computeStandard()

```
std::vector< PowerCurvePoint > PowerCurve::computeStandard (
    const RideData & rideData) [static]
```

Computes power duration curve for standard durations.

#### Parameters

|                 |                                |
|-----------------|--------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> data. |
|-----------------|--------------------------------|

**Returns**

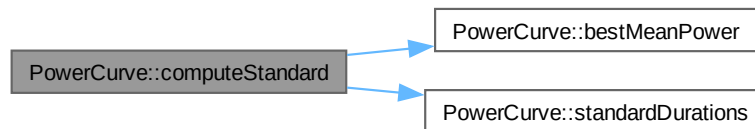
PDC points (omits durations longer than the ride).

Definition at line 47 of file [PowerCurve.cpp](#).

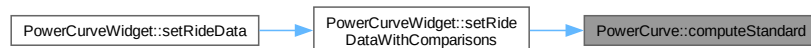
References [bestMeanPower\(\)](#), and [standardDurations\(\)](#).

Referenced by [PowerCurveWidget::setRideDataWithComparisons\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.85.2.3 standardDurations()**

```
std::vector< double > PowerCurve::standardDurations () [static]
```

Returns standard test durations in seconds.

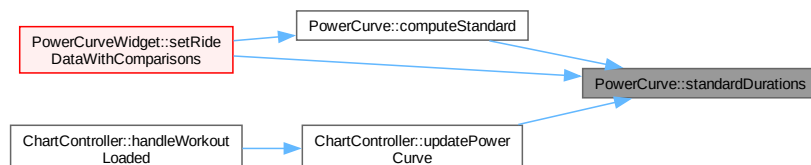
**Returns**

Durations from 5 seconds to 2 hours.

Definition at line 9 of file [PowerCurve.cpp](#).

Referenced by [computeStandard\(\)](#), [PowerCurveWidget::setRideDataWithComparisons\(\)](#), and [ChartController::updatePowerCurve\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/PowerCurve.h`
- `src/analysis/PowerCurve.cpp`

## 7.86 PowerCurvePoint Struct Reference

A single point on a power duration curve.

```
#include <PowerCurve.h>
```

### Public Attributes

- double [durationSeconds](#)  
*Duration in seconds.*
- double [bestPower](#)  
*Best mean power for duration in watts.*

### 7.86.1 Detailed Description

A single point on a power duration curve.

Definition at line 13 of file [PowerCurve.h](#).

### 7.86.2 Member Data Documentation

#### 7.86.2.1 bestPower

```
double PowerCurvePoint::bestPower
```

Best mean power for duration in watts.

Definition at line 19 of file [PowerCurve.h](#).

#### 7.86.2.2 durationSeconds

```
double PowerCurvePoint::durationSeconds
```

Duration in seconds.

Definition at line 16 of file [PowerCurve.h](#).

The documentation for this struct was generated from the following file:

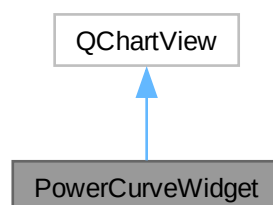
- `src/analysis/PowerCurve.h`

## 7.87 PowerCurveWidget Class Reference

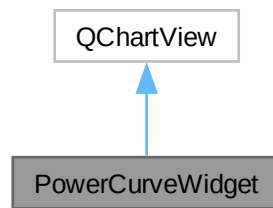
Displays power output over duration curve.

```
#include <PowerCurveWidget.h>
```

Inheritance diagram for PowerCurveWidget:



Collaboration diagram for PowerCurveWidget:



### Public Member Functions

- [PowerCurveWidget](#) (`QWidget *parent=nullptr`)  
*Constructs power curve chart widget.*
- void [setRideData](#) (const [RideData](#) &rideData, double ftp=0.0)  
*Sets ride data to display.*
- void [setRideDataWithComparisons](#) (const [RideData](#) &rideData, const `std::vector< PowerCurvePoint >` &last90, const `std::vector< PowerCurvePoint >` &currentYear, const `std::vector< PowerCurvePoint >` &allTime, double ftp=0.0)  
*Sets ride data with comparison curves.*
- void [clearChart](#) ()  
*Clears the chart.*

### 7.87.1 Detailed Description

Displays power output over duration curve.

Shows best mean power at various durations with optional comparison curves (last 90 days, current year, all-time).

Definition at line 17 of file [PowerCurveWidget.h](#).

### 7.87.2 Constructor & Destructor Documentation

#### 7.87.2.1 PowerCurveWidget()

```
PowerCurveWidget::PowerCurveWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs power curve chart widget.

#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 55 of file [PowerCurveWidget.cpp](#).

### 7.87.3 Member Function Documentation

#### 7.87.3.1 clearChart()

```
void PowerCurveWidget::clearChart ()
```

Clears the chart.

Definition at line 66 of file [PowerCurveWidget.cpp](#).

Referenced by [setRideDataWithComparisons\(\)](#).

Here is the caller graph for this function:



### 7.87.3.2 setRideData()

```
void PowerCurveWidget::setRideData (
    const RideData & rideData,
    double ftp = 0.0)
```

Sets ride data to display.

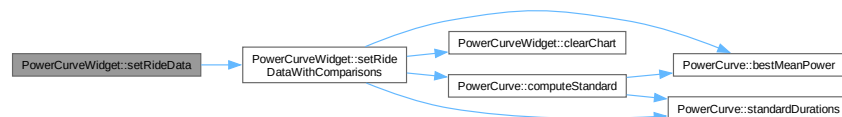
#### Parameters

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>rideData</i> | Activity data.                          |
| <i>ftp</i>      | Functional threshold power (default 0). |

Definition at line 74 of file [PowerCurveWidget.cpp](#).

References [setRideDataWithComparisons\(\)](#).

Here is the call graph for this function:



### 7.87.3.3 setRideDataWithComparisons()

```
void PowerCurveWidget::setRideDataWithComparisons (
    const RideData & rideData,
    const std::vector< PowerCurvePoint > & last90,
    const std::vector< PowerCurvePoint > & currentYear,
    const std::vector< PowerCurvePoint > & allTime,
    double ftp = 0.0)
```

Sets ride data with comparison curves.

#### Parameters

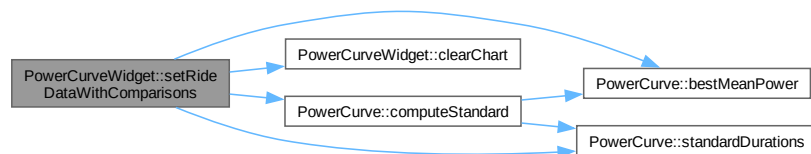
|                    |                                         |
|--------------------|-----------------------------------------|
| <i>rideData</i>    | Current activity data.                  |
| <i>last90</i>      | Power curve for last 90 days.           |
| <i>currentYear</i> | Power curve for current year.           |
| <i>allTime</i>     | Power curve for all time.               |
| <i>ftp</i>         | Functional threshold power (default 0). |

Definition at line 79 of file [PowerCurveWidget.cpp](#).

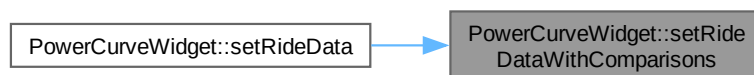
References [PowerCurve::bestMeanPower\(\)](#), [clearChart\(\)](#), [PowerCurve::computeStandard\(\)](#), and [PowerCurve::standardDurations\(\)](#).

Referenced by [setRideData\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

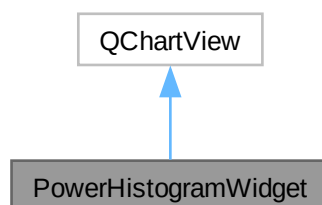
- `src/charts/PowerCurveWidget.h`
- `src/charts/PowerCurveWidget.cpp`

## 7.88 PowerHistogramWidget Class Reference

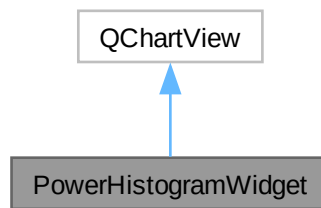
Displays power distribution histogram.

```
#include <PowerHistogramWidget.h>
```

Inheritance diagram for `PowerHistogramWidget`:



Collaboration diagram for PowerHistogramWidget:



## Public Member Functions

- [PowerHistogramWidget](#) (`QWidget *parent=nullptr`)  
*Constructs power histogram chart widget.*
- void [setRideData](#) (const [RideData](#) &rideData)  
*Sets ride data to display.*
- void [clearChart](#) ()  
*Clears the chart.*

### 7.88.1 Detailed Description

Displays power distribution histogram.  
Shows frequency distribution of power output across power range.  
Definition at line 15 of file [PowerHistogramWidget.h](#).

### 7.88.2 Constructor & Destructor Documentation

#### 7.88.2.1 PowerHistogramWidget()

```
PowerHistogramWidget::PowerHistogramWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs power histogram chart widget.

#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 17 of file [PowerHistogramWidget.cpp](#).

### 7.88.3 Member Function Documentation

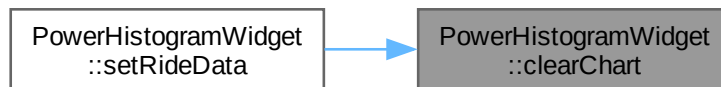
#### 7.88.3.1 clearChart()

```
void PowerHistogramWidget::clearChart ()
```

Clears the chart.  
Definition at line 28 of file [PowerHistogramWidget.cpp](#).  
Referenced by [setRideData\(\)](#).



Here is the caller graph for this function:



### 7.88.3.2 setRideData()

```
void PowerHistogramWidget::setRideData (  
    const RideData & rideData)
```

Sets ride data to display.

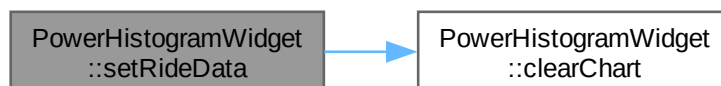
#### Parameters

|                 |                |
|-----------------|----------------|
| <i>rideData</i> | Activity data. |
|-----------------|----------------|

Definition at line 36 of file [PowerHistogramWidget.cpp](#).

References [clearChart\(\)](#), and [RideData::records](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

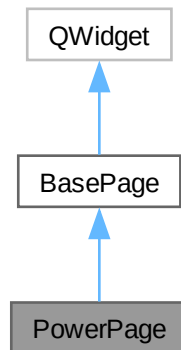
- `src/charts/PowerHistogramWidget.h`
- `src/charts/PowerHistogramWidget.cpp`

## 7.89 PowerPage Class Reference

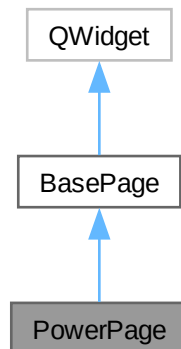
Top-level page for power analysis.

```
#include <PowerPage.h>
```

Inheritance diagram for PowerPage:



Collaboration diagram for PowerPage:



### Public Member Functions

- [PowerPage](#) (QTabWidget \*powerSubTabs, QWidget \*parent=nullptr)  
*Constructs the Power page.*

### Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()  
*Called by [MainWindow](#) when this page becomes the active page.*

#### 7.89.1 Detailed Description

Top-level page for power analysis.

Contains three sub-views accessed via an internal QTabWidget:

- Power Curve (PDC)
- Histogram
- Zones

Future additions: Critical Power, W', Power Profile, Power Trends.  
Definition at line 19 of file [PowerPage.h](#).

## 7.89.2 Constructor & Destructor Documentation

### 7.89.2.1 PowerPage()

```
PowerPage::PowerPage (
    QTabWidget * powerSubTabs,
    QWidget * parent = nullptr) [explicit]
```

Constructs the Power page.

#### Parameters

|                     |                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------|
| <i>powerSubTabs</i> | Pre-built QTabWidget containing Zones, Histogram, and Power Curve sub-tabs (ownership transfers here). |
| <i>parent</i>       | Parent widget.                                                                                         |

Accepts the pre-built power sub-tab widget (Zones / Histogram / Power Curve) and embeds it in this page's layout.

#### Parameters

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <i>powerSubTabs</i> | QTabWidget containing the power analysis sub-views. |
| <i>parent</i>       | Parent widget.                                      |

Definition at line 17 of file [PowerPage.cpp](#).

The documentation for this class was generated from the following files:

- [src/gui/pages/PowerPage.h](#)
- [src/gui/pages/PowerPage.cpp](#)

## 7.90 PowerZone Struct Reference

Represents a single power zone with time spent.

```
#include <FTPZones.h>
```

#### Public Attributes

- int [number](#)  
*Zone number (1-7).*
- std::string [name](#)  
*Zone name (Z1, Z2, etc.).*
- double [lowerPct](#)  
*Lower bound as fraction of FTP (inclusive).*
- double [upperPct](#)  
*Upper bound as fraction of FTP (exclusive), -1.0 for unbounded.*
- double [seconds](#) = 0.0  
*Time spent in zone in seconds.*

### 7.90.1 Detailed Description

Represents a single power zone with time spent.  
Definition at line 14 of file [FTPZones.h](#).

### 7.90.2 Member Data Documentation

#### 7.90.2.1 lowerPct

```
double PowerZone::lowerPct
```

Lower bound as fraction of FTP (inclusive).  
Definition at line 23 of file [FTPZones.h](#).

#### 7.90.2.2 name

```
std::string PowerZone::name
```

[Zone](#) name (Z1, Z2, etc.).  
Definition at line 20 of file [FTPZones.h](#).

#### 7.90.2.3 number

```
int PowerZone::number
```

[Zone](#) number (1-7).  
Definition at line 17 of file [FTPZones.h](#).

#### 7.90.2.4 seconds

```
double PowerZone::seconds = 0.0
```

Time spent in zone in seconds.  
Definition at line 29 of file [FTPZones.h](#).

#### 7.90.2.5 upperPct

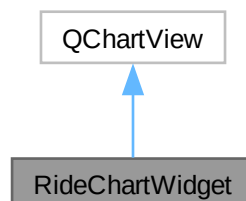
```
double PowerZone::upperPct
```

Upper bound as fraction of FTP (exclusive), -1.0 for unbounded.  
Definition at line 26 of file [FTPZones.h](#).  
The documentation for this struct was generated from the following file:

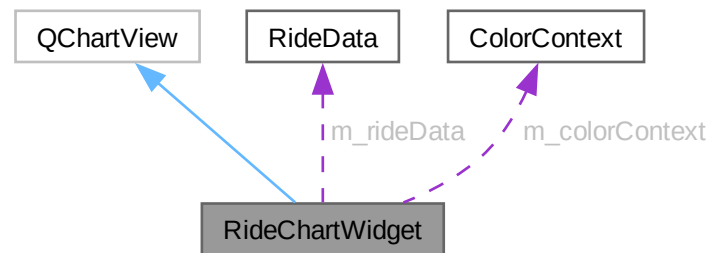
- [src/analysis/FTPZones.h](#)

## 7.91 RideChartWidget Class Reference

Interactive chart displaying ride data by selected metric.  
`#include <RideChartWidget.h>`  
Inheritance diagram for RideChartWidget:



Collaboration diagram for RideChartWidget:



### Public Slots

- void [setXRange](#) (double min, double max)  
*@slot Sets visible x-axis range.*
- void [setCrosshair](#) (double xMinutes)  
*@slot Sets crosshair position.*
- void [resetZoom](#) ()  
*@slot Resets zoom to full ride extent.*
- void [fitToData](#) ()  
*@slot Alias for [resetZoom\(\)](#).*

### Signals

- void [xRangeChanged](#) (double min, double max)  
*@signal Emitted when visible x range changes.*
- void [visibleRangeChanged](#) (double startMinutes, double endMinutes)  
*@signal Emitted when visible time range updates.*
- void [crosshairMoved](#) (double xMinutes)  
*@signal Emitted when crosshair moves.*
- void [intervalSelectionRequested](#) (double startSeconds, double endSeconds)  
*@signal Emitted when user selects time interval.*
- void [climbBoundaryEdited](#) (double oldStartSeconds, double oldEndSeconds, double newStartSeconds, double newEndSeconds)  
*@signal Emitted when climb boundaries are edited.*
- void [newClimbRequested](#) (double startSeconds, double endSeconds)  
*@signal Emitted when user creates new climb.*
- void [climbClicked](#) (int climbIndex)  
*@signal Emitted when climb is clicked.*

### Public Member Functions

- [RideChartWidget](#) (Metric metric, QWidget \*parent=nullptr)  
*Constructs ride chart for specified metric.*
- void [setRideData](#) (const [RideData](#) &rideData, ColorMetric colorMetric=ColorMetric::None, const [ColorContext](#) &colorContext={})  
*Sets ride data to display.*

- void [setPowerSmoothingSeconds](#) (int seconds)  
*Sets power smoothing window.*
- void [setAutoSmoothingEnabled](#) (bool enabled)  
*Enables/disables automatic smoothing.*
- void [setIntervals](#) (const std::vector< [Interval](#) > &intervals)  
*Sets interval overlays.*
- void [setClimbs](#) (const std::vector< [Climb](#) > &climbs)  
*Sets climb overlays.*
- void [setMetricOverlay](#) (ColorMetric metric, bool enabled)  
*Enables/disables metric overlay.*
- void [setSelectedClimbIndex](#) (int index)  
*Highlights a specific climb.*
- void [setClimbEditingEnabled](#) (bool enabled)  
*Enables/disables climb editing mode.*
- void [clearChart](#) ()  
*Clears the chart.*
- bool [hasData](#) () const  
*Checks if chart has data.*
- double [visibleRangeStartMinutes](#) () const  
*Gets visible range start time.*
- double [visibleRangeEndMinutes](#) () const  
*Gets end time of visible range in minutes from activity start.*

### Protected Member Functions

- void [wheelEvent](#) (QWheelEvent \*event) override  
*Handles wheel scroll for panning and zooming.*
- void [mousePressEvent](#) (QMouseEvent \*event) override  
*Handles mouse press for chart interaction.*
- void [mouseReleaseEvent](#) (QMouseEvent \*event) override  
*Handles mouse release for chart interaction.*
- void [mouseDoubleClickEvent](#) (QMouseEvent \*event) override  
*Handles double-click for interval creation.*
- void [mouseMoveEvent](#) (QMouseEvent \*event) override  
*Handles mouse movement for crosshair and selection.*
- void [leaveEvent](#) (QEvent \*event) override  
*Handles mouse leaving widget.*
- void [drawForeground](#) (QPainter \*painter, const QRectF &rect) override  
*Renders chart foreground elements.*
- void [resizeEvent](#) (QResizeEvent \*event) override  
*Handles widget resize events.*

### Private Types

- enum class [ClimbHandleDrag](#) { [None](#) , [Start](#) , [End](#) }

### Private Member Functions

- void [rebuildChart](#) (bool preserveXRange=true)
- std::vector< double > [computeSmoothedSeries](#) (Metric metric, int seconds) const
- std::vector< double > [computeSmoothedPowerSeries](#) (int seconds) const
- std::vector< int > [buildSampleIndices](#) (const std::vector< double > &values, int targetPointBudget) const
- void [clearBackgroundSeries](#) ()
- void [addZoneBackgroundBands](#) (double minY, double maxY, double maxXMinutes)
- void [addMetricOverlayBands](#) (double minY, double maxY)
- void [addIntervalBackgroundBands](#) (double minY, double maxY)
- void [addClimbBackgroundBands](#) (double minY, double maxY)
- void [updateXTicks](#) (double minVal, double maxVal)
- void [updateYAxis](#) ()
- void [updateYAxisForVisibleRange](#) (double xMinMin, double xMaxMin)

### Private Attributes

- Metric [m\\_metric](#)
- QLineSeries \* [m\\_series](#) = nullptr
- std::vector< QLineSeries \* > [m\\_rawSeries](#)
- std::vector< QLineSeries \* > [m\\_colorSeries](#)
- std::vector< QLineSeries \* > [m\\_overlaySeries](#)
- std::vector< QLineSeries \* > [m\\_referenceSeries](#)
- std::vector< QAreaSeries \* > [m\\_backgroundSeries](#)
- QValueAxis \* [m\\_axisX](#) = nullptr
- QCategoryAxis \* [m\\_axisXDisplay](#) = nullptr
- QValueAxis \* [m\\_axisY](#) = nullptr
- QValueAxis \* [m\\_axisYOverlay](#) = nullptr
- [RideData](#) [m\\_rideData](#)
- std::vector< [Interval](#) > [m\\_intervals](#)
- std::vector< [Climb](#) > [m\\_climbs](#)
- bool [m\\_metricOverlayEnabled](#) = false
- ColorMetric [m\\_metricOverlayMetric](#) = ColorMetric::None
- ColorMetric [m\\_colorMetric](#) = ColorMetric::None
- [ColorContext](#) [m\\_colorContext](#)
- std::unordered\_map< int, std::vector< double > > [m\\_smoothedSeriesCache](#)
- std::vector< QPointF > [m\\_tooltipPoints](#)
- std::vector< double > [m\\_tooltipRawValues](#)
- std::vector< double > [m\\_tooltipDisplayValues](#)
- int [m\\_requestedPowerSmoothingSeconds](#) = 0
- int [m\\_effectivePowerSmoothingSeconds](#) = 0
- bool [m\\_autoSmoothing](#) = false
- bool [m\\_isRebuilding](#) = false
- bool [m\\_syncing](#) = false
- bool [m\\_hasData](#) = false
- struct {
  - double [cachedMinX](#) = -1.0
  - double [cachedMaxX](#) = -1.0
  - double [cachedAxisMin](#) = 0.0
  - double [cachedAxisMax](#) = 1.0
  - bool [valid](#) = false
- [m\\_yAxisRangeCache](#)
- double [m\\_origMaxX](#) = 1.0
- double [m\\_crosshairX](#) = -1.0
- double [m\\_crosshairY](#) = std::numeric\_limits<double>::quiet\_NaN()

- bool `m_intervalSelecting` = false
- double `m_intervalStartMin` = -1.0
- double `m_intervalEndMin` = -1.0
- bool `m_climbEditingEnabled` = false
- int `m_selectedClimbIndex` = -1
- ClimbHandleDrag `m_climbHandleDrag` = ClimbHandleDrag::None
- double `m_dragOriginalClimbStartSec` = -1.0
- double `m_dragOriginalClimbEndSec` = -1.0
- bool `m_newClimbCreating` = false
- bool `m_newClimbDragging` = false
- double `m_newClimbStartSec` = -1.0
- double `m_newClimbCurrentSec` = -1.0
- double `m_dataMinY` = 0.0
- double `m_dataMaxY` = 1.0

### 7.91.1 Detailed Description

Interactive chart displaying ride data by selected metric.

Shows power, heart rate, cadence, speed, or altitude over time with overlays for intervals, climbs, and zone visualization.

Definition at line 29 of file [RideChartWidget.h](#).

### 7.91.2 Member Enumeration Documentation

#### 7.91.2.1 ClimbHandleDrag

```
enum class RideChartWidget::ClimbHandleDrag [strong], [private]
```

Definition at line 257 of file [RideChartWidget.h](#).

### 7.91.3 Constructor & Destructor Documentation

#### 7.91.3.1 RideChartWidget()

```
RideChartWidget::RideChartWidget (
    Metric metric,
    QWidget * parent = nullptr) [explicit]
```

Constructs ride chart for specified metric.

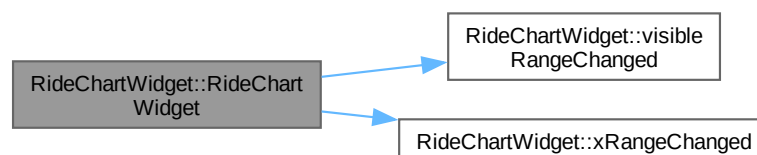
#### Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>metric</i> | Metric to display (Power, HeartRate, etc.). |
| <i>parent</i> | Parent widget.                              |

Definition at line 43 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#), and [xRangeChanged\(\)](#).

Here is the call graph for this function:





## 7.91.4 Member Function Documentation

### 7.91.4.1 addClimbBackgroundBands()

```
void RideChartWidget::addClimbBackgroundBands (
    double minY,
    double maxY) [private]
```

Definition at line 506 of file [RideChartWidget.cpp](#).

### 7.91.4.2 addIntervalBackgroundBands()

```
void RideChartWidget::addIntervalBackgroundBands (
    double minY,
    double maxY) [private]
```

Definition at line 395 of file [RideChartWidget.cpp](#).

### 7.91.4.3 addMetricOverlayBands()

```
void RideChartWidget::addMetricOverlayBands (
    double minY,
    double maxY) [private]
```

Definition at line 427 of file [RideChartWidget.cpp](#).

### 7.91.4.4 addZoneBackgroundBands()

```
void RideChartWidget::addZoneBackgroundBands (
    double minY,
    double maxY,
    double maxXMinutes) [private]
```

Definition at line 359 of file [RideChartWidget.cpp](#).

### 7.91.4.5 buildSampleIndices()

```
std::vector< int > RideChartWidget::buildSampleIndices (
    const std::vector< double > & values,
    int targetPointBudget) const [private]
```

Definition at line 262 of file [RideChartWidget.cpp](#).

### 7.91.4.6 clearBackgroundSeries()

```
void RideChartWidget::clearBackgroundSeries () [private]
```

Definition at line 349 of file [RideChartWidget.cpp](#).

### 7.91.4.7 clearChart()

```
void RideChartWidget::clearChart ()
```

Clears the chart.

Definition at line 1134 of file [RideChartWidget.cpp](#).

### 7.91.4.8 climbBoundaryEdited

```
void RideChartWidget::climbBoundaryEdited (
    double oldStartSeconds,
    double oldEndSeconds,
    double newStartSeconds,
    double newEndSeconds) [signal]
```

@signal Emitted when climb boundaries are edited.

**Parameters**

|                        |                                 |
|------------------------|---------------------------------|
| <i>oldStartSeconds</i> | Original start time in seconds. |
| <i>oldEndSeconds</i>   | Original end time in seconds.   |
| <i>newStartSeconds</i> | New start time in seconds.      |
| <i>newEndSeconds</i>   | New end time in seconds.        |

Referenced by [mouseReleaseEvent\(\)](#).

Here is the caller graph for this function:

**7.91.4.9 climbClicked**

```
void RideChartWidget::climbClicked (
    int climbIndex) [signal]
```

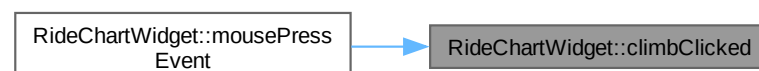
@signal Emitted when climb is clicked.

**Parameters**

|                   |                         |
|-------------------|-------------------------|
| <i>climbIndex</i> | Index of clicked climb. |
|-------------------|-------------------------|

Referenced by [mousePressEvent\(\)](#).

Here is the caller graph for this function:

**7.91.4.10 computeSmoothedPowerSeries()**

```
std::vector< double > RideChartWidget::computeSmoothedPowerSeries (
    int seconds) const [private]
```

Definition at line 187 of file [RideChartWidget.cpp](#).

**7.91.4.11 computeSmoothedSeries()**

```
std::vector< double > RideChartWidget::computeSmoothedSeries (
    Metric metric,
    int seconds) const [private]
```

Definition at line 192 of file [RideChartWidget.cpp](#).

#### 7.91.4.12 crosshairMoved

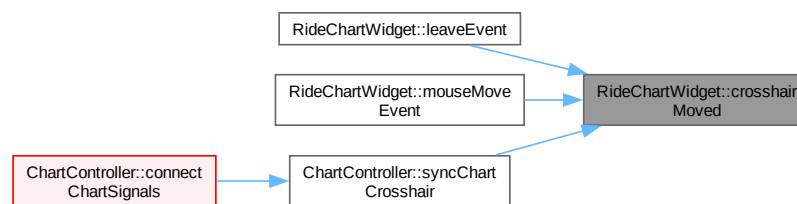
```
void RideChartWidget::crosshairMoved (
    double xMinutes) [signal]
```

@signal Emitted when crosshair moves.

##### Parameters

|                 |                            |
|-----------------|----------------------------|
| <i>xMinutes</i> | Crosshair time in minutes. |
|-----------------|----------------------------|

Referenced by [leaveEvent\(\)](#), [mouseMoveEvent\(\)](#), and [ChartController::syncChartCrosshair\(\)](#).  
Here is the caller graph for this function:



#### 7.91.4.13 drawForeground()

```
void RideChartWidget::drawForeground (
    QPainter * painter,
    const QRectF & rect) [override], [protected]
```

Renders chart foreground elements.

Definition at line 1709 of file [RideChartWidget.cpp](#).

References [Climb::endSeconds](#), and [Climb::startSeconds](#).

#### 7.91.4.14 fitToData

```
void RideChartWidget::fitToData () [slot]
```

@slot Alias for [resetZoom\(\)](#).

Definition at line 1233 of file [RideChartWidget.cpp](#).

References [resetZoom\(\)](#).

Here is the call graph for this function:



#### 7.91.4.15 hasData()

```
bool RideChartWidget::hasData () const [inline]
```

Checks if chart has data.

**Returns**

True if data is displayed.

Definition at line 103 of file [RideChartWidget.h](#).

**7.91.4.16 intervalSelectionRequested**

```
void RideChartWidget::intervalSelectionRequested (
    double startSeconds,
    double endSeconds) [signal]
```

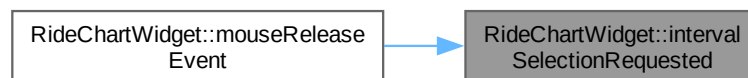
@signal Emitted when user selects time interval.

**Parameters**

|                     |                                            |
|---------------------|--------------------------------------------|
| <i>startSeconds</i> | Start time in seconds from activity start. |
| <i>endSeconds</i>   | End time in seconds from activity start.   |

Referenced by [mouseReleaseEvent\(\)](#).

Here is the caller graph for this function:

**7.91.4.17 leaveEvent()**

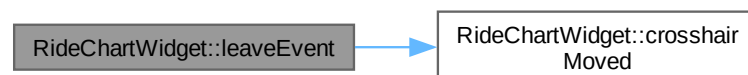
```
void RideChartWidget::leaveEvent (
    QEvent * event) [override], [protected]
```

Handles mouse leaving widget.

Definition at line 1691 of file [RideChartWidget.cpp](#).

References [crosshairMoved\(\)](#).

Here is the call graph for this function:

**7.91.4.18 mouseDoubleClickEvent()**

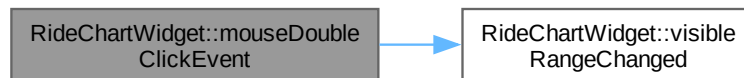
```
void RideChartWidget::mouseDoubleClickEvent (
    QMouseEvent * event) [override], [protected]
```

Handles double-click for interval creation.

Definition at line 1462 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Here is the call graph for this function:



#### 7.91.4.19 mouseMoveEvent()

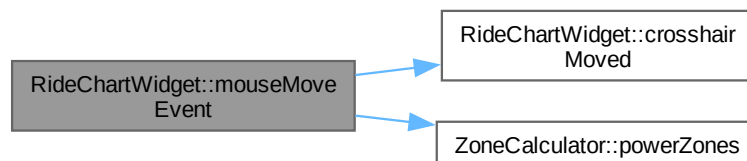
```
void RideChartWidget::mouseMoveEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse movement for crosshair and selection.

Definition at line 1497 of file [RideChartWidget.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [crosshairMoved\(\)](#), [Climb::endSeconds](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [Zone::maxValue](#), [Zone::minValue](#), [Zone::name](#), [RideRecord::power](#), [ZoneCalculator::powerZones\(\)](#), [RideRecord::speed](#), and [Climb::startSeconds](#).

Here is the call graph for this function:



#### 7.91.4.20 mousePressEvent()

```
void RideChartWidget::mousePressEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse press for chart interaction.

Definition at line 1278 of file [RideChartWidget.cpp](#).

References [climbClicked\(\)](#), [Climb::endSeconds](#), and [Climb::startSeconds](#).

Here is the call graph for this function:



#### 7.91.4.21 mouseReleaseEvent()

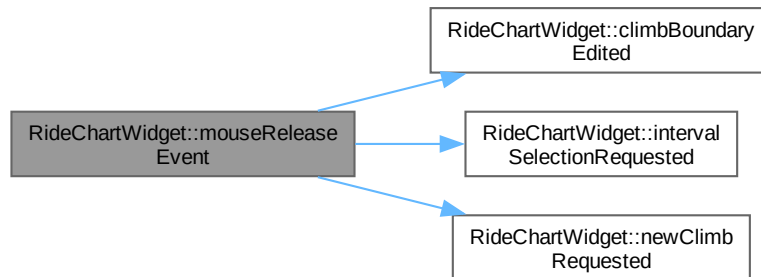
```
void RideChartWidget::mouseReleaseEvent (
    QMouseEvent * event) [override], [protected]
```

Handles mouse release for chart interaction.

Definition at line 1390 of file [RideChartWidget.cpp](#).

References [climbBoundaryEdited\(\)](#), [Climb::endSeconds](#), [intervalSelectionRequested\(\)](#), [newClimbRequested\(\)](#), and [Climb::startSeconds](#).

Here is the call graph for this function:



#### 7.91.4.22 newClimbRequested

```
void RideChartWidget::newClimbRequested (
    double startSeconds,
    double endSeconds) [signal]
```

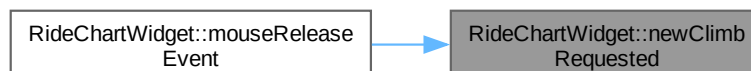
@signal Emitted when user creates new climb.

##### Parameters

|                     |                        |
|---------------------|------------------------|
| <i>startSeconds</i> | Start time in seconds. |
| <i>endSeconds</i>   | End time in seconds.   |

Referenced by [mouseReleaseEvent\(\)](#).

Here is the caller graph for this function:



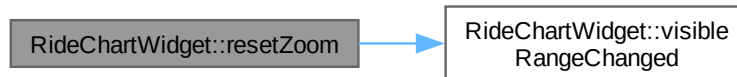
#### 7.91.4.23 rebuildChart()

```
void RideChartWidget::rebuildChart (
    bool preserveXRange = true) [private]
```

Definition at line 616 of file [RideChartWidget.cpp](#).

**7.91.4.24 resetZoom**

void RideChartWidget::resetZoom () [slot]  
 @slot Resets zoom to full ride extent.  
 Definition at line 1224 of file [RideChartWidget.cpp](#).  
 References [visibleRangeChanged\(\)](#).  
 Referenced by [fitToData\(\)](#).  
 Here is the call graph for this function:



Here is the caller graph for this function:

**7.91.4.25 resizeEvent()**

void RideChartWidget::resizeEvent (  
     QResizeEvent \* event) [override], [protected]  
 Handles widget resize events.  
 Definition at line 1123 of file [RideChartWidget.cpp](#).

**7.91.4.26 setAutoSmoothingEnabled()**

void RideChartWidget::setAutoSmoothingEnabled (  
     bool enabled)  
 Enables/disables automatic smoothing.

**Parameters**

|                |                                |
|----------------|--------------------------------|
| <i>enabled</i> | True to enable auto smoothing. |
|----------------|--------------------------------|

Definition at line 138 of file [RideChartWidget.cpp](#).

**7.91.4.27 setClimbEditingEnabled()**

void RideChartWidget::setClimbEditingEnabled (  
     bool enabled)  
 Enables/disables climb editing mode.

**Parameters**

|                |                         |
|----------------|-------------------------|
| <i>enabled</i> | True to enable editing. |
|----------------|-------------------------|

Definition at line 179 of file [RideChartWidget.cpp](#).

**7.91.4.28 setClimbs()**

```
void RideChartWidget::setClimbs (
    const std::vector< Climb > & climbs)
```

Sets climb overlays.

**Parameters**

|               |                  |
|---------------|------------------|
| <i>climbs</i> | Detected climbs. |
|---------------|------------------|

Definition at line 155 of file [RideChartWidget.cpp](#).

**7.91.4.29 setCrosshair**

```
void RideChartWidget::setCrosshair (
    double xMinutes) [slot]
```

@slot Sets crosshair position.

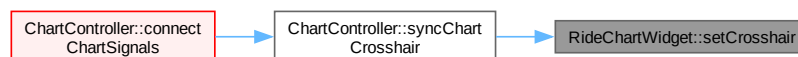
**Parameters**

|                 |                  |
|-----------------|------------------|
| <i>xMinutes</i> | Time in minutes. |
|-----------------|------------------|

Definition at line 1189 of file [RideChartWidget.cpp](#).

Referenced by [ChartController::syncChartCrosshair\(\)](#).

Here is the caller graph for this function:

**7.91.4.30 setIntervals()**

```
void RideChartWidget::setIntervals (
    const std::vector< Interval > & intervals)
```

Sets interval overlays.

**Parameters**

|                  |                          |
|------------------|--------------------------|
| <i>intervals</i> | Work/recovery intervals. |
|------------------|--------------------------|

Definition at line 148 of file [RideChartWidget.cpp](#).

**7.91.4.31 setMetricOverlay()**

```
void RideChartWidget::setMetricOverlay (
    ColorMetric metric,
    bool enabled)
```



Enables/disables metric overlay.

#### Parameters

|                |                       |
|----------------|-----------------------|
| <i>metric</i>  | Metric to overlay.    |
| <i>enabled</i> | True to show overlay. |

Definition at line 161 of file [RideChartWidget.cpp](#).

#### 7.91.4.32 setPowerSmoothingSeconds()

```
void RideChartWidget::setPowerSmoothingSeconds (  
    int seconds)
```

Sets power smoothing window.

#### Parameters

|                |                              |
|----------------|------------------------------|
| <i>seconds</i> | Smoothing window in seconds. |
|----------------|------------------------------|

Definition at line 127 of file [RideChartWidget.cpp](#).

#### 7.91.4.33 setRideData()

```
void RideChartWidget::setRideData (  
    const RideData & rideData,  
    ColorMetric colorMetric = ColorMetric::None,  
    const ColorContext & colorContext = {})
```

Sets ride data to display.

#### Parameters

|                     |                                               |
|---------------------|-----------------------------------------------|
| <i>rideData</i>     | <a href="#">Activity</a> data.                |
| <i>colorMetric</i>  | Metric to use for zone coloring.              |
| <i>colorContext</i> | <a href="#">Athlete</a> context for coloring. |

Definition at line 605 of file [RideChartWidget.cpp](#).

#### 7.91.4.34 setSelectedClimbIndex()

```
void RideChartWidget::setSelectedClimbIndex (  
    int index)
```

Highlights a specific climb.

#### Parameters

|              |                                            |
|--------------|--------------------------------------------|
| <i>index</i> | <a href="#">Climb</a> index (-1 to clear). |
|--------------|--------------------------------------------|

Definition at line 168 of file [RideChartWidget.cpp](#).

#### 7.91.4.35 setXRange

```
void RideChartWidget::setXRange (  
    double min,  
    double max) [slot]
```

@slot Sets visible x-axis range.

**Parameters**

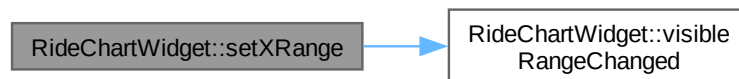
|            |                          |
|------------|--------------------------|
| <i>min</i> | Minimum time in minutes. |
| <i>max</i> | Maximum time in minutes. |

Definition at line 1181 of file [RideChartWidget.cpp](#).

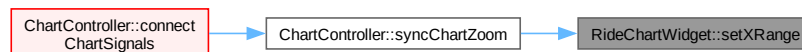
References [visibleRangeChanged\(\)](#).

Referenced by [ChartController::syncChartZoom\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.91.4.36 updateXTicks()**

```
void RideChartWidget::updateXTicks (
    double minVal,
    double maxVal) [private]
```

Definition at line 555 of file [RideChartWidget.cpp](#).

**7.91.4.37 updateYAxis()**

```
void RideChartWidget::updateYAxis () [private]
```

Definition at line 1062 of file [RideChartWidget.cpp](#).

**7.91.4.38 updateYAxisForVisibleRange()**

```
void RideChartWidget::updateYAxisForVisibleRange (
    double xMinMin,
    double xMaxMin) [private]
```

Definition at line 1075 of file [RideChartWidget.cpp](#).

**7.91.4.39 visibleRangeChanged**

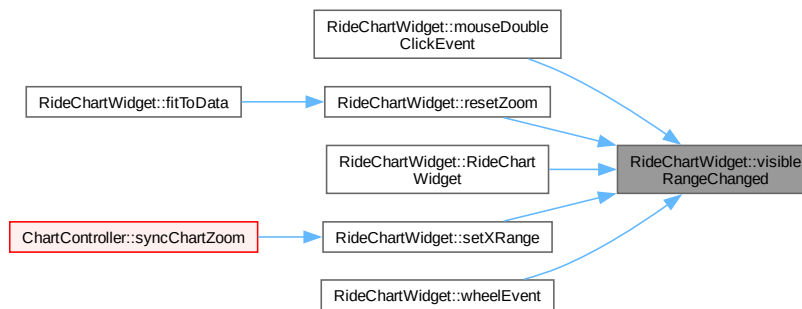
```
void RideChartWidget::visibleRangeChanged (
    double startMinutes,
    double endMinutes) [signal]
```

@signal Emitted when visible time range updates.

**Parameters**

|                     |                        |
|---------------------|------------------------|
| <i>startMinutes</i> | Start time in minutes. |
| <i>endMinutes</i>   | End time in minutes.   |

Referenced by [mouseDoubleClickEvent\(\)](#), [resetZoom\(\)](#), [RideChartWidget\(\)](#), [setXRange\(\)](#), and [wheelEvent\(\)](#).  
Here is the caller graph for this function:

**7.91.4.40 visibleRangeEndMinutes()**

```
double RideChartWidget::visibleRangeEndMinutes () const
```

Gets end time of visible range in minutes from activity start.

Definition at line 122 of file [RideChartWidget.cpp](#).

**7.91.4.41 visibleRangeStartMinutes()**

```
double RideChartWidget::visibleRangeStartMinutes () const
```

Gets visible range start time.

**Returns**

Start time in minutes.

Definition at line 117 of file [RideChartWidget.cpp](#).

**7.91.4.42 wheelEvent()**

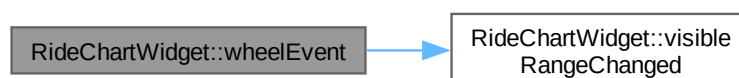
```
void RideChartWidget::wheelEvent (
    QWheelEvent * event) [override], [protected]
```

Handles wheel scroll for panning and zooming.

Definition at line 1238 of file [RideChartWidget.cpp](#).

References [visibleRangeChanged\(\)](#).

Here is the call graph for this function:



#### 7.91.4.43 xRangeChanged

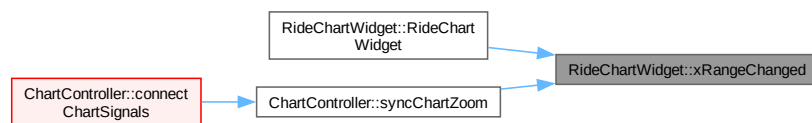
```
void RideChartWidget::xRangeChanged (
    double min,
    double max) [signal]
```

@signal Emitted when visible x range changes.

##### Parameters

|            |                          |
|------------|--------------------------|
| <i>min</i> | Minimum time in minutes. |
| <i>max</i> | Maximum time in minutes. |

Referenced by [RideChartWidget\(\)](#), and [ChartController::syncChartZoom\(\)](#).  
Here is the caller graph for this function:



### 7.91.5 Member Data Documentation

#### 7.91.5.1 cachedAxisMax

```
double RideChartWidget::cachedAxisMax = 1.0
```

Definition at line 246 of file [RideChartWidget.h](#).

#### 7.91.5.2 cachedAxisMin

```
double RideChartWidget::cachedAxisMin = 0.0
```

Definition at line 245 of file [RideChartWidget.h](#).

#### 7.91.5.3 cachedMaxX

```
double RideChartWidget::cachedMaxX = -1.0
```

Definition at line 244 of file [RideChartWidget.h](#).

#### 7.91.5.4 cachedMinX

```
double RideChartWidget::cachedMinX = -1.0
```

Definition at line 243 of file [RideChartWidget.h](#).

#### 7.91.5.5 m\_autoSmoothing

```
bool RideChartWidget::m_autoSmoothing = false [private]
```

Definition at line 235 of file [RideChartWidget.h](#).

#### 7.91.5.6 m\_axisX

```
QValueAxis* RideChartWidget::m_axisX = nullptr [private]
```

Definition at line 215 of file [RideChartWidget.h](#).

#### 7.91.5.7 m\_axisXDisplay

QCategoryAxis\* RideChartWidget::m\_axisXDisplay = nullptr [private]  
Definition at line 216 of file [RideChartWidget.h](#).

#### 7.91.5.8 m\_axisY

QValueAxis\* RideChartWidget::m\_axisY = nullptr [private]  
Definition at line 217 of file [RideChartWidget.h](#).

#### 7.91.5.9 m\_axisYOverlay

QValueAxis\* RideChartWidget::m\_axisYOverlay = nullptr [private]  
Definition at line 218 of file [RideChartWidget.h](#).

#### 7.91.5.10 m\_backgroundSeries

std::vector<QAreaSeries\*> RideChartWidget::m\_backgroundSeries [private]  
Definition at line 214 of file [RideChartWidget.h](#).

#### 7.91.5.11 m\_climbEditingEnabled

bool RideChartWidget::m\_climbEditingEnabled = false [private]  
Definition at line 255 of file [RideChartWidget.h](#).

#### 7.91.5.12 m\_climbHandleDrag

ClimbHandleDrag RideChartWidget::m\_climbHandleDrag = ClimbHandleDrag::None [private]  
Definition at line 263 of file [RideChartWidget.h](#).

#### 7.91.5.13 m\_climbs

std::vector<Climb> RideChartWidget::m\_climbs [private]  
Definition at line 221 of file [RideChartWidget.h](#).

#### 7.91.5.14 m\_colorContext

ColorContext RideChartWidget::m\_colorContext [private]  
Definition at line 225 of file [RideChartWidget.h](#).

#### 7.91.5.15 m\_colorMetric

ColorMetric RideChartWidget::m\_colorMetric = ColorMetric::None [private]  
Definition at line 224 of file [RideChartWidget.h](#).

#### 7.91.5.16 m\_colorSeries

std::vector<QLineSeries\*> RideChartWidget::m\_colorSeries [private]  
Definition at line 211 of file [RideChartWidget.h](#).

#### 7.91.5.17 m\_crosshairX

double RideChartWidget::m\_crosshairX = -1.0 [private]  
Definition at line 250 of file [RideChartWidget.h](#).

#### 7.91.5.18 m\_crosshairY

double RideChartWidget::m\_crosshairY = std::numeric\_limits<double>::quiet\_NaN() [private]  
Definition at line 251 of file [RideChartWidget.h](#).

#### 7.91.5.19 m\_dataMaxY

double RideChartWidget::m\_dataMaxY = 1.0 [private]  
Definition at line 271 of file [RideChartWidget.h](#).

#### 7.91.5.20 m\_dataMinY

double RideChartWidget::m\_dataMinY = 0.0 [private]  
Definition at line 270 of file [RideChartWidget.h](#).

#### 7.91.5.21 m\_dragOriginalClimbEndSec

double RideChartWidget::m\_dragOriginalClimbEndSec = -1.0 [private]  
Definition at line 265 of file [RideChartWidget.h](#).

#### 7.91.5.22 m\_dragOriginalClimbStartSec

double RideChartWidget::m\_dragOriginalClimbStartSec = -1.0 [private]  
Definition at line 264 of file [RideChartWidget.h](#).

#### 7.91.5.23 m\_effectivePowerSmoothingSeconds

int RideChartWidget::m\_effectivePowerSmoothingSeconds = 0 [private]  
Definition at line 234 of file [RideChartWidget.h](#).

#### 7.91.5.24 m\_hasData

bool RideChartWidget::m\_hasData = false [private]  
Definition at line 238 of file [RideChartWidget.h](#).

#### 7.91.5.25 m\_intervalEndMin

double RideChartWidget::m\_intervalEndMin = -1.0 [private]  
Definition at line 254 of file [RideChartWidget.h](#).

#### 7.91.5.26 m\_intervals

std::vector<[Interval](#)> RideChartWidget::m\_intervals [private]  
Definition at line 220 of file [RideChartWidget.h](#).

#### 7.91.5.27 m\_intervalSelecting

bool RideChartWidget::m\_intervalSelecting = false [private]  
Definition at line 252 of file [RideChartWidget.h](#).

#### 7.91.5.28 m\_intervalStartMin

double RideChartWidget::m\_intervalStartMin = -1.0 [private]  
Definition at line 253 of file [RideChartWidget.h](#).

#### 7.91.5.29 m\_isRebuilding

bool RideChartWidget::m\_isRebuilding = false [private]  
Definition at line 236 of file [RideChartWidget.h](#).

#### 7.91.5.30 m\_metric

Metric RideChartWidget::m\_metric [private]  
Definition at line 208 of file [RideChartWidget.h](#).

#### 7.91.5.31 m\_metricOverlayEnabled

`bool RideChartWidget::m_metricOverlayEnabled = false [private]`  
Definition at line 222 of file [RideChartWidget.h](#).

#### 7.91.5.32 m\_metricOverlayMetric

`ColorMetric RideChartWidget::m_metricOverlayMetric = ColorMetric::None [private]`  
Definition at line 223 of file [RideChartWidget.h](#).

#### 7.91.5.33 m\_newClimbCreating

`bool RideChartWidget::m_newClimbCreating = false [private]`  
Definition at line 266 of file [RideChartWidget.h](#).

#### 7.91.5.34 m\_newClimbCurrentSec

`double RideChartWidget::m_newClimbCurrentSec = -1.0 [private]`  
Definition at line 269 of file [RideChartWidget.h](#).

#### 7.91.5.35 m\_newClimbDragging

`bool RideChartWidget::m_newClimbDragging = false [private]`  
Definition at line 267 of file [RideChartWidget.h](#).

#### 7.91.5.36 m\_newClimbStartSec

`double RideChartWidget::m_newClimbStartSec = -1.0 [private]`  
Definition at line 268 of file [RideChartWidget.h](#).

#### 7.91.5.37 m\_origMaxX

`double RideChartWidget::m_origMaxX = 1.0 [private]`  
Definition at line 249 of file [RideChartWidget.h](#).

#### 7.91.5.38 m\_overlaySeries

`std::vector<QLineSeries*> RideChartWidget::m_overlaySeries [private]`  
Definition at line 212 of file [RideChartWidget.h](#).

#### 7.91.5.39 m\_rawSeries

`std::vector<QLineSeries*> RideChartWidget::m_rawSeries [private]`  
Definition at line 210 of file [RideChartWidget.h](#).

#### 7.91.5.40 m\_referenceSeries

`std::vector<QLineSeries*> RideChartWidget::m_referenceSeries [private]`  
Definition at line 213 of file [RideChartWidget.h](#).

#### 7.91.5.41 m\_requestedPowerSmoothingSeconds

`int RideChartWidget::m_requestedPowerSmoothingSeconds = 0 [private]`  
Definition at line 233 of file [RideChartWidget.h](#).

#### 7.91.5.42 m\_rideData

`RideData RideChartWidget::m_rideData [private]`  
Definition at line 219 of file [RideChartWidget.h](#).

**7.91.5.43 m\_selectedClimbIndex**

```
int RideChartWidget::m_selectedClimbIndex = -1 [private]
```

Definition at line 256 of file [RideChartWidget.h](#).

**7.91.5.44 m\_series**

```
QLineSeries* RideChartWidget::m_series = nullptr [private]
```

Definition at line 209 of file [RideChartWidget.h](#).

**7.91.5.45 m\_smoothedSeriesCache**

```
std::unordered_map<int, std::vector<double> > RideChartWidget::m_smoothedSeriesCache [mutable], [private]
```

Definition at line 229 of file [RideChartWidget.h](#).

**7.91.5.46 m\_syncing**

```
bool RideChartWidget::m_syncing = false [private]
```

Definition at line 237 of file [RideChartWidget.h](#).

**7.91.5.47 m\_tooltipDisplayValues**

```
std::vector<double> RideChartWidget::m_tooltipDisplayValues [private]
```

Definition at line 232 of file [RideChartWidget.h](#).

**7.91.5.48 m\_tooltipPoints**

```
std::vector<QPointF> RideChartWidget::m_tooltipPoints [private]
```

Definition at line 230 of file [RideChartWidget.h](#).

**7.91.5.49 m\_tooltipRawValues**

```
std::vector<double> RideChartWidget::m_tooltipRawValues [private]
```

Definition at line 231 of file [RideChartWidget.h](#).

**7.91.5.50 valid**

```
bool RideChartWidget::valid = false
```

Definition at line 247 of file [RideChartWidget.h](#).

The documentation for this class was generated from the following files:

- [src/charts/RideChartWidget.h](#)
- [src/charts/RideChartWidget.cpp](#)

**7.92 RideData Class Reference**

Container for all recorded samples and metadata for a single activity.

```
#include <RideData.h>
```

**Public Attributes**

- `std::vector< RideRecord > records`  
*Recorded samples.*
- `QString activityStartTimeUtc`  
*Activity start time (UTC, ISO 8601).*
- `QString activityEndTimeUtc`  
*Activity end time (UTC, ISO 8601).*
- `QString fileTimestampUtc`  
*File timestamp (UTC, ISO 8601).*



### 7.92.1 Detailed Description

Container for all recorded samples and metadata for a single activity.  
Definition at line 63 of file [RideData.h](#).

### 7.92.2 Member Data Documentation

#### 7.92.2.1 activityEndTimeUtc

`QString RideData::activityEndTimeUtc`

[Activity](#) end time (UTC, ISO 8601).

Definition at line 73 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.92.2.2 activityStartTimeUtc

`QString RideData::activityStartTimeUtc`

[Activity](#) start time (UTC, ISO 8601).

Definition at line 70 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.92.2.3 fileTimestampUtc

`QString RideData::fileTimestampUtc`

File timestamp (UTC, ISO 8601).

Definition at line 76 of file [RideData.h](#).

Referenced by [FitParser::load\(\)](#).

#### 7.92.2.4 records

`std::vector<RideRecord> RideData::records`

Recorded samples.

Definition at line 67 of file [RideData.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), [PowerCurve::bestMeanPower\(\)](#), [ClimbMetrics::build\(\)](#), [FTPZones::compute\(\)](#), [TrainingMetrics::compute\(\)](#), [ClimbDetector::computeAdaptiveSmoothingWindowsMeters\(\)](#), [ZoneCalculator::computeDistribution\(\)](#), [ClimbDetector::detect\(\)](#), [IntervalDetector::detect\(\)](#), [ZoneCalculator::hasMetricSamples\(\)](#), [FitParser::load\(\)](#), [TrainingLoad::normalizedPower\(\)](#), [ImportWorker::processJob\(\)](#), [AnalysisWorker::processTask\(\)](#), [PowerHistogramWidget::setRideData\(\)](#), [ClimbDetector::smoothAltitudeByDistance\(\)](#), [SmoothingEngine::smoothPower\(\)](#), and [ChartController::updatePowerCurve\(\)](#).

The documentation for this class was generated from the following file:

- `src/fit/RideData.h`

## 7.93 RideDataLoader Class Reference

Loads activity ride data samples from the database.

`#include <RideDataLoader.h>`

### Static Public Member Functions

- static [RideData loadActivitySamples](#) (int activityId, QSqlDatabase &db)

### 7.93.1 Detailed Description

Loads activity ride data samples from the database.

Definition at line 12 of file [RideDataLoader.h](#).

## 7.93.2 Member Function Documentation

### 7.93.2.1 loadActivitySamples()

```
RideData RideDataLoader::loadActivitySamples (
    int activityId,
    QSqlDatabase & db) [static]
```

Definition at line 8 of file [RideDataLoader.cpp](#).

The documentation for this class was generated from the following files:

- [src/model/RideDataLoader.h](#)
- [src/model/RideDataLoader.cpp](#)

## 7.94 RideDataSerializer Class Reference

Persists ride data to database.

```
#include <RideDataSerializer.h>
```

### Static Public Member Functions

- static int [saveRideToDatabase](#) (const [RideData](#) &rideData, const [WorkoutStatistics](#) &stats, int athleteId, const QString &filePath, const QString &fitHash, double normalizedPower, [DatabaseManager](#) &dbManager, [ActivityRepository](#) &activityRepo, [ImportRepository](#) &importRepo, QString \*errorOut=nullptr)
- static [RideData](#) [loadRideFromDatabase](#) (int activityId, [DatabaseManager](#) &dbManager)

### 7.94.1 Detailed Description

Persists ride data to database.

Saves parsed rides with computed statistics, detects duplicates, and logs all import attempts.

Definition at line 21 of file [RideDataSerializer.h](#).

## 7.94.2 Member Function Documentation

### 7.94.2.1 loadRideFromDatabase()

```
RideData RideDataSerializer::loadRideFromDatabase (
    int activityId,
    DatabaseManager & dbManager) [static]
```

Definition at line 171 of file [RideDataSerializer.cpp](#).

### 7.94.2.2 saveRideToDatabase()

```
int RideDataSerializer::saveRideToDatabase (
    const RideData & rideData,
    const WorkoutStatistics & stats,
    int athleteId,
    const QString & filePath,
    const QString & fitHash,
    double normalizedPower,
    DatabaseManager & dbManager,
    ActivityRepository & activityRepo,
    ImportRepository & importRepo,
    QString * errorOut = nullptr) [static]
```

Definition at line 17 of file [RideDataSerializer.cpp](#).

The documentation for this class was generated from the following files:

- [src/model/RideDataSerializer.h](#)
- [src/model/RideDataSerializer.cpp](#)

## 7.95 RideRecord Struct Reference

A single recorded sample from an activity recording.

```
#include <RideData.h>
```

### Public Attributes

- double [elapsedSeconds](#) = 0.0  
*Elapsed time in seconds from activity start.*
- double [latitude](#) = 0.0  
*GPS latitude in degrees.*
- double [longitude](#) = 0.0  
*GPS longitude in degrees.*
- double [power](#) = 0.0  
*Power in watts.*
- double [heartRate](#) = 0.0  
*Heart rate in bpm.*
- double [cadence](#) = 0.0  
*Cadence in rpm.*
- double [speed](#) = 0.0  
*Speed in km/h.*
- double [altitude](#) = 0.0  
*Altitude in meters.*
- bool [hasGps](#) = false  
*Whether GPS data is available.*
- bool [hasPower](#) = false  
*Whether power data is available.*
- bool [hasHeartRate](#) = false  
*Whether heart rate data is available.*
- bool [hasCadence](#) = false  
*Whether cadence data is available.*
- bool [hasSpeed](#) = false  
*Whether speed data is available.*
- bool [hasAltitude](#) = false  
*Whether altitude data is available.*

### 7.95.1 Detailed Description

A single recorded sample from an activity recording.

Contains time, position, power, heart rate, and other metrics recorded at one instant during an activity.

Definition at line 15 of file [RideData.h](#).

### 7.95.2 Member Data Documentation

#### 7.95.2.1 altitude

```
double RideRecord::altitude = 0.0
```

Altitude in meters.

Definition at line 39 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.2 cadence

```
double RideRecord::cadence = 0.0
```

Cadence in rpm.

Definition at line 33 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.3 elapsedSeconds

```
double RideRecord::elapsedSeconds = 0.0
```

Elapsed time in seconds from activity start.

Definition at line 18 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

### 7.95.2.4 hasAltitude

```
bool RideRecord::hasAltitude = false
```

Whether altitude data is available.

Definition at line 57 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [ClimbDetector::detect\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.5 hasCadence

```
bool RideRecord::hasCadence = false
```

Whether cadence data is available.

Definition at line 51 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.6 hasGps

```
bool RideRecord::hasGps = false
```

Whether GPS data is available.

Definition at line 42 of file [RideData.h](#).

Referenced by [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.7 hasHeartRate

```
bool RideRecord::hasHeartRate = false
```

Whether heart rate data is available.

Definition at line 48 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

### 7.95.2.8 hasPower

```
bool RideRecord::hasPower = false
```

Whether power data is available.

Definition at line 45 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMsg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

### 7.95.2.9 hasSpeed

```
bool RideRecord::hasSpeed = false
```

Whether speed data is available.

Definition at line 54 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

#### 7.95.2.10 heartRate

```
double RideRecord::heartRate = 0.0
```

Heart rate in bpm.

Definition at line 30 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

#### 7.95.2.11 latitude

```
double RideRecord::latitude = 0.0
```

GPS latitude in degrees.

Definition at line 21 of file [RideData.h](#).

Referenced by [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

#### 7.95.2.12 longitude

```
double RideRecord::longitude = 0.0
```

GPS longitude in degrees.

Definition at line 24 of file [RideData.h](#).

Referenced by [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

#### 7.95.2.13 power

```
double RideRecord::power = 0.0
```

Power in watts.

Definition at line 27 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [TrainingMetrics::compute\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), [VideoRenderJob::run\(\)](#), and [SmoothingEngine::smoothPower\(\)](#).

#### 7.95.2.14 speed

```
double RideRecord::speed = 0.0
```

Speed in km/h.

Definition at line 36 of file [RideData.h](#).

Referenced by [ClimbMetrics::build\(\)](#), [RideChartWidget::mouseMoveEvent\(\)](#), [FitRecordListener::OnMesg\(\)](#), [ImportWorker::processJob\(\)](#), and [VideoRenderJob::run\(\)](#).

The documentation for this struct was generated from the following file:

- [src/fit/RideData.h](#)

## 7.96 anonymous\_namespace{ClimbMetrics.cpp}::SliceStats Struct Reference

### Public Attributes

- double [avgPower](#) = 0.0
- double [avgHeartRate](#) = 0.0
- double [avgCadence](#) = 0.0

### 7.96.1 Detailed Description

Definition at line 12 of file [ClimbMetrics.cpp](#).

## 7.96.2 Member Data Documentation

### 7.96.2.1 avgCadence

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgCadence = 0.0`

Definition at line 16 of file [ClimbMetrics.cpp](#).

### 7.96.2.2 avgHeartRate

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgHeartRate = 0.0`

Definition at line 15 of file [ClimbMetrics.cpp](#).

### 7.96.2.3 avgPower

`double anonymous_namespace{ClimbMetrics.cpp}::SliceStats::avgPower = 0.0`

Definition at line 14 of file [ClimbMetrics.cpp](#).

The documentation for this struct was generated from the following file:

- [src/analysis/ClimbMetrics.cpp](#)

## 7.97 SmoothingEngine Class Reference

Provides data smoothing algorithms for activity metrics.

`#include <SmoothingEngine.h>`

### Static Public Member Functions

- static `std::vector< double > smoothPower` (const [RideData](#) &ride, int seconds)  
*Computes smoothed power series using rolling average.*

### 7.97.1 Detailed Description

Provides data smoothing algorithms for activity metrics.

Definition at line 13 of file [SmoothingEngine.h](#).

### 7.97.2 Member Function Documentation

#### 7.97.2.1 smoothPower()

```
std::vector< double > SmoothingEngine::smoothPower (
    const RideData & ride,
    int seconds) [static]
```

Computes smoothed power series using rolling average.

#### Parameters

|                |                                              |
|----------------|----------------------------------------------|
| <i>ride</i>    | <a href="#">Activity</a> data.               |
| <i>seconds</i> | Window width in seconds for rolling average. |

#### Returns

Smoothed power series (watts); NaN for invalid samples.

Definition at line 9 of file [SmoothingEngine.cpp](#).

References [RideRecord::elapsedSeconds](#), [RideRecord::hasPower](#), [RideRecord::power](#), and [RideData::records](#).

The documentation for this class was generated from the following files:

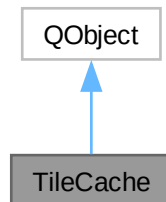
- [src/analysis/SmoothingEngine.h](#)
- [src/analysis/SmoothingEngine.cpp](#)

## 7.98 TileCache Class Reference

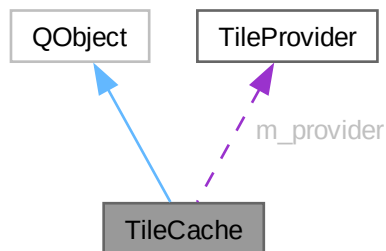
Caches map tiles in memory and on disk.

```
#include <TileCache.h>
```

Inheritance diagram for TileCache:



Collaboration diagram for TileCache:



### Classes

- struct [TileRequest](#)

### Signals

- void **tileLoaded** (int z, int x, int y)

### Public Member Functions

- [TileCache](#) (const [TileCacheConfig](#) &config={}, `QObject` \*parent=nullptr)
- `QPixmap` [tile](#) (int z, int x, int y)
- `QPixmap` [tileBlocking](#) (int z, int x, int y, bool allowNetwork=true)
- bool [isTileCachedOnDisk](#) (int z, int x, int y) const
- void [clearMemoryCache](#) ()
- void [setMemoryCacheSize](#) (int maxTiles)
- void [setMapStyle](#) (MapStyle style)
- MapStyle [mapStyle](#) () const
- `QString` [mapStyleName](#) () const

- QString [attribution](#) () const
- int [maxZoom](#) () const
- QString [diskCacheRoot](#) () const

#### Private Slots

- void [onReplyFinished](#) (QNetworkReply \*reply)

#### Private Member Functions

- QString [key](#) (int z, int x, int y) const
- QString [diskTilePathForRoot](#) (const QString &root, int z, int x, int y) const
- QString [diskTilePath](#) (int z, int x, int y) const
- void [dispatchNextDownloads](#) ()

#### Static Private Member Functions

- static QString [keyForStyle](#) (const QString &style, int z, int x, int y)
- static QString [diskTilePathForRoot](#) (const QString &root, const QString &style, int z, int x, int y)

#### Private Attributes

- QNetworkAccessManager [m\\_nam](#)
- QCache< QString, QPixmap > [m\\_cache](#)
- QSet< QString > [m\\_pending](#)
- QQueue< [TileRequest](#) > [m\\_downloadQueue](#)
- int [m\\_activeDownloads](#) = 0
- QString [m\\_diskCacheRoot](#)
- QStringList [m\\_diskFallbackRoots](#)
- MapStyle [m\\_mapStyle](#) = MapStyle::Light
- [TileProvider](#) [m\\_provider](#)

#### Static Private Attributes

- static constexpr int [kMaxConcurrentDownloads](#) = 6

### 7.98.1 Detailed Description

Caches map tiles in memory and on disk.

Provides tile lookup, network fetching, and memory/disk cache management with multiple map style support and fallback cache locations.

Definition at line 66 of file [TileCache.h](#).

### 7.98.2 Constructor & Destructor Documentation

#### 7.98.2.1 TileCache()

```
TileCache::TileCache (
    const TileCacheConfig & config = {},
    QObject * parent = nullptr) [explicit]
```

Definition at line 32 of file [TileCache.cpp](#).

### 7.98.3 Member Function Documentation

#### 7.98.3.1 attribution()

```
QString TileCache::attribution () const
```

Definition at line 127 of file [TileCache.cpp](#).



### 7.98.3.2 clearMemoryCache()

```
void TileCache::clearMemoryCache ()
```

Definition at line 86 of file [TileCache.cpp](#).

### 7.98.3.3 diskCacheRoot()

```
QString TileCache::diskCacheRoot () const
```

Definition at line 137 of file [TileCache.cpp](#).

### 7.98.3.4 diskTilePath()

```
QString TileCache::diskTilePath (
    int z,
    int x,
    int y) const [private]
```

Definition at line 69 of file [TileCache.cpp](#).

### 7.98.3.5 diskTilePathForRoot() [1/2]

```
QString TileCache::diskTilePathForRoot (
    const QString & root,
    const QString & style,
    int z,
    int x,
    int y) [static], [private]
```

Definition at line 60 of file [TileCache.cpp](#).

### 7.98.3.6 diskTilePathForRoot() [2/2]

```
QString TileCache::diskTilePathForRoot (
    const QString & root,
    int z,
    int x,
    int y) const [private]
```

Definition at line 55 of file [TileCache.cpp](#).

### 7.98.3.7 dispatchNextDownloads()

```
void TileCache::dispatchNextDownloads () [private]
```

Definition at line 248 of file [TileCache.cpp](#).

### 7.98.3.8 isTileCachedOnDisk()

```
bool TileCache::isTileCachedOnDisk (
    int z,
    int x,
    int y) const
```

Definition at line 74 of file [TileCache.cpp](#).

### 7.98.3.9 key()

```
QString TileCache::key (
    int z,
    int x,
    int y) const [private]
```

Definition at line 45 of file [TileCache.cpp](#).

#### 7.98.3.10 keyForStyle()

```
QString TileCache::keyForStyle (
    const QString & style,
    int z,
    int x,
    int y) [static], [private]
```

Definition at line 50 of file [TileCache.cpp](#).

#### 7.98.3.11 mapStyle()

```
MapStyle TileCache::mapStyle () const [inline]
```

Definition at line 84 of file [TileCache.h](#).

#### 7.98.3.12 mapStyleName()

```
QString TileCache::mapStyleName () const
```

Definition at line 122 of file [TileCache.cpp](#).

#### 7.98.3.13 maxZoom()

```
int TileCache::maxZoom () const
```

Definition at line 132 of file [TileCache.cpp](#).

#### 7.98.3.14 onReplyFinished

```
void TileCache::onReplyFinished (
    QNetworkReply * reply) [private], [slot]
```

Definition at line 270 of file [TileCache.cpp](#).

#### 7.98.3.15 setMapStyle()

```
void TileCache::setMapStyle (
    MapStyle style)
```

Definition at line 101 of file [TileCache.cpp](#).

#### 7.98.3.16 setMemoryCacheSize()

```
void TileCache::setMemoryCacheSize (
    int maxTiles)
```

Definition at line 96 of file [TileCache.cpp](#).

#### 7.98.3.17 tile()

```
QPixmap TileCache::tile (
    int z,
    int x,
    int y)
```

Definition at line 142 of file [TileCache.cpp](#).

#### 7.98.3.18 tileBlocking()

```
QPixmap TileCache::tileBlocking (
    int z,
    int x,
    int y,
    bool allowNetwork = true)
```

Definition at line 184 of file [TileCache.cpp](#).

## 7.98.4 Member Data Documentation

### 7.98.4.1 kMaxConcurrentDownloads

```
int TileCache::kMaxConcurrentDownloads = 6 [static], [constexpr], [private]
```

Definition at line 104 of file [TileCache.h](#).

### 7.98.4.2 m\_activeDownloads

```
int TileCache::m_activeDownloads = 0 [private]
```

Definition at line 122 of file [TileCache.h](#).

### 7.98.4.3 m\_cache

```
QCache<QString, QPixmap> TileCache::m_cache [private]
```

Definition at line 119 of file [TileCache.h](#).

### 7.98.4.4 m\_diskCacheRoot

```
QString TileCache::m_diskCacheRoot [private]
```

Definition at line 123 of file [TileCache.h](#).

### 7.98.4.5 m\_diskFallbackRoots

```
QStringList TileCache::m_diskFallbackRoots [private]
```

Definition at line 124 of file [TileCache.h](#).

### 7.98.4.6 m\_downloadQueue

```
QQueue<TileRequest> TileCache::m_downloadQueue [private]
```

Definition at line 121 of file [TileCache.h](#).

### 7.98.4.7 m\_mapStyle

```
MapStyle TileCache::m_mapStyle = MapStyle::Light [private]
```

Definition at line 125 of file [TileCache.h](#).

### 7.98.4.8 m\_nam

```
QNetworkAccessManager TileCache::m_nam [private]
```

Definition at line 118 of file [TileCache.h](#).

### 7.98.4.9 m\_pending

```
QSet<QString> TileCache::m_pending [private]
```

Definition at line 120 of file [TileCache.h](#).

### 7.98.4.10 m\_provider

```
TileProvider TileCache::m_provider [private]
```

Definition at line 126 of file [TileCache.h](#).

The documentation for this class was generated from the following files:

- [src/maps/TileCache.h](#)
- [src/maps/TileCache.cpp](#)

## 7.99 TileCacheConfig Struct Reference

Configuration for tile caching behavior.

```
#include <TileCache.h>
```

## Public Attributes

- int [maxTilesInMemory](#) = 512
- QString [diskCacheRoot](#)
- QStringList [diskFallbackRoots](#)

### 7.99.1 Detailed Description

Configuration for tile caching behavior.  
Definition at line 48 of file [TileCache.h](#).

### 7.99.2 Member Data Documentation

#### 7.99.2.1 diskCacheRoot

QString TileCacheConfig::diskCacheRoot  
Definition at line 54 of file [TileCache.h](#).

#### 7.99.2.2 diskFallbackRoots

QStringList TileCacheConfig::diskFallbackRoots  
Definition at line 57 of file [TileCache.h](#).

#### 7.99.2.3 maxTilesInMemory

int TileCacheConfig::maxTilesInMemory = 512  
Definition at line 51 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- src/maps/TileCache.h

## 7.100 TileProvider Struct Reference

Tile provider configuration.  
`#include <TileCache.h>`

## Public Attributes

- QString [name](#)
- QString [urlTemplate](#)
- int [maxZoom](#) = 18
- QString [attribution](#)

### 7.100.1 Detailed Description

Tile provider configuration.  
Definition at line 28 of file [TileCache.h](#).

### 7.100.2 Member Data Documentation

#### 7.100.2.1 attribution

QString TileProvider::attribution  
Definition at line 40 of file [TileCache.h](#).

#### 7.100.2.2 maxZoom

int TileProvider::maxZoom = 18  
Definition at line 37 of file [TileCache.h](#).

### 7.100.2.3 name

QString TileProvider::name

Definition at line 31 of file [TileCache.h](#).

### 7.100.2.4 urlTemplate

QString TileProvider::urlTemplate

Definition at line 34 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- src/maps/TileCache.h

## 7.101 TileCache::TileRequest Struct Reference

### Public Attributes

- int [z](#)
- int [x](#)
- int [y](#)

### 7.101.1 Detailed Description

Definition at line 102 of file [TileCache.h](#).

### 7.101.2 Member Data Documentation

#### 7.101.2.1 x

int TileCache::TileRequest::x

Definition at line 102 of file [TileCache.h](#).

#### 7.101.2.2 y

int TileCache::TileRequest::y

Definition at line 102 of file [TileCache.h](#).

#### 7.101.2.3 z

int TileCache::TileRequest::z

Definition at line 102 of file [TileCache.h](#).

The documentation for this struct was generated from the following file:

- src/maps/TileCache.h

## 7.102 VideoRenderer::TileViewport Struct Reference

### Public Member Functions

- bool [operator==](#) (const [TileViewport](#) &other) const

### Public Attributes

- int [tileX0](#) = 0
- int [tileY0](#) = 0
- int [nX](#) = 0
- int [nY](#) = 0

### 7.102.1 Detailed Description

Definition at line 14 of file [VideoRenderer.h](#).

### 7.102.2 Member Function Documentation

#### 7.102.2.1 operator==( )

```
bool VideoRenderer::TileViewport::operator== (
    const TileViewport & other) const [inline]
```

Definition at line 21 of file [VideoRenderer.h](#).

### 7.102.3 Member Data Documentation

#### 7.102.3.1 nX

```
int VideoRenderer::TileViewport::nX = 0
```

Definition at line 18 of file [VideoRenderer.h](#).

#### 7.102.3.2 nY

```
int VideoRenderer::TileViewport::nY = 0
```

Definition at line 19 of file [VideoRenderer.h](#).

#### 7.102.3.3 tileX0

```
int VideoRenderer::TileViewport::tileX0 = 0
```

Definition at line 16 of file [VideoRenderer.h](#).

#### 7.102.3.4 tileY0

```
int VideoRenderer::TileViewport::tileY0 = 0
```

Definition at line 17 of file [VideoRenderer.h](#).

The documentation for this struct was generated from the following file:

- [src/video/rendering/VideoRenderer.h](#)

## 7.103 TrainingLoad Class Reference

Computes training load metrics from activity power data.

```
#include <TrainingLoad.h>
```

### Static Public Member Functions

- static double [normalizedPower](#) (const [RideData](#) &rideData)  
*Calculates normalized power for an activity.*
- static double [intensityFactor](#) (double np, double ftp)  
*Calculates intensity factor.*
- static double [trainingStressScore](#) (double durationSeconds, double np, double ftp)  
*Calculates training stress score for an activity.*
- static std::vector< [FitnessMetricsPoint](#) > [fitnessTimeline](#) (const std::vector< [DailyLoadPoint](#) > &dailyLoads, int atlDays=7, int ctlDays=42)  
*Computes ATL/CTL/TSB sequence from daily training stress scores.*

### 7.103.1 Detailed Description

Computes training load metrics from activity power data.

Provides static methods to calculate normalized power, intensity factor, training stress score, and fitness timeline metrics (ATL, CTL, TSB).

Definition at line 43 of file [TrainingLoad.h](#).

## 7.103.2 Member Function Documentation

### 7.103.2.1 fitnessTimeline()

```
std::vector< FitnessMetricsPoint > TrainingLoad::fitnessTimeline (
    const std::vector< DailyLoadPoint > & dailyLoads,
    int atlDays = 7,
    int ctlDays = 42) [static]
```

Computes ATL/CTL/TSB sequence from daily training stress scores.

#### Parameters

|                   |                                                    |
|-------------------|----------------------------------------------------|
| <i>dailyLoads</i> | Daily TSS values.                                  |
| <i>atlDays</i>    | Acute training load window in days (default 7).    |
| <i>ctlDays</i>    | Chronic training load window in days (default 42). |

#### Returns

Timeline of fitness metrics.

Definition at line 54 of file [TrainingLoad.cpp](#).

References [FitnessMetricsPoint::atl](#), [FitnessMetricsPoint::ctl](#), and [FitnessMetricsPoint::tsb](#).

Referenced by [ChartController::updateFitnessChart\(\)](#).

Here is the caller graph for this function:



### 7.103.2.2 intensityFactor()

```
double TrainingLoad::intensityFactor (
    double np,
    double ftp) [static]
```

Calculates intensity factor.

#### Parameters

|            |                                      |
|------------|--------------------------------------|
| <i>np</i>  | Normalized power in watts.           |
| <i>ftp</i> | Functional threshold power in watts. |

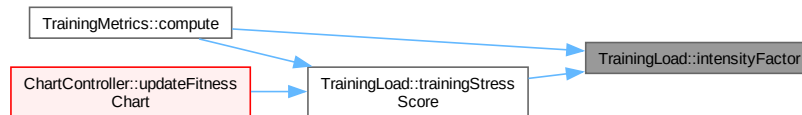
**Returns**

Intensity factor (NP/FTP ratio).

Definition at line 40 of file [TrainingLoad.cpp](#).

Referenced by [TrainingMetrics::compute\(\)](#), and [trainingStressScore\(\)](#).

Here is the caller graph for this function:

**7.103.2.3 normalizedPower()**

```
double TrainingLoad::normalizedPower (
    const RideData & rideData) [static]
```

Calculates normalized power for an activity.

**Parameters**

|                 |                                           |
|-----------------|-------------------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> data to analyze. |
|-----------------|-------------------------------------------|

**Returns**

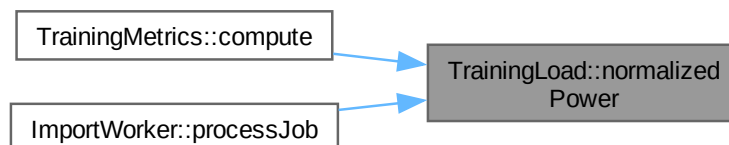
Normalized power in watts.

Definition at line 9 of file [TrainingLoad.cpp](#).

References [RideData::records](#).

Referenced by [TrainingMetrics::compute\(\)](#), and [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:

**7.103.2.4 trainingStressScore()**

```
double TrainingLoad::trainingStressScore (
    double durationSeconds,
    double np,
    double ftp) [static]
```

Calculates training stress score for an activity.



**Parameters**

|                        |                                               |
|------------------------|-----------------------------------------------|
| <i>durationSeconds</i> | <a href="#">Activity</a> duration in seconds. |
| <i>np</i>              | Normalized power in watts.                    |
| <i>ftp</i>             | Functional threshold power in watts.          |

**Returns**

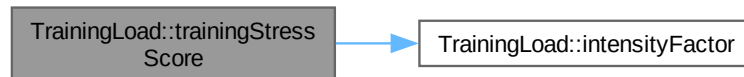
Training stress score.

Definition at line 46 of file [TrainingLoad.cpp](#).

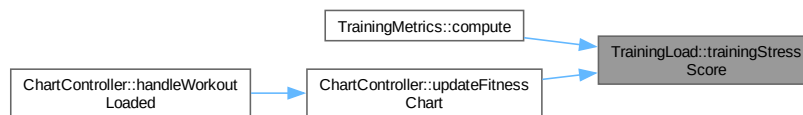
References [intensityFactor\(\)](#).

Referenced by [TrainingMetrics::compute\(\)](#), and [ChartController::updateFitnessChart\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/analysis/TrainingLoad.h`
- `src/analysis/TrainingLoad.cpp`

## 7.104 TrainingMetrics Class Reference

Computes training metrics from activity data.

```
#include <TrainingMetrics.h>
```

**Static Public Member Functions**

- static [TrainingMetricsSummary compute](#) (const [RideData](#) &rideData, double ftp, double duration↵  
Seconds=0.0)

*Computes training metrics for an activity.*

### 7.104.1 Detailed Description

Computes training metrics from activity data.

Definition at line 38 of file [TrainingMetrics.h](#).

## 7.104.2 Member Function Documentation

### 7.104.2.1 compute()

```
TrainingMetricsSummary TrainingMetrics::compute (
    const RideData & rideData,
    double ftp,
    double durationSeconds = 0.0) [static]
```

Computes training metrics for an activity.

#### Parameters

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| <i>rideData</i>        | Activity data.                                      |
| <i>ftp</i>             | Functional threshold power in watts.                |
| <i>durationSeconds</i> | Optional duration override (0 = use data duration). |

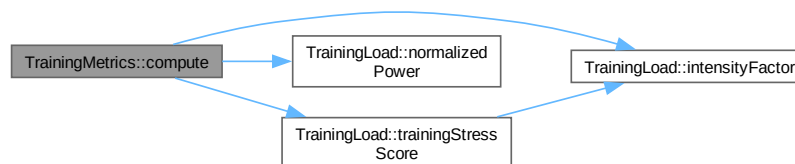
#### Returns

Metrics summary.

Definition at line 8 of file [TrainingMetrics.cpp](#).

References [TrainingMetricsSummary::averageHeartRate](#), [TrainingMetricsSummary::averagePower](#), [TrainingMetricsSummary::efficiency](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::heartRate](#), [TrainingLoad::intensityFactor\(\)](#), [TrainingMetricsSummary::intensityFactor](#), [TrainingLoad::normalizedPower\(\)](#), [TrainingMetricsSummary::normalizedPower](#), [RideRecord::power](#), [RideData::records](#), [TrainingLoad::trainingStressScore\(\)](#), [TrainingMetricsSummary::trainingStressScore](#), and [TrainingMetricsSummary::variabilityIndex](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [src/analysis/TrainingMetrics.h](#)
- [src/analysis/TrainingMetrics.cpp](#)

## 7.105 TrainingMetricsSummary Struct Reference

Summary of training-related metrics for an activity.

```
#include <TrainingMetrics.h>
```

#### Public Attributes

- double [normalizedPower](#) = 0.0  
*Normalized power in watts.*
- double [intensityFactor](#) = 0.0  
*Intensity factor (normalized power / FTP).*
- double [trainingStressScore](#) = 0.0  
*Training stress score (BikeScore).*

- double [variabilityIndex](#) = 0.0  
*Variability index (normalized power / average power).*
- double [efficiencyFactor](#) = 0.0  
*Efficiency factor (average power / average heart rate).*
- double [averagePower](#) = 0.0  
*Average power in watts.*
- double [averageHeartRate](#) = 0.0  
*Average heart rate in bpm.*

### 7.105.1 Detailed Description

Summary of training-related metrics for an activity.  
Definition at line 11 of file [TrainingMetrics.h](#).

### 7.105.2 Member Data Documentation

#### 7.105.2.1 averageHeartRate

`double TrainingMetricsSummary::averageHeartRate = 0.0`  
Average heart rate in bpm.  
Definition at line 32 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

#### 7.105.2.2 averagePower

`double TrainingMetricsSummary::averagePower = 0.0`  
Average power in watts.  
Definition at line 29 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

#### 7.105.2.3 efficiencyFactor

`double TrainingMetricsSummary::efficiencyFactor = 0.0`  
Efficiency factor (average power / average heart rate).  
Definition at line 26 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

#### 7.105.2.4 intensityFactor

`double TrainingMetricsSummary::intensityFactor = 0.0`  
Intensity factor (normalized power / FTP).  
Definition at line 17 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

#### 7.105.2.5 normalizedPower

`double TrainingMetricsSummary::normalizedPower = 0.0`  
Normalized power in watts.  
Definition at line 14 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

#### 7.105.2.6 trainingStressScore

`double TrainingMetricsSummary::trainingStressScore = 0.0`  
Training stress score (BikeScore).  
Definition at line 20 of file [TrainingMetrics.h](#).  
Referenced by [TrainingMetrics::compute\(\)](#).

### 7.105.2.7 variabilityIndex

`double TrainingMetricsSummary::variabilityIndex = 0.0`

Variability index (normalized power / average power).

Definition at line 23 of file [TrainingMetrics.h](#).

Referenced by [TrainingMetrics::compute\(\)](#).

The documentation for this struct was generated from the following file:

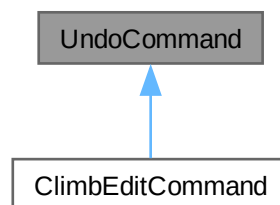
- `src/analysis/TrainingMetrics.h`

## 7.106 UndoCommand Class Reference

Abstract base class for all undo/redo operations.

`#include <UndoCommand.h>`

Inheritance diagram for UndoCommand:



### Public Member Functions

- virtual void [undo](#) ()=0  
*Reverses the command (undo operation).*
- virtual void [redo](#) ()=0  
*Re-applies the command (redo operation).*
- virtual QString [description](#) () const  
*Returns a human-readable description of the operation.*

### 7.106.1 Detailed Description

Abstract base class for all undo/redo operations.

Subclasses implement specific reversible edits by providing undo and redo methods.

Definition at line 13 of file [UndoCommand.h](#).

### 7.106.2 Member Function Documentation

#### 7.106.2.1 description()

`virtual QString UndoCommand::description () const [inline], [virtual]`

Returns a human-readable description of the operation.

Returns

Short description shown in menus/status bar.

Reimplemented in [ClimbEditCommand](#).

Definition at line 32 of file [UndoCommand.h](#).

### 7.106.2.2 redo()

```
virtual void UndoCommand::redo () [pure virtual]
```

Re-applies the command (redo operation).

Implemented in [ClimbEditCommand](#).

### 7.106.2.3 undo()

```
virtual void UndoCommand::undo () [pure virtual]
```

Reverses the command (undo operation).

Implemented in [ClimbEditCommand](#).

The documentation for this class was generated from the following file:

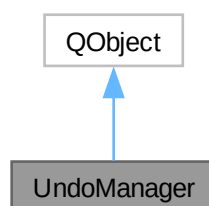
- `src/core/undo/UndoCommand.h`

## 7.107 UndoManager Class Reference

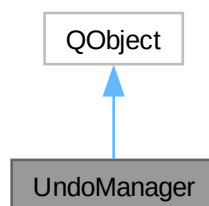
Central undo/redo stack manager.

```
#include <UndoManager.h>
```

Inheritance diagram for UndoManager:



Collaboration diagram for UndoManager:



### Public Slots

- void [undo](#) ()  
*Performs undo operation on the top command.*
- void [redo](#) ()  
*Performs redo operation on the next command.*

## Signals

- void **canUndoChanged** (bool canUndo)  
*Emitted when undo availability changes.*
- void **canRedoChanged** (bool canRedo)  
*Emitted when redo availability changes.*
- void **undoDescriptionChanged** (const QString &desc)  
*Emitted when undo description changes.*
- void **redoDescriptionChanged** (const QString &desc)  
*Emitted when redo description changes.*

## Public Member Functions

- [UndoManager](#) (int maxStackSize=50, QObject \*parent=nullptr)  
*Constructs the undo manager.*
- void [push](#) (std::unique\_ptr< [UndoCommand](#) > cmd)  
*Pushes a newly executed command onto the undo stack.*
- void [clear](#) ()  
*Clears all undo and redo history.*
- bool [canUndo](#) () const  
*Checks if undo is available.*
- bool [canRedo](#) () const  
*Checks if redo is available.*
- QString [undoDescription](#) () const  
*Gets description of the next undo operation.*
- QString [redoDescription](#) () const  
*Gets description of the next redo operation.*

## Private Member Functions

- void [notify](#) ()

## Private Attributes

- int [m\\_maxStackSize](#)
- std::deque< std::unique\_ptr< [UndoCommand](#) > > [m\\_undoStack](#)
- std::deque< std::unique\_ptr< [UndoCommand](#) > > [m\\_redoStack](#)

### 7.107.1 Detailed Description

Central undo/redo stack manager.

Manages a queue of reversible commands, supporting undo and redo operations with change notifications for UI updates.

Definition at line 20 of file [UndoManager.h](#).

### 7.107.2 Constructor & Destructor Documentation

#### 7.107.2.1 UndoManager()

```
UndoManager::UndoManager (
    int maxStackSize = 50,
    QObject * parent = nullptr) [explicit]
```

Constructs the undo manager.

**Parameters**

|                     |                                                    |
|---------------------|----------------------------------------------------|
| <i>maxStackSize</i> | Maximum number of commands to retain (default 50). |
| <i>parent</i>       | Qt parent object.                                  |

Definition at line 6 of file [UndoManager.cpp](#).

**7.107.3 Member Function Documentation****7.107.3.1 canRedo()**

```
bool UndoManager::canRedo () const
```

Checks if redo is available.

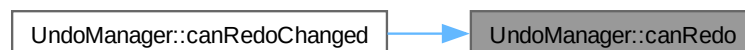
**Returns**

True if there are commands to redo.

Definition at line 32 of file [UndoManager.cpp](#).

Referenced by [canRedoChanged\(\)](#).

Here is the caller graph for this function:

**7.107.3.2 canUndo()**

```
bool UndoManager::canUndo () const
```

Checks if undo is available.

**Returns**

True if there are commands to undo.

Definition at line 31 of file [UndoManager.cpp](#).

Referenced by [canUndoChanged\(\)](#).

Here is the caller graph for this function:

**7.107.3.3 clear()**

```
void UndoManager::clear ()
```

Clears all undo and redo history.

Definition at line 24 of file [UndoManager.cpp](#).

#### 7.107.3.4 notify()

```
void UndoManager::notify () [private]
```

Definition at line 68 of file [UndoManager.cpp](#).

#### 7.107.3.5 push()

```
void UndoManager::push (
    std::unique_ptr< UndoCommand > cmd)
```

Pushes a newly executed command onto the undo stack.  
Clears the redo stack when a new command is pushed.

##### Parameters

|            |                                           |
|------------|-------------------------------------------|
| <i>cmd</i> | Unique pointer to the command to execute. |
|------------|-------------------------------------------|

Definition at line 11 of file [UndoManager.cpp](#).

#### 7.107.3.6 redo

```
void UndoManager::redo () [slot]
```

Performs redo operation on the next command.  
Definition at line 56 of file [UndoManager.cpp](#).

#### 7.107.3.7 redoDescription()

```
QString UndoManager::redoDescription () const
```

Gets description of the next redo operation.

##### Returns

Human-readable redo description.

Definition at line 39 of file [UndoManager.cpp](#).

#### 7.107.3.8 undo

```
void UndoManager::undo () [slot]
```

Performs undo operation on the top command.  
Definition at line 44 of file [UndoManager.cpp](#).

#### 7.107.3.9 undoDescription()

```
QString UndoManager::undoDescription () const
```

Gets description of the next undo operation.

##### Returns

Human-readable undo description.

Definition at line 34 of file [UndoManager.cpp](#).

### 7.107.4 Member Data Documentation

#### 7.107.4.1 m\_maxStackSize

```
int UndoManager::m_maxStackSize [private]
```

Definition at line 103 of file [UndoManager.h](#).

#### 7.107.4.2 m\_redoStack

```
std::deque<std::unique_ptr<UndoCommand> > UndoManager::m_redoStack [private]
```

Definition at line 105 of file [UndoManager.h](#).



### 7.107.4.3 m\_undoStack

`std::deque<std::unique_ptr<UndoCommand> > UndoManager::m_undoStack [private]`

Definition at line 104 of file [UndoManager.h](#).

The documentation for this class was generated from the following files:

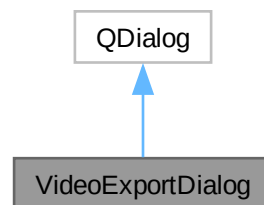
- `src/core/undo/UndoManager.h`
- `src/core/undo/UndoManager.cpp`

## 7.108 VideoExportDialog Class Reference

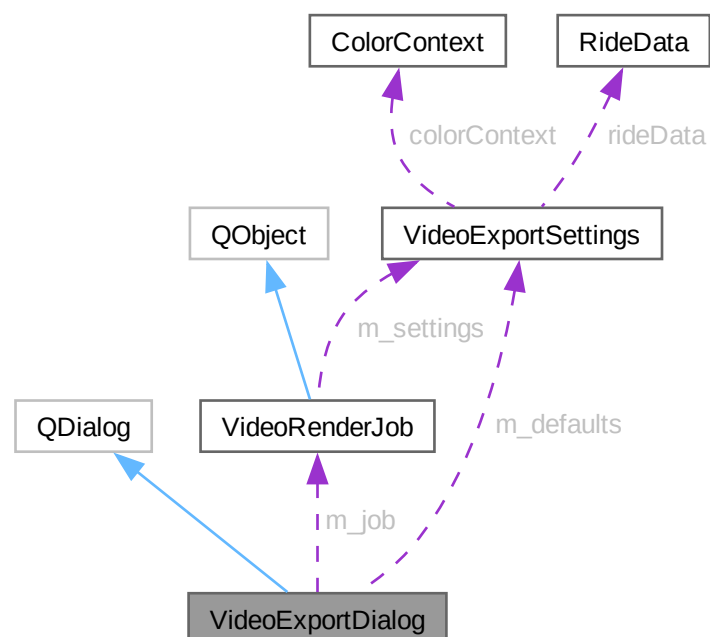
Dialog for configuring and launching activity video export.

`#include <VideoExportDialog.h>`

Inheritance diagram for VideoExportDialog:



Collaboration diagram for VideoExportDialog:



## Public Member Functions

- [VideoExportDialog](#) (const [VideoExportSettings](#) &defaults, QWidget \*parent=nullptr)  
*Constructs video export dialog.*

## Private Slots

- void [chooseOutputFile](#) ()
- void [startExport](#) ()
- void [cancelOrClose](#) ()
- void [onStageChanged](#) (const QString &stage)
- void [onProgressChanged](#) (int value, int maximum)
- void [onFrameProgressChanged](#) (int currentFrame, int totalFrames, const QString &etaText)
- void [onTileStatsChanged](#) (const QString &statsText)
- void [onFinished](#) (bool success, const QString &message, bool canceled)

## Private Member Functions

- [VideoExportSettings](#) [collectSettings](#) () const
- void [setAllOverlayChecks](#) (bool enabled)
- void [updateVideoLengthLabel](#) ()

## Private Attributes

- [VideoExportSettings](#) [m\\_defaults](#)
- QLineEdit \* [m\\_outputEdit](#) = nullptr
- QComboBox \* [m\\_resolutionCombo](#) = nullptr
- QSpinBox \* [m\\_fpsSpin](#) = nullptr
- QDoubleSpinBox \* [m\\_speedSpin](#) = nullptr
- QLabel \* [m\\_videoLengthLabel](#) = nullptr
- QRadioButton \* [m\\_selectedSegmentRadio](#) = nullptr
- QRadioButton \* [m\\_entireActivityRadio](#) = nullptr
- QComboBox \* [m\\_mapStyleCombo](#) = nullptr
- QSpinBox \* [m\\_fixedZoomSpin](#) = nullptr
- QComboBox \* [m\\_routeColorCombo](#) = nullptr
- QCheckBox \* [m\\_powerCheck](#) = nullptr
- QCheckBox \* [m\\_hrCheck](#) = nullptr
- QCheckBox \* [m\\_cadenceCheck](#) = nullptr
- QCheckBox \* [m\\_speedCheck](#) = nullptr
- QCheckBox \* [m\\_altitudeCheck](#) = nullptr
- QCheckBox \* [m\\_timeCheck](#) = nullptr
- QCheckBox \* [m\\_athleteCheck](#) = nullptr
- QCheckBox \* [m\\_telemetryPanelCheck](#) = nullptr
- QCheckBox \* [m\\_legendCheck](#) = nullptr
- QCheckBox \* [m\\_chartsCheck](#) = nullptr
- QCheckBox \* [m\\_deleteTemporaryTilesCheck](#) = nullptr
- QLabel \* [m\\_stageLabel](#) = nullptr
- QLabel \* [m\\_frameLabel](#) = nullptr
- QLabel \* [m\\_etaLabel](#) = nullptr
- QLabel \* [m\\_tileStatsLabel](#) = nullptr
- QProgressBar \* [m\\_progressBar](#) = nullptr
- QPushButton \* [m\\_exportButton](#) = nullptr
- QPushButton \* [m\\_cancelButton](#) = nullptr
- QThread \* [m\\_thread](#) = nullptr
- [VideoRenderJob](#) \* [m\\_job](#) = nullptr
- bool [m\\_running](#) = false

### 7.108.1 Detailed Description

Dialog for configuring and launching activity video export.

Provides UI for selecting output file, video format, map style, data overlays, and other video generation settings, then launches rendering job.

Definition at line 31 of file [VideoExportDialog.h](#).

### 7.108.2 Constructor & Destructor Documentation

#### 7.108.2.1 VideoExportDialog()

```
VideoExportDialog::VideoExportDialog (
    const VideoExportSettings & defaults,
    QWidget * parent = nullptr) [explicit]
```

Constructs video export dialog.

#### Parameters

|                 |                          |
|-----------------|--------------------------|
| <i>defaults</i> | Default export settings. |
| <i>parent</i>   | Parent widget.           |

Definition at line 85 of file [VideoExportDialog.cpp](#).

References [VideoExportSettings::fixedZoom](#), [VideoExportSettings::fps](#), [VideoExportSettings::height](#), [VideoExportSettings::mapStyle](#), [VideoExportSettings::outputPath](#), [VideoExportSettings::overlayAltitude](#), [VideoExportSettings::overlayAthleteName](#), [VideoExportSettings::overlayCadence](#), [VideoExportSettings::overlayCharts](#), [VideoExportSettings::overlayHeartRate](#), [VideoExportSettings::overlayPower](#), [VideoExportSettings::overlayRouteLegend](#), [VideoExportSettings::overlaySpeed](#), [VideoExportSettings::overlayTelemetryPanel](#), [VideoExportSettings::overlayTime](#), [VideoExportSettings::playbackSpeed](#), [VideoExportSettings::useSelectedSegment](#), and [VideoExportSettings::width](#).

#### 7.108.2.2 ~VideoExportDialog()

```
VideoExportDialog::~VideoExportDialog () [override]
```

Definition at line 294 of file [VideoExportDialog.cpp](#).

### 7.108.3 Member Function Documentation

#### 7.108.3.1 cancelOrClose

```
void VideoExportDialog::cancelOrClose () [private], [slot]
```

Definition at line 397 of file [VideoExportDialog.cpp](#).

#### 7.108.3.2 chooseOutputFile

```
void VideoExportDialog::chooseOutputFile () [private], [slot]
```

Definition at line 305 of file [VideoExportDialog.cpp](#).

#### 7.108.3.3 collectSettings()

```
VideoExportSettings VideoExportDialog::collectSettings () const [private]
```

Definition at line 316 of file [VideoExportDialog.cpp](#).

#### 7.108.3.4 onFinished

```
void VideoExportDialog::onFinished (
    bool success,
    const QString & message,
    bool canceled) [private], [slot]
```

Definition at line 432 of file [VideoExportDialog.cpp](#).

#### 7.108.3.5 onFrameProgressChanged

```
void VideoExportDialog::onFrameProgressChanged (
    int currentFrame,
    int totalFrames,
    const QString & etaText) [private], [slot]
```

Definition at line 420 of file [VideoExportDialog.cpp](#).

#### 7.108.3.6 onProgressChanged

```
void VideoExportDialog::onProgressChanged (
    int value,
    int maximum) [private], [slot]
```

Definition at line 414 of file [VideoExportDialog.cpp](#).

#### 7.108.3.7 onStageChanged

```
void VideoExportDialog::onStageChanged (
    const QString & stage) [private], [slot]
```

Definition at line 409 of file [VideoExportDialog.cpp](#).

#### 7.108.3.8 onTileStatsChanged

```
void VideoExportDialog::onTileStatsChanged (
    const QString & statsText) [private], [slot]
```

Definition at line 426 of file [VideoExportDialog.cpp](#).

#### 7.108.3.9 setAllOverlayChecks()

```
void VideoExportDialog::setAllOverlayChecks (
    bool enabled) [private]
```

Definition at line 453 of file [VideoExportDialog.cpp](#).

#### 7.108.3.10 startExport

```
void VideoExportDialog::startExport () [private], [slot]
```

Definition at line 355 of file [VideoExportDialog.cpp](#).

#### 7.108.3.11 updateVideoLengthLabel()

```
void VideoExportDialog::updateVideoLengthLabel () [private]
```

Definition at line 467 of file [VideoExportDialog.cpp](#).

### 7.108.4 Member Data Documentation

#### 7.108.4.1 m\_altitudeCheck

```
QCheckBox* VideoExportDialog::m_altitudeCheck = nullptr [private]
```

Definition at line 79 of file [VideoExportDialog.h](#).

#### 7.108.4.2 m\_athleteCheck

```
QCheckBox* VideoExportDialog::m_athleteCheck = nullptr [private]
```

Definition at line 81 of file [VideoExportDialog.h](#).

#### 7.108.4.3 m\_cadenceCheck

```
QCheckBox* VideoExportDialog::m_cadenceCheck = nullptr [private]
```

Definition at line 77 of file [VideoExportDialog.h](#).

#### 7.108.4.4 m\_cancelButton

QPushButton\* VideoExportDialog::m\_cancelButton = nullptr [private]  
Definition at line 94 of file [VideoExportDialog.h](#).

#### 7.108.4.5 m\_chartsCheck

QCheckBox\* VideoExportDialog::m\_chartsCheck = nullptr [private]  
Definition at line 84 of file [VideoExportDialog.h](#).

#### 7.108.4.6 m\_defaults

[VideoExportSettings](#) VideoExportDialog::m\_defaults [private]  
Definition at line 59 of file [VideoExportDialog.h](#).

#### 7.108.4.7 m\_deleteTemporaryTilesCheck

QCheckBox\* VideoExportDialog::m\_deleteTemporaryTilesCheck = nullptr [private]  
Definition at line 85 of file [VideoExportDialog.h](#).

#### 7.108.4.8 m\_entireActivityRadio

QRadioButton\* VideoExportDialog::m\_entireActivityRadio = nullptr [private]  
Definition at line 68 of file [VideoExportDialog.h](#).

#### 7.108.4.9 m\_etaLabel

QLabel\* VideoExportDialog::m\_etaLabel = nullptr [private]  
Definition at line 89 of file [VideoExportDialog.h](#).

#### 7.108.4.10 m\_exportButton

QPushButton\* VideoExportDialog::m\_exportButton = nullptr [private]  
Definition at line 93 of file [VideoExportDialog.h](#).

#### 7.108.4.11 m\_fixedZoomSpin

QSpinBox\* VideoExportDialog::m\_fixedZoomSpin = nullptr [private]  
Definition at line 71 of file [VideoExportDialog.h](#).

#### 7.108.4.12 m\_fpsSpin

QSpinBox\* VideoExportDialog::m\_fpsSpin = nullptr [private]  
Definition at line 63 of file [VideoExportDialog.h](#).

#### 7.108.4.13 m\_frameLabel

QLabel\* VideoExportDialog::m\_frameLabel = nullptr [private]  
Definition at line 88 of file [VideoExportDialog.h](#).

#### 7.108.4.14 m\_hrCheck

QCheckBox\* VideoExportDialog::m\_hrCheck = nullptr [private]  
Definition at line 76 of file [VideoExportDialog.h](#).

#### 7.108.4.15 m\_job

[VideoRenderJob](#)\* VideoExportDialog::m\_job = nullptr [private]  
Definition at line 97 of file [VideoExportDialog.h](#).

#### 7.108.4.16 m\_legendCheck

`QCheckBox* VideoExportDialog::m_legendCheck = nullptr [private]`  
Definition at line 83 of file [VideoExportDialog.h](#).

#### 7.108.4.17 m\_mapStyleCombo

`QComboBox* VideoExportDialog::m_mapStyleCombo = nullptr [private]`  
Definition at line 70 of file [VideoExportDialog.h](#).

#### 7.108.4.18 m\_outputEdit

`QLineEdit* VideoExportDialog::m_outputEdit = nullptr [private]`  
Definition at line 61 of file [VideoExportDialog.h](#).

#### 7.108.4.19 m\_powerCheck

`QCheckBox* VideoExportDialog::m_powerCheck = nullptr [private]`  
Definition at line 75 of file [VideoExportDialog.h](#).

#### 7.108.4.20 m\_progressBar

`QProgressBar* VideoExportDialog::m_progressBar = nullptr [private]`  
Definition at line 91 of file [VideoExportDialog.h](#).

#### 7.108.4.21 m\_resolutionCombo

`QComboBox* VideoExportDialog::m_resolutionCombo = nullptr [private]`  
Definition at line 62 of file [VideoExportDialog.h](#).

#### 7.108.4.22 m\_routeColorCombo

`QComboBox* VideoExportDialog::m_routeColorCombo = nullptr [private]`  
Definition at line 73 of file [VideoExportDialog.h](#).

#### 7.108.4.23 m\_running

`bool VideoExportDialog::m_running = false [private]`  
Definition at line 98 of file [VideoExportDialog.h](#).

#### 7.108.4.24 m\_selectedSegmentRadio

`QRadioButton* VideoExportDialog::m_selectedSegmentRadio = nullptr [private]`  
Definition at line 67 of file [VideoExportDialog.h](#).

#### 7.108.4.25 m\_speedCheck

`QCheckBox* VideoExportDialog::m_speedCheck = nullptr [private]`  
Definition at line 78 of file [VideoExportDialog.h](#).

#### 7.108.4.26 m\_speedSpin

`QDoubleSpinBox* VideoExportDialog::m_speedSpin = nullptr [private]`  
Definition at line 64 of file [VideoExportDialog.h](#).

#### 7.108.4.27 m\_stageLabel

`QLabel* VideoExportDialog::m_stageLabel = nullptr [private]`  
Definition at line 87 of file [VideoExportDialog.h](#).

**7.108.4.28 m\_telemetryPanelCheck**

QCheckBox\* VideoExportDialog::m\_telemetryPanelCheck = nullptr [private]  
 Definition at line 82 of file [VideoExportDialog.h](#).

**7.108.4.29 m\_thread**

QThread\* VideoExportDialog::m\_thread = nullptr [private]  
 Definition at line 96 of file [VideoExportDialog.h](#).

**7.108.4.30 m\_tileStatsLabel**

QLabel\* VideoExportDialog::m\_tileStatsLabel = nullptr [private]  
 Definition at line 90 of file [VideoExportDialog.h](#).

**7.108.4.31 m\_timeCheck**

QCheckBox\* VideoExportDialog::m\_timeCheck = nullptr [private]  
 Definition at line 80 of file [VideoExportDialog.h](#).

**7.108.4.32 m\_videoLengthLabel**

QLabel\* VideoExportDialog::m\_videoLengthLabel = nullptr [private]  
 Definition at line 65 of file [VideoExportDialog.h](#).

The documentation for this class was generated from the following files:

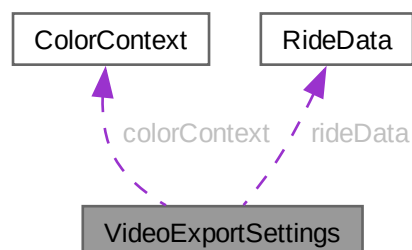
- [src/video/VideoExportDialog.h](#)
- [src/video/VideoExportDialog.cpp](#)

**7.109 VideoExportSettings Struct Reference**

Configuration for activity video export.

```
#include <VideoExportSettings.h>
```

Collaboration diagram for VideoExportSettings:

**Public Attributes**

- QString [outputPath](#)  
*Output file path.*
- QString [ffmpegPath](#)  
*FFmpeg executable path.*
- int [width](#) = 1920

- Output video width in pixels.*
  - int `height` = 1080
- Output video height in pixels.*
  - int `fps` = 30
- Output video frame rate in fps.*
  - double `playbackSpeed` = 3.0
- Playback speed multiplier.*
  - bool `useSelectedSegment` = true
- Whether to render only selected segment.*
  - double `segmentStartSeconds` = 0.0
- Segment start time in seconds.*
  - double `segmentEndSeconds` = 0.0
- Segment end time in seconds.*
  - MapStyle `mapStyle` = MapStyle::Light
- Map style for rendering.*
  - bool `autoZoom` = true
- Whether to enable automatic zoom adjustment.*
  - int `fixedZoom` = 18
- Fixed zoom level when autoZoom disabled.*
  - bool `followAthlete` = true
- Whether to center map on athlete position.*
  - ColorMetric `routeColorMetric` = ColorMetric::Power
- Color metric for route coloring.*
  - QColor `singleRouteColor` = QColor("#2563eb")
- Fixed route color when not using metric coloring.*
  - bool `overlayPower` = true
- Whether to overlay power data.*
  - bool `overlayHeartRate` = true
- Whether to overlay heart rate data.*
  - bool `overlayCadence` = true
- Whether to overlay cadence data.*
  - bool `overlaySpeed` = true
- Whether to overlay speed data.*
  - bool `overlayAltitude` = true
- Whether to overlay altitude data.*
  - bool `overlayTime` = true
- Whether to overlay elapsed time.*
  - bool `overlayAthleteName` = true
- Whether to overlay athlete name.*
  - bool `overlayTelemetryPanel` = true
- Whether to overlay telemetry panel.*
  - bool `overlayRouteLegend` = true
- Whether to overlay route legend.*
  - bool `overlayCharts` = true
- Whether to overlay metric charts.*
  - bool `deleteTemporaryTilesAfterExport` = true
- Whether to delete temporary tiles after export.*
  - QString `athleteName`
- Athlete name for display.*
  - QString `activityName`
- Activity name for display.*



- [ColorContext](#) colorContext

*Color context for metric coloring.*

- [RideData](#) rideData

*Activity ride data for rendering.*

### 7.109.1 Detailed Description

Configuration for activity video export.

Specifies output format, map rendering options, data overlays, and telemetry display settings for video generation.

Definition at line 19 of file [VideoExportSettings.h](#).

### 7.109.2 Member Data Documentation

#### 7.109.2.1 activityName

```
QString VideoExportSettings::activityName
```

[Activity](#) name for display.

Definition at line 103 of file [VideoExportSettings.h](#).

#### 7.109.2.2 athleteName

```
QString VideoExportSettings::athleteName
```

[Athlete](#) name for display.

Definition at line 100 of file [VideoExportSettings.h](#).

#### 7.109.2.3 autoZoom

```
bool VideoExportSettings::autoZoom = true
```

Whether to enable automatic zoom adjustment.

Definition at line 52 of file [VideoExportSettings.h](#).

#### 7.109.2.4 colorContext

```
ColorContext VideoExportSettings::colorContext
```

Color context for metric coloring.

Definition at line 106 of file [VideoExportSettings.h](#).

#### 7.109.2.5 deleteTemporaryTilesAfterExport

```
bool VideoExportSettings::deleteTemporaryTilesAfterExport = true
```

Whether to delete temporary tiles after export.

Definition at line 97 of file [VideoExportSettings.h](#).

#### 7.109.2.6 ffmpegPath

```
QString VideoExportSettings::ffmpegPath
```

FFmpeg executable path.

Definition at line 25 of file [VideoExportSettings.h](#).

#### 7.109.2.7 fixedZoom

```
int VideoExportSettings::fixedZoom = 18
```

Fixed zoom level when autoZoom disabled.

Definition at line 55 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.8 followAthlete

```
bool VideoExportSettings::followAthlete = true
```

Whether to center map on athlete position.

Definition at line 58 of file [VideoExportSettings.h](#).

#### 7.109.2.9 fps

```
int VideoExportSettings::fps = 30
```

Output video frame rate in fps.

Definition at line 34 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.10 height

```
int VideoExportSettings::height = 1080
```

Output video height in pixels.

Definition at line 31 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.11 mapStyle

```
MapStyle VideoExportSettings::mapStyle = MapStyle::Light
```

Map style for rendering.

Definition at line 49 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.12 outputPath

```
QString VideoExportSettings::outputPath
```

Output file path.

Definition at line 22 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.13 overlayAltitude

```
bool VideoExportSettings::overlayAltitude = true
```

Whether to overlay altitude data.

Definition at line 79 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.14 overlayAthleteName

```
bool VideoExportSettings::overlayAthleteName = true
```

Whether to overlay athlete name.

Definition at line 85 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.15 overlayCadence

```
bool VideoExportSettings::overlayCadence = true
```

Whether to overlay cadence data.

Definition at line 73 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.16 overlayCharts

```
bool VideoExportSettings::overlayCharts = true
```

Whether to overlay metric charts.

Definition at line 94 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.17 overlayHeartRate

```
bool VideoExportSettings::overlayHeartRate = true
```

Whether to overlay heart rate data.

Definition at line 70 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.18 overlayPower

```
bool VideoExportSettings::overlayPower = true
```

Whether to overlay power data.

Definition at line 67 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.19 overlayRouteLegend

```
bool VideoExportSettings::overlayRouteLegend = true
```

Whether to overlay route legend.

Definition at line 91 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.20 overlaySpeed

```
bool VideoExportSettings::overlaySpeed = true
```

Whether to overlay speed data.

Definition at line 76 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.21 overlayTelemetryPanel

```
bool VideoExportSettings::overlayTelemetryPanel = true
```

Whether to overlay telemetry panel.

Definition at line 88 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.22 overlayTime

```
bool VideoExportSettings::overlayTime = true
```

Whether to overlay elapsed time.

Definition at line 82 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.23 playbackSpeed

```
double VideoExportSettings::playbackSpeed = 3.0
```

Playback speed multiplier.

Definition at line 37 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.24 rideData

```
RideData VideoExportSettings::rideData
```

[Activity](#) ride data for rendering.

Definition at line 109 of file [VideoExportSettings.h](#).

#### 7.109.2.25 routeColorMetric

```
ColorMetric VideoExportSettings::routeColorMetric = ColorMetric::Power
```

Color metric for route coloring.

Definition at line 61 of file [VideoExportSettings.h](#).

#### 7.109.2.26 segmentEndSeconds

```
double VideoExportSettings::segmentEndSeconds = 0.0
```

Segment end time in seconds.

Definition at line 46 of file [VideoExportSettings.h](#).

#### 7.109.2.27 segmentStartSeconds

```
double VideoExportSettings::segmentStartSeconds = 0.0
```

Segment start time in seconds.

Definition at line 43 of file [VideoExportSettings.h](#).

#### 7.109.2.28 singleRouteColor

```
QColor VideoExportSettings::singleRouteColor = QColor("#2563eb")
```

Fixed route color when not using metric coloring.

Definition at line 64 of file [VideoExportSettings.h](#).

#### 7.109.2.29 useSelectedSegment

```
bool VideoExportSettings::useSelectedSegment = true
```

Whether to render only selected segment.

Definition at line 40 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

#### 7.109.2.30 width

```
int VideoExportSettings::width = 1920
```

Output video width in pixels.

Definition at line 28 of file [VideoExportSettings.h](#).

Referenced by [VideoExportDialog::VideoExportDialog\(\)](#).

The documentation for this struct was generated from the following file:

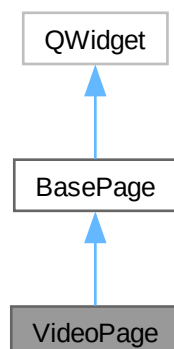
- [src/video/VideoExportSettings.h](#)

## 7.110 VideoPage Class Reference

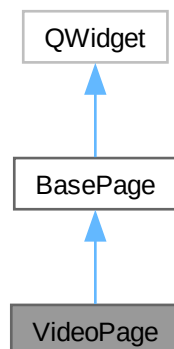
Top-level page for video creation and export.

```
#include <VideoPage.h>
```

Inheritance diagram for VideoPage:



Collaboration diagram for VideoPage:



### Signals

- void **previewRequested** ()  
*Emitted when the user requests a preview workflow.*
- void **renderRequested** ()  
*Emitted when the user requests a render workflow.*
- void **exportRequested** ()  
*Emitted when the user requests an export workflow.*

### Public Member Functions

- [VideoPage](#) (QWidget \*parent=nullptr)  
*Constructs the Video page.*

## Public Member Functions inherited from [BasePage](#)

- [BasePage](#) (QWidget \*parent=nullptr)
- virtual void [onActivated](#) ()

Called by [MainWindow](#) when this page becomes the active page.

### 7.110.1 Detailed Description

Top-level page for video creation and export.

Layout:

- Preview panel
- Render settings
- Output settings

Toolbar actions: Preview, Render, Export.

Currently wraps the existing "Create Video" dialog workflow. Future work: inline preview, render-progress bar, batch export.

Definition at line 20 of file [VideoPage.h](#).

### 7.110.2 Constructor & Destructor Documentation

#### 7.110.2.1 [VideoPage\(\)](#)

```
VideoPage::VideoPage (
    QWidget * parent = nullptr) [explicit]
```

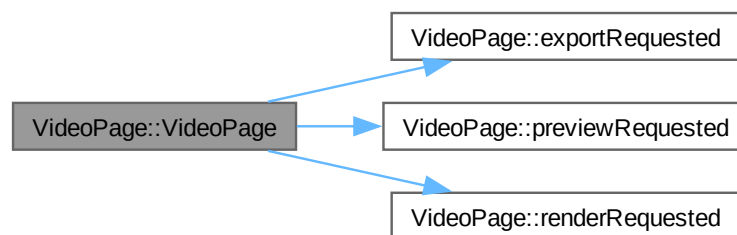
Constructs the Video page.

Shows a contextual toolbar and a three-section layout: Preview, Render Settings, and Output Settings.

Definition at line 19 of file [VideoPage.cpp](#).

References [exportRequested\(\)](#), [previewRequested\(\)](#), and [renderRequested\(\)](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

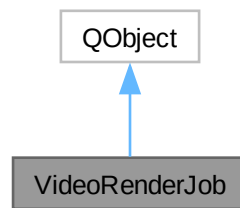
- `src/gui/pages/VideoPage.h`
- `src/gui/pages/VideoPage.cpp`

## 7.111 VideoRenderJob Class Reference

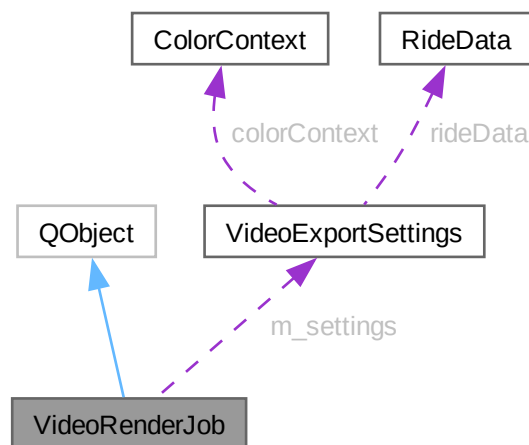
Renders activity video with map overlay and data visualization.

```
#include <VideoRenderJob.h>
```

Inheritance diagram for VideoRenderJob:



Collaboration diagram for VideoRenderJob:



## Signals

- void [stageChanged](#) (const QString &stage)  
@signal Emitted when render stage changes.
- void [progressChanged](#) (int value, int maximum)  
@signal Emitted when progress updates.
- void [frameProgressChanged](#) (int currentFrame, int totalFrames, const QString &etaText)  
@signal Emitted with frame-level progress.
- void [tileStatsChanged](#) (const QString &statsText)  
@signal Emitted when tile cache statistics update.
- void [finished](#) (bool success, const QString &message, bool canceled)  
@signal Emitted when rendering completes or fails.

## Public Member Functions

- [VideoRenderJob](#) ([VideoExportSettings](#) settings, QObject \*parent=nullptr)

*Constructs a video render job.*

- void [run](#) ()  
*@slot Runs the video rendering process.*
- void [cancel](#) ()  
*@slot Requests cancellation of rendering.*

### Private Attributes

- [VideoExportSettings](#) [m\\_settings](#)
- [QAtomicInteger](#)< int > [m\\_cancelRequested](#) { 0 }

## 7.111.1 Detailed Description

Renders activity video with map overlay and data visualization.

Generates video file from activity GPS and performance data, handling frame rendering, ffmpeg encoding, and progress reporting.

Definition at line 17 of file [VideoRenderJob.h](#).

## 7.111.2 Constructor & Destructor Documentation

### 7.111.2.1 VideoRenderJob()

```
VideoRenderJob::VideoRenderJob (
    VideoExportSettings settings,
    QObject * parent = nullptr) [explicit]
```

Constructs a video render job.

### Parameters

|                 |                  |
|-----------------|------------------|
| <i>settings</i> | Export settings. |
| <i>parent</i>   | Parent object.   |

Definition at line 74 of file [VideoRenderJob.cpp](#).

## 7.111.3 Member Function Documentation

### 7.111.3.1 cancel()

```
void VideoRenderJob::cancel ()
```

*@slot Requests cancellation of rendering.*

Definition at line 79 of file [VideoRenderJob.cpp](#).

### 7.111.3.2 finished

```
void VideoRenderJob::finished (
    bool success,
    const QString & message,
    bool canceled) [signal]
```

*@signal* Emitted when rendering completes or fails.

### Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>success</i>  | True if render completed successfully. |
| <i>message</i>  | Status or error message.               |
| <i>canceled</i> | True if render was canceled by user.   |

Referenced by [run\(\)](#).



Here is the caller graph for this function:



### 7.111.3.3 frameProgressChanged

```
void VideoRenderJob::frameProgressChanged (
    int currentFrame,
    int totalFrames,
    const QString & etaText) [signal]
```

@signal Emitted with frame-level progress.

#### Parameters

|                     |                           |
|---------------------|---------------------------|
| <i>currentFrame</i> | Current frame number.     |
| <i>totalFrames</i>  | Total number of frames.   |
| <i>etaText</i>      | Estimated time remaining. |

Referenced by [run\(\)](#).

Here is the caller graph for this function:



### 7.111.3.4 progressChanged

```
void VideoRenderJob::progressChanged (
    int value,
    int maximum) [signal]
```

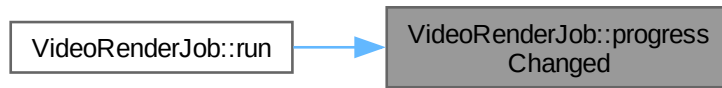
@signal Emitted when progress updates.

#### Parameters

|                |                         |
|----------------|-------------------------|
| <i>value</i>   | Current progress value. |
| <i>maximum</i> | Maximum progress value. |

Referenced by [run\(\)](#).

Here is the caller graph for this function:



### 7.111.3.5 run()

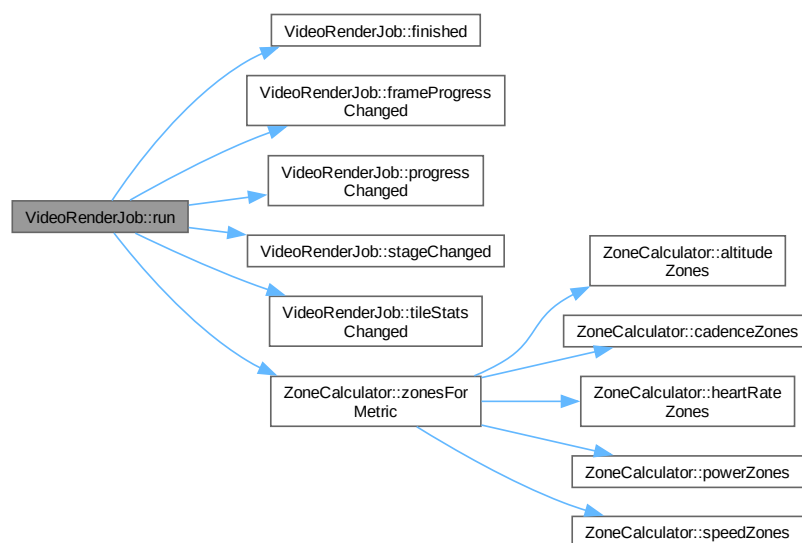
```
void VideoRenderJob::run ()
```

@slot Runs the video rendering process.

Definition at line 84 of file [VideoRenderJob.cpp](#).

References [RideRecord::altitude](#), [RideRecord::cadence](#), [VideoTileStats::downloadedTiles](#), [RideRecord::elapsedSeconds](#), [finished\(\)](#), [frameProgressChanged\(\)](#), [RideRecord::hasAltitude](#), [RideRecord::hasCadence](#), [RideRecord::hasGps](#), [RideRecord::hasHeartRate](#), [RideRecord::hasPower](#), [RideRecord::hasSpeed](#), [RideRecord::heartRate](#), [RideRecord::latitude](#), [RideRecord::longitude](#), [VideoTileStats::onDiskTiles](#), [RideRecord::power](#), [progressChanged\(\)](#), [VideoTileStats::requiredTiles](#), [RideRecord::speed](#), [stageChanged\(\)](#), [tileStatsChanged\(\)](#), and [ZoneCalculator::zonesForMetric\(\)](#).

Here is the call graph for this function:



### 7.111.3.6 stageChanged

```
void VideoRenderJob::stageChanged (
    const QString & stage) [signal]
```

@signal Emitted when render stage changes.

**Parameters**

|              |                               |
|--------------|-------------------------------|
| <i>stage</i> | Description of current stage. |
|--------------|-------------------------------|

Referenced by [run\(\)](#).

Here is the caller graph for this function:

**7.111.3.7 tileStatsChanged**

```
void VideoRenderJob::tileStatsChanged (
    const QString & statsText) [signal]
```

@signal Emitted when tile cache statistics update.

**Parameters**

|                  |                            |
|------------------|----------------------------|
| <i>statsText</i> | Formatted statistics text. |
|------------------|----------------------------|

Referenced by [run\(\)](#).

Here is the caller graph for this function:

**7.111.4 Member Data Documentation****7.111.4.1 m\_cancelRequested**

```
QAtomicInteger<int> VideoRenderJob::m_cancelRequested { 0 } [private]
```

Definition at line 67 of file [VideoRenderJob.h](#).

**7.111.4.2 m\_settings**

```
VideoExportSettings VideoRenderJob::m_settings [private]
```

Definition at line 66 of file [VideoRenderJob.h](#).

The documentation for this class was generated from the following files:

- `src/video/VideoRenderJob.h`
- `src/video/VideoRenderJob.cpp`

## 7.112 VideoTileId Struct Reference

Unique identifier for a map tile (z/x/y coordinates).

```
#include <VideoTileProvider.h>
```

### Public Member Functions

- bool `operator==` (const [VideoTileId](#) &other) const  
*Equality comparison operator.*

### Public Attributes

- int `z` = 0  
*Zoom level.*
- int `x` = 0  
*X coordinate.*
- int `y` = 0  
*Y coordinate.*

### 7.112.1 Detailed Description

Unique identifier for a map tile (z/x/y coordinates).

Definition at line 33 of file [VideoTileProvider.h](#).

### 7.112.2 Member Function Documentation

#### 7.112.2.1 `operator==()`

```
bool VideoTileId::operator== (
    const VideoTileId & other) const [inline]
```

Equality comparison operator.

Definition at line 45 of file [VideoTileProvider.h](#).

References [x](#), [y](#), and [z](#).

### 7.112.3 Member Data Documentation

#### 7.112.3.1 `x`

```
int VideoTileId::x = 0
```

X coordinate.

Definition at line 39 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

#### 7.112.3.2 `y`

```
int VideoTileId::y = 0
```

Y coordinate.

Definition at line 42 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

#### 7.112.3.3 `z`

```
int VideoTileId::z = 0
```

Zoom level.

Definition at line 36 of file [VideoTileProvider.h](#).

Referenced by [operator==\(\)](#).

The documentation for this struct was generated from the following file:

- `src/video/VideoTileProvider.h`

## 7.113 VideoTileIdHash Struct Reference

Hash function for [VideoTileId](#).

```
#include <VideoTileProvider.h>
```

### Public Member Functions

- `std::size_t operator() (const VideoTileId &id) const` noexcept  
*Computes hash for tile ID.*

### 7.113.1 Detailed Description

Hash function for [VideoTileId](#).

Definition at line 54 of file [VideoTileProvider.h](#).

### 7.113.2 Member Function Documentation

#### 7.113.2.1 operator>()

```
std::size_t VideoTileIdHash::operator() (
    const VideoTileId & id) const [inline], [noexcept]
```

Computes hash for tile ID.

Definition at line 57 of file [VideoTileProvider.h](#).

The documentation for this struct was generated from the following file:

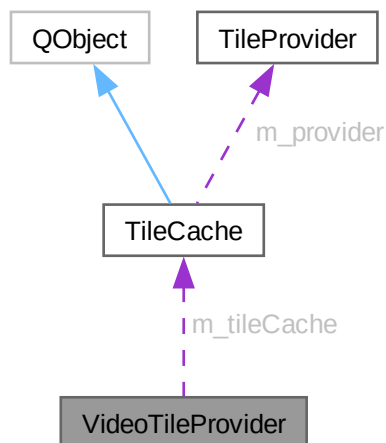
- `src/video/VideoTileProvider.h`

## 7.114 VideoTileProvider Class Reference

Manages map tiles for video export.

```
#include <VideoTileProvider.h>
```

Collaboration diagram for VideoTileProvider:



### Public Member Functions

- `VideoTileProvider (const VideoTileProviderConfig &config)`

- Constructs tile provider with configuration.*
- void [setRequiredTiles](#) (const std::unordered\_set< [VideoTileId](#), [VideoTileIdHash](#) > &required)  
*Sets the set of tiles required for video export.*
- int [requiredTileCount](#) () const  
*Gets count of required tiles.*
- int [requiredTilesAvailableOnDisk](#) () const  
*Gets count of required tiles available on disk.*
- QPixmap [getTile](#) (const [VideoTileId](#) &id, bool allowNetwork)  
*Retrieves a tile for rendering.*
- bool [isTileCachedOnDisk](#) (const [VideoTileId](#) &id) const  
*Checks if a tile is cached on disk.*
- [VideoTileStats](#) [stats](#) () const  
*Gets cache statistics.*
- void [clearMemoryCache](#) ()  
*Clears in-memory tile cache.*
- void [cleanupExportCache](#) ()  
*Cleans up export cache directory.*
- bool [shouldDeleteExportCacheAfterExport](#) () const  
*Checks if export cache should be deleted after export.*
- QString [exportCacheRoot](#) () const  
*Gets export cache root directory.*

### Private Attributes

- [TileCache](#) [m\\_tileCache](#)
- bool [m\\_deleteTemporaryTilesAfterExport](#) = true
- std::unordered\_set< [VideoTileId](#), [VideoTileIdHash](#) > [m\\_requiredTiles](#)
- int [m\\_requiredTilesOnDisk](#) = 0
- int [m\\_downloadedTiles](#) = 0

## 7.114.1 Detailed Description

Manages map tiles for video export.

Handles tile caching, download coordination, and cleanup for video rendering.

Definition at line 86 of file [VideoTileProvider.h](#).

## 7.114.2 Constructor & Destructor Documentation

### 7.114.2.1 VideoTileProvider()

```
VideoTileProvider::VideoTileProvider (
    const VideoTileProviderConfig & config) [explicit]
```

Constructs tile provider with configuration.

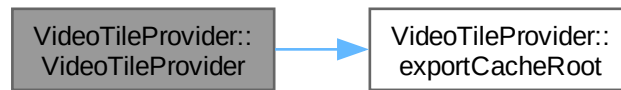
### Parameters

|               |                         |
|---------------|-------------------------|
| <i>config</i> | Provider configuration. |
|---------------|-------------------------|

Definition at line 30 of file [VideoTileProvider.cpp](#).

References [exportCacheRoot\(\)](#), and [VideoTileProviderConfig::style](#).

Here is the call graph for this function:



### 7.114.3 Member Function Documentation

#### 7.114.3.1 cleanupExportCache()

```
void VideoTileProvider::cleanupExportCache ()
```

Cleans up export cache directory.  
Definition at line 95 of file [VideoTileProvider.cpp](#).

#### 7.114.3.2 clearMemoryCache()

```
void VideoTileProvider::clearMemoryCache ()
```

Clears in-memory tile cache.  
Definition at line 90 of file [VideoTileProvider.cpp](#).

#### 7.114.3.3 exportCacheRoot()

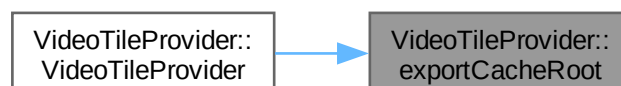
```
QString VideoTileProvider::exportCacheRoot () const
```

Gets export cache root directory.

##### Returns

Cache directory path.

Definition at line 110 of file [VideoTileProvider.cpp](#).  
Referenced by [VideoTileProvider\(\)](#).  
Here is the caller graph for this function:



#### 7.114.3.4 getTile()

```
QPixmap VideoTileProvider::getTile (
    const VideoTileId & id,
    bool allowNetwork)
```

Retrieves a tile for rendering.

**Parameters**

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>id</i>           | Tile identifier.                          |
| <i>allowNetwork</i> | True to fetch from network if not cached. |

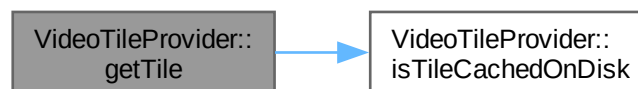
**Returns**

Tile pixmap, or empty if unavailable.

Definition at line 67 of file [VideoTileProvider.cpp](#).

References [isTileCachedOnDisk\(\)](#).

Here is the call graph for this function:

**7.114.3.5 isTileCachedOnDisk()**

```
bool VideoTileProvider::isTileCachedOnDisk (
    const VideoTileId & id) const
```

Checks if a tile is cached on disk.

**Parameters**

|           |                  |
|-----------|------------------|
| <i>id</i> | Tile identifier. |
|-----------|------------------|

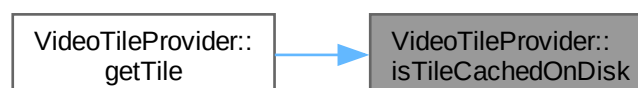
**Returns**

True if tile exists in disk cache.

Definition at line 85 of file [VideoTileProvider.cpp](#).

Referenced by [getTile\(\)](#).

Here is the caller graph for this function:

**7.114.3.6 requiredTileCount()**

```
int VideoTileProvider::requiredTileCount () const
```

Gets count of required tiles.



**Returns**

Number of required tiles.

Definition at line 57 of file [VideoTileProvider.cpp](#).

**7.114.3.7 requiredTilesAvailableOnDisk()**

```
int VideoTileProvider::requiredTilesAvailableOnDisk () const
```

Gets count of required tiles available on disk.

**Returns**

Number of tiles on disk cache.

Definition at line 62 of file [VideoTileProvider.cpp](#).

**7.114.3.8 setRequiredTiles()**

```
void VideoTileProvider::setRequiredTiles (
    const std::unordered_set< VideoTileId, VideoTileIdHash > & required)
```

Sets the set of tiles required for video export.

**Parameters**

|                 |                           |
|-----------------|---------------------------|
| <i>required</i> | Set of required tile IDs. |
|-----------------|---------------------------|

Definition at line 44 of file [VideoTileProvider.cpp](#).

**7.114.3.9 shouldDeleteExportCacheAfterExport()**

```
bool VideoTileProvider::shouldDeleteExportCacheAfterExport () const
```

Checks if export cache should be deleted after export.

**Returns**

True if temporary tiles should be cleaned up.

Definition at line 105 of file [VideoTileProvider.cpp](#).

**7.114.3.10 stats()**

```
VideoTileStats VideoTileProvider::stats () const
```

Gets cache statistics.

**Returns**

Stats including required, on-disk, and downloaded counts.

Definition at line 76 of file [VideoTileProvider.cpp](#).

References [VideoTileStats::downloadedTiles](#), [VideoTileStats::onDiskTiles](#), and [VideoTileStats::requiredTiles](#).

**7.114.4 Member Data Documentation****7.114.4.1 m\_deleteTemporaryTilesAfterExport**

```
bool VideoTileProvider::m_deleteTemporaryTilesAfterExport = true [private]
```

Definition at line 158 of file [VideoTileProvider.h](#).

**7.114.4.2 m\_downloadedTiles**

```
int VideoTileProvider::m_downloadedTiles = 0 [private]
```

Definition at line 161 of file [VideoTileProvider.h](#).

#### 7.114.4.3 m\_requiredTiles

`std::unordered_set<VideoTileId, VideoTileIdHash> VideoTileProvider::m_requiredTiles [private]`  
 Definition at line 159 of file [VideoTileProvider.h](#).

#### 7.114.4.4 m\_requiredTilesOnDisk

`int VideoTileProvider::m_requiredTilesOnDisk = 0 [private]`  
 Definition at line 160 of file [VideoTileProvider.h](#).

#### 7.114.4.5 m\_tileCache

`TileCache VideoTileProvider::m_tileCache [private]`  
 Definition at line 157 of file [VideoTileProvider.h](#).

The documentation for this class was generated from the following files:

- [src/video/VideoTileProvider.h](#)
- [src/video/VideoTileProvider.cpp](#)

## 7.115 VideoTileProviderConfig Struct Reference

Configuration for video tile provider.

`#include <VideoTileProvider.h>`

### Public Attributes

- `MapStyle style = MapStyle::Light`  
*Map style for video.*
- `int memoryCacheTiles = 96`  
*Maximum tiles to keep in memory.*
- `bool deleteTemporaryTilesAfterExport = true`  
*Whether to delete temporary tiles after export.*
- `QString exportCacheRoot`  
*Root directory for export cache.*

### 7.115.1 Detailed Description

Configuration for video tile provider.

Definition at line 15 of file [VideoTileProvider.h](#).

### 7.115.2 Member Data Documentation

#### 7.115.2.1 deleteTemporaryTilesAfterExport

`bool VideoTileProviderConfig::deleteTemporaryTilesAfterExport = true`

Whether to delete temporary tiles after export.

Definition at line 24 of file [VideoTileProvider.h](#).

#### 7.115.2.2 exportCacheRoot

`QString VideoTileProviderConfig::exportCacheRoot`

Root directory for export cache.

Definition at line 27 of file [VideoTileProvider.h](#).

#### 7.115.2.3 memoryCacheTiles

`int VideoTileProviderConfig::memoryCacheTiles = 96`

Maximum tiles to keep in memory.

Definition at line 21 of file [VideoTileProvider.h](#).

### 7.115.2.4 style

`MapStyle VideoTileProviderConfig::style = MapStyle::Light`

Map style for video.

Definition at line 18 of file [VideoTileProvider.h](#).

Referenced by [VideoTileProvider::VideoTileProvider\(\)](#).

The documentation for this struct was generated from the following file:

- `src/video/VideoTileProvider.h`

## 7.116 VideoTileStats Struct Reference

Tile availability statistics for export.

```
#include <VideoTileProvider.h>
```

### Public Attributes

- `int requiredTiles = 0`  
*Number of tiles required for video.*
- `int onDiskTiles = 0`  
*Number of tiles available on disk.*
- `int downloadedTiles = 0`  
*Number of tiles downloaded during export prep.*

### 7.116.1 Detailed Description

Tile availability statistics for export.

Definition at line 69 of file [VideoTileProvider.h](#).

### 7.116.2 Member Data Documentation

#### 7.116.2.1 downloadedTiles

```
int VideoTileStats::downloadedTiles = 0
```

Number of tiles downloaded during export prep.

Definition at line 78 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

#### 7.116.2.2 onDiskTiles

```
int VideoTileStats::onDiskTiles = 0
```

Number of tiles available on disk.

Definition at line 75 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

#### 7.116.2.3 requiredTiles

```
int VideoTileStats::requiredTiles = 0
```

Number of tiles required for video.

Definition at line 72 of file [VideoTileProvider.h](#).

Referenced by [VideoRenderJob::run\(\)](#), and [VideoTileProvider::stats\(\)](#).

The documentation for this struct was generated from the following file:

- `src/video/VideoTileProvider.h`

## 7.117 WeightEntry Struct Reference

Represents a historical weight entry.

```
#include <AthleteRepository.h>
```

## Public Attributes

- int `id` = -1  
*Unique weight entry identifier.*
- int `athleteld` = -1  
*Associated athlete ID.*
- double `weightKg` = 0.0  
*Body weight in kilograms.*
- QString `effectiveFrom`  
*Date weight becomes effective (ISO 8601 YYYY-MM-DD).*

### 7.117.1 Detailed Description

Represents a historical weight entry.

Definition at line 76 of file [AthleteRepository.h](#).

### 7.117.2 Member Data Documentation

#### 7.117.2.1 `athleteld`

```
int WeightEntry::athleteld = -1
```

Associated athlete ID.

Definition at line 82 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

#### 7.117.2.2 `effectiveFrom`

```
QString WeightEntry::effectiveFrom
```

Date weight becomes effective (ISO 8601 YYYY-MM-DD).

Definition at line 88 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

#### 7.117.2.3 `id`

```
int WeightEntry::id = -1
```

Unique weight entry identifier.

Definition at line 79 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::getWeightHistory\(\)](#).

#### 7.117.2.4 `weightKg`

```
double WeightEntry::weightKg = 0.0
```

Body weight in kilograms.

Definition at line 85 of file [AthleteRepository.h](#).

Referenced by [AthleteRepository::addWeightEntry\(\)](#), and [AthleteRepository::getWeightHistory\(\)](#).

The documentation for this struct was generated from the following file:

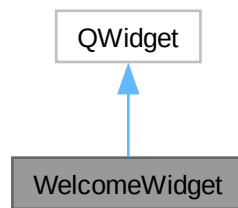
- `src/database/AthleteRepository.h`

## 7.118 WelcomeWidget Class Reference

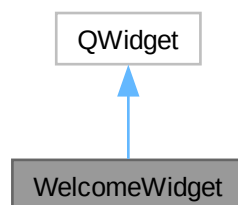
Dashboard welcome screen shown when no database is open or no activities exist.

```
#include <WelcomeWidget.h>
```

Inheritance diagram for WelcomeWidget:



Collaboration diagram for WelcomeWidget:



## Signals

- void **importRequested** ()  
*Emitted when the user clicks Import FIT Files.*
- void **openDatabaseRequested** ()  
*Emitted when the user clicks Open Database.*
- void **createDatabaseRequested** ()  
*Emitted when the user clicks Create Database.*
- void **manageAthletesRequested** ()  
*Emitted when the user clicks Manage Athletes.*

## Public Member Functions

- [WelcomeWidget](#) (QWidget \*parent=nullptr)  
*Constructs the welcome dashboard.*
- void [setFirstLaunch](#) (bool firstLaunch)  
*Switches the subtitle between first-launch and empty-database messages.*

## Private Attributes

- QLabel \* [m\\_subtitleLabel](#) = nullptr
- QPushButton \* [m\\_importButton](#) = nullptr
- QPushButton \* [m\\_openDbButton](#) = nullptr

- QPushButton \* [m\\_createDbButton](#) = nullptr
- QPushButton \* [m\\_manageAthletesButton](#) = nullptr

### 7.118.1 Detailed Description

Dashboard welcome screen shown when no database is open or no activities exist.

Layout:

- App title and subtitle
- Database row: Left: Recently Used Right: Pinned Databases
- Two-column row: Left: Quick Actions (Import / Open / Create / Manage Athletes) Right: Recent Activities (placeholder — populated in a future update)
- Sync Status section (Garmin Connect / Strava — coming in a future update)

The widget is hidden once the user has a database open with activities.

Definition at line 26 of file [WelcomeWidget.h](#).

### 7.118.2 Constructor & Destructor Documentation

#### 7.118.2.1 WelcomeWidget()

```
WelcomeWidget::WelcomeWidget (
    QWidget * parent = nullptr) [explicit]
```

Constructs the welcome dashboard.

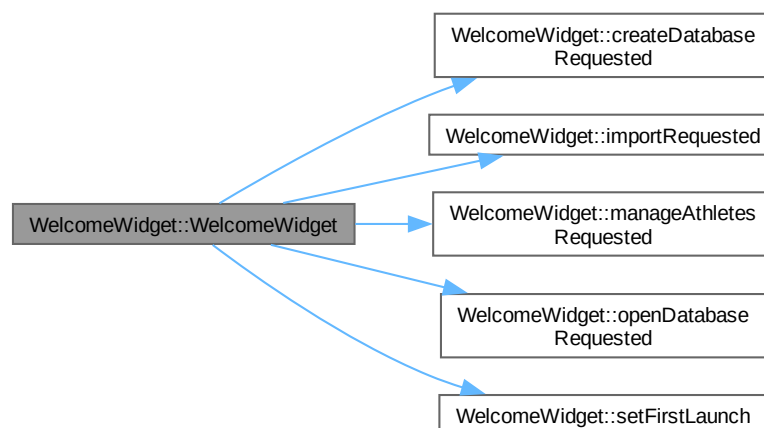
#### Parameters

|               |                |
|---------------|----------------|
| <i>parent</i> | Parent widget. |
|---------------|----------------|

Definition at line 70 of file [WelcomeWidget.cpp](#).

References [createDatabaseRequested\(\)](#), [importRequested\(\)](#), [manageAthletesRequested\(\)](#), [openDatabaseRequested\(\)](#), and [setFirstLaunch\(\)](#).

Here is the call graph for this function:



### 7.118.3 Member Function Documentation

#### 7.118.3.1 setFirstLaunch()

```
void WelcomeWidget::setFirstLaunch (
    bool firstLaunch)
```

Switches the subtitle between first-launch and empty-database messages.

##### Parameters

|                    |                                                        |
|--------------------|--------------------------------------------------------|
| <i>firstLaunch</i> | True = first-launch text; false = empty-database text. |
|--------------------|--------------------------------------------------------|

Definition at line 215 of file [WelcomeWidget.cpp](#).

Referenced by [WelcomeWidget\(\)](#).

Here is the caller graph for this function:



### 7.118.4 Member Data Documentation

#### 7.118.4.1 m\_createDbButton

```
QPushButton* WelcomeWidget::m_createDbButton = nullptr [private]
```

Definition at line 60 of file [WelcomeWidget.h](#).

#### 7.118.4.2 m\_importButton

```
QPushButton* WelcomeWidget::m_importButton = nullptr [private]
```

Definition at line 58 of file [WelcomeWidget.h](#).

#### 7.118.4.3 m\_manageAthletesButton

```
QPushButton* WelcomeWidget::m_manageAthletesButton = nullptr [private]
```

Definition at line 61 of file [WelcomeWidget.h](#).

#### 7.118.4.4 m\_openDbButton

```
QPushButton* WelcomeWidget::m_openDbButton = nullptr [private]
```

Definition at line 59 of file [WelcomeWidget.h](#).

#### 7.118.4.5 m\_subtitleLabel

```
QLabel* WelcomeWidget::m_subtitleLabel = nullptr [private]
```

Definition at line 57 of file [WelcomeWidget.h](#).

The documentation for this class was generated from the following files:

- `src/gui/WelcomeWidget.h`
- `src/gui/WelcomeWidget.cpp`

## 7.119 WorkoutAnalyzer Class Reference

Analyzes ride data to compute workout statistics.

```
#include <WorkoutAnalyzer.h>
```

## Public Member Functions

- [WorkoutStatistics analyze](#) (const [RideData](#) &rideData)

*Analyzes activity data to compute statistics.*

### 7.119.1 Detailed Description

Analyzes ride data to compute workout statistics.

Definition at line 53 of file [WorkoutAnalyzer.h](#).

### 7.119.2 Member Function Documentation

#### 7.119.2.1 analyze()

```
WorkoutStatistics WorkoutAnalyzer::analyze (
    const RideData & rideData)
```

Analyzes activity data to compute statistics.

#### Parameters

|                 |                                          |
|-----------------|------------------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> recording data. |
|-----------------|------------------------------------------|

#### Returns

Computed statistics.

Definition at line 19 of file [WorkoutAnalyzer.cpp](#).

References [WorkoutStatistics::averageCadence](#), [WorkoutStatistics::averageHeartRate](#), [WorkoutStatistics::averagePower](#), [WorkoutStatistics::averageSpeed](#), [WorkoutStatistics::durationSeconds](#), [WorkoutStatistics::elevationGainM](#), [WorkoutStatistics::maximumCadence](#), [WorkoutStatistics::maximumHeartRate](#), [WorkoutStatistics::maximumPower](#), [WorkoutStatistics::maximumSpeed](#), [WorkoutStatistics::movingTimeSeconds](#), [RideData::records](#), and [WorkoutStatistics::totalDistance](#).

Referenced by [ImportWorker::processJob\(\)](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [src/analysis/WorkoutAnalyzer.h](#)
- [src/analysis/WorkoutAnalyzer.cpp](#)

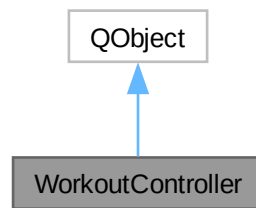
## 7.120 WorkoutController Class Reference

Manages activity loading, import, and analysis.

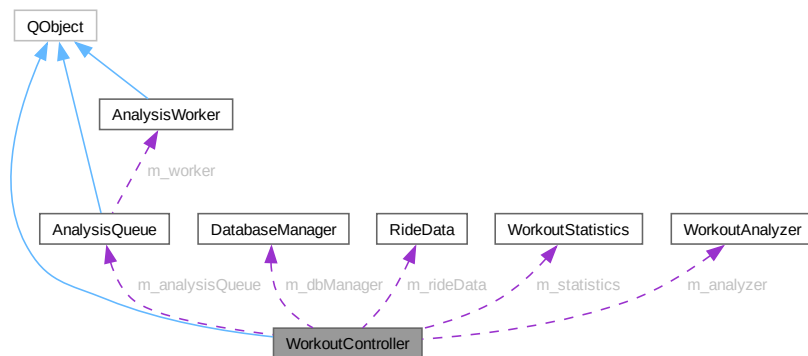
```
#include <WorkoutController.h>
```



Inheritance diagram for WorkoutController:



Collaboration diagram for WorkoutController:



## Signals

- void **workoutLoaded** ()
- void **activityImported** (int activityId)
- void **ftpChanged** (double ftp)

## Public Member Functions

- [WorkoutController](#) (QObject \*parent=nullptr)
- void [setFTP](#) (double ftp)
- double [ftp](#) () const
- void [setDatabaseManager](#) (DatabaseManager \*dbManager)
- void [setCurrentAthlete](#) (int athleteId)
- int [currentAthleteId](#) () const
- bool [loadFile](#) (const QString &path, QString &errorOut)
- int [importFile](#) (const QString &path, QString &errorOut, bool allowTimeOverlap=false, bool runAnalysis=true, const QString &importSource=QString(), DuplicateActivityInfo \*duplicateInfo=nullptr, ImportResult \*result=nullptr)
- bool [loadActivity](#) (int activityId, QString &errorOut)
- void [reloadClimbsFromDatabase](#) ()
- void [setAnalysisQueue](#) (AnalysisQueue \*queue)
- const [RideData](#) & [rideData](#) () const

- const [WorkoutStatistics](#) & [statistics](#) () const
- const std::vector< [ZoneDistribution](#) > & [powerZones](#) () const
- const std::vector< [PowerCurvePoint](#) > & [pdcPoints](#) () const
- const std::vector< [Interval](#) > & [intervals](#) () const
- const std::vector< [Climb](#) > & [climbs](#) () const
- double [normalizedPower](#) () const
- double [intensityFactor](#) () const
- double [trainingStressScore](#) () const
- double [variabilityIndex](#) () const
- double [efficiencyFactor](#) () const
- int [currentActivityId](#) () const

### Private Member Functions

- void [reanalyze](#) ()
- void [loadOrGenerateIntervals](#) ()
- void [loadOrGenerateClimbs](#) ()

### Private Attributes

- [AnalysisQueue](#) \* [m\\_analysisQueue](#) = nullptr
- double [m\\_ftp](#) = 250.0
- int [m\\_currentAthleteId](#) = -1
- int [m\\_currentActivityId](#) = -1
- [DatabaseManager](#) \* [m\\_dbManager](#) = nullptr
- [RideData](#) [m\\_rideData](#)
- [WorkoutStatistics](#) [m\\_statistics](#)
- [WorkoutAnalyzer](#) [m\\_analyzer](#)
- std::vector< [ZoneDistribution](#) > [m\\_powerZones](#)
- std::vector< [PowerCurvePoint](#) > [m\\_pdcPoints](#)
- std::vector< [Interval](#) > [m\\_intervals](#)
- std::vector< [Climb](#) > [m\\_climbs](#)
- double [m\\_np](#) = 0.0
- double [m\\_if](#) = 0.0
- double [m\\_tss](#) = 0.0
- double [m\\_vi](#) = 0.0
- double [m\\_ef](#) = 0.0

## 7.120.1 Detailed Description

Manages activity loading, import, and analysis.

Central controller for ride data loading, importing FIT files, and coordinating analysis operations including climb detection, interval detection, and metrics computation.

Definition at line 61 of file [WorkoutController.h](#).

## 7.120.2 Constructor & Destructor Documentation

### 7.120.2.1 WorkoutController()

```
WorkoutController::WorkoutController (
    QObject * parent = nullptr) [explicit]
```

Definition at line 54 of file [WorkoutController.cpp](#).

## 7.120.3 Member Function Documentation

### 7.120.3.1 climbs()

```
const std::vector< Climb > & WorkoutController::climbs () const [inline]
```

Definition at line 102 of file [WorkoutController.h](#).

### 7.120.3.2 currentActivityId()

```
int WorkoutController::currentActivityId () const [inline]
```

Definition at line 114 of file [WorkoutController.h](#).

### 7.120.3.3 currentAthleteId()

```
int WorkoutController::currentAthleteId () const [inline]
```

Definition at line 75 of file [WorkoutController.h](#).

### 7.120.3.4 efficiencyFactor()

```
double WorkoutController::efficiencyFactor () const [inline]
```

Definition at line 112 of file [WorkoutController.h](#).

### 7.120.3.5 ftp()

```
double WorkoutController::ftp () const [inline]
```

Definition at line 69 of file [WorkoutController.h](#).

### 7.120.3.6 importFile()

```
int WorkoutController::importFile (
    const QString & path,
    QString & errorOut,
    bool allowTimeOverlap = false,
    bool runAnalysis = true,
    const QString & importSource = QString(),
    DuplicateActivityInfo * duplicateInfo = nullptr,
    ImportResult * result = nullptr)
```

Definition at line 110 of file [WorkoutController.cpp](#).

### 7.120.3.7 intensityFactor()

```
double WorkoutController::intensityFactor () const [inline]
```

Definition at line 106 of file [WorkoutController.h](#).

### 7.120.3.8 intervals()

```
const std::vector< Interval > & WorkoutController::intervals () const [inline]
```

Definition at line 100 of file [WorkoutController.h](#).

### 7.120.3.9 loadActivity()

```
bool WorkoutController::loadActivity (
    int activityId,
    QString & errorOut)
```

Definition at line 225 of file [WorkoutController.cpp](#).

### 7.120.3.10 loadFile()

```
bool WorkoutController::loadFile (
    const QString & path,
    QString & errorOut)
```

Definition at line 92 of file [WorkoutController.cpp](#).

### 7.120.3.11 loadOrGenerateClimbs()

```
void WorkoutController::loadOrGenerateClimbs () [private]
```

Definition at line 365 of file [WorkoutController.cpp](#).

#### 7.120.3.12 loadOrGenerateIntervals()

void WorkoutController::loadOrGenerateIntervals () [private]  
Definition at line 293 of file [WorkoutController.cpp](#).

#### 7.120.3.13 normalizedPower()

double WorkoutController::normalizedPower () const [inline]  
Definition at line 104 of file [WorkoutController.h](#).

#### 7.120.3.14 pdcPoints()

const std::vector< [PowerCurvePoint](#) > & WorkoutController::pdcPoints () const [inline]  
Definition at line 98 of file [WorkoutController.h](#).

#### 7.120.3.15 powerZones()

const std::vector< [ZoneDistribution](#) > & WorkoutController::powerZones () const [inline]  
Definition at line 96 of file [WorkoutController.h](#).

#### 7.120.3.16 reanalyze()

void WorkoutController::reanalyze () [private]  
Definition at line 263 of file [WorkoutController.cpp](#).

#### 7.120.3.17 reloadClimbsFromDatabase()

void WorkoutController::reloadClimbsFromDatabase ()  
Definition at line 356 of file [WorkoutController.cpp](#).

#### 7.120.3.18 rideData()

const [RideData](#) & WorkoutController::rideData () const [inline]  
Definition at line 92 of file [WorkoutController.h](#).

#### 7.120.3.19 setAnalysisQueue()

void WorkoutController::setAnalysisQueue (  
    [AnalysisQueue](#) \* queue) [inline]  
Definition at line 90 of file [WorkoutController.h](#).

#### 7.120.3.20 setCurrentAthlete()

void WorkoutController::setCurrentAthlete (  
    int athleteId)  
Definition at line 71 of file [WorkoutController.cpp](#).

#### 7.120.3.21 setDatabaseManager()

void WorkoutController::setDatabaseManager (  
    [DatabaseManager](#) \* dbManager)  
Definition at line 66 of file [WorkoutController.cpp](#).

#### 7.120.3.22 setFTP()

void WorkoutController::setFTP (  
    double ftp)  
Definition at line 58 of file [WorkoutController.cpp](#).

### 7.120.3.23 statistics()

```
const WorkoutStatistics & WorkoutController::statistics () const [inline]
```

Definition at line 94 of file [WorkoutController.h](#).

### 7.120.3.24 trainingStressScore()

```
double WorkoutController::trainingStressScore () const [inline]
```

Definition at line 108 of file [WorkoutController.h](#).

### 7.120.3.25 variabilityIndex()

```
double WorkoutController::variabilityIndex () const [inline]
```

Definition at line 110 of file [WorkoutController.h](#).

## 7.120.4 Member Data Documentation

### 7.120.4.1 m\_analysisQueue

```
AnalysisQueue* WorkoutController::m_analysisQueue = nullptr [private]
```

Definition at line 130 of file [WorkoutController.h](#).

### 7.120.4.2 m\_analyzer

```
WorkoutAnalyzer WorkoutController::m_analyzer [private]
```

Definition at line 140 of file [WorkoutController.h](#).

### 7.120.4.3 m\_climbs

```
std::vector<Climb> WorkoutController::m_climbs [private]
```

Definition at line 144 of file [WorkoutController.h](#).

### 7.120.4.4 m\_currentActivityId

```
int WorkoutController::m_currentActivityId = -1 [private]
```

Definition at line 134 of file [WorkoutController.h](#).

### 7.120.4.5 m\_currentAthleteId

```
int WorkoutController::m_currentAthleteId = -1 [private]
```

Definition at line 133 of file [WorkoutController.h](#).

### 7.120.4.6 m\_dbManager

```
DatabaseManager* WorkoutController::m_dbManager = nullptr [private]
```

Definition at line 136 of file [WorkoutController.h](#).

### 7.120.4.7 m\_ef

```
double WorkoutController::m_ef = 0.0 [private]
```

Definition at line 150 of file [WorkoutController.h](#).

### 7.120.4.8 m\_ftp

```
double WorkoutController::m_ftp = 250.0 [private]
```

Definition at line 132 of file [WorkoutController.h](#).

### 7.120.4.9 m\_if

```
double WorkoutController::m_if = 0.0 [private]
```

Definition at line 147 of file [WorkoutController.h](#).

**7.120.4.10 m\_intervals**

`std::vector<Interval> WorkoutController::m_intervals [private]`  
 Definition at line 143 of file [WorkoutController.h](#).

**7.120.4.11 m\_np**

`double WorkoutController::m_np = 0.0 [private]`  
 Definition at line 146 of file [WorkoutController.h](#).

**7.120.4.12 m\_pdcPoints**

`std::vector<PowerCurvePoint> WorkoutController::m_pdcPoints [private]`  
 Definition at line 142 of file [WorkoutController.h](#).

**7.120.4.13 m\_powerZones**

`std::vector<ZoneDistribution> WorkoutController::m_powerZones [private]`  
 Definition at line 141 of file [WorkoutController.h](#).

**7.120.4.14 m\_rideData**

`RideData WorkoutController::m_rideData [private]`  
 Definition at line 138 of file [WorkoutController.h](#).

**7.120.4.15 m\_statistics**

`WorkoutStatistics WorkoutController::m_statistics [private]`  
 Definition at line 139 of file [WorkoutController.h](#).

**7.120.4.16 m\_tss**

`double WorkoutController::m_tss = 0.0 [private]`  
 Definition at line 148 of file [WorkoutController.h](#).

**7.120.4.17 m\_vi**

`double WorkoutController::m_vi = 0.0 [private]`  
 Definition at line 149 of file [WorkoutController.h](#).

The documentation for this class was generated from the following files:

- [src/gui/WorkoutController.h](#)
- [src/gui/WorkoutController.cpp](#)

**7.121 WorkoutPersistenceService Class Reference****Static Public Member Functions**

- static [RideData](#) [loadRide](#) (int activityId, [DatabaseManager](#) &dbManager)
- static int [saveRide](#) (const [RideData](#) &ride, const [WorkoutStatistics](#) &stats, int athleteId, const QString &sourcePath, const QString &fitHash, double normalizedPower, [DatabaseManager](#) &dbManager, [ActivityRepository](#) &activityRepository, [ImportRepository](#) &importRepository, QString \*errorOut)

**7.121.1 Detailed Description**

Definition at line 14 of file [WorkoutPersistenceService.h](#).

## 7.121.2 Member Function Documentation

### 7.121.2.1 loadRide()

```
RideData WorkoutPersistenceService::loadRide (
    int activityId,
    DatabaseManager & dbManager) [static]
```

Definition at line 8 of file [WorkoutPersistenceService.cpp](#).

### 7.121.2.2 saveRide()

```
int WorkoutPersistenceService::saveRide (
    const RideData & ride,
    const WorkoutStatistics & stats,
    int athleteId,
    const QString & sourcePath,
    const QString & fitHash,
    double normalizedPower,
    DatabaseManager & dbManager,
    ActivityRepository & activityRepository,
    ImportRepository & importRepository,
    QString * errorOut) [static]
```

Definition at line 13 of file [WorkoutPersistenceService.cpp](#).

The documentation for this class was generated from the following files:

- src/workouts/WorkoutPersistenceService.h
- src/workouts/WorkoutPersistenceService.cpp

## 7.122 WorkoutStatistics Struct Reference

Statistics summary for a workout activity.

```
#include <WorkoutAnalyzer.h>
```

### Public Attributes

- double [durationSeconds](#) = 0.0  
*Total elapsed time in seconds.*
- double [movingTimeSeconds](#) = 0.0  
*Moving time in seconds (speed > 2 km/h).*
- double [totalDistanceKm](#) = 0.0  
*Total distance in kilometers.*
- double [elevationGainM](#) = 0.0  
*Total elevation gain in meters.*
- double [averagePower](#) = 0.0  
*Average power in watts.*
- double [maximumPower](#) = 0.0  
*Maximum power in watts.*
- double [averageHeartRate](#) = 0.0  
*Average heart rate in bpm.*
- double [maximumHeartRate](#) = 0.0  
*Maximum heart rate in bpm.*
- double [averageCadence](#) = 0.0  
*Average cadence in rpm.*
- double [maximumCadence](#) = 0.0  
*Maximum cadence in rpm.*
- double [averageSpeed](#) = 0.0

- Average speed in km/h.*
- double `maximumSpeed` = 0.0
- Maximum speed in km/h.*

### 7.122.1 Detailed Description

Statistics summary for a workout activity.

Definition at line 11 of file [WorkoutAnalyzer.h](#).

### 7.122.2 Member Data Documentation

#### 7.122.2.1 averageCadence

```
double WorkoutStatistics::averageCadence = 0.0
```

Average cadence in rpm.

Definition at line 38 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.2 averageHeartRate

```
double WorkoutStatistics::averageHeartRate = 0.0
```

Average heart rate in bpm.

Definition at line 32 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.3 averagePower

```
double WorkoutStatistics::averagePower = 0.0
```

Average power in watts.

Definition at line 26 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.4 averageSpeed

```
double WorkoutStatistics::averageSpeed = 0.0
```

Average speed in km/h.

Definition at line 44 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.5 durationSeconds

```
double WorkoutStatistics::durationSeconds = 0.0
```

Total elapsed time in seconds.

Definition at line 14 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.6 elevationGainM

```
double WorkoutStatistics::elevationGainM = 0.0
```

Total elevation gain in meters.

Definition at line 23 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

#### 7.122.2.7 maximumCadence

```
double WorkoutStatistics::maximumCadence = 0.0
```

Maximum cadence in rpm.

Definition at line 41 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#).



### 7.122.2.8 maximumHeartRate

```
double WorkoutStatistics::maximumHeartRate = 0.0
```

Maximum heart rate in bpm.

Definition at line 35 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.122.2.9 maximumPower

```
double WorkoutStatistics::maximumPower = 0.0
```

Maximum power in watts.

Definition at line 29 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

### 7.122.2.10 maximumSpeed

```
double WorkoutStatistics::maximumSpeed = 0.0
```

Maximum speed in km/h.

Definition at line 47 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#).

### 7.122.2.11 movingTimeSeconds

```
double WorkoutStatistics::movingTimeSeconds = 0.0
```

Moving time in seconds (speed > 2 km/h).

Definition at line 17 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#).

### 7.122.2.12 totalDistanceKm

```
double WorkoutStatistics::totalDistanceKm = 0.0
```

Total distance in kilometers.

Definition at line 20 of file [WorkoutAnalyzer.h](#).

Referenced by [WorkoutAnalyzer::analyze\(\)](#), and [ImportWorker::processJob\(\)](#).

The documentation for this struct was generated from the following file:

- [src/analysis/WorkoutAnalyzer.h](#)

## 7.123 Zone Struct Reference

Represents a single training or power zone with bounds and color.

```
#include <ZoneDefinition.h>
```

### Public Attributes

- QString [name](#)  
*Zone name.*
- double [minValue](#) = 0.0  
*Minimum zone value (inclusive).*
- double [maxValue](#) = 0.0  
*Maximum zone value (exclusive).*
- QColor [color](#)  
*Zone color for display.*

### 7.123.1 Detailed Description

Represents a single training or power zone with bounds and color.

Definition at line 26 of file [ZoneDefinition.h](#).

## 7.123.2 Member Data Documentation

### 7.123.2.1 color

`QColor Zone::color`

[Zone](#) color for display.

Definition at line 38 of file [ZoneDefinition.h](#).

Referenced by [ChartController::updateZonesTab\(\)](#).

### 7.123.2.2 max**Value**

`double Zone::maxValue = 0.0`

Maximum zone value (exclusive).

Definition at line 35 of file [ZoneDefinition.h](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#).

### 7.123.2.3 min**Value**

`double Zone::minValue = 0.0`

Minimum zone value (inclusive).

Definition at line 32 of file [ZoneDefinition.h](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#).

### 7.123.2.4 name

`QString Zone::name`

[Zone](#) name.

Definition at line 29 of file [ZoneDefinition.h](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#), and [ChartController::updateZonesTab\(\)](#).

The documentation for this struct was generated from the following file:

- `src/core/zones/ZoneDefinition.h`

## 7.124 ZoneCalculator Class Reference

Calculates training zones for various metrics.

```
#include <ZoneCalculator.h>
```

### Static Public Member Functions

- static `std::vector< Zone > powerZones` (int ftp)  
*Computes power zones based on FTP.*
- static `std::vector< Zone > heartRateZones` (int heartRateMax, int heartRateRest=60, int thresholdHeartRate=0)  
*Computes heart rate zones.*
- static `std::vector< Zone > cadenceZones` ()  
*Computes cadence zones.*
- static `std::vector< Zone > speedZones` ()  
*Computes speed zones.*
- static `std::vector< Zone > altitudeZones` (double minAltitude, double maxAltitude)  
*Computes altitude zones.*
- static `std::vector< Zone > zonesForMetric` (ColorMetric metric, const [ColorContext](#) &context)  
*Gets zones for a specific metric.*
- static `std::vector< ZoneDistribution > computeDistribution` (const [RideData](#) &rideData, ColorMetric metric, const [ColorContext](#) &context)  
*Computes time/distance distribution across zones.*
- static bool `hasMetricSamples` (const [RideData](#) &rideData, ColorMetric metric)  
*Checks if activity has samples for a metric.*

### 7.124.1 Detailed Description

Calculates training zones for various metrics.

Provides zone definitions and distributions for power, heart rate, cadence, speed, and altitude.

Definition at line 16 of file [ZoneCalculator.h](#).

### 7.124.2 Member Function Documentation

#### 7.124.2.1 altitudeZones()

```
std::vector< Zone > ZoneCalculator::altitudeZones (
    double minAltitude,
    double maxAltitude) [static]
```

Computes altitude zones.

#### Parameters

|                    |                            |
|--------------------|----------------------------|
| <i>minAltitude</i> | Minimum altitude in range. |
| <i>maxAltitude</i> | Maximum altitude in range. |

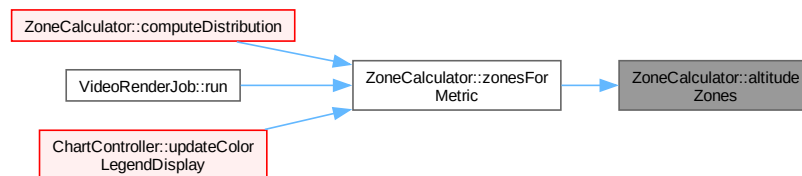
#### Returns

List of altitude zones.

Definition at line 112 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:



#### 7.124.2.2 cadenceZones()

```
std::vector< Zone > ZoneCalculator::cadenceZones () [static]
```

Computes cadence zones.

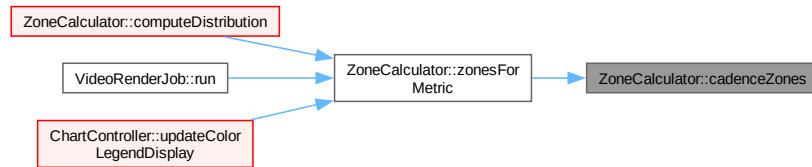
## Returns

List of cadence zones.

Definition at line 90 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:



### 7.124.2.3 computeDistribution()

```
std::vector< ZoneDistribution > ZoneCalculator::computeDistribution (
    const RideData & rideData,
    ColorMetric metric,
    const ColorContext & context) [static]
```

Computes time/distance distribution across zones.

## Parameters

|                 |                                  |
|-----------------|----------------------------------|
| <i>rideData</i> | <a href="#">Activity</a> data.   |
| <i>metric</i>   | Metric type.                     |
| <i>context</i>  | <a href="#">Athlete</a> context. |

## Returns

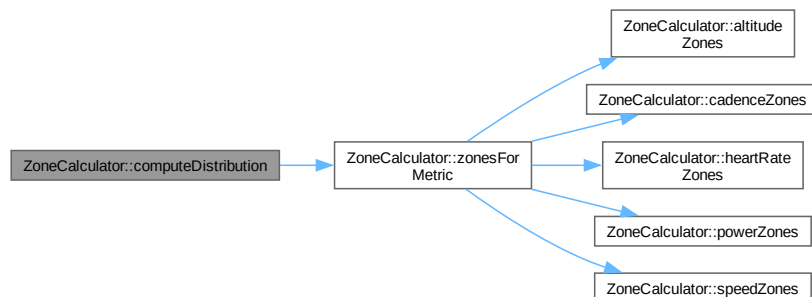
Distribution of time spent in each zone.

Definition at line 152 of file [ZoneCalculator.cpp](#).

References [RideData::records](#), and [zonesForMetric\(\)](#).

Referenced by [ChartController::updateZonesTab\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 7.124.2.4 hasMetricSamples()

```
bool ZoneCalculator::hasMetricSamples (
    const RideData & rideData,
    ColorMetric metric) [static]
```

Checks if activity has samples for a metric.

##### Parameters

|                 |                |
|-----------------|----------------|
| <i>rideData</i> | Activity data. |
| <i>metric</i>   | Metric type.   |

##### Returns

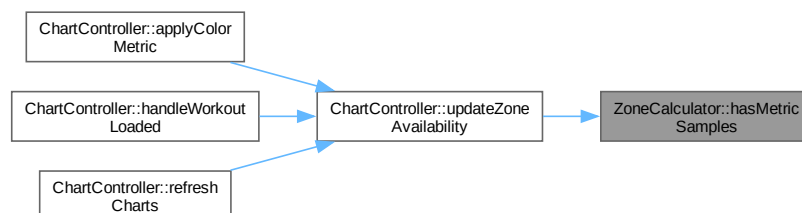
True if samples exist.

Definition at line 199 of file [ZoneCalculator.cpp](#).

References [RideData::records](#).

Referenced by [ChartController::updateZoneAvailability\(\)](#).

Here is the caller graph for this function:



#### 7.124.2.5 heartRateZones()

```
std::vector< Zone > ZoneCalculator::heartRateZones (
    int heartRateMax,
    int heartRateRest = 60,
    int thresholdHeartRate = 0) [static]
```

Computes heart rate zones.

##### Parameters

|                      |                                  |
|----------------------|----------------------------------|
| <i>heartRateMax</i>  | Maximum heart rate.              |
| <i>heartRateRest</i> | Resting heart rate (default 60). |

|                           |                           |
|---------------------------|---------------------------|
| <i>thresholdHeartRate</i> | Threshold HR (default 0). |
|---------------------------|---------------------------|

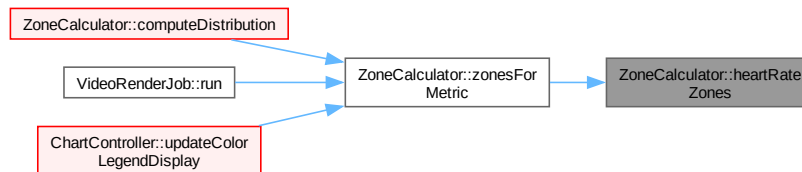
**Returns**

List of heart rate zones.

Definition at line 76 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.124.2.6 powerZones()**

```
std::vector< Zone > ZoneCalculator::powerZones (
    int ftp) [static]
```

Computes power zones based on FTP.

**Parameters**

|            |                                      |
|------------|--------------------------------------|
| <i>ftp</i> | Functional threshold power in watts. |
|------------|--------------------------------------|

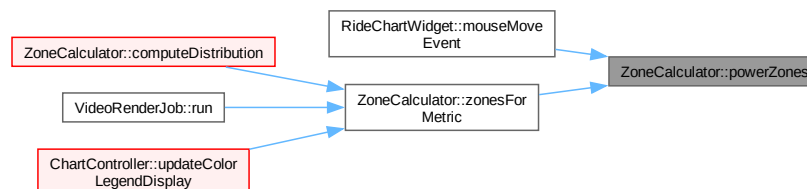
**Returns**

List of power zones.

Definition at line 62 of file [ZoneCalculator.cpp](#).

Referenced by [RideChartWidget::mouseMoveEvent\(\)](#), and [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.124.2.7 speedZones()**

```
std::vector< Zone > ZoneCalculator::speedZones () [static]
```

Computes speed zones.

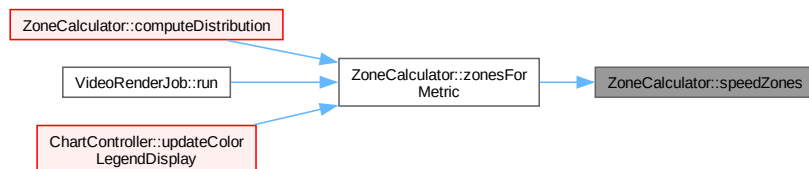
**Returns**

List of speed zones.

Definition at line 101 of file [ZoneCalculator.cpp](#).

Referenced by [zonesForMetric\(\)](#).

Here is the caller graph for this function:

**7.124.2.8 zonesForMetric()**

```
std::vector< Zone > ZoneCalculator::zonesForMetric (
    ColorMetric metric,
    const ColorContext & context) [static]
```

Gets zones for a specific metric.

**Parameters**

|                |                                                      |
|----------------|------------------------------------------------------|
| <i>metric</i>  | Metric type.                                         |
| <i>context</i> | <a href="#">Athlete</a> context (FTP, HR max, etc.). |

**Returns**

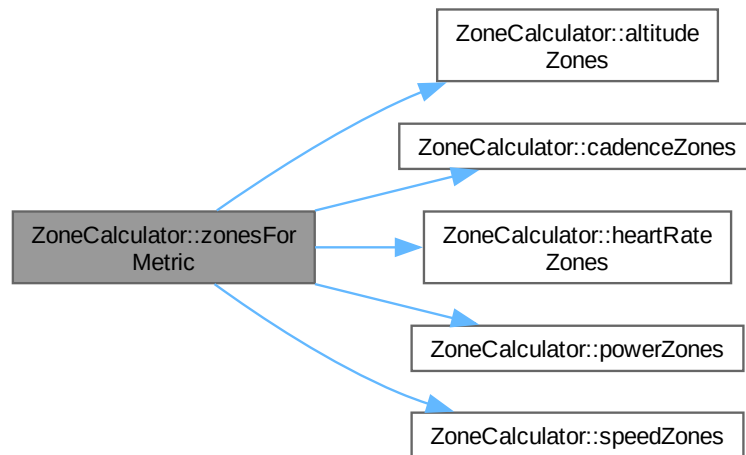
[Zone](#) definitions for the metric.

Definition at line 127 of file [ZoneCalculator.cpp](#).

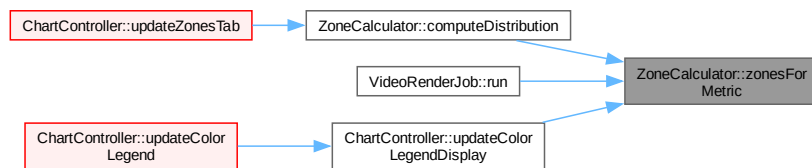
References [ColorContext::altitudeMax](#), [ColorContext::altitudeMin](#), [altitudeZones\(\)](#), [cadenceZones\(\)](#), [ColorContext::ftp](#), [ColorContext::heartRateMax](#), [ColorContext::heartRateRest](#), [heartRateZones\(\)](#), [powerZones\(\)](#), [speedZones\(\)](#), and [ColorContext::thresholdHeartRate](#).

Referenced by [computeDistribution\(\)](#), [VideoRenderJob::run\(\)](#), and [ChartController::updateColorLegendDisplay\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- `src/core/zones/ZoneCalculator.h`
- `src/core/zones/ZoneCalculator.cpp`

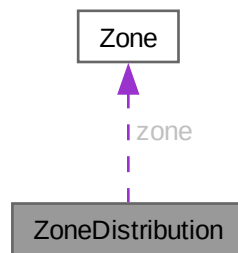
## 7.125 ZoneDistribution Struct Reference

Distribution of time or distance spent in a specific zone.

```
#include <ZoneDefinition.h>
```



Collaboration diagram for ZoneDistribution:



### Public Attributes

- [Zone zone](#)  
*Zone definition.*
- double [seconds](#) = 0.0  
*Time spent in zone (seconds).*
- double [percent](#) = 0.0  
*Percentage of total time in zone.*

### 7.125.1 Detailed Description

Distribution of time or distance spent in a specific zone.  
Definition at line 44 of file [ZoneDefinition.h](#).

### 7.125.2 Member Data Documentation

#### 7.125.2.1 percent

```
double ZoneDistribution::percent = 0.0
```

Percentage of total time in zone.  
Definition at line 53 of file [ZoneDefinition.h](#).  
Referenced by [ChartController::updateZonesTab\(\)](#).

#### 7.125.2.2 seconds

```
double ZoneDistribution::seconds = 0.0
```

Time spent in zone (seconds).  
Definition at line 50 of file [ZoneDefinition.h](#).  
Referenced by [ChartController::updateZonesTab\(\)](#).

#### 7.125.2.3 zone

```
Zone ZoneDistribution::zone
```

[Zone](#) definition.  
Definition at line 47 of file [ZoneDefinition.h](#).  
Referenced by [ChartController::updateZonesTab\(\)](#).

The documentation for this struct was generated from the following file:

- `src/core/zones/ZoneDefinition.h`



## Chapter 8

# File Documentation

### 8.1 ClimbCandidateFinder.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ClimbCandidateFinder.h"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007 #include <numbers>
00008
00009 namespace
00010 {
00011     double segmentMeters(double lat1, double lon1, double lat2, double lon2)
00012     {
00013         const double dLat = (lat2 - lat1) * std::numbers::pi_v<double> / 180.0;
00014         const double dLon = (lon2 - lon1) * std::numbers::pi_v<double> / 180.0;
00015         const double mLat = (lat1 + lat2) / 2.0 * std::numbers::pi_v<double> / 180.0;
00016         return std::hypot(dLat, dLon * std::cos(mLat)) * 6371000.0;
00017     }
00018 }
00019
00020 namespace ClimbCandidateFinder
00021 {
00022     std::vector<double> buildCumulativeDistanceMeters(const RideData& rideData)
00023     {
00024         const size_t n = rideData.records.size();
00025         std::vector<double> cumulative(n, 0.0);
00026         if (n == 0)
00027             return cumulative;
00028
00029         bool prevHasGps = false;
00030         double prevLat = 0.0;
00031         double prevLon = 0.0;
00032
00033         for (size_t i = 1; i < n; ++i)
00034         {
00035             const RideRecord& prev = rideData.records[i - 1];
00036             const RideRecord& now = rideData.records[i];
00037
00038             double segment = 0.0;
00039             if (now.hasGps)
00040             {
00041                 if (prevHasGps)
00042                     segment = segmentMeters(prevLat, prevLon, now.latitude, now.longitude);
00043                 prevHasGps = true;
00044                 prevLat = now.latitude;
00045                 prevLon = now.longitude;
00046             }
00047             else if (now.hasSpeed)
00048             {
00049                 const double dt = std::max(0.0, now.elapsedSeconds - prev.elapsedSeconds);
00050                 const double prevSpeed = prev.hasSpeed ? prev.speed : now.speed;
00051                 const double avgSpeedKmh = (now.speed + prevSpeed) / 2.0;
00052                 segment = avgSpeedKmh * (dt / 3600.0) * 1000.0;
00053             }
00054             cumulative[i] = cumulative[i - 1] + std::max(0.0, segment);
00055         }
00056         return cumulative;
00057     }
00058 }
00059
00060 }
```

## 8.2 ClimbCandidateFinder.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <vector>
00006
00007 #include "fit/RideData.h"
00008
00009 namespace ClimbCandidateFinder
00010 {
00011     std::vector<double> buildCumulativeDistanceMeters(const RideData& rideData);
00012 }

```

## 8.3 ClimbClassifier.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ClimbClassifier.h"
00004
00005 namespace ClimbClassifier
00006 {
00007     ClimbCategory categorize(double gainMeters,
00008                             double lengthKm,
00009                             double averageGrade)
00010     {
00011         if (gainMeters > 100.0 && lengthKm > 2.0 && averageGrade > 3.0)
00012             return ClimbCategory::Major;
00013
00014         if (gainMeters > 40.0 && lengthKm > 0.5 && averageGrade > 3.0)
00015             return ClimbCategory::Medium;
00016
00017         if (gainMeters > 10.0 && lengthKm > 0.1 && averageGrade > 5.0)
00018             return ClimbCategory::Punchy;
00019
00020         return ClimbCategory::FalsePositive;
00021     }
00022 }

```

## 8.4 ClimbClassifier.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "analysis/ClimbMetrics.h"
00006
00007 namespace ClimbClassifier
00008 {
00009     ClimbCategory categorize(double gainMeters,
00010                             double lengthKm,
00011                             double averageGrade);
00012 }

```

## 8.5 ClimbMerger.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ClimbMerger.h"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007
00008 namespace ClimbMerger
00009 {
00010     std::vector<Climb> merge(std::vector<Climb>& candidates,
00011                             const ClimbDetector::Config& config)
00012     {
00013         if (!config.enableClimbMerging || candidates.size() < 2)
00014             return candidates;
00015
00016         std::vector<Climb> merged;
00017         std::vector<bool> processed(candidates.size(), false);
00018
00019         for (size_t i = 0; i < candidates.size(); ++i)
00020         {

```

```

00021         if (processed[i])
00022             continue;
00023
00024         Climb combined = candidates[i];
00025         processed[i] = true;
00026
00027         size_t nextIdx = i + 1;
00028         while (nextIdx < candidates.size())
00029         {
00030             if (processed[nextIdx])
00031             {
00032                 ++nextIdx;
00033                 continue;
00034             }
00035
00036             const Climb& next = candidates[nextIdx];
00037
00038             const double gapMeters = next.startDistanceKm * 1000.0 - combined.endDistanceKm * 1000.0;
00039             const double elevationGapM = std::abs(next.startElevationM - combined.endElevationM);
00040             const bool overlaps = gapMeters <= 0.0;
00041
00042             if (overlaps ||
00043                 gapMeters <= config.mergeMaxGapMeters ||
00044                 elevationGapM <= config.mergeMaxGapElevationM)
00045             {
00046                 const double bridgingGainM = std::max(0.0, next.startElevationM -
combined.endElevationM);
00047                 combined.endSeconds = std::max(combined.endSeconds, next.endSeconds);
00048                 combined.endDistanceKm = std::max(combined.endDistanceKm, next.endDistanceKm);
00049                 combined.durationSeconds = combined.endSeconds - combined.startSeconds;
00050                 combined.lengthKm = combined.endDistanceKm - combined.startDistanceKm;
00051                 if (next.endDistanceKm >= combined.endDistanceKm)
00052                     combined.endElevationM = next.endElevationM;
00053
00054                 combined.elevationGainM += overlaps ? next.elevationGainM : bridgingGainM +
next.elevationGainM;
00055
00056                 if (combined.lengthKm > 0.0)
00057                     combined.averageGradient = (combined.elevationGainM / (combined.lengthKm *
1000.0)) * 100.0;
00058
00059                 processed[nextIdx] = true;
00060                 ++nextIdx;
00061             }
00062             else
00063             {
00064                 break;
00065             }
00066         }
00067         merged.push_back(combined);
00068     }
00069 }
00070
00071 return merged;
00072 }
00073 }

```

## 8.6 ClimbMerger.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <vector>
00006
00007 #include "analysis/ClimbDetector.h"
00008
00009 namespace ClimbMerger
00010 {
00011     std::vector<Climb> merge(std::vector<Climb>& candidates,
00012                             const ClimbDetector::Config& config);
00013 }

```

## 8.7 ClimbDetector.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbDetector.h"
00005
00006 #include "analysis/climb/ClimbCandidateFinder.h"

```

```

00007 #include "analysis/climb/ClimbClassifier.h"
00008 #include "analysis/climb/ClimbMerger.h"
00009
00010 #include <algorithm>
00011 #include <cmath>
00012 #include <limits>
00013
00014 namespace
00015 {
00016 bool isFinite(double value)
00017 {
00018     return std::isfinite(value);
00019 }
00020
00021 double clampValue(double value, double lo, double hi)
00022 {
00023     return std::max(lo, std::min(hi, value));
00024 }
00025
00026 std::vector<double> buildFilteredAltitudeMeters(
00027     const RideData& rideData,
00028     const std::vector<double>& cumulativeDistanceMeters,
00029     double maxVerticalSpeedMps)
00030 {
00031     const size_t n = rideData.records.size();
00032     std::vector<double> filtered(n, std::numeric_limits<double>::quiet_NaN());
00033     for (size_t i = 0; i < n; ++i)
00034     {
00035         if (rideData.records[i].hasAltitude)
00036             filtered[i] = rideData.records[i].altitude;
00037     }
00038
00039     if (n < 3 || maxVerticalSpeedMps <= 0.0)
00040         return filtered;
00041
00042     for (size_t i = 1; i + 1 < n; ++i)
00043     {
00044         if (!isFinite(filtered[i - 1]) || !isFinite(filtered[i]) || !isFinite(filtered[i + 1]))
00045             continue;
00046
00047         const RideRecord& prev = rideData.records[i - 1];
00048         const RideRecord& now = rideData.records[i];
00049         const RideRecord& next = rideData.records[i + 1];
00050
00051         const double dtPrev = now.elapsedSeconds - prev.elapsedSeconds;
00052         const double dtNext = next.elapsedSeconds - now.elapsedSeconds;
00053         if (dtPrev <= 0.0 || dtNext <= 0.0)
00054             continue;
00055
00056         const double vsPrev = (filtered[i] - filtered[i - 1]) / dtPrev;
00057         const double vsNext = (filtered[i + 1] - filtered[i]) / dtNext;
00058         const bool oppositeDirections = (vsPrev > 0.0 && vsNext < 0.0) || (vsPrev < 0.0 && vsNext >
00059 0.0);
00060         if (!oppositeDirections)
00061             continue;
00062
00063         if (std::abs(vsPrev) <= maxVerticalSpeedMps || std::abs(vsNext) <= maxVerticalSpeedMps)
00064             continue;
00065
00066         const double spanDist = cumulativeDistanceMeters[i + 1] - cumulativeDistanceMeters[i - 1];
00067         const double localDist = cumulativeDistanceMeters[i] - cumulativeDistanceMeters[i - 1];
00068         const double ratio = spanDist > 0.0 ? clampValue(localDist / spanDist, 0.0, 1.0) : 0.5;
00069         filtered[i] = filtered[i - 1] + (filtered[i + 1] - filtered[i - 1]) * ratio;
00070     }
00071     return filtered;
00072 }
00073
00074 double estimateSpeedKmhAt(
00075     const RideData& rideData,
00076     const std::vector<double>& cumulativeDistanceMeters,
00077     int index)
00078 {
00079     const int n = static_cast<int>(rideData.records.size());
00080     if (index < 0 || index >= n)
00081         return std::numeric_limits<double>::quiet_NaN();
00082
00083     const RideRecord& r = rideData.records[static_cast<size_t>(index)];
00084     if (r.hasSpeed && isFinite(r.speed) && r.speed >= 0.0)
00085         return r.speed;
00086
00087     const int prev = std::max(0, index - 1);
00088     const int next = std::min(n - 1, index + 1);
00089     if (next == prev)
00090         return std::numeric_limits<double>::quiet_NaN();
00091
00092     const double dt = rideData.records[static_cast<size_t>(next)].elapsedSeconds -

```

```

00093         rideData.records[static_cast<size_t>(prev)].elapsedSeconds;
00094     const double dd = cumulativeDistanceMeters[static_cast<size_t>(next)] -
00095         cumulativeDistanceMeters[static_cast<size_t>(prev)];
00096     if (dt <= 0.0 || dd < 0.0)
00097         return std::numeric_limits<double>::quiet_NaN();
00098
00099     return (dd / dt) * 3.6;
00100 }
00101 } // namespace
00102
00103 std::vector<double> ClimbDetector::buildCumulativeDistanceMeters(const RideData& rideData)
00104 {
00105     return ClimbCandidateFinder::buildCumulativeDistanceMeters(rideData);
00106 }
00107
00108 std::vector<double> ClimbDetector::smoothAltitudeByDistance(
00109     const RideData& rideData,
00110     const std::vector<double>& cumulativeDistanceMeters,
00111     double windowMeters,
00112     const std::vector<double>& adaptiveWindowMeters,
00113     double altitudeSpikeVerticalSpeedMps)
00114 {
00115     const size_t n = rideData.records.size();
00116     std::vector<double> smoothed(n, std::numeric_limits<double>::quiet_NaN());
00117     if (n == 0)
00118         return smoothed;
00119
00120     const std::vector<double> filteredAltitudeMeters = buildFilteredAltitudeMeters(
00121         rideData,
00122         cumulativeDistanceMeters,
00123         altitudeSpikeVerticalSpeedMps);
00124
00125     if (windowMeters <= 0.0)
00126     {
00127         for (size_t i = 0; i < n; ++i)
00128         {
00129             if (isFinite(filteredAltitudeMeters[i]))
00130                 smoothed[i] = filteredAltitudeMeters[i];
00131         }
00132         return smoothed;
00133     }
00134
00135     size_t left = 0;
00136     size_t right = 0;
00137
00138     for (size_t i = 0; i < n; ++i)
00139     {
00140         const double sampleWindow =
00141             (adaptiveWindowMeters.size() == n && isFinite(adaptiveWindowMeters[i]) &&
00142              adaptiveWindowMeters[i] > 0.0)
00143             ? adaptiveWindowMeters[i]
00144             : windowMeters;
00145         const double halfWindow = std::max(1.0, sampleWindow / 2.0);
00146         const double center = cumulativeDistanceMeters[i];
00147         while (left < n && cumulativeDistanceMeters[left] < center - halfWindow)
00148             ++left;
00149         while (right < n && cumulativeDistanceMeters[right] <= center + halfWindow)
00150             ++right;
00151
00152         double sumAlt = 0.0;
00153         int countAlt = 0;
00154         for (size_t j = left; j < right; ++j)
00155         {
00156             if (!isFinite(filteredAltitudeMeters[j]))
00157                 continue;
00158             sumAlt += filteredAltitudeMeters[j];
00159             ++countAlt;
00160         }
00161         if (countAlt > 0)
00162             smoothed[i] = sumAlt / static_cast<double>(countAlt);
00163     }
00164
00165     return smoothed;
00166 }
00167
00168 std::vector<double> ClimbDetector::computeAdaptiveSmoothingWindowsMeters(
00169     const RideData& rideData,
00170     const std::vector<double>& cumulativeDistanceMeters,
00171     const Config& config)
00172 {
00173     const size_t n = rideData.records.size();
00174     std::vector<double> windows(n, std::max(1.0, config.elevationSmoothingMeters));
00175     if (n == 0)
00176         return windows;
00177
00178     if (!config.useAdaptiveSmoothing)

```

```

00179         return windows;
00180
00181         const double minWin = std::max(1.0, std::min(config.adaptiveSmoothingMinMeters,
config.adaptiveSmoothingMaxMeters));
00182         const double maxWin = std::max(minWin, std::max(config.adaptiveSmoothingMinMeters,
config.adaptiveSmoothingMaxMeters));
00183
00184         for (int i = 0; i < static_cast<int>(n); ++i)
00185         {
00186             const double speedKmh = estimateSpeedKmhAt(rideData, cumulativeDistanceMeters, i);
00187             if (!isFinite(speedKmh))
00188             {
00189                 windows[static_cast<size_t>(i)] = clampValue(config.elevationSmoothingMeters, minWin,
maxWin);
00190                 continue;
00191             }
00192
00193             const double targetWindow = 20.0 + speedKmh * 1.8;
00194             windows[static_cast<size_t>(i)] = clampValue(targetWindow, minWin, maxWin);
00195         }
00196
00197         return windows;
00198     }
00199
00200     std::vector<double> ClimbDetector::computeLocalGradientPct (
00201         const std::vector<double>& smoothedAltitudeMeters,
00202         const std::vector<double>& cumulativeDistanceMeters,
00203         double halfWindowMeters)
00204     {
00205         const size_t n = smoothedAltitudeMeters.size();
00206         std::vector<double> gradient(n, std::numeric_limits<double>::quiet_NaN());
00207         if (n == 0)
00208             return gradient;
00209
00210         if (halfWindowMeters <= 0.0)
00211             halfWindowMeters = 1.0;
00212
00213         size_t left = 0;
00214         size_t right = 0;
00215
00216         for (size_t i = 0; i < n; ++i)
00217         {
00218             const double center = cumulativeDistanceMeters[i];
00219             while (left + 1 < n && cumulativeDistanceMeters[left + 1] <= center - halfWindowMeters)
00220                 ++left;
00221             while (right < n && cumulativeDistanceMeters[right] < center + halfWindowMeters)
00222                 ++right;
00223             if (right >= n)
00224                 right = n - 1;
00225
00226             if (!isFinite(smoothedAltitudeMeters[left]) || !isFinite(smoothedAltitudeMeters[right]))
00227                 continue;
00228
00229             const double distDiff = cumulativeDistanceMeters[right] - cumulativeDistanceMeters[left];
00230             if (distDiff <= 0.0)
00231                 continue;
00232
00233             const double altDiff = smoothedAltitudeMeters[right] - smoothedAltitudeMeters[left];
00234             gradient[i] = (altDiff / distDiff) * 100.0;
00235         }
00236
00237         return gradient;
00238     }
00239
00240     // Phase 2: Scoring-based detection
00241     double ClimbDetector::computeClimbScore (
00242         double gainMeters,
00243         double averageGrade,
00244         double distanceKm,
00245         double continuityRatio,
00246         double vam)
00247     {
00248         double score = gainMeters * 0.8 + averageGrade * 8.0 + distanceKm * 15.0 + continuityRatio * 20.0;
00249
00250         if (vam > 800.0)
00251             score += 10.0;
00252
00253         return score;
00254     }
00255
00256     // Phase 6: VAM computation
00257     double ClimbDetector::computeVAM(double elevationGainM, double durationSeconds)
00258     {
00259         if (durationSeconds <= 0.0)
00260             return 0.0;
00261
00262         const double durationHours = durationSeconds / 3600.0;

```



```

00263     if (durationHours <= 0.0)
00264         return 0.0;
00265
00266     return elevationGainM / durationHours;
00267 }
00268
00269 // Phase 1: Categorization
00270 ClimbCategory ClimbDetector::categorizeClimb(
00271     double gainMeters,
00272     double lengthKm,
00273     double averageGrade)
00274 {
00275     return ClimbClassifier::categorize(gainMeters, lengthKm, averageGrade);
00276 }
00277
00278 // Phase 7: Merge fragmented climbs
00279 std::vector<Climb> ClimbDetector::mergeFragmentedClimbs(
00280     std::vector<Climb>& candidates,
00281     const Config& config)
00282 {
00283     return ClimbMerger::merge(candidates, config);
00284 }
00285
00286 std::vector<Climb> ClimbDetector::detect(const RideData& rideData, const Config& config)
00287 {
00288     const int n = static_cast<int>(rideData.records.size());
00289     if (n < 3)
00290         return {};
00291
00292     bool hasAltitude = false;
00293     for (const RideRecord& r : rideData.records)
00294     {
00295         if (r.hasAltitude)
00296         {
00297             hasAltitude = true;
00298             break;
00299         }
00300     }
00301     if (!hasAltitude)
00302         return {};
00303
00304     const std::vector<double> cumulativeDistanceMeters = buildCumulativeDistanceMeters(rideData);
00305     const std::vector<double> adaptiveSmoothingWindows = computeAdaptiveSmoothingWindowsMeters(
00306         rideData,
00307         cumulativeDistanceMeters,
00308         config);
00309     const std::vector<double> smoothedAltitudeMeters = smoothAltitudeByDistance(
00310         rideData,
00311         cumulativeDistanceMeters,
00312         config.elevationSmoothingMeters,
00313         adaptiveSmoothingWindows,
00314         config.altitudeSpikeVerticalSpeedMps);
00315     const std::vector<double> gradient50mPct = computeLocalGradientPct(
00316         smoothedAltitudeMeters,
00317         cumulativeDistanceMeters,
00318         25.0);
00319     const std::vector<double> gradient100mPct = computeLocalGradientPct(
00320         smoothedAltitudeMeters,
00321         cumulativeDistanceMeters,
00322         50.0);
00323     const std::vector<double> gradient250mPct = computeLocalGradientPct(
00324         smoothedAltitudeMeters,
00325         cumulativeDistanceMeters,
00326         125.0);
00327
00328     auto isUphillSignal = [&](int idx)
00329     {
00330         const double g50 = gradient50mPct[static_cast<size_t>(idx)];
00331         const double g100 = gradient100mPct[static_cast<size_t>(idx)];
00332         const double g250 = gradient250mPct[static_cast<size_t>(idx)];
00333         return (isFinite(g100) && g100 >= config.startGradient) ||
00334             (isFinite(g250) && g250 >= config.startGradient) ||
00335             (isFinite(g50) && g50 >= config.startGradient + 1.5);
00336     };
00337
00338     auto isContinuationSignal = [&](int idx)
00339     {
00340         const double g50 = gradient50mPct[static_cast<size_t>(idx)];
00341         const double g100 = gradient100mPct[static_cast<size_t>(idx)];
00342         const double g250 = gradient250mPct[static_cast<size_t>(idx)];
00343         return (isFinite(g100) && g100 >= config.continueGradient) ||
00344             (isFinite(g250) && g250 >= config.startGradient) ||
00345             (isFinite(g50) && g50 >= config.startGradient + 2.0);
00346     };
00347
00348     std::vector<Climb> climbs;
00349     int i = 0;

```

```

00350
00351     while (i < n)
00352     {
00353         int triggerIdx = -1;
00354         for (; i < n; ++i)
00355         {
00356             if (isUphillSignal(i))
00357             {
00358                 triggerIdx = i;
00359                 break;
00360             }
00361         }
00362
00363         if (triggerIdx < 0)
00364             break;
00365
00366         int localMinIdx = triggerIdx;
00367         double localMinAlt = smoothedAltitudeMeters[static_cast<size_t>(triggerIdx)];
00368         if (!isFinite(localMinAlt))
00369         {
00370             i = triggerIdx + 1;
00371             continue;
00372         }
00373
00374         // Backtrack up to 1 km to anchor the climb start at the local minimum.
00375         for (int b = triggerIdx - 1; b >= 0; --b)
00376         {
00377             const double backtrackDist =
00378                 cumulativeDistanceMeters[static_cast<size_t>(triggerIdx)] -
00379                 cumulativeDistanceMeters[static_cast<size_t>(b)];
00380             if (backtrackDist > 1000.0)
00381                 break;
00382
00383             const double alt = smoothedAltitudeMeters[static_cast<size_t>(b)];
00384             if (isFinite(alt) && alt <= localMinAlt)
00385             {
00386                 localMinAlt = alt;
00387                 localMinIdx = b;
00388             }
00389         }
00390
00391         const int climbStart = localMinIdx;
00392
00393         double sustainedDistance = 0.0;
00394         double interruptionDistance = 0.0;
00395         double candidateGainM = 0.0;
00396         int confirmedIdx = -1;
00397         for (int j = std::max(triggerIdx, climbStart + 1); j < n; ++j)
00398         {
00399             const double segmentDist =
00400                 cumulativeDistanceMeters[static_cast<size_t>(j)] -
00401                 cumulativeDistanceMeters[static_cast<size_t>(j - 1)];
00402             const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(j - 1)];
00403             const double alt = smoothedAltitudeMeters[static_cast<size_t>(j)];
00404
00405             if (isFinite(prevAlt) && isFinite(alt))
00406                 candidateGainM += std::max(0.0, alt - prevAlt);
00407
00408             if (isContinuationSignal(j))
00409             {
00410                 sustainedDistance += std::max(0.0, segmentDist);
00411                 interruptionDistance = 0.0;
00412             }
00413             else
00414             {
00415                 interruptionDistance += std::max(0.0, segmentDist);
00416             }
00417
00418             if (interruptionDistance > config.maxStartInterruptionMeters)
00419                 break;
00420
00421             if (sustainedDistance >= config.minStartContinuousDistanceMeters &&
00422                 candidateGainM >= std::min(config.minElevationGainM, 5.0))
00423             {
00424                 confirmedIdx = j;
00425                 break;
00426             }
00427         }
00428
00429         if (confirmedIdx < 0)
00430         {
00431             i = triggerIdx + 1;
00432             continue;
00433         }
00434
00435         int peakIdx = climbStart;
00436         double peakAlt = localMinAlt;

```

```

00435     double plateauDistance = 0.0;
00436     int smallDipsCount = 0; // Phase 3: Track small dips
00437     double cumulativeGainM = 0.0;
00438
00439     int k = confirmedIdx;
00440     for (; k < n; ++k)
00441     {
00442         const double alt = smoothedAltitudeMeters[static_cast<size_t>(k)];
00443         if (!isFinite(alt))
00444             continue;
00445
00446         const double gradient = gradient100mPct[static_cast<size_t>(k)];
00447         const double segmentDist = (k > 0)
00448             ? std::max(0.0, cumulativeDistanceMeters[static_cast<size_t>(k)] -
cumulativeDistanceMeters[static_cast<size_t>(k - 1)])
00449             : 0.0;
00450         if (k > 0)
00451         {
00452             const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(k - 1)];
00453             if (isFinite(prevAlt) && isFinite(alt))
00454                 cumulativeGainM += std::max(0.0, alt - prevAlt);
00455         }
00456
00457         if (alt >= peakAlt)
00458         {
00459             peakAlt = alt;
00460             peakIdx = k;
00461             plateauDistance = 0.0;
00462             smallDipsCount = 0; // Reset dip counter on new peak
00463             continue;
00464         }
00465
00466         const double dipMeters = peakAlt - alt;
00467         const double dipDistance =
00468             cumulativeDistanceMeters[static_cast<size_t>(k)] -
cumulativeDistanceMeters[static_cast<size_t>(peakIdx)];
00469
00470         const double gainSoFar = std::max(0.0, peakAlt - localMinAlt);
00471         const double allowedDipMeters = std::max(
00472             config.maxDipMeters,
00473             std::max(gainSoFar, cumulativeGainM) * (config.maxDipPctOfGain / 100.0));
00474
00475         // Phase 3: Count small dips (< 5m) separately
00476         if (dipMeters <= config.smallDipThresholdMeters)
00477             smallDipsCount++;
00478
00479         if (isContinuationSignal(k))
00480         {
00481             plateauDistance = 0.0;
00482             smallDipsCount = 0; // Reset on uphill gradient
00483             continue;
00484         }
00485
00486         if (isFinite(gradient) && gradient >= config.plateauMinGradient)
00487             plateauDistance += segmentDist;
00488         else
00489             plateauDistance = config.maxPlateauDistanceMeters + 1.0;
00490
00491         // Phase 3: Allow multiple small dips before terminating
00492         const bool exceededDip =
00493             dipMeters > allowedDipMeters &&
00494             dipDistance > config.maxDipDistanceMeters &&
00495             smallDipsCount > config.maxSmallDipsCount;
00496         const bool plateauExpired =
00497             plateauDistance > config.maxPlateauDistanceMeters &&
00498             (!isFinite(gradient) || gradient < config.endGradient);
00499
00500         if (exceededDip || plateauExpired)
00501             break;
00502     }
00503
00504     const int climbEnd = peakIdx;
00505     i = std::max(k, climbEnd + 1);
00506
00507     if (climbEnd <= climbStart)
00508         continue;
00509
00510     Climb climb = ClimbMetrics::build(
00511         rideData,
00512         cumulativeDistanceMeters,
00513         smoothedAltitudeMeters,
00514         gradient100mPct,
00515         climbStart,
00516         climbEnd,
00517         static_cast<int>(climbs.size()) + 1);
00518
00519     // Phase 2: Compute composite climb score

```

```

00520         climb.climbScore = computeClimbScore(
00521             climb.elevationGainM,
00522             climb.averageGradient,
00523             climb.lengthKm,
00524             climb.gradePercentile / 100.0,
00525             climb.vam);
00526
00527         // Phase 1: Categorize the climb
00528         climb.climbCategory = categorizeClimb(
00529             climb.elevationGainM,
00530             climb.lengthKm,
00531             climb.averageGradient);
00532
00533         // Phase 2: Use scoring-based detection if enabled
00534         if (config.useScoringDetection)
00535         {
00536             const bool isPunchyClimb =
00537                 climb.elevationGainM >= 10.0 &&
00538                 climb.lengthKm >= 0.1 &&
00539                 climb.maximumGradient >= 5.0;
00540             const bool isRegularClimb =
00541                 climb.elevationGainM >= 25.0 &&
00542                 climb.lengthKm >= 0.5 &&
00543                 climb.averageGradient >= 3.0;
00544             const bool isLongShallowClimb =
00545                 climb.elevationGainM >= 40.0 &&
00546                 climb.lengthKm >= 1.0 &&
00547                 climb.averageGradient >= 1.5;
00548
00549             if (climb.climbScore < config.minClimbScore &&
00550                 !isPunchyClimb &&
00551                 !isRegularClimb &&
00552                 !isLongShallowClimb)
00553                 continue;
00554         }
00555         else
00556         {
00557             // Fallback to legacy threshold-based detection
00558             if (climb.lengthKm < config.minLengthKm)
00559                 continue;
00560             if (climb.elevationGainM < config.minElevationGainM)
00561                 continue;
00562             if (climb.averageGradient < config.minAverageGradient)
00563                 continue;
00564         }
00565
00566         climbs.push_back(climb);
00567     }
00568
00569     // Phase 7: Merge fragmented climbs if enabled
00570     if (config.enableClimbMerging)
00571     {
00572         climbs = mergeFragmentedClimbs(climbs, config);
00573     }
00574
00575     return climbs;
00576 }

```

## 8.8 ClimbDetector.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "analysis/ClimbMetrics.h"
00009
00016 struct ClimbDetectorConfig
00017 {
00019     double minLengthKm = 0.15;
00021     double minElevationGainM = 10.0;
00023     double minAverageGradient = 4.0;
00024
00026     double startGradient = 1.5;
00028     double continueGradient = 1.0;
00030     double endGradient = 0.5;
00032     double minStartContinuousDistanceMeters = 100.0;
00034     double maxStartInterruptionMeters = 30.0;
00035
00037     double maxDipMeters = 10.0;
00039     double maxDipDistanceMeters = 200.0;
00041     double maxDipPctOfGain = 10.0;

```

```

00043     double maxSmallDipsCount = 3;
00045     double smallDipThresholdMeters = 5.0;
00046
00048     double maxPlateauDistanceMeters = 500.0;
00050     double plateauMinGradient = -1.0;
00051
00053     double elevationSmoothingMeters = 50.0;
00055     bool useAdaptiveSmoothing = true;
00057     double adaptiveSmoothingMinMeters = 25.0;
00059     double adaptiveSmoothingMaxMeters = 100.0;
00061     double gradientWindowHalfMeters = 25.0;
00062
00064     double altitudeSpikeVerticalSpeedMps = 8.0;
00065
00067     bool useScoringDetection = true;
00069     double minClimbScore = 50.0;
00071     double gainScoreWeight = 0.5;
00073     double gradeScoreWeight = 4.0;
00075     double durationScoreWeight = 2.0;
00077     double vamScoreBoost = 800.0;
00078
00080     bool requireGradePersistence = true;
00082     double gradePersistenceThreshold = 4.0;
00084     double gradePersistenceMinMeters = 50.0;
00085
00087     bool enableClimbMerging = true;
00089     double mergeMaxGapMeters = 50.0;
00091     double mergeMaxGapElevationM = 10.0;
00092 };
00093
00100 class ClimbDetector
00101 {
00102 public:
00104     using Config = ClimbDetectorConfig;
00105
00112     static std::vector<Climb> detect(
00113         const RideData& rideData,
00114         const Config& config = Config{});
00115
00121     static std::vector<double> buildCumulativeDistanceMeters(const RideData& rideData);
00122
00132     static std::vector<double> smoothAltitudeByDistance(
00133         const RideData& rideData,
00134         const std::vector<double>& cumulativeDistanceMeters,
00135         double windowMeters,
00136         const std::vector<double>& adaptiveWindowMeters = {},
00137         double altitudeSpikeVerticalSpeedMps = 8.0);
00138
00146     static std::vector<double> computeAdaptiveSmoothingWindowsMeters(
00147         const RideData& rideData,
00148         const std::vector<double>& cumulativeDistanceMeters,
00149         const Config& config);
00150
00158     static std::vector<double> computeLocalGradientPct(
00159         const std::vector<double>& smoothedAltitudeMeters,
00160         const std::vector<double>& cumulativeDistanceMeters,
00161         double halfWindowMeters);
00162
00172     static double computeClimbScore(
00173         double gainMeters,
00174         double averageGrade,
00175         double distanceKm,
00176         double continuityRatio,
00177         double vam);
00178
00185     static double computeVAM(double elevationGainM, double durationSeconds);
00186
00194     static ClimbCategory categorizeClimb(
00195         double gainMeters,
00196         double lengthKm,
00197         double averageGrade);
00198
00205     static std::vector<Climb> mergeFragmentedClimbs(
00206         std::vector<Climb>& candidates,
00207         const Config& config);
00208 };

```

## 8.9 ClimbMetrics.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbMetrics.h"
00005

```

```

00006 #include <algorithm>
00007 #include <cmath>
00008 #include <limits>
00009
00010 namespace
00011 {
00012 struct SliceStats
00013 {
00014     double avgPower = 0.0;
00015     double avgHeartRate = 0.0;
00016     double avgCadence = 0.0;
00017 };
00018
00019 bool isFinite(double value)
00020 {
00021     return std::isfinite(value);
00022 }
00023
00024 double computeLinearRegressionSlope(
00025     const std::vector<double>& x,
00026     const std::vector<double>& y,
00027     double* interceptOut = nullptr)
00028 {
00029     if (x.size() != y.size() || x.size() < 2)
00030     {
00031         if (interceptOut)
00032             *interceptOut = 0.0;
00033         return 0.0;
00034     }
00035
00036     const double n = static_cast<double>(x.size());
00037     double sumX = 0.0;
00038     double sumY = 0.0;
00039     double sumXY = 0.0;
00040     double sumXX = 0.0;
00041
00042     for (size_t i = 0; i < x.size(); ++i)
00043     {
00044         sumX += x[i];
00045         sumY += y[i];
00046         sumXY += x[i] * y[i];
00047         sumXX += x[i] * x[i];
00048     }
00049
00050     const double denom = (n * sumXX) - (sumX * sumX);
00051     if (std::abs(denom) < 1e-9)
00052     {
00053         if (interceptOut)
00054             *interceptOut = sumY / n;
00055         return 0.0;
00056     }
00057
00058     const double slope = ((n * sumXY) - (sumX * sumY)) / denom;
00059     if (interceptOut)
00060         *interceptOut = (sumY - slope * sumX) / n;
00061     return slope;
00062 }
00063
00064 QString classifyClimbCategory(double gainM, double difficultyScore)
00065 {
00066     if (gainM >= 800.0 || difficultyScore >= 4500.0)
00067         return QStringLiteral("Mountain Pass");
00068     if (gainM >= 400.0)
00069         return QStringLiteral("Major Climb");
00070     if (gainM >= 150.0)
00071         return QStringLiteral("Climb");
00072     if (gainM >= 50.0)
00073         return QStringLiteral("Hill");
00074     return QStringLiteral("Bump");
00075 }
00076
00077 QString classifyClimbShape(
00078     const std::vector<double>& gradients,
00079     double lengthKm,
00080     double gainM,
00081     double avgGradient,
00082     double maxGradient)
00083 {
00084     if (gradients.size() < 4)
00085         return QStringLiteral("Steady");
00086
00087     double sum = 0.0;
00088     double sumSq = 0.0;
00089     double minG = gradients.front();
00090     double maxG = gradients.front();
00091     int rollingTransitions = 0;
00092     int prevBand = 0;

```

```

00093
00094     for (double g : gradients)
00095     {
00096         sum += g;
00097         sumSq += g * g;
00098         minG = std::min(minG, g);
00099         maxG = std::max(maxG, g);
00100
00101         const int band = (g >= 4.0) ? 1 : (g <= 1.0 ? -1 : 0);
00102         if (prevBand != 0 && band != 0 && band != prevBand)
00103             ++rollingTransitions;
00104         if (band != 0)
00105             prevBand = band;
00106     }
00107
00108     const double n = static_cast<double>(gradients.size());
00109     const double mean = sum / n;
00110     const double variance = std::max(0.0, (sumSq / n) - (mean * mean));
00111     const double stdDev = std::sqrt(variance);
00112
00113     const size_t chunk = std::max<size_t>(1, gradients.size() / 3);
00114     auto meanChunk = [&gradients](size_t begin, size_t end)
00115     {
00116         if (begin >= end || end > gradients.size())
00117             return 0.0;
00118         double s = 0.0;
00119         for (size_t i = begin; i < end; ++i)
00120             s += gradients[i];
00121         return s / static_cast<double>(end - begin);
00122     };
00123
00124     const double firstThird = meanChunk(0, std::min(gradients.size(), chunk));
00125     const double midThird = meanChunk(std::min(gradients.size(), chunk), std::min(gradients.size(), 2
* chunk));
00126     const double lastThird = meanChunk(std::min(gradients.size(), 2 * chunk), gradients.size());
00127
00128     if (lengthKm >= 8.0 && gainM >= 600.0 && avgGradient >= 5.0 && stdDev < 2.2)
00129         return QStringLiteral("Alpine");
00130     if (rollingTransitions >= 3 && stdDev >= 2.5)
00131         return QStringLiteral("Rolling");
00132     if (maxGradient >= 10.0 && (lastThird - firstThird >= 3.0 || maxGradient - mean >= 5.0))
00133         return QStringLiteral("Wall");
00134     if (stdDev >= 2.0 && (midThird + 1.0 < firstThird || midThird + 1.0 < lastThird) && (maxG - minG)
>= 5.0)
00135         return QStringLiteral("Staircase");
00136     if (stdDev <= 1.5)
00137         return QStringLiteral("Steady");
00138     return QStringLiteral("Mixed");
00139 }
00140
00141 SliceStats computeSliceStats(
00142     const RideData& rideData,
00143     int startIdx,
00144     int endIdx,
00145     const std::vector<double>& basis,
00146     double sliceStart,
00147     double sliceEnd)
00148 {
00149     SliceStats stats;
00150     if (startIdx < 0 || endIdx < startIdx || endIdx >= static_cast<int>(rideData.records.size()))
00151         return stats;
00152
00153     double sumPower = 0.0;
00154     int countPower = 0;
00155     double sumHr = 0.0;
00156     int countHr = 0;
00157     double sumCadence = 0.0;
00158     int countCadence = 0;
00159
00160     for (int i = startIdx; i <= endIdx; ++i)
00161     {
00162         const double v = basis[static_cast<size_t>(i)];
00163         if (!isFinite(v) || v < sliceStart || v > sliceEnd)
00164             continue;
00165
00166         const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00167         if (r.hasPower)
00168         {
00169             sumPower += r.power;
00170             ++countPower;
00171         }
00172         if (r.hasHeartRate)
00173         {
00174             sumHr += r.heartRate;
00175             ++countHr;
00176         }
00177         if (r.hasCadence)

```

```

00178         {
00179             sumCadence += r.cadence;
00180             ++countCadence;
00181         }
00182     }
00183
00184     if (countPower > 0)
00185         stats.avgPower = sumPower / static_cast<double>(countPower);
00186     if (countHr > 0)
00187         stats.avgHeartRate = sumHr / static_cast<double>(countHr);
00188     if (countCadence > 0)
00189         stats.avgCadence = sumCadence / static_cast<double>(countCadence);
00190
00191     return stats;
00192 }
00193
00194 double computeNormalizedPower(const std::vector<double>& power)
00195 {
00196     if (power.empty())
00197         return 0.0;
00198
00199     const int npWin = std::min(30, static_cast<int>(power.size()));
00200     if (npWin <= 0)
00201         return 0.0;
00202
00203     double rollingSum = 0.0;
00204     for (int i = 0; i < npWin; ++i)
00205         rollingSum += power[static_cast<size_t>(i)];
00206
00207     double sum4 = std::pow(rollingSum / static_cast<double>(npWin), 4.0);
00208     int count = 1;
00209
00210     for (int i = npWin; i < static_cast<int>(power.size()); ++i)
00211     {
00212         rollingSum += power[static_cast<size_t>(i)] - power[static_cast<size_t>(i - npWin)];
00213         sum4 += std::pow(rollingSum / static_cast<double>(npWin), 4.0);
00214         ++count;
00215     }
00216
00217     return std::pow(sum4 / static_cast<double>(count), 0.25);
00218 }
00219 } // namespace
00220
00221 Climb ClimbMetrics::build(
00222     const RideData& rideData,
00223     const std::vector<double>& cumulativeDistanceMeters,
00224     const std::vector<double>& smoothedAltitudeMeters,
00225     const std::vector<double>& localGradientPct,
00226     int startIdx,
00227     int endIdx,
00228     int climbOrdinal)
00229 {
00230     Climb climb;
00231     if (startIdx < 0 || endIdx < startIdx || endIdx >= static_cast<int>(rideData.records.size()))
00232         return climb;
00233
00234     const RideRecord& first = rideData.records[static_cast<size_t>(startIdx)];
00235     const RideRecord& last = rideData.records[static_cast<size_t>(endIdx)];
00236
00237     climb.startSeconds = first.elapsedSeconds;
00238     climb.endSeconds = last.elapsedSeconds;
00239     climb.durationSeconds = std::max(0.0, climb.endSeconds - climb.startSeconds);
00240
00241     climb.startDistanceKm = cumulativeDistanceMeters[static_cast<size_t>(startIdx)] / 1000.0;
00242     climb.endDistanceKm = cumulativeDistanceMeters[static_cast<size_t>(endIdx)] / 1000.0;
00243     climb.lengthKm = std::max(0.0, climb.endDistanceKm - climb.startDistanceKm);
00244
00245     double startAlt = first.hasAltitude ? first.altitude : std::numeric_limits<double>::quiet_NaN();
00246     double endAlt = last.hasAltitude ? last.altitude : std::numeric_limits<double>::quiet_NaN();
00247     if (isFinite(smoothedAltitudeMeters[static_cast<size_t>(startIdx)]))
00248         startAlt = smoothedAltitudeMeters[static_cast<size_t>(startIdx)];
00249     if (isFinite(smoothedAltitudeMeters[static_cast<size_t>(endIdx)]))
00250         endAlt = smoothedAltitudeMeters[static_cast<size_t>(endIdx)];
00251
00252     climb.startElevationM = isFinite(startAlt) ? startAlt : 0.0;
00253     climb.endElevationM = isFinite(endAlt) ? endAlt : climb.startElevationM;
00254
00255     double cumulativeGainM = 0.0;
00256     for (int i = startIdx + 1; i <= endIdx; ++i)
00257     {
00258         const double prevAlt = smoothedAltitudeMeters[static_cast<size_t>(i - 1)];
00259         const double currAlt = smoothedAltitudeMeters[static_cast<size_t>(i)];
00260         if (!isFinite(prevAlt) || !isFinite(currAlt))
00261             continue;
00262
00263         cumulativeGainM += std::max(0.0, currAlt - prevAlt);
00264     }

```



```

00265     climb.elevationGainM = cumulativeGainM;
00266
00267     const double lengthMeters = climb.lengthKm * 1000.0;
00268     if (lengthMeters > 0.0)
00269         climb.averageGradient = (climb.elevationGainM / lengthMeters) * 100.0;
00270
00271     double maxGradient = 0.0;
00272     int validGradientCount = 0;
00273     int climbingGradientCount = 0;
00274     for (int i = startIdx; i <= endIdx; ++i)
00275     {
00276         const double g = localGradientPct[static_cast<size_t>(i)];
00277         if (isFinite(g))
00278         {
00279             maxGradient = std::max(maxGradient, g);
00280             ++validGradientCount;
00281             if (g >= 1.5)
00282                 ++climbingGradientCount;
00283         }
00284     }
00285     climb.maximumGradient = maxGradient;
00286     if (validGradientCount > 0)
00287     {
00288         climb.gradePercentile =
00289             (static_cast<double>(climbingGradientCount) / static_cast<double>(validGradientCount)) *
00290             100.0;
00291     }
00292
00293     std::vector<double> powerSeries;
00294     powerSeries.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00295
00296     double powerSum = 0.0;
00297     int powerCount = 0;
00298     double hrSum = 0.0;
00299     int hrCount = 0;
00300     double cadenceSum = 0.0;
00301     int cadenceCount = 0;
00302     double speedSum = 0.0;
00303     int speedCount = 0;
00304
00305     for (int i = startIdx; i <= endIdx; ++i)
00306     {
00307         const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00308         powerSeries.push_back(r.hasPower ? r.power : 0.0);
00309
00310         if (r.hasPower)
00311         {
00312             powerSum += r.power;
00313             ++powerCount;
00314         }
00315         if (r.hasHeartRate)
00316         {
00317             hrSum += r.heartRate;
00318             ++hrCount;
00319         }
00320         if (r.hasCadence)
00321         {
00322             cadenceSum += r.cadence;
00323             ++cadenceCount;
00324         }
00325         if (r.hasSpeed)
00326         {
00327             speedSum += r.speed;
00328             ++speedCount;
00329         }
00330     }
00331
00332     if (powerCount > 0)
00333         climb.averagePower = powerSum / static_cast<double>(powerCount);
00334     if (hrCount > 0)
00335         climb.averageHeartRate = hrSum / static_cast<double>(hrCount);
00336     if (cadenceCount > 0)
00337         climb.averageCadence = cadenceSum / static_cast<double>(cadenceCount);
00338     if (speedCount > 0)
00339         climb.averageSpeed = speedSum / static_cast<double>(speedCount);
00340
00341     climb.normalizedPower = computeNormalizedPower(powerSeries);
00342
00343     const double durationHours = climb.durationSeconds / 3600.0;
00344     if (durationHours > 0.0)
00345         climb.vam = climb.elevationGainM / durationHours;
00346
00347     const bool canUseDistanceSlices =
00348         cumulativeDistanceMeters[static_cast<size_t>(endIdx)] >
00349         cumulativeDistanceMeters[static_cast<size_t>(startIdx)];
00350
00351     std::vector<double> basis(rideData.records.size(), 0.0);

```

```

00350     if (canUseDistanceSlices)
00351     {
00352         basis = cumulativeDistanceMeters;
00353     }
00354     else
00355     {
00356         for (size_t i = 0; i < rideData.records.size(); ++i)
00357             basis[i] = rideData.records[i].elapsedSeconds;
00358     }
00359
00360     const double spanStart = basis[static_cast<size_t>(startIdx)];
00361     const double spanEnd = basis[static_cast<size_t>(endIdx)];
00362     const double span = std::max(0.0, spanEnd - spanStart);
00363
00364     if (span > 0.0)
00365     {
00366         const double q0 = spanStart;
00367         const double q1 = spanStart + span * 0.25;
00368         const double q2 = spanStart + span * 0.50;
00369         const double q3 = spanStart + span * 0.75;
00370         const double q4 = spanEnd;
00371
00372         const SliceStats firstHalf = computeSliceStats(rideData, startIdx, endIdx, basis, q0, q2);
00373         const SliceStats secondHalf = computeSliceStats(rideData, startIdx, endIdx, basis, q2, q4);
00374         climb.powerFirstHalf = firstHalf.avgPower;
00375         climb.powerSecondHalf = secondHalf.avgPower;
00376
00377         const SliceStats firstQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q0, q1);
00378         const SliceStats secondQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q1, q2);
00379         const SliceStats thirdQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q2, q3);
00380         const SliceStats lastQuarter = computeSliceStats(rideData, startIdx, endIdx, basis, q3, q4);
00381
00382         climb.quarterMetrics[0] = { firstQuarter.avgPower, firstQuarter.avgHeartRate,
firstQuarter.avgCadence };
00383         climb.quarterMetrics[1] = { secondQuarter.avgPower, secondQuarter.avgHeartRate,
secondQuarter.avgCadence };
00384         climb.quarterMetrics[2] = { thirdQuarter.avgPower, thirdQuarter.avgHeartRate,
thirdQuarter.avgCadence };
00385         climb.quarterMetrics[3] = { lastQuarter.avgPower, lastQuarter.avgHeartRate,
lastQuarter.avgCadence };
00386
00387         if (firstQuarter.avgPower > 0.0)
00388         {
00389             climb.powerFadePct =
00390                 ((lastQuarter.avgPower - firstQuarter.avgPower) / firstQuarter.avgPower) * 100.0;
00391         }
00392
00393         if (firstQuarter.avgPower > 0.0 && lastQuarter.avgPower > 0.0 &&
00394             firstQuarter.avgHeartRate > 0.0 && lastQuarter.avgHeartRate > 0.0)
00395         {
00396             const double startRatio = firstQuarter.avgHeartRate / firstQuarter.avgPower;
00397             const double endRatio = lastQuarter.avgHeartRate / lastQuarter.avgPower;
00398             if (startRatio > 0.0)
00399                 climb.hrDriftPct = ((endRatio - startRatio) / startRatio) * 100.0;
00400         }
00401     }
00402
00403     std::vector<double> decoupleX;
00404     std::vector<double> decoupleY;
00405     decoupleX.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00406     decoupleY.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00407
00408     for (int i = startIdx; i <= endIdx; ++i)
00409     {
00410         const RideRecord& r = rideData.records[static_cast<size_t>(i)];
00411         if (!r.hasHeartRate || !r.hasPower || r.power <= 0.0 || r.heartRate <= 0.0)
00412             continue;
00413
00414         const double progress = (climb.durationSeconds > 0.0)
00415             ? (r.elapsedSeconds - climb.startSeconds) / climb.durationSeconds
00416             : 0.0;
00417         decoupleX.push_back(std::clamp(progress, 0.0, 1.0));
00418         decoupleY.push_back(r.heartRate / r.power);
00419     }
00420
00421     if (decoupleX.size() >= 6)
00422     {
00423         double intercept = 0.0;
00424         const double slope = computeLinearRegressionSlope(decoupleX, decoupleY, &intercept);
00425         const double startRatio = intercept;
00426         const double endRatio = intercept + slope;
00427         if (startRatio > 0.0)
00428             climb.aerobicDecouplingPct = ((endRatio - startRatio) / startRatio) * 100.0;
00429     }
00430     else
00431     {
00432         climb.aerobicDecouplingPct = climb.hrDriftPct;

```

```

00433     }
00434
00435     climb.hrDriftPct = climb.aerobicDecouplingPct;
00436
00437     climb.difficultyScore = climb.elevationGainM * climb.averageGradient;
00438
00439     std::vector<double> climbGradients;
00440     climbGradients.reserve(static_cast<size_t>(endIdx - startIdx + 1));
00441     for (int i = startIdx; i <= endIdx; ++i)
00442     {
00443         const double g = localGradientPct[static_cast<size_t>(i)];
00444         if (isFinite(g))
00445             climbGradients.push_back(g);
00446     }
00447
00448     climb.category = classifyClimbCategory(climb.elevationGainM, climb.difficultyScore);
00449     climb.shapeClass = classifyClimbShape(
00450         climbGradients,
00451         climb.lengthKm,
00452         climb.elevationGainM,
00453         climb.averageGradient,
00454         climb.maximumGradient);
00455     climb.name = QString("Climb %1").arg(climbOrdinal);
00456
00457     return climb;
00458 }

```

## 8.10 ClimbMetrics.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <vector>
00008
00009 #include <QString>
00010
00011 #include "fit/RideData.h"
00012
00016 enum class ClimbCategory
00017 {
00018     Major,          // >100m gain, >2km length, >3% avg grade
00019     Medium,         // >40m gain, >500m length, >3% avg grade
00020     Punchy,         // >10m gain, >100m length, >5% avg grade (short steep)
00021     FalsePositive // failed quality checks
00022 };
00023
00027 struct ClimbQuarterMetrics
00028 {
00030     double averagePower = 0.0;
00031
00033     double averageHeartRate = 0.0;
00034
00036     double averageCadence = 0.0;
00037 };
00038
00044 struct Climb
00045 {
00047     int id = -1;
00048
00050     double startSeconds = 0.0;
00051
00053     double endSeconds = 0.0;
00054
00056     double startDistanceKm = 0.0;
00057
00059     double endDistanceKm = 0.0;
00060
00062     double durationSeconds = 0.0;
00063
00065     double lengthKm = 0.0;
00066
00068     double elevationGainM = 0.0;
00069
00071     double startElevationM = 0.0;
00072
00074     double endElevationM = 0.0;
00075
00077     double averageGradient = 0.0;
00078
00080     double maximumGradient = 0.0;
00081

```

```

00083     double climbScore = 0.0;
00084
00086     double gradePercentile = 0.0;
00087
00089     ClimbCategory climbCategory = ClimbCategory::FalsePositive;
00090
00092     double averagePower = 0.0;
00093
00095     double averageWattsPerKg = 0.0;
00096
00098     double normalizedPower = 0.0;
00099
00101     double averageHeartRate = 0.0;
00102
00104     double averageCadence = 0.0;
00105
00107     double averageSpeed = 0.0;
00108
00110     double vam = 0.0;
00111
00113     double powerFirstHalf = 0.0;
00114
00116     double powerSecondHalf = 0.0;
00117
00119     double powerFadePct = 0.0;
00120
00122     double aerobicDecouplingPct = 0.0;
00123
00125     double hrDriftPct = 0.0;
00126
00128     double difficultyScore = 0.0;
00129
00131     std::array<ClimbQuarterMetrics, 4> quarterMetrics{};
00132
00134     QString category;
00135
00137     QString shapeClass;
00138
00140     QString wattsPerKgRank;
00141
00143     QString name;
00144 };
00145
00149 class ClimbMetrics
00150 {
00151 public:
00163     static Climb build(
00164         const RideData& rideData,
00165         const std::vector<double>& cumulativeDistanceMeters,
00166         const std::vector<double>& smoothedAltitudeMeters,
00167         const std::vector<double>& localGradientPct,
00168         int startIdx,
00169         int endIdx,
00170         int climbOrdinal);
00171 };

```

## 8.11 FTPZones.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FTPZones.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 FTPZones::ZoneArray FTPZones::defaultZones()
00010 {
00011     return {{
00012         { 1, "Active Recovery", 0.00, 0.55, 0.0 },
00013         { 2, "Endurance", 0.55, 0.75, 0.0 },
00014         { 3, "Tempo", 0.75, 0.90, 0.0 },
00015         { 4, "Threshold", 0.90, 1.05, 0.0 },
00016         { 5, "VO2 Max", 1.05, 1.20, 0.0 },
00017         { 6, "Anaerobic", 1.20, 1.50, 0.0 },
00018         { 7, "Neuromuscular", 1.50, -1.00, 0.0 },
00019     }};
00020 }
00021
00022 FTPZones::ZoneArray FTPZones::compute(
00023     const RideData& rideData, double ftp)
00024 {
00025     auto zones = defaultZones();
00026     if (ftp <= 0.0 || rideData.records.empty())

```

```

00027         return zones;
00028
00029     const auto& records = rideData.records;
00030     const int n = static_cast<int>(records.size());
00031
00032     for (int i = 0; i < n; ++i)
00033     {
00034         const auto& r = records[i];
00035         if (!r.hasPower) continue;
00036
00037         // Midpoint time weight
00038         double dt = 1.0;
00039         if (n > 1)
00040         {
00041             if (i == 0)
00042                 dt = records[1].elapsedSeconds - records[0].elapsedSeconds;
00043             else if (i == n - 1)
00044                 dt = records[n - 1].elapsedSeconds - records[n - 2].elapsedSeconds;
00045             else
00046                 dt = (records[i + 1].elapsedSeconds -
00047                     records[i - 1].elapsedSeconds) / 2.0;
00048         }
00049         dt = std::max(0.0, dt);
00050
00051         const double pct = r.power / ftp;
00052         for (auto& zone : zones)
00053         {
00054             if (pct >= zone.lowerPct &&
00055                 (zone.upperPct < 0.0 || pct < zone.upperPct))
00056             {
00057                 zone.seconds += dt;
00058                 break;
00059             }
00060         }
00061     }
00062
00063     return zones;
00064 }

```

## 8.12 FTPZones.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <string>
00008
00009 #include "fit/RideData.h"
00010
00014 struct PowerZone
00015 {
00017     int         number;
00018
00020     std::string name;
00021
00023     double      lowerPct;
00024
00026     double      upperPct;
00027
00029     double      seconds = 0.0;
00030 };
00031
00035 class FTPZones
00036 {
00037 public:
00039     static constexpr int kZoneCount = 7;
00040
00042     using ZoneArray = std::array<PowerZone, kZoneCount>;
00043
00048     static ZoneArray defaultZones();
00049
00056     static ZoneArray compute(const RideData& rideData, double ftp);
00057 };

```

## 8.13 IntervalDetector.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #include "IntervalDetector.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 std::vector<double> IntervalDetector::smoothPower(
00010     const RideData& rideData, int window)
00011 {
00012     const int n = static_cast<int>(rideData.records.size());
00013     std::vector<double> raw(n);
00014     for (int i = 0; i < n; ++i)
00015     {
00016         const auto& r = rideData.records[i];
00017         raw[i] = r.hasPower ? r.power : 0.0;
00018     }
00019
00020     const int half = window / 2;
00021     std::vector<double> out(n);
00022     for (int i = 0; i < n; ++i)
00023     {
00024         const int lo = std::max(0, i - half);
00025         const int hi = std::min(n - 1, i + half);
00026         double s = 0.0;
00027         for (int j = lo; j <= hi; ++j) s += raw[j];
00028         out[i] = s / (hi - lo + 1);
00029     }
00030
00031     return out;
00032 }
00033
00034 Interval IntervalDetector::buildInterval(
00035     const RideData& rideData,
00036     int startIdx, int endIdx,
00037     const std::string& label)
00038 {
00039     Interval iv;
00040     iv.label = label;
00041     iv.startSeconds = rideData.records[startIdx].elapsedSeconds;
00042     iv.endSeconds = rideData.records[endIdx].elapsedSeconds;
00043     iv.durationSeconds = iv.endSeconds - iv.startSeconds;
00044
00045     std::vector<double> pArr;
00046     double pSum = 0; int pN = 0;
00047     double hrSum = 0; int hrN = 0;
00048     double cadSum = 0; int cadN = 0;
00049     double spdSum = 0; int spdN = 0;
00050
00051     for (int i = startIdx; i <= endIdx; ++i)
00052     {
00053         const auto& r = rideData.records[i];
00054         pArr.push_back(r.hasPower ? r.power : 0.0);
00055
00056         if (r.hasPower)
00057         {
00058             pSum += r.power;
00059             iv.maximumPower = std::max(iv.maximumPower, r.power);
00060             ++pN;
00061         }
00062         if (r.hasHeartRate) { hrSum += r.heartRate; ++hrN; }
00063         if (r.hasCadence) { cadSum += r.cadence; ++cadN; }
00064         if (r.hasSpeed) { spdSum += r.speed; ++spdN; }
00065     }
00066
00067     if (pN > 0) iv.averagePower = pSum / pN;
00068     if (hrN > 0) iv.averageHeartRate = hrSum / hrN;
00069     if (cadN > 0) iv.averageCadence = cadSum / cadN;
00070     if (spdN > 0) iv.averageSpeed = spdSum / spdN;
00071
00072     // Normalised Power (30 s rolling)
00073     const int npWin = std::min(30, static_cast<int>(pArr.size()));
00074     if (npWin > 0)
00075     {
00076         double wSum = 0.0;
00077         for (int i = 0; i < npWin; ++i) wSum += pArr[i];
00078         double s4 = std::pow(wSum / npWin, 4.0);
00079         int cnt = 1;
00080         for (int i = npWin; i < static_cast<int>(pArr.size()); ++i)
00081         {
00082             wSum += pArr[i] - pArr[i - npWin];
00083             s4 += std::pow(wSum / npWin, 4.0);
00084             ++cnt;
00085         }
00086         iv.normalizedPower = std::pow(s4 / cnt, 0.25);
00087     }
00088
00089     return iv;
00090 }

```

```

00091
00092 std::vector<Interval> IntervalDetector::detect(
00093     const RideData& rideData, const Config& config)
00094 {
00095     const auto& records = rideData.records;
00096     if (records.empty()) return {};
00097
00098     bool hasPower = false;
00099     for (const auto& r : records)
00100         if (r.hasPower) { hasPower = true; break; }
00101     if (!hasPower) return {};
00102
00103     const double threshold =
00104         config ftp * config.workThresholdPct;
00105     const auto smoothed =
00106         smoothPower(rideData, config.smoothingWindow);
00107
00108     const int n = static_cast<int>(records.size());
00109     std::vector<Interval> result;
00110
00111     int segStart = 0;
00112     bool inWork = smoothed[0] >= threshold;
00113
00114     auto flush = [&](int endIdx)
00115     {
00116         if (endIdx < segStart) return;
00117         const double dur =
00118             records[endIdx].elapsedSeconds -
00119             records[segStart].elapsedSeconds;
00120
00121         if (inWork && dur >= config.minWorkSeconds)
00122             result.push_back(
00123                 buildInterval(rideData, segStart, endIdx, "Work"));
00124         else if (!inWork &&
00125             dur >= config.minRecoverySeconds &&
00126             !result.empty())
00127             result.push_back(
00128                 buildInterval(rideData, segStart, endIdx, "Recovery"));
00129     };
00130
00131     for (int i = 1; i < n; ++i)
00132     {
00133         const bool nowWork = smoothed[i] >= threshold;
00134         if (nowWork != inWork)
00135         {
00136             flush(i - 1);
00137             segStart = i;
00138             inWork = nowWork;
00139         }
00140     }
00141     flush(n - 1);
00142
00143     return result;
00144 }
00145
00146 std::vector<Interval> IntervalDetector::removeOverlappingAndDuplicates(
00147     std::vector<Interval> intervals)
00148 {
00149     if (intervals.size() <= 1)
00150         return intervals;
00151
00152     // Sort by start time
00153     std::sort(intervals.begin(), intervals.end(),
00154         [](const Interval& a, const Interval& b) {
00155             return a.startSeconds < b.startSeconds;
00156         });
00157
00158     std::vector<Interval> result;
00159     result.reserve(intervals.size());
00160
00161     // Merge overlapping/adjacent intervals and remove duplicates
00162     const double tolerance = 1.0; // 1 second boundary tolerance for duplicates
00163
00164     for (const Interval& current : intervals)
00165     {
00166         // Skip if this interval is a duplicate of the last one
00167         // (same start/end within tolerance)
00168         if (!result.empty())
00169         {
00170             const Interval& last = result.back();
00171             if (std::abs(last.startSeconds - current.startSeconds) <= tolerance &&
00172                 std::abs(last.endSeconds - current.endSeconds) <= tolerance)
00173             {
00174                 // Duplicate detected: keep the existing one
00175                 continue;
00176             }
00177         }

```

```

00178         // Check for overlap: if current starts before last ends, merge them
00179         if (current.startSeconds < last.endSeconds + tolerance)
00180         {
00181             // Merge by extending the end time and averaging metrics
00182             Interval& merged = result.back();
00183             merged.endSeconds = std::max(merged.endSeconds, current.endSeconds);
00184             merged.durationSeconds = merged.endSeconds - merged.startSeconds;
00185
00186             // Average the metrics proportionally (weighted by interval duration)
00187             const double lastDur = last.durationSeconds;
00188             const double currDur = current.durationSeconds;
00189             const double totalDur = lastDur + currDur;
00190
00191             if (totalDur > 0.0)
00192             {
00193                 merged.averagePower = (last.averagePower * lastDur + current.averagePower *
00194 currDur) / totalDur;
00195                 merged.maximumPower = std::max(last.maximumPower, current.maximumPower);
00196                 merged.normalizedPower = (last.normalizedPower * lastDur +
00197 current.normalizedPower * currDur) / totalDur;
00198                 merged.averageHeartRate = (last.averageHeartRate * lastDur +
00199 current.averageHeartRate * currDur) / totalDur;
00200                 merged.averageCadence = (last.averageCadence * lastDur + current.averageCadence
00201 * currDur) / totalDur;
00202                 merged.averageSpeed = (last.averageSpeed * lastDur + current.averageSpeed *
00203 currDur) / totalDur;
00204             }
00205             continue;
00206         }
00207         // No overlap/duplicate: add as new interval
00208         result.push_back(current);
00209     }
00210     return result;
00211 }

```

## 8.14 IntervalDetector.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <string>
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00010 struct Interval
00011 {
00012     double startSeconds = 0.0;
00013     double endSeconds = 0.0;
00014     double durationSeconds = 0.0;
00015     double averagePower = 0.0;
00016     double maximumPower = 0.0;
00017     double normalizedPower = 0.0;
00018     double averageHeartRate = 0.0;
00019     double averageCadence = 0.0;
00020     double averageSpeed = 0.0;
00021     std::string label;
00022 };
00023
00024 struct IntervalDetectorConfig
00025 {
00026     double ftp = 250.0;
00027     double workThresholdPct = 0.76;
00028     double minWorkSeconds = 60.0;
00029     double minRecoverySeconds = 20.0;
00030     int smoothingWindow = 30;
00031 }

```



```

00066 };
00067
00074 class IntervalDetector
00075 {
00076 public:
00077     using Config = IntervalDetectorConfig;
00078
00079     static std::vector<Interval> detect(
00080         const RideData& rideData,
00081         const Config& config = Config{});
00082
00083     static std::vector<Interval> removeOverlappingAndDuplicates(
00084         std::vector<Interval> intervals);
00085
00086 private:
00087     static std::vector<double> smoothPower(
00088         const RideData& rideData, int window);
00089
00090     static Interval buildInterval(
00091         const RideData& rideData,
00092         int startIdx, int endIdx,
00093         const std::string& label);
00094 };

```

## 8.15 PowerCurve.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerCurve.h"
00005
00006 #include <cmath>
00007 #include <vector>
00008
00009 std::vector<double> PowerCurve::standardDurations()
00010 {
00011     return {
00012         5, 10, 15, 30, 60,
00013         120, 180, 300, 600,
00014         1200, 1800, 3600, 5400, 7200
00015     };
00016 }
00017
00018 double PowerCurve::bestMeanPower(
00019     const RideData& rideData, double durationSeconds)
00020 {
00021     const auto& records = rideData.records;
00022     if (records.empty()) return 0.0;
00023
00024     std::vector<double> power;
00025     power.reserve(records.size());
00026     for (const auto& r : records)
00027         power.push_back(r.hasPower ? r.power : 0.0);
00028
00029     const int n = static_cast<int>(power.size());
00030     const int window = static_cast<int>(std::round(durationSeconds));
00031     if (window <= 0 || window > n) return 0.0;
00032
00033     double sum = 0.0;
00034     for (int i = 0; i < window; ++i)
00035         sum += power[i];
00036
00037     double best = sum;
00038     for (int i = window; i < n; ++i)
00039     {
00040         sum += power[i] - power[i - window];
00041         if (sum > best) best = sum;
00042     }
00043
00044     return best / window;
00045 }
00046
00047 std::vector<PowerCurvePoint> PowerCurve::computeStandard(
00048     const RideData& rideData)
00049 {
00050     const auto durations = standardDurations();
00051     std::vector<PowerCurvePoint> result;
00052     result.reserve(durations.size());
00053
00054     for (double d : durations)
00055     {
00056         const double p = bestMeanPower(rideData, d);
00057         if (p > 0.0)
00058             result.push_back({ d, p });
00059     }
00060 }

```

```

00059     }
00060
00061     return result;
00062 }

```

## 8.16 PowerCurve.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00013 struct PowerCurvePoint
00014 {
00016     double durationSeconds;
00017
00019     double bestPower;
00020 };
00021
00027 class PowerCurve
00028 {
00029 public:
00034     static std::vector<double> standardDurations();
00035
00042     static double bestMeanPower(
00043         const RideData& rideData, double durationSeconds);
00044
00050     static std::vector<PowerCurvePoint> computeStandard(
00051         const RideData& rideData);
00052 };

```

## 8.17 AnalysisQueue.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AnalysisQueue.h"
00005 #include "AnalysisWorker.h"
00006
00007 #include <QThread>
00008
00009 AnalysisQueue::AnalysisQueue(QObject* parent)
00010     : QObject(parent)
00011 {
00012     m_thread = new QThread(this);
00013     m_worker = new AnalysisWorker;
00014     m_worker->moveToThread(m_thread);
00015
00016     connect(m_worker, &AnalysisWorker::taskCompleted,
00017         this, &AnalysisQueue::onTaskCompleted);
00018     connect(m_worker, &AnalysisWorker::taskFailed,
00019         this, &AnalysisQueue::onTaskFailed);
00020
00021     m_thread->start();
00022 }
00023
00024 AnalysisQueue::~AnalysisQueue()
00025 {
00026     m_thread->quit();
00027     m_thread->wait(3000);
00028     delete m_worker;
00029 }
00030
00031 void AnalysisQueue::setDatabasePath(const QString& path)
00032 {
00033     m_dbPath = path;
00034     // Forward to worker on its own thread.
00035     QMetaObject::invokeMethod(m_worker, [this, path]() {
00036         m_worker->setDatabasePath(path);
00037     }, Qt::QueuedConnection);
00038 }
00039
00040 void AnalysisQueue::enqueue(int activityId)
00041 {
00042     m_queue.enqueue(activityId);
00043     emit pendingCountChanged(m_queue.size());
00044     if (!m_busy)

```

```

00045         processNext();
00046     }
00047
00048     int AnalysisQueue::pendingCount() const
00049     {
00050         return m_queue.size() + (m_busy ? 1 : 0);
00051     }
00052
00053     bool AnalysisQueue::isBusy() const
00054     {
00055         return m_busy;
00056     }
00057
00058     void AnalysisQueue::processNext()
00059     {
00060         if (m_queue.isEmpty() || m_busy)
00061             return;
00062
00063         m_busy = true;
00064         const int activityId = m_queue.dequeue();
00065         emit pendingCountChanged(m_queue.size());
00066
00067         QMetaObject::invokeMethod(m_worker, [this, activityId]() {
00068             m_worker->processTask(activityId);
00069         }, Qt::QueuedConnection);
00070     }
00071
00072     void AnalysisQueue::onTaskCompleted(int activityId)
00073     {
00074         m_busy = false;
00075         emit taskCompleted(activityId);
00076         processNext();
00077     }
00078
00079     void AnalysisQueue::onTaskFailed(int activityId, const QString& error)
00080     {
00081         m_busy = false;
00082         emit taskFailed(activityId, error);
00083         processNext();
00084     }

```

## 8.18 AnalysisQueue.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QObject>
00007 #include <QQueue>
00008 #include <QString>
00009
00010 class QThread;
00011 class AnalysisWorker;
00012
00019 class AnalysisQueue : public QObject
00020 {
00021     Q_OBJECT
00022 public:
00027     explicit AnalysisQueue(QObject* parent = nullptr);
00028     ~AnalysisQueue() override;
00029
00034     void setDatabasePath(const QString& path);
00035
00040     void enqueue(int activityId);
00041
00046     int pendingCount() const;
00047
00052     bool isBusy() const;
00053
00054 signals:
00057     void taskCompleted(int activityId);
00058
00062     void taskFailed(int activityId, const QString& error);
00063
00066     void pendingCountChanged(int count);
00067
00068 private slots:
00069     void onTaskCompleted(int activityId);
00070     void onTaskFailed(int activityId, const QString& error);
00071
00072 private:
00073     void processNext();
00074

```

```

00075     QThread*         m_thread = nullptr;
00076     AnalysisWorker*   m_worker = nullptr;
00077     QQueue<int>       m_queue;
00078     bool              m_busy   = false;
00079     QString           m_dbPath;
00080 };

```

## 8.19 AnalysisWorker.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AnalysisWorker.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlError>
00008 #include <QSqlQuery>
00009 #include <QThread>
00010 #include <QUuid>
00011
00012 namespace
00013 {
00014 void applySqlitePragmas(QSqlDatabase& db)
00015 {
00016     QSqlQuery q(db);
00017     q.exec("PRAGMA journal_mode=WAL");
00018     q.exec("PRAGMA synchronous=NORMAL");
00019     q.exec("PRAGMA temp_store=MEMORY");
00020     q.exec("PRAGMA foreign_keys=ON");
00021     q.exec("PRAGMA cache_size=-20000");
00022 }
00023 }
00024
00025 #include "analysis/ClimbDetector.h"
00026 #include "analysis/ClimbMetrics.h"
00027 #include "analysis/IntervalDetector.h"
00028 #include "core/AnalysisVersions.h"
00029 #include "core/ActivityAnalysisFlags.h"
00030 #include "database/ActivityRepository.h"
00031 #include "database/ClimbRepository.h"
00032 #include "database/DatabaseConnectionGuard.h"
00033 #include "database/IntervalRepository.h"
00034 #include "model/RideDataLoader.h"
00035 #include "utils/UuidGenerator.h"
00036
00037 AnalysisWorker::AnalysisWorker(QObject* parent)
00038     : QObject(parent)
00039 {}
00040
00041 void AnalysisWorker::setDatabasePath(const QString& path)
00042 {
00043     m_dbPath = path;
00044 }
00045
00046 void AnalysisWorker::processTask(int activityId)
00047 {
00048     if (m_dbPath.isEmpty())
00049     {
00050         emit taskFailed(activityId, QStringLiteral("No database path set.));
00051         return;
00052     }
00053
00054     // Use RAII guard to ensure connection cleanup even on exceptions
00055     const QString connName = QStringLiteral("AnalysisWorker_%1_%2_%3")
00056         .arg(activityId)
00057         .arg(reinterpret_cast<quintptr>(QThread::currentThread()))
00058         .arg(QUuid::createUuid().toString(QUuid::WithoutBraces));
00059
00060     try
00061     {
00062         DatabaseConnectionGuard guard(connName);
00063         QSqlDatabase db = guard.database();
00064         db.setDatabaseName(m_dbPath);
00065
00066         if (!db.open())
00067         {
00068             emit taskFailed(activityId, db.lastError().text());
00069             return;
00070         }
00071
00072         applySqlitePragmas(db);
00073
00074         // Load ride data
00075         RideData ride = RideDataLoader::loadActivitySamples(activityId, db);

```

```

00076
00077     if (ride.records.empty())
00078     {
00079         emit taskFailed(activityId, QStringLiteral("Activity has no samples."));
00080         return;
00081     }
00082
00083     ClimbRepository    climbRepo(db);
00084     IntervalRepository intervalRepo(db);
00085
00086     // Skip if already analyzed
00087     {
00088         QSqlQuery flagQ(db);
00089         flagQ.prepare("SELECT COALESCE(analysis_flags,0) FROM activities WHERE id=:id");
00090         flagQ.bindValue(":id", activityId);
00091         if (flagQ.exec() && flagQ.next())
00092         {
00093             const int flags = flagQ.value(0).toInt();
00094             const bool hasClimbs    = (flags & ActivityAnalysisFlags::HasClimbs)    != 0;
00095             const bool hasIntervals = (flags & ActivityAnalysisFlags::HasIntervals) != 0;
00096             if (hasClimbs && hasIntervals)
00097             {
00098                 emit taskCompleted(activityId);
00099                 return;
00100             }
00101         }
00102     }
00103
00104     // Detect and store climbs
00105     if (!climbRepo.hasClimbsEver(activityId))
00106     {
00107         ClimbDetector::Config climbCfg;
00108
00109         // Load per-activity settings if available
00110         {
00111             QSqlQuery settQ(db);
00112             settQ.prepare(
00113                 "SELECT climb_min_gain, climb_min_length, climb_min_gradient"
00114                 " FROM activity_analysis_settings WHERE activity_id=:id");
00115             settQ.bindValue(":id", activityId);
00116             if (settQ.exec() && settQ.next())
00117             {
00118                 if (!settQ.value(0).isNull()) climbCfg.minElevationGainM =
00119 settQ.value(0).toDouble();
00119                 if (!settQ.value(1).isNull()) climbCfg.minLengthKm      =
00120 settQ.value(1).toDouble();
00120                 if (!settQ.value(2).isNull()) climbCfg.minAverageGradient =
00121 settQ.value(2).toDouble();
00121             }
00122         }
00123
00124         const std::vector<Climb> climbs = ClimbDetector::detect(ride, climbCfg);
00125
00126         // Load deleted climbs for resurrection prevention
00127         const QList<ClimbRecord> deleted = climbRepo.deletedClimbsForActivity(activityId);
00128
00129         const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
00130         const std::vector<double> smoothed  = ClimbDetector::smoothAltitudeByDistance(
00131             ride, cumulative, climbCfg.elevationSmoothingMeters);
00132         const std::vector<double> gradient  = ClimbDetector::computeLocalGradientPct(
00133             smoothed, cumulative, climbCfg.gradientWindowHalfMeters);
00134
00135         int ordinal = 1;
00136         for (const Climb& c : climbs)
00137         {
00138             // Resurrection prevention
00139             bool overlapsDeleted = false;
00140             for (const ClimbRecord& d : deleted)
00141             {
00142                 if (c.startSeconds < d.endSeconds && c.endSeconds > d.startSeconds)
00143                 {
00144                     overlapsDeleted = true;
00145                     break;
00146                 }
00147             }
00148             if (overlapsDeleted) continue;
00149
00150             ClimbRecord record;
00151             record.activityId      = activityId;
00152             record.uuid            = UuidGenerator::create();
00153             record.startSeconds    = c.startSeconds;
00154             record.endSeconds      = c.endSeconds;
00155             record.originalStartSeconds = c.startSeconds;
00156             record.originalEndSeconds = c.endSeconds;
00157             record.name            = c.name;
00158             record.elevationGainM  = c.elevationGainM;
00159             record.lengthKm        = c.lengthKm;

```

```

00160         record.averageGradient      = c.averageGradient;
00161         record.avgPower              = c.averagePower;
00162         record.np                    = c.normalizedPower;
00163         record.avgHr                 = c.averageHeartRate;
00164         record.avgCadence            = c.averageCadence;
00165         record.vam                   = c.vam;
00166         record.source                 = QStringLiteral("auto");
00167         record.locked                 = false;
00168         record.deleted                = false;
00169         record.algorithmVersion       = AnalysisVersions::ClimbDetection;
00170         climbRepo.insertClimb(record);
00171         ++ordinal;
00172     }
00173
00174     // Mark HasClimbs flag
00175     QSqlQuery upFlags(db);
00176     upFlags.prepare(
00177         "UPDATE activities SET analysis_flags = COALESCE(analysis_flags,0) | :f WHERE
id=:id");
00178     upFlags.bindValue(":f", ActivityAnalysisFlags::HasClimbs);
00179     upFlags.bindValue(":id", activityId);
00180     upFlags.exec();
00181 }
00182
00183 // Detect and store intervals
00184 {
00185     const QList<IntervalRecord> existing = intervalRepo.listIntervalsForActivity(activityId);
00186     if (existing.isEmpty())
00187     {
00188         // Resolve FTP for this activity's date
00189         double ftp = 250.0;
00190         {
00191             QSqlQuery ftpQ(db);
00192             ftpQ.prepare(
00193                 "SELECT ah.ftp_watts FROM ftp_history ah"
00194                 " JOIN activities a ON a.athlete_id = ah.athlete_id"
00195                 " WHERE a.id=:id AND ah.effective_from <= SUBSTR(a.activity_start_time,1,10)"
00196                 " ORDER BY ah.effective_from DESC LIMIT 1");
00197             ftpQ.bindValue(":id", activityId);
00198             if (ftpQ.exec() && ftpQ.next())
00199                 ftp = ftpQ.value(0).toDouble();
00200         }
00201
00202         IntervalDetector::Config ivCfg;
00203         ivCfg.ftp = ftp;
00204         const std::vector<Interval> intervals = IntervalDetector::detect(ride, ivCfg);
00205
00206         for (const Interval& iv : intervals)
00207         {
00208             IntervalRecord record;
00209             record.activityId      = activityId;
00210             record.uuid             = UuidGenerator::create();
00211             record.startSample      = static_cast<int>(std::round(iv.startSeconds));
00212             record.endSample        = static_cast<int>(std::round(iv.endSeconds));
00213             record.name             = QString::fromStdString(iv.label);
00214             record.type             = QString::fromStdString(iv.label);
00215             record.avgPower         = iv.averagePower;
00216             record.np               = iv.normalizedPower;
00217             record.avgHr            = iv.averageHeartRate;
00218             record.avgCadence       = iv.averageCadence;
00219             record.source           = QStringLiteral("auto");
00220             record.locked           = false;
00221             record.deleted          = false;
00222             record.algorithmVersion = AnalysisVersions::IntervalDetection;
00223             intervalRepo.insertInterval(record);
00224         }
00225
00226         QSqlQuery upFlags(db);
00227         upFlags.prepare(
00228             "UPDATE activities SET analysis_flags = COALESCE(analysis_flags,0) | :f WHERE
id=:id");
00229         upFlags.bindValue(":f", ActivityAnalysisFlags::HasIntervals);
00230         upFlags.bindValue(":id", activityId);
00231         upFlags.exec();
00232     }
00233 }
00234
00235     emit taskCompleted(activityId);
00236 }
00237 catch (const std::exception& ex)
00238 {
00239     // Guard destructor will clean up connection automatically
00240     emit taskFailed(activityId, QString::fromStdString(ex.what()));
00241 }
00242 catch (...)
00243 {
00244     // Guard destructor will clean up connection automatically

```

```

00245         emit taskFailed(activityId, QStringLiteral("Unknown error during analysis.));
00246     }
00247 }

```

## 8.20 AnalysisWorker.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QObject>
00007 #include <QString>
00008
00015 class AnalysisWorker : public QObject
00016 {
00017     Q_OBJECT
00018 public:
00023     explicit AnalysisWorker(QObject* parent = nullptr);
00024
00029     void setDatabasePath(const QString& path);
00030
00031 public slots:
00036     void processTask(int activityId);
00037
00038 signals:
00041     void taskCompleted(int activityId);
00042
00046     void taskFailed(int activityId, const QString& error);
00047
00048 private:
00049     QString m_dbPath;
00050 };

```

## 8.21 SmoothingEngine.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "SmoothingEngine.h"
00005
00006 #include <cmath>
00007 #include <limits>
00008
00009 std::vector<double> SmoothingEngine::smoothPower(const RideData& ride, int seconds)
00010 {
00011     const int n = static_cast<int>(ride.records.size());
00012     std::vector<double> out(static_cast<size_t>(n), std::numeric_limits<double>::quiet_NaN());
00013     if (n == 0)
00014         return out;
00015
00016     if (seconds <= 0)
00017     {
00018         for (int i = 0; i < n; ++i)
00019         {
00020             const RideRecord& r = ride.records[static_cast<size_t>(i)];
00021             if (r.hasPower && r.power > 0.0)
00022                 out[static_cast<size_t>(i)] = r.power;
00023         }
00024         return out;
00025     }
00026
00027     const double halfWindow = seconds / 2.0;
00028     std::vector<double> prefixSum(static_cast<size_t>(n) + 1, 0.0);
00029     std::vector<int> prefixCount(static_cast<size_t>(n) + 1, 0);
00030
00031     for (int i = 0; i < n; ++i)
00032     {
00033         const RideRecord& r = ride.records[static_cast<size_t>(i)];
00034         const bool valid = r.hasPower && r.power > 0.0;
00035         prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] + (valid ? r.power :
00036 0.0);
00037         prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] + (valid ? 1 :
00038 0);
00039     }
00040
00041     int left = 0;
00042     int right = 0;
00043     for (int i = 0; i < n; ++i)
00044     {
00045         const RideRecord& r = ride.records[static_cast<size_t>(i)];

```

```

00044         const double t = r.elapsedSeconds;
00045
00046         while (left < n && ride.records[static_cast<size_t>(left)].elapsedSeconds < t - halfWindow)
00047             ++left;
00048         while (right < n && ride.records[static_cast<size_t>(right)].elapsedSeconds <= t + halfWindow)
00049             ++right;
00050
00051         if (!(r.hasPower && r.power > 0.0))
00052             continue;
00053
00054         const double sum = prefixSum[static_cast<size_t>(right)] -
prefixSum[static_cast<size_t>(left)];
00055         const int count = prefixCount[static_cast<size_t>(right)] -
prefixCount[static_cast<size_t>(left)];
00056         if (count > 0)
00057             out[static_cast<size_t>(i)] = sum / static_cast<double>(count);
00058     }
00059
00060     return out;
00061 }

```

## 8.22 SmoothingEngine.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00013 class SmoothingEngine
00014 {
00015 public:
00022     static std::vector<double> smoothPower(const RideData& ride, int seconds);
00023 };

```

## 8.23 TrainingLoad.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TrainingLoad.h"
00005
00006 #include <cmath>
00007 #include <vector>
00008
00009 double TrainingLoad::normalizedPower(const RideData& rideData)
00010 {
00011     const auto& records = rideData.records;
00012     if (records.empty()) return 0.0;
00013
00014     std::vector<double> power;
00015     power.reserve(records.size());
00016     for (const auto& r : records)
00017         power.push_back(r.hasPower ? r.power : 0.0);
00018
00019     constexpr int kWindow = 30;
00020     const int n = static_cast<int>(power.size());
00021     if (n < kWindow) return 0.0;
00022
00023     double windowSum = 0.0;
00024     for (int i = 0; i < kWindow; ++i)
00025         windowSum += power[i];
00026
00027     double sum4 = std::pow(windowSum / kWindow, 4.0);
00028     int count = 1;
00029
00030     for (int i = kWindow; i < n; ++i)
00031     {
00032         windowSum += power[i] - power[i - kWindow];
00033         sum4 += std::pow(windowSum / kWindow, 4.0);
00034         ++count;
00035     }
00036
00037     return std::pow(sum4 / count, 0.25);
00038 }
00039
00040 double TrainingLoad::intensityFactor(double np, double ftp)
00041 {

```



```

00042     if (ftp <= 0.0) return 0.0;
00043     return np / ftp;
00044 }
00045
00046 double TrainingLoad::trainingStressScore(
00047     double durationSeconds, double np, double ftp)
00048 {
00049     if (ftp <= 0.0) return 0.0;
00050     const double IF = intensityFactor(np, ftp);
00051     return (durationSeconds * np * IF) / (ftp * 3600.0) * 100.0;
00052 }
00053
00054 std::vector<FitnessMetricsPoint> TrainingLoad::fitnessTimeline(
00055     const std::vector<DailyLoadPoint>& dailyLoads,
00056     int atlDays,
00057     int ctlDays)
00058 {
00059     std::vector<FitnessMetricsPoint> out;
00060     out.reserve(dailyLoads.size());
00061
00062     if (dailyLoads.empty())
00063         return out;
00064
00065     const double atlAlpha = atlDays > 0 ? (2.0 / (atlDays + 1.0)) : 1.0;
00066     const double ctlAlpha = ctlDays > 0 ? (2.0 / (ctlDays + 1.0)) : 1.0;
00067
00068     double atl = 0.0;
00069     double ctl = 0.0;
00070     bool initialized = false;
00071
00072     for (const DailyLoadPoint& p : dailyLoads)
00073     {
00074         if (!initialized)
00075         {
00076             atl = p.tss;
00077             ctl = p.tss;
00078             initialized = true;
00079         }
00080         else
00081         {
00082             atl += atlAlpha * (p.tss - atl);
00083             ctl += ctlAlpha * (p.tss - ctl);
00084         }
00085
00086         FitnessMetricsPoint point;
00087         point.atl = atl;
00088         point.ctl = ctl;
00089         point.tsb = ctl - atl;
00090         out.push_back(point);
00091     }
00092
00093     return out;
00094 }

```

## 8.24 TrainingLoad.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "fit/RideData.h"
00009
00013 struct DailyLoadPoint
00014 {
00016     double tss = 0.0;
00017 };
00018
00025 struct FitnessMetricsPoint
00026 {
00028     double atl = 0.0;
00029
00031     double ctl = 0.0;
00032
00034     double tsb = 0.0;
00035 };
00036
00043 class TrainingLoad
00044 {
00045 public:
00051     static double normalizedPower(const RideData& rideData);
00052

```

```

00059     static double intensityFactor(double np, double ftp);
00060
00068     static double trainingStressScore(
00069         double durationSeconds, double np, double ftp);
00070
00078     static std::vector<FitnessMetricsPoint> fitnessTimeline(
00079         const std::vector<DailyLoadPoint>& dailyLoads,
00080         int atlDays = 7,
00081         int ctlDays = 42);
00082 };

```

## 8.25 TrainingMetrics.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TrainingMetrics.h"
00005
00006 #include "analysis/TrainingLoad.h"
00007
00008 TrainingMetricsSummary TrainingMetrics::compute(const RideData& rideData,
00009     double ftp,
00010     double durationSeconds)
00011 {
00012     TrainingMetricsSummary out;
00013
00014     if (durationSeconds <= 0.0 && !rideData.records.empty())
00015         durationSeconds = rideData.records.back().elapsedSeconds;
00016
00017     double powerSum = 0.0;
00018     int powerCount = 0;
00019     double hrSum = 0.0;
00020     int hrCount = 0;
00021
00022     for (const RideRecord& r : rideData.records)
00023     {
00024         if (r.hasPower && r.power > 0.0)
00025         {
00026             powerSum += r.power;
00027             ++powerCount;
00028         }
00029         if (r.hasHeartRate && r.heartRate > 0.0)
00030         {
00031             hrSum += r.heartRate;
00032             ++hrCount;
00033         }
00034     }
00035
00036     out.averagePower = powerCount > 0 ? (powerSum / powerCount) : 0.0;
00037     out.averageHeartRate = hrCount > 0 ? (hrSum / hrCount) : 0.0;
00038
00039     out.normalizedPower = TrainingLoad::normalizedPower(rideData);
00040     out.intensityFactor = TrainingLoad::intensityFactor(out.normalizedPower, ftp);
00041     out.trainingStressScore = TrainingLoad::trainingStressScore(durationSeconds, out.normalizedPower,
ftp);
00042
00043     if (out.averagePower > 0.0)
00044         out.variabilityIndex = out.normalizedPower / out.averagePower;
00045
00046     if (out.averageHeartRate > 0.0)
00047         out.efficiencyFactor = out.normalizedPower / out.averageHeartRate;
00048
00049     return out;
00050 }

```

## 8.26 TrainingMetrics.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007
00008 struct TrainingMetricsSummary
00009 {
00010     double normalizedPower = 0.0;
00011
00012     double intensityFactor = 0.0;
00013
00014     double trainingStressScore = 0.0;

```

```

00021
00023     double variabilityIndex = 0.0;
00024
00026     double efficiencyFactor = 0.0;
00027
00029     double averagePower = 0.0;
00030
00032     double averageHeartRate = 0.0;
00033 };
00034
00038 class TrainingMetrics
00039 {
00040 public:
00048     static TrainingMetricsSummary compute(const RideData& rideData,
00049                                           double ftp,
00050                                           double durationSeconds = 0.0);
00051 };

```

## 8.27 WorkoutAnalyzer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "WorkoutAnalyzer.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008 #include <numbers>
00009
00010 static double segmentKm(double lat1, double lon1, double lat2, double lon2)
00011 {
00012     constexpr double R = 6371.0;
00013     const double dLat = (lat2 - lat1) * std::numbers::pi_v<double> / 180.0;
00014     const double dLon = (lon2 - lon1) * std::numbers::pi_v<double> / 180.0;
00015     const double mLat = (lat1 + lat2) / 2.0 * std::numbers::pi_v<double> / 180.0;
00016     return std::hypot(dLat, dLon * std::cos(mLat)) * R;
00017 }
00018
00019 WorkoutStatistics WorkoutAnalyzer::analyze(const RideData& rideData)
00020 {
00021     WorkoutStatistics stats;
00022     if (rideData.records.empty()) return stats;
00023
00024     const auto& records = rideData.records;
00025     const int n = static_cast<int>(records.size());
00026
00027     double powerSum = 0, hrSum = 0, cadSum = 0, spdSum = 0;
00028     int powerN = 0, hrN = 0, cadN = 0, spdN = 0;
00029
00030     bool prevHasGps = false;
00031     double prevLat = 0, prevLon = 0;
00032     bool usedGps = false;
00033
00034     bool prevHasAlt = false;
00035     double prevAlt = 0;
00036
00037     for (int i = 0; i < n; ++i)
00038     {
00039         const auto& r = records[i];
00040         stats.durationSeconds = r.elapsedSeconds;
00041
00042         if (r.hasPower)
00043         {
00044             powerSum += r.power;
00045             stats.maximumPower = std::max(stats.maximumPower, r.power);
00046             ++powerN;
00047         }
00048
00049         if (r.hasHeartRate)
00050         {
00051             hrSum += r.heartRate;
00052             stats.maximumHeartRate = std::max(stats.maximumHeartRate, r.heartRate);
00053             ++hrN;
00054         }
00055
00056         if (r.hasCadence)
00057         {
00058             cadSum += r.cadence;
00059             stats.maximumCadence = std::max(stats.maximumCadence, r.cadence);
00060             ++cadN;
00061         }
00062
00063         if (r.hasSpeed)
00064         {

```

```

00065         spdSum += r.speed;
00066         stats.maximumSpeed = std::max(stats.maximumSpeed, r.speed);
00067         ++spdN;
00068
00069         if (r.speed > 2.0 && i > 0)
00070         {
00071             const double dt = r.elapsedSeconds - records[i-1].elapsedSeconds;
00072             stats.movingTimeSeconds += std::max(0.0, dt);
00073         }
00074     }
00075
00076     if (r.hasGps)
00077     {
00078         if (prevHasGps)
00079         {
00080             stats.totalDistanceKm +=
00081                 segmentKm(prevLat, prevLon, r.latitude, r.longitude);
00082             usedGps = true;
00083         }
00084         prevHasGps = true;
00085         prevLat = r.latitude;
00086         prevLon = r.longitude;
00087     }
00088
00089     if (r.hasAltitude)
00090     {
00091         if (prevHasAlt && r.altitude > prevAlt)
00092             stats.elevationGainM += r.altitude - prevAlt;
00093         prevHasAlt = true;
00094         prevAlt = r.altitude;
00095     }
00096 }
00097
00098 if (!usedGps && spdN > 0)
00099 {
00100     for (int i = 1; i < n; ++i)
00101     {
00102         const auto& r = records[i];
00103         const auto& prev = records[i-1];
00104         if (!r.hasSpeed) continue;
00105         const double dt = r.elapsedSeconds - prev.elapsedSeconds;
00106         const double spd = (r.speed + (prev.hasSpeed ? prev.speed : r.speed)) / 2.0;
00107         stats.totalDistanceKm += spd * std::max(0.0, dt) / 3600.0;
00108     }
00109 }
00110
00111 if (powerN) stats.averagePower = powerSum / powerN;
00112 if (hrN) stats.averageHeartRate = hrSum / hrN;
00113 if (cadN) stats.averageCadence = cadSum / cadN;
00114 if (spdN) stats.averageSpeed = spdSum / spdN;
00115
00116 return stats;
00117 }

```

## 8.28 WorkoutAnalyzer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007
00011 struct WorkoutStatistics
00012 {
00014     double durationSeconds = 0.0;
00015
00017     double movingTimeSeconds = 0.0;
00018
00020     double totalDistanceKm = 0.0;
00021
00023     double elevationGainM = 0.0;
00024
00026     double averagePower = 0.0;
00027
00029     double maximumPower = 0.0;
00030
00032     double averageHeartRate = 0.0;
00033
00035     double maximumHeartRate = 0.0;
00036
00038     double averageCadence = 0.0;
00039
00041     double maximumCadence = 0.0;

```

```

00042
00044     double averageSpeed      = 0.0;
00045
00047     double maximumSpeed      = 0.0;
00048 };
00049
00053 class WorkoutAnalyzer
00054 {
00055 public:
00061     WorkoutStatistics analyze(const RideData& rideData);
00062 };

```

## 8.29 RideChartDataBuilder.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "RideChartDataBuilder.h"
00004
00005 #include "core/zones/ColorProvider.h"
00006
00007 namespace RideChartDataBuilder
00008 {
00009     QString labelForMetric(Metric metric)
00010     {
00011         switch (metric)
00012         {
00013             case Metric::Power:      return "Power (W)";
00014             case Metric::HeartRate:  return "Heart Rate (bpm)";
00015             case Metric::Cadence:    return "Cadence (rpm)";
00016             case Metric::Speed:      return "Speed (km/h)";
00017             case Metric::Altitude:   return "Altitude (m)";
00018         }
00019         return {};
00020     }
00021
00022     QString labelForColorMetric(ColorMetric metric)
00023     {
00024         switch (metric)
00025         {
00026             case ColorMetric::Power: return "Power (W)";
00027             case ColorMetric::HeartRate: return "Heart Rate (bpm)";
00028             case ColorMetric::Cadence: return "Cadence (rpm)";
00029             case ColorMetric::Speed: return "Speed (km/h)";
00030             case ColorMetric::Altitude: return "Altitude (m)";
00031             case ColorMetric::Gradient: return "Gradient (%)";
00032             case ColorMetric::None: return "";
00033         }
00034         return "";
00035     }
00036
00037     Metric metricFromColorMetric(ColorMetric metric)
00038     {
00039         switch (metric)
00040         {
00041             case ColorMetric::Power: return Metric::Power;
00042             case ColorMetric::HeartRate: return Metric::HeartRate;
00043             case ColorMetric::Cadence: return Metric::Cadence;
00044             case ColorMetric::Speed: return Metric::Speed;
00045             case ColorMetric::Altitude: return Metric::Altitude;
00046             case ColorMetric::Gradient: return Metric::Gradient;
00047             case ColorMetric::None:
00048                 break;
00049         }
00050         return Metric::Power;
00051     }
00052
00053     double valueForMetric(const RideRecord& record, Metric metric)
00054     {
00055         switch (metric)
00056         {
00057             case Metric::Power:      return record.power;
00058             case Metric::HeartRate:  return record.heartRate;
00059             case Metric::Cadence:    return record.cadence;
00060             case Metric::Speed:      return record.speed;
00061             case Metric::Altitude:   return record.altitude;
00062         }
00063         return 0.0;
00064     }
00065
00066     bool isValidSample(const RideRecord& record, Metric metric)
00067     {
00068         switch (metric)
00069         {
00070             case Metric::Power:      return record.hasPower && record.power > 0.0;

```

```

00071         case Metric::HeartRate: return record.hasHeartRate;
00072         case Metric::Cadence:    return record.hasCadence;
00073         case Metric::Speed:      return record.hasSpeed;
00074         case Metric::Altitude:   return record.hasAltitude;
00075     }
00076     return false;
00077 }
00078
00079 int autoSmoothingSecondsForSpan(double spanMinutes)
00080 {
00081     if (spanMinutes < 5.0)    return 0;
00082     if (spanMinutes < 30.0)   return 5;
00083     if (spanMinutes < 120.0)  return 10;
00084     if (spanMinutes < 360.0)  return 30;
00085     return 60;
00086 }
00087
00088 QColor colorForMetricValue(ColorMetric metric,
00089                             const ColorContext& context,
00090                             const RideRecord& record)
00091 {
00092     switch (metric)
00093     {
00094         case ColorMetric::Power:
00095             return ColorProvider::colorForPower(record.power, context.ftp);
00096         case ColorMetric::HeartRate:
00097             return ColorProvider::colorForHeartRate(record.heartRate,
00098                                                         context.heartRateMax,
00099                                                         context.heartRateRest,
00100                                                         context.thresholdHeartRate);
00101         case ColorMetric::Cadence:
00102             return ColorProvider::colorForCadence(record.cadence);
00103         case ColorMetric::Speed:
00104             return ColorProvider::colorForSpeed(record.speed);
00105         case ColorMetric::Altitude:
00106             return ColorProvider::colorForAltitude(record.altitude,
00107                                                         context.altitudeMin,
00108                                                         context.altitudeMax);
00109         case ColorMetric::Gradient:
00110         case ColorMetric::None:
00111             return QColor("#2563eb");
00112     }
00113     return QColor("#2563eb");
00114 }
00115
00116 bool hasColorMetricValue(ColorMetric metric, const RideRecord& record)
00117 {
00118     switch (metric)
00119     {
00120         case ColorMetric::Power:      return record.hasPower && record.power > 0.0;
00121         case ColorMetric::HeartRate:   return record.hasHeartRate;
00122         case ColorMetric::Cadence:     return record.hasCadence;
00123         case ColorMetric::Speed:       return record.hasSpeed;
00124         case ColorMetric::Altitude:    return record.hasAltitude;
00125         case ColorMetric::Gradient:
00126         case ColorMetric::None:        return false;
00127     }
00128     return false;
00129 }
00130
00131 }
00132

```

## 8.30 RideChartDataBuilder.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QColor>
00006 #include <QString>
00007
00008 #include "charts/Metric.h"
00009 #include "core/zones/ZoneDefinition.h"
00010 #include "fit/RideData.h"
00011
00012 namespace RideChartDataBuilder
00013 {
00014     QString labelForMetric(Metric metric);
00015     QString labelForColorMetric(ColorMetric metric);
00016     Metric metricFromColorMetric(ColorMetric metric);
00017     double valueForMetric(const RideRecord& record, Metric metric);
00018     bool isValidSample(const RideRecord& record, Metric metric);
00019     int autoSmoothingSecondsForSpan(double spanMinutes);

```

```

00020 QColor colorForMetricValue(ColorMetric metric,
00021                             const ColorContext& context,
00022                             const RideRecord& record);
00023 bool hasColorMetricValue(ColorMetric metric, const RideRecord& record);
00024 }

```

## 8.31 FitnessChartWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FitnessChartWidget.h"
00005
00006 #include <QPen>
00007 #include <QtCharts/QChart>
00008 #include <QtCharts/QLineSeries>
00009 #include <QtCharts/QValueAxis>
00010
00011 #include <algorithm>
00012
00013 FitnessChartWidget::FitnessChartWidget(QWidget* parent)
00014     : QChartView(new QChart, parent)
00015 {
00016     m_ctlSeries = new QLineSeries;
00017     m_atlSeries = new QLineSeries;
00018     m_tsbSeries = new QLineSeries;
00019
00020     m_ctlSeries->setName("Fitness (CTL)");
00021     m_atlSeries->setName("Fatigue (ATL)");
00022     m_tsbSeries->setName("Form (TSB)");
00023
00024     m_ctlSeries->setPen(QPen(QColor("#16a34a"), 2.0));
00025     m_atlSeries->setPen(QPen(QColor("#dc2626"), 2.0));
00026     m_tsbSeries->setPen(QPen(QColor("#2563eb"), 1.8, Qt::DashLine));
00027
00028     m_axisX = new QValueAxis;
00029     m_axisY = new QValueAxis;
00030     m_axisX->setLabelFormat("%.0f");
00031     m_axisX->setTitleText("Day Index");
00032     m_axisY->setLabelFormat("%.0f");
00033     m_axisY->setTitleText("Load");
00034
00035     chart()->addSeries(m_ctlSeries);
00036     chart()->addSeries(m_atlSeries);
00037     chart()->addSeries(m_tsbSeries);
00038     chart()->addAxis(m_axisX, Qt::AlignBottom);
00039     chart()->addAxis(m_axisY, Qt::AlignLeft);
00040
00041     m_ctlSeries->attachAxis(m_axisX);
00042     m_ctlSeries->attachAxis(m_axisY);
00043     m_atlSeries->attachAxis(m_axisX);
00044     m_atlSeries->attachAxis(m_axisY);
00045     m_tsbSeries->attachAxis(m_axisX);
00046     m_tsbSeries->attachAxis(m_axisY);
00047
00048     chart()->setTitle("Fitness / Fatigue / Form");
00049     chart()->legend()->setVisible(true);
00050     chart()->setAnimationOptions(QChart::NoAnimation);
00051
00052     setRenderHint(QPainter::Antialiasing);
00053     setMinimumHeight(280);
00054 }
00055
00056 void FitnessChartWidget::setTimeline(const std::vector<FitnessMetricsPoint>& timeline)
00057 {
00058     m_ctlSeries->clear();
00059     m_atlSeries->clear();
00060     m_tsbSeries->clear();
00061
00062     if (timeline.empty())
00063     {
00064         m_axisX->setRange(0.0, 1.0);
00065         m_axisY->setRange(-10.0, 10.0);
00066         return;
00067     }
00068
00069     double minY = 1e9;
00070     double maxY = -1e9;
00071
00072     for (size_t i = 0; i < timeline.size(); ++i)
00073     {
00074         const double x = static_cast<double>(i);
00075         const auto& p = timeline[i];
00076

```

```

00077         m_ctlSeries->append(x, p.ctl);
00078         m_atlSeries->append(x, p.atl);
00079         m_tsbSeries->append(x, p.tsb);
00080
00081         minY = std::min(minY, std::min(p.tsb, std::min(p.ctl, p.atl)));
00082         maxY = std::max(maxY, std::max(p.tsb, std::max(p.ctl, p.atl)));
00083     }
00084
00085     if (maxY <= minY)
00086         maxY = minY + 1.0;
00087
00088     const double pad = std::max(5.0, (maxY - minY) * 0.1);
00089     m_axisX->setRange(0.0, static_cast<double>(timeline.size() - 1));
00090     m_axisY->setRange(minY - pad, maxY + pad);
00091 }
00092
00093 void FitnessChartWidget::clearChart()
00094 {
00095     m_ctlSeries->clear();
00096     m_atlSeries->clear();
00097     m_tsbSeries->clear();
00098     m_axisX->setRange(0.0, 1.0);
00099     m_axisY->setRange(-10.0, 10.0);
00100 }

```

## 8.32 FitnessChartWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include <vector>
00009
00010 #include "analysis/TrainingLoad.h"
00011
00012 class QLineSeries;
00013 class QValueAxis;
00014
00021 class FitnessChartWidget : public QChartView
00022 {
00023     Q_OBJECT
00024
00025 public:
00030     explicit FitnessChartWidget(QWidget* parent = nullptr);
00031
00036     void setTimeline(const std::vector<FitnessMetricsPoint>& timeline);
00037
00041     void clearChart();
00042
00043 private:
00044     QLineSeries* m_ctlSeries = nullptr;
00045     QLineSeries* m_atlSeries = nullptr;
00046     QLineSeries* m_tsbSeries = nullptr;
00047     QValueAxis* m_axisX = nullptr;
00048     QValueAxis* m_axisY = nullptr;
00049 };

```

## 8.33 Metric.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00009 enum class Metric
00010 {
00011     Power,
00012     HeartRate,
00013     Cadence,
00014     Speed,
00015     Altitude
00016 };

```



## 8.34 PowerCurveWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerCurveWidget.h"
00005
00006 #include <QtCharts/QCategoryAxis>
00007 #include <QtCharts/QAbstractSeries>
00008 #include <QtCharts/QChart>
00009 #include <QtCharts/QLineSeries>
00010 #include <QtCharts/QScatterSeries>
00011 #include <QtCharts/QValueAxis>
00012 #include <QCursor>
00013 #include <QPainter>
00014 #include <QToolTip>
00015
00016 #include <algorithm>
00017 #include <cmath>
00018
00019 static QString formatDurLabel(double seconds)
00020 {
00021     const int s = static_cast<int>(seconds);
00022     if (s < 60)
00023         return QString("%ls").arg(s);
00024     if (s < 3600)
00025         return QString("%lm").arg(s / 60);
00026     const int h = s / 3600;
00027     const int m = (s % 3600) / 60;
00028     return m > 0
00029         ? QString("%lh%2m").arg(h).arg(m, 2, 10, QChar('0'))
00030         : QString("%lh").arg(h);
00031 }
00032
00033 static std::vector<PowerCurvePoint> mergeWithDurations(const std::vector<double>& durations,
00034                                                         const std::vector<PowerCurvePoint>& points)
00035 {
00036     std::vector<PowerCurvePoint> out;
00037     out.reserve(durations.size());
00038     for (double d : durations)
00039     {
00040         double best = 0.0;
00041         for (const PowerCurvePoint& p : points)
00042         {
00043             if (std::abs(p.durationSeconds - d) < 0.5)
00044             {
00045                 best = p.bestPower;
00046                 break;
00047             }
00048         }
00049         if (best > 0.0)
00050             out.push_back({d, best});
00051     }
00052     return out;
00053 }
00054
00055 PowerCurveWidget::PowerCurveWidget(QWidget* parent)
00056     : QChartView(new QChart, parent)
00057 {
00058     chart()->setTitle("Power Duration Curve");
00059     chart()->legend()->show();
00060     chart()->setAnimationOptions(QChart::NoAnimation);
00061     chart()->setMargins(QMargins(4, 4, 4, 4));
00062     setRenderHint(QPainter::Antialiasing);
00063     setMinimumHeight(240);
00064 }
00065
00066 void PowerCurveWidget::clearChart()
00067 {
00068     chart()->removeAllSeries();
00069     const auto axes = chart()->axes();
00070     for (auto* ax : axes) chart()->removeAxis(ax);
00071     chart()->setTitle("Power Duration Curve");
00072 }
00073
00074 void PowerCurveWidget::setRideData(const RideData& rideData, double ftp)
00075 {
00076     setRideDataWithComparisons(rideData, {}, {}, {}, ftp);
00077 }
00078
00079 void PowerCurveWidget::setRideDataWithComparisons(const RideData& rideData,
00080                                                     const std::vector<PowerCurvePoint>& last90,
00081                                                     const std::vector<PowerCurvePoint>& currentYear,
00082                                                     const std::vector<PowerCurvePoint>& allTime,
00083                                                     double ftp)
00084 {

```

```

00085     clearChart();
00086
00087     const std::vector<double> durations = PowerCurve::standardDurations();
00088     auto pdcPts = mergeWithDurations(durations, PowerCurve::computeStandard(rideData));
00089     if (pdcPts.empty())
00090     {
00091         chart()->setTitle("Power Duration Curve (no power data)");
00092         return;
00093     }
00094
00095     // Use integer indices on X so QCategoryAxis gives clean duration labels
00096     auto* lineSeries = new QLineSeries;
00097     lineSeries->setName("Best Power");
00098
00099     auto* dotSeries = new QScatterSeries;
00100     dotSeries->setName("");
00101     dotSeries->setMarkerSize(9.0);
00102     dotSeries->setColor(QColor(255, 140, 0));
00103
00104     double maxPower = 0.0;
00105     int idx = 0;
00106     for (const auto& pt : pdcPts)
00107     {
00108         lineSeries->append(idx, pt.bestPower);
00109         dotSeries->append(idx, pt.bestPower);
00110         maxPower = std::max(maxPower, pt.bestPower);
00111         ++idx;
00112     }
00113
00114     chart()->addSeries(lineSeries);
00115     chart()->addSeries(dotSeries);
00116
00117     auto addComparisonSeries = [this, &durations, &maxPower](const QString& name,
00118                                                             const QColor& color,
00119                                                             Qt::PenStyle style,
00120                                                             const std::vector<PowerCurvePoint>& pts)
00121     {
00122         if (pts.empty())
00123             return;
00124
00125         const std::vector<PowerCurvePoint> merged = mergeWithDurations(durations, pts);
00126         if (merged.empty())
00127             return;
00128
00129         auto* s = new QLineSeries;
00130         s->setName(name);
00131         QPen pen(color);
00132         pen.setWidthF(1.4);
00133         pen.setStyle(style);
00134         s->setPen(pen);
00135
00136         int idx = 0;
00137         for (double d : durations)
00138         {
00139             for (const PowerCurvePoint& p : merged)
00140             {
00141                 if (std::abs(p.durationSeconds - d) < 0.5)
00142                 {
00143                     s->append(idx, p.bestPower);
00144                     maxPower = std::max(maxPower, p.bestPower);
00145                 }
00146             }
00147             ++idx;
00148         }
00149         chart()->addSeries(s);
00150     };
00151
00152     addComparisonSeries("Last 90 Days", QColor("#0ea5e9"), Qt::DashLine, last90);
00153     addComparisonSeries("Current Year", QColor("#14b8a6"), Qt::DotLine, currentYear);
00154     addComparisonSeries("All Time", QColor("#64748b"), Qt::DashDotLine, allTime);
00155
00156     // X axis: category axis with formatted duration labels
00157     auto* axisX = new QCategoryAxis;
00158     axisX->setTitleText("Duration");
00159     axisX->setGridLineVisible(true);
00160     axisX->setLabelsPosition(QCategoryAxis::AxisLabelsPositionOnValue);
00161     for (int i = 0; i < static_cast<int>(pdcPts.size()); ++i)
00162         axisX->append(formatDurLabel(pdcPts[i].durationSeconds), i);
00163     axisX->setRange(-0.5, idx - 0.5);
00164     chart()->addAxis(axisX, Qt::AlignBottom);
00165     for (QAbstractSeries* series : chart()->series())
00166         series->attachAxis(axisX);
00167
00168     // Y axis
00169     auto* axisY = new QValueAxis;
00170     axisY->setTitleText("Power (W)");
00171     axisY->setLabelFormat("%.0f");

```

```

00172     axisY->setGridLineVisible(true);
00173     axisY->setRange(0, maxPower * 1.1);
00174     chart()->addAxis(axisY, Qt::AlignLeft);
00175     for (QAbstractSeries* series : chart()->series())
00176         series->attachAxis(axisY);
00177
00178     // Optional FTP reference line
00179     if (ftp > 0.0 && ftp <= maxPower * 1.2)
00180     {
00181         auto* ftpLine = new QLineSeries;
00182         ftpLine->setName(QString("FTP (%1 W)").arg(ftp, 0, 'f', 0));
00183         ftpLine->setPen(QPen(Qt::red, 1.5, Qt::DashLine));
00184         ftpLine->append(-0.5, ftp);
00185         ftpLine->append(idx - 0.5, ftp);
00186         chart()->addSeries(ftpLine);
00187         ftpLine->attachAxis(axisX);
00188         ftpLine->attachAxis(axisY);
00189     }
00190
00191     // Tooltip for scatter points
00192     connect(dotSeries, &QScatterSeries::hovered,
00193            this, [pdcPts](const QPointF& point, bool state)
00194            {
00195                if (state)
00196                {
00197                    const int i =
00198                        static_cast<int>(std::round(point.x()));
00199                    if (i >= 0 && i < static_cast<int>(pdcPts.size()))
00200                        QToolTip::showText(
00201                            QCursor::pos(),
00202                            formatDurLabel(pdcPts[i].durationSeconds) +
00203                            "\n" +
00204                            QString("%1 W").arg(
00205                                pdcPts[i].bestPower, 0, 'f', 0));
00206                }
00207                else
00208                {
00209                    QToolTip::hideText();
00210                }
00211            });
00212
00213     const double p3m = PowerCurve::bestMeanPower(rideData, 180.0);
00214     const double p12m = PowerCurve::bestMeanPower(rideData, 720.0);
00215     double cp = 0.0;
00216     double wPrime = 0.0;
00217     if (p3m > 0.0 && p12m > 0.0)
00218     {
00219         cp = ((p12m * 720.0) - (p3m * 180.0)) / (720.0 - 180.0);
00220         wPrime = std::max(0.0, (p3m - cp) * 180.0);
00221     }
00222
00223     chart()->setTitle(
00224         QString("Power Duration Curve   CP %1 W   W' %2 kJ")
00225             .arg(cp, 0, 'f', 0)
00226             .arg(wPrime / 1000.0, 0, 'f', 1));
00227 }

```

## 8.35 PowerCurveWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include "analysis/PowerCurve.h"
00009 #include "fit/RideData.h"
00010
00017 class PowerCurveWidget : public QChartView
00018 {
00019     Q_OBJECT
00020 public:
00025     explicit PowerCurveWidget(QWidget* parent = nullptr);
00026
00032     void setRideData(const RideData& rideData, double ftp = 0.0);
00033
00042     void setRideDataWithComparisons(const RideData& rideData,
00043                                     const std::vector<PowerCurvePoint>& last90,
00044                                     const std::vector<PowerCurvePoint>& currentYear,
00045                                     const std::vector<PowerCurvePoint>& allTime,
00046                                     double ftp = 0.0);
00047
00051     void clearChart();

```

```
00052 };
```

## 8.36 PowerHistogramWidget.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PowerHistogramWidget.h"
00005
00006 #include <QtCharts/QBarCategoryAxis>
00007 #include <QtCharts/QBarSeries>
00008 #include <QtCharts/QBarSet>
00009 #include <QtCharts/QChart>
00010 #include <QtCharts/QValueAxis>
00011 #include <QPainter>
00012
00013 #include <algorithm>
00014 #include <cmath>
00015 #include <map>
00016
00017 PowerHistogramWidget::PowerHistogramWidget(QWidget* parent)
00018     : QChartView(new QChart, parent)
00019 {
00020     chart()->setTitle("Power Distribution");
00021     chart()->legend()->hide();
00022     chart()->setAnimationOptions(QChart::NoAnimation);
00023     chart()->setMargins(QMargins(4, 4, 4, 4));
00024     setRenderHint(QPainter::Antialiasing);
00025     setMinimumHeight(200);
00026 }
00027
00028 void PowerHistogramWidget::clearChart()
00029 {
00030     chart()->removeAllSeries();
00031     const auto axes = chart()->axes();
00032     for (auto* ax : axes) chart()->removeAxis(ax);
00033     chart()->setTitle("Power Distribution");
00034 }
00035
00036 void PowerHistogramWidget::setRideData(const RideData& rideData)
00037 {
00038     clearChart();
00039
00040     constexpr double kBucketW = 25.0;    // watts per bucket
00041
00042     std::map<int, double> buckets;        // bucket index → seconds
00043
00044     const int n = static_cast<int>(rideData.records.size());
00045     for (int i = 0; i < n; ++i)
00046     {
00047         const auto& r = rideData.records[i];
00048         if (!r.hasPower || r.power <= 0.0) continue;
00049
00050         double dt = 1.0;
00051         if (n > 1)
00052         {
00053             if (i == 0)
00054                 dt = rideData.records[1].elapsedSeconds -
00055                     rideData.records[0].elapsedSeconds;
00056             else if (i == n - 1)
00057                 dt = rideData.records[n - 1].elapsedSeconds -
00058                     rideData.records[n - 2].elapsedSeconds;
00059             else
00060                 dt = (rideData.records[i + 1].elapsedSeconds -
00061                     rideData.records[i - 1].elapsedSeconds) / 2.0;
00062         }
00063         dt = std::max(0.0, dt);
00064
00065         const int bucket = static_cast<int>(r.power / kBucketW);
00066         buckets[bucket] += dt;
00067     }
00068
00069     if (buckets.empty())
00070     {
00071         chart()->setTitle("Power Distribution (no data)");
00072         return;
00073     }
00074
00075     const int minBucket = buckets.begin()->first;
00076     const int maxBucket = buckets.rbegin()->first;
00077     const int numBuckets = maxBucket - minBucket + 1;
00078
00079     auto* barSet = new QBarSet("Time (s)");
00080     QStringList categories;
```

```

00081
00082 // Show axis labels every N buckets to avoid crowding
00083 const int labelStep = std::max(1, numBuckets / 10);
00084
00085 for (int b = minBucket; b <= maxBucket; ++b)
00086 {
00087     const double sec = buckets.count(b) ? buckets.at(b) : 0.0;
00088     *barSet << sec;
00089
00090     const int relIdx = b - minBucket;
00091     categories <<
00092         (relIdx % labelStep == 0
00093          ? QString::number(static_cast<int>(b * kBucketW))
00094          : QString(""));
00095 }
00096
00097 auto* series = new QBarSeries;
00098 series->append(barSet);
00099 chart()->addSeries(series);
00100
00101 auto* axisX = new QBarCategoryAxis;
00102 axisX->append(categories);
00103 axisX->setTitleText("Power (W)");
00104 axisX->setLabelsAngle(-45);
00105 chart()->addAxis(axisX, Qt::AlignBottom);
00106 series->attachAxis(axisX);
00107
00108 auto* axisY = new QValueAxis;
00109 axisY->setTitleText("Time (s)");
00110 axisY->setLabelFormat("%.0f");
00111 axisY->setGridLineVisible(true);
00112 chart()->addAxis(axisY, Qt::AlignLeft);
00113 series->attachAxis(axisY);
00114
00115 chart()->setTitle("Power Distribution");
00116 }

```

## 8.37 PowerHistogramWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include "fit/RideData.h"
00009
00015 class PowerHistogramWidget : public QChartView
00016 {
00017     Q_OBJECT
00018 public:
00023     explicit PowerHistogramWidget(QWidget* parent = nullptr);
00024
00029     void setRideData(const RideData& rideData);
00030
00034     void clearChart();
00035 };

```

## 8.38 RideChartRenderer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "RideChartRenderer.h"
00004
00005 #include <cmath>
00006
00007 namespace RideChartRenderer
00008 {
00009 double niceStep(double range)
00010 {
00011     if (range <= 0.0) return 1.0;
00012     const double rough = range / 8.0;
00013     const double magnitude = std::pow(10.0, std::floor(std::log10(rough)));
00014     const double residual = rough / magnitude;
00015     double step;
00016     if (residual < 1.5) step = 1.0;
00017     else if (residual < 3.5) step = 2.0;
00018     else if (residual < 7.5) step = 5.0;
00019     else step = 10.0;
00020     return step * magnitude;

```

```

00021 }
00022
00023 void niceRange(double dataMin, double dataMax,
00024               double& axisMin, double& axisMax)
00025 {
00026     const double range = dataMax - dataMin;
00027     const double step = niceStep(range > 0.0 ? range : dataMax);
00028     axisMin = (dataMin < step) ? 0.0 : std::floor(dataMin / step) * step;
00029     axisMax = std::ceil (dataMax / step) * step;
00030     if (axisMax <= axisMin) axisMax = axisMin + step;
00031 }
00032 }

```

## 8.39 RideChartRender.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 namespace RideChartRender
00006 {
00007     double niceStep(double range);
00008     void niceRange(double dataMin, double dataMax,
00009                   double& axisMin, double& axisMax);
00010 }

```

## 8.40 RideChartWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideChartWidget.h"
00005
00006 #include <QColor>
00007 #include <QCursor>
00008 #include <QAction>
00009 #include <QMenu>
00010 #include <QPen>
00011 #include <QMouseEvent>
00012 #include <QPainter>
00013 #include <QResizeEvent>
00014 #include <QToolTip>
00015 #include <QWheelEvent>
00016 #include <QtCharts/QCategoryAxis>
00017 #include <QtCharts/QAreaSeries>
00018 #include <QtCharts/QChart>
00019 #include <QtCharts/QLineSeries>
00020 #include <QtCharts/QValueAxis>
00021
00022 #include "analysis/TrainingLoad.h"
00023 #include "charts/data/RideChartDataBuilder.h"
00024 #include "charts/rendering/RideChartRender.h"
00025 #include "charts/selection/RideChartSelectionManager.h"
00026 #include "core/zones/ColorProvider.h"
00027 #include "core/zones/ZoneCalculator.h"
00028
00029 #include <algorithm>
00030 #include <array>
00031 #include <cmath>
00032 #include <limits>
00033
00034 namespace
00035 {
00036 }
00037
00038 using namespace RideChartDataBuilder;
00039 using namespace RideChartSelectionManager;
00040
00041 using namespace RideChartRender;
00042
00043 RideChartWidget::RideChartWidget(Metric metric, QWidget* parent)
00044     : QChartView(new QChart, parent)
00045     , m_metric(metric)
00046 {
00047     m_series = new QLineSeries;
00048     m_series->setUseOpenGL(false);
00049     m_series->setVisible(false);
00050
00051     m_axisX = new QValueAxis;
00052     m_axisX->setVisible(false);
00053 }

```

```

00054     m_axisXDisplay = new QCategoryAxis;
00055     m_axisXDisplay->setTitleText("Time");
00056     m_axisXDisplay->setGridLineVisible(true);
00057     m_axisXDisplay->setLabelsPosition(QCategoryAxis::AxisLabelsPositionOnValue);
00058
00059     m_axisY = new QValueAxis;
00060     m_axisY->setTitleText(labelForMetric(metric));
00061     m_axisY->setLabelFormat("%.0f");
00062     m_axisY->setGridLineVisible(true);
00063
00064     m_axisYOverlay = new QValueAxis;
00065     m_axisYOverlay->setVisible(false);
00066     m_axisYOverlay->setGridLineVisible(false);
00067     m_axisYOverlay->setLabelsColor(QColor("#475569"));
00068
00069     chart()->addSeries(m_series);
00070     chart()->addAxis(m_axisX, Qt::AlignBottom);
00071     chart()->addAxis(m_axisXDisplay, Qt::AlignBottom);
00072     chart()->addAxis(m_axisY, Qt::AlignLeft);
00073     chart()->addAxis(m_axisYOverlay, Qt::AlignRight);
00074     m_series->attachAxis(m_axisX);
00075     m_series->attachAxis(m_axisY);
00076
00077     chart()->setTitle(labelForMetric(metric));
00078     chart()->legend()->hide();
00079     chart()->setMargins(QMargins(4, 4, 4, 4));
00080     chart()->setAnimationOptions(QChart::NoAnimation);
00081
00082     // Horizontal-only rubber-band keeps Y-axis scale fixed
00083     setRubberBand(QChartView::HorizontalRubberBand);
00084     setRenderHint(QPainter::Antialiasing);
00085
00086     // Mouse tracking for crosshair (no button press required)
00087     setMouseTracking(true);
00088     viewport()->setMouseTracking(true);
00089
00090     // rangeChanged: clamp to [0, m_origMaxX], rebuild labels, emit for sync, rescale Y
00091     connect(m_axisX, &QValueAxis::rangeChanged,
00092             this, [this](double min, double max)
00093             {
00094                 const double finalMin = std::max(0.0, min);
00095                 const double finalMax = std::min(max, m_origMaxX);
00096
00097                 if (finalMin != min || finalMax != max)
00098                 {
00099                     QSignalBlocker blocker(m_axisX);
00100                     m_axisX->setRange(finalMin, finalMax);
00101                 }
00102
00103                 updateXTicks(finalMin, finalMax);
00104                 updateYAxisForVisibleRange(finalMin, finalMax);
00105
00106                 emit visibleRangeChanged(finalMin, finalMax);
00107
00108                 if (!m_syncing)
00109                     emit xRangeChanged(finalMin, finalMax);
00110             });
00111
00112     setMinimumHeight(220);
00113     setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);
00114
00115 }
00116
00117 double RideChartWidget::visibleRangeStartMinutes() const
00118 {
00119     return m_axisX ? m_axisX->min() : 0.0;
00120 }
00121
00122 double RideChartWidget::visibleRangeEndMinutes() const
00123 {
00124     return m_axisX ? m_axisX->max() : 0.0;
00125 }
00126
00127 void RideChartWidget::setPowerSmoothingSeconds(int seconds)
00128 {
00129     const int normalized = std::max(0, seconds);
00130     if (m_requestedPowerSmoothingSeconds == normalized)
00131         return;
00132
00133     m_requestedPowerSmoothingSeconds = normalized;
00134     if (m_metric == Metric::Power)
00135         rebuildChart();
00136 }
00137
00138 void RideChartWidget::setAutoSmoothingEnabled(bool enabled)
00139 {
00140     if (m_autoSmoothing == enabled)

```

```

00141         return;
00142
00143         m_autoSmoothing = enabled;
00144         if (m_metric == Metric::Power)
00145             rebuildChart();
00146     }
00147
00148 void RideChartWidget::setIntervals(const std::vector<Interval>& intervals)
00149 {
00150     m_intervals = intervals;
00151     if (m_metric == Metric::Power)
00152         rebuildChart();
00153 }
00154
00155 void RideChartWidget::setClimbs(const std::vector<Climb>& climbs)
00156 {
00157     m_climbs = climbs;
00158     rebuildChart();
00159 }
00160
00161 void RideChartWidget::setMetricOverlay(ColorMetric metric, bool enabled)
00162 {
00163     m_metricOverlayMetric = metric;
00164     m_metricOverlayEnabled = enabled && metric != ColorMetric::None;
00165     rebuildChart();
00166 }
00167
00168 void RideChartWidget::setSelectedClimbIndex(int index)
00169 {
00170     if (m_selectedClimbIndex == index)
00171         return;
00172
00173     m_selectedClimbIndex = index;
00174     // Selected climb fill is rendered as a background band, so rebuild is needed
00175     // to reflect table single-click selection changes.
00176     rebuildChart();
00177 }
00178
00179 void RideChartWidget::setClimbEditingEnabled(bool enabled)
00180 {
00181     m_climbEditingEnabled = enabled;
00182     if (!enabled)
00183         m_climbHandleDrag = ClimbHandleDrag::None;
00184     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
00185 }
00186
00187 std::vector<double> RideChartWidget::computeSmoothedPowerSeries(int seconds) const
00188 {
00189     return computeSmoothedSeries(Metric::Power, seconds);
00190 }
00191
00192 std::vector<double> RideChartWidget::computeSmoothedSeries(Metric metric, int seconds) const
00193 {
00194     const int n = static_cast<int>(m_rideData.records.size());
00195     std::vector<double> out(static_cast<size_t>(n), std::numeric_limits<double>::quiet_NaN());
00196     if (n == 0)
00197         return out;
00198
00199     // Check cache: key = (metric_id * 1000 + seconds)
00200     // Only cache if seconds > 0 (raw data doesn't need caching)
00201     if (seconds > 0)
00202     {
00203         const int metricId = static_cast<int>(metric);
00204         const int cacheKey = metricId * 1000 + seconds;
00205         auto cached = m_smoothedSeriesCache.find(cacheKey);
00206         if (cached != m_smoothedSeriesCache.end())
00207             return cached->second;
00208     }
00209
00210     if (seconds <= 0)
00211     {
00212         for (int i = 0; i < n; ++i)
00213         {
00214             const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];
00215             if (isValidSample(record, metric))
00216                 out[static_cast<size_t>(i)] = valueForMetric(record, metric);
00217         }
00218         return out;
00219     }
00220
00221     const double halfWindow = seconds / 2.0;
00222     std::vector<double> prefixSum(static_cast<size_t>(n) + 1, 0.0);
00223     std::vector<int> prefixCount(static_cast<size_t>(n) + 1, 0);
00224
00225     for (int i = 0; i < n; ++i)
00226     {
00227         const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];

```



```

00228         const bool valid = isValidSample(record, metric);
00229         prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] + (valid ?
valueForMetric(record, metric) : 0.0);
00230         prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] + (valid ? 1 :
0);
00231     }
00232
00233     int left = 0;
00234     int right = 0;
00235     for (int i = 0; i < n; ++i)
00236     {
00237         const RideRecord& record = m_rideData.records[static_cast<size_t>(i)];
00238         const double t = record.elapsedSeconds;
00239
00240         while (left < n && m_rideData.records[static_cast<size_t>(left)].elapsedSeconds < t -
halfWindow)
00241             ++left;
00242         while (right < n && m_rideData.records[static_cast<size_t>(right)].elapsedSeconds <= t +
halfWindow)
00243             ++right;
00244
00245         if (!isValidSample(record, metric))
00246             continue;
00247
00248         const double sum = prefixSum[static_cast<size_t>(right)] -
prefixSum[static_cast<size_t>(left)];
00249         const int count = prefixCount[static_cast<size_t>(right)] -
prefixCount[static_cast<size_t>(left)];
00250         if (count > 0)
00251             out[static_cast<size_t>(i)] = sum / static_cast<double>(count);
00252     }
00253
00254     // Store in cache
00255     const int metricId = static_cast<int>(metric);
00256     const int cacheKey = metricId * 1000 + seconds;
00257     m_smoothedSeriesCache[cacheKey] = out;
00258
00259     return out;
00260 }
00261
00262 std::vector<int> RideChartWidget::buildSampleIndices(const std::vector<double>& values,
00263 int targetPointBudget) const
00264 {
00265     std::vector<int> indices;
00266     const int total = static_cast<int>(values.size());
00267     if (total == 0)
00268         return indices;
00269
00270     if (total <= targetPointBudget || targetPointBudget <= 8)
00271     {
00272         indices.reserve(static_cast<size_t>(total));
00273         for (int i = 0; i < total; ++i)
00274             indices.push_back(i);
00275         return indices;
00276     }
00277
00278     const int bucketCount = std::max(1, targetPointBudget / 4);
00279     const double bucketSize = static_cast<double>(total) / static_cast<double>(bucketCount);
00280     indices.reserve(static_cast<size_t>(targetPointBudget));
00281
00282     for (int bucket = 0; bucket < bucketCount; ++bucket)
00283     {
00284         const int begin = static_cast<int>(std::floor(bucket * bucketSize));
00285         const int end = std::min(total, static_cast<int>(std::floor((bucket + 1) * bucketSize)));
00286         if (begin >= end)
00287             continue;
00288
00289         int first = -1;
00290         int last = -1;
00291         int minIdx = -1;
00292         int maxIdx = -1;
00293         double minVal = std::numeric_limits<double>::max();
00294         double maxVal = std::numeric_limits<double>::lowest();
00295
00296         for (int i = begin; i < end; ++i)
00297         {
00298             const double v = values[static_cast<size_t>(i)];
00299             if (std::isnan(v))
00300                 continue;
00301
00302             if (first < 0)
00303                 first = i;
00304             last = i;
00305             if (v < minVal)
00306             {
00307                 minVal = v;
00308                 minIdx = i;

```

```

00309         }
00310         if (v > maxVal)
00311         {
00312             maxVal = v;
00313             maxIdx = i;
00314         }
00315     }
00316
00317     if (first < 0)
00318         continue;
00319
00320     std::vector<int> bucketIndices = { first };
00321     if (minIdx >= 0 && minIdx != first)
00322         bucketIndices.push_back(minIdx);
00323     if (maxIdx >= 0 && maxIdx != first && maxIdx != minIdx)
00324         bucketIndices.push_back(maxIdx);
00325     if (last >= 0 && last != first && last != minIdx && last != maxIdx)
00326         bucketIndices.push_back(last);
00327
00328     std::sort(bucketIndices.begin(), bucketIndices.end());
00329     for (int idx : bucketIndices)
00330     {
00331         if (indices.empty() || indices.back() != idx)
00332             indices.push_back(idx);
00333     }
00334 }
00335
00336 if (!indices.empty() && indices.front() != 0)
00337     indices.insert(indices.begin(), 0);
00338 if (!indices.empty() && indices.back() != total - 1)
00339     indices.push_back(total - 1);
00340 if (indices.empty())
00341 {
00342     for (int i = 0; i < total; ++i)
00343         indices.push_back(i);
00344 }
00345
00346 return indices;
00347 }
00348
00349 void RideChartWidget::clearBackgroundSeries()
00350 {
00351     for (QAreaSeries* series : m_backgroundSeries)
00352     {
00353         chart()->removeSeries(series);
00354         delete series;
00355     }
00356     m_backgroundSeries.clear();
00357 }
00358
00359 void RideChartWidget::addZoneBackgroundBands(double minY, double maxY, double maxXMinutes)
00360 {
00361     if (m_metric != Metric::Power)
00362         return;
00363
00364     const std::vector<Zone> zones = ZoneCalculator::powerZones(m_colorContext.ftp);
00365     for (const Zone& zone : zones)
00366     {
00367         const double y0 = std::clamp(zone.minValue, minY, maxY);
00368         const double y1 = std::clamp(
00369             zone.maxValue >= std::numeric_limits<double>::max() / 2.0 ? maxY : zone.maxValue,
00370             minY,
00371             maxY);
00372         if (y1 <= y0)
00373             continue;
00374
00375         auto* upper = new QLineSeries;
00376         auto* lower = new QLineSeries;
00377         upper->append(0.0, y1);
00378         upper->append(maxXMinutes, y1);
00379         lower->append(0.0, y0);
00380         lower->append(maxXMinutes, y0);
00381
00382         auto* area = new QAreaSeries(upper, lower);
00383         QColor fill = zone.color;
00384         fill.setAlpha(24);
00385         area->setColor(fill);
00386         area->setPen(Qt::NoPen);
00387
00388         chart()->addSeries(area);
00389         area->attachAxis(m_axisX);
00390         area->attachAxis(m_axisY);
00391         m_backgroundSeries.push_back(area);
00392     }
00393 }
00394
00395 void RideChartWidget::addIntervalBackgroundBands(double minY, double maxY)

```

```

00396 {
00397     if (m_metric != Metric::Power || m_intervals.empty())
00398         return;
00399
00400     for (const Interval& iv : m_intervals)
00401     {
00402         const bool isWork = iv.label == "Work";
00403         const QColor base = isWork ? QColor("#dc2626") : QColor("#0284c7");
00404
00405         auto* upper = new QLineSeries;
00406         auto* lower = new QLineSeries;
00407         const double x0 = iv.startSeconds / 60.0;
00408         const double x1 = iv.endSeconds / 60.0;
00409         upper->append(x0, maxY);
00410         upper->append(x1, maxY);
00411         lower->append(x0, minY);
00412         lower->append(x1, minY);
00413
00414         auto* area = new QAreaSeries(upper, lower);
00415         QColor fill = base;
00416         fill.setAlpha(isWork ? 18 : 10);
00417         area->setColor(fill);
00418         area->setPen(Qt::NoPen);
00419
00420         chart()->addSeries(area);
00421         area->attachAxis(m_axisX);
00422         area->attachAxis(m_axisY);
00423         m_backgroundSeries.push_back(area);
00424     }
00425 }
00426
00427 void RideChartWidget::addMetricOverlayBands(double minY, double maxY)
00428 {
00429     if (!m_metricOverlayEnabled || m_metricOverlayMetric == ColorMetric::None)
00430         return;
00431
00432     const auto& records = m_rideData.records;
00433     if (records.size() < 2)
00434         return;
00435
00436     auto quantizeColor = [](const QColor& in)
00437     {
00438         auto q = [](int c) { return (c / 32) * 32; };
00439         return QColor(q(in.red()), q(in.green()), q(in.blue()));
00440     };
00441
00442     int segmentStart = -1;
00443     QColor current;
00444
00445     auto flush = [&](int startIndex, int endIndex, const QColor& color)
00446     {
00447         if (startIndex < 0 || endIndex <= startIndex)
00448             return;
00449
00450         const double x0 = records[static_cast<size_t>(startIndex)].elapsedSeconds / 60.0;
00451         const double x1 = records[static_cast<size_t>(endIndex)].elapsedSeconds / 60.0;
00452         if (x1 <= x0)
00453             return;
00454
00455         auto* upper = new QLineSeries;
00456         auto* lower = new QLineSeries;
00457         upper->append(x0, maxY);
00458         upper->append(x1, maxY);
00459         lower->append(x0, minY);
00460         lower->append(x1, minY);
00461
00462         auto* area = new QAreaSeries(upper, lower);
00463         QColor fill = color;
00464         fill.setAlpha(26);
00465         area->setColor(fill);
00466         area->setPen(Qt::NoPen);
00467         chart()->addSeries(area);
00468         area->attachAxis(m_axisX);
00469         area->attachAxis(m_axisY);
00470         m_backgroundSeries.push_back(area);
00471     };
00472
00473     for (int i = 0; i < static_cast<int>(records.size()); ++i)
00474     {
00475         const RideRecord& r = records[static_cast<size_t>(i)];
00476         if (!hasColorMetricValue(m_metricOverlayMetric, r))
00477         {
00478             if (segmentStart >= 0)
00479             {
00480                 flush(segmentStart, i, current);
00481                 segmentStart = -1;
00482             }

```

```

00483         continue;
00484     }
00485
00486     QColor c = quantizeColor(colorForMetricValue(m_metricOverlayMetric, m_colorContext, r));
00487     if (segmentStart < 0)
00488     {
00489         segmentStart = i;
00490         current = c;
00491         continue;
00492     }
00493
00494     if (c != current)
00495     {
00496         flush(segmentStart, i, current);
00497         segmentStart = i;
00498         current = c;
00499     }
00500 }
00501
00502 if (segmentStart >= 0)
00503     flush(segmentStart, static_cast<int>(records.size() - 1, current);
00504 }
00505
00506 void RideChartWidget::addClimbBackgroundBands(double minY, double maxY)
00507 {
00508     if (m_climbs.empty())
00509         return;
00510
00511     auto addBand = [this, minY, maxY](const Climb& climb, bool selected)
00512     {
00513         auto* upper = new QLineSeries;
00514         auto* lower = new QLineSeries;
00515         const double x0 = climb.startSeconds / 60.0;
00516         const double x1 = climb.endSeconds / 60.0;
00517         upper->append(x0, maxY);
00518         upper->append(x1, maxY);
00519         lower->append(x0, minY);
00520         lower->append(x1, minY);
00521
00522         auto* area = new QAreaSeries(upper, lower);
00523         QColor fill("#16a34a");
00524         fill.setAlpha(selected ? 54 : 20);
00525         area->setColor(fill);
00526         if (selected)
00527         {
00528             QPen border(QColor("#15803d"));
00529             border.setWidthF(1.5);
00530             area->setPen(border);
00531         }
00532         else
00533         {
00534             area->setPen(Qt::NoPen);
00535         }
00536
00537         chart()->addSeries(area);
00538         area->attachAxis(m_axisX);
00539         area->attachAxis(m_axisY);
00540         m_backgroundSeries.push_back(area);
00541     };
00542
00543     // Draw non-selected climbs first, then selected climb on top.
00544     for (int idx = 0; idx < static_cast<int>(m_climbs.size()); ++idx)
00545     {
00546         if (idx == m_selectedClimbIndex)
00547             continue;
00548         addBand(m_climbs[static_cast<size_t>(idx)], false);
00549     }
00550
00551     if (m_selectedClimbIndex >= 0 && m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
00552         addBand(m_climbs[static_cast<size_t>(m_selectedClimbIndex)], true);
00553 }
00554
00555 void RideChartWidget::updateXTicks(double minVal, double maxVal)
00556 {
00557     const QStringList existing = m_axisXDisplay->categoriesLabels();
00558     for (const auto& lbl : existing)
00559         m_axisXDisplay->remove(lbl);
00560
00561     const double span = maxVal - minVal;
00562     double interval;
00563     bool showSeconds = false;
00564
00565     if (span <= 1.0) { interval = 1.0 / 6.0; showSeconds = true; }
00566     else if (span <= 3.0) { interval = 0.5; showSeconds = true; }
00567     else if (span <= 8.0) { interval = 1.0; }
00568     else if (span <= 20.0) { interval = 2.0; }
00569     else if (span <= 40.0) { interval = 5.0; }

```

```

00570     else if (span <= 90.0) { interval = 10.0; }
00571     else if (span <= 180.0) { interval = 15.0; }
00572     else if (span <= 360.0) { interval = 30.0; }
00573     else { interval = 60.0; }
00574
00575     auto formatTime = [showSeconds](double minutes) -> QString
00576     {
00577         const int totalSec = static_cast<int>(std::round(minutes * 60.0));
00578         const int h = totalSec / 3600;
00579         const int m = (totalSec % 3600) / 60;
00580         const int s = totalSec % 60;
00581         if (h > 0)
00582             return showSeconds
00583                 ? QString("%1:%2:%3").arg(h).arg(m,2,10,QChar('0')).arg(s,2,10,QChar('0'))
00584                 : QString("%1:%2").arg(h).arg(m,2,10,QChar('0'));
00585         else
00586             return showSeconds
00587                 ? QString("%1:%2").arg(m).arg(s,2,10,QChar('0'))
00588                 : QString("%1").arg(m);
00589     };
00590
00591     double firstTick = std::ceil(minVal / interval) * interval;
00592     if (firstTick < 0.0) firstTick = 0.0;
00593
00594     for (double t = firstTick; t <= maxVal + interval * 1e-6; t += interval)
00595     {
00596         if (t < 0.0) continue;
00597         const QString label = formatTime(t);
00598         if (!m_axisXDisplay->categoriesLabels().contains(label))
00599             m_axisXDisplay->append(label, t);
00600     }
00601
00602     m_axisXDisplay->setRange(minVal, maxVal);
00603 }
00604
00605 void RideChartWidget::setRideData(const RideData& rideData,
00606                                   ColorMetric colorMetric,
00607                                   const ColorContext& colorContext)
00608 {
00609     m_rideData = rideData;
00610     m_colorMetric = colorMetric;
00611     m_colorContext = colorContext;
00612     m_smoothedSeriesCache.clear();
00613     rebuildChart(false);
00614 }
00615
00616 void RideChartWidget::rebuildChart(bool preserveXRange)
00617 {
00618     if (m_isRebuilding)
00619         return;
00620
00621     m_isRebuilding = true;
00622
00623     // Invalidate Y-axis range cache since data is changing
00624     m_yAxisRangeCache.valid = false;
00625
00626     const bool hadData = m_hasData;
00627     const double previousMinX = m_axisX->min();
00628     const double previousMaxX = m_axisX->max();
00629
00630     for (QLineSeries* series : m_rawSeries)
00631     {
00632         chart()->removeSeries(series);
00633         delete series;
00634     }
00635     m_rawSeries.clear();
00636
00637     for (QLineSeries* series : m_colorSeries)
00638     {
00639         chart()->removeSeries(series);
00640         delete series;
00641     }
00642     m_colorSeries.clear();
00643
00644     for (QLineSeries* series : m_overlaySeries)
00645     {
00646         chart()->removeSeries(series);
00647         delete series;
00648     }
00649     m_overlaySeries.clear();
00650
00651     for (QLineSeries* series : m_referenceSeries)
00652     {
00653         chart()->removeSeries(series);
00654         delete series;
00655     }
00656     m_referenceSeries.clear();

```

```

00657     clearBackgroundSeries();
00658
00659     m_axisYOverlay->setVisible(false);
00660     m_axisYOverlay->setTitleText("");
00661
00662     m_tooltipPoints.clear();
00663     m_tooltipRawValues.clear();
00664     m_tooltipDisplayValues.clear();
00665
00666     if (m_rideData.records.empty())
00667     {
00668         clearChart();
00669         m_isRebuilding = false;
00670         return;
00671     }
00672
00673     constexpr int kTargetPoints = 2600;
00674     const int total = static_cast<int>(m_rideData.records.size());
00675     const double visibleSpanMinutes = hadData ? std::max(0.0, previousMaxX - previousMinX) :
m_rideData.records.back().elapsedSeconds / 60.0;
00676     m_effectivePowerSmoothingSeconds = m_autoSmoothing
00677         ? autoSmoothingSecondsForSpan(visibleSpanMinutes)
00678         : m_requestedPowerSmoothingSeconds;
00679
00680     std::vector<double> primarySeries(total, std::numeric_limits<double>::quiet_NaN());
00681     if (m_effectivePowerSmoothingSeconds > 0)
00682     {
00683         // computeSmoothedSeries handles caching internally for all metrics
00684         primarySeries = computeSmoothedSeries(m_metric, m_effectivePowerSmoothingSeconds);
00685     }
00686     else
00687     {
00688         primarySeries = computeSmoothedSeries(m_metric, 0);
00689     }
00690
00691     const std::vector<int> sampleIndices = buildSampleIndices(primarySeries, kTargetPoints);
00692
00693     QList<QPointF> probePoints;
00694     probePoints.reserve(sampleIndices.size() + 2);
00695
00696     double sum = 0.0;
00697     double minY = std::numeric_limits<double>::max();
00698     double maxY = std::numeric_limits<double>::lowest();
00699     int validCount = 0;
00700
00701     QLineSeries* rawSeries = nullptr;
00702     QLineSeries* colorSeries = nullptr;
00703     QColor currentColor;
00704     bool hadValidPrimary = false;
00705     QPointF previousPrimaryPoint;
00706
00707     auto makeRawSeries = [this]()
00708     {
00709         auto* series = new QLineSeries;
00710         QPen pen(QColor(37, 99, 235, 60));
00711         pen.setWidthF(1.0);
00712         series->setPen(pen);
00713         chart()->addSeries(series);
00714         series->attachAxis(m_axisX);
00715         series->attachAxis(m_axisY);
00716         m_rawSeries.push_back(series);
00717         return series;
00718     };
00719
00720     auto makeColorSeries = [this](const QColor& color)
00721     {
00722         auto* series = new QLineSeries;
00723         QPen pen(color);
00724         pen.setWidthF(2.4);
00725         pen.setCapStyle(Qt::RoundCap);
00726         pen.setJoinStyle(Qt::RoundJoin);
00727         series->setPen(pen);
00728         chart()->addSeries(series);
00729         series->attachAxis(m_axisX);
00730         series->attachAxis(m_axisY);
00731         m_colorSeries.push_back(series);
00732         return series;
00733     };
00734
00735     const bool showRawOverlay = (m_metric == Metric::Power && m_effectivePowerSmoothingSeconds > 0);
00736
00737     for (int index : sampleIndices)
00738     {
00739         const RideRecord& record = m_rideData.records[static_cast<size_t>(index)];
00740         const bool rawValid = isValidSample(record, m_metric);
00741         const double rawValue = rawValid ? valueForMetric(record, m_metric) :
std::numeric_limits<double>::quiet_NaN();

```

```

00742
00743     const double primaryValue = primarySeries[static_cast<size_t>(index)];
00744     const bool primaryValid = !std::isnan(primaryValue);
00745
00746     if (showRawOverlay)
00747     {
00748         if (rawValid)
00749         {
00750             if (!rawSeries)
00751                 rawSeries = makeRawSeries();
00752             rawSeries->append(record.elapsedSeconds / 60.0, rawValue);
00753         }
00754         else
00755         {
00756             rawSeries = nullptr;
00757         }
00758     }
00759
00760     if (!primaryValid)
00761     {
00762         colorSeries = nullptr;
00763         hadValidPrimary = false;
00764         continue;
00765     }
00766
00767     const QPointF point(record.elapsedSeconds / 60.0, primaryValue);
00768     probePoints.append(point);
00769     m_tooltipPoints.push_back(point);
00770     m_tooltipRawValues.push_back(rawValue);
00771     m_tooltipDisplayValues.push_back(primaryValue);
00772     sum += primaryValue;
00773     minY = std::min(minY, primaryValue);
00774     maxY = std::max(maxY, primaryValue);
00775     ++validCount;
00776
00777     QColor edgeColor = QColor("#2563eb");
00778     if (m_colorMetric != ColorMetric::None && hasColorMetricValue(m_colorMetric, record))
00779         edgeColor = colorForMetricValue(m_colorMetric, m_colorContext, record);
00780
00781     if (!hadValidPrimary)
00782     {
00783         colorSeries = makeColorSeries(edgeColor);
00784         colorSeries->append(point);
00785         currentColor = edgeColor;
00786     }
00787     else if (!colorSeries || currentColor != edgeColor)
00788     {
00789         colorSeries = makeColorSeries(edgeColor);
00790         colorSeries->append(previousPrimaryPoint);
00791         colorSeries->append(point);
00792         currentColor = edgeColor;
00793     }
00794     else
00795     {
00796         colorSeries->append(point);
00797     }
00798
00799     previousPrimaryPoint = point;
00800     hadValidPrimary = true;
00801 }
00802
00803 m_series->replace(probePoints);
00804
00805 const double maxX = m_rideData.records.back().elapsedSeconds / 60.0;
00806 m_origMaxX = maxX;
00807 m_hasData = validCount > 0;
00808
00809 if (m_hasData)
00810 {
00811     m_dataMinY = minY;
00812     m_dataMaxY = maxY;
00813     updateYAxis();
00814
00815     addZoneBackgroundBands(m_dataMinY, m_dataMaxY, maxX);
00816     addMetricOverlayBands(m_dataMinY, m_dataMaxY);
00817     addIntervalBackgroundBands(m_dataMinY, m_dataMaxY);
00818     addClimbBackgroundBands(m_dataMinY, m_dataMaxY);
00819
00820     if (m_metricOverlayEnabled && m_metricOverlayMetric != ColorMetric::None)
00821     {
00822         const Metric overlayMetric = metricFromColorMetric(m_metricOverlayMetric);
00823         std::vector<double> overlayValues = computeSmoothedSeries(overlayMetric, 0);
00824         const std::vector<int> overlayIndices = buildSampleIndices(overlayValues, kTargetPoints);
00825
00826         double overlayMin = std::numeric_limits<double>::max();
00827         double overlayMax = std::numeric_limits<double>::lowest();
00828         QLineSeries* currentOverlay = nullptr;

```

```

00829     QColor currentOverlayColor;
00830     QPointF previousOverlayPoint;
00831     bool hadOverlayPoint = false;
00832
00833     auto makeOverlaySeries = [this](const QColor& color)
00834     {
00835         auto* series = new QLineSeries;
00836         QPen pen(color);
00837         pen.setWidthF(1.8);
00838         pen.setCapStyle(Qt::RoundCap);
00839         pen.setJoinStyle(Qt::RoundJoin);
00840         series->setPen(pen);
00841         chart()->addSeries(series);
00842         series->attachAxis(m_axisX);
00843         series->attachAxis(m_axisYOverlay);
00844         m_overlaySeries.push_back(series);
00845         return series;
00846     };
00847
00848     for (int idx : overlayIndices)
00849     {
00850         const RideRecord& r = m_rideData.records[static_cast<size_t>(idx)];
00851         if (!isValidSample(r, overlayMetric))
00852         {
00853             currentOverlay = nullptr;
00854             hadOverlayPoint = false;
00855             continue;
00856         }
00857
00858         const double y = overlayValues[static_cast<size_t>(idx)];
00859         if (std::isnan(y))
00860         {
00861             currentOverlay = nullptr;
00862             hadOverlayPoint = false;
00863             continue;
00864         }
00865
00866         overlayMin = std::min(overlayMin, y);
00867         overlayMax = std::max(overlayMax, y);
00868
00869         const QPointF point(r.elapsedSeconds / 60.0, y);
00870         const QColor segColor = colorForMetricValue(m_metricOverlayMetric, m_colorContext, r);
00871
00872         if (!currentOverlay)
00873         {
00874             currentOverlay = makeOverlaySeries(segColor);
00875             currentOverlayColor = segColor;
00876             currentOverlay->append(point);
00877         }
00878         else if (currentOverlayColor != segColor)
00879         {
00880             currentOverlay = makeOverlaySeries(segColor);
00881             currentOverlayColor = segColor;
00882             if (hadOverlayPoint)
00883                 currentOverlay->append(previousOverlayPoint);
00884             currentOverlay->append(point);
00885         }
00886         else
00887         {
00888             currentOverlay->append(point);
00889         }
00890
00891         previousOverlayPoint = point;
00892         hadOverlayPoint = true;
00893     }
00894
00895     if (!m_overlaySeries.empty())
00896     {
00897         // Keep metric overlay confined to the bottom third of the plot so
00898         // the altitude profile remains visually dominant.
00899         const double rawMin = overlayMin;
00900         const double rawMax = overlayMax;
00901         const double span = std::max(0.001, rawMax - rawMin);
00902
00903         // Extend the axis maximum by 2× the data span so the overlay line
00904         // sits in the lower third, but anchor ticks at the actual data
00905         // range so labels don't appear in the empty upper portion.
00906         m_axisYOverlay->setRange(rawMin, rawMax + 2.0 * span);
00907
00908         double niceMin = rawMin;
00909         double niceMax = rawMax;
00910         niceRange(rawMin, rawMax, niceMin, niceMax);
00911         const int desiredTicks = std::clamp(height() / 60, 3, 6);
00912         const double niceInterval = std::max(1.0, (niceMax - niceMin) / desiredTicks);
00913         m_axisYOverlay->setTickType(QValueAxis::TicksDynamic);
00914         m_axisYOverlay->setTickAnchor(niceMin);
00915         m_axisYOverlay->setTickInterval(niceInterval);

```



```

00916
00917         m_axisYOverlay->setTitleText(labelForColorMetric(m_metricOverlayMetric));
00918         m_axisYOverlay->setVisible(true);
00919     }
00920 }
00921
00922 if (m_metric == Metric::Power)
00923 {
00924     const double avg = sum / validCount;
00925     const double np = TrainingLoad::normalizedPower(m_rideData);
00926     auto addReferenceLine = [this, maxX](double y, const QColor& color, Qt::PenStyle style)
00927     {
00928         auto* series = new QLineSeries;
00929         QPen pen(color);
00930         pen.setWidthF(1.0);
00931         pen.setStyle(style);
00932         series->setPen(pen);
00933         series->append(0.0, y);
00934         series->append(maxX, y);
00935         chart()->addSeries(series);
00936         series->attachAxis(m_axisX);
00937         series->attachAxis(m_axisY);
00938         m_referenceSeries.push_back(series);
00939     };
00940
00941     addReferenceLine(m_colorContext.ftp, QColor("#7c3aed"), Qt::DashLine);
00942     addReferenceLine(avg, QColor("#0f766e"), Qt::DotLine);
00943     if (np > 0.0)
00944         addReferenceLine(np, QColor("#b45309"), Qt::DashDotLine);
00945
00946     // Mark best efforts by duration as vertical guide lines.
00947     const std::array<int, 9> bestDurations = { 1, 5, 15, 30, 60, 300, 600, 1200, 3600 };
00948     std::vector<double> rawPower(static_cast<size_t>(total), 0.0);
00949     std::vector<int> validMask(static_cast<size_t>(total), 0);
00950     for (int i = 0; i < total; ++i)
00951     {
00952         const RideRecord& rec = m_rideData.records[static_cast<size_t>(i)];
00953         if (rec.hasPower && rec.power > 0.0)
00954         {
00955             rawPower[static_cast<size_t>(i)] = rec.power;
00956             validMask[static_cast<size_t>(i)] = 1;
00957         }
00958     }
00959
00960     std::vector<double> prefixSum(static_cast<size_t>(total) + 1, 0.0);
00961     std::vector<int> prefixCount(static_cast<size_t>(total) + 1, 0);
00962     for (int i = 0; i < total; ++i)
00963     {
00964         prefixSum[static_cast<size_t>(i) + 1] = prefixSum[static_cast<size_t>(i)] +
00965 rawPower[static_cast<size_t>(i)];
00966         prefixCount[static_cast<size_t>(i) + 1] = prefixCount[static_cast<size_t>(i)] +
00967 validMask[static_cast<size_t>(i)];
00968     }
00969
00970     auto addBestMarker = [this](double xMinutes, double minV, double maxV, const QColor&
00971 color)
00972     {
00973         auto* series = new QLineSeries;
00974         QPen pen(color);
00975         pen.setWidthF(1.0);
00976         pen.setStyle(Qt::DashDotDotLine);
00977         series->setPen(pen);
00978         series->append(xMinutes, minV);
00979         series->append(xMinutes, maxV);
00980         chart()->addSeries(series);
00981         series->attachAxis(m_axisX);
00982         series->attachAxis(m_axisY);
00983         m_referenceSeries.push_back(series);
00984     };
00985
00986     for (int duration : bestDurations)
00987     {
00988         if (duration <= 0 || duration >= total)
00989             continue;
00990
00991         double bestAvg = -1.0;
00992         int bestEndIndex = -1;
00993         for (int end = duration - 1; end < total; ++end)
00994         {
00995             const int begin = end - duration + 1;
00996             const double winSum = prefixSum[static_cast<size_t>(end) + 1] -
00997 prefixSum[static_cast<size_t>(begin)];
00998             const int winCount = prefixCount[static_cast<size_t>(end) + 1] -
00999 prefixCount[static_cast<size_t>(begin)];
01000             if (winCount <= 0)
01001                 continue;

```

```

00998         const double avgPower = winSum / winCount;
00999         if (avgPower > bestAvg)
01000         {
01001             bestAvg = avgPower;
01002             bestEndIndex = end;
01003         }
01004     }
01005
01006     if (bestEndIndex < 0)
01007         continue;
01008
01009     const double xMinutes =
m_rideData.records[static_cast<size_t>(bestEndIndex)].elapsedSeconds / 60.0;
01010     QColor markerColor("#1d4ed8");
01011     markerColor.setAlpha(170);
01012     addBestMarker(xMinutes, m_dataMinY, m_dataMaxY, markerColor);
01013 }
01014
01015 const QString smoothingLabel = m_effectivePowerSmoothingSeconds <= 0
01016 ? QString("raw")
01017 : QString("%1s").arg(m_effectivePowerSmoothingSeconds);
01018
01019 chart()->setTitle(
01020     QString("%1 min %2 avg %3 max %4 FTP %5 NP %6 smoothing %7")
01021         .arg(labelForMetric(m_metric))
01022         .arg(minY, 0, 'f', 0)
01023         .arg(avg, 0, 'f', 0)
01024         .arg(maxY, 0, 'f', 0)
01025         .arg(m_colorContext.ftp)
01026         .arg(np, 0, 'f', 0)
01027         .arg(smoothingLabel));
01028 }
01029 else
01030 {
01031     const double avg = sum / validCount;
01032     chart()->setTitle(
01033         labelForMetric(m_metric) +
01034         QString(" min %1 avg %2 max %3 smoothing %4")
01035             .arg(minY, 0, 'f', 0)
01036             .arg(avg, 0, 'f', 0)
01037             .arg(maxY, 0, 'f', 0)
01038             .arg(m_effectivePowerSmoothingSeconds <= 0 ? QString("raw") :
01039                 QString("%1s").arg(m_effectivePowerSmoothingSeconds)));
01040 }
01041 else
01042 {
01043     m_axisY->setRange(0.0, 1.0);
01044     chart()->setTitle(labelForMetric(m_metric) + " (no data)");
01045 }
01046
01047 if (m_hasData)
01048 {
01049     if (preserveXRange && hadData && previousMaxX > previousMinX)
01050         m_axisX->setRange(std::max(0.0, previousMinX), std::min(previousMaxX, maxX));
01051     else
01052         m_axisX->setRange(0.0, maxX);
01053 }
01054 else
01055 {
01056     m_axisX->setRange(0.0, 1.0);
01057 }
01058
01059 m_isRebuilding = false;
01060 }
01061
01062 void RideChartWidget::updateYAxis()
01063 {
01064     if (!m_hasData && m_dataMaxY <= m_dataMinY) return;
01065
01066     double axisMin, axisMax;
01067     niceRange(m_dataMinY, m_dataMaxY, axisMin, axisMax);
01068     m_axisY->setRange(axisMin, axisMax);
01069
01070     // Tick count: ~one label per 40 px of chart height, clamped 4-15.
01071     const int ticks = std::clamp(height() / 40, 4, 15);
01072     m_axisY->setTickCount(ticks);
01073 }
01074
01075 void RideChartWidget::updateYAxisForVisibleRange(double xMinMin, double xMaxMin)
01076 {
01077     if (!m_hasData || m_tooltipPoints.empty()) return;
01078
01079     // Check cache: if same range, reuse calculation
01080     if (m_yAxisRangeCache.valid &&
01081         xMinMin == m_yAxisRangeCache.cachedMinX &&
01082         xMaxMin == m_yAxisRangeCache.cachedMaxX)

```

```

01083     {
01084         m_axisY->setRange(m_yAxisRangeCache.cachedAxisMin, m_yAxisRangeCache.cachedAxisMax);
01085         return;
01086     }
01087
01088     double visMin = std::numeric_limits<double>::max();
01089     double visMax = std::numeric_limits<double>::lowest();
01090
01091     for (const QPointF& pt : m_tooltipPoints)
01092     {
01093         if (pt.x() >= xMinMin && pt.x() <= xMaxMin)
01094         {
01095             visMin = std::min(visMin, pt.y());
01096             visMax = std::max(visMax, pt.y());
01097         }
01098     }
01099
01100     if (visMin > visMax)
01101     {
01102         // No points in view -- fall back to full range
01103         updateYAxis();
01104         return;
01105     }
01106
01107     double axisMin, axisMax;
01108     niceRange(visMin, visMax, axisMin, axisMax);
01109
01110     // Cache this result for rapid zoom operations
01111     m_yAxisRangeCache.cachedMinX = xMinMin;
01112     m_yAxisRangeCache.cachedMaxX = xMaxMin;
01113     m_yAxisRangeCache.cachedAxisMin = axisMin;
01114     m_yAxisRangeCache.cachedAxisMax = axisMax;
01115     m_yAxisRangeCache.valid = true;
01116
01117     m_axisY->setRange(axisMin, axisMax);
01118
01119     const int ticks = std::clamp(height() / 40, 4, 15);
01120     m_axisY->setTickCount(ticks);
01121 }
01122
01123 void RideChartWidget::resizeEvent(QResizeEvent* event)
01124 {
01125     QChartView::resizeEvent(event);
01126     // Only update tick count when we have data; don't change range on resize.
01127     if (m_hasData)
01128     {
01129         const int ticks = std::clamp(height() / 40, 4, 15);
01130         m_axisY->setTickCount(ticks);
01131     }
01132 }
01133
01134 void RideChartWidget::clearChart()
01135 {
01136     for (QLineSeries* series : m_rawSeries)
01137     {
01138         chart()->removeSeries(series);
01139         delete series;
01140     }
01141     m_rawSeries.clear();
01142
01143     for (QLineSeries* series : m_colorSeries)
01144     {
01145         chart()->removeSeries(series);
01146         delete series;
01147     }
01148     m_colorSeries.clear();
01149
01150     for (QLineSeries* series : m_overlaySeries)
01151     {
01152         chart()->removeSeries(series);
01153         delete series;
01154     }
01155     m_overlaySeries.clear();
01156
01157     for (QLineSeries* series : m_referenceSeries)
01158     {
01159         chart()->removeSeries(series);
01160         delete series;
01161     }
01162     m_referenceSeries.clear();
01163     clearBackgroundSeries();
01164     m_series->clear();
01165     m_smoothedSeriesCache.clear();
01166     m_yAxisRangeCache.valid = false;
01167     m_tooltipPoints.clear();
01168     m_tooltipRawValues.clear();
01169     m_tooltipDisplayValues.clear();

```

```

01170     m_hasData      = false;
01171     m_origMaxX     = 1.0;
01172     m_crosshairX   = -1.0;
01173     updateXTicks(0.0, 1.0);
01174     m_axisX->setRange(0.0, 1.0);
01175     m_axisY->setRange(0.0, 1.0);
01176     m_axisYOverlay->setVisible(false);
01177     m_axisYOverlay->setTitleText("");
01178     chart()->setTitle(labelForMetric(m_metric));
01179 }
01180
01181 void RideChartWidget::setXRange(double min, double max)
01182 {
01183     m_syncing = true;
01184     m_axisX->setRange(min, max);
01185     m_syncing = false;
01186     emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01187 }
01188
01189 void RideChartWidget::setCrosshair(double xMinutes)
01190 {
01191     m_crosshairX = xMinutes;
01192
01193     if (m_crosshairX < 0.0 || m_tooltipPoints.empty())
01194     {
01195         m_crosshairY = std::numeric_limits<double>::quiet_NaN();
01196         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01197         return;
01198     }
01199
01200     const auto it = std::lower_bound(
01201         m_tooltipPoints.begin(),
01202         m_tooltipPoints.end(),
01203         m_crosshairX,
01204         [](const QPointF& point, double x) { return point.x() < x; });
01205
01206     size_t index = 0;
01207     if (it == m_tooltipPoints.end())
01208         index = m_tooltipPoints.size() - 1;
01209     else
01210         index = static_cast<size_t>(std::distance(m_tooltipPoints.begin(), it));
01211
01212     if (index > 0 && index < m_tooltipPoints.size())
01213     {
01214         const double currDist = std::abs(m_tooltipPoints[index].x() - m_crosshairX);
01215         const double prevDist = std::abs(m_tooltipPoints[index - 1].x() - m_crosshairX);
01216         if (prevDist < currDist)
01217             --index;
01218     }
01219
01220     m_crosshairY = m_tooltipPoints[index].y();
01221     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01222 }
01223
01224 void RideChartWidget::resetZoom()
01225 {
01226     if (m_hasData)
01227     {
01228         m_axisX->setRange(0.0, m_origMaxX);
01229         emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01230     }
01231 }
01232
01233 void RideChartWidget::fitToData()
01234 {
01235     resetZoom();
01236 }
01237
01238 void RideChartWidget::wheelEvent(QWheelEvent* event)
01239 {
01240     if (m_newClimbCreating)
01241     {
01242         event->accept();
01243         return;
01244     }
01245
01246     if (m_climbEditingEnabled &&
01247         m_selectedClimbIndex >= 0 &&
01248         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01249     {
01250         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01251         const QPointF scenePos = mapToScene(event->position()).toPoint();
01252         if (hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit)
01253         {
01254             event->accept();
01255             return;
01256         }
01257     }

```

```

01257     }
01258
01259     const double factor = event->angleDelta().y() > 0 ? 1.15 : 1.0 / 1.15;
01260
01261     const double lo      = m_axisX->min();
01262     const double hi      = m_axisX->max();
01263     const double center = (lo + hi) / 2.0;
01264     const double half    = (hi - lo) / 2.0 / factor;
01265
01266     const double newMin = std::max(0.0, center - half);
01267     const double newMax = std::min(m_origMaxX, center + half);
01268
01269     if (newMax > newMin)
01270     {
01271         m_axisX->setRange(newMin, newMax);
01272         emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01273     }
01274
01275     event->accept();
01276 }
01277
01278 void RideChartWidget::mousePressEvent(QMouseEvent* event)
01279 {
01280     if (m_newClimbCreating && m_hasData && event->button() == Qt::LeftButton)
01281     {
01282         // Update current end position on press; mouseRelease will commit.
01283         const QPointF scenePos = mapToScene(event->pos());
01284         const QRectF plotArea = chart()->plotArea();
01285         const QPointF clampedPos(
01286             std::clamp(scenePos.x(), plotArea.left(), plotArea.right()),
01287             std::clamp(scenePos.y(), plotArea.top(), plotArea.bottom()));
01288         const double xMinutes = chart()->mapToValue(clampedPos, m_series).x();
01289         m_newClimbCurrentSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01290         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01291         event->accept();
01292         return;
01293     }
01294
01295     if (m_climbEditingEnabled &&
01296         m_hasData &&
01297         event->button() == Qt::RightButton)
01298     {
01299         const QPointF scenePos = mapToScene(event->pos());
01300         if (chart()->plotArea().contains(scenePos))
01301         {
01302             const double xMinutes = chart()->mapToValue(scenePos, m_series).x();
01303             const double anchorSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01304
01305             QMenu menu(this);
01306             QAction* newClimbAction = menu.addAction("New Climb");
01307             QAction* selected = menu.exec(event->globalPosition().toPoint());
01308             if (selected == newClimbAction)
01309             {
01310                 m_newClimbCreating = true;
01311                 m_newClimbDragging = true; // box visible immediately after menu close
01312                 m_newClimbStartSec = anchorSec;
01313                 m_newClimbCurrentSec = anchorSec;
01314                 viewport()->setCursor(Qt::SizeHorCursor);
01315                 setMouseTracking(true);
01316                 scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01317                 event->accept();
01318                 return;
01319             }
01320         }
01321     }
01322
01323     if (m_climbEditingEnabled &&
01324         m_hasData &&
01325         event->button() == Qt::LeftButton &&
01326         m_selectedClimbIndex >= 0 &&
01327         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01328     {
01329         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01330         const QPointF scenePos = mapToScene(event->pos());
01331         const ClimbHandleHitResult hit = hitTestClimbHandles(chart(), m_series, m_axisY, climb,
01332             scenePos);
01333         if (hit.hit)
01334         {
01335             m_climbHandleDrag = hit.startHandle ? ClimbHandleDrag::Start : ClimbHandleDrag::End;
01336             m_dragOriginalClimbStartSec = climb.startSeconds;
01337             m_dragOriginalClimbEndSec = climb.endSeconds;
01338             viewport()->setCursor(Qt::SizeHorCursor);
01339             event->accept();
01340             return;
01341         }
01342     }

```

```

01343 // Left-click inside a climb band → select it
01344 if (m_hasData && !m_climbs.empty() && event->button() == Qt::LeftButton)
01345 {
01346     const QPointF scenePos = mapToScene(event->pos());
01347     if (chart()->plotArea().contains(scenePos))
01348     {
01349         const double xSec = chart()->mapToValue(scenePos, m_series).x() * 60.0;
01350         for (int i = 0; i < static_cast<int>(m_climbs.size()); ++i)
01351         {
01352             const Climb& c = m_climbs[static_cast<size_t>(i)];
01353             if (xSec >= c.startSeconds && xSec <= c.endSeconds)
01354             {
01355                 emit climbClicked(i);
01356                 event->accept();
01357                 return;
01358             }
01359         }
01360     }
01361 }
01362
01363 if (m_metric == Metric::Power &&
01364     m_hasData &&
01365     event->button() == Qt::RightButton &&
01366     (event->modifiers() & Qt::ShiftModifier))
01367 {
01368     const QPointF scenePos = mapToScene(event->pos());
01369     if (chart()->plotArea().contains(scenePos))
01370     {
01371         const double x = chart()->mapToValue(scenePos, m_series).x();
01372         m_intervalSelecting = true;
01373         m_intervalStartMin = std::clamp(x, 0.0, m_origMaxX);
01374         m_intervalEndMin = m_intervalStartMin;
01375         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01376         event->accept();
01377         return;
01378     }
01379 }
01380
01381 if (event->button() == Qt::RightButton)
01382 {
01383     event->accept();
01384     return;
01385 }
01386
01387 QChartView::mousePressEvent(event);
01388 }
01389
01390 void RideChartWidget::mouseReleaseEvent(QMouseEvent* event)
01391 {
01392     if (m_newClimbCreating && event->button() == Qt::LeftButton)
01393     {
01394         if (m_newClimbDragging)
01395         {
01396             const double startSec = std::min(m_newClimbStartSec, m_newClimbCurrentSec);
01397             const double endSec = std::max(m_newClimbStartSec, m_newClimbCurrentSec);
01398             if (endSec - startSec >= 1.0)
01399                 emit newClimbRequested(startSec, endSec);
01400         }
01401
01402         m_newClimbCreating = false;
01403         m_newClimbDragging = false;
01404         m_newClimbStartSec = -1.0;
01405         m_newClimbCurrentSec = -1.0;
01406         viewport()->unsetCursor();
01407         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01408         event->accept();
01409         return;
01410     }
01411
01412     if (event->button() == Qt::LeftButton && m_climbHandleDrag != ClimbHandleDrag::None)
01413     {
01414         if (m_selectedClimbIndex >= 0 &&
01415             m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01416         {
01417             const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01418             if (std::abs(climb.startSeconds - m_dragOriginalClimbStartSec) > 1e-3 ||
01419                 std::abs(climb.endSeconds - m_dragOriginalClimbEndSec) > 1e-3)
01420             {
01421                 emit climbBoundaryEdited(
01422                     m_dragOriginalClimbStartSec,
01423                     m_dragOriginalClimbEndSec,
01424                     climb.startSeconds,
01425                     climb.endSeconds);
01426             }
01427         }
01428
01429         m_climbHandleDrag = ClimbHandleDrag::None;

```

```

01430         viewport()->unsetCursor();
01431         event->accept();
01432         return;
01433     }
01434
01435     if (m_intervalSelecting && event->button() == Qt::RightButton)
01436     {
01437         m_intervalSelecting = false;
01438         const double startMin = std::min(m_intervalStartMin, m_intervalEndMin);
01439         const double endMin = std::max(m_intervalStartMin, m_intervalEndMin);
01440         m_intervalStartMin = -1.0;
01441         m_intervalEndMin = -1.0;
01442         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01443
01444         const double startSec = startMin * 60.0;
01445         const double endSec = endMin * 60.0;
01446         if (endSec - startSec >= 1.0)
01447             emit intervalSelectionRequested(startSec, endSec);
01448
01449         event->accept();
01450         return;
01451     }
01452
01453     if (event->button() == Qt::RightButton)
01454     {
01455         event->accept();
01456         return;
01457     }
01458
01459     QChartView::mouseReleaseEvent(event);
01460 }
01461
01462 void RideChartWidget::mouseDoubleClickEvent(QMouseEvent* event)
01463 {
01464     if (event->button() == Qt::RightButton)
01465     {
01466         event->accept();
01467         return;
01468     }
01469
01470     if (m_newClimbCreating)
01471     {
01472         event->accept();
01473         return;
01474     }
01475
01476     if (m_climbEditingEnabled &&
01477         m_selectedClimbIndex >= 0 &&
01478         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01479     {
01480         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01481         const QPointF scenePos = mapToScene(event->pos());
01482         if (hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit)
01483         {
01484             event->accept();
01485             return;
01486         }
01487     }
01488
01489     if (m_hasData)
01490     {
01491         m_axisX->setRange(0.0, m_origMaxX);
01492         emit visibleRangeChanged(m_axisX->min(), m_axisX->max());
01493     }
01494     QChartView::mouseDoubleClickEvent(event);
01495 }
01496
01497 void RideChartWidget::mouseMoveEvent(QMouseEvent* event)
01498 {
01499     if (m_newClimbCreating)
01500     {
01501         const QPointF scenePos = mapToScene(event->pos());
01502         const QRectF plotArea = chart()->plotArea();
01503         const QPointF clampedChartPos(
01504             std::clamp(scenePos.x(), plotArea.left(), plotArea.right()),
01505             std::clamp(scenePos.y(), plotArea.top(), plotArea.bottom()));
01506         const double xMinutes = chart()->mapToValue(clampedChartPos, m_series).x();
01507         m_newClimbCurrentSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01508         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01509         event->accept();
01510         return;
01511     }
01512
01513     if (m_climbHandleDrag != ClimbHandleDrag::None &&
01514         m_selectedClimbIndex >= 0 &&
01515         m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01516     {

```

```

01517     const QPointF chartPos = mapToScene(event->pos());
01518     const QRectF plotArea = chart()->plotArea();
01519     const QPointF clampedChartPos(
01520         std::clamp(chartPos.x(), plotArea.left(), plotArea.right()),
01521         std::clamp(chartPos.y(), plotArea.top(), plotArea.bottom()));
01522     const double xMinutes = chart()->mapToValue(clampedChartPos, m_series).x();
01523     const double xSec = std::clamp(xMinutes * 60.0, 0.0, m_origMaxX * 60.0);
01524
01525     Climbe climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01526     const double oldStart = climb.startSeconds;
01527     const double oldEnd = climb.endSeconds;
01528     constexpr double kMinClimbDurationSec = 1.0;
01529
01530     if (m_climbHandleDrag == ClimbeHandleDrag::Start)
01531         climb.startSeconds = std::min(xSec, climb.endSeconds - kMinClimbDurationSec);
01532     else
01533         climb.endSeconds = std::max(xSec, climb.startSeconds + kMinClimbDurationSec);
01534
01535     (void)oldStart;
01536     (void)oldEnd;
01537     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01538     event->accept();
01539     return;
01540 }
01541
01542 if (m_climbEditingEnabled &&
01543     m_selectedClimbIndex >= 0 &&
01544     m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01545 {
01546     const Climbe climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01547     const QPointF scenePos = mapToScene(event->pos());
01548     const bool hoverHandle = hitTestClimbHandles(chart(), m_series, m_axisY, climb, scenePos).hit;
01549     if (hoverHandle)
01550     {
01551         viewport()->setCursor(Qt::SizeHorCursor);
01552         event->accept();
01553         return;
01554     }
01555     viewport()->unsetCursor();
01556 }
01557
01558 if (m_hasData)
01559 {
01560     const QPointF scenePos = mapToScene(event->pos());
01561     if (chart()->plotArea().contains(scenePos))
01562     {
01563         const double dataX = chart()->mapToValue(scenePos, m_series).x();
01564         m_crosshairX = std::clamp(dataX, 0.0, m_origMaxX);
01565         m_crosshairY = chart()->mapToValue(scenePos, m_series).y();
01566
01567         if (m_intervalSelecting)
01568         {
01569             m_intervalEndMin = m_crosshairX;
01570             scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01571             event->accept();
01572             return;
01573         }
01574
01575         emit crosshairMoved(m_crosshairX);
01576         scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01577
01578         if (!m_tooltipPoints.empty())
01579         {
01580             const auto it = std::lower_bound(
01581                 m_tooltipPoints.begin(),
01582                 m_tooltipPoints.end(),
01583                 m_crosshairX,
01584                 [](const QPointF& point, double x) { return point.x() < x; });
01585
01586             size_t index = 0;
01587             if (it == m_tooltipPoints.end())
01588                 index = m_tooltipPoints.size() - 1;
01589             else
01590                 index = static_cast<size_t>(std::distance(m_tooltipPoints.begin(), it));
01591
01592             if (index > 0 && index < m_tooltipPoints.size())
01593             {
01594                 const double currDist = std::abs(m_tooltipPoints[index].x() - m_crosshairX);
01595                 const double prevDist = std::abs(m_tooltipPoints[index - 1].x() - m_crosshairX);
01596                 if (prevDist < currDist)
01597                     --index;
01598             }
01599
01600             const QPointF& point = m_tooltipPoints[index];
01601             m_crosshairY = point.y();
01602             const int totalSec = static_cast<int>(point.x() * 60.0);
01603             const int h = totalSec / 3600;

```



```

01604         const int m = (totalSec % 3600) / 60;
01605         const int s = totalSec % 60;
01606         const double visibleMinutes = m_axisX->max() - m_axisX->min();
01607         const bool showSeconds = visibleMinutes <= 5.0;
01608
01609         QString timeStr;
01610         if (h > 0)
01611             timeStr = showSeconds
01612                 ? QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10,
01613                     QChar('0'))
01614                 : QString("%1:%2").arg(h).arg(m, 2, 10, QChar('0'));
01615         else
01616             timeStr = showSeconds
01617                 ? QString("%1:%2").arg(m).arg(s, 2, 10, QChar('0'))
01618                 : QString("%1 min").arg(m);
01619
01620         size_t nearestRecordIndex = 0;
01621         if (!m_rideData.records.empty())
01622         {
01623             const auto rit = std::lower_bound(
01624                 m_rideData.records.begin(),
01625                 m_rideData.records.end(),
01626                 point.x() * 60.0,
01627                 [](const RideRecord& rec, double sec) { return rec.elapsedSeconds < sec; });
01628             if (rit == m_rideData.records.end())
01629                 nearestRecordIndex = m_rideData.records.size() - 1;
01630             else
01631                 nearestRecordIndex =
01632                     static_cast<size_t>(std::distance(m_rideData.records.begin(), rit));
01633             if (nearestRecordIndex > 0 && nearestRecordIndex < m_rideData.records.size())
01634             {
01635                 const double currDist =
01636                     std::abs(m_rideData.records[nearestRecordIndex].elapsedSeconds - point.x() * 60.0);
01637                 const double prevDist = std::abs(m_rideData.records[nearestRecordIndex -
01638                     1].elapsedSeconds - point.x() * 60.0);
01639                 if (prevDist < currDist)
01640                     --nearestRecordIndex;
01641             }
01642         }
01643
01644         const RideRecord& rec = m_rideData.records[nearestRecordIndex];
01645
01646         double rolling30Power = 0.0;
01647         int rollingCount = 0;
01648         const size_t startIdx = nearestRecordIndex > 30 ? (nearestRecordIndex - 30) : 0;
01649         for (size_t i = startIdx; i <= nearestRecordIndex; ++i)
01650         {
01651             const RideRecord& rp = m_rideData.records[i];
01652             if (rp.hasPower && rp.power > 0.0)
01653             {
01654                 rolling30Power += rp.power;
01655                 ++rollingCount;
01656             }
01657         }
01658         if (rollingCount > 0)
01659             rolling30Power /= static_cast<double>(rollingCount);
01660
01661         QColor zoneColor = ColorProvider::colorForPower(rec.power, m_colorContext.ftp);
01662         const std::vector<Zone> powerZones = ZoneCalculator::powerZones(m_colorContext.ftp);
01663         QString zoneLabel = "-";
01664         for (size_t zi = 0; zi < powerZones.size(); ++zi)
01665         {
01666             const Zone& z = powerZones[zi];
01667             if (rec.power >= z.minValue && rec.power < z.maxValue)
01668             {
01669                 zoneLabel = QString("Z%1 %2").arg(static_cast<int>(zi) + 1).arg(z.name);
01670                 break;
01671             }
01672         }
01673
01674         QString tooltip;
01675         tooltip += QString("Time: %1").arg(timeStr);
01676         tooltip += QString("\nPower: %1 W").arg(rec.hasPower ? QString::number(rec.power, 'f',
01677             0) : "-");
01678         tooltip += QString("\n30s Power: %1 W").arg(rollingCount > 0 ?
01679             QString::number(rolling30Power, 'f', 0) : "-");
01680         tooltip += QString("\nHR: %1 bpm").arg(rec.hasHeartRate ?
01681             QString::number(rec.heartRate, 'f', 0) : "-");
01682         tooltip += QString("\nCadence: %1 rpm").arg(rec.hasCadence ?
01683             QString::number(rec.cadence, 'f', 0) : "-");
01684         tooltip += QString("\nSpeed: %1 km/h").arg(rec.hasSpeed ? QString::number(rec.speed,
01685             'f', 1) : "-");
01686         tooltip += QString("\nAltitude: %1 m").arg(rec.hasAltitude ?
01687             QString::number(rec.altitude, 'f', 0) : "-");
01688         tooltip += QString("\nZone: %1").arg(zoneLabel);
01689         tooltip += QString("\nColor: %1").arg(zoneColor.name());
01690

```

```

01681         if (m_metric == Metric::Power && m_effectivePowerSmoothingSeconds > 0)
01682             tooltip += QString("\nSmoothed: %1 W").arg(m_tooltipDisplayValues[index], 0, 'f',
01683     );
01684
01685         QToolTip::showText(QCursor::pos(), tooltip, this);
01686     }
01687 }
01688 QChartView::mouseMoveEvent(event);
01689 }
01690
01691 void RideChartWidget::leaveEvent(QEvent* event)
01692 {
01693     if (m_newClimbCreating && !m_newClimbDragging)
01694     {
01695         m_newClimbCreating = false;
01696         m_newClimbStartSec = -1.0;
01697         m_newClimbCurrentSec = -1.0;
01698     }
01699
01700     viewport()->unsetCursor();
01701     m_crosshairX = -1.0;
01702     m_crosshairY = std::numeric_limits<double>::quiet_NaN();
01703     QToolTip::hideText();
01704     emit crosshairMoved(-1.0);
01705     scene()->invalidate(chart()->plotArea(), QGraphicsScene::ForegroundLayer);
01706     QChartView::leaveEvent(event);
01707 }
01708
01709 void RideChartWidget::drawForeground(QPainter* painter, const QRectF&)
01710 {
01711     if (!m_hasData) return;
01712
01713     const QRectF plotArea = chart()->plotArea();
01714     painter->save();
01715
01716     // --- New-climb creation overlay (line + box) -----
01717     if (m_newClimbCreating && m_newClimbStartSec >= 0.0)
01718     {
01719         const double x0sec = std::min(m_newClimbStartSec, m_newClimbCurrentSec);
01720         const double x1sec = std::max(m_newClimbStartSec, m_newClimbCurrentSec);
01721         const QPointF p0 = chart()->mapToPosition(QPointF(x0sec / 60.0, m_axisY->min()), m_series);
01722         const QPointF p1 = chart()->mapToPosition(QPointF(x1sec / 60.0, m_axisY->min()), m_series);
01723         const double px0 = std::clamp(p0.x(), plotArea.left(), plotArea.right());
01724         const double px1 = std::clamp(p1.x(), plotArea.left(), plotArea.right());
01725
01726         painter->setPen(QPen(QColor("#16a34a"), 2.0, Qt::SolidLine));
01727
01728         constexpr double kBoxThreshold = 4.0; // px -- show box once wider than this
01729         if (px1 - px0 < kBoxThreshold)
01730         {
01731             // Just a vertical line at anchor
01732             painter->drawLine(QLineF(px0, plotArea.top(), px0, plotArea.bottom()));
01733         }
01734         else
01735         {
01736             QColor fill("#16a34a");
01737             fill.setAlpha(45);
01738             const QRectF box(QPointF(px0, plotArea.top()), QPointF(px1, plotArea.bottom()));
01739             painter->fillRect(box, fill);
01740             painter->drawRect(box);
01741         }
01742
01743         painter->restore();
01744         return; // skip crosshair while creating
01745     }
01746     // -----
01747
01748     if (m_crosshairX < 0.0)
01749     {
01750         painter->restore();
01751         return;
01752     }
01753
01754     const QPointF chartPos =
01755         chart()->mapToPosition(QPointF(m_crosshairX, m_axisY->min()), m_series);
01756
01757     if (chartPos.x() < plotArea.left() || chartPos.x() > plotArea.right())
01758     {
01759         painter->restore();
01760         return;
01761     }
01762
01763     painter->setPen(QPen(QColor(220, 220, 220, 200), 1, Qt::DashLine));
01764     painter->drawLine(QLineF(chartPos.x(), plotArea.top(),
01765                             chartPos.x(), plotArea.bottom()));
01766

```

```

01767     if (!std::isnan(m_crosshairY))
01768     {
01769         const QPointF yPos = chart()->mapToPosition(QPointF(m_axisX->min(), m_crosshairY), m_series);
01770         if (yPos.y() >= plotArea.top() && yPos.y() <= plotArea.bottom())
01771             painter->drawLine(QLineF(plotArea.left(), yPos.y(), plotArea.right(), yPos.y()));
01772     }
01773
01774     if (m_intervalSelecting && m_intervalStartMin >= 0.0 && m_intervalEndMin >= 0.0)
01775     {
01776         const double x0 = std::min(m_intervalStartMin, m_intervalEndMin);
01777         const double x1 = std::max(m_intervalStartMin, m_intervalEndMin);
01778         const QPointF p0 = chart()->mapToPosition(QPointF(x0, m_axisY->min()), m_series);
01779         const QPointF p1 = chart()->mapToPosition(QPointF(x1, m_axisY->min()), m_series);
01780         const QRectF selectionRect(
01781             QPointF(std::max(plotArea.left(), p0.x()), plotArea.top()),
01782             QPointF(std::min(plotArea.right(), p1.x()), plotArea.bottom()));
01783
01784         QColor fill("#0284c7");
01785         fill.setAlpha(30);
01786         painter->fillRect(selectionRect.normalized(), fill);
01787         painter->setPen(QPen(QColor("#0284c7"), 1.0, Qt::DashLine));
01788         painter->drawRect(selectionRect.normalized());
01789     }
01790
01791     if (m_selectedClimbIndex >= 0 && m_selectedClimbIndex < static_cast<int>(m_climbs.size()))
01792     {
01793         const Climb& climb = m_climbs[static_cast<size_t>(m_selectedClimbIndex)];
01794         const QPointF p0 = chart()->mapToPosition(QPointF(climb.startSeconds / 60.0, m_axisY->min()),
01795             m_series);
01796         const QPointF p1 = chart()->mapToPosition(QPointF(climb.endSeconds / 60.0, m_axisY->min()),
01797             m_series);
01798         if (p1.x() >= plotArea.left() && p0.x() <= plotArea.right())
01799         {
01800             painter->setPen(QPen(QColor("#16a34a"), 1.5, Qt::SolidLine));
01801             painter->setBrush(Qt::NoBrush);
01802             painter->drawLine(QLineF(p0.x(), plotArea.top(), p0.x(), plotArea.bottom()));
01803             painter->drawLine(QLineF(p1.x(), plotArea.top(), p1.x(), plotArea.bottom()));
01804
01805             if (m_climbEditingEnabled)
01806             {
01807                 painter->setBrush(QColor("#15803d"));
01808                 painter->setPen(QPen(Qt::white, 1.0));
01809                 painter->drawEllipse(QPointF(p0.x(), plotArea.top() + climbHandleCenterOffsetYPx()),
01810                     climbHandleRadiusPx(), climbHandleRadiusPx());
01811                 painter->drawEllipse(QPointF(p1.x(), plotArea.top() + climbHandleCenterOffsetYPx()),
01812                     climbHandleRadiusPx(), climbHandleRadiusPx());
01813             }
01814         }
01815     }
01816     painter->restore();
01817 }

```

## 8.41 RideChartWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QtCharts/QChartView>
00007
00008 #include <limits>
00009 #include <unordered_map>
00010 #include <vector>
00011
00012 #include "analysis/IntervalDetector.h"
00013 #include "analysis/ClimbMetrics.h"
00014 #include "charts/Metric.h"
00015 #include "core/zones/ZoneDefinition.h"
00016 #include "fit/RideData.h"
00017
00018 class QLineSeries;
00019 class QAreaSeries;
00020 class QValueAxis;
00021 class QCategoryAxis;
00022
00029 class RideChartWidget : public QChartView
00030 {
00031     Q_OBJECT
00032
00033 public:
00039     explicit RideChartWidget(Metric metric, QWidget* parent = nullptr);
00040
00047     void setRideData(const RideData& rideData,

```

```

00048         ColorMetric colorMetric = ColorMetric::None,
00049         const ColorContext& colorContext = {}));
00050
00055     void setPowerSmoothingSeconds(int seconds);
00056
00061     void setAutoSmoothingEnabled(bool enabled);
00062
00067     void setIntervals(const std::vector<Interval>& intervals);
00068
00073     void setClimbs(const std::vector<Climb>& climbs);
00074
00080     void setMetricOverlay(ColorMetric metric, bool enabled);
00081
00086     void setSelectedClimbIndex(int index);
00087
00092     void setClimbEditingEnabled(bool enabled);
00093
00097     void clearChart();
00098
00103     bool hasData() const { return m_hasData; }
00104
00109     double visibleRangeStartMinutes() const;
00111     double visibleRangeEndMinutes() const;
00112
00113 public slots:
00117     void setXRange(double min, double max);
00118
00121     void setCrosshair(double xMinutes);
00122
00124     void resetZoom();
00125
00127     void fitToData();
00128
00129 signals:
00133     void xRangeChanged(double min, double max);
00134
00138     void visibleRangeChanged(double startMinutes, double endMinutes);
00139
00142     void crosshairMoved(double xMinutes);
00143
00147     void intervalSelectionRequested(double startSeconds, double endSeconds);
00148
00154     void climbBoundaryEdited(
00155         double oldStartSeconds,
00156         double oldEndSeconds,
00157         double newStartSeconds,
00158         double newEndSeconds);
00159
00163     void newClimbRequested(double startSeconds, double endSeconds);
00164
00167     void climbClicked(int climbIndex);
00168
00169 protected:
00171     void wheelEvent(QWheelEvent* event) override;
00172
00174     void mousePressEvent(QMouseEvent* event) override;
00175
00177     void mouseReleaseEvent(QMouseEvent* event) override;
00178
00180     void mouseDoubleClickEvent(QMouseEvent* event) override;
00181
00183     void mouseMoveEvent(QMouseEvent* event) override;
00184
00186     void leaveEvent(QEvent* event) override;
00187
00189     void drawForeground(QPainter* painter, const QRectF& rect) override;
00190
00192     void resizeEvent(QResizeEvent* event) override;
00193
00194 private:
00195     void rebuildChart(bool preserveXRange = true);
00196     std::vector<double> computeSmoothedSeries(Metric metric, int seconds) const;
00197     std::vector<double> computeSmoothedPowerSeries(int seconds) const;
00198     std::vector<int> buildSampleIndices(const std::vector<double>& values, int targetPointBudget)
00199     const;
00199     void clearBackgroundSeries();
00200     void addZoneBackgroundBands(double minY, double maxY, double maxXMinutes);
00201     void addMetricOverlayBands(double minY, double maxY);
00202     void addIntervalBackgroundBands(double minY, double maxY);
00203     void addClimbBackgroundBands(double minY, double maxY);
00204     void updateXTicks(double minVal, double maxVal);
00205     void updateYAxis(); // recalculates nice range + tick count from full data
00206     void updateYAxisForVisibleRange(double xMinMin, double xMaxMin); // rescale to visible window
00207
00208     Metric m_metric;
00209     QLineSeries* m_series = nullptr;
00210     std::vector<QLineSeries*> m_rawSeries;

```

```

00211     std::vector<QLineSeries*> m_colorSeries;
00212     std::vector<QLineSeries*> m_overlaySeries;
00213     std::vector<QLineSeries*> m_referenceSeries;
00214     std::vector<QAreaSeries*> m_backgroundSeries;
00215     QValueAxis*      m_axisX      = nullptr;
00216     QCategoryAxis*   m_axisXDisplay = nullptr;
00217     QValueAxis*      m_axisY      = nullptr;
00218     QValueAxis*      m_axisYOverlay = nullptr;
00219     RideData         m_rideData;
00220     std::vector<Interval> m_intervals;
00221     std::vector<Climb> m_climbs;
00222     bool              m_metricOverlayEnabled = false;
00223     ColorMetric       m_metricOverlayMetric = ColorMetric::None;
00224     ColorMetric       m_colorMetric       = ColorMetric::None;
00225     ColorContext      m_colorContext;
00226
00227     // Cache for smoothed metric series: keyed by (metric * 1000 + smoothing_seconds)
00228     // Avoids recomputation when chart is redrawn or multiple overlays use same smoothing
00229     mutable std::unordered_map<int, std::vector<double>> m_smoothedSeriesCache;
00230     std::vector<QPointF> m_tooltipPoints;
00231     std::vector<double> m_tooltipRawValues;
00232     std::vector<double> m_tooltipDisplayValues;
00233     int                 m_requestedPowerSmoothingSeconds = 0;
00234     int                 m_effectivePowerSmoothingSeconds = 0;
00235     bool                m_autoSmoothing = false;
00236     bool                m_isRebuilding = false;
00237     bool                m_syncing      = false;
00238     bool                m_hasData      = false;
00239
00240     // Cache for Y-axis range calculations during zoom to avoid redundant recalculation
00241     mutable struct
00242     {
00243         double cachedMinX = -1.0;
00244         double cachedMaxX = -1.0;
00245         double cachedAxisMin = 0.0;
00246         double cachedAxisMax = 1.0;
00247         bool valid = false;
00248     } m_yAxisRangeCache;
00249     double m_origMaxX = 1.0;
00250     double m_crosshairX = -1.0;
00251     double m_crosshairY = std::numeric_limits<double>::quiet_NaN();
00252     bool m_intervalSelecting = false;
00253     double m_intervalStartMin = -1.0;
00254     double m_intervalEndMin = -1.0;
00255     bool m_climbEditingEnabled = false;
00256     int m_selectedClimbIndex = -1;
00257     enum class ClimbHandleDrag
00258     {
00259         None,
00260         Start,
00261         End
00262     };
00263     ClimbHandleDrag m_climbHandleDrag = ClimbHandleDrag::None;
00264     double m_dragOriginalClimbStartSec = -1.0;
00265     double m_dragOriginalClimbEndSec = -1.0;
00266     bool m_newClimbCreating = false;
00267     bool m_newClimbDragging = false;
00268     double m_newClimbStartSec = -1.0;
00269     double m_newClimbCurrentSec = -1.0;
00270     double m_dataMinY = 0.0; // raw min from ride data
00271     double m_dataMaxY = 1.0; // raw max from ride data
00272 };

```

## 8.42 RideChartSelectionManager.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "RideChartSelectionManager.h"
00004
00005 #include <QtCharts/QChart>
00006 #include <QtCharts/QLineSeries>
00007 #include <QtCharts/QValueAxis>
00008
00009 namespace
00010 {
00011     constexpr double kClimbHandleCenterOffsetYPx = 10.0;
00012     constexpr double kClimbHandleRadiusPx = 8.0;
00013     constexpr double kClimbHandleGrabRadiusPx = 24.0;
00014 }
00015
00016 namespace RideChartSelectionManager
00017 {
00018     ClimbHandleHitResult hitTestClimbHandles(
00019         QChart* chart,

```

```

00020     QLineSeries* series,
00021     const QValueAxis* axisY,
00022     const Climb& climb,
00023     const QPointF& scenePos)
00024 {
00025     if (!chart || !series || !axisY)
00026         return {};
00027
00028     const QRectF plotArea = chart->plotArea();
00029     if (!plotArea.contains(scenePos))
00030         return {};
00031
00032     QPointF startPos = chart->mapToPosition(QPointF(climb.startSeconds / 60.0, axisY->min()), series);
00033     QPointF endPos = chart->mapToPosition(QPointF(climb.endSeconds / 60.0, axisY->min()), series);
00034     startPos.setY(plotArea.top() + kClimbHandleCenterOffsetYPx);
00035     endPos.setY(plotArea.top() + kClimbHandleCenterOffsetYPx);
00036
00037     const double dxStart = scenePos.x() - startPos.x();
00038     const double dyStart = scenePos.y() - startPos.y();
00039     const double dxEnd = scenePos.x() - endPos.x();
00040     const double dyEnd = scenePos.y() - endPos.y();
00041     const double distStartSq = dxStart * dxStart + dyStart * dyStart;
00042     const double distEndSq = dxEnd * dxEnd + dyEnd * dyEnd;
00043     const double radiusSq = kClimbHandleGrabRadiusPx * kClimbHandleGrabRadiusPx;
00044
00045     if (distStartSq <= radiusSq || distEndSq <= radiusSq)
00046         return { true, distStartSq <= distEndSq };
00047
00048     return {};
00049 }
00050
00051 double climbHandleCenterOffsetYPx()
00052 {
00053     return kClimbHandleCenterOffsetYPx;
00054 }
00055
00056 double climbHandleRadiusPx()
00057 {
00058     return kClimbHandleRadiusPx;
00059 }
00060 }

```

## 8.43 RideChartSelectionManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QPointF>
00006
00007 #include "analysis/ClimbMetrics.h"
00008
00009 class QChart;
00010 class QLineSeries;
00011 class QValueAxis;
00012
00013 namespace RideChartSelectionManager
00014 {
00015     struct ClimbHandleHitResult
00016     {
00017         bool hit = false;
00018         bool startHandle = true;
00019     };
00020
00021     ClimbHandleHitResult hitTestClimbHandles(
00022         QChart* chart,
00023         QLineSeries* series,
00024         const QValueAxis* axisY,
00025         const Climb& climb,
00026         const QPointF& scenePos);
00027
00028     double climbHandleCenterOffsetYPx();
00029     double climbHandleRadiusPx();
00030 }

```

## 8.44 ActivityAnalysisFlags.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once

```

```

00005
00012 namespace ActivityAnalysisFlags
00013 {
00017     constexpr int None          = 0;
00018
00022     constexpr int HasClimbs     = 1 « 0;
00023
00027     constexpr int HasIntervals = 1 « 1;
00028
00032     constexpr int HasPowerCurve = 1 « 2;
00033
00037     constexpr int HasHistogram = 1 « 3;
00038
00042     constexpr int HasVideo     = 1 « 4;
00043 }

```

## 8.45 AnalysisVersions.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00012 namespace AnalysisVersions
00013 {
00017     constexpr int ClimbDetection = 1;
00018
00022     constexpr int IntervalDetection = 1;
00023 }

```

## 8.46 AppSettings.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AppSettings.h"
00005
00006 #include <QSettings>
00007
00008 AppSettings& AppSettings::instance()
00009 {
00010     static AppSettings settings;
00011     return settings;
00012 }
00013
00014 DateFormat AppSettings::dateFormat() const
00015 {
00016     QSettings settings("Fitlyzer", "FitlyzerC");
00017     const int rawValue = settings.value(dateFormatKey(),
00018         static_cast<int>(DateFormat::DD_MM_YYYY)).toInt();
00019     return sanitizeDateFormat(rawValue);
00020 }
00021 void AppSettings::setDateFormat(DateFormat format)
00022 {
00023     QSettings settings("Fitlyzer", "FitlyzerC");
00024     settings.setValue(dateFormatKey(), static_cast<int>(format));
00025 }
00026
00027 QString AppSettings::dateFormatKey()
00028 {
00029     return QStringLiteral("general/dateFormat");
00030 }
00031
00032 DateFormat AppSettings::sanitizeDateFormat(int rawValue)
00033 {
00034     switch (rawValue)
00035     {
00036     case static_cast<int>(DateFormat::DD_MM_YYYY):
00037         return DateFormat::DD_MM_YYYY;
00038     case static_cast<int>(DateFormat::YYYY_MM_DD):
00039         return DateFormat::YYYY_MM_DD;
00040     case static_cast<int>(DateFormat::DD_DOT_MM_DOT_YYYY):
00041         return DateFormat::DD_DOT_MM_DOT_YYYY;
00042     case static_cast<int>(DateFormat::MM_DD_YYYY):
00043         return DateFormat::MM_DD_YYYY;
00044     default:
00045         return DateFormat::DD_MM_YYYY;
00046     }
00047 }
00048

```

```

00049 int AppSettings::tileCacheSize() const
00050 {
00051     QSettings settings("Fitlyzer", "FitlyzerC");
00052     const int stored = settings.value(tileCacheSizeKey(), 512).toInt();
00053     // Clamp to known-good values in case of corruption.
00054     const int valid[] = {128, 256, 512, 1024, 2048};
00055     for (int v : valid)
00056         if (stored == v)
00057             return v;
00058     return 512;
00059 }
00060
00061 void AppSettings::setTileCacheSize(int tiles)
00062 {
00063     QSettings settings("Fitlyzer", "FitlyzerC");
00064     settings.setValue(tileCacheSizeKey(), tiles);
00065 }
00066
00067 QString AppSettings::tileCacheSizeKey()
00068 {
00069     return QStringLiteral("maps/tileCacheSize");
00070 }

```

## 8.47 AppSettings.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 class QSettings;
00009
00013 enum class DateFormat
00014 {
00015     DD_MM_YYYY = 0,
00016     YYYY_MM_DD,
00017     DD_DOT_MM_DOT_YYYY,
00018     MM_DD_YYYY
00019 };
00020
00026 class AppSettings
00027 {
00028 public:
00033     static AppSettings& instance();
00034
00039     DateFormat dateFormat() const;
00040
00045     void setDateFormat(DateFormat format);
00046
00051     int tileCacheSize() const;
00052
00057     void setTileCacheSize(int tiles);
00058
00059 private:
00060     AppSettings() = default;
00061
00062     static QString dateFormatKey();
00063     static DateFormat sanitizeDateFormat(int rawValue);
00064     static QString tileCacheSizeKey();
00065 };

```

## 8.48 DateFormatter.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DateFormatter.h"
00005
00006 #include "AppSettings.h"
00007
00008 #include <QStringList>
00009
00010 namespace
00011 {
00012     QString datePatternForFormat(DateFormat format)
00013     {
00014         switch (format)
00015         {
00016             case DateFormat::DD_MM_YYYY:

```



```

00017         return QStringLiteral("dd-MM-yyyy");
00018     case DateFormat::YYYY_MM_DD:
00019         return QStringLiteral("yyyy-MM-dd");
00020     case DateFormat::DD_DOT_MM_DOT_YYYY:
00021         return QStringLiteral("dd.MM.yyyy");
00022     case DateFormat::MM_DD_YYYY:
00023         return QStringLiteral("MM/dd/yyyy");
00024     }
00025
00026     return QStringLiteral("dd-MM-yyyy");
00027 }
00028 }
00029
00030 QString DateFormatter::formatDate(const QDate& date)
00031 {
00032     if (!date.isValid())
00033         return QStringLiteral("-");
00034
00035     return date.toString(datePatternForFormat(AppSettings::instance().dateFormat()));
00036 }
00037
00038 QString DateFormatter::formatDateTime(const QDateTime& dateTime)
00039 {
00040     if (!dateTime.isValid())
00041         return QStringLiteral("-");
00042
00043     return QStringLiteral("%1 %2")
00044         .arg(formatDate(dateTime.date()), dateTime.time().toString(QStringLiteral("HH:mm")));
00045 }
00046
00047 QString DateFormatter::toIsoDate(const QDate& date)
00048 {
00049     return date.toString(Qt::ISODate);
00050 }
00051
00052 QString DateFormatter::toIsoDateTime(const QDateTime& dateTime)
00053 {
00054     return dateTime.toString(Qt::ISODate);
00055 }
00056
00057 QDate DateFormatter::parseDate(const QString& text)
00058 {
00059     const QString trimmed = text.trimmed();
00060     if (trimmed.isEmpty())
00061         return QDate();
00062
00063     const QString activePattern = datePatternForFormat(AppSettings::instance().dateFormat());
00064
00065     QDate parsed = QDate::fromString(trimmed, activePattern);
00066     if (parsed.isValid())
00067         return parsed;
00068
00069     parsed = QDate::fromString(trimmed, Qt::ISODate);
00070     if (parsed.isValid())
00071         return parsed;
00072
00073     // Accept dates that may have been entered using any supported format.
00074     static const QStringList allPatterns = {
00075         QStringLiteral("dd-MM-yyyy"),
00076         QStringLiteral("yyyy-MM-dd"),
00077         QStringLiteral("dd.MM.yyyy"),
00078         QStringLiteral("MM/dd/yyyy")
00079     };
00080
00081     for (const QString& pattern : allPatterns)
00082     {
00083         parsed = QDate::fromString(trimmed, pattern);
00084         if (parsed.isValid())
00085             return parsed;
00086     }
00087
00088     return QDate();
00089 }
00090
00091 QString DateFormatter::qtDatePattern()
00092 {
00093     return datePatternForFormat(AppSettings::instance().dateFormat());
00094 }

```

## 8.49 DateFormatter.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QDateTime>
00008 #include <QString>
00009
00013 class DateFormatter
00014 {
00015 public:
00021     static QString formatDate(const QDate& date);
00022
00028     static QString formatDateTime(const QDateTime& dateTime);
00029
00035     static QString toIsoDate(const QDate& date);
00036
00042     static QString toIsoDateTime(const QDateTime& dateTime);
00043
00049     static QDate parseDate(const QString& text);
00050
00055     static QString qtDatePattern();
00056 };

```

## 8.50 ClimbEditCommand.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbEditCommand.h"
00005
00006 #include <QSqlDatabase>
00007
00008 ClimbEditCommand::ClimbEditCommand(const ClimbRecord& before,
00009                                     const ClimbRecord& after,
00010                                     const QString& dbConnectionName,
00011                                     std::function<void()> refreshCallback)
00012     : m_before(before)
00013     , m_after(after)
00014     , m_connName(dbConnectionName)
00015     , m_refresh(std::move(refreshCallback))
00016 {}
00017
00018 void ClimbEditCommand::undo()
00019 {
00020     apply(m_before);
00021 }
00022
00023 void ClimbEditCommand::redo()
00024 {
00025     apply(m_after);
00026 }
00027
00028 QString ClimbEditCommand::description() const
00029 {
00030     return QStringLiteral("Edit Climb Boundary");
00031 }
00032
00033 void ClimbEditCommand::apply(const ClimbRecord& record)
00034 {
00035     auto db = QSqlDatabase::database(m_connName);
00036     if (db.isValid() && db.isOpen())
00037     {
00038         ClimbRepository repo(db);
00039         repo.updateClimb(record);
00040     }
00041     if (m_refresh)
00042         m_refresh();
00043 }

```

## 8.51 ClimbEditCommand.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <functional>
00007
00008 #include <QSqlDatabase>
00009
00010 #include "UndoCommand.h"
00011 #include "database/ClimbRepository.h"

```

```

00012
00019 class ClimbEditCommand final : public UndoCommand
00020 {
00021 public:
00029     ClimbEditCommand(const ClimbRecord& before,
00030                       const ClimbRecord& after,
00031                       const QString& dbConnectionName,
00032                       std::function<void()> refreshCallback);
00033
00034     void undo() override;
00035     void redo() override;
00036     QString description() const override;
00037
00038 private:
00039     void apply(const ClimbRecord& record);
00040
00041     ClimbRecord m_before;
00042     ClimbRecord m_after;
00043     QString m_connName;
00044     std::function<void()> m_refresh;
00045 };

```

## 8.52 UndoCommand.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00013 class UndoCommand
00014 {
00015 public:
00016     virtual ~UndoCommand() = default;
00017
00021     virtual void undo() = 0;
00022
00026     virtual void redo() = 0;
00027
00032     virtual QString description() const { return QStringLiteral("Edit"); }
00033 };

```

## 8.53 UndoManager.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "UndoManager.h"
00005
00006 UndoManager::UndoManager(int maxStackSize, QObject* parent)
00007     : QObject(parent)
00008     , m_maxStackSize(maxStackSize)
00009 {}
00010
00011 void UndoManager::push(std::unique_ptr<UndoCommand> cmd)
00012 {
00013     m_redoStack.clear();
00014     m_undoStack.push_back(std::move(cmd));
00015
00016     // Trim oldest entries when the stack exceeds the limit.
00017     // std::deque::pop_front() is O(1), whereas vector::erase(begin()) is O(n).
00018     while (static_cast<int>(m_undoStack.size()) > m_maxStackSize)
00019         m_undoStack.pop_front();
00020
00021     notify();
00022 }
00023
00024 void UndoManager::clear()
00025 {
00026     m_undoStack.clear();
00027     m_redoStack.clear();
00028     notify();
00029 }
00030
00031 bool UndoManager::canUndo() const { return !m_undoStack.empty(); }
00032 bool UndoManager::canRedo() const { return !m_redoStack.empty(); }
00033
00034 QString UndoManager::undoDescription() const
00035 {
00036     return m_undoStack.empty() ? QString() : m_undoStack.back()->description();

```

```

00037 }
00038
00039 QString UndoManager::redoDescription() const
00040 {
00041     return m_redoStack.empty() ? QString() : m_redoStack.back()->description();
00042 }
00043
00044 void UndoManager::undo()
00045 {
00046     if (m_undoStack.empty())
00047         return;
00048
00049     auto cmd = std::move(m_undoStack.back());
00050     m_undoStack.pop_back();
00051     cmd->undo();
00052     m_redoStack.push_back(std::move(cmd));
00053     notify();
00054 }
00055
00056 void UndoManager::redo()
00057 {
00058     if (m_redoStack.empty())
00059         return;
00060
00061     auto cmd = std::move(m_redoStack.back());
00062     m_redoStack.pop_back();
00063     cmd->redo();
00064     m_undoStack.push_back(std::move(cmd));
00065     notify();
00066 }
00067
00068 void UndoManager::notify()
00069 {
00070     emit canUndoChanged(canUndo());
00071     emit canRedoChanged(canRedo());
00072     emit undoDescriptionChanged(undoDescription());
00073     emit redoDescriptionChanged(redoDescription());
00074 }

```

## 8.54 UndoManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <deque>
00007 #include <memory>
00008
00009 #include <QObject>
00010 #include <QString>
00011
00012 #include "UndoCommand.h"
00013
00020 class UndoManager : public QObject
00021 {
00022     Q_OBJECT
00023 public:
00029     explicit UndoManager(int maxStackSize = 50, QObject* parent = nullptr);
00030
00037     void push(std::unique_ptr<UndoCommand> cmd);
00038
00042     void clear();
00043
00048     bool canUndo() const;
00049
00054     bool canRedo() const;
00055
00060     QString undoDescription() const;
00061
00066     QString redoDescription() const;
00067
00068 public slots:
00072     void undo();
00073
00077     void redo();
00078
00079 signals:
00083     void canUndoChanged(bool canUndo);
00084
00088     void canRedoChanged(bool canRedo);
00089
00093     void undoDescriptionChanged(const QString& desc);
00094

```

```

00098     void redoDescriptionChanged(const QString& desc);
00099
00100 private:
00101     void notify();
00102
00103     int m_maxStackSize;
00104     std::deque<std::unique_ptr<UndoCommand> m_undoStack;
00105     std::deque<std::unique_ptr<UndoCommand> m_redoStack;
00106 };

```

## 8.55 ColorProvider.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ColorProvider.h"
00005
00006 #include "ZoneCalculator.h"
00007
00008 #include <algorithm>
00009
00010 namespace {
00011
00012 QColor interpolateColor(const QColor& a, const QColor& b, double t)
00013 {
00014     const double clamped = std::clamp(t, 0.0, 1.0);
00015     return QColor(
00016         static_cast<int>(a.red() + (b.red() - a.red()) * clamped),
00017         static_cast<int>(a.green() + (b.green() - a.green()) * clamped),
00018         static_cast<int>(a.blue() + (b.blue() - a.blue()) * clamped));
00019 }
00020
00021 }
00022
00023 QColor ColorProvider::colorForMetric(ColorMetric metric,
00024                                     double value,
00025                                     const ColorContext& context)
00026 {
00027     switch (metric)
00028     {
00029         case ColorMetric::Power:
00030             return colorForPower(value, context.ftp);
00031         case ColorMetric::HeartRate:
00032             return colorForHeartRate(value,
00033                                     context.heartRateMax,
00034                                     context.heartRateRest,
00035                                     context.thresholdHeartRate);
00036         case ColorMetric::Cadence:
00037             return colorForCadence(value);
00038         case ColorMetric::Speed:
00039             return colorForSpeed(value);
00040         case ColorMetric::Altitude:
00041             return colorForAltitude(value, context.altitudeMin, context.altitudeMax);
00042         case ColorMetric::Gradient:
00043         case ColorMetric::None:
00044             return QColor("#94a3b8");
00045     }
00046
00047     return QColor("#94a3b8");
00048 }
00049
00050 QColor ColorProvider::colorForPower(double watts, int ftp)
00051 {
00052     return colorForValue(watts, ZoneCalculator::powerZones(ftp));
00053 }
00054
00055 QColor ColorProvider::colorForHeartRate(double bpm,
00056                                         int heartRateMax,
00057                                         int heartRateRest,
00058                                         int thresholdHeartRate)
00059 {
00060     return colorForValue(
00061         bpm,
00062         ZoneCalculator::heartRateZones(heartRateMax, heartRateRest, thresholdHeartRate));
00063 }
00064
00065 QColor ColorProvider::colorForCadence(double cadence)
00066 {
00067     return colorForValue(cadence, ZoneCalculator::cadenceZones());
00068 }
00069
00070 QColor ColorProvider::colorForSpeed(double speedKmh)
00071 {
00072     return colorForValue(speedKmh, ZoneCalculator::speedZones());
00073 }

```

```

00073 }
00074
00075 QColor ColorProvider::colorForAltitude(double altitude,
00076                                       double minAltitude,
00077                                       double maxAltitude)
00078 {
00079     if (maxAltitude <= minAltitude)
00080         return QColor("#6fa8dc");
00081
00082     const double t = std::clamp((altitude - minAltitude) / (maxAltitude - minAltitude), 0.0, 1.0);
00083     if (t < 0.33)
00084         return interpolateColor(QColor("#6fa8dc"), QColor("#6aa84f"), t / 0.33);
00085     if (t < 0.66)
00086         return interpolateColor(QColor("#6aa84f"), QColor("#8d6e63"), (t - 0.33) / 0.33);
00087     return interpolateColor(QColor("#8d6e63"), QColor("#f5f5f5"), (t - 0.66) / 0.34);
00088 }
00089
00090 QColor ColorProvider::colorForValue(double value,
00091                                     const std::vector<Zone>& zones,
00092                                     const QColor& fallback)
00093 {
00094     for (const Zone& zone : zones)
00095     {
00096         if (value >= zone.minValue && value < zone.maxValue)
00097             return zone.color;
00098     }
00099
00100     return fallback;
00101 }

```

## 8.56 ColorProvider.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007
00008 #include <vector>
00009
00010 #include "ZoneDefinition.h"
00011
00012 class ColorProvider
00013 {
00014 public:
00015     static QColor colorForMetric(ColorMetric metric,
00016                                 double value,
00017                                 const ColorContext& context);
00018
00019     static QColor colorForPower(double watts, int ftp);
00020
00021     static QColor colorForHeartRate(double bpm,
00022                                     int heartRateMax,
00023                                     int heartRateRest = 60,
00024                                     int thresholdHeartRate = 0);
00025
00026     static QColor colorForCadence(double cadence);
00027
00028     static QColor colorForSpeed(double speedKmh);
00029
00030     static QColor colorForAltitude(double altitude,
00031                                    double minAltitude,
00032                                    double maxAltitude);
00033
00034     static QColor colorForValue(double value,
00035                                 const std::vector<Zone>& zones,
00036                                 const QColor& fallback = QColor("#94a3b8"));
00037 };

```

## 8.57 ZoneCalculator.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ZoneCalculator.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 namespace {

```

```

00010
00011 double valueForMetric(const RideRecord& record, ColorMetric metric)
00012 {
00013     switch (metric)
00014     {
00015         case ColorMetric::Power:      return record.power;
00016         case ColorMetric::HeartRate:  return record.heartRate;
00017         case ColorMetric::Cadence:    return record.cadence;
00018         case ColorMetric::Speed:      return record.speed;
00019         case ColorMetric::Altitude:   return record.altitude;
00020         case ColorMetric::Gradient:   return 0.0;
00021         case ColorMetric::None:       return 0.0;
00022     }
00023
00024     return 0.0;
00025 }
00026
00027 bool sampleAvailable(const RideRecord& record, ColorMetric metric)
00028 {
00029     switch (metric)
00030     {
00031         case ColorMetric::Power:      return record.hasPower;
00032         case ColorMetric::HeartRate:  return record.hasHeartRate;
00033         case ColorMetric::Cadence:    return record.hasCadence;
00034         case ColorMetric::Speed:      return record.hasSpeed;
00035         case ColorMetric::Altitude:   return record.hasAltitude;
00036         case ColorMetric::Gradient:   return false;
00037         case ColorMetric::None:       return false;
00038     }
00039
00040     return false;
00041 }
00042
00043 double sampleDurationSeconds(const std::vector<RideRecord>& records, int index)
00044 {
00045     const int size = static_cast<int>(records.size());
00046     if (size <= 1)
00047         return 1.0;
00048
00049     double dt = 1.0;
00050     if (index == 0)
00051         dt = records[1].elapsedSeconds - records[0].elapsedSeconds;
00052     else if (index == size - 1)
00053         dt = records[size - 1].elapsedSeconds - records[size - 2].elapsedSeconds;
00054     else
00055         dt = (records[index + 1].elapsedSeconds - records[index - 1].elapsedSeconds) / 2.0;
00056
00057     return std::max(0.0, dt);
00058 }
00059
00060 }
00061
00062 std::vector<Zone> ZoneCalculator::powerZones(int ftp)
00063 {
00064     const double threshold = std::max(ftp, 1);
00065     return {
00066         {"Z1 Recovery",      0.0,          threshold * 0.55, QColor("#b0b0b0")},
00067         {"Z2 Endurance",    threshold * 0.55, threshold * 0.75, QColor("#6fa8dc")},
00068         {"Z3 Tempo",        threshold * 0.75, threshold * 0.87, QColor("#6aa84f")},
00069         {"Z4 Threshold",    threshold * 0.87, threshold * 0.95, QColor("#ffd966")},
00070         {"Z5 VO2 Max",      threshold * 0.95, threshold * 1.05, QColor("#e69138")},
00071         {"Z6 Anaerobic",    threshold * 1.05, threshold * 1.20, QColor("#cc0000")},
00072         {"Z7 Neuromuscular", threshold * 1.20, DBL_MAX,      QColor("#741b47")},
00073     };
00074 }
00075
00076 std::vector<Zone> ZoneCalculator::heartRateZones(int heartRateMax,
00077 int,
00078 int thresholdHeartRate)
00079 {
00080     const double reference = std::max(thresholdHeartRate > 0 ? thresholdHeartRate : heartRateMax,
00081 120);
00082     return {
00083         {"Z1 Recovery",      0.0,          reference * 0.68, QColor("#b0b0b0")},
00084         {"Z2 Aerobic",      reference * 0.68, reference * 0.83, QColor("#6fa8dc")},
00085         {"Z3 Tempo",        reference * 0.83, reference * 0.94, QColor("#6aa84f")},
00086         {"Z4 Threshold",    reference * 0.94, reference * 1.03, QColor("#ffd966")},
00087         {"Z5 Max",          reference * 1.03, DBL_MAX,      QColor("#cc0000")},
00088     };
00089
00090 std::vector<Zone> ZoneCalculator::cadenceZones()
00091 {
00092     return {
00093         {"Low",              0.0,      70.0, QColor("#b0b0b0")},
00094         {"Steady",           70.0,     85.0, QColor("#6fa8dc")},
00095         {"Endurance",        85.0,     95.0, QColor("#6aa84f")},

```

```

00096         {"Fast",      95.0, 105.0, QColor("#ffd966")},
00097         {"Spin",      105.0, DBL_MAX, QColor("#cc0000")},
00098     };
00099 }
00100
00101 std::vector<Zone> ZoneCalculator::speedZones()
00102 {
00103     return {
00104         {"Easy",      0.0, 15.0, QColor("#6fa8dc")},
00105         {"Rolling",   15.0, 25.0, QColor("#6aa84f")},
00106         {"Tempo",     25.0, 35.0, QColor("#ffd966")},
00107         {"Fast",      35.0, 45.0, QColor("#e69138")},
00108         {"Descent",   45.0, DBL_MAX, QColor("#cc0000")},
00109     };
00110 }
00111
00112 std::vector<Zone> ZoneCalculator::altitudeZones(double minAltitude,
00113                                                double maxAltitude)
00114 {
00115     if (maxAltitude <= minAltitude)
00116         maxAltitude = minAltitude + 1.0;
00117
00118     const double span = maxAltitude - minAltitude;
00119     return {
00120         {"Low",      minAltitude, minAltitude + span * 0.25, QColor("#6fa8dc")},
00121         {"Rolling",  minAltitude + span * 0.25, minAltitude + span * 0.55, QColor("#6aa84f")},
00122         {"High",     minAltitude + span * 0.55, minAltitude + span * 0.80, QColor("#8d6e63")},
00123         {"Alpine",   minAltitude + span * 0.80, DBL_MAX, QColor("#f5f5f5")},
00124     };
00125 }
00126
00127 std::vector<Zone> ZoneCalculator::zonesForMetric(ColorMetric metric,
00128                                                  const ColorContext& context)
00129 {
00130     switch (metric)
00131     {
00132     case ColorMetric::Power:
00133         return powerZones(context.ftp);
00134     case ColorMetric::HeartRate:
00135         return heartRateZones(context.heartRateMax,
00136                               context.heartRateRest,
00137                               context.thresholdHeartRate);
00138     case ColorMetric::Cadence:
00139         return cadenceZones();
00140     case ColorMetric::Speed:
00141         return speedZones();
00142     case ColorMetric::Altitude:
00143         return altitudeZones(context.altitudeMin, context.altitudeMax);
00144     case ColorMetric::Gradient:
00145     case ColorMetric::None:
00146         return {};
00147     }
00148
00149     return {};
00150 }
00151
00152 std::vector<ZoneDistribution> ZoneCalculator::computeDistribution(
00153     const RideData& rideData,
00154     ColorMetric metric,
00155     const ColorContext& context)
00156 {
00157     std::vector<ZoneDistribution> result;
00158     const std::vector<Zone> zones = zonesForMetric(metric, context);
00159     result.reserve(zones.size());
00160     for (const Zone& zone : zones)
00161         result.push_back({zone, 0.0, 0.0});
00162
00163     if (metric == ColorMetric::None || rideData.records.empty() || result.empty())
00164         return result;
00165
00166     if (metric == ColorMetric::Gradient)
00167         return result;
00168
00169     double totalSeconds = 0.0;
00170     for (int index = 0; index < static_cast<int>(rideData.records.size()); ++index)
00171     {
00172         const RideRecord& record = rideData.records[static_cast<size_t>(index)];
00173         if (!sampleAvailable(record, metric))
00174             continue;
00175
00176         const double value = valueForMetric(record, metric);
00177         const double dt = sampleDurationSeconds(rideData.records, index);
00178         totalSeconds += dt;
00179
00180         for (ZoneDistribution& distribution : result)
00181         {
00182             if (value >= distribution.zone.minValue &&
00183                 value < distribution.zone.maxValue)

```



```

00183         {
00184             distribution.seconds += dt;
00185             break;
00186         }
00187     }
00188 }
00189
00190 if (totalSeconds > 0.0)
00191 {
00192     for (ZoneDistribution& distribution : result)
00193         distribution.percent = distribution.seconds / totalSeconds;
00194 }
00195
00196 return result;
00197 }
00198
00199 bool ZoneCalculator::hasMetricSamples(const RideData& rideData, ColorMetric metric)
00200 {
00201     if (metric == ColorMetric::None)
00202         return false;
00203
00204     for (const RideRecord& record : rideData.records)
00205     {
00206         if (sampleAvailable(record, metric))
00207             return true;
00208     }
00209
00210     return false;
00211 }

```

## 8.58 ZoneCalculator.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <vector>
00007
00008 #include "ZoneDefinition.h"
00009 #include "fit/RideData.h"
00010
00016 class ZoneCalculator
00017 {
00018 public:
00024     static std::vector<Zone> powerZones(int ftp);
00025
00033     static std::vector<Zone> heartRateZones(int heartRateMax,
00034                                             int heartRateRest = 60,
00035                                             int thresholdHeartRate = 0);
00036
00041     static std::vector<Zone> cadenceZones();
00042
00047     static std::vector<Zone> speedZones();
00048
00055     static std::vector<Zone> altitudeZones(double minAltitude,
00056                                           double maxAltitude);
00057
00064     static std::vector<Zone> zonesForMetric(ColorMetric metric,
00065                                             const ColorContext& context);
00066
00074     static std::vector<ZoneDistribution> computeDistribution(
00075         const RideData& rideData,
00076         ColorMetric metric,
00077         const ColorContext& context);
00078
00085     static bool hasMetricSamples(const RideData& rideData, ColorMetric metric);
00086 };

```

## 8.59 ZoneDefinition.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ZoneDefinition.h"
00005
00006 QString colorMetricDisplayName(ColorMetric metric)
00007 {
00008     switch (metric)
00009     {
00010         case ColorMetric::None:         return "None";

```

```

00011         case ColorMetric::Power:      return "Power";
00012         case ColorMetric::HeartRate:    return "Heart Rate";
00013         case ColorMetric::Cadence:      return "Cadence";
00014         case ColorMetric::Speed:        return "Speed";
00015         case ColorMetric::Altitude:     return "Altitude";
00016         case ColorMetric::Gradient:     return "Gradient";
00017     }
00018
00019     return "None";
00020 }
00021
00022 QString colorMetricUnit(ColorMetric metric)
00023 {
00024     switch (metric)
00025     {
00026         case ColorMetric::Power:      return "W";
00027         case ColorMetric::HeartRate:  return "bpm";
00028         case ColorMetric::Cadence:    return "rpm";
00029         case ColorMetric::Speed:      return "km/h";
00030         case ColorMetric::Altitude:   return "m";
00031         case ColorMetric::Gradient:   return {};
00032         case ColorMetric::None:       return {};
00033     }
00034
00035     return {};
00036 }

```

## 8.60 ZoneDefinition.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007 #include <QString>
00008
00012 enum class ColorMetric
00013 {
00014     None,
00015     Power,
00016     HeartRate,
00017     Cadence,
00018     Speed,
00019     Altitude,
00020     Gradient
00021 };
00022
00026 struct Zone
00027 {
00029     QString name;
00030
00032     double minValue = 0.0;
00033
00035     double maxValue = 0.0;
00036
00038     QColor color;
00039 };
00040
00044 struct ZoneDistribution
00045 {
00047     Zone zone;
00048
00050     double seconds = 0.0;
00051
00053     double percent = 0.0;
00054 };
00055
00059 struct ColorContext
00060 {
00062     int ftp = 250;
00063
00065     int heartRateMax = 190;
00066
00068     int heartRateRest = 60;
00069
00071     int thresholdHeartRate = 0;
00072
00074     double altitudeMin = 0.0;
00075
00077     double altitudeMax = 0.0;
00078
00080     bool hasAltitudeRange = false;
00081 };

```

```

00082
00088 QString colorMetricDisplayName(ColorMetric metric);
00089
00095 QString colorMetricUnit(ColorMetric metric);

```

## 8.61 ActivityAnalysisCacheRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityAnalysisCacheRepository.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlQuery>
00008
00009 ActivityAnalysisCacheRepository::ActivityAnalysisCacheRepository(QSqlDatabase& db)
00010     : m_db(db)
00011 {}
00012
00013 bool ActivityAnalysisCacheRepository::setCache(int activityId, const QString& key, const QByteArray&
value)
00014 {
00015     QSqlQuery q(m_db);
00016     q.prepare(
00017         "INSERT OR REPLACE INTO activity_analysis_cache(activity_id, cache_key, cache_value)"
00018         " VALUES(:id, :k, :v)");
00019     q.bindValue(":id", activityId);
00020     q.bindValue(":k", key);
00021     q.bindValue(":v", value);
00022     return q.exec();
00023 }
00024
00025 QByteArray ActivityAnalysisCacheRepository::getCache(int activityId, const QString& key)
00026 {
00027     QSqlQuery q(m_db);
00028     q.prepare("SELECT cache_value FROM activity_analysis_cache WHERE activity_id=:id AND
cache_key=:k");
00029     q.bindValue(":id", activityId);
00030     q.bindValue(":k", key);
00031     if (q.exec() && q.next())
00032         return q.value(0).toByteArray();
00033     return {};
00034 }
00035
00036 bool ActivityAnalysisCacheRepository::hasCache(int activityId, const QString& key)
00037 {
00038     QSqlQuery q(m_db);
00039     q.prepare("SELECT 1 FROM activity_analysis_cache WHERE activity_id=:id AND cache_key=:k LIMIT 1");
00040     q.bindValue(":id", activityId);
00041     q.bindValue(":k", key);
00042     return q.exec() && q.next();
00043 }
00044
00045 bool ActivityAnalysisCacheRepository::invalidateCache(int activityId)
00046 {
00047     QSqlQuery q(m_db);
00048     q.prepare("DELETE FROM activity_analysis_cache WHERE activity_id=:id");
00049     q.bindValue(":id", activityId);
00050     return q.exec();
00051 }
00052
00053 bool ActivityAnalysisCacheRepository::invalidateKey(int activityId, const QString& key)
00054 {
00055     QSqlQuery q(m_db);
00056     q.prepare("DELETE FROM activity_analysis_cache WHERE activity_id=:id AND cache_key=:k");
00057     q.bindValue(":id", activityId);
00058     q.bindValue(":k", key);
00059     return q.exec();
00060 }

```

## 8.62 ActivityAnalysisCacheRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QByteArray>
00007 #include <QString>
00008
00009 class QSqlDatabase;

```

```

00010
00017 class ActivityAnalysisCacheRepository
00018 {
00019 public:
00024     explicit ActivityAnalysisCacheRepository(QSqlDatabase& db);
00025
00033     bool        setCache(int activityId, const QString& key, const QByteArray& value);
00034
00041     QByteArray getCache(int activityId, const QString& key);
00042
00049     bool        hasCache(int activityId, const QString& key);
00050
00056     bool        invalidateCache(int activityId);
00057
00064     bool        invalidateKey(int activityId, const QString& key);
00065
00066 private:
00067     QSqlDatabase& m_db;
00068 };

```

## 8.63 ActivityAnalysisSettingsRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityAnalysisSettingsRepository.h"
00005
00006 #include <QSqlDatabase>
00007 #include <QSqlQuery>
00008
00009 ActivityAnalysisSettingsRepository::ActivityAnalysisSettingsRepository(QSqlDatabase& db)
00010     : m_db(db)
00011 {}
00012
00013 bool ActivityAnalysisSettingsRepository::insertOrUpdate(const ActivityAnalysisSettings& settings)
00014 {
00015     QSqlQuery q(m_db);
00016     q.prepare(
00017         "INSERT OR REPLACE INTO activity_analysis_settings("
00018         "    activity_id, climb_min_gain, climb_min_length, climb_min_gradient,"
00019         "    interval_work_threshold, interval_rest_threshold"
00020         ") VALUES ("
00021         "    :activity_id, :climb_min_gain, :climb_min_length, :climb_min_gradient,"
00022         "    :interval_work_threshold, :interval_rest_threshold"
00023         ")");
00024
00025     q.bindValue(":activity_id", settings.activityId);
00026     q.bindValue(":climb_min_gain", settings.climbMinGain);
00027     q.bindValue(":climb_min_length", settings.climbMinLength);
00028     q.bindValue(":climb_min_gradient", settings.climbMinGradient);
00029     q.bindValue(":interval_work_threshold", settings.intervalWorkThreshold);
00030     q.bindValue(":interval_rest_threshold", settings.intervalRestThreshold);
00031     return q.exec();
00032 }
00033
00034 ActivityAnalysisSettings ActivityAnalysisSettingsRepository::settingsForActivity(int activityId)
00035 {
00036     ActivityAnalysisSettings s;
00037     QSqlQuery q(m_db);
00038     q.prepare(
00039         "SELECT activity_id, climb_min_gain, climb_min_length, climb_min_gradient,"
00040         "    interval_work_threshold, interval_rest_threshold"
00041         " FROM activity_analysis_settings WHERE activity_id=:id");
00042     q.bindValue(":id", activityId);
00043     if (!q.exec() || !q.next())
00044         return s;
00045
00046     s.activityId = q.value(0).toInt();
00047     s.climbMinGain = q.value(1).toDouble();
00048     s.climbMinLength = q.value(2).toDouble();
00049     s.climbMinGradient = q.value(3).toDouble();
00050     s.intervalWorkThreshold = q.value(4).toDouble();
00051     s.intervalRestThreshold = q.value(5).toDouble();
00052     return s;
00053 }

```

## 8.64 ActivityAnalysisSettingsRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct ActivityAnalysisSettings
00015 {
00017     int     activityId           = -1;
00018
00020     double  climbMinGain        = 30.0;
00021
00023     double  climbMinLength      = 0.5;
00024
00026     double  climbMinGradient    = 3.0;
00027
00029     double  intervalWorkThreshold = 0.0;
00030
00032     double  intervalRestThreshold = 0.0;
00033
00037     bool    isValid() const { return activityId > 0; }
00038 };
00039
00045 class ActivityAnalysisSettingsRepository
00046 {
00047 public:
00052     explicit ActivityAnalysisSettingsRepository(QSqlDatabase& db);
00053
00059     bool insertOrUpdate(const ActivityAnalysisSettings& settings);
00060
00066     ActivityAnalysisSettings settingsForActivity(int activityId);
00067
00068 private:
00069     QSqlDatabase& m_db;
00070 };

```

## 8.65 ActivityRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010 #include <QVariant>
00011 #include <QStringList>
00012
00013 ActivityRepository::ActivityRepository(QSqlDatabase& db)
00014     : m_db(db)
00015 {}
00016
00017 // Helpers
00018
00019 static Activity activityFromQuery(QSqlQuery& q)
00020 {
00021     Activity a;
00022     int col = 0;
00023     a.id = q.value(col++).toInt();
00024     a.athleteId = q.value(col++).toInt();
00025     a.fitHash = q.value(col++).toString();
00026     a.fileName = q.value(col++).toString();
00027     a.sport = q.value(col++).toString();
00028     a.startTime = q.value(col++).toString();
00029     a.endTime = q.value(col++).toString();
00030     a.durationSec = q.value(col++).toDouble();
00031     a.distanceM = q.value(col++).toDouble();
00032     a.avgPower = q.value(col++).toDouble();
00033     a.maxPower = q.value(col++).toDouble();
00034     a.normalizedPower = q.value(col++).toDouble();
00035     a.avgHr = q.value(col++).toDouble();
00036     a.maxHr = q.value(col++).toDouble();
00037     a.avgCadence = q.value(col++).toDouble();
00038     a.avgSpeed = q.value(col++).toDouble();
00039     a.elevationGain = q.value(col++).toDouble();
00040     a.notes = q.value(col++).toString();
00041     a.weatherNotes = q.value(col++).toString();
00042     a.temperature = q.value(col++).toDouble();
00043     a.weather = q.value(col++).toString();
00044     a.wind = q.value(col++).toString();
00045     a.rpe = q.value(col++).toInt();

```

```

00046     a.fatigue           = q.value(col++).toInt();
00047     a.sleepHours        = q.value(col++).toDouble();
00048     a.weightKg           = q.value(col++).toDouble();
00049     a.bike               = q.value(col++).toString();
00050     a.equipment          = q.value(col++).toString();
00051     a.importedAt         = q.value(col++).toString();
00052     a.analysisVersion    = q.value(col++).toInt();
00053     a.fingerprint        = q.value(col++).toString();
00054     a.analysisFlags      = q.value(col++).toInt();
00055     a.importSource       = q.value(col++).toString();
00056     return a;
00057 }
00058
00059 static const char* kSelectCols =
00060     "id, athlete_id, fit_hash, file_name, sport, activity_start_time, end_time,"
00061     " duration_sec, distance_m, avg_power, max_power, normalized_power,"
00062     " avg_hr, max_hr, avg_cadence, avg_speed, elevation_gain,"
00063     " notes, weather_notes, temperature, weather, wind, rpe, fatigue, sleep,"
00064     " weight, bike, equipment, import_time,"
00065     " COALESCE(analysis_version,1), COALESCE(fingerprint,''),"
00066     " COALESCE(analysis_flags,0), COALESCE(import_source,')";
00067
00068 // Public API
00069
00070 int ActivityRepository::insertActivity(const Activity& a)
00071 {
00072     QSqlQuery q(m_db);
00073     q.prepare(
00074         "INSERT INTO activities("
00075         " athlete_id, fit_hash, file_name, sport, activity_start_time, end_time,"
00076         " duration_sec, distance_m, avg_power, max_power, normalized_power,"
00077         " avg_hr, max_hr, avg_cadence, avg_speed, elevation_gain,"
00078         " notes, weather_notes, temperature, weather, wind, rpe, fatigue, sleep,"
00079         " weight, bike, equipment, import_time"
00080         ") VALUES ("
00081         " :aid, :fh, :fn, :sport, :st, :et,"
00082         " :dur, :dist, :ap, :mp, :np,"
00083         " :ah, :mh, :ac, :as_, :eg,"
00084         " :notes, :wn, :temp, :weather, :wind, :rpe, :fatigue, :sleep,"
00085         " :weight, :bike, :equipment, :import_time"
00086         ")");
00087
00088     q.bindValue(":aid", a.athleteId);
00089     q.bindValue(":fh", a.fitHash.isEmpty() ? QVariant() : QVariant(a.fitHash));
00090     q.bindValue(":fn", a.fileName);
00091     q.bindValue(":sport", a.sport.isEmpty() ? QVariant() : QVariant(a.sport));
00092     q.bindValue(":st", a.startTime.isEmpty() ? QVariant() : QVariant(a.startTime));
00093     q.bindValue(":et", a.endTime.isEmpty() ? QVariant() : QVariant(a.endTime));
00094     q.bindValue(":dur", a.durationSec);
00095     q.bindValue(":dist", a.distanceM);
00096     q.bindValue(":ap", a.avgPower);
00097     q.bindValue(":mp", a.maxPower);
00098     q.bindValue(":np", a.normalizedPower);
00099     q.bindValue(":ah", a.avgHr);
00100     q.bindValue(":mh", a.maxHr);
00101     q.bindValue(":ac", a.avgCadence);
00102     q.bindValue(":as_", a.avgSpeed);
00103     q.bindValue(":eg", a.elevationGain);
00104     q.bindValue(":notes", a.notes);
00105     q.bindValue(":wn", a.weatherNotes);
00106     q.bindValue(":temp", a.temperature);
00107     q.bindValue(":weather", a.weather.isEmpty() ? QVariant() : QVariant(a.weather));
00108     q.bindValue(":wind", a.wind.isEmpty() ? QVariant() : QVariant(a.wind));
00109     q.bindValue(":rpe", a.rpe > 0 ? QVariant(a.rpe) : QVariant());
00110     q.bindValue(":fatigue", a.fatigue > 0 ? QVariant(a.fatigue) : QVariant());
00111     q.bindValue(":sleep", a.sleepHours > 0.0 ? QVariant(a.sleepHours) : QVariant());
00112     q.bindValue(":weight", a.weightKg > 0.0 ? QVariant(a.weightKg) : QVariant());
00113     q.bindValue(":bike", a.bike.isEmpty() ? QVariant() : QVariant(a.bike));
00114     q.bindValue(":equipment", a.equipment.isEmpty() ? QVariant() : QVariant(a.equipment));
00115     q.bindValue(":import_time", a.importedAt.isEmpty()
00116         ? QDateTime::currentDateTimeUtc().toString(Qt::ISODate)
00117         : a.importedAt);
00118
00119     if (!q.exec()) return -1;
00120     return q.lastInsertId().toInt();
00121 }
00122
00123 Activity ActivityRepository::getActivity(int activityId)
00124 {
00125     QSqlQuery q(m_db);
00126     q.prepare(QString("SELECT %1 FROM activities WHERE id=:id").arg(kSelectCols));
00127     q.bindValue(":id", activityId);
00128     q.exec();
00129     if (q.next()) return activityFromQuery(q);
00130     return {};
00131 }
00132

```

```

00133 QList<Activity> ActivityRepository::listActivities(int athleteId)
00134 {
00135     QList<Activity> result;
00136     QSqlQuery q(m_db);
00137     if (athleteId > 0)
00138     {
00139         q.prepare(QString("SELECT %1 FROM activities WHERE athlete_id=:aid"
00140             " ORDER BY activity_start_time DESC").arg(kSelectCols));
00141         q.bindValue(":aid", athleteId);
00142     }
00143     else
00144     {
00145         q.prepare(QString("SELECT %1 FROM activities ORDER BY activity_start_time DESC")
00146             .arg(kSelectCols));
00147     }
00148     q.exec();
00149     while (q.next())
00150         result.append(activityFromQuery(q));
00151     return result;
00152 }
00153
00154 bool ActivityRepository::deleteActivity(int activityId)
00155 {
00156     QSqlQuery q(m_db);
00157     q.prepare("DELETE FROM activities WHERE id=:id");
00158     q.bindValue(":id", activityId);
00159     return q.exec();
00160 }
00161
00162 bool ActivityRepository::hasFitHash(const QString& hash)
00163 {
00164     QSqlQuery q(m_db);
00165     q.prepare("SELECT 1 FROM activities WHERE fit_hash=:h LIMIT 1");
00166     q.bindValue(":h", hash);
00167     q.exec();
00168     return q.next();
00169 }
00170
00171 bool ActivityRepository::updateAnalysisFlags(int activityId, int flags)
00172 {
00173     QSqlQuery q(m_db);
00174     q.prepare("UPDATE activities SET analysis_flags=:f WHERE id=:id");
00175     q.bindValue(":f", flags);
00176     q.bindValue(":id", activityId);
00177     return q.exec();
00178 }
00179
00180 bool ActivityRepository::updateFingerprint(int activityId, const QString& fingerprint)
00181 {
00182     QSqlQuery q(m_db);
00183     q.prepare("UPDATE activities SET fingerprint=:fp WHERE id=:id");
00184     q.bindValue(":fp", fingerprint);
00185     q.bindValue(":id", activityId);
00186     return q.exec();
00187 }
00188
00189 bool ActivityRepository::hasFingerprintMatch(const QString& fingerprint)
00190 {
00191     if (fingerprint.isEmpty()) return false;
00192     QSqlQuery q(m_db);
00193     q.prepare("SELECT 1 FROM activities WHERE fingerprint=:fp LIMIT 1");
00194     q.bindValue(":fp", fingerprint);
00195     q.exec();
00196     return q.next();
00197 }
00198
00199 bool ActivityRepository::setImportSource(int activityId, const QString& source)
00200 {
00201     QSqlQuery q(m_db);
00202     q.prepare("UPDATE activities SET import_source=:s WHERE id=:id");
00203     q.bindValue(":s", source);
00204     q.bindValue(":id", activityId);
00205     return q.exec();
00206 }
00207
00208 bool ActivityRepository::updateNotes(int activityId,
00209     const QString& notes,
00210     const QString& weatherNotes)
00211 {
00212     QSqlQuery q(m_db);
00213     q.prepare("UPDATE activities SET notes=:n, weather_notes=:wn WHERE id=:id");
00214     q.bindValue(":n", notes);
00215     q.bindValue(":wn", weatherNotes);
00216     q.bindValue(":id", activityId);
00217     return q.exec();
00218 }
00219

```

```

00220 bool ActivityRepository::updateActivityProperties(int activityId, const Activity& a)
00221 {
00222     QSqlQuery q(m_db);
00223     q.prepare(
00224         "UPDATE activities SET "
00225         "notes=:notes, weather_notes=:wn, temperature=:temp, weather=:weather, wind=:wind,"
00226         "rpe=:rpe, fatigue=:fatigue, sleep=:sleep, weight=:weight, bike=:bike, equipment=:equipment "
00227         "WHERE id=:id");
00228     q.bindValue(":notes", a.notes);
00229     q.bindValue(":wn", a.weatherNotes);
00230     q.bindValue(":temp", a.temperature > 0.0 ? QVariant(a.temperature) : QVariant());
00231     q.bindValue(":weather", a.weather.isEmpty() ? QVariant() : QVariant(a.weather));
00232     q.bindValue(":wind", a.wind.isEmpty() ? QVariant() : QVariant(a.wind));
00233     q.bindValue(":rpe", a.rpe > 0 ? QVariant(a.rpe) : QVariant());
00234     q.bindValue(":fatigue", a.fatigue > 0 ? QVariant(a.fatigue) : QVariant());
00235     q.bindValue(":sleep", a.sleepHours > 0.0 ? QVariant(a.sleepHours) : QVariant());
00236     q.bindValue(":weight", a.weightKg > 0.0 ? QVariant(a.weightKg) : QVariant());
00237     q.bindValue(":bike", a.bike.isEmpty() ? QVariant() : QVariant(a.bike));
00238     q.bindValue(":equipment", a.equipment.isEmpty() ? QVariant() : QVariant(a.equipment));
00239     q.bindValue(":id", activityId);
00240     if (!q.exec())
00241         return false;
00242
00243     // Keep equipment usage in sync for service interval tracking.
00244     QSqlQuery details(m_db);
00245     details.prepare("SELECT athlete_id, distance_m, duration_sec FROM activities WHERE id=:id");
00246     details.bindValue(":id", activityId);
00247     if (!details.exec() || !details.next())
00248         return true;
00249
00250     const int athleteId = details.value(0).toInt();
00251     const double distanceKm = details.value(1).toDouble() / 1000.0;
00252     const double durationSec = details.value(2).toDouble();
00253
00254     QSqlQuery clearUsage(m_db);
00255     clearUsage.prepare("DELETE FROM equipment_usage WHERE activity_id=:id");
00256     clearUsage.bindValue(":id", activityId);
00257     clearUsage.exec();
00258
00259     QStringList equipmentNames;
00260     if (!a.bike.trimmed().isEmpty())
00261         equipmentNames << a.bike.trimmed();
00262     if (!a.equipment.trimmed().isEmpty())
00263     {
00264         const QStringList split = a.equipment.split(',', Qt::SkipEmptyParts);
00265         for (const QString& entry : split)
00266         {
00267             const QString trimmed = entry.trimmed();
00268             if (!trimmed.isEmpty())
00269                 equipmentNames << trimmed;
00270         }
00271     }
00272     equipmentNames.removeDuplicates();
00273
00274     for (const QString& name : equipmentNames)
00275     {
00276         int equipmentId = -1;
00277         QSqlQuery findEq(m_db);
00278         findEq.prepare("SELECT id FROM equipment WHERE athlete_id=:athlete_id AND name=:name LIMIT
1");
00279         findEq.bindValue(":athlete_id", athleteId);
00280         findEq.bindValue(":name", name);
00281         if (findEq.exec() && findEq.next())
00282         {
00283             equipmentId = findEq.value(0).toInt();
00284         }
00285         else
00286         {
00287             QSqlQuery insertEq(m_db);
00288             insertEq.prepare(
00289                 "INSERT INTO equipment(athlete_id, name, category, notes) "
00290                 "VALUES(:athlete_id, :name, :category, :notes)");
00291             insertEq.bindValue(":athlete_id", athleteId);
00292             insertEq.bindValue(":name", name);
00293             insertEq.bindValue(":category", name.compare(a.bike, Qt::CaseInsensitive) == 0 ? "Bike" :
"Component");
00294             insertEq.bindValue(":notes", QString());
00295             if (insertEq.exec())
00296                 equipmentId = insertEq.lastInsertId().toInt();
00297         }
00298
00299         if (equipmentId <= 0)
00300             continue;
00301
00302         QSqlQuery usage(m_db);
00303         usage.prepare(
00304             "INSERT INTO equipment_usage(equipment_id, activity_id, distance_km, duration_sec) "

```



```

00305         "VALUES(:equipment_id, :activity_id, :distance_km, :duration_sec)");
00306         usage.bindValue(":equipment_id", equipmentId);
00307         usage.bindValue(":activity_id", activityId);
00308         usage.bindValue(":distance_km", distanceKm);
00309         usage.bindValue(":duration_sec", durationSec);
00310         usage.exec();
00311     }
00312
00313     return true;
00314 }

```

## 8.66 ActivityRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00016 struct Activity
00017 {
00019     int      id              = -1;
00020
00022     int      athleteId      = -1;
00023
00025     QString  fitHash;
00026
00028     QString  fileName;
00029
00031     QString  sport;
00032
00034     QString  startTime;
00035
00037     QString  endTime;
00038
00040     double   durationSec     = 0.0;
00041
00043     double   distanceM       = 0.0;
00044
00046     double   avgPower        = 0.0;
00047
00049     double   maxPower        = 0.0;
00050
00052     double   normalizedPower = 0.0;
00053
00055     double   avgHr           = 0.0;
00056
00058     double   maxHr           = 0.0;
00059
00061     double   avgCadence      = 0.0;
00062
00064     double   avgSpeed        = 0.0;
00065
00067     double   elevationGain   = 0.0;
00068
00070     QString  notes;
00071
00073     QString  weatherNotes;
00074
00076     double   temperature     = 0.0;
00077
00079     QString  weather;
00080
00082     QString  wind;
00083
00085     int      rpe              = 0;
00086
00088     int      fatigue          = 0;
00089
00091     double   sleepHours      = 0.0;
00092
00094     double   weightKg        = 0.0;
00095
00097     QString  bike;
00098
00100     QString  equipment;
00101
00103     QString  importedAt;
00104
00106     int      analysisVersion = 1;

```

```

00107
00109     QString fingerprint;
00110
00112     int     analysisFlags    = 0;
00113
00115     QString importSource;
00116
00121     bool isValid() const { return id > 0; }
00122 };
00123
00129 class ActivityRepository
00130 {
00131 public:
00136     explicit ActivityRepository(QSqlDatabase& db);
00137
00143     int         insertActivity(const Activity& a);
00144
00150     Activity     getActivity(int activityId);
00151
00157     QList<Activity> listActivities(int athleteId);
00158
00164     bool         deleteActivity(int activityId);
00165
00171     bool         hasFitHash(const QString& hash);
00172
00180     bool         updateNotes(int activityId, const QString& notes,
00181                             const QString& weatherNotes);
00182
00189     bool         updateActivityProperties(int activityId, const Activity& a);
00190
00197     bool         updateAnalysisFlags(int activityId, int flags);
00198
00205     bool         updateFingerprint(int activityId, const QString& fingerprint);
00206
00212     bool         hasFingerprintMatch(const QString& fingerprint);
00213
00220     bool         setImportSource(int activityId, const QString& source);
00221
00222 private:
00223     QSqlDatabase& m_db;
00224 };

```

## 8.67 AthleteRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010
00011 AthleteRepository::AthleteRepository(QSqlDatabase& db)
00012     : m_db(db)
00013 {}
00014
00015 // Athletes
00016
00017 int AthleteRepository::insertAthlete(const Athlete& a)
00018 {
00019     QSqlQuery q(m_db);
00020     q.prepare(
00021         "INSERT INTO athletes(first_name, last_name, date_of_birth, email, height_cm, ftp,
00022         created_at)"
00023         " VALUES(:fn, :ln, :dob, :email, :h, :ftp, :ts)");
00024     q.bindValue(":fn", a.firstName());
00025     q.bindValue(":ln", a.lastName());
00026     q.bindValue(":dob", a.dateOfBirth());
00027     q.bindValue(":email", a.email());
00028     q.bindValue(":h", a.heightCm() > 0.0 ? QVariant(a.heightCm()) : QVariant());
00029     q.bindValue(":ftp", a.ftpWatts() > 0 ? a.ftpWatts() : 250);
00030     q.bindValue(":ts", QDateTime::currentDateTimeUtc().toString(Qt::ISODate));
00031
00032     if (!q.exec()) return -1;
00033     return q.lastInsertId().toInt();
00034 }
00035
00036 bool AthleteRepository::updateAthlete(const Athlete& a)
00037 {
00038     QSqlQuery q(m_db);
00039     q.prepare(
00040         "UPDATE athletes SET first_name=:fn, last_name=:ln, date_of_birth=:dob,"

```

```

00040         " email=:email, height_cm=:h, ftp=:ftp WHERE id=:id");
00041     q.bindValue(":fn",    a.firstName);
00042     q.bindValue(":ln",    a.lastName);
00043     q.bindValue(":dob",    a.dateOfBirth);
00044     q.bindValue(":email", a.email);
00045     q.bindValue(":h",      a.heightCm > 0.0 ? QVariant(a.heightCm) : QVariant());
00046     q.bindValue(":ftp",    a.ftpWatts > 0 ? a.ftpWatts : 250);
00047     q.bindValue(":id",     a.id);
00048     return q.exec();
00049 }
00050
00051 bool AthleteRepository::deleteAthlete(int athleteId)
00052 {
00053     QSqlQuery q(m_db);
00054     q.prepare("DELETE FROM athletes WHERE id=:id");
00055     q.bindValue(":id", athleteId);
00056     return q.exec();
00057 }
00058
00059 QList<Athlete> AthleteRepository::listAthletes()
00060 {
00061     QList<Athlete> result;
00062     QSqlQuery q(m_db);
00063     q.exec("SELECT id, first_name, last_name, date_of_birth, email, height_cm, ftp, created_at"
00064           " FROM athletes ORDER BY last_name, first_name");
00065     while (q.next())
00066     {
00067         Athlete a;
00068         a.id = q.value(0).toInt();
00069         a.firstName = q.value(1).toString();
00070         a.lastName = q.value(2).toString();
00071         a.dateOfBirth = q.value(3).toString();
00072         a.email = q.value(4).toString();
00073         a.heightCm = q.value(5).toDouble();
00074         a.ftpWatts = q.value(6).toInt() > 0 ? q.value(6).toInt() : 250;
00075         a.createdAt = q.value(7).toString();
00076         result.append(a);
00077     }
00078     return result;
00079 }
00080
00081 Athlete AthleteRepository::getAthlete(int athleteId)
00082 {
00083     QSqlQuery q(m_db);
00084     q.prepare("SELECT id, first_name, last_name, date_of_birth, email, height_cm, ftp, created_at"
00085           " FROM athletes WHERE id=:id");
00086     q.bindValue(":id", athleteId);
00087     q.exec();
00088     if (q.next())
00089     {
00090         Athlete a;
00091         a.id = q.value(0).toInt();
00092         a.firstName = q.value(1).toString();
00093         a.lastName = q.value(2).toString();
00094         a.dateOfBirth = q.value(3).toString();
00095         a.email = q.value(4).toString();
00096         a.heightCm = q.value(5).toDouble();
00097         a.ftpWatts = q.value(6).toInt() > 0 ? q.value(6).toInt() : 250;
00098         a.createdAt = q.value(7).toString();
00099         return a;
00100     }
00101     return {};
00102 }
00103
00104 // FTP history
00105
00106 int AthleteRepository::addFtpEntry(const FtpEntry& e)
00107 {
00108     QSqlQuery q(m_db);
00109     q.prepare(
00110         "INSERT INTO ftp_history(athlete_id, ftp_watts, effective_from, notes)"
00111         " VALUES(:aid, :w, :from, :notes)");
00112     q.bindValue(":aid", e.athleteId);
00113     q.bindValue(":w", e.ftpWatts);
00114     q.bindValue(":from", e.effectiveFrom);
00115     q.bindValue(":notes", e.notes);
00116     if (!q.exec()) return -1;
00117     return q.lastInsertId().toInt();
00118 }
00119
00120 bool AthleteRepository::deleteFtpEntry(int entryId)
00121 {
00122     QSqlQuery q(m_db);
00123     q.prepare("DELETE FROM ftp_history WHERE id=:id");
00124     q.bindValue(":id", entryId);
00125     return q.exec();
00126 }

```

```

00127
00128 QList<FtpEntry> AthleteRepository::getFtpHistory(int athleteId)
00129 {
00130     QList<FtpEntry> result;
00131     QSqlQuery q(m_db);
00132     q.prepare("SELECT id, athlete_id, ftp_watts, effective_from, notes"
00133             " FROM ftp_history WHERE athlete_id=:aid ORDER BY effective_from DESC");
00134     q.bindValue(":aid", athleteId);
00135     q.exec();
00136     while (q.next())
00137     {
00138         FtpEntry e;
00139         e.id = q.value(0).toInt();
00140         e.athleteId = q.value(1).toInt();
00141         e.ftpWatts = q.value(2).toInt();
00142         e.effectiveFrom = q.value(3).toString();
00143         e.notes = q.value(4).toString();
00144         result.append(e);
00145     }
00146     return result;
00147 }
00148
00149 int AthleteRepository::getFtpForDate(int athleteId, const QDate& date)
00150 {
00151     // Find the most recent FTP entry on or before the given date.
00152     QSqlQuery q(m_db);
00153     q.prepare(
00154         "SELECT ftp_watts FROM ftp_history"
00155         " WHERE athlete_id=:aid AND effective_from <= :date"
00156         " ORDER BY effective_from DESC LIMIT 1");
00157     q.bindValue(":aid", athleteId);
00158     q.bindValue(":date", date.toString(Qt::ISODate));
00159     q.exec();
00160     if (q.next())
00161         return q.value(0).toInt();
00162     return 0;
00163 }
00164
00165 // Weight history
00166
00167 int AthleteRepository::addWeightEntry(const WeightEntry& e)
00168 {
00169     QSqlQuery q(m_db);
00170     q.prepare(
00171         "INSERT INTO weight_history(athlete_id, weight_kg, effective_from)"
00172         " VALUES(:aid, :w, :from)");
00173     q.bindValue(":aid", e.athleteId);
00174     q.bindValue(":w", e.weightKg);
00175     q.bindValue(":from", e.effectiveFrom);
00176     if (!q.exec()) return -1;
00177     return q.lastInsertId().toInt();
00178 }
00179
00180 bool AthleteRepository::deleteWeightEntry(int entryId)
00181 {
00182     QSqlQuery q(m_db);
00183     q.prepare("DELETE FROM weight_history WHERE id=:id");
00184     q.bindValue(":id", entryId);
00185     return q.exec();
00186 }
00187
00188 QList<WeightEntry> AthleteRepository::getWeightHistory(int athleteId)
00189 {
00190     QList<WeightEntry> result;
00191     QSqlQuery q(m_db);
00192     q.prepare("SELECT id, athlete_id, weight_kg, effective_from"
00193             " FROM weight_history WHERE athlete_id=:aid ORDER BY effective_from DESC");
00194     q.bindValue(":aid", athleteId);
00195     q.exec();
00196     while (q.next())
00197     {
00198         WeightEntry e;
00199         e.id = q.value(0).toInt();
00200         e.athleteId = q.value(1).toInt();
00201         e.weightKg = q.value(2).toDouble();
00202         e.effectiveFrom = q.value(3).toString();
00203         result.append(e);
00204     }
00205     return result;
00206 }

```

## 8.68 AthleteRepository.h

```
00001 // SPDX-License-Identifier: GPL-3
```

```

00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QList>
00008 #include <QString>
00009
00010 class QSqlDatabase;
00011
00015 struct Athlete
00016 {
00018     int      id          = -1;
00019
00021     QString  firstName;
00022
00024     QString  lastName;
00025
00027     QString  dateOfBirth;
00028
00030     QString  email;
00031
00033     double   heightCm    = 0.0;
00034
00036     int      ftpWatts    = 250;
00037
00039     QString  createdAt;
00040
00044     QString  fullName() const { return firstName + " " + lastName; }
00045
00049     bool     isValid()  const { return id > 0; }
00050 };
00051
00055 struct FtpEntry
00056 {
00058     int      id          = -1;
00059
00061     int      athleteId   = -1;
00062
00064     int      ftpWatts    = 0;
00065
00067     QString  effectiveFrom;
00068
00070     QString  notes;
00071 };
00072
00076 struct WeightEntry
00077 {
00079     int      id          = -1;
00080
00082     int      athleteId   = -1;
00083
00085     double   weightKg     = 0.0;
00086
00088     QString  effectiveFrom;
00089 };
00090
00096 class AthleteRepository
00097 {
00098 public:
00103     explicit AthleteRepository(QSqlDatabase& db);
00104
00110     int      insertAthlete(const Athlete& a);
00111
00117     bool     updateAthlete(const Athlete& a);
00118
00124     bool     deleteAthlete(int athleteId);
00125
00130     QList<Athlete> listAthletes();
00131
00137     Athlete   getAthlete(int athleteId);
00138
00144     int      addFtpEntry(const FtpEntry& e);
00145
00151     bool     deleteFtpEntry(int entryId);
00152
00158     QList<FtpEntry> getFtpHistory(int athleteId);
00159
00166     int      getFtpForDate(int athleteId, const QDate& date);
00167
00173     int      addWeightEntry(const WeightEntry& e);
00174
00180     bool     deleteWeightEntry(int entryId);
00181
00187     QList<WeightEntry> getWeightHistory(int athleteId);
00188
00189 private:

```

```
00190     QSqlDatabase& m_db;
00191 };
```

## 8.69 ClimbRepository.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ClimbRepository.h"
00005 #include "SqlQueryHelper.h"
00006
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009 #include <QVariant>
00010
00011 namespace
00012 {
00013     const char* kSelectCols =
00014         "id, activity_id, COALESCE(uuid, ''), start_seconds, end_seconds, "
00015         " COALESCE(original_start_seconds, -1.0), COALESCE(original_end_seconds, -1.0), "
00016         " COALESCE(name, ''), COALESCE(elevation_gain_m, 0.0), COALESCE(length_km, 0.0), "
00017         " COALESCE(average_gradient, 0.0), "
00018         " COALESCE(avg_power, 0.0), COALESCE(np, 0.0), COALESCE(avg_hr, 0.0), "
00019         " COALESCE(avg_cadence, 0.0), COALESCE(vam, 0.0), "
00020         " COALESCE(source, 'auto'), COALESCE(locked, 0), COALESCE(deleted, 0), "
00021         " COALESCE(algorithm_version, 1), "
00022         " COALESCE(notes, ''), COALESCE(favorite, 0), COALESCE(rating, 0)";
00023
00024     ClimbRecord recordFromQuery(QSqlQuery& q)
00025     {
00026         ClimbRecord r;
00027         int col = 0;
00028         r.id = q.value(col++).toInt();
00029         r.activityId = q.value(col++).toInt();
00030         r.uuid = q.value(col++).toString();
00031         r.startSeconds = q.value(col++).toDouble();
00032         r.endSeconds = q.value(col++).toDouble();
00033         r.originalStartSeconds = q.value(col++).toDouble();
00034         r.originalEndSeconds = q.value(col++).toDouble();
00035         r.name = q.value(col++).toString();
00036         r.elevationGainM = q.value(col++).toDouble();
00037         r.lengthKm = q.value(col++).toDouble();
00038         r.averageGradient = q.value(col++).toDouble();
00039         r.avgPower = q.value(col++).toDouble();
00040         r.np = q.value(col++).toDouble();
00041         r.avgHr = q.value(col++).toDouble();
00042         r.avgCadence = q.value(col++).toDouble();
00043         r.vam = q.value(col++).toDouble();
00044         r.source = q.value(col++).toString();
00045         r.locked = q.value(col++).toInt() != 0;
00046         r.deleted = q.value(col++).toInt() != 0;
00047         r.algorithmVersion = q.value(col++).toInt();
00048         r.notes = q.value(col++).toString();
00049         r.favorite = q.value(col++).toInt() != 0;
00050         r.rating = q.value(col++).toInt();
00051         return r;
00052     }
00053 } // namespace
00054
00055 ClimbRepository::ClimbRepository(QSqlDatabase& db)
00056     : m_db(db)
00057 {}
00058
00059 int ClimbRepository::insertClimb(const ClimbRecord& climb)
00060 {
00061     QSqlQuery q(m_db);
00062     q.prepare(
00063         "INSERT INTO climbs("
00064         " activity_id, uuid, start_seconds, end_seconds, "
00065         " original_start_seconds, original_end_seconds, "
00066         " name, elevation_gain_m, length_km, average_gradient, "
00067         " avg_power, np, avg_hr, avg_cadence, vam, "
00068         " source, locked, deleted, algorithm_version, "
00069         " notes, favorite, rating "
00070         ") VALUES("
00071         " :activity_id, :uuid, :start_seconds, :end_seconds, "
00072         " :orig_start, :orig_end, "
00073         " :name, :elevation_gain_m, :length_km, :average_gradient, "
00074         " :avg_power, :np, :avg_hr, :avg_cadence, :vam, "
00075         " :source, :locked, :deleted, :algorithm_version, "
00076         " :notes, :favorite, :rating "
00077         ")");
00078
00079     q.bindValue(":activity_id", climb.activityId);
```

```

00080     q.bindValue(":uuid", climb.uuid.isEmpty() ? QVariant() : QVariant(climb.uuid));
00081     q.bindValue(":start_seconds", climb.startSeconds);
00082     q.bindValue(":end_seconds", climb.endSeconds);
00083     q.bindValue(":orig_start", climb.originalStartSeconds < 0 ? QVariant() :
    QVariant(climb.originalStartSeconds));
00084     q.bindValue(":orig_end", climb.originalEndSeconds < 0 ? QVariant() :
    QVariant(climb.originalEndSeconds));
00085     q.bindValue(":name", climb.name);
00086     q.bindValue(":elevation_gain_m", climb.elevationGainM);
00087     q.bindValue(":length_km", climb.lengthKm);
00088     q.bindValue(":average_gradient", climb.averageGradient);
00089     q.bindValue(":avg_power", climb.avgPower);
00090     q.bindValue(":np", climb.np);
00091     q.bindValue(":avg_hr", climb.avgHr);
00092     q.bindValue(":avg_cadence", climb.avgCadence);
00093     q.bindValue(":vam", climb.vam);
00094     q.bindValue(":source", climb.source);
00095     SqlQueryHelper::bindBoolValue(q, ":locked", climb.locked);
00096     SqlQueryHelper::bindBoolValue(q, ":deleted", climb.deleted);
00097     q.bindValue(":algorithm_version", climb.algorithmVersion);
00098     q.bindValue(":notes", climb.notes);
00099     q.bindValue(":favorite", climb.favorite ? 1 : 0);
00100     q.bindValue(":rating", climb.rating);
00101
00102     if (!q.exec())
00103         return -1;
00104     return q.lastInsertId().toInt();
00105 }
00106
00107 bool ClimbRepository::updateClimb(const ClimbRecord& climb)
00108 {
00109     QSqlQuery q(m_db);
00110     q.prepare(
00111         "UPDATE climbs SET"
00112         "  start_seconds = :start_seconds,"
00113         "  end_seconds   = :end_seconds,"
00114         "  name          = :name,"
00115         "  elevation_gain_m = :elevation_gain_m,"
00116         "  length_km      = :length_km,"
00117         "  average_gradient = :average_gradient,"
00118         "  avg_power      = :avg_power,"
00119         "  np            = :np,"
00120         "  avg_hr         = :avg_hr,"
00121         "  avg_cadence    = :avg_cadence,"
00122         "  vam           = :vam,"
00123         "  source         = :source,"
00124         "  locked         = :locked,"
00125         "  deleted        = :deleted,"
00126         "  notes          = :notes,"
00127         "  favorite       = :favorite,"
00128         "  rating         = :rating,"
00129         "  updated_at     = CURRENT_TIMESTAMP"
00130         " WHERE id = :id");
00131
00132     q.bindValue(":start_seconds", climb.startSeconds);
00133     q.bindValue(":end_seconds", climb.endSeconds);
00134     q.bindValue(":name", climb.name);
00135     q.bindValue(":elevation_gain_m", climb.elevationGainM);
00136     q.bindValue(":length_km", climb.lengthKm);
00137     q.bindValue(":average_gradient", climb.averageGradient);
00138     q.bindValue(":avg_power", climb.avgPower);
00139     q.bindValue(":np", climb.np);
00140     q.bindValue(":avg_hr", climb.avgHr);
00141     q.bindValue(":avg_cadence", climb.avgCadence);
00142     q.bindValue(":vam", climb.vam);
00143     q.bindValue(":source", climb.source);
00144     SqlQueryHelper::bindBoolValue(q, ":locked", climb.locked);
00145     SqlQueryHelper::bindBoolValue(q, ":deleted", climb.deleted);
00146     q.bindValue(":notes", climb.notes);
00147     q.bindValue(":favorite", climb.favorite ? 1 : 0);
00148     q.bindValue(":rating", climb.rating);
00149     q.bindValue(":id", climb.id);
00150     return q.exec();
00151 }
00152
00153 bool ClimbRepository::softDeleteClimb(int climbId)
00154 {
00155     QSqlQuery q(m_db);
00156     q.prepare("UPDATE climbs SET deleted=1, locked=1, updated_at=CURRENT_TIMESTAMP WHERE id=:id");
00157     q.bindValue(":id", climbId);
00158     return q.exec();
00159 }
00160
00161 bool ClimbRepository::hardDeleteClimb(int climbId)
00162 {
00163     QSqlQuery q(m_db);
00164     q.prepare("DELETE FROM climbs WHERE id=:id");

```

```

00165     q.bindValue(":id", climbId);
00166     return q.exec();
00167 }
00168
00169 bool ClimbRepository::deleteClimbsForActivity(int activityId)
00170 {
00171     QSqlQuery q(m_db);
00172     q.prepare("DELETE FROM climbs WHERE activity_id=:id");
00173     q.bindValue(":id", activityId);
00174     return q.exec();
00175 }
00176
00177 ClimbRecord ClimbRepository::getClimb(int climbId)
00178 {
00179     QSqlQuery q(m_db);
00180     q.prepare(QString("SELECT %1 FROM climbs WHERE id=:id").arg(kSelectCols));
00181     q.bindValue(":id", climbId);
00182     if (q.exec() && q.next())
00183         return recordFromQuery(q);
00184     return {};
00185 }
00186
00187 QList<ClimbRecord> ClimbRepository::climbsForActivity(int activityId)
00188 {
00189     QList<ClimbRecord> out;
00190     QSqlQuery q(m_db);
00191     q.prepare(QString("SELECT %1 FROM climbs WHERE activity_id=:id AND COALESCE(deleted,0)=0"
00192         " ORDER BY start_seconds ASC").arg(kSelectCols));
00193     q.bindValue(":id", activityId);
00194     if (!q.exec())
00195         return out;
00196     while (q.next())
00197         out.push_back(recordFromQuery(q));
00198     return out;
00199 }
00200
00201 QList<ClimbRecord> ClimbRepository::deletedClimbsForActivity(int activityId)
00202 {
00203     QList<ClimbRecord> out;
00204     QSqlQuery q(m_db);
00205     q.prepare(QString("SELECT %1 FROM climbs WHERE activity_id=:id AND COALESCE(deleted,0)=1"
00206         " ORDER BY start_seconds ASC").arg(kSelectCols));
00207     q.bindValue(":id", activityId);
00208     if (!q.exec())
00209         return out;
00210     while (q.next())
00211         out.push_back(recordFromQuery(q));
00212     return out;
00213 }
00214
00215 bool ClimbRepository::hasClimbs(int activityId)
00216 {
00217     QSqlQuery q(m_db);
00218     q.prepare(SqlQueryHelper::selectExistsNonDeleted("climbs"));
00219     q.bindValue(":id", activityId);
00220     return q.exec() && q.next();
00221 }
00222
00223 bool ClimbRepository::hasClimbsEver(int activityId)
00224 {
00225     QSqlQuery q(m_db);
00226     q.prepare(SqlQueryHelper::selectExistsEver("climbs"));
00227     q.bindValue(":id", activityId);
00228     return q.exec() && q.next();
00229 }
00230
00231 bool ClimbRepository::hasLockedClimbs(int activityId)
00232 {
00233     QSqlQuery q(m_db);
00234     q.prepare(SqlQueryHelper::selectExistsLocked("climbs"));
00235     q.bindValue(":id", activityId);
00236     return q.exec() && q.next();
00237 }

```

## 8.70 ClimbRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008

```



```

00009 class QSqlDatabase;
00010
00016 struct ClimbRecord
00017 {
00019     int      id                = -1;
00020
00022     int      activityId       = -1;
00023
00025     QString  uuid;
00026
00028     double   startSeconds     = 0.0;
00029
00031     double   endSeconds       = 0.0;
00032
00034     double   originalStartSeconds = -1.0;
00035
00037     double   originalEndSeconds = -1.0;
00038
00040     QString  name;
00041
00043     double   elevationGainM   = 0.0;
00044
00046     double   lengthKm         = 0.0;
00047
00049     double   averageGradient  = 0.0;
00050
00052     double   avgPower         = 0.0;
00053
00055     double   np               = 0.0;
00056
00058     double   avgHr            = 0.0;
00059
00061     double   avgCadence       = 0.0;
00062
00064     double   vam              = 0.0;
00065
00067     QString  source           = QStringLiteral("auto");
00068
00070     bool     locked           = false;
00071
00073     bool     deleted          = false;
00074
00076     int      algorithmVersion = 1;
00077
00079     QString  notes;
00080
00082     bool     favorite = false;
00083
00085     int      rating    = 0;
00086 };
00087
00093 class ClimbRepository
00094 {
00095 public:
00100     explicit ClimbRepository(QSqlDatabase& db);
00101
00107     int      insertClimb(const ClimbRecord& climb);
00108
00114     bool     updateClimb(const ClimbRecord& climb);
00115
00121     bool     softDeleteClimb(int climbId);
00122
00128     bool     hardDeleteClimb(int climbId);
00129
00135     bool     deleteClimbsForActivity(int activityId);
00136
00142     ClimbRecord      getClimb(int climbId);
00143
00149     QList<ClimbRecord>  climbsForActivity(int activityId);
00150
00156     QList<ClimbRecord>  deletedClimbsForActivity(int activityId);
00157
00163     bool     hasClimbs(int activityId);
00164
00170     bool     hasClimbsEver(int activityId);
00171
00177     bool     hasLockedClimbs(int activityId);
00178
00179 private:
00180     QSqlDatabase& m_db;
00181 };

```

## 8.71 DatabaseConnectionFactory.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "DatabaseConnectionFactory.h"
00004
00005 #include <QSqlDatabase>
00006 #include <QSqlError>
00007 #include <QSqlQuery>
00008 #include <QUuid>
00009
00010 namespace DatabaseConnectionFactory
00011 {
00012     QString createConnectionName()
00013     {
00014         return QUuid::createUuid().toString(QUuid::WithoutBraces);
00015     }
00016
00017     bool openSqlite(QSqlDatabase& db,
00018                   const QString& path,
00019                   const QString& connectionName,
00020                   QString* errorOut)
00021     {
00022         db = QSqlDatabase::addDatabase("QSQLITE", connectionName);
00023         db.setDatabaseName(path);
00024
00025         if (db.open())
00026             return true;
00027
00028         if (errorOut)
00029             *errorOut = db.lastError().text();
00030         QSqlDatabase::removeDatabase(connectionName);
00031         return false;
00032     }
00033
00034     void applySqlitePragmas(QSqlDatabase& db)
00035     {
00036         QSqlQuery q(db);
00037         q.exec("PRAGMA journal_mode=WAL");
00038         q.exec("PRAGMA synchronous=NORMAL");
00039         q.exec("PRAGMA temp_store=MEMORY");
00040         q.exec("PRAGMA foreign_keys=ON");
00041         q.exec("PRAGMA cache_size=-20000");
00042     }
00043 }

```

## 8.72 DatabaseConnectionFactory.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 class QSqlDatabase;
00008
00009 namespace DatabaseConnectionFactory
00010 {
00011     QString createConnectionName();
00012     bool openSqlite(QSqlDatabase& db,
00013                   const QString& path,
00014                   const QString& connectionName,
00015                   QString* errorOut);
00016     void applySqlitePragmas(QSqlDatabase& db);
00017 }

```

## 8.73 DatabaseConnectionGuard.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseConnectionGuard.h"
00005
00006 DatabaseConnectionGuard::DatabaseConnectionGuard(const QString& connectionName)
00007     : m_connectionName(connectionName)
00008 {
00009     // Create the connection
00010     if (!QSqlDatabase::contains(connectionName))
00011     {
00012         QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connectionName);

```

```

00013     }
00014 }
00015
00016 DatabaseConnectionGuard::~DatabaseConnectionGuard()
00017 {
00018     // Ensure connection is closed before removal
00019     if (QSqlDatabase::contains(m_connectionName))
00020     {
00021         {
00022             QSqlDatabase db = QSqlDatabase::database(m_connectionName, false);
00023             if (db.isOpen())
00024             {
00025                 db.close();
00026             }
00027         }
00028         // Remove the connection from Qt's registry
00029         QSqlDatabase::removeDatabase(m_connectionName);
00030     }
00031 }
00032
00033 QSqlDatabase DatabaseConnectionGuard::database() const
00034 {
00035     return QSqlDatabase::database(m_connectionName, false);
00036 }

```

## 8.74 DatabaseConnectionGuard.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QSqlDatabase>
00007 #include <QString>
00008
00015 class DatabaseConnectionGuard
00016 {
00017 public:
00022     explicit DatabaseConnectionGuard(const QString& connectionName);
00023     ~DatabaseConnectionGuard();
00024
00025     // Non-copyable
00026     DatabaseConnectionGuard(const DatabaseConnectionGuard&) = delete;
00027     DatabaseConnectionGuard& operator=(const DatabaseConnectionGuard&) = delete;
00028
00033     QSqlDatabase database() const;
00034
00035 private:
00036     QString m_connectionName;
00037 };

```

## 8.75 DatabaseManager.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseManager.h"
00005 #include "DatabaseSchema.h"
00006 #include "database/connection/DatabaseConnectionFactory.h"
00007 #include "database/migrations/DatabaseMigrationManager.h"
00008
00009 #include <QSqlError>
00010 #include <QSqlQuery>
00011
00012 DatabaseManager::DatabaseManager() = default;
00013
00014 DatabaseManager::~DatabaseManager()
00015 {
00016     close();
00017 }
00018
00019 // Private helpers
00020
00021 bool DatabaseManager::applySchema(QString* errorOut)
00022 {
00023     return DatabaseMigrationManager::applySchema(m_db, errorOut);
00024 }
00025
00026 bool DatabaseManager::upgradeSchema(int fromVersion, QString* errorOut)
00027 {
00028     return DatabaseMigrationManager::upgradeSchema(m_db, fromVersion, errorOut);

```

```

00029 }
00030
00031 // Public API
00032
00033 bool DatabaseManager::open(const QString& path, QString* errorOut)
00034 {
00035     close();
00036
00037     m_connectionName = DatabaseConnectionFactory::createConnectionName();
00038     if (!DatabaseConnectionFactory::openSqlite(m_db, path, m_connectionName, errorOut))
00039     {
00040         m_connectionName.clear();
00041         return false;
00042     }
00043
00044     DatabaseConnectionFactory::applySqlitePragmas(m_db);
00045
00046     // Check schema version
00047     QSqlQuery q(m_db);
00048     q.exec("SELECT version FROM schema_info ORDER BY rowid DESC LIMIT 1");
00049     if (q.next())
00050     {
00051         const int version = q.value(0).toInt();
00052         if (version < DatabaseSchema::kCurrentVersion)
00053         {
00054             if (!upgradeSchema(version, errorOut))
00055             {
00056                 cleanupFailedOpen();
00057                 return false;
00058             }
00059         }
00060     }
00061     else
00062     {
00063         // schema_info missing: fresh file or legacy -- apply schema
00064         if (!applySchema(errorOut))
00065         {
00066             cleanupFailedOpen();
00067             return false;
00068         }
00069     }
00070
00071     m_path = path;
00072     return true;
00073 }
00074
00075 void DatabaseManager::cleanupFailedOpen()
00076 {
00077     if (m_db.isOpen())
00078         m_db.close();
00079
00080     if (!m_connectionName.isEmpty())
00081     {
00082         QString conn = m_connectionName;
00083         m_db = QSqlDatabase();
00084         QSqlDatabase::removeDatabase(conn);
00085         m_connectionName.clear();
00086     }
00087 }
00088
00089 bool DatabaseManager::create(const QString& path, QString* errorOut)
00090 {
00091     // Sqlite creates the file if it doesn't exist -- same as open
00092     return open(path, errorOut);
00093 }
00094
00095 void DatabaseManager::close()
00096 {
00097     if (m_db.isOpen())
00098         m_db.close();
00099
00100     if (!m_connectionName.isEmpty())
00101     {
00102         QSqlDatabase::removeDatabase(m_connectionName);
00103         m_connectionName.clear();
00104     }
00105
00106     m_path.clear();
00107     m_db = QSqlDatabase(); // reset to default-constructed
00108 }
00109
00110 bool DatabaseManager::isOpen() const
00111 {
00112     return m_db.isValid() && m_db.isOpen();
00113 }
00114
00115 QSqlDatabase DatabaseManager::database() const

```

```

00116 {
00117     return m_db;
00118 }

```

## 8.76 DatabaseManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QSqlDatabase>
00007 #include <QString>
00008
00009 // DatabaseManager owns a single named QSqlDatabase connection.
00010 // Usage:
00011 //     DatabaseManager db;
00012 //     if (!db.open("/path/to/file.fitlyzerdb", &errorMsg)) { ... }
00013 //     db.database(); // use for queries
00020 class DatabaseManager
00021 {
00022 public:
00023     DatabaseManager();
00024     ~DatabaseManager();
00025
00032     bool open(const QString& path, QString* errorOut = nullptr);
00033
00040     bool create(const QString& path, QString* errorOut = nullptr);
00041
00045     void close();
00046
00051     bool isOpen() const;
00052
00057     QSqlDatabase database() const;
00058
00063     QString path() const { return m_path; }
00064
00065 private:
00066     bool applySchema(QString* errorOut);
00067     bool upgradeSchema(int fromVersion, QString* errorOut);
00068     void cleanupFailedOpen();
00069
00070     QString m_path;
00071     QString m_connectionName;
00072     QSqlDatabase m_db;
00073 };

```

## 8.77 DatabaseSchema.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "DatabaseSchema.h"
00005 // All definitions are header-only (constexpr / inline).

```

## 8.78 DatabaseSchema.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <array>
00007 #include <string_view>
00008
00009 namespace DatabaseSchema {
00010
00014 static constexpr int kCurrentVersion = 8;
00015
00022 inline constexpr std::array kStatements = {
00023     std::string_view{
00024         "CREATE TABLE IF NOT EXISTS schema_info ("
00025         "    version    INTEGER NOT NULL,"
00026         "    created_at TEXT    NOT NULL"
00027         ")"},
00028 },
00029     std::string_view{
00030         "CREATE TABLE IF NOT EXISTS athletes ("

```

```

00031         " id            INTEGER PRIMARY KEY AUTOINCREMENT,"
00032         " first_name     TEXT      NOT NULL,"
00033         " last_name      TEXT      NOT NULL,"
00034         " date_of_birth  TEXT,"
00035         " email          TEXT,"
00036         " height_cm      REAL,"
00037         " ftp            INTEGER NOT NULL DEFAULT 250,"
00038         " created_at     TEXT      NOT NULL"
00039     )"
00040 },
00041 std::string_view{
00042     "CREATE TABLE IF NOT EXISTS ftp_history ("
00043     " id            INTEGER PRIMARY KEY AUTOINCREMENT,"
00044     " athlete_id    INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00045     " ftp_watts      INTEGER NOT NULL,"
00046     " effective_from TEXT      NOT NULL,"
00047     " notes          TEXT"
00048     ")"
00049 },
00050 std::string_view{
00051     "CREATE TABLE IF NOT EXISTS weight_history ("
00052     " id            INTEGER PRIMARY KEY AUTOINCREMENT,"
00053     " athlete_id    INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00054     " weight_kg      REAL      NOT NULL,"
00055     " effective_from TEXT      NOT NULL"
00056     ")"
00057 },
00058 std::string_view{
00059     "CREATE TABLE IF NOT EXISTS activities ("
00060     " id            INTEGER PRIMARY KEY AUTOINCREMENT,"
00061     " athlete_id    INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00062     " fit_hash       TEXT      UNIQUE,"
00063     " file_name      TEXT,"
00064     " sport          TEXT,"
00065     " activity_start_time TEXT NOT NULL,"
00066     " end_time       TEXT,"
00067     " duration_sec   REAL,"
00068     " distance_m     REAL,"
00069     " avg_power       REAL,"
00070     " max_power       REAL,"
00071     " normalized_power REAL,"
00072     " avg_hr          REAL,"
00073     " max_hr          REAL,"
00074     " avg_cadence     REAL,"
00075     " avg_speed       REAL,"
00076     " elevation_gain  REAL,"
00077     " notes           TEXT,"
00078     " weather_notes   TEXT,"
00079     " temperature     REAL,"
00080     " weather         TEXT,"
00081     " wind            TEXT,"
00082     " rpe             INTEGER,"
00083     " fatigue         INTEGER,"
00084     " sleep           REAL,"
00085     " weight          REAL,"
00086     " bike            TEXT,"
00087     " equipment       TEXT,"
00088     " import_time     TEXT      NOT NULL,"
00089     " analysis_version INTEGER NOT NULL DEFAULT 1,"
00090     " fingerprint     TEXT,"
00091     " analysis_flags   INTEGER NOT NULL DEFAULT 0,"
00092     " import_source    TEXT"
00093     ")"
00094 },
00095 std::string_view{
00096     "CREATE INDEX IF NOT EXISTS idx_activities_activity_start_time"
00097     " ON activities(activity_start_time DESC)"
00098 },
00099 std::string_view{
00100     "CREATE INDEX IF NOT EXISTS idx_activities_athlete_start_time"
00101     " ON activities(athlete_id, activity_start_time DESC)"
00102 },
00103 std::string_view{
00104     "CREATE TABLE IF NOT EXISTS activity_samples ("
00105     " id            INTEGER PRIMARY KEY AUTOINCREMENT,"
00106     " activity_id    INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00107     " elapsed_seconds REAL      NOT NULL,"
00108     " power_total    REAL,"
00109     " heart_rate     REAL,"
00110     " cadence        REAL,"
00111     " speed          REAL,"
00112     " latitude       REAL,"
00113     " longitude      REAL,"
00114     " altitude       REAL,"
00115     " has_gps        INTEGER NOT NULL DEFAULT 0,"
00116     " has_power       INTEGER NOT NULL DEFAULT 0,"
00117     " has_heart_rate  INTEGER NOT NULL DEFAULT 0,"

```

```

00118         " has_cadence          INTEGER NOT NULL DEFAULT 0,"
00119         " has_speed            INTEGER NOT NULL DEFAULT 0,"
00120         " has_altitude         INTEGER NOT NULL DEFAULT 0"
00121     )"
00122 },
00123 std::string_view{
00124     "CREATE INDEX IF NOT EXISTS idx_activity_samples_activity_id"
00125     " ON activity_samples(activity_id)"
00126 },
00127 std::string_view{
00128     "CREATE INDEX IF NOT EXISTS idx_activity_samples_time"
00129     " ON activity_samples(activity_id, elapsed_seconds)"
00130 },
00131 std::string_view{
00132     "CREATE TABLE IF NOT EXISTS laps ("
00133     " id                INTEGER PRIMARY KEY AUTOINCREMENT,"
00134     " activity_id       INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00135     " lap_number        INTEGER NOT NULL,"
00136     " start_time        TEXT,"
00137     " duration_sec      REAL,"
00138     " distance_m         REAL,"
00139     " avg_power         REAL,"
00140     " avg_hr            REAL"
00141     ")"
00142 },
00143 std::string_view{
00144     "CREATE TABLE IF NOT EXISTS tags ("
00145     " id                INTEGER PRIMARY KEY AUTOINCREMENT,"
00146     " name              TEXT NOT NULL UNIQUE"
00147     ")"
00148 },
00149 std::string_view{
00150     "CREATE TABLE IF NOT EXISTS activity_tags ("
00151     " activity_id       INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00152     " tag_id            INTEGER NOT NULL REFERENCES tags(id) ON DELETE CASCADE,"
00153     " PRIMARY KEY (activity_id, tag_id)"
00154     ")"
00155 },
00156 std::string_view{
00157     "CREATE TABLE IF NOT EXISTS intervals ("
00158     " id                INTEGER PRIMARY KEY AUTOINCREMENT,"
00159     " activity_id       INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00160     " start_sample      INTEGER NOT NULL,"
00161     " end_sample        INTEGER NOT NULL,"
00162     " name              TEXT,"
00163     " type              TEXT,"
00164     " avg_power         REAL,"
00165     " np                REAL,"
00166     " avg_hr            REAL,"
00167     " avg_cadence       REAL,"
00168     " notes             TEXT,"
00169     " source            TEXT NOT NULL DEFAULT 'auto',"
00170     " locked            INTEGER NOT NULL DEFAULT 0,"
00171     " updated_at        TEXT,"
00172     " algorithm_version INTEGER NOT NULL DEFAULT 1,"
00173     " deleted           INTEGER NOT NULL DEFAULT 0,"
00174     " uuid              TEXT"
00175     ")"
00176 },
00177 std::string_view{
00178     "CREATE TABLE IF NOT EXISTS climbs ("
00179     " id                INTEGER PRIMARY KEY AUTOINCREMENT,"
00180     " activity_id       INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00181     " start_seconds     REAL NOT NULL,"
00182     " end_seconds       REAL NOT NULL,"
00183     " name              TEXT,"
00184     " elevation_gain_m  REAL,"
00185     " length_km         REAL,"
00186     " average_gradient  REAL,"
00187     " source            TEXT NOT NULL DEFAULT 'auto',"
00188     " locked            INTEGER NOT NULL DEFAULT 0,"
00189     " created_at        TEXT DEFAULT CURRENT_TIMESTAMP,"
00190     " updated_at        TEXT DEFAULT CURRENT_TIMESTAMP,"
00191     " algorithm_version INTEGER NOT NULL DEFAULT 1,"
00192     " deleted           INTEGER NOT NULL DEFAULT 0,"
00193     " uuid              TEXT,"
00194     " original_start_seconds REAL,"
00195     " original_end_seconds REAL,"
00196     " avg_power         REAL,"
00197     " np                REAL,"
00198     " avg_hr            REAL,"
00199     " avg_cadence       REAL,"
00200     " vam              REAL,"
00201     " notes             TEXT,"
00202     " favorite          INTEGER NOT NULL DEFAULT 0,"
00203     " rating            INTEGER NOT NULL DEFAULT 0"
00204     ")"

```

```

00205 },
00206 std::string_view{
00207     "CREATE INDEX IF NOT EXISTS idx_climbs_activity_id"
00208     " ON climbs(activity_id)"
00209 },
00210 std::string_view{
00211     "CREATE UNIQUE INDEX IF NOT EXISTS idx_activity_filehash"
00212     " ON activities(fit_hash) WHERE fit_hash IS NOT NULL"
00213 },
00214 std::string_view{
00215     "CREATE INDEX IF NOT EXISTS idx_activity_start_time"
00216     " ON activities(activity_start_time)"
00217 },
00218 std::string_view{
00219     "CREATE INDEX IF NOT EXISTS idx_activity_end_time"
00220     " ON activities(end_time)"
00221 },
00222 std::string_view{
00223     "CREATE INDEX IF NOT EXISTS idx_intervals_activity"
00224     " ON intervals(activity_id)"
00225 },
00226 std::string_view{
00227     "CREATE TABLE IF NOT EXISTS planned_workouts ("
00228     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00229     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00230     " workout_date TEXT NOT NULL,"
00231     " title TEXT NOT NULL,"
00232     " type TEXT,"
00233     " duration_sec REAL,"
00234     " target_tss REAL,"
00235     " notes TEXT"
00236     ") "
00237 },
00238 std::string_view{
00239     "CREATE INDEX IF NOT EXISTS idx_workouts_athlete_date"
00240     " ON planned_workouts(athlete_id, workout_date)"
00241 },
00242 std::string_view{
00243     "CREATE TABLE IF NOT EXISTS daily_metrics ("
00244     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00245     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00246     " metric_date TEXT NOT NULL,"
00247     " atl REAL NOT NULL DEFAULT 0,"
00248     " ctl REAL NOT NULL DEFAULT 0,"
00249     " tsb REAL NOT NULL DEFAULT 0"
00250     ") "
00251 },
00252 std::string_view{
00253     "CREATE UNIQUE INDEX IF NOT EXISTS idx_daily_metrics_athlete_date"
00254     " ON daily_metrics(athlete_id, metric_date)"
00255 },
00256 std::string_view{
00257     "CREATE TABLE IF NOT EXISTS equipment ("
00258     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00259     " athlete_id INTEGER NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,"
00260     " name TEXT NOT NULL,"
00261     " category TEXT NOT NULL,"
00262     " service_interval_km REAL,"
00263     " service_interval_hours REAL,"
00264     " notes TEXT"
00265     ") "
00266 },
00267 std::string_view{
00268     "CREATE TABLE IF NOT EXISTS equipment_usage ("
00269     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00270     " equipment_id INTEGER NOT NULL REFERENCES equipment(id) ON DELETE CASCADE,"
00271     " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00272     " distance_km REAL,"
00273     " duration_sec REAL"
00274     ") "
00275 },
00276 std::string_view{
00277     "CREATE TABLE IF NOT EXISTS imports ("
00278     " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00279     " file_name TEXT NOT NULL,"
00280     " imported_at TEXT NOT NULL,"
00281     " status TEXT NOT NULL,"
00282     " activity_id INTEGER,"
00283     " error_message TEXT"
00284     ") "
00285 },
00286 std::string_view{
00287     "CREATE INDEX IF NOT EXISTS idx_imports_date"
00288     " ON imports(imported_at)"
00289 },
00290 std::string_view{
00291     "CREATE TABLE IF NOT EXISTS activity_analysis_settings ("

```



```

00292         " activity_id          INTEGER PRIMARY KEY,"
00293         " climb_min_gain      REAL,"
00294         " climb_min_length    REAL,"
00295         " climb_min_gradient  REAL,"
00296         " interval_work_threshold REAL,"
00297         " interval_rest_threshold REAL,"
00298         " FOREIGN KEY(activity_id) REFERENCES activities(id) ON DELETE CASCADE"
00299         ") "
00300     },
00301     std::string_view{
00302         "CREATE TABLE IF NOT EXISTS activity_analysis_cache ("
00303         " activity_id  INTEGER NOT NULL,"
00304         " cache_key   TEXT    NOT NULL,"
00305         " cache_value  BLOB,"
00306         " PRIMARY KEY (activity_id, cache_key)"
00307         ") "
00308     },
00309     // Enable foreign-key enforcement (must be per-connection, set here via a dummy table)
00310     std::string_view{
00311         "PRAGMA foreign_keys = ON"
00312     },
00313 };
00314
00315 } // namespace DatabaseSchema

```

## 8.79 ImportRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ImportRepository.h"
00005
00006 #include <QDateTime>
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009
00010 ImportRepository::ImportRepository(QSqlDatabase& db)
00011     : m_db(db)
00012 {}
00013
00014 int ImportRepository::logImport(const ImportRecord& r)
00015 {
00016     QSqlQuery q(m_db);
00017     q.prepare(
00018         "INSERT INTO imports(file_name, imported_at, status, activity_id, error_message)"
00019         " VALUES(:fn, :ts, :st, :aid, :err)");
00020     q.bindValue(":fn", r.fileName());
00021     q.bindValue(":ts", r.importedAt().isEmpty()
00022         ? QDateTime::currentDateTimeUtc().toString(Qt::ISODate)
00023         : r.importedAt());
00024     q.bindValue(":st", r.status());
00025     q.bindValue(":aid", r.activityId() > 0 ? QVariant(r.activityId) : QVariant());
00026     q.bindValue(":err", r.errorMessage().isEmpty() ? QVariant() : QVariant(r.errorMessage));
00027     if (!q.exec()) return -1;
00028     return q.lastInsertId().toInt();
00029 }
00030
00031 QList<ImportRecord> ImportRepository::getRecentImports(int limit)
00032 {
00033     QList<ImportRecord> result;
00034     QSqlQuery q(m_db);
00035     q.prepare(
00036         "SELECT id, file_name, imported_at, status, activity_id, error_message"
00037         " FROM imports ORDER BY imported_at DESC LIMIT :lim");
00038     q.bindValue(":lim", limit);
00039     q.exec();
00040     while (q.next())
00041     {
00042         ImportRecord r;
00043         r.id = q.value(0).toInt();
00044         r.fileName = q.value(1).toString();
00045         r.importedAt = q.value(2).toString();
00046         r.status = q.value(3).toString();
00047         r.activityId = q.value(4).toInt();
00048         r.errorMessage = q.value(5).toString();
00049         result.append(r);
00050     }
00051     return result;
00052 }

```

## 8.80 ImportRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct ImportRecord
00015 {
00017     int      id          = -1;
00018
00020     QString  fileName;
00021
00023     QString  importedAt;
00024
00026     QString  status;
00027
00029     int      activityId   = -1;
00030
00032     QString  errorMessage;
00033 };
00034
00040 class ImportRepository
00041 {
00042 public:
00047     explicit ImportRepository(QSqlDatabase& db);
00048
00054     int      logImport(const ImportRecord& r);
00055
00061     QList<ImportRecord> getRecentImports(int limit = 50);
00062
00063 private:
00064     QSqlDatabase& m_db;
00065 };

```

## 8.81 IntervalRepository.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "IntervalRepository.h"
00005 #include "SqlQueryHelper.h"
00006
00007 #include <QSqlDatabase>
00008 #include <QSqlQuery>
00009
00010 IntervalRepository::IntervalRepository(QSqlDatabase& db)
00011     : m_db(db)
00012 {}
00013
00014 int IntervalRepository::insertInterval(const IntervalRecord& interval)
00015 {
00016     QSqlQuery q(m_db);
00017     q.prepare(
00018         "INSERT INTO intervals("
00019         "    activity_id, start_sample, end_sample, name, type,"
00020         "    avg_power, np, avg_hr, avg_cadence, notes,"
00021         "    source, locked, deleted, algorithm_version, uuid"
00022         ") VALUES("
00023         "    :activity_id, :start_sample, :end_sample, :name, :type,"
00024         "    :avg_power, :np, :avg_hr, :avg_cadence, :notes,"
00025         "    :source, :locked, :deleted, :algorithm_version, :uuid"
00026         ")");
00027
00028     q.bindValue(":activity_id", interval.activityId);
00029     q.bindValue(":start_sample", interval.startSample);
00030     q.bindValue(":end_sample", interval.endSample);
00031     q.bindValue(":name", interval.name);
00032     q.bindValue(":type", interval.type);
00033     q.bindValue(":avg_power", interval.avgPower);
00034     q.bindValue(":np", interval.np);
00035     q.bindValue(":avg_hr", interval.avgHr);
00036     q.bindValue(":avg_cadence", interval.avgCadence);
00037     q.bindValue(":notes", interval.notes);
00038     q.bindValue(":source", interval.source);
00039     SqlQueryHelper::bindBoolValue(q, ":locked", interval.locked);
00040     SqlQueryHelper::bindBoolValue(q, ":deleted", interval.deleted);
00041     q.bindValue(":algorithm_version", interval.algorithmVersion);

```

```

00042     q.bindValue(":uuid",                interval.uuid.isEmpty() ? QVariant() : QVariant(interval.uuid));
00043
00044     if (!q.exec()) return -1;
00045     return q.lastInsertId().toInt();
00046 }
00047
00048 bool IntervalRepository::updateInterval(const IntervalRecord& interval)
00049 {
00050     QSqlQuery q(m_db);
00051     q.prepare(
00052         "UPDATE intervals SET"
00053         "  start_sample = :start_sample,"
00054         "  end_sample   = :end_sample,"
00055         "  name         = :name,"
00056         "  type         = :type,"
00057         "  avg_power    = :avg_power,"
00058         "  np          = :np,"
00059         "  avg_hr       = :avg_hr,"
00060         "  avg_cadence  = :avg_cadence,"
00061         "  notes        = :notes,"
00062         "  source       = :source,"
00063         "  locked       = :locked,"
00064         "  deleted      = :deleted,"
00065         "  updated_at   = CURRENT_TIMESTAMP"
00066         " WHERE id = :id");
00067
00068     q.bindValue(":start_sample", interval.startSample);
00069     q.bindValue(":end_sample",   interval.endSample);
00070     q.bindValue(":name",        interval.name);
00071     q.bindValue(":type",        interval.type);
00072     q.bindValue(":avg_power",   interval.avgPower);
00073     q.bindValue(":np",          interval.np);
00074     q.bindValue(":avg_hr",      interval.avgHr);
00075     q.bindValue(":avg_cadence", interval.avgCadence);
00076     q.bindValue(":notes",       interval.notes);
00077     q.bindValue(":source",      interval.source);
00078     SqlQueryHelper::bindBoolValue(q, ":locked",    interval.locked);
00079     SqlQueryHelper::bindBoolValue(q, ":deleted",   interval.deleted);
00080     q.bindValue(":id",         interval.id);
00081     return q.exec();
00082 }
00083
00084 bool IntervalRepository::softDeleteInterval(int intervalId)
00085 {
00086     QSqlQuery q(m_db);
00087     q.prepare("UPDATE intervals SET deleted=1, locked=1, updated_at=CURRENT_TIMESTAMP WHERE id=:id");
00088     q.bindValue(":id", intervalId);
00089     return q.exec();
00090 }
00091
00092 bool IntervalRepository::deleteInterval(int intervalId)
00093 {
00094     QSqlQuery q(m_db);
00095     q.prepare("DELETE FROM intervals WHERE id=:id");
00096     q.bindValue(":id", intervalId);
00097     return q.exec();
00098 // deleteInterval remains a hard delete (used from UI when permanent removal is wanted)
00099 }
00100
00101 bool IntervalRepository::deleteIntervalsForActivity(int activityId)
00102 {
00103     QSqlQuery q(m_db);
00104     q.prepare("DELETE FROM intervals WHERE activity_id=:id");
00105     q.bindValue(":id", activityId);
00106     return q.exec();
00107 }
00108
00109 QList<IntervalRecord> IntervalRepository::listIntervalsForActivity(int activityId)
00110 {
00111     QList<IntervalRecord> out;
00112     QSqlQuery q(m_db);
00113     q.prepare(
00114         "SELECT id, activity_id, start_sample, end_sample, name, type,"
00115         "  avg_power, np, avg_hr, avg_cadence, notes,"
00116         "  COALESCE(source,'auto'), COALESCE(locked,0), COALESCE(deleted,0),"
00117         "  COALESCE(algorithm_version,1), COALESCE(uuid,'') "
00118         " FROM intervals WHERE activity_id=:id AND COALESCE(deleted,0)=0"
00119         " ORDER BY start_sample ASC");
00120     q.bindValue(":id", activityId);
00121     q.exec();
00122
00123     while (q.next())
00124     {
00125         IntervalRecord record;
00126         int col = 0;
00127         record.id = q.value(col++).toInt();
00128         record.activityId = q.value(col++).toInt();

```

```

00129         record.startSample      = q.value(col++).toInt();
00130         record.endSample        = q.value(col++).toInt();
00131         record.name              = q.value(col++).toString();
00132         record.type              = q.value(col++).toString();
00133         record.avgPower          = q.value(col++).toDouble();
00134         record.np                = q.value(col++).toDouble();
00135         record.avgHr             = q.value(col++).toDouble();
00136         record.avgCadence        = q.value(col++).toDouble();
00137         record.notes             = q.value(col++).toString();
00138         record.source            = q.value(col++).toString();
00139         record.locked            = q.value(col++).toInt() != 0;
00140         record.deleted           = q.value(col++).toInt() != 0;
00141         record.algorithmVersion  = q.value(col++).toInt();
00142         record.uuid              = q.value(col++).toString();
00143         out.push_back(record);
00144     }
00145     return out;
00146 }
00147
00148 bool IntervalRepository::hasLockedIntervals(int activityId)
00149 {
00150     QSqlQuery q(m_db);
00151     q.prepare(SqlQueryHelper::selectExistsLocked("intervals"));
00152     q.bindValue(":id", activityId);
00153     return q.exec() && q.next();
00154 }

```

## 8.82 IntervalRepository.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QList>
00007 #include <QString>
00008
00009 class QSqlDatabase;
00010
00014 struct IntervalRecord
00015 {
00017     int id = -1;
00018
00020     int activityId = -1;
00021
00023     int startSample = 0;
00024
00026     int endSample = 0;
00027
00029     QString name;
00030
00032     QString type;
00033
00035     double avgPower = 0.0;
00036
00038     double np = 0.0;
00039
00041     double avgHr = 0.0;
00042
00044     double avgCadence = 0.0;
00045
00047     QString notes;
00048
00050     QString source      = QStringLiteral("auto");
00051
00053     bool locked         = false;
00054
00056     bool deleted        = false;
00057
00059     int algorithmVersion = 1;
00060
00062     QString uuid;
00063 };
00064
00070 class IntervalRepository
00071 {
00072 public:
00077     explicit IntervalRepository(QSqlDatabase& db);
00078
00084     int insertInterval(const IntervalRecord& interval);
00085
00091     bool updateInterval(const IntervalRecord& interval);
00092
00094     bool softDeleteInterval(int intervalId);

```

```

00096     bool deleteInterval(int intervalId);
00097
00103     bool deleteIntervalsForActivity(int activityId);
00104
00106     QList<IntervalRecord> listIntervalsForActivity(int activityId);
00107
00113     bool hasLockedIntervals(int activityId);
00114
00115 private:
00116     QSqlDatabase& m_db;
00117 };

```

## 8.83 DatabaseMigrationManager.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "DatabaseMigrationManager.h"
00004
00005 #include "database/DatabaseSchema.h"
00006
00007 #include <QDateTime>
00008 #include <QSqlError>
00009 #include <QSqlQuery>
00010
00011 namespace
00012 {
00013     bool columnExists(QSqlDatabase& db, const QString& tableName, const QString& columnName)
00014     {
00015         QSqlQuery q(db);
00016         q.exec(QString("PRAGMA table_info(%1)").arg(tableName));
00017         while (q.next())
00018         {
00019             if (q.value(1).toString().compare(columnName, Qt::CaseInsensitive) == 0)
00020                 return true;
00021         }
00022         return false;
00023     }
00024
00025     bool ensureActivityTimeColumns(QSqlDatabase& db, QString* errorOut)
00026     {
00027         if (!columnExists(db, "activities", "activity_start_time"))
00028         {
00029             QSqlQuery q(db);
00030             if (!q.exec("ALTER TABLE activities ADD COLUMN activity_start_time TEXT"))
00031             {
00032                 if (errorOut)
00033                     *errorOut = q.lastError().text();
00034                 return false;
00035             }
00036         }
00037
00038         if (!columnExists(db, "activities", "import_time"))
00039         {
00040             QSqlQuery q(db);
00041             if (!q.exec("ALTER TABLE activities ADD COLUMN import_time TEXT"))
00042             {
00043                 if (errorOut)
00044                     *errorOut = q.lastError().text();
00045                 return false;
00046             }
00047         }
00048
00049         const bool hasLegacyStart = columnExists(db, "activities", "start_time");
00050         const bool hasLegacyImported = columnExists(db, "activities", "imported_at");
00051
00052         QString startExpr = QStringLiteral("activity_start_time = COALESCE(activity_start_time");
00053         if (hasLegacyStart)
00054             startExpr += QStringLiteral(", start_time");
00055         if (hasLegacyImported)
00056             startExpr += QStringLiteral(", imported_at");
00057         startExpr += QStringLiteral(", datetime('now'))");
00058
00059         QString importExpr = QStringLiteral("import_time = COALESCE(import_time");
00060         if (hasLegacyImported)
00061             importExpr += QStringLiteral(", imported_at");
00062         importExpr += QStringLiteral(", datetime('now'))");
00063
00064         {
00065             QSqlQuery q(db);
00066             if (!q.exec(QString("UPDATE activities SET %1, %2").arg(startExpr, importExpr)))
00067             {
00068                 if (errorOut)
00069                     *errorOut = q.lastError().text();
00070                 return false;
00071             }
00072         }
00073     }
00074 }

```

```

00071     }
00072 }
00073
00074 {
00075     QSqlQuery q(db);
00076     q.exec("CREATE INDEX IF NOT EXISTS idx_activities_activity_start_time ON
activities(activity_start_time DESC)");
00077 }
00078
00079     return true;
00080 }
00081 }
00082
00083 namespace DatabaseMigrationManager
00084 {
00085 bool applySchema(QSqlDatabase& db, QString* errorOut)
00086 {
00087     QSqlQuery q(db);
00088
00089     for (const auto& stmt : DatabaseSchema::kStatements)
00090     {
00091         const QString sql = QString::fromUtf8(stmt.data(), static_cast<qsize_t>(stmt.size()));
00092         if (!q.exec(sql))
00093         {
00094             if (errorOut)
00095                 *errorOut = q.lastError().text();
00096             return false;
00097         }
00098     }
00099
00100     if (!ensureActivityTimeColumns(db, errorOut))
00101         return false;
00102
00103     q.prepare("INSERT OR REPLACE INTO schema_info(version, created_at) VALUES(:v, :ts)");
00104     q.bindValue(":v", DatabaseSchema::kCurrentVersion);
00105     q.bindValue(":ts", QDateTime::currentDateTimeUtc().toString(Qt::ISODate));
00106     if (!q.exec())
00107     {
00108         if (errorOut)
00109             *errorOut = q.lastError().text();
00110         return false;
00111     }
00112
00113     return true;
00114 }
00115
00116 bool upgradeSchema(QSqlDatabase& db, int fromVersion, QString* errorOut)
00117 {
00118     if (fromVersion < 2 && !columnExists(db, "athletes", "ftp"))
00119     {
00120         QSqlQuery q(db);
00121         if (!q.exec("ALTER TABLE athletes ADD COLUMN ftp INTEGER NOT NULL DEFAULT 250"))
00122         {
00123             if (errorOut)
00124                 *errorOut = q.lastError().text();
00125             return false;
00126         }
00127     }
00128
00129     if (fromVersion < 3)
00130     {
00131         struct ActivityColumnDef
00132         {
00133             const char* name;
00134             const char* definition;
00135         };
00136
00137         static const ActivityColumnDef kActivityColumns[] = {
00138             {"temperature", "REAL"},
00139             {"weather", "TEXT"},
00140             {"wind", "TEXT"},
00141             {"rpe", "INTEGER"},
00142             {"fatigue", "INTEGER"},
00143             {"sleep", "REAL"},
00144             {"weight", "REAL"},
00145             {"bike", "TEXT"},
00146             {"equipment", "TEXT"},
00147         };
00148
00149         for (const ActivityColumnDef& col : kActivityColumns)
00150         {
00151             if (columnExists(db, "activities", col.name))
00152                 continue;
00153
00154             QSqlQuery q(db);
00155             const QString sql = QString("ALTER TABLE activities ADD COLUMN %1 %2")
                .arg(col.name)

```

```

00157         .arg(col.definition);
00158         if (!q.exec(sql))
00159         {
00160             if (errorOut)
00161                 *errorOut = q.lastError().text();
00162             return false;
00163         }
00164     }
00165 }
00166
00167 if (fromVersion < 4)
00168 {
00169     if (!ensureActivityTimeColumns(db, errorOut))
00170         return false;
00171 }
00172
00173 if (fromVersion < 5)
00174 {
00175     QSqlQuery q(db);
00176     q.exec(
00177         "CREATE INDEX IF NOT EXISTS idx_activities_athlete_start_time"
00178         " ON activities(athlete_id, activity_start_time DESC)");
00179 }
00180
00181 if (fromVersion < 6)
00182 {
00183     {
00184         QSqlQuery q(db);
00185         q.exec(
00186             "CREATE TABLE IF NOT EXISTS climbs ("
00187             " id INTEGER PRIMARY KEY AUTOINCREMENT,"
00188             " activity_id INTEGER NOT NULL REFERENCES activities(id) ON DELETE CASCADE,"
00189             " start_seconds REAL NOT NULL,"
00190             " end_seconds REAL NOT NULL,"
00191             " name TEXT,"
00192             " elevation_gain_m REAL,"
00193             " length_km REAL,"
00194             " average_gradient REAL,"
00195             " source TEXT NOT NULL DEFAULT 'auto',"
00196             " locked INTEGER NOT NULL DEFAULT 0,"
00197             " created_at TEXT DEFAULT CURRENT_TIMESTAMP,"
00198             " updated_at TEXT DEFAULT CURRENT_TIMESTAMP"
00199             " )");
00200     }
00201     {
00202         QSqlQuery q(db);
00203         q.exec("CREATE INDEX IF NOT EXISTS idx_climbs_activity_id ON climbs(activity_id)");
00204     }
00205
00206     struct ColDef { const char* name; const char* definition; };
00207     static const ColDef kIntervalCols[] = {
00208         { "source", "TEXT NOT NULL DEFAULT 'auto'" },
00209         { "locked", "INTEGER NOT NULL DEFAULT 0" },
00210         { "updated_at", "TEXT" },
00211     };
00212     for (const ColDef& col : kIntervalCols)
00213     {
00214         if (columnExists(db, "intervals", col.name))
00215             continue;
00216         QSqlQuery q(db);
00217         q.exec(QString("ALTER TABLE intervals ADD COLUMN %1 %2")
00218             .arg(col.name).arg(col.definition));
00219     }
00220
00221     {
00222         QSqlQuery q(db);
00223         q.exec(
00224             "CREATE UNIQUE INDEX IF NOT EXISTS idx_activity_filehash"
00225             " ON activities(fit_hash) WHERE fit_hash IS NOT NULL");
00226     }
00227     {
00228         QSqlQuery q(db);
00229         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_start_time ON
activities(activity_start_time)");
00230     }
00231     {
00232         QSqlQuery q(db);
00233         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_end_time ON activities(end_time)");
00234     }
00235 }
00236
00237 if (fromVersion < 7)
00238 {
00239     struct ColDef7a { const char* name; const char* def; };
00240     static const ColDef7a kActivityCols[] = {
00241         { "analysis_version", "INTEGER NOT NULL DEFAULT 1" },
00242         { "fingerprint", "TEXT" },

```

```

00243         { "analysis_flags",    "INTEGER NOT NULL DEFAULT 0" },
00244         { "import_source",      "TEXT" },
00245     };
00246     for (const ColDef7a& col : kActivityCols)
00247     {
00248         if (columnExists(db, "activities", col.name)) continue;
00249         QSqlQuery q(db);
00250         q.exec(QString("ALTER TABLE activities ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00251     }
00252
00253     struct ColDef7b { const char* name; const char* def; };
00254     static const ColDef7b kIntervalCols[] = {
00255         { "algorithm_version", "INTEGER NOT NULL DEFAULT 1" },
00256         { "deleted",           "INTEGER NOT NULL DEFAULT 0" },
00257         { "uuid",              "TEXT" },
00258     };
00259     for (const ColDef7b& col : kIntervalCols)
00260     {
00261         if (columnExists(db, "intervals", col.name)) continue;
00262         QSqlQuery q(db);
00263         q.exec(QString("ALTER TABLE intervals ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00264     }
00265
00266     struct ColDef7c { const char* name; const char* def; };
00267     static const ColDef7c kClimbCols[] = {
00268         { "algorithm_version", "INTEGER NOT NULL DEFAULT 1" },
00269         { "deleted",           "INTEGER NOT NULL DEFAULT 0" },
00270         { "uuid",              "TEXT" },
00271         { "original_start_seconds", "REAL" },
00272         { "original_end_seconds",   "REAL" },
00273         { "avg_power",             "REAL" },
00274         { "np",                    "REAL" },
00275         { "avg_hr",                "REAL" },
00276         { "avg_cadence",           "REAL" },
00277         { "vam",                   "REAL" },
00278         { "notes",                 "TEXT" },
00279         { "favorite",              "INTEGER NOT NULL DEFAULT 0" },
00280         { "rating",               "INTEGER NOT NULL DEFAULT 0" },
00281     };
00282     for (const ColDef7c& col : kClimbCols)
00283     {
00284         if (columnExists(db, "climbs", col.name)) continue;
00285         QSqlQuery q(db);
00286         q.exec(QString("ALTER TABLE climbs ADD COLUMN %1 %2").arg(col.name).arg(col.def));
00287     }
00288
00289     {
00290         QSqlQuery q(db);
00291         q.exec(
00292             "CREATE TABLE IF NOT EXISTS activity_analysis_settings ("
00293             "    activity_id          INTEGER PRIMARY KEY,"
00294             "    climb_min_gain      REAL,"
00295             "    climb_min_length   REAL,"
00296             "    climb_min_gradient REAL,"
00297             "    interval_work_threshold REAL,"
00298             "    interval_rest_threshold REAL,"
00299             "    FOREIGN KEY(activity_id) REFERENCES activities(id) ON DELETE CASCADE"
00300             ")");
00301     }
00302     {
00303         QSqlQuery q(db);
00304         q.exec(
00305             "CREATE TABLE IF NOT EXISTS activity_analysis_cache ("
00306             "    activity_id INTEGER NOT NULL,"
00307             "    cache_key   TEXT      NOT NULL,"
00308             "    cache_value BLOB,"
00309             "    PRIMARY KEY (activity_id, cache_key)"
00310             ")");
00311     }
00312 }
00313
00314 if (fromVersion < 8)
00315 {
00316     {
00317         QSqlQuery q(db);
00318         q.exec("CREATE INDEX IF NOT EXISTS idx_activity_samples_time"
00319             " ON activity_samples(activity_id, elapsed_seconds)");
00320     }
00321     {
00322         QSqlQuery q(db);
00323         q.exec("CREATE INDEX IF NOT EXISTS idx_imports_date"
00324             " ON imports(imported_at)");
00325     }
00326     {
00327         QSqlQuery q(db);
00328         q.exec("CREATE INDEX IF NOT EXISTS idx_workouts_athlete_date"
00329             " ON planned_workouts(athlete_id, workout_date)");

```



```

00330     }
00331 }
00332
00333     return applySchema(db, errorOut);
00334 }
00335 }

```

## 8.84 DatabaseMigrationManager.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 class QSqlDatabase;
00008
00009 namespace DatabaseMigrationManager
00010 {
00011     bool applySchema(QSqlDatabase& db, QString* errorOut);
00012     bool upgradeSchema(QSqlDatabase& db, int fromVersion, QString* errorOut);
00013 }

```

## 8.85 SqlQueryHelper.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <string_view>
00006
00007 #include <QSqlQuery>
00008 #include <QString>
00009
00010
00011 namespace SqlQueryHelper {
00012
00013     inline constexpr std::string_view kWhereNotDeleted = "COALESCE(deleted,0)=0";
00014
00015     inline constexpr std::string_view kWhereDeleted = "COALESCE(deleted,0)=1";
00016
00017     inline constexpr std::string_view kWhereLocked = "locked=1";
00018
00019     inline void bindBoolValue(QSqlQuery& query, const QString& placeholder, bool value)
00020     {
00021         query.bindValue(placeholder, value ? 1 : 0);
00022     }
00023
00024     inline QString selectExistsNonDeleted(const QString& tableName)
00025     {
00026         return QString("SELECT 1 FROM %1 WHERE activity_id=:id AND %2 LIMIT 1")
00027             .arg(tableName, QString::fromUtf8(kWhereNotDeleted.data(), kWhereNotDeleted.size()));
00028     }
00029
00030     inline QString selectExistsLocked(const QString& tableName)
00031     {
00032         return QString("SELECT 1 FROM %1 WHERE activity_id=:id AND %2 AND %3 LIMIT 1")
00033             .arg(tableName, QString::fromUtf8(kWhereLocked.data(), kWhereLocked.size()),
00034                 QString::fromUtf8(kWhereNotDeleted.data(), kWhereNotDeleted.size()));
00035     }
00036
00037     inline QString selectExistsEver(const QString& tableName)
00038     {
00039         return QString("SELECT 1 FROM %1 WHERE activity_id=:id LIMIT 1").arg(tableName);
00040     }
00041 } // namespace SqlQueryHelper

```

## 8.86 FitParser.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "FitParser.h"
00004 #include "FitRecordListener.h"
00005
00006 #include <QDateTime>

```

```

00008 #include <QTimeZone>
00009
00010 #include <fstream>
00011 #include <cmath>
00012 #include <numbers>
00013
00014 #include <fit_decode.hpp>
00015 #include <fit_file_id_mesg_listener.hpp>
00016 #include <fit_mesg_broadcaster.hpp>
00017 #include <fit_session_mesg_listener.hpp>
00018
00019 namespace
00020 {
00021   QString fitTimestampToIsoUtc(FIT_DATE_TIME timestamp)
00022   {
00023     if (timestamp == FIT_DATE_TIME_INVALID)
00024       return {};
00025
00026     // FIT epoch: 1989-12-31T00:00:00Z.
00027     static constexpr qint64 kFitEpochToUnixOffset = 631065600;
00028     const qint64 unixSeconds = kFitEpochToUnixOffset + static_cast<qint64>(timestamp);
00029     return QDateTime::fromSecsSinceEpoch(unixSeconds, QTimeZone::UTC).toString(Qt::ISODate);
00030   }
00031
00032   class FitSessionListener final : public fit::SessionMesgListener
00033   {
00034   public:
00035     void OnMesg(fit::SessionMesg& mesg) override
00036     {
00037       if (mesg.IsStartTimeValid())
00038         m_startTime = mesg.GetStartTime();
00039     }
00040
00041     FIT_DATE_TIME startTime() const { return m_startTime; }
00042
00043   private:
00044     FIT_DATE_TIME m_startTime = FIT_DATE_TIME_INVALID;
00045   };
00046
00047   class FitFileIdListener final : public fit::FileIdMesgListener
00048   {
00049   public:
00050     void OnMesg(fit::FileIdMesg& mesg) override
00051     {
00052       if (mesg.IsTimeCreatedValid())
00053         m_timeCreated = mesg.GetTimeCreated();
00054     }
00055
00056     FIT_DATE_TIME timeCreated() const { return m_timeCreated; }
00057
00058   private:
00059     FIT_DATE_TIME m_timeCreated = FIT_DATE_TIME_INVALID;
00060   };
00061 }
00062
00063 // Remove GPS points that imply speed > 150 km/h (equirectangular approximation)
00064 static void filterGpsOutliers(RideData& rideData)
00065 {
00066   constexpr double kMaxSpeedKmh = 150.0;
00067   constexpr double kMinSpeedKmh = 0.0;
00068
00069   bool prevValid = false;
00070   double prevLat = 0, prevLon = 0, prevSec = 0;
00071
00072   for (auto& r : rideData.records)
00073   {
00074     if (!r.hasGps)
00075     {
00076       prevValid = false;
00077       continue;
00078     }
00079
00080     // Additional sanity check on GPS coordinates
00081     if (r.latitude < -90.0 || r.latitude > 90.0 ||
00082         r.longitude < -180.0 || r.longitude > 180.0)
00083     {
00084       r.hasGps = false;
00085       prevValid = false;
00086       continue;
00087     }
00088
00089     if (prevValid)
00090     {
00091       const double dt = r.elapsedSeconds - prevSec;
00092       if (dt > 0.0 && dt < 3600.0) // Sanity: max 1 hour between samples
00093       {
00094         const double dLat = (r.latitude - prevLat) * std::numbers::pi_v<double> / 180.0;

```

```

00095         const double dLon = (r.longitude - prevLon) * std::numbers::pi_v<double> / 180.0;
00096         const double mLat = (r.latitude + prevLat) / 2.0 * std::numbers::pi_v<double> /
180.0;
00097         const double distKm =
00098             std::hypot(dLat, dLon * std::cos(mLat)) * 6371.0;
00099         const double speedKmh = distKm / (dt / 3600.0);
00100
00101         if (speedKmh > kMaxSpeedKmh || speedKmh < kMinSpeedKmh)
00102         {
00103             r.hasGps = false;
00104             prevValid = false;
00105             continue;
00106         }
00107     }
00108     else if (dt <= 0.0 || dt >= 3600.0)
00109     {
00110         // Invalid time jump; don't trust this GPS point
00111         r.hasGps = false;
00112         prevValid = false;
00113         continue;
00114     }
00115 }
00116
00117 prevValid = true;
00118 prevLat = r.latitude;
00119 prevLon = r.longitude;
00120 prevSec = r.elapsedSeconds;
00121 }
00122 }
00123
00124 RideData FitParser::load(const QString& fileName)
00125 {
00126     RideData rideData;
00127
00128     std::ifstream fitFile(fileName.toStdString(), std::ios::binary);
00129     if (!fitFile.is_open())
00130         throw std::runtime_error("Cannot open FIT file: " +
00131             fileName.toStdString());
00132
00133     fit::Decode decode;
00134     fit::MsgBroadcaster broadcaster;
00135
00136     FitRecordListener listener(rideData);
00137     FitSessionListener sessionListener;
00138     FitFileIdListener fileIdListener;
00139     broadcaster.AddListener((fit::RecordMsgListener&)listener);
00140     broadcaster.AddListener((fit::SessionMsgListener&)sessionListener);
00141     broadcaster.AddListener((fit::FileIdMsgListener&)fileIdListener);
00142
00143     if (!decode.CheckIntegrity(fitFile))
00144         throw std::runtime_error(
00145             "FIT integrity check failed.\n"
00146             "The file may be corrupted or was not fully written.");
00147
00148     fitFile.clear();
00149     fitFile.seekg(0);
00150
00151     decode.Read(fitFile, broadcaster, broadcaster);
00152
00153     if (rideData.records.empty())
00154         throw std::runtime_error(
00155             "No activity records found in this FIT file.\n"
00156             "The file may be a device configuration or course file, not an activity.");
00157
00158     // Count records with power or heart rate data (key fields for cycling analysis)
00159     int recordsWithData = 0;
00160     for (const auto& r : rideData.records)
00161     {
00162         if (r.hasPower || r.hasHeartRate)
00163             ++recordsWithData;
00164     }
00165
00166     // If less than 10% of records have key data, file is likely invalid
00167     if (recordsWithData == 0 || recordsWithData < static_cast<int>(rideData.records.size() * 0.1))
00168         throw std::runtime_error(
00169             "Activity file contains insufficient data.\n"
00170             "Expected power or heart rate data in most samples.");
00171
00172     filterGpsOutliers(rideData);
00173
00174     const FIT_DATE_TIME sessionStart = sessionListener.startTime();
00175     const FIT_DATE_TIME firstRecordTime = listener.firstTimestamp();
00176     const FIT_DATE_TIME lastRecordTime = listener.lastTimestamp();
00177     const FIT_DATE_TIME fileTime = fileIdListener.timeCreated();
00178
00179     FIT_DATE_TIME chosenStart = sessionStart;
00180     if (chosenStart == FIT_DATE_TIME_INVALID)

```

```

00181         chosenStart = firstRecordTime;
00182         if (chosenStart == FIT_DATE_TIME_INVALID)
00183             chosenStart = fileTime;
00184
00185         rideData.activityStartTimeUtc = fitTimestampToIsoUtc(chosenStart);
00186         rideData.activityEndTimeUtc = fitTimestampToIsoUtc(lastRecordTime != FIT_DATE_TIME_INVALID ?
lastRecordTime : chosenStart);
00187         rideData.fileTimestampUtc = fitTimestampToIsoUtc(fileTime);
00188
00189         if (rideData.activityStartTimeUtc.isEmpty())
00190             rideData.activityStartTimeUtc = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00191         if (rideData.activityEndTimeUtc.isEmpty())
00192             rideData.activityEndTimeUtc = rideData.activityStartTimeUtc;
00193
00194         return rideData;
00195     }

```

## 8.87 FitParser.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 #include "RideData.h"
00009
00013 class FitParser
00014 {
00015 public:
00021     RideData load(const QString& fileName);
00022
00023 private:
00024     static double semicirclesToDegrees(long value);
00025 };

```

## 8.88 FitRecordListener.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "FitRecordListener.h"
00005
00006 #include <algorithm>
00007
00008 FitRecordListener::FitRecordListener(
00009     RideData& rideData)
00010     : m_rideData(rideData)
00011 {
00012 }
00013
00014 void FitRecordListener::OnMsg(
00015     fit::RecordMsg& msg)
00016 {
00017     RideRecord record;
00018
00019     // GPS: validate coordinates are within valid range
00020     if (msg.IsPositionLatValid() &&
00021         msg.IsPositionLongValid())
00022     {
00023         const double lat = msg.GetPositionLat() * (180.0 / 2147483648.0);
00024         const double lon = msg.GetPositionLong() * (180.0 / 2147483648.0);
00025
00026         if (lat >= -90.0 && lat <= 90.0 && lon >= -180.0 && lon <= 180.0)
00027         {
00028             record.latitude = lat;
00029             record.longitude = lon;
00030             record.hasGps = true;
00031         }
00032     }
00033
00034     // Heart Rate: valid range 0-255 bpm
00035     if (msg.IsHeartRateValid())
00036     {
00037         const auto hr = msg.GetHeartRate();
00038         if (hr >= 0 && hr <= 255)
00039         {
00040             record.heartRate = hr;
00041             record.hasHeartRate = true;
00042         }
00043     }
00044 }

```

```

00043     }
00044
00045     // Cadence: valid range 0-255 rpm
00046     if (mesg.IsCadenceValid())
00047     {
00048         const auto cad = mesg.GetCadence();
00049         if (cad >= 0 && cad <= 255)
00050         {
00051             record.cadence = cad;
00052             record.hasCadence = true;
00053         }
00054     }
00055
00056     // Power: valid range 0-5000 watts
00057     if (mesg.IsPowerValid())
00058     {
00059         const auto pow = mesg.GetPower();
00060         if (pow >= 0 && pow <= 5000)
00061         {
00062             record.power = pow;
00063             record.hasPower = true;
00064         }
00065     }
00066
00067     // Altitude: valid range -500 to 10000 meters
00068     if (mesg.IsAltitudeValid())
00069     {
00070         const auto alt = mesg.GetAltitude();
00071         if (alt >= -500.0 && alt <= 10000.0)
00072         {
00073             record.altitude = alt;
00074             record.hasAltitude = true;
00075         }
00076     }
00077
00078     // Speed: valid range 0-150 km/h (converted from m/s)
00079     if (mesg.IsSpeedValid())
00080     {
00081         const auto spd = mesg.GetSpeed();
00082         const double spdKmh = spd * 3.6;
00083         if (spdKmh >= 0.0 && spdKmh <= 150.0)
00084         {
00085             record.speed = spdKmh;
00086             record.hasSpeed = true;
00087         }
00088     }
00089
00090     // Timestamp: validate monotonicity
00091     if (mesg.IsTimestampValid())
00092     {
00093         FIT_DATE_TIME ts = mesg.GetTimestamp();
00094
00095         if (m_firstTimestamp == FIT_DATE_TIME_INVALID)
00096             m_firstTimestamp = ts;
00097
00098         // Ensure monotonic timestamps; clamp to at least the last recorded elapsed time
00099         if (ts >= m_lastTimestamp)
00100         {
00101             m_lastTimestamp = ts;
00102             record.elapsedSeconds = static_cast<double>(ts - m_firstTimestamp);
00103         }
00104         else if (!m_rideData.records.empty())
00105         {
00106             // Backward timestamp: use previous elapsed + 1 second
00107             record.elapsedSeconds = m_rideData.records.back().elapsedSeconds + 1.0;
00108         }
00109         else
00110         {
00111             record.elapsedSeconds = 0.0;
00112         }
00113     }
00114     else if (!m_rideData.records.empty())
00115     {
00116         // No timestamp: continue sequence from last record
00117         record.elapsedSeconds =
00118             m_rideData.records.back().elapsedSeconds + 1.0;
00119     }
00120
00121     // Only add records that have at least one valid field
00122     if (record.hasGps || record.hasPower || record.hasHeartRate ||
00123         record.hasCadence || record.hasSpeed || record.hasAltitude)
00124     {
00125         m_rideData.records.push_back(record);
00126     }
00127 }

```

## 8.89 FitRecordListener.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "RideData.h"
00007
00008 #include <fit_record_mesg_listener.hpp>
00009
00015 class FitRecordListener :
00016     public fit::RecordMesgListener
00017 {
00018 public:
00023     explicit FitRecordListener(
00024         RideData& rideData);
00025
00030     void OnMesg(
00031         fit::RecordMesg& mesg) override;
00032
00037     FIT_DATE_TIME firstTimestamp() const { return m_firstTimestamp; }
00038
00043     FIT_DATE_TIME lastTimestamp() const { return m_lastTimestamp; }
00044
00045 private:
00046     RideData& m_rideData;
00047
00048     FIT_DATE_TIME m_firstTimestamp = FIT_DATE_TIME_INVALID;
00049     FIT_DATE_TIME m_lastTimestamp = FIT_DATE_TIME_INVALID;
00050 };

```

## 8.90 RideData.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <vector>
00008
00015 struct RideRecord
00016 {
00018     double elapsedSeconds = 0.0;
00019
00021     double latitude = 0.0;
00022
00024     double longitude = 0.0;
00025
00027     double power = 0.0;
00028
00030     double heartRate = 0.0;
00031
00033     double cadence = 0.0;
00034
00036     double speed = 0.0;
00037
00039     double altitude = 0.0;
00040
00042     bool hasGps = false;
00043
00045     bool hasPower = false;
00046
00048     bool hasHeartRate = false;
00049
00051     bool hasCadence = false;
00052
00054     bool hasSpeed = false;
00055
00057     bool hasAltitude = false;
00058 };
00059
00063 class RideData
00064 {
00065 public:
00067     std::vector<RideRecord> records;
00068
00070     QString activityStartTimeUtc;
00071
00073     QString activityEndTimeUtc;
00074
00076     QString fileTimestampUtc;

```

```
00077 };
```

## 8.91 fitlyzerC.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include <QApplication>
00005 #include <QIcon>
00006 #include <QPalette>
00007 #include <QStyleFactory>
00008
00009 #include "gui/MainWindow.h"
00010
00017 int main(int argc, char *argv[])
00018 {
00019     QApplication app(argc, argv);
00020     app.setOrganizationName("Fitlyzer");
00021     app.setApplicationName("FitlyzerC");
00022     app.setStyle(QStyleFactory::create("Fusion"));
00023
00024     QPalette lightPalette;
00025     lightPalette.setColor(QPalette::Window, QColor(245, 247, 250));
00026     lightPalette.setColor(QPalette::WindowText, QColor(31, 41, 55));
00027     lightPalette.setColor(QPalette::Base, QColor(255, 255, 255));
00028     lightPalette.setColor(QPalette::AlternateBase, QColor(241, 245, 249));
00029     lightPalette.setColor(QPalette::ToolTipBase, QColor(255, 255, 255));
00030     lightPalette.setColor(QPalette::ToolTipText, QColor(31, 41, 55));
00031     lightPalette.setColor(QPalette::Text, QColor(31, 41, 55));
00032     lightPalette.setColor(QPalette::Button, QColor(239, 243, 248));
00033     lightPalette.setColor(QPalette::ButtonText, QColor(31, 41, 55));
00034     lightPalette.setColor(QPalette::BrightText, Qt::white);
00035     lightPalette.setColor(QPalette::Highlight, QColor(37, 99, 235));
00036     lightPalette.setColor(QPalette::HighlightedText, Qt::white);
00037     app.setPalette(lightPalette);
00038
00039     app.setWindowIcon(QIcon(":/resources/icons/fitlyzer_logo.png"));
00040
00041     MainWindow window;
00042     window.show();
00043
00044     return app.exec();
00045 }
```

## 8.92 AboutDialog.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AboutDialog.h"
00005
00006 #include <QDesktopServices>
00007 #include <QDialogButtonBox>
00008 #include <QLabel>
00009 #include <QPushButton>
00010 #include <QUrl>
00011 #include <QVBoxLayout>
00012
00013 AboutDialog::AboutDialog(QWidget* parent)
00014 : QDialog(parent)
00015 {
00016     setWindowTitle("About FitlyzerC");
00017     setMinimumWidth(450);
00018
00019     auto* layout = new QVBoxLayout(this);
00020
00021     // Title
00022     auto* titleLabel = new QLabel("FitlyzerC", this);
00023     titleLabel->setStyleSheet("font-size: 18px; font-weight: bold; margin-bottom: 8px;");
00024     layout->addWidget(titleLabel);
00025
00026     // Description
00027     auto* descLabel = new QLabel(
00028         "FitlyzerC is a desktop application for analyzing cycling workouts and fitness data from "
00029         "FIT format files. It provides comprehensive training analytics, power curve analysis, "
00030         "and performance tracking for cyclists.",
00031         this);
00032     descLabel->setWordWrap(true);
00033     layout->addWidget(descLabel);
00034     layout->addSpacing(12);
00035 }
```

```

00036 // Contact & Links
00037 auto* contactLabel = new QLabel("Contact & Links:", this);
00038 contactLabel->setStyleSheet("font-weight: bold; margin-top: 8px;");
00039 layout->addWidget(contactLabel);
00040
00041 auto* emailLabel = new QLabel(
00042     "Email: <a href=\"mailto:email@larseberhart.com\">email@larseberhart.com</a>",
00043     this);
00044 emailLabel->setOpenExternalLinks(true);
00045 layout->addWidget(emailLabel);
00046
00047 auto* gitHubLabel = new QLabel(
00048     "GitHub: <a
href=\"https://github.com/larseberhart/fitlyzerC\">https://github.com/larseberhart/fitlyzerC</a>",
00049     this);
00050 gitHubLabel->setOpenExternalLinks(true);
00051 layout->addWidget(gitHubLabel);
00052
00053 layout->addSpacing(8);
00054 layout->addStretch();
00055
00056 // Close button
00057 auto* btnBox = new QDialogButtonBox(QDialogButtonBox::Close, this);
00058 connect(btnBox, &QDialogButtonBox::rejected, this, &QDialog::accept);
00059 layout->addWidget(btnBox);
00060 }

```

## 8.93 AboutDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00011 class AboutDialog : public QDialog
00012 {
00013     Q_OBJECT
00014
00015 public:
00020     explicit AboutDialog(QWidget* parent = nullptr);
00021 };

```

## 8.94 ActivityBrowser.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "ActivityBrowser.h"
00005 #include "EditNotesDialog.h"
00006 #include "core/settings/DateFormatter.h"
00007
00008 #include <QHeaderView>
00009 #include <QHBoxLayout>
00010 #include <QComboBox>
00011 #include <QLabel>
00012 #include <QLineEdit>
00013 #include <QMimeData>
00014 #include <QMenu>
00015 #include <QMessageBox>
00016 #include <QTableWidget>
00017 #include <QTableWidgetItem>
00018 #include <QUrl>
00019 #include <QVBoxLayout>
00020 #include <QFileInfo>
00021 #include <QDate>
00022
00023 namespace
00024 {
00025     constexpr int kNotesRole = Qt::UserRole + 1;
00026     constexpr int kWeatherRole = Qt::UserRole + 2;
00027     constexpr int kIsoDateRole = Qt::UserRole + 3;
00028
00029 class ActivityDateTableItem : public QTableWidgetItem
00030 {
00031 public:
00032     using QTableWidgetItem::QTableWidgetItem;
00033
00034     bool operator<(const QTableWidgetItem& other) const override
00035     {

```



```

00036     const QString lhsIso = data(kIsoDateRole).toString();
00037     const QString rhsIso = other.data(kIsoDateRole).toString();
00038
00039     const QDate lhsDate = QDate::fromString(lhsIso, Qt::ISODate);
00040     const QDate rhsDate = QDate::fromString(rhsIso, Qt::ISODate);
00041     if (lhsDate.isValid() && rhsDate.isValid())
00042         return lhsDate < rhsDate;
00043
00044     // Fall back to ISO lexical compare for partial values and then display text.
00045     const int isoCmp = QString::compare(lhsIso, rhsIso, Qt::CaseInsensitive);
00046     if (isoCmp != 0)
00047         return isoCmp < 0;
00048
00049     return QTableWidgetItem::operator<(other);
00050 }
00051 };
00052 }
00053
00054 static QString fmtDurAB(double seconds)
00055 {
00056     const int total = static_cast<int>(seconds);
00057     const int h = total / 3600;
00058     const int m = (total % 3600) / 60;
00059     const int s = total % 60;
00060     return h > 0
00061         ? QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00062         : QString("%1:%2").arg(m).arg(s, 2, 10, QChar('0'));
00063 }
00064
00065 ActivityBrowser::ActivityBrowser(DatabaseManager* dbManager, QWidget* parent)
00066     : QWidget(parent)
00067     , m_dbManager(dbManager)
00068 {
00069     setAcceptDrops(true);
00070
00071     auto* layout = new QVBoxLayout(this);
00072     layout->setContentsMargins(0, 0, 0, 0);
00073
00074     auto* filters = new QHBoxLayout;
00075     m_searchEdit = new QLineEdit(this);
00076     m_searchEdit->setPlaceholderText("Search activities, notes, or dates");
00077     m_searchEdit->setClearButtonEnabled(true);
00078     m_rangeFilter = new QComboBox(this);
00079     m_rangeFilter->addItem({"All", "Last 7 Days", "Last 30 Days", "This Year"});
00080     filters->addWidget(m_searchEdit, 1);
00081     filters->addWidget(m_rangeFilter);
00082     layout->addLayout(filters);
00083
00084     m_emptyStateLabel = new QLabel(this);
00085     m_emptyStateLabel->setWordWrap(true);
00086     m_emptyStateLabel->setAlignment(Qt::AlignCenter);
00087     m_emptyStateLabel->setText(
00088         "No activities yet.\n\nDrop .fit files here or use Import Activities to add rides.");
00089     m_emptyStateLabel->setStyleSheet(
00090         "QLabel { background: #f8fafc; color: #475569; border: 1px dashed #cbd5e1;"
00091         " padding: 20px; border-radius: 8px; font-size: 14px; }");
00092     layout->addWidget(m_emptyStateLabel);
00093
00094     m_table = new QTableWidgetItem(0, 8);
00095     m_table->setHorizontalHeaderLabels({
00096         "Date", "File", "Duration", "Distance (km)",
00097         "Avg W", "NP", "Avg HR", "Elevation (m)"
00098     });
00099     m_table->horizontalHeader()->setStretchLastSection(true);
00100     m_table->setSelectionBehavior(QAbstractItemView::SelectRows);
00101     m_table->setSelectionMode(QAbstractItemView::SingleSelection);
00102     m_table->setEditTriggers(QAbstractItemView::NoEditTriggers);
00103     m_table->setAlternatingRowColors(true);
00104     m_table->setSortingEnabled(true);
00105     m_table->setContextMenuPolicy(Qt::CustomContextMenu);
00106
00107     layout->addWidget(m_table);
00108
00109     connect(m_searchEdit, &QLineEdit::textChanged,
00110         this, [this] { applyFilters(); });
00111     connect(m_rangeFilter, qOverload<int>(&QComboBox::currentIndexChanged),
00112         this, [this](int) { applyFilters(); });
00113
00114     connect(m_table, &QTableWidgetItem::cellDoubleClicked,
00115         this, [this](int row, int)
00116     {
00117         auto* item = m_table->item(row, 0);
00118         if (!item) return;
00119         const int actId = item->data(Qt::UserRole).toInt();
00120         if (actId > 0)
00121             emit activitySelected(actId);
00122     });

```

```

00123
00124     connect(m_table, &QWidget::customContextMenuRequested,
00125             this, &ActivityBrowser::showContextMenu);
00126
00127     applyFilters();
00128 }
00129
00130 void ActivityBrowser::focusSearch()
00131 {
00132     if (!m_searchEdit)
00133         return;
00134
00135     m_searchEdit->setFocus();
00136     m_searchEdit->selectAll();
00137 }
00138
00139 void ActivityBrowser::dragEnterEvent(QDragEnterEvent* event)
00140 {
00141     if (!event->mimeTypeData()->hasUrls())
00142         return;
00143
00144     for (const QUrl& url : event->mimeTypeData()->urls())
00145     {
00146         if (!url.isLocalFile())
00147             continue;
00148         if (url.toLocalFile().toLower().endsWith(".fit"))
00149         {
00150             event->acceptProposedAction();
00151             return;
00152         }
00153     }
00154 }
00155
00156 void ActivityBrowser::dropEvent(QDropEvent* event)
00157 {
00158     if (!event->mimeTypeData()->hasUrls())
00159         return;
00160
00161     QStringList fitFiles;
00162     for (const QUrl& url : event->mimeTypeData()->urls())
00163     {
00164         if (!url.isLocalFile())
00165             continue;
00166         const QString path = url.toLocalFile();
00167         if (QFileInfo(path).suffix().compare("fit", Qt::CaseInsensitive) == 0)
00168             fitFiles « path;
00169     }
00170
00171     fitFiles.removeDuplicates();
00172     if (fitFiles.isEmpty())
00173         return;
00174
00175     emit fitFilesDropped(fitFiles);
00176     event->acceptProposedAction();
00177 }
00178
00179 void ActivityBrowser::refresh(int athleteId)
00180 {
00181     m_athleteId = athleteId;
00182     m_table->setSortingEnabled(false);
00183     m_table->setRowCount(0);
00184
00185     if (!m_dbManager || !m_dbManager->isOpen())
00186     {
00187         applyFilters();
00188         return;
00189     }
00190
00191     auto db = m_dbManager->database();
00192     ActivityRepository repo(db);
00193
00194     for (const Activity& a : repo.listActivities(athleteId))
00195     {
00196         const int row = m_table->rowCount();
00197         m_table->insertRow(row);
00198
00199         const QString isoDate = !a.startTime.isEmpty() ? a.startTime.left(10) : a.importedAt.left(10);
00200         const QDate parsedDate = QDate::fromString(isoDate, Qt::ISODate);
00201         const QString displayDate = parsedDate.isValid() ? DateFormatter::formatDate(parsedDate) :
isoDate;
00202
00203         auto* dateItem = new ActivityDateTableItem(displayDate);
00204         dateItem->setData(Qt::UserRole, a.id);
00205         dateItem->setData(kNotesRole, a.notes);
00206         dateItem->setData(kWeatherRole, a.weatherNotes);
00207         dateItem->setData(kIsoDateRole, isoDate);
00208         m_table->setItem(row, 0, dateItem);

```

```

00209
00210     m_table->setItem(row, 1, new QTableWidgetItem(a.fileName));
00211     m_table->setItem(row, 2, new QTableWidgetItem(fmtDurAB(a.durationSec)));
00212     m_table->setItem(row, 3, new QTableWidgetItem(
00213         a.distanceM > 0 ? QString::number(a.distanceM / 1000.0, 'f', 1) : ""));
00214     m_table->setItem(row, 4, new QTableWidgetItem(
00215         a.avgPower > 0 ? QString::number(a.avgPower, 'f', 0) : ""));
00216     m_table->setItem(row, 5, new QTableWidgetItem(
00217         a.normalizedPower > 0 ? QString::number(a.normalizedPower, 'f', 0) : ""));
00218     m_table->setItem(row, 6, new QTableWidgetItem(
00219         a.avgHr > 0 ? QString::number(a.avgHr, 'f', 0) : ""));
00220     m_table->setItem(row, 7, new QTableWidgetItem(
00221         a.elevationGain > 0 ? QString::number(a.elevationGain, 'f', 0) : ""));
00222     }
00223
00224     m_table->setSortingEnabled(true);
00225     m_table->resizeColumnsToContents();
00226     applyFilters();
00227 }
00228
00229 void ActivityBrowser::applyFilters()
00230 {
00231     if (!m_table)
00232         return;
00233
00234     const QString term = m_searchEdit ? m_searchEdit->text().trimmed().toLower() : QString();
00235     const QString range = m_rangeFilter ? m_rangeFilter->currentText() : QString("All");
00236     const QDate today = QDate::currentDate();
00237
00238     int visibleRows = 0;
00239     for (int row = 0; row < m_table->rowCount(); ++row)
00240     {
00241         bool matchesText = term.isEmpty();
00242         if (!matchesText)
00243         {
00244             QString rowText;
00245             for (int col = 0; col < m_table->columnCount(); ++col)
00246             {
00247                 auto* item = m_table->item(row, col);
00248                 if (item)
00249                     rowText += item->text().toLower() + " ";
00250             }
00251             auto* dateItem = m_table->item(row, 0);
00252             if (dateItem)
00253                 rowText += dateItem->data(kNotesRole).toString().toLower() + " ";
00254             matchesText = rowText.contains(term);
00255         }
00256
00257         bool matchesDate = true;
00258         auto* dateItem = m_table->item(row, 0);
00259         const QDate rowDate = dateItem
00260             ? QDate::fromString(dateItem->data(kIsoDateRole).toString(), Qt::ISODate)
00261             : QDate();
00262
00263         if (rowDate.isValid() && range != "All")
00264         {
00265             if (range == "Last 7 Days")
00266                 matchesDate = rowDate >= today.addDays(-7);
00267             else if (range == "Last 30 Days")
00268                 matchesDate = rowDate >= today.addDays(-30);
00269             else if (range == "This Year")
00270                 matchesDate = rowDate.year() == today.year();
00271         }
00272
00273         const bool visible = matchesText && matchesDate;
00274         m_table->setRowHidden(row, !visible);
00275         if (visible)
00276             ++visibleRows;
00277     }
00278
00279     if (m_emptyStateLabel)
00280     {
00281         const bool hasDatabase = m_dbManager && m_dbManager->isOpen();
00282         if (!hasDatabase)
00283         {
00284             m_emptyStateLabel->setText("Open or create a database to browse activities.");
00285         }
00286         else if (m_table->rowCount() == 0)
00287         {
00288             m_emptyStateLabel->setText(
00289                 "No activities yet.\n\nDrop .fit files here or use Import Activities to add rides.");
00290         }
00291         else if (visibleRows == 0)
00292         {
00293             m_emptyStateLabel->setText("No activities match the current search or date filter.");
00294         }
00295     }

```

```

00296         m_emptyStateLabel->setVisible(!hasDatabase || m_table->rowCount() == 0 || visibleRows == 0);
00297     }
00298
00299     m_table->setVisible(m_table->rowCount() > 0);
00300 }
00301
00302 void ActivityBrowser::showContextMenu(const QPoint& pos)
00303 {
00304     const int row = m_table->rowAt(pos.y());
00305     if (row < 0) return;
00306     auto* item = m_table->item(row, 0);
00307     if (!item) return;
00308     const int actId = item->data(Qt::UserRole).toInt();
00309     if (actId <= 0) return;
00310
00311     QMenu menu(this);
00312     auto* loadAct = menu.addAction("Load Activity");
00313     auto* notesAct = menu.addAction("Edit Notes...");
00314     menu.addSeparator();
00315     auto* deleteAct = menu.addAction("Delete Activity");
00316
00317     const QAction* chosen = menu.exec(m_table->viewport()->mapToGlobal(pos));
00318     if (chosen == loadAct)
00319         emit activitySelected(actId);
00320     else if (chosen == notesAct)
00321         editNotes(actId);
00322     else if (chosen == deleteAct)
00323         deleteActivity(actId);
00324 }
00325
00326 void ActivityBrowser::editNotes(int activityId)
00327 {
00328     if (!m_dbManager || !m_dbManager->isOpen()) return;
00329
00330     // Retrieve current notes from the row's stored data
00331     QString notes, weather;
00332     for (int row = 0; row < m_table->rowCount(); ++row)
00333     {
00334         auto* item = m_table->item(row, 0);
00335         if (item && item->data(Qt::UserRole).toInt() == activityId)
00336         {
00337             notes = item->data(kNotesRole).toString();
00338             weather = item->data(kWeatherRole).toString();
00339             break;
00340         }
00341     }
00342
00343     EditNotesDialog dlg(notes, weather, this);
00344     if (dlg.exec() != QDialog::Accepted) return;
00345
00346     auto db = m_dbManager->database();
00347     ActivityRepository repo(db);
00348     if (repo.updateNotes(activityId, dlg.notes(), dlg.weatherNotes()))
00349         refresh(m_athleteId); // reload to show updated data
00350     else
00351         QMessageBox::critical(this, "Error", "Failed to save notes.");
00352 }
00353
00354 void ActivityBrowser::deleteActivity(int activityId)
00355 {
00356     const auto ans = QMessageBox::question(
00357         this, "Delete Activity",
00358         "Delete this activity and all its samples?\nThis cannot be undone.",
00359         QMessageBox::Yes | QMessageBox::No);
00360     if (ans != QMessageBox::Yes) return;
00361
00362     if (!m_dbManager || !m_dbManager->isOpen()) return;
00363
00364     auto db = m_dbManager->database();
00365     ActivityRepository repo(db);
00366     if (repo.deleteActivity(activityId))
00367     {
00368         refresh(m_athleteId);
00369         emit activityDeleted(activityId);
00370     }
00371     else
00372         QMessageBox::critical(this, "Error", "Failed to delete activity.");
00373 }

```

## 8.95 ActivityBrowser.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #pragma once
00005
00006 #include <QDragEnterEvent>
00007 #include <QDropEvent>
00008 #include <QWidget>
00009
00010 #include "database/DatabaseManager.h"
00011 #include "database/ActivityRepository.h"
00012
00013 class QTableWidget;
00014 class QComboBox;
00015 class QLineEdit;
00016 class QLabel;
00017
00026 class ActivityBrowser : public QWidget
00027 {
00028     Q_OBJECT
00029 public:
00030     explicit ActivityBrowser(DatabaseManager* dbManager, QWidget* parent = nullptr);
00031
00032     void refresh(int athleteId = -1);
00033
00034     void focusSearch();
00035
00036 signals:
00037     // Emitted when an activity is selected in the table.
00038     void activitySelected(int activityId);
00039
00040     // Emitted when an activity is deleted.
00041     void activityDeleted(int activityId);
00042
00043     // Emitted when one or more local .fit files are dropped onto the widget.
00044     void fitFilesDropped(const QStringList& filePaths);
00045
00046 protected:
00047     void dragEnterEvent(QDragEnterEvent* event) override;
00048
00049     void dropEvent(QDropEvent* event) override;
00050
00051 private:
00052     void applyFilters();
00053     void showContextMenu(const QPoint& pos);
00054     void editNotes(int activityId);
00055     void deleteActivity(int activityId);
00056
00057     DatabaseManager* m_dbManager = nullptr;
00058     QLabel* m_emptyStateLabel = nullptr;
00059     QLineEdit* m_searchEdit = nullptr;
00060     QComboBox* m_rangeFilter = nullptr;
00061     QTableWidget* m_table = nullptr;
00062     int m_athleteId = -1;
00063 };

```

## 8.96 AthleteDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteDialog.h"
00005 #include "core/settings/DateFormatter.h"
00006
00007 #include <QDate>
00008 #include <QDialogButtonBox>
00009 #include <QDoubleSpinBox>
00010 #include <QFormLayout>
00011 #include <QGroupBox>
00012 #include <QHBoxLayout>
00013 #include <QHeaderView>
00014 #include <QLabel>
00015 #include <QLineEdit>
00016 #include <QMessageBox>
00017 #include <QPushButton>
00018 #include <QSpinBox>
00019 #include <QTableWidget>
00020 #include <QTableWidgetItem>
00021 #include <QVBoxLayout>
00022
00023 namespace
00024 {
00025     constexpr int kEntryIdRole = Qt::UserRole;
00026
00027     QString displayDateOrRaw(const QString& raw)
00028     {
00029         const QDate parsed = QDate::fromString(raw, Qt::ISODate);

```

```

00030     return parsed.isValid() ? DateFormatter::formatDate(parsed) : raw;
00031 }
00032
00033 QString toIsoOrRaw(const QString& text)
00034 {
00035     const QDate parsed = DateFormatter::parseDate(text);
00036     return parsed.isValid() ? DateFormatter::toIsoDate(parsed) : text.trimmed();
00037 }
00038 }
00039
00040 AthleteDialog::AthleteDialog(AthleteRepository& repo, int athleteId, QWidget* parent)
00041     : QDialog(parent)
00042     , m_repo(repo)
00043     , m_athleteId(athleteId)
00044 {
00045     setWindowTitle(athleteId < 0 ? "New Athlete" : "Edit Athlete");
00046     setMinimumWidth(520);
00047
00048     auto* mainLayout = new QVBoxLayout(this);
00049
00050     // Basic info
00051     auto* form = new QFormLayout;
00052     m_firstName = new QLineEdit;
00053     m_lastName = new QLineEdit;
00054     m_dob = new QLineEdit;
00055     m_dob->setPlaceholderText(QString("Date format: %1").arg(DateFormatter::qtDatePattern()));
00056     m_email = new QLineEdit;
00057     m_height = new QDoubleSpinBox;
00058     m_fftp = new QSpinBox;
00059     m_height->setRange(0.0, 250.0);
00060     m_height->setSuffix(" cm");
00061     m_height->setDecimals(1);
00062     m_height->setSpecialValueText("(not set)");
00063     m_fftp->setRange(100, 600);
00064     m_fftp->setValue(250);
00065     m_fftp->setSuffix(" W");
00066
00067     form->addRow("First name:", m_firstName);
00068     form->addRow("Last name:", m_lastName);
00069     form->addRow("Date of birth:", m_dob);
00070     form->addRow("Email:", m_email);
00071     form->addRow("Height:", m_height);
00072     form->addRow("FTP:", m_fftp);
00073     mainLayout->addLayout(form);
00074
00075     // FTP history
00076     auto* ftpGroup = new QGroupBox("FTP History");
00077     auto* ftpLayout = new QVBoxLayout(ftpGroup);
00078     m_ftpTable = new QTableWidget(0, 3);
00079     m_ftpTable->setHorizontalHeaderLabels({"Date", "FTP (W)", "Notes"});
00080     m_ftpTable->horizontalHeader()->setStretchLastSection(true);
00081     m_ftpTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00082     m_ftpTable->setMinimumHeight(120);
00083
00084     auto* ftpBtnBar = new QHBoxLayout;
00085     auto* addFtp = new QPushButton("Add");
00086     auto* delFtp = new QPushButton("Delete");
00087     ftpBtnBar->addWidget(addFtp);
00088     ftpBtnBar->addWidget(delFtp);
00089     ftpBtnBar->addStretch();
00090
00091     ftpLayout->addWidget(m_ftpTable);
00092     ftpLayout->addLayout(ftpBtnBar);
00093     mainLayout->addWidget(ftpGroup);
00094
00095     connect(addFtp, &QPushButton::clicked, this, &AthleteDialog::addFtpRow);
00096     connect(delFtp, &QPushButton::clicked, this, &AthleteDialog::deleteFtpRow);
00097
00098     // Weight history
00099     auto* wGroup = new QGroupBox("Weight History");
00100     auto* wLayout = new QVBoxLayout(wGroup);
00101     m_weightTable = new QTableWidget(0, 2);
00102     m_weightTable->setHorizontalHeaderLabels({"Date", "Weight (kg)"});
00103     m_weightTable->horizontalHeader()->setStretchLastSection(true);
00104     m_weightTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00105     m_weightTable->setMinimumHeight(100);
00106
00107     auto* wBtnBar = new QHBoxLayout;
00108     auto* addW = new QPushButton("Add");
00109     auto* delW = new QPushButton("Delete");
00110     wBtnBar->addWidget(addW);
00111     wBtnBar->addWidget(delW);
00112     wBtnBar->addStretch();
00113
00114     wLayout->addWidget(m_weightTable);
00115     wLayout->addLayout(wBtnBar);
00116     mainLayout->addWidget(wGroup);

```

```

00117
00118     connect(addW, &QPushButton::clicked, this, &AthleteDialog::addWeightRow);
00119     connect(delW, &QPushButton::clicked, this, &AthleteDialog::deleteWeightRow);
00120
00121     // Buttons
00122     auto* bbox = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
00123     mainLayout->addWidget(bbox);
00124
00125     connect(bbox, &QDialogButtonBox::accepted, this, &AthleteDialog::onAccepted);
00126     connect(bbox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00127
00128     if (m_athleteId > 0)
00129         loadAthlete();
00130 }
00131
00132 void AthleteDialog::loadAthlete()
00133 {
00134     Athlete a = m_repo.getAthlete(m_athleteId);
00135     m_firstName->setText(a.firstName);
00136     m_lastName->setText(a.lastName);
00137     m_dob->setText(displayDateOrRaw(a.dateOfBirth));
00138     m_email->setText(a.email);
00139     m_height->setValue(a.heightCm);
00140     m_ftp->setValue(a.ftpWatts > 0 ? a.ftpWatts : 250);
00141
00142     populateFtpTable(m_repo.getFtpHistory(m_athleteId));
00143     populateWeightTable(m_repo.getWeightHistory(m_athleteId));
00144 }
00145
00146 void AthleteDialog::populateFtpTable(const QList<FtpEntry>& entries)
00147 {
00148     m_ftpTable->setRowCount(0);
00149     for (const auto& e : entries)
00150     {
00151         const int row = m_ftpTable->rowCount();
00152         m_ftpTable->insertRow(row);
00153         m_ftpTable->setItem(row, 0, new QTableWidgetItem(displayDateOrRaw(e.effectiveFrom)));
00154         m_ftpTable->setItem(row, 1, new QTableWidgetItem(QString::number(e.ftpWatts)));
00155         m_ftpTable->setItem(row, 2, new QTableWidgetItem(e.notes));
00156         // Store the entry id in the row's first item
00157         m_ftpTable->item(row, 0)->setData(kEntryIdRole, e.id);
00158     }
00159 }
00160
00161 void AthleteDialog::populateWeightTable(const QList<WeightEntry>& entries)
00162 {
00163     m_weightTable->setRowCount(0);
00164     for (const auto& e : entries)
00165     {
00166         const int row = m_weightTable->rowCount();
00167         m_weightTable->insertRow(row);
00168         m_weightTable->setItem(row, 0, new QTableWidgetItem(displayDateOrRaw(e.effectiveFrom)));
00169         m_weightTable->setItem(row, 1, new QTableWidgetItem(
00170             QString::number(e.weightKg, 'f', 1)));
00171         m_weightTable->item(row, 0)->setData(kEntryIdRole, e.id);
00172     }
00173 }
00174
00175 void AthleteDialog::addFtpRow()
00176 {
00177     const int row = m_ftpTable->rowCount();
00178     m_ftpTable->insertRow(row);
00179     m_ftpTable->setItem(row, 0, new
00180     QTableWidgetItem(DateFormatter::formatDate(QDate::currentDate())));
00181     m_ftpTable->setItem(row, 1, new QTableWidgetItem("250"));
00182     m_ftpTable->item(row, 0)->setData(kEntryIdRole, -1); // new, no db id yet
00183     m_ftpTable->editItem(m_ftpTable->item(row, 0));
00184 }
00185
00186 void AthleteDialog::deleteFtpRow()
00187 {
00188     const int row = m_ftpTable->currentRow();
00189     if (row < 0) return;
00190     const int entryId = m_ftpTable->item(row, 0)->data(kEntryIdRole).toInt();
00191     if (entryId > 0 && m_athleteId > 0)
00192         m_repo.deleteFtpEntry(entryId);
00193     m_ftpTable->removeRow(row);
00194 }
00195
00196 void AthleteDialog::addWeightRow()
00197 {
00198     const int row = m_weightTable->rowCount();
00199     m_weightTable->insertRow(row);
00200     m_weightTable->setItem(row, 0, new
00201     QTableWidgetItem(DateFormatter::formatDate(QDate::currentDate())));
00202     m_weightTable->setItem(row, 1, new QTableWidgetItem("70.0"));

```

```

00202     m_weightTable->item(row, 0)->setData(kEntryIdRole, -1);
00203     m_weightTable->editItem(m_weightTable->item(row, 0));
00204 }
00205
00206 void AthleteDialog::deleteWeightRow()
00207 {
00208     const int row = m_weightTable->currentRow();
00209     if (row < 0) return;
00210     const int entryId = m_weightTable->item(row, 0)->data(kEntryIdRole).toInt();
00211     if (entryId > 0 && m_athleteId > 0)
00212         m_repo.deleteWeightEntry(entryId);
00213     m_weightTable->removeRow(row);
00214 }
00215
00216 void AthleteDialog::onAccepted()
00217 {
00218     if (m_firstName->text().trimmed().isEmpty() ||
00219         m_lastName->text().trimmed().isEmpty())
00220     {
00221         QMessageBox::warning(this, "Missing data",
00222             "First name and last name are required.");
00223         return;
00224     }
00225
00226     Athlete a;
00227     a.id = m_athleteId;
00228     a.firstName = m_firstName->text().trimmed();
00229     a.lastName = m_lastName->text().trimmed();
00230     a.dateOfBirth = toIsoOrRaw(m_dob->text());
00231     a.email = m_email->text().trimmed();
00232     a.heightCm = m_height->value();
00233     a.ftpWatts = m_ftp->value();
00234
00235     if (m_athleteId < 0)
00236         m_savedId = m_repo.insertAthlete(a);
00237     else
00238     {
00239         m_repo.updateAthlete(a);
00240         m_savedId = m_athleteId;
00241     }
00242
00243     if (m_savedId < 0)
00244     {
00245         QMessageBox::critical(this, "Error", "Failed to save athlete.");
00246         return;
00247     }
00248
00249     // Save new FTP entries (those with id == -1)
00250     for (int row = 0; row < m_ftpTable->rowCount(); ++row)
00251     {
00252         const int entryId = m_ftpTable->item(row, 0)->data(kEntryIdRole).toInt();
00253         if (entryId < 0)
00254         {
00255             FtpEntry e;
00256             e.athleteId = m_savedId;
00257             e.effectiveFrom = toIsoOrRaw(m_ftpTable->item(row, 0)->text());
00258             e.ftpWatts = m_ftpTable->item(row, 1)->text().toInt();
00259             e.notes = m_ftpTable->item(row, 2)->text();
00260             m_repo.addFtpEntry(e);
00261         }
00262     }
00263
00264     // Save new weight entries
00265     for (int row = 0; row < m_weightTable->rowCount(); ++row)
00266     {
00267         const int entryId = m_weightTable->item(row, 0)->data(kEntryIdRole).toInt();
00268         if (entryId < 0)
00269         {
00270             WeightEntry e;
00271             e.athleteId = m_savedId;
00272             e.effectiveFrom = toIsoOrRaw(m_weightTable->item(row, 0)->text());
00273             e.weightKg = m_weightTable->item(row, 1)->text().toDouble();
00274             m_repo.addWeightEntry(e);
00275         }
00276     }
00277
00278     accept();
00279 }

```

## 8.97 AthleteDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```



```

00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QList>
00008
00009 #include "database/AthleteRepository.h"
00010
00011 class QLineEdit;
00012 class QDoubleSpinBox;
00013 class QSpinBox;
00014 class QTableWidget;
00015
00021 class AthleteDialog : public QDialog
00022 {
00023     Q_OBJECT
00024 public:
00031     explicit AthleteDialog(AthleteRepository& repo,
00032                             int athleteId = -1,
00033                             QWidget* parent = nullptr);
00034
00039     int savedAthleteId() const { return m_savedId; }
00040
00041 private slots:
00042     void onAccepted();
00043     void addFtpRow();
00044     void deleteFtpRow();
00045     void addWeightRow();
00046     void deleteWeightRow();
00047
00048 private:
00049     void loadAthlete();
00050     void populateFtpTable(const QList<FtpEntry>& entries);
00051     void populateWeightTable(const QList<WeightEntry>& entries);
00052
00053     AthleteRepository& m_repo;
00054     int m_athleteId;
00055     int m_savedId = -1;
00056
00057     QLineEdit* m_firstName = nullptr;
00058     QLineEdit* m_lastName = nullptr;
00059     QLineEdit* m_dob = nullptr;
00060     QLineEdit* m_email = nullptr;
00061     QDoubleSpinBox* m_height = nullptr;
00062     QSpinBox* m_ftp = nullptr;
00063     QTableWidget* m_ftpTable = nullptr;
00064     QTableWidget* m_weightTable = nullptr;
00065 };

```

## 8.98 AthleteHeaderWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteHeaderWidget.h"
00005
00006 #include <QHBoxLayout>
00007 #include <QLabel>
00008
00009 AthleteHeaderWidget::AthleteHeaderWidget(QWidget* parent)
00010     : QWidget(parent)
00011 {
00012     setFixedHeight(40);
00013     setStyleSheet(
00014         "AthleteHeaderWidget {"
00015         "     background: #1e3a5f;"
00016         "     border-bottom: 1px solid #14274a;"
00017         "}");
00018
00019     auto* row = new QHBoxLayout(this);
00020     row->setContentsMargins(12, 0, 12, 0);
00021     row->setSpacing(0);
00022
00023     // Left: athlete name
00024     m_nameLabel = new QLabel("No athlete selected", this);
00025     m_nameLabel->setStyleSheet(
00026         "font-size: 14px; font-weight: 700; color: #0c4a6e;"
00027         " min-width: 160px;");
00028
00029     // Middle: FTP / activities / last ride
00030     m_statsLabel = new QLabel("FTP 0 W Activities 0 Last -", this);
00031     m_statsLabel->setAlignment(Qt::AlignCenter);
00032     m_statsLabel->setStyleSheet(
00033         "font-size: 12px; color: #374151;");
00034

```

```

00035 // Right: per-ride metrics
00036 m_metricsLabel = new QLabel("", this);
00037 m_metricsLabel->setAlignment(Qt::AlignRight | Qt::AlignVCenter);
00038 m_metricsLabel->setStyleSheet(
00039     "font-size: 12px; font-weight: 600; color: #1f2937;"
00040     " min-width: 160px;");
00041
00042 row->addWidget(m_nameLabel, 0);
00043 row->addWidget(m_statsLabel, 1);
00044 row->addWidget(m_metricsLabel, 0);
00045 }
00046
00047 void AthleteHeaderWidget::setSummary(const QString& athleteName,
00048                                     int ftpWatts,
00049                                     int activityCount,
00050                                     const QString& lastActivityDate)
00051 {
00052     m_nameLabel->setText(athleteName.isEmpty() ? QString("No athlete selected") : athleteName);
00053     m_statsLabel->setText(
00054         QString("FTP %1 W \u00b7 %2 activities \u00b7 Last %3")
00055             .arg(ftpWatts)
00056             .arg(activityCount)
00057             .arg(lastActivityDate));
00058 }
00059
00060 void AthleteHeaderWidget::setRideMetrics(const QString& metricsText)
00061 {
00062     m_metricsLabel->setText(metricsText);
00063 }

```

## 8.99 AthleteHeaderWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QWidget>
00007
00008 class QLabel;
00009
00010 class AthleteHeaderWidget : public QWidget
00011 {
00012     Q_OBJECT
00013
00014 public:
00015     explicit AthleteHeaderWidget(QWidget* parent = nullptr);
00016
00017     void setSummary(const QString& athleteName,
00018                   int ftpWatts,
00019                   int activityCount,
00020                   const QString& lastActivityDate);
00021
00022     void setRideMetrics(const QString& metricsText);
00023
00024 private:
00025     QLabel* m_nameLabel = nullptr; // left: athlete name
00026     QLabel* m_statsLabel = nullptr; // middle: FTP / activities / last ride
00027     QLabel* m_metricsLabel = nullptr; // right: NP / IF / TSS ...
00028 };

```

## 8.100 AthleteListDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "AthleteListDialog.h"
00005 #include "AthleteDialog.h"
00006 #include "core/settings/DateFormatter.h"
00007
00008 #include <QDate>
00009
00010 #include <QDialogButtonBox>
00011 #include <QHBoxLayout>
00012 #include <QHeaderView>
00013 #include <QMessageBox>
00014 #include <QPushButton>
00015 #include <QTableWidget>
00016 #include <QTableWidgetItem>
00017 #include <QVBoxLayout>
00018

```

```

00019 AthleteListDialog::AthleteListDialog(AthleteRepository& repo, QWidget* parent)
00020     : QDialog(parent)
00021     , m_repo(repo)
00022 {
00023     setWindowTitle("Athletes");
00024     setMinimumSize(460, 300);
00025
00026     auto* mainLayout = new QVBoxLayout(this);
00027
00028     m_table = new QTableWidget(0, 3);
00029     m_table->setHorizontalHeaderLabels({"Name", "Date of Birth", "Email"});
00030     m_table->horizontalHeader()->setStretchLastSection(true);
00031     m_table->setSelectionBehavior(QAbstractItemView::SelectRows);
00032     m_table->setSelectionMode(QAbstractItemView::SingleSelection);
00033     m_table->setEditTriggers(QAbstractItemView::NoEditTriggers);
00034     m_table->setAlternatingRowColors(true);
00035     mainLayout->addWidget(m_table);
00036
00037     // Buttons
00038     auto* btnBar = new QHBoxLayout;
00039     auto* newBtn = new QPushButton("New");
00040     m_editBtn = new QPushButton("Edit");
00041     m_deleteBtn = new QPushButton("Delete");
00042     m_selectBtn = new QPushButton("Select");
00043     m_selectBtn->setDefault(true);
00044
00045     btnBar->addWidget(newBtn);
00046     btnBar->addWidget(m_editBtn);
00047     btnBar->addWidget(m_deleteBtn);
00048     btnBar->addStretch();
00049     btnBar->addWidget(m_selectBtn);
00050     mainLayout->addLayout(btnBar);
00051
00052     auto* closeBtn = new QDialogButtonBox(QDialogButtonBox::Close);
00053     mainLayout->addWidget(closeBtn);
00054
00055     connect(newBtn, &QPushButton::clicked, this, &AthleteListDialog::newAthlete);
00056     connect(m_editBtn, &QPushButton::clicked, this, &AthleteListDialog::editAthlete);
00057     connect(m_deleteBtn, &QPushButton::clicked, this, &AthleteListDialog::deleteAthlete);
00058     connect(m_selectBtn, &QPushButton::clicked, this, &AthleteListDialog::selectAthlete);
00059     connect(closeBtn, &QDialogButtonBox::rejected, this, &QDialog::reject);
00060
00061     connect(m_table, &QTableWidget::cellDoubleClicked,
00062             this, [this](int, int){ editAthlete(); });
00063     connect(m_table, &QTableWidget::itemSelectionChanged, this, [this]{
00064         const bool hasSel = m_table->currentRow() >= 0;
00065         m_editBtn->setEnabled(hasSel);
00066         m_deleteBtn->setEnabled(hasSel);
00067         m_selectBtn->setEnabled(hasSel);
00068     });
00069
00070     m_editBtn->setEnabled(false);
00071     m_deleteBtn->setEnabled(false);
00072     m_selectBtn->setEnabled(false);
00073
00074     refresh();
00075 }
00076
00077 void AthleteListDialog::refresh()
00078 {
00079     m_table->setRowCount(0);
00080     for (const Athlete& a : m_repo.listAthletes())
00081     {
00082         const int row = m_table->rowCount();
00083         m_table->insertRow(row);
00084         auto* nameItem = new QTableWidgetItem(a.fullName());
00085         nameItem->setData(Qt::UserRole, a.id);
00086         m_table->setItem(row, 0, nameItem);
00087         const QDate dob = QDate::fromString(a.dateOfBirth, Qt::ISODate);
00088         const QString dobText = dob.isValid() ? DateFormatter::formatDate(dob) : a.dateOfBirth;
00089         m_table->setItem(row, 1, new QTableWidgetItem(dobText));
00090         m_table->setItem(row, 2, new QTableWidgetItem(a.email));
00091     }
00092 }
00093
00094 void AthleteListDialog::newAthlete()
00095 {
00096     AthleteDialog dlg(m_repo, -1, this);
00097     if (dlg.exec() == QDialog::Accepted)
00098         refresh();
00099 }
00100
00101 void AthleteListDialog::editAthlete()
00102 {
00103     const int row = m_table->currentRow();
00104     if (row < 0) return;
00105     const int athleteId = m_table->item(row, 0)->data(Qt::UserRole).toInt();

```

```

00106     AthleteDialog dlg(m_repo, athleteId, this);
00107     if (dlg.exec() == QDialog::Accepted)
00108         refresh();
00109 }
00110
00111 void AthleteListDialog::deleteAthlete()
00112 {
00113     const int row = m_table->currentRow();
00114     if (row < 0) return;
00115     const int athleteId = m_table->item(row, 0)->data(Qt::UserRole).toInt();
00116     const QString name = m_table->item(row, 0)->text();
00117
00118     const auto ans = QMessageBox::question(
00119         this, "Delete Athlete",
00120         QString("Delete \"%1\" and all their activities?").arg(name),
00121         QMessageBox::Yes | QMessageBox::No);
00122     if (ans != QMessageBox::Yes) return;
00123
00124     m_repo.deleteAthlete(athleteId);
00125     refresh();
00126 }
00127
00128 void AthleteListDialog::selectAthlete()
00129 {
00130     const int row = m_table->currentRow();
00131     if (row < 0) return;
00132     m_selectedId = m_table->item(row, 0)->data(Qt::UserRole).toInt();
00133     accept();
00134 }

```

## 8.101 AthleteListDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "database/AthleteRepository.h"
00009
00010 class QTableWidgetItem;
00011 class QPushButton;
00012
00019 class AthleteListDialog : public QDialog
00020 {
00021     Q_OBJECT
00022 public:
00023     explicit AthleteListDialog(AthleteRepository& repo,
00024                               QWidget* parent = nullptr);
00025
00026     int selectedAthleteId() const { return m_selectedId; }
00027
00028 private slots:
00029     void refresh();
00030     void newAthlete();
00031     void editAthlete();
00032     void deleteAthlete();
00033     void selectAthlete();
00034
00035 private:
00036     AthleteRepository& m_repo;
00037     int m_selectedId = -1;
00038
00039     QTableWidgetItem* m_table = nullptr;
00040     QPushButton* m_editBtn = nullptr;
00041     QPushButton* m_deleteBtn = nullptr;
00042     QPushButton* m_selectBtn = nullptr;
00043 };

```

## 8.102 CalendarWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "CalendarWidget.h"
00005
00006 #include "PlannedWorkoutDialog.h"
00007 #include "core/settings/DateFormatter.h"
00008 #include "database/DatabaseManager.h"
00009

```

```

00010 #include <QCalendarWidget>
00011 #include <QComboBox>
00012 #include <QDateEdit>
00013 #include <QHBoxLayout>
00014 #include <QLabel>
00015 #include <QListWidget>
00016 #include <QMessageBox>
00017 #include <QPushButton>
00018 #include <QSqlQuery>
00019 #include <QTextCharFormat>
00020 #include <QVBoxLayout>
00021
00022 CalendarWidget::CalendarWidget(QWidget* parent)
00023     : QWidget(parent)
00024 {
00025     auto* layout = new QVBoxLayout(this);
00026
00027     auto* top = new QHBoxLayout;
00028     top->addWidget(new QLabel("Color by:", this));
00029
00030     m_colorByCombo = new QComboBox(this);
00031     m_colorByCombo->addItem("TSS");
00032     m_colorByCombo->addItem("Duration");
00033     m_colorByCombo->addItem("Workout Type");
00034     top->addWidget(m_colorByCombo);
00035
00036     m_addButton = new QPushButton("Add", this);
00037     m_editButton = new QPushButton("Edit", this);
00038     m_deleteButton = new QPushButton("Delete", this);
00039     top->addWidget(m_addButton);
00040     top->addWidget(m_editButton);
00041     top->addWidget(m_deleteButton);
00042     top->addStretch(1);
00043
00044     m_calendar = new QCalendarWidget(this);
00045     m_calendar->setGridVisible(true);
00046
00047     m_dayList = new QListWidget(this);
00048     m_dayList->addItem("Planned workouts and completed activities will appear here.");
00049
00050     layout->addLayout(top);
00051     layout->addWidget(m_calendar);
00052     layout->addWidget(m_dayList);
00053
00054     connect(m_calendar, &QCalendarWidget::selectionChanged,
00055             this, &CalendarWidget::onDateChanged);
00056     connect(m_calendar, &QCalendarWidget::currentPageChanged,
00057             this, [this](int, int) { updateMonthFormatting(); });
00058     connect(m_colorByCombo, qOverload<int>(&QComboBox::currentIndexChanged),
00059             this, &CalendarWidget::recolorMonth);
00060     connect(m_addButton, &QPushButton::clicked,
00061             this, &CalendarWidget::addPlannedWorkout);
00062     connect(m_editButton, &QPushButton::clicked,
00063             this, &CalendarWidget::editSelectedWorkout);
00064     connect(m_deleteButton, &QPushButton::clicked,
00065             this, &CalendarWidget::deleteSelectedWorkout);
00066 }
00067
00068 void CalendarWidget::setDatabaseManager(DatabaseManager* dbManager)
00069 {
00070     m_dbManager = dbManager;
00071     refresh();
00072 }
00073
00074 void CalendarWidget::setAthleteId(int athleteId)
00075 {
00076     m_athleteId = athleteId;
00077     refresh();
00078 }
00079
00080 void CalendarWidget::refresh()
00081 {
00082     if (!m_calendar)
00083         return;
00084     reloadDayList(m_calendar->selectedDate());
00085     updateMonthFormatting();
00086 }
00087
00088 void CalendarWidget::onDateChanged()
00089 {
00090     if (!m_calendar)
00091         return;
00092     reloadDayList(m_calendar->selectedDate());
00093 }
00094
00095 void CalendarWidget::addPlannedWorkout()
00096 {

```

```

00097     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00098         return;
00099
00100     PlannedWorkoutDraft draft;
00101     draft.date = m_calendar ? m_calendar->selectedDate() : QDate::currentDate();
00102     PlannedWorkoutDialog dlg(draft, this);
00103     if (dlg.exec() != QDialog::Accepted)
00104         return;
00105
00106     const PlannedWorkoutDraft saved = dlg.draft();
00107     if (!saved.date.isValid() || saved.title.isEmpty())
00108         return;
00109
00110     QSqlQuery q(m_dbManager->database());
00111     q.prepare(
00112         "INSERT INTO planned_workouts(athlete_id, workout_date, title, type, duration_sec, target_tss,
notes) "
00113         "VALUES(:athlete_id, :workout_date, :title, :type, :duration_sec, :target_tss, :notes)");
00114     q.bindValue(":athlete_id", m_athleteId);
00115     q.bindValue(":workout_date", DateFormatter::toIsoDate(saved.date));
00116     q.bindValue(":title", saved.title);
00117     q.bindValue(":type", saved.type);
00118     q.bindValue(":duration_sec", saved.durationSeconds);
00119     q.bindValue(":target_tss", saved.targetTss);
00120     q.bindValue(":notes", saved.notes);
00121     if (!q.exec())
00122     {
00123         QMessageBox::critical(this, "Calendar", "Failed to add planned workout.");
00124         return;
00125     }
00126
00127     refresh();
00128 }
00129
00130 void CalendarWidget::editSelectedWorkout()
00131 {
00132     if (!m_dbManager || !m_dbManager->isOpen() || !m_dayList)
00133         return;
00134
00135     QListWidgetItem* item = m_dayList->currentItem();
00136     if (!item)
00137         return;
00138
00139     const int id = item->data(Qt::UserRole).toInt();
00140     if (id <= 0)
00141         return;
00142
00143     QSqlQuery q(m_dbManager->database());
00144     q.prepare(
00145         "SELECT workout_date, title, type, duration_sec, target_tss, notes "
00146         "FROM planned_workouts WHERE id=:id");
00147     q.bindValue(":id", id);
00148     if (!q.exec() || !q.next())
00149         return;
00150
00151     PlannedWorkoutDraft draft;
00152     draft.id = id;
00153     draft.date = QDate::fromString(q.value(0).toString(), Qt::ISODate);
00154     draft.title = q.value(1).toString();
00155     draft.type = q.value(2).toString();
00156     draft.durationSeconds = q.value(3).toDouble();
00157     draft.targetTss = q.value(4).toDouble();
00158     draft.notes = q.value(5).toString();
00159
00160     PlannedWorkoutDialog dlg(draft, this);
00161     if (dlg.exec() != QDialog::Accepted)
00162         return;
00163
00164     const PlannedWorkoutDraft saved = dlg.draft();
00165     QSqlQuery update(m_dbManager->database());
00166     update.prepare(
00167         "UPDATE planned_workouts SET workout_date=:workout_date, title=:title, type=:type, "
00168         "duration_sec=:duration_sec, target_tss=:target_tss, notes=:notes WHERE id=:id");
00169     update.bindValue(":workout_date", DateFormatter::toIsoDate(saved.date));
00170     update.bindValue(":title", saved.title);
00171     update.bindValue(":type", saved.type);
00172     update.bindValue(":duration_sec", saved.durationSeconds);
00173     update.bindValue(":target_tss", saved.targetTss);
00174     update.bindValue(":notes", saved.notes);
00175     update.bindValue(":id", id);
00176     if (!update.exec())
00177     {
00178         QMessageBox::critical(this, "Calendar", "Failed to update planned workout.");
00179         return;
00180     }
00181
00182     refresh();

```

```

00183 }
00184
00185 void CalendarWidget::deleteSelectedWorkout()
00186 {
00187     if (!m_dbManager || !m_dbManager->isOpen() || !m_dayList)
00188         return;
00189
00190     QListWidgetItem* item = m_dayList->currentItem();
00191     if (!item)
00192         return;
00193
00194     const int id = item->data(Qt::UserRole).toInt();
00195     if (id <= 0)
00196         return;
00197
00198     QSqlQuery q(m_dbManager->database());
00199     q.prepare("DELETE FROM planned_workouts WHERE id=:id");
00200     q.bindValue(":id", id);
00201     if (!q.exec())
00202     {
00203         QMessageBox::critical(this, "Calendar", "Failed to delete planned workout.");
00204         return;
00205     }
00206
00207     refresh();
00208 }
00209
00210 void CalendarWidget::recolorMonth()
00211 {
00212     updateMonthFormatting();
00213 }
00214
00215 void CalendarWidget::reloadDayList(const QDate& date)
00216 {
00217     if (!m_dayList)
00218         return;
00219
00220     m_dayList->clear();
00221     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00222     {
00223         m_dayList->addItem("Open a database and select an athlete.");
00224         return;
00225     }
00226
00227     QSqlQuery q(m_dbManager->database());
00228     q.prepare(
00229         "SELECT id, title, type, duration_sec, target_tss FROM planned_workouts "
00230         "WHERE athlete_id=:athlete_id AND workout_date=:workout_date ORDER BY id ASC");
00231     q.bindValue(":athlete_id", m_athleteId);
00232     q.bindValue(":workout_date", DateFormatter::toIsoDate(date));
00233     q.exec();
00234
00235     bool hasRows = false;
00236     while (q.next())
00237     {
00238         hasRows = true;
00239         const int id = q.value(0).toInt();
00240         const QString title = q.value(1).toString();
00241         const QString type = q.value(2).toString();
00242         const double durationSec = q.value(3).toDouble();
00243         const double tss = q.value(4).toDouble();
00244         const QString text = QString("%1 (%2, %3 h, TSS %4)")
00245             .arg(title)
00246             .arg(type.isEmpty() ? QStringLiteral("Planned") : type)
00247             .arg(durationSec / 3600.0, 0, 'f', 2)
00248             .arg(tss, 0, 'f', 0);
00249         auto* item = new QListWidgetItem(text, m_dayList);
00250         item->setData(Qt::UserRole, id);
00251     }
00252
00253     if (!hasRows)
00254         m_dayList->addItem("No planned workouts for this day.");
00255 }
00256
00257 void CalendarWidget::updateMonthFormatting()
00258 {
00259     if (!m_calendar)
00260         return;
00261
00262     const QTextCharFormat defaultFormat;
00263     const QDate shown = m_calendar->selectedDate().isValid() ? m_calendar->selectedDate() :
QDate::currentDate();
00264     const QDate monthStart(shown.year(), shown.month(), 1);
00265     const QDate monthEnd = monthStart.addMonths(1).addDays(-1);
00266
00267     for (QDate d = monthStart; d <= monthEnd; d = d.addDays(1))
00268         m_calendar->setTextFormat(d, defaultFormat);

```

```

00269
00270     if (!m_dbManager || !m_dbManager->isOpen() || m_athleteId <= 0)
00271         return;
00272
00273     QSqlQuery q(m_dbManager->database());
00274     q.prepare(
00275         "SELECT workout_date, SUM(target_tss), SUM(duration_sec), COUNT(*), MAX(type) "
00276         "FROM planned_workouts "
00277         "WHERE athlete_id=:athlete_id AND workout_date>=:start_date AND workout_date<=:end_date "
00278         "GROUP BY workout_date");
00279     q.bindValue(":athlete_id", m_athleteId);
00280     q.bindValue(":start_date", DateFormatter::toIsoDate(monthStart));
00281     q.bindValue(":end_date", DateFormatter::toIsoDate(monthEnd));
00282     q.exec();
00283
00284     while (q.next())
00285     {
00286         const QDate date = QDate::fromString(q.value(0).toString(), Qt::ISODate);
00287         const double totalTss = q.value(1).toDouble();
00288         const double totalSec = q.value(2).toDouble();
00289         const int count = q.value(3).toInt();
00290         const QString type = q.value(4).toString();
00291
00292         QColor bg("#dbeafe");
00293         const QString colorBy = m_colorByCombo ? m_colorByCombo->currentText() :
00294         QStringLiteral("TSS");
00295         if (colorBy == "TSS")
00296         {
00297             if (totalTss >= 200.0) bg = QColor("#991b1b");
00298             else if (totalTss >= 120.0) bg = QColor("#dc2626");
00299             else if (totalTss >= 70.0) bg = QColor("#f97316");
00300             else if (totalTss >= 30.0) bg = QColor("#facc15");
00301             else bg = QColor("#86efac");
00302         }
00303         else if (colorBy == "Duration")
00304         {
00305             const double hours = totalSec / 3600.0;
00306             if (hours >= 4.0) bg = QColor("#1d4ed8");
00307             else if (hours >= 2.0) bg = QColor("#2563eb");
00308             else if (hours >= 1.0) bg = QColor("#38bdf8");
00309             else bg = QColor("#bae6fd");
00310         }
00311         else
00312         {
00313             if (type.contains("VO2", Qt::CaseInsensitive)) bg = QColor("#ef4444");
00314             else if (type.contains("Threshold", Qt::CaseInsensitive)) bg = QColor("#f59e0b");
00315             else if (type.contains("Endurance", Qt::CaseInsensitive)) bg = QColor("#22c55e");
00316             else bg = QColor("#a3a3a3");
00317         }
00318         QTextCharFormat fmt;
00319         fmt.setBackground(bg);
00320         fmt.setForeground((bg.lightness() < 128) ? QColor("#f8f8fc") : QColor("#111827"));
00321         fmt.setToolTip(QString("%1 planned workouts").arg(count));
00322         m_calendar->setTextFormat(date, fmt);
00323     }
00324 }

```

## 8.103 CalendarWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QString>
00008 #include <QWidget>
00009
00010 class QCalendarWidget;
00011 class QComboBox;
00012 class QListWidget;
00013 class QPushButton;
00014 class QListWidgetItem;
00015 class DatabaseManager;
00016
00022 class CalendarWidget : public QWidget
00023 {
00024     Q_OBJECT
00025
00026 public:
00027     explicit CalendarWidget(QWidget* parent = nullptr);
00028
00029     void setDatabaseManager(DatabaseManager* dbManager);

```



```

00030
00031     void setAthleteId(int athleteId);
00032
00033     void refresh();
00034
00035 private slots:
00036     void onChanged();
00037     void addPlannedWorkout();
00038     void editSelectedWorkout();
00039     void deleteSelectedWorkout();
00040     void recolorMonth();
00041
00042 private:
00043     void reloadDayList(const QDate& date);
00044     void updateMonthFormatting();
00045
00046 private:
00047     QCalendarWidget* m_calendar = nullptr;
00048     QComboBox* m_colorByCombo = nullptr;
00049     QListWidget* m_dayList = nullptr;
00050     QPushButton* m_addButton = nullptr;
00051     QPushButton* m_editButton = nullptr;
00052     QPushButton* m_deleteButton = nullptr;
00053
00054     DatabaseManager* m_dbManager = nullptr;
00055     int m_athleteId = -1;
00056 };

```

## 8.104 ActivityViewController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ActivityViewController.h"
00004
00005 #include "../WorkoutController.h"
00006 #include "../../database/DatabaseManager.h"
00007 #include "../../database/ActivityRepository.h"
00008 #include "../../database/AthleteRepository.h"
00009 #include "../../core/settings/DateFormatter.h"
00010 #include "../AthleteHeaderWidget.h"
00011
00012 #include <QLabel>
00013 #include <QFileInfo>
00014
00021 ActivityViewController::ActivityViewController(
00022     WorkoutController* controller,
00023     DatabaseManager* dbManager,
00024     QObject* parent)
00025     : QObject(parent)
00026     , m_controller(controller)
00027     , m_dbManager(dbManager)
00028 {
00029     // Listen to workout controller signals
00030     if (m_controller)
00031     {
00032         connect(m_controller, &WorkoutController::workoutLoaded,
00033             this, &ActivityViewController::onWorkoutLoaded);
00034     }
00035 }
00036
00040 ActivityViewController::~ActivityViewController() = default;
00041
00045 void ActivityViewController::setAthleteContext(int athleteId, const QString& athleteName)
00046 {
00047     m_currentAthleteId = athleteId;
00048     m_currentAthleteName = athleteName;
00049 }
00050
00054 void ActivityViewController::setAthleteHeaderWidget(AthleteHeaderWidget* athleteHeader)
00055 {
00056     m_athleteHeader = athleteHeader;
00057 }
00058
00062 void ActivityViewController::setStatusLabels(
00063     QLabel* dbStatusLabel,
00064     QLabel* athleteStatusLabel,
00065     QLabel* activityCountLabel)
00066 {
00067     m_dbStatusLabel = dbStatusLabel;
00068     m_athleteStatusLabel = athleteStatusLabel;
00069     m_activityCountLabel = activityCountLabel;
00070 }
00071
00078 bool ActivityViewController::loadActivity(int activityId, QString& outError)

```

```

00079 {
00080     if (!m_controller)
00081     {
00082         outError = "Activity controller not initialized.";
00083         return false;
00084     }
00085
00086     if (!m_controller->loadActivity(activityId, outError))
00087         return false;
00088
00089     emit activityLoaded(activityId);
00090     return true;
00091 }
00092
00096 void ActivityViewController::refreshActivity()
00097 {
00098     updateActivityDisplay();
00099     emit activityDisplayUpdated();
00100 }
00101
00105 void ActivityViewController::updateActivityDisplay()
00106 {
00107     updateActivityMetadata();
00108     populateActivityDetails();
00109     syncAnalysisViews();
00110 }
00111
00115 void ActivityViewController::updateStatsDisplay()
00116 {
00117     updateActivityMetadata();
00118 }
00119
00123 void ActivityViewController::updateStatusBarInformation()
00124 {
00125     if (m_dbStatusLabel)
00126         m_dbStatusLabel->setText(
00127             m_dbManager && m_dbManager->isOpen()
00128             ? QString("Database: %1").arg(QFileInfo(m_dbManager->path()).fileName())
00129             : "Database: none");
00130
00131     if (m_athleteStatusLabel)
00132     {
00133         m_athleteStatusLabel->setText(
00134             m_currentAthleteName.isEmpty()
00135             ? "Athlete: none"
00136             : QString("Athlete: %1").arg(m_currentAthleteName));
00137     }
00138
00139     if (m_activityCountLabel)
00140     {
00141         int activityCount = 0;
00142         if (m_dbManager && m_dbManager->isOpen() && m_currentAthleteId > 0)
00143         {
00144             auto db = m_dbManager->database();
00145             ActivityRepository repo(db);
00146             activityCount = repo.listActivities(m_currentAthleteId).size();
00147         }
00148         m_activityCountLabel->setText(QString("Activities: %1").arg(activityCount));
00149     }
00150 }
00151
00155 void ActivityViewController::onWorkoutLoaded()
00156 {
00157     updateActivityDisplay();
00158     emit activityDisplayUpdated();
00159 }
00160
00164 void ActivityViewController::updateActivityMetadata()
00165 {
00166     if (!m_athleteHeader || !m_controller)
00167         return;
00168
00169     // TODO: Extract full metadata update from MainWindow::updateStatsLabel
00170     // This requires access to:
00171     // - m_currentAthleteId
00172     // - m_currentAthleteName
00173     // - Activity database queries
00174     // For now, this is a placeholder
00175 }
00176
00180 void ActivityViewController::populateActivityDetails()
00181 {
00182     // TODO: Extract detail panel population from MainWindow
00183 }
00184
00188 void ActivityViewController::syncAnalysisViews()
00189 {

```

```

00190     // TODO: Extract analysis view synchronization from MainWindow
00191     // This will delegate to ChartController and MapController once available
00192 }
00193
00197 QString ActivityViewController::formatActivityDateLabel(const QString& rawDate) const
00198 {
00199     QDate date = QDate::fromString(rawDate, Qt::ISODate);
00200     if (!date.isValid())
00201         return "-";
00202     return DateFormatter::toIsoDate(date);
00203 }

```

## 8.105 ActivityViewController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007 #include <memory>
00008
00009 class WorkoutController;
00010 class DatabaseManager;
00011
00021 class ActivityViewController : public QObject
00022 {
00023     Q_OBJECT
00024
00025 public:
00032     explicit ActivityViewController(
00033         WorkoutController* controller,
00034         DatabaseManager* dbManager,
00035         QObject* parent = nullptr);
00036
00040     ~ActivityViewController() override;
00041
00048     bool loadActivity(int activityId, QString& outError);
00049
00053     void refreshActivity();
00054
00060     void updateActivityDisplay();
00061
00067     void setAthleteContext(int athleteId, const QString& athleteName);
00068
00073     void setAthleteHeaderWidget(class AthleteHeaderWidget* athleteHeader);
00074
00081     void setStatusbarLabels(
00082         class QLabel* dbStatusLabel,
00083         class QLabel* athleteStatusLabel,
00084         class QLabel* activityCountLabel);
00085
00089     void updateStatsDisplay();
00090
00094     void updateStatusBarInformation();
00095
00096 signals:
00098     void activityLoaded(int activityId);
00099
00101     void activityDisplayUpdated();
00102
00104     void errorOccurred(const QString& message);
00105
00106 private slots:
00108     void onWorkoutLoaded();
00109
00110 private:
00112     void updateActivityMetadata();
00113
00115     void populateActivityDetails();
00116
00118     void syncAnalysisViews();
00119
00121     QString formatActivityDateLabel(const QString& rawDate) const;
00122
00123     // UI widget pointers
00124     class AthleteHeaderWidget* m_athleteHeader = nullptr;
00125     class QLabel* m_dbStatusLabel = nullptr;
00126     class QLabel* m_athleteStatusLabel = nullptr;
00127     class QLabel* m_activityCountLabel = nullptr;
00128
00129     // Athlete context
00130     int m_currentAthleteId = -1;
00131     QString m_currentAthleteName;

```

```

00132
00133     WorkoutController* m_controller;
00134     DatabaseManager* m_dbManager;
00135 };

```

## 8.106 ChartController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ChartController.h"
00004
00005 #include "../WorkoutController.h"
00006 #include "../../analysis/PowerCurve.h"
00007 #include "../../analysis/TrainingLoad.h"
00008 #include "../../core/zones/ZoneCalculator.h"
00009 #include "../../database/ActivityRepository.h"
00010 #include "../../database/DatabaseManager.h"
00011 #include "../../model/RideDataSerializer.h"
00012 #include "charts/RideChartWidget.h"
00013 #include "charts/PowerCurveWidget.h"
00014 #include "charts/PowerHistogramWidget.h"
00015 #include "charts/FitnessChartWidget.h"
00016 #include "core/zones/ZoneDefinition.h"
00017
00018 #include <QDate>
00019 #include <QTabWidget>
00020 #include <QStackedLayout>
00021 #include <QTableWidget>
00022 #include <QTableWidgetItem>
00023 #include <QComboBox>
00024 #include <QCheckBox>
00025 #include <QLayout>
00026 #include <QHBoxLayout>
00027 #include <QLabel>
00028 #include <QSpacerItem>
00029 #include <QSizePolicy>
00030
00031 #include <map>
00032 #include <cmath>
00033
00034 namespace
00035 {
00036     QString fmtDur(double seconds)
00037     {
00038         const int total = static_cast<int>(seconds);
00039         const int h = total / 3600;
00040         const int m = (total % 3600) / 60;
00041         const int s = total % 60;
00042         return h > 0
00043             ? QString("%1:%2:%3")
00044               .arg(h, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00045             : QString("%1:%2")
00046               .arg(m, 2, 10, QChar('0'));
00047     }
00048
00049     QString rangeLabelForZone(const Zone& zone, ColorMetric metric)
00050     {
00051         const QString unit = colorMetricUnit(metric);
00052         const QString minText = QString::number(zone.minValue, 'f', metric == ColorMetric::Speed ? 1 : 0);
00053         const QString maxText = QString::number(zone.maxValue, 'f', metric == ColorMetric::Speed ? 1 : 0);
00054
00055         if (zone.maxValue >= DBL_MAX / 2.0)
00056             return QString("> %1 %2").arg(minText, unit).trimmed();
00057
00058         return QString("%1 - %2 %3").arg(minText, maxText, unit).trimmed();
00059     }
00060 }
00061
00062 ChartController::ChartController(
00063     WorkoutController* controller,
00064     DatabaseManager* dbManager,
00065     QObject* parent)
00066     : QObject(parent)
00067     , m_controller(controller)
00068     , m_dbManager(dbManager)
00069 {
00070 }
00071
00072 ChartController::~ChartController() = default;
00073
00074 void ChartController::setChartWidgets(
00075     RideChartWidget* powerChart,
00076     RideChartWidget* hrChart,
00077     RideChartWidget* cadenceChart,

```

```

00090     RideChartWidget* speedChart,
00091     RideChartWidget* altitudeChart,
00092     PowerHistogramWidget* histogram,
00093     PowerCurveWidget* powerCurve,
00094     FitnessChartWidget* fitnessChart)
00095 {
00096     m_powerChart = powerChart;
00097     m_hrChart = hrChart;
00098     m_cadenceChart = cadenceChart;
00099     m_speedChart = speedChart;
00100     m_altitudeChart = altitudeChart;
00101     m_histogram = histogram;
00102     m_powerCurve = powerCurve;
00103     m_fitnessChart = fitnessChart;
00104
00105     connectChartSignals();
00106 }
00107
00111 void ChartController::setAnalysisTabWidgets(
00112     QTabWidget* analysisTabWidget,
00113     QStackedLayout* activityTabStack,
00114     QStackedLayout* zonesTabStack,
00115     QStackedLayout* histogramTabStack,
00116     QStackedLayout* powerCurveTabStack,
00117     QStackedLayout* calendarTabStack,
00118     QStackedLayout* fitnessTabStack)
00119 {
00120     m_analysisTabWidget = analysisTabWidget;
00121     m_activityTabStack = activityTabStack;
00122     m_zonesTabStack = zonesTabStack;
00123     m_histogramTabStack = histogramTabStack;
00124     m_powerCurveTabStack = powerCurveTabStack;
00125     m_calendarTabStack = calendarTabStack;
00126     m_fitnessTabStack = fitnessTabStack;
00127 }
00128
00132 void ChartController::setClimbingTabStack(QStackedLayout* climbingTabStack)
00133 {
00134     m_climbingTabStack = climbingTabStack;
00135 }
00136
00140 void ChartController::setZonesTable(QTableWidget* zonesTable)
00141 {
00142     m_zonesTable = zonesTable;
00143 }
00144
00148 void ChartController::setClimbCharts(
00149     RideChartWidget* climbPowerChart,
00150     RideChartWidget* climbHrChart,
00151     RideChartWidget* climbCadenceChart,
00152     RideChartWidget* climbSpeedChart,
00153     RideChartWidget* climbAltitudeChart)
00154 {
00155     m_climbPowerChart = climbPowerChart;
00156     m_climbHrChart = climbHrChart;
00157     m_climbCadenceChart = climbCadenceChart;
00158     m_climbSpeedChart = climbSpeedChart;
00159     m_climbAltitudeChart = climbAltitudeChart;
00160 }
00161
00165 void ChartController::setChartControls(
00166     QComboBox* colorMetricCombo,
00167     QComboBox* powerSmoothingCombo,
00168     QCheckBox* autoSmoothingCheck,
00169     QCheckBox* climbOverlayEnabledCheck,
00170     QComboBox* climbOverlayMetricCombo,
00171     QLayout* colorLegendLayout)
00172 {
00173     m_colorMetricCombo = colorMetricCombo;
00174     m_powerSmoothingCombo = powerSmoothingCombo;
00175     m_autoSmoothingCheck = autoSmoothingCheck;
00176     m_climbOverlayEnabledCheck = climbOverlayEnabledCheck;
00177     m_climbOverlayMetricCombo = climbOverlayMetricCombo;
00178     m_colorLegendLayout = colorLegendLayout;
00179 }
00180
00184 void ChartController::setAthleteId(int athleteId)
00185 {
00186     m_currentAthleteId = athleteId;
00187 }
00188
00192 void ChartController::updateCharts()
00193 {
00194     if (!m_controller)
00195         return;
00196
00197     const int colorMetricValue = m_colorMetricCombo ? m_colorMetricCombo->currentData().toInt() : 0;

```

```

00198     const ColorMetric colorMetric = static_cast<ColorMetric>(colorMetricValue);
00199
00200     if (!m_powerChart)
00201         return;
00202
00203     // Determine which climbs to show (filtered or all)
00204     const std::vector<Climb>& climbsForView = m_climbs.empty() ? m_controller->climbs() : m_climbs;
00205
00206     // Apply smoothing settings first
00207     applyChartSmoothing();
00208
00209     // Disable updates during population
00210     auto* parentWidget = qobject_cast<QWidget*>(parent());
00211     if (parentWidget)
00212         parentWidget->setUpdatesEnabled(false);
00213
00214     // Update main ride charts
00215     m_powerChart->setIntervals(m_controller->intervals());
00216     m_powerChart->setClimbs(climbsForView);
00217     if (m_hrChart) m_hrChart->setClimbs(climbsForView);
00218     if (m_cadenceChart) m_cadenceChart->setClimbs(climbsForView);
00219     if (m_speedChart) m_speedChart->setClimbs(climbsForView);
00220     if (m_altitudeChart) m_altitudeChart->setClimbs(climbsForView);
00221
00222     m_powerChart->setRideData(m_controller->rideData(), colorMetric, m_colorContext);
00223     if (m_hrChart) m_hrChart->setRideData(m_controller->rideData(), colorMetric, m_colorContext);
00224     if (m_cadenceChart) m_cadenceChart->setRideData(m_controller->rideData(), colorMetric,
m_colorContext);
00225     if (m_speedChart) m_speedChart->setRideData(m_controller->rideData(), colorMetric,
m_colorContext);
00226     if (m_altitudeChart) m_altitudeChart->setRideData(m_controller->rideData(), colorMetric,
m_colorContext);
00227
00228     // Update climb-specific charts if available
00229     if (m_climbPowerChart)
00230     {
00231         m_climbPowerChart->setClimbs(climbsForView);
00232         if (m_climbHrChart) m_climbHrChart->setClimbs(climbsForView);
00233         if (m_climbCadenceChart) m_climbCadenceChart->setClimbs(climbsForView);
00234         if (m_climbSpeedChart) m_climbSpeedChart->setClimbs(climbsForView);
00235         if (m_climbAltitudeChart) m_climbAltitudeChart->setClimbs(climbsForView);
00236
00237         m_climbPowerChart->setRideData(m_controller->rideData(), colorMetric, m_colorContext);
00238         if (m_climbHrChart) m_climbHrChart->setRideData(m_controller->rideData(), colorMetric,
m_colorContext);
00239         if (m_climbCadenceChart) m_climbCadenceChart->setRideData(m_controller->rideData(),
colorMetric, m_colorContext);
00240         if (m_climbSpeedChart) m_climbSpeedChart->setRideData(m_controller->rideData(), colorMetric,
m_colorContext);
00241         if (m_climbAltitudeChart) m_climbAltitudeChart->setRideData(m_controller->rideData(),
colorMetric, m_colorContext);
00242
00243         // Apply climb overlay if enabled
00244         if (m_climbAltitudeChart && m_climbOverlayEnabledCheck && m_climbOverlayMetricCombo)
00245         {
00246             const ColorMetric overlayMetric =
static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
00247             m_climbAltitudeChart->setMetricOverlay(
overlayMetric,
00249             m_climbOverlayEnabledCheck->isChecked());
00250         }
00251     }
00252
00253     if (parentWidget)
00254         parentWidget->setUpdatesEnabled(true);
00255
00256     emit chartsUpdated();
00257 }
00258
00262 void ChartController::refreshCharts()
00263 {
00264     updateCharts();
00265     updateColorLegend();
00266     updateZoneAvailability();
00267 }
00268
00272 void ChartController::resetAllZoom()
00273 {
00274     if (m_powerChart) m_powerChart->fitToData();
00275     if (m_hrChart) m_hrChart->fitToData();
00276     if (m_cadenceChart) m_cadenceChart->fitToData();
00277     if (m_speedChart) m_speedChart->fitToData();
00278     if (m_altitudeChart) m_altitudeChart->fitToData();
00279 }
00280
00284 void ChartController::updateHistogram()
00285 {

```

```

00286     if (!m_histogram || !m_controller)
00287         return;
00288
00289     m_histogram->setRideData(m_controller->rideData());
00290 }
00291
00295 void ChartController::updateFitnessChart()
00296 {
00297     if (!m_fitnessChart || !m_controller)
00298         return;
00299
00300     if (!m_dbManager || !m_dbManager->isOpen() || m_currentAthleteId <= 0)
00301     {
00302         m_fitnessChart->clearChart();
00303         return;
00304     }
00305
00306     auto db = m_dbManager->database();
00307     ActivityRepository repo(db);
00308     const QList<Activity> activities = repo.listActivities(m_currentAthleteId);
00309     if (activities.isEmpty())
00310     {
00311         m_fitnessChart->clearChart();
00312         return;
00313     }
00314
00315     std::map<QDate, double> tssPerDay;
00316     for (const Activity& activity : activities)
00317     {
00318         QString raw = !activity.startTime.isEmpty() ? activity.startTime.left(10) :
activity.importedAt.left(10);
00319         QDate day = QDate::fromString(raw, Qt::ISODate);
00320         if (!day.isValid())
00321             continue;
00322
00323         const double tss = TrainingLoad::trainingStressScore(
00324             activity.durationSec,
00325             activity.normalizedPower,
00326             m_controller->ftp());
00327         tssPerDay[day] += tss;
00328     }
00329
00330     if (tssPerDay.empty())
00331     {
00332         m_fitnessChart->clearChart();
00333         return;
00334     }
00335
00336     const QDate first = tssPerDay.begin()->first;
00337     const QDate last = tssPerDay.rbegin()->first;
00338     std::vector<DailyLoadPoint> daily;
00339     daily.reserve(static_cast<size_t>(first.daysTo(last) + 1));
00340     for (QDate day = first; day <= last; day = day.addDays(1))
00341     {
00342         DailyLoadPoint point;
00343         auto it = tssPerDay.find(day);
00344         if (it != tssPerDay.end())
00345             point.tss = it->second;
00346         daily.push_back(point);
00347     }
00348
00349     const std::vector<FitnessMetricsPoint> timeline = TrainingLoad::fitnessTimeline(daily);
00350     m_fitnessChart->setTimeline(timeline);
00351 }
00352
00356 void ChartController::updateAnalysisEmptyStates()
00357 {
00358     if (!m_controller)
00359         return;
00360
00361     const bool hasActivity = !m_controller->rideData().records.empty();
00362     const int page = hasActivity ? 1 : 0;
00363
00364     const std::initializer_list<QStackedLayout*> stacks =
00365     { m_activityTabStack,
00366       m_zonesTabStack,
00367       m_histogramTabStack,
00368       m_powerCurveTabStack,
00369       m_calendarTabStack,
00370       m_fitnessTabStack,
00371       m_climbingTabStack };
00372
00373     for (auto* stack : stacks)
00374     {
00375         if (stack)
00376             stack->setCurrentIndex(page);
00377     }

```

```

00378 }
00379
00383 void ChartController::syncChartZoom()
00384 {
00385     const std::initializer_list<RideChartWidget*> all =
00386         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00387
00388     for (auto* src : all)
00389     {
00390         if (!src)
00391             continue;
00392         for (auto* dst : all)
00393         {
00394             if (!dst || src == dst)
00395                 continue;
00396             connect(src, &RideChartWidget::xRangeChanged,
00397                 dst, &RideChartWidget::setXRange,
00398                 Qt::UniqueConnection);
00399         }
00400     }
00401 }
00402
00406 void ChartController::syncChartCrosshair()
00407 {
00408     const std::initializer_list<RideChartWidget*> all =
00409         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00410
00411     for (auto* src : all)
00412     {
00413         if (!src)
00414             continue;
00415         for (auto* dst : all)
00416         {
00417             if (!dst || src == dst)
00418                 continue;
00419             connect(src, &RideChartWidget::crosshairMoved,
00420                 dst, &RideChartWidget::setCrosshair,
00421                 Qt::UniqueConnection);
00422         }
00423     }
00424 }
00425
00429 void ChartController::connectChartSignals()
00430 {
00431     syncChartZoom();
00432     syncChartCrosshair();
00433 }
00434
00438 void ChartController::updateColorLegendDisplay()
00439 {
00440     if (!m_colorLegendLayout)
00441         return;
00442
00443     QWidget* legendHost = m_colorLegendLayout->parentWidget();
00444
00445     while (QLayoutItem* item = m_colorLegendLayout->takeAt(0))
00446     {
00447         if (QWidget* widget = item->widget())
00448             widget->deleteLater();
00449         delete item;
00450     }
00451
00452     const ColorMetric metric = static_cast<ColorMetric>(m_colorMetric);
00453     if (metric == ColorMetric::None)
00454     {
00455         auto* label = new QLabel(QStringLiteral("Coloring disabled"), legendHost);
00456         label->setStyleSheet(QStringLiteral("color: #64748b;"));
00457         m_colorLegendLayout->addWidget(label);
00458         m_colorLegendLayout->addItem(new QSpacerItem(0, 0, QSizePolicy::Expanding,
00459             QSizePolicy::Minimum));
00459         return;
00460     }
00461
00462     const std::vector<Zone> zones = ZoneCalculator::zonesForMetric(metric, m_colorContext);
00463     for (const Zone& zone : zones)
00464     {
00465         auto* swatch = new QLabel(legendHost);
00466         swatch->setFixedSize(12, 12);
00467         swatch->setStyleSheet(QString("background: %1; border: 1px solid rgba(15,23,42,0.15);")
00468             .arg(zone.color.name()));
00469
00470         auto* text = new QLabel(zone.name, legendHost);
00471
00472         auto* chip = new QWidget(legendHost);
00473         auto* chipLayout = new QHBoxLayout(chip);
00474         chipLayout->setContentsMargins(0, 0, 0, 0);
00475         chipLayout->setSpacing(4);

```



```

00476         chipLayout->addWidget(swatch);
00477         chipLayout->addWidget(text);
00478         m_colorLegendLayout->addWidget(chip);
00479     }
00480
00481     m_colorLegendLayout->addItem(new QSpacerItem(0, 0, QSizePolicy::Expanding, QSizePolicy::Minimum));
00482 }
00483
00487 int ChartController::currentColorMetric() const
00488 {
00489     return m_colorMetric;
00490 }
00491
00495 void ChartController::setIntervals(const std::vector<Interval>& intervals)
00496 {
00497     m_intervals = intervals;
00498 }
00499
00503 void ChartController::setClimbs(const std::vector<Climb>& climbs)
00504 {
00505     m_climbs = climbs;
00506 }
00507
00511 void ChartController::setColorMetric(int colorMetric)
00512 {
00513     m_colorMetric = colorMetric;
00514 }
00515
00519 void ChartController::setColorContext(const ColorContext& colorContext)
00520 {
00521     m_colorContext = colorContext;
00522 }
00523
00531 void ChartController::updateZoneAvailability()
00532 {
00533     if (!m_analysisTabWidget)
00534         return;
00535
00536     const bool hasPower = m_controller && m_controller->statistics().maximumPower > 0.0;
00537     const ColorMetric metric = static_cast<ColorMetric>(m_colorMetric);
00538     const bool hasSelectedMetric = m_controller
00539         && ZoneCalculator::hasMetricSamples(m_controller->rideData(), metric);
00540
00541     // Power page sub-tabs: [0] Zones, [1] Histogram, [2] Power Curve
00542     m_analysisTabWidget->setTabEnabled(0,
00543         metric != ColorMetric::None && hasSelectedMetric);
00544     m_analysisTabWidget->setTabEnabled(1, hasPower);
00545     m_analysisTabWidget->setTabEnabled(2, hasPower);
00546 }
00547
00551 void ChartController::handleWorkoutLoaded()
00552 {
00553     const bool hasPower = m_controller && m_controller->statistics().maximumPower > 0.0;
00554
00555     updateCharts();
00556     updateColorLegend();
00557     updateZoneAvailability();
00558
00559     if (hasPower)
00560     {
00561         updateZonesTab();
00562         updateHistogram();
00563         updatePowerCurve();
00564         updateFitnessChart();
00565     }
00566
00567     resetAllZoom();
00568 }
00569
00573 void ChartController::applyColorMetric()
00574 {
00575     updateCharts();
00576     updateColorLegend();
00577     updateZoneAvailability();
00578 }
00579
00583 void ChartController::updateZonesTab()
00584 {
00585     if (!m_zonesTable || !m_controller)
00586         return;
00587
00588     const ColorMetric metric = static_cast<ColorMetric>(m_colorMetric);
00589     const std::vector<ZoneDistribution> distributions = ZoneCalculator::computeDistribution(
00590         m_controller->rideData(),
00591         metric,
00592         m_colorContext);
00593 }

```

```

00594     m_zonesTable->setRowCount(static_cast<int>(distributions.size()));
00595
00596     for (int row = 0; row < static_cast<int>(distributions.size()); ++row)
00597     {
00598         const ZoneDistribution& distribution = distributions[static_cast<size_t>(row)];
00599
00600         auto mkItem = [](const QString& text)
00601         {
00602             auto* item = new QTableWidgetItem(text);
00603             item->setTextAlignment(Qt::AlignCenter);
00604             return item;
00605         };
00606
00607         auto* zoneItem = mkItem(QString("Z%1").arg(row + 1));
00608         zoneItem->setBackground(distribution.zone.color);
00609         zoneItem->setForeground(QColor("#111827"));
00610         m_zonesTable->setItem(row, 0, zoneItem);
00611
00612         auto* nameItem = new QTableWidgetItem(distribution.zone.name);
00613         nameItem->setTextAlignment(Qt::AlignLeft | Qt::AlignVCenter);
00614         m_zonesTable->setItem(row, 1, nameItem);
00615
00616         m_zonesTable->setItem(row, 2, mkItem(rangeLabelForZone(distribution.zone, metric)));
00617         m_zonesTable->setItem(row, 3, mkItem(fmtDur(distribution.seconds)));
00618         m_zonesTable->setItem(row, 4, mkItem(QString::number(distribution.percent * 100.0, 'f', 0) +
00619 "%");
00620     }
00621     m_zonesTable->resizeColumnsToContents();
00622 }
00623
00627 void ChartController::updatePowerCurve()
00628 {
00629     if (!m_powerCurve || !m_controller)
00630         return;
00631
00632     std::vector<PowerCurvePoint> last90;
00633     std::vector<PowerCurvePoint> currentYear;
00634     std::vector<PowerCurvePoint> allTime;
00635
00636     if (m_dbManager && m_dbManager->isOpen() && m_currentAthleteId > 0)
00637     {
00638         auto db = m_dbManager->database();
00639         ActivityRepository repo(db);
00640         const QList<Activity> activities = repo.listActivities(m_currentAthleteId);
00641         const std::vector<double> durations = PowerCurve::standardDurations();
00642
00643         auto aggregate = [&](auto predicate)
00644         {
00645             std::vector<double> best(durations.size(), 0.0);
00646             for (const Activity& activity : activities)
00647             {
00648                 QString raw = !activity.startTime.isEmpty() ? activity.startTime.left(10) :
00649 activity.importedAt.left(10);
00650                 const QDate day = QDate::fromString(raw, Qt::ISODate);
00651                 if (!predicate(day))
00652                     continue;
00653
00654                 const RideData ride = RideDataSerializer::loadRideFromDatabase(activity.id,
00655 *m_dbManager);
00656                 if (ride.records.empty())
00657                     continue;
00658
00659                 for (size_t i = 0; i < durations.size(); ++i)
00660                 {
00661                     const double p = PowerCurve::bestMeanPower(ride, durations[i]);
00662                     if (p > best[i])
00663                         best[i] = p;
00664                 }
00665
00666                 std::vector<PowerCurvePoint> out;
00667                 for (size_t i = 0; i < durations.size(); ++i)
00668                 {
00669                     if (best[i] > 0.0)
00670                         out.push_back({durations[i], best[i]});
00671                 }
00672                 return out;
00673             };
00674
00675             const QDate today = QDate::currentDate();
00676             const QDate ninetyAgo = today.addDays(-90);
00677             const int year = today.year();
00678
00679             last90 = aggregate([&](const QDate& day)
00680             {
00681                 return day.isValid() && day >= ninetyAgo;

```

```

00681         });
00682         currentYear = aggregate([&](const QDate& day)
00683         {
00684             return day.isValid() && day.year() == year;
00685         });
00686         allTime = aggregate([&](const QDate& day)
00687         {
00688             return day.isValid();
00689         });
00690     }
00691
00692     m_powerCurve->setRideDataWithComparisons(
00693         m_controller->rideData(),
00694         last90,
00695         currentYear,
00696         allTime,
00697         m_controller->ftp());
00698 }
00699
00700 void ChartController::updateColorLegend()
00701 {
00702     updateColorLegendDisplay();
00703 }
00704
00705 void ChartController::applyChartSmoothing()
00706 {
00707     const std::initializer_list<RideChartWidget*> all =
00708     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart,
00709       m_climbPowerChart, m_climbHrChart, m_climbCadenceChart, m_climbSpeedChart,
00710       m_climbAltitudeChart };
00711
00712     const int smoothingSeconds = m_powerSmoothingCombo ? m_powerSmoothingCombo->currentData().toInt()
00713     : 0;
00714     const bool autoSmoothing = m_autoSmoothingCheck ? m_autoSmoothingCheck->isChecked() : false;
00715
00716     for (auto* chart : all)
00717     {
00718         if (!chart)
00719             continue;
00720         chart->setPowerSmoothingSeconds(smoothingSeconds);
00721         chart->setAutoSmoothingEnabled(autoSmoothing);
00722     }
00723 }
00724
00725 }

```

## 8.107 ChartController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <vector>
00006
00007 #include <QObject>
00008 #include <QString>
00009
00010 #include "core/zones/ZoneDefinition.h"
00011
00012 class WorkoutController;
00013 class DatabaseManager;
00014 class RideChartWidget;
00015 class PowerCurveWidget;
00016 class PowerHistogramWidget;
00017 class FitnessChartWidget;
00018 class QTabWidget;
00019 class QStackedLayout;
00020 class QTableWidget;
00021 class QComboBox;
00022 class QCheckBox;
00023 class QLayout;
00024 class QWidget;
00025 class Interval;
00026 class Climb;
00027
00028 class ChartController : public QObject
00029 {
00030     Q_OBJECT
00031
00032 public:
00033     explicit ChartController(
00034         WorkoutController* controller,
00035         DatabaseManager* dbManager,
00036         QObject* parent = nullptr);
00037     ~ChartController() override;
00038 }

```

```

00045     void setChartWidgets (
00046         RideChartWidget* powerChart,
00047         RideChartWidget* hrChart,
00048         RideChartWidget* cadenceChart,
00049         RideChartWidget* speedChart,
00050         RideChartWidget* altitudeChart,
00051         PowerHistogramWidget* histogram,
00052         PowerCurveWidget* powerCurve,
00053         FitnessChartWidget* fitnessChart);
00054
00055     void setAnalysisTabWidgets (
00056         QTabWidget* analysisTabWidget,
00057         QStackedLayout* activityTabStack,
00058         QStackedLayout* zonesTabStack,
00059         QStackedLayout* histogramTabStack,
00060         QStackedLayout* powerCurveTabStack,
00061         QStackedLayout* calendarTabStack,
00062         QStackedLayout* fitnessTabStack);
00063
00064     void setClimbingTabStack (QStackedLayout* climbingTabStack);
00065
00066     void setZonesTable (QTableWidget* zonesTable);
00067
00068     void setClimbCharts (
00069         RideChartWidget* climbPowerChart,
00070         RideChartWidget* climbHrChart,
00071         RideChartWidget* climbCadenceChart,
00072         RideChartWidget* climbSpeedChart,
00073         RideChartWidget* climbAltitudeChart);
00074
00075     void setChartControls (
00076         QComboBox* colorMetricCombo,
00077         QComboBox* powerSmoothingCombo,
00078         QCheckBox* autoSmoothingCheck,
00079         QCheckBox* climbOverlayEnabledCheck,
00080         QComboBox* climbOverlayMetricCombo,
00081         QLayout* colorLegendLayout);
00082
00083     void setAthleteId (int athleteId);
00084
00085     void updateCharts ();
00086
00087     void refreshCharts ();
00088
00089     void resetAllZoom ();
00090
00091     void updateHistogram ();
00092
00093     void updateFitnessChart ();
00094
00095     void updateAnalysisEmptyStates ();
00096
00097     void updateZonesTab ();
00098
00099     void updatePowerCurve ();
00100
00101     void updateColorLegend ();
00102
00103     void applyChartSmoothing ();
00104
00105     void syncChartZoom ();
00106
00107     void syncChartCrosshair ();
00108
00109     void setIntervals (const std::vector<Interval>& intervals);
00110
00111     void setClimbs (const std::vector<Climb>& climbs);
00112
00113     void setColorMetric (int colorMetric);
00114
00115     void setColorContext (const ColorContext& colorContext);
00116
00117     void updateZoneAvailability ();
00118
00119     void handleWorkoutLoaded ();
00120
00121 signals:
00122     void chartsUpdated ();
00123
00124     void errorOccurred (const QString& message);
00125
00126 private:
00127     void connectChartSignals ();
00128
00129     void applyColorMetric ();
00130
00131     void updateColorLegendDisplay ();

```

```

00213
00215     int currentColorMetric() const;
00216
00217     // Data members
00218     WorkoutController* m_controller;
00219     DatabaseManager* m_dbManager;
00220
00221     // Main chart widgets
00222     RideChartWidget* m_powerChart = nullptr;
00223     RideChartWidget* m_hrChart = nullptr;
00224     RideChartWidget* m_cadenceChart = nullptr;
00225     RideChartWidget* m_speedChart = nullptr;
00226     RideChartWidget* m_altitudeChart = nullptr;
00227     PowerHistogramWidget* m_histogram = nullptr;
00228     PowerCurveWidget* m_powerCurve = nullptr;
00229     FitnessChartWidget* m_fitnessChart = nullptr;
00230
00231     // Analysis tab widgets
00232     QTabWidget* m_analysisTabWidget = nullptr;
00233     QStackedLayout* m_activityTabStack = nullptr;
00234     QStackedLayout* m_zonesTabStack = nullptr;
00235     QStackedLayout* m_histogramTabStack = nullptr;
00236     QStackedLayout* m_powerCurveTabStack = nullptr;
00237     QStackedLayout* m_calendarTabStack = nullptr;
00238     QStackedLayout* m_fitnessTabStack = nullptr;
00239     QStackedLayout* m_climbingTabStack = nullptr;
00240
00241     // Chart data
00242     std::vector<Interval> m_intervals;
00243     std::vector<Climb> m_climbs;
00244     int m_colorMetric = 0;
00245     ColorContext m_colorContext;
00246     bool m_smoothingEnabled = false;
00247     int m_smoothingAmount = 0;
00248
00249     // Additional chart widgets
00250     RideChartWidget* m_climbPowerChart = nullptr;
00251     RideChartWidget* m_climbHrChart = nullptr;
00252     RideChartWidget* m_climbCadenceChart = nullptr;
00253     RideChartWidget* m_climbSpeedChart = nullptr;
00254     RideChartWidget* m_climbAltitudeChart = nullptr;
00255     QTableWidget* m_zonesTable = nullptr;
00256
00257     // Chart controls
00258     QComboBox* m_colorMetricCombo = nullptr;
00259     QComboBox* m_powerSmoothingCombo = nullptr;
00260     QCheckBox* m_autoSmoothingCheck = nullptr;
00261     QCheckBox* m_climbOverlayEnabledCheck = nullptr;
00262     QComboBox* m_climbOverlayMetricCombo = nullptr;
00263     QLayout* m_colorLegendLayout = nullptr;
00264
00265     // Athlete context
00266     int m_currentAthleteId = -1;
00267 };

```

## 8.108 ImportController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportController.h"
00004
00005 #include "../WorkoutController.h"
00006 #include "../../database/DatabaseManager.h"
00007 #include "import/ImportQueue.h"
00008 #include "import/ImportProgressModel.h"
00009 #include "../ImportStatusWidget.h"
00010
00017 ImportController::ImportController(
00018     WorkoutController* controller,
00019     DatabaseManager* dbManager,
00020     QObject* parent)
00021     : QObject(parent)
00022     , m_controller(controller)
00023     , m_dbManager(dbManager)
00024 {
00025 }
00026
00030 ImportController::~ImportController() = default;
00031
00036 void ImportController::setImportQueue(ImportQueue* importQueue)
00037 {
00038     m_importQueue = importQueue;
00039 }
00040

```

```

00045 void ImportController::setProgressModel(ImportProgressModel* progressModel)
00046 {
00047     m_progressModel = progressModel;
00048 }
00049
00054 void ImportController::setStatusWidget(ImportStatusWidget* statusWidget)
00055 {
00056     m_statusWidget = statusWidget;
00057 }
00058
00066 void ImportController::queueFilesForImport(
00067     const QStringList& filePaths,
00068     int athleteId,
00069     bool showResultDialog,
00070     const QString& sourceLabel)
00071 {
00072     // TODO: Extract queue management logic from MainWindow
00073     emit filesQueued(filePaths.size(), sourceLabel);
00074 }
00075
00080 bool ImportController::canImportActivities() const
00081 {
00082     if (!m_dbManager || !m_controller)
00083         return false;
00084
00085     return m_dbManager->isOpen() && m_controller->currentAthleteId() > 0;
00086 }
00087
00091 void ImportController::updateImportAvailability()
00092 {
00093     const bool available = canImportActivities();
00094     emit importAvailabilityChanged(available);
00095     // TODO: Extract availability update logic from MainWindow
00096 }
00097
00102 ImportProgressModel* ImportController::progressModel() const
00103 {
00104     return m_progressModel;
00105 }
00106
00111 ImportQueue* ImportController::importQueue() const
00112 {
00113     return m_importQueue;
00114 }
00115
00119 void ImportController::finalizeBatches()
00120 {
00121     // TODO: Extract batch finalization logic from MainWindow
00122     emit importBatchesCompleted();
00123 }
00124
00128 void ImportController::onActivityImportComplete(int activityId)
00129 {
00130     emit activityImported(activityId);
00131 }

```

## 8.109 ImportController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007 #include <QStringList>
00008 #include <memory>
00009
00010 class WorkoutController;
00011 class DatabaseManager;
00012 class ImportQueue;
00013 class ImportProgressModel;
00014 class ImportStatusWidget;
00015
00027 class ImportController : public QObject
00028 {
00029     Q_OBJECT
00030
00031 public:
00032     explicit ImportController(
00033         WorkoutController* controller,
00034         DatabaseManager* dbManager,
00035         QObject* parent = nullptr);
00036
00046     ~ImportController() override;

```

```

00047
00052     void setImportQueue(ImportQueue* importQueue);
00053
00058     void setProgressModel(ImportProgressModel* progressModel);
00059
00064     void setStatusWidget(ImportStatusWidget* statusWidget);
00065
00073     void queueFilesForImport(
00074         const QStringList& filePaths,
00075         int athleteId,
00076         bool showResultDialog,
00077         const QString& sourceLabel);
00078
00083     bool canImportActivities() const;
00084
00088     void updateImportAvailability();
00089
00094     ImportProgressModel* progressModel() const;
00095
00100     ImportQueue* importQueue() const;
00101
00102 signals:
00104     void filesQueued(int count, const QString& source);
00105
00107     void activityImported(int activityId);
00108
00110     void importBatchesCompleted();
00111
00113     void importAvailabilityChanged(bool available);
00114
00116     void errorOccurred(const QString& message);
00117
00118 private:
00120     void finalizeBatches();
00121
00123     void onActivityImportComplete(int activityId);
00124
00125     WorkoutController* m_controller;
00126     DatabaseManager* m_dbManager;
00127     ImportQueue* m_importQueue = nullptr;
00128     ImportProgressModel* m_progressModel = nullptr;
00129     ImportStatusWidget* m_statusWidget = nullptr;
00130 };

```

## 8.110 MainWindowActions.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindowActions.h"
00004
00005 #include <QMainWindow>
00006 #include <QMenu>
00007 #include <QMenuBar>
00008 #include <QToolBar>
00009 #include <QAction>
00010 #include <QComboBox>
00011 #include <QLabel>
00012 #include <QStyle>
00013
00019 MainWindowActions::MainWindowActions(QMainWindow* mainWindow, QObject* parent)
00020     : QObject(parent)
00021     , m_mainWindow(mainWindow)
00022 {
00023 }
00024
00028 MainWindowActions::~MainWindowActions() = default;
00029
00033 void MainWindowActions::buildMenus()
00034 {
00035     if (!m_mainWindow)
00036         return;
00037
00038     createFileMenu();
00039     createEditMenu();
00040     createViewMenu();
00041     createAnalysisMenu();
00042     createToolsMenu();
00043     createHelpMenu();
00044 }
00045
00049 void MainWindowActions::buildToolbar()
00050 {
00051     if (!m_mainWindow)
00052         return;

```

```

00053
00054     // TODO: Extract toolbar creation from MainWindow
00055 }
00056
00060 void MainWindowActions::setupShortcuts()
00061 {
00062     // TODO: Extract keyboard shortcuts setup from MainWindow
00063 }
00064
00068 void MainWindowActions::createFileMenu()
00069 {
00070     if (!m_mainWindow)
00071         return;
00072
00073     QMenu* fileMenu = m_mainWindow->menuBar()->addMenu("&File");
00074
00075     // TODO: Extract File menu creation from MainWindow
00076 }
00077
00081 void MainWindowActions::createEditMenu()
00082 {
00083     if (!m_mainWindow)
00084         return;
00085
00086     QMenu* editMenu = m_mainWindow->menuBar()->addMenu("&Edit");
00087
00088     // TODO: Extract Edit menu creation from MainWindow
00089 }
00090
00094 void MainWindowActions::createViewMenu()
00095 {
00096     if (!m_mainWindow)
00097         return;
00098
00099     QMenu* viewMenu = m_mainWindow->menuBar()->addMenu("&View");
00100
00101     // TODO: Extract View menu creation from MainWindow
00102 }
00103
00107 void MainWindowActions::createAnalysisMenu()
00108 {
00109     if (!m_mainWindow)
00110         return;
00111
00112     QMenu* analysisMenu = m_mainWindow->menuBar()->addMenu("&Analysis");
00113
00114     // TODO: Extract Analysis menu creation from MainWindow
00115 }
00116
00120 void MainWindowActions::createToolsMenu()
00121 {
00122     if (!m_mainWindow)
00123         return;
00124
00125     QMenu* toolsMenu = m_mainWindow->menuBar()->addMenu("&Tools");
00126
00127     // TODO: Extract Tools menu creation from MainWindow
00128 }
00129
00133 void MainWindowActions::createHelpMenu()
00134 {
00135     if (!m_mainWindow)
00136         return;
00137
00138     QMenu* helpMenu = m_mainWindow->menuBar()->addMenu("&Help");
00139
00140     // TODO: Extract Help menu creation from MainWindow
00141 }

```

## 8.111 MainWindowActions.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QAction>
00007 #include <memory>
00008
00009 class QMainWindow;
00010 class QMenu;
00011 class QToolBar;
00012 class QComboBox;
00013 class QPushButton;

```



```

00014
00028 class MainWindowActions : public QObject
00029 {
00030     Q_OBJECT
00031
00032 public:
00038     explicit MainWindowActions(QMainWindow* mainWindow, QObject* parent = nullptr);
00039
00043     ~MainWindowActions() override;
00044
00048     void buildMenus();
00049
00053     void buildToolbar();
00054
00058     void setupShortcuts();
00059
00060     // Action accessors
00061     QAction* importAction() const { return m_importAct; }
00062     QAction* toolbarImportAction() const { return m_toolbarImportAct; }
00063     QAction* createVideoAction() const { return m_createVideoAct; }
00064     QAction* previewFitAction() const { return m_previewFitAct; }
00065     QMenu* recentDatabaseMenu() const { return m_recentDbMenu; }
00066
00067     // Toolbar widget accessors
00068     QComboBox* athleteCombo() const { return m_athleteCombo; }
00069
00070 signals:
00072     void importTriggered();
00073
00075     void createVideoTriggered();
00076
00078     void previewFitTriggered();
00079
00081     void openDatabaseRequested();
00082
00084     void createDatabaseRequested();
00085
00087     void manageAthletesRequested();
00088
00090     void openSettingsRequested();
00091
00093     void showAboutRequested();
00094
00096     void detectClimbsRequested();
00097
00099     void detectIntervalsRequested();
00100
00102     void athleteSelectionChanged(int athleteId);
00103
00104 private:
00106     void createFileMenu();
00107
00109     void createEditMenu();
00110
00112     void createViewMenu();
00113
00115     void createAnalysisMenu();
00116
00118     void createToolsMenu();
00119
00121     void createHelpMenu();
00122
00123     QMainWindow* m_mainWindow = nullptr;
00124
00125     // File menu actions
00126     QAction* m_importAct = nullptr;
00127     QAction* m_previewFitAct = nullptr;
00128     QAction* m_openDatabaseAct = nullptr;
00129     QAction* m_createDatabaseAct = nullptr;
00130     QMenu* m_recentDbMenu = nullptr;
00131     QAction* m_backupDatabaseAct = nullptr;
00132     QAction* m_exitAct = nullptr;
00133
00134     // Edit menu actions
00135     QAction* m_manageAthletesAct = nullptr;
00136     QAction* m_openSettingsAct = nullptr;
00137
00138     // View menu actions
00139     QAction* m_fullscreenAct = nullptr;
00140
00141     // Analysis menu actions
00142     QAction* m_detectClimbsAct = nullptr;
00143     QAction* m_detectIntervalsAct = nullptr;
00144     QAction* m_revertClimbAct = nullptr;
00145
00146     // Tools menu actions
00147     QAction* m_createVideoAct = nullptr;

```

```

00148
00149     // Help menu actions
00150     QAction* m_aboutAct = nullptr;
00151
00152     // Toolbar actions
00153     QAction* m_toolbarImportAct = nullptr;
00154     QAction* m_toolbarSearchAct = nullptr;
00155     QComboBox* m_athleteCombo = nullptr;
00156 };

```

## 8.112 MapController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MapController.h"
00004
00005 #include "../WorkoutController.h"
00006 #include "../../database/DatabaseManager.h"
00007 #include "maps/MapRenderer.h"
00008
00015 MapController::MapController(
00016     WorkoutController* controller,
00017     DatabaseManager* dbManager,
00018     QObject* parent)
00019     : QObject(parent)
00020     , m_controller(controller)
00021     , m_dbManager(dbManager)
00022 {
00023 }
00024
00028 MapController::~MapController() = default;
00029
00034 void MapController::setMapRenderer(MapRenderer* mapRenderer)
00035 {
00036     m_mapRenderer = mapRenderer;
00037     connectMapSignals();
00038 }
00039
00044 MapRenderer* MapController::mapRenderer() const
00045 {
00046     return m_mapRenderer;
00047 }
00048
00052 void MapController::updateMap()
00053 {
00054     // TODO: Extract map update logic from MainWindow
00055     emit mapUpdated();
00056 }
00057
00061 void MapController::refreshMap()
00062 {
00063     // TODO: Extract map refresh logic from MainWindow
00064     emit mapUpdated();
00065 }
00066
00070 void MapController::fitToTrack()
00071 {
00072     if (!m_mapRenderer)
00073         return;
00074     m_mapRenderer->fitToTrack();
00075 }
00076
00080 void MapController::fitToVisibleRange()
00081 {
00082     if (!m_mapRenderer)
00083         return;
00084     m_mapRenderer->fitToVisibleRange();
00085 }
00086
00090 void MapController::zoomIn()
00091 {
00092     if (!m_mapRenderer)
00093         return;
00094     m_mapRenderer->zoomIn();
00095 }
00096
00100 void MapController::zoomOut()
00101 {
00102     if (!m_mapRenderer)
00103         return;
00104     m_mapRenderer->zoomOut();
00105 }
00106
00111 void MapController::setCurrentTime(double timeSeconds)

```

```

00112 {
00113     if (!m_mapRenderer)
00114         return;
00115     m_mapRenderer->setCurrentTime(timeSeconds);
00116 }
00117
00123 void MapController::setVisibleTimeRange(double startSeconds, double endSeconds)
00124 {
00125     if (!m_mapRenderer)
00126         return;
00127     m_mapRenderer->setVisibleTimeRange(startSeconds, endSeconds);
00128 }
00129
00135 void MapController::highlightSegment(double startSeconds, double endSeconds)
00136 {
00137     // TODO: Extract segment highlighting from MainWindow
00138 }
00139
00143 void MapController::updateRouteColoring()
00144 {
00145     applyColorMetric();
00146 }
00147
00152 void MapController::setAutoRouteContrast(bool enabled)
00153 {
00154     if (!m_mapRenderer)
00155         return;
00156     m_mapRenderer->setAutoRouteContrast(enabled);
00157 }
00158
00162 void MapController::applyColorMetric()
00163 {
00164     // TODO: Extract color metric application from MainWindow
00165 }
00166
00170 void MapController::connectMapSignals()
00171 {
00172     // TODO: Extract signal connections from MainWindow
00173 }

```

## 8.113 MapController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <memory>
00007
00008 class WorkoutController;
00009 class DatabaseManager;
00010 class MapRenderer;
00011
00023 class MapController : public QObject
00024 {
00025     Q_OBJECT
00026
00027 public:
00034     explicit MapController(
00035         WorkoutController* controller,
00036         DatabaseManager* dbManager,
00037         QObject* parent = nullptr);
00038
00042     ~MapController() override;
00043
00048     void setMapRenderer(MapRenderer* mapRenderer);
00049
00054     MapRenderer* mapRenderer() const;
00055
00059     void updateMap();
00060
00064     void refreshMap();
00065
00069     void fitToTrack();
00070
00074     void fitToVisibleRange();
00075
00079     void zoomIn();
00080
00084     void zoomOut();
00085
00090     void setCurrentTime(double timeSeconds);
00091
00097     void setVisibleTimeRange(double startSeconds, double endSeconds);

```

```

00098
00104     void highlightSegment(double startSeconds, double endSeconds);
00105
00109     void updateRouteColoring();
00110
00115     void setAutoRouteContrast(bool enabled);
00116
00117 signals:
00119     void mapUpdated();
00120
00122     void errorOccurred(const QString& message);
00123
00124 private:
00126     void applyColorMetric();
00127
00129     void connectMapSignals();
00130
00131     WorkoutController* m_controller;
00132     DatabaseManager* m_dbManager;
00133     MapRenderer* m_mapRenderer = nullptr;
00134 };

```

## 8.114 NavigationController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "NavigationController.h"
00004
00005 #include <QSettings>
00006 #include <QStackedWidget>
00007
00012 NavigationController::NavigationController(QObject* parent)
00013     : QObject(parent)
00014 {
00015 }
00016
00020 NavigationController::~NavigationController() = default;
00021
00026 void NavigationController::setPageStack(QStackedWidget* stack)
00027 {
00028     m_pageStack = stack;
00029 }
00030
00038 void NavigationController::setNavigationSidebar(NavigationSidebar* sidebar)
00039 {
00040     m_sidebar = sidebar;
00041     if (!m_sidebar)
00042         return;
00043
00044     connect(m_sidebar, &NavigationSidebar::pageSelected,
00045            this, &NavigationController::navigateTo);
00046 }
00047
00054 void NavigationController::navigateTo(Page page)
00055 {
00056     m_currentPage = page;
00057
00058     if (m_pageStack)
00059         m_pageStack->setCurrentIndex(static_cast<int>(page));
00060
00061     if (m_sidebar)
00062     {
00063         // Block signals to avoid a re-entrant navigateTo() call.
00064         const QSignalBlocker blocker(m_sidebar);
00065         m_sidebar->setCurrentPage(page);
00066     }
00067
00068     emit pageChanged(page);
00069 }
00070
00074 NavigationController::Page NavigationController::currentPage() const
00075 {
00076     return m_currentPage;
00077 }
00078
00082 void NavigationController::saveNavigationState()
00083 {
00084     QSettings settings("Fitlyzer", "FitlyzerC");
00085     settings.setValue("activePage", static_cast<int>(m_currentPage));
00086 }
00087
00095 void NavigationController::restoreNavigationState()
00096 {
00097     QSettings settings("Fitlyzer", "FitlyzerC");

```

```

00098     const int raw = settings.value("activePage", static_cast<int>(Page::Activities)).toInt();
00099
00100     // Valid range: Activities (0) through Video (7).
00101     // Update the upper bound when new pages are appended to the Page enum.
00102     static constexpr int kFirstPage = static_cast<int>(Page::Activities);
00103     static constexpr int kLastPage  = static_cast<int>(Page::Video);
00104
00105     const auto page = (raw >= kFirstPage && raw <= kLastPage)
00106         ? static_cast<Page>(raw)
00107         : Page::Activities;
00108
00109     navigateTo(page);
00110     emit navigationStateRestored();
00111 }

```

## 8.115 NavigationController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <memory>
00007
00008 #include "../NavigationSidebar.h"
00009
00010 class QStackedWidget;
00011
00024 class NavigationController : public QObject
00025 {
00026     Q_OBJECT
00027
00028 public:
00030     using Page = NavigationSidebar::Page;
00031
00032     explicit NavigationController(QObject* parent = nullptr);
00033     ~NavigationController() override;
00034
00039     void setPageStack(QStackedWidget* stack);
00040
00048     void setNavigationSidebar(NavigationSidebar* sidebar);
00049
00054     void navigateTo(Page page);
00055
00057     Page currentPage() const;
00058
00060     void saveNavigationState();
00061
00063     void restoreNavigationState();
00064
00065 signals:
00067     void pageChanged(NavigationSidebar::Page page);
00068
00070     void navigationStateRestored();
00071
00072 private:
00073     QStackedWidget* m_pageStack = nullptr;
00074     NavigationSidebar* m_sidebar = nullptr;
00075
00076     Page m_currentPage = Page::Activities;
00077 };

```

## 8.116 CreateDatabaseDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "CreateDatabaseDialog.h"
00005
00006 #include <QCheckBox>
00007 #include <QDialogButtonBox>
00008 #include <QFileDialog>
00009 #include <QFileInfo>
00010 #include <QFormLayout>
00011 #include <QHBoxLayout>
00012 #include <QLabel>
00013 #include <QLineEdit>
00014 #include <QMessageBox>
00015 #include <QPushButton>
00016 #include <QVBoxLayout>
00017 #include <QDir>

```

```

00018
00019 CreateDatabaseDialog::CreateDatabaseDialog(const QString& defaultDirectory, QWidget* parent)
00020     : QDialog(parent)
00021 {
00022     setWindowTitle("Create Database");
00023     setMinimumWidth(520);
00024
00025     auto* mainLayout = new QVBoxLayout(this);
00026
00027     auto* intro = new QLabel(
00028         "Create a new training database and optionally add a default athlete.", this);
00029     intro->setWordWrap(true);
00030     mainLayout->addWidget(intro);
00031
00032     auto* form = new QFormLayout;
00033     m_nameEdit = new QLineEdit("fitlyzer", this);
00034     m_locationEdit = new QLineEdit(defaultDirectory, this);
00035     m_browseButton = new QPushButton("Browse...", this);
00036
00037     auto* locationRow = new QHBoxLayout;
00038     locationRow->addWidget(m_locationEdit, 1);
00039     locationRow->addWidget(m_browseButton);
00040
00041     form->addRow("Database name:", m_nameEdit);
00042     form->addRow("Location:", locationRow);
00043     mainLayout->addLayout(form);
00044
00045     m_createAthleteCheck = new QCheckBox("Create default athlete", this);
00046     m_createAthleteCheck->setChecked(true);
00047     mainLayout->addWidget(m_createAthleteCheck);
00048
00049     auto* athleteForm = new QFormLayout;
00050     m_firstNameEdit = new QLineEdit(this);
00051     m_lastNameEdit = new QLineEdit(this);
00052     m_firstNameEdit->setPlaceholderText("First name");
00053     m_lastNameEdit->setPlaceholderText("Last name");
00054     athleteForm->addRow("First name:", m_firstNameEdit);
00055     athleteForm->addRow("Last name:", m_lastNameEdit);
00056     mainLayout->addLayout(athleteForm);
00057
00058     m_buttonBox = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00059     mainLayout->addWidget(m_buttonBox);
00060
00061     connect(m_browseButton, &QPushButton::clicked, this, &CreateDatabaseDialog::browseLocation);
00062     connect(m_createAthleteCheck, &QCheckBox::toggled, this,
00063         &CreateDatabaseDialog::updateAthleteFields);
00064     connect(m_buttonBox, &QDialogButtonBox::accepted, this, &CreateDatabaseDialog::accept);
00065     connect(m_buttonBox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00066
00067     updateAthleteFields();
00068 }
00069
00070 QString CreateDatabaseDialog::databasePath() const
00071 {
00072     const QString dirPath = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00073     if (dirPath.isEmpty())
00074         return QString();
00075     return QDir(dirPath).filePath(normalizedDatabaseName());
00076 }
00077
00078 bool CreateDatabaseDialog::shouldCreateDefaultAthlete() const
00079 {
00080     return m_createAthleteCheck && m_createAthleteCheck->isChecked();
00081 }
00082
00083 QString CreateDatabaseDialog::defaultAthleteFirstName() const
00084 {
00085     return m_firstNameEdit ? m_firstNameEdit->text().trimmed() : QString();
00086 }
00087
00088 QString CreateDatabaseDialog::defaultAthleteLastName() const
00089 {
00090     return m_lastNameEdit ? m_lastNameEdit->text().trimmed() : QString();
00091 }
00092
00093 void CreateDatabaseDialog::browseLocation()
00094 {
00095     const QString current = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00096     const QString dir = QFileDialog::getExistingDirectory(this, "Choose Database Location", current);
00097     if (!dir.isEmpty() && m_locationEdit)
00098         m_locationEdit->setText(dir);
00099 }
00100
00101 void CreateDatabaseDialog::updateAthleteFields()
00102 {
00103     const bool enabled = shouldCreateDefaultAthlete();

```

```

00104     if (m_firstNameEdit)
00105         m_firstNameEdit->setEnabled(enabled);
00106     if (m_lastNameEdit)
00107         m_lastNameEdit->setEnabled(enabled);
00108 }
00109
00110 void CreateDatabaseDialog::accept()
00111 {
00112     const QString dirPath = m_locationEdit ? m_locationEdit->text().trimmed() : QString();
00113     if (dirPath.isEmpty())
00114     {
00115         QMessageBox::warning(this, "Missing Location", "Choose a folder for the new database.");
00116         return;
00117     }
00118
00119     const QFileInfo dirInfo(dirPath);
00120     if (!dirInfo.exists() || !dirInfo.isDir())
00121     {
00122         QMessageBox::warning(this, "Invalid Location", "Choose an existing folder for the new
database.");
00123         return;
00124     }
00125
00126     if (normalizedDatabaseName().isEmpty())
00127     {
00128         QMessageBox::warning(this, "Missing Name", "Enter a database name.");
00129         return;
00130     }
00131
00132     if (shouldCreateDefaultAthlete())
00133     {
00134         if (defaultAthleteFirstName().isEmpty() || defaultAthleteLastName().isEmpty())
00135         {
00136             QMessageBox::warning(this, "Missing Athlete Name",
"Enter a first and last name for the default athlete, or disable that option.");
00137             return;
00138         }
00139     }
00140 }
00141
00142 QDialog::accept();
00143 }
00144
00145 QString CreateDatabaseDialog::normalizedDatabaseName() const
00146 {
00147     QString name = m_nameEdit ? m_nameEdit->text().trimmed() : QString();
00148     if (name.endsWith(".fitlyzerdb", Qt::CaseInsensitive))
00149         return name;
00150     if (!name.isEmpty())
00151         name += ".fitlyzerdb";
00152     return name;
00153 }

```

## 8.117 CreateDatabaseDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 class QCheckBox;
00009 class QDialogButtonBox;
00010 class QLineEdit;
00011 class QPushButton;
00012
00018 class CreateDatabaseDialog : public QDialog
00019 {
00020     Q_OBJECT
00021
00022 public:
00023     explicit CreateDatabaseDialog(const QString& defaultDirectory, QWidget* parent = nullptr);
00024
00025     QString databasePath() const;
00026
00027     bool shouldCreateDefaultAthlete() const;
00028
00029     QString defaultAthleteFirstName() const;
00030
00031     QString defaultAthleteLastName() const;
00032
00033 private slots:
00034     void browseLocation();
00035     void updateAthleteFields();

```

```

00036     void accept() override;
00037
00038 private:
00039     QString normalizedDatabaseName() const;
00040
00041     QLineEdit* m_nameEdit = nullptr;
00042     QLineEdit* m_locationEdit = nullptr;
00043     QCheckBox* m_createAthleteCheck = nullptr;
00044     QLineEdit* m_firstNameEdit = nullptr;
00045     QLineEdit* m_lastNameEdit = nullptr;
00046     QPushButton* m_browseButton = nullptr;
00047     QDialogButtonBox* m_buttonBox = nullptr;
00048 };

```

## 8.118 EditActivityDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "EditActivityDialog.h"
00005
00006 #include <QDialogButtonBox>
00007 #include <QDoubleSpinBox>
00008 #include <QFormLayout>
00009 #include <QLineEdit>
00010 #include <QSpinBox>
00011 #include <QTextEdit>
00012 #include <QVBoxLayout>
00013
00014 EditActivityDialog::EditActivityDialog(const Activity& activity, QWidget* parent)
00015     : QDialog(parent)
00016     , m_initial(activity)
00017 {
00018     setWindowTitle("Activity Properties");
00019     setMinimumWidth(460);
00020
00021     auto* layout = new QVBoxLayout(this);
00022     auto* form = new QFormLayout;
00023
00024     m_temperatureSpin = new QDoubleSpinBox(this);
00025     m_temperatureSpin->setRange(-40.0, 60.0);
00026     m_temperatureSpin->setDecimals(1);
00027     m_temperatureSpin->setSuffix(" C");
00028     m_temperatureSpin->setValue(activity.temperature);
00029
00030     m_weatherEdit = new QLineEdit(activity.weather, this);
00031     m_windEdit = new QLineEdit(activity.wind, this);
00032
00033     m_rpeSpin = new QSpinBox(this);
00034     m_rpeSpin->setRange(0, 10);
00035     m_rpeSpin->setValue(activity.rpe);
00036
00037     m_fatigueSpin = new QSpinBox(this);
00038     m_fatigueSpin->setRange(0, 10);
00039     m_fatigueSpin->setValue(activity.fatigue);
00040
00041     m_sleepSpin = new QDoubleSpinBox(this);
00042     m_sleepSpin->setRange(0.0, 24.0);
00043     m_sleepSpin->setDecimals(1);
00044     m_sleepSpin->setSuffix(" h");
00045     m_sleepSpin->setValue(activity.sleepHours);
00046
00047     m_weightSpin = new QDoubleSpinBox(this);
00048     m_weightSpin->setRange(0.0, 200.0);
00049     m_weightSpin->setDecimals(1);
00050     m_weightSpin->setSuffix(" kg");
00051     m_weightSpin->setValue(activity.weightKg);
00052
00053     m_bikeEdit = new QLineEdit(activity.bike, this);
00054     m_equipmentEdit = new QLineEdit(activity.equipment, this);
00055
00056     m_notesEdit = new QTextEdit(this);
00057     m_notesEdit->setMinimumHeight(120);
00058     m_notesEdit->setPlainText(activity.notes);
00059
00060     form->addRow("Temperature:", m_temperatureSpin);
00061     form->addRow("Weather:", m_weatherEdit);
00062     form->addRow("Wind:", m_windEdit);
00063     form->addRow("RPE:", m_rpeSpin);
00064     form->addRow("Fatigue:", m_fatigueSpin);
00065     form->addRow("Sleep:", m_sleepSpin);
00066     form->addRow("Weight:", m_weightSpin);
00067     form->addRow("Bike:", m_bikeEdit);
00068     form->addRow("Equipment:", m_equipmentEdit);

```



```

00069     form->addRow("Notes:", m_notesEdit);
00070
00071     layout->addLayout(form);
00072
00073     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00074     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00075     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00076     layout->addWidget(buttons);
00077 }
00078
00079 Activity EditActivityDialog::updatedActivity() const
00080 {
00081     Activity out = m_initial;
00082     out.temperature = m_temperatureSpin ? m_temperatureSpin->value() : m_initial.temperature;
00083     out.weather = m_weatherEdit ? m_weatherEdit->text().trimmed() : m_initial.weather;
00084     out.wind = m_windEdit ? m_windEdit->text().trimmed() : m_initial.wind;
00085     out.rpe = m_rpeSpin ? m_rpeSpin->value() : m_initial.rpe;
00086     out.fatigue = m_fatigueSpin ? m_fatigueSpin->value() : m_initial.fatigue;
00087     out.sleepHours = m_sleepSpin ? m_sleepSpin->value() : m_initial.sleepHours;
00088     out.weightKg = m_weightSpin ? m_weightSpin->value() : m_initial.weightKg;
00089     out.bike = m_bikeEdit ? m_bikeEdit->text().trimmed() : m_initial.bike;
00090     out.equipment = m_equipmentEdit ? m_equipmentEdit->text().trimmed() : m_initial.equipment;
00091     out.notes = m_notesEdit ? m_notesEdit->toPlainText().trimmed() : m_initial.notes;
00092     return out;
00093 }

```

## 8.119 EditActivityDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "database/ActivityRepository.h"
00009
00010 class QDoubleSpinBox;
00011 class QLineEdit;
00012 class QTextEdit;
00013 class QSpinBox;
00014
00021 class EditActivityDialog : public QDialog
00022 {
00023     Q_OBJECT
00024
00025 public:
00026     explicit EditActivityDialog(const Activity& activity, QWidget* parent = nullptr);
00027
00028     Activity updatedActivity() const;
00029
00030 private:
00031     Activity m_initial;
00032
00033     QDoubleSpinBox* m_temperatureSpin = nullptr;
00034     QLineEdit* m_weatherEdit = nullptr;
00035     QLineEdit* m_windEdit = nullptr;
00036     QSpinBox* m_rpeSpin = nullptr;
00037     QSpinBox* m_fatigueSpin = nullptr;
00038     QDoubleSpinBox* m_sleepSpin = nullptr;
00039     QDoubleSpinBox* m_weightSpin = nullptr;
00040     QLineEdit* m_bikeEdit = nullptr;
00041     QLineEdit* m_equipmentEdit = nullptr;
00042     QTextEdit* m_notesEdit = nullptr;
00043 };

```

## 8.120 EditNotesDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "EditNotesDialog.h"
00005
00006 #include <QDialogButtonBox>
00007 #include <QFormLayout>
00008 #include <QLabel>
00009 #include <QLineEdit>
00010 #include <QTextEdit>
00011 #include <QVBoxLayout>
00012
00013 EditNotesDialog::EditNotesDialog(const QString& notes,

```

```

00014                                     const QString& weatherNotes,
00015                                     QWidget* parent)
00016     : QDialog(parent)
00017 {
00018     setWindowTitle("Edit Activity Notes");
00019     setMinimumWidth(420);
00020
00021     auto* layout = new QVBoxLayout(this);
00022
00023     auto* form = new QFormLayout;
00024     m_notes = new QTextEdit;
00025     m_notes->setPlainText(notes);
00026     m_notes->setMinimumHeight(120);
00027     form->addRow("Notes:", m_notes);
00028
00029     m_weather = new QLineEdit(weatherNotes);
00030     form->addRow("Weather:", m_weather);
00031
00032     layout->addLayout(form);
00033
00034     auto* bbox = new QDialogButtonBox(
00035         QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
00036     layout->addWidget(bbox);
00037
00038     connect(bbox, &QDialogButtonBox::accepted, this, &QDialog::accept);
00039     connect(bbox, &QDialogButtonBox::rejected, this, &QDialog::reject);
00040 }
00041
00042 QString EditNotesDialog::notes() const
00043 {
00044     return m_notes->toPlainText();
00045 }
00046
00047 QString EditNotesDialog::weatherNotes() const
00048 {
00049     return m_weather->text();
00050 }

```

## 8.121 EditNotesDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QString>
00008
00009 class QLineEdit;
00010 class QTextEdit;
00011
00015 class EditNotesDialog : public QDialog
00016 {
00017     Q_OBJECT
00018 public:
00019     explicit EditNotesDialog(const QString& notes,
00020                             const QString& weatherNotes,
00021                             QWidget* parent = nullptr);
00022
00023     QString notes() const;
00024
00025     QString weatherNotes() const;
00026
00027 private:
00028     QTextEdit* m_notes = nullptr;
00029     QLineEdit* m_weather = nullptr;
00030 };

```

## 8.122 ImportStatusWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportStatusWidget.h"
00004
00005 #include <QHBoxLayout>
00006 #include <QLabel>
00007 #include <QProgressBar>
00008
00009 #include "import/ImportProgressModel.h"
00010
00011 ImportStatusWidget::ImportStatusWidget(QWidget* parent)

```

```

00012     : QWidget (parent)
00013 {
00014     auto* row = new QHBoxLayout (this);
00015     row->setContentsMargins (0, 0, 0, 0);
00016     row->setSpacing (6);
00017
00018     m_titleLabel = new QLabel (this);
00019     m_titleLabel->setMinimumWidth (120);
00020
00021     m_progressBar = new QProgressBar (this);
00022     m_progressBar->setRange (0, 100);
00023     m_progressBar->setMinimumWidth (150);
00024     m_progressBar->setTextVisible (true);
00025
00026     m_detailLabel = new QLabel (this);
00027     m_detailLabel->setMinimumWidth (220);
00028
00029     row->addWidget (m_titleLabel);
00030     row->addWidget (m_progressBar);
00031     row->addWidget (m_detailLabel);
00032
00033     hide ();
00034 }
00035
00036 void ImportStatusWidget::setModel (ImportProgressModel* model)
00037 {
00038     if (m_model == model)
00039         return;
00040
00041     if (m_model)
00042         disconnect (m_model, nullptr, this, nullptr);
00043
00044     m_model = model;
00045     if (m_model)
00046         connect (m_model, &ImportProgressModel::changed, this, &ImportStatusWidget::updateFromModel);
00047
00048     updateFromModel ();
00049 }
00050
00051 void ImportStatusWidget::updateFromModel ()
00052 {
00053     if (!m_model || m_model->activeJobs () <= 0)
00054     {
00055         hide ();
00056         return;
00057     }
00058
00059     show ();
00060
00061     const int remaining = m_model->activeJobs ();
00062     const QString plural = remaining == 1 ? QStringLiteral ("file") : QStringLiteral ("files");
00063     m_titleLabel->setText (QString ("Importing %1 %2...").arg (remaining).arg (plural));
00064     m_progressBar->setValue (static_cast<int> (m_model->progress ()));
00065     m_detailLabel->setText (
00066         QString ("Current: %1 - %2")
00067             .arg (m_model->currentFileName ().isEmpty () ? QStringLiteral ("-") :
00068                 m_model->currentFileName ())
00069             .arg (m_model->statusText ().isEmpty () ? QStringLiteral ("Working") :
00070                 m_model->statusText ()) );
00071 }

```

## 8.123 ImportStatusWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QWidget>
00006
00007 class QLabel;
00008 class QProgressBar;
00009 class ImportProgressModel;
00010
00014 class ImportStatusWidget : public QWidget
00015 {
00016     Q_OBJECT
00017 public:
00018     explicit ImportStatusWidget (QWidget* parent = nullptr);
00019
00021     void setModel (ImportProgressModel* model);
00022
00023 private:
00024     void updateFromModel ();
00025

```

```

00026     ImportProgressModel* m_model = nullptr;
00027     QLabel* m_titleLabel = nullptr;
00028     QLabel* m_detailLabel = nullptr;
00029     QProgressBar* m_progressBar = nullptr;
00030 };

```

## 8.124 IntervalEditorDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "IntervalEditorDialog.h"
00005
00006 #include <QDialogButtonBox>
00007 #include <QDoubleSpinBox>
00008 #include <QFormLayout>
00009 #include <QLineEdit>
00010 #include <QTextEdit>
00011 #include <QVBoxLayout>
00012
00013 IntervalEditorDialog::IntervalEditorDialog(double startSeconds,
00014                                           double endSeconds,
00015                                           QWidget* parent)
00016     : QDialog(parent)
00017 {
00018     setWindowTitle("Create Interval");
00019     setMinimumWidth(420);
00020
00021     auto* layout = new QVBoxLayout(this);
00022     auto* form = new QFormLayout;
00023
00024     m_nameEdit = new QLineEdit("Manual Interval", this);
00025     m_typeEdit = new QLineEdit("Manual", this);
00026
00027     m_startSpin = new QDoubleSpinBox(this);
00028     m_startSpin->setRange(0.0, 86400.0);
00029     m_startSpin->setSuffix(" s");
00030     m_startSpin->setValue(startSeconds);
00031
00032     m_endSpin = new QDoubleSpinBox(this);
00033     m_endSpin->setRange(0.0, 86400.0);
00034     m_endSpin->setSuffix(" s");
00035     m_endSpin->setValue(endSeconds);
00036
00037     m_notesEdit = new QTextEdit(this);
00038     m_notesEdit->setMinimumHeight(100);
00039
00040     form->addRow("Name:", m_nameEdit);
00041     form->addRow("Type:", m_typeEdit);
00042     form->addRow("Start:", m_startSpin);
00043     form->addRow("End:", m_endSpin);
00044     form->addRow("Notes:", m_notesEdit);
00045
00046     layout->addLayout(form);
00047
00048     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00049     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00050     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00051     layout->addWidget(buttons);
00052 }
00053
00054 QString IntervalEditorDialog::intervalName() const
00055 {
00056     return m_nameEdit ? m_nameEdit->text().trimmed() : QString();
00057 }
00058
00059 QString IntervalEditorDialog::intervalType() const
00060 {
00061     return m_typeEdit ? m_typeEdit->text().trimmed() : QString();
00062 }
00063
00064 QString IntervalEditorDialog::intervalNotes() const
00065 {
00066     return m_notesEdit ? m_notesEdit->toPlainText().trimmed() : QString();
00067 }

```

## 8.125 IntervalEditorDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003

```

```

00004 #pragma once
00005
00006 #include <QDialog>
00007 #include <QString>
00008
00009 class QDoubleSpinBox;
00010 class QLineEdit;
00011 class QTextEdit;
00012
00018 class IntervalEditorDialog : public QDialog
00019 {
00020     Q_OBJECT
00021
00022 public:
00029     explicit IntervalEditorDialog(double startSeconds,
00030                                   double endSeconds,
00031                                   QWidget* parent = nullptr);
00032
00037     QString intervalName() const;
00038
00043     QString intervalType() const;
00044
00049     QString intervalNotes() const;
00050
00051 private:
00052     QLineEdit* m_nameEdit = nullptr;
00053     QLineEdit* m_typeEdit = nullptr;
00054     QTextEdit* m_notesEdit = nullptr;
00055     QDoubleSpinBox* m_startSpin = nullptr;
00056     QDoubleSpinBox* m_endSpin = nullptr;
00057 };

```

## 8.126 MainWindow.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MainWindow.h"
00005
00006 #include <QCloseEvent>
00007 #include <QFile>
00008 #include <QMessageBox>
00009 #include <QSettings>
00010 #include <QStatusBar>
00011 #include <QTimer>
00012 #include <QFileInfo>
00013 #include "ActivityBrowser.h"
00014 #include "NavigationSidebar.h"
00015 #include "pages/ActivitiesPage.h"
00016 #include "controllers/ActivityViewController.h"
00017 #include "controllers/ChartController.h"
00018 #include "controllers/MapController.h"
00019 #include "controllers/ImportController.h"
00020 #include "controllers/NavigationController.h"
00021 #include "controllers/MainWindowActions.h"
00022 #include "database/AthleteRepository.h"
00023 #include "import/ImportProgressModel.h"
00024
00025 static constexpr int kMaxRecentDatabases = 8;
00026
00027 // Constructor
00028
00029 MainWindow::MainWindow(QWidget* parent)
00030     : QMainWindow(parent)
00031 {
00032     setAcceptDrops(true);
00033
00034     m_controller = new WorkoutController(this);
00035     m_controller->setDatabaseManager(&m_dbManager);
00036
00037     buildUI();
00038
00039     // Instantiate and configure UI controllers.
00040     // m_navigationSidebar and m_pageStack are created inside buildUI().
00041     // NavigationController manages the page stack and sidebar connection.
00042     m_navigationController = new NavigationController(this);
00043     m_navigationController->setPageStack(m_pageStack);
00044     m_navigationController->setNavigationSidebar(m_navigationSidebar);
00045
00046     // Wire ActivitiesPage context toolbar signals.
00047     if (m_activitiesPage)
00048     {
00049         connect(m_activitiesPage, &ActivitiesPage::importRequested,
00050                 this, &MainWindow::importActivities);
00051     }

```

```

00051         connect(m_activitiesPage, &ActivitiesPage::detectClimbsRequested,
00052                 this, &MainWindow::triggerDetectClimbs);
00053         connect(m_activitiesPage, &ActivitiesPage::detectIntervalsRequested,
00054                 this, &MainWindow::triggerDetectIntervals);
00055         connect(m_activitiesPage, &ActivitiesPage::editActivityRequested,
00056                 this, &MainWindow::editCurrentActivityProperties);
00057         // Recalculate metrics: run analysis queue on selected activity.
00058         connect(m_activitiesPage, &ActivitiesPage::recalculateMetricsRequested,
00059                 this, [this]
00060         {
00061             if (m_controller && m_controller->currentActivityId() > 0 && m_analysisQueue)
00062                 m_analysisQueue->enqueue(m_controller->currentActivityId());
00063         });
00064     }
00065
00066     m_activityViewController = new ActivityViewController(m_controller, &m_dbManager, this);
00067     m_activityViewController->setAthleteHeaderWidget(m_athleteHeader);
00068     m_activityViewController->setStatusBarLabels(
00069         m_dbStatusLabel, m_athleteStatusLabel, m_activityCountLabel);
00070     m_activityViewController->setAthleteContext(m_currentAthleteId, m_currentAthleteName);
00071
00072     m_chartController = new ChartController(m_controller, &m_dbManager, this);
00073     m_chartController->setChartWidgets(
00074         m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart,
00075         m_histogram, m_pdcWidget, m_fitnessChart);
00076     m_chartController->setAnalysisTabWidgets(
00077         m_analysisTabWidget,
00078         m_activityTabStack, m_zonesTabStack, m_histogramTabStack,
00079         m_powerCurveTabStack, m_calendarTabStack, m_fitnessTabStack);
00080     m_chartController->setZonesTable(m_zonesTable);
00081     m_chartController->setClimbCharts(
00082         m_climbPowerChart, m_climbHrChart, m_climbCadenceChart,
00083         m_climbSpeedChart, m_climbAltitudeChart);
00084     m_chartController->setChartControls(
00085         m_colorMetricCombo, m_powerSmoothingCombo,
00086         m_autoSmoothingCheck, m_climbOverlayEnabledCheck,
00087         m_climbOverlayMetricCombo, m_colorLegendLayout);
00088     m_chartController->setAthleteId(m_currentAthleteId);
00089
00090     m_mapController = new MapController(m_controller, &m_dbManager, this);
00091     m_mapController->setMapRenderer(m_mapRenderer);
00092
00093     m_importController = new ImportController(m_controller, &m_dbManager, this);
00094     m_importController->setImportQueue(nullptr); // Set after queue is created below
00095     m_importController->setProgressModel(nullptr); // Set after model is created below
00096     m_importController->setStatusWidget(m_importStatusWidget);
00097
00098     m_actionsManager = new MainWindowActions(this, this);
00099
00100     connect(m_controller, &WorkoutController::workoutLoaded,
00101            this, &MainWindow::onWorkoutLoaded);
00102     connect(m_controller, &WorkoutController::activityImported,
00103            this, &MainWindow::onActivityImported);
00104
00105     m_undoManager = new UndoManager(50, this);
00106     m_analysisQueue = new AnalysisQueue(this);
00107     m_controller->setAnalysisQueue(m_analysisQueue);
00108     m_importQueue = new ImportQueue(this);
00109     m_importProgressModel = new ImportProgressModel(this);
00110     m_importProgressModel->attachQueue(m_importQueue);
00111
00112     // Wire import queue and model to import controller
00113     if (m_importController)
00114     {
00115         m_importController->setImportQueue(m_importQueue);
00116         m_importController->setProgressModel(m_importProgressModel);
00117     }
00118
00119     m_importRefreshTimer = new QTimer(this);
00120     m_importRefreshTimer->setSingleShot(true);
00121     m_importRefreshTimer->setInterval(500);
00122     connect(m_importRefreshTimer, &QTimer::timeout, this, [this]()
00123     {
00124         if (m_activityBrowser)
00125             m_activityBrowser->refresh(m_currentAthleteId);
00126         updateStatusLabel();
00127         updateStatusBarInfo();
00128         updateWelcomeScreenVisibility();
00129     });
00130
00131     // Status bar feedback from the analysis queue
00132     connect(m_analysisQueue, &AnalysisQueue::pendingCountChanged, this, [this](int count)
00133     {
00134         if (count > 0)
00135             statusBar()->showMessage(QString("Analyzing %1 activit%2 in background...")
00136                                     .arg(count).arg(count == 1 ? "y" : "ies"), 0);
00137         else

```

```

00138         statusBar()->clearMessage();
00139     });
00140     connect(m_analysisQueue, &AnalysisQueue::taskCompleted, this, [this](int activityId)
00141     {
00142         // If the completed task is the currently loaded activity, refresh climbing/intervals
00143         if (m_controller && m_controller->currentActivityId() == activityId)
00144         {
00145             m_controller->reloadClimbsFromDatabase();
00146             m_detectedClimbs = m_controller->climbs();
00147             applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00148             refreshClimbViews();
00149             updateIntervals();
00150         }
00151     });
00152
00153     connect(m_importQueue, &ImportQueue::jobFinished, this,
00154            [this](const QString&, const QString& batchId, const QString&, int activityId, int)
00155     {
00156         if (activityId > 0 && m_analysisQueue)
00157             m_analysisQueue->enqueue(activityId);
00158
00159         auto it = m_importBatchSummaries.find(batchId);
00160         if (it != m_importBatchSummaries.end())
00161             ++it->imported;
00162
00163         scheduleActivityBrowserRefresh();
00164         hideWelcomeScreen();
00165     });
00166
00167     connect(m_importQueue, &ImportQueue::jobFailed, this,
00168            [this](const QString&, const QString& batchId, const QString& filePath,
00169                const QString& reason, const QString& errorMessage, int)
00170     {
00171         auto it = m_importBatchSummaries.find(batchId);
00172         if (it != m_importBatchSummaries.end())
00173         {
00174             if (reason == QStringLiteral("duplicate") || reason == QStringLiteral("time_overlap"))
00175                 ++it->duplicates;
00176             else
00177                 ++it->failed;
00178
00179             if (!errorMessage.isEmpty())
00180                 it->errors << QString("%1: %2").arg(QFileInfo(filePath).fileName(), errorMessage);
00181         }
00182     });
00183
00184     connect(m_importQueue, &ImportQueue::queueIdle, this, [this]()
00185     {
00186         scheduleActivityBrowserRefresh();
00187         finalizeImportBatches();
00188     });
00189
00190     if (m_activityBrowser)
00191     {
00192         connect(m_activityBrowser, &ActivityBrowser::fitFilesDropped,
00193                this, &MainWindow::importFiles);
00194     }
00195
00196     setupShortcuts();
00197     loadSettings();
00198     applyChartHeight();
00199
00200     QSettings s("Fitlyzer", "FitlyzerC");
00201     if (!s.contains("firstLaunchCompleted"))
00202         s.setValue("firstLaunchCompleted", false);
00203     m_firstLaunchCompleted = s.value("firstLaunchCompleted", false).toBool();
00204
00205     // Re-open last database and athlete
00206     const QString lastDb = s.value("lastDatabase").toString();
00207     if (!lastDb.isEmpty())
00208     {
00209         QString err;
00210         if (!m_dbManager.open(lastDb, &err))
00211         {
00212             QMessageBox::warning(this, "Database",
00213                QString("Could not open last database:\n%1\n%2")
00214                    .arg(lastDb).arg(err));
00215         }
00216         else
00217         {
00218             m_controller->setDatabaseManager(&m_dbManager);
00219             if (m_analysisQueue)
00220                 m_analysisQueue->setDatabasePath(lastDb);
00221             if (m_importQueue)
00222                 m_importQueue->setDatabasePath(lastDb);
00223             const int aid = s.value("currentAthleteId", -1).toInt();
00224             if (aid > 0)

```

```

00225         {
00226             m_currentAthleteId = aid;
00227             m_controller->setCurrentAthlete(aid);
00228             auto db = m_dbManager.database();
00229             AthleteRepository repo(db);
00230             m_currentAthleteName = repo.getAthlete(aid).fullName();
00231         }
00232         updateAthleteLabel();
00233         refreshAthleteSelector();
00234         if (m_activityBrowser)
00235             m_activityBrowser->refresh(m_currentAthleteId);
00236
00237         const int lastActivityId = s.value("lastActivityId", -1).toInt();
00238         if (lastActivityId > 0)
00239         {
00240             QString loadErr;
00241             if (!m_controller->loadActivity(lastActivityId, loadErr))
00242                 s.remove("lastActivityId");
00243         }
00244     }
00245 }
00246
00247 updateRecentDatabaseMenu();
00248 refreshAthleteSelector();
00249 updateImportAvailability();
00250 updateStatsLabel();
00251 updateStatusBarInfo();
00252 updateWelcomeScreenVisibility();
00253 configureFolderWatcher();
00254 }
00255
00256 // Close event
00257
00258 void MainWindow::closeEvent(QCloseEvent* event)
00259 {
00260     saveSettings();
00261     QMainWindow::closeEvent(event);
00262 }
00263
00264 // UI Construction
00265
00266 // buildUI moved to MainWindowUiBuild.cpp
00267
00268 // Settings
00269
00270 // settings and database shell methods moved to MainWindowDatabase.cpp
00271
00272 // Slots
00273
00274 // athlete/settings shell methods moved to MainWindowAthlete.cpp
00275
00276 // Update helpers
00277

```

## 8.127 MainWindow.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QCheckBox>
00007 #include <QComboBox>
00008 #include <QDoubleSpinBox>
00009 #include <QDragEnterEvent>
00010 #include <QDropEvent>
00011 #include <QLabel>
00012 #include <QMainWindow>
00013 #include <QMenu>
00014 #include <QSplitter>
00015 #include <QStackedLayout>
00016 #include <QStackedWidget>
00017 #include <QPushButton>
00018 #include <QScrollArea>
00019 #include <QSet>
00020 #include <QHash>
00021 #include <QStringList>
00022 #include <QTabWidget>
00023 #include <QTableWidget>
00024 #include <QTextEdit>
00025 #include <QVBoxLayout>
00026
00027 #include "WorkoutController.h"
00028 #include "core/zones/ZoneDefinition.h"

```



```

00029 #include "analysis/queue/AnalysisQueue.h"
00030 #include "import/ImportQueue.h"
00031 #include "core/undo/UndoManager.h"
00032 #include "database/DatabaseManager.h"
00033 #include "database/ActivityRepository.h"
00034 #include "database/AthleteRepository.h"
00035 #include "database/ClimbRepository.h"
00036 #include "database/ImportRepository.h"
00037
00038 class RideChartWidget;
00039 class FitnessChartWidget;
00040 class PowerHistogramWidget;
00041 class PowerCurveWidget;
00042 class MapRenderer;
00043 class ActivityBrowser;
00044 class AthleteHeaderWidget;
00045 class CalendarWidget;
00046 class WelcomeWidget;
00047 class ImportStatusWidget;
00048 class ImportProgressModel;
00049 class QCloseEvent;
00050 class QFileSystemWatcher;
00051 class QMimeData;
00052 class QPoint;
00053 class QTimer;
00054 class QChartView;
00055 class NavigationSidebar;
00056
00063 class MainWindow : public QMainWindow
00064 {
00065     Q_OBJECT
00066
00067 public:
00072     explicit MainWindow(QWidget* parent = nullptr);
00073
00074 protected:
00076     void closeEvent(QCloseEvent* event) override;
00077
00079     bool eventFilter(QObject* watched, QEvent* event) override;
00080
00082     void dragEnterEvent(QDragEnterEvent* event) override;
00083
00085     void dropEvent(QDropEvent* event) override;
00086
00087 private slots:
00088     void importActivities();
00089     void previewFitFile();
00090     void onWorkoutLoaded();
00091     void onActivityImported(int activityId);
00092     void openDatabase();
00093     void createDatabase();
00094     void manageAthletes();
00095     void backupDatabase();
00096     void openSettingsDialog();
00097     void openRecentDatabase();
00098     void onAthleteSelectionChanged(int index);
00099     void createVideo();
00100     void showAbout();
00101     void triggerDetectClimbs();
00102     void triggerDetectIntervals();
00103     void revertSelectedClimbToAuto();
00104
00105 private:
00106     void buildUI();
00107     void buildToolBar();
00108     void buildStatusBar();
00109     void setupShortcuts();
00110     void saveSettings();
00111     void loadSettings();
00112     void addRecentDatabase(const QString& path);
00113     void updateRecentDatabaseMenu();
00114     void openDatabasePath(const QString& path);
00115
00119     bool canImportActivities() const;
00120
00122     void updateImportAvailability();
00123
00125     void importFiles(const QStringList& filePaths);
00126     void enqueueImportBatch(const QStringList& filePaths,
00127                             bool showResultDialog,
00128                             const QString& sourceLabel);
00129     QStringList prepareImportFileBatch(const QStringList& filePaths);
00130     QString importQueuedStatusMessage(int queuedCount) const;
00131
00133     QStringList fitFilesFromMimeData(const QMimeData* mimeData) const;
00134     void scheduleActivityBrowserRefresh();
00135

```

```

00137     void finalizeImportBatches();
00138     void refreshAthleteSelector();
00139     void resetAllZoom();
00140     void increaseChartHeight();
00141     void decreaseChartHeight();
00142     void applyChartHeight();
00143     void updateStatsLabel();
00144     void updateIntervals();
00145     void updateClimbingTab();
00146     void detectClimbsAndRefresh();
00147     void refreshClimbViews(double preferredStartSeconds = -1.0, double preferredEndSeconds = -1.0);
00148     void applyDetectedClimbsToCharts();
00149     void restoreClimbSelection(int previousRow, double preferredStartSeconds, double
preferredEndSeconds);
00150     void updateClimbQuarterCharts(const Climb* climb);
00151     double resolveActiveActivityWeightKg() const;
00152     void assignClimbWeightMetrics(Climb& climb, double riderWeightKg) const;
00153     void applyWeightMetricsToClimbs(std::vector<Climb>& climbs, double riderWeightKg) const;
00154     std::vector<int> selectedClimbIndicesFromTable() const;
00155     void onClimbTableContextMenuRequested(const QPoint& pos);
00156     void removeSelectedClimbs();
00157     void editSelectedClimbBoundaries();
00158     void joinSelectedClimbs();
00159     void updateClimbRowStyles();
00160     void onClimbSelectionChanged();
00161     void onClimbRowDoubleClicked(QTableWidgetItem* item);
00162     void onClimbBoundaryEdited(double oldStartSeconds, double oldEndSeconds, double newStartSeconds,
double newEndSeconds);
00163     void onNewClimbRequested(double startSeconds, double endSeconds);
00164     void onIntervalSelectionChanged();
00165     void onIntervalRowDoubleClicked(QTableWidgetItem* item);
00166     void navigateToInterval(double startSeconds, double endSeconds, bool exactZoom);
00167     void selectIntervalRowOffset(int delta);
00168     void updateIntervalRowStyles();
00169     void finalizeIntervalSelectionAndSummary(int previousRow);
00170     void updateIntervalSummary(double startSeconds, double endSeconds);
00171     void clearIntervalSummary();
00172     void updateLapsSummaryFromCurrentActivity();
00173     void updateActivityNotesFromCurrentActivity();
00174     bool ensureDatabase();
00175     bool ensureAthlete();
00176
00177     bool ensureImportReady();
00178     bool createOrOpenDefaultDatabase();
00179     QString defaultDatabasePathForAutoCreate() const;
00180     void showWelcomeScreen();
00181     void hideWelcomeScreen();
00182     void updateWelcomeScreenVisibility();
00183     int activityCount() const;
00184     void updateAthleteLabel();
00185     void updateStatusBarInfo();
00186     ColorMetric currentColorMetric() const;
00187     ColorContext buildColorContext() const;
00188     void applyChartPreset(int presetId);
00189     void editCurrentActivityProperties();
00190     double estimatedFtpFromCurrentRide() const;
00191     void applyEstimatedFtpForCurrentAthlete();
00192
00193     void importFilesInternal(const QStringList& filePaths,
                                bool showResultDialog,
                                const QString& sourceLabel);
00194
00195     void configureFolderWatcher();
00196     QStringList monitoredDirectories() const;
00197     void scanWatchDirectories(bool initialScan);
00198     void onWatchDirectoryChanged(const QString& path);
00199
00200     void syncAthleteContextToControllers();
00201
00202     void syncChartContextToController(bool includeLegendLayout);
00203
00204     void applyChartControlDrivenUpdates(bool includeLegendLayout, bool applySmoothing = false);
00205
00206     void applyChartAndMapUpdates(bool includeLegendLayout, bool applySmoothing);
00207
00208     void updateChartAnalysisEmptyStates();
00209
00210     void refreshMapRideDataFromCurrentState();
00211
00212     // -- Undo/Redo -----
00213     UndoManager* m_undoManager = nullptr;
00214
00215     // -- Background analysis queue -----
00216     AnalysisQueue* m_analysisQueue = nullptr;
00217
00218     // -- Background import queue -----
00219     ImportQueue* m_importQueue = nullptr;
00220     ImportProgressModel* m_importProgressModel = nullptr;

```

```

00233
00234 // -- Controller -----
00235 WorkoutController* m_controller = nullptr;
00236
00237 // -- UI Controllers (decomposed from MainWindow) -----
00238 class ActivityViewController* m_activityViewController = nullptr;
00239 class ChartController* m_chartController = nullptr;
00240 class MapController* m_mapController = nullptr;
00241 class ImportController* m_importController = nullptr;
00242 class NavigationController* m_navigationController = nullptr;
00243 NavigationSidebar* m_navigationSidebar = nullptr;
00244 class MainWindowActions* m_actionsManager = nullptr;
00245
00246 // -- Database -----
00247 DatabaseManager m_dbManager;
00248 int m_currentAthleteId = -1;
00249 QString m_currentAthleteName;
00250
00251 // -- Menu -----
00252 QMenu* m_athletesMenu = nullptr;
00253 QAction* m_athleteLabelAct = nullptr;
00254 QAction* m_importAct = nullptr;
00255 QAction* m_toolbarImportAct = nullptr;
00256 QAction* m_toolbarSearchAct = nullptr;
00257 QAction* m_previewAct = nullptr;
00258 QMenu* m_recentDbMenu = nullptr;
00259 QString m_lastOpenDir;
00260
00261 // -- Welcome / onboarding -----
00262 WelcomeWidget* m_welcomeWidget = nullptr;
00263 bool m_firstLaunchCompleted = false;
00264
00265 // -- Folder monitoring -----
00266 QFileSystemWatcher* m_watcher = nullptr;
00267 QTimer* m_watchRescanTimer = nullptr;
00268 QTimer* m_importRefreshTimer = nullptr;
00269 bool m_watchFolderEnabled = true;
00270 QString m_watchFolderPath;
00271 QSet<QString> m_knownWatchedFitFiles;
00272
00273 // -- Toolbar/Header -----
00274 QComboBox* m_athleteCombo = nullptr;
00275 QComboBox* m_colorMetricCombo = nullptr;
00276 QComboBox* m_chartPresetCombo = nullptr;
00277 QComboBox* m_powerSmoothingCombo = nullptr;
00278 QCheckBox* m_autoSmoothingCheck = nullptr;
00279 ImportStatusWidget* m_importStatusWidget = nullptr;
00280
00281 // -- Status bar -----
00282 QLabel* m_dbStatusLabel = nullptr;
00283 QLabel* m_athleteStatusLabel = nullptr;
00284 QLabel* m_activityCountLabel = nullptr;
00285
00286 // -- Stats panel -----
00287 AthleteHeaderWidget* m_athleteHeader = nullptr;
00288
00289 // -- Page navigation -----
00290 // m_pageStack holds one widget per NavigationSidebar::Page value.
00291 // The sidebar and NavigationController keep them in sync.
00292 QStackedWidget* m_pageStack = nullptr;
00293
00294 // Intermediate content widgets assembled in buildUI() and handed off to
00295 // the page widgets that are added to m_pageStack.
00296 QWidget* m_chartsPageContent = nullptr;
00297 QWidget* m_zonesPageContent = nullptr;
00298 QWidget* m_histogramPageContent = nullptr;
00299 QWidget* m_pdcPageContent = nullptr;
00300 QWidget* m_calendarPageContent = nullptr;
00301 QWidget* m_fitnessPageContent = nullptr;
00302 QWidget* m_climbsPageContent = nullptr;
00303 QWidget* m_intervalsPageContent = nullptr;
00304
00305 // -- Power sub-tabs (Zones/Histogram/Power Curve) -----
00306 QTabWidget* m_analysisTabWidget = nullptr;
00307 QStackedLayout* m_activityTabStack = nullptr;
00308 QStackedLayout* m_zonesTabStack = nullptr;
00309 QStackedLayout* m_histogramTabStack = nullptr;
00310 QStackedLayout* m_powerCurveTabStack = nullptr;
00311 QStackedLayout* m_calendarTabStack = nullptr;
00312 QStackedLayout* m_fitnessTabStack = nullptr;
00313 QStackedLayout* m_climbingTabStack = nullptr;
00314
00315 // Charts tab
00316 QScrollArea* m_chartScroll = nullptr;
00317 QWidget* m_chartsContainer = nullptr;
00318 QVBoxLayout* m_chartsLayout = nullptr;
00319 QSplitter* m_chartMapSplit = nullptr;

```

```

00320     int                m_chartHeight      = 220;
00321     RideChartWidget* m_powerChart        = nullptr;
00322     RideChartWidget* m_hrChart           = nullptr;
00323     RideChartWidget* m_cadenceChart      = nullptr;
00324     RideChartWidget* m_speedChart        = nullptr;
00325     RideChartWidget* m_altitudeChart     = nullptr;
00326
00327     // Climbing tab charts
00328     RideChartWidget* m_climbAltitudeChart = nullptr;
00329     RideChartWidget* m_climbPowerChart    = nullptr;
00330     RideChartWidget* m_climbHrChart       = nullptr;
00331     RideChartWidget* m_climbCadenceChart  = nullptr;
00332     RideChartWidget* m_climbSpeedChart    = nullptr;
00333     QChartView* m_climbQuarterPowerChart = nullptr;
00334     QChartView* m_climbQuarterHrChart    = nullptr;
00335     QChartView* m_climbQuarterCadenceChart = nullptr;
00336
00337     // Chart controls
00338     QPushButton* m_fitChartsButton        = nullptr;
00339     QPushButton* m_chartHeightIncreaseButton = nullptr;
00340     QPushButton* m_chartHeightDecreaseButton = nullptr;
00341     QPushButton* m_useEstimatedFtpButton   = nullptr;
00342     QWidget* m_colorLegendWidget          = nullptr;
00343     QHBoxLayout* m_colorLegendLayout       = nullptr;
00344
00345     // Metric visibility checkboxes
00346     QCheckBox* m_showPower      = nullptr;
00347     QCheckBox* m_showHR        = nullptr;
00348     QCheckBox* m_showCadence   = nullptr;
00349     QCheckBox* m_showSpeed     = nullptr;
00350     QCheckBox* m_showAltitude  = nullptr;
00351
00352     // Zones tab
00353     QTableWidgetItem* m_zonesTable = nullptr;
00354
00355     // Histogram tab
00356     PowerHistogramWidget* m_histogram = nullptr;
00357
00358     // Power curve tab
00359     PowerCurveWidget* m_pdcWidget = nullptr;
00360
00361     // Fitness tab
00362     FitnessChartWidget* m_fitnessChart = nullptr;
00363
00364     // Integrated activity panels
00365     QTableWidgetItem* m_intervalsTable = nullptr;
00366     QTableWidgetItem* m_lapsTable = nullptr;
00367     QTextEdit* m_activityNotesView = nullptr;
00368     QCheckBox* m_followIntervalSelectionCheck = nullptr;
00369     QLabel* m_intervalSummaryLabel = nullptr;
00370
00371     // Climbing tab
00372     QTableWidgetItem* m_climbsTable = nullptr;
00373     QLabel* m_climbSummaryLabel = nullptr;
00374     QDoubleSpinBox* m_climbMinLengthSpin = nullptr;
00375     QDoubleSpinBox* m_climbMinGainSpin = nullptr;
00376     QDoubleSpinBox* m_climbMinGradientSpin = nullptr;
00377     QDoubleSpinBox* m_climbStartGradientSpin = nullptr;
00378     QDoubleSpinBox* m_climbDipMetersSpin = nullptr;
00379     QDoubleSpinBox* m_climbDipDistanceSpin = nullptr;
00380     QDoubleSpinBox* m_climbSmoothingSpin = nullptr;
00381     QCheckBox* m_climbOverlayEnabledCheck = nullptr;
00382     QComboBox* m_climbOverlayMetricCombo = nullptr;
00383     std::vector<Climb> m_detectedClimbs;
00384     bool m_suppressClimbAutoZoom = false;
00385
00386     // Integrated map panel
00387     MapRenderer* m_mapRenderer = nullptr;
00388     QComboBox* m_mapStyleCombo = nullptr;
00389     QCheckBox* m_mapAutoContrastCheck = nullptr;
00390
00391     // Calendar tab
00392     CalendarWidget* m_calendarWidget = nullptr;
00393
00394     // Activities tab
00395     ActivityBrowser* m_activityBrowser = nullptr;
00396
00397     // -- Page references (for connecting signals / enabling actions) -----
00398     class ActivitiesPage* m_activitiesPage = nullptr;
00399
00400     struct ImportBatchSummary
00401     {
00402         int queued = 0;
00403         int imported = 0;
00404         int duplicates = 0;
00405         int failed = 0;
00406         bool showResultDialog = false;

```

```

00407         QString sourceLabel;
00408         QStringList errors;
00409     };
00410
00411     QHash<QString, ImportBatchSummary> m_importBatchSummaries;
00412 };

```

## 8.128 MainWindowAthlete.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QAction>
00006 #include <QComboBox>
00007 #include <QDialog>
00008 #include <QDialogButtonBox>
00009 #include <QDoubleSpinBox>
00010 #include <QFile>
00011 #include <QFileDialog>
00012 #include <QFormLayout>
00013 #include <QGroupBox>
00014 #include <QLabel>
00015 #include <QMessageBox>
00016 #include <QPushButton>
00017 #include <QScrollArea>
00018 #include <QSettings>
00019 #include <QSqlQuery>
00020 #include <QTabWidget>
00021 #include <QVBoxLayout>
00022
00023 #include "ActivityBrowser.h"
00024 #include "AthleteListDialog.h"
00025 #include "CalendarWidget.h"
00026 #include "core/settings/AppSettings.h"
00027 #include "maps/MapRenderer.h"
00028 #include "platform/Platform.h"
00029
00030 // Settings helpers
00031
00032 namespace ClimbDefaults
00033 {
00034     {
00035         static constexpr double kMinLength      = 0.15;
00036         static constexpr double kMinGain         = 10.0;
00037         static constexpr double kMinGradient     = 4.0;
00038         static constexpr double kStartGrad       = 1.5;
00039         static constexpr double kDipMeters       = 10.0;
00040         static constexpr double kDipDistance     = 200.0;
00041         static constexpr double kSmoothing       = 50.0;
00042     }
00043
00044 void MainWindow::manageAthletes()
00045 {
00046     if (!m_dbManager.isOpen())
00047     {
00048         QMessageBox::information(this, "No Database",
00049             "Please open or create a database first.");
00050         return;
00051     }
00052
00053     auto db = m_dbManager.database();
00054     AthleteRepository repo(db);
00055     AthleteListDialog dlg(repo, this);
00056     if (dlg.exec() == QDialog::Accepted)
00057     {
00058         m_currentAthleteId = dlg.selectedAthleteId();
00059         m_controller->setCurrentAthlete(m_currentAthleteId);
00060         m_currentAthleteName = repo.getAthlete(m_currentAthleteId).fullName();
00061         QSettings("Fitlyzer", "FitlyzerC")
00062             .setValue("currentAthleteId", m_currentAthleteId);
00063         updateAthleteLabel();
00064         refreshAthleteSelector();
00065         if (m_activityBrowser)
00066             m_activityBrowser->refresh(m_currentAthleteId);
00067         updateImportAvailability();
00068         updateStatusBarInfo();
00069         updateWelcomeScreenVisibility();
00070     }
00071 }
00072
00073 void MainWindow::openSettingsDialog()
00074 {
00075     // Dialog frame
00076     QDialog dialog(this);

```

```

00077     dialog.setWindowTitle("Settings");
00078     dialog.setModal(true);
00079     dialog.resize(540, 480);
00080
00081     auto* dlgLayout = new QVBoxLayout(&dialog);
00082     dlgLayout->setContentsMargins(12, 12, 12, 12);
00083     dlgLayout->setSpacing(8);
00084
00085     auto* tabs = new QTabWidget(&dialog);
00086
00087     // Tab: General
00088     {
00089         auto* page = new QWidget(tabs);
00090         auto* vl = new QVBoxLayout(page);
00091         vl->setContentsMargins(12, 12, 12, 12);
00092
00093         auto* generalGroup = new QGroupBox("General", page);
00094         auto* form = new QFormLayout(generalGroup);
00095
00096         auto* dateFormatCombo = new QComboBox(generalGroup);
00097         dateFormatCombo->addItem("DD-MM-YYYY (14-06-2026)", static_cast<int>(DateFormat::DD_MM_YYYY));
00098         dateFormatCombo->addItem("YYYY-MM-DD (2026-06-14)", static_cast<int>(DateFormat::YYYY_MM_DD));
00099         dateFormatCombo->addItem("DD.MM.YYYY (14.06.2026)",
static_cast<int>(DateFormat::DD_DOT_MM_DOT_YYYY));
00100         dateFormatCombo->addItem("MM/DD/YYYY (06/14/2026)", static_cast<int>(DateFormat::MM_DD_YYYY));
00101         const DateFormat currentFormat = AppSettings::instance().dateFormat();
00102         dateFormatCombo->setCurrentIndex(
            dateFormatCombo->findData(static_cast<int>(currentFormat)));
00103         form->addRow("Date Format:", dateFormatCombo);
00104
00105         vl->addWidget(generalGroup);
00106         vl->addStretch(1);
00107         tabs->addTab(page, "General");
00108
00109         // Store pointer for accept handler
00110         dialog.setProperty("dateFormatCombo",
QVariant::fromValue(static_cast<QObject*>(dateFormatCombo)));
00111     }
00112
00113     // Tab: Athletes
00114     {
00115         auto* page = new QWidget(tabs);
00116         auto* vl = new QVBoxLayout(page);
00117         vl->setContentsMargins(12, 12, 12, 12);
00118
00119         auto* lbl = new QLabel(
            "Athlete profile defaults and team settings are managed in this section.\n"
            "Use the main Athletes manager for profile CRUD.",
            page);
00120         lbl->setWordWrap(true);
00121         lbl->setStyleSheet("color: #64748b;");
00122         vl->addWidget(lbl);
00123         vl->addStretch(1);
00124
00125         tabs->addTab(page, "Athletes");
00126     }
00127
00128     // Tab: Import
00129     {
00130         auto* page = new QWidget(tabs);
00131         auto* vl = new QVBoxLayout(page);
00132         vl->setContentsMargins(12, 12, 12, 12);
00133
00134         auto* lbl = new QLabel(
            "Import defaults (auto-tagging, duplicate handling, file watchers)\n"
            "will be consolidated here.",
            page);
00135         lbl->setWordWrap(true);
00136         lbl->setStyleSheet("color: #64748b;");
00137         vl->addWidget(lbl);
00138         vl->addStretch(1);
00139
00140         tabs->addTab(page, "Import");
00141     }
00142
00143     // Tab: Maps
00144     {
00145         auto* page = new QWidget(tabs);
00146         auto* vl = new QVBoxLayout(page);
00147         vl->setContentsMargins(12, 12, 12, 12);
00148
00149         auto* mapsGroup = new QGroupBox("Maps", page);
00150         auto* mapsForm = new QFormLayout(mapsGroup);
00151
00152         auto* tileCacheCombo = new QComboBox(mapsGroup);
00153         tileCacheCombo->addItem("128 tiles (~32 MB)", 128);
00154         tileCacheCombo->addItem("256 tiles (~64 MB)", 256);
00155     }

```

```

00162         tileCacheCombo->addItem("512 tiles (~128 MB)", 512);
00163         tileCacheCombo->addItem("1024 tiles (~256 MB)", 1024);
00164         tileCacheCombo->addItem("2048 tiles (~512 MB)", 2048);
00165         const int currentCacheSize = AppSettings::instance().tileCacheSize();
00166         tileCacheCombo->setCurrentIndex(
00167             tileCacheCombo->findData(currentCacheSize) >= 0
00168             ? tileCacheCombo->findData(currentCacheSize) : 2);
00169         mapsForm->addRow("Tile Memory Cache:", tileCacheCombo);
00170
00171         vl->addWidget(mapsGroup);
00172         vl->addStretch(1);
00173         tabs->addTab(page, "Maps");
00174
00175         dialog.setProperty("tileCacheCombo",
00176             QVariant::fromValue(static_cast<QObject*>(tileCacheCombo)));
00177     }
00178     // Tab: Climb Detection
00179     QDoubleSpinBox *csMinLen, *csMinGain, *csMinGrad, *csStGrad,
00180         *csDipM, *csDipD, *csSmooth;
00181     {
00182         auto* page = new QWidget(tabs);
00183         auto* vl = new QVBoxLayout(page);
00184         vl->setContentsMargins(12, 12, 12, 12);
00185
00186         auto* climbGroup = new QGroupBox("Climb Detection Parameters", page);
00187         auto* climbForm = new QFormLayout(climbGroup);
00188
00189         QSettings cs("Fitlyzer", "FitlyzerC");
00190         auto makeSpin = [&cs, climbGroup](
00191             const QString& key, double defaultVal, double min, double max, int decimals,
00192             const QString& suffix) -> QDoubleSpinBox*
00193         {
00194             auto* spin = new QDoubleSpinBox(climbGroup);
00195             spin->setRange(min, max);
00196             spin->setDecimals(decimals);
00197             spin->setValue(cs.value(key, defaultVal).toDouble());
00198             if (!suffix.isEmpty()) spin->setSuffix(suffix);
00199             return spin;
00200         };
00201
00202         csMinLen = makeSpin("climb/minLength", ClimbDefaults::kMinLength, 0.1, 20.0, 2, "
00203 km");
00204         csMinGain = makeSpin("climb/minGain", ClimbDefaults::kMinGain, 1.0, 2000.0, 0, " m");
00205         csMinGrad = makeSpin("climb/minGradient", ClimbDefaults::kMinGradient, 0.5, 20.0, 1, " %");
00206         csStGrad = makeSpin("climb/startGrad", ClimbDefaults::kStartGrad, 0.5, 20.0, 1, " %");
00207         csDipM = makeSpin("climb/dipMeters", ClimbDefaults::kDipMeters, 0.0, 200.0, 1, " m");
00208         csDipD = makeSpin("climb/dipDistance", ClimbDefaults::kDipDistance, 10.0, 2000.0, 0, " m");
00209         csSmooth = makeSpin("climb/smoothing", ClimbDefaults::kSmoothing, 1.0, 500.0, 0, "");
00210
00211         climbForm->addRow("Minimum Length:", csMinLen);
00212         climbForm->addRow("Minimum Gain:", csMinGain);
00213         climbForm->addRow("Avg Gradient:", csMinGrad);
00214         climbForm->addRow("Start Gradient:", csStGrad);
00215         climbForm->addRow("Dip Elevation:", csDipM);
00216         climbForm->addRow("Dip Distance:", csDipD);
00217         climbForm->addRow("Smoothing:", csSmooth);
00218
00219         auto* restoreBtn = new QPushButton("Restore Defaults", climbGroup);
00220         climbForm->addRow("", restoreBtn);
00221         connect(restoreBtn, &QPushButton::clicked, this, [=]() {
00222             csMinLen->setValue(ClimbDefaults::kMinLength);
00223             csMinGain->setValue(ClimbDefaults::kMinGain);
00224             csMinGrad->setValue(ClimbDefaults::kMinGradient);
00225             csStGrad->setValue(ClimbDefaults::kStartGrad);
00226             csDipM->setValue(ClimbDefaults::kDipMeters);
00227             csDipD->setValue(ClimbDefaults::kDipDistance);
00228             csSmooth->setValue(ClimbDefaults::kSmoothing);
00229         });
00230         vl->addWidget(climbGroup);
00231         vl->addStretch(1);
00232         tabs->addTab(page, "Climb Detection");
00233     }
00234     // Tab: Analysis (stub)
00235     {
00236         auto* page = new QWidget(tabs);
00237         auto* vl = new QVBoxLayout(page);
00238         vl->setContentsMargins(12, 12, 12, 12);
00239         auto* lbl = new QLabel("Additional analysis preferences -- coming soon.", page);
00240         lbl->setStyleSheet("color: #64748b;");
00241         lbl->setWordWrap(true);
00242         vl->addWidget(lbl);
00243         vl->addStretch(1);
00244         tabs->addTab(page, "Analysis");
00245     }
00246

```

```

00247
00248 // Tab: Charts (stub)
00249 {
00250     auto* page = new QWidget(tabs);
00251     auto* vl = new QVBoxLayout(page);
00252     vl->setContentsMargins(12, 12, 12, 12);
00253     auto* lbl = new QLabel("Chart defaults and presets configuration -- coming soon.", page);
00254     lbl->setStyleSheet("color: #64748b;");
00255     lbl->setWordWrap(true);
00256     vl->addWidget(lbl);
00257     vl->addStretch(1);
00258     tabs->addTab(page, "Charts");
00259 }
00260
00261 // Tab: Video (stub)
00262 {
00263     auto* page = new QWidget(tabs);
00264     auto* vl = new QVBoxLayout(page);
00265     vl->setContentsMargins(12, 12, 12, 12);
00266     auto* lbl = new QLabel("Video export defaults -- coming soon.", page);
00267     lbl->setStyleSheet("color: #64748b;");
00268     lbl->setWordWrap(true);
00269     vl->addWidget(lbl);
00270     vl->addStretch(1);
00271     tabs->addTab(page, "Video");
00272 }
00273
00274 // Tab: Advanced (stub)
00275 {
00276     auto* page = new QWidget(tabs);
00277     auto* vl = new QVBoxLayout(page);
00278     vl->setContentsMargins(12, 12, 12, 12);
00279     auto* lbl = new QLabel("Advanced diagnostics and storage options -- coming soon.", page);
00280     lbl->setStyleSheet("color: #64748b;");
00281     lbl->setWordWrap(true);
00282     vl->addWidget(lbl);
00283     vl->addStretch(1);
00284     tabs->addTab(page, "Advanced");
00285 }
00286
00287 dlgLayout->addWidget(tabs, 1);
00288
00289 auto* buttons = new QDialogButtonBox(
00290     QDialogButtonBox::Ok | QDialogButtonBox::Cancel, &dialog);
00291 dlgLayout->addWidget(buttons);
00292
00293 connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
00294 connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
00295
00296 if (dialog.exec() != QDialog::Accepted)
00297     return;
00298
00299 // Retrieve current values before we possibly changed them via combo boxes.
00300 auto* dfCombo = qobject_cast<QComboBox*>(
00301     dialog.property("dateFormatCombo").value<QObject*>());
00302 auto* tcCombo = qobject_cast<QComboBox*>(
00303     dialog.property("tileCacheCombo").value<QObject*>());
00304
00305 // General: date format
00306 const DateFormat previousFormat = AppSettings::instance().dateFormat();
00307 if (dfCombo)
00308 {
00309     const DateFormat selectedFormat =
00310         static_cast<DateFormat>(dfCombo->currentData().toInt());
00311     if (selectedFormat != previousFormat)
00312     {
00313         AppSettings::instance().setDateFormat(selectedFormat);
00314         if (m_activityBrowser)
00315             m_activityBrowser->refresh(m_currentAthleteId);
00316         updateStatsLabel();
00317     }
00318 }
00319
00320 // Maps: tile cache size
00321 if (tcCombo)
00322 {
00323     const int selectedCacheSize = tcCombo->currentData().toInt();
00324     const int prevCacheSize = AppSettings::instance().tileCacheSize();
00325     if (selectedCacheSize != prevCacheSize)
00326     {
00327         AppSettings::instance().setTileCacheSize(selectedCacheSize);
00328         if (m_mapRenderer)
00329             m_mapRenderer->setTileCacheSize(selectedCacheSize);
00330     }
00331 }
00332
00333 // Climb detection: persist and sync live spinboxes.

```



```

00334     QSettings cs("Fitlyzer", "FitlyzerC");
00335     cs.setValue("climb/minLength", csMinLen->value());
00336     cs.setValue("climb/minGain", csMinGain->value());
00337     cs.setValue("climb/minGradient", csMinGrad->value());
00338     cs.setValue("climb/startGrad", csStGrad->value());
00339     cs.setValue("climb/dipMeters", csDipM->value());
00340     cs.setValue("climb/dipDistance", csDipD->value());
00341     cs.setValue("climb/smoothing", csSmooth->value());
00342
00343     auto applyIfChanged = [](QDoubleSpinBox* spin, double newVal)
00344     {
00345         if (spin && !qFuzzyCompare(spin->value(), newVal))
00346             spin->setValue(newVal);
00347     };
00348     applyIfChanged(m_climbMinLengthSpin, csMinLen->value());
00349     applyIfChanged(m_climbMinGainSpin, csMinGain->value());
00350     applyIfChanged(m_climbMinGradientSpin, csMinGrad->value());
00351     applyIfChanged(m_climbStartGradientSpin, csStGrad->value());
00352     applyIfChanged(m_climbDipMetersSpin, csDipM->value());
00353     applyIfChanged(m_climbDipDistanceSpin, csDipD->value());
00354     applyIfChanged(m_climbSmoothingSpin, csSmooth->value());
00355 }
00356
00357 void MainWindow::backupDatabase()
00358 {
00359     if (!m_dbManager.isOpen())
00360     {
00361         QMessageBox::information(this, "No Database",
00362             "No database is open.");
00363         return;
00364     }
00365     const QString startPath = Platform::defaultBackupPath(m_dbManager.path());
00366     const QString dest = QFileDialog::getSaveFileName(
00367         this, "Backup Database", startPath,
00368         "Fitlyzer Database (*.fitlyzerdb)");
00369     if (dest.isEmpty()) return;
00370
00371     // Checkpoint the WAL before copying so the backup file is self-contained.
00372     {
00373         auto db = m_dbManager.database();
00374         QSqlQuery q(db);
00375         q.exec("PRAGMA wal_checkpoint(FULL)");
00376     }
00377
00378     if (QFile::exists(dest) && !QFile::remove(dest))
00379     {
00380         QMessageBox::critical(this, "Backup Failed",
00381             "Cannot overwrite existing file:\n" + dest);
00382         return;
00383     }
00384
00385     if (QFile::copy(m_dbManager.path(), dest))
00386         QMessageBox::information(this, "Backup", "Backup saved to:\n" + dest);
00387     else
00388         QMessageBox::critical(this, "Backup Failed",
00389             "Could not copy database to:\n" + dest);
00390 }
00391
00392 void MainWindow::updateAthleteLabel()
00393 {
00394     if (m_athleteLabelAct)
00395         m_athleteLabelAct->setText(
00396             m_currentAthleteName.isEmpty()
00397             ? "(no athlete selected)"
00398             : "Current: " + m_currentAthleteName);
00399 }
00400
00401 void MainWindow::openRecentDatabase()
00402 {
00403     auto* action = qobject_cast<QAction*>(sender());
00404     if (!action) return;
00405     const QString path = action->data().toString();
00406     if (path.isEmpty()) return;
00407     openDatabasePath(path);
00408 }
00409
00410 void MainWindow::onAthleteSelectionChanged(int index)
00411 {
00412     if (!m_athleteCombo || index < 0)
00413         return;
00414
00415     const int athleteId = m_athleteCombo->currentData().toInt();
00416     if (athleteId <= 0)
00417     {
00418         m_currentAthleteId = -1;
00419         m_currentAthleteName.clear();
00420     }

```

```

00421     else
00422     {
00423         m_currentAthleteId = athleteId;
00424         m_currentAthleteName = m_athleteCombo->currentText();
00425         m_controller->setCurrentAthlete(athleteId);
00426     }
00427
00428     QSettings("Fitlyzer", "FitlyzerC").setValue("currentAthleteId", m_currentAthleteId);
00429     updateAthleteLabel();
00430     if (m_activityBrowser)
00431         m_activityBrowser->refresh(m_currentAthleteId);
00432     if (m_calendarWidget)
00433         m_calendarWidget->setAthleteId(m_currentAthleteId);
00434     updateChartAnalysisEmptyStates();
00435     updateImportAvailability();
00436     updateStatsLabel();
00437     updateStatusBarInfo();
00438     updateWelcomeScreenVisibility();
00439 }

```

## 8.129 MainWindowChartActions.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QDate>
00006 #include <QMessageBox>
00007 #include <QSignalBlocker>
00008 #include <QStatusBar>
00009
00010 #include "analysis/PowerCurve.h"
00011 #include "core/settings/DateFormatter.h"
00012 #include "database/AthleteRepository.h"
00013 #include "charts/RideChartWidget.h"
00014 #include "controllers/ChartController.h"
00015
00016 #include <algorithm>
00017 #include <cmath>
00018 #include <initializer_list>
00019
00020 namespace
00021 {
00022     static constexpr int kChartPresetCustom = 0;
00023     static constexpr int kChartPresetPowerAnalysis = 1;
00024     static constexpr int kChartPresetHeartRateFocus = 2;
00025     static constexpr int kChartPresetRaceReview = 3;
00026 }
00027
00028 void MainWindow::resetAllZoom()
00029 {
00030     const std::initializer_list<RideChartWidget*> all =
00031         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00032     for (auto* c : all)
00033         c->fitToData();
00034 }
00035
00036 void MainWindow::increaseChartHeight()
00037 {
00038     m_chartHeight += 40;
00039     applyChartHeight();
00040 }
00041
00042 void MainWindow::decreaseChartHeight()
00043 {
00044     m_chartHeight = std::max(120, m_chartHeight - 40);
00045     applyChartHeight();
00046 }
00047
00048 void MainWindow::applyChartHeight()
00049 {
00050     const std::initializer_list<RideChartWidget*> all =
00051         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00052
00053     int visibleCount = 0;
00054     for (auto* c : all)
00055     {
00056         c->setFixedHeight(m_chartHeight);
00057         if (c->isVisible())
00058             ++visibleCount;
00059     }
00060
00061     if (!m_chartsContainer)
00062         return;

```

```

00063
00064     const int spacing = m_chartsLayout ? m_chartsLayout->spacing() : 4;
00065     const int totalH = visibleCount * m_chartHeight
00066         + std::max(0, visibleCount - 1) * spacing;
00067     m_chartsContainer->setFixedHeight(std::max(totalH, 1));
00068
00069     if (m_chartsContainer)
00070         m_chartsContainer->setMaximumWidth(QWIDGETSIZE_MAX);
00071 }
00072
00073 void MainWindow::onWorkoutLoaded()
00074 {
00075     updateStatsLabel();
00076     updateChartAnalysisEmptyStates();
00077
00078     const bool hasPower = m_controller->statistics().maximumPower > 0.0;
00079     if (m_chartController)
00080     {
00081         syncChartContextToController(true);
00082         m_chartController->handleWorkoutLoaded();
00083     }
00084
00085     if (hasPower)
00086         updateIntervals();
00087
00088     updateClimbingTab();
00089     refreshMapRideDataFromCurrentState();
00090 }
00091
00092 void MainWindow::applyChartPreset(int presetId)
00093 {
00094     if (!m_colorMetricCombo || !m_powerSmoothingCombo || !m_autoSmoothingCheck)
00095         return;
00096
00097     if (presetId == kChartPresetCustom)
00098         return;
00099
00100     QSignalBlocker colorBlock(m_colorMetricCombo);
00101     QSignalBlocker smoothingBlock(m_powerSmoothingCombo);
00102     QSignalBlocker autoBlock(m_autoSmoothingCheck);
00103
00104     switch (presetId)
00105     {
00106     case kChartPresetPowerAnalysis:
00107         m_colorMetricCombo->setCurrentIndex(
00108             m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
00109         m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(5));
00110         m_autoSmoothingCheck->setChecked(false);
00111         m_showPower->setChecked(true);
00112         m_showHR->setChecked(true);
00113         m_showCadence->setChecked(true);
00114         m_showSpeed->setChecked(false);
00115         m_showAltitude->setChecked(false);
00116         break;
00117
00118     case kChartPresetHeartRateFocus:
00119         m_colorMetricCombo->setCurrentIndex(
00120             m_colorMetricCombo->findData(static_cast<int>(ColorMetric::HeartRate)));
00121         m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(10));
00122         m_autoSmoothingCheck->setChecked(true);
00123         m_showPower->setChecked(true);
00124         m_showHR->setChecked(true);
00125         m_showCadence->setChecked(false);
00126         m_showSpeed->setChecked(false);
00127         m_showAltitude->setChecked(false);
00128         break;
00129
00130     case kChartPresetRaceReview:
00131         m_colorMetricCombo->setCurrentIndex(
00132             m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
00133         m_powerSmoothingCombo->setCurrentIndex(m_powerSmoothingCombo->findData(30));
00134         m_autoSmoothingCheck->setChecked(true);
00135         m_showPower->setChecked(true);
00136         m_showHR->setChecked(true);
00137         m_showCadence->setChecked(true);
00138         m_showSpeed->setChecked(true);
00139         m_showAltitude->setChecked(true);
00140         break;
00141
00142     default:
00143         return;
00144     }
00145
00146     m_powerSmoothingCombo->setEnabled(!m_autoSmoothingCheck->isChecked());
00147     applyChartAndMapUpdates(true, true);
00148 }
00149

```

```

00150 double MainWindow::estimatedFtpFromCurrentRide() const
00151 {
00152     const double best20m = PowerCurve::bestMeanPower(m_controller->rideData(), 20.0 * 60.0);
00153     if (best20m <= 0.0)
00154         return 0.0;
00155     return best20m * 0.95;
00156 }
00157
00158 void MainWindow::applyEstimatedFtpForCurrentAthlete()
00159 {
00160     if (!m_dbManager.isOpen() || m_currentAthleteId <= 0)
00161         return;
00162
00163     const int estimated = static_cast<int>(std::round(estimatedFtpFromCurrentRide()));
00164     if (estimated <= 0)
00165     {
00166         QMessageBox::information(this, "Estimated FTP", "Insufficient power data to estimate FTP.");
00167         return;
00168     }
00169
00170     auto db = m_dbManager.database();
00171     AthleteRepository repo(db);
00172     Athlete athlete = repo.getAthlete(m_currentAthleteId);
00173     if (!athlete.isValid())
00174         return;
00175
00176     athlete.ftpWatts = estimated;
00177     if (!repo.updateAthlete(athlete))
00178     {
00179         QMessageBox::critical(this, "FTP Update", "Failed to update athlete FTP.");
00180         return;
00181     }
00182
00183     FtpEntry entry;
00184     entry.athleteId = m_currentAthleteId;
00185     entry.ftpWatts = estimated;
00186     entry.effectiveFrom = DateFormatter::toIsoDate(QDate::currentDate());
00187     entry.notes = "Estimated from best 20-minute power (95%).";
00188     repo.addFtpEntry(entry);
00189
00190     m_controller->setCurrentAthlete(m_currentAthleteId);
00191     updateStatsLabel();
00192     statusBar()->showMessage(QString("Estimated FTP applied: %1 W").arg(estimated), 3500);
00193 }

```

## 8.130 MainWindowCharts.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include "controllers/ChartController.h"
00006 #include "maps/MapRenderer.h"
00007
00008 void MainWindow::syncChartContextToController(bool includeLegendLayout)
00009 {
00010     if (!m_chartController)
00011         return;
00012
00013     const std::vector<Climb>& climbsForView =
00014         m_climbMinLengthSpin ? m_detectedClimbs : m_controller->climbs();
00015
00016     m_chartController->setClimbs(climbsForView);
00017     m_chartController->setColorMetric(static_cast<int>(currentColorMetric()));
00018     m_chartController->setColorContext(buildColorContext());
00019     m_chartController->setAthleteId(m_currentAthleteId);
00020     m_chartController->setZonesTable(m_zonesTable);
00021     m_chartController->setChartControls(
00022         m_colorMetricCombo, m_powerSmoothingCombo,
00023         m_autoSmoothingCheck, m_climbOverlayEnabledCheck,
00024         m_climbOverlayMetricCombo,
00025         includeLegendLayout ? m_colorLegendLayout : nullptr);
00026     m_chartController->setAnalysisTabWidgets(
00027         m_analysisTabWidget,
00028         m_activityTabStack,
00029         m_zonesTabStack,
00030         m_histogramTabStack,
00031         m_powerCurveTabStack,
00032         m_calendarTabStack,
00033         m_fitnessTabStack);
00034     m_chartController->setClimbingTabStack(m_climbingTabStack);
00035 }
00036
00037 void MainWindow::applyChartControlDrivenUpdates(bool includeLegendLayout, bool applySmoothing)

```

```

00038 {
00039     if (!m_chartController)
00040         return;
00041
00042     syncChartContextToController(includeLegendLayout);
00043     if (applySmoothing)
00044         m_chartController->applyChartSmoothing();
00045     m_chartController->updateColorLegend();
00046     m_chartController->updateZoneAvailability();
00047     m_chartController->updateCharts();
00048     m_chartController->updateZonesTab();
00049 }
00050
00051 void MainWindow::applyChartAndMapUpdates(bool includeLegendLayout, bool applySmoothing)
00052 {
00053     applyChartControlDrivenUpdates(includeLegendLayout, applySmoothing);
00054     refreshMapRideDataFromCurrentState();
00055 }
00056
00057 void MainWindow::updateChartAnalysisEmptyStates()
00058 {
00059     if (!m_chartController)
00060         return;
00061
00062     syncChartContextToController(false);
00063     m_chartController->updateAnalysisEmptyStates();
00064 }
00065
00066 void MainWindow::refreshMapRideDataFromCurrentState()
00067 {
00068     if (!m_mapRenderer)
00069         return;
00070
00071     m_mapRenderer->setRideData(m_controller->rideData(), currentColorMetric(), buildColorContext());
00072 }

```

## 8.131 MainWindowClimbInteractions.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QAction>
00006 #include <QFont>
00007 #include <QMenu>
00008 #include <QMessageBox>
00009 #include <QModelIndex>
00010 #include <QStatusBar>
00011 #include <QTableWidgetItem>
00012
00013 #include "analysis/ClimbDetector.h"
00014 #include "analysis/ClimbMetrics.h"
00015 #include "charts/RideChartWidget.h"
00016 #include "core/undo/ClimbEditCommand.h"
00017 #include "database/ClimbRepository.h"
00018 #include "fit/RideData.h"
00019 #include "maps/MapRenderer.h"
00020
00021 #include <algorithm>
00022 #include <cmath>
00023 #include <initializer_list>
00024 #include <limits>
00025 #include <memory>
00026 #include <vector>
00027
00028 namespace
00029 {
00030     static QString fmtDur(double seconds)
00031     {
00032         const int total = static_cast<int>(seconds);
00033         const int h = total / 3600;
00034         const int m = (total % 3600) / 60;
00035         const int s = total % 60;
00036         return h > 0
00037             ? QString("%1:%2:%3")
00038               .arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00039             : QString("%1:%2")
00040               .arg(m).arg(s, 2, 10, QChar('0'));
00041     }
00042
00043     static constexpr int kClimbStartRole = Qt::UserRole + 3;
00044     static constexpr int kClimbEndRole = Qt::UserRole + 4;
00045
00046     class ClimbNameTableWidgetItem final : public QTableWidgetItem

```

```

00047 {
00048 public:
00049     explicit ClimbNameTableItem(const QString& text)
00050         : QTableWidgetItem(text)
00051     {}
00052
00053     bool operator<(const QTableWidgetItem& other) const override
00054     {
00055         bool leftOk = false;
00056         bool rightOk = false;
00057         const int leftNumber = trailingNumber(text(), &leftOk);
00058         const int rightNumber = trailingNumber(other.text(), &rightOk);
00059
00060         if (leftOk && rightOk)
00061             return leftNumber < rightNumber;
00062
00063         return QString::localeAwareCompare(text(), other.text()) < 0;
00064     }
00065
00066 private:
00067     static int trailingNumber(const QString& text, bool* ok)
00068     {
00069         if (ok)
00070             *ok = false;
00071
00072         int i = text.size() - 1;
00073         while (i >= 0 && text[i].isSpace())
00074             --i;
00075
00076         if (i < 0 || !text[i].isDigit())
00077             return 0;
00078
00079         int end = i;
00080         while (i >= 0 && text[i].isDigit())
00081             --i;
00082
00083         const QString number = text.mid(i + 1, end - i);
00084         bool converted = false;
00085         const int value = number.toInt(&converted);
00086         if (ok)
00087             *ok = converted;
00088         return value;
00089     }
00090 };
00091
00092 class ClimbSortKeyTableItem final : public QTableWidgetItem
00093 {
00094 public:
00095     explicit ClimbSortKeyTableItem(const QString& text, double sortKey)
00096         : QTableWidgetItem(text)
00097         , m_sortKey(sortKey)
00098     {}
00099
00100     bool operator<(const QTableWidgetItem& other) const override
00101     {
00102         const auto* rhs = dynamic_cast<const ClimbSortKeyTableItem*>(&other);
00103         if (!rhs)
00104             return QString::localeAwareCompare(text(), other.text()) < 0;
00105
00106         const bool leftNaN = std::isnan(m_sortKey);
00107         const bool rightNaN = std::isnan(rhs->m_sortKey);
00108         if (leftNaN != rightNaN)
00109             return !leftNaN;
00110         if (leftNaN && rightNaN)
00111             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00112
00113         if (m_sortKey == rhs->m_sortKey)
00114             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00115         return m_sortKey < rhs->m_sortKey;
00116     }
00117
00118 private:
00119     double m_sortKey = std::numeric_limits<double>::quiet_NaN();
00120 };
00121
00122
00123 std::vector<int> MainWindow::selectedClimbIndicesFromTable() const
00124 {
00125     std::vector<int> indices;
00126     if (!m_climbsTable || !m_climbsTable->selectionModel())
00127         return indices;
00128
00129     const QModelIndexList rows = m_climbsTable->selectionModel()->selectedRows(0);
00130     for (const QModelIndex& modelIndex : rows)
00131     {
00132         auto* item = m_climbsTable->item(modelIndex.row(), 0);
00133         if (!item)

```

```

00134         continue;
00135
00136         const double selectedStart = item->data(kClimbStartRole).toDouble();
00137         const double selectedEnd = item->data(kClimbEndRole).toDouble();
00138
00139         for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00140         {
00141             const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
00142             if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
00143                 std::abs(c.endSeconds - selectedEnd) < 0.75)
00144             {
00145                 indices.push_back(i);
00146                 break;
00147             }
00148         }
00149     }
00150
00151     std::sort(indices.begin(), indices.end());
00152     indices.erase(std::unique(indices.begin(), indices.end()), indices.end());
00153     return indices;
00154 }
00155
00156 void MainWindow::onClimbTableContextMenuRequested(const QPoint& pos)
00157 {
00158     if (!m_climbsTable)
00159         return;
00160
00161     const int clickedRow = m_climbsTable->rowAt(pos.y());
00162     if (clickedRow >= 0 && !m_climbsTable->item(clickedRow, 0)->isSelected())
00163     {
00164         m_climbsTable->clearSelection();
00165         m_climbsTable->setCurrentCell(clickedRow, 0);
00166         m_climbsTable->selectRow(clickedRow);
00167     }
00168
00169     const std::vector<int> selected = selectedClimbIndicesFromTable();
00170     if (selected.empty())
00171         return;
00172
00173     QMenu menu(m_climbsTable);
00174     QAction* editAct = menu.addAction("Edit");
00175     QAction* removeAct = menu.addAction("Remove");
00176     QAction* joinAct = menu.addAction("Join");
00177
00178     editAct->setEnabled(selected.size() == 1);
00179     joinAct->setEnabled(selected.size() >= 2);
00180     removeAct->setEnabled(!selected.empty());
00181
00182     QAction* chosen = menu.exec(m_climbsTable->viewport()->mapToGlobal(pos));
00183     if (!chosen)
00184         return;
00185
00186     if (chosen == editAct)
00187         editSelectedClimbBoundaries();
00188     else if (chosen == removeAct)
00189         removeSelectedClimbs();
00190     else if (chosen == joinAct)
00191         joinSelectedClimbs();
00192 }
00193
00194 void MainWindow::removeSelectedClimbs()
00195 {
00196     std::vector<int> selected = selectedClimbIndicesFromTable();
00197     if (selected.empty())
00198         return;
00199
00200     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
00201     {
00202         auto db = m_dbManager.database();
00203         ClimbRepository repo(db);
00204         for (int idx : selected)
00205         {
00206             if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
00207             {
00208                 const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
00209                 if (climbId > 0)
00210                     repo.softDeleteClimb(climbId);
00211             }
00212         }
00213     }
00214
00215     std::sort(selected.rbegin(), selected.rend());
00216     for (int idx : selected)
00217     {
00218         if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
00219             m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
00220     }

```

```

00221
00222     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00223         m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
00224
00225     refreshClimbViews();
00226 }
00227
00228 void MainWindow::editSelectedClimbBoundaries()
00229 {
00230     std::vector<int> selected = selectedClimbIndicesFromTable();
00231     if (selected.size() != 1)
00232         return;
00233
00234     const int idx = selected.front();
00235     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
00236         return;
00237
00238     const Climb& climb = m_detectedClimbs[static_cast<size_t>(idx)];
00239
00240     if (m_climbAltitudeChart)
00241     {
00242         m_climbAltitudeChart->setClimbEditingEnabled(true);
00243         m_climbAltitudeChart->setSelectedClimbIndex(idx);
00244
00245         const double padding = std::max(5.0, climb.durationSeconds * 0.08);
00246         const double rangeStart = std::max(0.0, climb.startSeconds - padding);
00247         const double rangeEnd = climb.endSeconds + padding;
00248         m_climbAltitudeChart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
00249         m_climbAltitudeChart->setCrosshair(climb.startSeconds / 60.0);
00250         m_climbAltitudeChart->setFocus();
00251     }
00252
00253     statusBar()->showMessage(
00254         "Graphical edit enabled: drag the climb start/end handles in the elevation chart.",
00255         5000);
00256 }
00257
00258 void MainWindow::joinSelectedClimbs()
00259 {
00260     std::vector<int> selected = selectedClimbIndicesFromTable();
00261     if (selected.size() < 2)
00262         return;
00263
00264     std::sort(selected.begin(), selected.end());
00265     const int keepIdx = selected.front();
00266     if (keepIdx < 0 || keepIdx >= static_cast<int>(m_detectedClimbs.size()))
00267         return;
00268
00269     double newStart = m_detectedClimbs[static_cast<size_t>(keepIdx)].startSeconds;
00270     double newEnd = m_detectedClimbs[static_cast<size_t>(keepIdx)].endSeconds;
00271     for (int idx : selected)
00272     {
00273         if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
00274             continue;
00275         const Climb& c = m_detectedClimbs[static_cast<size_t>(idx)];
00276         newStart = std::min(newStart, c.startSeconds);
00277         newEnd = std::max(newEnd, c.endSeconds);
00278     }
00279
00280     const Climb keep = m_detectedClimbs[static_cast<size_t>(keepIdx)];
00281     onClimbBoundaryEdited(keep.startSeconds, keep.endSeconds, newStart, newEnd);
00282
00283     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
00284     {
00285         auto db = m_dbManager.database();
00286         ClimbRepository repo(db);
00287         std::sort(selected.rbegin(), selected.rend());
00288         for (int idx : selected)
00289         {
00290             if (idx == keepIdx)
00291                 continue;
00292             if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
00293             {
00294                 const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
00295                 if (climbId > 0)
00296                     repo.softDeleteClimb(climbId);
00297                 m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
00298             }
00299         }
00300     }
00301     else
00302     {
00303         std::sort(selected.rbegin(), selected.rend());
00304         for (int idx : selected)
00305         {
00306             if (idx == keepIdx)
00307                 continue;

```



```

00308         if (idx >= 0 && idx < static_cast<int>(m_detectedClimbs.size()))
00309             m_detectedClimbs.erase(m_detectedClimbs.begin() + idx);
00310     }
00311 }
00312
00313 for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00314     m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
00315
00316 refreshClimbViews(newStart, newEnd);
00317 }
00318
00319 void MainWindow::updateClimbRowStyles()
00320 {
00321     if (!m_climbsTable)
00322         return;
00323
00324     const int selectedRow = m_climbsTable->currentRow();
00325     for (int row = 0; row < m_climbsTable->rowCount(); ++row)
00326     {
00327         for (int col = 0; col < m_climbsTable->columnCount(); ++col)
00328         {
00329             auto* item = m_climbsTable->item(row, col);
00330             if (!item)
00331                 continue;
00332
00333             QFont font = item->font();
00334             font.setBold(row == selectedRow);
00335             item->setFont(font);
00336         }
00337     }
00338 }
00339
00340 void MainWindow::onClimbSelectionChanged()
00341 {
00342     if (!m_climbsTable)
00343         return;
00344
00345     const int row = m_climbsTable->currentRow();
00346     if (row < 0)
00347     {
00348         if (m_climbAltitudeChart)
00349             m_climbAltitudeChart->setSelectedClimbIndex(-1);
00350         updateClimbQuarterCharts(nullptr);
00351         updateClimbRowStyles();
00352         if (m_climbSummaryLabel)
00353             m_climbSummaryLabel->setText("Climb summary: select a climb");
00354         return;
00355     }
00356
00357     auto* firstItem = m_climbsTable->item(row, 0);
00358     if (!firstItem)
00359         return;
00360
00361     const double selectedStart = firstItem->data(kClimbStartRole).toDouble();
00362     const double selectedEnd = firstItem->data(kClimbEndRole).toDouble();
00363
00364     int climbIndex = -1;
00365     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00366     {
00367         const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
00368         if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
00369             std::abs(c.endSeconds - selectedEnd) < 0.75)
00370         {
00371             climbIndex = i;
00372             break;
00373         }
00374     }
00375
00376     if (climbIndex < 0)
00377     {
00378         if (m_climbAltitudeChart)
00379             m_climbAltitudeChart->setSelectedClimbIndex(-1);
00380         updateClimbQuarterCharts(nullptr);
00381         return;
00382     }
00383
00384     if (m_climbAltitudeChart)
00385         m_climbAltitudeChart->setSelectedClimbIndex(climbIndex);
00386
00387     const Climb& climb = m_detectedClimbs[static_cast<size_t>(climbIndex)];
00388     updateClimbQuarterCharts(&climb);
00389     const double padding = std::max(5.0, climb.durationSeconds * 0.08);
00390     const double startSeconds = std::max(0.0, climb.startSeconds - padding);
00391     const double endSeconds = climb.endSeconds + padding;
00392
00393     (void)startSeconds;
00394     (void)endSeconds;

```

```

00395
00396     if (m_climbSummaryLabel)
00397     {
00398         const QString wkgText = climb.averageWattsPerKg > 0.0
00399             ? QString::number(climb.averageWattsPerKg, 'f', 2)
00400             : QStringLiteral("\xE2\x80\x94");
00401         m_climbSummaryLabel->setText(
00402             QString("%1 | %2 | Gain %3 m | Avg %4% | W/kg %5 (%6) | VAM %7 m/h | Fade %8% | Decouple
%9% | %10 | %11")
00403                 .arg(climb.name)
00404                 .arg(fmtDur(climb.durationSeconds))
00405                 .arg(climb.elevationGainM, 0, 'f', 0)
00406                 .arg(climb.averageGradient, 0, 'f', 1)
00407                 .arg(wkgText)
00408                 .arg(climb.wattsPerKgRank.isEmpty() ? QStringLiteral("\xE2\x80\x94") :
climb.wattsPerKgRank)
00409                 .arg(climb.vam, 0, 'f', 0)
00410                 .arg(climb.powerFadePct, 0, 'f', 1)
00411                 .arg(climb.aerobicDecouplingPct, 0, 'f', 1)
00412                 .arg(climb.category)
00413                 .arg(climb.shapeClass));
00414     }
00415     updateClimbRowStyles();
00416 }
00417
00418 void MainWindow::onClimbRowDoubleClicked(QTableWidgetItem* item)
00419 {
00420     if (!item || !m_climbsTable)
00421         return;
00422
00423     const int row = item->row();
00424     auto* firstItem = m_climbsTable->item(row, 0);
00425     if (!firstItem)
00426         return;
00427
00428     const double selectedStart = firstItem->data(kClimbStartRole).toDouble();
00429     const double selectedEnd = firstItem->data(kClimbEndRole).toDouble();
00430
00431     int climbIndex = -1;
00432     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00433     {
00434         const Climbe c = m_detectedClimbs[static_cast<size_t>(i)];
00435         if (std::abs(c.startSeconds - selectedStart) < 0.75 &&
00436             std::abs(c.endSeconds - selectedEnd) < 0.75)
00437         {
00438             climbIndex = i;
00439             break;
00440         }
00441     }
00442
00443     if (climbIndex < 0)
00444         return;
00445
00446     const Climbe& climb = m_detectedClimbs[static_cast<size_t>(climbIndex)];
00447     const double padding = std::max(5.0, climb.durationSeconds * 0.08);
00448     const double rangeStart = std::max(0.0, climb.startSeconds - padding);
00449     const double rangeEnd = climb.endSeconds + padding;
00450
00451     const std::initializer_list<RideChartWidget*> all =
00452     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart,
00453       m_climbPowerChart, m_climbHrChart, m_climbCadenceChart, m_climbSpeedChart,
00454       m_climbAltitudeChart };
00455     for (auto* chart : all)
00456     {
00457         if (!chart)
00458             continue;
00459         chart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
00460         chart->setCrosshair(climb.startSeconds / 60.0);
00461     }
00462
00463     if (m_mapRenderer)
00464     {
00465         m_mapRenderer->setVisibleTimeRange(climb.startSeconds, climb.endSeconds);
00466         m_mapRenderer->fitToVisibleRange();
00467         m_mapRenderer->setCurrentTime(climb.startSeconds);
00468     }
00469 }
00470
00471 void MainWindow::onClimbBoundaryEdited(
00472     double oldStartSeconds,
00473     double oldEndSeconds,
00474     double newStartSeconds,
00475     double newEndSeconds)
00476 {
00477     if (m_detectedClimbs.empty())
00478         return;

```

```

00479
00480     int idx = -1;
00481     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00482     {
00483         const Climb& c = m_detectedClimbs[static_cast<size_t>(i)];
00484         if (std::abs(c.startSeconds - oldStartSeconds) < 0.75 &&
00485             std::abs(c.endSeconds - oldEndSeconds) < 0.75)
00486         {
00487             idx = i;
00488             break;
00489         }
00490     }
00491
00492     if (idx < 0)
00493         idx = m_climbsTable ? m_climbsTable->currentRow() : -1;
00494     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
00495         return;
00496
00497     const auto& ride = m_controller->rideData();
00498     if (ride.records.empty())
00499         return;
00500
00501     auto nearestIndexForSecond = [&ride](double sec)
00502     {
00503         auto it = std::lower_bound(
00504             ride.records.begin(),
00505             ride.records.end(),
00506             sec,
00507             [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
00508
00509         if (it == ride.records.end())
00510             return static_cast<int>(ride.records.size()) - 1;
00511
00512         int index = static_cast<int>(std::distance(ride.records.begin(), it));
00513         if (index > 0)
00514         {
00515             const double currDist = std::abs(ride.records[static_cast<size_t>(index)].elapsedSeconds -
00516 sec);
00517             const double prevDist = std::abs(ride.records[static_cast<size_t>(index -
00518 1)].elapsedSeconds - sec);
00519             if (prevDist < currDist)
00520                 --index;
00521         }
00522         return index;
00523     };
00524
00525     int startIdx = nearestIndexForSecond(newStartSeconds);
00526     int endIdx = nearestIndexForSecond(newEndSeconds);
00527     if (endIdx < startIdx)
00528         std::swap(startIdx, endIdx);
00529     if (endIdx <= startIdx)
00530         endIdx = std::min(static_cast<int>(ride.records.size()) - 1, startIdx + 1);
00531
00532     const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
00533     const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
00534         ride,
00535         cumulative,
00536         m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0);
00537     const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(smoothed, cumulative,
00538 25.0);
00539
00540     Climb rebuilt = ClimbMetrics::build(
00541         ride,
00542         cumulative,
00543         smoothed,
00544         gradient,
00545         startIdx,
00546         endIdx,
00547         idx + 1);
00548
00549     assignClimbWeightMetrics(rebuilt, resolveActiveActivityWeightKg());
00550
00551     rebuilt.name = m_detectedClimbs[static_cast<size_t>(idx)].name;
00552     rebuilt.id = m_detectedClimbs[static_cast<size_t>(idx)].id;
00553
00554     if (m_undoManager && m_dbManager.isOpen() && rebuilt.id > 0)
00555     {
00556         auto db = m_dbManager.database();
00557         ClimbRepository repo(db);
00558         const ClimbRecord beforeRecord = repo.getClimb(rebuilt.id);
00559         auto refresh = [this]() {
00560             m_controller->reloadClimbsFromDatabase();
00561             m_detectedClimbs = m_controller->climbs();
00562             applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00563             refreshClimbViews();
00564         };
00565         ClimbRecord afterRecord = beforeRecord;

```

```

00563         afterRecord.startSeconds = rebuilt.startSeconds;
00564         afterRecord.endSeconds   = rebuilt.endSeconds;
00565         afterRecord.elevationGainM = rebuilt.elevationGainM;
00566         afterRecord.lengthKm      = rebuilt.lengthKm;
00567         afterRecord.averageGradient = rebuilt.averageGradient;
00568         afterRecord.avgPower      = rebuilt.averagePower;
00569         afterRecord.np            = rebuilt.normalizedPower;
00570         afterRecord.avgHr         = rebuilt.averageHeartRate;
00571         afterRecord.avgCadence    = rebuilt.averageCadence;
00572         afterRecord.vam           = rebuilt.vam;
00573         afterRecord.source        = QStringLiteral("edited");
00574         afterRecord.locked        = true;
00575         m_undoManager->push(std::make_unique<ClimbEditCommand>(
00576             beforeRecord, afterRecord,
00577             m_dbManager.database().connectionName(),
00578             refresh));
00579     }
00580
00581     m_detectedClimbs[static_cast<size_t>(idx)] = rebuilt;
00582
00583     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
00584     {
00585         auto db = m_dbManager.database();
00586         ClimbRepository repo(db);
00587         const Climb& c = m_detectedClimbs[static_cast<size_t>(idx)];
00588         if (c.id > 0)
00589         {
00590             ClimbRecord record;
00591             record.id = c.id;
00592             record.activityId = m_controller->currentActivityId();
00593             record.startSeconds = c.startSeconds;
00594             record.endSeconds = c.endSeconds;
00595             record.name = c.name;
00596             record.elevationGainM = c.elevationGainM;
00597             record.lengthKm = c.lengthKm;
00598             record.averageGradient = c.averageGradient;
00599             record.source = QStringLiteral("edited");
00600             record.locked = true;
00601             repo.updateClimb(record);
00602         }
00603         else
00604         {
00605             ClimbRecord record;
00606             record.activityId = m_controller->currentActivityId();
00607             record.startSeconds = c.startSeconds;
00608             record.endSeconds = c.endSeconds;
00609             record.name = c.name;
00610             record.elevationGainM = c.elevationGainM;
00611             record.lengthKm = c.lengthKm;
00612             record.averageGradient = c.averageGradient;
00613             record.source = QStringLiteral("edited");
00614             record.locked = true;
00615             m_detectedClimbs[static_cast<size_t>(idx)].id = repo.insertClimb(record);
00616         }
00617     }
00618
00619     const auto applyClimbOverlays = [this](RideChartWidget* chart)
00620     {
00621         if (chart)
00622             chart->setClimbs(m_detectedClimbs);
00623     };
00624     applyClimbOverlays(m_powerChart);
00625     applyClimbOverlays(m_hrChart);
00626     applyClimbOverlays(m_cadenceChart);
00627     applyClimbOverlays(m_speedChart);
00628     applyClimbOverlays(m_altitudeChart);
00629     applyClimbOverlays(m_climbPowerChart);
00630     applyClimbOverlays(m_climbHrChart);
00631     applyClimbOverlays(m_climbCadenceChart);
00632     applyClimbOverlays(m_climbSpeedChart);
00633     applyClimbOverlays(m_climbAltitudeChart);
00634
00635     if (m_climbsTable)
00636     {
00637         auto setCell = [this, idx](int col, const QString& text)
00638         {
00639             auto* item = m_climbsTable->item(idx, col);
00640             if (!item)
00641             {
00642                 item = new QTableWidgetItem;
00643                 m_climbsTable->setItem(idx, col, item);
00644             }
00645             item->setText(text);
00646             item->setTextAlignment(Qt::AlignCenter);
00647         };
00648
00649         auto setNumericCell = [this, idx](int col, double value, int decimals, const QString& fallback

```

```

    = QStringLiteral("\xE2\x80\x94"), bool alwaysNumeric = false)
00650 {
00651     QTableWidgetItem* item = nullptr;
00652     if (alwaysNumeric || value > 0.0)
00653         item = new ClimbSortKeyTableItem(QString::number(value, 'f', decimals), value);
00654     else
00655         item = new ClimbSortKeyTableItem(fallback, std::numeric_limits<double>::quiet_NaN());
00656     item->setTextAlignment(Qt::AlignCenter);
00657     m_climbsTable->setItem(idx, col, item);
00658 };
00659
00660 auto* nameItem = m_climbsTable->item(idx, 0);
00661 if (!nameItem)
00662 {
00663     nameItem = new ClimbNameTableItem(rebuilt.name);
00664     m_climbsTable->setItem(idx, 0, nameItem);
00665 }
00666 nameItem->setData(kClimbStartRole, rebuilt.startSeconds);
00667 nameItem->setData(kClimbEndRole, rebuilt.endSeconds);
00668
00669 setNumericCell(1, rebuilt.lengthKm, 2, QStringLiteral("0.00"), true);
00670 setNumericCell(2, rebuilt.elevationGainM, 0, QStringLiteral("0"), true);
00671 setNumericCell(3, rebuilt.averageGradient, 1, QStringLiteral("0.0"), true);
00672 setNumericCell(4, rebuilt.maximumGradient, 1, QStringLiteral("0.0"), true);
00673 m_climbsTable->setItem(idx, 5, new ClimbSortKeyTableItem(fmtDur(rebuilt.durationSeconds),
rebuilt.durationSeconds));
00674 m_climbsTable->item(idx, 5)->setTextAlignment(Qt::AlignCenter);
00675 setNumericCell(6, rebuilt.averagePower, 0);
00676 setNumericCell(7, rebuilt.averageWattsPerKg, 2);
00677 setCell(8, rebuilt.wattsPerKgRank.isEmpty() ? QStringLiteral("\xE2\x80\x94") :
rebuilt.wattsPerKgRank);
00678 setNumericCell(9, rebuilt.normalizedPower, 0);
00679 setNumericCell(10, rebuilt.averageHeartRate, 0);
00680 setNumericCell(11, rebuilt.averageCadence, 0);
00681 setNumericCell(12, rebuilt.averageSpeed, 1);
00682 setNumericCell(13, rebuilt.vam, 0);
00683 setNumericCell(14, rebuilt.powerFadePct, 1, QStringLiteral("0.0"), true);
00684 setNumericCell(15, rebuilt.aerobicDecouplingPct, 1, QStringLiteral("0.0"), true);
00685 setNumericCell(16, rebuilt.difficultyScore, 0, QStringLiteral("0"), true);
00686 setCell(17, rebuilt.category.isEmpty() ? QStringLiteral("\xE2\x80\x94") : rebuilt.category);
00687 setCell(18, rebuilt.shapeClass.isEmpty() ? QStringLiteral("\xE2\x80\x94") :
rebuilt.shapeClass);
00688 }
00689
00690 onClimbSelectionChanged();
00691 }
00692
00693 void MainWindow::revertSelectedClimbToAuto()
00694 {
00695     std::vector<int> selected = selectedClimbIndicesFromTable();
00696     if (selected.size() != 1)
00697         return;
00698
00699     const int idx = selected.front();
00700     if (idx < 0 || idx >= static_cast<int>(m_detectedClimbs.size()))
00701         return;
00702
00703     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00704         return;
00705
00706     const int climbId = m_detectedClimbs[static_cast<size_t>(idx)].id;
00707     if (climbId <= 0)
00708         return;
00709
00710     auto db = m_dbManager.database();
00711     ClimbRepository repo(db);
00712     ClimbRecord record = repo.getClimb(climbId);
00713     if (record.id <= 0)
00714         return;
00715
00716     if (record.originalStartSeconds >= 0.0)
00717         record.startSeconds = record.originalStartSeconds;
00718     if (record.originalEndSeconds >= 0.0)
00719         record.endSeconds = record.originalEndSeconds;
00720     record.source = QStringLiteral("auto");
00721     record.locked = false;
00722     record.deleted = false;
00723
00724     if (!repo.updateClimb(record))
00725         return;
00726
00727     m_controller->reloadClimbsFromDatabase();
00728     m_detectedClimbs = m_controller->climbs();
00729     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00730     refreshClimbViews(record.startSeconds, record.endSeconds);
00731 }
00732

```

```

00733 void MainWindow::onNewClimbRequested(double startSeconds, double endSeconds)
00734 {
00735     if (!m_controller)
00736         return;
00737
00738     const auto& ride = m_controller->rideData();
00739     if (ride.records.empty())
00740         return;
00741
00742     const double orderedStart = std::min(startSeconds, endSeconds);
00743     const double orderedEnd = std::max(startSeconds, endSeconds);
00744     if (orderedEnd - orderedStart < 1.0)
00745         return;
00746
00747     auto nearestIndexForSecond = [&ride](double sec)
00748     {
00749         auto it = std::lower_bound(
00750             ride.records.begin(),
00751             ride.records.end(),
00752             sec,
00753             [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
00754
00755         if (it == ride.records.end())
00756             return static_cast<int>(ride.records.size()) - 1;
00757
00758         int index = static_cast<int>(std::distance(ride.records.begin(), it));
00759         if (index > 0)
00760         {
00761             const double currDist = std::abs(ride.records[static_cast<size_t>(index)].elapsedSeconds -
sec);
00762             const double prevDist = std::abs(ride.records[static_cast<size_t>(index -
1)].elapsedSeconds - sec);
00763             if (prevDist < currDist)
00764                 --index;
00765         }
00766         return index;
00767     };
00768
00769     int startIdx = nearestIndexForSecond(orderedStart);
00770     int endIdx = nearestIndexForSecond(orderedEnd);
00771     if (endIdx < startIdx)
00772         std::swap(startIdx, endIdx);
00773     if (endIdx <= startIdx)
00774         endIdx = std::min(static_cast<int>(ride.records.size()) - 1, startIdx + 1);
00775
00776     const std::vector<double> cumulative = ClimbDetector::buildCumulativeDistanceMeters(ride);
00777     const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
00778         ride,
00779         cumulative,
00780         m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0);
00781     const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(smoothed, cumulative,
25.0);
00782
00783     Climb newClimb = ClimbMetrics::build(
00784         ride,
00785         cumulative,
00786         smoothed,
00787         gradient,
00788         startIdx,
00789         endIdx,
00790         static_cast<int>(m_detectedClimbs.size()) + 1);
00791
00792     assignClimbWeightMetrics(newClimb, resolveActiveActivityWeightKg());
00793
00794     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
00795     {
00796         auto db = m_dbManager.database();
00797         ClimbRepository repo(db);
00798         ClimbRecord record;
00799         record.activityId = m_controller->currentActivityId();
00800         record.startSeconds = newClimb.startSeconds;
00801         record.endSeconds = newClimb.endSeconds;
00802         record.name = newClimb.name;
00803         record.elevationGainM = newClimb.elevationGainM;
00804         record.lengthKm = newClimb.lengthKm;
00805         record.averageGradient = newClimb.averageGradient;
00806         record.source = QStringLiteral("manual");
00807         record.locked = true;
00808         newClimb.id = repo.insertClimb(record);
00809     }
00810
00811     m_detectedClimbs.push_back(newClimb);
00812     std::sort(
00813         m_detectedClimbs.begin(),
00814         m_detectedClimbs.end(),
00815         [](const Climb& a, const Climb& b)
00816         {

```

```

00817         if (std::abs(a.startSeconds - b.startSeconds) > 0.1)
00818             return a.startSeconds < b.startSeconds;
00819         return a.endSeconds < b.endSeconds;
00820     });
00821
00822     for (int i = 0; i < static_cast<int>(m_detectedClimbs.size()); ++i)
00823         m_detectedClimbs[static_cast<size_t>(i)].name = QString("Climb %1").arg(i + 1);
00824
00825     refreshClimbViews(newClimb.startSeconds, newClimb.endSeconds);
00826 }

```

## 8.132 MainWindowClimbViews.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QBrush>
00006 #include <QColor>
00007 #include <QSignalBlocker>
00008 #include <QTableWidgetItem>
00009
00010 #include <QtCharts/QAbstractAxis>
00011 #include <QtCharts/QBarCategoryAxis>
00012 #include <QtCharts/QBarSeries>
00013 #include <QtCharts/QBarSet>
00014 #include <QtCharts/QChart>
00015 #include <QtCharts/QChartView>
00016 #include <QtCharts/QValueAxis>
00017
00018 #include "charts/RideChartWidget.h"
00019
00020 #include <algorithm>
00021 #include <cmath>
00022 #include <functional>
00023 #include <limits>
00024
00025 namespace
00026 {
00027     QString fmtDur(double seconds)
00028     {
00029         const int total = static_cast<int>(seconds);
00030         const int h = total / 3600;
00031         const int m = (total % 3600) / 60;
00032         const int s = total % 60;
00033         return h > 0
00034             ? QString("%1:%2:%3")
00035               .arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00036             : QString("%1:%2")
00037               .arg(m).arg(s, 2, 10, QChar('0'));
00038     }
00039
00040     static constexpr int kClimbStartRole = Qt::UserRole + 3;
00041     static constexpr int kClimbEndRole = Qt::UserRole + 4;
00042
00043     class ClimbNameTableWidgetItem final : public QTableWidgetItem
00044     {
00045     public:
00046         explicit ClimbNameTableWidgetItem(const QString& text)
00047             : QTableWidgetItem(text)
00048         {}
00049
00050         bool operator<(const QTableWidgetItem& other) const override
00051         {
00052             bool leftOk = false;
00053             bool rightOk = false;
00054             const int leftNumber = trailingNumber(text(), &leftOk);
00055             const int rightNumber = trailingNumber(other.text(), &rightOk);
00056
00057             if (leftOk && rightOk)
00058                 return leftNumber < rightNumber;
00059
00060             return QString::localeAwareCompare(text(), other.text()) < 0;
00061         }
00062
00063     private:
00064         static int trailingNumber(const QString& text, bool* ok)
00065         {
00066             if (ok)
00067                 *ok = false;
00068
00069             int i = text.size() - 1;
00070             while (i >= 0 && text[i].isSpace())
00071                 --i;

```

```

00072
00073     if (i < 0 || !text[i].isDigit())
00074         return 0;
00075
00076     int end = i;
00077     while (i >= 0 && text[i].isDigit())
00078         --i;
00079
00080     const QString number = text.mid(i + 1, end - i);
00081     bool converted = false;
00082     const int value = number.toInt(&converted);
00083     if (ok)
00084         *ok = converted;
00085     return value;
00086 }
00087 };
00088
00089 class ClimbSortKeyTableItem final : public QTableWidgetItem
00090 {
00091 public:
00092     explicit ClimbSortKeyTableItem(const QString& text, double sortKey)
00093         : QTableWidgetItem(text)
00094         , m_sortKey(sortKey)
00095     {}
00096
00097     bool operator<(const QTableWidgetItem& other) const override
00098     {
00099         const auto* rhs = dynamic_cast<const ClimbSortKeyTableItem*>(&other);
00100         if (!rhs)
00101             return QString::localeAwareCompare(text(), other.text()) < 0;
00102
00103         const bool leftNaN = std::isnan(m_sortKey);
00104         const bool rightNaN = std::isnan(rhs->m_sortKey);
00105         if (leftNaN != rightNaN)
00106             return !leftNaN;
00107         if (leftNaN && rightNaN)
00108             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00109
00110         if (m_sortKey == rhs->m_sortKey)
00111             return QString::localeAwareCompare(text(), rhs->text()) < 0;
00112         return m_sortKey < rhs->m_sortKey;
00113     }
00114
00115 private:
00116     double m_sortKey = std::numeric_limits<double>::quiet_NaN();
00117 };
00118 } // namespace
00119
00120 void MainWindow::updateClimbingTab()
00121 {
00122     if (!m_climbsTable)
00123         return;
00124
00125     const auto& ride = m_controller->rideData();
00126     if (ride.records.empty())
00127     {
00128         m_detectedClimbs.clear();
00129         m_climbsTable->setRowCount(0);
00130         updateClimbQuarterCharts(nullptr);
00131         if (m_climbSummaryLabel)
00132             m_climbSummaryLabel->setText("Climb summary: select a climb");
00133         return;
00134     }
00135
00136     m_detectedClimbs = m_controller->climbs();
00137     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00138     refreshClimbViews();
00139 }
00140
00141 void MainWindow::refreshClimbViews(double preferredStartSeconds, double preferredEndSeconds)
00142 {
00143     if (!m_climbsTable)
00144         return;
00145
00146     applyDetectedClimbsToCharts();
00147
00148     const int previousRow = m_climbsTable->currentRow();
00149     QSignalBlocker blocker(m_climbsTable);
00150     m_climbsTable->setSortingEnabled(false);
00151     m_climbsTable->setRowCount(static_cast<int>(m_detectedClimbs.size()));
00152
00153     auto mkSortKeyItem = [](const QString& text, double sortKey)
00154     {
00155         auto* item = new ClimbSortKeyTableItem(text, sortKey);
00156         item->setTextAlignment(Qt::AlignCenter);
00157         return item;
00158     };

```



```

00159
00160     auto mkNumericItem = [mkSortKeyItem](double value, int decimals, const QString& fallback =
    QStringLiteral("--"))
00161     {
00162         if (std::isnan(value))
00163             return mkSortKeyItem(fallback, std::numeric_limits<double>::quiet_NaN());
00164         return mkSortKeyItem(QString::number(value, 'f', decimals), value);
00165     };
00166
00167     for (int row = 0; row < static_cast<int>(m_detectedClimbs.size()); ++row)
00168     {
00169         const Climb& climb = m_detectedClimbs[static_cast<size_t>(row)];
00170         auto* nameItem = new ClimbNameTableItem(climb.name.isEmpty() ? QString("Climb %1").arg(row +
00171 1) : climb.name);
00172         nameItem->setData(kClimbStartRole, climb.startSeconds);
00173         nameItem->setData(kClimbEndRole, climb.endSeconds);
00174         nameItem->setBackground(QBrush(QColor(34, 197, 94, 45)));
00175         m_climbsTable->setItem(row, 0, nameItem);
00176         m_climbsTable->setItem(row, 1, mkNumericItem(climb.lengthKm, 2));
00177         m_climbsTable->setItem(row, 2, mkNumericItem(climb.elevationGainM, 0));
00178         m_climbsTable->setItem(row, 3, mkNumericItem(climb.averageGradient, 1));
00179         m_climbsTable->setItem(row, 4, mkNumericItem(climb.maximumGradient, 1));
00180         m_climbsTable->setItem(row, 5, mkSortKeyItem(fmtDur(climb.durationSeconds),
    climb.durationSeconds));
00181         m_climbsTable->setItem(row, 6, mkNumericItem(climb.averagePower > 0.0 ? climb.averagePower :
    std::numeric_limits<double>::quiet_NaN(), 0));
00182         m_climbsTable->setItem(row, 7, mkNumericItem(climb.averageWattsPerKg > 0.0 ?
    climb.averageWattsPerKg : std::numeric_limits<double>::quiet_NaN(), 2));
00183         m_climbsTable->setItem(row, 8, mkSortKeyItem(climb.wattsPerKgRank.isEmpty() ?
    QStringLiteral("--") : climb.wattsPerKgRank, row));
00184         m_climbsTable->setItem(row, 9, mkNumericItem(climb.normalizedPower > 0.0 ?
    climb.normalizedPower : std::numeric_limits<double>::quiet_NaN(), 0));
00185         m_climbsTable->setItem(row, 10, mkNumericItem(climb.averageHeartRate > 0.0 ?
    climb.averageHeartRate : std::numeric_limits<double>::quiet_NaN(), 0));
00186         m_climbsTable->setItem(row, 11, mkNumericItem(climb.averageCadence > 0.0 ?
    climb.averageCadence : std::numeric_limits<double>::quiet_NaN(), 0));
00187         m_climbsTable->setItem(row, 12, mkNumericItem(climb.averageSpeed > 0.0 ? climb.averageSpeed :
    std::numeric_limits<double>::quiet_NaN(), 1));
00188         m_climbsTable->setItem(row, 13, mkNumericItem(climb.vam > 0.0 ? climb.vam :
    std::numeric_limits<double>::quiet_NaN(), 0));
00189         m_climbsTable->setItem(row, 14, mkNumericItem(climb.powerFadePct, 1));
00190         m_climbsTable->setItem(row, 15, mkNumericItem(climb.aerobicDecouplingPct, 1));
00191         m_climbsTable->setItem(row, 16, mkNumericItem(climb.difficultyScore, 0));
00192         m_climbsTable->setItem(row, 17, mkSortKeyItem(climb.category.isEmpty() ? QStringLiteral("--")
    : climb.category, row));
00193         m_climbsTable->setItem(row, 18, mkSortKeyItem(climb.shapeClass.isEmpty() ?
    QStringLiteral("--") : climb.shapeClass, row));
00194     }
00195     m_climbsTable->resizeColumnsToContents();
00196
00197     m_climbsTable->setSortingEnabled(true);
00198     m_climbsTable->sortItems(0, Qt::AscendingOrder);
00199
00200     restoreClimbSelection(previousRow, preferredStartSeconds, preferredEndSeconds);
00201
00202     blocker.unblock();
00203     updateClimbRowStyles();
00204     m_suppressClimbAutoZoom = true;
00205     onClimbSelectionChanged();
00206     m_suppressClimbAutoZoom = false;
00207 }
00208
00209 void MainWindow::applyDetectedClimbsToCharts()
00210 {
00211     const auto applyClimbOverlays = [this](RideChartWidget* chart)
00212     {
00213         if (chart)
00214             chart->setClimbs(m_detectedClimbs);
00215     };
00216
00217     applyClimbOverlays(m_powerChart);
00218     applyClimbOverlays(m_hrChart);
00219     applyClimbOverlays(m_cadenceChart);
00220     applyClimbOverlays(m_speedChart);
00221     applyClimbOverlays(m_altitudeChart);
00222     applyClimbOverlays(m_climbPowerChart);
00223     applyClimbOverlays(m_climbHrChart);
00224     applyClimbOverlays(m_climbCadenceChart);
00225     applyClimbOverlays(m_climbSpeedChart);
00226     applyClimbOverlays(m_climbAltitudeChart);
00227 }
00228
00229 void MainWindow::restoreClimbSelection(int previousRow, double preferredStartSeconds, double
    preferredEndSeconds)
00230 {
00231     if (!m_climbsTable)

```

```

00232         return;
00233
00234     int targetRow = -1;
00235     if (preferredStartSeconds >= 0.0 && preferredEndSeconds >= 0.0)
00236     {
00237         for (int row = 0; row < m_climbsTable->rowCount(); ++row)
00238         {
00239             auto* item = m_climbsTable->item(row, 0);
00240             if (!item)
00241                 continue;
00242             const double s = item->data(kClimbStartRole).toDouble();
00243             const double e = item->data(kClimbEndRole).toDouble();
00244             if (std::abs(s - preferredStartSeconds) < 0.75 &&
00245                 std::abs(e - preferredEndSeconds) < 0.75)
00246             {
00247                 targetRow = row;
00248                 break;
00249             }
00250         }
00251     }
00252
00253     if (targetRow < 0 && m_climbsTable->rowCount() > 0)
00254         targetRow = std::clamp(previousRow >= 0 ? previousRow : 0, 0, m_climbsTable->rowCount() - 1);
00255
00256     if (targetRow >= 0)
00257         m_climbsTable->setCurrentCell(targetRow, 0);
00258 }
00259
00260 void MainWindow::updateClimbQuarterCharts(const Climb* climb)
00261 {
00262     auto updateChart = [climb](QChartView* view,
00263                                const QString& title,
00264                                const QColor& color,
00265                                const std::function<double(const ClimbQuarterMetrics*)>& metric,
00266                                int decimals,
00267                                const QString& axisLabel)
00268     {
00269         if (!view || !view->chart())
00270             return;
00271
00272         QChart* chart = view->chart();
00273         chart->setTitle(title);
00274         chart->removeAllSeries();
00275
00276         const auto oldAxes = chart->axes();
00277         for (QAbstractAxis* axis : oldAxes)
00278         {
00279             chart->removeAxis(axis);
00280             delete axis;
00281         }
00282
00283         auto* set = new QBarSet(title);
00284         set->setColor(color);
00285
00286         double maxValue = 1.0;
00287         for (int q = 0; q < 4; ++q)
00288         {
00289             double value = 0.0;
00290             if (climb)
00291             {
00292                 value = metric(climb->quarterMetrics[static_cast<size_t>(q)]);
00293                 if (!std::isfinite(value) || value < 0.0)
00294                     value = 0.0;
00295             }
00296
00297             *set << value;
00298             maxValue = std::max(maxValue, value);
00299         }
00300
00301         auto* series = new QBarSeries(chart);
00302         series->append(set);
00303         chart->addSeries(series);
00304
00305         auto* axisX = new QBarCategoryAxis(chart);
00306         axisX->append({ QStringLiteral("Q1"), QStringLiteral("Q2"), QStringLiteral("Q3"),
00307             QStringLiteral("Q4") });
00307         chart->addAxis(axisX, Qt::AlignBottom);
00308         series->attachAxis(axisX);
00309
00310         auto* axisY = new QValueAxis(chart);
00311         axisY->setRange(0.0, maxValue * 1.15);
00312         axisY->setLabelFormat(decimals > 0 ? QString("%%.1f").arg(decimals) : QString("%.0f"));
00313         axisY->setLabelText(axisLabel);
00314         chart->addAxis(axisY, Qt::AlignLeft);
00315         series->attachAxis(axisY);
00316     };
00317

```

```

00318     updateChart (
00319         m_climbQuarterPowerChart,
00320         QStringLiteral("Power by Quarter"),
00321         QColor(37, 99, 235),
00322         [](const ClimbQuarterMetrics& m) { return m.averagePower; },
00323         0,
00324         QStringLiteral("W"));
00325
00326     updateChart (
00327         m_climbQuarterHrChart,
00328         QStringLiteral("Heart Rate by Quarter"),
00329         QColor(220, 38, 38),
00330         [](const ClimbQuarterMetrics& m) { return m.averageHeartRate; },
00331         0,
00332         QStringLiteral("bpm"));
00333
00334     updateChart (
00335         m_climbQuarterCadenceChart,
00336         QStringLiteral("Cadence by Quarter"),
00337         QColor(5, 150, 105),
00338         [](const ClimbQuarterMetrics& m) { return m.averageCadence; },
00339         0,
00340         QStringLiteral("rpm"));
00341 }

```

## 8.133 MainWindowContext.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include "database/ActivityRepository.h"
00006 #include "database/AthleteRepository.h"
00007 #include "fit/RideData.h"
00008
00009 #include <algorithm>
00010 #include <limits>
00011
00012 void MainWindow::refreshAthleteSelector()
00013 {
00014     if (!m_athleteCombo)
00015         return;
00016
00017     const QSignalBlocker blocker(m_athleteCombo);
00018     m_athleteCombo->clear();
00019     m_athleteCombo->addItem("Select athlete...", -1);
00020
00021     if (m_dbManager.isOpen())
00022     {
00023         auto db = m_dbManager.database();
00024         AthleteRepository repo(db);
00025         for (const Athlete& athlete : repo.listAthletes())
00026             m_athleteCombo->addItem(athlete.fullName(), athlete.id);
00027     }
00028
00029     const int idx = m_athleteCombo->findData(m_currentAthleteId);
00030     if (idx >= 0)
00031     {
00032         m_athleteCombo->setCurrentIndex(idx);
00033     }
00034     else
00035     {
00036         m_athleteCombo->setCurrentIndex(0);
00037         m_currentAthleteId = -1;
00038         m_currentAthleteName.clear();
00039         m_controller->setCurrentAthlete(-1);
00040     }
00041 }
00042
00043 ColorMetric MainWindow::currentColorMetric() const
00044 {
00045     if (!m_colorMetricCombo)
00046         return ColorMetric::None;
00047
00048     return static_cast<ColorMetric>(m_colorMetricCombo->currentData().toInt());
00049 }
00050
00051 ColorContext MainWindow::buildColorContext() const
00052 {
00053     ColorContext context;
00054     context.ftp = static_cast<int>(m_controller->ftp());
00055
00056     if (m_dbManager.isOpen() && m_currentAthleteId > 0)
00057     {

```

```

00058     auto db = m_dbManager.database();
00059     AthleteRepository repo(db);
00060
00061     int resolvedFtp = 0;
00062     if (m_controller->currentActivityId() > 0)
00063     {
00064         ActivityRepository actRepo(db);
00065         const Activity activity = actRepo.getActivity(m_controller->currentActivityId());
00066         const QString rawDate = !activity.startTime.isEmpty()
00067             ? activity.startTime.left(10)
00068             : activity.importedAt.left(10);
00069         const QDate ftpDate = QDate::fromString(rawDate, Qt::ISODate);
00070         if (ftpDate.isValid())
00071             resolvedFtp = repo.getFtpForDate(m_currentAthleteId, ftpDate);
00072     }
00073
00074     if (resolvedFtp <= 0)
00075         resolvedFtp = repo.getAthlete(m_currentAthleteId).ftpWatts;
00076
00077     if (resolvedFtp > 0)
00078         context.ftp = resolvedFtp;
00079 }
00080
00081 int maxHeartRate = 0;
00082 double minAltitude = std::numeric_limits<double>::max();
00083 double maxAltitude = std::numeric_limits<double>::lowest();
00084 bool hasAltitude = false;
00085
00086 for (const RideRecord& record : m_controller->rideData().records)
00087 {
00088     if (record.hasHeartRate)
00089         maxHeartRate = std::max(maxHeartRate, static_cast<int>(record.heartRate));
00090     if (record.hasAltitude)
00091     {
00092         minAltitude = std::min(minAltitude, record.altitude);
00093         maxAltitude = std::max(maxAltitude, record.altitude);
00094         hasAltitude = true;
00095     }
00096 }
00097
00098 if (maxHeartRate > 0)
00099     context.heartRateMax = maxHeartRate;
00100 if (hasAltitude)
00101 {
00102     context.altitudeMin = minAltitude;
00103     context.altitudeMax = maxAltitude;
00104     context.hasAltitudeRange = true;
00105 }
00106
00107 return context;
00108 }

```

## 8.134 MainWindowDatabase.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QAction>
00006 #include <QFileDialog>
00007 #include <QFileInfo>
00008 #include <QMessageBox>
00009 #include <QSettings>
00010 #include <QStandardPaths>
00011 #include <QStatusBar>
00012
00013 #include "ActivityBrowser.h"
00014 #include "CalendarWidget.h"
00015 #include "CreateDatabaseDialog.h"
00016 #include "controllers/NavigationController.h"
00017 #include "database/AthleteRepository.h"
00018 #include "maps/TileCache.h"
00019 #include "platform/Platform.h"
00020
00021 namespace
00022 {
00023     static constexpr int kMaxRecentDatabases = 8;
00024
00025     static constexpr int kChartPresetCustom = 0;
00026
00027     // Legacy tab indices from pre-page-stack navigation.
00028     static constexpr int kLegacyTabActivities = 0;
00029     static constexpr int kLegacyTabAnalysis = 1;
00030 }

```

```

00033 // Legacy analysis sub-tab indices used when activeTab == Analysis.
00034 static constexpr int kLegacyAnalysisCharts = 0;
00035 static constexpr int kLegacyAnalysisZones = 1;
00036 static constexpr int kLegacyAnalysisHistogram = 2;
00037 static constexpr int kLegacyAnalysisPowerCurve = 3;
00038 static constexpr int kLegacyAnalysisCalendar = 4;
00039 static constexpr int kLegacyAnalysisFitness = 5;
00040 static constexpr int kLegacyAnalysisClimbing = 6;
00041
00042 // Current top-level page indices (NavigationSidebar::Page values).
00043 static constexpr int kPageActivities = 0;
00044 static constexpr int kPageCharts = 1;
00045 static constexpr int kPagePower = 2;
00046 static constexpr int kPageIntervals = 3;
00047 static constexpr int kPageClimbs = 4;
00048 static constexpr int kPageFitness = 5;
00049 static constexpr int kPageCalendar = 6;
00050 static constexpr int kPageVideo = 7;
00051
00053 int mapLegacyTabsToPageIndex(int legacyActiveTab, int legacyAnalysisTab)
00054 {
00055     if (legacyActiveTab == kLegacyTabActivities)
00056         return kPageActivities;
00057
00058     if (legacyActiveTab == kLegacyTabAnalysis)
00059     {
00060         switch (legacyAnalysisTab)
00061         {
00062             case kLegacyAnalysisCharts: return kPageCharts;
00063             case kLegacyAnalysisZones:
00064             case kLegacyAnalysisHistogram:
00065             case kLegacyAnalysisPowerCurve: return kPagePower;
00066             case kLegacyAnalysisCalendar: return kPageCalendar;
00067             case kLegacyAnalysisFitness: return kPageFitness;
00068             case kLegacyAnalysisClimbing: return kPageClimbs;
00069             default: return kPageCharts;
00070         }
00071     }
00072
00073     return kPageActivities;
00074 }
00075
00076
00077 void MainWindow::saveSettings()
00078 {
00079     QSettings s("Fitlyzer", "FitlyzerC");
00080     s.setValue("geometry", saveGeometry());
00081
00082     // Persist page-based navigation state. Keep legacy keys out of new writes.
00083     if (m_navigationController)
00084         m_navigationController->saveNavigationState();
00085
00086     if (m_analysisTabWidget)
00087         s.setValue("analysisTab", m_analysisTabWidget->currentIndex());
00088     if (m_colorMetricCombo)
00089         s.setValue("colorMetric", m_colorMetricCombo->currentData().toInt());
00090     if (m_powerSmoothingCombo)
00091         s.setValue("powerSmoothing", m_powerSmoothingCombo->currentData().toInt());
00092     if (m_chartPresetCombo)
00093         s.setValue("chartPreset", m_chartPresetCombo->currentData().toInt());
00094     if (m_autoSmoothingCheck)
00095         s.setValue("autoSmoothing", m_autoSmoothingCheck->isChecked());
00096     if (m_followIntervalSelectionCheck)
00097         s.setValue("followIntervalSelection", m_followIntervalSelectionCheck->isChecked());
00098     if (m_mapStyleCombo)
00099         s.setValue("mapStyle", m_mapStyleCombo->currentData().toInt());
00100     if (m_mapAutoContrastCheck)
00101         s.setValue("mapAutoRouteContrast", m_mapAutoContrastCheck->isChecked());
00102     s.setValue("lastOpenDir", m_lastOpenDir);
00103     s.setValue("showPower", m_showPower->isChecked());
00104     s.setValue("showHR", m_showHR->isChecked());
00105     s.setValue("showCadence", m_showCadence->isChecked());
00106     s.setValue("showSpeed", m_showSpeed->isChecked());
00107     s.setValue("showAltitude", m_showAltitude->isChecked());
00108     s.setValue("watchFolderEnabled", m_watchFolderEnabled);
00109     s.setValue("watchFolderPath", m_watchFolderPath);
00110     s.setValue("chartHeight", m_chartHeight);
00111     s.setValue("currentAthleteId", m_currentAthleteId);
00112     s.setValue("lastActivityId", m_controller ? m_controller->currentActivityId() : -1);
00113     if (m_dbManager.isOpen())
00114         s.setValue("lastDatabase", m_dbManager.path());
00115 }
00116
00117 void MainWindow::loadSettings()
00118 {
00119     QSettings s("Fitlyzer", "FitlyzerC");
00120

```

```

00121     if (s.contains("geometry"))
00122         restoreGeometry(s.value("geometry").toByteArray());
00123
00124     m_lastOpenDir = s.value("lastOpenDir", QString()).toString();
00125
00126     m_showPower->setChecked(s.value("showPower", true).toBool());
00127     m_showHR->setChecked(s.value("showHR", true).toBool());
00128     m_showCadence->setChecked(s.value("showCadence", true).toBool());
00129     m_showSpeed->setChecked(s.value("showSpeed", true).toBool());
00130     m_showAltitude->setChecked(s.value("showAltitude", true).toBool());
00131     m_watchFolderEnabled = s.value("watchFolderEnabled", true).toBool();
00132     m_watchFolderPath = s.value(
00133         "watchFolderPath",
00134         QStandardPaths::writableLocation(QStandardPaths::DownloadLocation)).toString();
00135
00136     m_chartHeight = std::clamp(s.value("chartHeight", 220).toInt(), 120, 600);
00137
00138     if (m_powerSmoothingCombo)
00139     {
00140         const int smoothing = s.value("powerSmoothing", 0).toInt();
00141         const int comboIndex = m_powerSmoothingCombo->findData(smoothing);
00142         if (comboIndex >= 0)
00143             m_powerSmoothingCombo->setCurrentIndex(comboIndex);
00144     }
00145     if (m_autoSmoothingCheck)
00146     {
00147         const bool autoSmoothing = s.value("autoSmoothing", false).toBool();
00148         m_autoSmoothingCheck->setChecked(autoSmoothing);
00149         if (m_powerSmoothingCombo)
00150             m_powerSmoothingCombo->setEnabled(!autoSmoothing);
00151     }
00152     if (m_followIntervalSelectionCheck)
00153     {
00154         m_followIntervalSelectionCheck->setChecked(
00155             s.value("followIntervalSelection", true).toBool());
00156     }
00157     if (m_mapStyleCombo)
00158     {
00159         const int styleValue = s.value("mapStyle", static_cast<int>(MapStyle::Light)).toInt();
00160         const int comboIndex = m_mapStyleCombo->findData(styleValue);
00161         if (comboIndex >= 0)
00162             m_mapStyleCombo->setCurrentIndex(comboIndex);
00163     }
00164     if (m_mapAutoContrastCheck)
00165     {
00166         m_mapAutoContrastCheck->setChecked(s.value("mapAutoRouteContrast", true).toBool());
00167     }
00168
00169     if (m_chartPresetCombo)
00170     {
00171         const int presetValue = s.value("chartPreset", kChartPresetCustom).toInt();
00172         const int comboIndex = m_chartPresetCombo->findData(presetValue);
00173         if (comboIndex >= 0)
00174             m_chartPresetCombo->setCurrentIndex(comboIndex);
00175     }
00176
00177     if (m_colorMetricCombo)
00178     {
00179         const int colorMetricValue = s.value("colorMetric",
00180 static_cast<int>(ColorMetric::Power)).toInt();
00181         const int comboIndex = m_colorMetricCombo->findData(colorMetricValue);
00182         if (comboIndex >= 0)
00183             m_colorMetricCombo->setCurrentIndex(comboIndex);
00184     }
00185     // Restore top-level page selection.
00186     // 1) Preferred: new page-based key handled by NavigationController.
00187     // 2) Fallback: migrate legacy activeTab/analysisTab keys.
00188     if (m_navigationController)
00189     {
00190         if (s.contains("activePage"))
00191         {
00192             m_navigationController->restoreNavigationState();
00193         }
00194         else if (s.contains("activeTab"))
00195         {
00196             const int legacyActiveTab = s.value("activeTab", kLegacyTabActivities).toInt();
00197             const int legacyAnalysisTab = s.value("analysisTab", kLegacyAnalysisCharts).toInt();
00198             const int pageIndex = mapLegacyTabsToPageIndex(legacyActiveTab, legacyAnalysisTab);
00199             m_navigationController->navigateTo(static_cast<NavigationController::Page>(pageIndex));
00200         }
00201     }
00202
00203     if (m_analysisTabWidget && s.contains("analysisTab"))
00204     {
00205         const int requested = s.value("analysisTab", 0).toInt();
00206         const int clamped = std::clamp(requested, 0, m_analysisTabWidget->count() - 1);

```

```

00207         m_analysisTabWidget->setCurrentIndex(clamped);
00208     }
00209 }
00210
00211 void MainWindow::addRecentDatabase(const QString& path)
00212 {
00213     QSettings s("Fitlyzer", "FitlyzerC");
00214     QStringList recent = s.value("recentDatabases").toStringList();
00215     recent.removeAll(path);
00216     recent.prepend(path);
00217     while (recent.size() > kMaxRecentDatabases)
00218         recent.removeLast();
00219     s.setValue("recentDatabases", recent);
00220 }
00221
00222 void MainWindow::updateRecentDatabaseMenu()
00223 {
00224     if (!m_recentDbMenu) return;
00225
00226     m_recentDbMenu->clear();
00227     QSettings s("Fitlyzer", "FitlyzerC");
00228     const QStringList recent = s.value("recentDatabases").toStringList();
00229
00230     for (const QString& path : recent)
00231     {
00232         auto* act = m_recentDbMenu->addAction(QFileInfo(path).fileName());
00233         act->setData(path);
00234         act->setStatusTip(path);
00235         connect(act, &QAction::triggered, this, &MainWindow::openRecentDatabase);
00236     }
00237
00238     if (recent.isEmpty())
00239     {
00240         auto* emptyAct = m_recentDbMenu->addAction("(none)");
00241         emptyAct->setEnabled(false);
00242     }
00243 }
00244
00245 void MainWindow::openDatabasePath(const QString& path)
00246 {
00247     QString err;
00248     if (!m_dbManager.open(path, &err))
00249     {
00250         QMessageBox::critical(this, "Database Error",
00251             "Could not open database:\n" + err);
00252         return;
00253     }
00254
00255     m_controller->setDatabaseManager(&m_dbManager);
00256     QSettings settings("Fitlyzer", "FitlyzerC");
00257     settings.setValue("lastDatabase", path);
00258     addRecentDatabase(path);
00259     updateRecentDatabaseMenu();
00260     if (m_analysisQueue)
00261         m_analysisQueue->setDatabasePath(path);
00262     if (m_importQueue)
00263         m_importQueue->setDatabasePath(path);
00264
00265     if (m_activityBrowser)
00266         m_activityBrowser->refresh(m_currentAthleteId);
00267
00268     if (m_calendarWidget)
00269     {
00270         m_calendarWidget->setDatabaseManager(&m_dbManager);
00271         m_calendarWidget->setAthleteId(m_currentAthleteId);
00272     }
00273
00274     refreshAthleteSelector();
00275     updateAthleteLabel();
00276     updateImportAvailability();
00277     updateStatsLabel();
00278     updateStatusBarInfo();
00279     updateWelcomeScreenVisibility();
00280     statusBar()->showMessage("Database opened.", 2500);
00281 }
00282
00283 void MainWindow::openDatabase()
00284 {
00285     const QString startDir = Platform::defaultDatabaseDirectory();
00286     const QString path = QFileDialog::getOpenFileName(
00287         this, "Open Database", startDir,
00288         "Fitlyzer Database (*.fitlyzerdb);;All Files (*)");
00289     if (path.isEmpty()) return;
00290     openDatabasePath(path);
00291 }
00292
00293 void MainWindow::createDatabase()

```

```

00294 {
00295     CreateDatabaseDialog dlg(Platform::defaultDatabaseDirectory(), this);
00296     if (dlg.exec() != QDialog::Accepted)
00297         return;
00298
00299     const QString path = dlg.databasePath();
00300     if (path.isEmpty())
00301         return;
00302
00303     QString err;
00304     if (!m_dbManager.create(path, &err))
00305     {
00306         QMessageBox::critical(this, "Database Error",
00307             "Could not create database:\n" + err);
00308         return;
00309     }
00310
00311     m_controller->setDatabaseManager(&m_dbManager);
00312     QSettings settings("Fitlyzer", "FitlyzerC");
00313     settings.setValue("lastDatabase", path);
00314     m_lastOpenDir = QFileInfo(path).absolutePath();
00315
00316     if (dlg.shouldCreateDefaultAthlete())
00317     {
00318         auto db = m_dbManager.database();
00319         AthleteRepository repo(db);
00320
00321         Athlete athlete;
00322         athlete.firstName = dlg.defaultAthleteFirstName();
00323         athlete.lastName = dlg.defaultAthleteLastName();
00324         const int athleteId = repo.insertAthlete(athlete);
00325         if (athleteId > 0)
00326         {
00327             m_currentAthleteId = athleteId;
00328             m_currentAthleteName = repo.getAthlete(athleteId).fullName();
00329             m_controller->setCurrentAthlete(athleteId);
00330             settings.setValue("currentAthleteId", athleteId);
00331         }
00332     }
00333
00334     addRecentDatabase(path);
00335     updateRecentDatabaseMenu();
00336     refreshAthleteSelector();
00337     updateAthleteLabel();
00338     updateImportAvailability();
00339     updateStatsLabel();
00340     updateStatusBarInfo();
00341     updateWelcomeScreenVisibility();
00342
00343     QMessageBox::information(this, "Database Created",
00344         "Database created successfully.\n" + path);
00345 }

```

## 8.135 MainWindowDetection.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QMessageBox>
00006 #include <QStatusBar>
00007
00008 #include "AboutDialog.h"
00009 #include "EditActivityDialog.h"
00010 #include "analysis/ClimbDetector.h"
00011 #include "analysis/IntervalDetector.h"
00012 #include "database/ActivityRepository.h"
00013 #include "database/ClimbRepository.h"
00014 #include "database/IntervalRepository.h"
00015
00016 #include <cmath>
00017 #include <vector>
00018
00019 void MainWindow::triggerDetectClimbs()
00020 {
00021     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00022     {
00023         QMessageBox::information(this, "Detect Climbs", "No activity loaded.");
00024         return;
00025     }
00026
00027     const auto& ride = m_controller->rideData();
00028     if (ride.records.empty())
00029         return;

```



```

00030
00031     auto db = m_dbManager.database();
00032     ClimbRepository repo(db);
00033     if (repo.hasLockedClimbs(m_controller->currentActivityId()))
00034     {
00035         const int answer = QMessageBox::warning(
00036             this,
00037             "Detect Climbs",
00038             "This activity contains manually edited climbs.\n\n"
00039             "Re-detecting climbs will overwrite all existing climbs,\n"
00040             "including manual changes.\n\nContinue?",
00041             QMessageBox::Cancel | QMessageBox::Yes,
00042             QMessageBox::Cancel);
00043         if (answer != QMessageBox::Yes)
00044             return;
00045     }
00046
00047     repo.deleteClimbsForActivity(m_controller->currentActivityId());
00048
00049     ClimbDetector::Config cfg;
00050     cfg.minLengthKm = m_climbMinLengthSpin ? m_climbMinLengthSpin->value() : 0.15;
00051     cfg.minElevationGainM = m_climbMinGainSpin ? m_climbMinGainSpin->value() : 10.0;
00052     cfg.minAverageGradient = m_climbMinGradientSpin ? m_climbMinGradientSpin->value() : 4.0;
00053     cfg.startGradient = m_climbStartGradientSpin ? m_climbStartGradientSpin->value() : 1.5;
00054     cfg.maxDipMeters = m_climbDipMetersSpin ? m_climbDipMetersSpin->value() : 10.0;
00055     cfg.maxDipDistanceMeters = m_climbDipDistanceSpin ? m_climbDipDistanceSpin->value() : 200.0;
00056     cfg.elevationSmoothingMeters = m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0;
00057
00058     m_detectedClimbs = ClimbDetector::detect(ride, cfg);
00059     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00060
00061     for (Climb& c : m_detectedClimbs)
00062     {
00063         ClimbRecord record;
00064         record.activityId = m_controller->currentActivityId();
00065         record.startSeconds = c.startSeconds;
00066         record.endSeconds = c.endSeconds;
00067         record.name = c.name;
00068         record.elevationGainM = c.elevationGainM;
00069         record.lengthKm = c.lengthKm;
00070         record.averageGradient = c.averageGradient;
00071         record.source = QStringLiteral("auto");
00072         record.locked = false;
00073         c.id = repo.insertClimb(record);
00074     }
00075
00076     refreshClimbViews();
00077     statusBar()->showMessage(
00078         QString("Detected %1 climb(s) and saved to database.").arg(m_detectedClimbs.size()),
00079         3000);
00080 }
00081
00082 void MainWindow::detectClimbsAndRefresh()
00083 {
00084     if (!m_climbsTable)
00085         return;
00086
00087     const auto& ride = m_controller->rideData();
00088     if (ride.records.empty())
00089     {
00090         m_detectedClimbs.clear();
00091         m_climbsTable->setRowCount(0);
00092         updateClimbQuarterCharts(nullptr);
00093         if (m_climbSummaryLabel)
00094             m_climbSummaryLabel->setText("Climb summary: select a climb");
00095         return;
00096     }
00097
00098     ClimbDetector::Config cfg;
00099     cfg.minLengthKm = m_climbMinLengthSpin ? m_climbMinLengthSpin->value() : 0.15;
00100     cfg.minElevationGainM = m_climbMinGainSpin ? m_climbMinGainSpin->value() : 10.0;
00101     cfg.minAverageGradient = m_climbMinGradientSpin ? m_climbMinGradientSpin->value() : 4.0;
00102     cfg.startGradient = m_climbStartGradientSpin ? m_climbStartGradientSpin->value() : 1.5;
00103     cfg.maxDipMeters = m_climbDipMetersSpin ? m_climbDipMetersSpin->value() : 10.0;
00104     cfg.maxDipDistanceMeters = m_climbDipDistanceSpin ? m_climbDipDistanceSpin->value() : 200.0;
00105     cfg.elevationSmoothingMeters = m_climbSmoothingSpin ? m_climbSmoothingSpin->value() : 50.0;
00106
00107     m_detectedClimbs = ClimbDetector::detect(ride, cfg);
00108     applyWeightMetricsToClimbs(m_detectedClimbs, resolveActiveActivityWeightKg());
00109
00110     refreshClimbViews();
00111 }
00112
00113 double MainWindow::resolveActiveActivityWeightKg() const
00114 {
00115     if (!m_dbManager.isOpen() || !m_controller || m_controller->currentActivityId() <= 0)
00116         return 0.0;

```

```

00117
00118     auto db = m_dbManager.database();
00119     ActivityRepository activityRepo(db);
00120     const Activity activity = activityRepo.getActivity(m_controller->currentActivityId());
00121
00122     if (activity.weightKg > 0.0)
00123         return activity.weightKg;
00124
00125     if (m_currentAthleteId <= 0)
00126         return 0.0;
00127
00128     const QString rawDate = !activity.startTime.isEmpty()
00129         ? activity.startTime.left(10)
00130         : activity.importedAt.left(10);
00131     const QDate activityDate = QDate::fromString(rawDate, Qt::ISODate);
00132     AthleteRepository athleteRepo(db);
00133     const QList<WeightEntry> weightHistory = athleteRepo.getWeightHistory(m_currentAthleteId);
00134
00135     if (weightHistory.isEmpty())
00136         return 0.0;
00137
00138     if (activityDate.isValid())
00139     {
00140         for (const WeightEntry& entry : weightHistory)
00141         {
00142             const QDate entryDate = QDate::fromString(entry.effectiveFrom, Qt::ISODate);
00143             if (!entryDate.isValid() || entryDate <= activityDate)
00144             {
00145                 if (entry.weightKg > 0.0)
00146                     return entry.weightKg;
00147             }
00148         }
00149     }
00150
00151     return weightHistory.front().weightKg > 0.0 ? weightHistory.front().weightKg : 0.0;
00152 }
00153
00154 void MainWindow::assignClimbWeightMetrics(Climb& climb, double riderWeightKg) const
00155 {
00156     climb.averageWattsPerKg = 0.0;
00157     climb.wattsPerKgRank.clear();
00158
00159     if (riderWeightKg <= 0.0 || climb.averagePower <= 0.0)
00160         return;
00161
00162     climb.averageWattsPerKg = climb.averagePower / riderWeightKg;
00163     const double wkg = climb.averageWattsPerKg;
00164
00165     if (wkg >= 4.5)
00166         climb.wattsPerKgRank = QStringLiteral("Elite");
00167     else if (wkg >= 3.8)
00168         climb.wattsPerKgRank = QStringLiteral("Very Strong");
00169     else if (wkg >= 3.2)
00170         climb.wattsPerKgRank = QStringLiteral("Strong");
00171     else if (wkg >= 2.5)
00172         climb.wattsPerKgRank = QStringLiteral("Moderate");
00173     else
00174         climb.wattsPerKgRank = QStringLiteral("Developing");
00175 }
00176
00177 void MainWindow::applyWeightMetricsToClimbs(std::vector<Climb>& climbs, double riderWeightKg) const
00178 {
00179     for (Climb& climb : climbs)
00180         assignClimbWeightMetrics(climb, riderWeightKg);
00181 }
00182
00183 void MainWindow::triggerDetectIntervals()
00184 {
00185     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00186     {
00187         QMessageBox::information(this, "Detect Intervals", "No activity loaded.");
00188         return;
00189     }
00190
00191     auto db = m_dbManager.database();
00192     IntervalRepository repo(db);
00193
00194     if (repo.hasLockedIntervals(m_controller->currentActivityId()))
00195     {
00196         const int answer = QMessageBox::warning(
00197             this,
00198             "Detect Intervals",
00199             "This activity contains manually edited intervals.\n\n"
00200             "Re-detecting intervals will overwrite all existing intervals,\n"
00201             "including manual changes.\n\nContinue?",
00202             QMessageBox::Cancel | QMessageBox::Yes,
00203             QMessageBox::Cancel);
    
```

```

00204         if (answer != QMessageBox::Yes)
00205             return;
00206     }
00207
00208     repo.deleteIntervalsForActivity(m_controller->currentActivityId());
00209
00210     IntervalDetector::Config cfg;
00211     cfg.ftp = m_controller->ftp();
00212     const std::vector<Interval> detected = IntervalDetector::detect(m_controller->rideData(), cfg);
00213
00214     const auto deduplicated = IntervalDetector::removeOverlappingAndDuplicates(detected);
00215
00216     for (const Interval& iv : deduplicated)
00217     {
00218         IntervalRecord record;
00219         record.activityId = m_controller->currentActivityId();
00220         record.startSample = static_cast<int>(std::round(iv.startSeconds));
00221         record.endSample = static_cast<int>(std::round(iv.endSeconds));
00222         record.name = QString::fromStdString(iv.label);
00223         record.type = QString::fromStdString(iv.label);
00224         record.avgPower = iv.averagePower;
00225         record.np = iv.normalizedPower;
00226         record.avgHr = iv.averageHeartRate;
00227         record.avgCadence = iv.averageCadence;
00228         record.source = QStringLiteral("auto");
00229         record.locked = false;
00230         repo.insertInterval(record);
00231     }
00232
00233     updateIntervals();
00234     statusBar()->showMessage(
00235         QString("Detected %1 interval(s) (after deduplication) and saved to
database.")
        .arg(deduplicated.size()),
00236         3000);
00237 }
00238
00239 void MainWindow::editCurrentActivityProperties()
00240 {
00241     if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00242         return;
00243
00244     auto db = m_dbManager.database();
00245     ActivityRepository repo(db);
00246     Activity activity = repo.getActivity(m_controller->currentActivityId());
00247     if (!activity.isValid())
00248         return;
00249
00250     EditActivityDialog dlg(activity, this);
00251     if (dlg.exec() != QDialog::Accepted)
00252         return;
00253
00254     const Activity updated = dlg.updatedActivity();
00255     if (!repo.updateActivityProperties(activity.id, updated))
00256     {
00257         QMessageBox::critical(this, "Error", "Failed to save activity properties.");
00258         return;
00259     }
00260
00261     statusBar()->showMessage("Activity properties updated.", 2500);
00262 }
00263
00264 void MainWindow::showAbout()
00265 {
00266     AboutDialog dlg(this);
00267     dlg.exec();
00268 }

```

## 8.136 MainWindowImport.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QSettings>
00006 #include <QFileDialog>
00007 #include <QFileInfo>
00008 #include <QMessageBox>
00009 #include <QMimeData>
00010 #include <QStatusBar>
00011 #include <QTimer>
00012 #include <QUrl>
00013 #include <QUuid>
00014
00015 #include "ActivityBrowser.h"

```

```

00016 #include "controllers/NavigationController.h"
00017
00018 void MainWindow::importFiles(const QStringList& filePaths)
00019 {
00020     importFilesInternal(filePaths, true, QStringLiteral("Manual import"));
00021 }
00022
00023 void MainWindow::importFilesInternal(const QStringList& filePaths,
00024                                     bool showResultDialog,
00025                                     const QString& sourceLabel)
00026 {
00027     const QStringList prepared = prepareImportFileBatch(filePaths);
00028     if (prepared.isEmpty())
00029         return;
00030
00031     enqueueImportBatch(prepared, showResultDialog, sourceLabel);
00032 }
00033
00034 QStringList MainWindow::prepareImportFileBatch(const QStringList& filePaths)
00035 {
00036     QStringList deduped = filePaths;
00037     deduped.removeDuplicates();
00038     if (deduped.isEmpty())
00039         return {};
00040
00041     if (!ensureImportReady())
00042     {
00043         updateImportAvailability();
00044         return {};
00045     }
00046
00047     if (!m_importQueue)
00048         return {};
00049
00050     return deduped;
00051 }
00052
00053 void MainWindow::enqueueImportBatch(const QStringList& filePaths,
00054                                     bool showResultDialog,
00055                                     const QString& sourceLabel)
00056 {
00057     if (!m_importQueue || filePaths.isEmpty())
00058         return;
00059
00060     const QString batchId = QUuid::createUuid().toString(QUuid::WithoutBraces);
00061     ImportBatchSummary summary;
00062     summary.queued = filePaths.size();
00063     summary.showResultDialog = showResultDialog;
00064     summary.sourceLabel = sourceLabel;
00065     m_importBatchSummaries.insert(batchId, summary);
00066
00067     const QString importSource = showResultDialog
00068         ? QStringLiteral("manual_import")
00069         : QStringLiteral("directory_import");
00070
00071     for (const QString& filePath : filePaths)
00072     {
00073         m_importQueue->enqueue(filePath,
00074                                QString::number(m_currentAthleteId),
00075                                batchId,
00076                                importSource);
00077     }
00078
00079     statusBar()->showMessage(importQueuedStatusMessage(filePaths.size()), 2500);
00080 }
00081
00082 QString MainWindow::importQueuedStatusMessage(int queuedCount) const
00083 {
00084     return QString("Queued %1 FIT file%2 for background import.")
00085         .arg(queuedCount)
00086         .arg(queuedCount == 1 ? QString() : QStringLiteral("s"));
00087 }
00088
00089 void MainWindow::scheduleActivityBrowserRefresh()
00090 {
00091     if (m_importRefreshTimer)
00092         m_importRefreshTimer->start();
00093 }
00094
00095 void MainWindow::finalizeImportBatches()
00096 {
00097     if (m_importBatchSummaries.isEmpty())
00098         return;
00099
00100     for (auto it = m_importBatchSummaries.begin(); it != m_importBatchSummaries.end(); ++it)
00101     {
00102         const ImportBatchSummary& summary = it.value();

```

```

00103     const QString label = summary.sourceLabel.isEmpty()
00104         ? QStringLiteral("Import")
00105         : summary.sourceLabel;
00106
00107     if (summary.showResultDialog)
00108     {
00109         QString text = QString("%1 files selected\n\nImported: %2\nDuplicates: %3\nFailed: %4")
00110             .arg(summary.queued)
00111             .arg(summary.imported)
00112             .arg(summary.duplicates)
00113             .arg(summary.failed);
00114
00115         if (!summary.errors.isEmpty())
00116             text += QString("\n\nErrors:\n%1").arg(summary.errors.join('\n'));
00117
00118         QMessageBox::information(this, "Import Results", text);
00119     }
00120     else
00121     {
00122         statusBar()->showMessage(
00123             QString("%1: imported %2, duplicates %3, failed %4")
00124                 .arg(label)
00125                 .arg(summary.imported)
00126                 .arg(summary.duplicates)
00127                 .arg(summary.failed),
00128             5000);
00129     }
00130 }
00131
00132 m_importBatchSummaries.clear();
00133 }
00134
00135 QStringList MainWindow::fitFilesFromMimeData(const QMimeData* mimeData) const
00136 {
00137     QStringList files;
00138     if (!mimeData || !mimeData->hasUrls())
00139         return files;
00140
00141     for (const QUrl& url : mimeData->urls())
00142     {
00143         if (!url.isLocalFile())
00144             continue;
00145         const QString path = url.toLocalFile();
00146         if (QFileInfo(path).suffix().compare("fit", Qt::CaseInsensitive) == 0)
00147             files << path;
00148     }
00149
00150     files.removeDuplicates();
00151     return files;
00152 }
00153
00154 void MainWindow::importActivities()
00155 {
00156     const QStringList files = QFileDialog::getOpenFileNames(
00157         this, "Import Activities",
00158         m_lastOpenDir,
00159         "FIT Files (*.fit);;All Files (*)");
00160
00161     if (files.isEmpty()) return;
00162     m_lastOpenDir = QFileInfo(files.front()).absolutePath();
00163     importFiles(files);
00164 }
00165
00166 void MainWindow::previewFitFile()
00167 {
00168     const QString fileName = QFileDialog::getOpenFileName(
00169         this, "Preview FIT File",
00170         m_lastOpenDir,
00171         "FIT Files (*.fit);;All Files (*)");
00172     if (fileName.isEmpty()) return;
00173     m_lastOpenDir = QFileInfo(fileName).absolutePath();
00174
00175     QString err;
00176     if (!m_controller->loadFile(fileName, err))
00177         QMessageBox::critical(this, "Preview Failed", err);
00178 }
00179
00180 void MainWindow::onActivityImported(int activityId)
00181 {
00182     QSettings("Fitlyzer", "FitlyzerC").setValue("lastActivityId", activityId);
00183     if (!m_firstLaunchCompleted)
00184     {
00185         m_firstLaunchCompleted = true;
00186         QSettings("Fitlyzer", "FitlyzerC").setValue("firstLaunchCompleted", true);
00187     }
00188
00189     if (m_activityBrowser)

```

```

00190     {
00191         m_activityBrowser->refresh(m_currentAthleteId);
00192         if (m_navigationController)
00193             m_navigationController->navigateTo(NavigationController::Page::Activities);
00194         else if (m_pageStack)
00195             m_pageStack->setCurrentIndex(0);
00196     }
00197
00198     updateStatusBarInfo();
00199     hideWelcomeScreen();
00200 }

```

## 8.137 MainWindowIntervals.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QBrush>
00006 #include <QColor>
00007 #include <QDateTime>
00008 #include <QFont>
00009 #include <QMessageBox>
00010 #include <QSignalBlocker>
00011 #include <QStatusBar>
00012 #include <QTableWidgetItem>
00013
00014 #include "charts/RideChartWidget.h"
00015 #include "core/settings/DateFormatter.h"
00016 #include "database/ActivityRepository.h"
00017 #include "database/IntervalRepository.h"
00018 #include "fit/RideData.h"
00019 #include "maps/MapRenderer.h"
00020
00021 #include <algorithm>
00022 #include <cmath>
00023 #include <initializer_list>
00024 #include <vector>
00025
00026 namespace
00027 {
00028     static QString fmtDur(double seconds)
00029     {
00030         const int total = static_cast<int>(seconds);
00031         const int h = total / 3600;
00032         const int m = (total % 3600) / 60;
00033         const int s = total % 60;
00034         return h > 0
00035             ? QString("%1:%2:%3")
00036               .arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'))
00037             : QString("%1:%2")
00038               .arg(m).arg(s, 2, 10, QChar('0'));
00039     }
00040
00041     static constexpr int kIntervalStartRole = Qt::UserRole + 1;
00042     static constexpr int kIntervalEndRole = Qt::UserRole + 2;
00043 }
00044
00045 void MainWindow::onIntervalSelectionChanged()
00046 {
00047     if (!m_intervalsTable)
00048         return;
00049
00050     const int row = m_intervalsTable->currentRow();
00051     if (row < 0)
00052     {
00053         updateIntervalRowStyles();
00054         clearIntervalSummary();
00055         return;
00056     }
00057
00058     auto* firstItem = m_intervalsTable->item(row, 0);
00059     if (!firstItem)
00060     {
00061         updateIntervalRowStyles();
00062         clearIntervalSummary();
00063         return;
00064     }
00065
00066     const QVariant startVar = firstItem->data(kIntervalStartRole);
00067     const QVariant endVar = firstItem->data(kIntervalEndRole);
00068     if (!startVar.isValid() || !endVar.isValid())
00069     {
00070         updateIntervalRowStyles();

```

```

00071         clearIntervalSummary();
00072         return;
00073     }
00074
00075     const double startSeconds = startVar.toDouble();
00076     const double endSeconds = endVar.toDouble();
00077
00078     updateIntervalRowStyles();
00079     updateIntervalSummary(startSeconds, endSeconds);
00080
00081     if (m_followIntervalSelectionCheck && m_followIntervalSelectionCheck->isChecked())
00082         navigateToInterval(startSeconds, endSeconds, false);
00083 }
00084
00085 void MainWindow::onIntervalRowDoubleClicked(QTableWidgetItem* item)
00086 {
00087     if (!item || !m_intervalsTable)
00088         return;
00089
00090     const int row = item->row();
00091     auto* firstItem = m_intervalsTable->item(row, 0);
00092     if (!firstItem)
00093         return;
00094
00095     const QVariant startVar = firstItem->data(kIntervalStartRole);
00096     const QVariant endVar = firstItem->data(kIntervalEndRole);
00097     if (!startVar.isValid() || !endVar.isValid())
00098         return;
00099
00100     if (m_followIntervalSelectionCheck && m_followIntervalSelectionCheck->isChecked())
00101         navigateToInterval(startVar.toDouble(), endVar.toDouble(), true);
00102 }
00103
00104 void MainWindow::navigateToInterval(double startSeconds, double endSeconds, bool exactZoom)
00105 {
00106     if (endSeconds <= startSeconds)
00107         return;
00108
00109     const double duration = endSeconds - startSeconds;
00110     const double padding = exactZoom ? 0.0 : std::max(duration * 0.10, 5.0);
00111     const double rangeStart = std::max(0.0, startSeconds - padding);
00112     const double rangeEnd = endSeconds + padding;
00113
00114     const std::initializer_list<RideChartWidget*> all =
00115     { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00116     for (auto* chart : all)
00117     {
00118         if (!chart)
00119             continue;
00120         chart->setXRange(rangeStart / 60.0, rangeEnd / 60.0);
00121         chart->setCrosshair(startSeconds / 60.0);
00122     }
00123
00124     if (m_mapRenderer)
00125     {
00126         m_mapRenderer->setVisibleTimeRange(startSeconds, endSeconds);
00127         m_mapRenderer->fitToVisibleRange();
00128         m_mapRenderer->setCurrentTime(startSeconds);
00129     }
00130
00131     const QString modeLabel = exactZoom ? "exact" : "padded";
00132     statusBar()->showMessage(
00133         QString("Interval inspection: %1 zoom (%2-%3)")
00134             .arg(modeLabel)
00135             .arg(fmtDur(rangeStart))
00136             .arg(fmtDur(rangeEnd)),
00137         2500);
00138 }
00139
00140 void MainWindow::selectIntervalRowOffset(int delta)
00141 {
00142     if (!m_intervalsTable)
00143         return;
00144
00145     const int rows = m_intervalsTable->rowCount();
00146     if (rows <= 0)
00147         return;
00148
00149     int row = m_intervalsTable->currentRow();
00150     if (row < 0)
00151         row = 0;
00152     else
00153         row = std::clamp(row + delta, 0, rows - 1);
00154
00155     m_intervalsTable->setCurrentCell(row, 0);
00156 }
00157

```

```

00158 void MainWindow::updateIntervalRowStyles()
00159 {
00160     if (!m_intervalsTable)
00161         return;
00162
00163     const int selectedRow = m_intervalsTable->currentRow();
00164     for (int row = 0; row < m_intervalsTable->rowCount(); ++row)
00165     {
00166         for (int col = 0; col < m_intervalsTable->columnCount(); ++col)
00167         {
00168             auto* item = m_intervalsTable->item(row, col);
00169             if (!item)
00170                 continue;
00171
00172             QFont font = item->font();
00173             font.setBold(row == selectedRow);
00174             item->setFont(font);
00175         }
00176     }
00177 }
00178
00179 void MainWindow::clearIntervalSummary()
00180 {
00181     if (m_intervalSummaryLabel)
00182         m_intervalSummaryLabel->setText("Interval summary: select an interval");
00183 }
00184
00185 void MainWindow::updateIntervalSummary(double startSeconds, double endSeconds)
00186 {
00187     if (!m_intervalSummaryLabel || endSeconds <= startSeconds)
00188     {
00189         clearIntervalSummary();
00190         return;
00191     }
00192
00193     const auto& records = m_controller->rideData().records;
00194     double sumPower = 0.0;
00195     int countPower = 0;
00196     double sumHr = 0.0;
00197     int countHr = 0;
00198     double sumCadence = 0.0;
00199     int countCadence = 0;
00200     double sumSpeed = 0.0;
00201     int countSpeed = 0;
00202     double elevationGain = 0.0;
00203     std::vector<double> powerValues;
00204
00205     bool havePreviousAltitude = false;
00206     double previousAltitude = 0.0;
00207
00208     for (const RideRecord& record : records)
00209     {
00210         if (record.elapsedSeconds < startSeconds || record.elapsedSeconds > endSeconds)
00211             continue;
00212
00213         if (record.hasPower)
00214         {
00215             sumPower += record.power;
00216             ++countPower;
00217             powerValues.push_back(record.power);
00218         }
00219         if (record.hasHeartRate)
00220         {
00221             sumHr += record.heartRate;
00222             ++countHr;
00223         }
00224         if (record.hasCadence)
00225         {
00226             sumCadence += record.cadence;
00227             ++countCadence;
00228         }
00229         if (record.hasSpeed)
00230         {
00231             sumSpeed += record.speed;
00232             ++countSpeed;
00233         }
00234         if (record.hasAltitude)
00235         {
00236             if (havePreviousAltitude)
00237             {
00238                 const double delta = record.altitude - previousAltitude;
00239                 if (delta > 0.0)
00240                     elevationGain += delta;
00241             }
00242             previousAltitude = record.altitude;
00243             havePreviousAltitude = true;
00244         }
00245     }

```



```

00245     }
00246
00247     double np = 0.0;
00248     if (!powerValues.empty())
00249     {
00250         double sum4 = 0.0;
00251         for (double p : powerValues)
00252             sum4 += std::pow(p, 4.0);
00253         np = std::pow(sum4 / static_cast<double>(powerValues.size()), 0.25);
00254     }
00255
00256     const double avgPower = countPower > 0 ? (sumPower / static_cast<double>(countPower)) : 0.0;
00257     const double avgHr = countHr > 0 ? (sumHr / static_cast<double>(countHr)) : 0.0;
00258     const double avgCadence = countCadence > 0 ? (sumCadence / static_cast<double>(countCadence)) :
0.0;
00259     const double avgSpeed = countSpeed > 0 ? (sumSpeed / static_cast<double>(countSpeed)) : 0.0;
00260
00261     m_intervalSummaryLabel->setText(
00262         QString("Interval %1 | Avg P %2 | NP %3 | HR %4 | Cad %5 | Speed %6 | Elev +%7")
00263             .arg(fmtDur(std::max(0.0, endSeconds - startSeconds)))
00264             .arg(avgPower > 0.0 ? QString("%1 W").arg(avgPower, 0, 'f', 0) : QString("\xE2\x80\x94"))
00265             .arg(np > 0.0 ? QString("%1 W").arg(np, 0, 'f', 0) : QString("\xE2\x80\x94"))
00266             .arg(avgHr > 0.0 ? QString("%1 bpm").arg(avgHr, 0, 'f', 0) : QString("\xE2\x80\x94"))
00267             .arg(avgCadence > 0.0 ? QString("%1 rpm").arg(avgCadence, 0, 'f', 0) :
QString("\xE2\x80\x94"))
00268             .arg(avgSpeed > 0.0 ? QString("%1 km/h").arg(avgSpeed, 0, 'f', 1) :
QString("\xE2\x80\x94"))
00269             .arg(QString("%1 m").arg(elevationGain, 0, 'f', 0)));
00270 }
00271
00272 void MainWindow::updateIntervals()
00273 {
00274     if (!m_intervalsTable)
00275         return;
00276
00277     const int previousRow = m_intervalsTable->currentRow();
00278     QSignalBlocker selectionBlocker(m_intervalsTable);
00279
00280     QList<IntervalRecord> records;
00281     if (m_dbManager.isOpen() && m_controller->currentActivityId() > 0)
00282     {
00283         auto db = m_dbManager.database();
00284         IntervalRepository repo(db);
00285         records = repo.listIntervalsForActivity(m_controller->currentActivityId());
00286     }
00287
00288     const auto populateIntervalsFromController = [this]()
00289     {
00290         const auto& ivs = m_controller->intervals();
00291         m_intervalsTable->setRowCount(static_cast<int>(ivs.size()));
00292
00293         for (int row = 0; row < static_cast<int>(ivs.size()); ++row)
00294         {
00295             const auto& iv = ivs[static_cast<size_t>(row)];
00296
00297             auto mkItem = [](const QString& text, Qt::Alignment align = Qt::AlignCenter)
00298             {
00299                 auto* item = new QTableWidgetItem(text);
00300                 item->setTextAlignment(align);
00301                 return item;
00302             };
00303
00304             auto* typeItem = mkItem(QString::fromStdString(iv.label));
00305             typeItem->setBackground(
00306                 iv.label == "Work"
00307                     ? QBrush(QColor(60, 110, 200, 100))
00308                     : QBrush(QColor(60, 170, 60, 100)));
00309             typeItem->setData(kIntervalStartRole, iv.startSeconds);
00310             typeItem->setData(kIntervalEndRole, iv.endSeconds);
00311             m_intervalsTable->setItem(row, 0, typeItem);
00312
00313             m_intervalsTable->setItem(row, 1, mkItem(fmtDur(iv.startSeconds)));
00314             m_intervalsTable->setItem(row, 2, mkItem(fmtDur(iv.durationSeconds)));
00315             m_intervalsTable->setItem(row, 3, mkItem(iv.averagePower > 0
00316                 ? QString("%1 W").arg(iv.averagePower, 0, 'f', 0) : "\xE2\x80\x94"));
00317             m_intervalsTable->setItem(row, 4, mkItem(iv.normalizedPower > 0
00318                 ? QString("%1 W").arg(iv.normalizedPower, 0, 'f', 0) : "\xE2\x80\x94"));
00319             const double intervalIf = (m_controller->ftp() > 0.0 && iv.normalizedPower > 0.0)
00320                 ? (iv.normalizedPower / m_controller->ftp()) : 0.0;
00321             m_intervalsTable->setItem(row, 5, mkItem(intervalIf > 0.0
00322                 ? QString::number(intervalIf, 'f', 2) : "\xE2\x80\x94"));
00323             m_intervalsTable->setItem(row, 6, mkItem(iv.maximumPower > 0
00324                 ? QString("%1 W").arg(iv.maximumPower, 0, 'f', 0) : "\xE2\x80\x94"));
00325             m_intervalsTable->setItem(row, 7, mkItem(iv.averageHeartRate > 0
00326                 ? QString("%1 bpm").arg(iv.averageHeartRate, 0, 'f', 0) : "\xE2\x80\x94"));
00327         }
00328     };

```

```

00329
00330     const auto populateIntervalsFromRecords = [this, &records]()
00331     {
00332         m_intervalsTable->setRowCount(records.size());
00333         for (int row = 0; row < records.size(); ++row)
00334         {
00335             const IntervalRecord& r = records[row];
00336             auto* typeItem = new QTableWidgetItem(r.type);
00337             typeItem->setTextAlignment(Qt::AlignCenter);
00338             typeItem->setData(kIntervalStartRole, static_cast<double>(r.startSample));
00339             typeItem->setData(kIntervalEndRole, static_cast<double>(r.endSample));
00340             m_intervalsTable->setItem(row, 0, typeItem);
00341             m_intervalsTable->setItem(row, 1, new QTableWidgetItem(fmtDur(r.startSample)));
00342             m_intervalsTable->setItem(row, 2, new QTableWidgetItem(fmtDur(std::max(0, r.endSample -
r.startSample))));
00343             m_intervalsTable->setItem(row, 3, new QTableWidgetItem(r.avgPower > 0 ? QString("%1
W").arg(r.avgPower, 0, 'f', 0) : "\xE2\x80\x94"));
00344             m_intervalsTable->setItem(row, 4, new QTableWidgetItem(r.np > 0 ? QString("%1
W").arg(r.np, 0, 'f', 0) : "\xE2\x80\x94"));
00345             const double intervalIf = (m_controller->ftp() > 0.0 && r.np > 0.0)
? (r.np / m_controller->ftp()) : 0.0;
00346             m_intervalsTable->setItem(row, 5, new QTableWidgetItem(intervalIf > 0.0 ?
QString::number(intervalIf, 'f', 2) : "\xE2\x80\x94"));
00347             m_intervalsTable->setItem(row, 6, new QTableWidgetItem("\xE2\x80\x94"));
00348             m_intervalsTable->setItem(row, 7, new QTableWidgetItem(r.avgHr > 0 ? QString("%1
bpm").arg(r.avgHr, 0, 'f', 0) : "\xE2\x80\x94"));
00349         }
00350     };
00351 };
00352
00353 if (records.isEmpty())
00354     populateIntervalsFromController();
00355 else
00356     populateIntervalsFromRecords();
00357
00358 m_intervalsTable->resizeColumnsToContents();
00359 finalizeIntervalSelectionAndSummary(previousRow);
00360
00361 updateLapsSummaryFromCurrentActivity();
00362 updateActivityNotesFromCurrentActivity();
00363 }
00364
00365 void MainWindow::updateLapsSummaryFromCurrentActivity()
00366 {
00367     if (!m_lapsTable)
00368         return;
00369
00370     m_lapsTable->setRowCount(1);
00371     m_lapsTable->setItem(0, 0, new QTableWidgetItem("1"));
00372     m_lapsTable->setItem(0, 1, new QTableWidgetItem("0:00"));
00373     m_lapsTable->setItem(0, 2, new
QTableWidgetItem(fmtDur(m_controller->statistics().durationSeconds)));
00374     m_lapsTable->resizeColumnsToContents();
00375 }
00376
00377 void MainWindow::finalizeIntervalSelectionAndSummary(int previousRow)
00378 {
00379     if (!m_intervalsTable)
00380         return;
00381
00382     if (m_intervalsTable->rowCount() > 0)
00383     {
00384         const int targetRow = std::clamp(previousRow >= 0 ? previousRow : 0, 0,
m_intervalsTable->rowCount() - 1);
00385         m_intervalsTable->setCurrentCell(targetRow, 0);
00386     }
00387
00388     updateIntervalRowStyles();
00389     if (m_intervalsTable->currentRow() >= 0)
00390         onIntervalSelectionChanged();
00391     else
00392         clearIntervalSummary();
00393 }
00394
00395 void MainWindow::updateActivityNotesFromCurrentActivity()
00396 {
00397     if (!m_activityNotesView || !m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00398         return;
00399
00400     auto db = m_dbManager.database();
00401     ActivityRepository repo(db);
00402     const Activity activity = repo.getActivity(m_controller->currentActivityId());
00403
00404     const QDateTime activityDateUtc = QDateTime::fromString(activity.startTime, Qt::ISODate);
00405     const QDateTime importDateUtc = QDateTime::fromString(activity.importedAt, Qt::ISODate);
00406     const QString activityDateText = activityDateUtc.isValid()
? DateFormatter::formatDateTime(activityDateUtc.toLocalTime())
: QStringLiteral("-");
00407
00408

```

```

00409     const QString importDateText = importDateUtc.isValid()
00410     ? DateFormatter::formatDateTime(importDateUtc.toLocalTime())
00411     : QStringLiteral("-");
00412
00413     m_activityNotesView->setPlainText (
00414         QString("Activity Date: %1\nImported: %2\n\nNotes:\n%3\n\nWeather: %4\nTemperature: %5 C\nRPE:
%6\nFatigue: %7\nSleep: %8 h\nBike: %9\nEquipment: %10")
00415         .arg(activityDateText)
00416         .arg(importDateText)
00417         .arg(activity.notes)
00418         .arg(activity.weather)
00419         .arg(activity.temperature, 0, 'f', 1)
00420         .arg(activity.rpe)
00421         .arg(activity.fatigue)
00422         .arg(activity.sleepHours, 0, 'f', 1)
00423         .arg(activity.bike)
00424         .arg(activity.equipment));
00425 }

```

## 8.138 MainWindowReadiness.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QDialog>
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QMessageBox>
00009 #include <QSettings>
00010 #include <QStandardPaths>
00011 #include <QStatusBar>
00012
00013 #include "AthleteDialog.h"
00014 #include "WelcomeWidget.h"
00015 #include "database/ActivityRepository.h"
00016 #include "database/AthleteRepository.h"
00017 #include "platform/Platform.h"
00018
00019 bool MainWindow::canImportActivities() const
00020 {
00021     return m_dbManager.isOpen() && m_currentAthleteId > 0;
00022 }
00023
00024 bool MainWindow::createOrOpenDefaultDatabase()
00025 {
00026     const QString path = defaultDatabasePathForAutoCreate();
00027     if (path.isEmpty())
00028         return false;
00029
00030     const QFileInfo info(path);
00031     QDir dir = info.dir();
00032     if (!dir.exists() && !dir.mkpath("."))
00033     {
00034         QMessageBox::critical(this, "Database Error",
00035             "Could not create default database folder:\n" + dir.absolutePath());
00036         return false;
00037     }
00038
00039     QString err;
00040     const bool ok = info.exists()
00041         ? m_dbManager.open(path, &err)
00042         : m_dbManager.create(path, &err);
00043     if (!ok)
00044     {
00045         QMessageBox::critical(this, "Database Error",
00046             QString("Could not open default database:\n%1\n%2").arg(path, err));
00047         return false;
00048     }
00049
00050     m_controller->setDatabaseManager(&m_dbManager);
00051     m_lastOpenDir = info.absolutePath();
00052     if (m_analysisQueue)
00053         m_analysisQueue->setDatabasePath(path);
00054     if (m_importQueue)
00055         m_importQueue->setDatabasePath(path);
00056
00057     QSettings settings("Fitlyzer", "FitlyzerC");
00058     settings.setValue("lastDatabase", path);
00059     addRecentDatabase(path);
00060     updateRecentDatabaseMenu();
00061     return true;
00062 }
00063

```

```

00064 bool MainWindow::ensureDatabase()
00065 {
00066     if (m_dbManager.isOpen())
00067         return true;
00068
00069     if (!createOrOpenDefaultDatabase())
00070         return false;
00071
00072     refreshAthleteSelector();
00073     updateAthleteLabel();
00074     updateImportAvailability();
00075     updateStatsLabel();
00076     updateStatusBarInfo();
00077     updateWelcomeScreenVisibility();
00078
00079     statusBar()->showMessage("Created default database for import.", 3500);
00080     return true;
00081 }
00082
00083 bool MainWindow::ensureAthlete()
00084 {
00085     if (!m_dbManager.isOpen())
00086         return false;
00087
00088     auto db = m_dbManager.database();
00089     AthleteRepository repo(db);
00090     QList<Athlete> athletes = repo.listAthletes();
00091
00092     bool createdDefaultAthlete = false;
00093     if (athletes.isEmpty())
00094     {
00095         Athlete athlete;
00096         athlete.firstName = "Default";
00097         athlete.lastName = "Athlete";
00098         const int createdId = repo.insertAthlete(athlete);
00099         if (createdId <= 0)
00100         {
00101             QMessageBox::critical(this, "Athlete Error",
00102                 "Could not create a default athlete for import.");
00103             return false;
00104         }
00105         athletes = repo.listAthletes();
00106         createdDefaultAthlete = true;
00107     }
00108
00109     bool hasCurrent = false;
00110     for (const Athlete& athlete : athletes)
00111     {
00112         if (athlete.id == m_currentAthleteId)
00113         {
00114             hasCurrent = true;
00115             break;
00116         }
00117     }
00118
00119     if (!hasCurrent)
00120     {
00121         m_currentAthleteId = athletes.first().id;
00122         m_currentAthleteName = athletes.first().fullName();
00123         m_controller->setCurrentAthlete(m_currentAthleteId);
00124         QSettings("Fitlyzer", "FitlyzerC").setValue("currentAthleteId", m_currentAthleteId);
00125     }
00126
00127     refreshAthleteSelector();
00128     updateAthleteLabel();
00129     updateImportAvailability();
00130     updateStatsLabel();
00131     updateStatusBarInfo();
00132
00133     if (createdDefaultAthlete)
00134     {
00135         const auto ans = QMessageBox::question(
00136             this,
00137             "Default Athlete Created",
00138             "A default athlete was created. Rename athlete now?",
00139             QMessageBox::Yes | QMessageBox::No,
00140             QMessageBox::Yes);
00141
00142         if (ans == QMessageBox::Yes)
00143         {
00144             AthleteDialog dlg(repo, m_currentAthleteId, this);
00145             if (dlg.exec() == QDialog::Accepted)
00146             {
00147                 const Athlete athlete = repo.getAthlete(m_currentAthleteId);
00148                 m_currentAthleteName = athlete.fullName();
00149                 refreshAthleteSelector();
00150                 updateAthleteLabel();

```

```

00151         updateStatusBarInfo();
00152     }
00153 }
00154 }
00155
00156     return m_currentAthleteId > 0;
00157 }
00158
00159 bool MainWindow::ensureImportReady()
00160 {
00161     return ensureDatabase() && ensureAthlete();
00162 }
00163
00164 QString MainWindow::defaultDatabasePathForAutoCreate() const
00165 {
00166     const QString documentsDir = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation);
00167     const QString baseDir = documentsDir.isEmpty()
00168         ? Platform::defaultDatabaseDirectory()
00169         : documentsDir;
00170     return QDir(baseDir).filePath("Fitlyzer/default.fitlyzerdb");
00171 }
00172
00173 void MainWindow::updateImportAvailability()
00174 {
00175     const bool enabled = canImportActivities();
00176     if (m_importAct)
00177         m_importAct->setEnabled(enabled);
00178     if (m_toolbarImportAct)
00179         m_toolbarImportAct->setEnabled(enabled);
00180
00181     if (enabled)
00182     {
00183         statusBar()->showMessage("Ready to import activities.", 2000);
00184     }
00185     else if (!m_dbManager.isOpen())
00186     {
00187         statusBar()->showMessage("Open or create a database to import activities.", 3000);
00188     }
00189     else
00190     {
00191         statusBar()->showMessage("Select an athlete to import activities.", 3000);
00192     }
00193 }
00194
00195 void MainWindow::showWelcomeScreen()
00196 {
00197     if (m_welcomeWidget)
00198         m_welcomeWidget->setVisible(true);
00199     if (m_pageStack)
00200         m_pageStack->setVisible(false);
00201 }
00202
00203 void MainWindow::hideWelcomeScreen()
00204 {
00205     if (m_welcomeWidget)
00206         m_welcomeWidget->setVisible(false);
00207     if (m_pageStack)
00208         m_pageStack->setVisible(true);
00209 }
00210
00211 int MainWindow::activityCount() const
00212 {
00213     if (!m_dbManager.isOpen())
00214         return 0;
00215
00216     auto db = m_dbManager.database();
00217     ActivityRepository repo(db);
00218     return repo.listActivities(-1).size();
00219 }
00220
00221 void MainWindow::updateWelcomeScreenVisibility()
00222 {
00223     if (!m_welcomeWidget)
00224         return;
00225
00226     m_welcomeWidget->setFirstLaunch(!m_firstLaunchCompleted);
00227     if (activityCount() == 0)
00228         showWelcomeScreen();
00229     else
00230         hideWelcomeScreen();
00231 }

```

## 8.139 MainWindowStatus.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QDate>
00006 #include <QFileInfo>
00007
00008 #include "AthleteHeaderWidget.h"
00009 #include "core/settings/DateFormatter.h"
00010 #include "database/ActivityRepository.h"
00011 #include "controllers/ActivityViewController.h"
00012
00013 #include <cmath>
00014
00015 namespace
00016 {
00017     QString formatActivityDateLabel(const QString& isoDate)
00018     {
00019         const QDate date = QDate::fromString(isoDate, Qt::ISODate);
00020         if (!date.isValid())
00021             return QStringLiteral("-");
00022
00023         const QDate today = QDate::currentDate();
00024         if (date == today)
00025             return QStringLiteral("Today");
00026         if (date == today.addDays(-1))
00027             return QStringLiteral("Yesterday");
00028
00029         return DateFormatter::formatDate(date);
00030     }
00031 } // namespace
00032
00033 void MainWindow::updateStatusBarInfo()
00034 {
00035     if (m_dbStatusLabel)
00036         m_dbStatusLabel->setText (
00037             m_dbManager.isOpen()
00038             ? QString("Database: %1").arg(QFileInfo(m_dbManager.path()).fileName())
00039             : "Database: none");
00040
00041     if (m_athleteStatusLabel)
00042         m_athleteStatusLabel->setText (
00043             m_currentAthleteName.isEmpty()
00044             ? "Athlete: none"
00045             : QString("Athlete: %1").arg(m_currentAthleteName));
00046
00047     int activityCount = 0;
00048     if (m_dbManager.isOpen())
00049     {
00050         auto db = m_dbManager.database();
00051         ActivityRepository repo(db);
00052         activityCount = repo.listActivities(m_currentAthleteId).size();
00053     }
00054
00055     if (m_activityCountLabel)
00056         m_activityCountLabel->setText (QString("Activities: %1").arg(activityCount));
00057
00058     // Sync athlete context to activity controller for status display
00059     syncAthleteContextToControllers();
00060 }
00061
00062 void MainWindow::syncAthleteContextToControllers()
00063 {
00064     if (m_activityViewController)
00065     {
00066         m_activityViewController->setAthleteContext(m_currentAthleteId, m_currentAthleteName);
00067         m_activityViewController->updateStatusBarInformation();
00068     }
00069 }
00070
00071 void MainWindow::updateStatsLabel()
00072 {
00073     if (!m_athleteHeader)
00074         return;
00075
00076     QString athleteName = m_currentAthleteName.isEmpty() ? "No athlete selected" :
00077         m_currentAthleteName;
00078     int activityCount = 0;
00079     QString lastActivity = "-";
00080
00081     if (m_dbManager.isOpen() && m_currentAthleteId > 0)
00082     {
00083         auto db = m_dbManager.database();
00084         ActivityRepository actRepo(db);

```

```

00084         const QList<Activity> activities = actRepo.listActivities(m_currentAthleteId);
00085         activityCount = activities.size();
00086         if (!activities.isEmpty())
00087         {
00088             const QString rawDate = !activities.first().startTime.isEmpty()
00089                 ? activities.first().startTime.left(10)
00090                 : activities.first().importedAt.left(10);
00091             lastActivity = formatActivityDateLabel(rawDate);
00092         }
00093     }
00094
00095     const int ftpWatts = static_cast<int>(m_controller->ftp());
00096     const double estimatedFtp = estimatedFtpFromCurrentRide();
00097     const int estimatedFtpRounded = estimatedFtp > 0.0 ? static_cast<int>(std::round(estimatedFtp)) :
0;
00098     const int ftpDelta = estimatedFtpRounded - ftpWatts;
00099     m_athleteHeader->setSummary(athleteName, ftpWatts, activityCount, lastActivity);
00100
00101     if (m_useEstimatedFtpButton)
00102         m_useEstimatedFtpButton->setEnabled(m_currentAthleteId > 0 && estimatedFtpRounded > 0);
00103
00104     const QString ftpSummary = estimatedFtpRounded > 0
00105         ? QString("FTP %1 Est %2 \u0394%3")
00106           .arg(ftpWatts)
00107           .arg(estimatedFtpRounded)
00108           .arg(ftpDelta >= 0 ? QString("+%1").arg(ftpDelta) : QString::number(ftpDelta))
00109         : QString();
00110
00111     const QString rideText = QString("NP %1 W IF %2 TSS %3 VI %4 EF %5%6")
00112         .arg(m_controller->normalizedPower(), 0, 'f', 0)
00113         .arg(m_controller->intensityFactor(), 0, 'f', 2)
00114         .arg(m_controller->trainingStressScore(), 0, 'f', 0)
00115         .arg(m_controller->variabilityIndex(), 0, 'f', 2)
00116         .arg(m_controller->efficiencyFactor(), 0, 'f', 2)
00117         .arg(ftpSummary.isEmpty() ? QString() : QString(" ") + ftpSummary);
00118     m_athleteHeader->setRideMetrics(rideText);
00119 }

```

## 8.140 MainWindowUi.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QAction>
00006 #include <QComboBox>
00007 #include <QEvent>
00008 #include <QKeySequence>
00009 #include <QLabel>
00010 #include <QMenu>
00011 #include <QStatusBar>
00012 #include <QStyle>
00013 #include <QToolBar>
00014 #include <QToolButton>
00015
00016 #include "ActivityBrowser.h"
00017 #include "ImportStatusWidget.h"
00018 #include "NavigationSidebar.h"
00019
00020 #include <cmath>
00021
00022 void MainWindow::buildToolBar()
00023 {
00024     // Global application toolbar.
00025     // Contextual actions (per-page) live in each page widget's own toolbar.
00026     auto* tb = addToolBar("Main");
00027     tb->setMovable(false);
00028     tb->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00029
00030     // Import
00031     m_toolbarImportAct = new QAction("Import", this);
00032     m_toolbarImportAct->setIcon(style()->standardIcon(QStyle::SP_DialogOpenButton));
00033     m_toolbarImportAct->setEnabled(false);
00034     m_toolbarImportAct->setToolTip("Import FIT files (Ctrl+O)");
00035     connect(m_toolbarImportAct, &QAction::triggered, this, &MainWindow::importActivities);
00036     tb->addAction(m_toolbarImportAct);
00037
00038     tb->addSeparator();
00039
00040     tb->addSeparator();
00041
00042     // Analyze dropdown
00043     auto* analyzeMenu = new QMenu(tb);
00044

```

```

00045     auto* detectClimbsAct = new QAction("Detect Climbs", analyzeMenu);
00046     connect(detectClimbsAct, &QAction::triggered, this, &MainWindow::triggerDetectClimbs);
00047     analyzeMenu->addAction(detectClimbsAct);
00048
00049     auto* detectIntervalsAct = new QAction("Detect Intervals", analyzeMenu);
00050     connect(detectIntervalsAct, &QAction::triggered, this, &MainWindow::triggerDetectIntervals);
00051     analyzeMenu->addAction(detectIntervalsAct);
00052
00053     auto* analyzeBtn = new QToolButton(tb);
00054     analyzeBtn->setText("Analyze");
00055     analyzeBtn->setIcon(style()->standardIcon(QStyle::SP_BrowserReload));
00056     analyzeBtn->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00057     analyzeBtn->setPopupMode(QToolButton::MenuButtonPopup);
00058     analyzeBtn->setMenu(analyzeMenu);
00059     connect(analyzeBtn, &QToolButton::clicked, this, &MainWindow::triggerDetectClimbs);
00060     tb->addWidget(analyzeBtn);
00061
00062     // Compare
00063     auto* compareAct = new QAction("Compare", this);
00064     compareAct->setIcon(style()->standardIcon(QStyle::SP_ArrowUp));
00065     compareAct->setToolTip("Open power comparison workflows on the Power page");
00066     connect(compareAct, &QAction::triggered, this, [this]
00067     {
00068         if (m_navigationSidebar)
00069             m_navigationSidebar->setCurrentPage(NavigationSidebar::Page::Power);
00070     });
00071     tb->addAction(compareAct);
00072
00073     // Export
00074     auto* exportAct = new QAction("Export", this);
00075     exportAct->setIcon(style()->standardIcon(QStyle::SP_DialogSaveButton));
00076     exportAct->setToolTip("Open video export workflows");
00077     connect(exportAct, &QAction::triggered, this, [this]
00078     {
00079         if (m_navigationSidebar)
00080             m_navigationSidebar->setCurrentPage(NavigationSidebar::Page::Video);
00081         createVideo();
00082     });
00083     tb->addAction(exportAct);
00084
00085     // Settings
00086     auto* settingsAct = new QAction("Settings", this);
00087     settingsAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogInfoView));
00088     settingsAct->setToolTip("Open Settings (Ctrl+,)");
00089     connect(settingsAct, &QAction::triggered, this, &MainWindow::openSettingsDialog);
00090     tb->addAction(settingsAct);
00091
00092     tb->addSeparator();
00093     tb->addWidget(new QLabel("Athlete:"));
00094
00095     m_athleteCombo = new QComboBox(tb);
00096     m_athleteCombo->setMinimumWidth(220);
00097     connect(m_athleteCombo, &QComboBox::currentIndexChanged,
00098             this, &MainWindow::onAthleteSelectionChanged);
00099     tb->addWidget(m_athleteCombo);
00100
00101     tb->addSeparator();
00102     tb->addWidget(new QLabel(" "));
00103
00104     m_importStatusWidget = new ImportStatusWidget(tb);
00105     if (m_importProgressModel)
00106         m_importStatusWidget->setModel(m_importProgressModel);
00107     tb->addWidget(m_importStatusWidget);
00108 }
00109
00110 void MainWindow::buildStatusBar()
00111 {
00112     m_dbStatusLabel = new QLabel("Database: none");
00113     m_athleteStatusLabel = new QLabel("Athlete: none");
00114     m_activityCountLabel = new QLabel("Activities: 0");
00115
00116     statusBar()->addPermanentWidget(m_dbStatusLabel);
00117     statusBar()->addPermanentWidget(m_athleteStatusLabel);
00118     statusBar()->addPermanentWidget(m_activityCountLabel);
00119 }
00120
00121 void MainWindow::setupShortcuts()
00122 {
00123     auto* fsAct = new QAction(this);
00124     fsAct->setShortcut(Qt::Key_F11);
00125     connect(fsAct, &QAction::triggered, this, [this]
00126     {
00127         isFullScreen() ? showNormal() : showFullScreen();
00128     });
00129     addAction(fsAct);
00130
00131     auto* resetAct = new QAction(this);

```



```

00132     resetAct->setShortcut(QKeySequence("Ctrl+0"));
00133     connect(resetAct, &QAction::triggered, this, &MainWindow::resetAllZoom);
00134     addAction(resetAct);
00135
00136     auto* undoAct = new QAction(this);
00137     undoAct->setShortcut(QKeySequence::Undo);
00138     connect(undoAct, &QAction::triggered, m_undoManager, &UndoManager::undo);
00139     addAction(undoAct);
00140
00141     auto* redoAct = new QAction(this);
00142     redoAct->setShortcut(QKeySequence::Redo);
00143     connect(redoAct, &QAction::triggered, m_undoManager, &UndoManager::redo);
00144     addAction(redoAct);
00145
00146     if (m_intervalsTable)
00147     {
00148         auto* prevIntervalAct = new QAction(m_intervalsTable);
00149         prevIntervalAct->setShortcut(Qt::Key_Up);
00150         prevIntervalAct->setShortcutContext(Qt::WidgetShortcut);
00151         connect(prevIntervalAct, &QAction::triggered, this, [this]
00152         {
00153             selectIntervalRowOffset(-1);
00154         });
00155         m_intervalsTable->addAction(prevIntervalAct);
00156
00157         auto* nextIntervalAct = new QAction(m_intervalsTable);
00158         nextIntervalAct->setShortcut(Qt::Key_Down);
00159         nextIntervalAct->setShortcutContext(Qt::WidgetShortcut);
00160         connect(nextIntervalAct, &QAction::triggered, this, [this]
00161         {
00162             selectIntervalRowOffset(1);
00163         });
00164         m_intervalsTable->addAction(nextIntervalAct);
00165     }
00166 }
00167
00168 bool MainWindow::eventFilter(QObject* watched, QEvent* event)
00169 {
00170     if (m_chartScroll && watched == m_chartScroll->viewport()
00171         && event->type() == QEvent::Resize)
00172     {
00173         applyChartHeight();
00174     }
00175
00176     if (m_chartMapSplit && watched == m_chartMapSplit && event->type() == QEvent::Resize)
00177     {
00178         const QList<int> sizes = m_chartMapSplit->sizes();
00179         if (sizes.size() == 2 && std::abs(sizes[0] - sizes[1]) > 1)
00180         {
00181             const int totalWidth = sizes[0] + sizes[1];
00182             if (totalWidth > 0)
00183                 m_chartMapSplit->setSizes({ totalWidth / 2, totalWidth / 2 });
00184         }
00185     }
00186
00187     return QMainWindow::eventFilter(watched, event);
00188 }

```

## 8.141 MainWindowUiBuild.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QAbstractItemView>
00006 #include <QAction>
00007 #include <QApplication>
00008 #include <QCheckBox>
00009 #include <QComboBox>
00010 #include <QDate>
00011 #include <QDialog>
00012 #include <QDoubleSpinBox>
00013 #include <QHBoxLayout>
00014 #include <QHeaderView>
00015 #include <QIcon>
00016 #include <QLabel>
00017 #include <QMenu>
00018 #include <QMenuBar>
00019 #include <QMessageBox>
00020 #include <QPainter>
00021 #include <QPushButton>
00022 #include <QSettings>
00023 #include <QSizePolicy>
00024 #include <QSplitter>

```

```

00025 #include <QStackedLayout>
00026 #include <QStackedWidget>
00027 #include <QStyle>
00028 #include <QTableWidget>
00029 #include <QTextEdit>
00030 #include <QStatusBar>
00031 #include <QTimer>
00032 #include <QToolButton>
00033 #include <QVBoxLayout>
00034 #include <QWidget>
00035
00036 #include <QtCharts/QBarCategoryAxis>
00037 #include <QtCharts/QBarSeries>
00038 #include <QtCharts/QBarSet>
00039 #include <QtCharts/QChart>
00040 #include <QtCharts/QChartView>
00041 #include <QtCharts/QValueAxis>
00042
00043 #include "AboutDialog.h"
00044 #include "ActivityBrowser.h"
00045 #include "AthleteHeaderWidget.h"
00046 #include "CalendarWidget.h"
00047 #include "IntervalEditorDialog.h"
00048 #include "NavigationSidebar.h"
00049 #include "WelcomeWidget.h"
00050 #include "pages/ActivitiesPage.h"
00051 #include "pages/CalendarPage.h"
00052 #include "pages/ChartsPage.h"
00053 #include "pages/ClimbsPage.h"
00054 #include "pages/FitnessPage.h"
00055 #include "pages/IntervalsPage.h"
00056 #include "pages/PowerPage.h"
00057 #include "pages/VideoPage.h"
00058 #include "analysis/IntervalDetector.h"
00059 #include "charts/FitnessChartWidget.h"
00060 #include "charts/PowerCurveWidget.h"
00061 #include "charts/PowerHistogramWidget.h"
00062 #include "charts/RideChartWidget.h"
00063 #include "database/IntervalRepository.h"
00064 #include "maps/MapRenderer.h"
00065 #include "controllers/ChartController.h"
00066
00067 #include <algorithm>
00068 #include <cmath>
00069 #include <numeric>
00070
00071 namespace
00072 {
00073     enum ChartPresetId
00074     {
00075         ChartPresetCustom = 0,
00076         ChartPresetPowerAnalysis,
00077         ChartPresetHeartRateFocus,
00078         ChartPresetRaceReview
00079     };
00080
00081     static constexpr int kClimbStartRole = Qt::UserRole + 3;
00082     static constexpr int kClimbEndRole = Qt::UserRole + 4;
00083
00084     QWidget* wrapWithNoActivityState(
00085         QWidget* content,
00086         QStackedLayout** outStack,
00087         const QString& message = QStringLiteral("No Activity Selected"))
00088     {
00089         auto* root = new QWidget;
00090         auto* stack = new QStackedLayout(root);
00091         stack->setContentsMargins(0, 0, 0, 0);
00092
00093         auto* emptyPage = new QWidget(root);
00094         auto* emptyLayout = new QVBoxLayout(emptyPage);
00095         emptyLayout->setContentsMargins(0, 0, 0, 0);
00096         auto* label = new QLabel(message, emptyPage);
00097         label->setAlignment(Qt::AlignCenter);
00098         label->setStyleSheet("color: #64748b; font-size: 16px; font-weight: 600;");
00099         emptyLayout->addWidget(label, 1);
00100
00101         stack->addWidget(emptyPage);
00102         stack->addWidget(content);
00103         stack->setCurrentIndex(0);
00104
00105         if (outStack)
00106             *outStack = stack;
00107         return root;
00108     }
00109 } // namespace
00110
00111 void MainWindow::buildUI()

```

```

00112 {
00113     {
00114         auto* fileMenu = menuBar()->addMenu("&File");
00115
00116         auto* openDbAct = new QAction("Open Database...", this);
00117         connect(openDbAct, &QAction::triggered, this, &MainWindow::openDatabase);
00118         fileMenu->addAction(openDbAct);
00119
00120         auto* newDbAct = new QAction("Create Database...", this);
00121         connect(newDbAct, &QAction::triggered, this, &MainWindow::createDatabase);
00122         fileMenu->addAction(newDbAct);
00123
00124         auto* backupAct = new QAction("Backup Database...", this);
00125         connect(backupAct, &QAction::triggered, this, &MainWindow::backupDatabase);
00126         fileMenu->addAction(backupAct);
00127
00128         fileMenu->addSeparator();
00129         m_recentDbMenu = fileMenu->addMenu("Recent Databases");
00130
00131         fileMenu->addSeparator();
00132
00133         auto* exitAct = new QAction("E&xit", this);
00134         exitAct->setShortcut(QKeySequence::Quit);
00135         connect(exitAct, &QAction::triggered, qApp, &QApplication::quit);
00136         fileMenu->addAction(exitAct);
00137
00138         auto* actsMenu = menuBar()->addMenu("&Activities");
00139
00140         m_importAct = new QAction("Import Activities...", this);
00141         m_importAct->setShortcut(QKeySequence::Open);
00142         m_importAct->setEnabled(false);
00143         connect(m_importAct, &QAction::triggered, this, &MainWindow::importActivities);
00144         actsMenu->addAction(m_importAct);
00145
00146         m_athletesMenu = menuBar()->addMenu("&Athletes");
00147
00148         auto* manageAct = new QAction("Manage Athletes...", this);
00149         connect(manageAct, &QAction::triggered, this, &MainWindow::manageAthletes);
00150         m_athletesMenu->addAction(manageAct);
00151
00152         auto* toolsMenu = menuBar()->addMenu("&Tools");
00153         m_previewAct = new QAction("Preview FIT File...", this);
00154         connect(m_previewAct, &QAction::triggered, this, &MainWindow::previewFitFile);
00155         toolsMenu->addAction(m_previewAct);
00156         toolsMenu->addSeparator();
00157         auto* settingsAct = new QAction("Settings...", this);
00158         settingsAct->setShortcut(QKeySequence::Preferences);
00159         connect(settingsAct, &QAction::triggered, this, &MainWindow::openSettingsDialog);
00160         toolsMenu->addAction(settingsAct);
00161
00162         auto* viewMenu = menuBar()->addMenu("&View");
00163         viewMenu->addAction("Reset Zoom", QKeySequence("Ctrl+0"), this, &MainWindow::resetAllZoom);
00164         viewMenu->addAction("Increase Chart Height", this, &MainWindow::increaseChartHeight);
00165         viewMenu->addAction("Decrease Chart Height", this, &MainWindow::decreaseChartHeight);
00166         auto* helpMenu = menuBar()->addMenu("&Help");
00167         helpMenu->addAction("&About", this, &MainWindow::showAbout);
00168     }
00169
00170     auto* central = new QWidget(this);
00171     auto* mainLayout = new QVBoxLayout(central);
00172     mainLayout->setSpacing(6);
00173     mainLayout->setContentsMargins(8, 8, 8, 8);
00174
00175     m_athleteHeader = new AthleteHeaderWidget(this);
00176     mainLayout->addWidget(m_athleteHeader);
00177
00178     m_welcomeWidget = new WelcomeWidget(this);
00179     mainLayout->addWidget(m_welcomeWidget, 1);
00180     m_welcomeWidget->setVisible(false);
00181     connect(m_welcomeWidget, &WelcomeWidget::importRequested, this, &MainWindow::importActivities);
00182     connect(m_welcomeWidget, &WelcomeWidget::openDatabaseRequested, this, &MainWindow::openDatabase);
00183     connect(m_welcomeWidget, &WelcomeWidget::createDatabaseRequested, this,
&MainWindow::createDatabase);
00184     connect(m_welcomeWidget, &WelcomeWidget::manageAthletesRequested, this,
&MainWindow::manageAthletes);
00185
00186     // Color-by metric control bar -- placed at the top of ChartsPage.
00187     auto* colorBar = new QHBoxLayout;
00188     colorBar->setContentsMargins(0, 0, 0, 0);
00189     colorBar->addWidget(new QLabel("Color By:"));
00190
00191     m_colorMetricCombo = new QComboBox(this);
00192     auto addColorMetric = [this](ColorMetric metric)
00193     {
00194         m_colorMetricCombo->addItem(colorMetricDisplayName(metric), static_cast<int>(metric));
00195     };
00196     addColorMetric(ColorMetric::None);

```

```

00197     addColorMetric(ColorMetric::Power);
00198     addColorMetric(ColorMetric::HeartRate);
00199     addColorMetric(ColorMetric::Cadence);
00200     addColorMetric(ColorMetric::Speed);
00201     addColorMetric(ColorMetric::Altitude);
00202     m_colorMetricCombo->setCurrentIndex(
00203         m_colorMetricCombo->findData(static_cast<int>(ColorMetric::Power)));
00204     colorBar->addWidget(m_colorMetricCombo);
00205     colorBar->addStretch(1);
00206
00207     {
00208         m_activityBrowser = new ActivityBrowser(&m_dbManager, this);
00209
00210         connect(m_activityBrowser, &ActivityBrowser::activitySelected,
00211             this, [this](int activityId)
00212             {
00213                 QString err;
00214                 if (!m_controller->loadActivity(activityId, err))
00215                     QMessageBox::critical(this, "Load Error", err);
00216                 else
00217                 {
00218                     QSettings("Fitlyzer", "FitlyzerC").setValue("lastActivityId", activityId);
00219                     // Navigate to Charts page -- the sidebar and NavigationController
00220                     // keep the page stack in sync.
00221                     if (m_navigationSidebar)
00222                         m_navigationSidebar->setCurrentPage(NavigationSidebar::Page::Charts);
00223                 }
00224             });
00225
00226         connect(m_activityBrowser, &ActivityBrowser::activityDeleted,
00227             this, [this](int activityId)
00228             {
00229                 (void)activityId;
00230                 updateStatsLabel();
00231                 updateStatusBarInfo();
00232                 updateWelcomeScreenVisibility();
00233             });
00234     }
00235
00236     {
00237         m_showPower = new QCheckBox("Power");
00238         m_showPower->setChecked(true);
00239         m_showHR = new QCheckBox("Heart Rate");
00240         m_showHR->setChecked(true);
00241         m_showCadence = new QCheckBox("Cadence");
00242         m_showCadence->setChecked(true);
00243         m_showSpeed = new QCheckBox("Speed");
00244         m_showSpeed->setChecked(true);
00245         m_showAltitude = new QCheckBox("Altitude");
00246         m_showAltitude->setChecked(true);
00247
00248         auto* metricsBtn = new QToolButton;
00249         metricsBtn->setText("Metrics");
00250         metricsBtn->setPopupMode(QToolButton::InstantPopup);
00251         auto* metricsMenu = new QMenu(metricsBtn);
00252         auto addMetricAction = [metricsMenu](const QString& title, QCheckBox* box)
00253         {
00254             auto* act = metricsMenu->addAction(title);
00255             act->setCheckable(true);
00256             act->setChecked(box->isChecked());
00257             QObject::connect(act, &QAction::toggled, box, &QCheckBox::setChecked);
00258             QObject::connect(box, &QCheckBox::toggled, act, &QAction::setChecked);
00259         };
00260         addMetricAction("Power", m_showPower);
00261         addMetricAction("Heart Rate", m_showHR);
00262         addMetricAction("Cadence", m_showCadence);
00263         addMetricAction("Speed", m_showSpeed);
00264         addMetricAction("Altitude", m_showAltitude);
00265         metricsBtn->setMenu(metricsMenu);
00266
00267         m_powerSmoothingCombo = new QComboBox(this);
00268         m_powerSmoothingCombo->addItem("Raw", 0);
00269         m_powerSmoothingCombo->addItem("3 sec", 3);
00270         m_powerSmoothingCombo->addItem("5 sec", 5);
00271         m_powerSmoothingCombo->addItem("10 sec", 10);
00272         m_powerSmoothingCombo->addItem("30 sec", 30);
00273         m_powerSmoothingCombo->addItem("60 sec", 60);
00274
00275         m_autoSmoothingCheck = new QCheckBox("Auto Smoothing", this);
00276         m_autoSmoothingCheck->setChecked(false);
00277
00278         m_fitChartsButton = new QPushButton("Reset Zoom");
00279         m_chartHeightIncreaseButton = new QPushButton("Increase Height");
00280         m_chartHeightDecreaseButton = new QPushButton("Decrease Height");
00281         m_useEstimatedFtpButton = new QPushButton("Use Estimated FTP");
00282         auto* editActivityButton = new QPushButton("Activity Properties");
00283         m_fitChartsButton->setFixedWidth(110);

```

```

00284     m_chartHeightIncreaseButton->setFixedWidth(130);
00285     m_chartHeightDecreaseButton->setFixedWidth(130);
00286     m_useEstimatedFtpButton->setFixedWidth(150);
00287     editActivityButton->setFixedWidth(150);
00288     m_fitChartsButton->setToolTip("Reset all chart zoom to full ride (Ctrl+0)");
00289     m_chartHeightIncreaseButton->setToolTip("Increase chart height");
00290     m_chartHeightDecreaseButton->setToolTip("Decrease chart height");
00291     m_useEstimatedFtpButton->setToolTip("Estimate FTP as 95% of best 20-minute power");
00292     m_useEstimatedFtpButton->setEnabled(false);
00293
00294     connect(m_fitChartsButton, &QPushButton::clicked,
00295             this, &MainWindow::resetAllZoom);
00296     connect(m_chartHeightIncreaseButton, &QPushButton::clicked,
00297             this, &MainWindow::increaseChartHeight);
00298     connect(m_chartHeightDecreaseButton, &QPushButton::clicked,
00299             this, &MainWindow::decreaseChartHeight);
00300     connect(editActivityButton, &QPushButton::clicked,
00301             this, &MainWindow::editCurrentActivityProperties);
00302     connect(m_useEstimatedFtpButton, &QPushButton::clicked,
00303             this, &MainWindow::applyEstimatedFtpForCurrentAthlete);
00304
00305     auto* ctrlBar = new QHBoxLayout;
00306     ctrlBar->setContentsMargins(0, 0, 0, 0);
00307     ctrlBar->addWidget(metricsBtn);
00308     ctrlBar->addSpacing(10);
00309     ctrlBar->addWidget(new QLabel("Preset:"));
00310
00311     m_chartPresetCombo = new QComboBox(this);
00312     m_chartPresetCombo->addItem("Custom", ChartPresetCustom);
00313     m_chartPresetCombo->addItem("Power Analysis", ChartPresetPowerAnalysis);
00314     m_chartPresetCombo->addItem("Heart Rate Focus", ChartPresetHeartRateFocus);
00315     m_chartPresetCombo->addItem("Race Review", ChartPresetRaceReview);
00316     m_chartPresetCombo->setCurrentIndex(0);
00317     ctrlBar->addWidget(m_chartPresetCombo);
00318
00319     ctrlBar->addSpacing(10);
00320     ctrlBar->addWidget(new QLabel("Power Smoothing:"));
00321     ctrlBar->addWidget(m_powerSmoothingCombo);
00322     ctrlBar->addWidget(m_autoSmoothingCheck);
00323     ctrlBar->addStretch(1);
00324     ctrlBar->addSpacing(8);
00325     ctrlBar->addWidget(m_fitChartsButton);
00326     ctrlBar->addWidget(m_chartHeightIncreaseButton);
00327     ctrlBar->addWidget(m_chartHeightDecreaseButton);
00328     ctrlBar->addWidget(m_useEstimatedFtpButton);
00329     ctrlBar->addWidget(editActivityButton);
00330
00331     m_powerChart = new RideChartWidget(Metric::Power);
00332     m_hrChart = new RideChartWidget(Metric::HeartRate);
00333     m_cadenceChart = new RideChartWidget(Metric::Cadence);
00334     m_speedChart = new RideChartWidget(Metric::Speed);
00335     m_altitudeChart = new RideChartWidget(Metric::Altitude);
00336
00337     connect(m_powerSmoothingCombo,
00338             qOverload<int>(&QComboBox::currentIndexChanged),
00339             this,
00340             [this](int)
00341             {
00342                 if (!m_powerSmoothingCombo)
00343                     return;
00344                 if (!m_chartController)
00345                     return;
00346                 syncChartContextToController(false);
00347                 m_chartController->applyChartSmoothing();
00348             });
00349
00350     connect(m_autoSmoothingCheck, &QCheckBox::toggled,
00351             this, [this](bool checked)
00352             {
00353                 if (!m_powerSmoothingCombo)
00354                     return;
00355                 m_powerSmoothingCombo->setEnabled(!checked);
00356                 if (!m_chartController)
00357                     return;
00358                 syncChartContextToController(false);
00359                 m_chartController->applyChartSmoothing();
00360             });
00361
00362     connect(m_chartPresetCombo,
00363             qOverload<int>(&QComboBox::currentIndexChanged),
00364             this,
00365             [this](int)
00366             {
00367                 if (!m_chartPresetCombo)
00368                     return;
00369                 applyChartPreset(m_chartPresetCombo->currentData().toInt());
00370             });

```

```

00371
00372     m_chartsContainer = new QWidget;
00373     m_chartsLayout = new QVBoxLayout(m_chartsContainer);
00374     m_chartsLayout->setSpacing(4);
00375     m_chartsLayout->setContentsMargins(0, 0, 0, 0);
00376     m_chartsLayout->addWidget(m_powerChart);
00377     m_chartsLayout->addWidget(m_hrChart);
00378     m_chartsLayout->addWidget(m_cadenceChart);
00379     m_chartsLayout->addWidget(m_speedChart);
00380     m_chartsLayout->addWidget(m_altitudeChart);
00381     m_chartsLayout->addStretch();
00382
00383     m_chartScroll = new QScrollArea;
00384     m_chartScroll->setWidget(m_chartsContainer);
00385     m_chartScroll->setWidgetResizable(true);
00386     m_chartScroll->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
00387     m_chartScroll->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOn);
00388     m_chartScroll->viewport()->installEventFilter(this);
00389
00390     auto* chartsTab = new QWidget;
00391     auto* chartsVL = new QVBoxLayout(chartsTab);
00392     chartsVL->setContentsMargins(0, 0, 0, 0);
00393     chartsVL->setSpacing(4);
00394     chartsVL->addLayout(ctrlBar);
00395
00396     // m_colorLegendWidget is placed in the Charts page header bar (colorBarWidget),
00397     // not inline in chartsTab, so it must not be reparented here.
00398     m_colorLegendWidget = new QWidget(this);
00399     m_colorLegendLayout = new QHBoxLayout(m_colorLegendWidget);
00400     m_colorLegendLayout->setContentsMargins(0, 0, 0, 0);
00401     m_colorLegendLayout->setSpacing(8);
00402     m_mapRenderer = new MapRenderer;
00403
00404     auto* mapPanel = new QWidget(chartsTab);
00405     auto* mapVL = new QVBoxLayout(mapPanel);
00406     mapVL->setContentsMargins(0, 0, 0, 0);
00407     mapVL->setSpacing(4);
00408     auto* mapBtnBar = new QHBoxLayout;
00409     auto* mapFitBtn = new QPushButton("Fit to Track", mapPanel);
00410     auto* mapFitSelBtn = new QPushButton("Fit Map To Selection", mapPanel);
00411     auto* mapZoomIn = new QPushButton("+", mapPanel);
00412     auto* mapZoomOut = new QPushButton("-", mapPanel);
00413     m_mapStyleCombo = new QComboBox(mapPanel);
00414     m_mapStyleCombo->addItem("Light", static_cast<int>(MapStyle::Light));
00415     m_mapStyleCombo->addItem("Street Map", static_cast<int>(MapStyle::Street));
00416     m_mapStyleCombo->addItem("Satellite", static_cast<int>(MapStyle::Satellite));
00417     m_mapStyleCombo->addItem("Dark", static_cast<int>(MapStyle::Dark));
00418     m_mapStyleCombo->addItem("Terrain", static_cast<int>(MapStyle::Terrain));
00419     m_mapStyleCombo->addItem("Topographic", static_cast<int>(MapStyle::Topographic));
00420     m_mapStyleCombo->addItem("Minimal", static_cast<int>(MapStyle::Minimal));
00421     m_mapAutoContrastCheck = new QCheckBox("Auto route contrast", mapPanel);
00422     m_mapAutoContrastCheck->setChecked(true);
00423     mapZoomIn->setFixedWidth(32);
00424     mapZoomOut->setFixedWidth(32);
00425     connect(mapFitBtn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::fitToTrack);
00426     connect(mapFitSelBtn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::fitToVisibleRange);
00427     connect(mapZoomIn, &QPushButton::clicked, m_mapRenderer, &MapRenderer::zoomIn);
00428     connect(mapZoomOut, &QPushButton::clicked, m_mapRenderer, &MapRenderer::zoomOut);
00429     connect(m_mapStyleCombo, qOverload<int>(&QComboBox::currentIndexChanged), this, [this](int)
00430     {
00431         if (!m_mapRenderer || !m_mapStyleCombo)
00432             return;
00433         const MapStyle style = static_cast<MapStyle>(m_mapStyleCombo->currentData().toInt());
00434         m_mapRenderer->setMapStyle(style);
00435     });
00436     connect(m_mapAutoContrastCheck, &QCheckBox::toggled, this, [this](bool checked)
00437     {
00438         if (m_mapRenderer)
00439             m_mapRenderer->setAutoRouteContrast(checked);
00440     });
00441     mapBtnBar->addWidget(mapFitBtn);
00442     mapBtnBar->addWidget(mapFitSelBtn);
00443     mapBtnBar->addWidget(mapZoomIn);
00444     mapBtnBar->addWidget(mapZoomOut);
00445     mapBtnBar->addSpacing(8);
00446     mapBtnBar->addWidget(new QLabel("Map Style:", mapPanel));
00447     mapBtnBar->addWidget(m_mapStyleCombo);
00448     mapBtnBar->addWidget(m_mapAutoContrastCheck);
00449     mapBtnBar->addStretch(1);
00450     mapVL->addLayout(mapBtnBar);
00451     mapVL->addWidget(m_mapRenderer, 1);
00452
00453     m_intervalsTable = new QTableWidgetItem(0, 8, chartsTab);
00454     m_intervalsTable->setHorizontalHeaderLabels(
00455         { "Type", "Start", "Duration", "Avg W", "NP", "IF", "Max W", "Avg HR" });
00456     m_intervalsTable->horizontalHeader()->setStretchLastSection(true);
00457     m_intervalsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);

```

```

00458     m_intervalsTable->setAlternatingRowColors(true);
00459     m_intervalsTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00460     m_intervalsTable->setSelectionMode(QAbstractItemView::SingleSelection);
00461     m_intervalsTable->setStyleSheet(
00462         "QTableWidget::item:selected { background-color: #bfbdbe; color: #0f172a; }");
00463     connect(m_intervalsTable, &QTableWidget::itemSelectionChanged,
00464         this, &MainWindow::onIntervalSelectionChanged);
00465     connect(m_intervalsTable, &QTableWidget::itemDoubleClicked,
00466         this, &MainWindow::onIntervalRowDoubleClicked);
00467
00468     m_lapsTable = new QTableWidget(0, 3, chartsTab);
00469     m_lapsTable->setHorizontalHeaderLabels({ "Lap", "Start", "Duration" });
00470     m_lapsTable->horizontalHeader()->setStretchLastSection(true);
00471     m_lapsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
00472
00473     m_activityNotesView = new QTextEdit(chartsTab);
00474     m_activityNotesView->setReadOnly(true);
00475     m_activityNotesView->setPlaceholderText("Activity notes and conditions");
00476
00477     auto* intervalsPanel = new QWidget(chartsTab);
00478     auto* intervalsVL = new QVBoxLayout(intervalsPanel);
00479     intervalsVL->setContentsMargins(0, 0, 0, 0);
00480     intervalsVL->setSpacing(4);
00481     auto* intervalsLabel = new QLabel("Intervals", intervalsPanel);
00482     intervalsLabel->setStyleSheet("font-weight: 600;");
00483     intervalsVL->addWidget(intervalsLabel);
00484     auto* intervalsToolBar = new QHBoxLayout;
00485     auto* saveAutoBtn = new QPushButton("Save Auto Intervals", intervalsPanel);
00486     auto* addManualBtn = new QPushButton("Add Manual Interval", intervalsPanel);
00487     m_followIntervalSelectionCheck = new QCheckBox("Follow interval selection", intervalsPanel);
00488     m_followIntervalSelectionCheck->setChecked(true);
00489     intervalsToolBar->addWidget(saveAutoBtn);
00490     intervalsToolBar->addWidget(addManualBtn);
00491     intervalsToolBar->addWidget(m_followIntervalSelectionCheck);
00492     intervalsToolBar->addStretch(1);
00493     intervalsVL->addLayout(intervalsToolBar);
00494     intervalsVL->addWidget(m_intervalsTable, 1);
00495     m_intervalSummaryLabel = new QLabel("Interval summary: select an interval", intervalsPanel);
00496     m_intervalSummaryLabel->setStyleSheet("color: #334155;");
00497     intervalsVL->addWidget(m_intervalSummaryLabel);
00498
00499     auto* lapsPanel = new QWidget(chartsTab);
00500     auto* lapsVL = new QVBoxLayout(lapsPanel);
00501     lapsVL->setContentsMargins(0, 0, 0, 0);
00502     lapsVL->setSpacing(4);
00503     auto* lapsLabel = new QLabel("Laps", lapsPanel);
00504     lapsLabel->setStyleSheet("font-weight: 600;");
00505     lapsVL->addWidget(lapsLabel);
00506     lapsVL->addWidget(m_lapsTable, 1);
00507
00508     auto* notesPanel = new QWidget(chartsTab);
00509     auto* notesVL = new QVBoxLayout(notesPanel);
00510     notesVL->setContentsMargins(0, 0, 0, 0);
00511     notesVL->setSpacing(4);
00512     auto* notesLabel = new QLabel("Notes", notesPanel);
00513     notesLabel->setStyleSheet("font-weight: 600;");
00514     notesVL->addWidget(notesLabel);
00515     notesVL->addWidget(m_activityNotesView, 1);
00516
00517     auto* topSplit = new QSplitter(Qt::Horizontal, chartsTab);
00518     m_chartMapSplit = topSplit;
00519     topSplit->addWidget(m_chartScroll);
00520     topSplit->addWidget(mapPanel);
00521     topSplit->setStretchFactor(0, 1);
00522     topSplit->setStretchFactor(1, 1);
00523     topSplit->setCollapsible(0, false);
00524     topSplit->setCollapsible(1, false);
00525
00526     QTimer::singleShot(0, topSplit, [topSplit]()
00527     {
00528         const QList<int> sizes = topSplit->sizes();
00529         const int totalWidth = std::accumulate(sizes.begin(), sizes.end(), 0);
00530         if (totalWidth > 0)
00531             topSplit->setSizes({ totalWidth / 2, totalWidth / 2 });
00532     });
00533
00534     auto* bottomSplit = new QSplitter(Qt::Horizontal, chartsTab);
00535     bottomSplit->addWidget(lapsPanel);
00536     bottomSplit->addWidget(notesPanel);
00537     bottomSplit->setStretchFactor(0, 1);
00538     bottomSplit->setStretchFactor(1, 2);
00539
00540     // Intervals now live on the dedicated Intervals page.
00541     m_intervalsPageContent = intervalsPanel;
00542
00543     auto* activitySplit = new QSplitter(Qt::Vertical, chartsTab);
00544     activitySplit->addWidget(topSplit);

```



```

00545     activitySplit->addWidget(bottomSplit);
00546     activitySplit->setStretchFactor(0, 3);
00547     activitySplit->setStretchFactor(1, 2);
00548
00549     chartsVL->addWidget(activitySplit, 1);
00550
00551     // Wrap the charts/map/intervals content in an empty-state guard.
00552     // The resulting widget becomes the content area of ChartsPage.
00553     // Intervals panel remains here for chart-context navigation (interval
00554     // selection zooms the chart). IntervalsPage shows a standalone view.
00555     m_chartsPageContent = wrapWithNoActivityState(chartsTab, &m_activityTabStack);
00556
00557     auto makeVisToggle = [this](QCheckBox* cb, RideChartWidget* chart)
00558     {
00559         connect(cb, &QCheckBox::toggled, this, [this, chart](bool v)
00560         {
00561             chart->setVisible(v);
00562             if (m_chartsContainer)
00563                 m_chartsContainer->adjustSize();
00564         });
00565     };
00566     makeVisToggle(m_showPower, m_powerChart);
00567     makeVisToggle(m_showHR, m_hrChart);
00568     makeVisToggle(m_showCadence, m_cadenceChart);
00569     makeVisToggle(m_showSpeed, m_speedChart);
00570     makeVisToggle(m_showAltitude, m_altitudeChart);
00571
00572     const std::initializer_list<RideChartWidget*> all =
00573         { m_powerChart, m_hrChart, m_cadenceChart, m_speedChart, m_altitudeChart };
00574
00575     for (auto* src : all)
00576     {
00577         connect(src, &RideChartWidget::crosshairMoved,
00578             this, [this](double xMinutes)
00579             {
00580                 if (!m_mapRenderer)
00581                     return;
00582                 m_mapRenderer->setCurrentTime(xMinutes < 0.0 ? -1.0 : xMinutes * 60.0);
00583             });
00584     }
00585
00586     connect(m_powerChart, &RideChartWidget::visibleRangeChanged,
00587         this, [this](double startMinutes, double endMinutes)
00588         {
00589             if (!m_mapRenderer)
00590                 return;
00591             m_mapRenderer->setVisibleTimeRange(startMinutes * 60.0, endMinutes * 60.0);
00592         });
00593
00594     auto syncChartsToMapSelection = [this](double startSeconds, double endSeconds)
00595     {
00596         const double startMinutes = std::min(startSeconds, endSeconds) / 60.0;
00597         const double endMinutes = std::max(startSeconds, endSeconds) / 60.0;
00598         m_powerChart->setXRange(startMinutes, endMinutes);
00599         m_hrChart->setXRange(startMinutes, endMinutes);
00600         m_cadenceChart->setXRange(startMinutes, endMinutes);
00601         m_speedChart->setXRange(startMinutes, endMinutes);
00602         m_altitudeChart->setXRange(startMinutes, endMinutes);
00603     };
00604
00605     connect(m_mapRenderer, &MapRenderer::segmentSelectionChanged,
00606         this, [syncChartsToMapSelection](double startSeconds, double endSeconds)
00607         {
00608             syncChartsToMapSelection(startSeconds, endSeconds);
00609         });
00610
00611     connect(m_mapRenderer, &MapRenderer::segmentSelectionFinished,
00612         this, [syncChartsToMapSelection](double startSeconds, double endSeconds)
00613         {
00614             syncChartsToMapSelection(startSeconds, endSeconds);
00615         });
00616
00617     connect(m_powerChart, &RideChartWidget::intervalSelectionRequested,
00618         this, [this](double startSec, double endSec)
00619         {
00620             if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00621                 return;
00622
00623             IntervalEditorDialog dlg(startSec, endSec, this);
00624             if (dlg.exec() != QDialog::Accepted)
00625                 return;
00626
00627             auto db = m_dbManager.database();
00628             IntervalRepository repo(db);
00629
00630             const auto& records = m_controller->rideData().records;
00631             double sumPower = 0.0;

```



```

00632         int countPower = 0;
00633         double sumHr = 0.0;
00634         int countHr = 0;
00635         double sumCadence = 0.0;
00636         int countCadence = 0;
00637         std::vector<double> powerValues;
00638         for (const RideRecord& r : records)
00639         {
00640             if (r.elapsedSeconds < startSec || r.elapsedSeconds > endSec)
00641                 continue;
00642             if (r.hasPower)
00643             {
00644                 sumPower += r.power;
00645                 ++countPower;
00646                 powerValues.push_back(r.power);
00647             }
00648             if (r.hasHeartRate)
00649             {
00650                 sumHr += r.heartRate;
00651                 ++countHr;
00652             }
00653             if (r.hasCadence)
00654             {
00655                 sumCadence += r.cadence;
00656                 ++countCadence;
00657             }
00658         }
00659
00660         double np = 0.0;
00661         if (!powerValues.empty())
00662         {
00663             double sum4 = 0.0;
00664             for (double p : powerValues)
00665                 sum4 += std::pow(p, 4.0);
00666             np = std::pow(sum4 / static_cast<double>(powerValues.size()), 0.25);
00667         }
00668
00669         IntervalRecord record;
00670         record.activityId = m_controller->currentActivityId();
00671         record.startSample = static_cast<int>(std::round(startSec));
00672         record.endSample = static_cast<int>(std::round(endSec));
00673         record.name = dlg.intervalName().isEmpty() ? QStringLiteral("Manual Interval") :
dlg.intervalName();
00674         record.type = dlg.intervalType().isEmpty() ? QStringLiteral("Manual") :
dlg.intervalType();
00675         record.avgPower = countPower > 0 ? (sumPower / static_cast<double>(countPower)) : 0.0;
00676         record.np = np;
00677         record.avgHr = countHr > 0 ? (sumHr / static_cast<double>(countHr)) : 0.0;
00678         record.avgCadence = countCadence > 0 ? (sumCadence / static_cast<double>(countCadence)) :
0.0;
00679         record.notes = dlg.intervalNotes();
00680         record.source = QStringLiteral("manual");
00681         record.locked = true;
00682         repo.insertInterval(record);
00683         updateIntervals();
00684         statusBar()->showMessage("Manual interval saved from chart selection.", 2200);
00685     });
00686
00687     connect(saveAutoBtn, &QPushButton::clicked, this, [this]
00688     {
00689         if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)
00690             return;
00691
00692         auto db = m_dbManager.database();
00693         IntervalRepository repo(db);
00694         repo.deleteIntervalsForActivity(m_controller->currentActivityId());
00695         for (const Interval& iv : m_controller->intervals())
00696         {
00697             IntervalRecord record;
00698             record.activityId = m_controller->currentActivityId();
00699             record.startSample = static_cast<int>(std::round(iv.startSeconds));
00700             record.endSample = static_cast<int>(std::round(iv.endSeconds));
00701             record.name = QString::fromStdString(iv.label);
00702             record.type = QString::fromStdString(iv.label);
00703             record.avgPower = iv.averagePower;
00704             record.np = iv.normalizedPower;
00705             record.avgHr = iv.averageHeartRate;
00706             record.avgCadence = iv.averageCadence;
00707             repo.insertInterval(record);
00708         }
00709         updateIntervals();
00710         statusBar()->showMessage("Auto intervals saved.", 2000);
00711     });
00712
00713     connect(addManualBtn, &QPushButton::clicked, this, [this]
00714     {
00715         if (!m_dbManager.isOpen() || m_controller->currentActivityId() <= 0)

```

```

00716         return;
00717         const auto& records = m_controller->rideData().records;
00718         if (records.size() < 2)
00719             return;
00720
00721         const double startSec = records[records.size() / 4].elapsedSeconds;
00722         const double endSec = records[records.size() / 4 + records.size() / 10].elapsedSeconds;
00723
00724         IntervalEditorDialog dlg(startSec, endSec, this);
00725         if (dlg.exec() != QDialog::Accepted)
00726             return;
00727
00728         auto db = m_dbManager.database();
00729         IntervalRepository repo(db);
00730         IntervalRecord record;
00731         record.activityId = m_controller->currentActivityId();
00732         record.startSample = static_cast<int>(std::round(startSec));
00733         record.endSample = static_cast<int>(std::round(endSec));
00734         record.name = dlg.intervalName();
00735         record.type = dlg.intervalType().isEmpty() ? QStringLiteral("Manual") :
00736             dlg.intervalType();
00737         record.notes = dlg.intervalNotes();
00738         record.source = QStringLiteral("manual");
00739         record.locked = true;
00740         repo.insertInterval(record);
00741         updateIntervals();
00742         statusBar()->showMessage("Manual interval saved.", 2000);
00743     });
00744
00745     {
00746         m_zonesTable = new QTableWidget(0, 5);
00747         m_zonesTable->setHorizontalHeaderLabels(
00748             { "Zone", "Name", "Range", "Time in Zone", "%" });
00749         m_zonesTable->horizontalHeader()->setStretchLastSection(true);
00750         m_zonesTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
00751         m_zonesTable->setAlternatingRowColors(true);
00752         m_zonesTable->setSelectionMode(QAbstractItemView::SingleSelection);
00753
00754         auto* w = new QWidget;
00755         auto* vl = new QVBoxLayout(w);
00756         vl->addWidget(m_zonesTable);
00757         // Store wrapped zones widget for the Power page.
00758         m_zonesPageContent = wrapWithNoActivityState(w, &m_zonesTabStack);
00759     }
00760
00761     {
00762         m_histogram = new PowerHistogramWidget;
00763         // Store wrapped histogram widget for the Power page.
00764         m_histogramPageContent = wrapWithNoActivityState(m_histogram, &m_histogramTabStack);
00765     }
00766
00767     {
00768         m_pdcWidget = new PowerCurveWidget;
00769         // Store wrapped power-curve widget for the Power page.
00770         m_pdcPageContent = wrapWithNoActivityState(m_pdcWidget, &m_powerCurveTabStack);
00771     }
00772
00773     {
00774         m_calendarWidget = new CalendarWidget(this);
00775         m_calendarWidget->setDatabaseManager(&m_dbManager);
00776         m_calendarWidget->setAthleteId(m_currentAthleteId);
00777         // Store wrapped calendar widget for CalendarPage.
00778         m_calendarPageContent = wrapWithNoActivityState(m_calendarWidget, &m_calendarTabStack);
00779     }
00780
00781     {
00782         m_fitnessChart = new FitnessChartWidget(this);
00783         // Store wrapped fitness chart for FitnessPage.
00784         m_fitnessPageContent = wrapWithNoActivityState(m_fitnessChart, &m_fitnessTabStack);
00785     }
00786
00787     {
00788         auto* climbingTab = new QWidget(this);
00789         auto* climbingVL = new QVBoxLayout(climbingTab);
00790         climbingVL->setContentsMargins(0, 0, 0, 0);
00791         climbingVL->setSpacing(6);
00792
00793         m_climbAltitudeChart = new RideChartWidget(Metric::Altitude, climbingTab);
00794         m_climbPowerChart = new RideChartWidget(Metric::Power, climbingTab);
00795         m_climbHrChart = new RideChartWidget(Metric::HeartRate, climbingTab);
00796         m_climbCadenceChart = new RideChartWidget(Metric::Cadence, climbingTab);
00797         m_climbSpeedChart = new RideChartWidget(Metric::Speed, climbingTab);
00798         m_climbAltitudeChart->setClimbEditingEnabled(true);
00799         m_climbAltitudeChart->setMinimumHeight(320);
00800         m_climbAltitudeChart->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
00801         m_climbPowerChart->hide();

```

```

00802         m_climbHrChart->hide();
00803         m_climbCadenceChart->hide();
00804         m_climbSpeedChart->hide();
00805
00806         auto* climbCharts = new QWidget(climbingTab);
00807         auto* climbChartsVL = new QVBoxLayout(climbCharts);
00808         climbChartsVL->setContentsMargins(0, 0, 0, 0);
00809         climbChartsVL->setSpacing(4);
00810         climbChartsVL->addWidget(m_climbAltitudeChart);
00811
00812         auto createQuarterChart = [climbingTab](const QString& title, const QColor& color)
00813         {
00814             auto* view = new QChartView(new QChart, climbingTab);
00815             view->setRenderHint(QPainter::Antialiasing);
00816             view->setMinimumHeight(160);
00817             view->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Preferred);
00818             view->chart()->setTitle(title);
00819             view->chart()->setAnimationOptions(QChart::NoAnimation);
00820             view->chart()->legend()->hide();
00821             view->chart()->setBackgroundVisible(false);
00822             view->setStyleSheet("background: #ffffff; border: 1px solid #e2e8f0; border-radius:
6px;");
00823
00824             auto* set = new QBarSet(title);
00825             set->setColor(color);
00826             *set << 0.0 << 0.0 << 0.0 << 0.0;
00827
00828             auto* series = new QBarSeries(view->chart());
00829             series->append(set);
00830             view->chart()->addSeries(series);
00831
00832             auto* axisX = new QBarCategoryAxis(view->chart());
00833             axisX->append({ "Q1", "Q2", "Q3", "Q4" });
00834             view->chart()->addAxis(axisX, Qt::AlignBottom);
00835             series->attachAxis(axisX);
00836
00837             auto* axisY = new QValueAxis(view->chart());
00838             axisY->setRange(0.0, 1.0);
00839             view->chart()->addAxis(axisY, Qt::AlignLeft);
00840             series->attachAxis(axisY);
00841
00842             return view;
00843         };
00844
00845         auto* quarterCharts = new QWidget(climbingTab);
00846         auto* quarterChartsHL = new QHBoxLayout(quarterCharts);
00847         quarterChartsHL->setContentsMargins(0, 0, 0, 0);
00848         quarterChartsHL->setSpacing(6);
00849
00850         m_climbQuarterPowerChart = createQuarterChart(QStringLiteral("Power by Quarter"), QColor(37,
99, 235));
00851         m_climbQuarterHrChart = createQuarterChart(QStringLiteral("Heart Rate by Quarter"),
QColor(220, 38, 38));
00852         m_climbQuarterCadenceChart = createQuarterChart(QStringLiteral("Cadence by Quarter"),
QColor(5, 150, 105));
00853
00854         quarterChartsHL->addWidget(m_climbQuarterPowerChart, 1);
00855         quarterChartsHL->addWidget(m_climbQuarterHrChart, 1);
00856         quarterChartsHL->addWidget(m_climbQuarterCadenceChart, 1);
00857         climbChartsVL->addWidget(quarterCharts);
00858
00859         m_climbMinLengthSpin = new QDoubleSpinBox(climbingTab);
00860         m_climbMinLengthSpin->setRange(0.1, 20.0);
00861         m_climbMinLengthSpin->setDecimals(2);
00862         m_climbMinLengthSpin->setSingleStep(0.1);
00863         m_climbMinLengthSpin->setValue(0.15);
00864
00865         m_climbMinGainSpin = new QDoubleSpinBox(climbingTab);
00866         m_climbMinGainSpin->setRange(1.0, 2000.0);
00867         m_climbMinGainSpin->setDecimals(0);
00868         m_climbMinGainSpin->setSingleStep(5.0);
00869         m_climbMinGainSpin->setValue(10.0);
00870
00871         m_climbMinGradientSpin = new QDoubleSpinBox(climbingTab);
00872         m_climbMinGradientSpin->setRange(0.5, 20.0);
00873         m_climbMinGradientSpin->setDecimals(1);
00874         m_climbMinGradientSpin->setSingleStep(0.5);
00875         m_climbMinGradientSpin->setValue(4.0);
00876
00877         m_climbStartGradientSpin = new QDoubleSpinBox(climbingTab);
00878         m_climbStartGradientSpin->setRange(0.5, 20.0);
00879         m_climbStartGradientSpin->setDecimals(1);
00880         m_climbStartGradientSpin->setSingleStep(0.5);
00881         m_climbStartGradientSpin->setValue(1.5);
00882
00883         m_climbDipMetersSpin = new QDoubleSpinBox(climbingTab);
00884         m_climbDipMetersSpin->setRange(0.0, 200.0);

```

```

00885     m_climbDipMetersSpin->setDecimals(1);
00886     m_climbDipMetersSpin->setSingleStep(1.0);
00887     m_climbDipMetersSpin->setValue(10.0);
00888
00889     m_climbDipDistanceSpin = new QDoubleSpinBox(climbingTab);
00890     m_climbDipDistanceSpin->setRange(10.0, 2000.0);
00891     m_climbDipDistanceSpin->setDecimals(0);
00892     m_climbDipDistanceSpin->setSingleStep(10.0);
00893     m_climbDipDistanceSpin->setValue(200.0);
00894
00895     m_climbSmoothingSpin = new QDoubleSpinBox(climbingTab);
00896     m_climbSmoothingSpin->setRange(1.0, 500.0);
00897     m_climbSmoothingSpin->setDecimals(0);
00898     m_climbSmoothingSpin->setSingleStep(5.0);
00899     m_climbSmoothingSpin->setValue(50.0);
00900
00901     // Override spinbox defaults with persisted settings (set in Settings dialog).
00902     {
00903         QSettings cs("Fitlyzer", "FitlyzerC");
00904         m_climbMinLengthSpin->setValue( cs.value("climb/minLength", 0.15).toDouble());
00905         m_climbMinGainSpin->setValue( cs.value("climb/minGain", 10.0).toDouble());
00906         m_climbMinGradientSpin->setValue( cs.value("climb/minGradient", 4.0).toDouble());
00907         m_climbStartGradientSpin->setValue(cs.value("climb/startGrad", 1.5).toDouble());
00908         m_climbDipMetersSpin->setValue( cs.value("climb/dipMeters", 10.0).toDouble());
00909         m_climbDipDistanceSpin->setValue( cs.value("climb/dipDistance", 200.0).toDouble());
00910         m_climbSmoothingSpin->setValue( cs.value("climb/smoothing", 50.0).toDouble());
00911     }
00912
00913     m_climbOverlayEnabledCheck = new QCheckBox("Overlay", climbingTab);
00914     m_climbOverlayEnabledCheck->setChecked(false);
00915     m_climbOverlayEnabledCheck->setStyleSheet(
00916         "QCheckBox { color: #0f172a; font-weight: 600; }"
00917         "QCheckBox::indicator { width: 16px; height: 16px; border: 2px solid #334155; background:
00918         #ffffff; }"
00919         "QCheckBox::indicator:checked { background: #0f172a; border-color: #0f172a; }");
00920     m_climbOverlayMetricCombo = new QComboBox(climbingTab);
00921     m_climbOverlayMetricCombo->addItem("Power", static_cast<int>(ColorMetric::Power));
00922     m_climbOverlayMetricCombo->addItem("Heart Rate", static_cast<int>(ColorMetric::HeartRate));
00923     m_climbOverlayMetricCombo->addItem("Cadence", static_cast<int>(ColorMetric::Cadence));
00924     m_climbOverlayMetricCombo->addItem("Speed", static_cast<int>(ColorMetric::Speed));
00925     m_climbOverlayMetricCombo->setCurrentIndex(0);
00926     m_climbOverlayMetricCombo->setEnabled(false);
00927
00928     m_climbsTable = new QTableWidgetItem(0, 19, climbingTab);
00929     m_climbsTable->setHorizontalHeaderLabels(
00930         { "Name", "Length (km)", "Gain (m)", "Avg Gradient %", "Max Gradient %",
00931         "Duration", "Avg Power", "W/kg", "W/kg Rank", "NP", "Avg HR", "Avg Cadence", "Avg
00932         Speed", "VAM", "Fade %", "Decouple %", "Difficulty", "Category", "Shape" });
00933     m_climbsTable->horizontalHeader()->setStretchLastSection(true);
00934     m_climbsTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
00935     m_climbsTable->setSelectionBehavior(QAbstractItemView::SelectRows);
00936     m_climbsTable->setSelectionMode(QAbstractItemView::ExtendedSelection);
00937     m_climbsTable->setAlternatingRowColors(true);
00938     m_climbsTable->setSortingEnabled(true);
00939     m_climbsTable->sortItems(0, Qt::AscendingOrder);
00940     m_climbsTable->setContextMenuPolicy(Qt::CustomContextMenu);
00941
00942     m_climbSummaryLabel = new QLabel("Climb summary: select a climb", climbingTab);
00943     m_climbSummaryLabel->setStyleSheet("color: #334155;");
00944
00945     auto* climbOverlayBar = new QWidget(climbingTab);
00946     auto* climbOverlayHL = new QHBoxLayout(climbOverlayBar);
00947     climbOverlayHL->setContentsMargins(0, 0, 0, 0);
00948     climbOverlayHL->setSpacing(6);
00949     climbOverlayHL->addWidget(new QLabel("Chart Overlay:", climbOverlayBar));
00950     climbOverlayHL->addWidget(m_climbOverlayEnabledCheck);
00951     climbOverlayHL->addWidget(m_climbOverlayMetricCombo);
00952     climbOverlayHL->addStretch(1);
00953
00954     auto* climbsBottom = new QWidget(climbingTab);
00955     auto* climbsBottomVL = new QVBoxLayout(climbsBottom);
00956     climbsBottomVL->setContentsMargins(0, 0, 0, 0);
00957     climbsBottomVL->setSpacing(4);
00958     climbsBottomVL->addWidget(new QLabel("Recognized Climbs", climbsBottom));
00959     climbsBottomVL->addWidget(m_climbsTable, 1);
00960     climbsBottomVL->addWidget(m_climbSummaryLabel);
00961
00962     auto* climbingSplit = new QSplitter(Qt::Vertical, climbingTab);
00963     climbingSplit->addWidget(climbCharts);
00964     climbingSplit->addWidget(climbsBottom);
00965     climbingSplit->setStretchFactor(0, 3);
00966     climbingSplit->setStretchFactor(1, 2);
00967     climbingSplit->setCollapsible(0, false);
00968     climbingSplit->setCollapsible(1, false);
00969
00970     QTimer::singleShot(0, climbingSplit, [climbingSplit]()
00971     {

```

```

00970         const QList<int> sizes = climbingSplit->sizes();
00971         const int total = std::accumulate(sizes.begin(), sizes.end(), 0);
00972         if (total > 0)
00973             climbingSplit->setSizes({ (total * 60) / 100, (total * 40) / 100 });
00974     });
00975
00976     climbingVL->addWidget(climbOverlayBar);
00977     climbingVL->addWidget(climbingSplit, 1);
00978
00979     auto wireDetectionChange = [this](QDoubleSpinBox* spin)
00980     {
00981         connect(spin, qOverload<double>(&QDoubleSpinBox::valueChanged),
00982                 this, [this](double)
00983                 {
00984                     detectClimbsAndRefresh();
00985                 });
00986     };
00987
00988     wireDetectionChange(m_climbMinLengthSpin);
00989     wireDetectionChange(m_climbMinGainSpin);
00990     wireDetectionChange(m_climbMinGradientSpin);
00991     wireDetectionChange(m_climbStartGradientSpin);
00992     wireDetectionChange(m_climbDipMetersSpin);
00993     wireDetectionChange(m_climbDipDistanceSpin);
00994     wireDetectionChange(m_climbSmoothingSpin);
00995
00996     connect(m_climbOverlayEnabledCheck, &QCheckBox::toggled, this, [this](bool checked)
00997     {
00998         if (!m_climbOverlayMetricCombo || !m_climbAltitudeChart)
00999             return;
01000         m_climbOverlayMetricCombo->setEnabled(checked);
01001         const ColorMetric metric =
01002             static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
01003         m_climbAltitudeChart->setMetricOverlay(metric, checked);
01004     });
01005     connect(m_climbOverlayMetricCombo, qOverload<int>(&QComboBox::currentIndexChanged), this,
01006             [this](int)
01007     {
01008         if (!m_climbOverlayEnabledCheck || !m_climbOverlayEnabledCheck->isChecked() ||
01009             !m_climbAltitudeChart || !m_climbOverlayMetricCombo)
01010             return;
01011         const ColorMetric metric =
01012             static_cast<ColorMetric>(m_climbOverlayMetricCombo->currentData().toInt());
01013         m_climbAltitudeChart->setMetricOverlay(metric, true);
01014     });
01015
01016     connect(m_climbsTable, &QTableWidget::itemSelectionChanged,
01017             this, &MainWindow::onClimbSelectionChanged);
01018     connect(m_climbsTable, &QTableWidget::itemDoubleClicked,
01019             this, &MainWindow::onClimbRowDoubleClicked);
01020     connect(m_climbsTable, &QWidget::customContextMenuRequested,
01021             this, &MainWindow::onClimbTableContextMenuRequested);
01022     connect(m_climbAltitudeChart, &RideChartWidget::climbBoundaryEdited,
01023             this, &MainWindow::onClimbBoundaryEdited);
01024     connect(m_climbAltitudeChart, &RideChartWidget::newClimbRequested,
01025             this, &MainWindow::onNewClimbRequested);
01026     connect(m_climbAltitudeChart, &RideChartWidget::climbClicked,
01027             this, [this](int climbIndex)
01028     {
01029         if (!m_climbsTable || climbIndex < 0 || climbIndex >=
01030             static_cast<int>(m_detectedClimbs.size()))
01031             return;
01032         const Climbe c = m_detectedClimbs[static_cast<size_t>(climbIndex)];
01033         for (int row = 0; row < m_climbsTable->rowCount(); ++row)
01034         {
01035             auto* item = m_climbsTable->item(row, 0);
01036             if (!item)
01037                 continue;
01038             if (std::abs(item->data(kClimbStartRole).toDouble() - c.startSeconds) < 0.75 &&
01039                 std::abs(item->data(kClimbEndRole).toDouble() - c.endSeconds) < 0.75)
01040             {
01041                 m_climbsTable->setCurrentCell(row, 0);
01042                 m_climbsTable->scrollToItem(item);
01043                 break;
01044             }
01045         }
01046     });
01047
01048     const std::initializer_list<RideChartWidget*> climbChartsAll =
01049     { m_climbAltitudeChart, m_climbPowerChart, m_climbHrChart, m_climbCadenceChart,
01050       m_climbSpeedChart };
01051     for (auto* src : climbChartsAll)
01052         for (auto* dst : climbChartsAll)
01053             if (src != dst)
01054                 connect(src, &RideChartWidget::xRangeChanged, dst, &RideChartWidget::setXRange);
01055

```

```

01051         for (auto* src : climbChartsAll)
01052             for (auto* dst : climbChartsAll)
01053                 if (src != dst)
01054                     connect(src, &RideChartWidget::crosshairMoved, dst,
&RideChartWidget::setCrosshair);
01055
01056         updateClimbQuarterCharts(nullptr);
01057
01058         // Wrap the climbing content in an empty-state guard for ClimbsPage.
01059         m_climbsPageContent = wrapWithNoActivityState(climbingTab, &m_climbingTabStack);
01060     }
01061
01062     updateChartAnalysisEmptyStates();
01063
01064     // Assemble the page stack
01065     // Each entry at index N matches NavigationSidebar::Page enum value N.
01066     // Pages that are not yet fully migrated use PlaceholderPage.
01067
01068     m_navigationSidebar = new NavigationSidebar(central);
01069     m_pageStack = new QStackedWidget(central);
01070
01071     // [0] Activities
01072     m_activitiesPage = new ActivitiesPage(m_activityBrowser, m_pageStack);
01073     m_pageStack->addWidget(m_activitiesPage);
01074
01075     // [1] Charts -- ride charts, map, intervals, laps, and notes
01076     {
01077         // Assemble a header widget containing the Color By control and the
01078         // color-legend strip. This becomes ChartsPage's top bar.
01079         auto* chartsColorBar = new QWidget;
01080         auto* chartsColorBarHL = new QHBoxLayout(chartsColorBar);
01081         chartsColorBarHL->setContentsMargins(0, 0, 0, 0);
01082         chartsColorBarHL->addLayout(colorBar);
01083         chartsColorBarHL->addWidget(m_colorLegendWidget);
01084         m_pageStack->addWidget(new ChartsPage(chartsColorBar, m_chartsPageContent, m_pageStack));
01085     }
01086
01087     // [2] Power -- Zones, Histogram, Power Curve (sub-tabbed)
01088     {
01089         m_analysisTabWidget = new QTabWidget(m_pageStack);
01090         m_analysisTabWidget->addTab(m_zonesPageContent, "Zones");
01091         m_analysisTabWidget->addTab(m_histogramPageContent, "Histogram");
01092         m_analysisTabWidget->addTab(m_pdcPageContent, "Power Curve");
01093         m_pageStack->addWidget(new PowerPage(m_analysisTabWidget, m_pageStack));
01094     }
01095
01096     // [3] Intervals -- dedicated page hosting interval controls and table.
01097     m_pageStack->addWidget(new IntervalsPage(m_intervalsPageContent, m_pageStack));
01098
01099     // [4] Climbs -- climb charts + climbs table
01100     {
01101         auto* climbsPage = new ClimbsPage(m_climbsPageContent, m_pageStack);
01102         connect(climbsPage, &ClimbsPage::addRequested, this, [this]() {
01103             {
01104                 if (m_climbAltitudeChart)
01105                     m_climbAltitudeChart->setFocus();
01106                 statusBar()->showMessage("Use the climb chart to create a new climb boundary.", 2500);
01107             }
01108             connect(climbsPage, &ClimbsPage::editRequested, this,
&MainWindow::editSelectedClimbBoundaries);
01109             connect(climbsPage, &ClimbsPage::mergeRequested, this, &MainWindow::joinSelectedClimbs);
01110             connect(climbsPage, &ClimbsPage::deleteRequested, this, &MainWindow::removeSelectedClimbs);
01111             connect(climbsPage, &ClimbsPage::undoRequested, this, [this]() {
01112                 {
01113                     if (m_undoManager)
01114                         m_undoManager->undo();
01115                 }
01116                 connect(climbsPage, &ClimbsPage::redoRequested, this, [this]() {
01117                     {
01118                         if (m_undoManager)
01119                             m_undoManager->redo();
01120                     }
01121                     m_pageStack->addWidget(climbsPage);
01122                 }
01123             }
01124
01125             // [5] Fitness -- CTL/ATL/TSB and FTP history
01126             m_pageStack->addWidget(new FitnessPage(m_fitnessChart, m_pageStack));
01127
01128             // [6] Calendar -- activity and workout planning
01129             m_pageStack->addWidget(new CalendarPage(m_calendarWidget, m_pageStack));
01130
01131             // [7] Video -- video creation and export page
01132             {
01133                 auto* videoPage = new VideoPage(m_pageStack);
01134                 connect(videoPage, &VideoPage::previewRequested, this, &MainWindow::createVideo);
01135                 connect(videoPage, &VideoPage::renderRequested, this, &MainWindow::createVideo);
01136                 connect(videoPage, &VideoPage::exportRequested, this, &MainWindow::createVideo);
01137             }
01138         }
01139     }

```

```

01136         m_pageStack->addWidget(videoPage);
01137     }
01138
01139     // Sidebar (fixed width, left) | page stack (stretching, right)
01140     auto* contentRow = new QHBoxLayout;
01141     contentRow->setContentsMargins(0, 0, 0, 0);
01142     contentRow->setSpacing(0);
01143     contentRow->addWidget(m_navigationSidebar);
01144     contentRow->addWidget(m_pageStack, 1);
01145     mainLayout->addLayout(contentRow, 1);
01146
01147     setCentralWidget(central);
01148
01149     buildToolBar();
01150     buildStatusBar();
01151
01152     connect(m_colorMetricCombo,
01153             qOverload<int>(&QComboBox::currentIndexChanged),
01154             this,
01155             [this](int)
01156             {
01157                 applyChartAndMapUpdates(true, false);
01158             });
01159
01160     resize(1050, 860);
01161     setWindowTitle("Fitlyzer C++");
01162     setWindowIcon(QIcon(":/resources/icons/fitlyzer_logo.png"));
01163 }

```

## 8.142 MainWindowVideo.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QDate>
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QMessageBox>
00009
00010 #include "charts/RideChartWidget.h"
00011 #include "core/settings/DateFormatter.h"
00012 #include "database/ActivityRepository.h"
00013 #include "database/AthleteRepository.h"
00014 #include "maps/MapRenderer.h"
00015 #include "platform/Platform.h"
00016 #include "utils/FfmpegPath.h"
00017 #include "video/VideoExportDialog.h"
00018 #include "video/VideoExportSettings.h"
00019
00020 #include <algorithm>
00021
00022 namespace
00023 {
00024     QString sanitizeVideoFilePart(QString text)
00025     {
00026         static const QString invalid = QStringLiteral("\\\\/:*?\"<>|");
00027         for (const QChar c : invalid)
00028             text.replace(c, '-');
00029         text = text.simplified().replace(' ', '-');
00030         while (text.contains("--"))
00031             text.replace("--", "-");
00032         return text.trimmed();
00033     }
00034 } // namespace
00035
00036 void MainWindow::createVideo()
00037 {
00038     const RideData& rideData = m_controller->rideData();
00039     if (rideData.records.empty())
00040     {
00041         QMessageBox::information(this, "Create Video", "Load an activity before creating a video.");
00042         return;
00043     }
00044
00045     if (m_controller->currentActivityId() <= 0 || !m_dbManager.isOpen())
00046     {
00047         QMessageBox::information(this, "Create Video", "Video export is available for saved activities.");
00048         return;
00049     }
00050
00051     auto db = m_dbManager.database();
00052     ActivityRepository actRepo(db);

```



```

00053     const Activity activity = actRepo.getActivity(m_controller->currentActivityId());
00054
00055     QString activityName = QFileInfo(activity.fileName()).completeBaseName();
00056     if (activityName.isEmpty())
00057         activityName = activity.sport;
00058     if (activityName.isEmpty())
00059         activityName = QStringLiteral("Ride");
00060
00061     QString athleteName = m_currentAthleteName.trimmed();
00062     if (athleteName.isEmpty() && m_currentAthleteId > 0)
00063     {
00064         AthleteRepository athleteRepo(db);
00065         athleteName = athleteRepo.getAthlete(m_currentAthleteId).fullName().trimmed();
00066     }
00067     if (athleteName.isEmpty())
00068         athleteName = QStringLiteral("Athlete");
00069
00070     const QDate date = !activity.startTime.isEmpty()
00071         ? QDate::fromString(activity.startTime.left(10), Qt::ISODate)
00072         : QDate::fromString(activity.importedAt.left(10), Qt::ISODate);
00073     const QString dateText = DateFormatter::toIsoDate(date.isValid() ? date : QDate::currentDate());
00074
00075     const QString defaultName = QString("%1-%2-%3.mp4")
00076         .arg(dateText,
00077             sanitizeVideoFilePart(activityName),
00078             sanitizeVideoFilePart(athleteName));
00079     const QString defaultOutput = QDir(Platform::defaultDatabaseDirectory()).filePath(defaultName);
00080
00081     const double visibleStartMin = m_powerChart ? m_powerChart->visibleRangeStartMinutes() : 0.0;
00082     const double visibleEndMin = m_powerChart ? m_powerChart->visibleRangeEndMinutes() : 0.0;
00083
00084     VideoExportSettings defaults;
00085     defaults.outputPath = defaultOutput;
00086     defaults.ffmpegPath = resolveFfmpegExecutablePath();
00087     defaults.width = 1920;
00088     defaults.height = 1080;
00089     defaults.fps = 30;
00090     defaults.playbackSpeed = 3.0;
00091     defaults.useSelectedSegment = true;
00092     defaults.segmentStartSeconds = std::max(0.0, std::min(visibleStartMin, visibleEndMin) * 60.0);
00093     defaults.segmentEndSeconds = std::max(defaults.segmentStartSeconds,
00094         std::max(visibleStartMin, visibleEndMin) * 60.0);
00095     defaults.mapStyle = m_mapRenderer ? m_mapRenderer->mapStyle() : MapStyle::Light;
00096     defaults.autoZoom = true;
00097     defaults.fixedZoom = 18;
00098     defaults.followAthlete = true;
00099     defaults.routeColorMetric = currentColorMetric() == ColorMetric::None
00100         ? ColorMetric::Power
00101         : currentColorMetric();
00102     defaults.athleteName = athleteName;
00103     defaults.activityName = activityName;
00104     defaults.colorContext = buildColorContext();
00105     defaults.rideData = rideData;
00106
00107     if (defaults.segmentEndSeconds - defaults.segmentStartSeconds < 1.0)
00108     {
00109         defaults.segmentStartSeconds = 0.0;
00110         defaults.segmentEndSeconds = rideData.records.back().elapsedSeconds;
00111     }
00112
00113     VideoExportDialog dialog(defaults, this);
00114     dialog.exec();
00115 }

```

## 8.143 MainWindowWatch.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "MainWindow.h"
00004
00005 #include <QDir>
00006 #include <QDirIterator>
00007 #include <QFileInfo>
00008 #include <QFileSystemWatcher>
00009 #include <QMimeData>
00010 #include <QStandardPaths>
00011 #include <QTimer>
00012
00013 #include <algorithm>
00014
00015 void MainWindow::dragEnterEvent(QDragEnterEvent* event)
00016 {
00017     if (!event)
00018         return;

```



```

00019
00020     if (!fitFilesFromMimeData(event->mimeType().isEmpty()))
00021         event->acceptProposedAction();
00022 }
00023
00024 void MainWindow::dropEvent(QDropEvent* event)
00025 {
00026     if (!event)
00027         return;
00028
00029     const QStringList files = fitFilesFromMimeData(event->mimeType());
00030     if (files.isEmpty())
00031         return;
00032
00033     importFiles(files);
00034     event->acceptProposedAction();
00035 }
00036
00037 void MainWindow::configureFolderWatcher()
00038 {
00039     if (!m_watcher)
00040     {
00041         m_watcher = new QFileSystemWatcher(this);
00042         connect(m_watcher, &QFileSystemWatcher::directoryChanged,
00043             this, &MainWindow::onWatchDirectoryChanged);
00044     }
00045
00046     if (!m_watchRescanTimer)
00047     {
00048         m_watchRescanTimer = new QTimer(this);
00049         m_watchRescanTimer->setSingleShot(true);
00050         m_watchRescanTimer->setInterval(700);
00051         connect(m_watchRescanTimer, &QTimer::timeout, this, [this]
00052             {
00053                 scanWatchDirectories(false);
00054             });
00055     }
00056
00057     const QStringList current = m_watcher->directories();
00058     if (!current.isEmpty())
00059         m_watcher->removePaths(current);
00060
00061     m_knownWatchedFitFiles.clear();
00062     if (!m_watchFolderEnabled)
00063         return;
00064
00065     const QStringList dirs = monitoredDirectories();
00066     QStringList validDirs;
00067     for (const QString& dirPath : dirs)
00068     {
00069         const QFileInfo info(dirPath);
00070         if (info.exists() && info.isDir())
00071             validDirs << info.absoluteFilePath();
00072     }
00073
00074     validDirs.removeDuplicates();
00075     if (!validDirs.isEmpty())
00076     {
00077         m_watcher->addPaths(validDirs);
00078         scanWatchDirectories(true);
00079     }
00080 }
00081
00082 QStringList MainWindow::monitoredDirectories() const
00083 {
00084     QStringList dirs;
00085     const QString downloads = QStandardPaths::writableLocation(QStandardPaths::DownloadLocation);
00086     const QString primary = m_watchFolderPath.trimmed().isEmpty() ? downloads : m_watchFolderPath;
00087     if (!primary.isEmpty())
00088         dirs << QDir::cleanPath(primary);
00089
00090     const QString home = QDir::homePath();
00091     const QStringList candidates = {
00092         QDir(home).filePath("Downloads/Garmin"),
00093         QDir(home).filePath("Downloads/Garmin Export"),
00094         QDir(home).filePath("Garmin/Exports")
00095     };
00096     for (const QString& candidate : candidates)
00097     {
00098         if (QFileInfo::exists(candidate))
00099             dirs << QDir::cleanPath(candidate);
00100     }
00101
00102     dirs.removeDuplicates();
00103     return dirs;
00104 }
00105

```

```

00106 void MainWindow::scanWatchDirectories(bool initialScan)
00107 {
00108     if (!m_watchFolderEnabled)
00109         return;
00110
00111     const QStringList dirs = monitoredDirectories();
00112     QSet<QString> currentSet;
00113
00114     for (const QString& dirPath : dirs)
00115     {
00116         QDirIterator it(
00117             dirPath,
00118             QStringList() << "*.fit" << "*.FIT",
00119             QDir::Files,
00120             QDirIterator::Subdirectories);
00121         while (it.hasNext())
00122             currentSet.insert(QDir::cleanPath(it.next()));
00123     }
00124
00125     if (initialScan)
00126     {
00127         m_knownWatchedFitFiles = currentSet;
00128         return;
00129     }
00130
00131     QStringList newFiles;
00132     for (const QString& path : currentSet)
00133     {
00134         if (!m_knownWatchedFitFiles.contains(path))
00135             newFiles << path;
00136     }
00137
00138     m_knownWatchedFitFiles = currentSet;
00139     if (newFiles.isEmpty())
00140         return;
00141
00142     std::sort(newFiles.begin(), newFiles.end());
00143     importFilesInternal(newFiles, false, QStringLiteral("Folder watcher"));
00144 }
00145
00146 void MainWindow::onWatchDirectoryChanged(const QString& path)
00147 {
00148     (void)path;
00149     if (!m_watchRescanTimer)
00150         return;
00151     m_watchRescanTimer->start();
00152 }

```

## 8.144 NavigationSidebar.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "NavigationSidebar.h"
00005
00006 #include <QListWidget>
00007 #include <QVBoxLayout>
00008
00009 NavigationSidebar::NavigationSidebar(QWidget* parent)
00010 : QWidget(parent)
00011 {
00012     setObjectName("navigationSidebar");
00013     setStyleSheet(
00014         "QWidget#navigationSidebar {"
00015             "    background: #1e293b;"
00016         "}"
00017         "QListWidget {"
00018             "    background: transparent;"
00019             "    border: none;"
00020             "    outline: none;"
00021             "    color: #94a3b8;"
00022             "    font-size: 13px;"
00023             "    font-weight: 500;"
00024         "}"
00025         "QListWidget::item {"
00026             "    padding: 9px 14px;"
00027             "    border-radius: 6px;"
00028             "    margin: 1px 6px;"
00029         "}"
00030         "QListWidget::item:selected {"
00031             "    background: #334155;"
00032             "    color: #f1f5f9;"
00033         "}"
00034         "QListWidget::item: hover:!selected {"

```

```

00035         " background: #273449;"
00036         " color: #e2e8f0;"
00037         "});
00038
00039     auto* layout = new QVBoxLayout(this);
00040     layout->setContentsMargins(0, 8, 0, 8);
00041
00042     m_list = new QListWidget(this);
00043     // Items must be added in the same order as the Page enum.
00044     m_list->addItem("Activities");
00045     m_list->addItem("Charts");
00046     m_list->addItem("Power");
00047     m_list->addItem("Intervals");
00048     m_list->addItem("Climbs");
00049     m_list->addItem("Fitness");
00050     m_list->addItem("Calendar");
00051     m_list->addItem("Video");
00052     m_list->setCurrentRow(static_cast<int>(Page::Activities));
00053     m_list->setFixedWidth(170);
00054     layout->addWidget(m_list);
00055
00056     connect(m_list, &QListWidget::currentRowChanged, this, [this](int row)
00057     {
00058         if (row < 0)
00059             return;
00060         emit pageSelected(static_cast<Page>(row));
00061     });
00062 }
00063
00064 void NavigationSidebar::setCurrentPage(Page page)
00065 {
00066     if (!m_list)
00067         return;
00068     m_list->setCurrentRow(static_cast<int>(page));
00069 }
00070
00071 NavigationSidebar::Page NavigationSidebar::currentPage() const
00072 {
00073     if (!m_list)
00074         return Page::Activities;
00075     const int row = m_list->currentRow();
00076     return row < 0 ? Page::Activities : static_cast<Page>(row);
00077 }

```

## 8.145 NavigationSidebar.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QWidget>
00007
00008 class QListWidget;
00009 class QListWidgetItem;
00010
00011 class NavigationSidebar : public QWidget
00012 {
00013     Q_OBJECT
00014
00015 public:
00016     enum class Page
00017     {
00018         Activities = 0,
00019         Charts,
00020         Power,
00021         Intervals,
00022         Climbs,
00023         Fitness,
00024         Calendar,
00025         Video
00026     };
00027
00028     explicit NavigationSidebar(QWidget* parent = nullptr);
00029
00030     void setCurrentPage(Page page);
00031
00032     Page currentPage() const;
00033
00034 signals:
00035     void pageSelected(NavigationSidebar::Page page);
00036
00037 private:
00038     QListWidget* m_list = nullptr;

```

```
00056 };
```

## 8.146 ActivitiesPage.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ActivitiesPage.h"
00004
00005 #include <QAction>
00006 #include <QMenu>
00007 #include <QStyle>
00008 #include <QToolBar>
00009 #include <QToolButton>
00010 #include <QVBoxLayout>
00011
00012 #include "../ActivityBrowser.h"
00013
00023 ActivitiesPage::ActivitiesPage(ActivityBrowser* browser, QWidget* parent)
00024     : BasePage(parent)
00025 {
00026     auto* layout = new QVBoxLayout(this);
00027     layout->setContentsMargins(0, 0, 0, 0);
00028     layout->setSpacing(0);
00029
00030     // Context toolbar
00031     m_toolbar = new QToolBar(this);
00032     m_toolbar->setMovable(false);
00033     m_toolbar->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00034     m_toolbar->setIconSize(QSize(16, 16));
00035
00036     // Import
00037     auto* importAct = new QAction("Import", this);
00038     importAct->setIcon(style()->standardIcon(QStyle::SP_DialogOpenButton));
00039     importAct->setToolTip("Import FIT files");
00040     connect(importAct, &QAction::triggered, this, &ActivitiesPage::importRequested);
00041     m_toolbar->addAction(importAct);
00042
00043     m_toolbar->addSeparator();
00044
00045     // Analyze dropdown
00046     auto* analyzeMenu = new QMenu(this);
00047
00048     auto* detectClimbsAct = new QAction("Detect Climbs", analyzeMenu);
00049     connect(detectClimbsAct, &QAction::triggered,
00050         this, &ActivitiesPage::detectClimbsRequested);
00051     analyzeMenu->addAction(detectClimbsAct);
00052
00053     auto* detectIntervalsAct = new QAction("Detect Intervals", analyzeMenu);
00054     connect(detectIntervalsAct, &QAction::triggered,
00055         this, &ActivitiesPage::detectIntervalsRequested);
00056     analyzeMenu->addAction(detectIntervalsAct);
00057
00058     analyzeMenu->addSeparator();
00059
00060     auto* recalcAct = new QAction("Recalculate Metrics", analyzeMenu);
00061     connect(recalcAct, &QAction::triggered,
00062         this, &ActivitiesPage::recalculateMetricsRequested);
00063     analyzeMenu->addAction(recalcAct);
00064
00065     auto* analyzeBtn = new QToolButton(m_toolbar);
00066     analyzeBtn->setText("Analyze");
00067     analyzeBtn->setIcon(style()->standardIcon(QStyle::SP_BrowserReload));
00068     analyzeBtn->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00069     analyzeBtn->setPopupMode(QToolButton::MenuButtonPopup);
00070     analyzeBtn->setMenu(analyzeMenu);
00071     // Default click triggers Detect Climbs
00072     connect(analyzeBtn, &QToolButton::clicked,
00073         this, &ActivitiesPage::detectClimbsRequested);
00074     m_toolbar->addWidget(analyzeBtn);
00075
00076     m_toolbar->addSeparator();
00077
00078     // Edit
00079     auto* editAct = new QAction("Edit", this);
00080     editAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogDetailedView));
00081     editAct->setToolTip("Edit selected activity properties");
00082     connect(editAct, &QAction::triggered, this, &ActivitiesPage::editActivityRequested);
00083     m_toolbar->addAction(editAct);
00084
00085     layout->addWidget(m_toolbar);
00086
00087     // Activity browser (takes remaining height)
00088     if (browser)
00089         layout->addWidget(browser, 1);
```

```
00090 }
```

## 8.147 ActivitiesPage.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class ActivityBrowser;
00008 class QAction;
00009 class QToolBar;
00010
00023 class ActivitiesPage : public BasePage
00024 {
00025     Q_OBJECT
00026
00027 public:
00033     explicit ActivitiesPage(ActivityBrowser* browser, QWidget* parent = nullptr);
00034
00036     QToolBar* toolbar() const { return m_toolbar; }
00037
00038 signals:
00040     void importRequested();
00041
00043     void detectClimbsRequested();
00044
00046     void detectIntervalsRequested();
00047
00049     void recalculateMetricsRequested();
00050
00052     void editActivityRequested();
00053
00054 private:
00055     QToolBar* m_toolbar = nullptr;
00056 };
```

## 8.148 BasePage.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QWidget>
00006
00032 class BasePage : public QWidget
00033 {
00034     Q_OBJECT
00035
00036 public:
00037     explicit BasePage(QWidget* parent = nullptr) : QWidget(parent) {}
00038
00045     virtual void onActivated() {}
00046 };
```

## 8.149 CalendarPage.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "CalendarPage.h"
00004
00005 #include <QVBoxLayout>
00006
00007 #include "../CalendarWidget.h"
00008
00014 CalendarPage::CalendarPage(CalendarWidget* calendarWidget, QWidget* parent)
00015     : BasePage(parent)
00016 {
00017     auto* layout = new QVBoxLayout(this);
00018     layout->setContentsMargins(0, 0, 0, 0);
00019     layout->setSpacing(0);
00020     if (calendarWidget)
00021         layout->addWidget(calendarWidget);
00022 }
```

## 8.150 CalendarPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class CalendarWidget;
00008
00018 class CalendarPage : public BasePage
00019 {
00020     Q_OBJECT
00021
00022 public:
00028     explicit CalendarPage(CalendarWidget* calendarWidget, QWidget* parent = nullptr);
00029 };

```

## 8.151 ChartsPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ChartsPage.h"
00004
00005 #include <QVBoxLayout>
00006
00019 ChartsPage::ChartsPage(QWidget* colorBarWidget,
00020                        QWidget* contentWidget,
00021                        QWidget* parent)
00022     : BasePage(parent)
00023 {
00024     auto* layout = new QVBoxLayout(this);
00025     layout->setContentsMargins(0, 0, 0, 0);
00026     layout->setSpacing(4);
00027
00028     if (colorBarWidget)
00029         layout->addWidget(colorBarWidget);
00030
00031     if (contentWidget)
00032         layout->addWidget(contentWidget, 1);
00033 }

```

## 8.152 ChartsPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class QWidget;
00008
00020 class ChartsPage : public BasePage
00021 {
00022     Q_OBJECT
00023
00024 public:
00032     explicit ChartsPage(QWidget* colorBarWidget,
00033                        QWidget* contentWidget,
00034                        QWidget* parent = nullptr);
00035 };

```

## 8.153 ClimbsPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ClimbsPage.h"
00004
00005 #include <QAction>
00006 #include <QStyle>
00007 #include <QToolBar>
00008 #include <QVBoxLayout>
00009
00019 ClimbsPage::ClimbsPage(QWidget* contentWidget,
00020                        QWidget* parent)
00021     : BasePage(parent)
00022 {

```

```

00023     auto* layout = new QVBoxLayout(this);
00024     layout->setContentsMargins(0, 0, 0, 0);
00025     layout->setSpacing(0);
00026
00027     m_toolbar = new QToolBar(this);
00028     m_toolbar->setMovable(false);
00029     m_toolbar->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00030     m_toolbar->setIconSize(QSize(16, 16));
00031
00032     auto* addAct = new QAction("Add", this);
00033     addAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogNewFolder));
00034     connect(addAct, &QAction::triggered, this, &ClimbsPage::addRequested);
00035     m_toolbar->addAction(addAct);
00036
00037     auto* editAct = new QAction("Edit", this);
00038     editAct->setIcon(style()->standardIcon(QStyle::SP_FileDialogDetailedView));
00039     connect(editAct, &QAction::triggered, this, &ClimbsPage::editRequested);
00040     m_toolbar->addAction(editAct);
00041
00042     auto* mergeAct = new QAction("Merge", this);
00043     mergeAct->setIcon(style()->standardIcon(QStyle::SP_ArrowRight));
00044     connect(mergeAct, &QAction::triggered, this, &ClimbsPage::mergeRequested);
00045     m_toolbar->addAction(mergeAct);
00046
00047     auto* deleteAct = new QAction("Delete", this);
00048     deleteAct->setIcon(style()->standardIcon(QStyle::SP_TrashIcon));
00049     connect(deleteAct, &QAction::triggered, this, &ClimbsPage::deleteRequested);
00050     m_toolbar->addAction(deleteAct);
00051
00052     m_toolbar->addSeparator();
00053
00054     auto* undoAct = new QAction("Undo", this);
00055     undoAct->setIcon(style()->standardIcon(QStyle::SP_ArrowBack));
00056     connect(undoAct, &QAction::triggered, this, &ClimbsPage::undoRequested);
00057     m_toolbar->addAction(undoAct);
00058
00059     auto* redoAct = new QAction("Redo", this);
00060     redoAct->setIcon(style()->standardIcon(QStyle::SP_ArrowForward));
00061     connect(redoAct, &QAction::triggered, this, &ClimbsPage::redoRequested);
00062     m_toolbar->addAction(redoAct);
00063
00064     layout->addWidget(m_toolbar);
00065
00066     if (contentWidget)
00067         layout->addWidget(contentWidget, 1);
00068 }

```

## 8.154 ClimbsPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class QWidget;
00008 class QToolBar;
00009
00018 class ClimbsPage : public BasePage
00019 {
00020     Q_OBJECT
00021
00022 public:
00028     explicit ClimbsPage(QWidget* contentWidget,
00029                         QWidget* parent = nullptr);
00030
00031 signals:
00032     void addRequested();
00033     void editRequested();
00034     void mergeRequested();
00035     void deleteRequested();
00036     void undoRequested();
00037     void redoRequested();
00038
00039 private:
00040     QToolBar* m_toolbar = nullptr;
00041 };

```

## 8.155 FitnessPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3

```

```

00002
00003 #include "FitnessPage.h"
00004
00005 #include <QVBoxLayout>
00006
00007 #include "../charts/FitnessChartWidget.h"
00008
00014 FitnessPage::FitnessPage(FitnessChartWidget* fitnessChart, QWidget* parent)
00015     : BasePage(parent)
00016 {
00017     auto* layout = new QVBoxLayout(this);
00018     layout->setContentsMargins(0, 0, 0, 0);
00019     layout->setSpacing(0);
00020     if (fitnessChart)
00021         layout->addWidget(fitnessChart);
00022 }

```

## 8.156 FitnessPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class FitnessChartWidget;
00008
00018 class FitnessPage : public BasePage
00019 {
00020     Q_OBJECT
00021
00022 public:
00028     explicit FitnessPage(FitnessChartWidget* fitnessChart, QWidget* parent = nullptr);
00029 };

```

## 8.157 IntervalsPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "IntervalsPage.h"
00004
00005 #include <QLabel>
00006 #include <QVBoxLayout>
00007
00018 IntervalsPage::IntervalsPage(QWidget* intervalsContent, QWidget* parent)
00019     : BasePage(parent)
00020 {
00021     auto* layout = new QVBoxLayout(this);
00022     layout->setContentsMargins(0, 0, 0, 0);
00023     layout->setSpacing(0);
00024
00025     if (intervalsContent)
00026     {
00027         layout->addWidget(intervalsContent);
00028     }
00029     else
00030     {
00031         // Placeholder: intervals are accessible from the Charts page.
00032         auto* label = new QLabel(
00033             "Interval analysis is available on the Charts page.\n"
00034             "Select an interval there to zoom the chart to that effort.",
00035             this);
00036         label->setAlignment(Qt::AlignCenter);
00037         label->setWordWrap(true);
00038         label->setStyleSheet("color: #64748b; font-size: 15px;");
00039         layout->addWidget(label);
00040     }
00041 }

```

## 8.158 IntervalsPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class QTableWidget;

```



```

00008 class QLabel;
00009 class QCheckBox;
00010 class QStackedLayout;
00011
00023 class IntervalsPage : public BasePage
00024 {
00025     Q_OBJECT
00026
00027 public:
00033     explicit IntervalsPage(QWidget* intervalsContent, QWidget* parent = nullptr);
00034 };

```

## 8.159 PlaceholderPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 #include <QLabel>
00008 #include <QVBoxLayout>
00009
00016 class PlaceholderPage : public BasePage
00017 {
00018     Q_OBJECT
00019
00020 public:
00026     explicit PlaceholderPage(const QString& text, QWidget* parent = nullptr)
00027         : BasePage(parent)
00028     {
00029         auto* layout = new QVBoxLayout(this);
00030         auto* label = new QLabel(text, this);
00031         label->setAlignment(Qt::AlignCenter);
00032         label->setStyleSheet("color: #64748b; font-size: 16px; font-weight: 600;");
00033         layout->addWidget(label);
00034     }
00035 };

```

## 8.160 PowerPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "PowerPage.h"
00004
00005 #include <QTabWidget>
00006 #include <QVBoxLayout>
00007
00017 PowerPage::PowerPage(QTabWidget* powerSubTabs, QWidget* parent)
00018     : BasePage(parent)
00019 {
00020     auto* layout = new QVBoxLayout(this);
00021     layout->setContentsMargins(0, 0, 0, 0);
00022     layout->setSpacing(0);
00023     if (powerSubTabs)
00024         layout->addWidget(powerSubTabs);
00025 }

```

## 8.161 PowerPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00007 class QTabWidget;
00008
00019 class PowerPage : public BasePage
00020 {
00021     Q_OBJECT
00022
00023 public:
00030     explicit PowerPage(QTabWidget* powerSubTabs, QWidget* parent = nullptr);
00031 };

```

## 8.162 VideoPage.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "VideoPage.h"
00004
00005 #include <QAction>
00006 #include <QFrame>
00007 #include <QHBoxLayout>
00008 #include <QLabel>
00009 #include <QStyle>
00010 #include <QToolBar>
00011 #include <QVBoxLayout>
00012
00019 VideoPage::VideoPage(QWidget* parent)
00020     : BasePage(parent)
00021 {
00022     auto* layout = new QVBoxLayout(this);
00023     layout->setContentsMargins(0, 0, 0, 0);
00024     layout->setSpacing(6);
00025
00026     // Context toolbar
00027     auto* toolbar = new QToolBar(this);
00028     toolbar->setMovable(false);
00029     toolbar->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
00030     toolbar->setIconSize(QSize(16, 16));
00031
00032     auto* previewAct = new QAction("Preview", this);
00033     previewAct->setIcon(style()->standardIcon(QStyle::SP_MediaPlay));
00034     previewAct->setToolTip("Open preview workflow");
00035     connect(previewAct, &QAction::triggered, this, &VideoPage::previewRequested);
00036     toolbar->addAction(previewAct);
00037
00038     auto* renderAct = new QAction("Render", this);
00039     renderAct->setIcon(style()->standardIcon(QStyle::SP_BrowserReload));
00040     renderAct->setToolTip("Open render workflow");
00041     connect(renderAct, &QAction::triggered, this, &VideoPage::renderRequested);
00042     toolbar->addAction(renderAct);
00043
00044     auto* exportAct = new QAction("Export", this);
00045     exportAct->setIcon(style()->standardIcon(QStyle::SP_DialogSaveButton));
00046     exportAct->setToolTip("Open export workflow");
00047     connect(exportAct, &QAction::triggered, this, &VideoPage::exportRequested);
00048     toolbar->addAction(exportAct);
00049
00050     layout->addWidget(toolbar);
00051
00052     auto makeCard = [this](const QString& title, const QString& body)
00053     {
00054         auto* card = new QFrame(this);
00055         card->setFrameShape(QFrame::StyledPanel);
00056         card->setStyleSheet(
00057             "QFrame {"
00058             "    background: #ffffff;"
00059             "    border: 1px solid #e2e8f0;"
00060             "    border-radius: 8px;"
00061             "}");
00062
00063         auto* vl = new QVBoxLayout(card);
00064         vl->setContentsMargins(14, 12, 14, 12);
00065         vl->setSpacing(6);
00066
00067         auto* heading = new QLabel(title, card);
00068         heading->setStyleSheet("font-size: 15px; font-weight: 700; color: #0f172a;");
00069         vl->addWidget(heading);
00070
00071         auto* text = new QLabel(body, card);
00072         text->setWordWrap(true);
00073         text->setStyleSheet("font-size: 13px; color: #475569;");
00074         vl->addWidget(text);
00075
00076         return card;
00077     };
00078
00079     auto* previewCard = makeCard(
00080         "Preview",
00081         "Scrub and validate overlays before rendering. Use Preview to launch the existing workflow.");
00082     auto* renderCard = makeCard(
00083         "Render Settings",
00084         "Resolution, frame rate, and overlay composition are configured in the render flow.");
00085     auto* outputCard = makeCard(
00086         "Output Settings",
00087         "Choose destination, codec profile, and export naming in the export flow.");
00088
00089     layout->addWidget(previewCard);
00090

```

```

00091     auto* row = new QHBoxLayout;
00092     row->setContentsMargins(0, 0, 0, 0);
00093     row->setSpacing(6);
00094     row->addWidget(renderCard, 1);
00095     row->addWidget(outputCard, 1);
00096     layout->addLayout(row, 1);
00097 }

```

## 8.163 VideoPage.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include "BasePage.h"
00006
00020 class VideoPage : public BasePage
00021 {
00022     Q_OBJECT
00023
00024 public:
00025     explicit VideoPage(QWidget* parent = nullptr);
00026
00027 signals:
00029     void previewRequested();
00030
00032     void renderRequested();
00033
00035     void exportRequested();
00036 };

```

## 8.164 PlannedWorkoutDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "PlannedWorkoutDialog.h"
00005 #include "core/settings/DateFormatter.h"
00006
00007 #include <QDate>
00008 #include <QDateEdit>
00009 #include <QDialogButtonBox>
00010 #include <QDoubleSpinBox>
00011 #include <QFormLayout>
00012 #include <QLineEdit>
00013 #include <QTextEdit>
00014 #include <QVBoxLayout>
00015
00016 PlannedWorkoutDialog::PlannedWorkoutDialog(QWidget* parent)
00017     : PlannedWorkoutDialog(PlannedWorkoutDraft{}, parent)
00018 {}
00019
00020 PlannedWorkoutDialog::PlannedWorkoutDialog(const PlannedWorkoutDraft& draft, QWidget* parent)
00021     : QDialog(parent)
00022 {
00023     setWindowTitle("Planned Workout");
00024     setMinimumWidth(420);
00025
00026     auto* layout = new QVBoxLayout(this);
00027     auto* form = new QFormLayout;
00028
00029     m_dateEdit = new QDateEdit(this);
00030     m_dateEdit->setCalendarPopup(true);
00031     m_dateEdit->setDisplayFormat(DateFormatter::qtDatePattern());
00032     m_dateEdit->setDate(draft.date.isValid() ? draft.date : QDate::currentDate());
00033
00034     m_titleEdit = new QLineEdit(draft.title, this);
00035     m_typeEdit = new QLineEdit(draft.type, this);
00036
00037     m_durationSpin = new QDoubleSpinBox(this);
00038     m_durationSpin->setRange(0.0, 12.0);
00039     m_durationSpin->setDecimals(2);
00040     m_durationSpin->setSuffix(" h");
00041     m_durationSpin->setValue(draft.durationSeconds / 3600.0);
00042
00043     m_tssSpin = new QDoubleSpinBox(this);
00044     m_tssSpin->setRange(0.0, 1000.0);
00045     m_tssSpin->setDecimals(1);
00046     m_tssSpin->setValue(draft.targetTss);
00047
00048     m_notesEdit = new QTextEdit(this);

```

```

00049     m_notesEdit->setMinimumHeight(90);
00050     m_notesEdit->setPlainText(draft.notes);
00051
00052     m_id = draft.id;
00053
00054     form->addRow("Date:", m_dateEdit);
00055     form->addRow("Title:", m_titleEdit);
00056     form->addRow("Type:", m_typeEdit);
00057     form->addRow("Duration:", m_durationSpin);
00058     form->addRow("Target TSS:", m_tssSpin);
00059     form->addRow("Notes:", m_notesEdit);
00060
00061     layout->addLayout(form);
00062
00063     auto* buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
00064     connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
00065     connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
00066     layout->addWidget(buttons);
00067 }
00068
00069 PlannedWorkoutDraft PlannedWorkoutDialog::draft() const
00070 {
00071     PlannedWorkoutDraft out;
00072     out.id = m_id;
00073     out.date = m_dateEdit ? m_dateEdit->date() : QDate();
00074     out.title = m_titleEdit ? m_titleEdit->text().trimmed() : QString();
00075     out.type = m_typeEdit ? m_typeEdit->text().trimmed() : QString();
00076     out.durationSeconds = (m_durationSpin ? m_durationSpin->value() : 0.0) * 3600.0;
00077     out.targetTss = m_tssSpin ? m_tssSpin->value() : 0.0;
00078     out.notes = m_notesEdit ? m_notesEdit->toPlainText().trimmed() : QString();
00079     return out;
00080 }

```

## 8.165 PlannedWorkoutDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include <QDate>
00009 #include <QString>
00010
00014 struct PlannedWorkoutDraft
00015 {
00016     // Unique identifier (-1 if new).
00017     int id = -1;
00018
00019     // Planned workout date.
00020     QDate date;
00021
00022     // Workout title.
00023     QString title;
00024
00025     // Workout type (for example Recovery, Threshold).
00026     QString type;
00027
00028     // Planned duration in seconds.
00029     double durationSeconds = 0.0;
00030
00031     // Target Training Stress Score.
00032     double targetTss = 0.0;
00033
00034     // Additional notes.
00035     QString notes;
00036 };
00037
00038 class QDateEdit;
00039 class QDoubleSpinBox;
00040 class QLineEdit;
00041 class QTextEdit;
00042
00046 class PlannedWorkoutDialog : public QDialog
00047 {
00048     Q_OBJECT
00049
00050 public:
00051     explicit PlannedWorkoutDialog(QWidget* parent = nullptr);
00052
00053     explicit PlannedWorkoutDialog(const PlannedWorkoutDraft& draft, QWidget* parent = nullptr);
00054
00055     PlannedWorkoutDraft draft() const;

```

```

00056
00057 private:
00058     QDateEdit* m_dateEdit = nullptr;
00059     QLineEdit* m_titleEdit = nullptr;
00060     QLineEdit* m_typeEdit = nullptr;
00061     QDoubleSpinBox* m_durationSpin = nullptr;
00062     QDoubleSpinBox* m_tssSpin = nullptr;
00063     QTextEdit* m_notesEdit = nullptr;
00064     int m_id = -1;
00065 };

```

## 8.166 WelcomeWidget.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "WelcomeWidget.h"
00005
00006 #include <QFrame>
00007 #include <QHBoxLayout>
00008 #include <QLabel>
00009 #include <QPushButton>
00010 #include <QVBoxLayout>
00011
00012 // Helpers
00013
00014 namespace
00015 {
00017 QFrame* makeCard(QWidget* parent)
00018 {
00019     auto* card = new QFrame(parent);
00020     card->setStyleSheet(
00021         "QFrame {"
00022         "    background: #ffffff;"
00023         "    border: 1px solid #e2e8f0;"
00024         "    border-radius: 10px;"
00025         "}");
00026     return card;
00027 }
00028
00030 QLabel* makeHeading(const QString& text, QWidget* parent)
00031 {
00032     auto* lbl = new QLabel(text, parent);
00033     lbl->setStyleSheet("font-size: 13px; font-weight: 700; color: #475569; "
00034         "text-transform: uppercase; letter-spacing: 0.05em;");
00035     return lbl;
00036 }
00037
00039 QPushButton* makePrimaryBtn(const QString& text, QWidget* parent)
00040 {
00041     auto* btn = new QPushButton(text, parent);
00042     btn->setMinimumHeight(36);
00043     btn->setStyleSheet(
00044         "QPushButton {"
00045         "    background: #1d4ed8; color: white; border: none; border-radius: 8px;"
00046         "    padding: 6px 14px; font-weight: 600; font-size: 13px;"
00047         "}"
00048         "QPushButton:hover { background: #1e40af; }"
00049         "QPushButton:pressed { background: #1e3a8a; }");
00050     return btn;
00051 }
00052
00054 QPushButton* makeSecondaryBtn(const QString& text, QWidget* parent)
00055 {
00056     auto* btn = new QPushButton(text, parent);
00057     btn->setMinimumHeight(36);
00058     btn->setStyleSheet(
00059         "QPushButton {"
00060         "    background: #f1f5f9; color: #0f172a; border: 1px solid #cbd5e1;"
00061         "    border-radius: 8px; padding: 6px 14px; font-size: 13px;"
00062         "}"
00063         "QPushButton:hover { background: #e2e8f0; }");
00064     return btn;
00065 }
00066 } // namespace
00067
00068 // WelcomeWidget
00069
00070 WelcomeWidget::WelcomeWidget(QWidget* parent)
00071 : QWidget(parent)
00072 {
00073     // Root gradient background
00074     setStyleSheet(
00075         "WelcomeWidget {"

```

```

00076         " background: qlineargradient(x1:0, y1:0, x2:1, y2:1,"
00077         " stop:0 #f8fafa, stop:1 #eef2ff);"
00078     "});";
00079
00080     auto* rootLayout = new QVBoxLayout(this);
00081     rootLayout->setContentsMargins(24, 24, 24, 24);
00082     rootLayout->setSpacing(20);
00083
00084     // App title
00085     auto* titleLabel = new QLabel("FitlyzerC", this);
00086     titleLabel->setStyleSheet(
00087         "font-size: 32px; font-weight: 800; color: #0f172a;");
00088     titleLabel->setAlignment(Qt::AlignCenter);
00089     rootLayout->addWidget(titleLabel);
00090
00091     m_subtitleLabel = new QLabel(this);
00092     m_subtitleLabel->setWordWrap(true);
00093     m_subtitleLabel->setAlignment(Qt::AlignCenter);
00094     m_subtitleLabel->setStyleSheet("font-size: 14px; color: #64748b;");
00095     rootLayout->addWidget(m_subtitleLabel);
00096
00097     // Database row: Recently Used | Pinned Databases
00098     {
00099         auto* dbRow = new QHBoxLayout;
00100         dbRow->setSpacing(16);
00101
00102         auto* recentCard = makeCard(this);
00103         auto* recentVL = new QVBoxLayout(recentCard);
00104         recentVL->setContentsMargins(16, 14, 16, 14);
00105         recentVL->setSpacing(6);
00106         recentVL->addWidget(makeHeading("Recently Used", recentCard));
00107         auto* recentPlaceholder = new QLabel(
00108             "Recent databases will appear here once opened.", recentCard);
00109         recentPlaceholder->setWordWrap(true);
00110         recentPlaceholder->setStyleSheet("color: #94a3b8; font-size: 12px;");
00111         recentVL->addWidget(recentPlaceholder);
00112         recentVL->addStretch(1);
00113
00114         auto* pinnedCard = makeCard(this);
00115         auto* pinnedVL = new QVBoxLayout(pinnedCard);
00116         pinnedVL->setContentsMargins(16, 14, 16, 14);
00117         pinnedVL->setSpacing(6);
00118         pinnedVL->addWidget(makeHeading("Pinned Databases", pinnedCard));
00119         auto* pinnedPlaceholder = new QLabel(
00120             "Pin frequently used athlete databases for quick access.", pinnedCard);
00121         pinnedPlaceholder->setWordWrap(true);
00122         pinnedPlaceholder->setStyleSheet("color: #94a3b8; font-size: 12px;");
00123         pinnedVL->addWidget(pinnedPlaceholder);
00124         pinnedVL->addStretch(1);
00125
00126         dbRow->addWidget(recentCard, 1);
00127         dbRow->addWidget(pinnedCard, 1);
00128         rootLayout->addLayout(dbRow);
00129     }
00130
00131     // Two-column row: Quick Actions | Recent Activities
00132     auto* columnsLayout = new QHBoxLayout;
00133     columnsLayout->setSpacing(16);
00134
00135     // Left: Quick Actions
00136     {
00137         auto* card = makeCard(this);
00138         auto* vl = new QVBoxLayout(card);
00139         vl->setContentsMargins(16, 16, 16, 16);
00140         vl->setSpacing(8);
00141
00142         vl->addWidget(makeHeading("Quick Actions", card));
00143
00144         m_importButton = makePrimaryBtn("Import FIT Files", card);
00145         m_openDbButton = makeSecondaryBtn("Open Database", card);
00146         m_createDbButton = makeSecondaryBtn("Create Database", card);
00147         m_manageAthletesButton = makeSecondaryBtn("Manage Athletes", card);
00148
00149         vl->addWidget(m_importButton);
00150         vl->addWidget(m_openDbButton);
00151         vl->addWidget(m_createDbButton);
00152         vl->addWidget(m_manageAthletesButton);
00153         vl->addStretch(1);
00154
00155         columnsLayout->addWidget(card, 1);
00156     }
00157
00158     // Right: Recent Activities (placeholder)
00159     {
00160         auto* card = makeCard(this);
00161         auto* vl = new QVBoxLayout(card);
00162         vl->setContentsMargins(16, 16, 16, 16);

```

```

00163         vl->setSpacing(8);
00164
00165         vl->addWidget(makeHeading("Recent Activities", card));
00166
00167         auto* placeholder = new QLabel(
00168             "Your recently imported activities\nwill appear here.", card);
00169         placeholder->setAlignment(Qt::AlignCenter);
00170         placeholder->setWordWrap(true);
00171         placeholder->setStyleSheet("color: #94a3b8; font-size: 13px;");
00172         vl->addWidget(placeholder, 1);
00173
00174         columnsLayout->addWidget(card, 2);
00175     }
00176
00177     rootLayout->addLayout(columnsLayout, 1);
00178
00179     // Sync Status (future: Garmin Connect / Strava)
00180     {
00181         auto* card = makeCard(this);
00182         auto* vl = new QVBoxLayout(card);
00183         vl->setContentsMargins(16, 12, 16, 12);
00184         vl->setSpacing(4);
00185
00186         vl->addWidget(makeHeading("Sync Status", card));
00187
00188         auto makeSyncRow = [&vl, card](const QString& name)
00189         {
00190             auto* row = new QHBoxLayout;
00191             auto* nameLabel = new QLabel(name, card);
00192             nameLabel->setStyleSheet("font-size: 13px; color: #334155;");
00193             auto* statusLabel = new QLabel("Not connected -- coming in a future update", card);
00194             statusLabel->setStyleSheet("font-size: 12px; color: #94a3b8;");
00195             row->addWidget(nameLabel);
00196             row->addStretch(1);
00197             row->addWidget(statusLabel);
00198             vl->addLayout(row);
00199         };
00200         makeSyncRow("Garmin Connect");
00201         makeSyncRow("Strava");
00202
00203         rootLayout->addWidget(card);
00204     }
00205
00206     // Connect signals
00207     connect(m_importButton, &QPushButton::clicked, this, &WelcomeWidget::importRequested);
00208     connect(m_openDbButton, &QPushButton::clicked, this, &WelcomeWidget::openDatabaseRequested);
00209     connect(m_createDbButton, &QPushButton::clicked, this, &WelcomeWidget::createDatabaseRequested);
00210     connect(m_manageAthletesButton, &QPushButton::clicked, this,
00211         &WelcomeWidget::manageAthletesRequested);
00212
00213     setFirstLaunch(true);
00214 }
00215 void WelcomeWidget::setFirstLaunch(bool firstLaunch)
00216 {
00217     if (!m_subtitleLabel)
00218         return;
00219
00220     if (firstLaunch)
00221     {
00222         m_subtitleLabel->setText(
00223             "Get started: create or open a database, then import your FIT files.");
00224     }
00225     else
00226     {
00227         m_subtitleLabel->setText(
00228             "No activities found. Import a FIT file or open a different database.");
00229     }
00230 }

```

## 8.167 WelcomeWidget.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QWidget>
00006
00007 class QLabel;
00008 class QPushButton;
00009
00026 class WelcomeWidget : public QWidget
00027 {
00028     Q_OBJECT

```

```

00029
00030 public:
00031     explicit WelcomeWidget(QWidget* parent = nullptr);
00032
00033     void setFirstLaunch(bool firstLaunch);
00034
00035 signals:
00036     void importRequested();
00037
00038     void openDatabaseRequested();
00039
00040     void createDatabaseRequested();
00041
00042     void manageAthletesRequested();
00043
00044 private:
00045     QLabel* m_subtitleLabel = nullptr;
00046     QPushButton* m_importButton = nullptr;
00047     QPushButton* m_openDbButton = nullptr;
00048     QPushButton* m_createDbButton = nullptr;
00049     QPushButton* m_manageAthletesButton = nullptr;
00050 };

```

## 8.168 WorkoutController.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "WorkoutController.h"
00004
00005 #include <QCryptographicHash>
00006 #include <QFile>
00007 #include <QFileInfo>
00008 #include <QSqlQuery>
00009
00010 #include <algorithm>
00011 #include <cmath>
00012
00013 #include "core/ActivityAnalysisFlags.h"
00014 #include "core/AnalysisVersions.h"
00015 #include "database/ActivityAnalysisSettingsRepository.h"
00016 #include "utils/UuidGenerator.h"
00017 #include "analysis/ClimbDetector.h"
00018 #include "analysis/ClimbMetrics.h"
00019 #include "analysis/TrainingLoad.h"
00020 #include "analysis/TrainingMetrics.h"
00021 #include "core/zones/ZoneCalculator.h"
00022 #include "database/ClimbRepository.h"
00023 #include "database/DatabaseManager.h"
00024 #include "database/IntervalRepository.h"
00025 #include "fit/FitParser.h"
00026 #include "model/RideDataSerializer.h"
00027 #include "workouts/WorkoutPersistenceService.h"
00028
00029 namespace
00030 {
00031     QDate activityDateForFtp(const Activity& activity)
00032     {
00033         const QString rawDate = !activity.startTime.isEmpty()
00034             ? activity.startTime.left(10)
00035             : activity.importedAt.left(10);
00036         return QDate::fromString(rawDate, Qt::ISODate);
00037     }
00038
00039     int resolveAthleteFtpForDate(AthleteRepository& repo, int athleteId, const QDate& date)
00040     {
00041         if (date.isValid())
00042         {
00043             const int historicalFtp = repo.getFtpForDate(athleteId, date);
00044             if (historicalFtp > 0)
00045                 return historicalFtp;
00046         }
00047         const Athlete athlete = repo.getAthlete(athleteId);
00048         return athlete.ftpWatts > 0 ? athlete.ftpWatts : 250;
00049     }
00050
00051 WorkoutController::WorkoutController(QObject* parent)
00052     : QObject(parent)
00053 {}
00054
00055 void WorkoutController::setFTP(double ftp)
00056 {

```



```

00060     m_ftp = ftp;
00061     if (!m_rideData.records.empty())
00062         reanalyze();
00063     emit ftpChanged(ftp);
00064 }
00065
00066 void WorkoutController::setDatabaseManager(DatabaseManager* dbManager)
00067 {
00068     m_dbManager = dbManager;
00069 }
00070
00071 void WorkoutController::setCurrentAthlete(int athleteId)
00072 {
00073     m_currentAthleteId = athleteId;
00074     if (m_dbManager && m_dbManager->isOpen() && athleteId > 0)
00075     {
00076         auto db = m_dbManager->database();
00077         QDate ftpDate = QDate::currentDate();
00078         if (m_currentActivityId > 0)
00079         {
00080             ActivityRepository actRepo(db);
00081             const Activity activity = actRepo.getActivity(m_currentActivityId);
00082             const QDate activityDate = activityDateForFtp(activity);
00083             if (activityDate.isValid())
00084                 ftpDate = activityDate;
00085         }
00086
00087         AthleteRepository repo(db);
00088         setFTP(static_cast<double>(resolveAthleteFtpForDate(repo, athleteId, ftpDate)));
00089     }
00090 }
00091
00092 bool WorkoutController::loadFile(const QString& path, QString& errorOut)
00093 {
00094     try
00095     {
00096         FitParser parser;
00097         m_rideData = parser.load(path);
00098         m_currentActivityId = -1;
00099         reanalyze();
00100         emit workoutLoaded();
00101         return true;
00102     }
00103     catch (const std::exception& ex)
00104     {
00105         errorOut = QString::fromLocal8Bit(ex.what());
00106         return false;
00107     }
00108 }
00109
00110 int WorkoutController::importFile(const QString& path, QString& errorOut,
00111                                   bool allowTimeOverlap,
00112                                   bool runAnalysis,
00113                                   const QString& importSource,
00114                                   DuplicateActivityInfo* duplicateInfo,
00115                                   ImportResult* result)
00116 {
00117     if (!m_dbManager || !m_dbManager->isOpen())
00118     {
00119         errorOut = "No database is open.";
00120         if (result) *result = ImportResult::Failed;
00121         return -1;
00122     }
00123     if (m_currentAthleteId < 0)
00124     {
00125         errorOut = "No athlete selected.";
00126         if (result) *result = ImportResult::Failed;
00127         return -1;
00128     }
00129
00130     RideData ride;
00131     try
00132     {
00133         FitParser parser;
00134         ride = parser.load(path);
00135     }
00136     catch (const std::exception& ex)
00137     {
00138         errorOut = QString::fromLocal8Bit(ex.what());
00139         if (result) *result = ImportResult::Failed;
00140         return -1;
00141     }
00142
00143     QFile f(path);
00144     QString fitHash;
00145     if (f.open(QIODevice::ReadOnly))
00146     {

```

```

00147         fitHash = QCryptographicHash::hash(f.readAll(), QCryptographicHash::Sha256)
00148             .toHex();
00149         f.close();
00150     }
00151
00152     // Level 2: Time-overlap detection
00153     if (!allowTimeOverlap && !ride.activityStartTimeUtc.isEmpty())
00154     {
00155         auto db = m_dbManager->database();
00156         QSqlQuery q(db);
00157         q.prepare(
00158             "SELECT id, activity_start_time, end_time, duration_sec "
00159             "FROM activities WHERE athlete_id=:aid AND activity_start_time=:st LIMIT 1");
00160         q.bindValue(":aid", m_currentAthleteId);
00161         q.bindValue(":st", ride.activityStartTimeUtc);
00162         if (q.exec() && q.next())
00163         {
00164             if (duplicateInfo)
00165             {
00166                 duplicateInfo->existingActivityId = q.value(0).toInt();
00167                 duplicateInfo->existingStartUtc = q.value(1).toString();
00168                 duplicateInfo->existingDurationSeconds = q.value(3).toDouble();
00169                 duplicateInfo->newStartUtc = ride.activityStartTimeUtc;
00170                 duplicateInfo->newDurationSeconds =
00171                     ride.records.empty() ? 0.0 : ride.records.back().elapsedSeconds;
00172             }
00173
00174             // Return TimeOverlap result with descriptive error message
00175             errorOut = QString("A duplicate activity with the same start time already exists");
00176             if (result) *result = ImportResult::TimeOverlap;
00177             return -1;
00178         }
00179     }
00180
00181     WorkoutAnalyzer analyzer;
00182     WorkoutStatistics stats = analyzer.analyze(ride);
00183     const double np = TrainingLoad::normalizedPower(ride);
00184
00185     auto db = m_dbManager->database();
00186     ActivityRepository actRepo(db);
00187     ImportRepository impRepo(db);
00188
00189     const int activityId = WorkoutPersistenceService::saveRide(
00190         ride, stats, m_currentAthleteId, path, fitHash,
00191         np, *m_dbManager, actRepo, impRepo, &errorOut);
00192
00193     if (activityId < 0)
00194     {
00195         if (result) *result = ImportResult::Failed;
00196         return -1;
00197     }
00198
00199     // Store fingerprint (sport + start_time + rounded duration + rounded distance)
00200     {
00201         QCryptographicHash fpHasher(QCryptographicHash::Sha256);
00202         fpHasher.addData(QStringLiteral("Cycling").toUtf8());
00203         fpHasher.addData(ride.activityStartTimeUtc.toUtf8());
00204         fpHasher.addData(QString::number(static_cast<int>(stats.durationSeconds)).toUtf8());
00205         fpHasher.addData(QString::number(static_cast<int>(stats.totalDistanceKm * 1000.0)).toUtf8());
00206         const QString fingerprint = fpHasher.result().toHex();
00207         actRepo.updateFingerprint(activityId, fingerprint);
00208     }
00209
00210     // Store import source
00211     if (!importSource.isEmpty())
00212         actRepo.setImportSource(activityId, importSource);
00213
00214     emit activityImported(activityId);
00215
00216     if (runAnalysis)
00217         loadActivity(activityId, errorOut);
00218     else if (m_analysisQueue)
00219         m_analysisQueue->enqueue(activityId);
00220
00221     if (result) *result = ImportResult::Success;
00222     return activityId;
00223 }
00224
00225 bool WorkoutController::loadActivity(int activityId, QString& errorOut)
00226 {
00227     if (!m_dbManager || !m_dbManager->isOpen())
00228     {
00229         errorOut = "No database is open.";
00230         return false;
00231     }
00232
00233     m_rideData = WorkoutPersistenceService::loadRide(activityId, *m_dbManager);

```

```

00234     if (m_rideData.records.empty())
00235     {
00236         errorOut = "Activity has no samples in the database.";
00237         return false;
00238     }
00239
00240     m_currentActivityId = activityId;
00241
00242     if (m_currentAthleteId > 0)
00243     {
00244         auto db = m_dbManager->database();
00245         ActivityRepository actRepo(db);
00246         const Activity activity = actRepo.getActivity(activityId);
00247         const QDate ftpDate = activityDateForFtp(activity);
00248
00249         AthleteRepository repo(db);
00250         m_ftp = resolveAthleteFtpForDate(
00251             repo,
00252             m_currentAthleteId,
00253             ftpDate.isValid() ? ftpDate : QDate::currentDate());
00254     }
00255
00256     reanalyze();
00257     loadOrGenerateIntervals();
00258     loadOrGenerateClimbs();
00259     emit workoutLoaded();
00260     return true;
00261 }
00262
00263 void WorkoutController::reanalyze()
00264 {
00265     m_statistics = m_analyzer.analyze(m_rideData);
00266     const TrainingMetricsSummary metrics = TrainingMetrics::compute(
00267         m_rideData,
00268         m_ftp,
00269         m_statistics.durationSeconds);
00270
00271     m_np = metrics.normalizedPower;
00272     m_if = metrics.intensityFactor;
00273     m_tss = metrics.trainingStressScore;
00274     m_vi = metrics.variabilityIndex;
00275     m_ef = metrics.efficiencyFactor;
00276
00277     ColorContext colorContext;
00278     colorContext.ftp = static_cast<int>(m_ftp);
00279     m_powerZones = ZoneCalculator::computeDistribution(
00280         m_rideData,
00281         ColorMetric::Power,
00282         colorContext);
00283     m_pdcPoints = PowerCurve::computeStandard(m_rideData);
00284
00285     IntervalDetector::Config cfg;
00286     cfg.ftp = m_ftp;
00287     m_intervals = IntervalDetector::detect(m_rideData, cfg);
00288
00289     ClimbDetector::Config climbCfg;
00290     m_climbs = ClimbDetector::detect(m_rideData, climbCfg);
00291 }
00292
00293 void WorkoutController::loadOrGenerateIntervals()
00294 {
00295     if (!m_dbManager || !m_dbManager->isOpen() || m_currentActivityId <= 0)
00296         return;
00297
00298     auto db = m_dbManager->database();
00299     IntervalRepository repo(db);
00300     const QList<IntervalRecord> stored = repo.listIntervalsForActivity(m_currentActivityId);
00301
00302     if (!stored.isEmpty())
00303     {
00304         m_intervals.clear();
00305         m_intervals.reserve(static_cast<size_t>(stored.size()));
00306
00307         for (const IntervalRecord& record : stored)
00308         {
00309             Interval iv;
00310             iv.startSeconds = record.startSample;
00311             iv.endSeconds = record.endSample;
00312             iv.durationSeconds = std::max(0.0, iv.endSeconds - iv.startSeconds);
00313             iv.averagePower = record.avgPower;
00314             iv.normalizedPower = record.np;
00315             iv.averageHeartRate = record.avgHr;
00316             iv.averageCadence = record.avgCadence;
00317             iv.label = record.type.isEmpty() ? "Interval" : record.type.toString();
00318             m_intervals.push_back(iv);
00319         }
00320         return;

```

```

00321     }
00322
00323     if (m_intervals.empty())
00324         return;
00325
00326     for (const Interval& iv : m_intervals)
00327     {
00328         IntervalRecord record;
00329         record.activityId = m_currentActivityId;
00330         record.startSample = static_cast<int>(std::round(iv.startSeconds));
00331         record.endSample = static_cast<int>(std::round(iv.endSeconds));
00332         record.name       = QString::fromStdString(iv.label);
00333         record.type       = QString::fromStdString(iv.label);
00334         record.avgPower   = iv.averagePower;
00335         record.np         = iv.normalizedPower;
00336         record.avgHr      = iv.averageHeartRate;
00337         record.avgCadence = iv.averageCadence;
00338         record.source     = QStringLiteral("auto");
00339         record.locked     = false;
00340         record.deleted    = false;
00341         record.algorithmVersion = AnalysisVersions::IntervalDetection;
00342         record.uuid       = UuidGenerator::create();
00343         repo.insertInterval(record);
00344     }
00345
00346     // Mark HasIntervals flag
00347     {
00348         auto db2 = m_dbManager->database();
00349         ActivityRepository actRepo2(db2);
00350         const Activity actForFlags = actRepo2.getActivity(m_currentActivityId);
00351         actRepo2.updateAnalysisFlags(m_currentActivityId,
00352                                     actForFlags.analysisFlags |
ActivityAnalysisFlags::HasIntervals);
00353     }
00354 }
00355
00356 void WorkoutController::reloadClimbsFromDatabase()
00357 {
00358     if (m_dbManager && m_dbManager->isOpen() && m_currentActivityId > 0)
00359     {
00360         m_climbs.clear();
00361         loadOrGenerateClimbs();
00362     }
00363 }
00364
00365 void WorkoutController::loadOrGenerateClimbs()
00366 {
00367     if (!m_dbManager || !m_dbManager->isOpen() || m_currentActivityId <= 0)
00368         return;
00369
00370     auto db = m_dbManager->database();
00371     ClimbRepository repo(db);
00372     ActivityRepository actRepo(db);
00373
00374     // Use HasClimbs flag as an authoritative "analysis was ever run" sentinel.
00375     // This prevents auto re-detect when the user has deliberately deleted all climbs.
00376     const Activity activity = actRepo.getActivity(m_currentActivityId);
00377     const bool analysisEverRun = (activity.analysisFlags & ActivityAnalysisFlags::HasClimbs) != 0
                                || repo.hasClimbsEver(m_currentActivityId);
00378
00379     if (analysisEverRun)
00380     {
00381         // Load non-deleted climbs from DB; rebuild full metrics from ride data.
00382         const QList<ClimbRecord> stored = repo.climbsForActivity(m_currentActivityId);
00383         m_climbs.clear();
00384         m_climbs.reserve(static_cast<size_t>(stored.size()));
00385
00386         if (m_rideData.records.empty())
00387             return;
00388
00389         // Build analysis vectors once with default config
00390         ClimbDetector::Config cfg;
00391         const std::vector<double> cumulative =
ClimbDetector::buildCumulativeDistanceMeters(m_rideData);
00392         const std::vector<double> smoothed = ClimbDetector::smoothAltitudeByDistance(
00393             m_rideData, cumulative, cfg.elevationSmoothingMeters);
00394         const std::vector<double> gradient = ClimbDetector::computeLocalGradientPct(
00395             smoothed, cumulative, cfg.gradientWindowHalfMeters);
00396
00397         auto nearestIdx = [this](double sec) -> int
00398         {
00399             auto it = std::lower_bound(
00400                 m_rideData.records.begin(), m_rideData.records.end(), sec,
00401                 [](const RideRecord& r, double s) { return r.elapsedSeconds < s; });
00402             if (it == m_rideData.records.end())
00403                 return static_cast<int>(m_rideData.records.size()) - 1;
00404             int idx = static_cast<int>(std::distance(m_rideData.records.begin(), it));
00405

```

```

00406         if (idx > 0)
00407         {
00408             const double curr =
std::abs(m_rideData.records[static_cast<size_t>(idx)].elapsedSeconds - sec);
00409             const double prev = std::abs(m_rideData.records[static_cast<size_t>(idx -
1)].elapsedSeconds - sec);
00410             if (prev < curr) --idx;
00411         }
00412         return idx;
00413     };
00414
00415     for (int ordinal = 0; ordinal < stored.size(); ++ordinal)
00416     {
00417         const ClimbRecord& record = stored[ordinal];
00418         int startIdx = nearestIdx(record.startSeconds);
00419         int endIdx = nearestIdx(record.endSeconds);
00420         if (endIdx < startIdx) std::swap(startIdx, endIdx);
00421         if (endIdx <= startIdx)
00422             endIdx = std::min(static_cast<int>(m_rideData.records.size()) - 1, startIdx + 1);
00423
00424         Climb c = ClimbMetrics::build(m_rideData, cumulative, smoothed, gradient,
00425                                     startIdx, endIdx, ordinal + 1);
00426         c.id = record.id;
00427         if (!record.name.isEmpty())
00428             c.name = record.name;
00429         m_climbs.push_back(c);
00430     }
00431     return;
00432 }
00433
00434 // First time: m_climbs was populated by reanalyze(). Check for resurrection, then persist.
00435 const QList<ClimbRecord> deletedClimbs = repo.deletedClimbsForActivity(m_currentActivityId);
00436
00437 for (Climb& c : m_climbs)
00438 {
00439     // Resurrection prevention: skip if overlaps a previously deleted climb
00440     bool overlapsDeleted = false;
00441     for (const ClimbRecord& d : deletedClimbs)
00442     {
00443         if (c.startSeconds < d.endSeconds && c.endSeconds > d.startSeconds)
00444         {
00445             overlapsDeleted = true;
00446             break;
00447         }
00448     }
00449     if (overlapsDeleted) continue;
00450
00451     ClimbRecord record;
00452     record.activityId = m_currentActivityId;
00453     record.uuid = UuidGenerator::create();
00454     record.startSeconds = c.startSeconds;
00455     record.endSeconds = c.endSeconds;
00456     record.originalStartSeconds = c.startSeconds;
00457     record.originalEndSeconds = c.endSeconds;
00458     record.name = c.name;
00459     record.elevationGainM = c.elevationGainM;
00460     record.lengthKm = c.lengthKm;
00461     record.averageGradient = c.averageGradient;
00462     record.avgPower = c.averagePower;
00463     record.np = c.normalizedPower;
00464     record.avgHr = c.averageHeartRate;
00465     record.avgCadence = c.averageCadence;
00466     record.vam = c.vam;
00467     record.source = QStringLiteral("auto");
00468     record.locked = false;
00469     record.algorithmVersion = AnalysisVersions::ClimbDetection;
00470     c.id = repo.insertClimb(record);
00471 }
00472
00473 // Mark HasClimbs flag
00474 if (!m_climbs.empty())
00475     actRepo.updateAnalysisFlags(m_currentActivityId,
00476                                activity.analysisFlags | ActivityAnalysisFlags::HasClimbs);
00477 }

```

## 8.169 WorkoutController.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDate>
00007 #include <QObject>

```

```

00008 #include <QString>
00009
00010 #include "analysis/IntervalDetector.h"
00011 #include "analysis/ClimbDetector.h"
00012 #include "analysis/PowerCurve.h"
00013 #include "analysis/TrainingLoad.h"
00014 #include "analysis/TrainingMetrics.h"
00015 #include "analysis/WorkoutAnalyzer.h"
00016 #include "core/zones/ZoneDefinition.h"
00017 #include "analysis/queue/AnalysisQueue.h"
00018 #include "core/AnalysisVersions.h"
00019 #include "core/ActivityAnalysisFlags.h"
00020 #include "database/ActivityRepository.h"
00021 #include "database/AthleteRepository.h"
00022 #include "database/ClimbRepository.h"
00023 #include "database/ImportRepository.h"
00024 #include "database/IntervalRepository.h"
00025 #include "fit/RideData.h"
00026
00027 class DatabaseManager;
00028
00029 enum class ImportResult
00030 {
00031     Success,
00032     Duplicate,
00033     TimeOverlap,
00034     Failed
00035 };
00036
00037 struct DuplicateActivityInfo
00038 {
00039     // Existing conflicting activity ID.
00040     int existingActivityId = -1;
00041
00042     // Existing activity start time in UTC ISO format.
00043     QString existingStartUtc;
00044
00045     // Existing activity duration in seconds.
00046     double existingDurationSeconds = 0.0;
00047
00048     // Incoming activity start time in UTC ISO format.
00049     QString newStartUtc;
00050
00051     // Incoming activity duration in seconds.
00052     double newDurationSeconds = 0.0;
00053 };
00054
00061 class WorkoutController : public QObject
00062 {
00063     Q_OBJECT
00064 public:
00065     explicit WorkoutController(QObject* parent = nullptr);
00066
00067     void setFTP(double ftp);
00068
00069     double ftp() const { return m_ftp; }
00070
00071     void setDatabaseManager(DatabaseManager* dbManager);
00072
00073     void setCurrentAthlete(int athleteId);
00074
00075     int currentAthleteId() const { return m_currentAthleteId; }
00076
00077     bool loadFile(const QString& path, QString& errorOut);
00078
00079     int importFile(const QString& path, QString& errorOut,
00080                  bool allowTimeOverlap = false,
00081                  bool runAnalysis = true,
00082                  const QString& importSource = QString(),
00083                  DuplicateActivityInfo* duplicateInfo = nullptr,
00084                  ImportResult* result = nullptr);
00085
00086     bool loadActivity(int activityId, QString& errorOut);
00087
00088     void reloadClimbsFromDatabase();
00089
00090     void setAnalysisQueue(AnalysisQueue* queue) { m_analysisQueue = queue; }
00091
00092     const RideData& rideData() const { return m_rideData; }
00093
00094     const WorkoutStatistics& statistics() const { return m_statistics; }
00095
00096     const std::vector<ZoneDistribution>& powerZones() const { return m_powerZones; }
00097
00098     const std::vector<PowerCurvePoint>& pdcPoints() const { return m_pdcPoints; }
00099
00100     const std::vector<Interval>& intervals() const { return m_intervals; }

```

```

00101
00102     const std::vector<Climb>&          climbs()          const { return m_climbs; }
00103
00104     double normalizedPower()          const { return m_np; }
00105
00106     double intensityFactor()          const { return m_if; }
00107
00108     double trainingStressScore()      const { return m_tss; }
00109
00110     double variabilityIndex()         const { return m_vi; }
00111
00112     double efficiencyFactor()         const { return m_ef; }
00113
00114     int currentActivityId()           const { return m_currentActivityId; }
00115
00116 signals:
00117     // Emitted when activity finishes loading.
00118     void workoutLoaded();
00119
00120     // Emitted when activity is imported.
00121     void activityImported(int activityId);
00122
00123     // Emitted when FTP changes.
00124     void ftpChanged(double ftp);
00125
00126 private:
00127     void reanalyze();
00128     void loadOrGenerateIntervals();
00129     void loadOrGenerateClimbs();
00130     AnalysisQueue* m_analysisQueue = nullptr;
00131
00132     double m_ftp = 250.0;
00133     int m_currentAthleteId = -1;
00134     int m_currentActivityId = -1;
00135
00136     DatabaseManager* m_dbManager = nullptr;
00137
00138     RideData m_rideData;
00139     WorkoutStatistics m_statistics;
00140     WorkoutAnalyzer m_analyzer;
00141     std::vector<ZoneDistribution> m_powerZones;
00142     std::vector<PowerCurvePoint> m_pdcPoints;
00143     std::vector<Interval> m_intervals;
00144     std::vector<Climb> m_climbs;
00145
00146     double m_np = 0.0;
00147     double m_if = 0.0;
00148     double m_tss = 0.0;
00149     double m_vi = 0.0;
00150     double m_ef = 0.0;
00151 };

```

## 8.170 ImportJob.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportJob.h"

```

## 8.171 ImportJob.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00010 struct ImportJob
00011 {
00013     enum class State
00014     {
00015         Queued,
00016         Parsing,
00017         Storing,
00018         Completed,
00019         Failed
00020     };
00021
00022     QString jobId;
00023     QString batchId;
00024     QString filePath;
00025     QString athleteId;

```

```

00026     QString importSource;
00027     State state = State::Queued;
00028     double progress = 0.0;
00029     QString errorMessage;
00030 };

```

## 8.172 ImportProgressModel.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportProgressModel.h"
00004
00005 #include <QFileInfo>
00006
00007 #include "ImportQueue.h"
00008
00009 ImportProgressModel::ImportProgressModel(QObject* parent)
00010     : QObject(parent)
00011 {}
00012
00013 void ImportProgressModel::attachQueue(ImportQueue* queue)
00014 {
00015     if (!queue)
00016         return;
00017
00018     connect(queue, &ImportQueue::jobStarted, this,
00019             [this](const QString&, const QString&, const QString& filePath, int activeCount)
00020             {
00021                 m_activeJobs = activeCount;
00022                 m_progress = 0.0;
00023                 m_currentFileName = QFileInfo(filePath).fileName();
00024                 m_statusText = QStringLiteral("Starting");
00025                 emit changed();
00026             });
00027
00028     connect(queue, &ImportQueue::jobProgress, this,
00029             [this](const QString&, const QString&, const QString& filePath,
00030                   double percentage, const QString& statusText, int activeCount)
00031             {
00032                 m_activeJobs = activeCount;
00033                 m_progress = percentage;
00034                 m_currentFileName = QFileInfo(filePath).fileName();
00035                 m_statusText = statusText;
00036                 emit changed();
00037             });
00038
00039     connect(queue, &ImportQueue::jobFinished, this,
00040             [this](const QString&, const QString&, const QString&, int, int activeCount)
00041             {
00042                 m_activeJobs = activeCount;
00043                 ++m_completedJobs;
00044                 if (m_activeJobs <= 0)
00045                 {
00046                     m_progress = 100.0;
00047                     m_statusText = QStringLiteral("Done");
00048                 }
00049                 emit changed();
00050             });
00051
00052     connect(queue, &ImportQueue::jobFailed, this,
00053             [this](const QString&, const QString&, const QString&, const QString&, const QString&, int
00054                   activeCount)
00055             {
00056                 m_activeJobs = activeCount;
00057                 ++m_failedJobs;
00058                 if (m_activeJobs <= 0)
00059                     m_statusText = QStringLiteral("Finished with errors");
00060                 emit changed();
00061             });
00062
00063     connect(queue, &ImportQueue::queueIdle, this, [this]()
00064             {
00065                 m_activeJobs = 0;
00066                 m_currentFileName.clear();
00067                 m_statusText.clear();
00068                 m_progress = 0.0;
00069                 emit changed();
00070             });

```



## 8.173 ImportProgressModel.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007
00008 class ImportQueue;
00009
00015 class ImportProgressModel : public QObject
00016 {
00017     Q_OBJECT
00018 public:
00019     explicit ImportProgressModel(QObject* parent = nullptr);
00020
00022     void attachQueue(ImportQueue* queue);
00023
00024     int activeJobs() const { return m_activeJobs; }
00025     int completedJobs() const { return m_completedJobs; }
00026     int failedJobs() const { return m_failedJobs; }
00027     double progress() const { return m_progress; }
00028     QString currentFileName() const { return m_currentFileName; }
00029     QString statusText() const { return m_statusText; }
00030
00031 signals:
00033     void changed();
00034
00035 private:
00036     int m_activeJobs = 0;
00037     int m_completedJobs = 0;
00038     int m_failedJobs = 0;
00039     double m_progress = 0.0;
00040     QString m_currentFileName;
00041     QString m_statusText;
00042 };

```

## 8.174 ImportQueue.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportQueue.h"
00004
00005 #include <QThread>
00006 #include <QUuid>
00007
00008 #include "ImportWorker.h"
00009
00010 ImportQueue::ImportQueue(QObject* parent)
00011     : QObject(parent)
00012 {
00013     m_thread = new QThread(this);
00014     m_worker = new ImportWorker;
00015     m_worker->moveToThread(m_thread);
00016
00017     connect(m_worker, &ImportWorker::progressChanged,
00018             this, &ImportQueue::onWorkerProgress);
00019     connect(m_worker, &ImportWorker::completed,
00020             this, &ImportQueue::onWorkerCompleted);
00021     connect(m_worker, &ImportWorker::failed,
00022             this, &ImportQueue::onWorkerFailed);
00023
00024     m_thread->start();
00025 }
00026
00027 ImportQueue::~ImportQueue()
00028 {
00029     m_thread->quit();
00030     m_thread->wait(3000);
00031     delete m_worker;
00032 }
00033
00034 void ImportQueue::setDatabasePath(const QString& path)
00035 {
00036     m_dbPath = path;
00037     QMetaObject::invokeMethod(m_worker, [this, path]()
00038     {
00039         m_worker->setDatabasePath(path);
00040     }, Qt::QueuedConnection);
00041 }
00042
00043 void ImportQueue::enqueue(const QString& filePath,

```

```

00044         const QString& athleteId,
00045         const QString& batchId,
00046         const QString& importSource)
00047 {
00048     ImportJob job;
00049     job.jobId = QUuid::createUuid().toString(QUuid::WithoutBraces);
00050     job.batchId = batchId;
00051     job.filePath = filePath;
00052     job.athleteId = athleteId;
00053     job.importSource = importSource;
00054     job.state = ImportJob::State::Queued;
00055     job.progress = 0.0;
00056
00057     m_queue.enqueue(job);
00058
00059     emit jobQueued(job.jobId, job.batchId, job.filePath, activeCountUnsafe());
00060     emit queueStateChanged(m_busy ? 1 : 0, m_queue.size());
00061
00062     if (!m_busy)
00063         processNext();
00064 }
00065
00066 int ImportQueue::pendingCount() const
00067 {
00068     return activeCountUnsafe();
00069 }
00070
00071 bool ImportQueue::isBusy() const
00072 {
00073     return m_busy;
00074 }
00075
00076 void ImportQueue::onWorkerProgress(const QString& jobId,
00077                                   const QString& batchId,
00078                                   const QString& filePath,
00079                                   double percentage,
00080                                   const QString& statusText)
00081 {
00082     emit jobProgress(jobId, batchId, filePath, percentage, statusText, activeCountUnsafe());
00083 }
00084
00085 void ImportQueue::onWorkerCompleted(const QString& jobId,
00086                                    const QString& batchId,
00087                                    const QString& filePath,
00088                                    int activityId)
00089 {
00090     m_busy = false;
00091     emit jobFinished(jobId, batchId, filePath, activityId, activeCountUnsafe());
00092     emit queueStateChanged(0, m_queue.size());
00093
00094     if (!m_queue.isEmpty())
00095     {
00096         processNext();
00097         return;
00098     }
00099
00100     emit queueIdle();
00101 }
00102
00103 void ImportQueue::onWorkerFailed(const QString& jobId,
00104                                 const QString& batchId,
00105                                 const QString& filePath,
00106                                 const QString& reason,
00107                                 const QString& errorMessage)
00108 {
00109     m_busy = false;
00110     emit jobFailed(jobId, batchId, filePath, reason, errorMessage, activeCountUnsafe());
00111     emit queueStateChanged(0, m_queue.size());
00112
00113     if (!m_queue.isEmpty())
00114     {
00115         processNext();
00116         return;
00117     }
00118
00119     emit queueIdle();
00120 }
00121
00122 void ImportQueue::processNext()
00123 {
00124     if (m_busy || m_queue.isEmpty())
00125         return;
00126
00127     m_busy = true;
00128     m_runningJob = m_queue.dequeue();
00129
00130     emit jobStarted(m_runningJob.jobId, m_runningJob.batchId, m_runningJob.filePath,

```

```

        activeCountUnsafe());
00131     emit queueStateChanged(1, m_queue.size());
00132
00133     const ImportJob job = m_runningJob;
00134     QMetaObject::invokeMethod(m_worker, [this, job]()
00135     {
00136         m_worker->processJob(job);
00137     }, Qt::QueuedConnection);
00138 }
00139
00140 int ImportQueue::activeCountUnsafe() const
00141 {
00142     return m_queue.size() + (m_busy ? 1 : 0);
00143 }

```

## 8.175 ImportQueue.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QQueue>
00007 #include <QString>
00008
00009 #include "ImportJob.h"
00010
00011 class ImportWorker;
00012 class QThread;
00013
00020 class ImportQueue : public QObject
00021 {
00022     Q_OBJECT
00023 public:
00024     explicit ImportQueue(QObject* parent = nullptr);
00025     ~ImportQueue() override;
00026
00028     void setDatabasePath(const QString& path);
00029
00035     void enqueue(const QString& filePath,
00036                 const QString& athleteId,
00037                 const QString& batchId,
00038                 const QString& importSource);
00039
00041     int pendingCount() const;
00042
00044     bool isBusy() const;
00045
00046 signals:
00048     void jobQueued(const QString& jobId,
00049                  const QString& batchId,
00050                  const QString& filePath,
00051                  int activeCount);
00052
00054     void jobStarted(const QString& jobId,
00055                   const QString& batchId,
00056                   const QString& filePath,
00057                   int activeCount);
00058
00060     void jobProgress(const QString& jobId,
00061                    const QString& batchId,
00062                    const QString& filePath,
00063                    double percentage,
00064                    const QString& statusText,
00065                    int activeCount);
00066
00068     void jobFinished(const QString& jobId,
00069                    const QString& batchId,
00070                    const QString& filePath,
00071                    int activityId,
00072                    int activeCount);
00073
00078     void jobFailed(const QString& jobId,
00079                  const QString& batchId,
00080                  const QString& filePath,
00081                  const QString& reason,
00082                  const QString& errorMessage,
00083                  int activeCount);
00084
00086     void queueStateChanged(int runningCount, int pendingCount);
00087
00089     void queueIdle();
00090
00091 private slots:

```

```

00092     void onWorkerProgress(const QString& jobId,
00093                           const QString& batchId,
00094                           const QString& filePath,
00095                           double percentage,
00096                           const QString& statusText);
00097
00098     void onWorkerCompleted(const QString& jobId,
00099                           const QString& batchId,
00100                           const QString& filePath,
00101                           int activityId);
00102
00103     void onWorkerFailed(const QString& jobId,
00104                        const QString& batchId,
00105                        const QString& filePath,
00106                        const QString& reason,
00107                        const QString& errorMessage);
00108
00109 private:
00110     void processNext();
00111     int activeCountUnsafe() const;
00112
00113     QThread* m_thread = nullptr;
00114     ImportWorker* m_worker = nullptr;
00115     QQueue<ImportJob> m_queue;
00116     bool m_busy = false;
00117     QString m_dbPath;
00118     ImportJob m_runningJob;
00119 };

```

## 8.176 ImportWorker.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "ImportWorker.h"
00004
00005 #include <QCryptographicHash>
00006 #include <QDateTime>
00007 #include <QFile>
00008 #include <QFileInfo>
00009 #include <QIODevice>
00010 #include <QSqlDatabase>
00011 #include <QSqlError>
00012 #include <QSqlQuery>
00013 #include <QUuid>
00014
00015 #include "analysis/TrainingLoad.h"
00016 #include "analysis/WorkoutAnalyzer.h"
00017 #include "database/ActivityRepository.h"
00018 #include "database/DatabaseConnectionGuard.h"
00019 #include "database/ImportRepository.h"
00020 #include "fit/FitParser.h"
00021
00022 namespace
00023 {
00024 void applySqlitePragmas(QSqlDatabase& db)
00025 {
00026     QSqlQuery q(db);
00027     q.exec("PRAGMA journal_mode=WAL");
00028     q.exec("PRAGMA synchronous=NORMAL");
00029     q.exec("PRAGMA temp_store=MEMORY");
00030     q.exec("PRAGMA foreign_keys=ON");
00031     q.exec("PRAGMA cache_size=-20000");
00032 }
00033 }
00034
00035 ImportWorker::ImportWorker(QObject* parent)
00036 : QObject(parent)
00037 {}
00038
00039 void ImportWorker::setDatabasePath(const QString& path)
00040 {
00041     m_dbPath = path;
00042 }
00043
00044 void ImportWorker::processJob(const ImportJob& job)
00045 {
00046     if (m_dbPath.isEmpty())
00047     {
00048         emit failed(job.jobId, job.batchId, job.filePath,
00049                    QStringLiteral("error"), QStringLiteral("No database path set."));
00050         return;
00051     }
00052
00053     const int athleteId = job.athleteId.toInt();

```

```

00054     if (athleteId <= 0)
00055     {
00056         emit failed(job.jobId, job.batchId, job.filePath,
00057                     QStringLiteral("error"), QStringLiteral("No athlete selected.));
00058         return;
00059     }
00060
00061     const QString connName =
00062     QStringLiteral("ImportWorker_%1").arg(QUuid::createUuid().toString(QUuid::WithoutBraces));
00063
00064     try
00065     {
00066         DatabaseConnectionGuard guard(connName);
00067         QSqlDatabase db = guard.database();
00068         db.setDatabaseName(m_dbPath);
00069         if (!db.open())
00070         {
00071             emit failed(job.jobId, job.batchId, job.filePath,
00072                         QStringLiteral("error"), db.lastError().text());
00073             return;
00074         }
00075
00076         applySqlitePragmas(db);
00077         emit progressChanged(job.jobId, job.batchId, job.filePath, 5.0, QStringLiteral("Opening
00078         file"));
00079
00080         QString fitHash;
00081         QFile file(job.filePath);
00082         if (file.open(QIODevice::ReadOnly))
00083         {
00084             fitHash = QCryptographicHash::hash(file.readAll(), QCryptographicHash::Sha256).toHex();
00085             file.close();
00086         }
00087
00088         ActivityRepository activityRepo(db);
00089         ImportRepository importRepo(db);
00090
00091         const auto emitLoggedFailure = [&](const QString& reason, const QString& message)
00092         {
00093             ImportRecord rec;
00094             rec.fileName = QFile::fileName(job.filePath);
00095             rec.status = reason == QStringLiteral("duplicate") || reason ==
00096             QStringLiteral("time_overlap")
00097             ? QStringLiteral("duplicate")
00098             : QStringLiteral("error");
00099             rec.errorMessage = message;
00100             importRepo.logImport(rec);
00101
00102             emit failed(job.jobId, job.batchId, job.filePath, reason, message);
00103         };
00104
00105         if (!fitHash.isEmpty() && activityRepo.hasFitHash(fitHash))
00106         {
00107             emitLoggedFailure(QStringLiteral("duplicate"),
00108                             QStringLiteral("Already imported (duplicate file).));
00109             return;
00110         }
00111
00112         emit progressChanged(job.jobId, job.batchId, job.filePath, 15.0, QStringLiteral("Parsing
00113         FIT"));
00114
00115         FitParser parser;
00116         RideData ride = parser.load(job.filePath);
00117
00118         if (!ride.activityStartTimeUtc.isEmpty())
00119         {
00120             QSqlQuery overlapQ(db);
00121             overlapQ.prepare(
00122                 "SELECT id FROM activities "
00123                 "WHERE athlete_id=:aid AND activity_start_time=:st LIMIT 1");
00124             overlapQ.bindValue(":aid", athleteId);
00125             overlapQ.bindValue(":st", ride.activityStartTimeUtc);
00126             if (overlapQ.exec() && overlapQ.next())
00127             {
00128                 emitLoggedFailure(QStringLiteral("time_overlap"),
00129                                 QStringLiteral("Potential duplicate activity (matching start
00130             time).));
00131                 return;
00132             }
00133         }
00134
00135         emit progressChanged(job.jobId, job.batchId, job.filePath, 72.0, QStringLiteral("Storing
00136         activity"));
00137
00138         WorkoutAnalyzer analyzer;
00139         const WorkoutStatistics stats = analyzer.analyze(ride);
00140         const double np = TrainingLoad::normalizedPower(ride);

```

```

00135
00136     const QString now = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00137
00138     Activity act;
00139     act.athleteId = athleteId;
00140     act.fitHash = fitHash;
00141     act.fileName = QFile::fileName(job.filePath).fileName();
00142     act.sport = QStringLiteral("Cycling");
00143     act.startTime = ride.activityStartTimeUtc.isEmpty() ? now : ride.activityStartTimeUtc;
00144     act.endTime = ride.activityEndTimeUtc.isEmpty() ? act.startTime : ride.activityEndTimeUtc;
00145     act.durationSec = stats.durationSeconds;
00146     act.distanceM = stats.totalDistanceKm * 1000.0;
00147     act.avgPower = stats.averagePower;
00148     act.maxPower = stats.maximumPower;
00149     act.normalizedPower = np;
00150     act.avgHr = stats.averageHeartRate;
00151     act.maxHr = stats.maximumHeartRate;
00152     act.avgCadence = stats.averageCadence;
00153     act.avgSpeed = stats.averageSpeed;
00154     act.elevationGain = stats.elevationGainM;
00155     act.importedAt = now;
00156
00157     if (!db.transaction())
00158     {
00159         emitLoggedFailure(QStringLiteral("error"), db.lastError().text());
00160         return;
00161     }
00162
00163     const int activityId = activityRepo.insertActivity(act);
00164     if (activityId <= 0)
00165     {
00166         db.rollback();
00167         emitLoggedFailure(QStringLiteral("error"), QStringLiteral("Failed to insert activity."));
00168         return;
00169     }
00170
00171     QCryptographicHash fingerprintHasher(QCryptographicHash::Sha256);
00172     fingerprintHasher.addData(QStringLiteral("Cycling").toUtf8());
00173     fingerprintHasher.addData(ride.activityStartTimeUtc.toUtf8());
00174     fingerprintHasher.addData(QString::number(static_cast<int>(stats.durationSeconds)).toUtf8());
00175     fingerprintHasher.addData(QString::number(static_cast<int>(stats.totalDistanceKm *
1000.0)).toUtf8());
00176     activityRepo.updateFingerprint(activityId, fingerprintHasher.result().toHex());
00177
00178     QSqlQuery sampleQ(db);
00179     sampleQ.prepare(
00180         "INSERT INTO activity_samples("
00181         "    activity_id, elapsed_seconds,"
00182         "    power_total, heart_rate, cadence, speed,"
00183         "    latitude, longitude, altitude,"
00184         "    has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00185         ") VALUES("
00186         "    :aid, :es,"
00187         "    :pw, :hr, :cad, :spd,"
00188         "    :lat, :lon, :alt,"
00189         "    :hgps, :hpw, :hhr, :hcad, :hspd, :halt"
00190         ")");
00191
00192     QVariantList aids, es_, pw_, hr_, cad_, spd_;
00193     QVariantList lat_, lon_, alt_;
00194     QVariantList hgps, hpw, hhr, hcad, hspd, halt;
00195
00196     for (const RideRecord& r : ride.records)
00197     {
00198         aids << activityId;
00199         es_ << r.elapsedSeconds;
00200         pw_ << (r.hasPower ? QVariant(r.power) : QVariant());
00201         hr_ << (r.hasHeartRate ? QVariant(r.heartRate) : QVariant());
00202         cad_ << (r.hasCadence ? QVariant(r.cadence) : QVariant());
00203         spd_ << (r.hasSpeed ? QVariant(r.speed) : QVariant());
00204         lat_ << (r.hasGps ? QVariant(r.latitude) : QVariant());
00205         lon_ << (r.hasGps ? QVariant(r.longitude) : QVariant());
00206         alt_ << (r.hasAltitude ? QVariant(r.altitude) : QVariant());
00207         hgps << (r.hasGps ? 1 : 0);
00208         hpw << (r.hasPower ? 1 : 0);
00209         hhr << (r.hasHeartRate ? 1 : 0);
00210         hcad << (r.hasCadence ? 1 : 0);
00211         hspd << (r.hasSpeed ? 1 : 0);
00212         halt << (r.hasAltitude ? 1 : 0);
00213     }
00214
00215     sampleQ.bindValue(":aid", aids);
00216     sampleQ.bindValue(":es", es_);
00217     sampleQ.bindValue(":pw", pw_);
00218     sampleQ.bindValue(":hr", hr_);
00219     sampleQ.bindValue(":cad", cad_);
00220     sampleQ.bindValue(":spd", spd_);

```

```

00221         sampleQ.bindValue(":lat", lat_);
00222         sampleQ.bindValue(":lon", lon_);
00223         sampleQ.bindValue(":alt", alt_);
00224         sampleQ.bindValue(":hgps", hgps);
00225         sampleQ.bindValue(":hpw", hpw);
00226         sampleQ.bindValue(":hhr", hhr);
00227         sampleQ.bindValue(":hcad", hcad);
00228         sampleQ.bindValue(":hspd", hspd);
00229         sampleQ.bindValue(":halt", halt);
00230
00231         if (!sampleQ.execBatch())
00232         {
00233             db.rollback();
00234             emitLoggedFailure(QStringLiteral("error"),
00235                               QStringLiteral("Failed to insert samples:
%1").arg(sampleQ.lastError().text()));
00236             return;
00237         }
00238
00239         // Store optional import source metadata.
00240         if (!job.importSource.isEmpty())
00241             activityRepo.setImportSource(activityId, job.importSource);
00242
00243         emit progressChanged(job.jobId, job.batchId, job.filePath, 96.0,
00244                             QStringLiteral("Committing"));
00245
00246         if (!db.commit())
00247         {
00248             db.rollback();
00249             emitLoggedFailure(QStringLiteral("error"), db.lastError().text());
00250             return;
00251         }
00252         ImportRecord rec;
00253         rec.fileName = act.fileName;
00254         rec.status = QStringLiteral("success");
00255         rec.activityId = activityId;
00256         importRepo.logImport(rec);
00257
00258         emit progressChanged(job.jobId, job.batchId, job.filePath, 100.0, QStringLiteral("Done"));
00259         emit completed(job.jobId, job.batchId, job.filePath, activityId);
00260     }
00261     catch (const std::exception& ex)
00262     {
00263         emit failed(job.jobId, job.batchId, job.filePath,
00264                     QStringLiteral("error"), QString::fromLocal8Bit(ex.what()));
00265     }
00266     catch (...)
00267     {
00268         emit failed(job.jobId, job.batchId, job.filePath,
00269                     QStringLiteral("error"), QStringLiteral("Unknown import error.));
00270     }
00271 }

```

## 8.177 ImportWorker.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QObject>
00006 #include <QString>
00007
00008 #include "ImportJob.h"
00009
00016 class ImportWorker : public QObject
00017 {
00018     Q_OBJECT
00019 public:
00020     explicit ImportWorker(QObject* parent = nullptr);
00021
00022     void setDatabasePath(const QString& path);
00023
00024 public slots:
00025     void processJob(const ImportJob& job);
00026
00027 signals:
00031     void progressChanged(const QString& jobId,
00032                          const QString& batchId,
00033                          const QString& filePath,
00034                          double percentage,
00035                          const QString& statusText);
00036
00038     void completed(const QString& jobId,

```

```

00039             const QString& batchId,
00040             const QString& filePath,
00041             int activityId);
00042
00043 void failed(const QString& jobId,
00044            const QString& batchId,
00045            const QString& filePath,
00046            const QString& reason,
00047            const QString& errorMessage);
00048
00049 private:
00050     QString m_dbPath;
00051 };

```

## 8.178 DiskTileCache.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "DiskTileCache.h"
00004
00005 #include <QDir>
00006 #include <QFileInfo>
00007
00008 namespace DiskTileCache
00009 {
00010     QString diskTilePathForRoot(const QString& root,
00011                                const QString& style,
00012                                int z,
00013                                int x,
00014                                int y)
00015     {
00016         return QDir(root).filePath(QString("%1/%2/%3/%4.png")
00017                                     .arg(style)
00018                                     .arg(z)
00019                                     .arg(x)
00020                                     .arg(y));
00021     }
00022
00023 bool isTileCachedOnDisk(const QString& primaryRoot,
00024                        const QStringList& fallbackRoots,
00025                        const QString& style,
00026                        int z,
00027                        int x,
00028                        int y)
00029 {
00030     if (QFileInfo::exists(diskTilePathForRoot(primaryRoot, style, z, x, y)))
00031         return true;
00032
00033     for (const QString& fallbackRoot : fallbackRoots)
00034     {
00035         if (fallbackRoot.trimmed().isEmpty())
00036             continue;
00037         if (QFileInfo::exists(diskTilePathForRoot(fallbackRoot, style, z, x, y)))
00038             return true;
00039     }
00040
00041     return false;
00042 }
00043 }

```

## 8.179 DiskTileCache.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006 #include <QStringList>
00007
00008 namespace DiskTileCache
00009 {
00010     QString diskTilePathForRoot(const QString& root,
00011                                const QString& style,
00012                                int z,
00013                                int x,
00014                                int y);
00015
00016 bool isTileCachedOnDisk(const QString& primaryRoot,
00017                        const QStringList& fallbackRoots,
00018                        const QString& style,
00019                        int z,

```



```

00020             int x,
00021             int y);
00022 }

```

## 8.180 GpsCacheBuilder.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "GpsCacheBuilder.h"
00004
00005 namespace GpsCacheBuilder
00006 {
00007 GpsCacheData build(const RideData& rideData)
00008 {
00009     GpsCacheData cache;
00010     cache.records.reserve(rideData.records.size());
00011
00012     for (const auto& r : rideData.records)
00013     {
00014         if (!r.hasGps)
00015             continue;
00016
00017         cache.records.push_back(&r);
00018         if (!cache.firstRecord)
00019             cache.firstRecord = &r;
00020         cache.lastRecord = &r;
00021     }
00022
00023     return cache;
00024 }
00025 }

```

## 8.181 GpsCacheBuilder.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <vector>
00006
00007 #include "fit/RideData.h"
00008
00009 struct GpsCacheData
00010 {
00011     std::vector<const RideRecord*> records;
00012     const RideRecord* firstRecord = nullptr;
00013     const RideRecord* lastRecord = nullptr;
00014 };
00015
00016 namespace GpsCacheBuilder
00017 {
00018 GpsCacheData build(const RideData& rideData);
00019 }

```

## 8.182 MapFitMath.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MapFitMath.h"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008
00009 namespace
00010 {
00011 constexpr double kPi = 3.14159265358979323846;
00012 constexpr int kTilePx = 256;
00013 constexpr double kMinLatSpan = 0.001;
00014 constexpr double kMinLonSpan = 0.001;
00015 constexpr double kMaxMercatorLat = 85.051129;
00016
00017 double normalizeLongitude(double lon)
00018 {
00019     double normalized = std::fmod(lon + 180.0, 360.0);
00020     if (normalized < 0.0)
00021         normalized += 360.0;
00022     return normalized - 180.0;
00023 }

```

```

00023 }
00024
00025 double latToTileY(double lat)
00026 {
00027     const double clamped = std::clamp(lat, -kMaxMercatorLat, kMaxMercatorLat);
00028     const double rad = clamped * kPi / 180.0;
00029     return (1.0 - std::log(std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0;
00030 }
00031
00032 double tileYToLat(double tileY)
00033 {
00034     const double mercN = kPi * (1.0 - 2.0 * tileY);
00035     return std::atan(std::sinh(mercN)) * 180.0 / kPi;
00036 }
00037 }
00038
00039 namespace MapFitMath
00040 {
00041 FitResult computeFitToBounds(double minLat,
00042                               double maxLat,
00043                               double minLon,
00044                               double maxLon,
00045                               int viewportWidth,
00046                               int viewportHeight,
00047                               int maxProviderZoom)
00048 {
00049     if (maxLat < minLat)
00050         std::swap(minLat, maxLat);
00051     if (maxLon < minLon)
00052         std::swap(minLon, maxLon);
00053
00054     if (maxLat - minLat < kMinLatSpan)
00055     {
00056         const double center = (minLat + maxLat) / 2.0;
00057         minLat = center - kMinLatSpan / 2.0;
00058         maxLat = center + kMinLatSpan / 2.0;
00059     }
00060     if (maxLon - minLon < kMinLonSpan)
00061     {
00062         const double center = (minLon + maxLon) / 2.0;
00063         minLon = center - kMinLonSpan / 2.0;
00064         maxLon = center + kMinLonSpan / 2.0;
00065     }
00066
00067     minLat = std::clamp(minLat, -kMaxMercatorLat, kMaxMercatorLat);
00068     maxLat = std::clamp(maxLat, -kMaxMercatorLat, kMaxMercatorLat);
00069     minLon = normalizeLongitude(minLon);
00070     maxLon = normalizeLongitude(maxLon);
00071
00072     FitResult result;
00073     result.minLat = minLat;
00074     result.maxLat = maxLat;
00075     result.minLon = minLon;
00076     result.maxLon = maxLon;
00077
00078     double lonSpan = result.maxLon - result.minLon;
00079     if (lonSpan < 0.0)
00080         lonSpan += 360.0;
00081     const bool wrapsDateline = lonSpan > 180.0;
00082     if (wrapsDateline)
00083         lonSpan = 360.0 - lonSpan;
00084     result.lonSpanDegrees = lonSpan;
00085
00086     const double northTileY = latToTileY(result.maxLat);
00087     const double southTileY = latToTileY(result.minLat);
00088     const double centerTileY = (northTileY + southTileY) / 2.0;
00089     result.centerLat = std::clamp(tileYToLat(centerTileY), -kMaxMercatorLat, kMaxMercatorLat);
00090
00091     if (wrapsDateline)
00092     {
00093         double shiftedMin = result.minLon;
00094         double shiftedMax = result.maxLon;
00095         if (shiftedMin < 0.0)
00096             shiftedMin += 360.0;
00097         if (shiftedMax < 0.0)
00098             shiftedMax += 360.0;
00099         if (shiftedMin > shiftedMax)
00100             std::swap(shiftedMin, shiftedMax);
00101         result.centerLon = normalizeLongitude((shiftedMin + shiftedMax) / 2.0);
00102     }
00103     else
00104     {
00105         result.centerLon = normalizeLongitude((result.minLon + result.maxLon) / 2.0);
00106     }
00107
00108     const int w = std::max(viewportWidth, 800);
00109     const int h = std::max(viewportHeight, 400);

```

```

00110
00111     if (result.maxLat <= result.minLat || result.lonSpanDegrees <= 0.0)
00112     {
00113         result.zoom = std::min(15, maxProviderZoom);
00114         return result;
00115     }
00116
00117     const int cappedMaxZoom = std::clamp(maxProviderZoom, 1, 18);
00118     result.zoom = 1;
00119     for (int z = cappedMaxZoom; z >= 1; --z)
00120     {
00121         const double n = std::pow(2.0, z);
00122         const double pixW = (result.lonSpanDegrees / 360.0) * n * kTilePx;
00123         const double pixH = std::abs(southTileY - northTileY) * n * kTilePx;
00124         if (pixW <= w * 0.85 && pixH <= h * 0.85)
00125         {
00126             result.zoom = z;
00127             break;
00128         }
00129     }
00130
00131     return result;
00132 }
00133 }

```

## 8.183 MapFitMath.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 namespace MapFitMath
00007 {
00011 struct FitResult
00012 {
00014     double minLat = 0.0;
00015
00017     double maxLat = 0.0;
00018
00020     double minLon = 0.0;
00021
00023     double maxLon = 0.0;
00024
00026     double centerLat = 0.0;
00027
00029     double centerLon = 0.0;
00030
00032     double lonSpanDegrees = 0.0;
00033
00035     int zoom = 1;
00036 };
00037
00049 FitResult computeFitToBounds(double minLat,
00050                             double maxLat,
00051                             double minLon,
00052                             double maxLon,
00053                             int viewportWidth,
00054                             int viewportHeight,
00055                             int maxProviderZoom);
00056 }

```

## 8.184 MapRenderer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "MapRenderer.h"
00005
00006 #include <QLineF>
00007 #include <QMouseEvent>
00008 #include <QPainter>
00009 #include <QPainterPath>
00010 #include <QWheelEvent>
00011
00012 #include "core/settings/AppSettings.h"
00013 #include "core/zones/ColorProvider.h"
00014 #include "maps/cache/GpsCacheBuilder.h"
00015 #include "maps/MapFitMath.h"
00016 #include "maps/rendering/RouteColorizer.h"
00017

```

```

00018 #include <algorithm>
00019 #include <cmath>
00020 #include <limits>
00021
00022 using namespace RouteColorizer;
00023
00025 static constexpr double kPi      = 3.14159265358979323846;
00026
00028 static constexpr int    kTilePx = 256;
00029
00031 static constexpr qreal  kMarkerHitSizePx = 24.0;
00032
00034 static constexpr qreal  kMarkerRadiusPx = 6.5;
00035
00037 static constexpr qreal  kNearestRecordMaxDistancePx = 40.0;
00038
00040 static constexpr int    kEdgePanMargin = 60;
00041
00042 QPointF MapRenderer::latLonToTile(double lat, double lon, int zoom)
00043 {
00044     const double n    = std::pow(2.0, zoom);
00045     const double x    = (lon + 180.0) / 360.0 * n;
00046     const double rad = lat * kPi / 180.0;
00047     const double y    = (1.0 - std::log(
00048         std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0 * n;
00049     return { x, y };
00050 }
00051
00052 QPointF MapRenderer::tileToScreen(const QPointF& tilePt) const
00053 {
00054     const auto centre = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00055     return {
00056         width() / 2.0 + (tilePt.x() - centre.x()) * kTilePx,
00057         height() / 2.0 + (tilePt.y() - centre.y()) * kTilePx
00058     };
00059 }
00060
00061 MapRenderer::MapRenderer(QWidget* parent)
00062     : QWidget(parent)
00063 {
00064     setMinimumHeight(300);
00065
00066     // Apply persisted tile cache size.
00067     m_tileCache.setMemoryCacheSize(AppSettings::instance().tileCacheSize());
00068
00069     // Coalesce tile-download repaints: many tiles arriving at once
00070     // produce one repaint 50 ms after the last arrival, not one per tile.
00071     m_tileRepaintTimer.setSingleShot(true);
00072     m_tileRepaintTimer.setInterval(50);
00073     connect(&m_tileRepaintTimer, &QTimer::timeout, this, [this] { update(); });
00074
00075     connect(&m_tileCache, &TileCache::tileLoaded,
00076         this, [this](int, int, int) { m_tileRepaintTimer.start(); });
00077 }
00078
00079 // Precompute per-segment base colors (excluding map-style adjustment which changes
00080 // per-repaint). Called once on setRideData() and whenever the color mode changes.
00081 void MapRenderer::rebuildSegmentColors()
00082 {
00083     if (m_gpsRecords.size() < 2 || m_routeColorMode == RouteColorMode::None)
00084     {
00085         m_segmentColors.clear();
00086         return;
00087     }
00088
00089     m_segmentColors.resize(m_gpsRecords.size() - 1);
00090
00091     for (size_t i = 0; i + 1 < m_gpsRecords.size(); ++i)
00092     {
00093         const RideRecord& cur  = *m_gpsRecords[i];
00094         const RideRecord& next = *m_gpsRecords[i + 1];
00095
00096         double value = 0.0;
00097         bool hasValue = false;
00098
00099         switch (m_routeColorMode)
00100         {
00101             case RouteColorMode::None: break;
00102             case RouteColorMode::Power:
00103                 if (cur.hasPower) { value = cur.power; hasValue = true; }
00104                 else if (next.hasPower) { value = next.power; hasValue = true; }
00105                 break;
00106             case RouteColorMode::HeartRate:
00107                 if (cur.hasHeartRate) { value = cur.heartRate; hasValue = true; }
00108                 else if (next.hasHeartRate) { value = next.heartRate; hasValue = true; }
00109                 break;
00110             case RouteColorMode::Cadence:

```

```

00111         if (cur.hasCadence) { value = cur.cadence; hasValue = true; }
00112         else if (next.hasCadence) { value = next.cadence; hasValue = true; }
00113         break;
00114     case RouteColorMode::Speed:
00115         if (cur.hasSpeed) { value = cur.speed; hasValue = true; }
00116         else if (next.hasSpeed) { value = next.speed; hasValue = true; }
00117         break;
00118     case RouteColorMode::Altitude:
00119         if (cur.hasAltitude) { value = cur.altitude; hasValue = true; }
00120         else if (next.hasAltitude) { value = next.altitude; hasValue = true; }
00121         break;
00122     case RouteColorMode::Gradient:
00123         if (cur.hasGps && next.hasGps && cur.hasAltitude && next.hasAltitude)
00124         {
00125             const double lat1 = cur.latitude * kPi / 180.0;
00126             const double lat2 = next.latitude * kPi / 180.0;
00127             const double dLat = lat2 - lat1;
00128             const double dLon = (next.longitude - cur.longitude) * kPi / 180.0;
00129             const double a = std::sin(dLat / 2.0) * std::sin(dLat / 2.0)
00130                 + std::cos(lat1) * std::cos(lat2)
00131                 * std::sin(dLon / 2.0) * std::sin(dLon / 2.0);
00132             const double dist = 2.0 * 6371000.0
00133                 * std::atan2(std::sqrt(a), std::sqrt(std::max(0.0, 1.0 - a)));
00134             const double elev = next.altitude - cur.altitude;
00135             if (dist > 0.1) { value = (elev / dist) * 100.0; hasValue = true; }
00136         }
00137         break;
00138     }
00139     m_segmentColors[i] = hasValue
00140         ? (m_routeColorMode == RouteColorMode::Gradient
00141            ? gradientColorForSlope(value)
00142            : ColorProvider::colorForMetric(
00143                routeModeToMetric(m_routeColorMode), value, m_colorContext))
00144         : QColor(); // invalid = no data
00145 }
00146 }
00147 }
00148
00149 void MapRenderer::rebuildGpsCache()
00150 {
00151     const GpsCacheData cache = GpsCacheBuilder::build(m_rideData);
00152     m_gpsRecords = cache.records;
00153     m_firstGpsRecord = cache.firstRecord;
00154     m_lastGpsRecord = cache.lastRecord;
00155 }
00156
00157 // Nearest GPS record to a given elapsed time: O(log N).
00158 const RideRecord* MapRenderer::gpsRecordAtTime(double elapsedSeconds) const
00159 {
00160     if (m_gpsRecords.empty())
00161         return nullptr;
00162
00163     auto it = std::lower_bound(
00164         m_gpsRecords.begin(), m_gpsRecords.end(), elapsedSeconds,
00165         [](const RideRecord* r, double t) { return r->elapsedSeconds < t; });
00166
00167     if (it == m_gpsRecords.end())
00168         return m_gpsRecords.back();
00169     if (it == m_gpsRecords.begin())
00170         return m_gpsRecords.front();
00171
00172     auto prev = std::prev(it);
00173     return (std::abs((*prev)->elapsedSeconds - elapsedSeconds) <=
00174         std::abs(*it->elapsedSeconds - elapsedSeconds))
00175         ? *prev : *it;
00176 }
00177
00178 void MapRenderer::setRideData(const RideData& rideData,
00179     ColorMetric colorMetric,
00180     const ColorContext& colorContext)
00181 {
00182     m_rideData = rideData;
00183     m_hasGps = false;
00184     m_userMovedMap = false;
00185     m_autoFitVisibleRange = true;
00186     m_visibleStartSeconds = -1.0;
00187     m_visibleEndSeconds = -1.0;
00188     m_currentTimeSeconds = -1.0;
00189     m_highlightElapsedSeconds = -1.0;
00190     m_dragHandle = DragHandle::None;
00191     m_startMarkerRect = QRectF();
00192     m_endMarkerRect = QRectF();
00193     m_draggingSelection = false;
00194     m_routeColorMode = colorMetric == ColorMetric::None
00195         ? RouteColorMode::None
00196         : colorMetric == ColorMetric::Power
00197         ? RouteColorMode::Power

```

```

00198         : colorMetric == ColorMetric::HeartRate
00199         ? RouteColorMode::HeartRate
00200         : colorMetric == ColorMetric::Cadence
00201         ? RouteColorMode::Cadence
00202         : colorMetric == ColorMetric::Speed
00203         ? RouteColorMode::Speed
00204         : RouteColorMode::Altitude;
00205     m_colorContext = colorContext;
00206
00207     rebuildGpsCache();
00208     m_hasGps = !m_gpsRecords.empty();
00209     rebuildSegmentColors();
00210
00211     if (m_hasGps)
00212         fitToTrack();
00213
00214     update();
00215 }
00216
00217 void MapRenderer::setVisibleTimeRange(double startSeconds, double endSeconds)
00218 {
00219     m_visibleStartSeconds = std::min(startSeconds, endSeconds);
00220     m_visibleEndSeconds = std::max(startSeconds, endSeconds);
00221
00222     if (m_autoFitVisibleRange && !m_userMovedMap)
00223         fitToVisibleRange();
00224
00225     update();
00226 }
00227
00228 void MapRenderer::setCurrentTime(double seconds)
00229 {
00230     m_currentTimeSeconds = seconds;
00231     update();
00232 }
00233
00234 void MapRenderer::setRouteColorMode(RouteColorMode mode, const ColorContext& colorContext)
00235 {
00236     m_routeColorMode = mode;
00237     m_colorContext = colorContext;
00238     rebuildSegmentColors();
00239     update();
00240 }
00241
00242 void MapRenderer::setMapStyle(MapStyle style)
00243 {
00244     if (m_tileCache.mapStyle() == style)
00245         return;
00246
00247     m_tileCache.setMapStyle(style);
00248     m_zoom = std::clamp(m_zoom, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00249     update();
00250 }
00251
00252 void MapRenderer::setAutoRouteContrast(bool enabled)
00253 {
00254     if (m_autoRouteContrast == enabled)
00255         return;
00256     m_autoRouteContrast = enabled;
00257     update();
00258 }
00259
00260 void MapRenderer::setTileCacheSize(int maxTiles)
00261 {
00262     m_tileCache.setMemoryCacheSize(maxTiles);
00263 }
00264
00265 QColor MapRenderer::adjustRouteColorForStyle(const QColor& color) const
00266 {
00267     if (!m_autoRouteContrast)
00268         return color;
00269
00270     QColor adjusted = color;
00271     switch (m_tileCache.mapStyle())
00272     {
00273     case MapStyle::Dark:
00274     case MapStyle::Satellite:
00275         adjusted = adjusted.lighter(145);
00276         break;
00277     case MapStyle::Light:
00278     case MapStyle::Street:
00279     case MapStyle::Minimal:
00280         adjusted = adjusted.darker(120);
00281         break;
00282     case MapStyle::Terrain:
00283     case MapStyle::Topographic:
00284         adjusted = adjusted.lighter(120);

```

```

00285         break;
00286     }
00287
00288     return adjusted;
00289 }
00290
00291 const RideRecord* MapRenderer::nearestGpsRecord(const QPointF& screenPos) const
00292 {
00293     if (m_gpsRecords.empty())
00294         return nullptr;
00295
00296     const RideRecord* nearest = nullptr;
00297     double bestDistance = std::numeric_limits<double>::max();
00298
00299     for (const RideRecord* record : m_gpsRecords)
00300     {
00301         const QPointF point = tileToScreen(latLonToTile(record->latitude, record->longitude, m_zoom));
00302         const double distance = QLineF(screenPos, point).length();
00303         if (distance < bestDistance)
00304         {
00305             bestDistance = distance;
00306             nearest = record;
00307         }
00308     }
00309
00310     if (bestDistance > kNearestRecordMaxDistancePx)
00311         return nullptr;
00312
00313     return nearest;
00314 }
00315
00316 void MapRenderer::fitToTrack()
00317 {
00318     if (!m_hasGps) return;
00319
00320     double minLat = 90.0;
00321     double maxLat = -90.0;
00322     double minLon = 180.0;
00323     double maxLon = -180.0;
00324
00325     for (const RideRecord* record : m_gpsRecords)
00326     {
00327         minLat = std::min(minLat, record->latitude);
00328         maxLat = std::max(maxLat, record->latitude);
00329         minLon = std::min(minLon, record->longitude);
00330         maxLon = std::max(maxLon, record->longitude);
00331     }
00332
00333     m_userMovedMap = false;
00334     m_autoFitVisibleRange = true;
00335
00336     fitToBounds(minLat, maxLat, minLon, maxLon, true);
00337     update();
00338 }
00339
00340 void MapRenderer::fitToVisibleRange()
00341 {
00342     if (!m_hasGps)
00343         return;
00344
00345     double minLat = 90.0;
00346     double maxLat = -90.0;
00347     double minLon = 180.0;
00348     double maxLon = -180.0;
00349     int count = 0;
00350
00351     // Use lower_bound to skip records before the visible range start.
00352     auto rangeBegin = m_gpsRecords.begin();
00353     if (m_visibleStartSeconds >= 0.0)
00354     {
00355         rangeBegin = std::lower_bound(
00356             m_gpsRecords.begin(), m_gpsRecords.end(), m_visibleStartSeconds,
00357             [](const RideRecord* r, double t) { return r->elapsedSeconds < t; });
00358     }
00359
00360     for (auto it = rangeBegin; it != m_gpsRecords.end(); ++it)
00361     {
00362         const RideRecord* record = *it;
00363         if (m_visibleEndSeconds >= 0.0 && record->elapsedSeconds > m_visibleEndSeconds)
00364             break;
00365
00366         minLat = std::min(minLat, record->latitude);
00367         maxLat = std::max(maxLat, record->latitude);
00368         minLon = std::min(minLon, record->longitude);
00369         maxLon = std::max(maxLon, record->longitude);
00370         ++count;
00371     }

```

```

00372
00373     if (count < 1)
00374     {
00375         fitToTrack();
00376         return;
00377     }
00378
00379     m_userMovedMap = false;
00380     m_autoFitVisibleRange = true;
00381
00382     fitToBounds(minLat, maxLat, minLon, maxLon, false);
00383     update();
00384 }
00385
00386 void MapRenderer::fitToBounds(double minLat, double maxLat, double minLon, double maxLon, bool
updateMinZoom)
00387 {
00388     if (!m_hasGps)
00389         return;
00390
00391     const auto fit = MapFitMath::computeFitToBounds(minLat,
00392                                                     maxLat,
00393                                                     minLon,
00394                                                     maxLon,
00395                                                     width(),
00396                                                     height(),
00397                                                     std::min(18, m_tileCache.maxZoom()));
00398
00399     m_minLat = fit.minLat;
00400     m_maxLat = fit.maxLat;
00401     m_minLon = fit.minLon;
00402     m_maxLon = fit.maxLon;
00403     m_centerLat = fit.centerLat;
00404     m_centerLon = fit.centerLon;
00405     m_zoom = fit.zoom;
00406
00407     if (updateMinZoom)
00408         m_minZoom = m_zoom; // track-level minimum zoom; selection fit must not tighten this floor
00409 }
00410
00411 void MapRenderer::zoomIn()
00412 {
00413     m_zoom = std::clamp(m_zoom + 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00414     m_userMovedMap = true;
00415     m_autoFitVisibleRange = false;
00416     update();
00417 }
00418
00419 void MapRenderer::zoomOut()
00420 {
00421     m_zoom = std::clamp(m_zoom - 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00422     m_userMovedMap = true;
00423     m_autoFitVisibleRange = false;
00424     update();
00425 }
00426
00427 void MapRenderer::setHighlightedElapsedSeconds(double elapsedSeconds)
00428 {
00429     m_highlightElapsedSeconds = elapsedSeconds;
00430     m_currentTimeSeconds = elapsedSeconds;
00431     update();
00432 }
00433
00434 void MapRenderer::paintEvent(QPaintEvent*)
00435 {
00436     QPainter p(this);
00437     p.setRenderHint(QPainter::Antialiasing);
00438     p.fillRect(rect(), QColor(200, 200, 200));
00439
00440     if (!m_hasGps)
00441     {
00442         p.drawText(rect(), Qt::AlignCenter, "No GPS data in this file");
00443         return;
00444     }
00445
00446     const auto centreTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00447     const int tileX0 = static_cast<int>(std::floor(
00448         centreTile.x() - static_cast<double>(width()) / (2.0 * kTilePx))) - 1;
00449     const int tileY0 = static_cast<int>(std::floor(
00450         centreTile.y() - static_cast<double>(height()) / (2.0 * kTilePx))) - 1;
00451     const int nX = static_cast<int>(std::ceil(
00452         static_cast<double>(width()) / kTilePx)) + 3;
00453     const int nY = static_cast<int>(std::ceil(
00454         static_cast<double>(height()) / kTilePx)) + 3;
00455     const int maxTile = (1 « m_zoom) - 1;
00456
00457     for (int ty = 0; ty <= nY; ++ty)

```



```

00458     {
00459         for (int tx = 0; tx <= nX; ++tx)
00460         {
00461             const int tileX = tileX0 + tx;
00462             const int tileY = tileY0 + ty;
00463             if (tileX < 0 || tileX > maxTile || tileY < 0 || tileY > maxTile)
00464                 continue;
00465
00466             const QPointF topLeft = tileToScreen({ (double)tileX, (double)tileY });
00467             const QPixmap px = m_tileCache.tile(m_zoom, tileX, tileY);
00468
00469             if (px.isNull())
00470                 p.fillRect(QRectF(topLeft, QSizeF(kTilePx, kTilePx)), QColor(210,210,210));
00471             else
00472                 p.drawPixmap(static_cast<int>(topLeft.x()),
00473                             static_cast<int>(topLeft.y()),
00474                             kTilePx, kTilePx, px);
00475         }
00476     }
00477
00478     // Use precomputed GPS record list -- no per-frame allocation.
00479     const auto& gpsRecords = m_gpsRecords;
00480
00481     auto segmentVisible = [this](double startSec, double endSec)
00482     {
00483         if (m_visibleStartSeconds < 0.0 || m_visibleEndSeconds < 0.0)
00484             return true;
00485         const double lo = std::min(m_visibleStartSeconds, m_visibleEndSeconds);
00486         const double hi = std::max(m_visibleStartSeconds, m_visibleEndSeconds);
00487         return endSec >= lo && startSec <= hi;
00488     };
00489
00490     auto drawSegment = [&](const QPointF& a, const QPointF& b, const QColor& color, qreal width)
00491     {
00492         p.setPen(QPen(color, width));
00493         p.drawLine(a, b);
00494     };
00495
00496     // O(log N) time lookup using precomputed sorted GPS records.
00497     auto gpsPointAtTime = [this](double elapsedSeconds) -> const RideRecord*
00498     {
00499         return gpsRecordAtTime(elapsedSeconds);
00500     };
00501
00502     const bool hasVisibleRange = m_visibleStartSeconds >= 0.0 && m_visibleEndSeconds >= 0.0;
00503     const QColor mutedGrey = [&]()
00504     {
00505         QColor c("#d1d5db");
00506         c.setAlpha(80);
00507         return c;
00508     }();
00509
00510     if (m_routeColorMode == RouteColorMode::None)
00511     {
00512         QPainterPath path;
00513         bool first = true;
00514         for (const RideRecord* record : gpsRecords)
00515         {
00516             const auto sp = tileToScreen(latLonToTile(record->latitude, record->longitude, m_zoom));
00517             if (first) { path.moveTo(sp); first = false; }
00518             else { path.lineTo(sp); }
00519         }
00520         p.setPen(QPen(mutedGrey, 1.5));
00521         p.drawPath(path);
00522
00523         QPainterPath visiblePath;
00524         first = true;
00525         for (size_t index = 1; index < gpsRecords.size(); ++index)
00526         {
00527             const RideRecord& prev = *gpsRecords[index - 1];
00528             const RideRecord& curr = *gpsRecords[index];
00529             if (!segmentVisible(prev.elapsedSeconds, curr.elapsedSeconds))
00530                 continue;
00531
00532             const auto sp = tileToScreen(latLonToTile(prev.latitude, prev.longitude, m_zoom));
00533             const auto ep = tileToScreen(latLonToTile(curr.latitude, curr.longitude, m_zoom));
00534             if (first) { visiblePath.moveTo(sp); first = false; }
00535             visiblePath.lineTo(ep);
00536         }
00537         p.setPen(QPen(adjustRouteColorForStyle(QColor(37, 99, 235)), 4.5));
00538         p.drawPath(visiblePath);
00539     }
00540     else
00541     {
00542         for (size_t index = 1; index < gpsRecords.size(); ++index)
00543         {
00544             const RideRecord& prev = *gpsRecords[index - 1];

```

```

00545         const RideRecord& curr = *gpsRecords[index];
00546         const QPointF prevPoint = tileToScreen(latLonToTile(prev.latitude, prev.longitude,
m_zoom));
00547         const QPointF currPoint = tileToScreen(latLonToTile(curr.latitude, curr.longitude,
m_zoom));
00548
00549         // Look up precomputed base color; apply style adjustment per-paint.
00550         const QColor& baseColor = m_segmentColors[index - 1];
00551         const QColor color = baseColor.isValid()
00552             ? adjustRouteColorForStyle(baseColor)
00553             : QColor("#cbd5e1");
00554
00555         if (!segmentVisible(prev.elapsedSeconds, curr.elapsedSeconds))
00556         {
00557             drawSegment(prevPoint, currPoint, mutedGrey, 1.5);
00558         }
00559         else
00560         {
00561             drawSegment(prevPoint, currPoint, mutedGrey, 1.5);
00562             drawSegment(prevPoint, currPoint, color, 4.5);
00563         }
00564     }
00565 }
00566
00567 m_startMarkerRect = QRectF();
00568 m_endMarkerRect = QRectF();
00569
00570 if (hasVisibleRange)
00571 {
00572     const RideRecord* startRecord = gpsPointAtTime(m_visibleStartSeconds);
00573     const RideRecord* endRecord = gpsPointAtTime(m_visibleEndSeconds);
00574
00575     auto drawMarker = [&](const RideRecord* record, const QColor& color, QRectF* markerRect)
00576     {
00577         if (!record)
00578             return;
00579         const QPointF point = tileToScreen(latLonToTile(record->latitude, record->longitude,
m_zoom));
00580         if (markerRect)
00581         {
00582             *markerRect = QRectF(point.x() - kMarkerHitSizePx / 2.0,
00583                 point.y() - kMarkerHitSizePx / 2.0,
00584                 kMarkerHitSizePx,
00585                 kMarkerHitSizePx);
00586         }
00587         p.setPen(QPen(QColor("ffffff"), 2.0));
00588         p.setBrush(color);
00589         p.drawEllipse(point, kMarkerRadiusPx, kMarkerRadiusPx);
00590     };
00591
00592     drawMarker(startRecord, QColor("#22c55e"), &m_startMarkerRect);
00593     drawMarker(endRecord, QColor("#ef4444"), &m_endMarkerRect);
00594 }
00595
00596 auto drawDot = [&](double lat, double lon, QColor c)
00597 {
00598     const auto sp = tileToScreen(latLonToTile(lat, lon, m_zoom));
00599     p.setPen(QPen(Qt::white, 2.0));
00600     p.setBrush(c);
00601     p.drawEllipse(sp, 6.0, 6.0);
00602 };
00603
00604 if (m_firstGpsRecord)
00605     drawDot(m_firstGpsRecord->latitude, m_firstGpsRecord->longitude, QColor(50,180,50));
00606 if (m_lastGpsRecord)
00607     drawDot(m_lastGpsRecord->latitude, m_lastGpsRecord->longitude, QColor(220,50,50));
00608
00609 if (m_currentTimeSeconds >= 0.0)
00610 {
00611     const RideRecord* nearest = gpsRecordAtTime(m_currentTimeSeconds);
00612     if (nearest)
00613     {
00614         const QPointF hp = tileToScreen(latLonToTile(nearest->latitude, nearest->longitude,
m_zoom));
00615         p.setPen(QPen(QColor("#f8f8f8"), 2.0));
00616         p.setBrush(QColor("#111827"));
00617         p.drawEllipse(hp, 7.0, 7.0);
00618         p.setPen(QPen(QColor("#ef4444"), 2.0));
00619         p.setBrush(Qt::NoBrush);
00620         p.drawEllipse(hp, 11.0, 11.0);
00621     }
00622 }
00623
00624 p.setPen(QColor(60, 60, 60));
00625 p.setFont(QFont("Arial", 8));
00626 p.drawText(rect().adjusted(0,0,-4,-4),
Qt::AlignBottom | Qt::AlignRight,

```

```

00628         m_tileCache.attribution());
00629     }
00630
00631 void MapRenderer::wheelEvent(QWheelEvent* event)
00632 {
00633     const int delta = event->angleDelta().y() > 0 ? 1 : -1;
00634     const int oldZoom = m_zoom;
00635     const int newZoom = std::clamp(m_zoom + delta, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00636     if (newZoom == oldZoom)
00637     {
00638         event->accept();
00639         return;
00640     }
00641
00642     const QPointF cursor = event->position();
00643     const QPointF oldCenterTile = latLonToTile(m_centerLat, m_centerLon, oldZoom);
00644     const QPointF screenOffset(
00645         (cursor.x() - width() / 2.0) / kTilePx,
00646         (cursor.y() - height() / 2.0) / kTilePx
00647     );
00648
00649     // World position under the cursor before zoom.
00650     const QPointF anchorTileOld = oldCenterTile + screenOffset;
00651     const double nOld = std::pow(2.0, oldZoom);
00652     const double anchorLon = anchorTileOld.x() / nOld * 360.0 - 180.0;
00653     const double anchorMercN = kPi * (1.0 - 2.0 * anchorTileOld.y() / nOld);
00654     const double anchorLat = std::atan(std::sinh(anchorMercN)) * 180.0 / kPi;
00655
00656     // Recompute center at new zoom so the same world point stays under cursor.
00657     const QPointF anchorTileNew = latLonToTile(anchorLat, anchorLon, newZoom);
00658     const QPointF newCenterTile = anchorTileNew - screenOffset;
00659     const double nNew = std::pow(2.0, newZoom);
00660     m_centerLon = newCenterTile.x() / nNew * 360.0 - 180.0;
00661     const double centerMercN = kPi * (1.0 - 2.0 * newCenterTile.y() / nNew);
00662     m_centerLat = std::atan(std::sinh(centerMercN)) * 180.0 / kPi;
00663     m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00664     m_zoom = newZoom;
00665     m_userMovedMap = true;
00666     m_autoFitVisibleRange = false;
00667
00668     update();
00669     event->accept();
00670 }
00671
00672 void MapRenderer::mouseDoubleClickEvent(QMouseEvent* event)
00673 {
00674     if (!m_hasGps || event->button() != Qt::LeftButton)
00675     {
00676         QWidget::mouseDoubleClickEvent(event);
00677         return;
00678     }
00679
00680     const QPointF centreTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00681     const double clickedTileX = centreTile.x() + (event->position().x() - width() / 2.0) / kTilePx;
00682     const double clickedTileY = centreTile.y() + (event->position().y() - height() / 2.0) / kTilePx;
00683
00684     const double n = std::pow(2.0, m_zoom);
00685     m_centerLon = clickedTileX / n * 360.0 - 180.0;
00686     const double mercN = kPi * (1.0 - 2.0 * clickedTileY / n);
00687     m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00688     m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00689
00690     m_zoom = std::clamp(m_zoom + 1, m_minZoom, std::min(18, m_tileCache.maxZoom()));
00691     m_panning = false;
00692     m_userMovedMap = true;
00693     m_autoFitVisibleRange = false;
00694     update();
00695     event->accept();
00696 }
00697
00698 void MapRenderer::mousePressEvent(QMouseEvent* event)
00699 {
00700     if (event->button() == Qt::LeftButton)
00701     {
00702         const QPointF pos = event->position();
00703         if (m_startMarkerRect.contains(pos))
00704         {
00705             m_dragHandle = DragHandle::Start;
00706             m_draggingSelection = true;
00707             m_panning = false;
00708             event->accept();
00709             return;
00710         }
00711
00712         if (m_endMarkerRect.contains(pos))
00713         {
00714             m_dragHandle = DragHandle::End;

```

```

00715         m_draggingSelection = true;
00716         m_panning = false;
00717         event->accept();
00718         return;
00719     }
00720
00721     m_panning = true;
00722     m_panStart = event->position();
00723     event->accept();
00724     return;
00725 }
00726
00727 QWidget::mousePressEvent(event);
00728 }
00729
00730 void MapRenderer::mouseMoveEvent(QMouseEvent* event)
00731 {
00732     if (m_draggingSelection)
00733     {
00734         const QPointF pos = event->position();
00735
00736         double panX = 0.0;
00737         double panY = 0.0;
00738         if (pos.x() < kEdgePanMargin)
00739             panX = -0.05;
00740         else if (pos.x() > width() - kEdgePanMargin)
00741             panX = 0.05;
00742
00743         if (pos.y() < kEdgePanMargin)
00744             panY = -0.05;
00745         else if (pos.y() > height() - kEdgePanMargin)
00746             panY = 0.05;
00747
00748         if (panX != 0.0 || panY != 0.0)
00749         {
00750             const QPointF prevTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00751             const QPointF newTile = prevTile + QPointF(panX, panY);
00752             const double n = std::pow(2.0, m_zoom);
00753             m_centerLon = newTile.x() / n * 360.0 - 180.0;
00754             const double mercN = kPi * (1.0 - 2.0 * newTile.y() / n);
00755             m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00756             m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00757             m_userMovedMap = true;
00758             m_autoFitVisibleRange = false;
00759         }
00760
00761         const RideRecord* record = nearestGpsRecord(pos);
00762         if (!record)
00763         {
00764             update();
00765             event->accept();
00766             return;
00767         }
00768
00769         if (m_dragHandle == DragHandle::Start)
00770             m_visibleStartSeconds = std::min(record->elapsedSeconds, m_visibleEndSeconds);
00771         else if (m_dragHandle == DragHandle::End)
00772             m_visibleEndSeconds = std::max(record->elapsedSeconds, m_visibleStartSeconds);
00773
00774         emit segmentSelectionChanged(m_visibleStartSeconds, m_visibleEndSeconds);
00775         update();
00776         event->accept();
00777         return;
00778     }
00779
00780     if (!m_panning)
00781     {
00782         QWidget::mouseMoveEvent(event);
00783         return;
00784     }
00785
00786     const QPointF delta = event->position() - m_panStart;
00787     m_panStart = event->position();
00788
00789     const auto prevTile = latLonToTile(m_centerLat, m_centerLon, m_zoom);
00790     const double newTileX = prevTile.x() - delta.x() / kTilePx;
00791     const double newTileY = prevTile.y() - delta.y() / kTilePx;
00792
00793     const double n = std::pow(2.0, m_zoom);
00794     m_centerLon = newTileX / n * 360.0 - 180.0;
00795     const double mercN = kPi * (1.0 - 2.0 * newTileY / n);
00796     m_centerLat = std::atan(std::sinh(mercN)) * 180.0 / kPi;
00797     m_centerLat = std::clamp(m_centerLat, -85.051129, 85.051129);
00798     m_userMovedMap = true;
00799     m_autoFitVisibleRange = false;
00800     update();
00801     event->accept();

```

```

00802 }
00803
00804 void MapRenderer::mouseReleaseEvent(QMouseEvent* event)
00805 {
00806     if (event->button() == Qt::LeftButton)
00807     {
00808         if (m_draggingSelection)
00809         {
00810             emit segmentSelectionFinished(m_visibleStartSeconds, m_visibleEndSeconds);
00811             m_draggingSelection = false;
00812             m_dragHandle = DragHandle::None;
00813             event->accept();
00814             return;
00815         }
00816
00817         m_panning = false;
00818         event->accept();
00819         return;
00820     }
00821
00822     QWidget::mouseReleaseEvent(event);
00823 }

```

## 8.185 MapRenderer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QPointF>
00007 #include <QRectF>
00008 #include <QTimer>
00009 #include <QWidget>
00010
00011 #include "core/zones/ZoneDefinition.h"
00012 #include "fit/RideData.h"
00013 #include "maps/TileCache.h"
00014
00015 enum class RouteColorMode
00016 {
00017     None,
00018     Power,
00019     HeartRate,
00020     Cadence,
00021     Speed,
00022     Altitude,
00023     Gradient
00024 };
00025
00026 class MapRenderer : public QWidget
00027 {
00028     Q_OBJECT
00029 public:
00030     explicit MapRenderer(QWidget* parent = nullptr);
00031
00032     void setRideData(const RideData& rideData,
00033                     ColorMetric colorMetric = ColorMetric::None,
00034                     const ColorContext& colorContext = {});
00035     void setVisibleTimeRange(double startSeconds, double endSeconds);
00036     void setCurrentTime(double seconds);
00037     void setRouteColorMode(RouteColorMode mode, const ColorContext& colorContext = {});
00038     void setMapStyle(MapStyle style);
00039     MapStyle mapStyle() const { return m_tileCache.mapStyle(); }
00040     void setAutoRouteContrast(bool enabled);
00041     bool autoRouteContrast() const { return m_autoRouteContrast; }
00042     void setTileCacheSize(int maxTiles);
00043
00044 signals:
00045     // Emitted when segment selection changes while dragging.
00046     void segmentSelectionChanged(double startSeconds, double endSeconds);
00047
00048     // Emitted when segment selection completes.
00049     void segmentSelectionFinished(double startSeconds, double endSeconds);
00050
00051 public slots:
00052     void fitToTrack();
00053
00054     void fitToVisibleRange();
00055
00056     void zoomIn();
00057
00058     void zoomOut();
00059 }

```

```

00069     void setHighlightedElapsedSeconds(double elapsedSeconds);
00070
00071 protected:
00072     void paintEvent(QPaintEvent* event) override;
00073
00074     void wheelEvent(QWheelEvent* event) override;
00075
00076     void mouseDoubleClickEvent(QMouseEvent* event) override;
00077
00078     void mousePressEvent(QMouseEvent* event) override;
00079
00080     void mouseMoveEvent(QMouseEvent* event) override;
00081
00082     void mouseReleaseEvent(QMouseEvent* event) override;
00083
00084 private:
00085     enum class DragHandle
00086     {
00087         None,
00088         Start,
00089         End
00090     };
00091
00092     static QPointF latLonToTile(double lat, double lon, int zoom);
00093     void fitToBounds(double minLat, double maxLat, double minLon, double maxLon, bool updateMinZoom);
00094     QPointF tileToScreen(const QPointF& tilePt) const;
00095     QColor adjustRouteColorForStyle(const QColor& color) const;
00096     const RideRecord* nearestGpsRecord(const QPointF& screenPos) const;
00097     void rebuildGpsCache();
00098     void rebuildSegmentColors();
00099     const RideRecord* gpsRecordAtTime(double elapsedSeconds) const;
00100
00101     TileCache m_tileCache;
00102     RideData m_rideData;
00103     // WARNING: m_gpsRecords contains raw pointers into m_rideData.records.
00104     // Any mutation of m_rideData.records requires calling rebuildGpsCache().
00105     // Pre-filtered, time-ordered GPS-only records. Rebuilt once in setRideData().
00106     std::vector<const RideRecord*> m_gpsRecords;
00107     // Per-segment base colors (before map-style adjustment). One per GPS segment.
00108     // Invalid QColor means no data for that segment.
00109     std::vector<QColor> m_segmentColors;
00110     const RideRecord* m_firstGpsRecord = nullptr;
00111     const RideRecord* m_lastGpsRecord = nullptr;
00112     bool m_hasGps = false;
00113     RouteColorMode m_routeColorMode = RouteColorMode::None;
00114     ColorContext m_colorContext;
00115
00116     double m_minLat = 90.0, m_maxLat = -90.0;
00117     double m_minLon = 180.0, m_maxLon = -180.0;
00118
00119     double m_centerLat = 51.5;
00120     double m_centerLon = 0.0;
00121     int m_zoom = 13;
00122     int m_minZoom = 1; // set by fitToTrack(); prevents zooming out past track extent
00123
00124     QPointF m_panStart;
00125     bool m_panning = false;
00126     DragHandle m_dragHandle = DragHandle::None;
00127     QRectF m_startMarkerRect;
00128     QRectF m_endMarkerRect;
00129     bool m_draggingSelection = false;
00130     bool m_userMovedMap = false;
00131     bool m_autoFitVisibleRange = true;
00132     bool m_autoRouteContrast = true;
00133     double m_highlightElapsedSeconds = -1.0;
00134     double m_visibleStartSeconds = -1.0;
00135     double m_visibleEndSeconds = -1.0;
00136     double m_currentTimeSeconds = -1.0;
00137     QTimer m_tileRepaintTimer;
00138 };

```

## 8.186 TileDownloader.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "TileDownloader.h"
00004
00005 #include <QUrl>
00006
00007 namespace TileDownloader
00008 {
00009     QString tileUrl(const QString& urlTemplate, int z, int x, int y)
00010     {
00011         return QString(urlTemplate)

```

```

00012         .replace("{z}", QString::number(z))
00013         .replace("{x}", QString::number(x))
00014         .replace("{y}", QString::number(y));
00015     }
00016
00017     QNetworkRequest buildRequest(const QString& urlTemplate, int z, int x, int y)
00018     {
00019         return QNetworkRequest(QUrl(tileUrl(urlTemplate, z, x, y)));
00020     }
00021 }

```

## 8.187 TileDownloader.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QNetworkRequest>
00006 #include <QString>
00007
00008 namespace TileDownloader
00009 {
00010     QString tileUrl(const QString& urlTemplate, int z, int x, int y);
00011     QNetworkRequest buildRequest(const QString& urlTemplate, int z, int x, int y);
00012 }

```

## 8.188 TileProviderRegistry.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "TileProviderRegistry.h"
00004
00005 namespace TileProviderRegistry
00006 {
00007     QString styleKey(MapStyle style)
00008     {
00009         switch (style)
00010         {
00011             case MapStyle::Street:      return "street";
00012             case MapStyle::Light:       return "light";
00013             case MapStyle::Dark:        return "dark";
00014             case MapStyle::Satellite:   return "satellite";
00015             case MapStyle::Terrain:     return "terrain";
00016             case MapStyle::Topographic: return "topographic";
00017             case MapStyle::Minimal:     return "minimal";
00018         }
00019         return "street";
00020     }
00021
00022     QString styleDisplayName(MapStyle style)
00023     {
00024         switch (style)
00025         {
00026             case MapStyle::Street:      return "Street Map";
00027             case MapStyle::Light:       return "Light";
00028             case MapStyle::Dark:        return "Dark";
00029             case MapStyle::Satellite:   return "Satellite";
00030             case MapStyle::Terrain:     return "Terrain";
00031             case MapStyle::Topographic: return "Topographic";
00032             case MapStyle::Minimal:     return "Minimal";
00033         }
00034         return "Street Map";
00035     }
00036
00037     TileProvider providerForStyle(MapStyle style)
00038     {
00039         switch (style)
00040         {
00041             case MapStyle::Street:
00042                 return {
00043                     "OpenStreetMap Standard",
00044                     "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
00045                     19,
00046                     "© OpenStreetMap contributors"
00047                 };
00048             case MapStyle::Light:
00049                 return {
00050                     "Carto Positron",
00051                     "https://a.basemaps.cartocdn.com/light_all/{z}/{x}/{y}.png",
00052                     20,
00053                     "© OpenStreetMap contributors © CARTO"
00054                 };
00055         }
00056     }
00057 }

```

```

00054         };
00055         case MapStyle::Dark:
00056             return {
00057                 "Carto Dark Matter",
00058                 "https://a.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}.png",
00059                 20,
00060                 "© OpenStreetMap contributors © CARTO"
00061             };
00062         case MapStyle::Satellite:
00063             return {
00064                 "Esri World Imagery",
00065                 "https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}",
00066                 18,
00067                 "Tiles © Esri"
00068             };
00069         case MapStyle::Terrain:
00070             return {
00071                 "OpenTopoMap",
00072                 "https://a.tile.opentopomap.org/{z}/{x}/{y}.png",
00073                 17,
00074                 "© OpenStreetMap contributors, SRTM | © OpenTopoMap"
00075             };
00076         case MapStyle::Topographic:
00077             return {
00078                 "OpenTopoMap",
00079                 "https://a.tile.opentopomap.org/{z}/{x}/{y}.png",
00080                 17,
00081                 "© OpenStreetMap contributors, SRTM | © OpenTopoMap"
00082             };
00083         case MapStyle::Minimal:
00084             return {
00085                 "Carto Positron No Labels",
00086                 "https://a.basemaps.cartocdn.com/light_nolabels/{z}/{x}/{y}.png",
00087                 20,
00088                 "© OpenStreetMap contributors © CARTO"
00089             };
00090     }
00091     return {
00092         "OpenStreetMap Standard",
00093         "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
00094         19,
00095         "© OpenStreetMap contributors"
00096     };
00097 }
00098 }
00099 }

```

## 8.189 TileProviderRegistry.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 #include "maps/TileCache.h"
00008
00009 namespace TileProviderRegistry
00010 {
00011     QString styleKey(MapStyle style);
00012     QString styleDisplayName(MapStyle style);
00013     TileProvider providerForStyle(MapStyle style);
00014 }

```

## 8.190 RouteColorizer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "RouteColorizer.h"
00004
00005 namespace RouteColorizer
00006 {
00007     QColor gradientColorForSlope(double percent)
00008     {
00009         if (percent <= -8.0) return QColor("#7c2d12");
00010         if (percent <= -4.0) return QColor("#ea580c");
00011         if (percent <= -1.0) return QColor("#f59e0b");
00012         if (percent < 1.0) return QColor("#94a3b8");
00013         if (percent < 4.0) return QColor("#22c55e");
00014         if (percent < 8.0) return QColor("#16a34a");

```



```

00015     return QColor("#15803d");
00016 }
00017
00018 ColorMetric routeModeToMetric(RouteColorMode mode)
00019 {
00020     switch (mode)
00021     {
00022         case RouteColorMode::Power:      return ColorMetric::Power;
00023         case RouteColorMode::HeartRate:  return ColorMetric::HeartRate;
00024         case RouteColorMode::Cadence:    return ColorMetric::Cadence;
00025         case RouteColorMode::Speed:      return ColorMetric::Speed;
00026         case RouteColorMode::Altitude:   return ColorMetric::Altitude;
00027         case RouteColorMode::None:
00028         case RouteColorMode::Gradient:
00029             return ColorMetric::None;
00030     }
00031     return ColorMetric::None;
00032 }
00033 }
00034 }

```

## 8.191 RouteColorizer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QColor>
00006
00007 #include "maps/MapRenderer.h"
00008
00009 namespace RouteColorizer
00010 {
00011     QColor gradientColorForSlope(double percent);
00012     ColorMetric routeModeToMetric(RouteColorMode mode);
00013 }

```

## 8.192 TileCache.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "TileCache.h"
00005
00006 #include <QDir>
00007 #include <QEventLoop>
00008 #include <QFileInfo>
00009 #include <QImage>
00010 #include <QNetworkReply>
00011 #include <QNetworkRequest>
00012 #include <QStandardPaths>
00013 #include <QUrl>
00014
00015 #include <algorithm>
00016
00017 #include "maps/cache/DiskTileCache.h"
00018 #include "maps/network/TileDownloader.h"
00019 #include "maps/providers/TileProviderRegistry.h"
00020
00021 namespace
00022 {
00023     QString defaultDiskCacheRoot()
00024     {
00025         const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00026         return appData.isEmpty()
00027             ? QDir::home().filePath(".fitlyzerc/tiles")
00028             : QDir(appData).filePath("tiles");
00029     }
00030 }
00031
00032 TileCache::TileCache(const TileCacheConfig& config, QObject* parent)
00033     : QObject(parent)
00034     , m_cache(std::max(1, config.maxTilesInMemory))
00035     , m_diskCacheRoot(config.diskCacheRoot.trimmed().isEmpty()
00036         ? defaultDiskCacheRoot()
00037         : config.diskCacheRoot)
00038     , m_diskFallbackRoots(config.diskFallbackRoots)
00039 {
00040     m_provider = TileProviderRegistry::providerForStyle(m_mapStyle);
00041     connect(&m_nam, &QNetworkAccessManager::finished,
00042         this, &TileCache::onReplyFinished);

```

```

00043 }
00044
00045 QString TileCache::key(int z, int x, int y) const
00046 {
00047     return keyForStyle(TileProviderRegistry::styleKey(m_mapStyle), z, x, y);
00048 }
00049
00050 QString TileCache::keyForStyle(const QString& style, int z, int x, int y)
00051 {
00052     return QString("%1/%2/%3/%4").arg(style).arg(z).arg(x).arg(y);
00053 }
00054
00055 QString TileCache::diskTilePathForRoot(const QString& root, int z, int x, int y) const
00056 {
00057     return diskTilePathForRoot(root, TileProviderRegistry::styleKey(m_mapStyle), z, x, y);
00058 }
00059
00060 QString TileCache::diskTilePathForRoot(const QString& root,
00061                                       const QString& style,
00062                                       int z,
00063                                       int x,
00064                                       int y)
00065 {
00066     return DiskTileCache::diskTilePathForRoot(root, style, z, x, y);
00067 }
00068
00069 QString TileCache::diskTilePath(int z, int x, int y) const
00070 {
00071     return diskTilePathForRoot(m_diskCacheRoot, z, x, y);
00072 }
00073
00074 bool TileCache::isTileCachedOnDisk(int z, int x, int y) const
00075 {
00076     z = std::clamp(z, 1, m_provider.maxZoom);
00077     return DiskTileCache::isTileCachedOnDisk(
00078         m_diskCacheRoot,
00079         m_diskFallbackRoots,
00080         TileProviderRegistry::styleKey(m_mapStyle),
00081         z,
00082         x,
00083         y);
00084 }
00085
00086 void TileCache::clearMemoryCache()
00087 {
00088     m_cache.clear();
00089     m_pending.clear();
00090     m_downloadQueue.clear();
00091     // m_activeDownloads is not reset here: in-flight replies will still arrive
00092     // and decrement it correctly. dispatchNextDownloads() will be a no-op since
00093     // the queue is now empty.
00094 }
00095
00096 void TileCache::setMemoryCacheSize(int maxTiles)
00097 {
00098     m_cache.setMaxCost(std::max(1, maxTiles));
00099 }
00100
00101 void TileCache::setMapStyle(MapStyle style)
00102 {
00103     if (m_mapStyle == style)
00104         return;
00105
00106     // Cancel in-flight downloads from the previous style to reduce stale work.
00107     const auto activeReplies = m_nam.findChildren<QNetworkReply*>();
00108     for (QNetworkReply* reply : activeReplies)
00109     {
00110         if (reply)
00111         {
00112             reply->setProperty("styleChangeAbort", true);
00113             reply->abort();
00114         }
00115     }
00116
00117     m_mapStyle = style;
00118     m_provider = TileProviderRegistry::providerForStyle(style);
00119     clearMemoryCache();
00120 }
00121
00122 QString TileCache::mapStyleName() const
00123 {
00124     return TileProviderRegistry::styleDisplayName(m_mapStyle);
00125 }
00126
00127 QString TileCache::attribution() const
00128 {
00129     return m_provider.attribution;

```

```

00130 }
00131
00132 int TileCache::maxZoom() const
00133 {
00134     return m_provider.maxZoom;
00135 }
00136
00137 QString TileCache::diskCacheRoot() const
00138 {
00139     return m_diskCacheRoot;
00140 }
00141
00142 QPixmap TileCache::tile(int z, int x, int y)
00143 {
00144     z = std::clamp(z, 1, m_provider.maxZoom);
00145     const QString k = key(z, x, y);
00146
00147     if (auto* px = m_cache.object(k))
00148         return *px;
00149
00150     const QString diskPath = diskTilePath(z, x, y);
00151     QPixmap diskPixmap;
00152     if (diskPixmap.load(diskPath))
00153     {
00154         m_cache.insert(k, new QPixmap(diskPixmap));
00155         return diskPixmap;
00156     }
00157
00158     for (const QString& fallbackRoot : m_diskFallbackRoots)
00159     {
00160         if (fallbackRoot.trimmed().isEmpty())
00161             continue;
00162
00163         const QString fallbackPath = diskTilePathForRoot(fallbackRoot, z, x, y);
00164         if (!diskPixmap.load(fallbackPath))
00165             continue;
00166
00167         const QFileInfo fi(diskPath);
00168         QDir().mkpath(fi.path());
00169         diskPixmap.save(diskPath);
00170         m_cache.insert(k, new QPixmap(diskPixmap));
00171         return diskPixmap;
00172     }
00173
00174     if (!m_pending.contains(k))
00175     {
00176         m_pending.insert(k);
00177         m_downloadQueue.enqueue({z, x, y});
00178         dispatchNextDownloads();
00179     }
00180
00181     return {}; // empty until the download completes
00182 }
00183
00184 QPixmap TileCache::tileBlocking(int z, int x, int y, bool allowNetwork)
00185 {
00186     z = std::clamp(z, 1, m_provider.maxZoom);
00187     const QString k = key(z, x, y);
00188
00189     if (auto* px = m_cache.object(k))
00190         return *px;
00191
00192     const QString diskPath = diskTilePath(z, x, y);
00193     QPixmap diskPixmap;
00194     if (diskPixmap.load(diskPath))
00195     {
00196         m_cache.insert(k, new QPixmap(diskPixmap));
00197         return diskPixmap;
00198     }
00199
00200     for (const QString& fallbackRoot : m_diskFallbackRoots)
00201     {
00202         if (fallbackRoot.trimmed().isEmpty())
00203             continue;
00204
00205         const QString fallbackPath = diskTilePathForRoot(fallbackRoot, z, x, y);
00206         if (!diskPixmap.load(fallbackPath))
00207             continue;
00208
00209         const QFileInfo fi(diskPath);
00210         QDir().mkpath(fi.path());
00211         diskPixmap.save(diskPath);
00212         m_cache.insert(k, new QPixmap(diskPixmap));
00213         return diskPixmap;
00214     }
00215
00216     if (!allowNetwork)

```

```

00217         return {};
00218
00219     QNetworkAccessManager nam;
00220     QNetworkRequest req = TileDownloader::buildRequest(m_provider.urlTemplate, z, x, y);
00221     req.setHeader(QNetworkRequest::UserAgentHeader, "FitlyzerC/1.0");
00222     req.setAttribute(
00223         QNetworkRequest::RedirectPolicyAttribute,
00224         QNetworkRequest::NoLessSafeRedirectPolicy);
00225
00226     QNetworkReply* reply = nam.get(req);
00227     QEventLoop loop;
00228     connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
00229     loop.exec();
00230
00231     QPixmap px;
00232     if (reply->error() == QNetworkReply::NoError)
00233     {
00234         const QByteArray bytes = reply->readAll();
00235         if (px.loadFromData(bytes))
00236         {
00237             const QFileInfo fi(diskPath);
00238             QDir().mkpath(fi.path());
00239             px.save(diskPath);
00240             m_cache.insert(k, new QPixmap(px));
00241         }
00242     }
00243
00244     reply->deleteLater();
00245     return px;
00246 }
00247
00248 void TileCache::dispatchNextDownloads()
00249 {
00250     while (m_activeDownloads < kMaxConcurrentDownloads && !m_downloadQueue.isEmpty())
00251     {
00252         const TileRequest req = m_downloadQueue.dequeue();
00253
00254         QNetworkRequest netReq = TileDownloader::buildRequest(m_provider.urlTemplate, req.z, req.x,
00255             req.y);
00256         netReq.setHeader(QNetworkRequest::UserAgentHeader, "FitlyzerC/1.0");
00257         netReq.setAttribute(
00258             QNetworkRequest::RedirectPolicyAttribute,
00259             QNetworkRequest::NoLessSafeRedirectPolicy);
00260
00261         auto* reply = m_nam.get(netReq);
00262         reply->setProperty("z", req.z);
00263         reply->setProperty("x", req.x);
00264         reply->setProperty("y", req.y);
00265         reply->setProperty("style", TileProviderRegistry::styleKey(m_mapStyle));
00266         ++m_activeDownloads;
00267     }
00268 }
00269
00270 void TileCache::onReplyFinished(QNetworkReply* reply)
00271 {
00272     reply->deleteLater();
00273
00274     // Safely decrement active downloads counter. Aborted replies or any other
00275     // completion will trigger this, so guard against going negative.
00276     if (m_activeDownloads > 0)
00277         --m_activeDownloads;
00278
00279     const int z = reply->property("z").toInt();
00280     const int x = reply->property("x").toInt();
00281     const int y = reply->property("y").toInt();
00282     const QString replyStyle = reply->property("style").toString();
00283     const bool wasAbortedForStyleChange = reply->property("styleChangeAbort").toBool();
00284     const QString k = keyForStyle(replyStyle, z, x, y);
00285     m_pending.remove(k);
00286
00287     // Always redispach so queued tiles are sent as slots free up.
00288     dispatchNextDownloads();
00289
00290     // Skip processing aborted or stale-style replies
00291     if (wasAbortedForStyleChange || replyStyle != TileProviderRegistry::styleKey(m_mapStyle))
00292         return;
00293
00294     if (reply->error() != QNetworkReply::NoError)
00295         return;
00296
00297     QPixmap px;
00298     if (!px.loadFromData(reply->readAll()))
00299         return;
00300
00301     const QString diskPath = diskTilePathForRoot(m_diskCacheRoot, replyStyle, z, x, y);
00302     const QFileInfo fi(diskPath);

```

```

00303     QDir().mkpath(fi.path());
00304     px.save(diskPath);
00305
00306     m_cache.insert(k, new QPixmap(px));
00307     emit tileLoaded(z, x, y);
00308 }

```

## 8.193 TileCache.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QCache>
00007 #include <QNetworkAccessManager>
00008 #include <QObject>
00009 #include <QPixmap>
00010 #include <QQueue>
00011 #include <QSet>
00012 #include <QString>
00013 #include <QStringList>
00014 enum class MapStyle
00015 {
00016     Street = 0,
00017     Light = 1,
00018     Dark = 4,
00019     Satellite = 5,
00020     Terrain = 6,
00021     Topographic = 7,
00022     Minimal = 8
00023 };
00024
00028 struct TileProvider
00029 {
00030     // Provider name.
00031     QString name;
00032
00033     // URL template with {z}/{x}/{y} placeholders.
00034     QString urlTemplate;
00035
00036     // Maximum zoom level.
00037     int maxZoom = 18;
00038
00039     // Attribution text for provider.
00040     QString attribution;
00041 };
00042
00043 class QNetworkReply;
00044
00048 struct TileCacheConfig
00049 {
00050     // Maximum tiles to hold in memory.
00051     int maxTilesInMemory = 512;
00052
00053     // Primary disk cache directory.
00054     QString diskCacheRoot;
00055
00056     // Additional fallback cache directories.
00057     QStringList diskFallbackRoots;
00058 };
00059
00066 class TileCache : public QObject
00067 {
00068     Q_OBJECT
00069 public:
00070     explicit TileCache(const TileCacheConfig& config = {}, QObject* parent = nullptr);
00071
00072     QPixmap tile(int z, int x, int y);
00073
00074     QPixmap tileBlocking(int z, int x, int y, bool allowNetwork = true);
00075
00076     bool isTileCachedOnDisk(int z, int x, int y) const;
00077
00078     void clearMemoryCache();
00079
00080     void setMemoryCacheSize(int maxTiles);
00081
00082     void setMapStyle(MapStyle style);
00083
00084     MapStyle mapStyle() const { return m_mapStyle; }
00085
00086     QString mapStyleName() const;
00087

```

```

00088     QString attribution() const;
00089
00090     int maxZoom() const;
00091
00092     QString diskCacheRoot() const;
00093
00094 signals:
00095     // Emitted when a tile finishes loading.
00096     void tileLoaded(int z, int x, int y);
00097
00098 private slots:
00099     void onReplyFinished(QNetworkReply* reply);
00100
00101 private:
00102     struct TileRequest { int z, x, y; };
00103
00104     static constexpr int kMaxConcurrentDownloads = 6;
00105
00106     QString key(int z, int x, int y) const;
00107     static QString keyForStyle(const QString& style, int z, int x, int y);
00108     QString diskTilePathForRoot(const QString& root, int z, int x, int y) const;
00109     static QString diskTilePathForRoot(const QString& root,
00110                                       const QString& style,
00111                                       int z,
00112                                       int x,
00113                                       int y);
00114     QString diskTilePath(int z, int x, int y) const;
00115
00116     void dispatchNextDownloads();
00117
00118     QNetworkAccessManager    m_nam;
00119     QCache<QString, QPixmap>  m_cache;
00120     QSet<QString>             m_pending;
00121     QQueue<TileRequest>      m_downloadQueue;
00122     int                       m_activeDownloads = 0;
00123     QString                   m_diskCacheRoot;
00124     QStringList               m_diskFallbackRoots;
00125     MapStyle                  m_mapStyle = MapStyle::Light;
00126     TileProvider              m_provider;
00127 };

```

## 8.194 RideDataLoader.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideDataLoader.h"
00005
00006 #include <QSqlQuery>
00007
00008 RideData RideDataLoader::loadActivitySamples(int activityId, QSqlDatabase& db)
00009 {
00010     RideData ride;
00011
00012     // Pre-reserve vector to avoid repeated reallocations during large sample loads
00013     {
00014         QSqlQuery countQ(db);
00015         countQ.prepare("SELECT COUNT(*) FROM activity_samples WHERE activity_id=:id");
00016         countQ.bindValue(":id", activityId);
00017         if (countQ.exec() && countQ.next())
00018         {
00019             const int sampleCount = countQ.value(0).toInt();
00020             if (sampleCount > 0)
00021                 ride.records.reserve(sampleCount);
00022         }
00023     }
00024
00025     // Load all samples with forward-only mode for memory efficiency
00026     QSqlQuery samplesQuery(db);
00027     samplesQuery.prepare(
00028         "SELECT elapsed_seconds, power_total, heart_rate, cadence, speed,"
00029         " latitude, longitude, altitude,"
00030         " has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00031         " FROM activity_samples WHERE activity_id=:id ORDER BY elapsed_seconds ASC");
00032     samplesQuery.bindValue(":id", activityId);
00033     samplesQuery.setForwardOnly(true);
00034
00035     if (samplesQuery.exec())
00036     {
00037         while (samplesQuery.next())
00038         {
00039             RideRecord rec;
00040             rec.elapsedSeconds = samplesQuery.value(0).toDouble();
00041             rec.power           = samplesQuery.value(1).toDouble();

```

```

00042         rec.heartRate      = samplesQuery.value(2).toDouble();
00043         rec.cadence         = samplesQuery.value(3).toDouble();
00044         rec.speed           = samplesQuery.value(4).toDouble();
00045         rec.latitude        = samplesQuery.value(5).toDouble();
00046         rec.longitude       = samplesQuery.value(6).toDouble();
00047         rec.altitude        = samplesQuery.value(7).toDouble();
00048         rec.hasGps          = samplesQuery.value(8).toInt() != 0;
00049         rec.hasPower        = samplesQuery.value(9).toInt() != 0;
00050         rec.hasHeartRate    = samplesQuery.value(10).toInt() != 0;
00051         rec.hasCadence      = samplesQuery.value(11).toInt() != 0;
00052         rec.hasSpeed        = samplesQuery.value(12).toInt() != 0;
00053         rec.hasAltitude     = samplesQuery.value(13).toInt() != 0;
00054         ride.records.push_back(rec);
00055     }
00056 }
00057
00058 return ride;
00059 }

```

## 8.195 RideDataLoader.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include "fit/RideData.h"
00007 #include <QSqlDatabase>
00008
00012 class RideDataLoader
00013 {
00014 public:
00015     static RideData loadActivitySamples(int activityId, QSqlDatabase& db);
00016 };

```

## 8.196 RideDataSerializer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "RideDataSerializer.h"
00005 #include "database/DatabaseManager.h"
00006 #include "model/RideDataLoader.h"
00007
00008 #include <QDateTime>
00009 #include <QFileInfo>
00010 #include <QSqlDatabase>
00011 #include <QSqlError>
00012 #include <QSqlQuery>
00013 #include <QVariantList>
00014
00015 // Save
00016
00017 int RideDataSerializer::saveRideToDatabase(
00018     const RideData& rideData,
00019     const WorkoutStatistics& stats,
00020     int athleteId,
00021     const QString& filePath,
00022     const QString& fitHash,
00023     double normalizedPower,
00024     DatabaseManager& dbManager,
00025     ActivityRepository& activityRepo,
00026     ImportRepository& importRepo,
00027     QString* errorOut)
00028 {
00029     QSqlDatabase db = dbManager.database();
00030
00031     // Duplicate check
00032     if (!fitHash.isEmpty() && activityRepo.hasFitHash(fitHash))
00033     {
00034         ImportRecord rec;
00035         rec.fileName = QFileInfo(filePath).fileName();
00036         rec.status = "duplicate";
00037         importRepo.logImport(rec);
00038
00039         if (errorOut)
00040             *errorOut = "Already imported (duplicate file).";
00041         return -1;
00042     }
00043
00044     // Build Activity record

```

```

00045     const QString now = QDateTime::currentDateTimeUtc().toString(Qt::ISODate);
00046
00047     Activity act;
00048     act.athleteId      = athleteId;
00049     act.fitHash         = fitHash;
00050     act.fileName       = QFileInfo(filePath).fileName();
00051     act.sport           = "Cycling";
00052     act.startTime      = rideData.activityStartTimeUtc.isEmpty() ? now :
rideData.activityStartTimeUtc;
00053     act.endTime        = rideData.activityEndTimeUtc.isEmpty() ? act.startTime :
rideData.activityEndTimeUtc;
00054     act.durationSec     = stats.durationSeconds;
00055     act.distanceM       = stats.totalDistanceKm * 1000.0;
00056     act.avgPower        = stats.averagePower;
00057     act.maxPower        = stats.maximumPower;
00058     act.normalizedPower = normalizedPower;
00059     act.avgHr           = stats.averageHeartRate;
00060     act.maxHr           = stats.maximumHeartRate;
00061     act.avgCadence      = stats.averageCadence;
00062     act.avgSpeed        = stats.averageSpeed;
00063     act.elevationGain   = stats.elevationGainM;
00064     act.importedAt      = now;
00065
00066     // Transaction: activity row + all samples
00067     if (!db.transaction())
00068     {
00069         if (errorOut) *errorOut = db.lastError().text();
00070         return -1;
00071     }
00072
00073     const int activityId = activityRepo.insertActivity(act);
00074     if (activityId < 0)
00075     {
00076         db.rollback();
00077         ImportRecord rec;
00078         rec.fileName    = act.fileName;
00079         rec.status      = "error";
00080         rec.errorMessage = "Failed to insert activity row.";
00081         importRepo.logImport(rec);
00082         if (errorOut) *errorOut = rec.errorMessage;
00083         return -1;
00084     }
00085
00086     // Bulk-insert samples via execBatch
00087     QSqlQuery q(db);
00088     q.prepare(
00089         "INSERT INTO activity_samples("
00090         "  activity_id, elapsed_seconds,"
00091         "  power_total, heart_rate, cadence, speed,"
00092         "  latitude, longitude, altitude,"
00093         "  has_gps, has_power, has_heart_rate, has_cadence, has_speed, has_altitude"
00094         ") VALUES ("
00095         "  :aid, :es,"
00096         "  :pw, :hr, :cad, :spd,"
00097         "  :lat, :lon, :alt,"
00098         "  :hgps, :hpw, :hhr, :hcad, :hspd, :halt"
00099         ")");
00100
00101     QVariantList aids, es_, pw_, hr_, cad_, spd_;
00102     QVariantList lat_, lon_, alt_;
00103     QVariantList hgps, hpw, hhr, hcad, hspd, halt;
00104
00105     for (const auto& r : rideData.records)
00106     {
00107         aids << activityId;
00108         es_ << r.elapsedSeconds;
00109         pw_ << (r.hasPower ? QVariant(r.power) : QVariant());
00110         hr_ << (r.hasHeartRate ? QVariant(r.heartRate) : QVariant());
00111         cad_ << (r.hasCadence ? QVariant(r.cadence) : QVariant());
00112         spd_ << (r.hasSpeed ? QVariant(r.speed) : QVariant());
00113         lat_ << (r.hasGps ? QVariant(r.latitude) : QVariant());
00114         lon_ << (r.hasGps ? QVariant(r.longitude) : QVariant());
00115         alt_ << (r.hasAltitude ? QVariant(r.altitude) : QVariant());
00116         hgps << (r.hasGps ? 1 : 0);
00117         hpw << (r.hasPower ? 1 : 0);
00118         hhr << (r.hasHeartRate ? 1 : 0);
00119         hcad << (r.hasCadence ? 1 : 0);
00120         hspd << (r.hasSpeed ? 1 : 0);
00121         halt << (r.hasAltitude ? 1 : 0);
00122     }
00123
00124     q.bindValue(":aid", aids);
00125     q.bindValue(":es", es_);
00126     q.bindValue(":pw", pw_);
00127     q.bindValue(":hr", hr_);
00128     q.bindValue(":cad", cad_);
00129     q.bindValue(":spd", spd_);

```



```

00130     q.bindValue(":lat", lat_);
00131     q.bindValue(":lon", lon_);
00132     q.bindValue(":alt", alt_);
00133     q.bindValue(":hgps", hgps);
00134     q.bindValue(":hpw", hpw);
00135     q.bindValue(":hhr", hhr);
00136     q.bindValue(":hcad", hcad);
00137     q.bindValue(":hspd", hspd);
00138     q.bindValue(":halt", halt);
00139
00140     if (!q.execBatch())
00141     {
00142         db.rollback();
00143         ImportRecord rec;
00144         rec.fileName = act.fileName;
00145         rec.status = "error";
00146         rec.errorMessage = "Failed to insert samples: " + q.lastError().text();
00147         importRepo.logImport(rec);
00148         if (errorOut) *errorOut = rec.errorMessage;
00149         return -1;
00150     }
00151
00152     if (!db.commit())
00153     {
00154         db.rollback();
00155         if (errorOut) *errorOut = db.lastError().text();
00156         return -1;
00157     }
00158
00159     // Log success
00160     ImportRecord rec;
00161     rec.fileName = act.fileName;
00162     rec.status = "success";
00163     rec.activityId = activityId;
00164     importRepo.logImport(rec);
00165
00166     return activityId;
00167 }
00168
00169 // Load
00170
00171 RideData RideDataSerializer::loadRideFromDatabase(int activityId,
00172                                                    DatabaseManager& dbManager)
00173 {
00174     QSqlDatabase db = dbManager.database();
00175     return RideDataLoader::loadActivitySamples(activityId, db);
00176 }

```

## 8.197 RideDataSerializer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 #include "database/ActivityRepository.h"
00009 #include "database/ImportRepository.h"
00010 #include "analysis/WorkoutAnalyzer.h"
00011 #include "fit/RideData.h"
00012
00013 class DatabaseManager;
00014
00021 class RideDataSerializer
00022 {
00023 public:
00024     static int saveRideToDatabase(
00025         const RideData& rideData,
00026         const WorkoutStatistics& stats,
00027         int athleteId,
00028         const QString& filePath,
00029         const QString& fitHash, // SHA-256 hex of the .fit file bytes
00030         double normalizedPower,
00031         DatabaseManager& dbManager,
00032         ActivityRepository& activityRepo,
00033         ImportRepository& importRepo,
00034         QString* errorOut = nullptr);
00035
00036     static RideData loadRideFromDatabase(int activityId, DatabaseManager& dbManager);
00037 };

```

## 8.198 Platform.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QStandardPaths>
00009
00010 namespace Platform
00011 {
00012 QString applicationDataDirectory()
00013 {
00014     QString dir = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00015     if (dir.isEmpty())
00016         dir = QDir::homePath() + "/FitlyzerC";
00017
00018     QDir().mkpath(dir);
00019     return dir;
00020 }
00021
00022 QString defaultDatabaseDirectory()
00023 {
00024     const QString documentsDir = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation);
00025     if (!documentsDir.isEmpty())
00026         return documentsDir;
00027
00028     return applicationDataDirectory();
00029 }
00030
00031 QString defaultDatabasePath()
00032 {
00033     return QDir(defaultDatabaseDirectory()).filePath("fitlyzer.fitlyzerdb");
00034 }
00035
00036 QString defaultBackupPath(const QString& databasePath)
00037 {
00038     if (databasePath.isEmpty())
00039         return QDir(defaultDatabaseDirectory()).filePath("fitlyzer-backup.fitlyzerdb");
00040
00041     const QFileInfo info(databasePath);
00042     const QString baseName = info.completeBaseName().isEmpty()
00043         ? QStringLiteral("fitlyzer")
00044         : info.completeBaseName();
00045     return info.dir().filePath(baseName + "-backup.fitlyzerdb");
00046 }
00047 }

```

## 8.199 Platform.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007
00008 namespace Platform
00009 {
00010 QString applicationDataDirectory();
00011
00012 QString defaultDatabaseDirectory();
00013
00014 QString defaultDatabasePath();
00015
00016 QString defaultBackupPath(const QString& databasePath);
00017
00018 void openInFileManager(const QString& path);
00019 }

```

## 8.200 Platform\_linux.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>

```

```
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_LINUX)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

## 8.201 Platform\_macos.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_MACOS)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

## 8.202 Platform\_windows.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "Platform.h"
00005
00006 #include <QDesktopServices>
00007 #include <QFileInfo>
00008 #include <QUrl>
00009
00010 #if defined(Q_OS_WIN)
00011
00012 namespace Platform
00013 {
00014 void openInFileManager(const QString& path)
00015 {
00016     const QFileInfo info(path);
00017     const QString target = info.isDir() ? info.absoluteFilePath() : info.absolutePath();
00018     QDesktopServices::openUrl(QUrl::fromLocalFile(target));
00019 }
00020 }
00021
00022 #endif
```

## 8.203 FfmpegPath.cpp

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "FfmpegPath.h"
00004
00005 #include <QCoreApplication>
00006 #include <QDir>
00007 #include <QFileInfo>
00008 #include <QStringList>
00009 #include <QStandardPaths>
```

```

00010
00011 namespace
00012 {
00013 QString absoluteExecutablePath(const QString& candidate)
00014 {
00015     if (candidate.isEmpty())
00016         return {};
00017
00018     const QFileInfo info(candidate);
00019     if (info.exists() && info.isFile())
00020         return info.absoluteFilePath();
00021
00022     return {};
00023 }
00024 }
00025
00026 QString resolveFfmpegExecutablePath()
00027 {
00028 #ifdef Q_OS_WIN
00029     const QString bundledWindowsPath = absoluteExecutablePath(
00030         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg.exe"));
00031     if (!bundledWindowsPath.isEmpty())
00032         return bundledWindowsPath;
00033 #endif
00034
00035 #ifdef FITLYZER_FFmpeg_PATH
00036     const QString configured = QString::fromUtf8(FITLYZER_FFmpeg_PATH);
00037     const QStringList candidates = {
00038         configured,
00039         QDir(QCoreApplication::applicationDirPath()).filePath(configured),
00040         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg.exe"),
00041         QDir(QCoreApplication::applicationDirPath()).filePath("ffmpeg"),
00042         QDir(QCoreApplication::applicationDirPath()).filePath("../Resources/ffmpeg"),
00043         QDir(QCoreApplication::applicationDirPath()).filePath("../Resources/ffmpeg.exe")
00044     };
00045
00046     for (const QString& candidate : candidates)
00047     {
00048         const QString resolved = absoluteExecutablePath(candidate);
00049         if (!resolved.isEmpty())
00050             return resolved;
00051     }
00052 #endif
00053
00054     const QString systemFfmpeg = QStandardPaths::findExecutable(QStringLiteral("ffmpeg"));
00055     if (!systemFfmpeg.isEmpty())
00056         return systemFfmpeg;
00057
00058     return QStringLiteral("ffmpeg");
00059 }

```

## 8.204 FfmpegPath.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 QString resolveFfmpegExecutablePath();

```

## 8.205 UuidGenerator.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <QUuid>
00008
00009 namespace UuidGenerator
00010 {
00011     inline QString create()
00012     {
00013         return QUuid::createUuid().toString(QUuid::WithoutBraces);
00014     }
00015 }
00016

```

## 8.206 VideoEncoder.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "VideoEncoder.h"
00004
00005 #include <QProcess>
00006
00007 #include "video/VideoExportSettings.h"
00008
00009 namespace VideoEncoder
00010 {
00011 QStringList ffmpegArguments(const VideoExportSettings& settings, int fps)
00012 {
00013     return {
00014         "-y",
00015         "-f", "rawvideo",
00016         "-pixel_format", "rgba",
00017         "-video_size", QString("%1x%2").arg(settings.width).arg(settings.height),
00018         "-framerate", QString::number(fps),
00019         "-i", "-",
00020         "-an",
00021         "-c:v", "libx264",
00022         "-preset", "medium",
00023         "-crf", "20",
00024         "-pix_fmt", "yuv420p",
00025         settings.outputPath
00026     };
00027 }
00028
00029 bool startFfmpeg(QProcess& process,
00030                 const QString& ffmpegPath,
00031                 const VideoExportSettings& settings,
00032                 int fps,
00033                 QString& errorMessage)
00034 {
00035     process.setProgram(ffmpegPath);
00036     process.setArguments(ffmpegArguments(settings, fps));
00037     process.setProcessChannelMode(QProcess::MergedChannels);
00038     process.start();
00039
00040     if (process.waitForStarted())
00041         return true;
00042
00043     const QString detail = process.errorString().trimmed();
00044     if (detail.isEmpty())
00045         errorMessage = QString("Failed to start FFmpeg at '%1'.").arg(ffmpegPath);
00046     else
00047         errorMessage = QString("Failed to start FFmpeg at '%1': %2").arg(ffmpegPath, detail);
00048     return false;
00049 }
00050 }

```

## 8.207 VideoEncoder.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006 #include <QStringList>
00007
00008 class QProcess;
00009 struct VideoExportSettings;
00010
00011 namespace VideoEncoder
00012 {
00013 QStringList ffmpegArguments(const VideoExportSettings& settings, int fps);
00014 bool startFfmpeg(QProcess& process,
00015                 const QString& ffmpegPath,
00016                 const VideoExportSettings& settings,
00017                 int fps,
00018                 QString& errorMessage);
00019 }

```

## 8.208 OverlayRenderer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "OverlayRenderer.h"

```

```

00004
00005 #include <cmath>
00006 #include <limits>
00007
00008 namespace OverlayRenderer
00009 {
00010 QString formatDuration(double seconds)
00011 {
00012     const int total = static_cast<int>(std::round(std::max(0.0, seconds)));
00013     const int h = total / 3600;
00014     const int m = (total % 3600) / 60;
00015     const int s = total % 60;
00016     if (h > 0)
00017         return QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00018     return QString("%1:%2").arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00019 }
00020
00021 QString formatLegendValue(double value, ColorMetric metric)
00022 {
00023     switch (metric)
00024     {
00025         case ColorMetric::Speed:
00026             return QString::number(value, 'f', 1);
00027         case ColorMetric::Power:
00028         case ColorMetric::HeartRate:
00029         case ColorMetric::Cadence:
00030         case ColorMetric::Altitude:
00031             return QString::number(static_cast<int>(std::round(value)));
00032         case ColorMetric::Gradient:
00033         case ColorMetric::None:
00034             return QString::number(value, 'f', 1);
00035     }
00036
00037     return QString::number(value, 'f', 1);
00038 }
00039
00040 QString zoneLegendText(const Zone& zone, ColorMetric metric)
00041 {
00042     const QString unit = colorMetricUnit(metric);
00043     const bool hasUnit = !unit.isEmpty();
00044     const QString valueSuffix = hasUnit ? QString(" %1").arg(unit) : QString();
00045     const double hugeThreshold = std::numeric_limits<double>::max() * 0.5;
00046     const bool openEnded = !std::isfinite(zone.maxValue) || zone.maxValue >= hugeThreshold;
00047
00048     if (openEnded)
00049         return QString("%1 > %2%3").arg(zone.name, formatLegendValue(zone.minValue, metric),
00050 valueSuffix);
00051     if (zone.minValue <= 0.0)
00052         return QString("%1 < %2%3").arg(zone.name, formatLegendValue(zone.maxValue, metric),
00053 valueSuffix);
00054     return QString("%1 %2-%3%4")
00055         .arg(zone.name,
00056             formatLegendValue(zone.minValue, metric),
00057             formatLegendValue(zone.maxValue, metric),
00058             valueSuffix);
00059 }
00060 }

```

## 8.209 OverlayRenderer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 #include "core/zones/ZoneDefinition.h"
00008
00009 namespace OverlayRenderer
00010 {
00011 QString formatDuration(double seconds);
00012 QString formatLegendValue(double value, ColorMetric metric);
00013 QString zoneLegendText(const Zone& zone, ColorMetric metric);
00014 }

```

## 8.210 VideoRenderer.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "VideoRenderer.h"
00004

```

```

00005 #include <algorithm>
00006 #include <cmath>
00007 #include <limits>
00008
00009 namespace
00010 {
00011     constexpr int kTilePx = 256;
00012     constexpr double kPi = 3.14159265358979323846;
00013 }
00014
00015 namespace VideoRenderer
00016 {
00017     TileViewport computeTileViewport(const QPointF& centerTile, int width, int height)
00018     {
00019         TileViewport viewport;
00020         viewport.tileX0 = static_cast<int>(std::floor(centerTile.x() - static_cast<double>(width) / (2.0 *
00021 kTilePx))) - 1;
00022         viewport.tileY0 = static_cast<int>(std::floor(centerTile.y() - static_cast<double>(height) / (2.0
00023 * kTilePx))) - 1;
00024         viewport.nX = static_cast<int>(std::ceil(static_cast<double>(width) / kTilePx)) + 3;
00025         viewport.nY = static_cast<int>(std::ceil(static_cast<double>(height) / kTilePx)) + 3;
00026         return viewport;
00027     }
00028
00029     std::vector<VideoTileId> tilesForViewport(const TileViewport& viewport, int zoom)
00030     {
00031         std::vector<VideoTileId> tiles;
00032         const int maxTile = (1 << zoom) - 1;
00033         tiles.reserve(static_cast<size_t>((viewport.nX + 1) * (viewport.nY + 1)));
00034
00035         for (int ty = 0; ty <= viewport.nY; ++ty)
00036         {
00037             for (int tx = 0; tx <= viewport.nX; ++tx)
00038             {
00039                 const int tileX = viewport.tileX0 + tx;
00040                 const int tileY = viewport.tileY0 + ty;
00041                 if (tileX < 0 || tileY < 0 || tileX > maxTile || tileY > maxTile)
00042                     continue;
00043                 tiles.push_back({ zoom, tileX, tileY });
00044             }
00045         }
00046         return tiles;
00047     }
00048
00049     QPointF latLonToTile(double lat, double lon, int zoom)
00050     {
00051         const double n = std::pow(2.0, zoom);
00052         const double x = (lon + 180.0) / 360.0 * n;
00053         const double rad = lat * kPi / 180.0;
00054         const double y = (1.0 - std::log(std::tan(rad) + 1.0 / std::cos(rad)) / kPi) / 2.0 * n;
00055         return { x, y };
00056     }
00057
00058     QPointF tileToScreen(const QPointF& tilePt, const QPointF& centerTile, int width, int height)
00059     {
00060         return {
00061             width / 2.0 + (tilePt.x() - centerTile.x()) * kTilePx,
00062             height / 2.0 + (tilePt.y() - centerTile.y()) * kTilePx
00063         };
00064     }
00065
00066     int nearestRecordIndex(const std::vector<RideRecord>& records, double elapsedSeconds)
00067     {
00068         if (records.empty())
00069             return -1;
00070
00071         int best = 0;
00072         double bestDist = std::numeric_limits<double>::max();
00073         for (int i = 0; i < static_cast<int>(records.size()); ++i)
00074         {
00075             const double dist = std::abs(records[static_cast<size_t>(i)].elapsedSeconds - elapsedSeconds);
00076             if (dist < bestDist)
00077             {
00078                 bestDist = dist;
00079                 best = i;
00080             }
00081         }
00082         return best;
00083     }
00084
00085     bool interpolateGpsAtTime(const std::vector<const RideRecord*>& gpsRecords,
00086                             double elapsedSeconds,
00087                             double& outLat,
00088                             double& outLon)
00089     {
00090         if (gpsRecords.empty())

```

```

00090         return false;
00091
00092     if (elapsedSeconds <= gpsRecords.front()->elapsedSeconds)
00093     {
00094         outLat = gpsRecords.front()->latitude;
00095         outLon = gpsRecords.front()->longitude;
00096         return true;
00097     }
00098
00099     if (elapsedSeconds >= gpsRecords.back()->elapsedSeconds)
00100     {
00101         outLat = gpsRecords.back()->latitude;
00102         outLon = gpsRecords.back()->longitude;
00103         return true;
00104     }
00105
00106     auto it = std::lower_bound(
00107         gpsRecords.begin(),
00108         gpsRecords.end(),
00109         elapsedSeconds,
00110         [](const RideRecord* rec, double t)
00111         {
00112             return rec->elapsedSeconds < t;
00113         });
00114
00115     if (it == gpsRecords.begin())
00116     {
00117         outLat = (*it)->latitude;
00118         outLon = (*it)->longitude;
00119         return true;
00120     }
00121
00122     const RideRecord* b = *it;
00123     const RideRecord* a = *(it - 1);
00124     const double dt = b->elapsedSeconds - a->elapsedSeconds;
00125     if (dt <= 1e-6)
00126     {
00127         outLat = a->latitude;
00128         outLon = a->longitude;
00129         return true;
00130     }
00131
00132     const double alpha = std::clamp((elapsedSeconds - a->elapsedSeconds) / dt, 0.0, 1.0);
00133     outLat = a->latitude + (b->latitude - a->latitude) * alpha;
00134     outLon = a->longitude + (b->longitude - a->longitude) * alpha;
00135     return true;
00136 }
00137 }

```

## 8.211 VideoRenderer.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QPointF>
00006
00007 #include <vector>
00008
00009 #include "fit/RideData.h"
00010 #include "video/VideoTileProvider.h"
00011
00012 namespace VideoRenderer
00013 {
00014     struct TileViewport
00015     {
00016         int tileX0 = 0;
00017         int tileY0 = 0;
00018         int nX = 0;
00019         int nY = 0;
00020
00021         bool operator==(const TileViewport& other) const
00022         {
00023             return tileX0 == other.tileX0 &&
00024                 tileY0 == other.tileY0 &&
00025                 nX == other.nX &&
00026                 nY == other.nY;
00027         }
00028     };
00029
00030     TileViewport computeTileViewport(const QPointF& centerTile, int width, int height);
00031     std::vector<VideoTileId> tilesForViewport(const TileViewport& viewport, int zoom);
00032     QPointF latLonToTile(double lat, double lon, int zoom);
00033     QPointF tileToScreen(const QPointF& tilePt, const QPointF& centerTile, int width, int height);

```



```

00034 int nearestRecordIndex(const std::vector<RideRecord>& records, double elapsedSeconds);
00035 bool interpolateGpsAtTime(const std::vector<const RideRecord*>& gpsRecords,
00036                          double elapsedSeconds,
00037                          double& outLat,
00038                          double& outLon);
00039 }

```

## 8.212 VideoExportDialog.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoExportDialog.h"
00005
00006 #include "core/settings/DateFormatter.h"
00007
00008 #include <QCheckBox>
00009 #include <QComboBox>
00010 #include <QDate>
00011 #include <QDialogButtonBox>
00012 #include <QDoubleSpinBox>
00013 #include <QFileDialog>
00014 #include <QFileInfo>
00015 #include <QFormLayout>
00016 #include <QGridLayout>
00017 #include <QGroupBox>
00018 #include <QHBoxLayout>
00019 #include <QLabel>
00020 #include <QLineEdit>
00021 #include <QMessageBox>
00022 #include <QProgressBar>
00023 #include <QPushButton>
00024 #include <QRadioButton>
00025 #include <QScrollArea>
00026 #include <QSettings>
00027 #include <QSpinBox>
00028 #include <QThread>
00029 #include <QVBoxLayout>
00030
00031 #include "video/VideoRenderJob.h"
00032
00033 namespace
00034 {
00035 QString mapStyleName(MapStyle style)
00036 {
00037     switch (style)
00038     {
00039         case MapStyle::Street: return "Street";
00040         case MapStyle::Light: return "Light";
00041         case MapStyle::Dark: return "Dark";
00042         case MapStyle::Satellite: return "Satellite";
00043         case MapStyle::Terrain: return "Terrain";
00044         case MapStyle::Topographic: return "Topographic";
00045         case MapStyle::Minimal: return "Minimal";
00046     }
00047     return "Street";
00048 }
00049
00050 QString makeDefaultFilename(const VideoExportSettings& defaults)
00051 {
00052     QString activity = defaults.activityName.trimmed();
00053     if (activity.isEmpty())
00054         activity = "Ride";
00055
00056     QString athlete = defaults.athleteName.trimmed();
00057     if (athlete.isEmpty())
00058         athlete = "Athlete";
00059
00060     const QString date = DateFormatter::toIsoDate(QDate::currentDate());
00061     QString name = QString("%1-%2-%3.mp4").arg(date, activity, athlete);
00062
00063     static const QString invalid = QStringLiteral("\\/:*?\"<>|");
00064     for (const QChar c : invalid)
00065         name.replace(c, '-');
00066
00067     while (name.contains("--"))
00068         name.replace("--", "-");
00069
00070     return name;
00071 }
00072
00073 QString formatVideoLength(double seconds)
00074 {
00075     const int total = static_cast<int>(std::round(std::max(0.0, seconds)));

```

```

00076     const int h = total / 3600;
00077     const int m = (total % 3600) / 60;
00078     const int s = total % 60;
00079     if (h > 0)
00080         return QString("%1:%2:%3").arg(h).arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00081     return QString("%1:%2").arg(m, 2, 10, QChar('0')).arg(s, 2, 10, QChar('0'));
00082 }
00083 }
00084
00085 VideoExportDialog::VideoExportDialog(const VideoExportSettings& defaults, QWidget* parent)
00086     : QDialog(parent)
00087     , m_defaults(defaults)
00088 {
00089     setWindowTitle("Create Video");
00090     setModal(true);
00091     resize(620, 720);
00092
00093     auto* root = new QVBoxLayout(this);
00094     auto* scrollArea = new QScrollArea(this);
00095     scrollArea->setWidgetResizable(true);
00096     scrollArea->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
00097
00098     auto* contentWidget = new QWidget(scrollArea);
00099     auto* contentLayout = new QVBoxLayout(contentWidget);
00100     contentLayout->setContentsMargins(0, 0, 0, 0);
00101     contentLayout->setSpacing(root->spacing());
00102
00103     scrollArea->setWidget(contentWidget);
00104
00105     auto* outputGroup = new QGroupBox("General", contentWidget);
00106     auto* outputLayout = new QGridLayout(outputGroup);
00107     m_outputEdit = new QLineEdit(outputGroup);
00108     if (defaults.outputPath.isEmpty())
00109         m_outputEdit->setText(makeDefaultFilename(defaults));
00110     else
00111         m_outputEdit->setText(defaults.outputPath);
00112
00113     auto* browseButton = new QPushButton("Browse...", outputGroup);
00114     connect(browseButton, &QPushButton::clicked, this, &VideoExportDialog::chooseOutputFile);
00115
00116     outputLayout->addWidget(new QLabel("Output file:"), 0, 0);
00117     outputLayout->addWidget(m_outputEdit, 0, 1);
00118     outputLayout->addWidget(browseButton, 0, 2);
00119
00120     m_selectedSegmentRadio = new QRadioButton("Current selected segment", outputGroup);
00121     m_entireActivityRadio = new QRadioButton("Entire activity", outputGroup);
00122     m_selectedSegmentRadio->setChecked(defaults.useSelectedSegment);
00123     m_entireActivityRadio->setChecked(!defaults.useSelectedSegment);
00124     outputLayout->addWidget(m_selectedSegmentRadio, 1, 0, 1, 3);
00125     outputLayout->addWidget(m_entireActivityRadio, 2, 0, 1, 3);
00126
00127     auto* videoGroup = new QGroupBox("Video", contentWidget);
00128     auto* videoLayout = new QFormLayout(videoGroup);
00129
00130     m_resolutionCombo = new QComboBox(videoGroup);
00131     m_resolutionCombo->addItem("1280x720", QSize(1280, 720));
00132     m_resolutionCombo->addItem("1920x1080", QSize(1920, 1080));
00133     m_resolutionCombo->addItem("2560x1440", QSize(2560, 1440));
00134     m_resolutionCombo->addItem("3840x2160", QSize(3840, 2160));
00135
00136     const int defaultResIndex = m_resolutionCombo->findData(QSize(defaults.width, defaults.height));
00137     m_resolutionCombo->setCurrentIndex(defaultResIndex >= 0 ? defaultResIndex : 1);
00138
00139     m_fpsSpin = new QSpinBox(videoGroup);
00140     m_fpsSpin->setRange(15, 60);
00141     m_fpsSpin->setValue(defaults.fps <= 0 ? 30 : defaults.fps);
00142
00143     m_speedSpin = new QDoubleSpinBox(videoGroup);
00144     m_speedSpin->setRange(1.0, 20.0);
00145     m_speedSpin->setSingleStep(0.5);
00146     m_speedSpin->setValue(defaults.playbackSpeed < 1.0 ? 3.0 : defaults.playbackSpeed);
00147     m_speedSpin->setSuffix("x");
00148     m_speedSpin->setToolTip("Acceleration factor: 1x = real-time, 5x = 5 seconds FIT per 1 second video.");
00149
00150     videoLayout->addRow("Resolution:", m_resolutionCombo);
00151     videoLayout->addRow("FPS:", m_fpsSpin);
00152     videoLayout->addRow("Playback speed:", m_speedSpin);
00153     m_videoLengthLabel = new QLabel(videoGroup);
00154     videoLayout->addRow("Video Length:", m_videoLengthLabel);
00155     videoLayout->addRow("Format:", new QLabel("MP4 (H.264)", videoGroup));
00156
00157     connect(m_selectedSegmentRadio, &QRadioButton::toggled, this,
00158         &VideoExportDialog::updateVideoLengthLabel);
00158     connect(m_entireActivityRadio, &QRadioButton::toggled, this,
00159         &VideoExportDialog::updateVideoLengthLabel);
00159     connect(m_speedSpin, qOverload<double>(&QDoubleSpinBox::valueChanged), this,

```

```

        &VideoExportDialog::updateVideoLengthLabel);
00160
00161     auto* mapGroup = new QGroupBox("Map", contentWidget);
00162     auto* mapLayout = new QGridLayout(mapGroup);
00163
00164     m_mapStyleCombo = new QComboBox(mapGroup);
00165     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Street), static_cast<int>(MapStyle::Street));
00166     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Light), static_cast<int>(MapStyle::Light));
00167     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Dark), static_cast<int>(MapStyle::Dark));
00168     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Satellite),
static_cast<int>(MapStyle::Satellite));
00169     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Terrain), static_cast<int>(MapStyle::Terrain));
00170     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Topographic),
static_cast<int>(MapStyle::Topographic));
00171     m_mapStyleCombo->addItem(mapStyleName(MapStyle::Minimal), static_cast<int>(MapStyle::Minimal));
00172     const int styleIndex = m_mapStyleCombo->findData(static_cast<int>(defaults.mapStyle));
00173     m_mapStyleCombo->setCurrentIndex(styleIndex >= 0 ? styleIndex : 0);
00174
00175     m_fixedZoomSpin = new QSpinBox(mapGroup);
00176     m_fixedZoomSpin->setRange(1, 18);
00177     m_fixedZoomSpin->setValue(defaults.fixedZoom <= 0 ? 18 : defaults.fixedZoom);
00178
00179     mapLayout->addWidget(new QLabel("Style:", 0, 0);
00180     mapLayout->addWidget(m_mapStyleCombo, 0, 1, 1, 2);
00181     mapLayout->addWidget(new QLabel("Zoom:", 1, 0);
00182     mapLayout->addWidget(m_fixedZoomSpin, 1, 1);
00183     auto* followLabel = new QLabel("Follow athlete: enabled", mapGroup);
00184     mapLayout->addWidget(followLabel, 2, 0, 1, 3);
00185
00186     auto* routeGroup = new QGroupBox("Route", contentWidget);
00187     auto* routeLayout = new QFormLayout(routeGroup);
00188     m_routeColorCombo = new QComboBox(routeGroup);
00189     m_routeColorCombo->addItem("FTP Zone", static_cast<int>(ColorMetric::Power));
00190     m_routeColorCombo->addItem("Power", static_cast<int>(ColorMetric::Power));
00191     m_routeColorCombo->addItem("Heart Rate", static_cast<int>(ColorMetric::HeartRate));
00192     m_routeColorCombo->addItem("Cadence", static_cast<int>(ColorMetric::Cadence));
00193     m_routeColorCombo->addItem("Speed", static_cast<int>(ColorMetric::Speed));
00194     m_routeColorCombo->addItem("Altitude", static_cast<int>(ColorMetric::Altitude));
00195     m_routeColorCombo->addItem("Single Color", static_cast<int>(ColorMetric::None));
00196     routeLayout->addRow("Color route by:", m_routeColorCombo);
00197
00198     auto* telemetryGroup = new QGroupBox("Telemetry Overlays", contentWidget);
00199     auto* telemetryLayout = new QGridLayout(telemetryGroup);
00200
00201     m_powerCheck = new QCheckBox("Power", telemetryGroup);
00202     m_hrCheck = new QCheckBox("Heart Rate", telemetryGroup);
00203     m_cadenceCheck = new QCheckBox("Cadence", telemetryGroup);
00204     m_speedCheck = new QCheckBox("Speed", telemetryGroup);
00205     m_altitudeCheck = new QCheckBox("Altitude", telemetryGroup);
00206     m_timeCheck = new QCheckBox("Time", telemetryGroup);
00207     m_athleteCheck = new QCheckBox("Athlete Name", telemetryGroup);
00208     m_telemetryPanelCheck = new QCheckBox("Telemetry Box", telemetryGroup);
00209     m_legendCheck = new QCheckBox("Route Legend", telemetryGroup);
00210     m_chartsCheck = new QCheckBox("Metric Charts", telemetryGroup);
00211
00212     m_powerCheck->setChecked(defaults.overlayPower);
00213     m_hrCheck->setChecked(defaults.overlayHeartRate);
00214     m_cadenceCheck->setChecked(defaults.overlayCadence);
00215     m_speedCheck->setChecked(defaults.overlaySpeed);
00216     m_altitudeCheck->setChecked(defaults.overlayAltitude);
00217     m_timeCheck->setChecked(defaults.overlayTime);
00218     m_athleteCheck->setChecked(defaults.overlayAthleteName);
00219     m_telemetryPanelCheck->setChecked(defaults.overlayTelemetryPanel);
00220     m_legendCheck->setChecked(defaults.overlayRouteLegend);
00221     m_chartsCheck->setChecked(defaults.overlayCharts);
00222
00223     telemetryLayout->addWidget(m_powerCheck, 0, 0);
00224     telemetryLayout->addWidget(m_hrCheck, 0, 1);
00225     telemetryLayout->addWidget(m_cadenceCheck, 0, 2);
00226     telemetryLayout->addWidget(m_speedCheck, 1, 0);
00227     telemetryLayout->addWidget(m_altitudeCheck, 1, 1);
00228     telemetryLayout->addWidget(m_timeCheck, 1, 2);
00229     telemetryLayout->addWidget(m_athleteCheck, 2, 0);
00230     telemetryLayout->addWidget(m_telemetryPanelCheck, 2, 1);
00231     telemetryLayout->addWidget(m_legendCheck, 2, 2);
00232     telemetryLayout->addWidget(m_chartsCheck, 3, 0);
00233
00234     auto* overlayButtons = new QHBoxLayout;
00235     auto* enableAllOverlaysButton = new QPushButton("Enable All", telemetryGroup);
00236     auto* disableAllOverlaysButton = new QPushButton("Disable All", telemetryGroup);
00237     connect(enableAllOverlaysButton, &QPushButton::clicked, this, [this]() {
setAllOverlayChecks(true); });
00238     connect(disableAllOverlaysButton, &QPushButton::clicked, this, [this]() {
setAllOverlayChecks(false); });
00239     overlayButtons->addWidget(enableAllOverlaysButton);
00240     overlayButtons->addWidget(disableAllOverlaysButton);
00241     overlayButtons->addStretch(1);

```

```

00242     telemetryLayout->addLayout (overlayButtons, 4, 0, 1, 3);
00243
00244     auto* tileGroup = new QGroupBox("Tiles", contentWidget);
00245     auto* tileLayout = new QVBoxLayout(tileGroup);
00246     m_deleteTemporaryTilesCheck = new QCheckBox("Delete temporary video tiles after export",
tileGroup);
00247     m_deleteTemporaryTilesCheck->setToolTip(
00248         "When enabled, tiles downloaded during export are removed after export finishes or is
canceled.");
00249     QSettings settings("Fitlyzer", "FitlyzerC");
00250     m_deleteTemporaryTilesCheck->setChecked(
00251         settings.value("video/deleteTemporaryTilesAfterExport", true).toBool());
00252     tileLayout->addWidget(m_deleteTemporaryTilesCheck);
00253
00254     auto* progressGroup = new QGroupBox("Export Progress", contentWidget);
00255     auto* progressLayout = new QVBoxLayout(progressGroup);
00256     m_stageLabel = new QLabel("Ready", progressGroup);
00257     m_frameLabel = new QLabel("Frame: -", progressGroup);
00258     m_etaLabel = new QLabel("ETA: -", progressGroup);
00259     m_tileStatsLabel = new QLabel("Tiles: required - | on-disk - | downloaded -", progressGroup);
00260     m_progressBar = new QProgressBar(progressGroup);
00261     m_progressBar->setRange(0, 100);
00262     m_progressBar->setValue(0);
00263
00264     progressLayout->addWidget(m_stageLabel);
00265     progressLayout->addWidget(m_frameLabel);
00266     progressLayout->addWidget(m_etaLabel);
00267     progressLayout->addWidget(m_tileStatsLabel);
00268     progressLayout->addWidget(m_progressBar);
00269
00270     auto* buttons = new QHBoxLayout;
00271     buttons->addStretch(1);
00272     m_exportButton = new QPushButton("Export", this);
00273     m_cancelButton = new QPushButton("Cancel", this);
00274     buttons->addWidget(m_exportButton);
00275     buttons->addWidget(m_cancelButton);
00276
00277     connect(m_exportButton, &QPushButton::clicked, this, &VideoExportDialog::startExport);
00278     connect(m_cancelButton, &QPushButton::clicked, this, &VideoExportDialog::cancelOrClose);
00279
00280     contentLayout->addWidget(outputGroup);
00281     contentLayout->addWidget(videoGroup);
00282     contentLayout->addWidget(mapGroup);
00283     contentLayout->addWidget(routeGroup);
00284     contentLayout->addWidget(telemetryGroup);
00285     contentLayout->addWidget(tileGroup);
00286     contentLayout->addWidget(progressGroup);
00287     contentLayout->addStretch(1);
00288     root->addWidget(scrollArea);
00289     root->addLayout(buttons);
00290
00291     updateVideoLengthLabel();
00292 }
00293
00294 VideoExportDialog::~VideoExportDialog()
00295 {
00296     if (m_job)
00297         m_job->cancel();
00298     if (m_thread)
00299     {
00300         m_thread->quit();
00301         m_thread->wait();
00302     }
00303 }
00304
00305 void VideoExportDialog::chooseOutputFile()
00306 {
00307     const QString path = QFileDialog::getSaveFileName(
00308         this,
00309         "Export Video",
00310         m_outputEdit->text().trimmed(),
00311         "MP4 Video (*.mp4)");
00312     if (!path.isEmpty())
00313         m_outputEdit->setText(path);
00314 }
00315
00316 VideoExportSettings VideoExportDialog::collectSettings() const
00317 {
00318     VideoExportSettings s = m_defaults;
00319     s.outputPath = m_outputEdit->text().trimmed();
00320
00321     const QSize res = m_resolutionCombo->currentData().toSize();
00322     s.width = res.width();
00323     s.height = res.height();
00324     s.fps = m_fpsSpin->value();
00325     s.playbackSpeed = m_speedSpin->value();
00326

```

```

00327     s.useSelectedSegment = m_selectedSegmentRadio->isChecked();
00328
00329     s.mapStyle = static_cast<MapStyle>(m_mapStyleCombo->currentData().toInt());
00330     s.autoZoom = false;
00331     s.fixedZoom = m_fixedZoomSpin->value();
00332
00333     s.routeColorMetric = static_cast<ColorMetric>(m_routeColorCombo->currentData().toInt());
00334
00335     s.overlayPower = m_powerCheck->isChecked();
00336     s.overlayHeartRate = m_hrCheck->isChecked();
00337     s.overlayCadence = m_cadenceCheck->isChecked();
00338     s.overlaySpeed = m_speedCheck->isChecked();
00339     s.overlayAltitude = m_altitudeCheck->isChecked();
00340     s.overlayTime = m_timeCheck->isChecked();
00341     s.overlayAthleteName = m_athleteCheck->isChecked();
00342     s.overlayTelemetryPanel = m_telemetryPanelCheck->isChecked();
00343     s.overlayRouteLegend = m_legendCheck->isChecked();
00344     s.overlayCharts = m_chartsCheck->isChecked();
00345     s.deleteTemporaryTilesAfterExport = m_deleteTemporaryTilesCheck
00346         ? m_deleteTemporaryTilesCheck->isChecked()
00347         : true;
00348
00349     QSettings appSettings("Fitlyzer", "FitlyzerC");
00350     appSettings.setValue("video/deleteTemporaryTilesAfterExport", s.deleteTemporaryTilesAfterExport);
00351
00352     return s;
00353 }
00354
00355 void VideoExportDialog::startExport()
00356 {
00357     if (m_running)
00358         return;
00359
00360     VideoExportSettings settings = collectSettings();
00361     if (settings.outputPath.isEmpty())
00362     {
00363         QMessageBox::warning(this, "Create Video", "Choose an output file.");
00364         return;
00365     }
00366
00367     if (!settings.outputPath.endsWith(".mp4", Qt::CaseInsensitive))
00368         settings.outputPath += ".mp4";
00369
00370     m_thread = new QThread(this);
00371     m_job = new VideoRenderJob(settings);
00372     m_job->moveToThread(m_thread);
00373
00374     connect(m_thread, &QThread::started, m_job, &VideoRenderJob::run);
00375     connect(m_job, &VideoRenderJob::stageChanged, this, &VideoExportDialog::onStageChanged);
00376     connect(m_job, &VideoRenderJob::progressChanged, this, &VideoExportDialog::onProgressChanged);
00377     connect(m_job, &VideoRenderJob::frameProgressChanged, this,
00378         &VideoExportDialog::onFrameProgressChanged);
00379     connect(m_job, &VideoRenderJob::tileStatsChanged, this, &VideoExportDialog::onTileStatsChanged);
00380     connect(m_job, &VideoRenderJob::finished, this, &VideoExportDialog::onFinished);
00381     connect(m_job, &VideoRenderJob::finished, m_thread, &QThread::quit);
00382     connect(m_thread, &QThread::finished, m_job, &QObject::deleteLater);
00383     connect(m_thread, &QThread::finished, m_thread, &QObject::deleteLater);
00384
00385     m_running = true;
00386     m_exportButton->setEnabled(false);
00387     m_cancelButton->setText("Cancel Export");
00388
00389     m_stageLabel->setText("Preparing data...");
00390     m_frameLabel->setText("Frame: 0 / 0");
00391     m_etaLabel->setText("ETA: --:--");
00392     m_tileStatsLabel->setText("Tiles: required - | on-disk - | downloaded -");
00393     m_progressBar->setValue(0);
00394
00395     m_thread->start();
00396 }
00397
00398 void VideoExportDialog::cancelOrClose()
00399 {
00400     if (m_running && m_job)
00401     {
00402         m_job->cancel();
00403         m_cancelButton->setEnabled(false);
00404         return;
00405     }
00406     reject();
00407 }
00408
00409 void VideoExportDialog::onStageChanged(const QString& stage)
00410 {
00411     m_stageLabel->setText(stage);
00412 }

```

```

00413
00414 void VideoExportDialog::onProgressChanged(int value, int maximum)
00415 {
00416     m_progressBar->setRange(0, std::max(1, maximum));
00417     m_progressBar->setValue(value);
00418 }
00419
00420 void VideoExportDialog::onFrameProgressChanged(int currentFrame, int totalFrames, const QString&
etaText)
00421 {
00422     m_frameLabel->setText(QString("Frame: %1 / %2").arg(currentFrame).arg(totalFrames));
00423     m_etaLabel->setText(QString("ETA: %1").arg(etaText));
00424 }
00425
00426 void VideoExportDialog::onTileStatsChanged(const QString& statsText)
00427 {
00428     if (m_tileStatsLabel)
00429         m_tileStatsLabel->setText(statsText);
00430 }
00431
00432 void VideoExportDialog::onFinished(bool success, const QString& message, bool canceled)
00433 {
00434     m_running = false;
00435     m_exportButton->setEnabled(true);
00436     m_cancelButton->setEnabled(true);
00437     m_cancelButton->setText("Close");
00438
00439     m_thread = nullptr;
00440     m_job = nullptr;
00441
00442     if (success)
00443     {
00444         QMessageBox::information(this, "Create Video", message);
00445         accept();
00446         return;
00447     }
00448
00449     if (!canceled)
00450         QMessageBox::critical(this, "Create Video", message);
00451 }
00452
00453 void VideoExportDialog::setAllOverlayChecks(bool enabled)
00454 {
00455     m_powerCheck->setChecked(enabled);
00456     m_hrCheck->setChecked(enabled);
00457     m_cadenceCheck->setChecked(enabled);
00458     m_speedCheck->setChecked(enabled);
00459     m_altitudeCheck->setChecked(enabled);
00460     m_timeCheck->setChecked(enabled);
00461     m_athleteCheck->setChecked(enabled);
00462     m_telemetryPanelCheck->setChecked(enabled);
00463     m_legendCheck->setChecked(enabled);
00464     m_chartsCheck->setChecked(enabled);
00465 }
00466
00467 void VideoExportDialog::updateVideoLengthLabel()
00468 {
00469     double realSeconds = 0.0;
00470     if (m_selectedSegmentRadio->isChecked())
00471     {
00472         realSeconds = std::max(0.0, m_defaults.segmentEndSeconds - m_defaults.segmentStartSeconds);
00473     }
00474     else if (!m_defaults.rideData.records.empty())
00475     {
00476         realSeconds = std::max(0.0, m_defaults.rideData.records.back().elapsedSeconds);
00477     }
00478
00479     const double speed = std::max(1.0, m_speedSpin->value());
00480     const double videoSeconds = realSeconds / speed;
00481     m_videoLengthLabel->setText(
00482         QString("Real %1 / Video %2")
00483         .arg(formatVideoLength(realSeconds), formatVideoLength(videoSeconds)));
00484 }

```

## 8.213 VideoExportDialog.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QDialog>
00007
00008 #include "video/VideoExportSettings.h"

```

```

00009
00010 class QButtonGroup;
00011 class QCheckBox;
00012 class QComboBox;
00013 class QDoubleSpinBox;
00014 class QGroupBox;
00015 class QLabel;
00016 class QLineEdit;
00017 class QProgressBar;
00018 class QPushButton;
00019 class QRadioButton;
00020 class QSpinBox;
00021 class QThread;
00022
00023 class VideoRenderJob;
00024
00031 class VideoExportDialog : public QDialog
00032 {
00033     Q_OBJECT
00034
00035 public:
00041     explicit VideoExportDialog(const VideoExportSettings& defaults, QWidget* parent = nullptr);
00042     ~VideoExportDialog() override;
00043
00044 private slots:
00045     void chooseOutputFile();
00046     void startExport();
00047     void cancelOrClose();
00048     void onStageChanged(const QString& stage);
00049     void onProgressChanged(int value, int maximum);
00050     void onFrameProgressChanged(int currentFrame, int totalFrames, const QString& etaText);
00051     void onTileStatsChanged(const QString& statsText);
00052     void onFinished(bool success, const QString& message, bool canceled);
00053
00054 private:
00055     VideoExportSettings collectSettings() const;
00056     void setAllOverlayChecks(bool enabled);
00057     void updateVideoLengthLabel();
00058
00059     VideoExportSettings m_defaults;
00060
00061     QLineEdit* m_outputEdit = nullptr;
00062     QComboBox* m_resolutionCombo = nullptr;
00063     QSpinBox* m_fpsSpin = nullptr;
00064     QDoubleSpinBox* m_speedSpin = nullptr;
00065     QLabel* m_videoLengthLabel = nullptr;
00066
00067     QRadioButton* m_selectedSegmentRadio = nullptr;
00068     QRadioButton* m_entireActivityRadio = nullptr;
00069
00070     QComboBox* m_mapStyleCombo = nullptr;
00071     QSpinBox* m_fixedZoomSpin = nullptr;
00072
00073     QComboBox* m_routeColorCombo = nullptr;
00074
00075     QCheckBox* m_powerCheck = nullptr;
00076     QCheckBox* m_hrCheck = nullptr;
00077     QCheckBox* m_cadenceCheck = nullptr;
00078     QCheckBox* m_speedCheck = nullptr;
00079     QCheckBox* m_altitudeCheck = nullptr;
00080     QCheckBox* m_timeCheck = nullptr;
00081     QCheckBox* m_athleteCheck = nullptr;
00082     QCheckBox* m_telemetryPanelCheck = nullptr;
00083     QCheckBox* m_legendCheck = nullptr;
00084     QCheckBox* m_chartsCheck = nullptr;
00085     QCheckBox* m_deleteTemporaryTilesCheck = nullptr;
00086
00087     QLabel* m_stageLabel = nullptr;
00088     QLabel* m_frameLabel = nullptr;
00089     QLabel* m_etaLabel = nullptr;
00090     QLabel* m_tileStatsLabel = nullptr;
00091     QProgressBar* m_progressBar = nullptr;
00092
00093     QPushButton* m_exportButton = nullptr;
00094     QPushButton* m_cancelButton = nullptr;
00095
00096     QThread* m_thread = nullptr;
00097     VideoRenderJob* m_job = nullptr;
00098     bool m_running = false;
00099 };

```

## 8.214 VideoExportSettings.h

```
00001 // SPDX-License-Identifier: GPL-3
```

```

00002
00003
00004 #pragma once
00005
00006 #include <QColor>
00007 #include <QString>
00008
00009 #include "core/zones/ZoneDefinition.h"
00010 #include "fit/RideData.h"
00011 #include "maps/TileCache.h"
00012
00019 struct VideoExportSettings
00020 {
00022     QString outputPath;
00023
00025     QString ffmpegPath;
00026
00028     int width = 1920;
00029
00031     int height = 1080;
00032
00034     int fps = 30;
00035
00037     double playbackSpeed = 3.0;
00038
00040     bool useSelectedSegment = true;
00041
00043     double segmentStartSeconds = 0.0;
00044
00046     double segmentEndSeconds = 0.0;
00047
00049     MapStyle mapStyle = MapStyle::Light;
00050
00052     bool autoZoom = true;
00053
00055     int fixedZoom = 18;
00056
00058     bool followAthlete = true;
00059
00061     ColorMetric routeColorMetric = ColorMetric::Power;
00062
00064     QColor singleRouteColor = QColor("#2563eb");
00065
00067     bool overlayPower = true;
00068
00070     bool overlayHeartRate = true;
00071
00073     bool overlayCadence = true;
00074
00076     bool overlaySpeed = true;
00077
00079     bool overlayAltitude = true;
00080
00082     bool overlayTime = true;
00083
00085     bool overlayAthleteName = true;
00086
00088     bool overlayTelemetryPanel = true;
00089
00091     bool overlayRouteLegend = true;
00092
00094     bool overlayCharts = true;
00095
00097     bool deleteTemporaryTilesAfterExport = true;
00098
00100     QString athleteName;
00101
00103     QString activityName;
00104
00106     ColorContext colorContext;
00107
00109     RideData rideData;
00110 };

```

## 8.215 VideoRenderJob.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoRenderJob.h"
00005
00006 #include <QDateTime>
00007 #include <QDir>
00008 #include <QElapsedTimer>

```



```

00009 #include <QFileInfo>
00010 #include <QImage>
00011 #include <QPainter>
00012 #include <QPainterPath>
00013 #include <QPen>
00014 #include <QProcess>
00015 #include <QSet>
00016 #include <QStringList>
00017
00018 #include "core/zones/ColorProvider.h"
00019 #include "core/zones/ZoneCalculator.h"
00020 #include "maps/MapFitMath.h"
00021 #include "video/VideoTileProvider.h"
00022 #include "video/encoding/VideoEncoder.h"
00023 #include "video/overlay/OverlayRenderer.h"
00024 #include "video/rendering/VideoRenderer.h"
00025 #include "utils/FfmpegPath.h"
00026
00027 #include <algorithm>
00028 #include <cmath>
00029 #include <limits>
00030 #include <unordered_set>
00031
00032 namespace
00033 {
00034     static constexpr int kTilePx = 256;
00035     static constexpr int kVideoExportTileCache = 96;
00036
00037     QColor routeColor(const RideRecord& rec,
00038                      ColorMetric metric,
00039                      const ColorContext& context,
00040                      const QColor& singleColor)
00041     {
00042         if (metric == ColorMetric::None)
00043             return singleColor;
00044         return ColorProvider::colorForMetric(metric,
00045                                             metric == ColorMetric::Power ? rec.power :
00046                                             metric == ColorMetric::HeartRate ? rec.heartRate :
00047                                             metric == ColorMetric::Cadence ? rec.cadence :
00048                                             metric == ColorMetric::Speed ? rec.speed :
00049                                             metric == ColorMetric::Altitude ? rec.altitude : 0.0,
00050                                             context);
00051     }
00052
00053     class ExportTileCleanupGuard
00054     {
00055     public:
00056         explicit ExportTileCleanupGuard(VideoTileProvider& provider)
00057             : m_provider(provider)
00058         {}
00059
00060         ~ExportTileCleanupGuard()
00061         {
00062             m_provider.clearMemoryCache();
00063             m_provider.cleanupExportCache();
00064         }
00065
00066     private:
00067         VideoTileProvider& m_provider;
00068     };
00069 }
00070
00071 using namespace OverlayRenderer;
00072 using namespace VideoRenderer;
00073
00074 VideoRenderJob::VideoRenderJob(VideoExportSettings settings, QObject* parent)
00075     : QObject(parent)
00076     , m_settings(std::move(settings))
00077 {}
00078
00079 void VideoRenderJob::cancel()
00080 {
00081     m_cancelRequested.storeRelease(1);
00082 }
00083
00084 void VideoRenderJob::run()
00085 {
00086     const auto canceled = [this]() { return m_cancelRequested.loadAcquire() != 0; };
00087
00088     if (m_settings.rideData.records.empty())
00089     {
00090         emit finished(false, "No ride data to export.", false);
00091         return;
00092     }
00093
00094     QString ffmpegPath = m_settings.ffmpegPath.trimmed();
00095     if (ffmpegPath.isEmpty())

```

```

00096         ffmpegPath = resolveFfmpegExecutablePath();
00097
00098     const QFileInfo outInfo(m_settings.outputPath);
00099     QDir().mkpath(outInfo.absolutePath());
00100
00101     double startSec = std::max(0.0, m_settings.segmentStartSeconds);
00102     double endSec = std::max(startSec, m_settings.segmentEndSeconds);
00103     if (!m_settings.useSelectedSegment)
00104     {
00105         startSec = 0.0;
00106         endSec = m_settings.rideData.records.back().elapsedSeconds;
00107     }
00108
00109     if (endSec <= startSec)
00110     {
00111         emit finished(false, "Selected segment is empty.", false);
00112         return;
00113     }
00114
00115     const int fps = std::max(1, m_settings.fps);
00116     const double playbackSpeed = std::max(1.0, m_settings.playbackSpeed);
00117
00118     std::vector<const RideRecord> gpsRecords;
00119     gpsRecords.reserve(m_settings.rideData.records.size());
00120     double minLat = 90.0;
00121     double maxLat = -90.0;
00122     double minLon = 180.0;
00123     double maxLon = -180.0;
00124
00125     for (const RideRecord& rec : m_settings.rideData.records)
00126     {
00127         if (!rec.hasGps)
00128             continue;
00129         if (rec.elapsedSeconds < startSec || rec.elapsedSeconds > endSec)
00130             continue;
00131
00132         gpsRecords.push_back(&rec);
00133         minLat = std::min(minLat, rec.latitude);
00134         maxLat = std::max(maxLat, rec.latitude);
00135         minLon = std::min(minLon, rec.longitude);
00136         maxLon = std::max(maxLon, rec.longitude);
00137     }
00138
00139     if (gpsRecords.size() < 2)
00140     {
00141         emit finished(false, "Video export requires GPS data in the selected segment.", false);
00142         return;
00143     }
00144
00145     // If the selected segment begins before the first GPS fix, start rendering at
00146     // the first GPS timestamp so the route animation starts immediately.
00147     startSec = std::max(startSec, gpsRecords.front()->elapsedSeconds);
00148     if (endSec <= startSec)
00149     {
00150         emit finished(false, "Selected segment has no duration after GPS alignment.", false);
00151         return;
00152     }
00153
00154     const double sourceDuration = endSec - startSec;
00155     const double outputDuration = sourceDuration / playbackSpeed;
00156     const int totalFrames = std::max(1, static_cast<int>(std::ceil(outputDuration * fps)));
00157
00158     emit stageChanged("Preparing data...");
00159     emit progressChanged(0, totalFrames);
00160
00161     const bool showAnyMetricChart = m_settings.overlayCharts &&
00162         (m_settings.overlayPower ||
00163          m_settings.overlayHeartRate ||
00164          m_settings.overlayCadence ||
00165          m_settings.overlaySpeed ||
00166          m_settings.overlayAltitude);
00167     const int mapHeight = showAnyMetricChart
00168         ? static_cast<int>(std::round(m_settings.height * 0.72))
00169         : m_settings.height;
00170     const int chartHeight = showAnyMetricChart
00171         ? std::max(120, m_settings.height - mapHeight)
00172         : 0;
00173     const double uiScale = std::clamp(static_cast<double>(m_settings.height) / 720.0, 0.85, 2.6);
00174
00175     const int telemetryMargin = static_cast<int>(std::round(18.0 * uiScale));
00176     const int telemetryWidth = static_cast<int>(std::round(320.0 * uiScale));
00177     const int telemetryHeight = static_cast<int>(std::round(180.0 * uiScale));
00178     const int telemetryLeftPadding = static_cast<int>(std::round(12.0 * uiScale));
00179     const int telemetryTopBaseline = static_cast<int>(std::round(26.0 * uiScale));
00180     const int telemetryLineStep = static_cast<int>(std::round(20.0 * uiScale));
00181     const int telemetryFontPx = static_cast<int>(std::round(12.0 * uiScale));
00182

```

```

00183     const int athleteWidth = static_cast<int>(std::round(344.0 * uiScale));
00184     const int athleteHeight = static_cast<int>(std::round(42.0 * uiScale));
00185     const int athleteInset = static_cast<int>(std::round(12.0 * uiScale));
00186     const int athleteFontPx = static_cast<int>(std::round(13.0 * uiScale));
00187
00188     const int chartSidePadding = static_cast<int>(std::round(12.0 * uiScale));
00189     const int chartTopPadding = static_cast<int>(std::round(4.0 * uiScale));
00190     const int chartBottomPadding = static_cast<int>(std::round(8.0 * uiScale));
00191     const int chartLabelFontPx = static_cast<int>(std::round(9.0 * uiScale));
00192     const int chartLabelBaseline = static_cast<int>(std::round(14.0 * uiScale));
00193
00194     const int legendInset = static_cast<int>(std::round(8.0 * uiScale));
00195     const int legendGap = static_cast<int>(std::round(8.0 * uiScale));
00196     const int legendSwatchSize = static_cast<int>(std::round(12.0 * uiScale));
00197     const int legendTitleFontPx = static_cast<int>(std::round(9.0 * uiScale));
00198     const int legendLabelFontPx = static_cast<int>(std::round(8.5 * uiScale));
00199
00200     const std::vector<Zone> routeLegendZones =
ZoneCalculator::zonesForMetric(m_settings.routeColorMetric,
00201 m_settings.colorContext);
00202     const bool showRouteLegend = m_settings.overlayRouteLegend &&
00203         m_settings.routeColorMetric != ColorMetric::None &&
00204         !routeLegendZones.empty();
00205
00206     VideoTileProvider tileProvider(VideoTileProviderConfig{
00207         m_settings.mapStyle,
00208         kVideoExportTileCache,
00209         m_settings.deleteTemporaryTilesAfterExport,
00210         QString()
00211     });
00212     ExportTileCleanupGuard tileCleanupGuard(tileProvider);
00213
00214     int zoom = std::clamp(m_settings.fixedZoom, 1, 18);
00215     if (m_settings.autoZoom)
00216     {
00217         const auto fit = MapFitMath::computeFitToBounds(minLat,
00218                                                         maxLat,
00219                                                         minLon,
00220                                                         maxLon,
00221                                                         m_settings.width,
00222                                                         mapHeight,
00223                                                         18);
00224         zoom = fit.zoom;
00225     }
00226
00227     std::vector<QPointF> gpsTilePoints;
00228     gpsTilePoints.reserve(gpsRecords.size());
00229     for (const RideRecord* rec : gpsRecords)
00230         gpsTilePoints.push_back(latLonToTile(rec->latitude, rec->longitude, zoom));
00231
00232     emit stageChanged("Analyzing required tiles...");
00233
00234     std::unordered_set<VideoTileId, VideoTileIdHash> requiredTiles;
00235     requiredTiles.reserve(static_cast<size_t>(std::max(256, totalFrames / 3)));
00236     for (int frame = 0; frame < totalFrames; ++frame)
00237     {
00238         if (canceled())
00239         {
00240             emit finished(false, "Export canceled.", true);
00241             return;
00242         }
00243
00244         const double sourceSeconds = std::min(
00245             endSec,
00246             startSec + (static_cast<double>(frame) / fps) * playbackSpeed);
00247         double centerLat = 0.0;
00248         double centerLon = 0.0;
00249         if (!interpolateGpsAtTime(gpsRecords, sourceSeconds, centerLat, centerLon))
00250             continue;
00251
00252         const QPointF centerTile = latLonToTile(centerLat, centerLon, zoom);
00253         const TileViewport viewport = computeTileViewport(centerTile, m_settings.width, mapHeight);
00254         const std::vector<VideoTileId> tiles = tilesForViewport(viewport, zoom);
00255         requiredTiles.insert(tiles.begin(), tiles.end());
00256
00257         if (frame % 120 == 0)
00258             emit progressChanged(frame, totalFrames);
00259     }
00260
00261     tileProvider.setRequiredTiles(requiredTiles);
00262     {
00263         const VideoTileStats s = tileProvider.stats();
00264         emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded 0")
00265                                 .arg(s.requiredTiles)
00266                                 .arg(s.onDiskTiles));
00267     }

```

```

00268     emit stageChanged(QString("Analyzed %1 required tiles (%2 available on disk)")
00269         .arg(tileProvider.requiredTileCount())
00270         .arg(tileProvider.requiredTilesAvailableOnDisk()));
00271
00272     emit stageChanged("Encoding video...");
00273
00274     QProcess ffmpeg;
00275     QString ffmpegStartError;
00276     if (!VideoEncoder::startFfmpeg(ffmpeg, ffmpegPath, m_settings, fps, ffmpegStartError))
00277     {
00278         emit finished(false, ffmpegStartError, false);
00279         return;
00280     }
00281
00282     static constexpr double kChartWindowBackSeconds = 180.0;
00283     static constexpr double kChartWindowForwardSeconds = 30.0;
00284
00285     QElapsedTimer timer;
00286     timer.start();
00287
00288     TileViewport lastViewport;
00289     bool hasLastViewport = false;
00290     std::vector<VideoTileId> viewportTiles;
00291     int currentGpsIndex = 0;
00292
00293     for (int frame = 0; frame < totalFrames; ++frame)
00294     {
00295         if (canceled())
00296         {
00297             ffmpeg.kill();
00298             ffmpeg.waitForFinished();
00299             emit finished(false, "Export canceled.", true);
00300             return;
00301         }
00302
00303         const double sourceSeconds = std::min(
00304             endSec,
00305             startSec + (static_cast<double>(frame) / fps) * playbackSpeed);
00306         const int recIndex = nearestRecordIndex(m_settings.rideData.records, sourceSeconds);
00307         if (recIndex < 0)
00308             continue;
00309
00310         const RideRecord& now = m_settings.rideData.records[static_cast<size_t>(recIndex)];
00311         double centerLat = 0.0;
00312         double centerLon = 0.0;
00313         if (!interpolateGpsAtTime(gpsRecords, sourceSeconds, centerLat, centerLon))
00314             continue;
00315         const QPointF centerTile = latLonToTile(centerLat, centerLon, zoom);
00316
00317         const TileViewport viewport = computeTileViewport(centerTile, m_settings.width, mapHeight);
00318         if (!hasLastViewport || !viewport == lastViewport)
00319         {
00320             viewportTiles = tilesForViewport(viewport, zoom);
00321             lastViewport = viewport;
00322             hasLastViewport = true;
00323         }
00324
00325         while (currentGpsIndex + 1 < static_cast<int>(gpsRecords.size()) &&
00326             gpsRecords[static_cast<size_t>(currentGpsIndex + 1)]->elapsedSeconds <= sourceSeconds)
00327         {
00328             ++currentGpsIndex;
00329         }
00330
00331         QImage image(m_settings.width, m_settings.height, QImage::Format_RGBA8888);
00332         image.fill(QColor("#0f172a"));
00333
00334         QPainter p(&image);
00335         p.setRenderHint(QPainter::Antialiasing);
00336
00337         const QRect mapRect(0, 0, m_settings.width, mapHeight);
00338         p.fillRect(mapRect, QColor("#111827"));
00339
00340         for (const VideoTileId& tileId : viewportTiles)
00341         {
00342             const QPointF topLeft = tileToScreen({ static_cast<double>(tileId.x),
00343                 static_cast<double>(tileId.y) },
00344                 centerTile,
00345                 m_settings.width,
00346                 mapHeight);
00347             const QPixmap px = tileProvider.getTile(tileId, true);
00348             if (px.isNull())
00349                 p.fillRect(QRectF(topLeft, QSizeF(kTilePx, kTilePx)), QColor("#334155"));
00350             else
00351                 p.drawPixmap(static_cast<int>(topLeft.x()), static_cast<int>(topLeft.y()), kTilePx,
00352                     kTilePx, px);
00353         }
00354     }
00355

```

```

00353         p.setPen(QPen(QColor(220, 220, 220, 110), 1.5));
00354         for (size_t i = 1; i < gpsTilePoints.size(); ++i)
00355         {
00356             const QPointF ap = tileToScreen(gpsTilePoints[i - 1], centerTile, m_settings.width,
mapHeight);
00357             const QPointF bp = tileToScreen(gpsTilePoints[i], centerTile, m_settings.width,
mapHeight);
00358             p.drawLine(ap, bp);
00359         }
00360
00361         const size_t maxColoredSegment = static_cast<size_t>(std::clamp(currentGpsIndex + 1, 1,
static_cast<int>(gpsRecords.size()) - 1));
00362         for (size_t i = 1; i <= maxColoredSegment; ++i)
00363         {
00364             const RideRecord& a = *gpsRecords[i - 1];
00365             const QPointF ap = tileToScreen(gpsTilePoints[i - 1], centerTile, m_settings.width,
mapHeight);
00366             const QPointF bp = tileToScreen(gpsTilePoints[i], centerTile, m_settings.width,
mapHeight);
00367             const QColor c = routeColor(a,
                                m_settings.routeColorMetric,
                                m_settings.colorContext,
                                m_settings.singleRouteColor);
00370             p.setPen(QPen(c, 4.0, Qt::SolidLine, Qt::RoundCap, Qt::RoundJoin));
00371             p.drawLine(ap, bp);
00372         }
00373     }
00374 }
00375
00376 if (currentGpsIndex + 1 < static_cast<int>(gpsRecords.size()))
00377 {
00378     const RideRecord& a = *gpsRecords[static_cast<size_t>(currentGpsIndex)];
00379     const RideRecord& b = *gpsRecords[static_cast<size_t>(currentGpsIndex + 1)];
00380     if (sourceSeconds > a.elapsedSeconds && sourceSeconds < b.elapsedSeconds)
00381     {
00382         const double dt = b.elapsedSeconds - a.elapsedSeconds;
00383         const double alpha = dt > 1e-6
00384             ? std::clamp((sourceSeconds - a.elapsedSeconds) / dt, 0.0, 1.0)
00385             : 0.0;
00386         const QPointF ap = tileToScreen(gpsTilePoints[static_cast<size_t>(currentGpsIndex)],
                                centerTile,
                                m_settings.width,
                                mapHeight);
00387         const QPointF bp = tileToScreen(gpsTilePoints[static_cast<size_t>(currentGpsIndex +
1)],
                                centerTile,
                                m_settings.width,
                                mapHeight);
00388         const QPointF ip = ap + (bp - ap) * alpha;
00389         const QColor c = routeColor(a,
                                m_settings.routeColorMetric,
                                m_settings.colorContext,
                                m_settings.singleRouteColor);
00390         p.setPen(QPen(c, 4.0, Qt::SolidLine, Qt::RoundCap, Qt::RoundJoin));
00391         p.drawLine(ap, ip);
00392     }
00393 }
00394
00395 const QPointF currentPos = tileToScreen(latLonToTile(centerLat, centerLon, zoom),
                                centerTile,
                                m_settings.width,
                                mapHeight);
00396
00397 p.setPen(QPen(QColor("#ffffff"), 2.0));
00398 p.setBrush(QColor("#ef4444"));
00399 p.drawEllipse(currentPos, 7.0, 7.0);
00400
00401 if (showRouteLegend)
00402 {
00403     const bool hasAthleteBox = m_settings.overlayAthleteName;
00404     const int reservedLeft = telemetryMargin + telemetryWidth + telemetryMargin;
00405     const int reservedRight = hasAthleteBox
00406         ? (athleteWidth + telemetryMargin + telemetryMargin)
00407         : telemetryMargin;
00408     int legendLeft = reservedLeft;
00409     int legendRight = m_settings.width - reservedRight;
00410     int legendY = telemetryMargin;
00411
00412     if (legendRight - legendLeft < static_cast<int>(std::round(260.0 * uiScale)))
00413     {
00414         legendLeft = telemetryMargin;
00415         legendRight = m_settings.width - telemetryMargin;
00416         legendY = telemetryMargin + std::max(telemetryHeight, hasAthleteBox ? athleteHeight :
0) + legendGap;
00417     }
00418
00419     const QString legendTitle = QString("Track Color:
%1").arg(colorMetricDisplayName(m_settings.routeColorMetric));

```

```

00432     p.setFont(QFont("Helvetica", legendTitleFontPx, QFont::DemiBold));
00433     const QFontMetrics titleFm = p.fontMetrics();
00434
00435     p.setFont(QFont("Helvetica", legendLabelFontPx, QFont::Normal));
00436     const QFontMetrics labelFm = p.fontMetrics();
00437
00438     const int legendWidth = std::max(120, legendRight - legendLeft);
00439     const int contentWidth = std::max(80, legendWidth - (2 * legendInset));
00440     const int rowHeight = std::max(legendSwatchSize + 2, labelFm.height() + 2);
00441
00442     struct LegendLayoutItem
00443     {
00444         const Zone* zone = nullptr;
00445         QString label;
00446         int row = 0;
00447         int x = 0;
00448         int textWidth = 0;
00449     };
00450
00451     std::vector<LegendLayoutItem> layoutItems;
00452     layoutItems.reserve(routeLegendZones.size());
00453     int currentRow = 0;
00454     int currentX = 0;
00455     for (const Zone& zone : routeLegendZones)
00456     {
00457         const QString label = zoneLegendText(zone, m_settings.routeColorMetric);
00458         const int textW = labelFm.horizontalAdvance(label);
00459         const int itemW = legendSwatchSize + legendGap + textW + legendInset;
00460
00461         if (currentX > 0 && currentX + itemW > contentWidth)
00462         {
00463             ++currentRow;
00464             currentX = 0;
00465         }
00466
00467         layoutItems.push_back({ &zone, label, currentRow, currentX, textW });
00468         currentX += itemW;
00469     }
00470
00471     const int rows = std::max(1, currentRow + 1);
00472     const int titleHeight = titleFm.height();
00473     const int legendHeight = (2 * legendInset) + titleHeight + legendGap + (rows * rowHeight);
00474
00475     QRect legendRect(legendLeft,
00476                     legendY,
00477                     legendWidth,
00478                     legendHeight);
00479
00480     p.setPen(Qt::NoPen);
00481     p.setBrush(QColor(15, 23, 42, 170));
00482     p.drawRoundedRect(legendRect, 8.0, 8.0);
00483
00484     p.setPen(QColor("#cbd5e1"));
00485     p.setFont(QFont("Helvetica", legendTitleFontPx, QFont::DemiBold));
00486     const QRect titleRect(legendRect.left() + legendInset,
00487                          legendRect.top() + legendInset,
00488                          legendRect.width() - (2 * legendInset),
00489                          titleHeight);
00490     p.drawText(titleRect, Qt::AlignVCenter | Qt::AlignLeft, legendTitle);
00491
00492     p.setFont(QFont("Helvetica", legendLabelFontPx, QFont::Normal));
00493     for (const LegendLayoutItem& item : layoutItems)
00494     {
00495         const int rowTop = titleRect.bottom() + legendGap + (item.row * rowHeight);
00496         QRect swatchRect(legendRect.left() + legendInset + item.x,
00497                        rowTop + (rowHeight - legendSwatchSize) / 2,
00498                        legendSwatchSize,
00499                        legendSwatchSize);
00500         p.setPen(QPen(QColor(255, 255, 255, 90), 1.0));
00501         p.setBrush(item.zone->color);
00502         p.drawRoundedRect(swatchRect, 2.0, 2.0);
00503
00504         p.setPen(QColor("#f1f5f9"));
00505         p.drawText(QRect(swatchRect.right() + legendGap,
00506                        rowTop,
00507                        item.textWidth,
00508                        rowHeight),
00509                  Qt::AlignVCenter | Qt::AlignLeft,
00510                  item.label);
00511     }
00512 }
00513
00514 const bool hasTelemetryValue = m_settings.overlayTime ||
00515 (m_settings.overlayPower && now.hasPower) ||
00516 (m_settings.overlayHeartRate && now.hasHeartRate) ||
00517 (m_settings.overlayCadence && now.hasCadence) ||
00518 (m_settings.overlaySpeed && now.hasSpeed) ||

```

```

00519         (m_settings.overlayAltitude && now.hasAltitude);
00520
00521     if (m_settings.overlayTelemetryPanel && hasTelemetryValue)
00522     {
00523         QRect telemetryRect(telemetryMargin, telemetryMargin, telemetryWidth, telemetryHeight);
00524         p.setPen(Qt::NoPen);
00525         p.setBrush(QColor(15, 23, 42, 165));
00526         p.drawRoundedRect(telemetryRect, 8.0, 8.0);
00527
00528         p.setPen(QColor("#e2e8f0"));
00529         p.setFont(QFont("Helvetica", telemetryFontPx, QFont::DemiBold));
00530         int y = telemetryRect.top() + telemetryTopBaseline;
00531         auto drawLine = [&](const QString& line)
00532         {
00533             p.drawText(telemetryRect.left() + telemetryLeftPadding, y, line);
00534             y += telemetryLineStep;
00535         };
00536
00537         if (m_settings.overlayTime)
00538             drawLine(QString("Time %1").arg(formatDuration(sourceSeconds - startSec)));
00539         if (m_settings.overlayPower && now.hasPower)
00540             drawLine(QString("Power %1 W").arg(static_cast<int>(std::round(now.power))));
00541         if (m_settings.overlayHeartRate && now.hasHeartRate)
00542             drawLine(QString("HR %1 bpm").arg(static_cast<int>(std::round(now.heartRate))));
00543         if (m_settings.overlayCadence && now.hasCadence)
00544             drawLine(QString("Cad %1 rpm").arg(static_cast<int>(std::round(now.cadence))));
00545         if (m_settings.overlaySpeed && now.hasSpeed)
00546             drawLine(QString("Speed %1 km/h").arg(now.speed, 0, 'f', 1));
00547         if (m_settings.overlayAltitude && now.hasAltitude)
00548             drawLine(QString("Alt %1 m").arg(now.altitude, 0, 'f', 0));
00549     }
00550
00551     if (m_settings.overlayAthleteName)
00552     {
00553         p.setPen(Qt::NoPen);
00554         p.setBrush(QColor(15, 23, 42, 150));
00555         QRect athleteRect(m_settings.width - athleteWidth - telemetryMargin,
00556                           telemetryMargin,
00557                           athleteWidth,
00558                           athleteHeight);
00559         p.drawRoundedRect(athleteRect, 8.0, 8.0);
00560         p.setPen(QColor("#f8f9fa"));
00561         p.setFont(QFont("Helvetica", athleteFontPx, QFont::Bold));
00562         p.drawText(athleteRect.adjusted(athleteInset, 0, -athleteInset, 0), Qt::AlignVCenter |
00563                  Qt::AlignRight, m_settings.athleteName);
00564     }
00565
00566     if (showAnyMetricChart)
00567     {
00568         const QRect chartRect(0, mapHeight, m_settings.width, chartHeight);
00569         p.fillRect(chartRect, QColor("#020617"));
00570
00571         auto drawMetricChart = [&](int row,
00572                                    int rows,
00573                                    const QString& label,
00574                                    auto hasMetric,
00575                                    auto metricValue,
00576                                    const QColor& color)
00577         {
00578             const int rowHeight = chartRect.height() / std::max(1, rows);
00579             const QRect rowRect(chartRect.left() + chartSidePadding,
00580                                 chartRect.top() + row * rowHeight + chartTopPadding,
00581                                 chartRect.width() - (2 * chartSidePadding),
00582                                 rowHeight - chartBottomPadding);
00583
00584             p.setPen(QColor("#94a3b8"));
00585             p.setFont(QFont("Helvetica", chartLabelFontPx, QFont::DemiBold));
00586             p.drawText(rowRect.adjusted(0, 0, 0, -rowRect.height() + chartLabelBaseline), label);
00587
00588             const double windowStart = sourceSeconds - kChartWindowBackSeconds;
00589             const double windowEnd = sourceSeconds + kChartWindowForwardSeconds;
00590             const double visibleDataStart = std::max(startSec, windowStart);
00591             const double visibleDataEnd = std::min(endSec, windowEnd);
00592
00593             double minV = std::numeric_limits<double>::max();
00594             double maxV = std::numeric_limits<double>::lowest();
00595             std::vector<QPointF> pts;
00596             for (const RideRecord& rec : m_settings.rideData.records)
00597             {
00598                 if (rec.elapsedSeconds < visibleDataStart || rec.elapsedSeconds > visibleDataEnd)
00599                     continue;
00600                 if (!hasMetric(rec))
00601                     continue;
00602                 const double v = metricValue(rec);
00603                 minV = std::min(minV, v);
00604                 maxV = std::max(maxV, v);
00605                 pts.emplace_back(rec.elapsedSeconds, v);
00606             }
00607         };
00608     }

```

```

00605     }
00606
00607     if (pts.size() < 2)
00608         return;
00609
00610     if (std::abs(maxV - minV) < 0.001)
00611         maxV = minV + 1.0;
00612
00613     QPainterPath path;
00614     bool first = true;
00615     for (const QPointF& pt : pts)
00616     {
00617         const double nx = (pt.x() - windowStart) / std::max(0.001, windowEnd - windowStart);
00618         const double ny = (pt.y() - minV) / (maxV - minV);
00619         const QPointF sp(rowRect.left() + nx * rowRect.width(),
00620             rowRect.bottom() - ny * rowRect.height());
00621         if (first)
00622         {
00623             path.moveTo(sp);
00624             first = false;
00625         }
00626         else
00627         {
00628             path.lineTo(sp);
00629         }
00630     }
00631
00632     p.setPen(QPen(color, 1.8));
00633     p.drawPath(path);
00634
00635     const double cursorX = rowRect.left() + ((sourceSeconds - windowStart) / std::max(0.001,
00636         windowEnd - windowStart)) * rowRect.width();
00637     p.setPen(QPen(QColor("#e2e8f0"), 1.0, Qt::DashLine));
00638     p.drawLine(QPointF(cursorX, rowRect.top()), QPointF(cursorX, rowRect.bottom()));
00639     };
00640
00641     struct ChartSpec
00642     {
00643         bool enabled;
00644         QString label;
00645         std::function<bool(const RideRecord&> hasMetric;
00646             std::function<double(const RideRecord&> metricValue;
00647                 QColor color;
00648     };
00649
00650     const std::vector<ChartSpec> chartSpecs = {
00651         { m_settings.overlayPower,
00652             "Power",
00653             [](const RideRecord& r) { return r.hasPower; },
00654             [](const RideRecord& r) { return r.power; },
00655             QColor("#ef4444") },
00656         { m_settings.overlayHeartRate,
00657             "Heart Rate",
00658             [](const RideRecord& r) { return r.hasHeartRate; },
00659             [](const RideRecord& r) { return r.heartRate; },
00660             QColor("#f97316") },
00661         { m_settings.overlayCadence,
00662             "Cadence",
00663             [](const RideRecord& r) { return r.hasCadence; },
00664             [](const RideRecord& r) { return r.cadence; },
00665             QColor("#22c55e") },
00666         { m_settings.overlaySpeed,
00667             "Speed",
00668             [](const RideRecord& r) { return r.hasSpeed; },
00669             [](const RideRecord& r) { return r.speed; },
00670             QColor("#38bdf8") },
00671         { m_settings.overlayAltitude,
00672             "Altitude",
00673             [](const RideRecord& r) { return r.hasAltitude; },
00674             [](const RideRecord& r) { return r.altitude; },
00675             QColor("#c084fc") }
00676     };
00677
00678     int enabledCount = 0;
00679     for (const ChartSpec& spec : chartSpecs)
00680     {
00681         if (spec.enabled)
00682             ++enabledCount;
00683     }
00684
00685     int row = 0;
00686     for (const ChartSpec& spec : chartSpecs)
00687     {
00688         if (!spec.enabled)
00689             continue;
00690         drawMetricChart(row,
00691             enabledCount,

```



```

00691                                     spec.label,
00692                                     spec.hasMetric,
00693                                     spec.metricValue,
00694                                     spec.color);
00695                                     ++row;
00696                                     }
00697                             }
00698
00699         p.end();
00700
00701         if (ffmpeg.write(reinterpret_cast<const char*>(image.constBits()), image.sizeInBytes()) < 0)
00702         {
00703             ffmpeg.kill();
00704             ffmpeg.waitForFinished();
00705             emit finished(false, "FFmpeg pipe write failed.", false);
00706             return;
00707         }
00708
00709         const qint64 elapsedMs = timer.elapsed();
00710         const int done = frame + 1;
00711         const double avgPerFrameMs = done > 0 ? static_cast<double>(elapsedMs) /
static_cast<double>(done) : 0.0;
00712         const double remainingMs = avgPerFrameMs * static_cast<double>(totalFrames - done);
00713         const QString eta = formatDuration(remainingMs / 1000.0);
00714
00715         if (done == 1 || done % 60 == 0 || done == totalFrames)
00716         {
00717             const VideoTileStats s = tileProvider.stats();
00718             emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded %3")
00719                                     .arg(s.requiredTiles)
00720                                     .arg(s.onDiskTiles)
00721                                     .arg(s.downloadedTiles));
00722         }
00723
00724         emit progressChanged(done, totalFrames);
00725         emit frameProgressChanged(done, totalFrames, eta);
00726     }
00727
00728     ffmpeg.closeWriteChannel();
00729     if (!ffmpeg.waitForFinished(-1))
00730     {
00731         emit finished(false, "FFmpeg did not finish cleanly.", false);
00732         return;
00733     }
00734
00735     if (ffmpeg.exitStatus() != QProcess::NormalExit || ffmpeg.exitCode() != 0)
00736     {
00737         const QString err = QString::fromLocal8Bit(ffmpeg.readAllStandardOutput());
00738         emit finished(false, QString("FFmpeg failed: %1").arg(err.left(500)), false);
00739         return;
00740     }
00741
00742     emit stageChanged("Finalizing...");
00743     emit progressChanged(totalFrames, totalFrames);
00744     {
00745         const VideoTileStats s = tileProvider.stats();
00746         emit tileStatsChanged(QString("Tiles: required %1 | on-disk %2 | downloaded %3")
00747                                 .arg(s.requiredTiles)
00748                                 .arg(s.onDiskTiles)
00749                                 .arg(s.downloadedTiles));
00750
00751         emit finished(
00752             true,
00753             QString("Video exported to %1\n\nTile stats: required %2, on-disk %3, downloaded %4")
00754                 .arg(m_settings.outputPath)
00755                 .arg(s.requiredTiles)
00756                 .arg(s.onDiskTiles)
00757                 .arg(s.downloadedTiles),
00758             false);
00759     }

```

## 8.216 VideoRenderJob.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QAtomicInteger>
00007 #include <QObject>
00008
00009 #include "video/VideoExportSettings.h"
00010
00017 class VideoRenderJob : public QObject

```

```

00018 {
00019     Q_OBJECT
00020
00021 public:
00022     explicit VideoRenderJob(VideoExportSettings settings, QObject* parent = nullptr);
00028
00032     void run();
00033
00037     void cancel();
00038
00039 signals:
00042     void stageChanged(const QString& stage);
00043
00047     void progressChanged(int value, int maximum);
00048
00053     void frameProgressChanged(int currentFrame, int totalFrames, const QString& etaText);
00054
00057     void tileStatsChanged(const QString& statsText);
00058
00063     void finished(bool success, const QString& message, bool canceled);
00064
00065 private:
00066     VideoExportSettings m_settings;
00067     QAtomicInteger<int> m_cancelRequested { 0 };
00068 };

```

## 8.217 VideoTileProvider.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #include "VideoTileProvider.h"
00005
00006 #include <QDir>
00007 #include <QStandardPaths>
00008
00009 #include <algorithm>
00010
00011 namespace
00012 {
00013     QString defaultVideoExportCacheRoot()
00014     {
00015         const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00016         return appData.isEmpty()
00017             ? QDir::home().filePath(".fitlyzerc/video-export-cache")
00018             : QDir(appData).filePath("video-export-cache");
00019     }
00020
00021     QString defaultPrimaryTileCacheRoot()
00022     {
00023         const QString appData = QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
00024         return appData.isEmpty()
00025             ? QDir::home().filePath(".fitlyzerc/tiles")
00026             : QDir(appData).filePath("tiles");
00027     }
00028 }
00029
00030 VideoTileProvider::VideoTileProvider(const VideoTileProviderConfig& config)
00031 : m_tileCache(
00032     TileCacheConfig{
00033         std::max(1, config.memoryCacheTiles),
00034         config.exportCacheRoot.trimmed().isEmpty()
00035             ? defaultVideoExportCacheRoot()
00036             : config.exportCacheRoot,
00037         QStringList{ defaultPrimaryTileCacheRoot() }
00038     })
00039 , m_deleteTemporaryTilesAfterExport(config.deleteTemporaryTilesAfterExport)
00040 {
00041     m_tileCache.setMapStyle(config.style);
00042 }
00043
00044 void VideoTileProvider::setRequiredTiles(const std::unordered_set<VideoTileId, VideoTileIdHash>&
required)
00045 {
00046     m_requiredTiles = required;
00047     m_requiredTilesOnDisk = 0;
00048     m_downloadedTiles = 0;
00049
00050     for (const VideoTileId& tileId : m_requiredTiles)
00051     {
00052         if (m_tileCache.isTileCachedOnDisk(tileId.z, tileId.x, tileId.y))
00053             ++m_requiredTilesOnDisk;
00054     }
00055 }

```

```

00056
00057 int VideoTileProvider::requiredTileCount() const
00058 {
00059     return static_cast<int>(m_requiredTiles.size());
00060 }
00061
00062 int VideoTileProvider::requiredTilesAvailableOnDisk() const
00063 {
00064     return m_requiredTilesOnDisk;
00065 }
00066
00067 QPixmap VideoTileProvider::getTile(const VideoTileId& id, bool allowNetwork)
00068 {
00069     const bool cachedBefore = isTileCachedOnDisk(id);
00070     const QPixmap px = m_tileCache.tileBlocking(id.z, id.x, id.y, allowNetwork);
00071     if (allowNetwork && !cachedBefore && !px.isNull() && isTileCachedOnDisk(id))
00072         ++m_downloadedTiles;
00073     return px;
00074 }
00075
00076 VideoTileStats VideoTileProvider::stats() const
00077 {
00078     VideoTileStats s;
00079     s.requiredTiles = static_cast<int>(m_requiredTiles.size());
00080     s.onDiskTiles = m_requiredTilesOnDisk;
00081     s.downloadedTiles = m_downloadedTiles;
00082     return s;
00083 }
00084
00085 bool VideoTileProvider::isTileCachedOnDisk(const VideoTileId& id) const
00086 {
00087     return m_tileCache.isTileCachedOnDisk(id.z, id.x, id.y);
00088 }
00089
00090 void VideoTileProvider::clearMemoryCache()
00091 {
00092     m_tileCache.clearMemoryCache();
00093 }
00094
00095 void VideoTileProvider::cleanupExportCache()
00096 {
00097     if (!m_deleteTemporaryTilesAfterExport)
00098         return;
00099
00100     QDir cacheRoot(m_tileCache.diskCacheRoot());
00101     if (cacheRoot.exists())
00102         cacheRoot.removeRecursively();
00103 }
00104
00105 bool VideoTileProvider::shouldDeleteExportCacheAfterExport() const
00106 {
00107     return m_deleteTemporaryTilesAfterExport;
00108 }
00109
00110 QString VideoTileProvider::exportCacheRoot() const
00111 {
00112     return m_tileCache.diskCacheRoot();
00113 }

```

## 8.218 VideoTileProvider.h

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003
00004 #pragma once
00005
00006 #include <QString>
00007 #include <unordered_set>
00008 #include <vector>
00009
00010 #include "maps/TileCache.h"
00011
00015 struct VideoTileProviderConfig
00016 {
00018     MapStyle style = MapStyle::Light;
00019
00021     int memoryCacheTiles = 96;
00022
00024     bool deleteTemporaryTilesAfterExport = true;
00025
00027     QString exportCacheRoot;
00028 };
00029
00033 struct VideoTileId

```

```

00034 {
00036     int z = 0;
00037
00039     int x = 0;
00040
00042     int y = 0;
00043
00045     bool operator==(const VideoTileId& other) const
00046     {
00047         return z == other.z && x == other.x && y == other.y;
00048     }
00049 };
00050
00054 struct VideoTileIdHash
00055 {
00057     std::size_t operator()(const VideoTileId& id) const noexcept
00058     {
00059         const std::size_t hz = static_cast<std::size_t>(id.z);
00060         const std::size_t hx = static_cast<std::size_t>(id.x);
00061         const std::size_t hy = static_cast<std::size_t>(id.y);
00062         return (hz * 73856093u) ^ (hx * 19349663u) ^ (hy * 83492791u);
00063     }
00064 };
00065
00069 struct VideoTileStats
00070 {
00072     int requiredTiles = 0;
00073
00075     int onDiskTiles = 0;
00076
00078     int downloadedTiles = 0;
00079 };
00080
00086 class VideoTileProvider
00087 {
00088 public:
00093     explicit VideoTileProvider(const VideoTileProviderConfig& config);
00094
00099     void setRequiredTiles(const std::unordered_set<VideoTileId, VideoTileIdHash>& required);
00100
00105     int requiredTileCount() const;
00106
00111     int requiredTilesAvailableOnDisk() const;
00112
00119     QPixmap getTile(const VideoTileId& id, bool allowNetwork);
00120
00126     bool isTileCachedOnDisk(const VideoTileId& id) const;
00127
00132     VideoTileStats stats() const;
00133
00137     void clearMemoryCache();
00138
00142     void cleanupExportCache();
00143
00148     bool shouldDeleteExportCacheAfterExport() const;
00149
00154     QString exportCacheRoot() const;
00155
00156 private:
00157     TileCache m_tileCache;
00158     bool m_deleteTemporaryTilesAfterExport = true;
00159     std::unordered_set<VideoTileId, VideoTileIdHash> m_requiredTiles;
00160     int m_requiredTilesOnDisk = 0;
00161     int m_downloadedTiles = 0;
00162 };

```

## 8.219 WorkoutPersistenceService.cpp

```

00001 // SPDX-License-Identifier: GPL-3
00002
00003 #include "WorkoutPersistenceService.h"
00004
00005 #include "database/DatabaseManager.h"
00006 #include "model/RideDataSerializer.h"
00007
00008 RideData WorkoutPersistenceService::loadRide(int activityId, DatabaseManager& dbManager)
00009 {
00010     return RideDataSerializer::loadRideFromDatabase(activityId, dbManager);
00011 }
00012
00013 int WorkoutPersistenceService::saveRide(const RideData& ride,
00014                                         const WorkoutStatistics& stats,
00015                                         int athleteId,
00016                                         const QString& sourcePath,

```

```
00017         const QString& fitHash,
00018         double normalizedPower,
00019         DatabaseManager& dbManager,
00020         ActivityRepository& activityRepository,
00021         ImportRepository& importRepository,
00022         QString* errorOut)
00023 {
00024     return RideDataSerializer::saveRideToDatabase(
00025         ride,
00026         stats,
00027         athleteId,
00028         sourcePath,
00029         fitHash,
00030         normalizedPower,
00031         dbManager,
00032         activityRepository,
00033         importRepository,
00034         errorOut);
00035 }
```

## 8.220 WorkoutPersistenceService.h

```
00001 // SPDX-License-Identifier: GPL-3
00002
00003 #pragma once
00004
00005 #include <QString>
00006
00007 #include "analysis/WorkoutAnalyzer.h"
00008 #include "database/ActivityRepository.h"
00009 #include "database/ImportRepository.h"
00010 #include "fit/RideData.h"
00011
00012 class DatabaseManager;
00013
00014 class WorkoutPersistenceService
00015 {
00016 public:
00017     static RideData loadRide(int activityId, DatabaseManager& dbManager);
00018
00019     static int saveRide(const RideData& ride,
00020         const WorkoutStatistics& stats,
00021         int athleteId,
00022         const QString& sourcePath,
00023         const QString& fitHash,
00024         double normalizedPower,
00025         DatabaseManager& dbManager,
00026         ActivityRepository& activityRepository,
00027         ImportRepository& importRepository,
00028         QString* errorOut);
00029 };
```



# Index

- ~AnalysisQueue
  - AnalysisQueue, [89](#)
- ~DatabaseConnectionGuard
  - DatabaseConnectionGuard, [190](#)
- ~DatabaseManager
  - DatabaseManager, [192](#)
- ~ExportTileCleanupGuard
  - anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard, [204](#)
- ~ImportQueue
  - ImportQueue, [237](#)
- ~VideoExportDialog
  - VideoExportDialog, [389](#)
- AboutDialog, [53](#)
  - AboutDialog, [54](#)
- absoluteExecutablePath
  - anonymous\_namespace{FfmpegPath.cpp}, [34](#)
- accept
  - CreateDatabaseDialog, [188](#)
- activeCountUnsafe
  - ImportQueue, [237](#)
- activeJobs
  - ImportProgressModel, [233](#)
- Activities
  - NavigationSidebar, [326](#)
- ActivitiesPage, [54](#)
  - ActivitiesPage, [56](#)
  - m\_toolbar, [57](#)
  - toolbar, [57](#)
- Activity, [57](#)
  - analysisFlags, [59](#)
  - analysisVersion, [59](#)
  - athleteId, [59](#)
  - avgCadence, [59](#)
  - avgHr, [59](#)
  - avgPower, [59](#)
  - avgSpeed, [59](#)
  - bike, [59](#)
  - distanceM, [60](#)
  - durationSec, [60](#)
  - elevationGain, [60](#)
  - endTime, [60](#)
  - equipment, [60](#)
  - fatigue, [60](#)
  - fileName, [60](#)
  - fingerprint, [60](#)
  - fitHash, [61](#)
  - id, [61](#)
  - importedAt, [61](#)
  - importSource, [61](#)
  - isValid, [59](#)
  - maxHr, [61](#)
  - maxPower, [61](#)
  - normalizedPower, [61](#)
  - notes, [61](#)
  - rpe, [61](#)
  - sleepHr, [62](#)
  - sport, [62](#)
  - startTime, [62](#)
  - temperature, [62](#)
  - weather, [62](#)
  - weatherNotes, [62](#)
  - weightKg, [62](#)
  - wind, [62](#)
- ActivityAnalysisCacheRepository, [63](#)
  - ActivityAnalysisCacheRepository, [63](#)
  - getCache, [63](#)
  - hasCache, [64](#)
  - invalidateCache, [64](#)
  - invalidateKey, [64](#)
  - m\_db, [65](#)
  - setCache, [65](#)
- ActivityAnalysisFlags, [27](#)
  - HasClimbs, [27](#)
  - HasHistogram, [27](#)
  - HasIntervals, [27](#)
  - HasPowerCurve, [28](#)
  - HasVideo, [28](#)
  - None, [28](#)
- ActivityAnalysisSettings, [65](#)
  - activityId, [66](#)
  - climbMinGain, [66](#)
  - climbMinGradient, [66](#)
  - climbMinLength, [66](#)
  - intervalRestThreshold, [66](#)
  - intervalWorkThreshold, [67](#)
  - isValid, [66](#)
- ActivityAnalysisSettingsRepository, [67](#)
  - ActivityAnalysisSettingsRepository, [67](#)
  - insertOrUpdate, [68](#)
  - m\_db, [68](#)
  - settingsForActivity, [68](#)
- ActivityBrowser, [68](#)
  - ActivityBrowser, [70](#)
  - applyFilters, [70](#)
  - deleteActivity, [70](#)
  - dragEnterEvent, [70](#)
  - dropEvent, [70](#)

- editNotes, 70
- focusSearch, 70
- m\_athleteId, 71
- m\_dbManager, 71
- m\_emptyStateLabel, 71
- m\_rangeFilter, 71
- m\_searchEdit, 71
- m\_table, 71
- refresh, 70
- showContextMenu, 71
- activityCount
  - MainWindow, 272
- activityDateForFtp
  - anonymous\_namespace{WorkoutController.cpp}, 46
- activityEndTimeUtc
  - RideData, 363
- activityId
  - ActivityAnalysisSettings, 66
  - ClimbRecord, 168
  - ImportRecord, 241
  - IntervalRecord, 257
- activityName
  - VideoExportSettings, 395
- ActivityRepository, 73
  - ActivityRepository, 73
  - deleteActivity, 74
  - getActivity, 74
  - hasFingerprintMatch, 74
  - hasFitHash, 74
  - insertActivity, 75
  - listActivities, 75
  - m\_db, 78
  - setImportSource, 76
  - updateActivityProperties, 77
  - updateAnalysisFlags, 77
  - updateFingerprint, 77
  - updateNotes, 78
- activityStartTimeUtc
  - RideData, 363
- ActivityViewController, 79
  - ActivityViewController, 81
  - formatActivityDateLabel, 81
  - loadActivity, 81
  - m\_activityCountLabel, 87
  - m\_athleteHeader, 87
  - m\_athleteStatusLabel, 87
  - m\_controller, 87
  - m\_currentAthleteId, 87
  - m\_currentAthleteName, 87
  - m\_dbManager, 87
  - m\_dbStatusLabel, 87
  - onWorkoutLoaded, 82
  - populateActivityDetails, 83
  - refreshActivity, 83
  - setAthleteContext, 83
  - setAthleteHeaderWidget, 84
  - setStatusbarLabels, 84
  - syncAnalysisViews, 84
  - updateActivityDisplay, 85
  - updateActivityMetadata, 85
  - updateStatsDisplay, 86
  - updateStatusBarInformation, 86
- adaptiveSmoothingMaxMeters
  - ClimbDetectorConfig, 155
- adaptiveSmoothingMinMeters
  - ClimbDetectorConfig, 155
- addClimbBackgroundBands
  - RideChartWidget, 347
- addFtpEntry
  - AthleteRepository, 110
- addFtpRow
  - AthleteDialog, 103
- addIntervalBackgroundBands
  - RideChartWidget, 347
- addMetricOverlayBands
  - RideChartWidget, 347
- addPlannedWorkout
  - CalendarWidget, 120
- addRecentDatabase
  - MainWindow, 272
- addWeightEntry
  - AthleteRepository, 111
- addWeightRow
  - AthleteDialog, 103
- addZoneBackgroundBands
  - RideChartWidget, 347
- adjustRouteColorForStyle
  - MapRenderer, 314
- aerobicDecouplingPct
  - Climb, 143
- algorithmVersion
  - ClimbRecord, 168
  - IntervalRecord, 257
- altitude
  - RideRecord, 365
- altitudeMax
  - ColorContext, 184
- altitudeMin
  - ColorContext, 184
- altitudeSpikeVerticalSpeedMps
  - ClimbDetectorConfig, 155
- altitudeZones
  - ZoneCalculator, 429
- analysisFlags
  - Activity, 59
- AnalysisQueue, 87
  - ~AnalysisQueue, 89
  - AnalysisQueue, 89
  - enqueue, 90
  - isBusy, 90
  - m\_busy, 92
  - m\_dbPath, 92
  - m\_queue, 92
  - m\_thread, 92
  - m\_worker, 92



- onTaskCompleted, 90
- onTaskFailed, 90
- pendingCount, 90
- pendingCountChanged, 91
- processNext, 91
- setDatabasePath, 91
- taskCompleted, 91
- taskFailed, 92
- analysisVersion
  - Activity, 59
- AnalysisVersions, 28
  - ClimbDetection, 28
  - IntervalDetection, 28
- AnalysisWorker, 93
  - AnalysisWorker, 94
  - m\_dbPath, 96
  - processTask, 94
  - setDatabasePath, 95
  - taskCompleted, 95
  - taskFailed, 96
- analyze
  - WorkoutAnalyzer, 418
- anonymous\_namespace{ActivityBrowser.cpp}, 29
  - kIsoDateRole, 29
  - kNotesRole, 29
  - kWeatherRole, 29
- anonymous\_namespace{ActivityBrowser.cpp}::ActivityDateTableItem, 72
  - operator<, 72
- anonymous\_namespace{AnalysisWorker.cpp}, 29
  - applySqlitePragmas, 29
- anonymous\_namespace{AthleteDialog.cpp}, 29
  - displayDateOrRaw, 30
  - kEntryIdRole, 30
  - toISOString, 30
- anonymous\_namespace{ChartController.cpp}, 30
  - fmtDur, 30
  - rangeLabelForZone, 30
- anonymous\_namespace{ClimbCandidateFinder.cpp}, 30
  - segmentMeters, 30
- anonymous\_namespace{ClimbDetector.cpp}, 31
  - buildFilteredAltitudeMeters, 31
  - clampValue, 31
  - estimateSpeedKmhAt, 31
  - isFinite, 31
- anonymous\_namespace{ClimbMetrics.cpp}, 31
  - classifyClimbCategory, 32
  - classifyClimbShape, 32
  - computeLinearRegressionSlope, 32
  - computeNormalizedPower, 32
  - computeSliceStats, 32
  - isFinite, 32
- anonymous\_namespace{ClimbMetrics.cpp}::SliceStats, 367
  - avgCadence, 368
  - avgHeartRate, 368
  - avgPower, 368
- anonymous\_namespace{ClimbRepository.cpp}, 33
  - kSelectCols, 33
  - recordFromQuery, 33
- anonymous\_namespace{ColorProvider.cpp}, 33
  - interpolateColor, 33
- anonymous\_namespace{DatabaseMigrationManager.cpp}, 33
  - columnExists, 34
  - ensureActivityTimeColumns, 34
- anonymous\_namespace{DateFormatter.cpp}, 34
  - datePatternForFormat, 34
- anonymous\_namespace{FfmpegPath.cpp}, 34
  - absoluteExecutablePath, 34
- anonymous\_namespace{FitParser.cpp}, 34
  - fitTimestampToIsoUtc, 35
- anonymous\_namespace{FitParser.cpp}::FitFileIdListener, 205
  - m\_timeCreated, 206
  - OnMesg, 206
  - timeCreated, 206
- anonymous\_namespace{FitParser.cpp}::FitSessionListener, 217
  - m\_startTime, 218
  - OnMesg, 217
  - startTime, 218
- anonymous\_namespace{ImportWorker.cpp}, 35
  - applySqlitePragmas, 35
- anonymous\_namespace{MainWindowChartActions.cpp}, 35
  - kChartPresetCustom, 35
  - kChartPresetHeartRateFocus, 35
  - kChartPresetPowerAnalysis, 35
  - kChartPresetRaceReview, 35
- anonymous\_namespace{MainWindowClimbInteractions.cpp}, 36
  - fmtDur, 36
  - kClimbEndRole, 36
  - kClimbStartRole, 36
- anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem, 164
  - ClimbNameTableItem, 164
  - operator<, 165
  - trailingNumber, 165
- anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem, 178
  - ClimbSortKeyTableItem, 179
  - m\_sortKey, 179
  - operator<, 179
- anonymous\_namespace{MainWindowClimbViews.cpp}, 36
  - fmtDur, 37
  - kClimbEndRole, 37
  - kClimbStartRole, 37
- anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem, 165
  - ClimbNameTableItem, 166
  - operator<, 166
  - trailingNumber, 166

anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem, 180  
     ClimbSortKeyTableItem, 180  
     m\_sortKey, 181  
     operator<, 181  
 anonymous\_namespace{MainWindowDatabase.cpp}, 37  
     kChartPresetCustom, 38  
     kLegacyAnalysisCalendar, 38  
     kLegacyAnalysisCharts, 38  
     kLegacyAnalysisClimbing, 38  
     kLegacyAnalysisFitness, 38  
     kLegacyAnalysisHistogram, 38  
     kLegacyAnalysisPowerCurve, 38  
     kLegacyAnalysisZones, 38  
     kLegacyTabActivities, 38  
     kLegacyTabAnalysis, 38  
     kMaxRecentDatabases, 39  
     kPageActivities, 39  
     kPageCalendar, 39  
     kPageCharts, 39  
     kPageClimbs, 39  
     kPageFitness, 39  
     kPageIntervals, 39  
     kPagePower, 39  
     kPageVideo, 39  
     mapLegacyTabsToPageIndex, 38  
 anonymous\_namespace{MainWindowIntervals.cpp}, 39  
     fmtDur, 40  
     kIntervalEndRole, 40  
     kIntervalStartRole, 40  
 anonymous\_namespace{MainWindowStatus.cpp}, 40  
     formatActivityDateLabel, 40  
 anonymous\_namespace{MainWindowUiBuild.cpp}, 40  
     ChartPresetId, 41  
     kClimbEndRole, 41  
     kClimbStartRole, 41  
     wrapWithNoActivityState, 41  
 anonymous\_namespace{MainWindowVideo.cpp}, 41  
     sanitizeVideoFilePart, 41  
 anonymous\_namespace{MapFitMath.cpp}, 41  
     kMaxMercatorLat, 42  
     kMinLatSpan, 42  
     kMinLonSpan, 42  
     kPi, 42  
     kTilePx, 42  
     latToTileY, 42  
     normalizeLongitude, 42  
     tileYToLat, 42  
 anonymous\_namespace{RideChartSelectionManager.cpp}, 43  
     kClimbHandleCenterOffsetYPx, 43  
     kClimbHandleGrabRadiusPx, 43  
     kClimbHandleRadiusPx, 43  
 anonymous\_namespace{RideChartWidget.cpp}, 43  
 anonymous\_namespace{TileCache.cpp}, 43  
     defaultDiskCacheRoot, 43  
 anonymous\_namespace{VideoExportDialog.cpp}, 43  
     kKeyValTableLength, 44  
     makeDefaultFilename, 44  
     mapStyleName, 44  
 anonymous\_namespace{VideoRenderer.cpp}, 44  
     kPi, 44  
     kTilePx, 44  
 anonymous\_namespace{VideoRenderJob.cpp}, 44  
     kTilePx, 45  
     kVideoExportTileCache, 45  
     routeColor, 45  
 anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCleanupGuard, 204  
     ~ExportTileCleanupGuard, 204  
     ExportTileCleanupGuard, 204  
     m\_provider, 205  
 anonymous\_namespace{VideoTileProvider.cpp}, 45  
     defaultPrimaryTileCacheRoot, 45  
     defaultVideoExportCacheRoot, 45  
 anonymous\_namespace{WelcomeWidget.cpp}, 45  
     makeCard, 46  
     makeHeading, 46  
     makePrimaryBtn, 46  
     makeSecondaryBtn, 46  
 anonymous\_namespace{WorkoutController.cpp}, 46  
     activityDateForFtp, 46  
     resolveAthleteFtpForDate, 46  
 anonymous\_namespace{ZoneCalculator.cpp}, 47  
     sampleAvailable, 47  
     sampleDurationSeconds, 47  
     valueForMetric, 47  
 applicationDataDirectory  
     Platform, 48  
 apply  
     ClimbEditCommand, 161  
 applyChartAndMapUpdates  
     MainWindow, 272  
 applyChartControlDrivenUpdates  
     MainWindow, 273  
 applyChartHeight  
     MainWindow, 273  
 applyChartPreset  
     MainWindow, 273  
 applyChartSmoothing  
     ChartController, 125  
 applyColorMetric  
     ChartController, 125  
     MapController, 307  
 applyDetectedClimbsToCharts  
     MainWindow, 274  
 applyEstimatedFtpForCurrentAthlete  
     MainWindow, 274  
 applyFilters  
     ActivityBrowser, 70  
 applySchema  
     DatabaseManager, 192  
 applySqlitePragmas  
     anonymous\_namespace{AnalysisWorker.cpp}, 29  
     anonymous\_namespace{ImportWorker.cpp}, 35

- applyWeightMetricsToClimbs
  - MainWindow, 274
- AppSettings, 97
  - dateFormat, 97
  - dateFormatKey, 97
  - instance, 97
  - sanitizeDateFormat, 98
  - setDateFormat, 98
  - setTileCacheSize, 98
  - tileCacheSize, 98
  - tileCacheSizeKey, 98
- assignClimbWeightMetrics
  - MainWindow, 274
- Athlete, 99
  - createdAt, 100
  - dateOfBirth, 100
  - email, 100
  - firstName, 100
  - ftpWatts, 100
  - fullName, 99
  - heightCm, 100
  - id, 101
  - isValid, 100
  - lastName, 101
- athleteCombo
  - MainWindowActions, 299
- AthleteDialog, 101
  - addFtpRow, 103
  - addWeightRow, 103
  - AthleteDialog, 103
  - deleteFtpRow, 103
  - deleteWeightRow, 103
  - loadAthlete, 103
  - m\_athleteld, 104
  - m\_dob, 104
  - m\_email, 104
  - m\_firstName, 104
  - m\_ftp, 104
  - m\_ftpTable, 104
  - m\_height, 104
  - m\_lastName, 105
  - m\_repo, 105
  - m\_savedId, 105
  - m\_weightTable, 105
  - onAccepted, 103
  - populateFtpTable, 104
  - populateWeightTable, 104
  - savedAthleteld, 104
- AthleteHeaderWidget, 105
  - AthleteHeaderWidget, 106
  - m\_metricsLabel, 107
  - m\_nameLabel, 107
  - m\_statsLabel, 107
  - setRideMetrics, 106
  - setSummary, 106
- athleteld
  - Activity, 59
  - FtpEntry, 218
  - ImportJob, 231
  - WeightEntry, 414
- AthleteListDialog, 107
  - AthleteListDialog, 108
  - deleteAthlete, 108
  - editAthlete, 108
  - m\_deleteBtn, 109
  - m\_editBtn, 109
  - m\_repo, 109
  - m\_selectBtn, 109
  - m\_selectedId, 109
  - m\_table, 109
  - newAthlete, 108
  - refresh, 108
  - selectAthlete, 108
  - selectedAthleteld, 109
- athleteName
  - VideoExportSettings, 395
- AthleteRepository, 109
  - addFtpEntry, 110
  - addWeightEntry, 111
  - AthleteRepository, 110
  - deleteAthlete, 111
  - deleteFtpEntry, 111
  - deleteWeightEntry, 112
  - getAthlete, 112
  - getFtpFromDate, 112
  - getFtpHistory, 113
  - getWeightHistory, 113
  - insertAthlete, 113
  - listAthletes, 114
  - m\_db, 114
  - updateAthlete, 114
- atl
  - FitnessMetricsPoint, 209
- attachQueue
  - ImportProgressModel, 233
- attribution
  - TileCache, 370
  - TileProvider, 374
- autoRouteContrast
  - MapRenderer, 314
- autoZoom
  - VideoExportSettings, 395
- averageCadence
  - Climb, 143
  - ClimbQuarterMetrics, 167
  - Interval, 249
  - WorkoutStatistics, 426
- averageGradient
  - Climb, 143
  - ClimbRecord, 169
- averageHeartRate
  - Climb, 143
  - ClimbQuarterMetrics, 167
  - Interval, 249
  - TrainingMetricsSummary, 381
  - WorkoutStatistics, 426

- averagePower
  - Climb, [143](#)
  - ClimbQuarterMetrics, [167](#)
  - Interval, [249](#)
  - TrainingMetricsSummary, [381](#)
  - WorkoutStatistics, [426](#)
- averageSpeed
  - Climb, [143](#)
  - Interval, [249](#)
  - WorkoutStatistics, [426](#)
- averageWattsPerKg
  - Climb, [143](#)
- avgCadence
  - Activity, [59](#)
  - anonymous\_namespace{ClimbMetrics.cpp}::SliceStats, [368](#)
  - ClimbRecord, [169](#)
  - IntervalRecord, [257](#)
- avgHeartRate
  - anonymous\_namespace{ClimbMetrics.cpp}::SliceStats, [368](#)
- avgHr
  - Activity, [59](#)
  - ClimbRecord, [169](#)
  - IntervalRecord, [257](#)
- avgPower
  - Activity, [59](#)
  - anonymous\_namespace{ClimbMetrics.cpp}::SliceStats, [368](#)
  - ClimbRecord, [169](#)
  - IntervalRecord, [257](#)
- avgSpeed
  - Activity, [59](#)
- backupDatabase
  - MainWindow, [274](#)
- BasePage, [115](#)
  - BasePage, [116](#)
  - onActivated, [116](#)
- batchId
  - ImportJob, [231](#)
- bestMeanPower
  - PowerCurve, [332](#)
- bestPower
  - PowerCurvePoint, [334](#)
- bike
  - Activity, [59](#)
- bindBoolValue
  - SqlQueryHelper, [49](#)
- browseLocation
  - CreateDatabaseDialog, [188](#)
- build
  - ClimbMetrics, [163](#)
- buildColorContext
  - MainWindow, [274](#)
- buildCumulativeDistanceMeters
  - ClimbDetector, [148](#)
- buildFilteredAltitudeMeters
  - anonymous\_namespace{ClimbDetector.cpp}, [31](#)
- buildInterval
  - IntervalDetector, [251](#)
- buildMenus
  - MainWindowActions, [299](#)
- buildSampleIndices
  - RideChartWidget, [347](#)
- buildStatusBar
  - MainWindow, [274](#)
- buildToolBar
  - MainWindow, [274](#)
  - MainWindowActions, [300](#)
- buildUI
  - MainWindow, [274](#)
- cachedAxisMax
  - RideChartWidget, [358](#)
- cachedAxisMin
  - RideChartWidget, [358](#)
- cachedMaxX
  - RideChartWidget, [358](#)
- cachedMinX
  - RideChartWidget, [358](#)
- cadence
  - RideRecord, [365](#)
- cadenceZones
  - ZoneCalculator, [429](#)
- Calendar
  - NavigationSidebar, [326](#)
- CalendarPage, [116](#)
  - CalendarPage, [118](#)
- CalendarWidget, [118](#)
  - addPlannedWorkout, [120](#)
  - CalendarWidget, [120](#)
  - deleteSelectedWorkout, [120](#)
  - editSelectedWorkout, [120](#)
  - m\_addButton, [121](#)
  - m\_athleteId, [121](#)
  - m\_calendar, [121](#)
  - m\_colorByCombo, [121](#)
  - m\_dayList, [121](#)
  - m\_dbManager, [121](#)
  - m\_deleteButton, [121](#)
  - m\_editButton, [121](#)
  - onDateChanged, [120](#)
  - recolorMonth, [120](#)
  - refresh, [120](#)
  - reloadDayList, [120](#)
  - setAthleteId, [120](#)
  - setDatabaseManager, [120](#)
  - updateMonthFormatting, [120](#)
- cancel
  - VideoRenderJob, [402](#)
- cancelOrClose
  - VideoExportDialog, [389](#)
- canImportActivities
  - ImportController, [225](#)
  - MainWindow, [274](#)
- canRedo
  - UndoManager, [385](#)

- canUndo
  - UndoManager, 385
- categorizeClimb
  - ClimbDetector, 148
- category
  - Climb, 144
- centerLat
  - MapFitMath::FitResult, 216
- centerLon
  - MapFitMath::FitResult, 216
- ChartController, 121
  - applyChartSmoothing, 125
  - applyColorMetric, 125
  - ChartController, 125
  - connectChartSignals, 126
  - currentColorMetric, 126
  - handleWorkoutLoaded, 126
  - m\_activityTabStack, 136
  - m\_altitudeChart, 136
  - m\_analysisTabWidget, 137
  - m\_autoSmoothingCheck, 137
  - m\_cadenceChart, 137
  - m\_calendarTabStack, 137
  - m\_climbAltitudeChart, 137
  - m\_climbCadenceChart, 137
  - m\_climbHrChart, 137
  - m\_climbingTabStack, 137
  - m\_climbOverlayEnabledCheck, 137
  - m\_climbOverlayMetricCombo, 137
  - m\_climbPowerChart, 137
  - m\_climbs, 137
  - m\_climbSpeedChart, 138
  - m\_colorContext, 138
  - m\_colorLegendLayout, 138
  - m\_colorMetric, 138
  - m\_colorMetricCombo, 138
  - m\_controller, 138
  - m\_currentAthletId, 138
  - m\_dbManager, 138
  - m\_fitnessChart, 138
  - m\_fitnessTabStack, 138
  - m\_histogram, 138
  - m\_histogramTabStack, 138
  - m\_hrChart, 139
  - m\_intervals, 139
  - m\_powerChart, 139
  - m\_powerCurve, 139
  - m\_powerCurveTabStack, 139
  - m\_powerSmoothingCombo, 139
  - m\_smoothingAmount, 139
  - m\_smoothingEnabled, 139
  - m\_speedChart, 139
  - m\_zonesTable, 139
  - m\_zonesTabStack, 139
  - refreshCharts, 127
  - resetAllZoom, 127
  - setAnalysisTabWidgets, 128
  - setAthletId, 128
  - setChartControls, 128
  - setChartWidgets, 128
  - setClimbCharts, 129
  - setClimbingTabStack, 129
  - setClimbs, 129
  - setColorContext, 129
  - setColorMetric, 130
  - setIntervals, 130
  - setZonesTable, 130
  - syncChartCrosshair, 130
  - syncChartZoom, 131
  - updateAnalysisEmptyStates, 131
  - updateCharts, 131
  - updateColorLegend, 132
  - updateColorLegendDisplay, 133
  - updateFitnessChart, 133
  - updateHistogram, 134
  - updatePowerCurve, 134
  - updateZoneAvailability, 135
  - updateZonesTab, 136
- ChartPresetId
  - anonymous\_namespace{MainWindowUiBuild.cpp}, 41
- Charts
  - NavigationSidebar, 326
- ChartsPage, 140
  - ChartsPage, 141
- chooseOutputFile
  - VideoExportDialog, 389
- clampValue
  - anonymous\_namespace{ClimbDetector.cpp}, 31
- classifyClimbCategory
  - anonymous\_namespace{ClimbMetrics.cpp}, 32
- classifyClimbShape
  - anonymous\_namespace{ClimbMetrics.cpp}, 32
- cleanupExportCache
  - VideoTileProvider, 409
- cleanupFailedOpen
  - DatabaseManager, 192
- clear
  - UndoManager, 385
- clearBackgroundSeries
  - RideChartWidget, 347
- clearChart
  - FitnessChartWidget, 208
  - PowerCurveWidget, 335
  - PowerHistogramWidget, 338
  - RideChartWidget, 347
- clearIntervalSummary
  - MainWindow, 275
- clearMemoryCache
  - TileCache, 370
  - VideoTileProvider, 409
- Climb, 141
  - aerobicDecouplingPct, 143
  - averageCadence, 143
  - averageGradient, 143
  - averageHeartRate, 143

- averagePower, 143
- averageSpeed, 143
- averageWattsPerKg, 143
- category, 144
- climbCategory, 144
- climbScore, 144
- difficultyScore, 144
- durationSeconds, 144
- elevationGainM, 144
- endDistanceKm, 144
- endElevationM, 144
- endSeconds, 145
- gradePercentile, 145
- hrDriftPct, 145
- id, 145
- lengthKm, 145
- maximumGradient, 145
- name, 145
- normalizedPower, 145
- powerFadePct, 145
- powerFirstHalf, 146
- powerSecondHalf, 146
- quarterMetrics, 146
- shapeClass, 146
- startDistanceKm, 146
- startElevationM, 146
- startSeconds, 146
- vam, 146
- wattsPerKgRank, 147
- climbBoundaryEdited
  - RideChartWidget, 347
- climbCategory
  - Climb, 144
- climbClicked
  - RideChartWidget, 348
- ClimbDefaults, 47
  - kDipDistance, 47
  - kDipMeters, 47
  - kMinGain, 48
  - kMinGradient, 48
  - kMinLength, 48
  - kSmoothing, 48
  - kStartGrad, 48
- ClimbDetection
  - AnalysisVersions, 28
- ClimbDetector, 147
  - buildCumulativeDistanceMeters, 148
  - categorizeClimb, 148
  - computeAdaptiveSmoothingWindowsMeters, 149
  - computeClimbScore, 149
  - computeLocalGradientPct, 150
  - computeVAM, 150
  - Config, 148
  - detect, 151
  - mergeFragmentedClimbs, 152
  - smoothAltitudeByDistance, 153
- ClimbDetectorConfig, 154
  - adaptiveSmoothingMaxMeters, 155
  - adaptiveSmoothingMinMeters, 155
  - altitudeSpikeVerticalSpeedMps, 155
  - continueGradient, 155
  - durationScoreWeight, 156
  - elevationSmoothingMeters, 156
  - enableClimbMerging, 156
  - endGradient, 156
  - gainScoreWeight, 156
  - gradePersistenceMinMeters, 156
  - gradePersistenceThreshold, 156
  - gradeScoreWeight, 156
  - gradientWindowHalfMeters, 156
  - maxDipDistanceMeters, 157
  - maxDipMeters, 157
  - maxDipPctOfGain, 157
  - maxPlateauDistanceMeters, 157
  - maxSmallDipsCount, 157
  - maxStartInterruptionMeters, 157
  - mergeMaxGapElevationM, 157
  - mergeMaxGapMeters, 157
  - minAverageGradient, 157
  - minClimbScore, 158
  - minElevationGainM, 158
  - minLengthKm, 158
  - minStartContinuousDistanceMeters, 158
  - plateauMinGradient, 158
  - requireGradePersistence, 158
  - smallDipThresholdMeters, 158
  - startGradient, 158
  - useAdaptiveSmoothing, 159
  - useScoringDetection, 159
  - vamScoreBoost, 159
- ClimbEditCommand, 159
  - apply, 161
  - ClimbEditCommand, 160
  - description, 161
  - m\_after, 161
  - m\_before, 161
  - m\_connName, 161
  - m\_refresh, 162
  - redo, 161
  - undo, 161
- ClimbHandleDrag
  - RideChartWidget, 346
- ClimbMetrics, 162
  - build, 163
- climbMinGain
  - ActivityAnalysisSettings, 66
- climbMinGradient
  - ActivityAnalysisSettings, 66
- climbMinLength
  - ActivityAnalysisSettings, 66
- ClimbNameTableItem
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem, 164
  - anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem, 166
- ClimbQuarterMetrics, 167

- averageCadence, 167
- averageHeartRate, 167
- averagePower, 167
- ClimbRecord, 167
  - activityId, 168
  - algorithmVersion, 168
  - averageGradient, 169
  - avgCadence, 169
  - avgHr, 169
  - avgPower, 169
  - deleted, 169
  - elevationGainM, 169
  - endSeconds, 169
  - favorite, 169
  - id, 170
  - lengthKm, 170
  - locked, 170
  - name, 170
  - notes, 170
  - np, 170
  - originalEndSeconds, 170
  - originalStartSeconds, 170
  - rating, 170
  - source, 171
  - startSeconds, 171
  - uuid, 171
  - vam, 171
- ClimbRepository, 171
  - ClimbRepository, 172
  - climbsForActivity, 172
  - deleteClimbsForActivity, 172
  - deletedClimbsForActivity, 173
  - getClimb, 173
  - hardDeleteClimb, 174
  - hasClimbs, 174
  - hasClimbsEver, 174
  - hasLockedClimbs, 175
  - insertClimb, 176
  - m\_db, 178
  - softDeleteClimb, 177
  - updateClimb, 177
- Climbs
  - NavigationSidebar, 326
- climbs
  - WorkoutController, 420
- climbScore
  - Climb, 144
- climbsForActivity
  - ClimbRepository, 172
- ClimbSortKeyTableItem
  - anonymous\_namespace(MainWindowClimbInteractions.cpp)::ClimbSortKeyTableItem, 179
  - anonymous\_namespace(MainWindowClimbViews.cpp)::ClimbSortKeyTableItem, 180
- ClimbsPage, 181
  - ClimbsPage, 183
  - m\_toolbar, 183
- close
  - DatabaseManager, 192
  - closeEvent
    - MainWindow, 275
  - collectSettings
    - VideoExportDialog, 389
  - color
    - Zone, 428
  - ColorContext, 183
    - altitudeMax, 184
    - altitudeMin, 184
    - ftp, 184
    - hasAltitudeRange, 184
    - heartRateMax, 184
    - heartRateRest, 184
    - thresholdHeartRate, 184
  - colorContext
    - VideoExportSettings, 395
  - colorForAltitude
    - ColorProvider, 185
  - colorForCadence
    - ColorProvider, 185
  - colorForHeartRate
    - ColorProvider, 185
  - colorForMetric
    - ColorProvider, 185
  - colorForPower
    - ColorProvider, 185
  - colorForSpeed
    - ColorProvider, 186
  - colorForValue
    - ColorProvider, 186
  - ColorProvider, 185
    - colorForAltitude, 185
    - colorForCadence, 185
    - colorForHeartRate, 185
    - colorForMetric, 185
    - colorForPower, 185
    - colorForSpeed, 186
    - colorForValue, 186
  - columnExists
    - anonymous\_namespace(DatabaseMigrationManager.cpp), 34
  - Completed
    - ImportJob, 231
  - completedJobs
    - ImportProgressModel, 234
  - compute
    - FTPZones, 220
    - TrainingMetrics, 380
  - computeAdaptiveSmoothingWindowsMeters
    - ClimbSortKeyTableItem, 179
  - computeClimbScore
    - ClimbSortKeyTableItem, 180
  - computeDistribution
    - ZoneCalculator, 430
  - computeLinearRegressionSlope
    - anonymous\_namespace(ClimbMetrics.cpp), 32
  - computeLocalGradientPct



- ClimbDetector, 150
- computeNormalizedPower
  - anonymous\_namespace{ClimbMetrics.cpp}, 32
- computeSliceStats
  - anonymous\_namespace{ClimbMetrics.cpp}, 32
- computeSmoothedPowerSeries
  - RideChartWidget, 348
- computeSmoothedSeries
  - RideChartWidget, 348
- computeStandard
  - PowerCurve, 332
- computeVAM
  - ClimbDetector, 150
- Config
  - ClimbDetector, 148
  - IntervalDetector, 251
- configureFolderWatcher
  - MainWindow, 275
- connectChartSignals
  - ChartController, 126
- connectMapSignals
  - MapController, 307
- continueGradient
  - ClimbDetectorConfig, 155
- create
  - DatabaseManager, 192
- createAnalysisMenu
  - MainWindowActions, 300
- createdAt
  - Athlete, 100
- createDatabase
  - MainWindow, 275
- CreateDatabaseDialog, 186
  - accept, 188
  - browseLocation, 188
  - CreateDatabaseDialog, 187
  - databasePath, 188
  - defaultAthleteFirstName, 188
  - defaultAthleteLastName, 188
  - m\_browseButton, 188
  - m\_buttonBox, 188
  - m\_createAthleteCheck, 188
  - m\_firstNameEdit, 188
  - m\_lastNameEdit, 189
  - m\_locationEdit, 189
  - m\_nameEdit, 189
  - normalizedDatabaseName, 188
  - shouldCreateDefaultAthlete, 188
  - updateAthleteFields, 188
- createEditMenu
  - MainWindowActions, 301
- createFileMenu
  - MainWindowActions, 301
- createHelpMenu
  - MainWindowActions, 301
- createOrOpenDefaultDatabase
  - MainWindow, 275
- createToolsMenu
  - MainWindowActions, 301
- createVideo
  - MainWindow, 275
- createVideoAction
  - MainWindowActions, 302
- createViewMenu
  - MainWindowActions, 302
- crosshairMoved
  - RideChartWidget, 348
- ctl
  - FitnessMetricsPoint, 209
- currentActivityId
  - WorkoutController, 420
- currentAthleteId
  - WorkoutController, 421
- currentColorMetric
  - ChartController, 126
  - MainWindow, 275
- currentFileName
  - ImportProgressModel, 234
- currentPage
  - NavigationController, 322
  - NavigationSidebar, 326
- DailyLoadPoint, 189
  - tss, 189
- database
  - DatabaseConnectionGuard, 190
  - DatabaseManager, 193
- DatabaseConnectionGuard, 189
  - ~DatabaseConnectionGuard, 190
- database, 190
  - DatabaseConnectionGuard, 190
  - m\_connectionName, 191
- DatabaseManager, 191
  - ~DatabaseManager, 192
  - applySchema, 192
  - cleanupFailedOpen, 192
  - close, 192
  - create, 192
  - database, 193
  - isOpen, 193
  - m\_connectionName, 195
  - m\_db, 195
  - m\_path, 195
  - open, 193
  - path, 194
  - upgradeSchema, 194
- databasePath
  - CreateDatabaseDialog, 188
- date
  - PlannedWorkoutDraft, 331
- dateFormat
  - AppSettings, 97
- dateFormatKey
  - AppSettings, 97
- DateFormatter, 195
  - formatDate, 195
  - formatDateTime, 196



- parseDate, 196
  - qtDatePattern, 197
  - tolsoDate, 198
  - tolsoDateTime, 198
- dateOfBirth
  - Athlete, 100
- datePatternForFormat
  - anonymous\_namespace{DateFormatter.cpp}, 34
- decreaseChartHeight
  - MainWindow, 275
- defaultAthleteFirstName
  - CreateDatabaseDialog, 188
- defaultAthleteLastName
  - CreateDatabaseDialog, 188
- defaultBackupPath
  - Platform, 48
- defaultDatabaseDirectory
  - Platform, 48
- defaultDatabasePath
  - Platform, 49
- defaultDatabasePathForAutoCreate
  - MainWindow, 275
- defaultDiskCacheRoot
  - anonymous\_namespace{TileCache.cpp}, 43
- defaultPrimaryTileCacheRoot
  - anonymous\_namespace{VideoTileProvider.cpp}, 45
- defaultVideoExportCacheRoot
  - anonymous\_namespace{VideoTileProvider.cpp}, 45
- defaultZones
  - FTPZones, 220
- deleteActivity
  - ActivityBrowser, 70
  - ActivityRepository, 74
- deleteAthlete
  - AthleteListDialog, 108
  - AthleteRepository, 111
- deleteClimbsForActivity
  - ClimbRepository, 172
- deleted
  - ClimbRecord, 169
  - IntervalRecord, 257
- deletedClimbsForActivity
  - ClimbRepository, 173
- deleteFtpEntry
  - AthleteRepository, 111
- deleteFtpRow
  - AthleteDialog, 103
- deleteInterval
  - IntervalRepository, 260
- deleteIntervalsForActivity
  - IntervalRepository, 260
- deleteSelectedWorkout
  - CalendarWidget, 120
- deleteTemporaryTilesAfterExport
  - VideoExportSettings, 395
  - VideoTileProviderConfig, 412
- deleteWeightEntry
  - AthleteRepository, 112
- deleteWeightRow
  - AthleteDialog, 103
- description
  - ClimbEditCommand, 161
  - UndoCommand, 382
- detect
  - ClimbDetector, 151
  - IntervalDetector, 251
- detectClimbsAndRefresh
  - MainWindow, 275
- difficultyScore
  - Climb, 144
- diskCacheRoot
  - TileCache, 371
  - TileCacheConfig, 374
- diskFallbackRoots
  - TileCacheConfig, 374
- diskTilePath
  - TileCache, 371
- diskTilePathForRoot
  - TileCache, 371
- dispatchNextDownloads
  - TileCache, 371
- displayDateOrRaw
  - anonymous\_namespace{AthleteDialog.cpp}, 30
- distanceM
  - Activity, 60
- downloadedTiles
  - VideoTileStats, 413
- draft
  - PlannedWorkoutDialog, 329
- dragEnterEvent
  - ActivityBrowser, 70
  - MainWindow, 276
- DragHandle
  - MapRenderer, 314
- drawForeground
  - RideChartWidget, 349
- dropEvent
  - ActivityBrowser, 70
  - MainWindow, 276
- DuplicateActivityInfo, 199
  - existingActivityId, 199
  - existingDurationSeconds, 199
  - existingStartUtc, 199
  - newDurationSeconds, 199
  - newStartUtc, 199
- duplicates
  - MainWindow::ImportBatchSummary, 222
- durationScoreWeight
  - ClimbDetectorConfig, 156
- durationSec
  - Activity, 60
- durationSeconds
  - Climb, 144
  - Interval, 250

- PlannedWorkoutDraft, [331](#)
- PowerCurvePoint, [334](#)
- WorkoutStatistics, [426](#)
- EditActivityDialog, [199](#)
  - EditActivityDialog, [201](#)
  - m\_bikeEdit, [201](#)
  - m\_equipmentEdit, [201](#)
  - m\_fatigueSpin, [201](#)
  - m\_initial, [201](#)
  - m\_notesEdit, [201](#)
  - m\_rpeSpin, [201](#)
  - m\_sleepSpin, [201](#)
  - m\_temperatureSpin, [201](#)
  - m\_weatherEdit, [201](#)
  - m\_weightSpin, [201](#)
  - m\_windEdit, [202](#)
  - updatedActivity, [201](#)
- editAthlete
  - AthleteListDialog, [108](#)
- editCurrentActivityProperties
  - MainWindow, [276](#)
- editNotes
  - ActivityBrowser, [70](#)
- EditNotesDialog, [202](#)
  - EditNotesDialog, [203](#)
  - m\_notes, [203](#)
  - m\_weather, [203](#)
  - notes, [203](#)
  - weatherNotes, [203](#)
- editSelectedClimbBoundaries
  - MainWindow, [276](#)
- editSelectedWorkout
  - CalendarWidget, [120](#)
- effectiveFrom
  - FtpEntry, [218](#)
  - WeightEntry, [414](#)
- efficiencyFactor
  - TrainingMetricsSummary, [381](#)
  - WorkoutController, [421](#)
- elapsedSeconds
  - RideRecord, [366](#)
- elevationGain
  - Activity, [60](#)
- elevationGainM
  - Climb, [144](#)
  - ClimbRecord, [169](#)
  - WorkoutStatistics, [426](#)
- elevationSmoothingMeters
  - ClimbDetectorConfig, [156](#)
- email
  - Athlete, [100](#)
- enableClimbMerging
  - ClimbDetectorConfig, [156](#)
- endDistanceKm
  - Climb, [144](#)
- endElevationM
  - Climb, [144](#)
- endGradient
  - ClimbDetectorConfig, [156](#)
- endSample
  - IntervalRecord, [257](#)
- endSeconds
  - Climb, [145](#)
  - ClimbRecord, [169](#)
  - Interval, [250](#)
- endTime
  - Activity, [60](#)
- enqueue
  - AnalysisQueue, [90](#)
  - ImportQueue, [237](#)
- enqueueImportBatch
  - MainWindow, [276](#)
- ensureActivityTimeColumns
  - anonymous\_namespace{DatabaseMigrationManager.cpp}, [34](#)
- ensureAthlete
  - MainWindow, [277](#)
- ensureDatabase
  - MainWindow, [277](#)
- ensureImportReady
  - MainWindow, [277](#)
- equipment
  - Activity, [60](#)
- errorMessage
  - ImportJob, [231](#)
  - ImportRecord, [241](#)
- errors
  - MainWindow::ImportBatchSummary, [222](#)
- estimatedFtpFromCurrentRide
  - MainWindow, [277](#)
- estimateSpeedKmhAt
  - anonymous\_namespace{ClimbDetector.cpp}, [31](#)
- eventFilter
  - MainWindow, [277](#)
- existingActivityId
  - DuplicateActivityInfo, [199](#)
- existingDurationSeconds
  - DuplicateActivityInfo, [199](#)
- existingStartUtc
  - DuplicateActivityInfo, [199](#)
- exportCacheRoot
  - VideoTileProvider, [409](#)
  - VideoTileProviderConfig, [412](#)
- ExportTileCleanupGuard
  - anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCleanupG  
[204](#)
- Failed
  - ImportJob, [231](#)
- failed
  - ImportWorker, [247](#)
  - MainWindow::ImportBatchSummary, [222](#)
- failedJobs
  - ImportProgressModel, [234](#)
- fatigue
  - Activity, [60](#)
- favorite

- ClimbRecord, 169
- ffmpegPath
  - VideoExportSettings, 395
- fileName
  - Activity, 60
  - ImportRecord, 241
- filePath
  - ImportJob, 231
- fileTimestampUtc
  - RideData, 363
- finalizeBatches
  - ImportController, 225
- finalizeImportBatches
  - MainWindow, 277
- finalizeIntervalSelectionAndSummary
  - MainWindow, 277
- fingerprint
  - Activity, 60
- finished
  - VideoRenderJob, 402
- firstName
  - Athlete, 100
- firstRecord
  - GpsCacheData, 221
- firstTimestamp
  - FitRecordListener, 214
- fitFilesFromMimeData
  - MainWindow, 277
- fitHash
  - Activity, 61
- FitlyzerC, 1
- Fitness
  - NavigationSidebar, 326
- FitnessChartWidget, 206
  - clearChart, 208
  - FitnessChartWidget, 207
  - m\_atlSeries, 208
  - m\_axisX, 208
  - m\_axisY, 208
  - m\_ctlSeries, 208
  - m\_tsbSeries, 208
  - setTimeline, 208
- FitnessMetricsPoint, 208
  - atl, 209
  - ctl, 209
  - tsb, 209
- FitnessPage, 209
  - FitnessPage, 211
- fitnessTimeline
  - TrainingLoad, 377
- FitParser, 211
  - load, 211
- FitRecordListener, 212
  - firstTimestamp, 214
  - FitRecordListener, 214
  - lastTimestamp, 214
  - m\_firstTimestamp, 215
  - m\_lastTimestamp, 215
  - m\_rideData, 215
  - OnMesg, 214
- fitTimestampToIsoUtc
  - anonymous\_namespace{FitParser.cpp}, 35
- fitToBounds
  - MapRenderer, 314
- fitToData
  - RideChartWidget, 349
- fitToTrack
  - MapController, 307
  - MapRenderer, 314
- fitToVisibleRange
  - MapController, 308
  - MapRenderer, 314
- fixedZoom
  - VideoExportSettings, 395
- fmtDur
  - anonymous\_namespace{ChartController.cpp}, 30
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}, 36
  - anonymous\_namespace{MainWindowClimbViews.cpp}, 37
  - anonymous\_namespace{MainWindowIntervals.cpp}, 40
- focusSearch
  - ActivityBrowser, 70
- followAthlete
  - VideoExportSettings, 395
- formatActivityDateLabel
  - ActivityViewController, 81
  - anonymous\_namespace{MainWindowStatus.cpp}, 40
- formatDate
  - DateFormatter, 195
- formatDateTime
  - DateFormatter, 196
- formatVideoLength
  - anonymous\_namespace{VideoExportDialog.cpp}, 44
- fps
  - VideoExportSettings, 396
- frameProgressChanged
  - VideoRenderJob, 403
- ftp
  - ColorContext, 184
  - IntervalDetectorConfig, 253
  - WorkoutController, 421
- FtpEntry, 218
  - athleteId, 218
  - effectiveFrom, 218
  - ftpWatts, 218
  - id, 219
  - notes, 219
- ftpWatts
  - Athlete, 100
  - FtpEntry, 218
- FTPZones, 219
  - compute, 220

- defaultZones, [220](#)
- kZoneCount, [221](#)
- ZoneArray, [219](#)
- fullName
  - Athlete, [99](#)
- gainScoreWeight
  - ClimbDetectorConfig, [156](#)
- getActivity
  - ActivityRepository, [74](#)
- getAthlete
  - AthleteRepository, [112](#)
- getCache
  - ActivityAnalysisCacheRepository, [63](#)
- getClimb
  - ClimbRepository, [173](#)
- getFtpForDate
  - AthleteRepository, [112](#)
- getFtpHistory
  - AthleteRepository, [113](#)
- getRecentImports
  - ImportRepository, [242](#)
- getTile
  - VideoTileProvider, [409](#)
- getWeightHistory
  - AthleteRepository, [113](#)
- GpsCacheData, [221](#)
  - firstRecord, [221](#)
  - lastRecord, [221](#)
  - records, [221](#)
- gpsRecordAtTime
  - MapRenderer, [315](#)
- gradePercentile
  - Climb, [145](#)
- gradePersistenceMinMeters
  - ClimbDetectorConfig, [156](#)
- gradePersistenceThreshold
  - ClimbDetectorConfig, [156](#)
- gradeScoreWeight
  - ClimbDetectorConfig, [156](#)
- gradientWindowHalfMeters
  - ClimbDetectorConfig, [156](#)
- handleWorkoutLoaded
  - ChartController, [126](#)
- hardDeleteClimb
  - ClimbRepository, [174](#)
- hasAltitude
  - RideRecord, [366](#)
- hasAltitudeRange
  - ColorContext, [184](#)
- hasCache
  - ActivityAnalysisCacheRepository, [64](#)
- hasCadence
  - RideRecord, [366](#)
- HasClimbs
  - ActivityAnalysisFlags, [27](#)
- hasClimbs
  - ClimbRepository, [174](#)
- hasClimbsEver
  - ClimbRepository, [174](#)
- hasData
  - RideChartWidget, [349](#)
- hasFingerprintMatch
  - ActivityRepository, [74](#)
- hasFitHash
  - ActivityRepository, [74](#)
- hasGps
  - RideRecord, [366](#)
- hasHeartRate
  - RideRecord, [366](#)
- HasHistogram
  - ActivityAnalysisFlags, [27](#)
- HasIntervals
  - ActivityAnalysisFlags, [27](#)
- hasLockedClimbs
  - ClimbRepository, [175](#)
- hasLockedIntervals
  - IntervalRepository, [260](#)
- hasMetricSamples
  - ZoneCalculator, [431](#)
- hasPower
  - RideRecord, [366](#)
- HasPowerCurve
  - ActivityAnalysisFlags, [28](#)
- hasSpeed
  - RideRecord, [366](#)
- HasVideo
  - ActivityAnalysisFlags, [28](#)
- heartRate
  - RideRecord, [367](#)
- heartRateMax
  - ColorContext, [184](#)
- heartRateRest
  - ColorContext, [184](#)
- heartRateZones
  - ZoneCalculator, [431](#)
- height
  - VideoExportSettings, [396](#)
- heightCm
  - Athlete, [100](#)
- hideWelcomeScreen
  - MainWindow, [278](#)
- highlightSegment
  - MapController, [308](#)
- hit
  - RideChartSelectionManager::ClimbHandleHitResult, [162](#)
- hrDriftPct
  - Climb, [145](#)
- id
  - Activity, [61](#)
  - Athlete, [101](#)
  - Climb, [145](#)
  - ClimbRecord, [170](#)
  - FtpEntry, [219](#)
  - ImportRecord, [241](#)

- IntervalRecord, [258](#)
- PlannedWorkoutDraft, [331](#)
- WeightEntry, [414](#)
- importAction
  - MainWindowActions, [302](#)
- importActivities
  - MainWindow, [278](#)
- ImportController, [223](#)
  - canImportActivities, [225](#)
  - finalizeBatches, [225](#)
  - ImportController, [225](#)
  - importQueue, [226](#)
  - m\_controller, [229](#)
  - m\_dbManager, [229](#)
  - m\_importQueue, [229](#)
  - m\_progressModel, [230](#)
  - m\_statusWidget, [230](#)
  - onActivityImportComplete, [226](#)
  - progressModel, [227](#)
  - queueFilesForImport, [227](#)
  - setImportQueue, [228](#)
  - setProgressModel, [228](#)
  - setStatusWidget, [229](#)
  - updateImportAvailability, [229](#)
- imported
  - MainWindow::ImportBatchSummary, [222](#)
- importedAt
  - Activity, [61](#)
  - ImportRecord, [241](#)
- importFile
  - WorkoutController, [421](#)
- importFiles
  - MainWindow, [278](#)
- importFilesInternal
  - MainWindow, [279](#)
- ImportJob, [230](#)
  - athleteId, [231](#)
  - batchId, [231](#)
  - Completed, [231](#)
  - errorMessage, [231](#)
  - Failed, [231](#)
  - filePath, [231](#)
  - importSource, [231](#)
  - jobId, [231](#)
  - Parsing, [231](#)
  - progress, [231](#)
  - Queued, [231](#)
  - State, [231](#)
  - state, [232](#)
  - Storing, [231](#)
- ImportProgressModel, [232](#)
  - activeJobs, [233](#)
  - attachQueue, [233](#)
  - completedJobs, [234](#)
  - currentFileName, [234](#)
  - failedJobs, [234](#)
  - ImportProgressModel, [233](#)
  - m\_activeJobs, [234](#)
  - m\_completedJobs, [234](#)
  - m\_currentFileName, [235](#)
  - m\_failedJobs, [235](#)
  - m\_progress, [235](#)
  - m\_statusText, [235](#)
  - progress, [234](#)
  - statusText, [234](#)
- ImportQueue, [235](#)
  - ~ImportQueue, [237](#)
  - activeCountUnsafe, [237](#)
  - enqueue, [237](#)
  - ImportQueue, [237](#)
  - isBusy, [238](#)
  - jobFailed, [238](#)
  - m\_busy, [240](#)
  - m\_dbPath, [240](#)
  - m\_queue, [240](#)
  - m\_runningJob, [240](#)
  - m\_thread, [240](#)
  - m\_worker, [240](#)
  - onWorkerCompleted, [239](#)
  - onWorkerFailed, [239](#)
  - onWorkerProgress, [239](#)
  - pendingCount, [239](#)
  - processNext, [239](#)
  - setDatabasePath, [239](#)
- importQueue
  - ImportController, [226](#)
- importQueuedStatusMessage
  - MainWindow, [279](#)
- ImportRecord, [240](#)
  - activityId, [241](#)
  - errorMessage, [241](#)
  - fileName, [241](#)
  - id, [241](#)
  - importedAt, [241](#)
  - status, [241](#)
- ImportRepository, [241](#)
  - getRecentImports, [242](#)
  - ImportRepository, [242](#)
  - logImport, [242](#)
  - m\_db, [243](#)
- importSource
  - Activity, [61](#)
  - ImportJob, [231](#)
- ImportStatusWidget, [243](#)
  - ImportStatusWidget, [244](#)
  - m\_detailLabel, [245](#)
  - m\_model, [245](#)
  - m\_progressBar, [245](#)
  - m\_titleLabel, [245](#)
  - setModel, [244](#)
  - updateFromModel, [245](#)
- ImportWorker, [245](#)
  - failed, [247](#)
  - ImportWorker, [247](#)
  - m\_dbPath, [248](#)
  - processJob, [247](#)

- setDatabasePath, 248
- increaseChartHeight
  - MainWindow, 279
- insertActivity
  - ActivityRepository, 75
- insertAthlete
  - AthleteRepository, 113
- insertClimb
  - ClimbRepository, 176
- insertInterval
  - IntervalRepository, 261
- insertOrUpdate
  - ActivityAnalysisSettingsRepository, 68
- instance
  - AppSettings, 97
- intensityFactor
  - TrainingLoad, 377
  - TrainingMetricsSummary, 381
  - WorkoutController, 421
- interpolateColor
  - anonymous\_namespace{ColorProvider.cpp}, 33
- Interval, 249
  - averageCadence, 249
  - averageHeartRate, 249
  - averagePower, 249
  - averageSpeed, 249
  - durationSeconds, 250
  - endSeconds, 250
  - label, 250
  - maximumPower, 250
  - normalizedPower, 250
  - startSeconds, 250
- IntervalDetection
  - AnalysisVersions, 28
- IntervalDetector, 250
  - buildInterval, 251
  - Config, 251
  - detect, 251
  - removeOverlappingAndDuplicates, 252
  - smoothPower, 252
- IntervalDetectorConfig, 252
  - ftp, 253
  - minRecoverySeconds, 253
  - minWorkSeconds, 253
  - smoothingWindow, 253
  - workThresholdPct, 253
- IntervalEditorDialog, 253
  - IntervalEditorDialog, 255
  - intervalName, 255
  - intervalNotes, 255
  - intervalType, 255
  - m\_endSpin, 255
  - m\_nameEdit, 255
  - m\_notesEdit, 255
  - m\_startSpin, 256
  - m\_typeEdit, 256
- intervalName
  - IntervalEditorDialog, 255
- intervalNotes
  - IntervalEditorDialog, 255
- IntervalRecord, 256
  - activityId, 257
  - algorithmVersion, 257
  - avgCadence, 257
  - avgHr, 257
  - avgPower, 257
  - deleted, 257
  - endSample, 257
  - id, 258
  - locked, 258
  - name, 258
  - notes, 258
  - np, 258
  - source, 258
  - startSample, 258
  - type, 259
  - uuid, 259
- IntervalRepository, 259
  - deleteInterval, 260
  - deleteIntervalsForActivity, 260
  - hasLockedIntervals, 260
  - insertInterval, 261
  - IntervalRepository, 260
  - listIntervalsForActivity, 262
  - m\_db, 263
  - softDeleteInterval, 262
  - updateInterval, 262
- intervalRestThreshold
  - ActivityAnalysisSettings, 66
- Intervals
  - NavigationSidebar, 326
- intervals
  - WorkoutController, 421
- intervalSelectionRequested
  - RideChartWidget, 350
- IntervalsPage, 263
  - IntervalsPage, 264
- intervalType
  - IntervalEditorDialog, 255
- intervalWorkThreshold
  - ActivityAnalysisSettings, 67
- invalidateCache
  - ActivityAnalysisCacheRepository, 64
- invalidateKey
  - ActivityAnalysisCacheRepository, 64
- isBusy
  - AnalysisQueue, 90
  - ImportQueue, 238
- isFinite
  - anonymous\_namespace{ClimbDetector.cpp}, 31
  - anonymous\_namespace{ClimbMetrics.cpp}, 32
- isOpen
  - DatabaseManager, 193
- isTileCachedOnDisk
  - TileCache, 371
  - VideoTileProvider, 410

- isValid
  - Activity, [59](#)
  - ActivityAnalysisSettings, [66](#)
  - Athlete, [100](#)
- jobFailed
  - ImportQueue, [238](#)
- jobId
  - ImportJob, [231](#)
- joinSelectedClimbs
  - MainWindow, [279](#)
- kChartPresetCustom
  - anonymous\_namespace{MainWindowChartActions.cpp}, [35](#)
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kChartPresetHeartRateFocus
  - anonymous\_namespace{MainWindowChartActions.cpp}, [35](#)
- kChartPresetPowerAnalysis
  - anonymous\_namespace{MainWindowChartActions.cpp}, [35](#)
- kChartPresetRaceReview
  - anonymous\_namespace{MainWindowChartActions.cpp}, [35](#)
- kClimbEndRole
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}, [36](#)
  - anonymous\_namespace{MainWindowClimbViews.cpp}, [37](#)
  - anonymous\_namespace{MainWindowUiBuild.cpp}, [41](#)
- kClimbHandleCenterOffsetYPx
  - anonymous\_namespace{RideChartSelectionManager.cpp}, [43](#)
- kClimbHandleGrabRadiusPx
  - anonymous\_namespace{RideChartSelectionManager.cpp}, [43](#)
- kClimbHandleRadiusPx
  - anonymous\_namespace{RideChartSelectionManager.cpp}, [43](#)
- kClimbStartRole
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}, [36](#)
  - anonymous\_namespace{MainWindowClimbViews.cpp}, [37](#)
  - anonymous\_namespace{MainWindowUiBuild.cpp}, [41](#)
- kDipDistance
  - ClimbDefaults, [47](#)
- kDipMeters
  - ClimbDefaults, [47](#)
- kEntryIdRole
  - anonymous\_namespace{AthleteDialog.cpp}, [30](#)
- key
  - TileCache, [371](#)
- keyForStyle
  - TileCache, [371](#)
- kIntervalEndRole
  - anonymous\_namespace{MainWindowIntervals.cpp}, [40](#)
- kIntervalStartRole
  - anonymous\_namespace{MainWindowIntervals.cpp}, [40](#)
- kIsoDateRole
  - anonymous\_namespace{ActivityBrowser.cpp}, [29](#)
- kLegacyAnalysisCalendar
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisCharts
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisClimbing
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisFitness
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisHistogram
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisPowerCurve
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyAnalysisZones
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyTabActivities
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kLegacyTabAnalysis
  - anonymous\_namespace{MainWindowDatabase.cpp}, [38](#)
- kMaxConcurrentDownloads
  - TileCache, [373](#)
- kMaxMercatorLat
  - anonymous\_namespace{MapFitMath.cpp}, [42](#)
- kMaxRecentDatabases
  - anonymous\_namespace{MainWindowDatabase.cpp}, [39](#)
- kMinGain
  - ClimbDefaults, [48](#)
- kMinGradient
  - ClimbDefaults, [48](#)
- kMinLatSpan
  - anonymous\_namespace{MapFitMath.cpp}, [42](#)
- kMinLength
  - ClimbDefaults, [48](#)
- kMinLonSpan
  - anonymous\_namespace{MapFitMath.cpp}, [42](#)
- kNotesRole
  - anonymous\_namespace{ActivityBrowser.cpp}, [29](#)
- kPageActivities
  - anonymous\_namespace{MainWindowDatabase.cpp}, [39](#)
- kPageCalendar



- anonymous\_namespace{MainWindowDatabase.cpp}, 39
- kPageCharts
  - anonymous\_namespace{MainWindowDatabase.cpp},lengthKm 39
- kPageClimbs
  - anonymous\_namespace{MainWindowDatabase.cpp},listActivities 39
- kPageFitness
  - anonymous\_namespace{MainWindowDatabase.cpp}, 39
- kPageIntervals
  - anonymous\_namespace{MainWindowDatabase.cpp},load 39
- kPagePower
  - anonymous\_namespace{MainWindowDatabase.cpp}, 39
- kPageVideo
  - anonymous\_namespace{MainWindowDatabase.cpp}, 39
- kPi
  - anonymous\_namespace{MapFitMath.cpp}, 42
  - anonymous\_namespace{VideoRenderer.cpp}, 44
- kSelectCols
  - anonymous\_namespace{ClimbRepository.cpp}, 33
- kSmoothing
  - ClimbDefaults, 48
- kStartGrad
  - ClimbDefaults, 48
- kTilePx
  - anonymous\_namespace{MapFitMath.cpp}, 42
  - anonymous\_namespace{VideoRenderer.cpp}, 44
  - anonymous\_namespace{VideoRenderJob.cpp}, 45
- kVideoExportTileCache
  - anonymous\_namespace{VideoRenderJob.cpp}, 45
- kWeatherRole
  - anonymous\_namespace{ActivityBrowser.cpp}, 29
- kWhereDeleted
  - SqlQueryHelper, 51
- kWhereLocked
  - SqlQueryHelper, 51
- kWhereNotDeleted
  - SqlQueryHelper, 52
- kZoneCount
  - FTPZones, 221
- label
  - Interval, 250
- lastName
  - Athlete, 101
- lastRecord
  - GpsCacheData, 221
- lastTimestamp
  - FitRecordListener, 214
- latitude
  - RideRecord, 367
- latLonToTile
  - MapRenderer, 315
- latToTileY
  - anonymous\_namespace{MapFitMath.cpp}, 42
  - leaveEvent
  - RideChartWidget, 350
  - Climb, 145
  - ClimbRecord, 170
  - ActivityRepository, 75
  - listAthletes
  - AthleteRepository, 114
  - listIntervalsForActivity
  - IntervalRepository, 262
  - FitParser, 211
  - loadActivity
  - ActivityViewController, 81
  - WorkoutController, 421
  - loadActivitySamples
  - RideDataLoader, 364
  - loadAthlete
  - AthleteDialog, 103
  - loadFile
  - WorkoutController, 421
  - loadOrGenerateClimbs
  - WorkoutController, 421
  - loadOrGenerateIntervals
  - WorkoutController, 421
  - loadRide
  - WorkoutPersistenceService, 425
  - loadRideFromDatabase
  - RideDataSerializer, 364
  - loadSettings
  - MainWindow, 279
  - locked
  - ClimbRecord, 170
  - IntervalRecord, 258
  - logImport
  - ImportRepository, 242
  - longitude
  - RideRecord, 367
  - lonSpanDegrees
  - MapFitMath::FitResult, 216
  - lowerPct
  - PowerZone, 342
  - m\_aboutAct
  - MainWindowActions, 303
  - m\_actionsManager
  - MainWindow, 286
  - m\_activeDownloads
  - TileCache, 373
  - m\_activeJobs
  - ImportProgressModel, 234
  - m\_activitiesPage
  - MainWindow, 286
  - m\_activityBrowser
  - MainWindow, 286
  - m\_activityCountLabel
  - ActivityViewController, 87



- MainWindow, 286
- m\_activityNotesView
  - MainWindow, 286
- m\_activityTabStack
  - ChartController, 136
  - MainWindow, 286
- m\_activityViewController
  - MainWindow, 286
- m\_addButton
  - CalendarWidget, 121
- m\_after
  - ClimbEditCommand, 161
- m\_altitudeChart
  - ChartController, 136
  - MainWindow, 286
- m\_altitudeCheck
  - VideoExportDialog, 390
- m\_analysisQueue
  - MainWindow, 287
  - WorkoutController, 423
- m\_analysisTabWidget
  - ChartController, 137
  - MainWindow, 287
- m\_analyzer
  - WorkoutController, 423
- m\_athleteCheck
  - VideoExportDialog, 390
- m\_athleteCombo
  - MainWindow, 287
  - MainWindowActions, 303
- m\_athleteHeader
  - ActivityViewController, 87
  - MainWindow, 287
- m\_athleteld
  - ActivityBrowser, 71
  - AthleteDialog, 104
  - CalendarWidget, 121
- m\_athleteLabelAct
  - MainWindow, 287
- m\_athletesMenu
  - MainWindow, 287
- m\_athleteStatusLabel
  - ActivityViewController, 87
  - MainWindow, 287
- m\_atlSeries
  - FitnessChartWidget, 208
- m\_autoFitVisibleRange
  - MapRenderer, 317
- m\_autoRouteContrast
  - MapRenderer, 317
- m\_autoSmoothing
  - RideChartWidget, 358
- m\_autoSmoothingCheck
  - ChartController, 137
  - MainWindow, 287
- m\_axisX
  - FitnessChartWidget, 208
  - RideChartWidget, 358
- m\_axisXDisplay
  - RideChartWidget, 358
- m\_axisY
  - FitnessChartWidget, 208
  - RideChartWidget, 359
- m\_axisYOverlay
  - RideChartWidget, 359
- m\_backgroundSeries
  - RideChartWidget, 359
- m\_backupDatabaseAct
  - MainWindowActions, 303
- m\_before
  - ClimbEditCommand, 161
- m\_bikeEdit
  - EditActivityDialog, 201
- m\_browseButton
  - CreateDatabaseDialog, 188
- m\_busy
  - AnalysisQueue, 92
  - ImportQueue, 240
- m\_buttonBox
  - CreateDatabaseDialog, 188
- m\_cache
  - TileCache, 373
- m\_cadenceChart
  - ChartController, 137
  - MainWindow, 287
- m\_cadenceCheck
  - VideoExportDialog, 390
- m\_calendar
  - CalendarWidget, 121
- m\_calendarPageContent
  - MainWindow, 287
- m\_calendarTabStack
  - ChartController, 137
  - MainWindow, 287
- m\_calendarWidget
  - MainWindow, 287
- m\_cancelButton
  - VideoExportDialog, 390
- m\_cancelRequested
  - VideoRenderJob, 405
- m\_centerLat
  - MapRenderer, 317
- m\_centerLon
  - MapRenderer, 317
- m\_chartController
  - MainWindow, 288
- m\_chartHeight
  - MainWindow, 288
- m\_chartHeightDecreaseButton
  - MainWindow, 288
- m\_chartHeightIncreaseButton
  - MainWindow, 288
- m\_chartMapSplit
  - MainWindow, 288
- m\_chartPresetCombo
  - MainWindow, 288

- m\_chartsCheck
  - VideoExportDialog, 391
- m\_chartsContainer
  - MainWindow, 288
- m\_chartScroll
  - MainWindow, 288
- m\_chartsLayout
  - MainWindow, 288
- m\_chartsPageContent
  - MainWindow, 288
- m\_climbAltitudeChart
  - ChartController, 137
  - MainWindow, 288
- m\_climbCadenceChart
  - ChartController, 137
  - MainWindow, 288
- m\_climbDipDistanceSpin
  - MainWindow, 289
- m\_climbDipMetersSpin
  - MainWindow, 289
- m\_climbEditingEnabled
  - RideChartWidget, 359
- m\_climbHandleDrag
  - RideChartWidget, 359
- m\_climbHrChart
  - ChartController, 137
  - MainWindow, 289
- m\_climbingTabStack
  - ChartController, 137
  - MainWindow, 289
- m\_climbMinGainSpin
  - MainWindow, 289
- m\_climbMinGradientSpin
  - MainWindow, 289
- m\_climbMinLengthSpin
  - MainWindow, 289
- m\_climbOverlayEnabledCheck
  - ChartController, 137
  - MainWindow, 289
- m\_climbOverlayMetricCombo
  - ChartController, 137
  - MainWindow, 289
- m\_climbPowerChart
  - ChartController, 137
  - MainWindow, 289
- m\_climbQuarterCadenceChart
  - MainWindow, 289
- m\_climbQuarterHrChart
  - MainWindow, 289
- m\_climbQuarterPowerChart
  - MainWindow, 290
- m\_climbs
  - ChartController, 137
  - RideChartWidget, 359
  - WorkoutController, 423
- m\_climbSmoothingSpin
  - MainWindow, 290
- m\_climbsPageContent
  - MainWindow, 290
- m\_climbSpeedChart
  - ChartController, 138
  - MainWindow, 290
- m\_climbsTable
  - MainWindow, 290
- m\_climbStartGradientSpin
  - MainWindow, 290
- m\_climbSummaryLabel
  - MainWindow, 290
- m\_colorByCombo
  - CalendarWidget, 121
- m\_colorContext
  - ChartController, 138
  - MapRenderer, 317
  - RideChartWidget, 359
- m\_colorLegendLayout
  - ChartController, 138
  - MainWindow, 290
- m\_colorLegendWidget
  - MainWindow, 290
- m\_colorMetric
  - ChartController, 138
  - RideChartWidget, 359
- m\_colorMetricCombo
  - ChartController, 138
  - MainWindow, 290
- m\_colorSeries
  - RideChartWidget, 359
- m\_completedJobs
  - ImportProgressModel, 234
- m\_connectionName
  - DatabaseConnectionGuard, 191
  - DatabaseManager, 195
- m\_connName
  - ClimbEditCommand, 161
- m\_controller
  - ActivityViewController, 87
  - ChartController, 138
  - ImportController, 229
  - MainWindow, 290
  - MapController, 311
- m\_createAthleteCheck
  - CreateDatabaseDialog, 188
- m\_createDatabaseAct
  - MainWindowActions, 303
- m\_createDbButton
  - WelcomeWidget, 417
- m\_createVideoAct
  - MainWindowActions, 303
- m\_crosshairX
  - RideChartWidget, 359
- m\_crosshairY
  - RideChartWidget, 359
- m\_ctlSeries
  - FitnessChartWidget, 208
- m\_currentActivityId
  - WorkoutController, 423

- m\_currentAthleteld
  - ActivityViewController, [87](#)
  - ChartController, [138](#)
  - MainWindow, [290](#)
  - WorkoutController, [423](#)
- m\_currentAthleteName
  - ActivityViewController, [87](#)
  - MainWindow, [291](#)
- m\_currentFileName
  - ImportProgressModel, [235](#)
- m\_currentPage
  - NavigationController, [324](#)
- m\_currentTimeSeconds
  - MapRenderer, [317](#)
- m\_dataMaxY
  - RideChartWidget, [359](#)
- m\_dataMinY
  - RideChartWidget, [360](#)
- m\_dateEdit
  - PlannedWorkoutDialog, [330](#)
- m\_dayList
  - CalendarWidget, [121](#)
- m\_db
  - ActivityAnalysisCacheRepository, [65](#)
  - ActivityAnalysisSettingsRepository, [68](#)
  - ActivityRepository, [78](#)
  - AthleteRepository, [114](#)
  - ClimbRepository, [178](#)
  - DatabaseManager, [195](#)
  - ImportRepository, [243](#)
  - IntervalRepository, [263](#)
- m\_dbManager
  - ActivityBrowser, [71](#)
  - ActivityViewController, [87](#)
  - CalendarWidget, [121](#)
  - ChartController, [138](#)
  - ImportController, [229](#)
  - MainWindow, [291](#)
  - MapController, [311](#)
  - WorkoutController, [423](#)
- m\_dbPath
  - AnalysisQueue, [92](#)
  - AnalysisWorker, [96](#)
  - ImportQueue, [240](#)
  - ImportWorker, [248](#)
- m\_dbStatusLabel
  - ActivityViewController, [87](#)
  - MainWindow, [291](#)
- m\_defaults
  - VideoExportDialog, [391](#)
- m\_deleteBtn
  - AthleteListDialog, [109](#)
- m\_deleteButton
  - CalendarWidget, [121](#)
- m\_deleteTemporaryTilesAfterExport
  - VideoTileProvider, [411](#)
- m\_deleteTemporaryTilesCheck
  - VideoExportDialog, [391](#)
- m\_detailLabel
  - ImportStatusWidget, [245](#)
- m\_detectClimbsAct
  - MainWindowActions, [303](#)
- m\_detectedClimbs
  - MainWindow, [291](#)
- m\_detectIntervalsAct
  - MainWindowActions, [303](#)
- m\_diskCacheRoot
  - TileCache, [373](#)
- m\_diskFallbackRoots
  - TileCache, [373](#)
- m\_dob
  - AthleteDialog, [104](#)
- m\_downloadedTiles
  - VideoTileProvider, [411](#)
- m\_downloadQueue
  - TileCache, [373](#)
- m\_draggingSelection
  - MapRenderer, [317](#)
- m\_dragHandle
  - MapRenderer, [317](#)
- m\_dragOriginalClimbEndSec
  - RideChartWidget, [360](#)
- m\_dragOriginalClimbStartSec
  - RideChartWidget, [360](#)
- m\_durationSpin
  - PlannedWorkoutDialog, [330](#)
- m\_editBtn
  - AthleteListDialog, [109](#)
- m\_editButton
  - CalendarWidget, [121](#)
- m\_ef
  - WorkoutController, [423](#)
- m\_effectivePowerSmoothingSeconds
  - RideChartWidget, [360](#)
- m\_email
  - AthleteDialog, [104](#)
- m\_emptyStateLabel
  - ActivityBrowser, [71](#)
- m\_endMarkerRect
  - MapRenderer, [318](#)
- m\_endSpin
  - IntervalEditorDialog, [255](#)
- m\_entireActivityRadio
  - VideoExportDialog, [391](#)
- m\_equipmentEdit
  - EditActivityDialog, [201](#)
- m\_etaLabel
  - VideoExportDialog, [391](#)
- m\_exitAct
  - MainWindowActions, [303](#)
- m\_exportButton
  - VideoExportDialog, [391](#)
- m\_failedJobs
  - ImportProgressModel, [235](#)
- m\_fatigueSpin
  - EditActivityDialog, [201](#)

- m\_firstGpsRecord
  - MapRenderer, 318
- m\_firstLaunchCompleted
  - MainWindow, 291
- m\_firstName
  - AthleteDialog, 104
- m\_firstNameEdit
  - CreateDatabaseDialog, 188
- m\_firstTimestamp
  - FitRecordListener, 215
- m\_fitChartsButton
  - MainWindow, 291
- m\_fitnessChart
  - ChartController, 138
  - MainWindow, 291
- m\_fitnessPageContent
  - MainWindow, 291
- m\_fitnessTabStack
  - ChartController, 138
  - MainWindow, 291
- m\_fixedZoomSpin
  - VideoExportDialog, 391
- m\_followIntervalSelectionCheck
  - MainWindow, 291
- m\_fpsSpin
  - VideoExportDialog, 391
- m\_frameLabel
  - VideoExportDialog, 391
- m\_ftp
  - AthleteDialog, 104
  - WorkoutController, 423
- m\_ftpTable
  - AthleteDialog, 104
- m\_fullscreenAct
  - MainWindowActions, 303
- m\_gpsRecords
  - MapRenderer, 318
- m\_hasData
  - RideChartWidget, 360
- m\_hasGps
  - MapRenderer, 318
- m\_height
  - AthleteDialog, 104
- m\_highlightElapsedSeconds
  - MapRenderer, 318
- m\_histogram
  - ChartController, 138
  - MainWindow, 291
- m\_histogramPageContent
  - MainWindow, 291
- m\_histogramTabStack
  - ChartController, 138
  - MainWindow, 292
- m\_hrChart
  - ChartController, 139
  - MainWindow, 292
- m\_hrCheck
  - VideoExportDialog, 391
- m\_id
  - PlannedWorkoutDialog, 330
- m\_if
  - WorkoutController, 423
- m\_importAct
  - MainWindow, 292
  - MainWindowActions, 303
- m\_importBatchSummaries
  - MainWindow, 292
- m\_importButton
  - WelcomeWidget, 417
- m\_importController
  - MainWindow, 292
- m\_importProgressModel
  - MainWindow, 292
- m\_importQueue
  - ImportController, 229
  - MainWindow, 292
- m\_importRefreshTimer
  - MainWindow, 292
- m\_importStatusWidget
  - MainWindow, 292
- m\_initial
  - EditActivityDialog, 201
- m\_intervalEndMin
  - RideChartWidget, 360
- m\_intervals
  - ChartController, 139
  - RideChartWidget, 360
  - WorkoutController, 423
- m\_intervalSelecting
  - RideChartWidget, 360
- m\_intervalsPageContent
  - MainWindow, 292
- m\_intervalsTable
  - MainWindow, 292
- m\_intervalStartMin
  - RideChartWidget, 360
- m\_intervalSummaryLabel
  - MainWindow, 292
- m\_isRebuilding
  - RideChartWidget, 360
- m\_job
  - VideoExportDialog, 391
- m\_knownWatchedFitFiles
  - MainWindow, 293
- m\_lapsTable
  - MainWindow, 293
- m\_lastGpsRecord
  - MapRenderer, 318
- m\_lastName
  - AthleteDialog, 105
- m\_lastNameEdit
  - CreateDatabaseDialog, 189
- m\_lastOpenDir
  - MainWindow, 293
- m\_lastTimestamp
  - FitRecordListener, 215

- m\_legendCheck
  - VideoExportDialog, 391
- m\_list
  - NavigationSideBar, 327
- m\_locationEdit
  - CreateDatabaseDialog, 189
- m\_mainWindow
  - MainWindowActions, 304
- m\_manageAthletesAct
  - MainWindowActions, 304
- m\_manageAthletesButton
  - WelcomeWidget, 417
- m\_mapAutoContrastCheck
  - MainWindow, 293
- m\_mapController
  - MainWindow, 293
- m\_mapRenderer
  - MainWindow, 293
  - MapController, 311
- m\_mapStyle
  - TileCache, 373
- m\_mapStyleCombo
  - MainWindow, 293
  - VideoExportDialog, 392
- m\_maxLat
  - MapRenderer, 318
- m\_maxLon
  - MapRenderer, 318
- m\_maxStackSize
  - UndoManager, 386
- m\_metric
  - RideChartWidget, 360
- m\_metricOverlayEnabled
  - RideChartWidget, 360
- m\_metricOverlayMetric
  - RideChartWidget, 361
- m\_metricsLabel
  - AthleteHeaderWidget, 107
- m\_minLat
  - MapRenderer, 318
- m\_minLon
  - MapRenderer, 318
- m\_minZoom
  - MapRenderer, 318
- m\_model
  - ImportStatusWidget, 245
- m\_nam
  - TileCache, 373
- m\_nameEdit
  - CreateDatabaseDialog, 189
  - IntervalEditorDialog, 255
- m\_nameLabel
  - AthleteHeaderWidget, 107
- m\_navigationController
  - MainWindow, 293
- m\_navigationSideBar
  - MainWindow, 293
- m\_newClimbCreating
  - RideChartWidget, 361
- m\_newClimbCurrentSec
  - RideChartWidget, 361
- m\_newClimbDragging
  - RideChartWidget, 361
- m\_newClimbStartSec
  - RideChartWidget, 361
- m\_notes
  - EditNotesDialog, 203
- m\_notesEdit
  - EditActivityDialog, 201
  - IntervalEditorDialog, 255
  - PlannedWorkoutDialog, 330
- m\_np
  - WorkoutController, 424
- m\_openDatabaseAct
  - MainWindowActions, 304
- m\_openDbButton
  - WelcomeWidget, 417
- m\_openSettingsAct
  - MainWindowActions, 304
- m\_origMaxX
  - RideChartWidget, 361
- m\_outputEdit
  - VideoExportDialog, 392
- m\_overlaySeries
  - RideChartWidget, 361
- m\_pageStack
  - MainWindow, 293
  - NavigationController, 324
- m\_panning
  - MapRenderer, 318
- m\_panStart
  - MapRenderer, 319
- m\_path
  - DatabaseManager, 195
- m\_pdcPageContent
  - MainWindow, 293
- m\_pdcPoints
  - WorkoutController, 424
- m\_pdcWidget
  - MainWindow, 293
- m\_pending
  - TileCache, 373
- m\_powerChart
  - ChartController, 139
  - MainWindow, 294
- m\_powerCheck
  - VideoExportDialog, 392
- m\_powerCurve
  - ChartController, 139
- m\_powerCurveTabStack
  - ChartController, 139
  - MainWindow, 294
- m\_powerSmoothingCombo
  - ChartController, 139
  - MainWindow, 294
- m\_powerZones

- WorkoutController, 424
- m\_previewAct
  - MainWindow, 294
- m\_previewFitAct
  - MainWindowActions, 304
- m\_progress
  - ImportProgressModel, 235
- m\_progressBar
  - ImportStatusWidget, 245
  - VideoExportDialog, 392
- m\_progressModel
  - ImportController, 230
- m\_provider
  - anonymous\_namespace{VideoRenderJob.cpp}::ExportTileCache, 109
  - 205
  - TileCache, 373
- m\_queue
  - AnalysisQueue, 92
  - ImportQueue, 240
- m\_rangeFilter
  - ActivityBrowser, 71
- m\_rawSeries
  - RideChartWidget, 361
- m\_recentDbMenu
  - MainWindow, 294
  - MainWindowActions, 304
- m\_redoStack
  - UndoManager, 386
- m\_referenceSeries
  - RideChartWidget, 361
- m\_refresh
  - ClimbEditCommand, 162
- m\_repo
  - AthleteDialog, 105
  - AthleteListDialog, 109
- m\_requestedPowerSmoothingSeconds
  - RideChartWidget, 361
- m\_requiredTiles
  - VideoTileProvider, 411
- m\_requiredTilesOnDisk
  - VideoTileProvider, 412
- m\_resolutionCombo
  - VideoExportDialog, 392
- m\_revertClimbAct
  - MainWindowActions, 304
- m\_rideData
  - FitRecordListener, 215
  - MapRenderer, 319
  - RideChartWidget, 361
  - WorkoutController, 424
- m\_routeColorCombo
  - VideoExportDialog, 392
- m\_routeColorMode
  - MapRenderer, 319
- m\_rpeSpin
  - EditActivityDialog, 201
- m\_running
  - VideoExportDialog, 392
- m\_runningJob
  - ImportQueue, 240
- m\_savedId
  - AthleteDialog, 105
- m\_searchEdit
  - ActivityBrowser, 71
- m\_segmentColors
  - MapRenderer, 319
- m\_selectBtn
  - AthleteListDialog, 109
- m\_selectedClimbIndex
  - RideChartWidget, 361
- m\_selectedId
  - AthleteListDialog, 109
- m\_selectedSegmentRadio
  - VideoExportDialog, 392
- m\_series
  - RideChartWidget, 362
- m\_settings
  - VideoRenderJob, 405
- m\_showAltitude
  - MainWindow, 294
- m\_showCadence
  - MainWindow, 294
- m\_showHR
  - MainWindow, 294
- m\_showPower
  - MainWindow, 294
- m\_showSpeed
  - MainWindow, 294
- m\_sidebar
  - NavigationController, 324
- m\_sleepSpin
  - EditActivityDialog, 201
- m\_smoothedSeriesCache
  - RideChartWidget, 362
- m\_smoothingAmount
  - ChartController, 139
- m\_smoothingEnabled
  - ChartController, 139
- m\_sortKey
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKey, 179
  - anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKey, 181
- m\_speedChart
  - ChartController, 139
  - MainWindow, 294
- m\_speedCheck
  - VideoExportDialog, 392
- m\_speedSpin
  - VideoExportDialog, 392
- m\_stageLabel
  - VideoExportDialog, 392
- m\_startMarkerRect
  - MapRenderer, 319
- m\_startSpin
  - IntervalEditorDialog, 256

- m\_startTime
  - anonymous\_namespace{FitParser.cpp}::FitSessionLister, 218
- m\_statistics
  - WorkoutController, 424
- m\_statsLabel
  - AthleteHeaderWidget, 107
- m\_statusText
  - ImportProgressModel, 235
- m\_statusWidget
  - ImportController, 230
- m\_subtitleLabel
  - WelcomeWidget, 417
- m\_suppressClimbAutoZoom
  - MainWindow, 294
- m\_syncing
  - RideChartWidget, 362
- m\_table
  - ActivityBrowser, 71
  - AthleteListDialog, 109
- m\_telemetryPanelCheck
  - VideoExportDialog, 392
- m\_temperatureSpin
  - EditActivityDialog, 201
- m\_thread
  - AnalysisQueue, 92
  - ImportQueue, 240
  - VideoExportDialog, 393
- m\_tileCache
  - MapRenderer, 319
  - VideoTileProvider, 412
- m\_tileRepaintTimer
  - MapRenderer, 319
- m\_tileStatsLabel
  - VideoExportDialog, 393
- m\_timeCheck
  - VideoExportDialog, 393
- m\_timeCreated
  - anonymous\_namespace{FitParser.cpp}::FitFileIdListener, 206
- m\_titleEdit
  - PlannedWorkoutDialog, 330
- m\_titleLabel
  - ImportStatusWidget, 245
- m\_toolbar
  - ActivitiesPage, 57
  - ClimbsPage, 183
- m\_toolbarImportAct
  - MainWindow, 295
  - MainWindowActions, 304
- m\_toolbarSearchAct
  - MainWindow, 295
  - MainWindowActions, 304
- m\_tooltipDisplayValues
  - RideChartWidget, 362
- m\_tooltipPoints
  - RideChartWidget, 362
- m\_tooltipRawValues
  - RideChartWidget, 362
- IntervalSeries
  - FitnessChartWidget, 208
- m\_tss
  - WorkoutController, 424
- m\_tssSpin
  - PlannedWorkoutDialog, 330
- m\_typeEdit
  - IntervalEditorDialog, 256
  - PlannedWorkoutDialog, 330
- m\_undoManager
  - MainWindow, 295
- m\_undoStack
  - UndoManager, 386
- m\_useEstimatedFtpButton
  - MainWindow, 295
- m\_userMovedMap
  - MapRenderer, 319
- m\_vi
  - WorkoutController, 424
- m\_videoLengthLabel
  - VideoExportDialog, 393
- m\_visibleEndSeconds
  - MapRenderer, 319
- m\_visibleStartSeconds
  - MapRenderer, 319
- m\_watcher
  - MainWindow, 295
- m\_watchFolderEnabled
  - MainWindow, 295
- m\_watchFolderPath
  - MainWindow, 295
- m\_watchRescanTimer
  - MainWindow, 295
- m\_weather
  - EditNotesDialog, 203
- m\_weatherEdit
  - EditActivityDialog, 201
- m\_weightSpin
  - EditActivityDialog, 201
- m\_weightTable
  - AthleteDialog, 105
- m\_welcomeWidget
  - MainWindow, 295
- m\_windEdit
  - EditActivityDialog, 202
- m\_worker
  - AnalysisQueue, 92
  - ImportQueue, 240
- m\_zonesPageContent
  - MainWindow, 295
- m\_zonesTable
  - ChartController, 139
  - MainWindow, 295
- m\_zonesTabStack
  - ChartController, 139
  - MainWindow, 295
- m\_zoom



- MapRenderer, 319
- MainWindow, 265
  - activityCount, 272
  - addRecentDatabase, 272
  - applyChartAndMapUpdates, 272
  - applyChartControlDrivenUpdates, 273
  - applyChartHeight, 273
  - applyChartPreset, 273
  - applyDetectedClimbsToCharts, 274
  - applyEstimatedFtpForCurrentAthlete, 274
  - applyWeightMetricsToClimbs, 274
  - assignClimbWeightMetrics, 274
  - backupDatabase, 274
  - buildColorContext, 274
  - buildStatusBar, 274
  - buildToolBar, 274
  - buildUI, 274
  - canImportActivities, 274
  - clearIntervalSummary, 275
  - closeEvent, 275
  - configureFolderWatcher, 275
  - createDatabase, 275
  - createOrOpenDefaultDatabase, 275
  - createVideo, 275
  - currentColorMetric, 275
  - decreaseChartHeight, 275
  - defaultDatabasePathForAutoCreate, 275
  - detectClimbsAndRefresh, 275
  - dragEnterEvent, 276
  - dropEvent, 276
  - editCurrentActivityProperties, 276
  - editSelectedClimbBoundaries, 276
  - enqueueImportBatch, 276
  - ensureAthlete, 277
  - ensureDatabase, 277
  - ensureImportReady, 277
  - estimatedFtpFromCurrentRide, 277
  - eventFilter, 277
  - finalizeImportBatches, 277
  - finalizeIntervalSelectionAndSummary, 277
  - fitFilesFromMimeData, 277
  - hideWelcomeScreen, 278
  - importActivities, 278
  - importFiles, 278
  - importFilesInternal, 279
  - importQueuedStatusMessage, 279
  - increaseChartHeight, 279
  - joinSelectedClimbs, 279
  - loadSettings, 279
  - m\_actionsManager, 286
  - m\_activitiesPage, 286
  - m\_activityBrowser, 286
  - m\_activityCountLabel, 286
  - m\_activityNotesView, 286
  - m\_activityTabStack, 286
  - m\_activityViewController, 286
  - m\_altitudeChart, 286
  - m\_analysisQueue, 287
  - m\_analysisTabWidget, 287
  - m\_athleteCombo, 287
  - m\_athleteHeader, 287
  - m\_athleteLabelAct, 287
  - m\_athletesMenu, 287
  - m\_athleteStatusLabel, 287
  - m\_autoSmoothingCheck, 287
  - m\_cadenceChart, 287
  - m\_calendarPageContent, 287
  - m\_calendarTabStack, 287
  - m\_calendarWidget, 287
  - m\_chartController, 288
  - m\_chartHeight, 288
  - m\_chartHeightDecreaseButton, 288
  - m\_chartHeightIncreaseButton, 288
  - m\_chartMapSplit, 288
  - m\_chartPresetCombo, 288
  - m\_chartsContainer, 288
  - m\_chartScroll, 288
  - m\_chartsLayout, 288
  - m\_chartsPageContent, 288
  - m\_climbAltitudeChart, 288
  - m\_climbCadenceChart, 288
  - m\_climbDipDistanceSpin, 289
  - m\_climbDipMetersSpin, 289
  - m\_climbHrChart, 289
  - m\_climbingTabStack, 289
  - m\_climbMinGainSpin, 289
  - m\_climbMinGradientSpin, 289
  - m\_climbMinLengthSpin, 289
  - m\_climbOverlayEnabledCheck, 289
  - m\_climbOverlayMetricCombo, 289
  - m\_climbPowerChart, 289
  - m\_climbQuarterCadenceChart, 289
  - m\_climbQuarterHrChart, 289
  - m\_climbQuarterPowerChart, 290
  - m\_climbSmoothingSpin, 290
  - m\_climbsPageContent, 290
  - m\_climbSpeedChart, 290
  - m\_climbsTable, 290
  - m\_climbStartGradientSpin, 290
  - m\_climbSummaryLabel, 290
  - m\_colorLegendLayout, 290
  - m\_colorLegendWidget, 290
  - m\_colorMetricCombo, 290
  - m\_controller, 290
  - m\_currentAthleteId, 290
  - m\_currentAthleteName, 291
  - m\_dbManager, 291
  - m\_dbStatusLabel, 291
  - m\_detectedClimbs, 291
  - m\_firstLaunchCompleted, 291
  - m\_fitChartsButton, 291
  - m\_fitnessChart, 291
  - m\_fitnessPageContent, 291
  - m\_fitnessTabStack, 291
  - m\_followIntervalSelectionCheck, 291
  - m\_histogram, 291



- [m\\_histogramPageContent](#), 291
- [m\\_histogramTabStack](#), 292
- [m\\_hrChart](#), 292
- [m\\_importAct](#), 292
- [m\\_importBatchSummaries](#), 292
- [m\\_importController](#), 292
- [m\\_importProgressModel](#), 292
- [m\\_importQueue](#), 292
- [m\\_importRefreshTimer](#), 292
- [m\\_importStatusWidget](#), 292
- [m\\_intervalsPageContent](#), 292
- [m\\_intervalsTable](#), 292
- [m\\_intervalSummaryLabel](#), 292
- [m\\_knownWatchedFitFiles](#), 293
- [m\\_lapsTable](#), 293
- [m\\_lastOpenDir](#), 293
- [m\\_mapAutoContrastCheck](#), 293
- [m\\_mapController](#), 293
- [m\\_mapRenderer](#), 293
- [m\\_mapStyleCombo](#), 293
- [m\\_navigationController](#), 293
- [m\\_navigationSidebar](#), 293
- [m\\_pageStack](#), 293
- [m\\_pdcPageContent](#), 293
- [m\\_pdcWidget](#), 293
- [m\\_powerChart](#), 294
- [m\\_powerCurveTabStack](#), 294
- [m\\_powerSmoothingCombo](#), 294
- [m\\_previewAct](#), 294
- [m\\_recentDbMenu](#), 294
- [m\\_showAltitude](#), 294
- [m\\_showCadence](#), 294
- [m\\_showHR](#), 294
- [m\\_showPower](#), 294
- [m\\_showSpeed](#), 294
- [m\\_speedChart](#), 294
- [m\\_suppressClimbAutoZoom](#), 294
- [m\\_toolbarImportAct](#), 295
- [m\\_toolbarSearchAct](#), 295
- [m\\_undoManager](#), 295
- [m\\_useEstimatedFtpButton](#), 295
- [m\\_watcher](#), 295
- [m\\_watchFolderEnabled](#), 295
- [m\\_watchFolderPath](#), 295
- [m\\_watchRescanTimer](#), 295
- [m\\_welcomeWidget](#), 295
- [m\\_zonesPageContent](#), 295
- [m\\_zonesTable](#), 295
- [m\\_zonesTabStack](#), 295
- [MainWindow](#), 271
- [manageAthletes](#), 280
- [monitoredDirectories](#), 280
- [navigateToInterval](#), 280
- [onActivityImported](#), 280
- [onAthleteSelectionChanged](#), 280
- [onClimbBoundaryEdited](#), 280
- [onClimbRowDoubleClicked](#), 280
- [onClimbSelectionChanged](#), 280
- [onClimbTableContextMenuRequested](#), 280
- [onIntervalRowDoubleClicked](#), 280
- [onIntervalSelectionChanged](#), 281
- [onNewClimbRequested](#), 281
- [onWatchDirectoryChanged](#), 281
- [onWorkoutLoaded](#), 281
- [openDatabase](#), 281
- [openDatabasePath](#), 281
- [openRecentDatabase](#), 281
- [openSettingsDialog](#), 281
- [prepareImportFileBatch](#), 281
- [previewFitFile](#), 281
- [refreshAthleteSelector](#), 281
- [refreshClimbViews](#), 282
- [refreshMapRideDataFromCurrentState](#), 282
- [removeSelectedClimbs](#), 282
- [resetAllZoom](#), 282
- [resolveActiveActivityWeightKg](#), 282
- [restoreClimbSelection](#), 282
- [revertSelectedClimbToAuto](#), 282
- [saveSettings](#), 282
- [scanWatchDirectories](#), 283
- [scheduleActivityBrowserRefresh](#), 283
- [selectedClimbIndicesFromTable](#), 283
- [selectIntervalRowOffset](#), 283
- [setupShortcuts](#), 283
- [showAbout](#), 283
- [showWelcomeScreen](#), 283
- [syncAthleteContextToControllers](#), 283
- [syncChartContextToController](#), 283
- [triggerDetectClimbs](#), 284
- [triggerDetectIntervals](#), 284
- [updateActivityNotesFromCurrentActivity](#), 284
- [updateAthleteLabel](#), 284
- [updateChartAnalysisEmptyStates](#), 284
- [updateClimbingTab](#), 284
- [updateClimbQuarterCharts](#), 284
- [updateClimbRowStyles](#), 285
- [updateImportAvailability](#), 285
- [updateIntervalRowStyles](#), 285
- [updateIntervals](#), 285
- [updateIntervalSummary](#), 285
- [updateLapsSummaryFromCurrentActivity](#), 285
- [updateRecentDatabaseMenu](#), 286
- [updateStatsLabel](#), 286
- [updateStatusBarInfo](#), 286
- [updateWelcomeScreenVisibility](#), 286
- [MainWindow::ImportBatchSummary](#), 222
  - [duplicates](#), 222
  - [errors](#), 222
  - [failed](#), 222
  - [imported](#), 222
  - [queued](#), 222
  - [showResultDialog](#), 222
  - [sourceLabel](#), 222
- [MainWindowActions](#), 296
  - [athleteCombo](#), 299
  - [buildMenus](#), 299

- buildToolBar, 300
- createAnalysisMenu, 300
- createEditMenu, 301
- createFileMenu, 301
- createHelpMenu, 301
- createToolsMenu, 301
- createVideoAction, 302
- createViewMenu, 302
- importAction, 302
- m\_aboutAct, 303
- m\_athleteCombo, 303
- m\_backupDatabaseAct, 303
- m\_createDatabaseAct, 303
- m\_createVideoAct, 303
- m\_detectClimbsAct, 303
- m\_detectIntervalsAct, 303
- m\_exitAct, 303
- m\_fullscreenAct, 303
- m\_importAct, 303
- m\_mainWindow, 304
- m\_manageAthletesAct, 304
- m\_openDatabaseAct, 304
- m\_openSettingsAct, 304
- m\_previewFitAct, 304
- m\_recentDbMenu, 304
- m\_revertClimbAct, 304
- m\_toolbarImportAct, 304
- m\_toolbarSearchAct, 304
- MainWindowActions, 299
- previewFitAction, 302
- recentDatabaseMenu, 302
- setupShortcuts, 302
- toolbarImportAction, 303
- makeCard
  - anonymous\_namespace{WelcomeWidget.cpp}, 46
- makeDefaultFilename
  - anonymous\_namespace{VideoExportDialog.cpp}, 44
- makeHeading
  - anonymous\_namespace{WelcomeWidget.cpp}, 46
- makePrimaryBtn
  - anonymous\_namespace{WelcomeWidget.cpp}, 46
- makeSecondaryBtn
  - anonymous\_namespace{WelcomeWidget.cpp}, 46
- manageAthletes
  - MainWindow, 280
- MapController, 304
  - applyColorMetric, 307
  - connectMapSignals, 307
  - fitToTrack, 307
  - fitToVisibleRange, 308
  - highlightSegment, 308
  - m\_controller, 311
  - m\_dbManager, 311
  - m\_mapRenderer, 311
  - MapController, 307
  - mapRenderer, 308
  - refreshMap, 308
  - setAutoRouteContrast, 309
  - setCurrentTime, 309
  - setMapRenderer, 309
  - setVisibleTimeRange, 310
  - updateMap, 310
  - updateRouteColoring, 310
  - zoomIn, 311
  - zoomOut, 311
- MapFitMath::FitResult, 215
  - centerLat, 216
  - centerLon, 216
  - lonSpanDegrees, 216
  - maxLat, 216
  - maxLon, 216
  - minLat, 216
  - minLon, 216
  - zoom, 216
- mapLegacyTabsToPageIndex
  - anonymous\_namespace{MainWindowDatabase.cpp}, 38
- MapRenderer, 311
  - adjustRouteColorForStyle, 314
  - autoRouteContrast, 314
  - DragHandle, 314
  - fitToBounds, 314
  - fitToTrack, 314
  - fitToVisibleRange, 314
  - gpsRecordAtTime, 315
  - latLonToTile, 315
  - m\_autoFitVisibleRange, 317
  - m\_autoRouteContrast, 317
  - m\_centerLat, 317
  - m\_centerLon, 317
  - m\_colorContext, 317
  - m\_currentTimeSeconds, 317
  - m\_draggingSelection, 317
  - m\_dragHandle, 317
  - m\_endMarkerRect, 318
  - m\_firstGpsRecord, 318
  - m\_gpsRecords, 318
  - m\_hasGps, 318
  - m\_highlightElapsedSeconds, 318
  - m\_lastGpsRecord, 318
  - m\_maxLat, 318
  - m\_maxLon, 318
  - m\_minLat, 318
  - m\_minLon, 318
  - m\_minZoom, 318
  - m\_panning, 318
  - m\_panStart, 319
  - m\_rideData, 319
  - m\_routeColorMode, 319
  - m\_segmentColors, 319
  - m\_startMarkerRect, 319
  - m\_tileCache, 319
  - m\_tileRepaintTimer, 319
  - m\_userMovedMap, 319
  - m\_visibleEndSeconds, 319

- m\_visibleStartSeconds, 319
- m\_zoom, 319
- MapRenderer, 314
- mapStyle, 315
- mouseDoubleClickEvent, 315
- mouseMoveEvent, 315
- mousePressEvent, 315
- mouseReleaseEvent, 315
- nearestGpsRecord, 315
- paintEvent, 315
- rebuildGpsCache, 315
- rebuildSegmentColors, 316
- setAutoRouteContrast, 316
- setCurrentTime, 316
- setHighlightedElapsedSeconds, 316
- setMapStyle, 316
- setRideData, 316
- setRouteColorMode, 316
- setTileCacheSize, 316
- setVisibleTimeRange, 316
- tileToScreen, 316
- wheelEvent, 317
- zoomIn, 317
- zoomOut, 317
- mapRenderer
  - MapController, 308
- mapStyle
  - MapRenderer, 315
  - TileCache, 372
  - VideoExportSettings, 396
- mapStyleName
  - anonymous\_namespace{VideoExportDialog.cpp}, 44
  - TileCache, 372
- maxDipDistanceMeters
  - ClimbDetectorConfig, 157
- maxDipMeters
  - ClimbDetectorConfig, 157
- maxDipPctOfGain
  - ClimbDetectorConfig, 157
- maxHr
  - Activity, 61
- maximumCadence
  - WorkoutStatistics, 426
- maximumGradient
  - Climb, 145
- maximumHeartRate
  - WorkoutStatistics, 426
- maximumPower
  - Interval, 250
  - WorkoutStatistics, 427
- maximumSpeed
  - WorkoutStatistics, 427
- maxLat
  - MapFitMath::FitResult, 216
- maxLon
  - MapFitMath::FitResult, 216
- maxPlateauDistanceMeters
  - ClimbDetectorConfig, 157
- maxPower
  - Activity, 61
- maxSmallDipsCount
  - ClimbDetectorConfig, 157
- maxStartInterruptionMeters
  - ClimbDetectorConfig, 157
- maxTilesInMemory
  - TileCacheConfig, 374
- maxValue
  - Zone, 428
- maxZoom
  - TileCache, 372
  - TileProvider, 374
- memoryCacheTiles
  - VideoTileProviderConfig, 412
- mergeFragmentedClimbs
  - ClimbDetector, 152
- mergeMaxGapElevationM
  - ClimbDetectorConfig, 157
- mergeMaxGapMeters
  - ClimbDetectorConfig, 157
- minAverageGradient
  - ClimbDetectorConfig, 157
- minClimbScore
  - ClimbDetectorConfig, 158
- minElevationGainM
  - ClimbDetectorConfig, 158
- minLat
  - MapFitMath::FitResult, 216
- minLengthKm
  - ClimbDetectorConfig, 158
- minLon
  - MapFitMath::FitResult, 216
- minRecoverySeconds
  - IntervalDetectorConfig, 253
- minStartContinuousDistanceMeters
  - ClimbDetectorConfig, 158
- minValue
  - Zone, 428
- minWorkSeconds
  - IntervalDetectorConfig, 253
- monitoredDirectories
  - MainWindow, 280
- mouseDoubleClickEvent
  - MapRenderer, 315
  - RideChartWidget, 350
- mouseMoveEvent
  - MapRenderer, 315
  - RideChartWidget, 351
- mousePressEvent
  - MapRenderer, 315
  - RideChartWidget, 351
- mouseReleaseEvent
  - MapRenderer, 315
  - RideChartWidget, 351
- movingTimeSeconds
  - WorkoutStatistics, 427

- name
  - Climb, [145](#)
  - ClimbRecord, [170](#)
  - IntervalRecord, [258](#)
  - PowerZone, [342](#)
  - TileProvider, [374](#)
  - Zone, [428](#)
- navigateTo
  - NavigationController, [322](#)
- navigateToInterval
  - MainWindow, [280](#)
- NavigationController, [320](#)
  - currentPage, [322](#)
  - m\_currentPage, [324](#)
  - m\_pageStack, [324](#)
  - m\_sidebar, [324](#)
  - navigateTo, [322](#)
  - NavigationController, [321](#)
  - Page, [321](#)
  - restoreNavigationState, [323](#)
  - saveNavigationState, [323](#)
  - setNavigationSidebar, [323](#)
  - setPageStack, [324](#)
- NavigationSidebar, [325](#)
  - Activities, [326](#)
  - Calendar, [326](#)
  - Charts, [326](#)
  - Climbs, [326](#)
  - currentPage, [326](#)
  - Fitness, [326](#)
  - Intervals, [326](#)
  - m\_list, [327](#)
  - NavigationSidebar, [326](#)
  - Page, [326](#)
  - Power, [326](#)
  - setCurrentPage, [326](#)
  - Video, [326](#)
- nearestGpsRecord
  - MapRenderer, [315](#)
- newAthlete
  - AthleteListDialog, [108](#)
- newClimbRequested
  - RideChartWidget, [352](#)
- newDurationSeconds
  - DuplicateActivityInfo, [199](#)
- newStartUtc
  - DuplicateActivityInfo, [199](#)
- None
  - ActivityAnalysisFlags, [28](#)
- normalizedDatabaseName
  - CreateDatabaseDialog, [188](#)
- normalizedPower
  - Activity, [61](#)
  - Climb, [145](#)
  - Interval, [250](#)
  - TrainingLoad, [378](#)
  - TrainingMetricsSummary, [381](#)
  - WorkoutController, [422](#)
- normalizeLongitude
  - anonymous\_namespace{MapFitMath.cpp}, [42](#)
- notes
  - Activity, [61](#)
  - ClimbRecord, [170](#)
  - EditNotesDialog, [203](#)
  - FtpEntry, [219](#)
  - IntervalRecord, [258](#)
  - PlannedWorkoutDraft, [331](#)
- notify
  - UndoManager, [385](#)
- np
  - ClimbRecord, [170](#)
  - IntervalRecord, [258](#)
- number
  - PowerZone, [342](#)
- nX
  - VideoRenderer::TileViewport, [376](#)
- nY
  - VideoRenderer::TileViewport, [376](#)
- onAccepted
  - AthleteDialog, [103](#)
- onActivated
  - BasePage, [116](#)
- onActivityImportComplete
  - ImportController, [226](#)
- onActivityImported
  - MainWindow, [280](#)
- onAthleteSelectionChanged
  - MainWindow, [280](#)
- onClimbBoundaryEdited
  - MainWindow, [280](#)
- onClimbRowDoubleClicked
  - MainWindow, [280](#)
- onClimbSelectionChanged
  - MainWindow, [280](#)
- onClimbTableContextMenuRequested
  - MainWindow, [280](#)
- onDateChanged
  - CalendarWidget, [120](#)
- onDiskTiles
  - VideoTileStats, [413](#)
- onFinished
  - VideoExportDialog, [389](#)
- onFrameProgressChanged
  - VideoExportDialog, [389](#)
- onIntervalRowDoubleClicked
  - MainWindow, [280](#)
- onIntervalSelectionChanged
  - MainWindow, [281](#)
- OnMsg
  - anonymous\_namespace{FitParser.cpp}::FitFileIdListener, [206](#)
  - anonymous\_namespace{FitParser.cpp}::FitSessionListener, [217](#)
  - FitRecordListener, [214](#)
- onNewClimbRequested
  - MainWindow, [281](#)

- onProgressChanged
  - VideoExportDialog, 390
- onReplyFinished
  - TileCache, 372
- onStageChanged
  - VideoExportDialog, 390
- onTaskCompleted
  - AnalysisQueue, 90
- onTaskFailed
  - AnalysisQueue, 90
- onTileStatsChanged
  - VideoExportDialog, 390
- onWatchDirectoryChanged
  - MainWindow, 281
- onWorkerCompleted
  - ImportQueue, 239
- onWorkerFailed
  - ImportQueue, 239
- onWorkerProgress
  - ImportQueue, 239
- onWorkoutLoaded
  - ActivityViewController, 82
  - MainWindow, 281
- open
  - DatabaseManager, 193
- openDatabase
  - MainWindow, 281
- openDatabasePath
  - MainWindow, 281
- openRecentDatabase
  - MainWindow, 281
- openSettingsDialog
  - MainWindow, 281
- operator<
  - anonymous\_namespace{ActivityBrowser.cpp}::ActivityBrowserTableItem, 72
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbNameTableItem, 165
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbSortKeyTableItem, 179
  - anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbNameTableItem, 166
  - anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbSortKeyTableItem, 181
- operator()
  - VideoTileIdHash, 407
- operator==
  - VideoRenderer::TileViewport, 376
  - VideoTileId, 406
- originalEndSeconds
  - ClimbRecord, 170
- originalStartSeconds
  - ClimbRecord, 170
- outputPath
  - VideoExportSettings, 396
- overlayAltitude
  - VideoExportSettings, 396
- overlayAthleteName
  - VideoExportSettings, 396
- overlayCadence
  - VideoExportSettings, 396
- overlayCharts
  - VideoExportSettings, 396
- overlayHeartRate
  - VideoExportSettings, 397
- overlayPower
  - VideoExportSettings, 397
- overlayRouteLegend
  - VideoExportSettings, 397
- overlaySpeed
  - VideoExportSettings, 397
- overlayTelemetryPanel
  - VideoExportSettings, 397
- overlayTime
  - VideoExportSettings, 397
- Page
  - NavigationController, 321
  - NavigationSidebar, 326
- paintEvent
  - MapRenderer, 315
- parseDate
  - DateFormatter, 196
- Parsing
  - ImportJob, 231
- path
  - DatabaseManager, 194
- pdCPoints
  - WorkoutController, 422
- pendingCount
  - AnalysisQueue, 90
  - ImportQueue, 239
- pendingCountChanged
  - AnalysisQueue, 91
- percent
  - PlaceholderPage, 327
  - PlaceholderPage, 328
- PlannedWorkoutDialog
  - draft, 329
  - m\_dateEdit, 330
  - m\_durationSpin, 330
  - m\_id, 330
  - m\_notesEdit, 330
  - m\_titleEdit, 330
  - m\_tssSpin, 330
  - m\_typeEdit, 330
  - PlannedWorkoutDialog, 329
- PlannedWorkoutDraft, 330
  - date, 331
  - durationSeconds, 331
  - id, 331
  - notes, 331
  - targetTss, 331
  - title, 331
  - type, 331
- plateauMinGradient

- ClimbDetectorConfig, 158
- Platform, 48
  - applicationDataDirectory, 48
  - defaultBackupPath, 48
  - defaultDatabaseDirectory, 48
  - defaultDatabasePath, 49
- playbackSpeed
  - VideoExportSettings, 397
- populateActivityDetails
  - ActivityViewController, 83
- populateFtpTable
  - AthleteDialog, 104
- populateWeightTable
  - AthleteDialog, 104
- Power
  - NavigationSidebar, 326
- power
  - RideRecord, 367
- PowerCurve, 331
  - bestMeanPower, 332
  - computeStandard, 332
  - standardDurations, 333
- PowerCurvePoint, 334
  - bestPower, 334
  - durationSeconds, 334
- PowerCurveWidget, 334
  - clearChart, 335
  - PowerCurveWidget, 335
  - setRideData, 336
  - setRideDataWithComparisons, 336
- powerFadePct
  - Climb, 145
- powerFirstHalf
  - Climb, 146
- PowerHistogramWidget, 337
  - clearChart, 338
  - PowerHistogramWidget, 338
  - setRideData, 339
- PowerPage, 339
  - PowerPage, 341
- powerSecondHalf
  - Climb, 146
- PowerZone, 341
  - lowerPct, 342
  - name, 342
  - number, 342
  - seconds, 342
  - upperPct, 342
- powerZones
  - WorkoutController, 422
  - ZoneCalculator, 432
- prepareImportFileBatch
  - MainWindow, 281
- previewFitAction
  - MainWindowActions, 302
- previewFitFile
  - MainWindow, 281
- processJob
  - ImportWorker, 247
- processNext
  - AnalysisQueue, 91
  - ImportQueue, 239
- processTask
  - AnalysisWorker, 94
- progress
  - ImportJob, 231
  - ImportProgressModel, 234
- progressChanged
  - VideoRenderJob, 403
- progressModel
  - ImportController, 227
- push
  - UndoManager, 386
- qtDatePattern
  - DateFormatter, 197
- quarterMetrics
  - Climb, 146
- Queued
  - ImportJob, 231
- queued
  - MainWindow::ImportBatchSummary, 222
- queueFilesForImport
  - ImportController, 227
- rangeLabelForZone
  - anonymous\_namespace{ChartController.cpp}, 30
- rating
  - ClimbRecord, 170
- reanalyze
  - WorkoutController, 422
- rebuildChart
  - RideChartWidget, 352
- rebuildGpsCache
  - MapRenderer, 315
- rebuildSegmentColors
  - MapRenderer, 316
- recentDatabaseMenu
  - MainWindowActions, 302
- recolorMonth
  - CalendarWidget, 120
- recordFromQuery
  - anonymous\_namespace{ClimbRepository.cpp}, 33
- records
  - GpsCacheData, 221
  - RideData, 363
- redo
  - ClimbEditCommand, 161
  - UndoCommand, 382
  - UndoManager, 386
- redoDescription
  - UndoManager, 386
- refresh
  - ActivityBrowser, 70
  - AthleteListDialog, 108
  - CalendarWidget, 120
- refreshActivity

- ActivityViewController, [83](#)
- refreshAthleteSelector
  - MainWindow, [281](#)
- refreshCharts
  - ChartController, [127](#)
- refreshClimbViews
  - MainWindow, [282](#)
- refreshMap
  - MapController, [308](#)
- refreshMapRideDataFromCurrentState
  - MainWindow, [282](#)
- reloadClimbsFromDatabase
  - WorkoutController, [422](#)
- reloadDayList
  - CalendarWidget, [120](#)
- removeOverlappingAndDuplicates
  - IntervalDetector, [252](#)
- removeSelectedClimbs
  - MainWindow, [282](#)
- requiredTileCount
  - VideoTileProvider, [410](#)
- requiredTiles
  - VideoTileStats, [413](#)
- requiredTilesAvailableOnDisk
  - VideoTileProvider, [411](#)
- requireGradePersistence
  - ClimbDetectorConfig, [158](#)
- resetAllZoom
  - ChartController, [127](#)
  - MainWindow, [282](#)
- resetZoom
  - RideChartWidget, [352](#)
- resizeEvent
  - RideChartWidget, [353](#)
- resolveActiveActivityWeightKg
  - MainWindow, [282](#)
- resolveAthleteFtpForDate
  - anonymous\_namespace{WorkoutController.cpp}, [46](#)
- restoreClimbSelection
  - MainWindow, [282](#)
- restoreNavigationState
  - NavigationController, [323](#)
- revertSelectedClimbToAuto
  - MainWindow, [282](#)
- RideChartSelectionManager::ClimbHandleHitResult, [162](#)
  - hit, [162](#)
  - startHandle, [162](#)
- RideChartWidget, [342](#)
  - addClimbBackgroundBands, [347](#)
  - addIntervalBackgroundBands, [347](#)
  - addMetricOverlayBands, [347](#)
  - addZoneBackgroundBands, [347](#)
  - buildSampleIndices, [347](#)
  - cachedAxisMax, [358](#)
  - cachedAxisMin, [358](#)
  - cachedMaxX, [358](#)
  - cachedMinX, [358](#)
  - clearBackgroundSeries, [347](#)
  - clearChart, [347](#)
  - climbBoundaryEdited, [347](#)
  - climbClicked, [348](#)
  - ClimbHandleDrag, [346](#)
  - computeSmoothedPowerSeries, [348](#)
  - computeSmoothedSeries, [348](#)
  - crosshairMoved, [348](#)
  - drawForeground, [349](#)
  - fitToData, [349](#)
  - hasData, [349](#)
  - intervalSelectionRequested, [350](#)
  - leaveEvent, [350](#)
  - m\_autoSmoothing, [358](#)
  - m\_axisX, [358](#)
  - m\_axisXDisplay, [358](#)
  - m\_axisY, [359](#)
  - m\_axisYOverlay, [359](#)
  - m\_backgroundSeries, [359](#)
  - m\_climbEditingEnabled, [359](#)
  - m\_climbHandleDrag, [359](#)
  - m\_climbs, [359](#)
  - m\_colorContext, [359](#)
  - m\_colorMetric, [359](#)
  - m\_colorSeries, [359](#)
  - m\_crosshairX, [359](#)
  - m\_crosshairY, [359](#)
  - m\_dataMaxY, [359](#)
  - m\_dataMinY, [360](#)
  - m\_dragOriginalClimbEndSec, [360](#)
  - m\_dragOriginalClimbStartSec, [360](#)
  - m\_effectivePowerSmoothingSeconds, [360](#)
  - m\_hasData, [360](#)
  - m\_intervalEndMin, [360](#)
  - m\_intervals, [360](#)
  - m\_intervalSelecting, [360](#)
  - m\_intervalStartMin, [360](#)
  - m\_isRebuilding, [360](#)
  - m\_metric, [360](#)
  - m\_metricOverlayEnabled, [360](#)
  - m\_metricOverlayMetric, [361](#)
  - m\_newClimbCreating, [361](#)
  - m\_newClimbCurrentSec, [361](#)
  - m\_newClimbDragging, [361](#)
  - m\_newClimbStartSec, [361](#)
  - m\_origMaxX, [361](#)
  - m\_overlaySeries, [361](#)
  - m\_rawSeries, [361](#)
  - m\_referenceSeries, [361](#)
  - m\_requestedPowerSmoothingSeconds, [361](#)
  - m\_rideData, [361](#)
  - m\_selectedClimbIndex, [361](#)
  - m\_series, [362](#)
  - m\_smoothedSeriesCache, [362](#)
  - m\_syncing, [362](#)
  - m\_tooltipDisplayValues, [362](#)
  - m\_tooltipPoints, [362](#)



- m\_tooltipRawValues, 362
- mouseDoubleClickEvent, 350
- mouseMoveEvent, 351
- mousePressEvent, 351
- mouseReleaseEvent, 351
- newClimbRequested, 352
- rebuildChart, 352
- resetZoom, 352
- resizeEvent, 353
- RideChartWidget, 346
- setAutoSmoothingEnabled, 353
- setClimbEditingEnabled, 353
- setClimbs, 354
- setCrosshair, 354
- setIntervals, 354
- setMetricOverlay, 354
- setPowerSmoothingSeconds, 355
- setRideData, 355
- setSelectedClimbIndex, 355
- setXRange, 355
- updateXTicks, 356
- updateYAxis, 356
- updateYAxisForVisibleRange, 356
- valid, 362
- visibleRangeChanged, 356
- visibleRangeEndMinutes, 357
- visibleRangeStartMinutes, 357
- wheelEvent, 357
- xRangeChanged, 357
- RideData, 362
  - activityEndTimeUtc, 363
  - activityStartTimeUtc, 363
  - fileTimestampUtc, 363
  - records, 363
- rideData
  - VideoExportSettings, 397
  - WorkoutController, 422
- RideDataLoader, 363
  - loadActivitySamples, 364
- RideDataSerializer, 364
  - loadRideFromDatabase, 364
  - saveRideToDatabase, 364
- RideRecord, 365
  - altitude, 365
  - cadence, 365
  - elapsedSeconds, 366
  - hasAltitude, 366
  - hasCadence, 366
  - hasGps, 366
  - hasHeartRate, 366
  - hasPower, 366
  - hasSpeed, 366
  - heartRate, 367
  - latitude, 367
  - longitude, 367
  - power, 367
  - speed, 367
- routeColor
  - anonymous\_namespace{VideoRenderJob.cpp}, 45
- routeColorMetric
  - VideoExportSettings, 397
- rpe
  - Activity, 61
- run
  - VideoRenderJob, 404
- sampleAvailable
  - anonymous\_namespace{ZoneCalculator.cpp}, 47
- sampleDurationSeconds
  - anonymous\_namespace{ZoneCalculator.cpp}, 47
- sanitizeDateFormat
  - AppSettings, 98
- sanitizeVideoFilePart
  - anonymous\_namespace{MainWindowVideo.cpp}, 41
- savedAthletId
  - AthleteDialog, 104
- saveNavigationState
  - NavigationController, 323
- saveRide
  - WorkoutPersistenceService, 425
- saveRideToDatabase
  - RideDataSerializer, 364
- saveSettings
  - MainWindow, 282
- scanWatchDirectories
  - MainWindow, 283
- scheduleActivityBrowserRefresh
  - MainWindow, 283
- seconds
  - PowerZone, 342
  - ZoneDistribution, 435
- segmentEndSeconds
  - VideoExportSettings, 398
- segmentMeters
  - anonymous\_namespace{ClimbCandidateFinder.cpp}, 30
- segmentStartSeconds
  - VideoExportSettings, 398
- selectAthlete
  - AthleteListDialog, 108
- selectedAthletId
  - AthleteListDialog, 109
- selectedClimbIndicesFromTable
  - MainWindow, 283
- selectExistsEver
  - SqlQueryHelper, 50
- selectExistsLocked
  - SqlQueryHelper, 50
- selectExistsNonDeleted
  - SqlQueryHelper, 51
- selectIntervalRowOffset
  - MainWindow, 283
- setAllOverlayChecks
  - VideoExportDialog, 390
- setAnalysisQueue
  - WorkoutController, 422



- setAnalysisTabWidgets
  - ChartController, 128
- setAthleteContext
  - ActivityViewController, 83
- setAthleteHeaderWidget
  - ActivityViewController, 84
- setAthleteld
  - CalendarWidget, 120
  - ChartController, 128
- setAutoRouteContrast
  - MapController, 309
  - MapRenderer, 316
- setAutoSmoothingEnabled
  - RideChartWidget, 353
- setCache
  - ActivityAnalysisCacheRepository, 65
- setChartControls
  - ChartController, 128
- setChartWidgets
  - ChartController, 128
- setClimbCharts
  - ChartController, 129
- setClimbEditingEnabled
  - RideChartWidget, 353
- setClimbingTabStack
  - ChartController, 129
- setClimbs
  - ChartController, 129
  - RideChartWidget, 354
- setColorContext
  - ChartController, 129
- setColorMetric
  - ChartController, 130
- setCrosshair
  - RideChartWidget, 354
- setCurrentAthlete
  - WorkoutController, 422
- setCurrentPage
  - NavigationSidebar, 326
- setCurrentTime
  - MapController, 309
  - MapRenderer, 316
- setDatabaseManager
  - CalendarWidget, 120
  - WorkoutController, 422
- setDatabasePath
  - AnalysisQueue, 91
  - AnalysisWorker, 95
  - ImportQueue, 239
  - ImportWorker, 248
- setDateFormat
  - AppSettings, 98
- setFirstLaunch
  - WelcomeWidget, 417
- setFTP
  - WorkoutController, 422
- setHighlightedElapsedSeconds
  - MapRenderer, 316
- setImportQueue
  - ImportController, 228
- setImportSource
  - ActivityRepository, 76
- setIntervals
  - ChartController, 130
  - RideChartWidget, 354
- setMapRenderer
  - MapController, 309
- setMapStyle
  - MapRenderer, 316
  - TileCache, 372
- setMemoryCacheSize
  - TileCache, 372
- setMetricOverlay
  - RideChartWidget, 354
- setModel
  - ImportStatusWidget, 244
- setNavigationSidebar
  - NavigationController, 323
- setPageStack
  - NavigationController, 324
- setPowerSmoothingSeconds
  - RideChartWidget, 355
- setProgressModel
  - ImportController, 228
- setRequiredTiles
  - VideoTileProvider, 411
- setRideData
  - MapRenderer, 316
  - PowerCurveWidget, 336
  - PowerHistogramWidget, 339
  - RideChartWidget, 355
- setRideDataWithComparisons
  - PowerCurveWidget, 336
- setRideMetrics
  - AthleteHeaderWidget, 106
- setRouteColorMode
  - MapRenderer, 316
- setSelectedClimbIndex
  - RideChartWidget, 355
- setStatusBarLabels
  - ActivityViewController, 84
- setStatusWidget
  - ImportController, 229
- setSummary
  - AthleteHeaderWidget, 106
- setTileCacheSize
  - AppSettings, 98
  - MapRenderer, 316
- setTimeline
  - FitnessChartWidget, 208
- settingsForActivity
  - ActivityAnalysisSettingsRepository, 68
- setupShortcuts
  - MainWindow, 283
  - MainWindowActions, 302
- setVisibleTimeRange

- MapController, 310
- MapRenderer, 316
- setXRange
  - RideChartWidget, 355
- setZonesTable
  - ChartController, 130
- shapeClass
  - Climb, 146
- shouldCreateDefaultAthlete
  - CreateDatabaseDialog, 188
- shouldDeleteExportCacheAfterExport
  - VideoTileProvider, 411
- showAbout
  - MainWindow, 283
- showContextMenu
  - ActivityBrowser, 71
- showResultDialog
  - MainWindow::ImportBatchSummary, 222
- showWelcomeScreen
  - MainWindow, 283
- singleRouteColor
  - VideoExportSettings, 398
- sleepHours
  - Activity, 62
- smallDipThresholdMeters
  - ClimbDetectorConfig, 158
- smoothAltitudeByDistance
  - ClimbDetector, 153
- SmoothingEngine, 368
  - smoothPower, 368
- smoothingWindow
  - IntervalDetectorConfig, 253
- smoothPower
  - IntervalDetector, 252
  - SmoothingEngine, 368
- softDeleteClimb
  - ClimbRepository, 177
- softDeleteInterval
  - IntervalRepository, 262
- source
  - ClimbRecord, 171
  - IntervalRecord, 258
- sourceLabel
  - MainWindow::ImportBatchSummary, 222
- speed
  - RideRecord, 367
- speedZones
  - ZoneCalculator, 432
- sport
  - Activity, 62
- SqlQueryHelper, 49
  - bindBoolValue, 49
  - kWhereDeleted, 51
  - kWhereLocked, 51
  - kWhereNotDeleted, 52
  - selectExistsEver, 50
  - selectExistsLocked, 50
  - selectExistsNonDeleted, 51
- src/analysis/climb/ClimbCandidateFinder.cpp, 437
- src/analysis/climb/ClimbCandidateFinder.h, 438
- src/analysis/climb/ClimbClassifier.cpp, 438
- src/analysis/climb/ClimbClassifier.h, 438
- src/analysis/climb/ClimbMerger.cpp, 438
- src/analysis/climb/ClimbMerger.h, 439
- src/analysis/ClimbDetector.cpp, 439
- src/analysis/ClimbDetector.h, 446
- src/analysis/ClimbMetrics.cpp, 447
- src/analysis/ClimbMetrics.h, 453
- src/analysis/FTPZones.cpp, 454
- src/analysis/FTPZones.h, 455
- src/analysis/IntervalDetector.cpp, 455
- src/analysis/IntervalDetector.h, 458
- src/analysis/PowerCurve.cpp, 459
- src/analysis/PowerCurve.h, 460
- src/analysis/queue/AnalysisQueue.cpp, 460
- src/analysis/queue/AnalysisQueue.h, 461
- src/analysis/queue/AnalysisWorker.cpp, 462
- src/analysis/queue/AnalysisWorker.h, 465
- src/analysis/SmoothingEngine.cpp, 465
- src/analysis/SmoothingEngine.h, 466
- src/analysis/TrainingLoad.cpp, 466
- src/analysis/TrainingLoad.h, 467
- src/analysis/TrainingMetrics.cpp, 468
- src/analysis/TrainingMetrics.h, 468
- src/analysis/WorkoutAnalyzer.cpp, 469
- src/analysis/WorkoutAnalyzer.h, 470
- src/charts/data/RideChartDataBuilder.cpp, 471
- src/charts/data/RideChartDataBuilder.h, 472
- src/charts/FitnessChartWidget.cpp, 473
- src/charts/FitnessChartWidget.h, 474
- src/charts/Metric.h, 474
- src/charts/PowerCurveWidget.cpp, 475
- src/charts/PowerCurveWidget.h, 477
- src/charts/PowerHistogramWidget.cpp, 478
- src/charts/PowerHistogramWidget.h, 479
- src/charts/rendering/RideChartRenderer.cpp, 479
- src/charts/rendering/RideChartRenderer.h, 480
- src/charts/RideChartWidget.cpp, 480
- src/charts/RideChartWidget.h, 501
- src/charts/selection/RideChartSelectionManager.cpp, 503
- src/charts/selection/RideChartSelectionManager.h, 504
- src/core/ActivityAnalysisFlags.h, 504
- src/core/AnalysisVersions.h, 505
- src/core/settings/AppSettings.cpp, 505
- src/core/settings/AppSettings.h, 506
- src/core/settings/DateFormatter.cpp, 506
- src/core/settings/DateFormatter.h, 507
- src/core/undo/ClimbEditCommand.cpp, 508
- src/core/undo/ClimbEditCommand.h, 508
- src/core/undo/UndoCommand.h, 509
- src/core/undo/UndoManager.cpp, 509
- src/core/undo/UndoManager.h, 510
- src/core/zones/ColorProvider.cpp, 511
- src/core/zones/ColorProvider.h, 512
- src/core/zones/ZoneCalculator.cpp, 512

src/core/zones/ZoneCalculator.h, 515  
src/core/zones/ZoneDefinition.cpp, 515  
src/core/zones/ZoneDefinition.h, 516  
src/database/ActivityAnalysisCacheRepository.cpp, 517  
src/database/ActivityAnalysisCacheRepository.h, 517  
src/database/ActivityAnalysisSettingsRepository.cpp, 518  
src/database/ActivityAnalysisSettingsRepository.h, 518  
src/database/ActivityRepository.cpp, 519  
src/database/ActivityRepository.h, 523  
src/database/AthleteRepository.cpp, 524  
src/database/AthleteRepository.h, 526  
src/database/ClimbRepository.cpp, 528  
src/database/ClimbRepository.h, 530  
src/database/connection/DatabaseConnectionFactory.cpp, 532  
src/database/connection/DatabaseConnectionFactory.h, 532  
src/database/DatabaseConnectionGuard.cpp, 532  
src/database/DatabaseConnectionGuard.h, 533  
src/database/DatabaseManager.cpp, 533  
src/database/DatabaseManager.h, 535  
src/database/DatabaseSchema.cpp, 535  
src/database/DatabaseSchema.h, 535  
src/database/ImportRepository.cpp, 539  
src/database/ImportRepository.h, 540  
src/database/IntervalRepository.cpp, 540  
src/database/IntervalRepository.h, 542  
src/database/migrations/DatabaseMigrationManager.cpp, 543  
src/database/migrations/DatabaseMigrationManager.h, 547  
src/database/SqlQueryHelper.h, 547  
src/fit/FitParser.cpp, 547  
src/fit/FitParser.h, 550  
src/fit/FitRecordListener.cpp, 550  
src/fit/FitRecordListener.h, 552  
src/fit/RideData.h, 552  
src/fityzerC.cpp, 553  
src/gui/AboutDialog.cpp, 553  
src/gui/AboutDialog.h, 554  
src/gui/ActivityBrowser.cpp, 554  
src/gui/ActivityBrowser.h, 558  
src/gui/AthleteDialog.cpp, 559  
src/gui/AthleteDialog.h, 562  
src/gui/AthleteHeaderWidget.cpp, 563  
src/gui/AthleteHeaderWidget.h, 564  
src/gui/AthleteListDialog.cpp, 564  
src/gui/AthleteListDialog.h, 566  
src/gui/CalendarWidget.cpp, 566  
src/gui/CalendarWidget.h, 570  
src/gui/controllers/ActivityViewController.cpp, 571  
src/gui/controllers/ActivityViewController.h, 573  
src/gui/controllers/ChartController.cpp, 574  
src/gui/controllers/ChartController.h, 581  
src/gui/controllers/ImportController.cpp, 583  
src/gui/controllers/ImportController.h, 584  
src/gui/controllers/MainWindowActions.cpp, 585  
src/gui/controllers/MainWindowActions.h, 586  
src/gui/controllers/MapController.cpp, 588  
src/gui/controllers/MapController.h, 589  
src/gui/controllers/NavigationController.cpp, 590  
src/gui/controllers/NavigationController.h, 591  
src/gui/CreateDatabaseDialog.cpp, 591  
src/gui/CreateDatabaseDialog.h, 593  
src/gui/EditActivityDialog.cpp, 594  
src/gui/EditActivityDialog.h, 595  
src/gui/EditNotesDialog.cpp, 595  
src/gui/EditNotesDialog.h, 596  
src/gui/ImportStatusWidget.cpp, 596  
src/gui/ImportStatusWidget.h, 597  
src/gui/IntervalEditorDialog.cpp, 598  
src/gui/IntervalEditorDialog.h, 598  
src/gui/MainWindow.cpp, 599  
src/gui/MainWindow.h, 602  
src/gui/MainWindowAthlete.cpp, 607  
src/gui/MainWindowChartActions.cpp, 612  
src/gui/MainWindowCharts.cpp, 614  
src/gui/MainWindowClimbInteractions.cpp, 615  
src/gui/MainWindowClimbViews.cpp, 625  
src/gui/MainWindowContext.cpp, 629  
src/gui/MainWindowDatabase.cpp, 630  
src/gui/MainWindowDetection.cpp, 634  
src/gui/MainWindowImport.cpp, 637  
src/gui/MainWindowIntervals.cpp, 640  
src/gui/MainWindowReadiness.cpp, 645  
src/gui/MainWindowStatus.cpp, 648  
src/gui/MainWindowUi.cpp, 649  
src/gui/MainWindowUiBuild.cpp, 651  
src/gui/MainWindowVideo.cpp, 665  
src/gui/MainWindowWatch.cpp, 666  
src/gui/NavigationSidebar.cpp, 668  
src/gui/NavigationSidebar.h, 669  
src/gui/pages/ActivitiesPage.cpp, 670  
src/gui/pages/ActivitiesPage.h, 671  
src/gui/pages/BasePage.h, 671  
src/gui/pages/CalendarPage.cpp, 671  
src/gui/pages/CalendarPage.h, 672  
src/gui/pages/ChartsPage.cpp, 672  
src/gui/pages/ChartsPage.h, 672  
src/gui/pages/ClimbsPage.cpp, 672  
src/gui/pages/ClimbsPage.h, 673  
src/gui/pages/FitnessPage.cpp, 673  
src/gui/pages/FitnessPage.h, 674  
src/gui/pages/IntervalsPage.cpp, 674  
src/gui/pages/IntervalsPage.h, 674  
src/gui/pages/PlaceholderPage.h, 675  
src/gui/pages/PowerPage.cpp, 675  
src/gui/pages/PowerPage.h, 675  
src/gui/pages/VideoPage.cpp, 676  
src/gui/pages/VideoPage.h, 677  
src/gui/PlannedWorkoutDialog.cpp, 677  
src/gui/PlannedWorkoutDialog.h, 678  
src/gui/WelcomeWidget.cpp, 679  
src/gui/WelcomeWidget.h, 681  
src/gui/WorkoutController.cpp, 682

- src/gui/WorkoutController.h, 687
- src/import/ImportJob.cpp, 689
- src/import/ImportJob.h, 689
- src/import/ImportProgressModel.cpp, 690
- src/import/ImportProgressModel.h, 691
- src/import/ImportQueue.cpp, 691
- src/import/ImportQueue.h, 693
- src/import/ImportWorker.cpp, 694
- src/import/ImportWorker.h, 697
- src/maps/cache/DiskTileCache.cpp, 698
- src/maps/cache/DiskTileCache.h, 698
- src/maps/cache/GpsCacheBuilder.cpp, 699
- src/maps/cache/GpsCacheBuilder.h, 699
- src/maps/MapFitMath.cpp, 699
- src/maps/MapFitMath.h, 701
- src/maps/MapRenderer.cpp, 701
- src/maps/MapRenderer.h, 711
- src/maps/network/TileDownloader.cpp, 712
- src/maps/network/TileDownloader.h, 713
- src/maps/providers/TileProviderRegistry.cpp, 713
- src/maps/providers/TileProviderRegistry.h, 714
- src/maps/rendering/RouteColorizer.cpp, 714
- src/maps/rendering/RouteColorizer.h, 715
- src/maps/TileCache.cpp, 715
- src/maps/TileCache.h, 719
- src/model/RideDataLoader.cpp, 720
- src/model/RideDataLoader.h, 721
- src/model/RideDataSerializer.cpp, 721
- src/model/RideDataSerializer.h, 723
- src/platform/Platform.cpp, 724
- src/platform/Platform.h, 724
- src/platform/Platform\_linux.cpp, 724
- src/platform/Platform\_macos.cpp, 725
- src/platform/Platform\_windows.cpp, 725
- src/utils/FfmpegPath.cpp, 725
- src/utils/FfmpegPath.h, 726
- src/utils/UuidGenerator.h, 726
- src/video/encoding/VideoEncoder.cpp, 727
- src/video/encoding/VideoEncoder.h, 727
- src/video/overlay/OverlayRenderer.cpp, 727
- src/video/overlay/OverlayRenderer.h, 728
- src/video/rendering/VideoRenderer.cpp, 728
- src/video/rendering/VideoRenderer.h, 730
- src/video/VideoExportDialog.cpp, 731
- src/video/VideoExportDialog.h, 736
- src/video/VideoExportSettings.h, 737
- src/video/VideoRenderJob.cpp, 738
- src/video/VideoRenderJob.h, 747
- src/video/VideoTileProvider.cpp, 748
- src/video/VideoTileProvider.h, 749
- src/workouts/WorkoutPersistenceService.cpp, 750
- src/workouts/WorkoutPersistenceService.h, 751
- stageChanged
  - VideoRenderJob, 404
- standardDurations
  - PowerCurve, 333
- startDistanceKm
  - Climb, 146
- startElevationM
  - Climb, 146
- startExport
  - VideoExportDialog, 390
- startGradient
  - ClimbDetectorConfig, 158
- startHandle
  - RideChartSelectionManager::ClimbHandleHitResult, 162
- startSample
  - IntervalRecord, 258
- startSeconds
  - Climb, 146
  - ClimbRecord, 171
  - Interval, 250
- startTime
  - Activity, 62
  - anonymous\_namespace{FitParser.cpp}::FitSessionListener, 218
- State
  - ImportJob, 231
- state
  - ImportJob, 232
- statistics
  - WorkoutController, 422
- stats
  - VideoTileProvider, 411
- status
  - ImportRecord, 241
- statusText
  - ImportProgressModel, 234
- Storing
  - ImportJob, 231
- style
  - VideoTileProviderConfig, 412
- syncAnalysisViews
  - ActivityViewController, 84
- syncAthleteContextToControllers
  - MainWindow, 283
- syncChartContextToController
  - MainWindow, 283
- syncChartCrosshair
  - ChartController, 130
- syncChartZoom
  - ChartController, 131
- targetTss
  - PlannedWorkoutDraft, 331
- taskCompleted
  - AnalysisQueue, 91
  - AnalysisWorker, 95
- taskFailed
  - AnalysisQueue, 92
  - AnalysisWorker, 96
- temperature
  - Activity, 62
- thresholdHeartRate
  - ColorContext, 184
- tile

- TileCache, 372
- tileBlocking
  - TileCache, 372
- TileCache, 369
  - attribution, 370
  - clearMemoryCache, 370
  - diskCacheRoot, 371
  - diskTilePath, 371
  - diskTilePathForRoot, 371
  - dispatchNextDownloads, 371
  - isTileCachedOnDisk, 371
  - key, 371
  - keyForStyle, 371
  - kMaxConcurrentDownloads, 373
  - m\_activeDownloads, 373
  - m\_cache, 373
  - m\_diskCacheRoot, 373
  - m\_diskFallbackRoots, 373
  - m\_downloadQueue, 373
  - m\_mapStyle, 373
  - m\_nam, 373
  - m\_pending, 373
  - m\_provider, 373
  - mapStyle, 372
  - mapStyleName, 372
  - maxZoom, 372
  - onReplyFinished, 372
  - setMapStyle, 372
  - setMemoryCacheSize, 372
  - tile, 372
  - tileBlocking, 372
  - TileCache, 370
- TileCache::TileRequest, 375
  - x, 375
  - y, 375
  - z, 375
- TileCacheConfig, 373
  - diskCacheRoot, 374
  - diskFallbackRoots, 374
  - maxTilesInMemory, 374
- tileCacheSize
  - AppSettings, 98
- tileCacheSizeKey
  - AppSettings, 98
- TileProvider, 374
  - attribution, 374
  - maxZoom, 374
  - name, 374
  - urlTemplate, 375
- tileStatsChanged
  - VideoRenderJob, 405
- tileToScreen
  - MapRenderer, 316
- tileX0
  - VideoRenderer::TileViewport, 376
- tileY0
  - VideoRenderer::TileViewport, 376
- tileYToLat
  - anonymous\_namespace{MapFitMath.cpp}, 42
- timeCreated
  - anonymous\_namespace{FitParser.cpp}::FitFileIdListener, 206
- title
  - PlannedWorkoutDraft, 331
- toISOString
  - DateFormatter, 198
- toISOStringDate
  - DateFormatter, 198
- toISOStringRaw
  - anonymous\_namespace{AthleteDialog.cpp}, 30
- toolbar
  - ActivitiesPage, 57
- toolbarImportAction
  - MainWindowActions, 303
- totalDistanceKm
  - WorkoutStatistics, 427
- trailingNumber
  - anonymous\_namespace{MainWindowClimbInteractions.cpp}::ClimbName, 165
  - anonymous\_namespace{MainWindowClimbViews.cpp}::ClimbName, 166
- TrainingLoad, 376
  - fitnessTimeline, 377
  - intensityFactor, 377
  - normalizedPower, 378
  - trainingStressScore, 378
- TrainingMetrics, 379
  - compute, 380
- TrainingMetricsSummary, 380
  - averageHeartRate, 381
  - averagePower, 381
  - efficiencyFactor, 381
  - intensityFactor, 381
  - normalizedPower, 381
  - trainingStressScore, 381
  - variabilityIndex, 381
- trainingStressScore
  - TrainingLoad, 378
  - TrainingMetricsSummary, 381
  - WorkoutController, 423
- triggerDetectClimbs
  - MainWindow, 284
- triggerDetectIntervals
  - MainWindow, 284
- tsb
  - FitnessMetricsPoint, 209
- tss
  - DailyLoadPoint, 189
- type
  - IntervalRecord, 259
  - PlannedWorkoutDraft, 331
- undo
  - ClimbEditCommand, 161
  - UndoCommand, 383
  - UndoManager, 386
  - UndoCommand, 382

- description, 382
- redo, 382
- undo, 383
- undoDescription
  - UndoManager, 386
- UndoManager, 383
  - canRedo, 385
  - canUndo, 385
  - clear, 385
  - m\_maxStackSize, 386
  - m\_redoStack, 386
  - m\_undoStack, 386
  - notify, 385
  - push, 386
  - redo, 386
  - redoDescription, 386
  - undo, 386
  - undoDescription, 386
  - UndoManager, 384
- updateActivityDisplay
  - ActivityViewController, 85
- updateActivityMetadata
  - ActivityViewController, 85
- updateActivityNotesFromCurrentActivity
  - MainWindow, 284
- updateActivityProperties
  - ActivityRepository, 77
- updateAnalysisEmptyStates
  - ChartController, 131
- updateAnalysisFlags
  - ActivityRepository, 77
- updateAthlete
  - AthleteRepository, 114
- updateAthleteFields
  - CreateDatabaseDialog, 188
- updateAthleteLabel
  - MainWindow, 284
- updateChartAnalysisEmptyStates
  - MainWindow, 284
- updateCharts
  - ChartController, 131
- updateClimb
  - ClimbRepository, 177
- updateClimbingTab
  - MainWindow, 284
- updateClimbQuarterCharts
  - MainWindow, 284
- updateClimbRowStyles
  - MainWindow, 285
- updateColorLegend
  - ChartController, 132
- updateColorLegendDisplay
  - ChartController, 133
- updatedActivity
  - EditActivityDialog, 201
- updateFingerprint
  - ActivityRepository, 77
- updateFitnessChart
  - ChartController, 133
- updateFromModel
  - ImportStatusWidget, 245
- updateHistogram
  - ChartController, 134
- updateImportAvailability
  - ImportController, 229
  - MainWindow, 285
- updateInterval
  - IntervalRepository, 262
- updateIntervalRowStyles
  - MainWindow, 285
- updateIntervals
  - MainWindow, 285
- updateIntervalSummary
  - MainWindow, 285
- updateLapsSummaryFromCurrentActivity
  - MainWindow, 285
- updateMap
  - MapController, 310
- updateMonthFormatting
  - CalendarWidget, 120
- updateNotes
  - ActivityRepository, 78
- updatePowerCurve
  - ChartController, 134
- updateRecentDatabaseMenu
  - MainWindow, 286
- updateRouteColoring
  - MapController, 310
- updateStatsDisplay
  - ActivityViewController, 86
- updateStatsLabel
  - MainWindow, 286
- updateStatusBarInfo
  - MainWindow, 286
- updateStatusBarInformation
  - ActivityViewController, 86
- updateVideoLengthLabel
  - VideoExportDialog, 390
- updateWelcomeScreenVisibility
  - MainWindow, 286
- updateXTicks
  - RideChartWidget, 356
- updateYAxis
  - RideChartWidget, 356
- updateYAxisForVisibleRange
  - RideChartWidget, 356
- updateZoneAvailability
  - ChartController, 135
- updateZonesTab
  - ChartController, 136
- upgradeSchema
  - DatabaseManager, 194
- upperPct
  - PowerZone, 342
- urlTemplate
  - TileProvider, 375



- useAdaptiveSmoothing
  - ClimbDetectorConfig, 159
- useScoringDetection
  - ClimbDetectorConfig, 159
- useSelectedSegment
  - VideoExportSettings, 398
- uuid
  - ClimbRecord, 171
  - IntervalRecord, 259
- valid
  - RideChartWidget, 362
- valueForMetric
  - anonymous\_namespace{ZoneCalculator.cpp}, 47
- vam
  - Climb, 146
  - ClimbRecord, 171
- vamScoreBoost
  - ClimbDetectorConfig, 159
- variabilityIndex
  - TrainingMetricsSummary, 381
  - WorkoutController, 423
- Video
  - NavigationSidebar, 326
- VideoExportDialog, 387
  - ~VideoExportDialog, 389
  - cancelOrClose, 389
  - chooseOutputFile, 389
  - collectSettings, 389
  - m\_altitudeCheck, 390
  - m\_athleteCheck, 390
  - m\_cadenceCheck, 390
  - m\_cancelButton, 390
  - m\_chartsCheck, 391
  - m\_defaults, 391
  - m\_deleteTemporaryTilesCheck, 391
  - m\_entireActivityRadio, 391
  - m\_etaLabel, 391
  - m\_exportButton, 391
  - m\_fixedZoomSpin, 391
  - m\_fpsSpin, 391
  - m\_frameLabel, 391
  - m\_hrCheck, 391
  - m\_job, 391
  - m\_legendCheck, 391
  - m\_mapStyleCombo, 392
  - m\_outputEdit, 392
  - m\_powerCheck, 392
  - m\_progressBar, 392
  - m\_resolutionCombo, 392
  - m\_routeColorCombo, 392
  - m\_running, 392
  - m\_selectedSegmentRadio, 392
  - m\_speedCheck, 392
  - m\_speedSpin, 392
  - m\_stageLabel, 392
  - m\_telemetryPanelCheck, 392
  - m\_thread, 393
  - m\_tileStatsLabel, 393
  - m\_timeCheck, 393
  - m\_videoLengthLabel, 393
  - onFinished, 389
  - onFrameProgressChanged, 389
  - onProgressChanged, 390
  - onStageChanged, 390
  - onTileStatsChanged, 390
  - setAllOverlayChecks, 390
  - startExport, 390
  - updateVideoLengthLabel, 390
  - VideoExportDialog, 389
- VideoExportSettings, 393
  - activityName, 395
  - athleteName, 395
  - autoZoom, 395
  - colorContext, 395
  - deleteTemporaryTilesAfterExport, 395
  - ffmpegPath, 395
  - fixedZoom, 395
  - followAthlete, 395
  - fps, 396
  - height, 396
  - mapStyle, 396
  - outputPath, 396
  - overlayAltitude, 396
  - overlayAthleteName, 396
  - overlayCadence, 396
  - overlayCharts, 396
  - overlayHeartRate, 397
  - overlayPower, 397
  - overlayRouteLegend, 397
  - overlaySpeed, 397
  - overlayTelemetryPanel, 397
  - overlayTime, 397
  - playbackSpeed, 397
  - rideData, 397
  - routeColorMetric, 397
  - segmentEndSeconds, 398
  - segmentStartSeconds, 398
  - singleRouteColor, 398
  - useSelectedSegment, 398
  - width, 398
- VideoPage, 398
  - VideoPage, 400
- VideoRenderer::TileViewport, 375
  - nX, 376
  - nY, 376
  - operator==, 376
  - tileX0, 376
  - tileY0, 376
- VideoRenderJob, 400
  - cancel, 402
  - finished, 402
  - frameProgressChanged, 403
  - m\_cancelRequested, 405
  - m\_settings, 405
  - progressChanged, 403
  - run, 404

- stageChanged, 404
- tileStatsChanged, 405
- VideoRenderJob, 402
- VideoTileId, 406
  - operator==, 406
  - x, 406
  - y, 406
  - z, 406
- VideoTileIdHash, 407
  - operator(), 407
- VideoTileProvider, 407
  - cleanupExportCache, 409
  - clearMemoryCache, 409
  - exportCacheRoot, 409
  - getTile, 409
  - isTileCachedOnDisk, 410
  - m\_deleteTemporaryTilesAfterExport, 411
  - m\_downloadedTiles, 411
  - m\_requiredTiles, 411
  - m\_requiredTilesOnDisk, 412
  - m\_tileCache, 412
  - requiredTileCount, 410
  - requiredTilesAvailableOnDisk, 411
  - setRequiredTiles, 411
  - shouldDeleteExportCacheAfterExport, 411
  - stats, 411
  - VideoTileProvider, 408
- VideoTileProviderConfig, 412
  - deleteTemporaryTilesAfterExport, 412
  - exportCacheRoot, 412
  - memoryCacheTiles, 412
  - style, 412
- VideoTileStats, 413
  - downloadedTiles, 413
  - onDiskTiles, 413
  - requiredTiles, 413
- visibleRangeChanged
  - RideChartWidget, 356
- visibleRangeEndMinutes
  - RideChartWidget, 357
- visibleRangeStartMinutes
  - RideChartWidget, 357
- wattsPerKgRank
  - Climb, 147
- weather
  - Activity, 62
- weatherNotes
  - Activity, 62
  - EditNotesDialog, 203
- WeightEntry, 413
  - athleteId, 414
  - effectiveFrom, 414
  - id, 414
  - weightKg, 414
- weightKg
  - Activity, 62
  - WeightEntry, 414
- WelcomeWidget, 414
  - m\_createDbButton, 417
  - m\_importButton, 417
  - m\_manageAthletesButton, 417
  - m\_openDbButton, 417
  - m\_subtitleLabel, 417
  - setFirstLaunch, 417
  - WelcomeWidget, 416
- wheelEvent
  - MapRenderer, 317
  - RideChartWidget, 357
- width
  - VideoExportSettings, 398
- wind
  - Activity, 62
- WorkoutAnalyzer, 417
  - analyze, 418
- WorkoutController, 418
  - climbs, 420
  - currentActivityId, 420
  - currentAthleteId, 421
  - efficiencyFactor, 421
  - ftp, 421
  - importFile, 421
  - intensityFactor, 421
  - intervals, 421
  - loadActivity, 421
  - loadFile, 421
  - loadOrGenerateClimbs, 421
  - loadOrGenerateIntervals, 421
  - m\_analysisQueue, 423
  - m\_analyzer, 423
  - m\_climbs, 423
  - m\_currentActivityId, 423
  - m\_currentAthleteId, 423
  - m\_dbManager, 423
  - m\_ef, 423
  - m\_ftp, 423
  - m\_if, 423
  - m\_intervals, 423
  - m\_np, 424
  - m\_pdcPoints, 424
  - m\_powerZones, 424
  - m\_rideData, 424
  - m\_statistics, 424
  - m\_tss, 424
  - m\_vi, 424
  - normalizedPower, 422
  - pdcPoints, 422
  - powerZones, 422
  - reanalyze, 422
  - reloadClimbsFromDatabase, 422
  - rideData, 422
  - setAnalysisQueue, 422
  - setCurrentAthlete, 422
  - setDatabaseManager, 422
  - setFTP, 422
  - statistics, 422
  - trainingStressScore, 423



- variabilityIndex, [423](#)
- WorkoutController, [420](#)
- WorkoutPersistenceService, [424](#)
  - loadRide, [425](#)
  - saveRide, [425](#)
- WorkoutStatistics, [425](#)
  - averageCadence, [426](#)
  - averageHeartRate, [426](#)
  - averagePower, [426](#)
  - averageSpeed, [426](#)
  - durationSeconds, [426](#)
  - elevationGainM, [426](#)
  - maximumCadence, [426](#)
  - maximumHeartRate, [426](#)
  - maximumPower, [427](#)
  - maximumSpeed, [427](#)
  - movingTimeSeconds, [427](#)
  - totalDistanceKm, [427](#)
- workThresholdPct
  - IntervalDetectorConfig, [253](#)
- wrapWithNoActivityState
  - anonymous\_namespace{MainWindowUiBuild.cpp}, [41](#)
- x
  - TileCache::TileRequest, [375](#)
  - VideoTileId, [406](#)
- xRangeChanged
  - RideChartWidget, [357](#)
- y
  - TileCache::TileRequest, [375](#)
  - VideoTileId, [406](#)
- z
  - TileCache::TileRequest, [375](#)
  - VideoTileId, [406](#)
- Zone, [427](#)
  - color, [428](#)
  - maxValue, [428](#)
  - minValue, [428](#)
  - name, [428](#)
- zone
  - ZoneDistribution, [435](#)
- ZoneArray
  - FTPZones, [219](#)
- ZoneCalculator, [428](#)
  - altitudeZones, [429](#)
  - cadenceZones, [429](#)
  - computeDistribution, [430](#)
  - hasMetricSamples, [431](#)
  - heartRateZones, [431](#)
  - powerZones, [432](#)
  - speedZones, [432](#)
  - zonesForMetric, [433](#)
- ZoneDistribution, [434](#)
  - percent, [435](#)
  - seconds, [435](#)
  - zone, [435](#)
- zonesForMetric
  - ZoneCalculator, [433](#)
- zoom
  - MapFitMath::FitResult, [216](#)
- zoomIn
  - MapController, [311](#)
  - MapRenderer, [317](#)
- zoomOut
  - MapController, [311](#)
  - MapRenderer, [317](#)